Directum RX

Руководство разработчика

Версия 25.2

© Компания Directum, 2025

Содержание

Введение ... 8

Основные понятия ... 10

Процесс разработки ... 21

- Среда разработки Directum RX ... 22
  - Сохранение изменений ... 25
  - Разрешение конфликтов ... 28
  - Настройка внешнего инструмента сравнения ... 31
  - Настройка учетных записей Git ... 33
  - Использование шаблонов кода ... 34
  - Настройка цветовой схемы для редактора кода ... 35
  - Поиск по коду и замена ... 36
  - Серверная сборка мусора ... 38
- Наследование ... 39
  - Что можно изменять в наследнике ... 40
  - Схема наследования договорных документов ... 42
  - Пример переопределения функции ... 44
- Перекрытие ... 46
  - Что можно изменять в перекрытии ... 47
  - Рекомендации по разработке перекрытий ... 50
  - Объединение изменений в перекрытиях ... 53
  - Пример перекрытия типа документа и модуля ... 55
- Что использовать: наследование или перекрытие ... 61
- Создание и публикация настроек по умолчанию ... 63
- Сборка решений ... 64
- Публикация ... 66
  - Публикация с помощью среды разработки ... 69
  - Публикация через Directum Launcher ... 73
  - Публикация с помощью командной строки ... 75
  - Публикация в распределенную систему ... 80
  - Настройка параметров публикации и клиента ... 81
  - Структура пакета развертывания ... 82
- Экспорт разработки ... 84
  - Экспорт через среду разработки ... 85
  - Невизуальный экспорт через среду разработки ... 87
  - Невизуальный экспорт через DeploymentToolCore ... 89
- Импорт разработки ... 92
- Отладка программного кода ... 93
  - Точки останова ... 96
  - Выполнение отладки ... 97
- Горячие клавиши ... 100
- Особенности разработки для Linux ... 103

Программный код ... 104

- Виды кода ... 105
- Функции ... 106
  - Атрибуты ... 109
  - Модификаторы ... 113
  - Создание функций модуля ... 115
  - Создание функций типов сущностей ... 116

Создание функций инициализации ... 119
Создание функций вычисляемых выражений ... 121
Структуры ... 125
Константы ... 130
Правила оформления ... 133
Группировка программного кода ... 135
Разрешенные и запрещенные конструкции ... 138
Классы Sungero ... 138
Классы .NET ... 139
Язык C# ... 142
LINQ и лямбда-выражения ... 145
Элементы разработки ... 148
Решение ... 148
Редактор решения ... 149
Публикация ... 152
Сторонние библиотеки ... 153
Сторонние компоненты ... 155
Наборы виджетов ... 158
Папки Мой Директум ... 162
Модуль ... 162
Редактор модуля ... 163
Обложка ... 167
Публикация ... 174
Сторонние библиотеки ... 181
Изолированные области ... 183
Папки ... 187
Отображение ... 201
Виджеты ... 202
Фоновые процессы ... 232
Асинхронные обработчики ... 237
Типы сущностей ... 241
Редактор типа сущности ... 243
Авторизация ... 252
Свойства ... 262
Действия ... 285
Форма ... 292
Списки ... 329
Стили ... 335
События типов сущностей ... 341
Порядок выполнения событий ... 343
Серверные события ... 347
Клиентские события ... 372
Разделяемые события ... 377
Параметр Params ... 379
Отключение событий сущностей и свойств ... 380
Разработка справочников ... 381
Наследуемые операции авторизации ... 385
Наследуемые действия ... 385
Разработка документов ... 386
Наследуемые операции авторизации ... 390
Наследуемые действия ... 390
Строгий доступ к документам ... 393
Шифрование документов ... 396
Ограничение на работу с документами вне системы ... 397

Разработка задач и заданий ... 398
Настройка блоков бизнес-процессов ... 402
Настройка схемы задачи ... 426
Настройка формы для задач и заданий ... 452
Работа с вложениями ... 454
Результаты выполнения заданий ... 464
Наследуемые операции авторизации ... 467
Наследуемые действия ... 467
Копирование свойств и действий ... 471
Добавление SQL-запроса ... 472
Настройка панели фильтрации ... 474
Добавление панели фильтрации ... 476
Копирование панели фильтрации ... 477
Скрытие панели фильтрации ... 478
Создание группы контролов ... 478
Добавление набора флажков ... 479
Добавление ссылки на сущность ... 482
Добавление периода дат ... 483
Рекомендации по настройке панели фильтрации ... 485
Отчеты ... 491
Редактор отчета ... 493
Дизайнер отчета ... 495
Настройка макета отчета ... 511
События отчета ... 550
Порядок выполнения отчета ... 551
Копирование отчета ... 552
Рекомендации по разработке отчетов в Linux ... 552
Ресурсы ... 554
Пример локализации действия на ленте ... 557
Пример локализации сообщения валидации ... 559
Примеры изменения локализации в перекрытых элементах разработки ... 560
Рекомендации по разработке для масштабируемых систем ... 562
Быстродействие операций ... 563
Индексы ... 564
Прикладные кэши ... 565
Отключение проверки прав доступа ... 565
Параметры фильтрации ... 567
Получение большого объема данных ... 567
Получение связанных документов ... 568
Оптимизация построения отчетов ... 570
Обращения к веб-серверу ... 570
Объединение вычислений ... 571
«Жадная» загрузка свойств ... 571
Проверка блокировки сущности в действиях ... 572
Кэширование действий с помощью параметра Params ... 573
Проверка фильтров в списках и папках ... 574
Оптимизация длительных процессов ... 574
Транзакции и сессии с большим количеством объектов ... 576
Запуск большого количества длительных асинхронных обработчиков ... 577
Ограничение количества ролей для пользователя ... 578
Назначение большого количества прав на типы документов ... 578
Назначение прав группе пользователей (экземплярные права) ... 579
Массовое удаление документов ... 581
Версионирование функций инициализации ... 581
Массовое удаление субъектов прав ... 583

Оптимизация работы с сущностями ... 584
Блокировка сущности перед изменением ... 584
Вызов асинхронного обработчика при сохранении карточки ... 588
Разработка карточек объектов с большой табличной частью ... 589
Оптимизация события «Возможность выполнения» (CanExecute) ... 590
Работа с большим количеством свойств типа сущности ... 592
Большое количество подписей на документе ... 593
Ускорение массового подписания документов ... 594
Оптимизация выполнения блока «Мониторинг» ... 596
Сервис интеграции ... 596
Ограничение количества объектов для передачи ... 596
Запросы на выполнение длительных операций ... 600
Примеры разработки ... 601
Разработка модуля и типа документа ... 601
Шаг 1. Создание решения, модуля и справочников ... 602
Шаг 2. Создание типа договора ... 604
Шаг 3. Доработка вычисляемой папки в перекрытии модуля ... 606
Шаг 4. Настройка обложки модуля ... 610
Шаг 5. Инициализация модуля ... 611
Шаг 6. Добавление роли согласования ... 615
Шаг 7. Добавление условия согласования ... 621
Документы ... 628
Как подготовить новый тип документа к работе ... 629
Как добавить новый тип документа в окно смены типа ... 632
Как изменить длину содержания в карточке документа ... 633
Как изменить формирование имени документа ... 637
Как добавить контрол состояния с информацией о задачах по документу ... 640
Как запретить отправку договора на согласование, если не указана сумма ... 643
Как изменить срок возврата договора при выдаче сотруднику ... 644
Как отфильтровать список шаблонов, доступных при создании документа ... 645
Как настроить склонение имени в шаблонах документов ... 646
Как добавить сводку по новому типу документа ... 648
Как изменить отметку об ЭП ... 653
Как создать постраничную отметку для парафирования договорных документов ... 658
Как создать отметку об оплате входящих счетов ... 664
Как настроить работу редактора отметок ... 669
Как получить данные без учета прав доступа ... 670
Как создать операцию авторизации для типа документа ... 673
Как сгенерировать отчет и записать его в версию документа ... 675
Как настроить шифрование договоров ... 677
Как настроить строгий доступ к документу ... 679
Как запретить получение документов от конкретного филиала из сервиса обмена ... 680
Согласование по регламенту ... 681
Как добавить результат выполнения в задание на согласование ... 682
Как добавить роль согласования с несколькими участниками ... 686
Как добавить критерий в правило согласования ... 690
Как проинформировать согласующего об ограничениях в полномочиях ... 693
Как зафиксировать дополнительную информацию о согласовании в тексте задания ... 695
Как в задаче по регламенту заполнить поле «Адресат» значением из нового типа документа ... 696
Как создать новый тип сценария для правила согласования ... 699
Задачи и задания ... 704
Как создать версию схемы задачи ... 704
Как в задании скрыть результат выполнения ... 705
Как скрыть блок в контроле состояния ... 706
Как добавить в задание возможность запрашивать продление срока ... 710
Доступность, обязательность, видимость свойств ... 711

Как сделать свойство обязательным ... 711
Как сделать свойство обязательным в зависимости от значения другого свойства ... 712
Как сделать свойство доступным в зависимости от прав доступа пользователя ... 714
Как управлять доступностью нового свойства, если оно зависит от существующего свойства ... 716
Как скрыть свойство из области регистрации в карточке документа ... 717
Как выбрать закрытую запись справочника ... 718
Интеллектуальная обработка документов ... 718
Как настроить распознавание нового типа документов ... 720
Как добавить дополнительный классификатор ... 724
Как изменить стиль подсветки выделенного факта в области предпросмотра ... 728
Обработчики событий ... 731
Как добавить действие в историю работы с сущностью ... 732
Передача данных между событиями одной сущности ... 733
Передача данных между событиями разных сущностей ... 735
Как определить, где находится код ... 735
Папки и списки ... 737
Как создать вычисляемую папку, которая доступна только руководителям ... 738
Как добавить условия фильтрации для списка «Входящие документы» ... 741
Как добавить условия фильтрации для папки «Документы к возврату» ... 743
Как настроить валидацию панели фильтрации для входящих документов ... 747
Как настроить список для двух типов документов с одинаковыми свойствами ... 757
Как ограничить видимость сотрудников в списках ... 761
Виджеты ... 762
Как создать виджет с действиями ... 763
Примеры событий для кольцевой диаграммы ... 771
Примеры событий для круговой диаграммы ... 773
Примеры событий для гистограммы ... 777
Примеры событий для линейного графика ... 781
Как добавить параметр-перечисления ... 784
Как добавить параметр логического типа ... 786
Как создать набор виджетов, который доступен только руководителям ... 788
Скрипты конвертации ... 791
Появилось свойство логического типа ... 791
Свойство стало обязательным ... 792
Изменился тип данных свойства ... 792
Локализация ... 794
Как локализовать отображаемое имя, подсказку и описание действия ... 794
Как добавить сообщение валидации и локализовать его ... 796
Как изменить локализацию в перекрытых элементах разработки ... 797
Структура хранения данных ... 799
Формат имен таблиц и колонок ... 799
Колонки свойств-компонент ... 803
Уникальность таблиц и колонок ... 803
Уникальность и ограничения кодов ... 804
Примеры системных таблиц ... 805
Информация о предыдущих бинарных данных документа ... 807
Особенности разработки для мобильных решений ... 809
Поддержка типов задач в NOMAD ... 809
Поддержка типов заданий в NOMAD ... 810
Свойства ... 810
Группы вложений ... 810
Действия ... 811
Диалоги ... 811

Подавляемые диалоги ... 812
Офлайн-режим ... 813
Подписание ... 813
Работа с карточками документов и справочников в NOMAD ... 814
Настройка задач и заданий в файле SungeroAdapter.config ... 814
task – описание задачи ... 815
job – описание задания или уведомления ... 816
action – описание действия ... 816
result – вариант выполнения задания с дополнительной логикой ... 819
autoRightsAttachmentGroup – группа вложений, на документы из которой автоматически назначаются права ... 820
ask – запрашиваемые параметры ... 820
If – дополнительное действие запрашиваемого параметра ... 824
condition – условия, наложенные на действие ... 825
param – свойство ... 829
taskDialog – диалог с выбором варианта для автоматической обработки ... 830
button – кнопка диалога ... 830
Типы ... 831
Бизнес-процессы ... 831
Документы ... 832
Флаги ... 833
Примеры доработок в файле SungeroAdapter.config ... 834
Как добавить вкладку «Резолюции» в собственное задание ... 834
Как добавить подписание усиленной ЭП в собственное задание ... 836
Как настроить подписание групп вложений в зависимости от результата выполнения ... 836
Как поддержать работу с нестандартными поручениями ... 838

Введение

Directum RX предоставляет возможности модификации системы. Для этого используется среда разработки **Directum RX Development Studio**, с помощью которой вы можете развивать систему под потребности компании, если возможностей стандартной настройки недостаточно. Создавайте новые объекты или дорабатывайте уже существующие. Такой подход позволяет расширять функциональность системы, сохраняя при этом все ее качества.

Для работы со средой разработки необходимо приобрести лицензию.

В этой главе вы найдете информацию о всех этапах разработки, рекомендации и примеры кода, использование объектной модели и особенности написания программного кода.

Установка среды разработки

1. Изучите архитектуру системы: какие есть средства разработки, из чего состоит модель разработки, какая используется схема работы с исходными кодами.
2. Ознакомьтесь с требованиями к компьютеру, на который будет устанавливаться среда разработки.
3. Установите среду разработки с помощью инструмента Directum Launcher.
4. Настройте учетные записи для совместной работы нескольких прикладных разработчиков. По умолчанию используется система управления репозиториями Gitea. Она устанавливается с помощью программы установки. Первая учетная запись создается при установке Gitea, а остальные – в параметрах среды разработки.

   Для работы можно использовать любую другую систему управления репозиториями. В этом случае нужно самостоятельно установить ее и настроить учетные записи.

Перед началом разработки

1. Ознакомьтесь с механизмами наследования и перекрытия. Разработка ведется не с нуля, в поставку системы входят готовые модули, документы, справочники, задачи, задания и отчеты со стандартными свойствами и логикой поведения. Их можно менять с помощью наследования и перекрытий, что позволяет добавлять новые типы сущностей и модифицировать стандартные под потребности компании.
2. Посмотрите обучающие видеоролики с основами разработки и примерами модификации системы.
3. Ознакомьтесь с особенностями написания программного кода, правилами оформления, разрешенными и запрещенными конструкциями.
4. Ознакомьтесь с основными понятиями, которые используются в Directum RX и в среде разработки.
5. Посмотрите список горячих клавиш среды разработки. После запоминания даже части комбинаций возрастает скорость разработки, ведь прибегать к использованию мыши приходится намного меньше.

Модификация системы

1. При разработке следуйте общему порядку модификации и публикации изменений в продуктивную систему.

2. Используйте разделы справки:
   - Элементы разработки – особенности разработки решений, модулей, задач и заданий, документов, справочников, отчетов. Также в главе собраны рекомендации, которые помогут органично встроить ваши модификации в стандартное решение. Таким образом при работе с системой не будет возникать ощущения разнородности ее частей;
   - Рекомендации по разработке для масштабируемых систем – приемы проектирования и разработки для систем с большими объемами данных и количеством пользователей. Рекомендации помогут оптимизировать длительные процессы, обеспечить быстродействие операций, повысить скорость отклика системы и др.;
   - Объектная модель – описания классов и методов для работы с задачами и заданиями, отчетами, датой и временем, правами доступа, подписями и др. Также содержит описание прикладных модулей и схемы связей между их типами сущностей;
   - Примеры разработки – примеры, созданные на основе опыта прикладных разработчиков. Если вы только приступаете к разработке, то примеры помогут погрузиться и разобраться с доработками системы, чтобы написать собственный код. Не нужно долго придумывать что-то свое, так как можно делать по аналогии. Опытные разработчики найдут примеры доработки сложных процессов: различные варианты доработки задачи на согласование по регламенту и др;
   - Структура хранения данных – информация о том, как генерируются таблицы в структуре данных, форматы имен таблиц и колонок, примеры системных таблиц.
3. Учитывайте особенности разработки для Linux. Среда разработки устанавливается на компьютер разработчика с операционной системой Microsoft Windows, поэтому разработчик может собрать и опубликовать изменения в локальную систему только на Windows. Для тестирования своих доработок в системе на Linux нужны дополнительные действия.
4. Ознакомьтесь с особенностями разработки для мобильных решений.

Основные понятия

DeploymentTool

Утилита управления разработкой для публикации *пакета разработки* в *продуктивную систему* Directum RX и *инициализации* модулей и применения настроек бизнес-процессов. Устанавливается вместе с платформой системы.

Low-code

Подход, который предполагает использование предметно-ориентированной среды разработки для модификации системы. В Directum RX разработка ведется в Development Studio, которая позволяет задействовать редакторы свойств, действий, событий, отчетов, задавать свой код, а также подключать сторонние сборки.

No-code

Подход, который предполагает настройку системы в веб-клиенте визуальными средствами, без разработки кода. Для настройки используются готовые элементы: блоки схем бизнес-процессов, функции вычисляемых выражений и т.д. Как правило, настройку выполняют аналитики. В Directum RX таким образом настраиваются бизнес-процессы, шаблоны документов, структура компании и т.д.

OData

Открытый веб-протокол для запроса и обновления данных. Используется, чтобы с помощью HTTP-запросов обмениваться данными с Directum RX в формате JSON.

Авторизация

Процесс проверки прав пользователя перед выполнением действий в системе. Права могут быть выданы пользователю, группе или роли, в которую он входит.

Асинхронный обработчик

Обработчик, который позволяет:

- вынести длительные, ресурсоемкие операции, не требующие немедленного возврата результата, с веб-сервера на отдельный сервис **Worker**;
- одновременно выполнить вычисления в несколько потоков (распараллеливание вычислений).

Благодаря асинхронным обработчикам можно ускорить исполнение тяжелых операций, снизить нагрузку на веб-сервер, а также ускорить отклик системы для конечного пользователя.

Базовое решение

Стандартное решение Directum RX, которое разрабатывает вендор, компания Directum. Оно используется при модификации системы – на его основе, используя инъекцию, перекрытие и наследование, можно создавать собственные решения.

В среде разработки элементы базового решения доступны только для просмотра.

Базовый слой разработки

Слой разработки, на котором располагаются *базовые решения*. Например, это решения вендора. Модификация разработки под потребности компании выполняется на *рабочем слое*, при этом решения на базовом слое остаются неизменными.

Базовый тип сущности

Тип сущности, от которого в среде разработки можно создать наследников (новые типы сущностей). В клиентском приложении сущность базового типа создавать нельзя. Например, есть базовый тип справочника «Правило согласования» (ApprovalRuleBase). От него создано два наследника: «Правила согласования» (ApprovalRule) и «Правила согласования договоров» (ContractsApprovalRule). Пользователи в системе работают только с наследниками: конкретными правилами согласования. По умолчанию в среде разработки уже есть базовые типы сущностей: «Электронный документ» (Sungero.Content.ElectronicDocument), «Задача» (Sungero.Workflow.Task), «Задание» (Sungero.Workflow.Assignment), «Базовый договор» (ContractBase) и т.д. Вы можете разработать свои базовые типы, но только для документов и справочников.

Блок в схеме задачи

Визуальный элемент, который используется при разработке *схемы задачи*, например блок «Скрипт», «Мониторинг», «Задача», «Задание», «Начало процесса», «Конец процесса» и т.д. У каждого блока есть идентификатор, заголовок, набор свойств и событий.

Бэнд

Область страницы отчета, является контейнером для объектов отчета. Например, на бэнде «Данные» располагаются объекты «Таблица» и «Текст» для отображения данных отчета.

Валидация

Проверка состояния объекта перед дальнейшей работой с ним. С помощью валидации пользователю сообщается, что в данных есть ошибки, неточности, особенности, на которые он может повлиять.

В Directum RX есть несколько способов реализации валидации: сообщения, привязанные к свойствам; сообщения, привязанные к карточке; диалоги и всплывающие сообщения.

Виджет

Графический элемент, который используется для анализа и мониторинга работ в системе, быстрого доступа к спискам, папкам и отчетам. На виджетах отображаются:

- действия в виде ссылок, при нажатии на которые открывается список сущностей или выполняется другой программный код, написанный разработчиком;
- диаграммы, на которых отображается процент исполнительской дисциплины сотрудников, количество выполненных заданий и т.д.

«Жадная» загрузка свойств

Метод оптимизации программного кода для уменьшения количества обращений к веб-серверу. В среде разработки «жадная» загрузка используется для свойств, у которых установлен флажок **Загружать значения сразу**. При получении сущности с веб-сервера свойство загружается вместе с сущностью. Его можно сразу отобразить пользователю или использовать в вычислениях.

Зависимости модулей

Список модулей, программный код или элементы разработки которых используются в текущем модуле. Например, добавьте зависимость, если в своем модуле вы:

- использовали публичную функцию модуля «Настройки документооборота»;
- создали свойство с типом **Ссылка** и в поле ***Тип сущности** выбрали тип сущности «Сотрудники» из модуля «Компания».

Рекомендуется добавлять модуль в список зависимостей сразу после использования его элементов в своем модуле. Если зависимости не указаны, то при публикации возникнут ошибки.

Зависимости решений

Список модулей, которые используются в вашем решении. В него входят:

- модулей *базового решения*, которые перекрыты в вашем решении;
- модулей базового и ваших решений, программный код или элементы разработки которых используются в перекрытиях модулей или функциях вашего решения.

Список заполняется в редакторе решения.

Инициализация

Процесс приведения системы в состояние готовности к использованию. Функции инициализации модулей *базового решения* Directum RX запускаются автоматически после установки или обновления системы и используются для ее первоначальной настройки. Например, с помощью функций инициализации происходит:

- заполнение справочников **Валюты**, **Виды документов**, **Правила согласования** значениями по умолчанию;
- создание предопределенных групп пользователей и ролей – «Делопроизводители», «Ответственные за договоры», «Регистраторы входящих документов» и другие;
- выдача необходимых прав пользователям, группам или ролям на типы справочников, типы документов, вычисляемые папки и отчеты.

Интеграционный API

Серверные и разделяемые функции сущностей, модулей и решений Directum RX, а также репозитории типов сущностей, к которым можно обратится извне.

Источник данных (для отчетов)

Объект, из которого можно получить данные для формирования отчета. Источник данных описывает тип и структуру предоставляемых в отчете данных. Источники бывают двух видов: на основании типа сущности и SQL-источник данных.

Исходный код

Код, который вручную пишут разработчики в редакторах, например, при добавлении

события. А также код, который генерируется автоматически на основании параметров, выбранных в редакторах элементов разработки, и кода разработчиков.

Контрол

Элемент интерфейса, предназначенный для визуального отображения значений свойств *типов сущностей* в карточках. В проводнике системы контролы отображаются, например, в виде полей, выпадающих списков, флажков и переключателей.

Локальный веб-сервер

Веб-сервер, который обеспечивает работу *локальной системы*. В локальной системе разработчик проверяет свои изменения перед обновлением продуктивной системы. Локальный веб-сервер и локальная система устанавливаются только на компьютере разработчика.

Локальная система

Система Directum RX, в которой разработчик может проверить свои изменения до публикации в *продуктивную систему*. Устанавливается на компьютере разработчика. Чтобы запустить локальную систему, нужно в *среде разработки* на панели инструментов использовать действия из выпадающего списка **Запустить веб-клиент**.

Локализация системы

Перевод пользовательского интерфейса на другой язык. Например, если в системе планируется работать на русском и английском языках, то названия модулей, типов сущностей, полей, текстовые сообщения и другое необходимо указать на обоих языках. Для локализации используется редактор ресурсов.

Локальный репозиторий

Папка на компьютере разработчика, в которую копируются *исходные коды* из *удаленного репозитория*. По умолчанию на компьютере разработчика создано два локальных репозитория. В первом хранятся исходные коды разрабатываемых решений, во втором – исходные коды *базовых решений*. Разработчики изменяют исходные коды разрабатываемых решений. Чтобы изменения стали доступны другим разработчикам, они отправляют их в удаленный репозиторий.

Макет отчета

Шаблон для настройки внешнего вида отчета. Макет разрабатывается с помощью дизайнера отчетов FastReport.Net.

Модуль

Самостоятельная часть системы, решающая бизнес-задачи некоторой предметной области. Например, модуль «Договоры» позволяет автоматизировать работу с договорными документами. Модуль состоит из типов сущностей, вычисляемых папок и отчетов. В своем решении можно создать новый модуль или перекрыть существующий.

Метаданные

Структурированный набор данных об элементах системы. Метаданные позволяют описать тип сущности в одном месте и затем использовать это описание при написании бизнес-логики, визуальном проектировании карточек, создании структуры БД и др. На этапе разработки метаданные хранятся в отдельных файлах в формате JSON. Разработчик не правит файлы метаданных вручную, они меняются автоматически при написании кода в среде разработки.

Метаданные OData

XML-файл с информацией о функциях и репозиториях типов сущностей платформы и прикладной части Directum RX, которые доступны через сервис интеграции.

Наследование

Механизм, который позволяет создавать свои типы сущностей на основе *базовых типов*. При этом выделяют базовые типы сущностей платформы, базовые типы сущностей на уровне прикладной разработки и базовые типы сущностей, которые вы можете создать сами. Новые типы сущностей наследуют от базовых структуру данных, свойства, действия, форму, логику работы и отображение. Если в базовый тип внести изменения, то они появятся и в наследниках. Таким образом, сокращается время на его разработку, новый тип не нужно разрабатывать с нуля. Кроме этого, уменьшается дублирование кода между схожими типами сущностей.

Обложка модуля

Визуальное представление содержимого *модуля*. Обложка отображается при выборе модуля в проводнике Directum RX. Она предоставляет доступ к функциональности модуля и упрощает обучение. На обложке отображается общая информация о модуле и часто используемые действия с пояснениями к ним.

Объект авторизации

То, над чем выполняется действие. Например, запись справочника **Сотрудники**, тип документа «Договор», функция регистрации или публикации разработки.

Операция авторизации

Действие, которое *субъект* может выполнить над *объектом*. Например, для записи справочника **Сотрудники** это может быть операция удаления, для типа документа «Договор» – операция создания нового документа, для функций регистрации или публикации разработки – операция выполнения.

Пакет разработки

Архив с расширением ***.dat**, который может содержать исполняемые файлы, исходные коды, настройки бизнес-процессов, скрипты проверок, конвертации или описание ручных этапов публикации. Пакет разработки с исполняемыми файлами используется для импорта новых или измененных решений в продуктивную систему с помощью утилиты DeploymentToolCore. Пакет с исходными кодами используется для обмена разработкой. Кроме того, пакет разработки может передаваться как отладочный и использоваться для отладки программного кода, когда решение публикуется на сервере с операционной системой Linux.

Пакет развертывания

Архив с прикладной разработкой, который создается автоматически при публикации в среде разработки или при использовании утилиты DeploymentToolCore. В среде разработки пакет создается на основе *исходных кодов*, а при публикации с помощью утилиты – на основе *пакета разработки*.

В пакете развертывания содержатся бинарные файлы публикуемых модулей, XML-файл с описанием пакета, конфигурационные файлы разработки для веб-сервера и веб-клиента, а также интерфейсная сборка и скрипты публикации. Во время публикации пакет развертывания отправляется на веб-сервер.

Параметры отчета

Переменные, на основании которых формируется отчет. Значения параметров запрашиваются у пользователя с помощью диалоговых окон или вычисляются на основе других параметров и данных отчета.

Перекрытие

Механизм, который позволяет на своем слое разработки безопасно изменять модули и типы сущностей *базового решения*.

Например, чтобы в карточку справочника базового решения добавить контрол, нужно создать свое решение, перекрыть в нем необходимый тип справочника и добавить недостающий контрол на форму. В результате в клиентском приложении карточка справочника отображается с новым контролом (перекрытый справочник подменит справочник базового решения).

Так как перекрытые модули и типы сущностей хранятся на отдельном слое разработки, можно безопасно вносить в них любые изменения, не нарушая при этом логику работы базового решения. При необходимости изменения в перекрытых модулях и типах сущностей можно отменить или удалить.

Правило согласования

Справочник ApprovalRuleBase, в котором администраторы настраивают порядок обработки разных видов документов согласно *регламентам* компании. В карточке правила в Directum RX:

- задаются критерии подбора и фильтрации правил для конкретного документа;
- настраивается *схема согласования*. Т.е. кому придут задания на согласование и в каком порядке;
- указывается информация о том, кто будет ответственным за доработку документа.

Продуктивная система

Система Directum RX, с которой работают сотрудники компании. После модификации системы разработчик переносит изменения в продуктивную систему. Для этого нужно создать *пакет разработки* с исполняемыми файлами и обновить продуктивную систему с помощью утилиты *DeploymentToolCore*.

Публикация решений

Процесс установки новых решений, обновления существующих и удаления ранее установленных решений в *локальной* или *продуктивной* системе.

Для публикации в локальную систему нужно в *среде разработки* на панели инструментов использовать действие **Опубликовать...** или в главном меню в выпадающем списке **Сборка** выбрать пункт **Опубликовать выборочно...**.

Для публикации в продуктивную систему необходимо создать *пакет разработки* с исполняемыми файлами и использовать утилиту *DeploymentToolCore*.

Рабочий слой разработки

Слой разработки, на котором создается новое *решение*. На рабочем слое с помощью механизма *перекрытия* можно безопасно изменять модули и типы сущностей *базового решения*, а с помощью *наследования* – в модуле решения создавать свои типы сущностей на основе *базовых типов*.

Регламент

Порядок обработки документов, принятый и утвержденный в организации.

Репозиторий типа сущности

Основная точка входа для работы с сущностями. Репозиторий позволяет создавать сущности конкретного типа, получать одну или сразу несколько сущностей, а также содержит несколько вспомогательных методов: работа с ресурсами, работа с правами от типа сущности и т.д. Репозиторий для типа сущности находится в классе **<Код_компании>.<Имя_модуля>.<Имя типа сущности во множественном числе>**. Например, для типа сущности **Book** из модуля **MyModule** репозиторий будет называться **Sungero.MyModule.Books**.

Ресурсы

Статическое содержимое приложения: изображения и наборы *строк локализации*. В среде разработки для *локализации* пользовательского интерфейса используются редакторы прикладных и системных ресурсов.

Редактор системных ресурсов необходим для локализации названий *модулей*, *типов сущностей*, групп контролов, действий на ленте и т.д. Редактор прикладных – для локализации диалоговых окон, сообщений *валидации* и для добавления иконок в контролах состояния.

Решение

Часть прикладной разработки, которая объединяет между собой логически связанные модули, типы сущностей и отчеты. В решении можно создавать свои модули и перекрывать модули и типы сущностей базовых решений.

Решения не отображаются в проводнике Directum RX. Пользователь работает только с элементами решения: модулями, документами, задачами и т.д.

Сборка решений

Процесс, который выполнятся перед *публикацией* разработки в *локальную систему*. Необходим для проверки, что в изменениях нет ошибок и несовместимостей.

Свойство-коллекция

Свойство *типа сущности*, которое содержит набор дочерних свойств. В проводнике

системы свойство-коллекция может отображаться, например, в виде таблицы или выпадающего списка.

Сервис интеграции

Сервис системы Directum RX, который позволяет по веб-протоколу *OData* обмениваться данными с другими системами. Иначе говоря, сервис интеграции – это посредник между внешней системой и Directum RX. Сервис получает HTTP-запросы OData, «расшифровывает» их, определяет, что нужно сделать в Directum RX, получает ответы и передает их внешним системам.

Скрипт конвертации

Последовательность действий, которая вносит изменения в структуру и содержимое базы данных. Скрипты выполняются при обновлении системы Directum RX с версии на версию. Они изменяют базу данных таким образом, чтобы она соответствовала новым требованиям. В скриптах можно добавлять и удалять колонки таблиц и сами таблицы, создавать предопределенные данные, удалять индексы и т.д. Все скрипты конвертации делятся по:

- модулям или решениям;
- используемой СУБД (Microsoft SQL Server и СУБД на основе PostgreSQL);
- времени выполнения (скрипты **до публикации** и скрипты **после публикации**).

Скрипт проверки

Последовательность действий, которая проверяет, что данные модуля или решения можно конвертировать. Среди скриптов проверок бывают:

- **критичные**. Выполняются перед публикацией модуля. Если при проверке появятся ошибки, то публикация будет прервана;
- **некритичные**. Выполняются перед публикацией модуля, после критичных проверок. Если при проверке появятся ошибки, будет выведено соответствующее предупреждение. При этом публикация будет продолжена.

Скрипт ручного этапа

Последовательность действий для поддержки совместимости между разными базами данных. Например, в разработке для типа сущности добавили новое необязательное свойство и есть функция, которая синхронизирует его значение между базами данных. В базе данных Directum RX свойство появится после публикации, а в базе данных сторонней системы требуется создать свойство вручную. Для этого нужно написать соответствующий скрипт ручного этапа по созданию этого свойства в сторонней базе данных. Скрипты ручных этапов по времени выполнения делятся на скрипты, которые выполняются **до публикации** и **после публикации**.

Слой разработки

Разработанное *решение*, которое *перекрывает* типы сущностей или модули *базового решения*. Использование слоев в модели разработки позволяет безопасно адаптировать под себя версию системы вендора. Базовые решения и решения заказчика находятся на разных слоях разработки. Все, что создаете и изменяете в своих решениях, сохраняется только на *рабочем слое* и не влияет на *базовый слой*. Это облегчает процесс обновления решений на новые версии.

Например, в своем решении вы *перекрыли* справочник **Регионы** из стандартного

решения Directum RX и добавили в него контрол. В новой версии решения вендор тоже добавил в этот справочник новый контрол. После обновления системы в перекрытом справочнике останется ваш контрол и появится новый, который добавил вендор. При необходимости в перекрытом справочнике можно скрыть контрол вендора.

Среда разработки Directum RX (Development Studio)

Полноценная IDE с визуальными редакторами форм и процессов, современным редактором кода и инструментами отладки. Дает возможность тонко настраивать процессы в системе под сложные регламенты и расширять функциональность, если возможностей стандартной настройки недостаточно. Разработка ведется не с нуля, в поставку системы входят готовые модули, документы, справочники, задачи, задания и отчеты со стандартными свойствами и логикой поведения. Их можно менять с помощью *наследования* и *перекрытий*, что позволяет добавлять новые типы сущностей и модифицировать стандартные под потребности компании. Такой подход позволит расширять функциональность системы, сохраняя при этом все ее качества.

Сторонняя библиотека

Структурированный набор *сторонних сборок*, а также файлов конфигурации, отладочной информации и других файлов, необходимых для работы сборок.

Сторонняя сборка

.NET-сборка, которую разработчик может подключить к модулю и распространять вместе с ним. Подключение сторонних сборок расширяет функциональность системы. Например, можно подключить сборки для работы со сторонними сервисами или библиотеками для интеграции со сканерами штрихкодов (считывание кода товара, документа).

Строка локализации

Элемент интерфейса, который может отображаться для пользователя на разных языках. Например, заголовок справочника, название действия на ленте, текст сообщения *валидации*. Строки локализации можно использовать в программном коде вместо явного задания текста. В проводнике системы отображается то значение строки локализации, которое соответствует установленному в системе языку.

Субъект прав

Тот, кто выполняет действие. Например, пользователь, группа или роль, в которую он входит. В Directum RX все субъекты прав являются наследниками IRecipient.

Сущность

Объект, с которым пользователь работает в клиентском приложении Directum RX. Например, сущностью является конкретный документ, запись справочника, задача, задание.

Схема задачи

Графическое представление, которое состоит из последовательных этапов работ, описывающих бизнес-процессы компании. Настраивается для каждого типа задачи в узле «Схема». Согласно схеме в рамках задачи с помощью сервисов WorkflowProcessService и WorkflowBlockService создаются другие задачи, задания, уведомления и задания на приемку.

Схема правила согласования

Графическое представление *этапов согласования*, которые пройдет документ при отправке задачей на согласование по регламенту. Настраивается в проводнике Directum RX в карточке *правила согласования* на закладке «Схема». Согласно схеме по каждому этапу создаются задания соответствующих типов и вычисляются исполнители.

Схема правила согласования используется только в типе «Задача на согласование по регламенту», в которой содержатся задания для большинства ситуаций, возникающих при согласовании документа: регистрация, печать, подписание, согласование с руководителем, отправка контрагенту, рассмотрение и т.д. Какие конкретно из этих заданий будут созданы, зависит от схемы правила согласования.

Сценарий

Этап с вычислением, в котором выполняется предопределенная логика. Сценарий создается в среде разработки и добавляется на схему правила согласования. Например, таким образом создан сценарий «Рассмотрение несколькими адресатами». Сценарии упрощают разработку правил согласования, позволяют вызывать нужную логику на любом этапе согласования.

Тип сущности

Каждая сущность относится к определенному типу сущности. Выделяют типы документов, справочников, задач, заданий, заданий на приемку и уведомлений. У каждого типа сущности есть редактор, в котором его можно доработать под потребности компании. Например, в среде разработки создан тип документа «Договор». В его редакторе созданы свойства и действия, настроена форма карточки, разработана логика работы сущности и т.д. В клиентском приложении пользователи работают с сущностями данного типа – конкретными договорами.

Точка останова

Маркер, которым помечается строка кода. При отладке выполнение кода приостановится, когда будет достигнута строка с маркером.

Удаленный репозиторий

Папка на сервере Git, в которой хранятся оригиналы *исходных кодов*. По умолчанию на сервере создано два удаленных репозитория. В первом хранятся исходные коды разработанных решений, во втором – исходные коды *базовых решений*. Разработчики отправляют в удаленные репозитории изменения и забирают из них обновления. Исходные коды в репозитории с базовыми решениями изменяются только при обновлении системы на новую версию.

Условие согласования

Визуальный элемент, который используется для создания ветвлений в *схеме правила согласования*. Информация об условиях хранится в справочнике ConditionBase. По умолчанию справочник содержит предопределенные условия, например «Сумма», «Валюта», «Адресат», «Вид документа» и т.д. Условия позволяют в зависимости от свойств конкретного документа или состава участников согласования более гибко настраивать последовательность этапов в *правиле*. Например, у документа могут быть разные этапы согласования в зависимости от того, кто является адресатом документа.

Разработчик может добавлять новые условия. При этом нужно учитывать рекомендации.

Фоновый процесс

Процесс для выполнения работ в Directum RX, которые запускаются по заданному расписанию. Например, фоновые процессы используются для автоматической рассылки уведомлений о завершении сроков действия договоров или периодической синхронизации данных с сервисами обмена электронными юридически значимыми документами.

Фоновые процессы настраиваются в редакторе модуля в узле «Фоновые процессы». Вы можете изменять процессы стандартного решения Directum RX, отключать и создавать свои.

Этап согласования

Визуальный элемент, который используется при настройке *схемы правила согласования*. Информация об этапах хранится в справочнике ApprovalStage. Справочник по умолчанию содержит этапы, которые используются в предопределенных правилах согласования. У каждого этапа есть параметры и настройки, от которых зависит поведение конкретных заданий в рамках задачи на согласование по регламенту.

Разработчик может с помощью функций инициализации создавать свои предопределенные этапы. Вносить изменения в сам тип справочника ApprovalStage не рекомендуется, так как это может повлиять на работоспособность задачи на согласование по регламенту.

Процесс разработки

Базовое решение Sungero.Directum RX – стандартная прикладная часть системы Directum RX, с помощью которой автоматизированы процессы работы с официальными документами классического делопроизводства, финансово-хозяйственными документами и договорами, документами для совещаний и проектов, обмен электронными юридически значимыми документами с контрагентами и многое другое.

Прикладную часть системы легко настроить под особенности организации без программирования. Однако, если требуется глубокая адаптация под бизнес-задачи, например, из-за специфики отрасли, тогда адаптируйте базовое решение под себя с помощью среды разработки.

Порядок модификации элементов разработки и публикации их в продуктивную систему представлен на схеме. Для проверки и отладки изменений используется отдельная *локальная система* Directum RX. Пользователи при этом работают в *продуктивной системе*.

1. Запустите среду разработки. Перед тем, как вносить изменения, получите последнюю версию исходного кода с сервера. Для этого на панели инструментов нажмите на кнопку **Получить изменения с сервера**. Это уменьшит число конфликтов при сохранении изменений.
2. Создайте свое решение, в котором будете выполнять все доработки:
   - перекройте модули и типы сущностей *базовых решений*, чтобы внести в них изменения;
   - разработайте свои модули и отчеты, создайте типы сущностей с помощью механизма наследования. При разработке учитывайте, что некоторые элементы быстрее создать копированием существующих. Так можно создать отчет, панель фильтрации в списках и вычисляемых папках, свойства и действия типов сущностей. Также вы можете добавить сторонние компоненты, чтобы с их помощью адаптировать интерфейс веб-клиента под потребности компании.

   ПРИМЕЧАНИЕ. Если используются мобильные приложения Directum Solo или Directum Jazz, следуйте рекомендациям по модификации прикладной разработки. Это необходимо, чтобы реализовать поддержку собственной разработки в мобильных приложениях.
3. Соберите решения, чтобы проверить, что код написан верно и в нем нет несовместимостей. Если сборка выполнилась успешно, зафиксируйте и отправьте изменения на сервер Git, чтобы другие разработчики увидели его.
4. Опубликуйте решения в локальную систему Directum RX для проверки работы изменений. Откроется проводник веб-клиента.
5. Проверьте в локальной системе опубликованные изменения. При этом вы можете использовать разные конфигурации: упрощенную, чтобы быстро проверить разработанное решение, или полную для просмотра того, как решение работает в условиях, близких к реальным. Подробнее см. в руководстве администратора, раздел «Конфигурирование настроек» (Windows, Linux).

При необходимости выполните отладку, исправьте разработку и опубликуйте снова.

6. Создайте настройки по умолчанию для бизнес-процессов и интерфейса.
7. Создайте пакет разработки с исполняемыми файлами для обновления продуктивной системы.

   ВАЖНО. Перед публикацией создайте резервную копию БД. Это позволит восстановить данные при возникновении ошибок.
8. Опубликуйте пакет разработки в продуктивную систему с помощью Directum Launcher.
9. Для работы с обновленной продуктивной системой пользователь перезапускает клиентское приложение. Чтобы у пользователя появлялось сообщение о перезапуске клиентского приложения, установите соответствующий флажок в редакторе решения, в узле «Публикация».
10. Изучите особенности разработки для Linux, если нужно протестировать модификации в этой операционной системе.

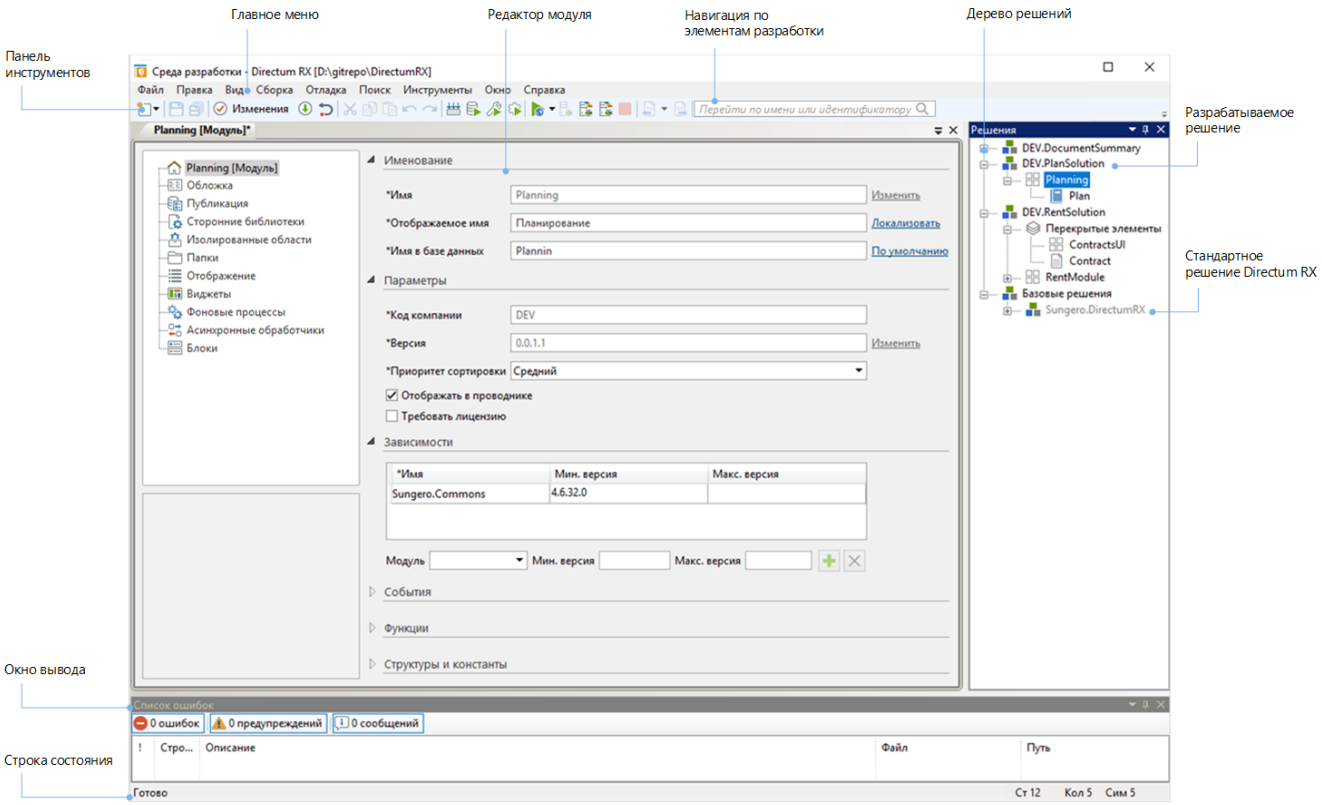
С помощью среды разработки вы также можете настроить интеграцию Directum RX с другими информационными системами вашей компании и таким образом создать единую корпоративную экосистему. Для обращения к внешним приложениям на компьютере или к локальным файлам в операционной системе можно использовать утилиту RxCmd.

Среда разработки Directum RX

Среда разработки Directum RX (Development Studio) – это инструмент, с помощью которого можно:

- создавать решения, модули и отчеты;
- создавать типы сущностей с помощью механизма *наследования*;
- *перекрывать* модули и типы сущностей *базовых решений*, чтобы вносить в них изменения.

Окно среды разработки выглядит следующим образом:



Главное меню. Содержит пункты меню, предназначенные для выполнения различных действий в среде разработки. Например, в пункте меню **Инструменты**, **Параметры...** доступны действия для настройки учетных записей Git, цветовой схемы и шаблонов кода.

Редактор модуля (решения, типа сущности или отчета). У каждого элемента разработки есть свой редактор. В редакторе задаются настройки, добавляются функции и события. Чтобы его открыть, в дереве решений щелкните два раза левой кнопкой мыши по названию решения, модуля, типа сущности или отчета.

Навигация по элементам разработки. Поле используется, чтобы быстро перейти к любому элементу разработки из дерева решений и открыть его редактор. Введите в поле имя, отображаемое имя или локализованный вариант отображаемого имени элемента разработки, например, **ApprovalTask**, **Rule-based approval task** или **Задача на согласование по регламенту**. В поле также можно указать идентификатор элемента разработки. Например, это полезно, если идентификатор зафиксирован в сообщении лог-файла и нужно перейти к соответствующем редактору. В результате поиска появится список подходящих элементов. После выбора элемента откроется его редактор. Чтобы быстро переместить курсор в поле навигации, используйте клавиши CTRL+N.

Дерево решений. В дереве отображается список разрабатываемых и *базовых решений*. В узлах разрабатываемых решений отображаются перекрытия элементов базовых решений и созданные модули, типы сущностей и отчеты. Базовые решения выделены серым цветом и доступны только для просмотра.

Для элементов дерева доступно контекстное меню, с помощью которого можно открыть на редактирование или чтение редактор элемента, открыть SQL-запросы, создать или удалить элемент разработки. Состав пунктов контекстного меню зависит от выбранного элемента.

Окно вывода. В окне отображается дополнительная информация и предупреждения, например результаты поиска.

Строка состояния. В строке отображается информация о выполненных действиях в среде разработки, например сообщения об успешной *сборке*, *публикации* решений, о том, что изменения отправлены на сервер Git.

Панель инструментов. Содержит кнопки для выполнения основных действий. Список часто используемых действий приведен в таблице:

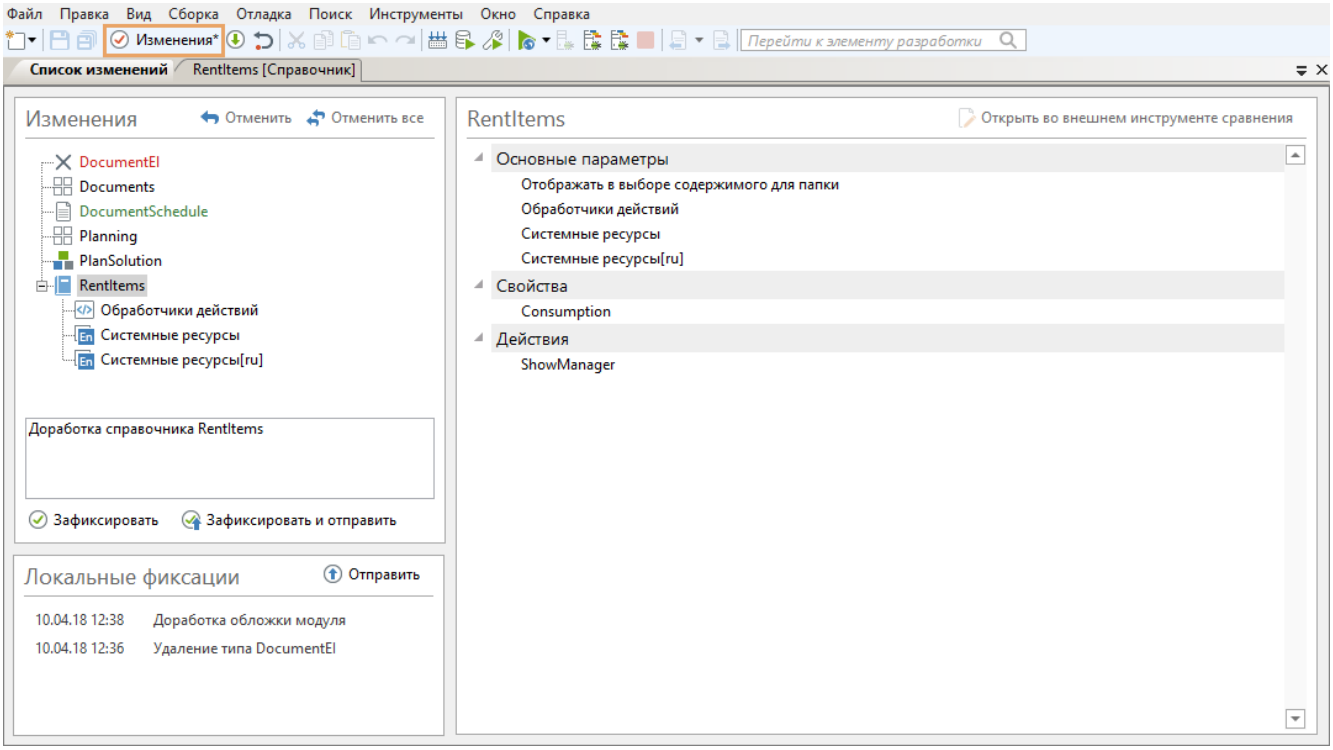
Действие	Описание
Создание элементов разработки	
Создать	Создание решений, модулей, отчетов и типов сущностей: справочников, документов, задач, заданий, заданий на приемку, уведомлений. Доступность пунктов зависит от того, какой элемент разработки выделен в дереве решений
Сохранение и отмена изменений	
Изменения*	Просмотр изменений перед их фиксацией, отправкой на сервер или отменой. После создания, изменения или удаления элементов разработки рядом с названием кнопки появляется символ *
Получить изменения с сервера	Копирование *исходных кодов* с сервера Git на компьютер разработчика
Сбросить локальные изменения и фиксации	Отмена локальных изменений и фиксаций. Кнопка удаляет все локальные изменения. Если разработка до этого не была отправлена на сервер Git, то она будет потеряна
Публикация	
Собрать	Сборка измененных решений и модулей. Выполняется, например, для проверки корректности кода разработки
Опубликовать...	Публикация изменений в решениях и модулях, опубликованных ранее. Выполняется для проверки работоспособности решений в локальной системе Directum RX
Запустить инициализацию	Инициализация опубликованных модулей в локальной системе Directum RX
Применить настройки на сервер	Загрузить настройки по умолчанию для бизнес-процессов на сервер из исходных кодов
Проверка и отладка	
Запустить клиент	В выпадающем списке доступны пункты: • **Запустить веб-клиент**. Откроется веб-клиент локальной системы Directum RX от имени пользователя, под которым система запускалась в последний раз; • **Запустить фоновый процесс...**. Откроется окно для принудительного запуска или отладки фоновых процессов
Отладить веб-сервер Отладить общий сервис Отладить сервис интеграции	Отладка веб-сервера, общего сервиса и сервиса интеграции. Во время отладки для изменения доступен только проверяемый элемент разработки. Редакторы остальных элементов открываются на просмотр, их изменение станет доступно после прерывания отладки
Остановить отладку	Завершение отладки

При запуске Directum RX Development Studio в фоне генерируется прикладной код. В это время часть объектов в дереве решений доступна только для просмотра. С теми объектами, которые загрузились, можно работать, остальные отображаются серым цветом. Сборка решений, экспорт и импорт пакетов разработки, а также публикация становятся доступны только после полного завершения фоновой генерации метаданных.

ПРИМЕЧАНИЕ. Если после закрытия Development Studio содержимое локального репозитория не менялось, и версия среды разработки не обновлялась, то при следующем запуске этап генерации исходных кодов пропускается. Также для ускорения запуска и оптимизации работы Development Studio по умолчанию включена серверная сборка мусора.

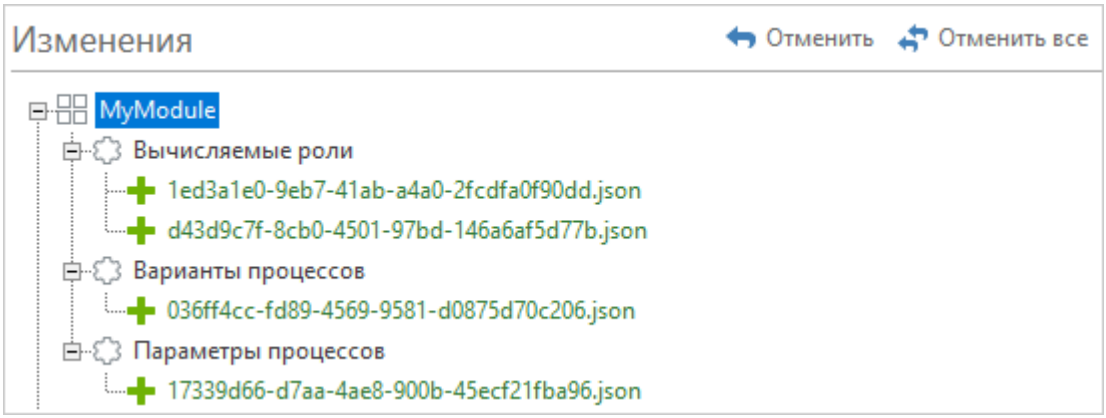
Сохранение изменений

После создания, изменения или удаления элементов разработки на панели инструментов рядом с кнопкой **Изменения** появляется символ *. При нажатии на кнопку открывается список изменений:



В области «Изменения» в виде дерева отображаются измененные решения, модули, отчеты и типы сущностей. Новые элементы выделяются зеленым цветом, удаленные – красным, измененные – черным. Дочерние узлы появляются только у измененных элементов, в которых дорабатывался программный код, скрипты публикации, sql-запросы или локализация.

Кроме того, в области отображаются измененные файлы с настройками бизнес-процессов:



Изменения можно просмотреть, отменить, зафиксировать или отправить на сервер Git.

ПРИМЕЧАНИЕ. Информация об изменениях, отправленных на сервер, записывается в историю. По комментариям в истории можно выяснить, что было изменено, кем и когда. Чтобы посмотреть историю, в главном меню последовательно выберите пункты **Инструменты**, **История изменений**.

Просмотр изменений

Для просмотра изменений выделите в дереве решение, модуль, отчет или тип сущности. В области справа отобразятся все его изменения. Например, свойства, форма, основные параметры и т.д. Если в дереве изменений выделить дочерний узел, то в области справа отобразятся изменения в программном коде. Новые строки кода выделяются зеленым цветом, а удаленные – красным.

ПРИМЕЧАНИЕ. Чтобы детальнее посмотреть, как изменился программный код после редактирования, используйте внешний инструмент для сравнения файлов. Для этого выделите дочерний узел в дереве изменений или выделите пункт в области справа и нажмите на кнопку **Открыть во внешнем инструменте сравнения**. По умолчанию откроется программа WinMerge. Вместо WinMerge можно использовать другой инструмент сравнения, например vsDiffMerge или KDiff3. Подробнее см. раздел «Настройка внешнего инструмента сравнения».

Фиксирование и отправка изменений

Перед отправкой изменений на сервер Git запустите сборку, чтобы проверить, что изменения написаны верно и в них нет несовместимостей.

СОВЕТ. Отправляйте изменения на сервер Git как можно чаще. При отправке небольшого количества изменений, конфликты легче исправлять.

Если вы внесли изменения, которые нужны для работы других разработчиков, зафиксируйте и отправьте изменения на сервер Git. Для этого:

1. Введите комментарий с кратким описанием изменений. Комментарий будет отображаться в истории изменений.
2. Нажмите на кнопку **Зафиксировать и отправить**.

Перед отправкой изменений на сервер Git среда разработки копирует на компьютер разработчика последнюю актуальную версию *исходного кода* и сравнивает изменения. Если внесены схожие изменения в один и тот же элемент разработки, возникнет конфликт. Среда разработки разрешит конфликт автоматически либо предложит разработчику разрешить конфликт вручную.

Если часть разработки готова, и вы хотите проверить, что она совместима с изменениями на сервере:

1. Введите комментарий с кратким описанием изменений и нажмите на кнопку **Зафиксировать**.

   В области «Локальные фиксации» появится запись с комментарием и временем создания фиксации.
2. На панели инструментов нажмите на кнопку **Получить изменения с сервера**.
3. Если возникли конфликты и они разрешены автоматически, проверьте, что внесенные изменения верны. Оставшиеся конфликты разрешите вручную. Затем отправьте изменения на сервер. Если конфликтов нет, то вы можете сделать несколько фиксаций, а после отправить их все вместе на сервер.

   ВАЖНО. Чтобы сократить количество конфликтов, рекомендуется сразу после каждой фиксации отправлять изменения на сервер.
4. Чтобы отправить изменения на сервер, в области «Локальные фиксации» нажмите на кнопку **Отправить**. Изменения, отправленные на сервер, другие разработчики смогут скопировать на свой компьютер.

Отмена изменений

Чтобы отменить изменения во всех элементах разработки, которые отображаются в области «Изменения», нажмите на кнопку **Отменить все**.

Чтобы отменить изменения в конкретном решении, модуле, типе сущности или отчете, выделите элемент в дереве и нажмите на кнопку **Отменить**. Будут удалены все изменения, которые вы сделали в элементе разработки. Например, свойства, действия, основные параметры. Чтобы отменить только часть изменений, вернитесь в соответствующий редактор и удалите доработки вручную.

ВАЖНО. Если элемент разработки был переименован, то в списке изменений отображаются два элемента: с предыдущим и новым именем. Чтобы вернуть прежнее имя, по очереди выделите элемент с новым и предыдущим именем и нажмите на кнопку **Отменить**. Если отменить изменение только для одного элемента, при дальнейшей работе возникнет ошибка.

Разрешение конфликтов

Если в один и тот же элемент разработки внесены схожие изменения, возникает конфликт. Это может произойти при слиянии изменений в одной ветке разработки или при слиянии нескольких веток с помощью Git Extensions.

При получении изменений с сервера Git и перед их отправкой на сервер среда Directum RX Development Studio копирует на компьютер разработчика последнюю актуальную версию *исходного кода* и сравнивает изменения. Наличие конфликтов также проверяется при каждом запуске среды разработки, большинство из них разрешаются автоматически. Внешние инструменты сравнения используются в оставшихся случаях, когда среда разработки предлагает разрешить конфликты вручную.

Чтобы минимизировать количество конфликтов рекомендуется:

- всегда получать исходный код с сервера Git перед тем, как вносить изменения в разработку;
- не изменять одновременно с другими разработчиками один и тот же отчет, схему задачи, форму, ленту, набор виджетов, внешний вид списка и папки, а также внешний вид вычисляемой папки;
- после добавления, переименования, удаления или изменения элементов разработки сразу отправлять изменения на сервер Git;
- перед переключением на другую ветку закрывать среду разработки.

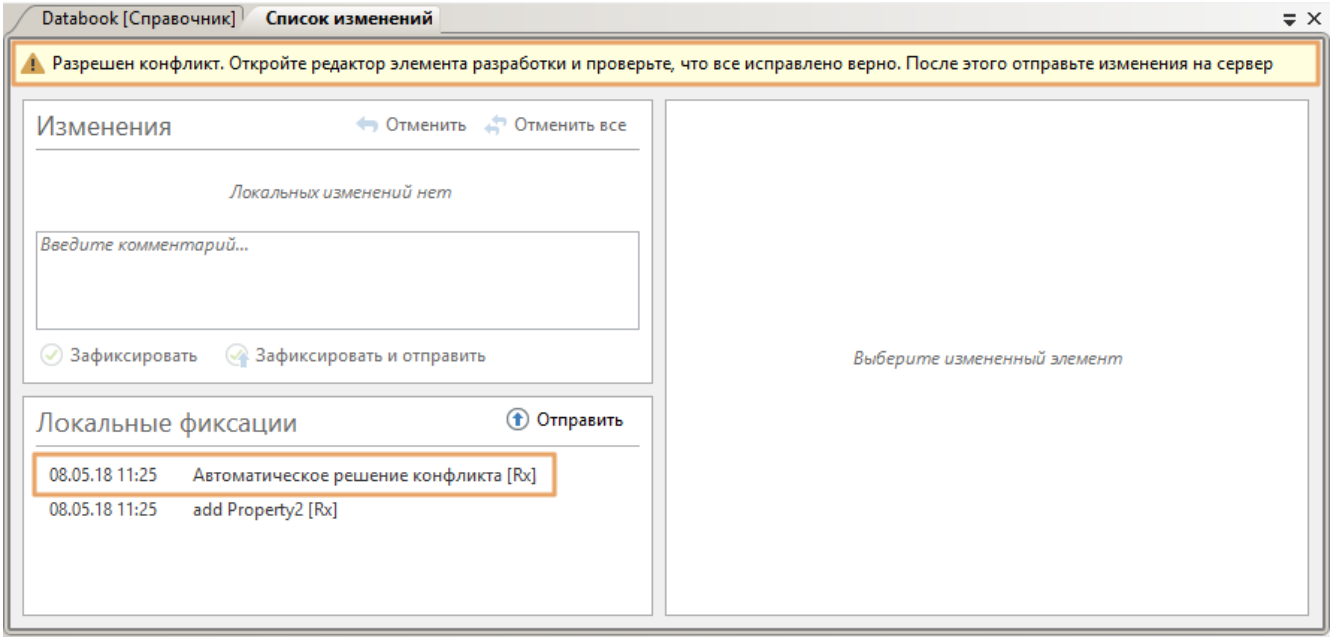
Автоматическое разрешение конфликтов

Конфликты разрешаются автоматически:

- при слиянии файлов ресурсов (*.resx);
- при слиянии файлов метаданных (*.mtd), например, если одновременно добавляются свойства или публичные функции. Кроме того, конфликты разрешаются, когда в структуре метаданных важен порядок элементов или зависимость между ними. К таким случаям относятся изменения, вносимые в разработку виджетов, вычисляемых папок и т.д.

ПРИМЕЧАНИЕ. Конфликты, которые возникают при слиянии файлов исходного кода (*.cs), разрешаются только вручную.

Если конфликты разрешены автоматически, предупреждение и информация о них выводится в списке изменений:



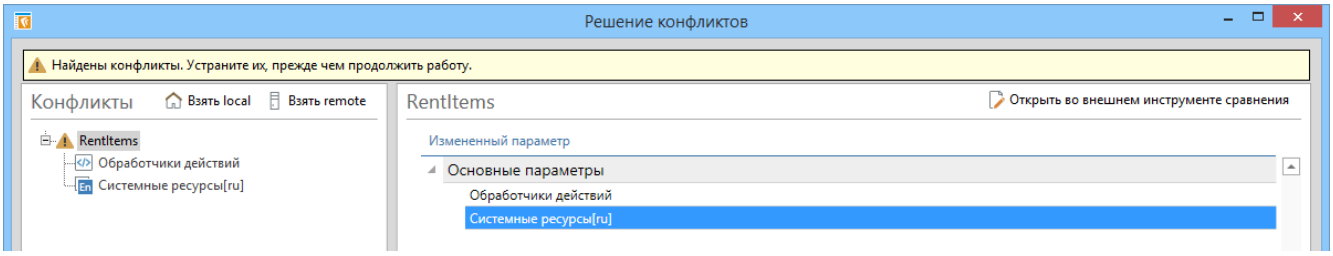
В этом случае:

1. Откройте редактор элемента разработки и проверьте, что автоматически внесенные изменения верны. Это рекомендуется сделать, чтобы гарантировать корректность кода. Определить измененные элементы разработки можно в программе Git Extensions на закладке Diff.
2. Если изменения верны, отправьте изменения на сервер. Если нет, исправьте их и затем отправьте.

Ручное разрешение конфликтов

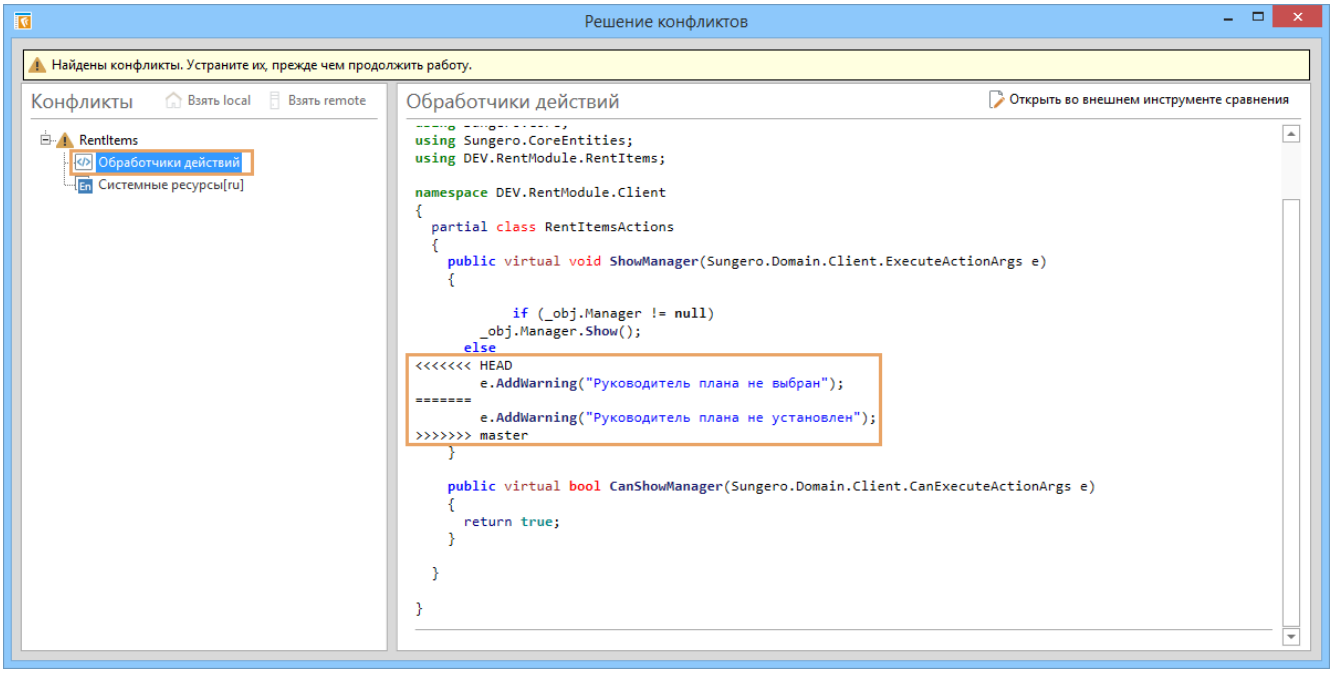
Если открылось окно с конфликтами:

1. В области «Конфликты» выделите решение, модуль, отчет или тип сущности. В области справа отобразятся параметры, в которых возник конфликт. Например, свойства, форма, обработчики событий, системные *ресурсы* и т.д.:



2. В дереве у элемента разработки отображаются дочерние узлы, если конфликт возник в программном коде, скриптах публикации, sql-запросах или локализации.

   Чтобы сравнить ваши изменения с изменениями, сохраненными на сервере Git, выделите узел в дереве. Справа отобразятся строки, в которых возник конфликт:



   Строки с конфликтом начинаются с символов <<<<<<< HEAD и заканчиваются символами >>>>>>> master. До знаков ======= отображаются ваши изменения, после – изменения, сохраненные на сервере. Строки в области справа доступны только для просмотра.

3. Чтобы разрешить конфликты в программном коде, объедините изменения во внешнем инструменте для сравнения. Для этого выделите дочерний узел в дереве и нажмите на кнопку **Открыть во внешнем инструменте сравнения**. По умолчанию откроется программа WinMerge. В ней отображаются изменения, которые:

   - были на сервере до внесения изменений вами и другим разработчиком;
   - добавил и отправил на сервер другой разработчик;
   - внесли вы.

   ПРИМЕЧАНИЕ. Вместо WinMerge можно использовать другой инструмент сравнения, например vsDiffMerge или KDiff3. Подробнее см. раздел «Настройка внешнего инструмента сравнения».

4. Если после разрешения конфликтов в коде остались другие конфликты, например, в схеме задачи, выберите, какие изменения использовать: ваши или сохраненные на сервере. Для этого выделите решение, модуль, отчет или тип сущности в дереве. Чтобы для разрешения остальных конфликтов использовать ваши изменения нажмите на кнопку **Взять local**. Чтобы использовать изменения с сервера – **Взять remote**.

   СОВЕТ. Если конфликты возникли в изменениях формы, ленты, отчета, схемы задачи, набора виджетов, внешнего вида списка, папки или вычисляемой папки, возьмите изменения с сервера. После разрешения всех конфликтов и фиксации изменений, откройте редактор элемента и при необходимости добавьте свои изменения заново.

5. После разрешения всех конфликтов появится окно с соответствующим сообщением. Нажмите в нем на кнопку **Зафиксировать**. Окно разрешения конфликтов закроется. В окне «Список изменений» в области «Локальные фиксации» появится запись с вашим комментарием к фиксации и запись о разрешении конфликта.
6. Если после разрешения конфликтов в окне вывода среды разработки появились сообщения об ошибках, исправьте их.

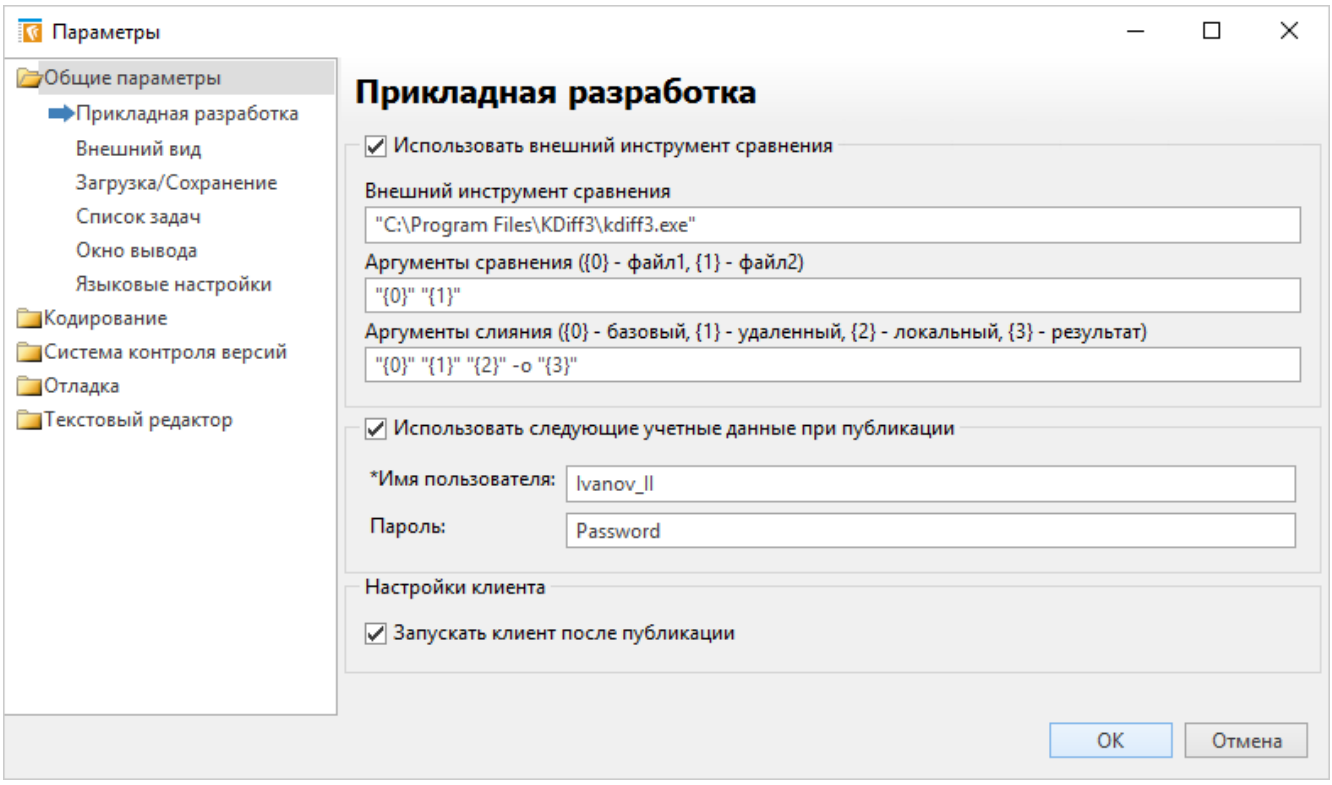
   СОВЕТ. В окне вывода щелкните два раза левой кнопкой мыши по тексту сообщения. Откроется редактор элемента разработки, в котором возникла ошибка.
7. Откройте редакторы элементов разработки и проверьте изменения, которые внесли при разрешении конфликтов.
8. Отправьте изменения на сервер. Для этого в окне «Список изменений», в области «Локальные фиксации» нажмите на кнопку **Отправить**.

Настройка внешнего инструмента сравнения

По умолчанию в качестве внешнего инструмента сравнения изменений используется программа WinMerge. Она автоматически устанавливается вместе со средой разработки и не требует дополнительной настройки. Вместо WinMerge можно использовать другие программы, например vsDiffMerge или KDiff3.

Чтобы настроить внешний инструмент сравнения:

1. В главном меню последовательно выберите пункты **Инструменты**, **Параметры...**.
2. В узле **Прикладная разработка** установите флажок **Использовать внешний инструмент сравнения** и заполните поля:



Внешний инструмент сравнения. Путь к исполняемому файлу программы.

Аргументы сравнения. Аргументы, которые задают правило сравнения двух файлов.

Аргументы слияния. Аргументы, которые задают правило слияния файлов.

Подробнее об аргументах сравнения и слияния см. в документации используемого инструмента сравнения.

3. Нажмите на кнопку **OK**.

Настройка vsDiffMerge

Внешний инструмент сравнения:
"C:\Program Files\Microsoft Visual Studio\2022\Community\Common7\IDE\CommonExtensions\Microsoft\TeamFoundation\Team Explorer\vsdiffmerge.exe"

Аргументы сравнения: "{0}" "{1}" /t

Аргументы слияния: "{1}" "{2}" "{0}" "{3}" /m /t

Настройка KDiff3

Внешний инструмент сравнения: "C:\Program Files\KDiff3\kdiff3.exe"

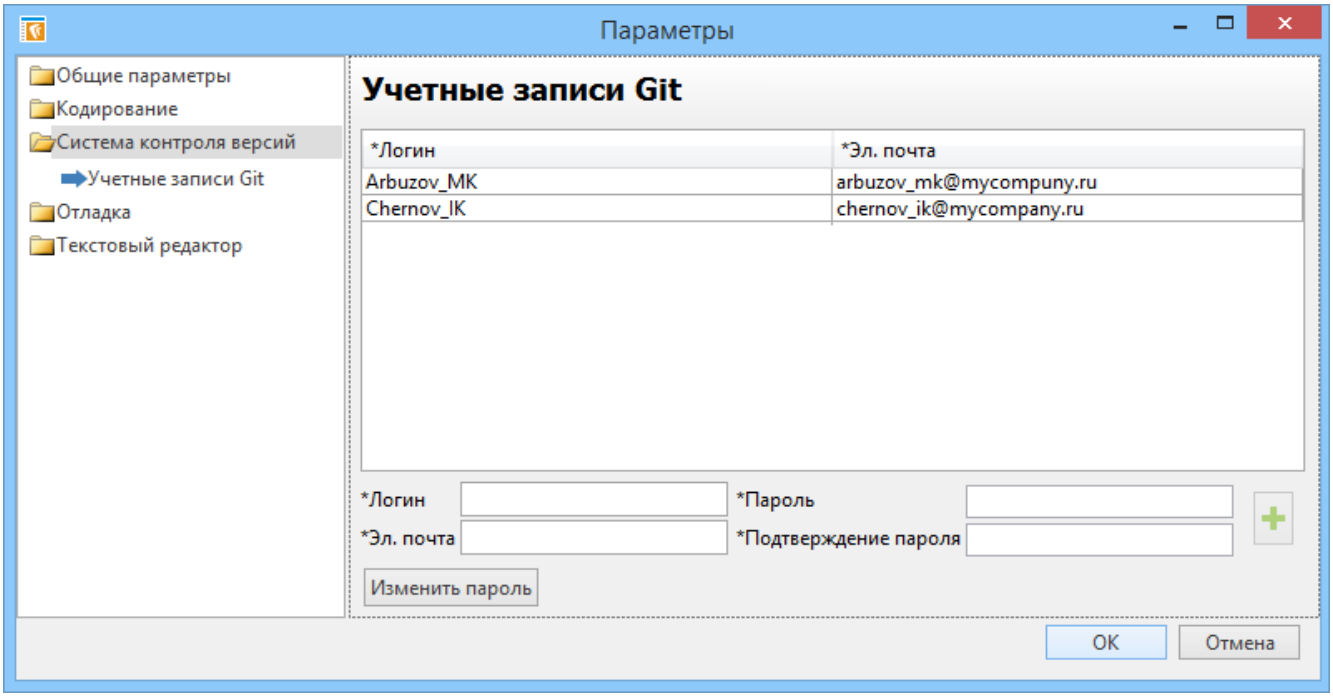
Аргументы сравнения: "{0}" "{1}"

Аргументы слияния: "{0}" "{1}" "{2}" -o "{3}"

Настройка учетных записей Git

Для совместной работы в Git нескольких прикладных разработчиков нужно установить систему управления репозиториями и создать для всех разработчиков учетные записи. По умолчанию используется система управления репозиториями Gitea. Она устанавливается с помощью программы установки. Первая учетная запись создается при установке Gitea, а остальные – в параметрах:

1. В главном меню последовательно выберите пункты **Инструменты**, **Параметры...** и перейдите в узел **Система контроля версий**:



2. Заполните поля ***Логин**, ***Эл. почта**, ***Пароль** и ***Подтверждение пароля**.

   ВАЖНО. Логин может состоять из букв латинского алфавита, цифр и символов «.», «_», «-». Пароль может состоять из букв латинского алфавита, цифр и любых символов. Минимальная длина пароля – 6 символов.

3. Чтобы добавить запись, нажмите на кнопку .

При первом запуске среды разработки появится окно аутентификации. Введите логин и пароль учетной записи Git. При следующем запуске окно аутентификации не появится. При необходимости разработчик может изменить пароль в параметрах среды разработки с помощью кнопки **Изменить пароль**. После смены пароля нужно вновь пройти аутентификацию.

Для работы с Git можно использовать любую другую систему управления репозиториями. В этом случае нужно самостоятельно установить систему и настроить учетные записи.

Использование шаблонов кода

При написании кода можно использовать шаблон – фрагмент кода, который автоматически генерируется при вводе определенного набора символов. Шаблоны кода позволяют упростить и ускорить процесс разработки, избавляют от необходимости каждый раз прописывать стандартные конструкции. Например, для автогенерации оператора **if ... else** достаточно ввести в редакторе кода строку «ifelse» и нажать клавишу TAB.

Чтобы добавить шаблон кода:

1. Введите набор символов, который соответствует шаблону кода.

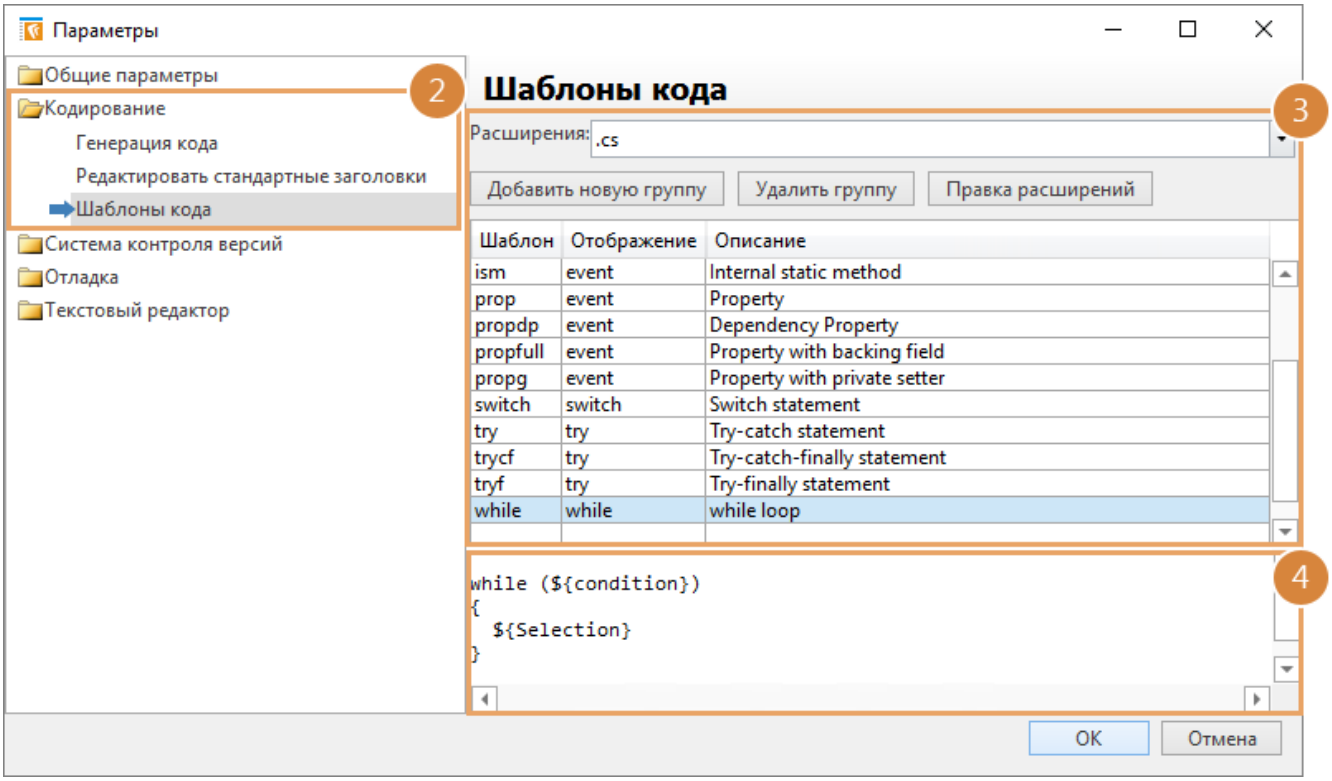
   ПРИМЕЧАНИЕ. Чтобы посмотреть список всех шаблонов, воспользуйтесь горячими клавишами CTRL+J.

2. Зажмите клавишу TAB.

В результате в редактор кода добавится соответствующая шаблону кода конструкция.

Если нужного шаблона нет, вы можете добавить свой. Для этого:

1. В главном меню последовательно выберите пункты **Инструменты**, **Параметры...**.
2. В открывшемся окне выберите пункт **Кодирование**, затем узел «Шаблоны кода»:



3. Проверьте, что в выпадающем списке **Расширения** выбрано значение **.cs**, и заполните поля:

   Шаблон. Строка, которая в сочетании с клавишей TAB добавляет фрагмент кода.

   Описание. Краткая информация о шаблоне кода. Отображается при вызове списка всех доступных шаблонов.

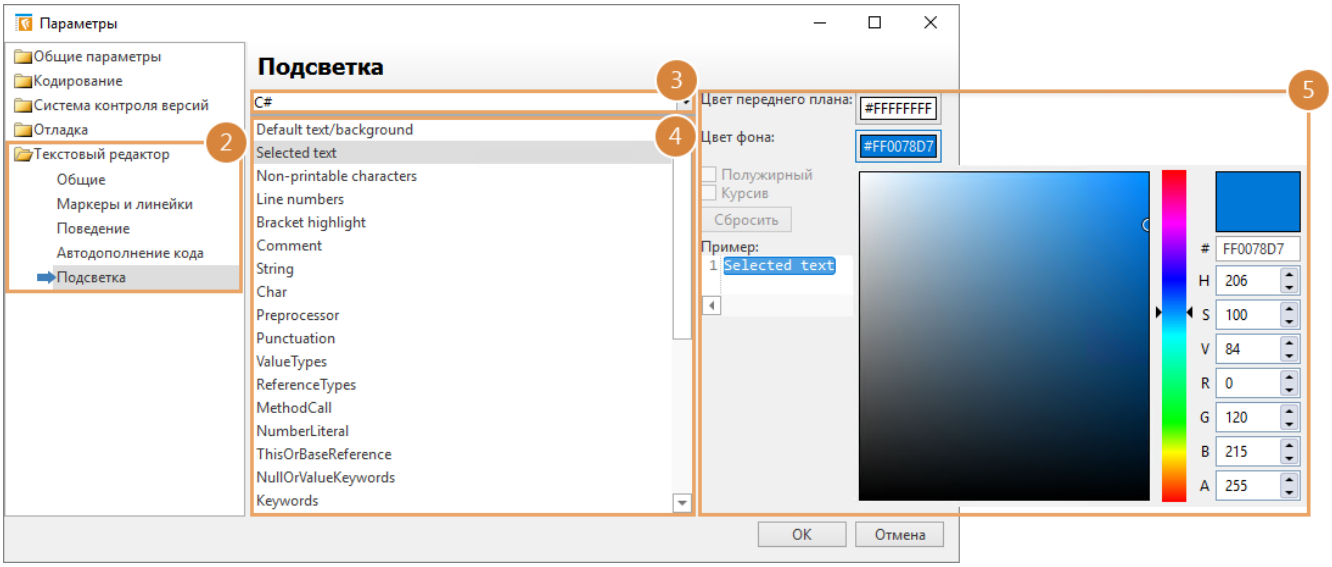
4. Задайте фрагмент кода в поле под таблицей. Он добавляется при вызове шаблона кода.
5. Нажмите на кнопку **OK**.

Настройка цветовой схемы для редактора кода

В редакторе кода уже есть стандартная подсветка. При необходимости ее можно изменить. Для точечных изменений используйте ручную настройку, а для более масштабных – импортируйте цветовую схему.

Ручная настройка

1. В главном меню последовательно выберите пункты **Инструменты**, **Параметры...**.
2. В открывшемся окне выберите пункт **Текстовый редактор**, затем узел «Подсветка»:



3. В выпадающем списке выберите значение **C#**. Отобразится список элементов кода для настройки.
4. Из списка выберите элемент кода, отображение которого вы хотите настроить.
5. Настройте цветовую схему. Для этого заполните поля:
   - **Цвет переднего плана** – цвет текста;
   - **Цвет фона**, которым будет выделяться фрагмент кода.

     ПРИМЕЧАНИЕ. Цвета можно задать с помощью палитры или кодировки.
   - **Полужирный**. Установите флажок, если к фрагменту кода нужно применить полужирное начертание;
   - **Курсив**. Установите флажок, если к фрагменту кода нужно применить курсив.
6. Нажмите на кнопку **ОК**.

Импорт цветовой схемы

1. Подготовьте цветовую схему.

   ПРИМЕЧАНИЕ. Пример готовой цветовой схемы можно скачать с сайта сообщества Directum Club из статьи «Цветовая схема для редактора кода в Sungero Development Studio».
2. Закройте среду разработки.

3. Создайте резервную копию файла DevelopmentStudioProperties.xml.

   Путь до файла: %APPDATA%\DIRECTUM Company\DirectumRX\Sungero Development Studio\DevelopmentStudioProperties.xml
4. Откройте исходный файл DevelopmentStudioProperties.xml.
5. Найдите тег **<ArrayOfCustomizedHighlightingColor>**.
6. Замените значения тега данными вашей цветовой схемы.

Поиск по коду и замена

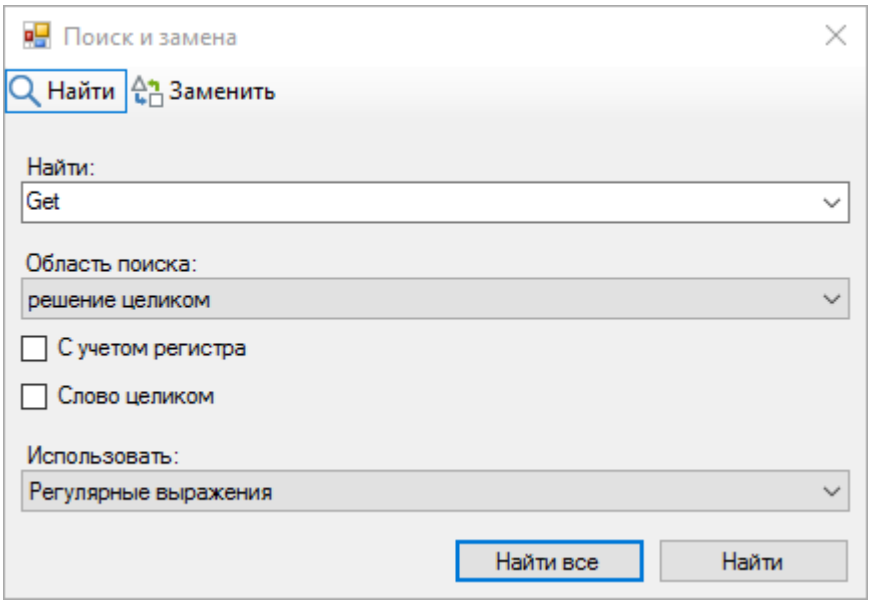
При работе с большим кодом для лучшей навигации рекомендуется использовать поиск по коду. Благодаря ему можно быстро найти нужный фрагмент, а также при необходимости его заменить.

Поиск

Чтобы запустить поиск по коду:

1. В главном меню последовательно выберите пункты **Поиск**, **Найти** или **Найти в решении**.

   Также можно воспользоваться горячими клавишами CTRL+F или CTRL+SHIFT+F.
2. В окне «Поиск и замена» заполните поля:



Найти. Фрагмент кода или название ресурса, который нужно найти.

Область поиска. Возможные значения:

- **текущий документ**. Значение по умолчанию;
- **выделенный фрагмент**;
- **все открытые документы**;
- **решение целиком**.

С учетом регистра. Установите флажок, если при поиске необходимо учесть регистр символов, введенных в поле **Найти**.

Слово целиком. Установите флажок, если необходимо найти значение поля **Найти** в том виде, как оно указано. Иначе в результатах поиска также отобразятся и его вхождения в другие слова.

Использовать дополнительные условия поиска. Возможные значения:

- **Стандартный поиск**. Текстовый поиск без специальных символов. Значение по умолчанию.

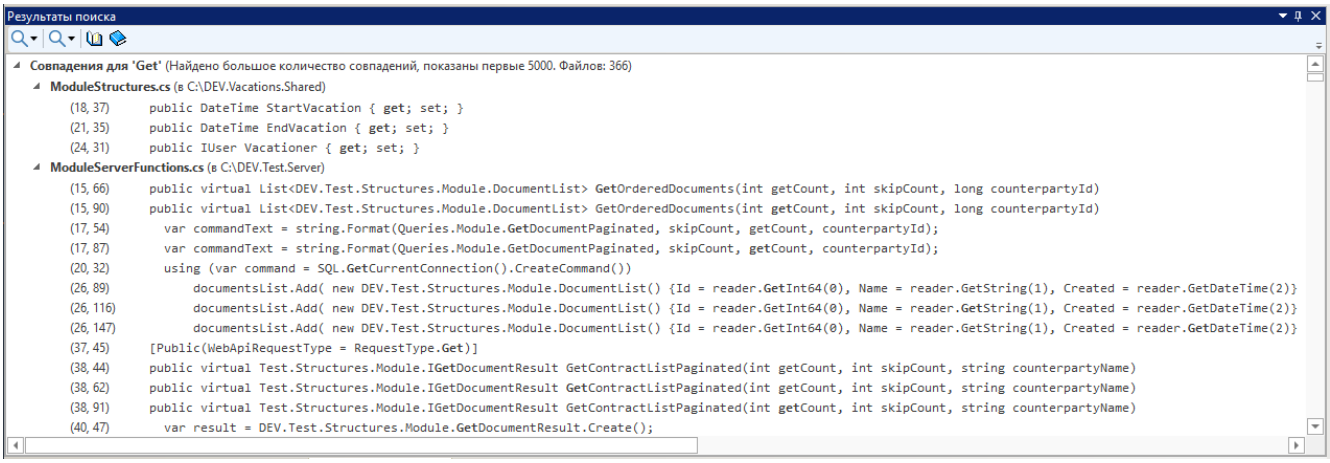
  Например, при поиске строки «Get» результатом может быть переменная **getCount** или функция **GetCurrentConnection**.

- **Регулярные выражения**. Поиск с использованием сложных конструкций. Например, если в поле **Найти** ввести строку «D[a-z](10)ent», найдутся все вхождения, начинающиеся с D и заканчивающиеся на ent. Между ними допускается не более 10 символов от a до z. Результатами поиска может быть как переменная **Document**, так и справочник **Department**.

  Подробнее о регулярных выражениях см. в документации Microsoft статью «Элементы языка регулярных выражений – краткий справочник».

- **Знаки подстановки**. При поиске используются знаки подстановки – **?** или *****, вместо которых в значении поля **Найти** подставляется один символ или их последовательность соответственно.

3. Запустите поиск. Для этого нажмите на одну из кнопок:
   - **Найти** – курсор переместится к подходящему значению. Если результатов поиска нет, появится сообщение об этом;
   - **Найти все** – результаты поиска отобразятся в окне вывода:



Окно «Результаты поиска» содержит инструменты для работы с результатами поиска. При нажатии на кнопку вы можете выбрать варианты сортировки результатов:

- **Плоско** – показывает результаты сплошным списком;
- **По файлам** – показывает результаты с фильтрацией по файлам. При этом сначала выводятся файлы с рабочего слоя, а потом с базового.

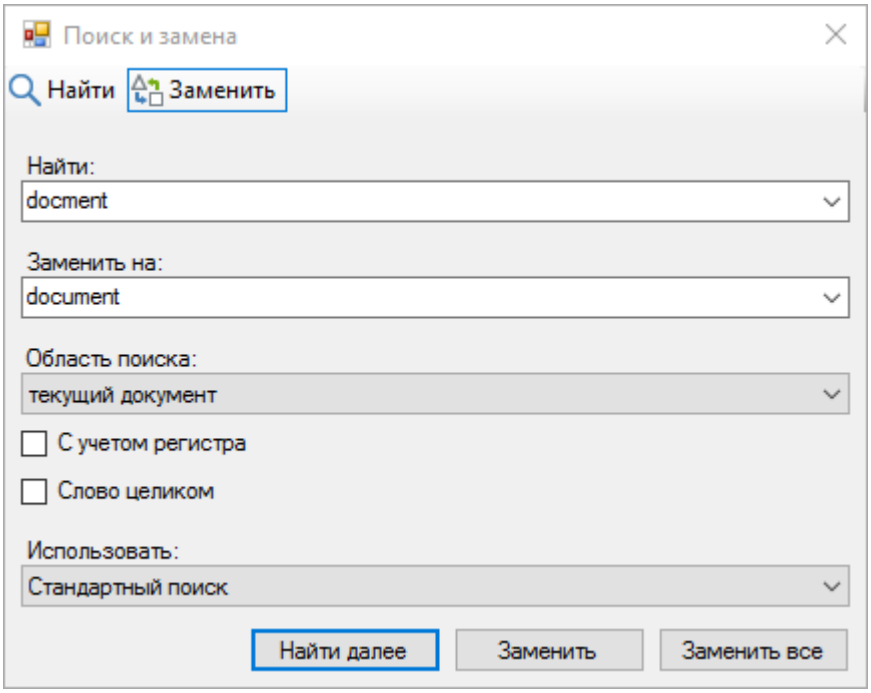
Также для удобства по кнопке вы можете свернуть результаты поиска, а по кнопке – развернуть.

Замена

При необходимости найденный фрагмент можно быстро заменить. Например, это полезно, если во время написания кода разработчик в нескольких местах допустил опечатку.

Чтобы воспользоваться заменой:

1. В главном меню последовательно выберите пункты **Поиск**, **Заменить...**. Также можно применить горячие клавиши CTRL+H.
2. В окне «Поиск и замена» заполните поля:



Найти. Фрагмент кода или название ресурса, который необходимо заменить.

Заменить на. Строка, на которую необходимо заменить результаты поиска.

Остальные поля заполняются аналогично полям при поиске.

Серверная сборка мусора

По умолчанию в среде разработки включена серверная сборка мусора (Server Garbage Collection). За счет этого ускоряется запуск и оптимизируется работа Development Studio. Чем больше объем исходного кода, тем сильнее заметно ускорение. На больших объемах среда разработки запускается быстрее в 2-4 раза.

Серверная сборка мусора настраивается в конфигурационном файле DevelopmentStudio.exe.config в параметре **gcServer**. Путь к файлу: <Папка с Directum Launcher>\etc_builds\developmentstudio\bin. Возможные значения параметра:

- **true** – используется серверная сборка мусора. Значение по умолчанию, менять не рекомендуется;
- **false** – используется сборка мусора на рабочей станции.

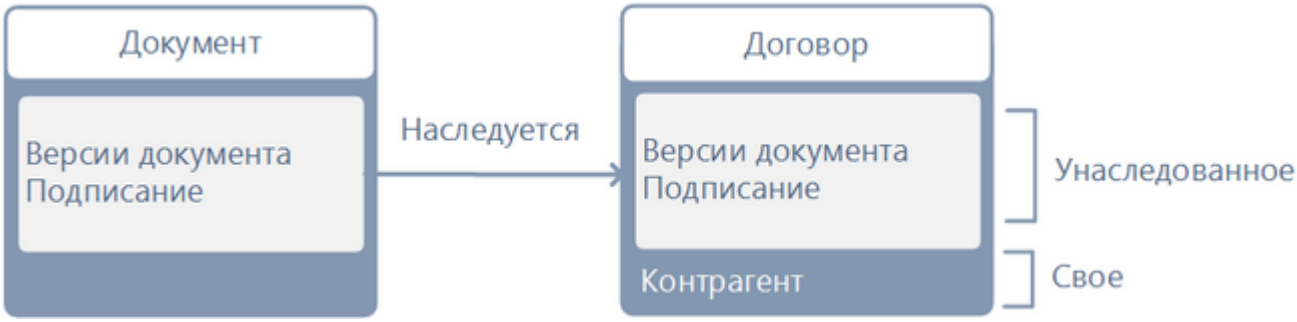
Пример настройки:

```
<gcServer enabled="true"/>
```

Наследование

Механизм наследования позволяет на *рабочем слое разработки* в модуле решения создавать новые типы сущностей, которые наследуют структуру данных, свойства, действия, форму, логику работы и отображение от других типов сущностей. Благодаря механизму наследования уменьшается дублирование кода между схожими типами.

Тип сущности, от которого наследуются, называется *базовым типом сущности*. При создании типа сущности для него всегда указывается базовый тип. Например, при создании договора в качестве базового типа выбирается документ. Договор наследует от базового типа документа версии и возможность их подписания. При этом в договор добавляется свое свойство **Контрагент** (организация, персона или банк, с которым заключается договор):

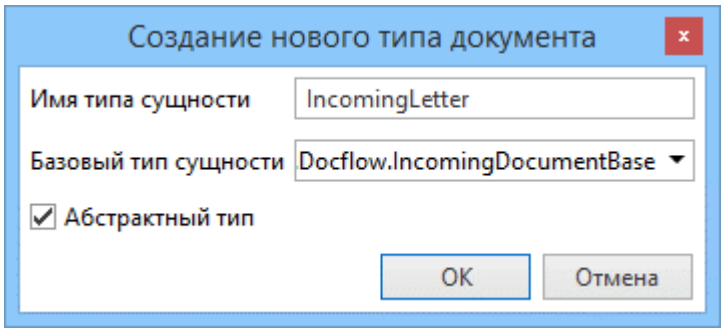


Выбрав базовую сущность, разработчик определяет поведение и возможности сущности, ее «специализацию», при этом может расширить и изменить унаследованное. Подробнее см. раздел «Что можно изменять в наследнике».

В системе есть:

- **базовые типы сущностей платформы**. Например, в среде разработки есть стандартные базовые типы «Задача» (Sungero.Workflow.Task), «Задание» (Sungero.Workflow.Assignment), «Уведомление» (Sungero.Workflow.Notice), «Задание на приемку» (Sungero.Workflow.ReviewAssignment), «Электронный документ» (Sungero.Content.ElectronicDocument), «Справочник» (Sungero.CoreEntities.DatabookEntry) и т.д. На их основе можно создать свои типы сущностей;
- **базовые типы сущностей на уровне прикладной разработки**. Например, в стандартном решении Directum RX есть базовый тип документа «Базовый договор» (ContractBase). На его основе можно создать типы документов «Договор доходный», «Договор расходный» и т.д. При наследовании от прикладных абстрактных типов стоит учитывать, как с ними будут работать отчеты, задачи, списки и вычисляемые папки базовых решений. Например, наследники, созданные на основе базового типа:
  - «Входящие документы» (IncomingDocumentBase) попадают в список «Входящие документы» модуля «Делопроизводство» и в отчет «Журнал входящих документов»;
  - «Исходящие документы» (OutgoingDocumentBase) попадают в список «Исходящие документы» модуля «Делопроизводство»;
  - «Внутренние документы» (InternalDocumentBase) попадают в список «Внутренние документы» модуля «Делопроизводство» и в отчет «Журнал внутренних документов»;

  - «Бухгалтерские документы» (AccountingDocumentBase) попадают в список «Финансовые документы» модуля «Финансовый архив», а также доступны при выгрузке документов из финансового архива;
  - «Договоры» (ContractBase) попадают в папку «Реестр договоров» модуля «Договоры», учитываются в рассылке о завершении срока действия договоров, а также доступны при выгрузке договоров из финансового архива.
- **базовые типы сущностей ваших решений**. Например, вы создали базовый тип документа «Внутренние документы» (MyInternalDocumentBase). На его основе можно создать типы документов «Заявление на отпуск», «Распоряжение» и т.д. Создавать базовые типы можно только справочников и документов. Для этого при создании типа сущности выберите любой существующий базовый тип справочника или документа, на основании которого будет создан новый тип, и установите флажок **Абстрактный тип**.



ВАЖНО. От созданного базового типа в дальнейшем можно создавать только наследников. В клиентском приложении справочники и документы базового типа нельзя создавать. Например, для правил согласования создан базовый тип ApprovalRuleBase. От него создано два наследника: «Правила согласования» (ApprovalRule) и «Правила согласования договоров» (ContractsApprovalRule). Пользователи в системе работают только с наследниками.

Что можно изменять в наследнике

Элементы, полученные при наследовании, можно изменять в редакторе типа сущности. К ним относятся:

Элементы	Что можно делать в наследнике?
Параметры типов сущностей. Находятся на первой закладке в редакторе типа сущности	Изменять имя, отображаемое имя, дополнительную информацию и другие параметры на закладке
Свойства	Добавлять новые и изменять существующие свойства. В существующих свойствах можно изменять только часть настроек, например видимость, доступность, отображаемое имя
Действия	Добавлять новые и изменять существующие действия. В существующих действиях можно изменять только часть настроек, например, отображаемое имя, описание и подсказку. В наследнике можно частично или полностью изменить логику базового действия, а также логику доступности действия

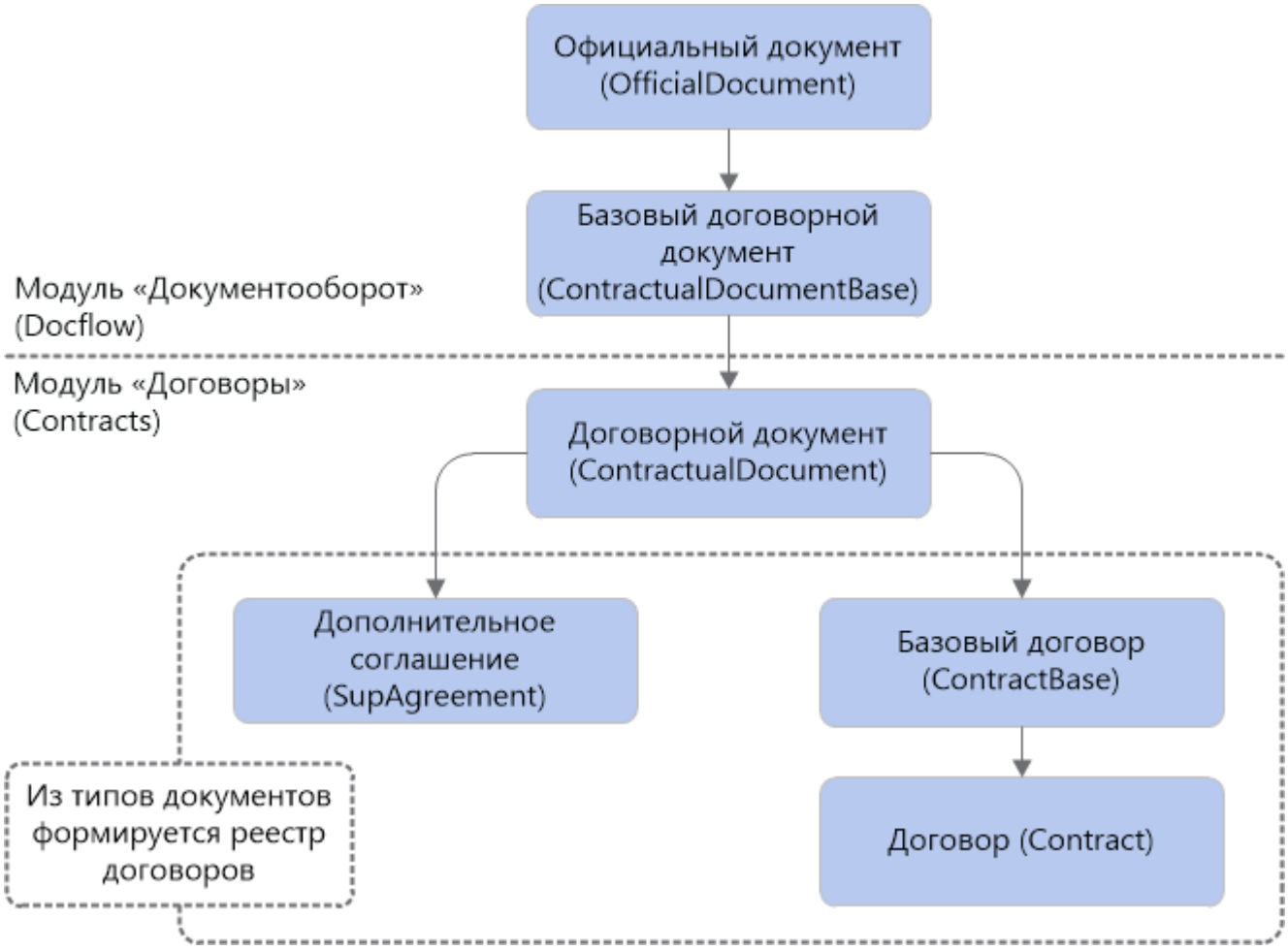
Элементы	Что можно делать в наследнике?
Форма	Переопределять карточку типа сущности: вкладки, группы контролов, ленту. Также можно переопределять область вложений для задач, заданий, заданий на приемку и уведомлений
Списки	Изменять внешний вид списков и папок, панель фильтрации в списке, ленту списка, контекстное меню. При необходимости можно отключить отображение всей панели фильтрации или отдельных групп и контролов
Стили	Изменять существующие стили ПРИМЕЧАНИЕ. При наследовании нельзя изменять приоритет и настройки стилей, созданных на предметном слое. Для стилей, созданных на прикладном уровне, можно изменять все, кроме имени
Операции авторизации	Добавлять операции авторизации и изменять отображаемое имя существующих операций авторизации. ПРИМЕЧАНИЕ. В среде разработки можно добавить ограниченное количество операций авторизации. Если количество добавлений исчерпано, кнопка **Добавить** в узле «Авторизация» становится недоступна
Функции	Добавлять и переопределять функции. Переопределять можно только те наследуемые функции, для которых в базовом типе сущности указан модификатор **virtual**. Чтобы переопределить функцию базового типа сущности, необходимо в наследниках объявить эту функцию с модификатором **override**

События	Добавлять новые и переопределять существующие обработчики событий. Для частичного переопределения обработчика события добавляется новая логика и вызывается базовый обработчик события (в начале, в конце или в середине кода, в зависимости от того, когда он должен выполниться). Если необходимо полностью переопределить логику обработчика события, то базовый обработчик вызывать не нужно.
	Для вызова базового обработчика события используется синтаксис:
	`base.<Имя события>(<Параметры события>);`
	Например, при частичном переопределении код обработчика события может быть следующим:
	`public override void Saving(Sungero.Domain.SavingEventArgs e)`
	`{`
	`  // Вызов базового обработчика события "До сохранения (в транзакции)".`
	`  base.Saving(e);`
	`  // Добавление новой логики обработчика события.`
	`  _obj.Name = "Default";`
	`}`

После изменения наследуемого свойства, действия и стиля рядом с ним появляется значок • . При этом значок отображаться не будет, если изменить только *локализацию* свойства или действия.Чтобы отменять изменения свойства или стиля, сделанные на слое, нажмите на кнопку ↩.

Схема наследования договорных документов

В разделе представлена схема наследования договорных документов. Схема позволяет определить *базовые типы* документов, от которых наследуются свойства, действия, форма, списки, операции авторизации и стили отображения в списках при создании нового типа договора.



Официальный документ (OfficialDocument)

Базовый тип документа в системе. Все типы договорных документов являются его наследниками.

Содержит общую логику, характерную для всех документов в системе:

- подписание;
- регистрация и нумерация;
- проверка уникальности;
- журнал выдачи бумажных экземпляров документов;
- отметка о помещении в дело;
- контроль возврата документа от контрагента;
- мониторинг состояния документа;
- отслеживание жизненного цикла документа.

Базовый договорной документ (ContractualDocumentBase)

Базовый тип договорного документа. Введен как связка между договорными документами модуля «Договоры» и официальным документом. Определяет общую логику договорных документов – договоров, дополнительных соглашений.

Документы, которые наследуются от данного типа, реализуют общую логику работы с контрагентом.

Особенности:

- появляется информация об организации, с которой заключается договорной документ;
- добавляется контактное лицо, которое подписывает документ.

Договорной документ (ContractualDocument)

Базовый тип договорного документа модуля «Договоры». Введен для создания различных договорных документов путем наследования. В системе таким образом реализованы договор, дополнительное соглашение.

Особенности:

- появляются свойства, характеризующие сроки действия документа и ответственных за документ с обоих сторон, может указываться сумма и валюта;
- реализована базовая карточка документа;
- переопределена логика проверки права подписи, характерная для договорных документов;
- реестр договоров строится на основе его наследников: базового договора (ContractBase) и дополнительного соглашения (SupAgreement). Таким образом, в реестр договоров попадают все договоры (перекрытые в решении и наследники от абстрактного договора) и все дополнительные соглашения.

Базовый договор (ContractBase)

Базовый тип, наследник базового типа договорного документа (ContractualDocument). Введен для определения логики договоров, используется как точка расширения для создания новых типов договоров путем наследования. Наследники показываются в списке «Реестр договоров» модуля «Договоры», учитываются в рассылке о завершении срока действия договоров, будут доступны при выгрузке договоров из финансового архива. Пример возможного наследника: договор закупки.

Особенности:

- добавлены свойства, характерные для договоров: **Типовой**, **С автопролонгацией**, ***Дней для завершения**;
- доопределена логика создания и сохранения, проверки дублей и фильтрации категорий договоров;
- переопределена логика автоформирования имени договора, обязательность полей и их доступность, подбор правил согласования.

Договор (Contract)

Выделен, как конечная реализация типа базового договора (ContractBase) в стандартном решении Directum RX. Особенностей реализации не содержит, полностью наследует поведение абстрактного договора.

Дополнительное соглашение (SupAgreement)

Документ, который прилагается к договору. В дополнительном соглашении фиксируются изменения в действующих условиях. Может дополнять, аннулировать или изменять соответствующие пункты договора. Является наследником договорного документа (ContractualDocument).

Особенности: добавлено свойство **Договор**. В нем указывается ссылка на договор, к которому прилагается дополнительное соглашение.

Пример переопределения функции

Ситуация

Есть базовый тип документа «Официальный документ» (OfficialDocument), у которого в событии До сохранения вызывается функция FillName(). С помощью этой функции автоматически заполняется имя документа, если для вида документа администратор установил флажок **Формировать имя документа автоматически**.

```
// Вызов функции для формирования имени документа в событии "До сохранения".
Functions.OfficialDocument.FillName(_obj);
```

Имя документа составляется по формату <Вид документа> <ИД документа>.

```
// Код функции.
public virtual void FillName()
{
  _obj.Name = _obj.DocumentKind + " " + _obj.Id.ToString();
}
```

Предположим, у типа документа OfficialDocument есть два наследника. Необходимо, чтобы для каждого наследника имя документа формировалось следующим образом:

- <Вид документа> <ИД документа> <Номер регистрации> для первого типа документа;
- <Вид документа> №<Номер> <Имя контрагента> для второго типа документа.

Решение

Сначала рассмотрим, как изменить формат имени для первого типа документа на следующий: <Вид документа> <ИД документа> <Номер регистрации>.

Для этого необходимо переопределить логику функции **FillName()** базового типа:

1. В редакторе типа документа, который является наследником, добавьте разделяемую функцию **FillName()** с модификатором **override**:

```
public override void FillName()
```

```
{
  base.FillName();
  _obj.Name += " " + _obj.RegistrationNumber.ToString();
}
```

К формату имени добавляется номер регистрации. Таким образом к логике функции базового типа сущности добавляется новая логика, поэтому добавлен вызов базового обработчика события.

2. Перейдите в узел «Свойства». В событии Изменение значения свойства вызовите функцию **FillName()** для тех свойств, значения которых влияют на имя документа.

   В данном случае только для свойства **Рег. №** (RegistrationNumber):

```
public virtual void RegistrationNumberChanged
(DirRX.AppModule.Shared.StringPropertyChangedEventArgs)
{
  FillName();
}
```

Теперь рассмотрим, как изменить формат имени для второго типа документа на следующий: <Вид документа> №<Номер> <Имя контрагента>. С таким форматом могут быть, например, заявки на проверку контрагента, которые используются при заключении договоров. Предполагается, что для второго типа документа уже добавлено свойство **Контрагент** (Counterparty).

Для изменения формата имени второго типа документа необходимо полностью переопределить логику функции **FillName()** базового типа:

1. В редакторе типа документа, который является наследником, добавьте разделяемую функцию **FillName()** с модификатором **override**:

```
public override void FillName()
  {
    //Получить вид документа.
    var documentKind = _obj.DocumentKind;
    //Очистить имя.
    if (documentKind != null && !documentKind.GenerateDocumentName.Value &&
  _obj.Name ==
  Sungero.Docflow.OfficialDocuments.Resources.DocumentNameAutotext)
      _obj.Name = string.Empty;
    if (documentKind == null || !documentKind.GenerateDocumentName.Value)
      return;
    var name = string.Empty;
    // Заполнить имя в формате: <Вид документа> №<Номер> <Имя контрагента>.
    using (TenantInfo.Culture.SwitchTo())
    {
      // Добавить к имени №<Номер>.
      if (!string.IsNullOrWhiteSpace(_obj.RegistrationNumber))
        name += Sungero.Docflow.OfficialDocuments.Resources.Number +
  _obj.RegistrationNumber;
      // Добавить к имени <Имя контрагента>.
      if (_obj.Counterparty != null)
        name += " " + _obj.Counterparty.DisplayValue;
    }
    if (string.IsNullOrWhiteSpace(name))
      name = Sungero.Docflow.OfficialDocuments.Resources.DocumentNameAutotext;
```

```
    else if (_obj.DocumentKind != null)
      name = _obj.DocumentKind.ShortName + name;
    _obj.Name =
  Sungero.Docflow.PublicFunctions.Module.TrimSpecialSymbols(name);
  }
```

Так как логика функции базового типа сущности полностью переопределяется, то базовый обработчик события вызывать не нужно.

2. Перейдите в узел «Свойства». В событии Изменение значения свойства вызовите функцию **FillName()** для тех свойств, значения которых влияют на имя документа.

   В данном случае для двух свойств:

   - **Рег. №** (RegistrationNumber):

```
public                                virtual                                void
   RegistrationNumberChanged
   (DirRX.AppModule.Shared.StringPropertyChangedEventArgs)
      {
        FillName();
      }
```

   - **Контрагент** (Counterparty):

```
public                                virtual                                void
   CounterpartyChanged
   (DirRX.AppModule.Shared.CounterpartyRequestCounterpartyChangedEventArgs e)
      {
        FillName();
      }
```

Перекрытие

Механизм перекрытия позволяет на *рабочем слое разработки* безопасно изменять модули и типы сущностей *базового решения*: добавлять дополнительные свойства в карточку, скрывать действия на ленте, изменять логику работы.

Например, чтобы в карточку справочника базового решения добавить *контрол*, нужно создать свое решение, перекрыть в нем необходимый тип справочника и добавить недостающий контрол на форму. В результате в клиентском приложении карточка справочника станет отображаться с новым контролом (перекрытый справочник подменит справочник базового решения).

Так как перекрытые модули и типы сущностей хранятся на отдельном слое разработки, вы можете безопасно вносить в них любые изменения, не нарушая при этом логику работы базового решения. Это значит, что при обновлении Directum RX изменения новой версии безопасно объединятся с вашими изменениями. Предположим, что в новой версии базового решения Directum RX на ленте в карточке справочника появилась кнопка. В своем решении ранее вы перекрыли этот справочник и добавили на вкладку контрол. После обновления в карточке справочника будет отображаться ваш контрол на вкладке и кнопка на ленте из базового решения.

Если в новой версии базового решения Directum RX изменится та же часть, которую вы доработали в своем решении, у пользователей будут отображаться только ваши изменения. Например, в своем решении вы переопределили клиентскую функцию типа справочника. В новой версии базового решения разработчики изменили код этой же функции. После

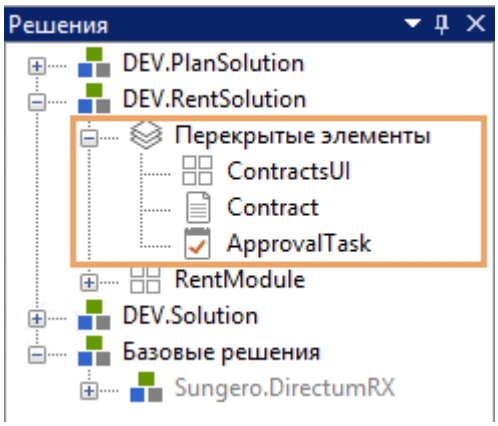
обновления функция будет работать так, как вы ее определили. Если вы хотите, чтобы функция работала, как в базовом решении, удалите код вашей функции или вручную перенесите доработки из функции базового решения. Чтобы не перерабатывать свои перекрытия после каждого обновления, ознакомьтесь с рекомендациями.

ПРИМЕЧАНИЕ. Иногда на *рабочем слое разработки* нужно базовый модуль или тип сущности одновременно перекрыть в нескольких решениях. Чтобы одновременно опубликовать такие решения с помощью среды разработки в *локальную систему*, в конфигурационный файл config.yml добавьте параметр REBUILD_WORK_INHERITANCE_CHAINS со значением **true**.

Создание перекрытия

Чтобы перекрыть элемент разработки:

1. Выделите решение, в котором хотите создать перекрытие, и в контекстном меню последовательно выберите пункты **Перекрыть**, **Модуль** или **Перекрыть**, **<Имя типа сущности>**, например, **Документ**.
2. В открывшемся окне в выпадающем списке выберите модуль, тип сущности или базовый тип сущности.
3. Нажмите на кнопку **ОК**. Элемент появится в дереве решений в разделе «Перекрытые элементы»:



Откроется редактор перекрытого элемента.

4. После перекрытия модуля добавьте его в список зависимостей решения. Если зависимости не указаны, то при публикации возникнут ошибки.
5. Внесите необходимые изменения. Подробнее см. раздел «Что можно изменять в перекрытии».

Что можно изменять в перекрытии

Перекрытие модуля

Элементы	Что можно делать в перекрытии?
Параметры модуля.	Изменять отображаемое имя, приоритет сортировки, отображение в проводнике, локализацию и другие параметры на закладке.
Задаются в узле «Модуль»	При этом следует учитывать особенности:

Элементы	Что можно делать в перекрытии?
	• в перекрытии модуля нельзя изменить версию и признак **Требовать лицензию**. Они устанавливаются автоматически из версии модуля базового решения, который был перекрыт; • при изменении локализации важно, чтобы сообщения валидации при сохранении данных корректно работали в невизуальном режиме, например, при сохранении документа в задачах
Функции	Добавлять и переопределять функции. Переопределять можно только те функции, для которых в перекрываемом модуле указан модификатор **virtual**. Чтобы переопределить функцию, необходимо в перекрытом модуле объявить эту функцию с модификатором **override**
Обложка модуля	Дорабатывать *обложку*. При перекрытии созданные на предметном слое вкладки, группы и действия удалить нельзя. Их можно только переместить или скрыть с обложки. ПРИМЕЧАНИЕ. Обложки рекомендуется настраивать в веб-клиенте. Их редактирование в среде разработки оставлено для совместимости
Вычисляемые папки	Создавать новые папки и настраивать для них панель фильтрации. Изменять папки, которые уже есть в перекрытом модуле и скрывать для них панель фильтрации или отключать отображение отдельных групп и контролей
Отображение	• Изменять состав и порядок расположения типов сущностей и вычисляемых папок модуля в проводнике системы, а также отображение отчетов модуля и типов сущностей модуля на обложке или по ссылке **Все отчеты**. ПРИМЕЧАНИЕ. Обложки рекомендуется настраивать в веб-клиенте. Их редактирование в среде разработки оставлено для совместимости. • Отображать или скрывать отчеты, которые отображаются по ссылке **Все отчеты** на обложке модуля
Виджеты	Изменять и скрывать существующие виджеты
Фоновые процессы	Изменять отображаемое имя и описание фонового процесса, расписание запуска, переопределить логику выполнения процесса с помощью обработчика события «Выполнение» или отключить процесс
Асинхронные обработчики	Изменять интервал повтора, а также обработчик события **Выполнение**

Подробнее см. пример перекрытия модуля «Договоры», изменения вычисляемой папки «Реестр договоров» и настройка валидации панели фильтрации для входящих документов.

ПРИМЕЧАНИЕ. В редакторе перекрытого модуля нет таблицы с зависимостями модуля и узлов «Публикация» и «Сторонние библиотеки». Данные настройки нужно изменять в редакторе вашего решения, в котором создано перекрытие модуля.

Перекрытие типа сущности

Элементы	Что можно делать в перекрытии?
Параметры типов сущностей. Задаются в редакторе в узле	Изменять отображаемое имя, дополнительную информацию, локализацию и другие параметры на закладке

Элементы	Что можно делать в перекрытии?
с названием типа сущности	
Функции	Добавлять и переопределять функции. Переопределять можно только те функции, для которых в перекрываемом типе сущности указан модификатор **virtual**. Чтобы переопределить функцию, необходимо в перекрытом типе сущности объявить эту функцию с модификатором **override**. Подробнее см. переопределение функции базового типа документа **OfficialDocument**
Операции авторизации	Добавлять операции авторизации и изменять отображаемое имя существующих операций авторизации. ПРИМЕЧАНИЕ. В среде разработки можно добавить ограниченное количество операций авторизации. Если количество добавлений исчерпано, кнопка **Добавить** в узле «Авторизация» становится недоступна
Свойства	Добавлять новые и изменять существующие свойства. В существующих свойствах можно изменять только часть настроек, например, видимость, доступность, отображаемое имя. При добавлении новых обязательных свойств рекомендуется программно указывать значения по умолчанию, которые пользователи увидят в карточке и смогут изменить. Это необходимо, чтобы невизуальная логика работы системы отрабатывала без ошибок
Действия	Добавлять новые и изменять существующие действия. В существующих действиях можно изменять только часть настроек, например, отображаемое имя, описание, подсказку, частично или полностью изменить логику выполнения базового действия. Для частичного переопределения обработчика действия добавляется новая логика и вызывается базовый обработчик действия в начале, в конце или в середине кода, в зависимости от того, когда он должен выполниться. Если необходимо полностью переопределить логику обработчика действия, то базовый обработчик вызывать не нужно. Для вызова базового обработчика действия используется синтаксис: `base.<Имя действия>(<Параметры действия>);` ПРИМЕЧАНИЕ. Обработчик «Выполнение» для действия типа сущности может быть переопределен при перекрытии, опираясь на базовую реализацию. Однако, если в параметрах действия в выпадающем списке ***Доступно в** выбрано значение **Дочерней коллекции** и установлен флажок **Доступно при отсутствии выделенных записей**, то базовая реализация недоступна через ключевое слово **base**. Код действия из родительского элемента придется скопировать
Форма	Переопределять карточку типа сущности: вкладки, группы контролов, ленту. В область вложений для задач, заданий, заданий на приемку и уведомлений можно добавлять новые группы и изменять видимость, доступность и обработчики событий существующих групп. Не рекомендуется скрывать с формы карточки основные контролы и группы, например, **Область регистрации**, **Наша организация**, **Подразделение**. Они влияют на процессы регистрации, нумерации и согласования по регламенту в стандартной функциональности
Списки	Изменять внешний вид списков, панель фильтрации в списке, ленту списка, контекстное меню. Отключать отображение всей панели фильтрации или отдельных групп и контролов
Стили	Изменять существующие стили.

Элементы	Что можно делать в перекрытии?
	ПРИМЕЧАНИЕ. При перекрытии нельзя изменять приоритет и настройки стилей, созданных на предметном слое. Для стилей, созданных на прикладном уровне, можно изменять все, кроме имени

События	Добавлять новые и переопределять существующие обработчики событий. Частичное и полное переопределение обработчика события делается аналогично переопределению обработчика действия.
	Например, при частичном переопределении код обработчика события может быть следующим:
	`public override void Saving(Sungero.Domain.SavingEventArgs e)`
	`{`
	`  // Вызов базового обработчика события "До сохранения (в транзакции)".`
	`  base.Saving(e);`
	
	`  // Добавление новой логики обработчика события.`
	`  _obj.Name = "Default";`
	`}`

Схема задачи	Изменять схему задачи и обработчики событий блоков схем
Результаты выполнения заданий	Добавлять новые результаты выполнения. Изменять отображаемое имя, подсказку, описание, иконки и обработчики результатов выполнения заданий. Нельзя удалять существующие результаты выполнения

После изменения перекрытого свойства, действия и стиля рядом с ним появляется значок •. При этом значок отображаться не будет, если изменить только *локализацию* свойства или действия.Чтобы отменять изменения свойства или стиля, сделанные на слое, нажмите на кнопку ↩.

Подробнее см. пример перекрытия типа документа «Договор». Порядок перекрытия остальных типов сущностей аналогичен.

При развертывании среды разработки важно установить флажок **Уникальные имена при перекрытии**. Благодаря ему в перекрытых модулях и типах сущностей к именам свойств, действий и другим настройкам дописывается постфикс с кодом компании. Например, перекрыт справочник с именем **ContractCategory**, в него добавлено свойство **Property1**, при этом код компании **Company**. В результате у добавленного свойства будет имя **Property1Company**. В среде разработки постфикс недоступен для изменения и удаления. При обновлении Directum RX на новую версию он помогает избежать конфликтов между одинаковыми именами в базовом решении и вашем. Если флажок **Уникальные имена при перекрытии** не установлен, то нужно вручную отслеживать, чтобы имена элементов в вашем решении и в базовом не совпадали. Подробнее см. «Directum RX 25.2. Инструкция по установке», входит в комплект поставки.

Рекомендации по разработке перекрытий

Общие рекомендации

- После перекрытия типа сущности или модуля следите, чтобы сохранялась работоспособность стандартной разработки, в том числе при выходе новых версий. *Базовое решение* Directum RX постоянно развивается: в него добавляются новые типы документов, задач, справочников и т.д. Если перекрытие модуля или типа сущности

базового решения неправильное, то после каждого обновления стандартной версии перекрытие придется значительно перерабатывать;

- При перекрытии модулей учитывайте их связь с другими модулями. Изменение одного может повлиять на связанные. Например, изменение в логике регистрации документов может повлиять на интеграцию с системами электронного обмена документами;
- Перекрытие и стандартная разработка должны быть единым целостным решением. Для пользователей нет деления на стандартную разработку и перекрытие, поэтому при работе с системой у них не должно возникать ощущения разнородности отдельных частей. Работа в системе проходит проще и удобнее, когда ее элементы выглядят одинаково, и одни и те же действия выполняются однотипно. Чтобы перекрытие было единообразным со стандартной разработкой следуйте рекомендациям, которые перечислены в разделе «Элементы разработки»;
- Иногда на *рабочем слое разработки* нужно базовый модуль или тип сущности одновременно перекрыть в нескольких решениях. Чтобы опубликовать такие решения с помощью среды разработки в *локальную систему*, в конфигурационный файл config.yml добавьте параметр REBUILD_WORK_INHERITANCE_CHAINS со значением **true**.

Перед разработкой дополнительно изучите особенности, которые описаны в разделах:

- перекрытие задачи на согласование по регламенту
- перекрытие обложки модуля
- перекрытие формы типа сущности
- обновление схемы в перекрытии типа задачи

Перекрытие задачи на согласование по регламенту

В перекрытии типа «Задача на согласование по регламенту» (ApprovalTask) можно изменять обработчики блоков схемы и группы вложений, но нельзя изменять схему. Изменение схемы недоступно, так как в *базовом решении* Directum RX для типа «Задача на согласование по регламенту» установлен флажок **Запретить создавать версии схемы задачи в перекрытиях**. Ограничение добавлено, так как схема задачи на согласование по регламенту сложная, с большим количеством зависимостей, доработка схемы может сломать основную логику работы системы. Способ модификации задачи на согласование по регламенту зависит от того, какие доработки требуются.

Если нужно добавить новую роль согласования, **не перекрывайте задачу** на согласование по регламенту. Подробнее см. пример добавления роли.

Перекройте задачу на согласование, если нужно добавить:

- новые действия на ленту, при этом не нужно добавлять новые результаты выполнения. Подробнее см. пример «Как добавить результат выполнения в задание на согласование»;
- *контролы* в карточки задач или заданий;
- этап согласования со специальной карточкой и лентой и с результатами выполнения «Выполнить» и «На доработку».

Создайте новый тип задачи, если:

- нужно добавить новые результаты выполнения заданий и ветки схемы;
- бизнес-логика специфична и сильно отличается от стандартного согласования по регламенту;

- трудоемкость перекрытия и обновления на новые версии выше, чем разработка новой задачи. Как правило проще и быстрее создать новый тип задачи под конкретную бизнес-логику, чем перекрывать задачу на согласование по регламенту и актуализировать перекрытие при обновлениях стандартной версии системы.

После создания типа задачи нужно добавить ее отображение на вкладке «Задачи» в карточке документа и настроить для заданий вычисляемую папку в папке «Входящие».

СОВЕТ. В базовых решениях флажок **Запретить создавать версии схемы задачи в перекрытиях** может быть установлен для любых типов задач. При их перекрытии придерживайтесь тех же рекомендаций, что и при перекрытии задачи на согласование по регламенту.

Перекрытие обложки модуля

ПРИМЕЧАНИЕ. Обложки рекомендуется настраивать в веб-клиенте. Их редактирование в среде разработке оставлено для совместимости.

При перекрытии модуля не рекомендуется убирать с обложки стандартные группы действий, так как некоторые возможности доступны только с обложки. Например, только с обложки можно открыть список «Ожидают подписания» в финансовом архиве. Кроме этого, если в стандартной версии в группу добавят новое действие, после обновления вы его не увидите, так как в вашем перекрытии группа будет скрыта. Придется вручную добавлять новое действие на обложку в перекрытом модуле.

Если импортируется несколько *базовых решений*, в которых перекрыты одни и те же модули, то действия и группы на обложках этих модулей объединяются.

Перекрытие формы типа сущности

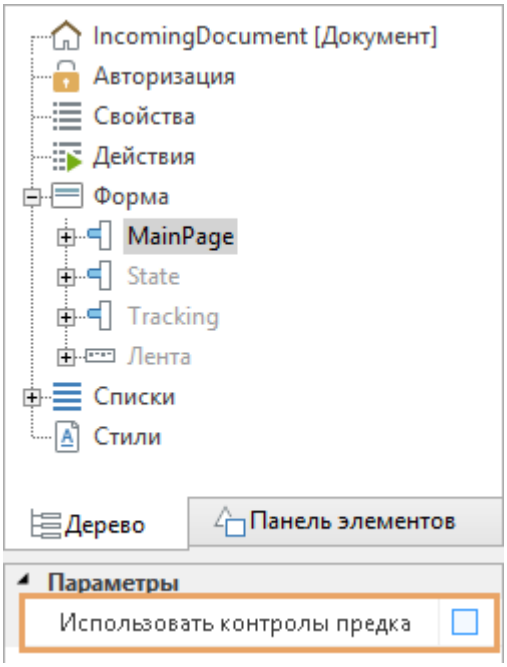
При создании или перекрытии типа сущности от родительского типа автоматически наследуются вкладки карточки. В дереве типа сущности они выделяются серым цветом и по умолчанию доступны только для просмотра. Также наследуются системные вкладки «Доступ» и «История», а для документов дополнительно наследуются вкладки «Версии» и «Связи». Системные вкладки недоступны для изменения и не отображаются в дереве типа сущности.

ПРИМЕЧАНИЕ. При создании типа справочника от базового типа Sungero.CoreEntities.DatabookEntry автоматически наследуется основная вкладка с одной группой и контролом. Вкладка доступна для изменения.

При необходимости можно изменить вкладки, полученные при наследовании и перекрытии. Для этого:

1. В дереве типа сущности в узле «Форма» выделите нужную вкладку.

2. В свойствах элемента снимите флажок **Использовать контролы предка**.



В результате наследуемые элементы вкладки можно будет изменить или удалить.

Чтобы вернуть внешний вид вкладки, установите флажок **Использовать контролы предка**. Вкладка станет такой же, как в родительском типе сущности.

ВАЖНО. Если флажок снят, то все последующие изменения вкладки в родительском типе сущности не будут отображаться на доработанной вкладке. Т.е. если в родительском типе на вкладку добавят новые контролы или удалят существующие, изменения не появятся на вашей форме. Такие доработки придется переносить вручную.

Чтобы при обновлении базовых решений не дорабатывать форму вручную, рекомендуем добавлять новые элементы на новых вкладках. Если все-таки нужно добавить элемент на существующую вкладку, то снимите флажок **Использовать контролы предка** для группы, а не для вкладки в целом. Тогда на вкладке не будут отображаться только те изменения базового решения, которые появятся в той же группе.

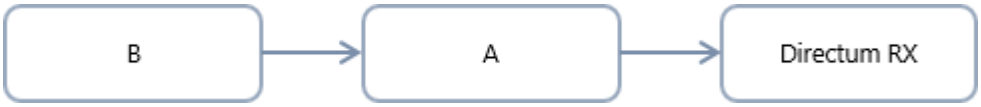
Объединение изменений в перекрытиях

Если в среду разработки импортируется несколько *базовых решений* с перекрытиями одинаковых элементов разработки, то изменения в этих перекрытиях объединяются. Например, у компании есть базовое решение Directum RX. Она приобретает два решения с заказной разработкой, назовем их A и B. И в том, и в другом решении перекрыта задача на ознакомление, но сделаны разные доработки. Оба решения передаются компании в пакетах разработки как базовые решения. При импорте пакетов, чтобы объединить изменения, среда разработки выстраивает цепочку зависимостей между решениями.

В начале цепочки ставится решение, в котором находится родительский элемент разработки, на основе которого делались перекрытия. В описанном примере в начале цепочки будет базовое решение Directum RX. Остальные решения сортируются по уникальному идентификатору (GUID). Предположим, что у решений A и B следующие идентификаторы:

Решение	GUID
A	3d68235c-ccba-4a34-b192-9f7674c372c9
B	d8575dc6-5a3b-43a2-903b-8a7e2d46cebd

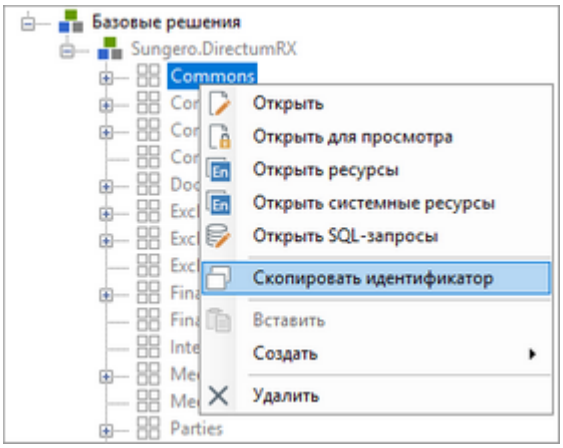
Раньше в цепочке встает решение, GUID которой по алфавиту меньше:



После построения цепочки для каждого перекрытого типа сущности или модуля заполняется BaseGuid – GUID типа сущности или модуля, который стоит в цепочке раньше. Если позже импортируется еще одно решение с перекрытием того же элемента разработки, то цепочка перестраивается и для перекрытий указывается актуальный BaseGuid.

ПОДРОБНЕЕ

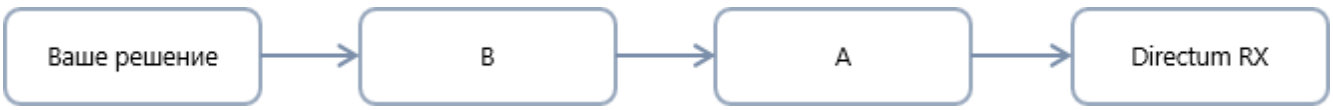
Если вы хотите узнать идентификатор (GUID) импортированного элемента разработки, выделите его в дереве решений, в контекстном меню выберите пункт **Скопировать идентификатор** и вставьте в любой редактор.



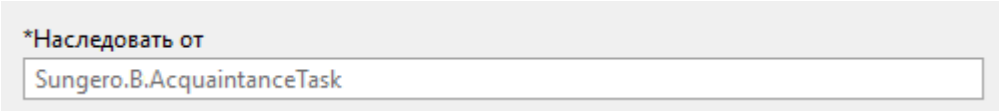
Информация о BaseGuid хранится только в метаданных элемента разработки.

Если после импорта базовых решений на своем слое еще раз перекрыть тот же элемент разработки, то ваше решение будет последним в цепочке. В редакторе перекрытого типа сущности в поле **Наследовать от** будет указано это решение. Т.е. если в нашем примере на слое перекрыть задачу на ознакомление, то:

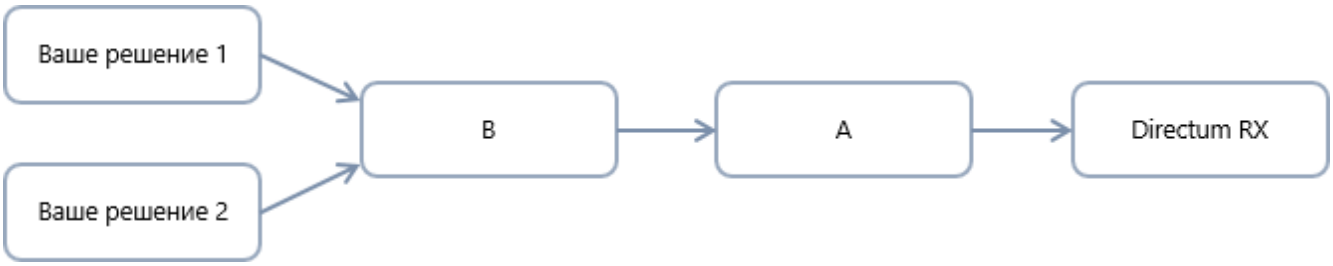
- цепочка решений будет выглядеть так:



- в редакторе перекрытой задачи в поле **Наследовать от** будет указано имя предпоследнего решения в цепочке:



Цепочка решений с одинаковыми перекрытиями строится только на базовом слое. Если на своем слое создать несколько решений с одинаковыми перекрытиями, то схема будет следующей:



Чтобы перенести доработки решений (базовых и ваших) в продуктивную систему, нужно создать *пакет разработки* и включить в него необходимые решения. После чего обновить продуктивную систему с помощью утилиты DeploymentToolCore.

Чтобы обновить на новую версию одно или все базовые решения, понадобится выполнить те же действия: импортировать обновленные пакеты разработки в Directum RX Development Studio, создать новый пакет разработки и обновить продуктивную систему с помощью утилиты DeploymentToolCore.

Пример перекрытия типа документа и модуля

Ситуация

Предположим, в карточку договора рядом с полем ***Контрагент** нужно добавить флажок **Резидент**. Пользователи будут его устанавливать, если договор заключается с контрагентом, зарегистрированным на территории Российской Федерации. Чтобы посмотреть список всех договоров, заключенных с резидентами, на *обложку* модуля «Договоры» нужно добавить соответствующие действие.

ВАЖНО. Обложки рекомендуется настраивать в веб-клиенте. Пример см. в руководстве администратора, в разделе «Пример настройки модуля и стартовой страницы». Далее показана доработка обложки через среду разработки: такая возможность оставлена для совместимости.

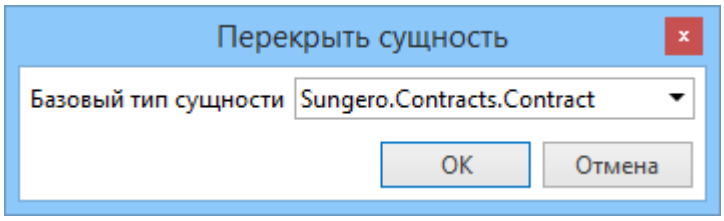
Решение

Шаг 1. Перекройте тип документа «Договор» и добавьте в карточку свойство **Резидент**.

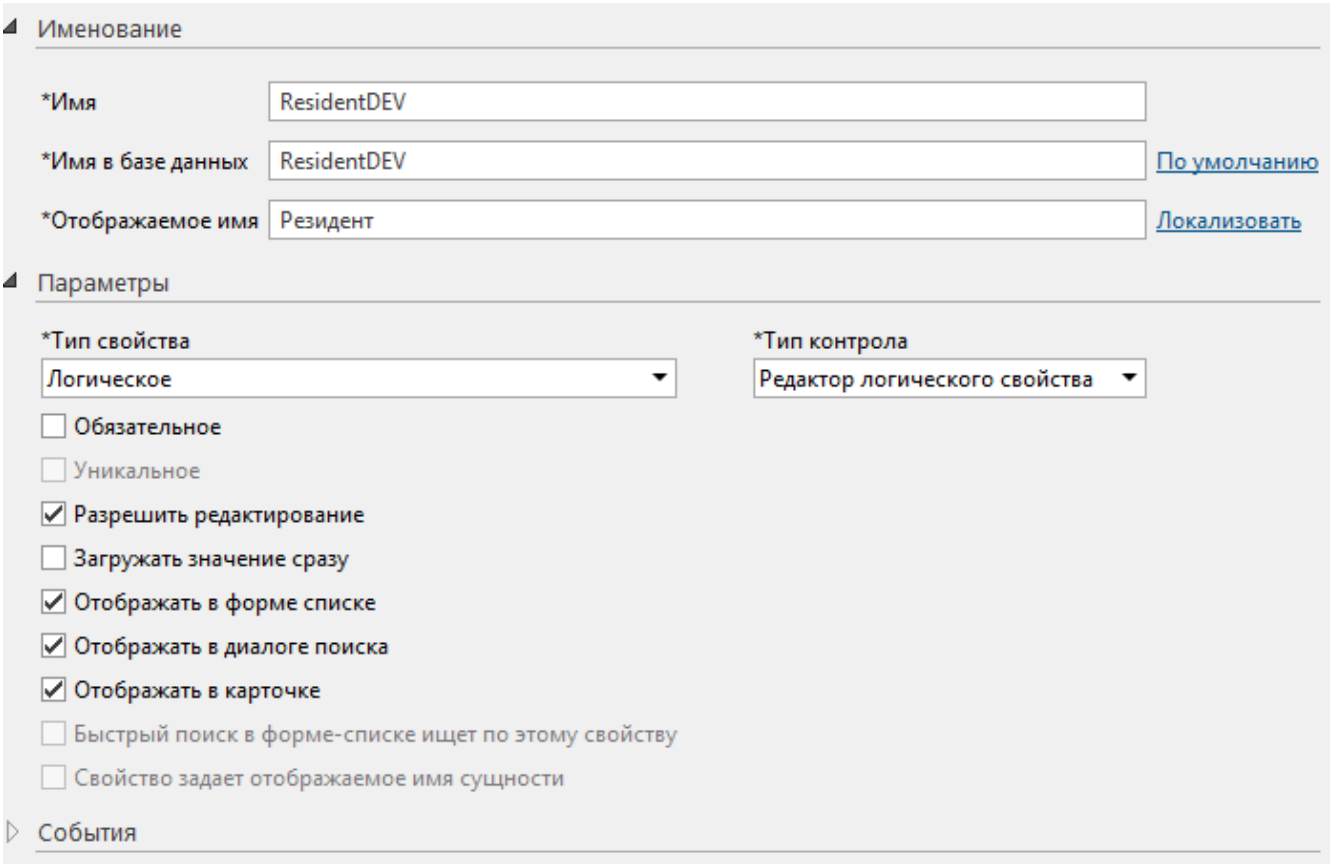
Шаг 2. Перекройте модуль «Договоры», создайте серверную функцию, чтобы получать список всех договоров с резидентами и добавьте на обложку модуля действия для открытия списка.

Шаг 1. Перекрытие типа документа

1. Создайте решение.
2. В контекстном меню решения последовательно выберите пункты **Перекрыть**, **Документ**.
3. В открывшемся окне в выпадающем списке выберите тип сущности Sungero.Contracts.Contract из *базового решения* Directum RX.

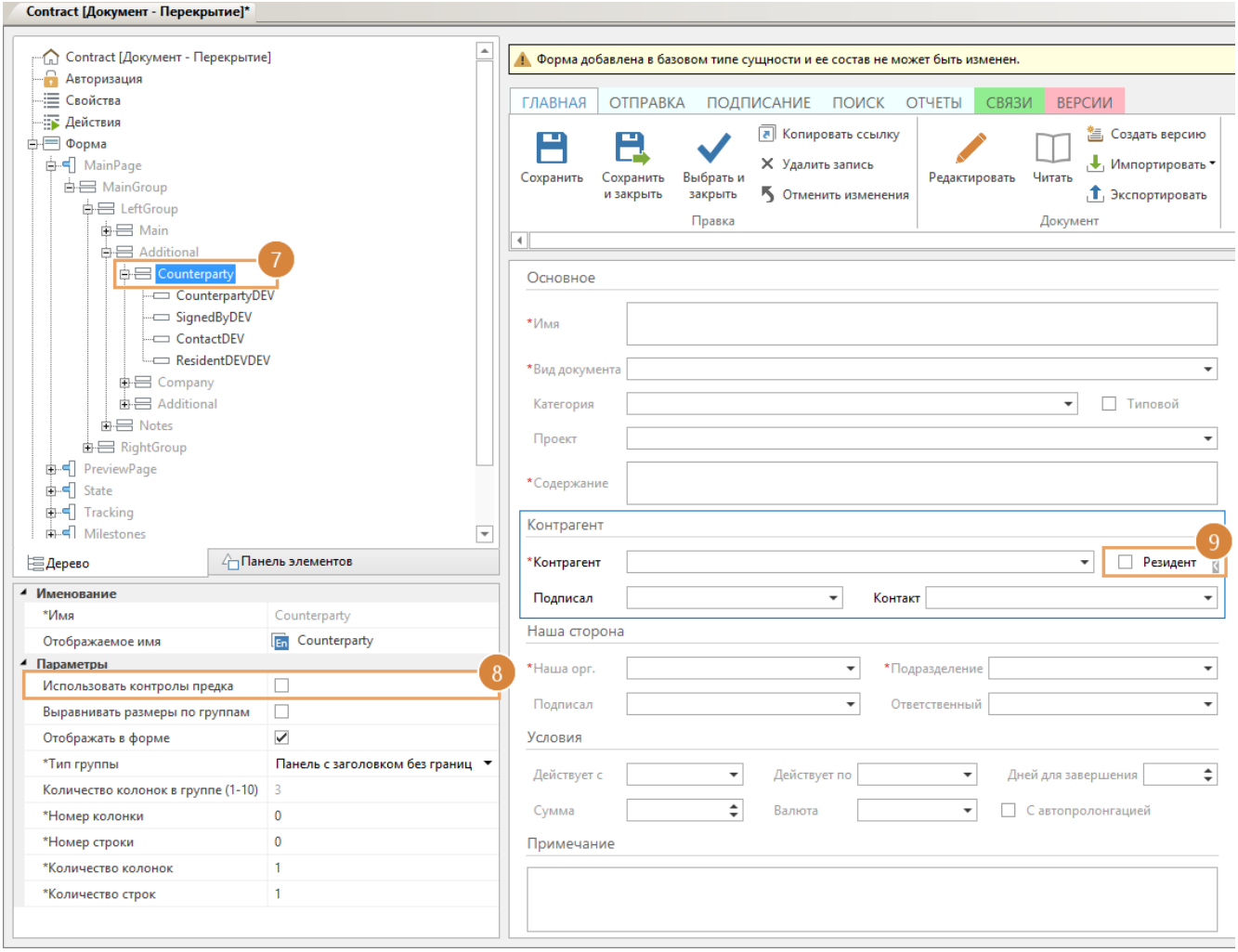


4. Нажмите на кнопку **ОК**. Документ появится в дереве решений в разделе «Перекрытые элементы». Откроется его редактор.
5. Перейдите в редактор созданного решения и добавьте зависимость от модуля «Договоры» (Sungero.Contracts). Если зависимости не указаны, при *публикации* могут возникнуть ошибки.
6. Вернитесь в редактор перекрытого типа документа и в узле «Свойства» добавьте свойство **Резидент** (Resident) с типом Логическое:



ПРИМЕЧАНИЕ. К имени свойства автоматически добавляется постфикс с кодом компании (DEV), так как при установке среды разработки был установлен флажок **Уникальные имена при перекрытии**.

7. Перейдите в узел «Форма». Щелкните левой кнопкой мыши по группе Counterparty («Контрагент») на форме. В результате группа выделится синим цветом в дереве элементов и синей рамкой на форме.



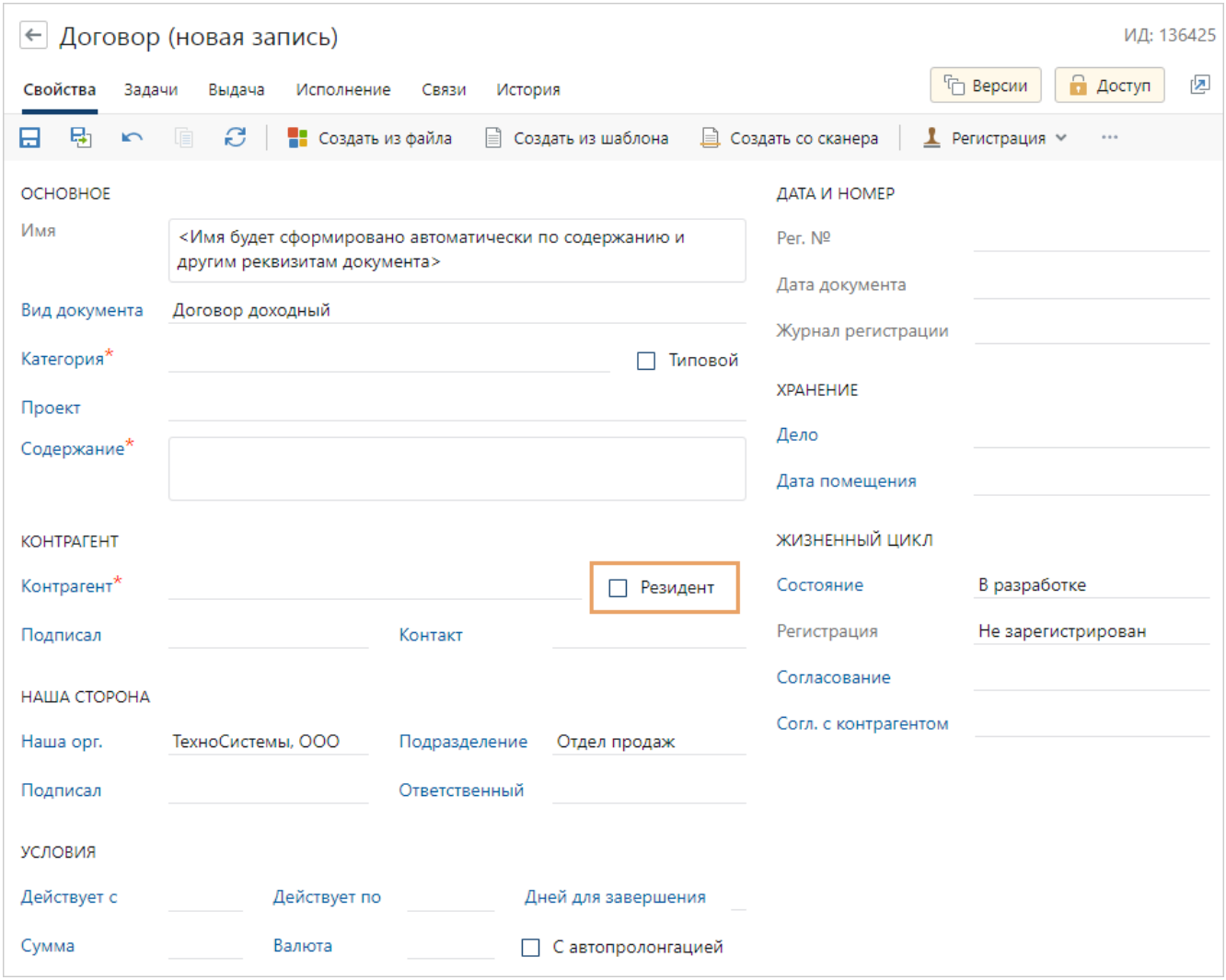
8. В редакторе свойств группы снимите флажок **Использовать контролы предка**. Все последующие изменения родительского типа сущности не будут применяться к текущему. Т.е. если в базовом решении в типе договора в группу добавят новые контролы или удалят существующие, изменения не появятся в вашем типе договора.

   ВАЖНО. Если необходимо изменить всю карточку документа, в дереве элементов выделите элемент **MainPage** и в редакторе свойств снимите флажок **Использовать контролы предка**.

9. В группу добавьте контрол и свяжите его с созданным свойством.
10. При необходимости измените размер остальных контролов в группе.
11. Чтобы при создании договора флажок **Резидент** по умолчанию был снят, перейдите в первый узел редактора договора (Contract), установите флажок рядом с серверным событием **Создание** и перейдите по ссылке. В открывшемся редакторе вставьте код:

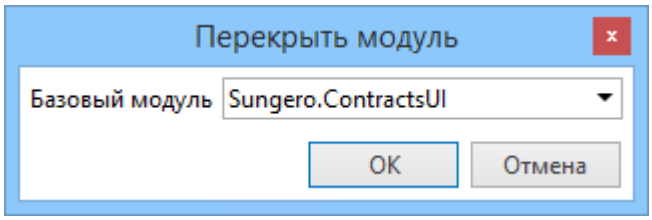
```
public override void Created(Sungero.Domain.CreatedEventArgs e)
    {
      base.Created(e);
      _obj.ResidentDEV = false;
    }
```

12. Опубликуйте разработку, запустите клиент и создайте договор. В карточке договора появится флажок **Резидент**.



Шаг 2. Перекрытие модуля

1. В контекстном меню решения, созданного на шаг 1, последовательно выберите пункты **Перекрыть**, **Модуль**.
2. В открывшемся окне в выпадающем списке выберите модуль «Договоры» (Sungero.ContractsUI) из *базового решения* Directum RX.



3. Нажмите на кнопку **OK**. Модуль появится в дереве решений в разделе «Перекрытые элементы». Откроется его редактор.
4. Перейдите в редактор вашего решения и добавьте зависимость от модуля Sungero.ContractsUI. Если зависимости не указаны, при *публикации* могут возникнуть ошибки.

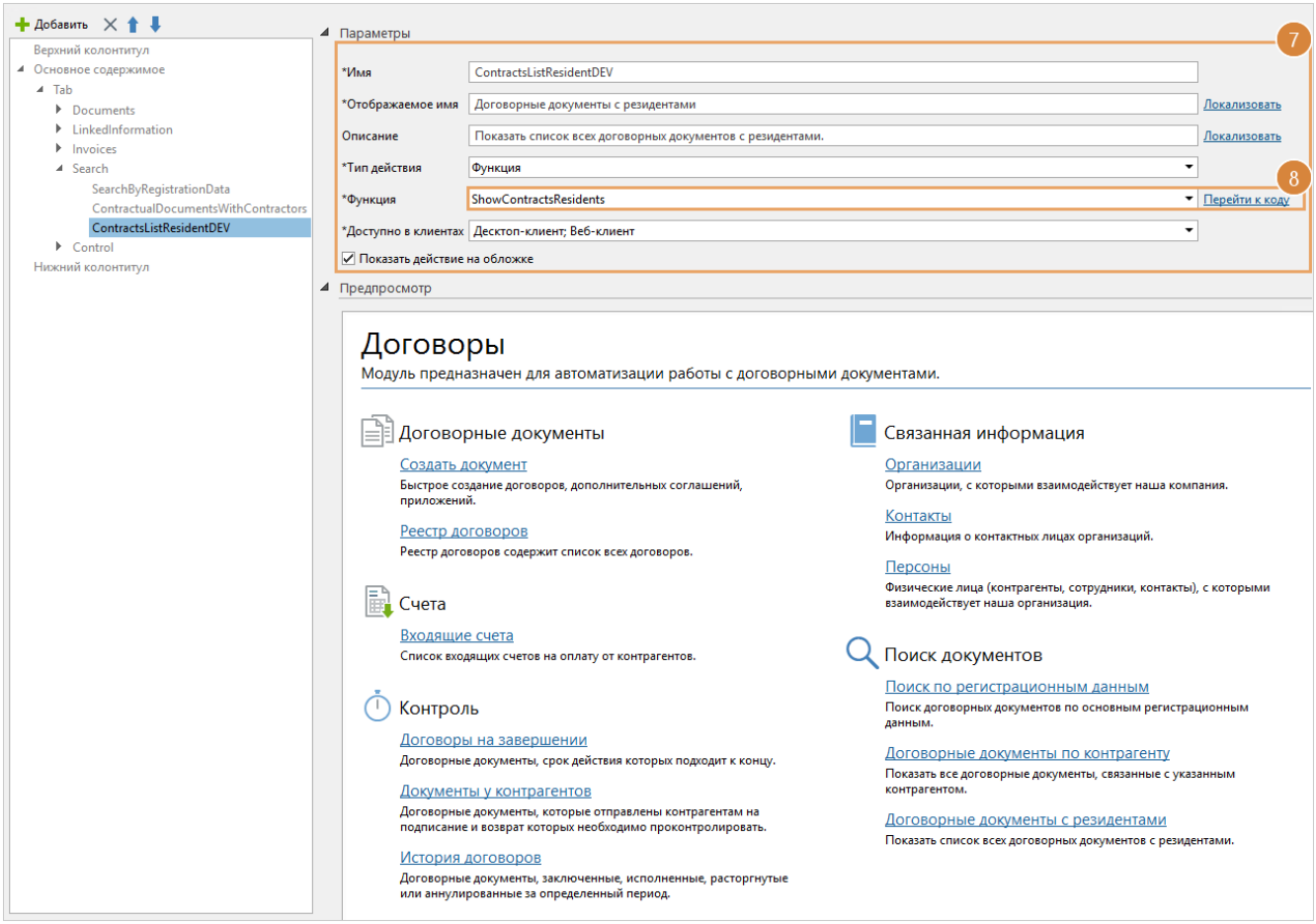
5. Чтобы получить список всех договоров с резидентами, перейдите в первый узел редактора модуля «Договоры» (ContractUI).
6. В группе «Функции» перейдите по ссылке **Добавить** рядом с серверной функцией. В открывшемся редакторе вставьте код:

```
[Remote]
    public IQueryable<DEV.PerekrytiiaExample.IContract> GetContractsResidents()
    {
      return DEV.PerekrytiiaExample.Contracts.GetAll(t => t.ResidentDEV.Value);
    }
```

7. Перейдите в узел «Обложка» и в группу «Поиск документов» (Search) добавьте действие для открытия списка всех договоров с резидентами. Заполните параметры:
   - ***Имя** – ContractsListResident;

     ПРИМЕЧАНИЕ. К имени автоматически добавляется постфикс с кодом компании (DEV), так как при установке среды разработки был установлен флажок **Уникальные имена при перекрытии**.
   - ***Отображаемое имя** – Договорные документы с резидентами;
   - **Описание** – Показать список всех договорных документов с резидентами.;
   - ***Тип действия** – Функция;
   - ***Доступно в клиентах** – Веб-клиент, Десктоп-клиент.

   Проверьте, что установлен флажок **Показать действие на обложке**.



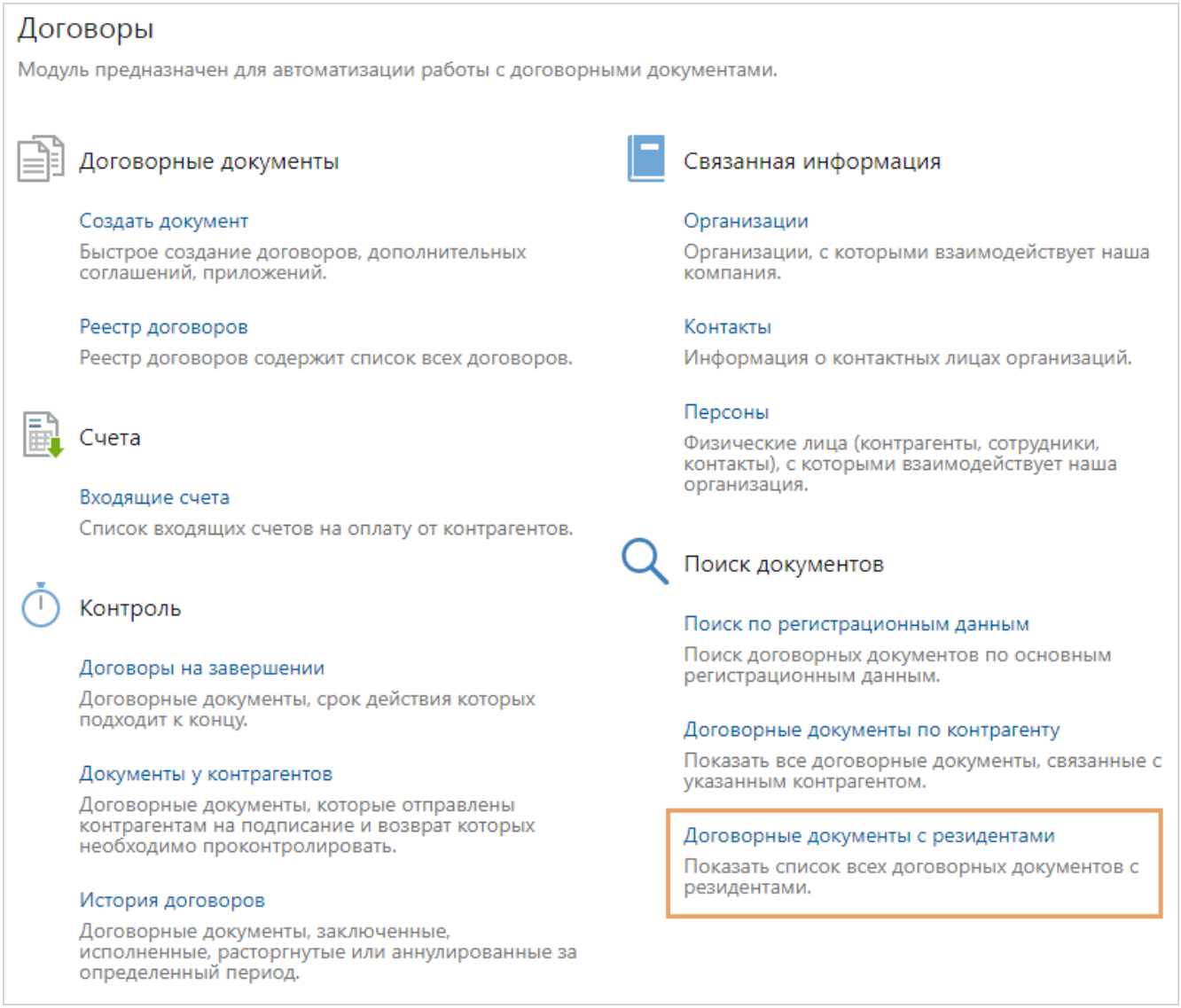
8. Рядом с полем ***Функция** перейдите по ссылке **Перейти к коду**. В открывшемся редакторе вставьте код:

```
public virtual void ShowContractsResidents()
```

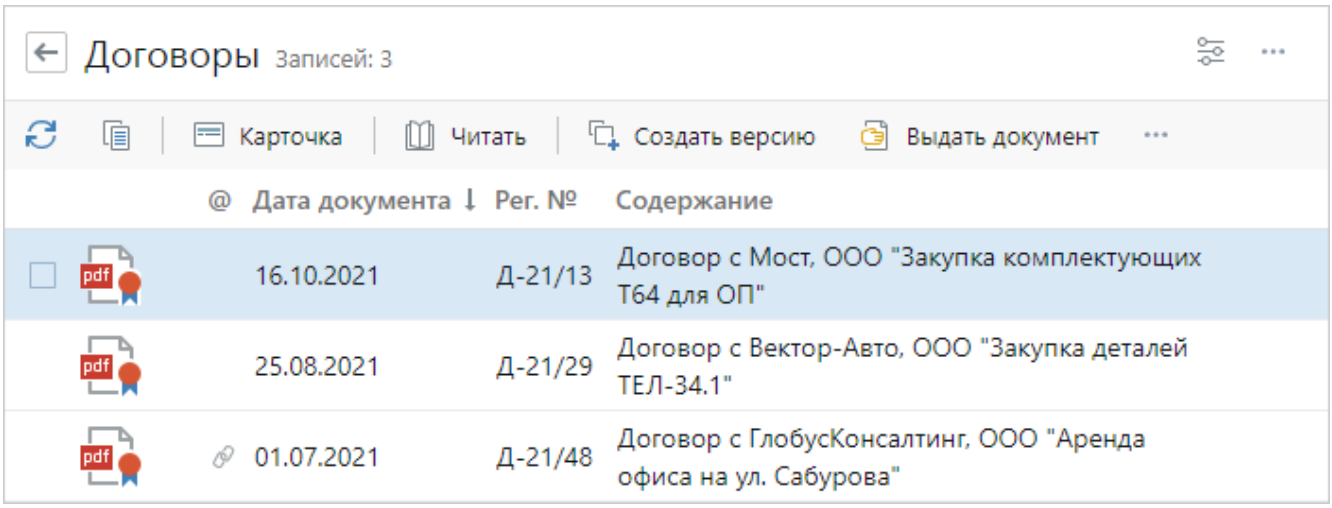
```
{
  Functions.Module.Remote.GetContractsResidents().Show();
}
```

После этого вернитесь в узел «Обложка» и проверьте, что в поле ***Функция** отображается имя функции, которую вы написали (ShowContractsResidents).

9. Опубликуйте разработку, запустите клиент, создайте несколько договоров с резидентами.
10. Перейдите на обложку модуля «Договоры» и нажмите на ссылку **Договорные документы с резидентами**.



Откроется список договоров с резидентами, которые вы создали.



Что использовать: наследование или перекрытие

Наследование – это механизм, который позволяет в модуле решения создавать свои типы сущностей на основе *базовых типов*. Новые типы сущностей наследуют от базовых структуру данных, свойства, действия, формы, логику работы и отображение. Если в базовый тип внести изменения, то они появятся и в наследниках. Таким образом, сокращается время на разработку, так как новый тип сущности не нужно разрабатывать с нуля. Кроме того, уменьшается дублирование кода между схожими типами сущностей.

Перекрытие – это механизм, который позволяет безопасно изменять модули и типы сущностей *базового решения*. При необходимости изменения в перекрытых модулях и типах сущностей можно отменить или удалить. В этом случае система продолжит работу с модулями и типами сущностей базового решения.

Наследование и перекрытие выполняются на *рабочем слое разработки*.

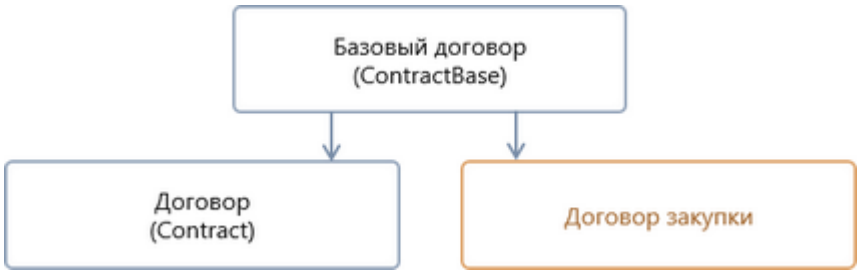
Также можно перекрывать базовые типы документов и справочников. Удобно использовать такое перекрытие, когда нужно внести идентичные изменения в существующие документы или справочники, созданные на основе одних и тех же родительских типов. Например, когда необходимо добавить одно и то же свойство в карточку или действие на ленту. Чтобы изменить наследников, нужно найти ближайший общий базовый тип сущности, перекрыть его и один раз добавить нужные свойства или действия. Изменения появятся у всех наследников на вашем и базовом слое.

Перекрытия подменяют элементы разработки стандартного решения Directum RX. В результате конечные пользователи могут работать в системе только с измененными модулями и типами сущностей. Наследование оставляет стандартные элементы разработки неизменными и добавляет новые. Таким образом, конечные пользователи могут работать в системе и с новыми типами сущностями, и со стандартными.

Чтобы детальнее разобраться, когда использовать перекрытие, а когда наследование, рассмотрим примеры.

Пример наследования

В стандартном решении Directum RX есть тип документа «Договор» (Sungero.Contracts.Contract). Он создан на основании базового типа документа «Базовый договор» (Sungero.Contracts.ContractBase) и наследует от него структуру данных, свойства, действия, формы, логику работы и т.д. В своем решении разработчик на основе базового типа «Базовый договор» создал наследника «Договор закупки». Добавил в карточку созданного документа логическое свойство **Особый контроль**. В результате в клиентском приложении теперь можно создать обычный договор и договор закупки с новым свойством.

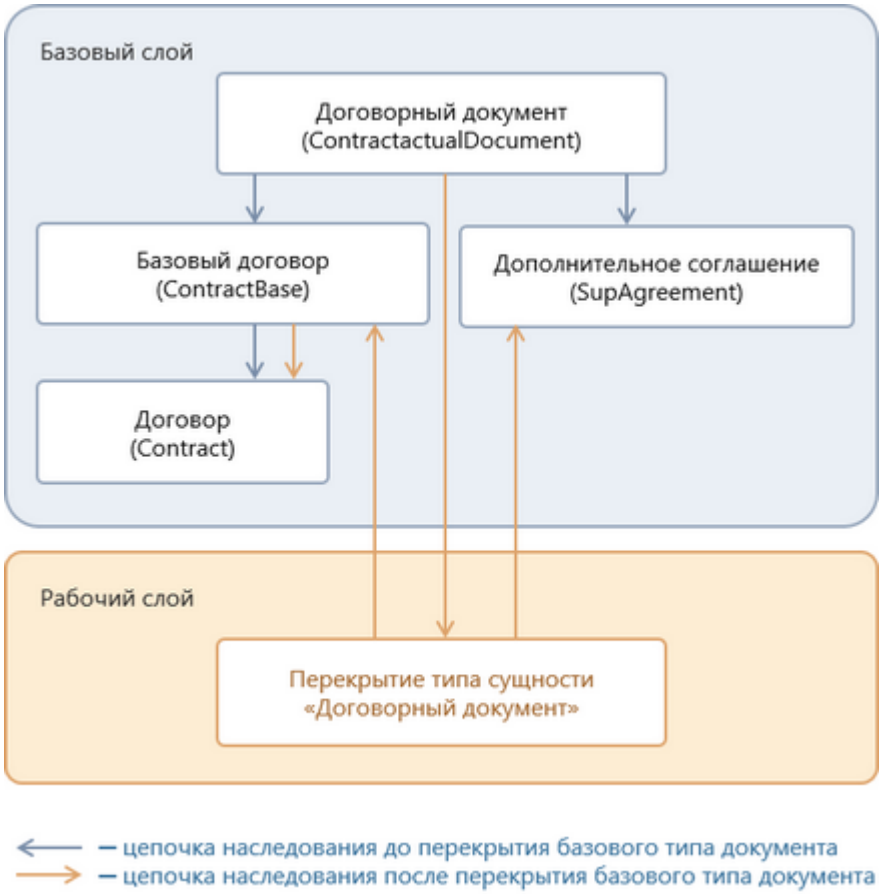


Пример перекрытия

В своем решении разработчик перекрыл тип документа «Договор» (Sungero.Contracts.Contract) и добавил в карточку договора логическое свойство **Особый контроль**. В результате в клиентском приложении карточка договора стала отображаться с новым свойством (перекрытый документ подменил документ базового решения).

Пример перекрытия базового типа сущности

В своем решении разработчик перекрыл *базовый тип* документа «Договорной документ» (ContractualDocument) и добавил в его карточку логическое свойство **Особый контроль**. В результате в системе у наследников договорного документа – договора (Contract) и дополнительного соглашения (SupAgreement) – в карточках появилось новое свойство.



Как видно на схеме, перекрытый базовый тип сущности находится на *рабочем слое*, а у конечных типов сущностей изменяется цепочка наследования.

Создание и публикация настроек по умолчанию

Настройки, заданные в локальной системе, сохраняются в базу данных и в исходные коды, и используются по умолчанию. При сборке решения заданные настройки попадают в пакет разработки и при публикации применяются в продуктивной системе. Подробнее см. в руководстве «No-code: бизнес-процессы и интерфейс», раздел «Типы настроек и их жизненный цикл».

ПРИМЕЧАНИЕ. Через среду разработки можно применять настройки только на *локальном веб-сервере*.

Чтобы задать настройки по умолчанию:

1. Опубликуйте разработку в локальную систему, если это не было сделано ранее.
2. Запустите инициализацию по кнопке **Запустить инициализацию** на панели инструментов.

3. Нажмите на кнопку **Применить настройки на сервер**.

   ПРИМЕЧАНИЕ. Если при применении настроек возникает ошибка «Используется не локальный веб-сервер», то в конфигураторе Directum Launcher исправьте значения параметров **SAVE_NOCODE_SETTINGS_TO_SOURCES** и **DEV_STUDIO_CONFIG_PATH**. Подробнее см. в руководстве администратора, раздел «Среда разработки».

4. В веб-клиенте локальной системы настройте бизнес-процессы и интерфейс. Для настройки используются те же инструменты, что и в продуктивной системе. Подробнее см. в руководстве «No-code: бизнес-процессы и интерфейс», раздел «Типы настроек и их жизненный цикл».

   Настройки, заданные по умолчанию, попадают в пакет разработки, поэтому их нужно связать с модулем системы. Для этого в карточке настройки на вкладке «Хранение» установите флажок **Настройка по умолчанию** и в поле **Сохранить в модуле системы** укажите модуль, к которому относится настройка.

   ПРИМЕЧАНИЕ. Поле отображается только в веб-клиенте локальной системы, а в продуктивной – скрыто.

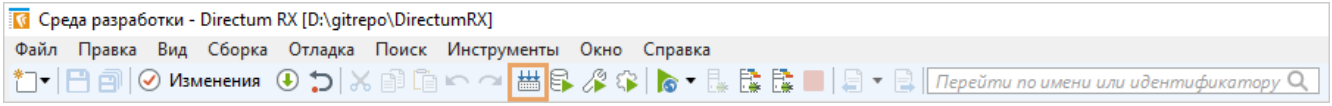
5. Создайте пакет разработки.

При публикации созданного пакета разработки в продуктивной системе заданные настройки по умолчанию применятся для бизнес-процессов и интерфейса.

Сборка решений

Чтобы проверить, что изменения написаны верно и в них нет несовместимостей, выполните предварительную сборку решения. Для этого:

1. На панели инструментов среды разработки Directum RX нажмите на кнопку **Собрать**:



   Среда разработки полностью пересоберет решения и модули при первой сборке после установки системы и после импорта *базовых решений*.

   В остальных случаях среда разработки пересоберет только изменения, которые появились в решениях и модулях. Поэтому сборка проходит быстрее.

2. Если сборка выполнится с ошибками, то появится соответствующее сообщение. Исправьте ошибки, которые отобразятся в окне вывода, и повторите сборку.

   СОВЕТ. В окне вывода щелкните два раза левой кнопкой мыши по тексту сообщения. Откроется редактор кода, в котором вы увидите строку с ошибкой.

3. Если сборка прошла успешно, в строке состояния среды разработки Directum RX появится соответствующее сообщение.

4. Опубликуйте решения и модули, чтобы проверить изменения в *локальной системе* Directum RX.

Если сборку нужно отменить, нажмите на клавиши CTRL+BREAK. Результат зависит от ситуации:

- сборку остановили сразу после запуска – она завершается моментально. Например, ее запустили случайно и нужно тут же отменить;

- сборка уже выполняется – отмена занимает дополнительное время, в результате появляется соответствующее сообщение;
- сборка завершилась, и запустились публикация или экспорт разработки – для отмены может потребоваться больше времени. В ряде случаев публикация и экспорт должны завершиться до конца, и остановить их уже нельзя.

Устранение неисправностей при сборке

Ошибка генерации файлов с расширением *g.cs

Иногда при сборке решения могут возникнуть ошибки компиляции *g.cs-файлов кода на языке C#. Они автоматически создаются при компиляции и нужны, например, для привязки данных в WPF.

Ошибка может возникнуть, даже если основной код собирается верно.

Решение

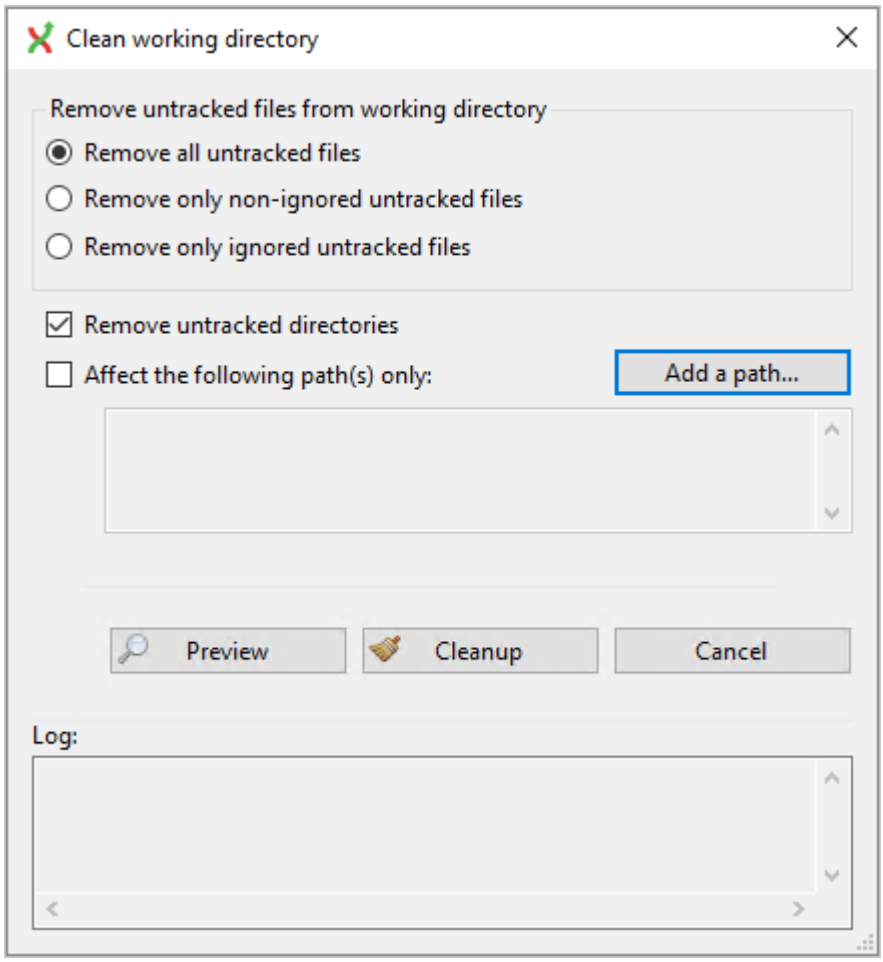
Часто ошибки генерации *g.cs-файлов связаны с использованием оператора **using** при обращении к классам и репозиториям. Поэтому перед устранением неисправности замените **using** на полный путь, например **Sungero.Company.Employees.GetAll()**.

Для исправления ошибки необходимо очистить рабочий репозиторий. Для этого воспользуйтесь программой Git Extensions или командной строкой. В результате при следующей сборке файлы с расширением *g.cs сгенерируются заново.

Очистка рабочего репозитория с помощью Git Extensions

1. Закройте среду разработки.
2. Запустите программу Git Extensions.
3. В главном меню последовательно выберите пункты **Commands**, **Clean working directory**.

4. В открывшемся окне «Clean working directory» проверьте, что установлены переключатель **Remove all untracked files** и флажок **Remove untracked directories**:



5. Нажмите на кнопку **Cleanup**.
6. Запустите среду разработки.
7. Загрузите последнюю версию исходного кода. Для этого на панели инструментов нажмите на кнопку **Получить изменения с сервера**.

Очистка рабочего репозитория с помощью командной строки

1. Закройте среду разработки.
2. Выполните команду:

   ```
   git clean -xdf
   ```

3. Запустите среду разработки.
4. Загрузите последнюю версию исходного кода. Для этого на панели инструментов нажмите на кнопку **Получить изменения с сервера**.

Публикация

Публикация – это процесс установки новых решений, обновления существующих и удаления ранее установленных решений в *локальной* или *продуктивной* системе.

ВАЖНО. Публиковать решения в продуктивную систему рекомендуется в нерабочее время. Во время публикации веб-сервер перезагружается и недоступен для пользователей. А если сотрудник работал с элементами системы, которые доработали в публикуемом решении, то после публикации в системе могут возникнуть конфликты. На период публикации администратор может включить режим технического обслуживания.

Перед публикацией в продуктивную систему создайте резервную копию БД. Это позволит восстановить данные при возникновении ошибок.

Если в среде разработки удалить один или несколько модулей из состава решения, то при публикации они удалятся и из решения в системе.

После публикации в локальную или продуктивную систему проверьте, что изменения не повлияли на работоспособность веб-сервера и сервисов Directum RX. Подробнее см. в разделе «Проверка работоспособности сервисов» (Windows, Linux).

Публикация с помощью среды разработки

С помощью среды разработки изменения публикуют в локальную систему. Для этого на панели инструментов используют действие **Опубликовать...** или в главном меню в выпадающем списке **Сборка** пункт **Опубликовать выборочно...**.

Процесс публикации в локальную систему состоит из этапов:

1. В среде разработки на основе *исходных кодов* создается пакет развертывания.

   ВАЖНО. Перед публикацией убедитесь, что размер одного пакета не превышает 1 ГБ.
2. Пакет развертывания отправляется на веб-сервер.
3. В базе данных обновляются структуры и после этого в нее копируются обновленные файлы модулей.
4. Веб-сервер запускает механизм обновления прикладной разработки.
5. Если разработчик изменял не только прикладной код и ресурсы, то веб-сервер и сервисы рестартуются. Примеры, когда рестарт не происходит, см в разделе «Публикация в локальную систему».
6. Запускается клиент, если в параметрах среды разработки установлен флажок **Запускать клиент после публикации**.

Публикация с помощью Directum Launcher

Публикация запускается в инструменте Directum Launcher на странице **Публикация**.

С помощью Directum Launcher публикуют изменения в продуктивную систему.

Процесс публикации в продуктивную систему состоит из этапов:

1. Разработчик создает *пакет разработки*.

   ВАЖНО. Перед публикацией убедитесь, что размер одного пакета не превышает 1 ГБ.
2. На основе пакета разработки создается *пакет развертывания* и отправляется на веб-сервер.
3. В базе данных обновляются структуры и копируются обновленные файлы модулей.
4. Веб-сервер запускает механизм обновления прикладной разработки.
5. Веб-сервер и сервисы перезапускаются.

Разница в процессах публикации с помощью среды разработки и Directum Launcher

Предположим, в системе установлены решения 2, 3 и 4. Нужно установить решение 1 и обновить решения 2 и 3.

В среде разработки после публикации появится решение 1, обновятся решения 2 и 3 и удалится решение 4. Чтобы решение 4 осталось в системе, нужно перед публикацией его явно выбрать в списке публикуемых решений.

При публикации с помощью инструмента Directum Launcher в системе появится решение 1, обновятся решения 2 и 3 и останется решение 4. Никаких дополнительных действий для этого делать не придется.

Механизм обновления прикладной разработки

Веб-сервер для обновления прикладной разработки запрашивает данные из БД. Обновление выполняется, если:

- сработал таймер обновления веб-сервера;
- выполнен старт веб-сервера.

При обновлении веб-сервер получает из БД данные о прикладной разработке, и полученные файлы помещаются в папку <ServerPath>\bin\AppliedModules. Если содержимое этой папки меняется, то выполняется отложенный автоматический рестарт веб-сервера, затем новая конфигурация прикладной разработки загружается на веб-сервер.

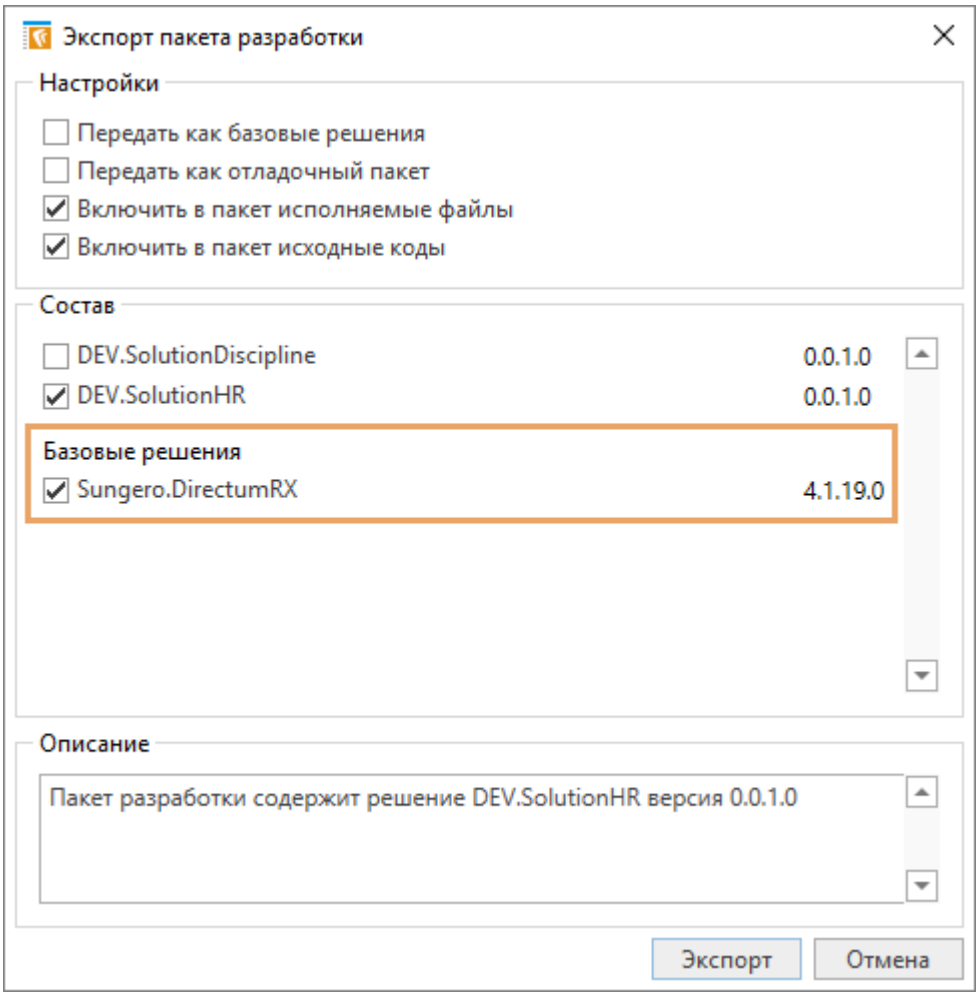
Устранение неисправностей при публикации

Ошибка «Базовое соединение закрыто: Непредвиденная ошибка при приеме»

Ситуация возможна, если при разработке были перекрыты абстрактные типы сущностей, например «Входящий документ» (IncomingDocumentBase).

Решение

При экспорте пакета разработки обязательно включите в него базовые решения. Для этого в окне экспорта установите флажок рядом с базовым решением:



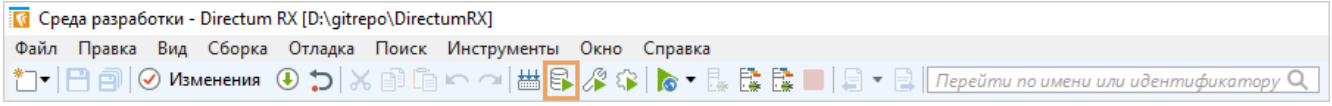
Публикация с помощью среды разработки

Быстрая публикация

СОВЕТ. Если вы публикуете решения в первый раз после установки среды разработки, проверьте параметры публикации.

Для проверки изменений опубликуйте решения в локальную систему Directum RX. Для этого:

1. На панели инструментов среды разработки Directum RX нажмите на кнопку **Опубликовать...**.



После нажатия на кнопку откроется окно «Публикация решений», если:

- публикация запускается в первый раз после установки среды разработки;
- были созданы новые решения;
- в нескольких опубликованных решениях появились перекрытия одних и тех же элементов разработки.

В остальных случаях запустится публикация. При повторном нажатии на кнопку запустится публикация опубликованных в последний раз модулей.

2. Если публикация выполнится с ошибками, то появится соответствующее сообщение. Исправьте ошибки и повторите публикацию. В этом случае откроется окно «Публикация решений».

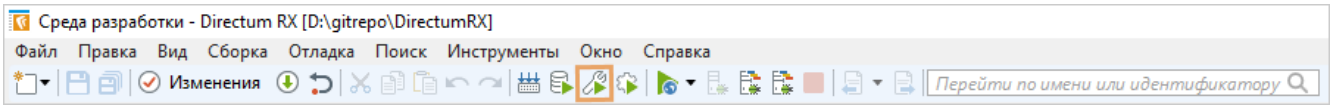
   СОВЕТ. Если ошибки отобразятся в окне вывода, щелкните два раза левой кнопкой мыши по тексту сообщения. Откроется редактор кода, в котором вы увидите строку с ошибкой.

3. Если в параметрах установлен флажок **Запускать клиент после публикации**, то после публикации автоматически запустится веб-клиент. Если флажок не установлен, запустите веб-клиент вручную. Для этого на панели инструментов нажмите на кнопку **Запустить веб-клиент**.

4. При первом входе в веб-клиент появится окно аутентификации. Введите имя пользователя Administrator и пароль – 11111. Работу под пользователем Administrator рекомендуется сократить до минимума, так как он не является сотрудником и имеет ряд ограничений при работе с системой. Поэтому создайте учетную запись, включите ее в роль «Администраторы» и используйте для дальнейшей работы.

5. Проверьте внесенные изменения в веб-клиенте. При необходимости выполните отладку, скорректируйте разработку и снова опубликуйте ее.

   Если отладка выполняется от имени пользователя, который входит в роль «Администраторы», то в локальной системе видны все изменения. Обычные пользователи увидят только те изменения, на которые у них есть права. Например, если вы создали отчет, но не выдали на него права, то в системе запустить отчет смогут только администраторы. Как правило, права выдаются в функциях инициализации для ролей, в которые входят сотрудники. Таким образом, чтобы проверить внесенные изменения в локальной системе от имени обычного пользователя, выдайте права на созданные типы сущностей в функциях инициализации и после публикации запустите инициализацию по кнопке **Запустить инициализацию** на панели инструментов:



6. Для обновления продуктивной системы создайте *пакет разработки*.

Повторная публикация проходит быстро, например, если разработчик изменял только прикладной код, формы в карточках и строки локализации. На скорость влияет то, что в этом случае веб-сервер не перезапускается.

Быстрая публикация без перезапуска веб-сервера	Публикация с перезапуском веб-сервера
• изменение прикладного кода, в том числе создание или удаление	• создание отчета;

Быстрая публикация без перезапуска веб-сервера	Публикация с перезапуском веб-сервера
обработчиков и функций; • изменение формы в карточках; • изменение существующих строк локализации и иконок; • изменения фонов для обложек модулей; • настройка колонок в списках и папках; • переключение версии схемы задачи; • все остальные изменения, не перечисленные в правой колонке	• создание публичной функции или изменение ее сигнатуры; • переименование решения, модуля, типа сущности; • изменение способа авторизации в типах сущностей; • изменение схемы задачи, а также создание и удаление версий схемы; • изменения, требующие генерации структуры данных при публикации: состав модулей и типов сущностей, состав свойств в типах сущностей, имена таблиц и полей в базе данных для типов сущностей, параметр «Уникальное» в свойствах, скрипты публикации

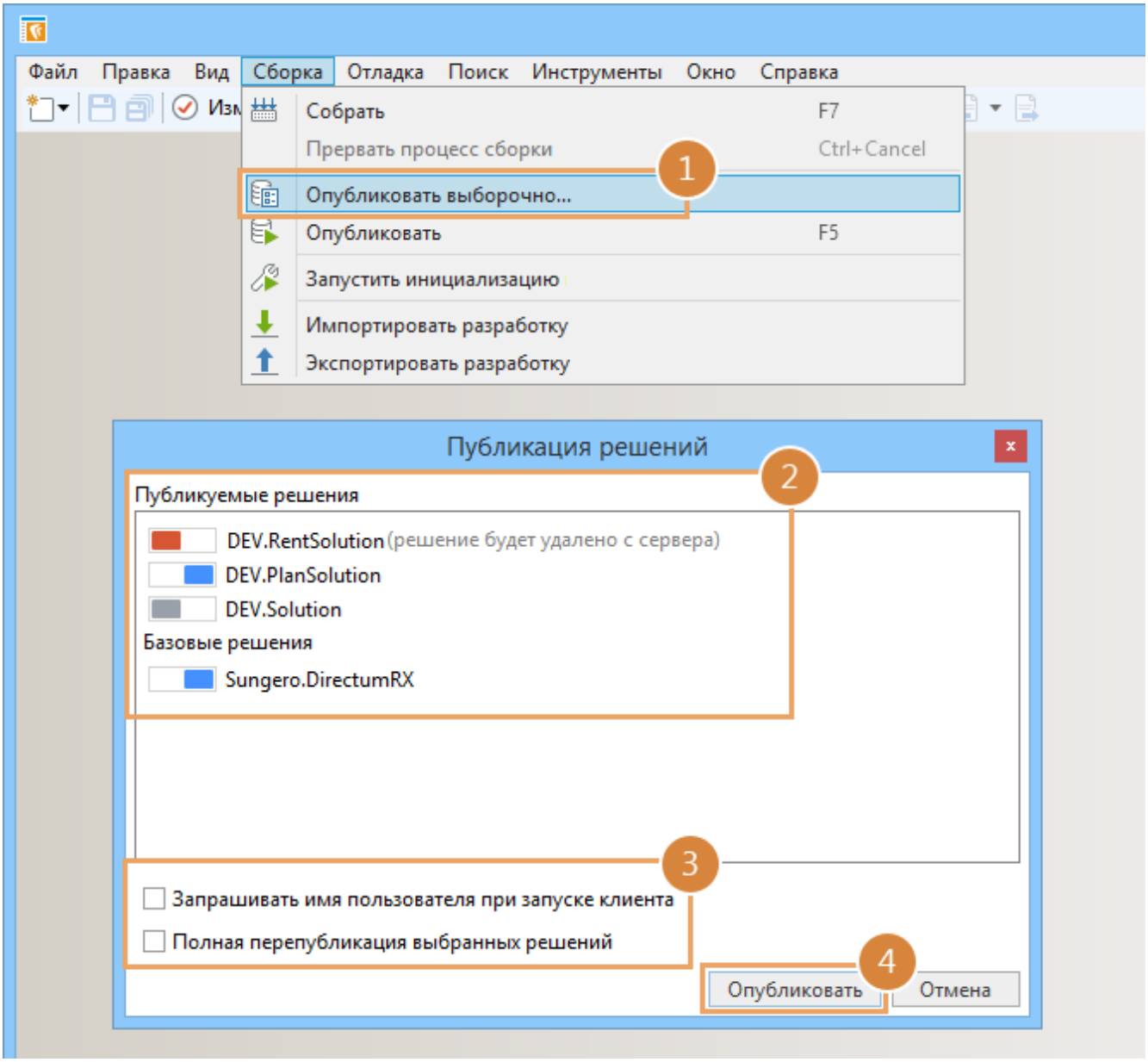
ПРИМЕЧАНИЕ. Публикация проходит быстро, если общий сервис выполняет функции четырех сервисов: сервис выполнения блоков задач, сервис асинхронных событий, сервис отчетов, сервис виджетов.

Публикация с выбором решений

После установки среды разработки, создания решения или перекрытия одних и тех же элементов разработки окно «Публикация решений» откроется автоматически. У опубликованных решений установлено синее положение ползунка, у неопубликованных – серое.

Чтобы опубликовать ранее неопубликованные решения или удалить опубликованные, можно открыть окно «Публикация решений» вручную. Для этого:

1. В главном меню среды разработки последовательно выберите пункты **Сборка**, **Опубликовать выборочно...**:



2. Настройте положение ползунков:
   - для решений, которые нужно опубликовать, измените серое положение ползунка на синее. После публикации элементы разработки и изменения выбранных решений появятся в клиентском приложении.

     ВАЖНО. Для *базовых решений* положение ползунка всегда должно быть синим.
   - для решений, которые нужно удалить, измените синее положение ползунка на красное. После публикации элементы разработки выбранных решений перестанут отображаться в клиентском приложении.

3. При необходимости установите флажок:
   - **Запрашивать имя пользователя при запуске клиента**, чтобы после завершения публикации при запуске клиентского приложения локальной системы появилось окно аутентификации. Если флажок не установлен, то клиентское приложение запускается от имени пользователя, под которым система запускалась в последний раз;
   - **Полная перепубликация выбранных решений**, чтобы переопубликовать выбранные решения полностью, а не только измененные части. Например, установите флажок после обновления среды разработки на новую версию.
4. Нажмите на кнопку **Опубликовать**.

При публикации с выбором решений всегда перезапускается веб-сервер, поэтому публикация проходит медленно.

Публикация через Directum Launcher

Для запуска публикации разработки используется инструмент **Directum Launcher**. Она выполняется на сервере Directum RX для публикации разработки в *продуктивную систему*, инициализации модулей и применения настроек бизнес-процессов по умолчанию. В этом случае публикацию рекомендуется выполнять в нерабочее время.

Если прикладная разработка уже установлена и ее нужно обновить, не меняя версию компонента Platform, то на период публикации администратор может включить режим технического обслуживания.

При публикации Directum Launcher запускает утилиту **DeploymentToolCore**. Поэтому убедитесь, что заданы ее настройки. Подробнее см. в руководстве администратора в разделе «Утилита DeploymentToolCore» (Windows, Linux).

ПРИМЕЧАНИЕ. В операционной системе Linux утилита **DeploymentToolCore** запускается в docker-контейнере.

Чтобы опубликовать пакет разработки:

1. Запустите Directum Launcher. Для этого:

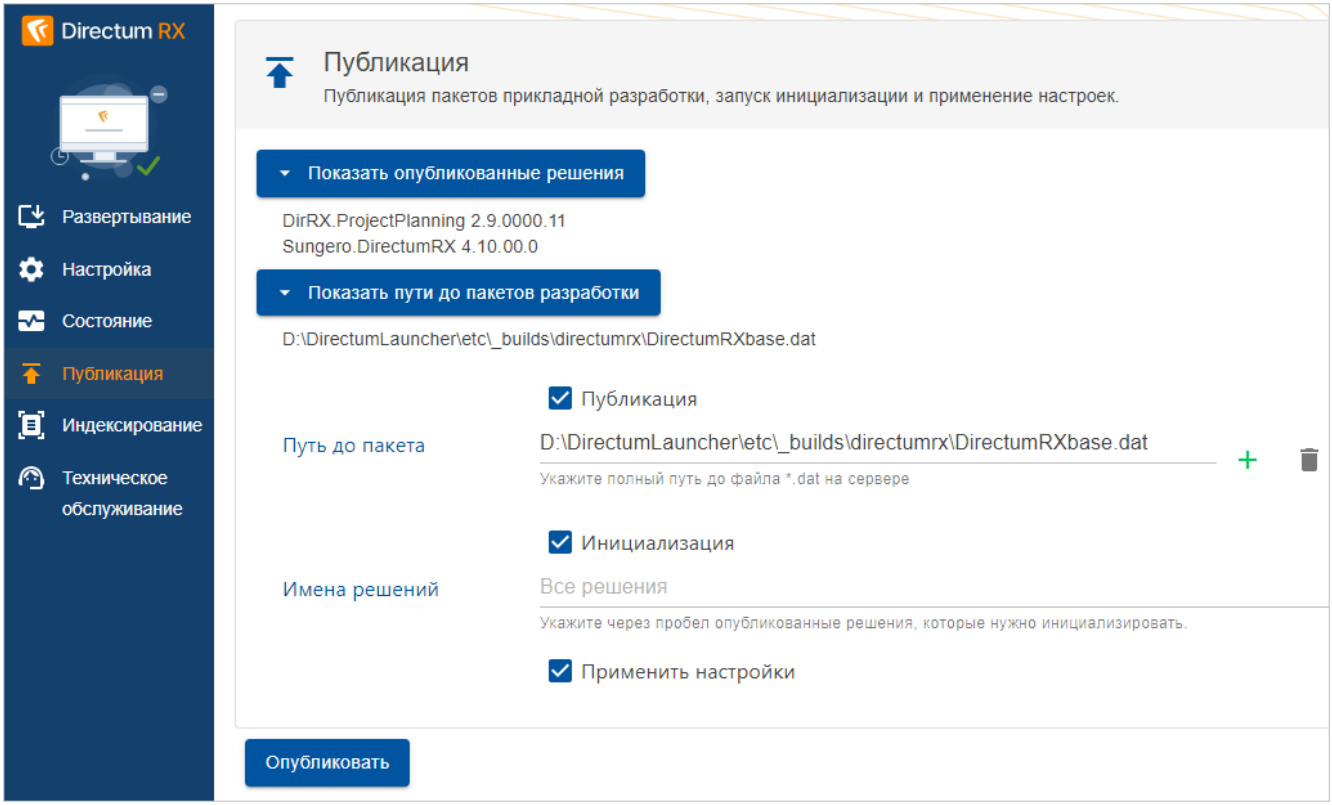
   Windows от имени администратора запустите файл DirectumLauncher.exe;

   Linux выполните команду:

   ```
   ./DirectumLauncher --host=0.0.0.0
   ```

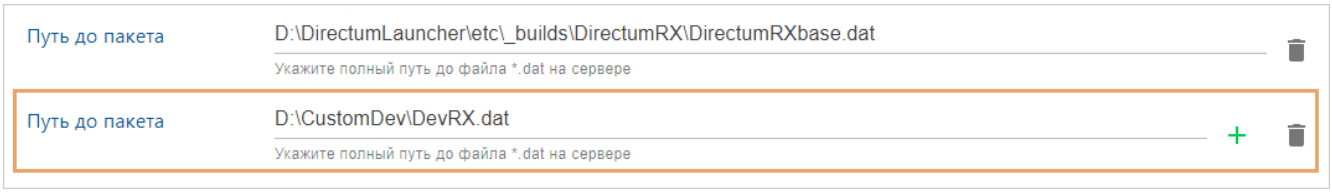
   Подробнее см. инструкцию по установке, раздел «Серверная часть Directum RX» (Windows, Linux).

2. Перейдите на страницу **Публикация** и укажите настройки:



Публикация. Установите флажок, чтобы запустить публикацию.

Путь до пакета. Укажите путь до пакета разработки. Если разработка состоит из нескольких пакетов или нужно опубликовать пакет другого решения, то нажмите на кнопку + и в появившемся поле укажите путь до пакета:



Таким же образом добавьте все необходимые пакеты. Они должны включать в себя исходные коды. Если нужно удалить поле с пакетом, нажмите на кнопку .

Список доступных пакетов разработки можно посмотреть по кнопке **Показать пути до пакетов разработки**. Отобразятся пакеты из добавленных компонентов.

Инициализация. Установите флажок, если нужно запустить инициализацию решений. Если нужно инициализировать только выбранные решения, укажите их в поле **Имена решений**. Иначе оставьте поле пустым, тогда инициализация выполнится для всех опубликованных решений.

Имена решений. Через пробел укажите имена решений, для которых нужно выполнить инициализацию. Список опубликованных решений можно получить по кнопке **Показать опубликованные решения**.

СОВЕТ. Если решение ранее не публиковалось, то получите его имя с помощью команды **dt get_deployed_solutions** либо не заполняйте поле, чтобы проинициализировались все решения.

Применить настройки. Установите флажок, если нужно применить настройки бизнес-процессов по умолчанию.

3. Нажмите на кнопку **Опубликовать**.

В результате публикуется прикладная разработка, выполняется инициализация и применяются настройки по умолчанию, если установлены соответствующие флажки.

При необходимости публикацию можно запустить через командную строку.

Публикация с помощью командной строки

Публикацию можно запустить с помощью инструмента **Directum Launcher** через командную строку, вызвав утилиту **DeploymentToolCore**. Перед первой публикацией убедитесь, что она настроена. Подробнее см. в руководстве администратора в разделе «Утилита DeploymentToolCore» (Windows, Linux).

ПРИМЕЧАНИЕ. При необходимости вы можете выполнить публикацию с помощью **DeploymentToolCore** без использования Directum Launcher, запустив утилиту через Docker.

При запуске публикации можно использовать основные и дополнительные команды. При возникновении ошибок определите причину по коду возврата.

Запуск публикации

1. Запустите командную строку от имени администратора и перейдите в папку с Directum Launcher.

   ВАЖНО. Утилита **DeploymentToolCore** добавляется вместе с компонентом платформы системы. Поэтому утилиту необходимо запускать из папки с инструментом Directum Launcher, с помощью которого установили систему. Иначе возникнет ошибка.

2. Опубликуйте разработку. Для этого выполните команду:

   Windows

   ```
   do dt deploy --package="<Путь к пакету 1>;<Путь к пакету 2>;<Путь к пакету N>"
   ```

   Linux

   ```
   ./do.sh dt deploy --package="<Путь к пакету 1>;<Путь к пакету 2>;<Путь к пакету N>"
   ```

   В параметре **--package** укажите публикуемые пакеты через точку с запятой.

   Пример команды:

   Windows

   ```
   do dt deploy --package="D:\CustomDev\Dev1.dat;D:\CustomDev\Dev2.dat"
   ```

   Linux

   ```
   ./do.sh dt deploy --package="/srv/CustomDev/Dev1.dat;/srv/CustomDev/Dev2.dat"
   ```

   ПРИМЕЧАНИЕ. Если в базе данных уже есть публикуемая версия прикладной разработки, то повторно она не опубликуется. Чтобы это сделать принудительно, выполните команду с параметром **--force**.

В результате выполнения команды публикуется разработка, инициализируются модули и применяются настройки бизнес-процессов по умолчанию.

СОВЕТ. При публикации прикладной разработки система недоступна. Если нужно указать несколько пакетов разработки, то затрачивается время на их объединение, что может дополнительно увеличить период простоя. Чтобы ускорить обновление, с помощью команды **dt merge_packages** заранее подготовьте объединенный пакет с нужными решениями, а затем при проведении плановых работ опубликуйте его.

При публикации используются имя пользователя и пароль, указанные в конфигурационном файле config.yml. Если публикацию необходимо запустить от имени другого пользователя, укажите параметры **--user** и **--password**:

Windows

```
do dt deploy --package="<Путь к пакету 1>;<Путь к пакету N>" --user="<Имя пользователя>" --password="<Пароль пользователя>"
```

Linux

```
./do.sh dt deploy --package="<Путь к пакету 1>;<Путь к пакету N>" --user="<Имя пользователя>" --password="<Пароль пользователя>"
```

ВАЖНО. Логины и пароли необходимо записывать в двойных кавычках, если они содержат спецсимволы или пробелы, например, логин **"Service User"** и пароль **"~!@55555"**. Если кавычки уже используются в пароле, его необходимо экранировать. Например, пароль вида **pas~!@"88** записываем как **"pas~!@""88"**.

Основные и дополнительные команды утилиты расширяют ее возможности. При этом некоторые выполняемые действия можно запустить разными командами.

Основные команды утилиты

В таблице приведены основные команды утилиты. В операционных системах Microsoft Windows и Linux формат команд одинаковый, отличается только способ их вызова:

Windows

```
do <команда>
```

Linux

```
./do.sh <команда>
```

Команда	Описание
dt export-package --export_package <dev_package> --configuration <configuration> --root <path> [--repositories <paths>]	Создать пакет разработки
dt merge_packages "<Путь к объединенному пакету>" --packages="<Пути к пакетам>"	Объединить пакеты разработки без публикации. В параметре **--packages** через точку с запятой укажите пути к пакетам и их названия. Пример команды: ./do.sh dt merge_packages "/app/union.dat" --packages="/app/first.dat;/app/second.dat"
dt merge_packages "<Путь к объединенному пакету>" -- package_from_component ="<Названия компонентов>"	Объединить пакеты разработки без публикации. В параметре **-- package_from_component** укажите названия компонентов с нужными пакетами.

Команда	Описание
	Пример команды: ./do.sh dt merge_packages "/app" --package_from_component="base;memo;agile;genai"
dt increment_version --root <path> --repositories <paths>	Увеличить номер версии модулей и решений
dt set_version --version <version> --root <path> --repositories <paths>	Задать номер версии модулей и решений
dt deploy --package="<Путь к пакету>"	Запустить публикацию и инициализацию разработки, применить настройки бизнес-процессов по умолчанию. Если пакет уже опубликован, то он повторно не публикуется
dt deploy --package="<Путь к пакету>" --force	Запустить публикацию принудительно, независимо от того, установлено решение или нет. Например, если прикладная разработка уже есть и для устранения инцидентов ее нужно перепубликовать. При этом также выполняется инициализация разработки и применяются настройки бизнес-процессов по умолчанию
dt deploy --package="<Путь к пакету>" --init	Запустить публикацию и инициализацию разработки. Если нужно выполнить публикацию без инициализации и применения настроек, например при обновлении прикладной разработки в рамках версии, то в ключе **--init** укажите значение **False**: dt deploy --package="<Путь к пакету>" --init=False
dt deploy --package="<Путь к пакету>" --settings	Запустить публикацию разработки и применить настройки бизнес-процессов и интерфейса по умолчанию
dt deploy --package="<Путь к пакету>" --dev	Запустить публикацию разработки
dt deploy --package="<Путь к пакету>" --distributed	Запустить публикацию в распределенную систему
dt deploy --init	Запустить инициализацию разработки.
dt deploy --settings	Применить настройки бизнес-процессов по умолчанию. Настройки могут содержать ссылки на сущности, которые создаются при инициализации, поэтому параметр выполняется после параметра **--init**
dt get_deployed_solutions	Получить список опубликованных решений
dt get_applied_solutions_info	Получить информацию о решениях из пакета разработки
dt init_and_apply_settings	Запустить инициализацию разработки и применить настройки бизнес-процессов по умолчанию
dt init	Запустить инициализацию
dt remove_solutions --solution_names="<Имена удаляемых решений>"	Удалить решения. В параметре **--solution_names** перечислите имена удаляемых решений через пробел, например: dt remove_solutions --solution_names="<Solution1 Solution2>"

Команда	Описание
	Если удалить решение необходимо от имени другого пользователя, укажите параметры --user и --password

Дополнительные команды утилиты

С помощью параметра **--command** можно указать дополнительные параметры запуска. В операционных системах Microsoft Windows и Linux формат команд одинаковый, отличается только способ их вызова:

Windows

```
do dt run --command="<Параметры>"
```

Linux

```
./do.sh dt run --command="<Параметры>"
```

ВАЖНО. Если логины и пароли содержат спецсимволы и пробелы, то перед двойными кавычками нужно добавить экранирующий символ ****. Например, логин **\"Service User\"** и пароль **\"~!@55555\"**. Если в пароле используются кавычки, то их необходимо также экранировать с помощью символа **"**, то есть указать две двойные кавычки. Например, пароль вида **1Qwerty"** нужно записать как **\"1Qwerty""\"**. Пример команды на Windows:

```
do dt run --command="-n \"Service User\" -p \"1Qwerty""\""
```

Параметр	Описание
-n (или --name) -p (или --password)	Имя и пароль пользователя для публикации. Пример: do dt run --command="-n username -p password -x"
-d (или --development-package)	Путь к публикуемому пакету разработки
-m (или --merge-only)	Путь для сохранения объединенного пакета разработки из нескольких пакетов. При выполнении команды публикация не запускается. Пример: do dt run --command="-d D:\CustomDev\first.dat D:\CustomDev\second.dat -m D:\CustomDev\union.dat" Если название объединенного пакета не задано, а указана только папка, то имя пакета генерируется по шаблону: MergedPackage-<yyyy-MM-dd_HH-mm-ss>.dat
-x (или --initialize)	Запустить инициализацию. По умолчанию инициализируются модули всех решений. Если инициализацию нужно выполнить только для определенных решений, через пробел укажите их названия. Можно указывать или оба параметра **-d** и **-x**, или один из них
-s (или --apply-settings)	Применить настройки бизнес-процессов и интерфейса по умолчанию. Настройки могут содержать ссылки на сущности, которые создаются при инициализации, поэтому параметр **-s** выполняется после **-x**
--settings-localization="<Путь к XLSX-файлу со строками>"	Импортировать строки локализации настроек
-l (или --list-deployed-solutions)	Отобразить список всех опубликованных модулей

Параметр	Описание
-r (или --remove-solutions)	Имена решений, которые необходимо удалить. Указываются через пробел
-y (или --settings)	Путь до конфигурационного файла утилиты
--distributed	Публикация в распределенную систему. Пример: do dt run --command="-n Administrator -p 11111 -d "D:\CustomDev\DevPackage.dat" --distributed" Параметр **--distributed** нельзя использовать совместно с ключами **-x** и **-r**
--help	Показать список и описание всех параметров утилиты. Пример: do dt run --command="--help"

Коды возврата ошибок

При возникновении ошибки выводится код возврата, который позволяет определить ее причину:

Код возврата	Описание
0	Ошибок не было
1	Ошибка возникла до отправки пакета разработки на сервер, например, во время: • проверки аргументов командной строки; • запроса метаданных с сервера; • чтения пакета разработки; • формирования пакета развертывания
2	Ошибка возникла при передаче пакета разработки на сервер или при публикации, например: • сетевая ошибка при отправке пакета развертывания на сервер; • ошибка публикации пакета, когда сервер недоступен; • ошибка публикации пакета, когда версия утилиты **DeploymentToolCore** отличается от версии веб-сервера. Публикация запускается, если версии совпадают на уровне редакции релиза (Directum RX 25.2.0.XXXX)
3	Ошибка возникла во время инициализации модулей при: • отправке запроса на инициализацию; • выполнении инициализации модулей на веб-сервере
4	Ошибка возникла при применении настроек по умолчанию из пакета разработки
5	Ошибка возникла при импорте настроек
6	Ошибка возникла при экспорте настроек
7	Ошибка возникла при экспорте пакета разработки
8	Ошибка возникла при изменении номера версии модулей и решений

Публикация в распределенную систему

В распределенных системах для масштабирования используется отказоустойчивый кластер. При такой конфигурации сервисы дублируются, поэтому для публикации изменений разработки нужно обновить каждый узел кластера. По умолчанию при публикации пакета разработки с помощью утилиты **DeploymentToolCore** все узлы отключаются одновременно, и система становится недоступна.

Иногда необходимо опубликовать разработку в рабочее время. Кроме того, есть компании, в которых серверная инфраструктура работает круглосуточно и нет возможности остановить ее даже на короткое время. В таких случаях публикацию рекомендуется запускать через утилиту **DeploymentToolCore** с параметром **--distributed**. Это позволяет без перезагрузки узлов и остановки системы вносить небольшие изменения в разработку: новое расположение элементов на панели действий и форме, изменение обработчиков и функций, строк локализации, иконок и т.д.

ВАЖНО. Публикация с параметром **--distributed** недоступна, если внесенные изменения требуют генерацию структуры данных: создание нового типа сущности, изменение типа данных и т.п.

Перед первой публикацией

Убедитесь, что в конфигурационном файле haproxy.cfg балансировщика нагрузки HAProxy для узлов с веб-сервером и сервисом интеграции указана проверка работоспособности **httpchk**. Если узел не отвечает на HTTP-запросы, то считается недоступным. В этом случае клиентские запросы перенаправляются на рабочие узлы.

Далее в примере вместо значений, выделенных **жирным** шрифтом, укажите свои значения:

```
backend rx-nodes-api
    option httpchk
    http-check send meth GET uri /client/api/health hdr host
rx.example.com
    balance roundrobin
    server rx1 192.168.1.30:443 check
    server rx2 192.168.1.31:443 check
backend integration-nodes-backend
    option httpchk
    http-check send meth GET uri /integration/health hdr host
rx.example.com
    balance roundrobin
    server rx1 192.168.1.30:443 check
    server rx2 192.168.1.31:443 check
```

Подробнее о настройке балансировщика см. раздел «Настройка HAProxy» (Windows, Linux).

ПРИМЕЧАНИЕ. Если в качестве балансировщика вместо HAProxy используется, например, Nginx, то проверка доступности по HTTP-запросам настраивается аналогично. Подробнее см. документацию на используемое программное обеспечение.

Запуск публикации в распределенную систему

1. Запустите командную строку от имени администратора и перейдите в папку с Directum Launcher.

   ВАЖНО. Утилиту **DeploymentToolCore** необходимо запускать из папки с инструментом Directum Launcher, с помощью которого установили систему.

2. Выполните команду:

   Windows

   ```
   do dt deploy --package="<Путь к пакету>" --distributed
   ```

   Linux

   ```
   ./do.sh dt deploy --package="<Путь к пакету>" --distributed
   ```

   Примеры команд:

   Windows

   ```
   do dt deploy --package="D:\CustomDev\DevPackage.dat" --distributed
   ```

   Linux

   ```
   ./do.sh dt deploy --package="srv/CustomDev/DevPackage.dat" --distributed
   ```

В результате, если:

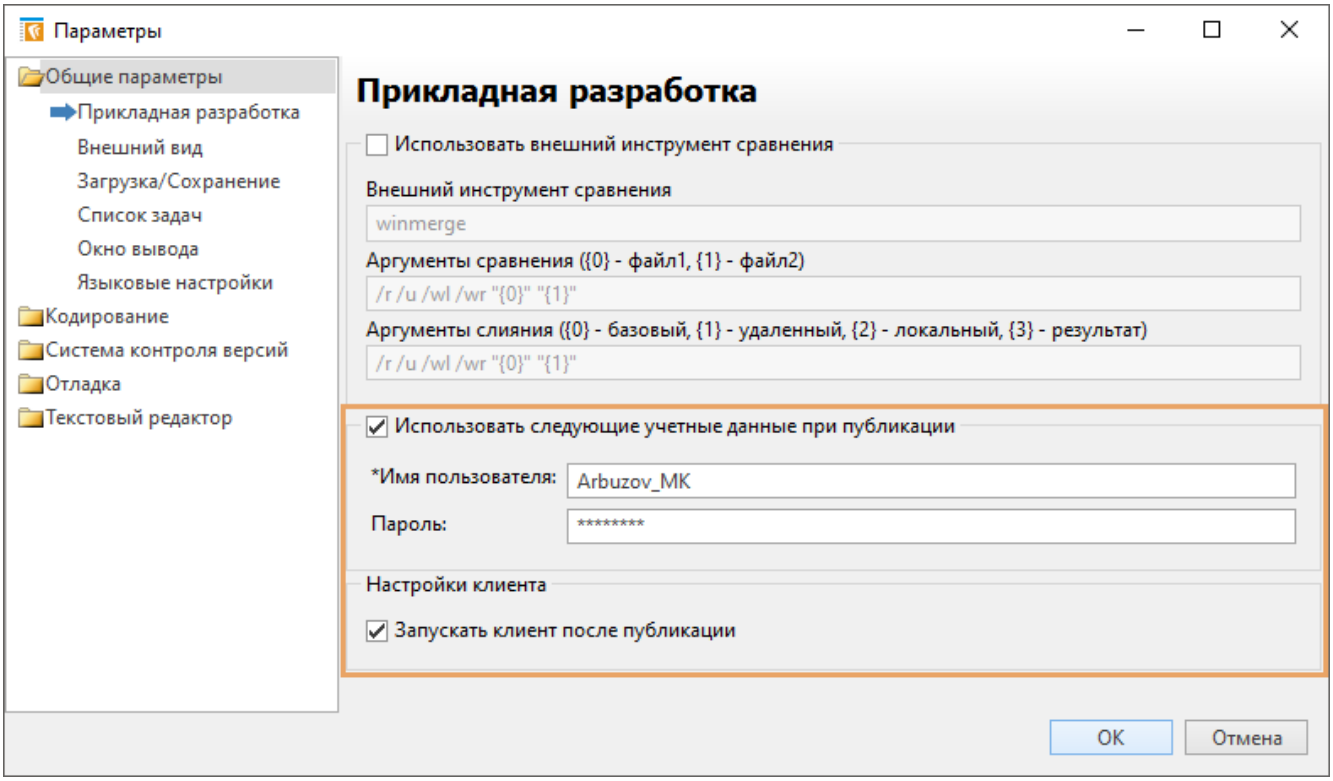
- **генерация структуры данных не требуется**, то запускается публикация в распределенную систему. При публикации изменения вносятся последовательно на каждом узле, то есть становится недоступен только обновляемый узел. Балансировщик нагрузки автоматически перенаправляет клиентские запросы на рабочие узлы, поэтому обновление происходит незаметно для пользователей. Процесс завершается после публикации изменений на всех узлах;
- **генерация структуры данных требуется**, то выводится предупреждение, публикация в распределенную систему не запускается. В этом случае запустите стандартную публикацию, без использования ключа **--distributed**. Подробнее см. раздел «Публикация с помощью командной строки».

Настройка параметров публикации и клиента

В параметрах можно указать учетные данные пользователя для публикации решений и признак, нужно ли автоматически запускать веб-клиент после публикации.

1. В главном меню последовательно выберите пункты **Инструменты**, **Параметры...**.

2. В узле **Прикладная разработка** задайте настройки:



Использовать следующие учетные данные при публикации. Установите флажок, чтобы указать данные пользователя, от имени которого по умолчанию будет запускаться публикация решений. Станут доступны поля ***Имя пользователя** и **Пароль**. Укажите учетные данные пользователя. Если после запуска среды разработки вы уже публиковали решения, то изменения вступят в силу после перезапуска среды.

ВАЖНО. Пользователь должен входить в предопределенную роль «Администраторы» или «Менеджеры конфигураций».

Запускать клиент после публикации. Установите флажок, если хотите, чтобы после публикации автоматически запускался веб-клиент.

3. Нажмите на кнопку **ОК**.

Структура пакета развертывания

Во время публикации *пакет развертывания* отправляется на веб-сервер. В нем содержатся:

- бинарные файлы публикуемых модулей
- интерфейсная сборка
- конфигурационные файлы разработки для веб-сервера и клиента
- скрипты публикации модулей
- xml-файл с описанием пакета

Бинарные файлы публикуемых модулей

Модули системы – это совокупность бинарных файлов (.NET-сборок), которые подгружаются в клиентские или серверные процессы и становятся доступными для использования. Для каждого модуля определены следующие библиотеки:

- <Полное имя модуля>.ClientBase.dll – подключается на сервере и на клиенте. Содержит обработчики клиентских событий, аргументы событий, клиентские функции, иконки и настройки внешнего вида списков;
- <Полное имя модуля>.Server.dll – подключается только на сервере. Содержит серверный прикладной код, схемы задач, разметку отчетов, сделанную в дизайнере;
- <Полное имя модуля>.Shared.dll – подключается на сервере и на клиенте. Содержит разделяемый прикладной код, метаданные элементов разработки и *строки локализации*, указанные в колонке Default;
- <Полное имя модуля>.Shared.resources.dll – дополнительная библиотека (satellite assembly), которая содержит локализованные ресурсы. Для каждого языка собирается своя ресурсная сборка и располагается в папке с именем, соответствующим коду языка;
- *сторонние сборки*, необходимые для работы модуля.

Интерфейсная сборка

Интерфейсная сборка – это библиотека Sungero.Domain.Interfaces.dll, которая обеспечивает взаимодействие между разными модулями. По сути это .NET-сборка, внутри которой объявлены интерфейсы, реализуемые классами сущностей всех установленных в системе модулей. Сборка подключается к клиентским и серверным процессам.

Конфигурационные файлы разработки для веб-сервера и клиента

Конфигурационные файлы (development.config) – файлы, которые используются для корректной работы модулей в системе. Их цель: сконфигурировать Unity-контейнер для классов сущностей. Соответственно, для веб-сервера и для клиента используются разные конфигурационные файлы, так как серверные и клиентские классы сущностей объявлены в различных сборках.

Конфигурационные файлы перегенерируется каждый раз при создании пакета развертывания по тем же метаданным, по которым создается интерфейсная сборка.

Скрипты публикации

Скрипты публикации делятся на:

- скрипты конвертации – предназначены для настройки конвертации данных модуля или решения с одной версии на другую;
- скрипты проверок – предназначены для проверки того, что данные модуля или решения можно конвертировать;
- скрипты ручных этапов – предназначены для поддержки совместимости между разными базами данных.

Подробнее см. раздел «Скрипты конвертации, проверок и ручных этапов».

XML-файл с описанием пакета

В файле находится список всех модулей, которые при публикации нужно установить, обновить или удалить.

Экспорт разработки

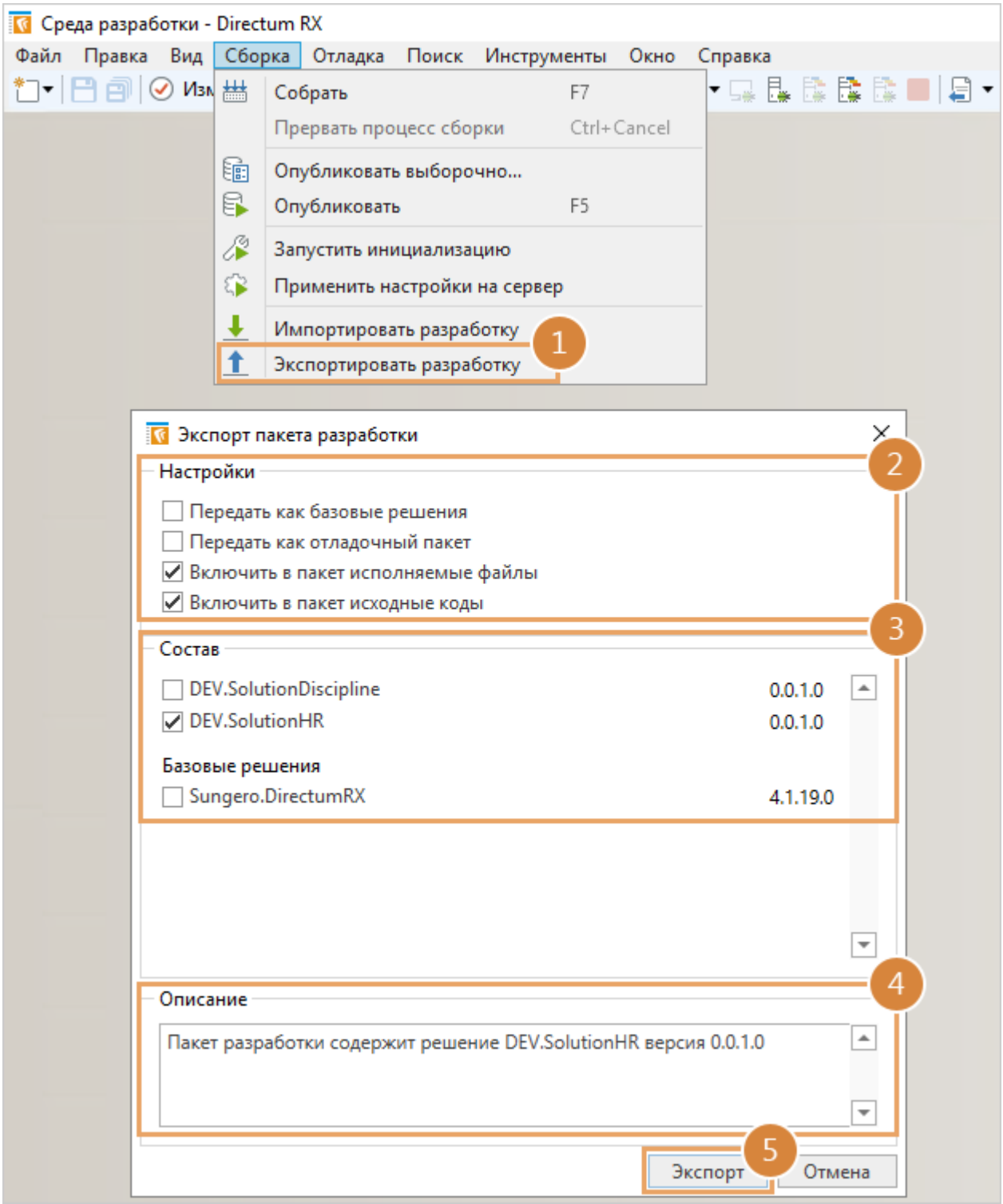
Исходные коды можно передавать из одной среды разработки в другую, например, чтобы обновить элементы *базовых решений* на новую версию системы, передать разработанные решения в стороннюю компанию для дальнейшей модификации или отладить решение на сервере с операционной системой Linux. Вместе с исходными кодами передаются настройки по умолчанию для бизнес-процессов и интерфейса.

Предположим, в среде разработки Directum RX в новом решении создан модуль и с помощью перекрытия изменено несколько справочников базовых решений. Кроме того, в решении мог быть создан тип задачи, для которого настроена схема процесса по умолчанию. Чтобы передать разработку, создайте *пакет разработки* с исходными кодами и настройками по умолчанию. Для этого экспортируйте разработку одним из способов:

- в визуальном режиме через среду разработки;
- в невизуальном режиме через среду разработки;
- в невизуальном режиме через утилиту DeploymentToolCore.

Экспорт через среду разработки

1. В главном меню среды разработки Directum RX последовательно выберите пункты **Сборка**, **Экспортировать разработку**:



2. В открывшемся окне задайте настройки:
   - **Передать как базовые решения**. Установите флажок, если необходимо, чтобы решения после импорта разработки добавились в среду разработки как *базовые*. Элементы базовых решений можно изменять только на *слое* в своем решении при помощи *перекрытия*;

- **Передать как отладочный пакет**. Установите флажок, если пакет разработки собирается для отладки решения. При публикации отладочного пакета в Directum RX в лог-файл веб-сервера записывается информация о номерах строк кода, в которых возникает ошибка. Кроме этого, к процессу отладки, запущенному на сервере, можно будет подключиться с другого компьютера;
- **Включить в пакет исполняемые файлы**. Установите флажок, если пакет разработки собирается для обновления продуктивной системы. Решения, импортированные из пакета, станут доступны для работы в клиентском приложении;
- **Включить в пакет исходные коды**. Установите флажок, если пакет разработки собирается для обмена разработкой. Решения, импортированные из пакета, станут доступны для изменения в среде разработки. Настройки по умолчанию для бизнес-процессов добавляются в пакет разработки всегда, даже если флажок не установлен.

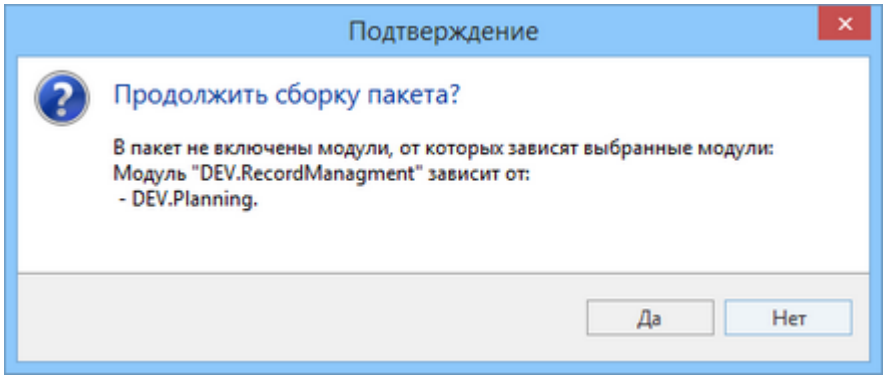
ПРИМЕЧАНИЕ. Если пакет разработки не является отладочным пакетом и в него помещены исполняемые файлы решения, то после успешного создания пакета разработки номер версии решения увеличится на 1.

3. Определите состав решений, которые нужно включить в пакет разработки. Установите рядом с ними флажки. Если выбранные решения зависят от других решений, установите флажки и для них.

   ВАЖНО. Если в решении были перекрыты абстрактные типы сущностей, например «Входящий документ» (IncomingDocumentBase), обязательно включите в пакет разработки базовые решения.

4. При необходимости добавьте текстовое описание. Оно отображается при импорте пакета в среду разработки Directum RX.
5. Нажмите на кнопку **Экспорт**.

   ВАЖНО. Если для выбранных решений или модулей настроены зависимости от модулей, которые не включены в пакет, появится предупреждение.



   Выберите одно из действий:

   Да. В пакет разработки добавятся исполняемые файлы решений и модулей, от которых зависят выбранные. Настройка создания пакета разработки продолжится.

   Нет. Настройка создания пакета разработки приостановится. Для продолжения в окне создания пакета разработки добавьте решения и модули, от которых зависят выбранные.

6. В открывшемся окне выберите папку для сохранения пакета разработки и введите имя пакета. Нажмите на кнопку **Сохранить**. Начнется создание пакета разработки.

После успешного создания пакета разработки появится соответствующее сообщение. В результате создадутся:

- файл пакета разработки с расширением ***.dat**;
- XML-файл с именем пакета разработки, в котором содержится информация о пакете.

Невизуальный экспорт через среду разработки

Пакет разработки можно создать в невизуальном режиме, например с помощью командной строки, PowerShell или программы FAR Manager. Для этого на локальном компьютере должна быть установлена среда разработки.

1. Создайте XML-файл конфигурации пакета разработки.

   СОВЕТ. При создании пакета разработки в визуальном режиме XML-файл генерируется автоматически. Можете использовать его, если ранее создавали пакет.

   Структура файла:

```
<?xml version="1.0"?>
   <DevelopmentPackageInfo
   xmlns:xsd="http://www.w3.org/2001/XMLSchema"
   xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
   <IsDebugPackage>{Передать как отладочный пакет}
   </IsDebugPackage>
     <PackageModules>
       <PackageModuleItem>
         <Id>{Идентификатор экспортируемого решения}
         </Id>
         <Name>{Код компании.Имя решения}
         </Name>
         <Version>{Версия решения}</Version>
         <IncludeAssemblies>{Включить в пакет исполняемые файлы}
         </IncludeAssemblies>
         <IncludeSources>{Включить в пакет исходные коды}
         </IncludeSources>
         <IsSolution>{Является решением}
         </IsSolution>
         <IsPreviousLayerModule>{Передать как базовые решения}
         </IsPreviousLayerModule>
       </PackageModuleItem>
   <!--Список модулей для экспорта -->
       <PackageModuleItem>
         <Id>{Идентификатор экспортируемого решения}
          </Id>
         <SolutionId>{Идентификатор экспортируемого модуля}
         </SolutionId>
         <Name>{Код компании.Имя модуля}
         </Name>
         <Version>{Версия модуля}</Version>
         <IncludeAssemblies>{Включить в пакет исполняемые файлы}
         </IncludeAssemblies>
         <IncludeSources>{Включить в пакет исходные коды}
         </IncludeSources>
         <IsSolution>{Является решением}
         </IsSolution>
         <IsPreviousLayerModule>{Передать как базовые решения}
         </IsPreviousLayerModule>
       </PackageModuleItem>
     </PackageModules>
   …
   </DevelopmentPackageInfo>
```

   Пример:

```
<!--Экспорт решения, которое состоит из одного модуля -->
    <?xml version="1.0"?>
    <DevelopmentPackageInfo
    xmlns:xsd="http://www.w3.org/2001/XMLSchema"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
    <IsDebugPackage>false</IsDebugPackage>
      <PackageModules>
        <PackageModuleItem>
          <Id>ccaf5fa7-4108-422d-bed4-1d4ea46488af</Id>
          <Name>DEV.RentSolution</Name>
          <Version>0.0.1.0</Version>
    <IncludeAssemblies>true</IncludeAssemblies>
          <IncludeSources>false</IncludeSources>
          <IsSolution>true</IsSolution>
          <IsPreviousLayerModule>false
          </IsPreviousLayerModule>
        </PackageModuleItem>
    <!--Данные модуля -->
        <PackageModuleItem>
          <Id>9dc3d9e2-9698-4643-ad95-d72cb55a2bb8</Id>
          <SolutionId>ccaf5fa7-4108-422d-bed4-1d4ea46488af</SolutionId>
          <Name>DEV.RentModule</Name>
          <Version>0.0.1.0</Version>
          <IncludeAssemblies>true
          </IncludeAssemblies>
          <IncludeSources>false
          </IncludeSources>
          <IsSolution>false
          </IsSolution>
          <IsPreviousLayerModule>false
          </IsPreviousLayerModule>
        </PackageModuleItem>
      </PackageModules>
    </DevelopmentPackageInfo>
```

ПРИМЕЧАНИЕ. Чтобы скопировать идентификатор решения или модуля, выделите его в дереве решений и в контекстном меню выберите пункт **Скопировать идентификатор**.

Если в решение входит несколько модулей, то добавьте секцию `<PackageModuleItem>` для каждого из них. При этом для решения и всех входящих в него модулей должны совпадать значения параметров **Включить в пакет исполняемые файлы**, **Включить в пакет исходные коды**, **Передать как базовые решения**.

2. С помощью командной строки откройте содержимое папки, в которой находится среда разработки Directum RX. Пример пути до папки: %PROGRAMFILES%\Directum Company\Sungero Development Studio.
3. Передайте параметры команды для утилиты DevelopmentStudio.exe в формате:

```
Start /w DevelopmentStudio.exe -c <configuration> -d <dev_package> --
increment-version <true/false> –PassThru
```

/w – флаг командной строки, который позволяет не дожидаться завершения всех дочерних процессов и тем самым ускоряет процесс создания пакета. Аналог команды в PowerShell – $process.WaitForExit();

-c (или **--configuration**) <**configuration**> – путь к созданному XML-файлу конфигурации пакета разработки. Обязательный параметр;

-d (или **--development-package**) <**dev_package**> – путь к создаваемому файлу с пакетом разработки. Обязательный параметр;

--increment-version <**true/false**> – признак, который определяет нужно увеличивать номер версии модулей и решений или нет. Если выбрано значение:

- **True** – номер версии у всех модулей и решений увеличится;
- **False** – номер версии у всех модулей и решений не увеличится.

Если параметр в команде не используется, то номер версии увеличится только у тех модулей и решений, для которых в XML-файле параметр <IsDebugPackage> имеет значение **False**, а параметр <IncludeAssemblies> имеет значение **True**.

-v (или **--set-version**) <**version**> – номер версии, которую надо задать для решения и его модулей, входящих в пакет разработки. Номер версии изменяется у решений и модулей, для которых в XML-файле параметр <IsDebugPackage> имеет значение **False**, а параметр <IncludeAssemblies> имеет значение **True**.

Пример команды для cmd:

```
Start /w DevelopmentStudio.exe -c "C:\Package\Development_package.xml" -d
"C:\Package\Development_package.dat" --increment-version “false”
```

Пример команды для PowerShell:

```
$process = $(Start-Process -FilePath DevelopmentStudio.exe -ArgumentList "-c
""C:\Package\Development_package.xml"" -d ""C:
\Package\Development_package.dat"" --increment-version false" -PassThru)
$process.WaitForExit()
```

Невизуальный экспорт через DeploymentToolCore

Пакет разработки можно создать без установки среды разработки, например, если его нужно экспортировать на компьютере с ОС Linux. Для этого используется команда инструмента Directum Launcher.

ПРИМЕЧАНИЕ. При необходимости вы можете выполнить экспорт с помощью DeploymentToolCore без использования Directum Launcher, запустив утилиту через Docker.

В зависимости от операционной системы способ вызова команды отличается:

Linux

```
./do.sh <команда>
```

Windows

```
do <команда>
```

Далее приведены команды для выполнения на компьютере с операционной системой Linux.

Чтобы создать пакет разработки:

1. Получите файлы с исходными кодами, например через Git.
2. Создайте XML-файл конфигурации пакета разработки.

   СОВЕТ. При создании пакета разработки в визуальном режиме XML-файл генерируется автоматически. Можете использовать его, если ранее создавали пакет.

   Структура файла:

```
<?xml version="1.0"?>
  <DevelopmentPackageInfo
  xmlns:xsd="http://www.w3.org/2001/XMLSchema"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
  <IsDebugPackage>{Передать как отладочный пакет}
```

```
  </IsDebugPackage>
    <PackageModules>
      <PackageModuleItem>
        <Id>{Идентификатор экспортируемого решения}
        </Id>
        <Name>{Код компании.Имя решения}
        </Name>
        <Version>{Версия решения}</Version>
        <IncludeAssemblies>{Включить в пакет исполняемые файлы}
        </IncludeAssemblies>
        <IncludeSources>{Включить в пакет исходные коды}
        </IncludeSources>
        <IsSolution>{Является решением}
        </IsSolution>
        <IsPreviousLayerModule>{Передать как базовые решения}
        </IsPreviousLayerModule>
      </PackageModuleItem>
  <!--Список модулей для экспорта -->
      <PackageModuleItem>
        <Id>{Идентификатор экспортируемого решения}
         </Id>
        <SolutionId>{Идентификатор экспортируемого модуля}
        </SolutionId>
        <Name>{Код компании.Имя модуля}
        </Name>
        <Version>{Версия модуля}</Version>
        <IncludeAssemblies>{Включить в пакет исполняемые файлы}
        </IncludeAssemblies>
        <IncludeSources>{Включить в пакет исходные коды}
        </IncludeSources>
        <IsSolution>{Является решением}
        </IsSolution>
        <IsPreviousLayerModule>{Передать как базовые решения}
        </IsPreviousLayerModule>
      </PackageModuleItem>
    </PackageModules>
  …
  </DevelopmentPackageInfo>
```

Пример:

```
<!--Экспорт решения, которое состоит из одного модуля -->
    <?xml version="1.0"?>
    <DevelopmentPackageInfo
    xmlns:xsd="http://www.w3.org/2001/XMLSchema"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
    <IsDebugPackage>false</IsDebugPackage>
      <PackageModules>
        <PackageModuleItem>
          <Id>ccaf5fa7-4108-422d-bed4-1d4ea46488af</Id>
          <Name>DEV.RentSolution</Name>
          <Version>0.0.1.0</Version>
    <IncludeAssemblies>true</IncludeAssemblies>
          <IncludeSources>false</IncludeSources>
          <IsSolution>true</IsSolution>
          <IsPreviousLayerModule>false
          </IsPreviousLayerModule>
        </PackageModuleItem>
    <!--Данные модуля -->
        <PackageModuleItem>
          <Id>9dc3d9e2-9698-4643-ad95-d72cb55a2bb8</Id>
          <SolutionId>ccaf5fa7-4108-422d-bed4-1d4ea46488af</SolutionId>
          <Name>DEV.RentModule</Name>
          <Version>0.0.1.0</Version>
          <IncludeAssemblies>true
          </IncludeAssemblies>
          <IncludeSources>false
          </IncludeSources>
```

```
      <IsSolution>false
      </IsSolution>
      <IsPreviousLayerModule>false
      </IsPreviousLayerModule>
    </PackageModuleItem>
  </PackageModules>
</DevelopmentPackageInfo>
```

3. В командной строке перейдите в папку с Directum Launcher и выполните команду:

```
/do.sh dt export-package --export_package <dev_package> --configuration
<configuration> --root <path> [--repositories <paths>]
```

Где:

--export-package <dev_package> – путь к создаваемому файлу с пакетом разработки. Обязательный параметр;

--configuration <configuration> – путь к созданному XML-файлу конфигурации пакета разработки. Обязательный параметр;

--root <path> – путь до корневой папки, в которой хранятся репозитории. Обязательный параметр;

--repositories <paths> или **--work <paths>** – имена папок с репозиториями базового и рабочего слоя соответственно. Должен быть задан один из параметров.

ВАЖНО. При выполнении команды на Linux в ключах **--export-package** и **--configuration** необходимо указать пути к файлам, находящимся в одной и той же папке.

Пример команды:

```
./do.sh dt export-package --export_package /home/user/CustomDev/DevRX.dat --
configuration /home/user/CustomDev/DevRX.xml --root /home/user --
repositories Base
```

4. При необходимости измените номер версии модулей и решений из указанных репозиториев. Для этого выполните одну из команд:

- увеличить номер версии.

```
./do.sh dt increment_version --root <path> --repositories <paths>
```

Пример команды:

```
./do.sh dt increment_version --root /home/user --repositories Base
```

- задать номер версии.

```
./do.sh dt set_version --version <version> --root <path> --repositories
<paths>
```

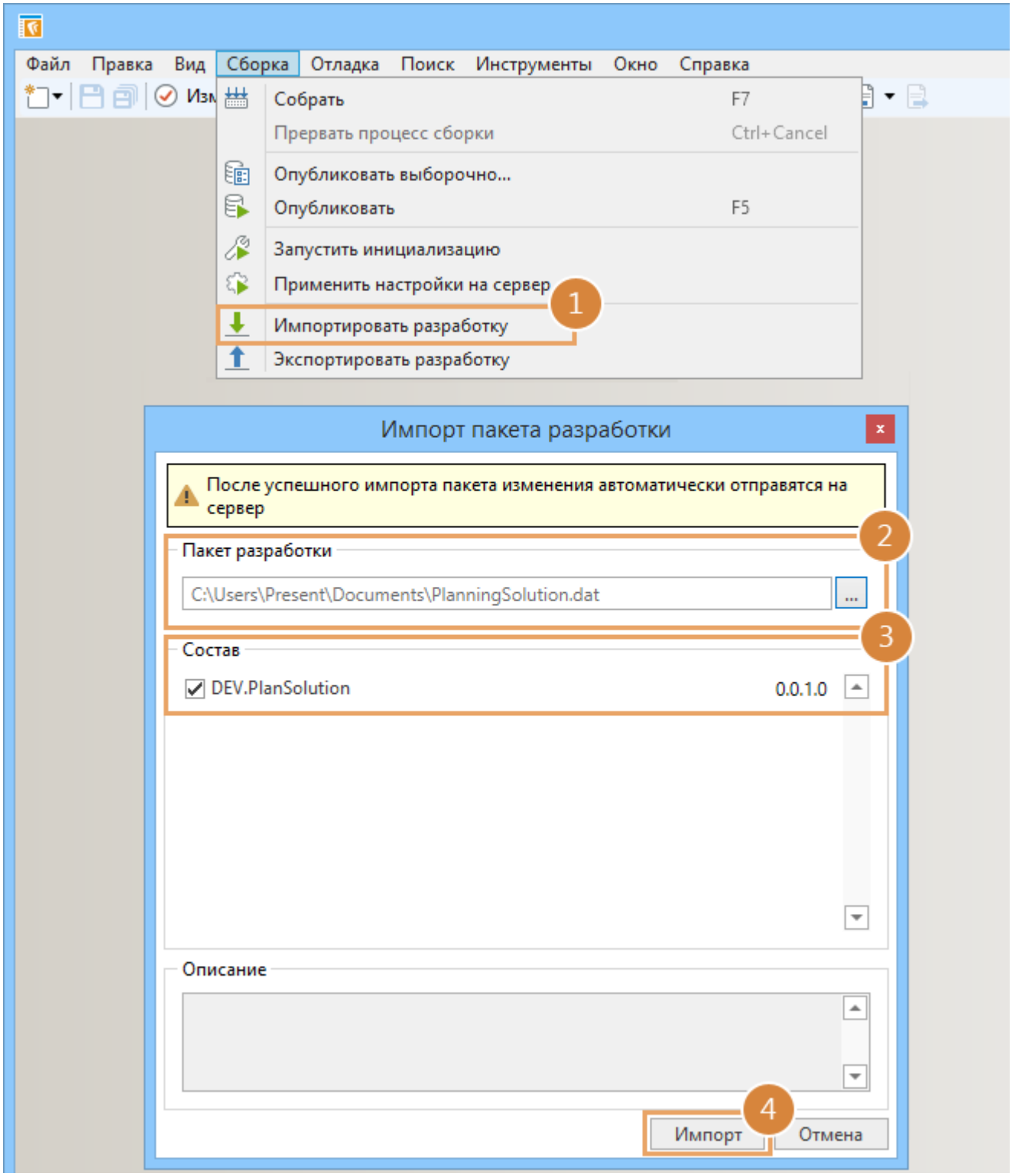
Пример команды:

```
./do.sh dt set_version --version 0.0.0.1 --root /home/user --repositories
Base
```

Импорт разработки

Если вам передали пакет разработки с новыми или измененными решениями, импортируйте его в свою среду разработки:

1. В главном меню среды разработки Directum RX последовательно выберите пункты **Сборка**, **Импортировать разработку**:



2. В открывшемся окне в поле **Пакет разработки** укажите путь к файлу *пакета разработки* с исходными кодами. Если исходных кодов в пакете нет, то при импорте появится соответствующее сообщение.

3. Определите состав решений, которые нужно перенести в среду разработки. Установите рядом с ними флажки. Если решения зависят от других решений, установите флажки и для них.
4. Нажмите на кнопку **Импорт**.

После успешного импорта пакета разработки появится соответствующее сообщение. В результате:

- исходные коды из пакета разработки сохранятся в *локальном репозитории* и отправятся на сервер с Git;
- импортированные решения появятся в дереве решений в среде разработки.

Особенности импорта базовых решений

После импорта пакета разработки с *базовыми решениями* обновите *локальную систему* Directum RX. Для этого:

1. Запустите публикацию решений.
2. Запустите инициализацию модулей.
3. Примените настройки по умолчанию для бизнес-процессов по кнопке **Применить настойки на сервер** на панели инструментов.

Если импортируется несколько базовых решений, в которых перекрыты одни и те же типы сущностей или модули, то изменения в перекрытиях объединятся.

Отладка программного кода

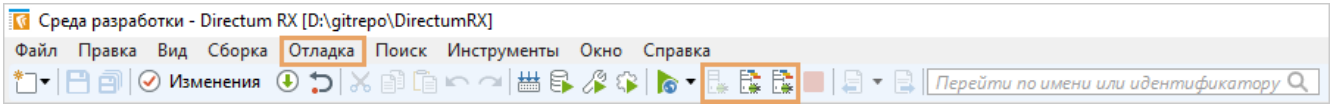
Отладка используется для поиска и устранения ошибок в программном коде и позволяет:

- выполнять программный код до *точек останова* и по шагам;
- просматривать значения переменных и стек вызова функций.

Разработка и отладка приложения выполняются в следующем порядке:

1. В среде разработки Directum RX создаются или изменяются решения, модули, типы сущностей и отчеты.
2. Выполняется сборка и публикация решений в *локальную систему* Directum RX для проверки их работы.
3. В локальной системе проверяется работа опубликованных решений.
4. Если возникает исключение или код работает некорректно, то выполняется отладка.
5. Найденные во время отладки ошибки исправляются, после чего отладка прекращается.
6. Решения повторно публикуются и проверяются в локальной системе.

Для запуска отладки используются кнопки на панели инструментов или пункт главного меню **Отладка**:



Кнопки и пункты меню доступны, когда запущены веб-сервер, общий сервис или сервис интеграции.

ПРИМЕЧАНИЕ. Отладка общего сервиса возможна, если он выполняет функции четырех сервисов: сервис выполнения блоков задач, сервис асинхронных событий, сервис отчетов, сервис виджетов.

Для отладки нужна привилегия «Отладка программ» (SeDebugPrivilege) в операционной системе Microsoft Windows. По умолчанию привилегия есть у локального администратора. Также ее можно выдать отдельно, не включая пользователя в группу администраторов.

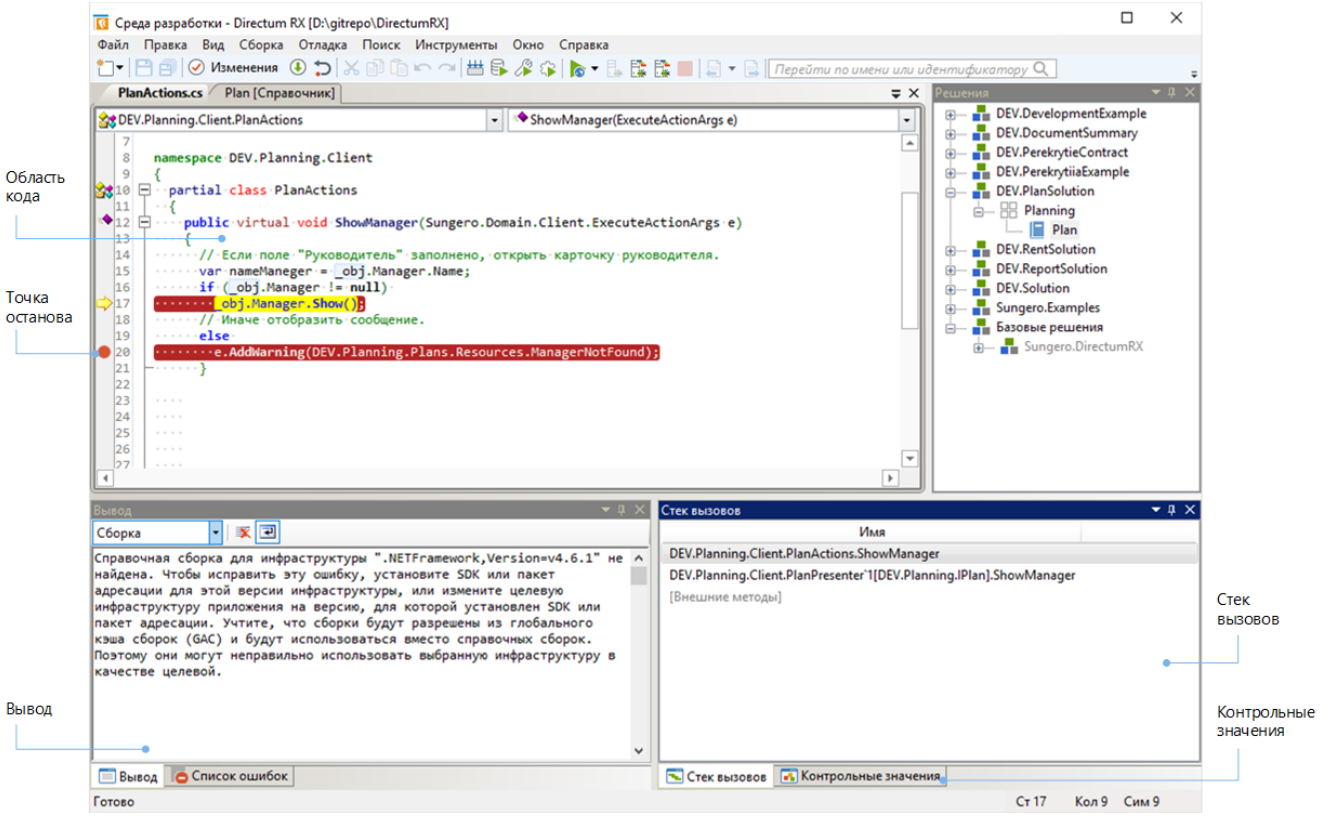
Чтобы отладить:

- клиентский и серверный код, нажмите на кнопку **Отладить веб-сервер**;
- функции инициализации, последовательно нажмите на кнопки **Отладить веб-сервер** и **Запустить инициализацию**;
- схемы задач и асинхронные обработчики, нажмите на кнопку **Отладить общий сервис**;
- интеграцию с другими системами, нажмите на кнопку **Отладить сервис интеграции**;
- фоновые процессы, в выпадающем списке **Запустить клиент** выберите пункт **Запустить фоновый процесс...**. В открывшемся окне выберите фоновый процесс и нажмите на кнопку **Отладка**.

ВАЖНО. Недоступна одновременная отладка разных видов кода, а также отладка кода изолированных областей.

Во время отладки для изменения доступен только проверяемый элемент разработки. Редакторы остальных элементов открываются на просмотр. Чтобы открыть редактор элемента на изменение, остановите отладку с помощью кнопки и переоткройте редактор, если ранее он был открыт на просмотр.

При отладке окно среды разработки Directum RX имеет вид:



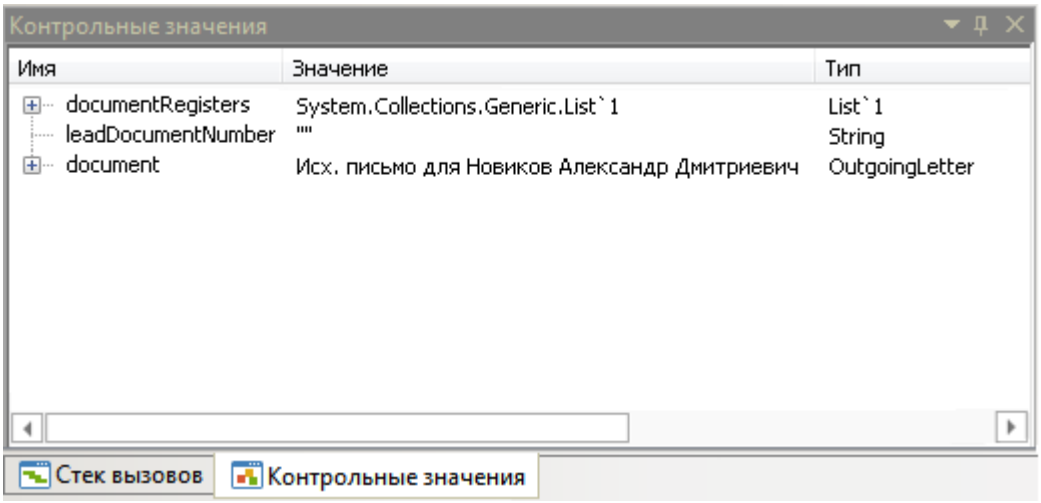
Область кода. Содержит тексты вычислений. Используется для редактирования программного кода, установки точек останова, наблюдения за выполнением кода. При преднамеренной остановке работы приложения и последовательном выполнении кода текущая выполняемая строка отмечается стрелкой и выделяется желтым цветом.

Панель «Вывод». Содержит информацию о текущем состоянии выполняемой программы, предупреждения, информацию об исключениях при отладке и публикации решений.

Панель «Стек вызовов». Предназначена для отображения последовательности вызова функций, выполненных при отладке. На панели отображаются имена всех вызванных функций. Для дополнительной информации можно включить отображение имен и значений аргументов. Для этого в контекстном меню панели выберите пункт **Показать имена аргументов** и/или **Показать значения аргументов**.

Из панели «Стек вызовов» можно перейти к программному коду, откуда был выполнен вызов конкретной функции или процедуры. Для этого щелкните левой кнопкой мыши по имени нужной функции или процедуры.

Панель «Контрольные значения». Предназначена для формирования списка переменных, значения которых необходимо отслеживать при отладке. Позволяет узнать значения конкретных переменных в определенный момент выполнения программного кода.



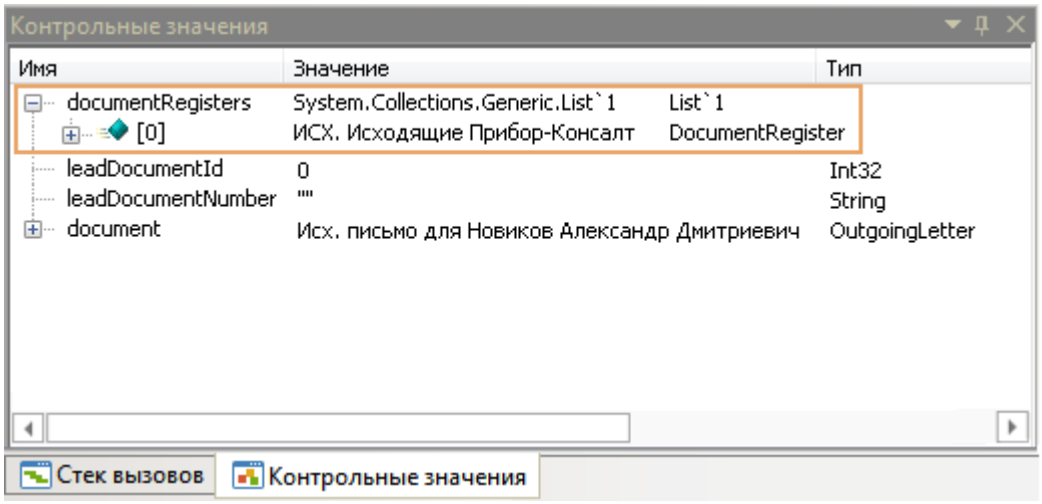
На панели отображаются столбцы:

- **Имя** переменной;
- **Значение** переменной на текущем шаге выполнения кода. Значения доступны для просмотра и копирования;
- **Тип** значения переменной.

Список переменных формируется вручную перед началом отладки. Чтобы добавить переменную на панель, воспользуйтесь одним из способов:

- в контекстном меню панели выберите пункт **Добавить контрольное значение** и в открывшемся окне введите имя переменной;
- выделите переменную в редакторе кода и в контекстном меню выберите пункт **Добавить контрольное значение**;
- выделите переменную в редакторе кода и переместите на панель с помощью механизма Drag&Drop.

У каждой переменной можно посмотреть свойства и их значения. Для этого нажмите на кнопку ⊞ рядом с переменной.

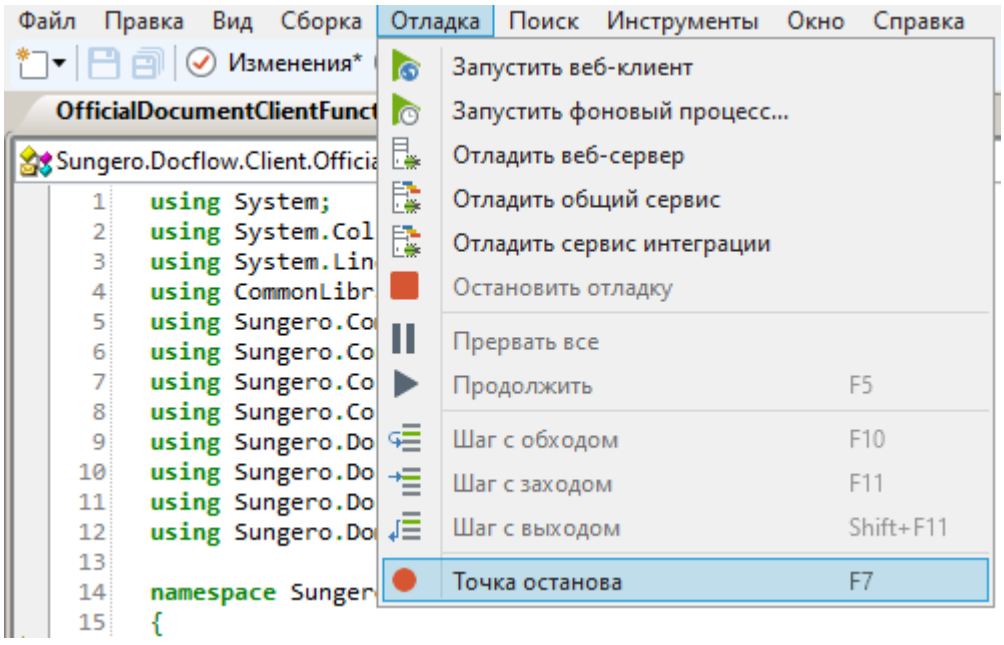


Точки останова

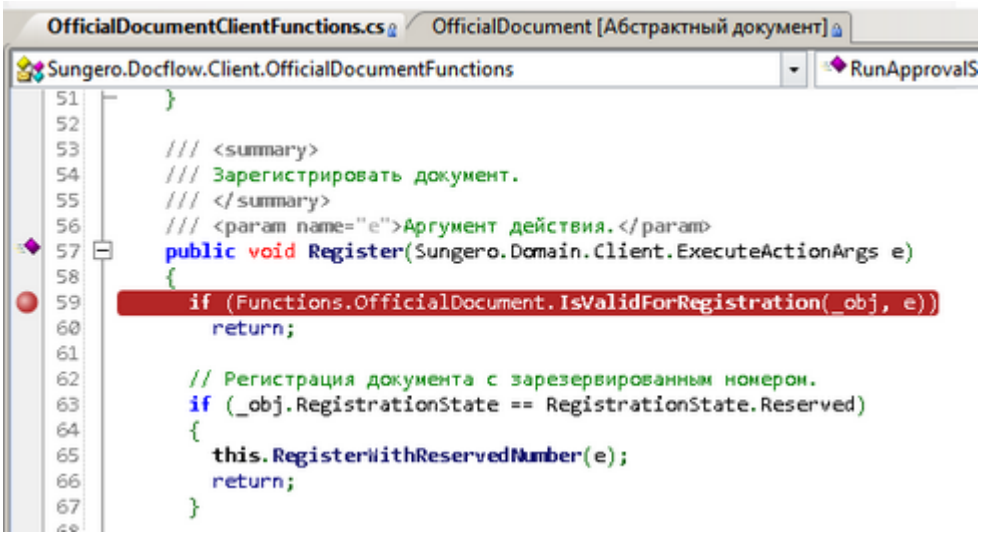
Для отладки определенной части кода используются *точки останова*. В этом случае при отладке программный код будет выполняться до точки останова. Также доступна установка нескольких точек останова, чтобы приостанавливать выполнение программы в разных местах программного кода.

Чтобы установить в программном коде точку останова, воспользуйтесь одним из способов:

- установите курсор в нужную строку кода и нажмите клавишу F7;
- щелкните левой кнопкой мыши на поле слева рядом с нужной строкой кода;
- установите курсор в нужную строку кода и в главном меню среды разработки Directum RX выберите пункт **Точка останова**:



В результате строка кода будет выделена красным цветом, слева отобразится значок точки останова:



При отладке выполнение программного кода приостановится, когда будет достигнута точка останова. Откроется окно редактора кода с выделенной точкой останова.

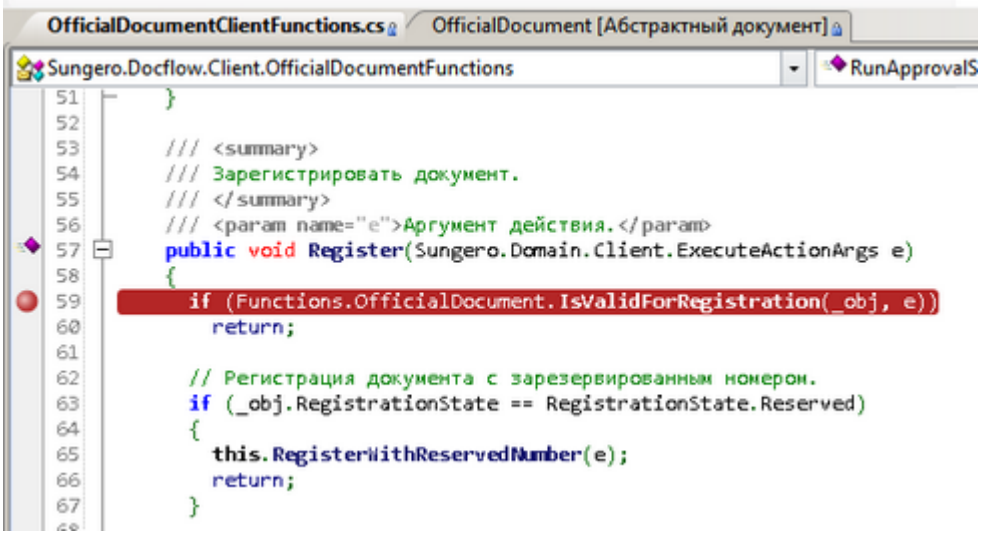
Выполнение отладки

В разделе описан порядок отладки клиентского кода. Отладка серверного кода, схем задач, асинхронных обработчиков, фоновых процессов и сервиса интеграции выполняется аналогично.

Предположим, что в среде разработки Directum RX создан модуль, в котором выполняется регистрация документов. При проверке работы в модуле обнаружилось, что при регистрации появляется исключение и регистрационный номер не отображается на карточках документов. Чтобы определить причины и исправить ошибку, выполните отладку клиентского кода. Для этого:

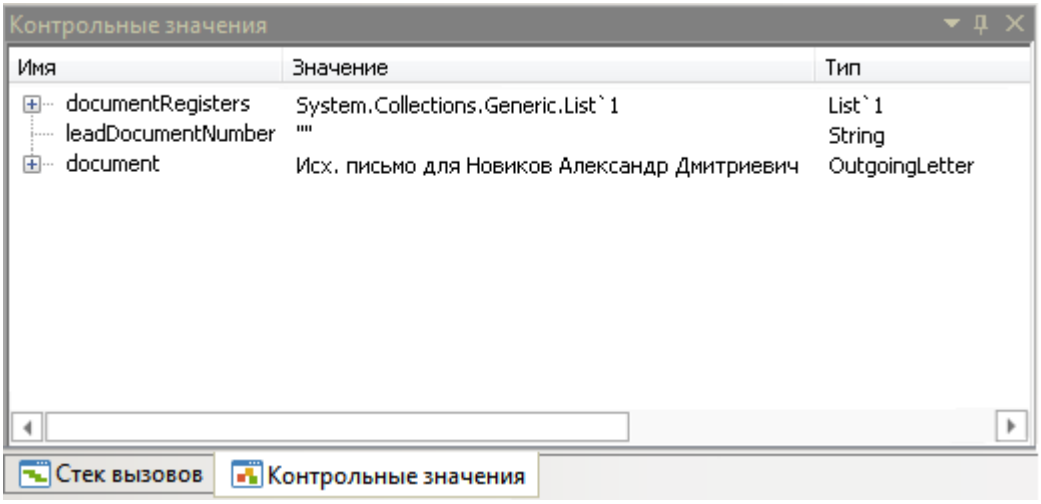
1. Откройте редактор типа документа, в котором не отображается регистрационный номер.
2. В группе «Функции» перейдите по ссылке **Клиентские**. Откроется редактор программного кода, в котором будут находиться все клиентские функции типа сущности.

3. В редакторе кода установите *точки останова* в том месте, в котором предположительно может вызываться исключение. В результате строка кода будет выделена красным цветом, а слева от нее отобразится значок точки останова:

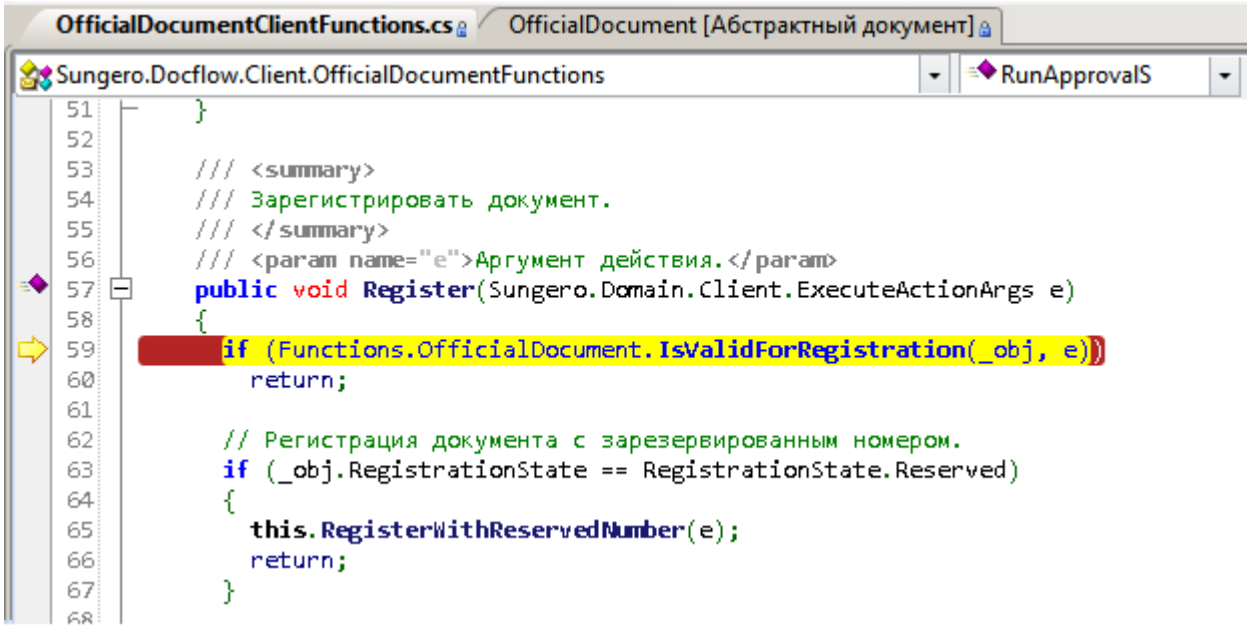


4. Чтобы отслеживать изменение значений переменных в процессе работы системы, добавьте нужные переменные на панель «Контрольные значения».

   При отладке будут отображаться значения добавленных переменных в текущий момент выполнения программного кода, например:

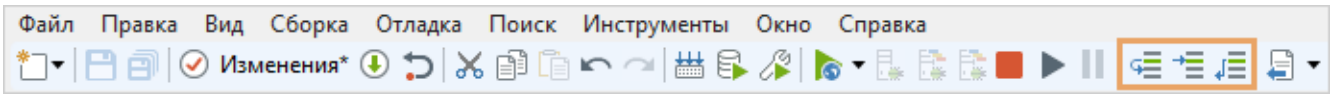


5. Запустите веб-клиент. Для этого на панели инструментов нажмите на кнопку **Запустить веб-клиент**. Откроется проводник локальной системы Directum RX от имени пользователя, под которым система запускалась в последний раз.
6. Запустите отладку клиентского кода. Для этого на панели инструментов нажмите на кнопку **Отладить веб-сервер**.
7. В веб-клиенте смоделируйте ситуацию, которая приведет к выполнению нужной части кода. Например, в карточке документа на закладке **Главная** в группе **Регистрация** нажмите на кнопку **Зарегистрировать**. Когда программа достигнет точки останова, выполнение приостановится. Откроется окно редактора кода с выделенной точкой останова. Следующая строка кода, которая будет выполнена, будет помечена желтым цветом:



ПРИМЕЧАНИЕ. Если не устанавливать точки останова, то после запуска отладки выполнение программы приостановится на первой обнаруженной ошибке. Если ошибок нет, то программа выполнится без остановок.

8. Чтобы последовательно выполнять строки программного кода, на панели инструментов нажмите на одну из кнопок:



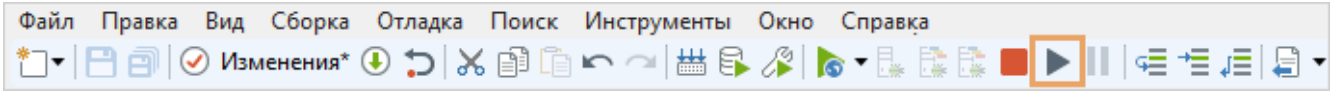
Шаг с обходом или нажмите клавишу F10. Будет выполнена текущая строка кода и переход к следующей. Если в исполняемой строке содержится функция, то она будет полностью выполнена.

Шаг с заходом или нажмите клавишу F11. Будет выполнена текущая строка кода и переход к следующей. Если в исполняемой строке содержится функция, то будет выполнен переход в начало этой функции.

Шаг с выходом или нажмите клавиши SHIFT+F11. Будет выполнена текущая функция и переход к основному коду.

ПРИМЕЧАНИЕ. Кнопки становятся доступными при запуске отладки.

При необходимости для повторного запуска приложения на панели инструментов нажмите на кнопку **Продолжить выполнение отлаживаемого процесса** или нажмите клавишу F5.



9. При обнаружении ошибки скорректируйте разработку.
10. Опубликуйте модуль или решение в локальную систему Directum RX и проверьте результаты разработки.
11. При необходимости повторите отладку.

Горячие клавиши

В среде разработки доступны следующие горячие клавиши:

Часто используемые комбинации клавиш

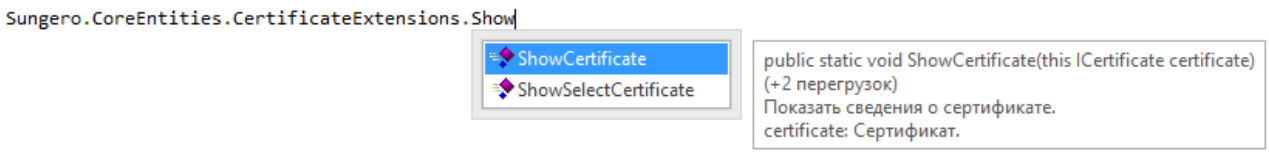
F7	Собрать решение
F5	Опубликовать решение
CTRL+SPACE	Вызвать список для автодополнения кода (Intellisense)
CTRL+I	Форматировать документ
CTRL+R	Создать ресурс или изменить его, если ресурс уже создан
CTRL+/	Закомментировать/раскомментировать выделенный фрагмент
CTRL+F (CTRL+SHIFT+F), CTRL+H	Открыть окно «Поиск и замена»

Справка

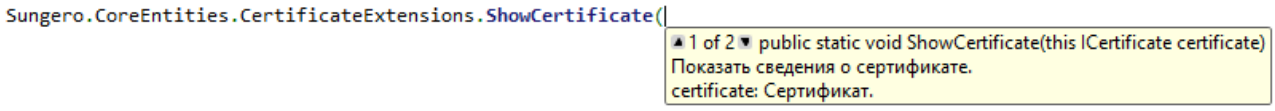
F1 – вызвать справку по работе с текущим окном. Например, если вызвать справку из окна «Редактирование ресурса», то откроется раздел «Пример локализации сообщения валидации».

Работа с кодом

CTRL+SPACE – вызвать список для автодополнения кода (Intellisense). Используйте горячие клавиши в любом месте кода, появится список допустимых членов типа или пространства имен. Также список появится автоматически, если после имени любого объекта поставить точку. Справа отобразятся параметры выделенного члена и описание.



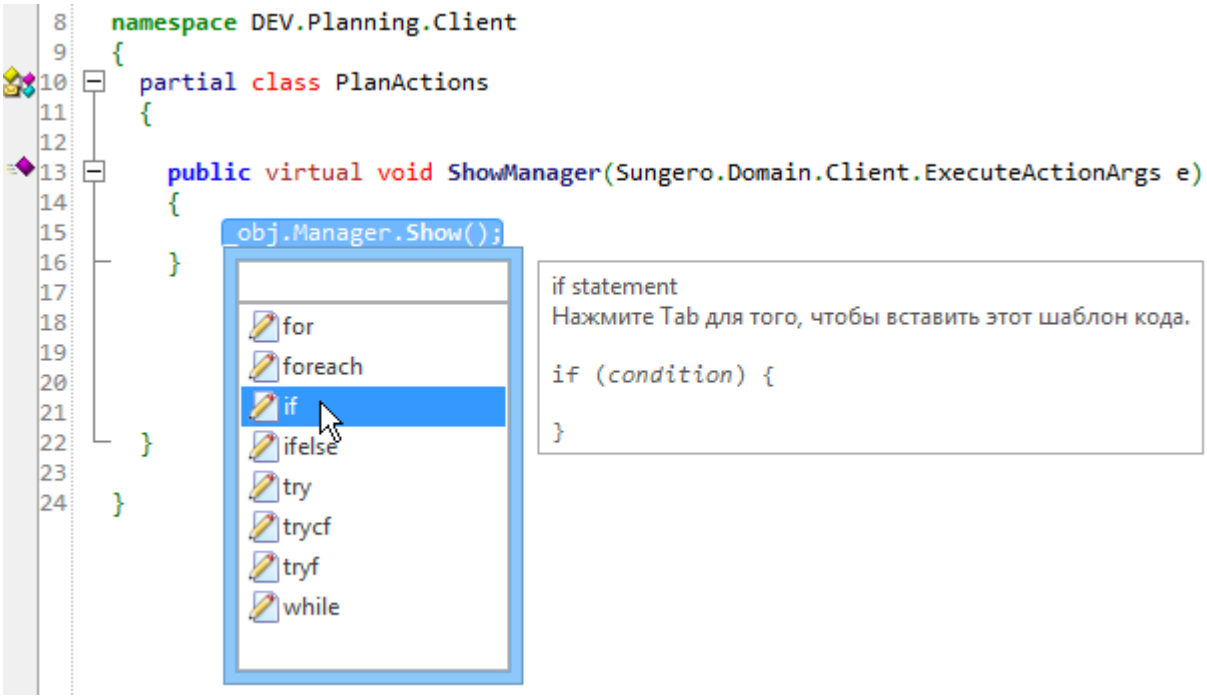
Выберите имя функции (метода) и установите открывающуюся скобку. Автоматически появляется подсказка с краткими сведениями о параметрах:



Чтобы посмотреть параметры перегруженных функций и методов, используйте клавиши СТРЕЛКА ВВЕРХ и СТРЕЛКА ВНИЗ. Чтобы вызвать подсказку с краткими сведениями о параметрах вручную, используйте сочетание клавиш CTRL+SHIFT+SPACE в области параметров.

CTRL+I – форматировать документ. Установите курсор в любом месте документа и используйте сочетание клавиш. К коду автоматически применится оформление и выравнивание на основании настроек, заданных в параметрах среды разработки. Если форматирование нужно применить только к части кода, выделите его в редакторе и используйте сочетание клавиш.

CTRL+J – заключить выделенный код в сниппет (фрагмент кода): for, foreach, if, ifelse, try, trycf, tryf, while. Подробнее см. в документации Microsoft статью «Фрагменты кода C#».



CTRL+SHIFT+G – сгенерировать произвольный GUID, уникальный 128-битный идентификатор. Например, для создания константы.

CTRL+R – создать ресурс или изменить его, если ресурс уже создан. Установите курсор на *строке локализации* и используйте сочетание клавиш. Откроется окно для редактирования ресурса.

CTRL+W – выделить блок кода, в котором установлен курсор. Каждое нажатие клавиш увеличивает область выделения.

CTRL+/ (в английской раскладке) – закомментировать или раскомментировать выделенный фрагмент.

/// – написать комментарий к коду в XML-формате:

```
/// <summary>
///
/// </summary>
/// <param name=""></param>
/// <returns></returns>
```

Содержимое комментария зависит от кода. Например, если в коде реализован метод без параметров, то при вводе /// структура комментария будет без строки `/// <param name=""></param>`. А если в коде метода указан параметр, то его название автоматически укажется в теге `<param>`, например, `/// <param name="responsible"></param>`.

Поиск в коде

CTRL+F (CTRL+SHIFT+F) – открыть окно поиска по коду и ресурсам. Введите текст и выберите область поиска: в текущем открытом документе, в выделенном фрагменте, во всех открытых документах или в решении целиком. Для навигации по совпадениям используйте клавишу F3.

F3 – найти следующее совпадение из результатов последнего поиска. Если поиск не запускали, откроется окно для поиска строки в текущем документе.

CTRL+F3 – найти следующее вхождение выделенной строки в текущем документе. Курсор переместится на первое найденное совпадение.

CTRL+H – заменить строку. Откроется окно для поиска и замены строки. Замена работает только в строках кода. Для замены строк в ресурсах используйте редакторы системных и прикладных ресурсов.

CTRL+E – запустить инкрементный поиск (поиск с постепенным уточнением). В редакторе кода курсор мыши заменится на значок . Начните вводить текст, например, message. Выполнится поиск вниз от текущего положения в документе и подсветится первое найденное совпадение. То есть сперва подсветится первое найденное слово, начинающееся на m, потом на me и так далее. Чтобы отменить поиск, нажмите левую кнопку мыши внутри редактора либо клавишу ESC.

CTRL+SHIFT+E – запустить инкрементный поиск в обратном порядке. В редакторе кода курсор мыши заменится на значок .Совпадения ищутся вверх от текущего положения в документе. Чтобы отменить поиск, нажмите левую кнопку мыши внутри редактора либо клавишу ESC.

Навигация по коду

CTRL+N – переместить курсор в поле для навигации по элементам разработки.

CTRL+клавиша минус (-) – Назад. Используется для перехода к предыдущему активному редактору кода в истории навигации. Если вкладка с редактором была закрыта, она откроется заново.

CTRL+клавиша плюс (+) – Вперед. Используется для перехода к следующему активному редактору кода в истории навигации.

CTRL+M – установить закладку рядом с номером строки кода. Закладки помогают быстро перемещаться между блоками кода.

CTRL+, (в английской раскладке) – перейти к предыдущей закладке.

CTRL+. (в английской раскладке) – перейти к следующей закладке.

CTRL+G – перейти к…. Используется для перехода к введенному в поле номеру строки, названию класса или файла.

CTRL+ENTER – перейти к определению метода, функции, класса и т.д. Установите курсор на название объекта и используйте сочетание горячих клавиш. Откроется редактор кода с определением объекта.

F12 – найти все ссылки, где используется метод, функция, класс, свойство и т.д. Установите курсор на имени объекта и используйте горячую клавишу. В окне результатов поиска отобразится список файлов, где используется объект.

F6 – найти перегрузки и наследников для объекта, на котором установлен курсор.

CTRL+B – перейти на парную фигурную скобку.

CTRL+SHIFT+M – свернуть или развернуть блок кода.

CTRL+SHIFT+L – свернуть или развернуть все блоки кода.

CTRL+SHIFT+P – свернуть определения методов, функций, классов и т.д.

Сборка и публикация решений

F7 – собрать решения.

CTRL+BREAK – прервать процесс сборки.

F5 – опубликовать решения.

Отладка программного кода

F5 – продолжить отладку.

F10 – выполнить шаг с обходом.

F11 – выполнить шаг с заходом.

SHIFT+F11 – выполнить шаг с выходом.

Особенности разработки для Linux

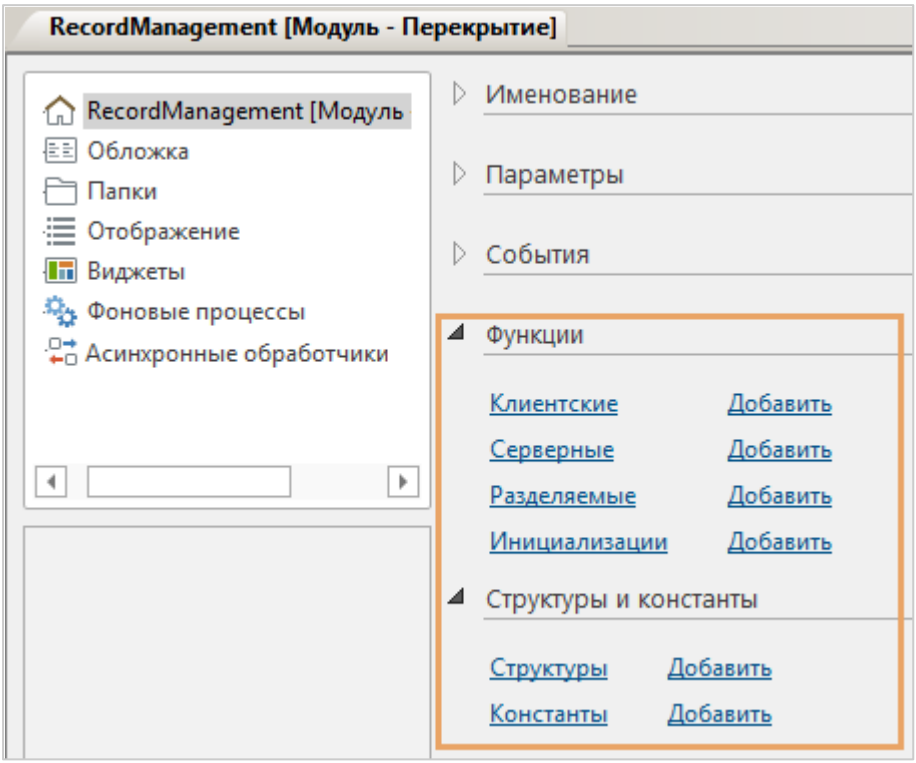
Среда разработки устанавливается на компьютер разработчика с операционной системой Microsoft Windows. Соответственно разработчик может собрать и опубликовать изменения в локальную систему только на Windows. Чтобы протестировать свои доработки в системе на Linux:

1. Разверните тестовую систему Directum RX на отдельном сервере с Linux.
2. В среде разработки создайте отладочный пакет.
3. Опубликуйте пакет разработки с помощью утилиты DeploymentToolCore.
4. Проверьте доработки. Используйте лог-файлы веб-сервера: при публикации отладочного пакета в них записывается информация о номерах строк кода, в которых возникла ошибка.
5. Если все работает корректно, создайте пакет разработки и опубликуйте его в продуктивную систему. Иначе в среде разработки внесите изменения, пересоберите отладочный пакет и протестируйте работу на Linux еще раз.

Программный код

Для написания программного кода в среде разработки используется язык C# с некоторыми ограничениями, которые накладываются платформой Sungero. При возникновении вопросов по синтаксису C#, обращайтесь, например, к справочнику на сайте Microsoft.

Программный код пишется в отдельном редакторе кода, к которому можно перейти из редакторов элементов разработки. Например, для добавления функций модуля, констант и структур, перейдите по соответствующим ссылкам:



В программном коде используются следующие элементы:

- Функции – именованные последовательности операторов, предназначенные для многократного использования;
- Структуры – объекты, которые состоят из нескольких логически связанных между собой полей разного типа. Например, в структуре можно хранить персональную информацию о сотруднике: ФИО (текст), дату рождения (дата и время), паспортные данные (число);
- Константы – заданные значения, которые не могут изменяться при выполнении программы. Используются, например, когда необходимо зафиксировать срок рассмотрения документа или имя нового типа прав.

Для реализации логики работы с сущностью используются события типов сущностей. Они могут выполняться как на веб-сервере, так и в клиентском приложении.

При необходимости вы можете группировать в отдельных файлах и папках программный код функций, структур или констант. Например, вы можете объединить все серверные функции типа сущности **OfficialDocument**, которые вызываются при подписании документа. Благодаря этому улучшается структурированность кода и упрощается навигация по нему.

При написании программного кода используется объектная модель.

Дополнительно при разработке учитывайте:

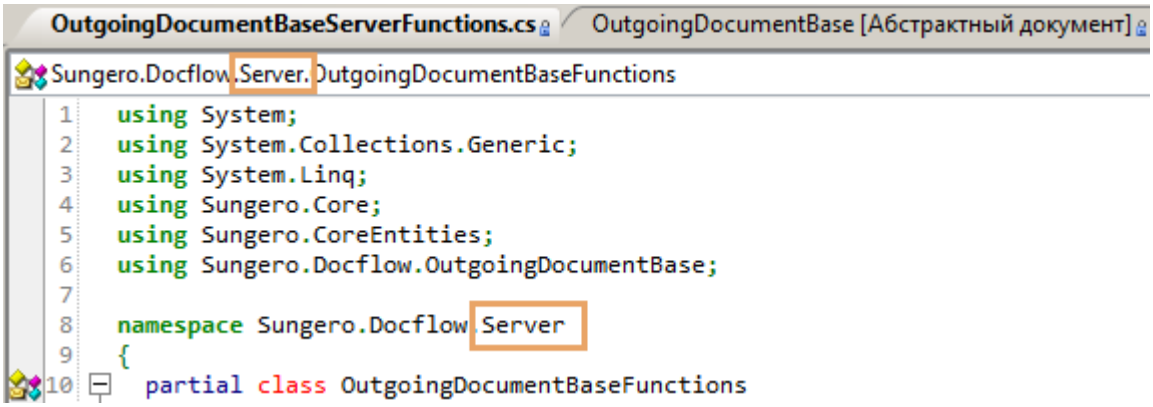
- не рекомендуется использовать оператор OR в коде обработчика события фильтрации, через который идет обращение к большим спискам (более 1 млн. записей);
- не используйте SQL-запросы, вместо них данные можно получать LINQ-запросами;
- сразу пишите английскую локализацию. В дальнейшем, если понадобится менять состав поддерживаемых языков локализации, это поможет сэкономить ресурсы.

ПРИМЕЧАНИЕ. При завершении работы с типом сущности, модулем или решением закрываются также и вкладки кода, связанные с ними. Например, при закрытии редактора типа сущности OfficialDocument закрываются редакторы серверного, клиентского, разделяемого кода, структур, констант и т.д., если они были открыты.

Как правило, пользователю удобнее работать с системой, в которой каждое действие выполняется мгновенно, то есть занимает менее 1 секунды. Если простое действие в системе, например открытие списка, выполняется 2,5 секунды, то оно выглядит тормозящим, более 3 секунд – медленным. Чтобы действия выполнялись быстро, необходимо писать оптимальный код. Подробнее см. рекомендации по разработке для масштабируемых систем.

Виды кода

Программный код делится на серверный, клиентский и разделяемый. При необходимости в редакторе кода вы можете посмотреть, какой код используется в данный момент:



Серверный код

Выполняется на сервере. К серверному коду относятся, например:

- обработчики событий сущности;
- обработчики событий фильтрации выбора из списка;
- серверные функции.

Так как серверный код выполняется на веб-сервере, в нем не должно быть логики, которая связана с пользовательским интерфейсом (GUI). Это означает, что в серверном коде запрещено показывать окна, выводить диалоги.

Серверный код выполняется от имени пользователя, который совершает действия в веб-клиенте. Например, сохраняет карточку или выбирает разработанный отчет на обложке модуля.

При написании серверного кода необходимо учитывать, что доступ к файловым и сетевым ресурсам из кода может ограничиваться политиками безопасности, и учетная запись, с правами которой работает веб-сервер, может иметь недостаточно прав для доступа к тем или иным файлам.

Клиентский код

Выполняется в клиентском приложении. К клиентскому коду относятся, например:

- обработчики событий действий сущности;
- обработчики события на изменение в контроле для свойств сущности;
- обработчики событий формы;
- клиентские функции.

Клиентский код, в отличие от серверного, позволяет обращаться к пользовательскому интерфейсу, выводить окна сообщений и диалоги. При этом для клиентского кода существуют ограничения, связанные с трехзвенной архитектурой и оптимизацией работы системы через интернет. Из кода нельзя:

- выполнять SQL-запросы;
- напрямую создавать, удалять и получать сущности (для этого используются серверные функции).

Для выполнения данных задач используются серверные функции. В серверную функцию добавляется атрибут **Remote**, с помощью которого ее можно будет вызывать в клиентском коде.

ВАЖНО. Серверные функции с небольшими вычислениями рекомендуется группировать в одну функцию, чтобы минимизировать количество обращений к веб-серверу.

Разделяемый код

Разделяемый код может выполняться как на сервере, так и в клиентском приложении.

К разделяемому коду относятся, например:

- обработчики события на изменение значения свойства;
- обработчики событий на добавление и удаление для свойств-коллекций.

На разделяемый код накладываются ограничения одновременно и серверного, и клиентского кода. Используется, когда не нужно вызывать диалоги и напрямую создавать, удалять и получать сущности. Например, когда нужно выполнить какие-либо вычисления: получить срок выдачи документа, ожидаемый срок по задаче. Такие вычисления можно будет вызывать и в серверном, и в клиентском приложении.

Функции

В системе доступно создание функций в редакторах модулей и типов сущностей.

Функции модуля создаются, когда предполагается, что одна и та же логика будет выполняться в нескольких типах сущностей или отчетах модуля.

Функции типов сущностей создаются, когда предполагается, что они будут вызываться в разных местах программного кода. Например, в других функциях этого же типа сущности или событиях.

Функции делятся на клиентские, серверные, разделяемые. Также есть функции инициализации и функции вычисляемых выражений. Серверные и разделяемые функции модулей можно сделать интеграционными.

Клиентские функции

Выполняются в клиентском приложении. Используются для обращения к пользовательскому интерфейсу, вывода диалогов и сообщений.

Функции можно выбирать при настройке действий в обложках модулей. Чтобы в редакторе обложек отображалось локализованное значение имени и описания действия, создайте клиентскую функцию с атрибутом **LocalizeFunction**.

ПРИМЕЧАНИЕ. Для настройки действий в обложке модуля проводника доступны клиентские функции с типом возвращаемого значения **void**, **List<IEntity>**, **IQuieryable<IEntity>**.

Пример. Клиентская функция модуля Company

```
   /// <summary>
/// Показать настройки видимости организационной структуры.
/// </summary>
public virtual void ShowVisibilitySettings()
{
  if (!VisibilitySettings.AccessRights.CanUpdate())
  {
    Dialogs.ShowMessage(Resources.VisibilitySettingsNotAvailable);
    return;
  }
  Functions.Module.Remote.GetVisibilitySettings().Show();
}
```

Серверные функции

Выполняются на сервере. Используются для получения, создания и удаления сущностей. В качестве параметров и типа возвращаемого значения для серверных функций используются только:

- простые типы:

short, short?	Guid, Guid?	double, double?
int, int?	bool, bool?	Uri
long, long?	decimal, decimal?	string
char, char?	DateTime, DateTime?	

- типы сущностей. Например, IEmployee, ICompany, IContract;
- список List. Содержит простые типы или сущности. Например, List<int>, List<ICompany>. Используется, если функция возвращает небольшой список;
- параметризованный тип IQueryable (запрос на получение данных). Содержит только сущности. Например, IQueryable<IContractualDocument>. Используется при работе с

большими объемами данных. В этом случае данные с сервера подгружаются порциями. Функция вызывается при загрузке каждой порции. Это позволяет работать с данными без ожидания полной загрузки с сервера.

ВАЖНО. В серверных функциях не рекомендуется выполнять длительные расчеты. Также не рекомендуется часто вызывать серверные функции из клиентского приложения.

Пример 1. Действие CreateProjectDocument в справочнике ProjectCore

```
   // Клиентский код, вызывающий серверную функцию.
public virtual void CreateProjectDocument(Sungero.Domain.Client.ExecuteActionArgs
e)
{
  var document = Functions.ProjectCore.Remote.CreateProjectDocument(_obj);
  document.Show();
}
```

Пример 2. Серверная функция справочника ProjectCore

```
   /// <summary>
/// Создать документ по проекту.
/// </summary>
/// <returns>Документ.</returns>
[Remote]
public IOfficialDocument CreateProjectDocument()
{
  var document = ProjectDocuments.Create();
  document.Project = _obj;
  return document;
}
```

Разделяемые функции

Выполняются как на сервере, так и в клиентском приложении. Могут использоваться, например, для изменения доступности, обязательности и видимости свойств в зависимости от значения другого свойства. Вызываются в событиях «Изменение значения свойства» и «Обновление формы».

Пример 1. Разделяемая функция справочника DocumentRegister (Журналы регистрации)

```
   /// <summary>
/// Установить обязательность свойств в зависимости от заполненных данных.
/// </summary>
public virtual void SetRequiredProperties()
{
  _obj.State.Properties.RegistrationGroup.IsRequired =
  _obj.Info.Properties.RegistrationGroup.IsRequired ||
  _obj.RegisterType == RegisterType.Registration;
}
```

В примере свойство **Группа регистрации** должно быть обязательным для заполнения, если установлен признак обязательности свойства или если в свойстве **Тип журнала** указано значение «Регистрация». Данные вычисления должны сработать при изменении свойства **Тип журнала**, а также при обновлении формы, например, чтобы при отмене изменений корректно отрабатывалась смена доступности, обязательности и видимости свойств.

Пример 2. Событие «Изменение значение свойства» свойства RegisterType (Тип журнала) справочника DocumentRegister

```
    public                                  virtual                                  void
        RegisterTypeChanged
        (Sungero.Domain.Shared.EnumerationPropertyChangedEventArgs e)
{
  . . .
  Functions.DocumentRegister.SetRequiredProperties(_obj);
}
```

Пример 3. Событие «Обновление формы» справочника DocumentRegister

```
    public override void Refresh(Sungero.Presentation.FormRefreshEventArgs e)
{
  Functions.DocumentRegister.SetRequiredProperties(_obj);
. . .
}
```

Функции инициализации

Функции инициализации выполняются на сервере. Создаются только в редакторах модулей. Используются для заполнения справочников значениями по умолчанию, создания сущностей, для которых в редакторе типа сущности установлен флажок **Создавать только программно**, выдачи пользователям прав доступа на вычисляемые папки модуля, создания предопределенных групп пользователей и ролей и т.д.

Функции вычисляемых выражений

Функции используются для преобразования данных при настройке вычисляемых выражений. Например, можно создать функцию, которая укажет ФИО сотрудника в винительном падеже или текущую дату сдвинет на заданное количество рабочих дней. Подробнее см. раздел «Создание функций вычисляемых выражений».

Атрибуты

Атрибуты предназначены для указания способа вызова функций. Например, с помощью атрибутов серверную функцию можно вызывать в клиентском коде, или функцию одного модуля вызывать из функции другого модуля.

Существуют следующие атрибуты:

- **Public**. Указывается, если функцию, структуру или константу нужно использовать в других модулях или решениях. Атрибут можно использовать в функциях инициализации;
- **[Public(WebApiRequestType = RequestType.Get)]** и **[Public(WebApiRequestType = RequestType.Post)]**. Указываются для создания функций интеграции.

  ПРИМЕЧАНИЕ. Атрибуты можно использовать только в функциях модуля.
- **Remote**. Указывается при создании серверной функции, которая будет вызываться в клиентском коде.

Для атрибута **Remote** можно дополнительно указать параметры:

- **PackResultEntityEagerly**. Добавляется, чтобы функция возвращала сущность вместе со свойствами (*«жадная» загрузка*). Например, чтобы при создании задачи сразу за один запрос загружались значения полей и вложения. Возможные значения: **true, false**. Указывается в формате: `[Remote(PackResultEntityEagerly = true)]`;
- **IsPure**. Рекомендуется использовать для оптимальной работы функций, которые не вносят изменения в данные. Например, для функций получения объекта или выборки данных. Возможные значения: **true**, **false**. Указывается в формате: `[Remote(IsPure = true)]`.

  Например, добавьте параметр в серверную функцию получения данных, которая будет вызываться в клиентском событии карточки или действия сущности. Если параметр не указать, то при публикации могут возникнуть ошибки;
- **UseTransaction**. Отвечает за выполнение функции в одной транзакции. Возможные значения: **true**, **false**. Указывается в формате: `[Remote(UseTransaction = true)]`.

  Значение по умолчанию – **true**. В этом случае процесс сохранения нескольких сущностей проходит в одной транзакции. Ошибка при сохранении одной сущности отменит транзакцию, и все последующие изменения не сохранятся. Рекомендуется указывать значение **false**, если нужно сохранить большое количество сущностей, например в цикле. Тогда ошибка при сохранении одной сущности не повлияет на сохранение остальных;

- **Hyperlink**. Указывается при создании клиентских функций, которые выполняются при нажатии на ссылку. Используйте атрибут, например, когда при нажатии на ссылку необходимо открыть объект внешней системы, или в текст задачи добавить ссылку, при нажатии на которую откроется список или карточка типа сущности.

  ПРИМЕЧАНИЕ. Атрибут можно использовать только в функциях модуля.

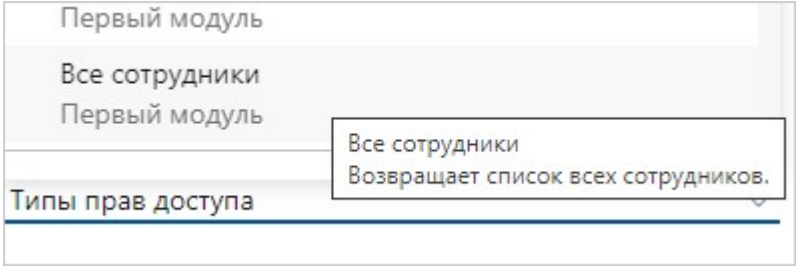
  Для корректного отображения ссылки задайте ее название в редакторе ресурсов. В параметрах атрибута укажите имя добавленного ресурса:

  ```
  [Hyperlink(DisplayNameResource="<Имя ресурса>")]
  ```

  Для вызова функции используйте класс Hyperlinks:

  ```
  var url = Hyperlinks.Functions.<Имя клиентской функции>();
  ```

- **LocalizeFunction**. Указывается при создании клиентских функций, связанных с действиями обложек модулей, а также для серверных функций без параметров, которые возвращают модель контрола состояния **StateView**. Используйте атрибут, чтобы локализовать имя и описание функции, которые отображаются в редакторе форм для контрола состояния и в редакторе обложек:

  Атрибут указывается в формате: `[LocalizeFunction("<Отображаемое имя функции>", "<Описание>")]`.

  Пример:

  ```
  [LocalizeFunction("Все сотрудники", "Возвращает список всех сотрудников.")]
  ```

```
public IQueryable<IEmployee> EmployeesAll()
{
  return Employees.GetAll();
 }
```

Вместо имени и описания функции также можно передавать ресурсы:

```
[LocalizeFunction("EmployeesAll_ResourceKey",
   "EmployeesAll_DescriptionResourceKey")]
public IQueryable<IEmployee> EmployeesAll()
{
  return Employees.GetAll();
 }
```

- **Converter**. Указывается при создании серверных функций сущности, которые преобразуют один объект в другой по заданной логике. Используйте атрибут, например, если в шаблоне документа с автозаполняемыми полями нужно настроить параметры для изменения значений: преобразования числа в строку, склонения по падежам, преобразования строки (изменение регистра, взятие первой буквы и т.д.).

  В атрибуте можно указывать только один параметр:

  ```
  [Converter("<Имя ресурса>")]
  ```

  Для создания функции-конвертера добавьте серверный метод с модификатором **public** и заключите его в атрибут **Converter[]**, а также добавьте ресурсы для отображаемого значения функции-конвертера в системе.

  ПРИМЕЧАНИЕ. Атрибут можно использовать только в статических функциях типов сущностей.

- **ExpressionElement**. Указывается при создании функций вычисляемых выражений.

Атрибуты указываются в квадратных скобках перед объявлением функции. Если функция будет вызываться в клиентском коде и использоваться в других модулях, то атрибуты **Remote** и **Public** указываются через запятую перед ее объявлением.

Пример 1. Серверная функция модуля Company

```
    /// <summary>
/// Получить подразделение по id.
/// </summary>
/// <param name="id">Id.</param>
/// <returns>Подразделение.</returns>
[Public, Remote(IsPure = true)]
public static IDepartment GetDepartmentById(long id)
{
  return Departments.Get(id);
}

// Пример вызова функции из клиентского кода модуля Shell:
// var department =
Sungero.Company.PublicFunctions.Module.Remote.GetDepartmentById(departmentId);
```

Пример 2. Серверная функция модуля RecordManagement

```
    /// <summary>
/// Создать задачу на ознакомление с документом.
/// </summary>
/// <param name="document">Документ, который отправляется на
ознакомление.</param>
```

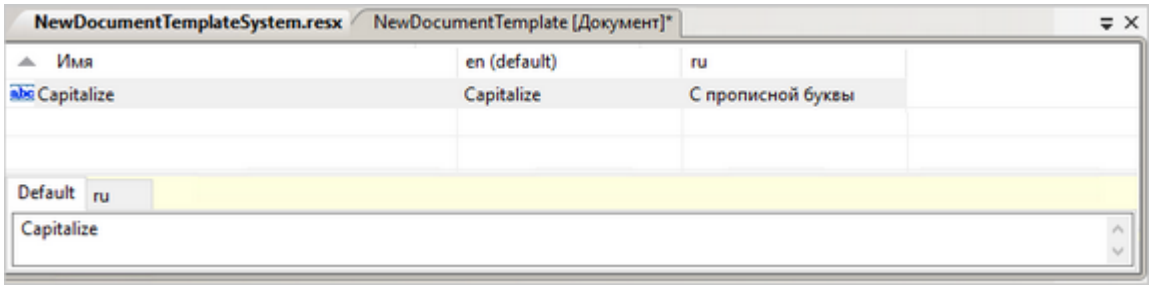
```
/// <returns>Задача на ознакомление по документу.</returns>
[Remote(PackResultEntityEagerly = true), Public]
public static IAcquaintanceTask CreateAcquaintanceTask(IOfficialDocument
document)
{
  var newAcqTask = AcquaintanceTasks.Create();
  newAcqTask.DocumentGroup.OfficialDocuments.Add(document);
  return newAcqTask;
}
```

Пример 3. Серверная функция модуля RecordManagement

```
   /// <summary>
/// Получить списки ознакомления.
/// </summary>
/// <returns>Списки ознакомления.</returns>
[Public, Remote(IsPure = true)]
public IQueryable<IAcquaintanceList> GetAcquaintanceLists()
{
  return AcquaintanceLists.GetAll()
      .Where(a => a.Status ==
Sungero.RecordManagement.AcquaintanceList.Status.Active);
}
```

Пример 4.Серверная функция капитализации слова (написания с прописной буквы)

Добавление ресурса для отображения в системе значения «С прописной буквы» функции капитализации слова:



Добавление ресурса для функции:

```
   [Converter("Capitalize")]
public static string Capitalize(string target)
{
  if (string.IsNullOrEmpty(target))
    return target;
  return target.Substring(0, 1).ToUpper(CultureInfo.CurrentCulture) +
(target.Length > 1 ? target.Substring(1) : string.Empty);
}
```

Модификаторы

Модификаторы предназначены для задания уровня доступности функции, который определяет возможность использования функции из другого кода.

При объявлении функций можно использовать модификаторы:

- **public**. Указывается, чтобы задать к функции общий доступ из любого другого кода с максимальными правами без ограничений. Модификатор автоматически добавляется в определении функции при ее создании;
- **virtual**. Указывается, чтобы функцию можно было переопределять в наследниках или перекрывать. Модификатор можно использовать в функциях инициализации.

  Пример объявления:

  ```
  public virtual void FullName();
  ```

- **override**. Указывается при переопределении функции базового типа сущности. Переопределять можно только те наследуемые функции, для которых в базовом типе сущности указан модификатор **virtual**. Чтобы переопределить функцию, в наследнике объявите эту функцию с модификатором **override**.

  Пример объявления:

  ```
  public override void FullName();
  ```

  Подробнее см. переопределение функции базового типа документа OfficialDocument.

  ВАЖНО. Функции перекрытых модулей, для которых указан модификатор **virtual**, также можно переопределять с помощью модификатора **override**.

- **static**. Указывается, когда при вызове функции типа сущности не нужно создавать экземпляр конкретной сущности. Параметр **_obj** будет недоступен.

  ВАЖНО. Модификатор **static** можно применять для функций типов сущностей. Для функций модуля не обязательно указывать модификатор, т.к. они считаются статическими, даже если модификатор **static** не указан. При этом в работе с отчетами есть особенность: если в тексте объекта отчета используется серверная функция модуля, то в ее объявлении нужно явно указать модификатор **static**.

  Пример объявления:

  ```
  public static IContractualDocument
    GetContractualDocumentIgnoreAccessRights(long documentId).
  ```

Пример 1. Создание функции в типе сущности «Официальный документ»

Предположим, что для типа сущности **Официальный документ** необходимо добавить серверную функцию для получения списка заданий на возврат документа. При этом для функции есть условия:

- будет вызываться из клиентского кода. Поэтому перед объявлением функции указывается атрибут **Remote**;
- функция не вносит изменения в данные. Поэтому перед объявлением функции указывается параметр **IsPure**;
- не нужен доступ к экземпляру сущности. Поэтому в объявлении функции указывается модификатор **static**.

Модификатор **public** в объявлении функции добавится автоматически.

```
   /// <summary>
/// Получить задания на возврат по документу.
/// </summary>
/// <param name="returnTask">Задача.</param>
/// <returns>Задания на возврат.</returns>
[Remote(IsPure = true)]
public static List<Sungero.Workflow.IAssignment>
GetReturnAssignments(Sungero.Workflow.ITask returnTask)
{
  return GetReturnAssignments(new List<Sungero.Workflow.ITask>()
{ returnTask });
}
```

Пример 2. Создание функции в модуле «Делопроизводство»

Предположим, что в модуле «Делопроизводство» необходимо добавить серверную функцию для создания поручения. При этом есть условия:

- функция будет вызываться из клиентского кода и использоваться в функции другого модуля. Поэтому перед объявлением функции указываются атрибуты **Remote** и **Public**;
- карточка поручения должна загружаться вместе с вложениями за один запрос с веб-сервера. Поэтому перед объявлением функции указывается параметр **PackResultEntityEagerly**;
- чтобы функцию можно было переопределять, в объявлении указывается модификатор **virtual**.

В объявлении функции модификатор **static** можно не указывать, т.к. это функция модуля, которая по умолчанию считается статической.

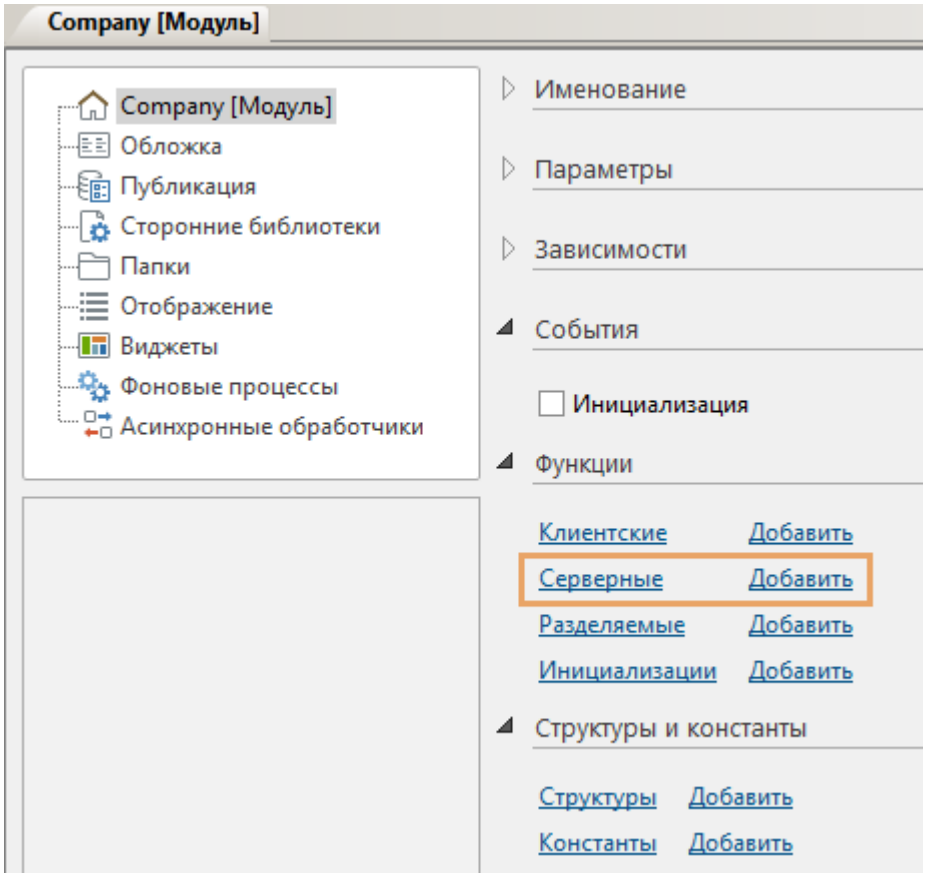
Модификатор **public** в объявлении функции добавится автоматически.

```
/// <summary>
/// Серверная функция для создания поручения по документу.
/// </summary>
/// <param name="document">Документ на рассмотрение.</param>
/// <returns>Поручение по документу.</returns>
[Remote(PackResultEntityEagerly = true), Public]
public virtual IActionItemExecutionTask
CreateActionItemExecution(IOfficialDocument document)
{
  return CreateActionItemExecution(document, Assignments.Null);
}
```

Создание функций модуля

Предположим, что необходимо создать серверную функцию. После этого нужно создать клиентскую функцию и вызвать из нее серверную. Для этого:

1. В редакторе модуля перейдите в узел «Модуль».
2. В группе «Функции» перейдите по ссылке **Добавить** рядом с серверной функцией.



3. В открывшемся редакторе кода доработайте имя функции, добавьте комментарий с описанием и напишите ее код. При необходимости, добавьте тип возвращаемого значения, входные параметры и атрибуты функции. В примере кода, который написан ниже, используется атрибут **Remote**, так как далее серверная функция вызывается в клиентском коде.

   ВАЖНО. Если серверная функция будет использоваться в выражении объекта отчета, то в объявлении функции явно укажите модификатор **static**.

```
/// <summary>
/// Серверная функция для создания организации.
/// </summary>
[Remote]
public static ICompany CreateCompany()
{
   return Companies.Create();
}
```

4. Сохраните изменения.
5. В группе «Функции» перейдите по ссылке **Добавить** рядом с клиентской функцией.

6. В открывшемся редакторе кода доработайте имя клиентской функции, добавьте комментарий с описанием и вызов серверной функции. Чтобы вызвать серверную функцию используйте формат: **Functions.Module.Remote.<Имя функции>(<Параметры>)**. В формате используется **Remote**, потому что из клиентского кода вызывается серверная функция. Формат вызова других функций описан ниже.

```
/// <summary>
/// Клиентская функция для создания организации.
/// </summary>
public static void CreateCompany()
{
  Functions.Module.Remote.CreateCompany().Show();
}
```

Чтобы посмотреть или изменить существующий код, в группе «Функции» перейдите по ссылке с названием функции.

Формат вызова функций

Функции вызываются по формату **Functions.Module.<Имя функции>(<Параметры>)**. В формат вызова могут добавляться атрибуты:

- **Remote**, если в клиентском коде вызывается серверная функция. Формат вызова: Functions.Module.**Remote**.<Имя функции>(<Параметры>);
- **Public**, если функция вызывается в функции другого модуля. При этом формат вызова функции меняется: Module.**PublicFunctions**.<Имя сущности>.<Имя функции>(<Параметры>).

Формат вызова можно сократить, если функция вызывается:

- из функции типа сущности. **Functions** можно не указывать;
- из функции того же модуля, в котором она создана. **Functions.Module** можно не указывать.

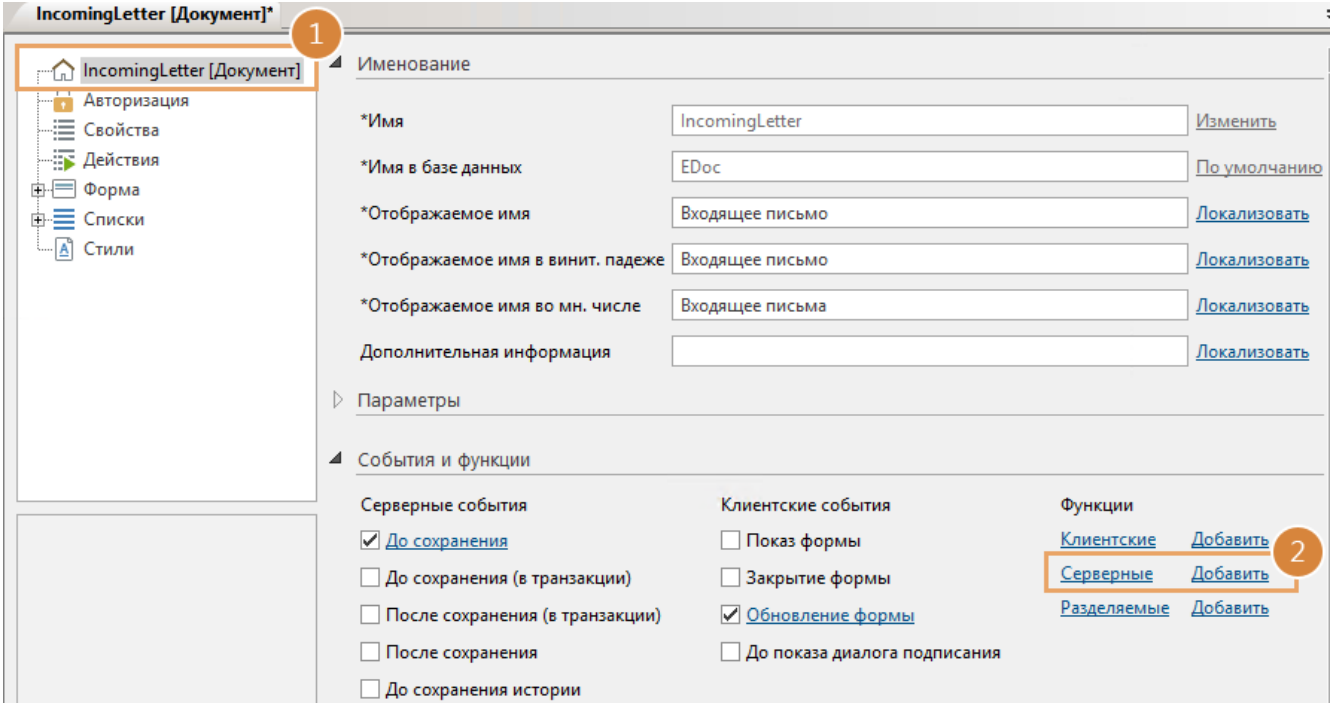
Создание функций типов сущностей

Предположим, что для входящего письма необходимо искать дубли документов и показывать их в списке. Для выполнения задачи необходимо:

- создать серверную функцию, которая будет искать дубли документа;
- в обработчике действия вызвать серверную функцию.

Для этого:

1. Откройте узел «Документ» в редакторе типа сущности «Входящее письмо».

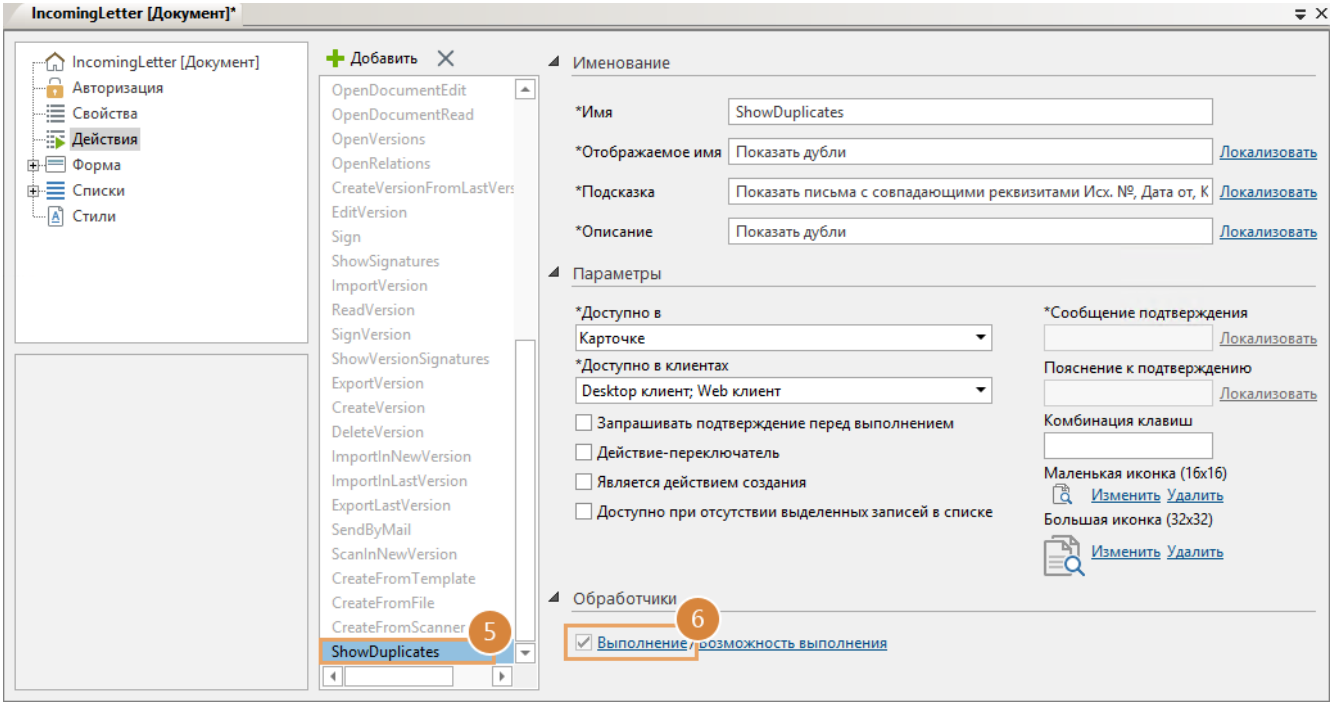


2. В группе «Функции» перейдите по ссылке **Добавить** рядом с серверной функцией.
3. В открывшемся редакторе доработайте имя функции, добавьте комментарий с описанием и напишите ее код. При необходимости добавьте тип возвращаемого значения, входные параметры, атрибуты и модификаторы функции.

```
   /// <summary>
/// Серверная функция для получения дублей входящего письма.
/// </summary>
/// <param name="letter">Письмо для проверки.</param>
/// <param name="documentKind">Вид письма.</param>
/// <param name="businessUnit">Наша организация.</param>
/// <param name="inNumber">Номер входящего документа.</param>
/// <param name="dated">Дата письма.</param>
/// <param name="correspondent">Корреспондент.</param>
/// <returns>Письма, дублирующие текущее.</returns>
[Remote(IsPure = true)]
public static IQueryable<IIncomingLetter> GetDuplicate(IIncomingLetter letter,
Sungero.Docflow.IDocumentKind documentKind, Company.IBusinessUnit businessUnit,
string inNumber,DateTime? dated, Parties.ICounterparty correspondent)
{
  return IncomingLetters.GetAll()
  .Where(l => documentKind == l.DocumentKind)
  .Where(l => dated.HasValue && dated == l.Dated)
  .Where(l => businessUnit != null && businessUnit == l.BusinessUnit)
  .Where(l => inNumber != null && inNumber == l.InNumber)
  .Where(l => correspondent != null && correspondent == l.Correspondent)
  .Where(l => !Equals(letter, l));
}
```

4. Сохраните изменения.

5. Перейдите в узел «Действия» и выберите действие **ShowDuplicates**.



6. Для действия добавьте обработчик события **Выполнение**. Для этого в группе «Обработчики» установите флажок и перейдите по ссылке **Выполнение**.
7. В открывшемся редакторе добавьте комментарий с описанием и вызов серверной функции. В формат вызова перед именем функции добавьте атрибут **Remote**.

```
/// <summary>
    /// Клиентская функция для формирования списка дублей входящего письма.
    /// </summary>
    public virtual void ShowDuplicates(Sungero.Domain.Client.ExecuteActionArgs
    e)
    {
      var duplicates = Functions.IncomingLetter.Remote.GetDuplicate(_obj,
    _obj.DocumentKind, _obj.BusinessUnit, _obj.InNumber, _obj.Dated,
    _obj.Correspondent);
      if (duplicates.Any())
        duplicates.Show();
      else
        Dialogs.NotifyMessage(IncomingLetters.Resources.DuplicateNotFound);
    }
```

8. Сохраните изменения.

Формат вызова функций

Функции вызываются по формату **Functions.<Имя типа сущности>.<Имя функции>(<параметры>)**. В формат вызова могут добавляться атрибуты:

- **Remote**, если в клиентском коде вызывается серверная функция. Формат вызова: Functions.<Имя типа сущности>.**Remote**.<Имя функции>(<Параметры>);
- **Public**, если функция вызывается в функции другого модуля. Формат вызова: **PublicFunctions**.<Имя типа сущности>.<Имя функции>(<Параметры>).

Создание функций инициализации

Инициализация – это процесс приведения системы в состояние готовности к использованию. Функции инициализации модулей *базового решения* Directum RX запускаются автоматически после установки или обновления системы и используются для ее первоначальной настройки. Например, с помощью функций инициализации происходит:

- заполнение справочников Валюты, Виды документов, Правила согласования значениями по умолчанию;
- создание предопределенных групп пользователей и ролей – «Делопроизводители», «Ответственные за договоры», «Регистраторы входящих документов» и другие;
- выдача необходимых прав пользователям, группам или ролям на типы справочников, типы документов, вычисляемые папки и отчеты.

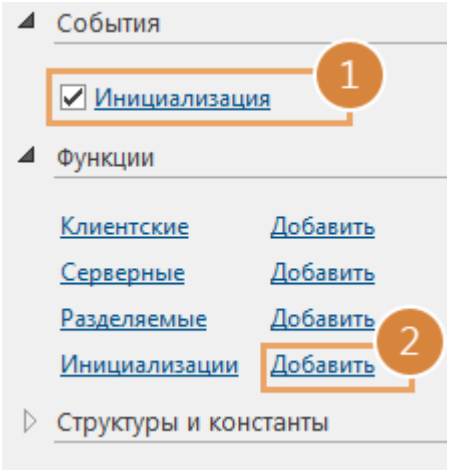
В среде разработки вы можете перекрыть стандартные модули и переопределить их функции инициализации, а также добавить функции инициализации для своих модулей.

Функции инициализации выполняются на веб-сервере и создаются только в редакторах модулей.

СОВЕТ. Если ваша система содержит большой объем данных и количество пользователей, то для инициализации модулей рекомендуется использовать версионирование функций инициализации.

Чтобы добавить логику инициализации:

1. В редакторе модуля в группе «События» установите флажок рядом с событием **Инициализация**:



2. В группе «Функции» перейдите по ссылке **Добавить** рядом с функциями инициализации. Откроется редактор, в котором сгенерируется следующий код:

```
public override void Initializing(Sungero.Domain.ModuleInitializingEventArgs e)
    {
    }

    /// <summary>
    ///
    /// </summary>
    public void Function1()
    {
```

```
}
```

3. Напишите код для функции **Function1()**. Например, программно создайте сущности или выдайте права доступа по умолчанию на вычисляемые папки модуля. Рекомендуется задавать в программном коде функций условия для проверки уникальности создаваемых данных.

   СОВЕТ. Чтобы при публикации отображались сообщения о порядке инициализации модулей, добавьте в код события или в функции инициализации метод **InitializationLogger.Debug("Текст сообщения")**. В тексте сообщения напишите, что будет выполняться в данный момент, например, «Создание роли «Инициатор закупки».

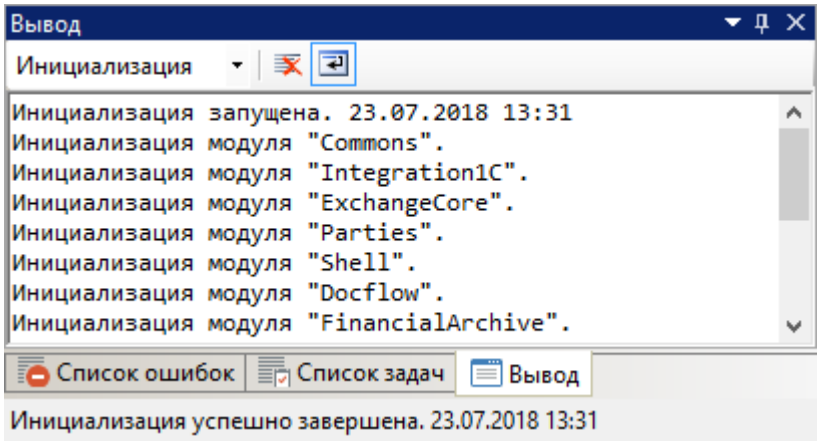
4. Вызовите добавленную функцию в событии **Initializing()**.

   Если функций несколько, то вызовите их в том порядке, в котором они должны выполняться.

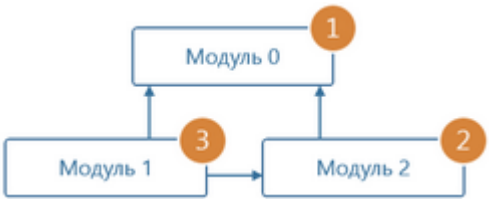
   ПРИМЕЧАНИЕ. Примеры создания и вызова функций инициализации см. в разделах «Пример создания вычисляемой папки», «Инициализация модуля», «Добавление роли согласования».

5. Опубликуйте решения, для модулей которых были добавлены функции инициализации, и запустите инициализацию по кнопке **Запустить инициализацию** на панели инструментов.

6. В окне вывода и в строке состояния среды разработки отобразятся сообщения о порядке инициализации модулей:



   ВАЖНО. Сперва инициализацию проходят все модули, у которых нет зависимостей. Далее последовательно выполняется инициализация модулей, работа которых связана с работой других модулей. Например, модуль 1 зависит от модулей 0 и 2. А модуль 2 зависит только от модуля 0. В результате порядок инициализации будет следующим: Модуль 0, Модуль 2, Модуль 1.



7. Если при выполнении инициализации возникнет ошибка, появится соответствующее сообщение. Все предопределенные данные, созданные до момента ошибки, будут удалены.

Чтобы устранить ошибки, выполните отладку. Для этого установите точку останова и последовательно на панели инструментов нажмите на кнопки **Отладить веб-сервер**, **Запустить инициализацию**.

8. Проверьте результат выполнения функций инициализации в локальной системе Directum RX.

Инициализация разработанных модулей в продуктивной системе Directum RX выполняется с помощью инструмента Directum Launcher.

Создание функций вычисляемых выражений

Функции используются для преобразования данных при настройке вычисляемых выражений. Созданные функции отображаются в выпадающем списке с элементами выражения. При создании функций учитывайте их особенности и рекомендации по именованию.

Особенности создания функций

Функции вычисляемых выражений:

- создаются в серверном коде, помечаются атрибутом ExpressionElement;
- должны быть статичными и публичными, а также возвращать результат;
- могут содержать любое число параметров. При этом:
  - первый параметр функции определяет, для какого контекста эта функция может быть использована. В данном случае контекст – тип свойства или результата другой функции, следом за которой может быть выбрана текущая функция;
  - при создании функции первый параметр заполняется автоматически. Поэтому, если в коде функции есть только один параметр, то в вычисляемом выражении она отображается без параметров. Если в коде задано два параметра, то в вычисляемом выражении отображается только второй;
  - функции могут быть без параметров, то есть вызываться без контекста. С таких функций может начинаться вычисляемое выражение, а также они могут быть начальными для параметров функций.

Если вычисляемые функции общие и не привязаны к какой-то конкретной сущности, то их рекомендуется создавать в модуле. Если функции используются только для конкретного типа сущности, то их рекомендуется создать в серверных функциях этого типа сущности.

Формат атрибута ExpressionElement

Атрибут указывается в формате:

```
[ExpressionElement("<Имя функции>", "<Описание функции>", "", "<Описание параметра 2>", "<Описание параметра 3>", "<Описание параметра N>")]
```

Где:

- **Имя функции** – название функции, отображаемое в выпадающем списке при заполнении вычисляемого выражения. Обязательный параметр;
- **Описание функции** – подсказка по использованию функции;

- **Описание параметра 2...N** – подсказки с описанием параметров. Описания задаются в порядке следования параметров.

ПРИМЕЧАНИЕ. Первый параметр подставляется автоматически, задавать для него описание не нужно. Вместо описания укажите только двойные кавычки "". Описание задается со второго параметра.

Общую информацию об атрибутах см. в разделе «Атрибуты».

Порядок создания функции

Предположим, необходимо написать функцию, которая возвращает руководителя инициатора задачи. Для этого:

1. Создайте серверную функцию модуля.

   ПРИМЕЧАНИЕ. Функцию рекомендуется создавать в модуле, так как в данном случае она не привязана к определенному типу сущности.

2. В редакторе кода укажите нужное имя функции, добавьте комментарий и напишите код. Перед объявлением функции укажите атрибут **ExpressionElement**:

```
[ExpressionElement("Руководитель пользователя", "Возвращает CEO пользователя")]
    public static Sungero.CoreEntities.IRecipient
    CEOofAuthor(Sungero.Workflow.ITask task)
    {
      return
    Sungero.Company.Employees.Get(task.Author.Id).Department.BusinessUnit.CEO;
    }
```

3. Сохраните изменения.

В результате созданную функцию можно выбрать при создании вычисляемых выражений.

ВАЖНО. Если функцию нужно удалить, то в вычисляемых выражениях ее необходимо заменить на другую, иначе при получении значения возникнет ошибка.

Если функция должна быть доступна только в выражении для конкретного типа документа, то ее рекомендуется создавать в серверных функциях этого типа. Например, чтобы получить сумму договора, используйте код:

```
   [ExpressionElement("Получить сумму")]
public static double? GetSum(IContract doc)
{
  return doc.TotalAmount;
}
```

Локализация функции

Название и описание функции, а также описания ее параметров можно локализовать. Для этого:

1. В атрибуте **ExpressionElement** установите курсор на нужное описание.
2. Нажмите сочетание клавиш CTRL+R или воспользуйтесь пунктом контекстного меню **Создать ресурс**.

3. В открывшемся окне заполните поля ***Имя ресурса**, ***en (default)**, ***ru** и нажмите на кнопку **Сохранить**. Подробнее см. раздел «Ресурсы». Текст описания заменится на полный путь до ресурса. В редакторе ресурсов элемента разработки появится строка с ресурсом.
4. В отличие от других элементов разработки, в атрибуте **ExpressionElement** используется только имя ресурса, а не полный путь. Поэтому замените его на имя ресурса в двойных кавычках:

```
[ExpressionElement("Ceo_of_author", "Ceo_of_author_description")]
    public static Sungero.CoreEntities.IRecipient
    CEOofAuthor(Sungero.Workflow.ITask task)
    {
      return
    Sungero.Company.Employees.Get(task.Author.Id).Department.BusinessUnit.CEO;
    }
```

В результате после публикации наименование и описание функции будет отображаться в выражениях с учетом выбранной локализации.

Рекомендации по именованию функций вычисляемых выражений

Из наименования и подсказок к функции *аналитику* должно быть понятно, как она работает и какие у нее параметры. Кроме того, для удобства использования создаваемые функции должны называться единообразно. Чтобы достичь этого, соблюдайте рекомендации.

Общие советы

При создании функций вычисляемых выражений рекомендуется:

- наименование функции писать с большой буквы, на естественном языке с пробелами между словами;
- использовать короткое наименование функции, чтобы итоговое выражение тоже получилось коротким;
- в наименовании отражать ключевую суть функции, опустить детали;
- в наименовании использовать общепринятые сокращения, например, «КЭДО», «НДС», «ч.», «руб.», а также сокращения, принятые внутри компании.

Именование в зависимости от типа возвращаемого значения

Ссылка на объект

Если функция возвращает ссылку на объект, то в качестве наименования рекомендуется использовать существительное в единственном числе в именительном падеже.

Примеры:

- Задача → Контролер → **Непосредственный руководитель**
- Задача → Сотрудник → **Ресурс-менеджер**
- Задача → Адресат → **Помощник**

Коллекция элементов

Если функция возвращает коллекцию элементов, то в качестве наименования рекомендуется использовать существительное во множественном числе в именительном падеже. При этом в наименовании рекомендуется опустить слово «коллекция», чтобы не увеличивать длину вычисляемого выражения.

Примеры:

- Задача → **Все участники**
- Задача → **Участники повторного ознакомления**
- Задача → **Приложения**

Коллекция элементов (с уточнением)

Функция может возвращать коллекцию, но при этом быть доступна только в определенном типе задач и/или уточнять контекст для коллекции только определенного типа. В этом случае рекомендуется, чтобы наименование содержало существительное во множественном числе и суть уточнения.

Примеры:

- Задача → Участники ознакомления → **Участники с КЭДО**
- Задача → Участники ознакомления → **Участники без КЭДО**
- Задача → **Участники повторного ознакомления**

Преобразование данных

В вычисляемых выражениях могут использоваться функции с преобразованием данных. Такие функции не только возвращают существующие в системе данные, но и изменяют их. В том числе могут быть получены новые данные другого типа. Рекомендуется в наименование функции включить краткую суть преобразования.

Примеры:

- Задача → Согласующие → **Список ФИО в текст**
- Задача → Кандидат → **Фамилия и инициалы**
- Задача → Тема в переписке → **Обрезать до N символов**

Целое значение

Если функция возвращает целое значение, то рекомендуется в наименование включить существительное и единицу измерения.

Примеры:

- Задача → Вложения → **Количество элементов**
- Задача → Инициатор → **Стаж, лет**
- Задача → **Относительный срок, ч.**

Логическое значение

Если функция возвращает логическое значение, то наименование рекомендуется указывать в виде вопроса.

Примеры:

- Задача → Документ на ознакомление → Все → Первый элемент → **Подписан?**
- Задача → Сотрудник → Подразделение → **Производственное?**
- Задача → **Задачи созданы для всех адресатов?**
- Задача → **Это подзадача?**

Подсказка для функций

Подсказка включает в себя краткое описание функции и параметров, если они используются. Описание функции рекомендуется начинать с глагола в неопределенной форме.

Полное описание функции задается с помощью атрибута **ExpressionElement** в формате:

```
[ExpressionElement("<Имя функции>", "<Описание функции>", "", "<Описание параметра 2>", "<Описание параметра 3>", "<Описание параметра N>")]
```

Пример атрибута для функции с входными параметрами:

```
[ExpressionElement("Обрезать до N символов", "Обрезать строку до N символов", "", "Количество символов")]
```

Примеры атрибутов для функций без входных параметров:

```
[ExpressionElement("В верхнем регистре", "Перевести строку в верхний регистр")]
[ExpressionElement("Участники ознакомления", "Получить список участников ознакомления")]
```

ПРИМЕЧАНИЕ. На практике вместо явного задания текста рекомендуется использовать строки локализации.

Структуры

Структуры – это объекты, которые состоят из нескольких логически связанных между собой полей разного типа. Например, в структуре можно хранить персональную информацию о сотруднике: ФИО (string), дату рождения (DateTime), паспортные данные (int).

Полями в структурах могут быть:

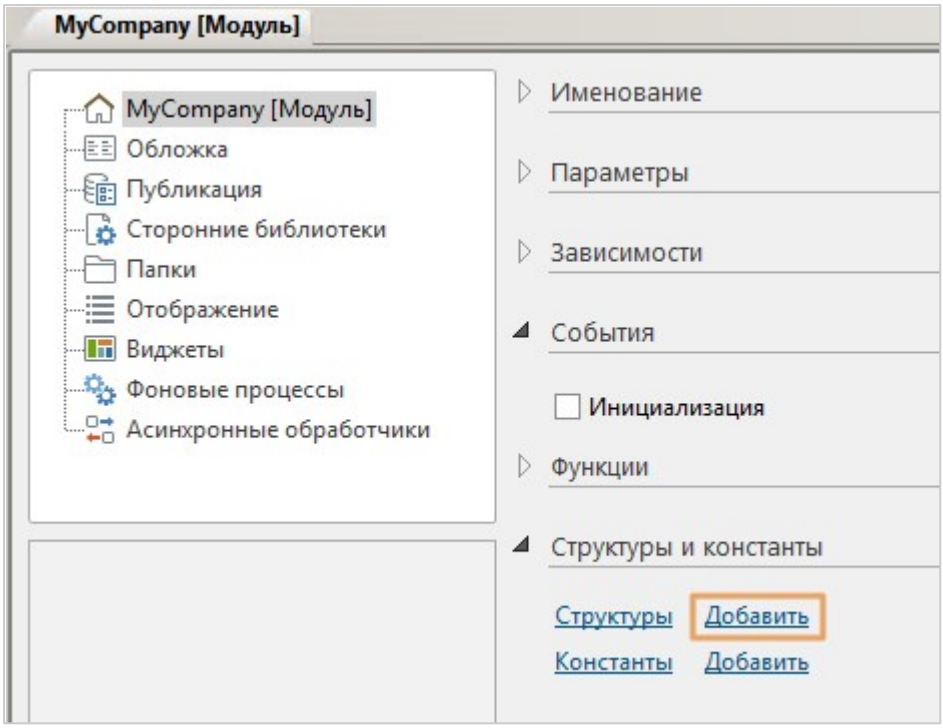
- простые типы:

short, short?	Guid, Guid?	double, double?
int, int?	bool, bool?	Uri
long, long?	decimal, decimal?	string
char, char?	DateTime, DateTime?	

- перечисления: Enumeration, Enumeration?;
- ссылки на типы сущностей: IUser, ICompany и т.п.;
- интерфейсы структур и списки интерфейсов структур;
- списки простых типов или типов сущностей: List<int>, List<ICompany> и т.п.

Создание структуры

1. В редакторе решения, модуля, типа сущности или отчета в группе «Структуры и константы» перейдите по ссылке **Добавить**.



2. В открывшемся редакторе кода доработайте имя структуры, добавьте поля и напишите комментарии, например:

```
/// <summary>
   /// Замещение.
   /// </summary>
   partial class Substitution
   {
     /// Замещающий (кто).
     public IUser User { get; set; }

     /// Замещаемый (кого).
     public IUser SubstitutedUser { get; set; }
   }
```

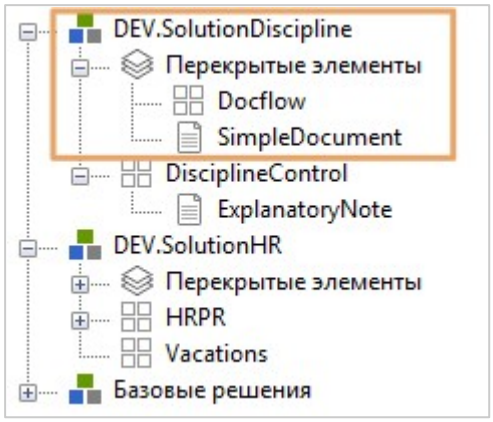
3. Сохраните изменения.

ВАЖНО. В перекрытиях и наследниках можно добавлять новые структуры, но нельзя переопределять структуры, которые добавлены в родительском модуле или типе сущности.

Структура без атрибута Public

Если у структуры **нет атрибута Public** и она добавлена в редакторе решения, то создавать экземпляры структуры и использовать их:

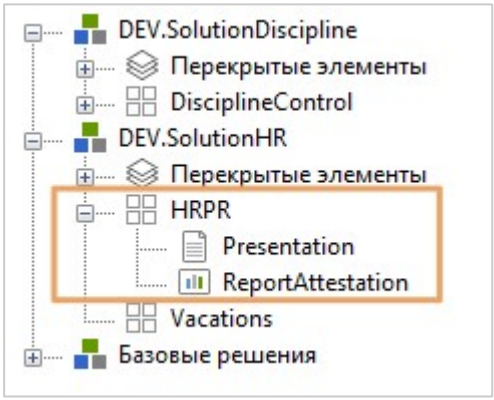
- **можно** в коде этого решения и перекрытиях, которые созданы в этом решении. Например, структура создана в решении SolutionDiscipline. Ее можно использовать в этом же решении и его перекрытиях;



- **нельзя** в коде модулей, типов сущностей и отчетов этого решения и в других решениях.

Если у структуры **нет атрибута Public** и она добавлена в редакторе модуля, типа сущности или отчета, то создавать экземпляры структуры и использовать их:

- **можно** в рамках этого модуля. Например, структура добавлена в редакторе типа документа с именем Presentation. Создать экземпляры структуры можно в функции модуля HRPR, в событии отчета ReportAttestation, в действии типа документа Presentation и т.п.



- **нельзя** в решении, где находится модуль, в перекрытиях, а также в других модулях и решениях.

Например, создан модуль с именем HRPR. В него добавлена структура с данными для расчета исполнительской дисциплины сотрудников подразделений:

```
   /// <summary>
/// Исполнительская дисциплина подразделения.
/// </summary>
partial class DepartmentDiscipline
{
  /// <summary>
  /// Процент исполнительской дисциплины сотрудников.
```

```
  /// </summary>
  public int? Discipline { get; set; }

  /// <summary>
  /// Подразделение.
  /// </summary>
  public Sungero.Company.IDepartment Department { get; set; }
}
```

Структура используется в серверной функции этого же модуля:

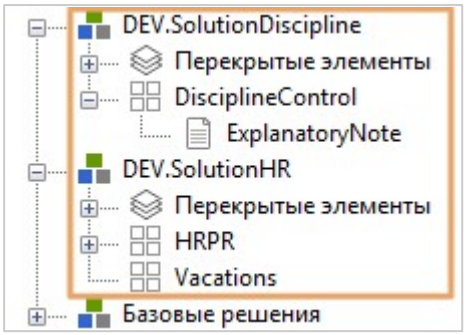
```
   public                          List<Structures.Module.DepartmentDiscipline>
      GetDepartmentsDiscipline()
{
  /// Берутся задания сотрудников и группируются по подразделениям.
  var assignments = Assignments.GetAll().GroupBy(a =>
a.Performer.Department).ToList();
  /// Возвращается исполнительская дисциплина сотрудников.
  return assignments.Select(assignmentsGroup =>
Structures
.Module
.DepartmentDiscipline
.Create(this.CalculateDiscipline(assignmentsGroup.ToList()),
assignmentsGroup.Key)).OrderBy(s => s.Discipline).ToList();
}
```

Структура с атрибутом Public

Атрибут **Public** добавляют, если нужно создавать и использовать экземпляры структуры в других решениях, модулях, типах сущностей и отчетах. Например, в модуле Vacations (Отпуска) добавлена структура:

```
   /// <summary>
/// Отпуск.
/// </summary>
[Public]
partial class Vacation
{
  // Дата начала отпуска.
  public DateTime StartVacation { get; set; }

  //Дата завершения отпуска.
  public DateTime EndVacation { get; set; }

  /// Отпускник.
  public IUser Vacationer { get; set; }

}
```

Ее можно использовать в рамках этого модуля и в других модулях, решениях и перекрытиях:



ВАЖНО. Чтобы обратиться к структуре с атрибутом **Public**, нужно использовать интерфейс структуры. Например, в модуле HRPR добавлена серверная функция, в которой создается экземпляр структуры из модуля Vacations:

```
   public Vacations.Structures.Module.IVacation CreateVacation()
{
  var StartDate = Calendar.Today.NextWorkingDay();
  var EndDate = StartDate.AddWorkingDays(7);
  var vacation = Vacations.Structures.Module.Vacation.Create(StartDate,
EndDate, Users.Current);
  return vacation;
}
```

Интерфейс структуры используется в типе возвращаемого значения: написано Structures.Module.IVacation, а не Structures.Module.Vacation.

Формат создания экземпляра структуры

Чтобы создать экземпляр структуры, используйте метод **Create()** с параметрами или без них. Параметрами являются поля структуры. Формат вызова метода зависит от того, в каком редакторе элемента разработки добавлена структура.

Структура добавлена в редакторе	Формат метода для создания экземпляра структуры
Решения или модуля	Structures.Module.<Имя структуры>.Create(<поля структуры>)
Перекрытого модуля	Structures.<Имя перекрытого модуля>.Module.<Имя структуры>.Create (<поля структуры>)
Перекрытого типа сущности	Structures.<Имя перекрытого модуля>.<Имя перекрытого типа сущности>.<Имя структуры>.Create(<поля структуры>)
Типа сущности	Structures.<Имя типа сущности>.<Имя структуры>.Create(<поля структуры>)
Отчета	Structures.Report.<Имя структуры>.Create(<поля структуры>)

Константы

Константы представляют собой заданные значения, которые не могут изменяться при выполнении программы. Используются, например, когда необходимо зафиксировать срок рассмотрения документа или имя нового типа прав.

Особенности использования:

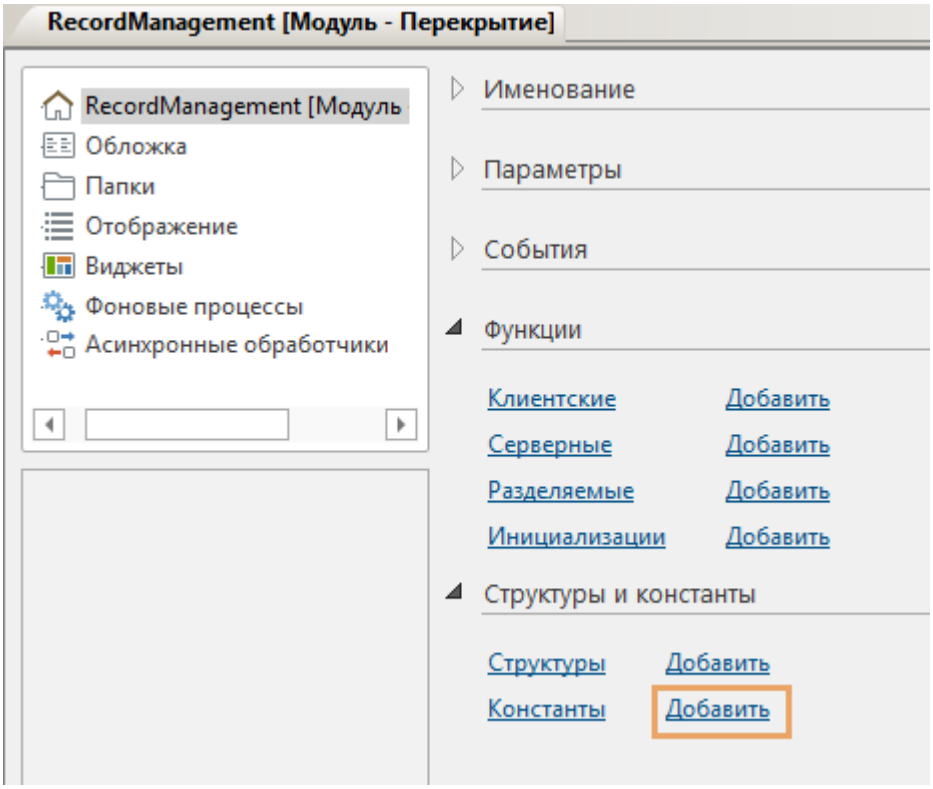
- константы добавляются в программный код решений, модулей, типов сущностей и отчетов. При этом константы можно будет использовать только в рамках того модуля, в котором они созданы. Константы, созданные в решении, можно использовать только в коде решения и его перекрытиях;
- если константу необходимо использовать в других решениях или модулях, то при добавлении константы указывается атрибут **Public**;
- константы нельзя переопределять или наследовать;
- в качестве констант можно указывать только:
  - простые типы:

short, short?	Guid, Guid?	double, double?
int, int?	bool, bool?	Uri
long, long?	decimal, decimal?	string
char, char?	DateTime, DateTime?	

    При объявлении константы указывается модификатор **const**;
  - идентификаторы (Guid). При объявлении константы указывается **static readonly Guid**.

Чтобы добавить константу:

1. В редакторе решения, модуля, типа сущности или отчета в группе «Структуры и константы» перейдите по ссылке **Добавить**.



2. В открывшемся редакторе кода доработайте имя константы, тип и допишите комментарии:

```
/// <summary>
    // Имя типа прав "Регистрация".
    /// </summary>
    public const string RegistrationRightType = "Регистрация";

    [Public]
    /// <summary>
    // GUID роли "Руководители наших организаций".
    /// </summary>
    public static readonly Guid BusinessUnitHeadsRole = Guid.Parse("03C7A126-
    83DE-4F8F-908B-3ACB868E30C5");
```

ВАЖНО. Для группировки логически связанных между собой констант вы можете создавать вложенные классы.

Например, все константы с именами типов прав можно сгруппировать в класс TypeRights, а константы с именами ролей – в класс Role:

```
  public static class Module
{
  public static class TypeRights
  {
    // Имя типа прав "Регистрация".
    public const string RegistrationRightType = "Регистрация";
    // Имя типа прав "Удаление".
    public const string DeleteRightType = "Удаление";
  }
  public static class Role
```

```
  {
    // Имя роли "Делопроизводители".
    public const string RoleNameClerks = "Делопроизводители";
    // Имя роли "Ответственные за настройку регистрации".
    public const string RoleNameRegistrationManagers = "Ответственные за
настройку регистрации";
  }
}
```

3. Сохраните изменения.
4. Перейдите в программный код, в котором будет использоваться константа. Для обращения к константам используются следующие форматы:
   - Constants.Module.<Имя константы> – для констант модуля или решения;
   - Constants.<Имя перекрытого модуля>.Module.<Имя константы> – для констант перекрытого модуля;
   - Constants.<Имя перекрытого модуля>.<Имя перекрытого типа сущности>.<Имя константы> – для констант перекрытого типа сущности;
   - Constants.<Имя типа сущности>.<Имя константы> – для констант типов сущностей;
   - Constants.Report.<Имя константы> – для констант отчетов;
   - <Имя модуля или решения>.PublicConstants.Module.<Имя константы> – для констант, доступных в других модулях или решениях;
   - <Имя модуля>.PublicConstants.<Имя типа сущности>.<Имя константы> – для констант типов сущностей, доступных в других модулях.

   ВАЖНО. Если константы сгруппированы в классы, то в формат перед именем константы добавляется имя класса.

   Формат обращения к константе класса TypeRights (см. пример выше) выглядит следующим образом: **Constants.Module.TypeRights.RegistrationRightType**

Пример. Использование константы в модуле

В модуль «Делопроизводство» добавлена константа для фиксирования срока рассмотрения документа в днях.

```
   // Срок рассмотрения документа по умолчанию в днях.
public const int DocumentReviewDefaultDays = 3;
```

Используем добавленную константу в серверной функции:

```
   /// <summary>
/// Получить константу срока рассмотрения документа по умолчанию в днях.
/// </summary>
/// <returns> Константа срока рассмотрения документа по умолчанию в
днях.</returns>
[Remote]
public virtual int GetDocumentReviewDefaultDays()
{
  return Constants.Module.DocumentReviewDefaultDays;
}
```

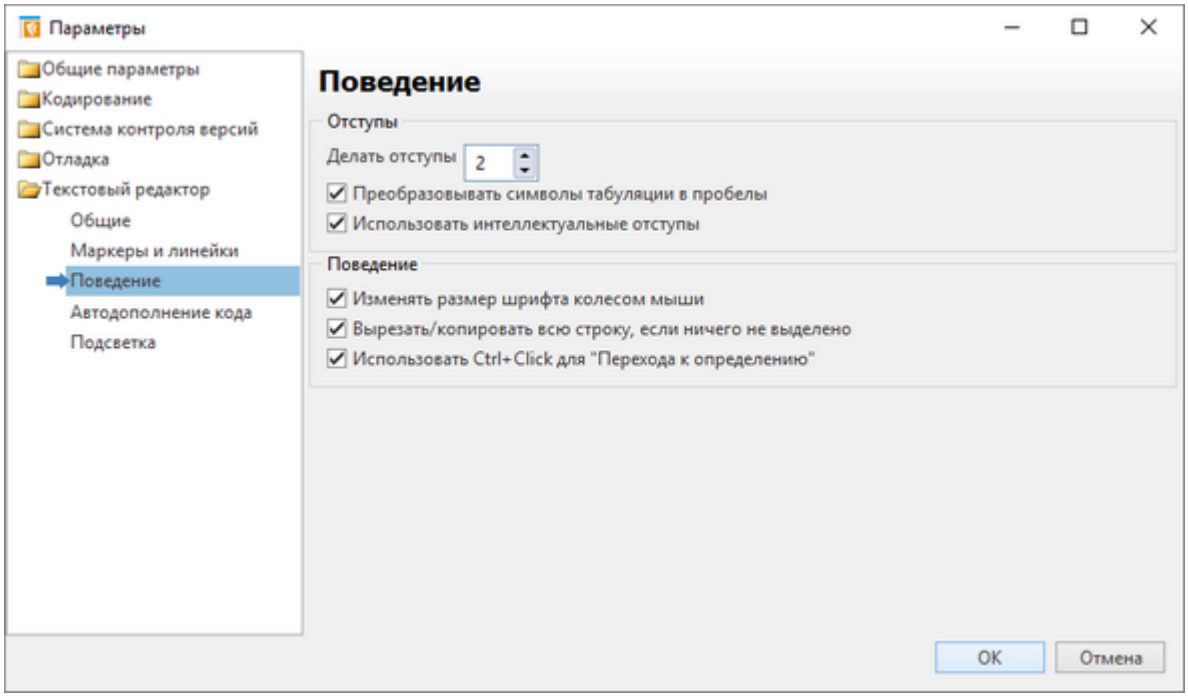
Правила оформления

Общие правила по оформлению программного кода:

- имена элементов разработки, свойств и действий задавайте как можно короче. При этом они должны быть осмысленными и говорящими, так как в дальнейшем это упростит читаемость кода и понимание другими разработчиками;
- для идентификаторов переменных, параметров используйте нотацию camelCasing: первое слово пишется с маленькой буквы, остальные слова – с большой. Для всех остальных – PascalCasing, в том числе для задания имен свойствам сущности. Подробнее см. в документации Microsoft статью «Соглашения о написании прописными буквами»;
- сокращения в именах идентификаторов не должны влиять на понятность имени. Не используйте аббревиатуры или неполные слова в идентификаторах, если только они не являются общепринятыми;
- имя сущности задавайте в единственном числе. Так как в среде разработки автоматически генерируется пространство имен репозитория с именем сущности во множественном числе;
- избегайте лишних скобок, обрамляющих выражения целиком;
- содержимое класса или функции должно быть сдвинуто на два пробела относительно объявления:

```
partial class ContractFunctions
{
  /// <summary>
  /// Показывать сводку по документу в заданиях на согласование и подписание.
  /// </summary>
  /// <returns>True, если в заданиях нужно показывать сводку по
документу.</returns>
  [Public]
  public override bool NeedViewDocumentSummary()
  {
    return true;
  }
  ...
}
```

Для автоматического форматирования кода используйте сочетание клавиш CTRL+I, предварительно настроив отступы в среде разработки. Для этого в меню последовательно выберите пункты: **Инструменты**, далее **Параметры**, закладка **Текстовый редактор**, **Поведение**, укажите необходимые настройки:



- переменные рекомендуется объявлять с помощью ключевого слова **var** вместо явного указания типа:

```
public static bool IncludedInRole(string roleName)
{
  var ceoRole = Roles.GetAll(r => r.Name == roleName).FirstOrDefault();
  if (ceoRole == null)
  {
    return false;
    var employee = Users.Current;
    return employee.IncludedIn(ceoRole);
  }
}
```

Тип допускается указывать явно, когда это необходимо для корректной сборки и выполнения кода.

- пишите комментарии к коду в XML-формате. Чтобы быстро написать комментарий по формату, в среде разработки Directum RX введите ///. В результате автоматически вставится структура комментария:

```
/// <summary>
///
/// </summary>
/// <param name=""></param>
/// <returns></returns>
```

Содержимое комментария зависит от кода. Например, если в коде реализован метод без параметров, то при вводе /// структура комментария будет без строки `/// <param name=""></param>`. А если в коде метода указан параметр, то его название автоматически укажется в теге `<param>`, например, `/// <param name="responsible"></param>`.

Группировка программного кода

При модификации системы в файлы с исходным кодом добавляются функции, структуры и константы. Иногда их может быть много, из-за чего ухудшается навигация по коду, а работа с файлами замедляется. В таких случаях вы можете группировать программный код решений, модулей или типов сущностей в отдельных файлах и папках. При этом программный код переносится из файлов с исходным кодом в созданные файлы. Например, вы можете объединить все серверные функции типа сущности **OfficialDocument**, которые вызываются при подписании документа. Благодаря этому улучшается структурированность кода и упрощается навигация по нему.

ВАЖНО. Группировать можно только код одного типа, например только клиентские функции или только структуры.

Чтобы сгруппировать программный код:

1. Перейдите в папку %<Папка с локальными репозиториями>%\work\<Код компании>.<Название решения или модуля>. В ней в зависимости от группируемого кода выберите папку:
   - клиентские функции – <Код компании>.<Название решения или модуля>.ClientBase;
   - серверные функции – <Код компании>.<Название решения или модуля >.Server;
   - разделяемые функции, структуры и константы – <Код компании>.<Название решения или модуля >.Shared.
2. Если группируется код типа сущности, то в открывшейся папке перейдите в папку с его названием.
3. В зависимости от типа группируемого кода создайте файл:
   - структуры – *Structures.cs
   - константы – *Constants.cs;
   - клиентские функции типа сущности или модуля/решения – *ClientFunctions.cs или *ModuleClientFunctions.cs;
   - серверные функции типа сущности или модуля/решения – *ServerFunctions.cs или *ModuleServerFunctions.cs;
   - разделяемые функции типа сущности или модуля/решения – *SharedFunctions.cs или *ModuleSharedFunctions.cs.

   Например, для группировки серверных функций типа сущности создайте файл SigningServerFunctions.cs.

   При необходимости вы можете создавать собственные папки и уже в них размещать файлы с группируемым кодом. Например, в папке с серверными функциями модуля создайте папку SigningDocument. Перейдите в нее и уже в ней создайте файлы с функциями, которые работают с типом сущности **OfficialDocument**.

   ВАЖНО. Имена файлов и папок должны состоять из букв латинского алфавита и цифр.
4. Перезапустите среду разработки.
5. С помощью механизма Drag&Drop переместите созданный файл в окно среды разработки. В результате он и редактор решения, модуля или типа сущности, к которому относится файл, откроются на редактирование.

6. В файле подключите библиотеки, необходимые для выполнения функций.

   ВАЖНО. В файле укажите пространство имен и класс, в которых переносимые функции были объявлены изначально. При объявлении класса используйте ключевое слово **partial**.

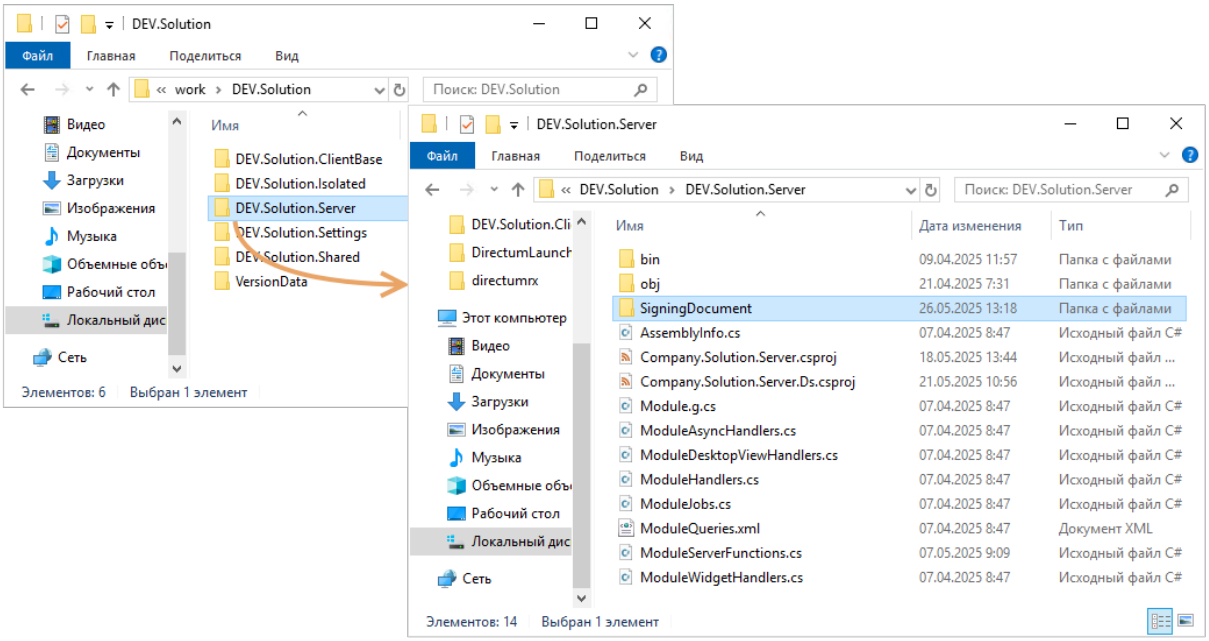
7. Из файла с исходным кодом переместите функции в созданный класс. При необходимости вы также можете написать новые функции.
8. Сохраните изменения. Чтобы они появились у других пользователей, зафиксируйте и отправьте их на сервер Git.

При вызове подобного кода, например обработчиком события, система самостоятельно найдет его в созданном файле и выполнит.

Пример группировки программного кода функций решения

Предположим, что в решении **Solution** необходимо сгруппировать все серверные функции типа сущности **OfficialDocument**, которые вызываются при подписании документа. Для этого:

1. Перейдите в папку, в которой хранятся серверные обработчики модуля – %<Папка с локальным репозиторием Work>%\DEV.Solution\DEV.Solution.Server.
2. Создайте папку SigningDocument и перейдите в нее:



3. Создайте файл SigningServerFunctions.cs.
4. Перезапустите среду разработки.
5. С помощью механизма Drag&Drop переместите файл в окно среды разработки.
6. В открывшемся окне редактирования подключите необходимые библиотеки, укажите пространство имен и класс, в которых функции были объявлены изначально. Переместите функции в созданный класс:

```
using System;
   using CommonLibrary;
   using Sungero.Company;
   using Sungero.Content;
   using Sungero.CoreEntities;
```

```
using Sungero.Docflow.OfficialDocument;
using Sungero.DocflowApproval;
using Sungero.Parties;
using Sungero.Workflow;

namespace Sungero.Docflow.Server
{
  partial class OfficialDocumentFunctions
  {
    // Проверить наличие права подписи со всеми сотрудниками.
    [Remote(IsPure = true)]
    public virtual bool SignatorySettingWithAllUsersExist()
    {
      var settings =
Functions.OfficialDocument.GetSignatureSettingsQuery(_obj);
      return settings.Any(s => s.Recipient.Sid ==
Constants.OfficialDocument.AllUsersSid);
    }

    // Проверить наличие права подписи у сотрудника.
    [Public, Remote(IsPure = true)]
    public virtual bool CanSignByEmployee(IEmployee employee)
    {
      if (_obj == null || employee == null)
        return false;
      return this.GetSignatureSettingsByEmployee(employee).Any();
    }

    // Заполнить статус согласования "Подписан".
    [Remote]
    public void SetInternalApprovalStateToSigned()
    {
      if (_obj.InternalApprovalState == InternalApprovalState.Aborted ||
          _obj.InternalApprovalState == InternalApprovalState.Signed)
        return;
      if (Equals(_obj.InternalApprovalState, InternalApprovalState.Signed))
        return;
      if (!_obj.AccessRights.CanUpdate())
      {
        using (var session = new Sungero.Domain.Session())
        {
          Functions.Module.AddFullAccessRightsInSession(session, _obj);
          _obj.InternalApprovalState = InternalApprovalState.Signed;
          _obj.Save();
        }
      }
      else
        _obj.InternalApprovalState = InternalApprovalState.Signed;
    }

    // Заполнить подписывающего в карточке документа.
    [Public, Remote]
    public virtual void SetDocumentSignatory(IEmployee employee)
    {
      if (Equals(_obj.OurSignatory, employee))
        return;
```

```
            if (!_obj.AccessRights.CanUpdate())
            {
              using (var session = new Sungero.Domain.Session())
              {
                Functions.Module.AddFullAccessRightsInSession(session, _obj);
                _obj.OurSignatory = employee;
                _obj.Save();
              }
            }
            else
            {
              _obj.OurSignatory = employee;
              _obj.Save();
            }
          }
        }
      }
```

7. Сохраните изменения.

Разрешенные и запрещенные конструкции

В разделе описаны разрешенные и запрещенные конструкции в прикладном программном коде:

- классы Sungero
- классы .NET
- язык C#

Запрещенные конструкции распространяются на любой программный код, если явно не указано, в каком коде запрещено использовать: серверный, клиентский, разделяемый.

Классы Sungero

Разрешенные классы	Запрещенные классы
Классы из пространства имен всех прикладных модулей: Sungero.Company; Sungero.Commons и т.д.	Классы всех остальных пространств имен Sungero.*
Классы из пространства имен всех предметных модулей: Sungero.Content; Sungero.Workflow	
Классы из системных пространств имен. Разрешается использовать только Sungero.Core и Sungero.CoreEntities	

ВАЖНО. Запрещенные классы использовать нельзя, т.к. при обновлении системы не гарантируется совместимость кода.

Классы .NET

Разрешенные классы	Запрещенные классы
Классы простых типов (System.Int, System.String, System.DateTime и т.д.)	Класс System.Tuple
Классы Generic-коллекций	Класс System.Globalization.CultureInfo
Методы расширения System.Linq	Класс System.Convert
Класс System.Math	Прочие классы .NET

Разрешенные классы

Классы простых типов (System.Int, System.String, System.DateTime и т.д.)

Разрешается использовать простые типы, включая nullable, а также методы из их классов. Например, разрешается использовать методы:

- объекта String:

```
str.Substring(0, 5);
str.Split('|');
str.Trim();
str.Replace('|', ';');
str.IsNullOrWhiteSpace();
```

- класса String:

```
string.Format("{0} {1}", str1, str2);
string.Join("\n", messages);
```

- объекта DateTime:

```
date.AddDays(1);
date.AddMonths(1);
date.ToShortDateString()
```

ВАЖНО. Запрещается получать текущее время через класс **DateTime**. Вместо **DateTime.Today** используйте Calendar.Today, а вместо **DateTime.Now** – **Calendar.Now**. Запрет на использование этих свойств через класс **DateTime** связан с тем, что они возвращают дату и время в часовом поясе, установленном на ПК пользователя, а не в часовом поясе веб-сервера.

Также запрещается использовать свойство **DateTime.Kind** со значением **Local**, так как в этом случае дата автоматически преобразуется с учетом часового пояса,

установленного на ПК пользователя, а не часового пояса веб-сервера. В дальнейшем это приводит к искажению данных.

Классы Generic-коллекций

Разрешается использовать классы коллекций и методы, которые они предоставляют:

- System.Collections.Generic.Dictionary;
- System.Collections.Generic.List;
- System.Array.

Разрешается использовать методы коллекций:

```
dictionary.TryGetValue("key1", out Value);
list.Contains(range);
```

Разрешается передавать коллекции в функциях:

```
Dictionary<string, DateTime?> GetPlanActualDates(string planKasId, string
planStageKasId);
List<DateTime?> GetPlanActualDates(string planKasId, string planStageKasId);
```

Методы расширения System.Linq

Разрешается использовать любые методы расширения LINQ: **First()**, **OrderBy()**, **Any()** и т.д. Примеры см. в разделе «LINQ и лямбда-выражения».

При этом есть исключение. В клиентском или серверном коде при обращении к списку сущностей через репозиторий запрещается использовать следующие методы расширения:

```
.Concat()
.Distinct()
.GroupBy(p => p.Id)
.Last()
.LastOrDefault()
.Max(p => p.Number)
.OfType<IEmployee>()
.SelectMany(p => p.Versions)
.Sum(p => p.Number)
.Union()
```

Класс System.Math

Разрешается использовать методы класса Math. Например:

```
var result = Math.Round(value, 2);
```

Запрещенные классы

Класс System.Tuple

Запрещается использовать кортежи – класс Tuple:

```
var result = new Tuple<long, DateTime>(id, date);
```

Вместо класса Tuple используйте структуры. По сравнению с кортежами они обеспечивают хорошо читаемый код и удобное использование в функциях.

Класс System.Globalization.CultureInfo

Запрещается использовать класс CultureInfo, т.к. в разных условиях он возвращает разные значения. Например, если серверные и клиентские региональные настройки отличаются, то не гарантируется, какие из них используются при выполнении кода.

Если необходимо локализовать строку по региональным настройкам веб-сервера, то используйте конструкцию:

```
   using(TenantInfo.Culture.SwitchTo())
{
  // Локализация строки.
}
```

Класс System.Convert

Запрещается использовать методы класса Convert:

```
System.Convert.ToDateTime("01/02/03");
```

```
System.Convert.ToUInt32("-5")
```

Ограничение связано с тем, что методы класса зависят от региональных настроек и не всегда могут корректно сконвертировать значение.

Для конвертации используйте методы простых типов:

```
DateTime.Parse("01/02/03");
```

```
DateTime.TryParseExact("MM/DD/YYYY");
```

```
int.Parse("-5");
```

При этом метод `DateTime.TryParse` использовать запрещается.

Прочие классы .NET

Запрещается использовать все классы .NET, кроме разрешенных. Например, запрещается работать с:

- файлами, папками, путями;
- информацией о типах (System.Reflection). Например, запрещено получать тип объекта с помощью конструкции typeof(IEmployee);

- Window, MessageBox, Control (System.Windows);
- xml-данными, сериализацией (System.Xml);
- потоками (System.Threading);
- ADO.NET (System.Data).

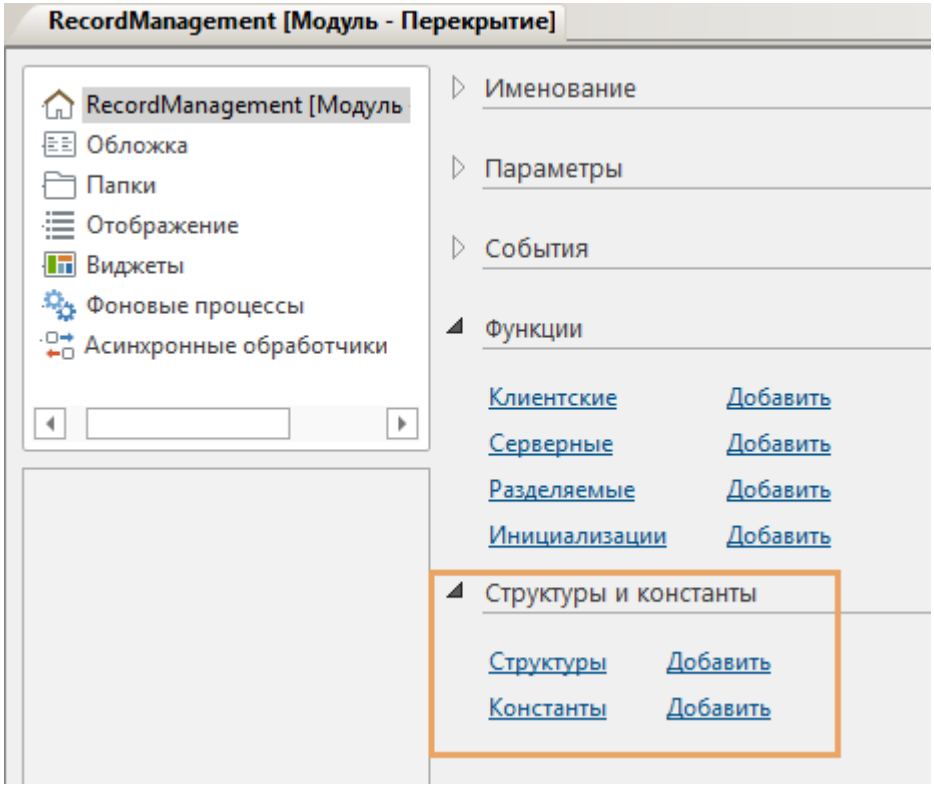
Язык C#

Разрешенные возможности	Запрещенные возможности
Создание структур и констант	Использование анонимных типов
Генерация и перехватывание исключений	Приведение к типу is, as
	Создание объектов с помощью new
	Использование статических полей
	Модификатор параметра out

Разрешенные возможности

Создание структур и констант

Разрешается создавать структуры и константы в программном коде решений, модулей, типов сущностей и отчетов. При этом добавлять структуры и константы можно только с помощью ссылок в группе «Структуры и константы»:



Генерация и перехватывание исключений

Разрешается генерировать и перехватывать прикладные исключения:

```
   try
{
    throw AppliedCodeException.Create(Resources.Message,
      Plans.Resource.Description);
}
catch (AppliedCodeException ex)
{
}
```

Также разрешается перехватывать предметные исключения, если вы точно знаете о том, что это за исключение. При этом запрещается генерировать родные исключения .NET:

```
   try
{
    throw new ArgumentOutOfRangeException();
}
catch (ArgumentOutOfRangeException ex)
{
}
```

Запрещается перехватывать и гасить все исключения:

```
   catch (Exception ex)
{
}
```

Для обработки и передачи исключений разрешается использовать конструкцию:

```
   catch (Exception ex)
{
  throw ex;
}
```

Запрещенные возможности

Анонимные типы

Запрещается использовать анонимные типы:

```
var obj = new { Name = 'Name1' };
```

```
.Select(p => new { Name = p.Name })
```

Ограничение связано с тем, что они не передаются между сервером и клиентом.

Вместо анонимных типов используйте структуры.

Приведение к типу is, as

Запрещается проверка типа:

```
if (entity is IEmployee)
```

Запрещается приведение к типу:

```
var employee = entity as IEmployee;
```

Ограничение связано с использованием ORM NHibernate: возможна ситуация, когда сущность приводится к интерфейсу, который она не реализует. В этом случае код не работает.

Вместо этого используйте методы безопасной проверки и приведения к типу Employees.Is(entity) и Employees.As(entity).

Создание объектов с помощью new

Запрещается создавать объекты с помощью оператора **new**:

```
var result = new Tuple<long, DateTime>(id, date);
```

```
var ex = new ArgumentOutOfRangeException("date");
```

Через оператор **new** разрешается создавать только коллекции. В остальных случаях используйте метод создания, который есть в указанном классе, например Create().

Использование статических полей

Запрещается использовать статические поля в прикладном коде:

```
static List<double?> list = new List<double?>();
```

Поля не поддерживаются на уровне платформы. Вместо них используйте свойства.

Модификатор параметра out

Запрещается использовать модификатор параметра **out**:
```
public bool TryGetEmployeeForTask(out Sungero.Company.IEmployee employee)
```

Модификатор не поддерживается в среде разработки. Вместо него используйте структуры.

LINQ и лямбда-выражения

LINQ (Language-Integrated Query) – универсальный язык запросов для каждого типа источника данных. В качестве источника данных может выступать объект, реализующий интерфейс IEnumerable (стандартные коллекции, массивы и т.д.), набор данных DataSet, документ XML. Вне зависимости от типа источника LINQ позволяет применить ко всем один и тот же подход для выборки данных. Таким образом, при использовании LINQ разработчикам не приходится учить новый язык источника данных или формата данных, для которых требуется обеспечить поддержку.

Рассмотрим простой пример выборки из массива строк, которые начинаются на определенную букву – например, на «С».

Пример 1. Выборка с использованием оператора foreach

```
   string[] cities = { "Москва", "Кострома", "Иркутск", "Сургут", "Сочи"};
// List<T> - наследник IEnumerable<T>.
var selectedCities = new List<string>();
foreach (string c in cities)
{
   if (c.ToUpper().StartsWith("C"))
       selectedCities.Add(c);
}
```

Тот же пример можно реализовать с помощью LINQ.

Пример 2. Использование LINQ

```
   string[] cities = { "Москва", "Кострома", "Иркутск", "Сургут", "Сочи"};
IEnumerable<string> selectedCities =
// Определить каждый объект как «c».
                  from c in cities
// Отфильтровать по критерию.
                  where c.ToUpper().StartsWith("C")
// Выбрать объект.
                  select c;
```

LINQ преобразует запросы в удобную языковую конструкцию, которая применяется аналогично методам, классам и событиям.

Методы расширения LINQ

Для создания запроса LINQ, кроме стандартного синтаксиса from in select, можно использовать специальные методы расширения, которые определены для интерфейса IEnumerable. Например:

- **Any** – проверяет существование хотя бы одного элемента в последовательности;
- **First** – возвращает первый элемент последовательности;
- **FirstOrDefault** – возвращает первый элемент последовательности или значение по умолчанию, если ни одного элемента не найдено;
- **OrderBy** – сортировка элементов в порядке возрастания;
- **Select** – определяет тип значений, которые будут получены при выполнении запроса;
- **Single** – возвращает единственный конкретный элемент последовательности;
- **SingleOrDefault** – возвращает единственный конкретный элемент последовательности или значение по умолчанию, если этот элемент не найден. Генерирует исключение, если в последовательности содержится более одного элемента;
- **Take** – возвращает указанное число подряд идущих элементов с начала последовательности;
- **ToList** – создает объект List<T> из IEnumerable<T>;
- **Where** – определяет фильтр элементов источника на основе одного или нескольких логических выражений, разделенных операторами логического И и ИЛИ (&& или ||).

Как правило, эти методы реализуют ту же функциональность, что и операторы LINQ типа where. Подробнее о том, какие методы расширения **System.Linq** доступны для использования, см. в разделе «Методы расширения System.Linq».

При написании LINQ-запросов рекомендуется использовать лямбда-выражения.

Лямбда-выражение – это блок кода (выражение или оператор кода), который рассматривается как объект. Лямбда-выражения можно передавать в качестве аргументов или возвращать в качестве значений из вызовов функций. Для объявления лямбда-выражения используется оператор **=>**. Например, используются конструкции:

```
   .Where(p => p != null)
.Where(p => Function(p))
.Select(p => p.Name)
.Take(10)
.OrderBy(p => p.Date)
```

Рассмотрим пример запроса, использующего метод расширения. Он эквивалентен двум примерам выше. В качестве аргумента метод принимает лямбда-выражение.

Пример 3. Использование лямбда-выражений

```
   string[] cities = { "Москва", "Кострома", "Иркутск", "Сургут", "Сочи"};
IEnumerable<string> selectedCities = cities.Where(c =>
c.ToUpper().StartsWith("С"));
```

Чтобы создать лямбда-выражение, необходимо указать входные параметры, если они есть, с левой стороны оператора **=>**, и поместить блок выражений или операторов с другой стороны. Например, лямбда-выражение **x => 2*x** задает параметр с именем **x** и возвращает значение **2** умноженное на **x**. Оператор **=>** имеет такой же приоритет, как и присваивание (**=**). При написании лямбда-выражений обычно не требуется указывать тип входных параметров.

Подробнее о лямбда-выражениях см. в документации Microsoft статью «Лямбда-выражения (Руководство по программированию в C#)».

Элементы разработки

В разделе описано, как создавать новые решения, модули, отчеты и типы сущностей.

Решение

Решение – часть системы, которая объединяет между собой логически связанные модули, типы сущностей и отчеты. В своем решении можно:

- создавать модули и отчеты;
- *перекрывать* модули и типы сущностей базовых решений, чтобы вносить в них изменения.

Решение может содержать только перекрытые типы сущностей. Создание новых типов, в том числе унаследованных, без привязки к модулю недоступно. При этом в модуле решения можно создавать:

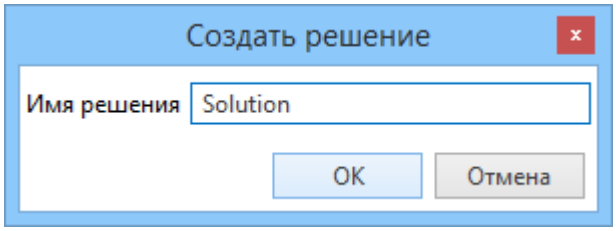
- базовые типы сущностей на основании базовых типов платформы;
- типы сущностей на основании базовых типов платформы, прикладного уровня и своих.

Новое решение создается на *рабочем слое разработки*. При создании типов сущностей используется механизм *наследования*. Подробнее про уровни разработки см. в разделе «Модель разработки».

Решения в проводнике Directum RX не отображаются. Пользователь работает только с элементами решения, например, с модулями, документами, задачами.

Создание решения

1. В контекстном меню дерева решений выберите пункт **Создать решение**.
2. В открывшемся окне укажите имя решения. Имя должно быть уникальным, состоять из букв латинского алфавита и цифр.

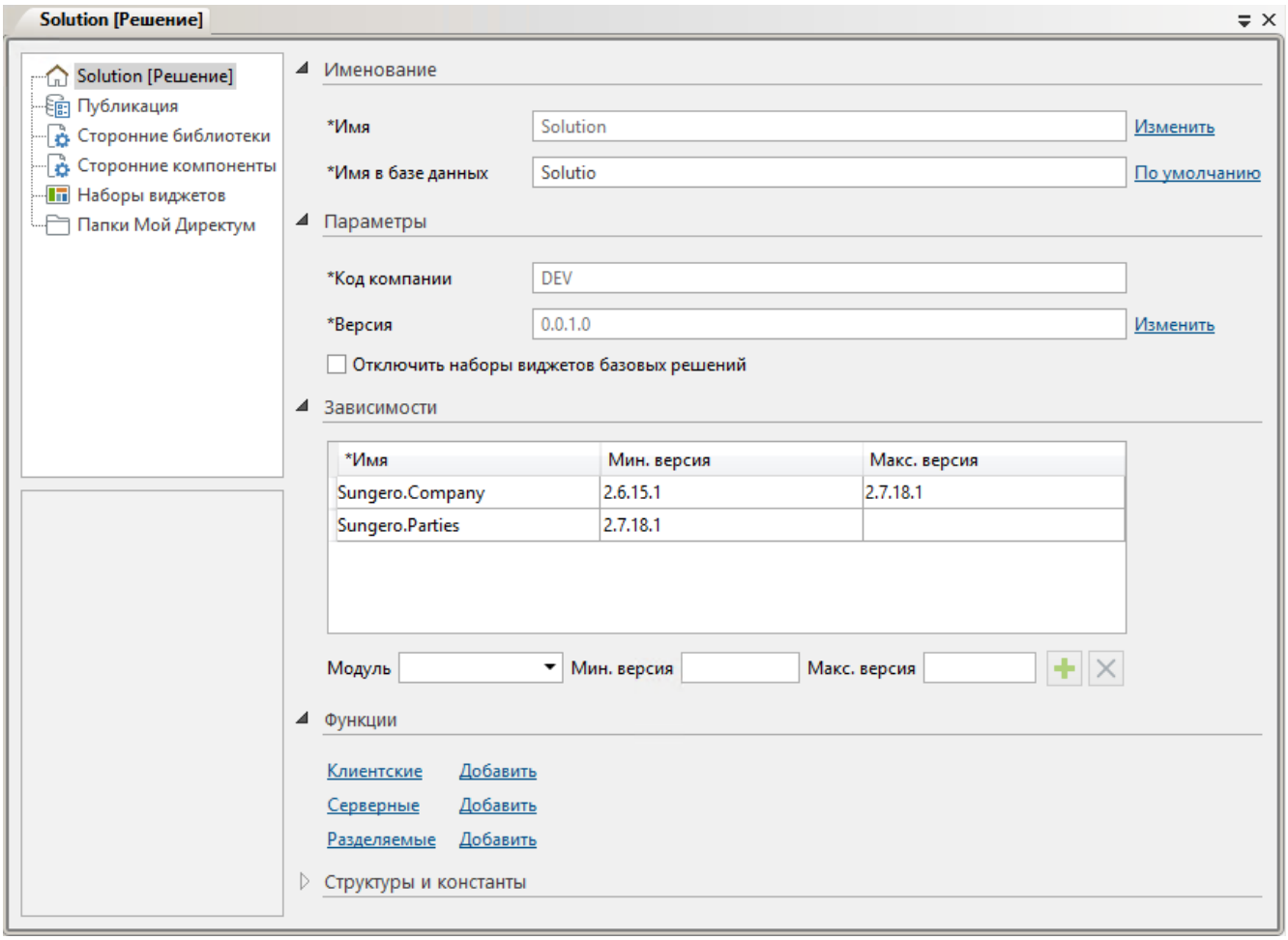


3. Нажмите на кнопку **ОК**. В дереве появится новое решение.
4. Заполните необходимые параметры в редакторе решения.
5. Создайте в решении необходимые модули, типы сущностей, отчеты или добавьте перекрытия типов сущностей или модулей *базовых решений*.
6. При необходимости настройте наборы виджетов и задайте расположение папок в **Мой Директум**.

Редактор решения

Чтобы задать или изменить настройки решения, щелкните два раза левой кнопкой мыши по его названию в дереве решений. Откроется редактор, в котором задаются свойства решения, настройки публикации для *перекрытий модулей*, подключаются сторонние библиотеки и сторонние компоненты, создаются наборы виджетов, добавляются функции.

При открытии редактора отображается основной узел «Решение», в котором задаются параметры, зависимости перекрытий модулей, функции, структуры и константы.



Именование

***Имя**. Указывается при создании решения. Имя решения должно быть уникальным, состоять из букв латинского алфавита и цифр. При необходимости имя решения можно изменить. Для этого сохраните изменения в редакторе, перейдите по ссылке **Изменить** и задайте новое имя. После изменения имени доработайте программный код, в котором оно используется.

***Имя в базе данных**. Используется при генерации таблиц для перекрытий в базе данных. Заполняется автоматически. При необходимости имя в базе данных можно изменить. Для этого введите нужное значение вручную. Имя решения должно быть уникальным и состоять из букв латинского алфавита. Чтобы вернуть первоначальное значение, нажмите **По умолчанию**. После изменения имени решения в базе данных, доработайте программный код и SQL-запросы, в которых оно используется.

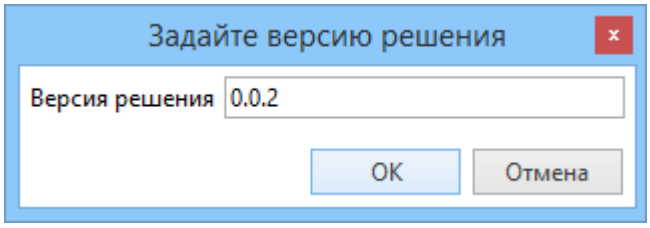
Параметры

***Код компании**. Используется при написании программного кода и генерации уникальных имен таблиц в базе данных. По коду можно определить, какая компания разработала решение. Заполняется автоматически и доступен только для чтения.

***Версия**. Номер версии решения. Заполняется автоматически при создании решения. Последняя цифра версии увеличивается на 1 после каждого создания пакета разработки с исполняемыми файлами.

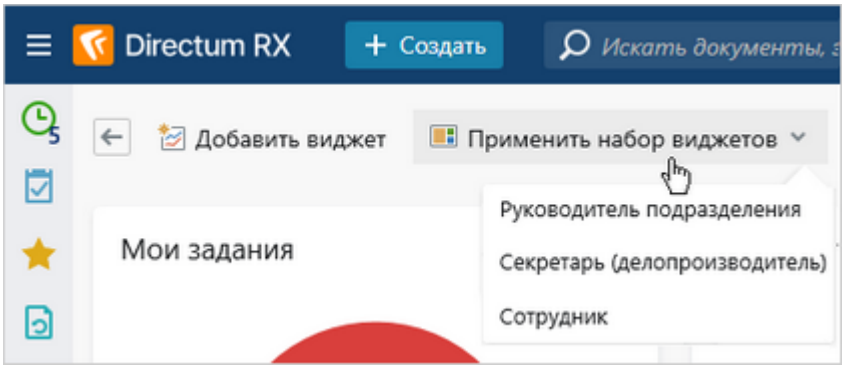
Измените номер вручную, например, если для версии решения нужно добавить скрипт конвертации. Для этого:

1. Откройте редактор решения и перейдите по ссылке **Изменить** рядом с полем **Версия***.
2. В открывшемся окне заполните поле **Версия решения**:



   Для изменения доступны только первые три цифры номера. Номер версии можно только увеличивать.
3. Нажмите на кнопку **OK**. В результате будет увеличена версия решения и всех его модулей.

Отключить наборы виджетов базовых решений. Наборы виджетов, разработанные в *базовых решениях*, будут скрыты в списке наборов, доступных пользователю для выбора в проводнике системы:



ПРИМЕЧАНИЕ. Если набор скрыт, то виджеты, которые в него входят, останутся доступны пользователям для добавления в область виджетов.

Зависимости

Зависимости. Список, который состоит из:

- модулей *базового решения*, которые перекрыты в вашем решении;
- модулей базового и ваших решений, программный код или элементы разработки которых используются в перекрытиях модулей или функциях вашего решения.

Рекомендуется добавлять модуль в список зависимостей сразу после использования в своем решении. Если зависимости не указаны, то при публикации возникнут ошибки.

Например, если в своем решении вы перекрыли модуль «Компания», добавьте модуль базового решения в список зависимостей. Для этого:

1. В выпадающем списке **Модуль** выберите модуль «Компания» (Sungero.Company). В поле **Мин. версия** автоматически подставится текущая версия модуля. Рекомендуется оставлять значение по умолчанию.
2. Нажмите на кнопку . В табличной части «Зависимости» отобразится добавленная зависимость.

   Если зависимость добавлена по ошибке, выделите ее в табличной части и нажмите на кнопку .

Не рекомендуется удалять зависимости из списка, даже если элементы модуля больше не используются.

СОВЕТ. После добавления зависимости запустите публикацию с выбором и проверьте, что для решений, модули которых указаны в качестве зависимых, установились синие положения ползунков. Это необходимо для корректной публикации решения.

Функции

Добавление клиентских, серверных и разделяемых функций. Функции создаются в решении тогда, когда предполагается, что одна и та же логика будет выполняться в нескольких местах, например, в нескольких перекрытиях модулей. Функции решения создаются аналогично функциям модуля.

Структуры и константы

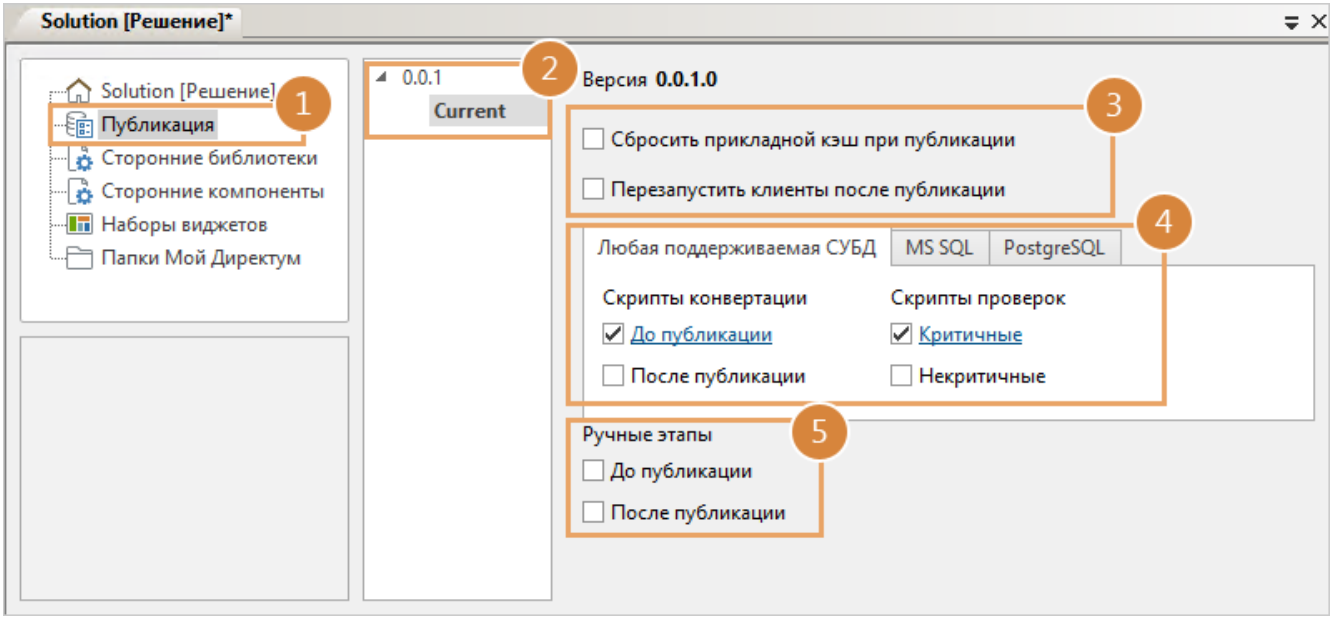
Добавление структур и констант, которые в последующем можно использовать в функциях и обработчиках событий.

Публикация

Публикация представляет собой обновление разработки на веб-сервере. Настройки публикации задаются в редакторе решения или модуля. Включают добавление скриптов конвертации, проверок и ручных этапов, которые нужно выполнить до или после публикации. Если изменился перекрытый модуль, то задайте настройки в редакторе решения. Если изменения произошли в разработке, которую сделали вы сами, задайте настройки в редакторе соответствующего модуля.

Чтобы задать настройки публикации:

1. В редакторе решения или модуля перейдите в узел «Публикация».



2. В узле **Current**, который соответствует текущей версии, укажите настройки. Если нужно логически отделить скрипт конвертации по версии, увеличьте номер версии решения и после этого добавьте необходимый скрипт.

   ВАЖНО. Если собирается пакет разработки с исполняемыми файлами, то последняя цифра версии автоматически увеличивается на 1. Например, с **0.0.2.0** до **0.0.2.1**.

   При необходимости можно посмотреть скрипты предыдущих версий. Для этого раскройте узел версии, выше текущей.

3. При необходимости установите флажки:

   Сбросить прикладной кэш при публикации, если в программном коде изменилась работа с кэшем. Например, изменилось вычисление для виджетов модулей решения.

   Перезапустить клиенты после публикации, если необходимо, чтобы после обновления продуктивной системы перезапустились клиентские приложения, которые были открыты. Для пользователей, у которых запущено клиентское приложение в момент обновления, появится сообщение о том, что приложение необходимо перезапустить. В этом случае после публикации у пользователей появится сообщение о том, что необходимо обновить страницу браузера.

   Устанавливать флажок для локальной системы не требуется. Клиентские приложения локальной системы, в которых разработчик проверяет изменения перед отправкой в удаленный репозиторий, после публикации перезапускаются самостоятельно.

4. Добавьте скрипты. Для этого выберите вкладку:
   - «Любая поддерживаемая СУБД» – если синтаксис запросов подходит для PostgreSQL/Postgres Pro/Jatoba и Microsoft SQL Server;
   - «MS SQL» и «PostgreSQL» – если синтаксис запросов отличается. Добавьте на каждой вкладке свой скрипт запроса, в дальнейшем система сама определит нужный скрипт в зависимости от того, какая СУБД используется.

   На вкладках рядом с нужным названием скрипта установите флажок. Станет доступна ссылка, перейдите по ней. Откроется редактор SQL-запросов. В редакторе напишите код SQL-запроса.

   Чтобы удалить скрипт, снимите флажок.
5. Если у вас настроена синхронизация с другими базами данных, добавьте ручные этапы. В тексте запроса укажите скрипт для конвертации другой базы данных.
6. Сохраните изменения.

Сторонние библиотеки

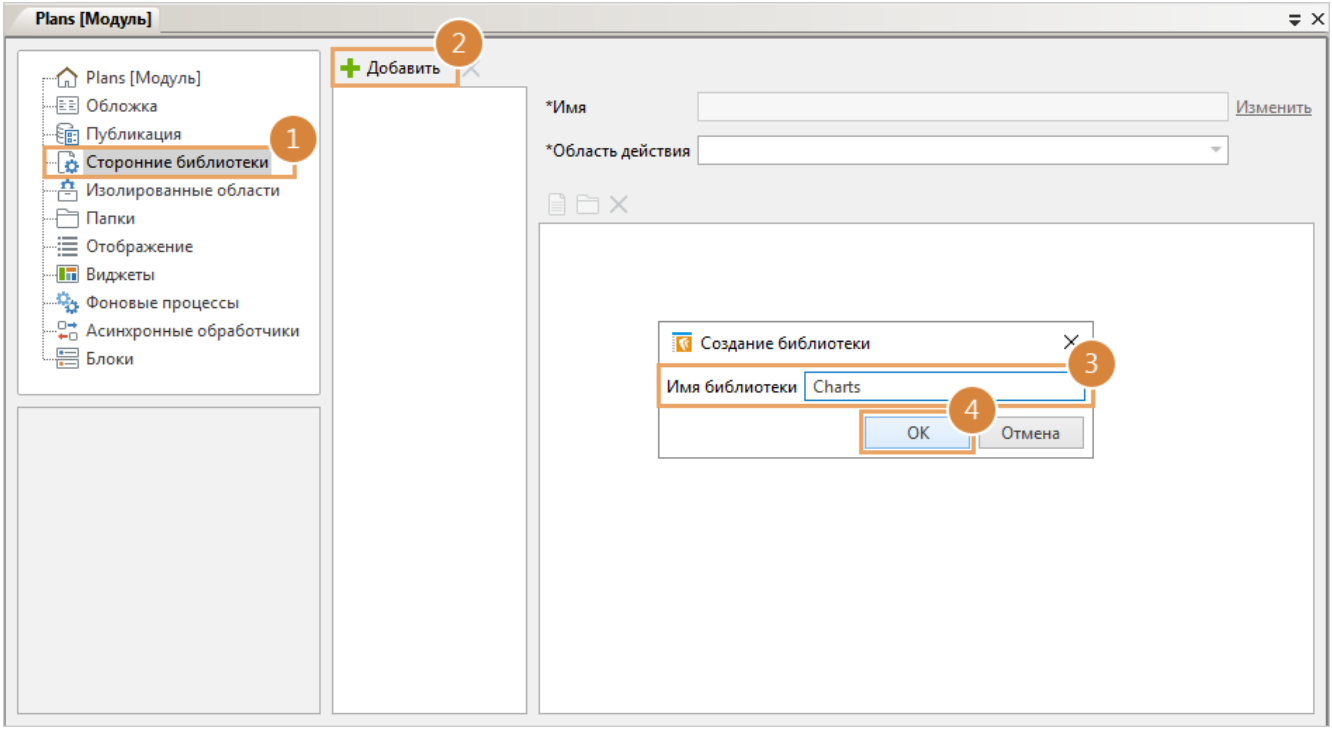
Для расширения возможностей системы доступно добавление *сторонних библиотек*. Вы можете реализовать необходимую функциональность в виде отдельной .NET-библиотеки и подключить ее к нужному решению или модулю. После добавления библиотеки *сторонние сборки*, которые входят в ее состав, можно использовать в программном коде.

Выполнение кода сторонней сборки можно перенести на сервис изолированного кода. Для этого нужно подключить стороннюю библиотеку к изолированной области. Тогда код сторонней сборки будет выполняться в отдельном процессе. Работа других сервисов продолжится, даже если в стороннем коде возникнут ошибки.

ВАЖНО. Сторонние библиотеки для прикладной разработки Directum RX должны быть собраны под .NET Standard 2.0.

Чтобы добавить стороннюю библиотеку:

1. В редакторе решения или модуля перейдите в узел «Сторонние библиотеки».



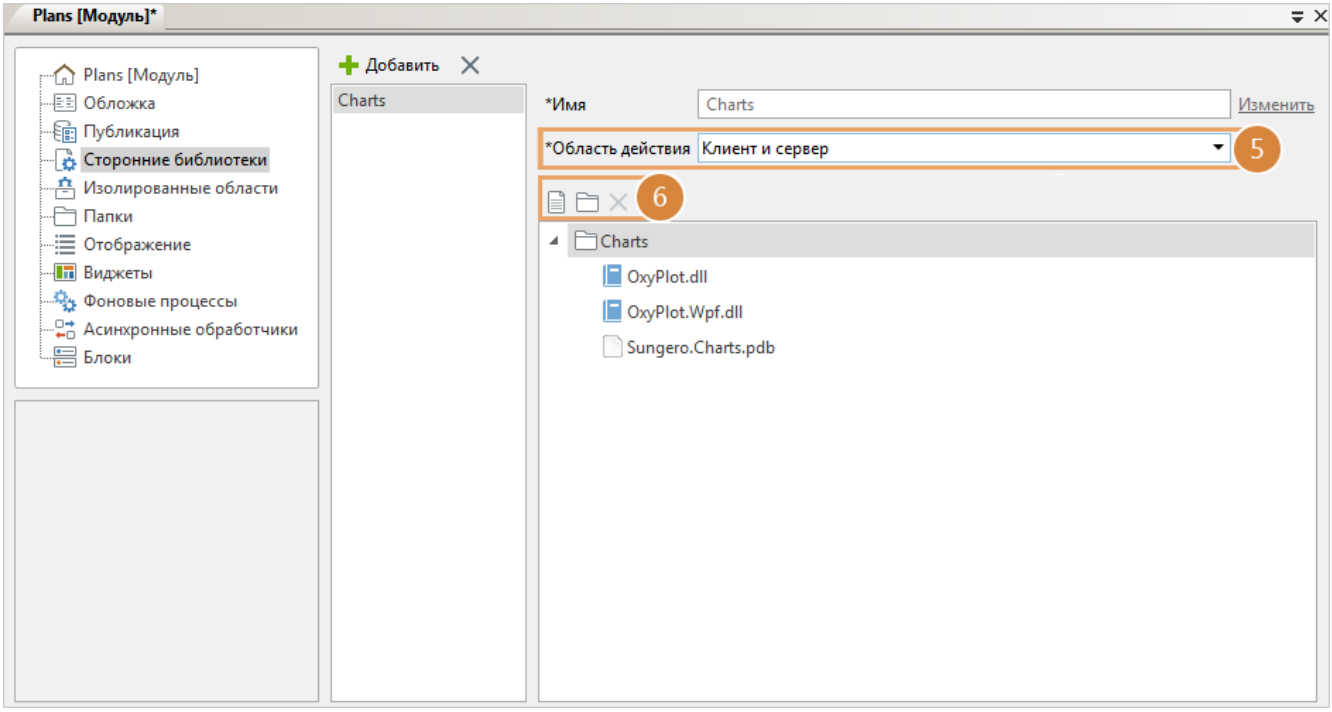
2. Нажмите на кнопку **Добавить**.
3. В открывшемся окне укажите имя библиотеки. Оно должно быть уникальным, состоять из букв латинского алфавита и цифр.
4. Нажмите на кнопку **ОК**.
5. Заполните поле ***Область действия**, по которому определяется, в каком программном коде будут доступны сборки библиотеки. Возможные значения:

   WPF Клиент. Настройка является устаревшей и оставлена для совместимости.

   Базовый клиент. Сборки доступны в клиентском коде, предназначенном для веб-клиента.

   Сервер и изолированные области. Сборки доступны в серверном коде, например в событиях сущности. Также сборки доступны в коде изолированной области.

Клиент и сервер. Сборки доступны в любом программном коде.



6. Добавьте содержимое библиотеки. Для этого воспользуйтесь кнопками:
   - для добавления содержимого всей папки, выбранной с локального диска;
   - для добавления определенного файла в выбранную папку библиотеки.

   ПРИМЕЧАНИЕ. Запрещается использование сторонних сборок размером более 15 МБ.

После загрузки в области предпросмотра отображаются все добавленные сборки и файлы библиотеки. Сборки, которые можно использовать в программном коде, отмечены значком . Остальные файлы библиотеки отмечены значком .

ВАЖНО. Сторонняя библиотека может использоваться только в рамках того решения или модуля, в который она добавлена. Чтобы использовать библиотеку в нескольких модулях, добавьте ее в каждый модуль и заполните одинаковым набором файлов и сборок одной версии.

Сторонние компоненты

Чтобы адаптировать интерфейс веб-клиента под потребности компании, вы можете использовать свои контролы. Например, в карточку документа добавить контрол, который отображает состояние жизненного цикла. Для этого разработайте свой контрол и добавьте файлы стороннего компонента в среду разработки. Веб-приложение стороннего компонента размещается на стороне веб-сервера Directum RX и публикуется с пакетом прикладной разработки.

Сторонний компонент может содержать один или несколько контролов.

Разработка контрола

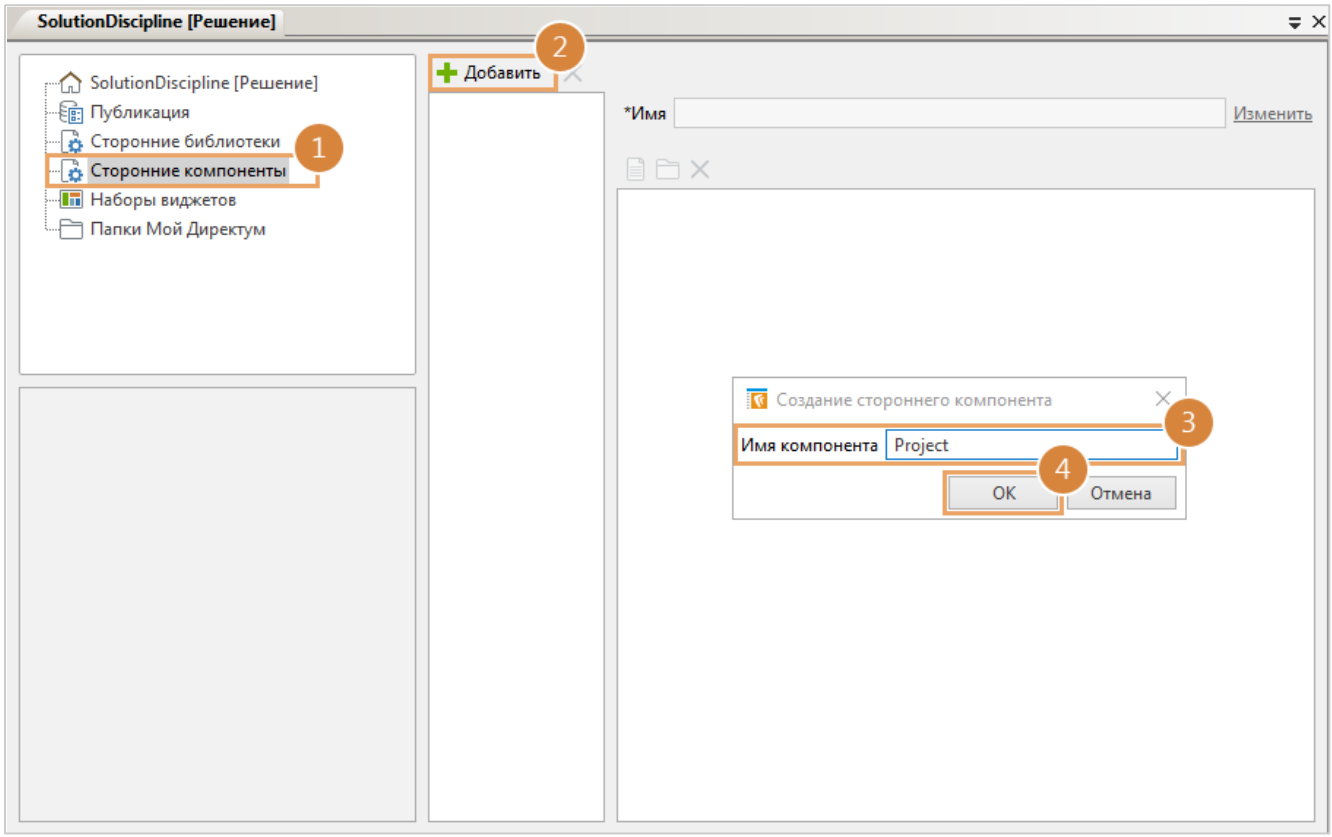
В репозитории GitHub доступен пример стороннего компонента с контролами – одностраничное веб-приложение, которое можно встроить в веб-клиент. Пример написан с использованием библиотеки React. Чтобы разработать свои контролы, скачайте пример стороннего компонента из репозитория и доработайте его.

ПРИМЕЧАНИЕ. Чтобы в сторонний компонент передать настройки веб-сервера, используйте параметр **REMOTE_COMPONENT_SETTINGS**. Подробнее см. в руководстве администратора, раздел «Веб-сервер» (Linux, Windows).

Добавление стороннего компонента

Чтобы добавить сторонний компонент:

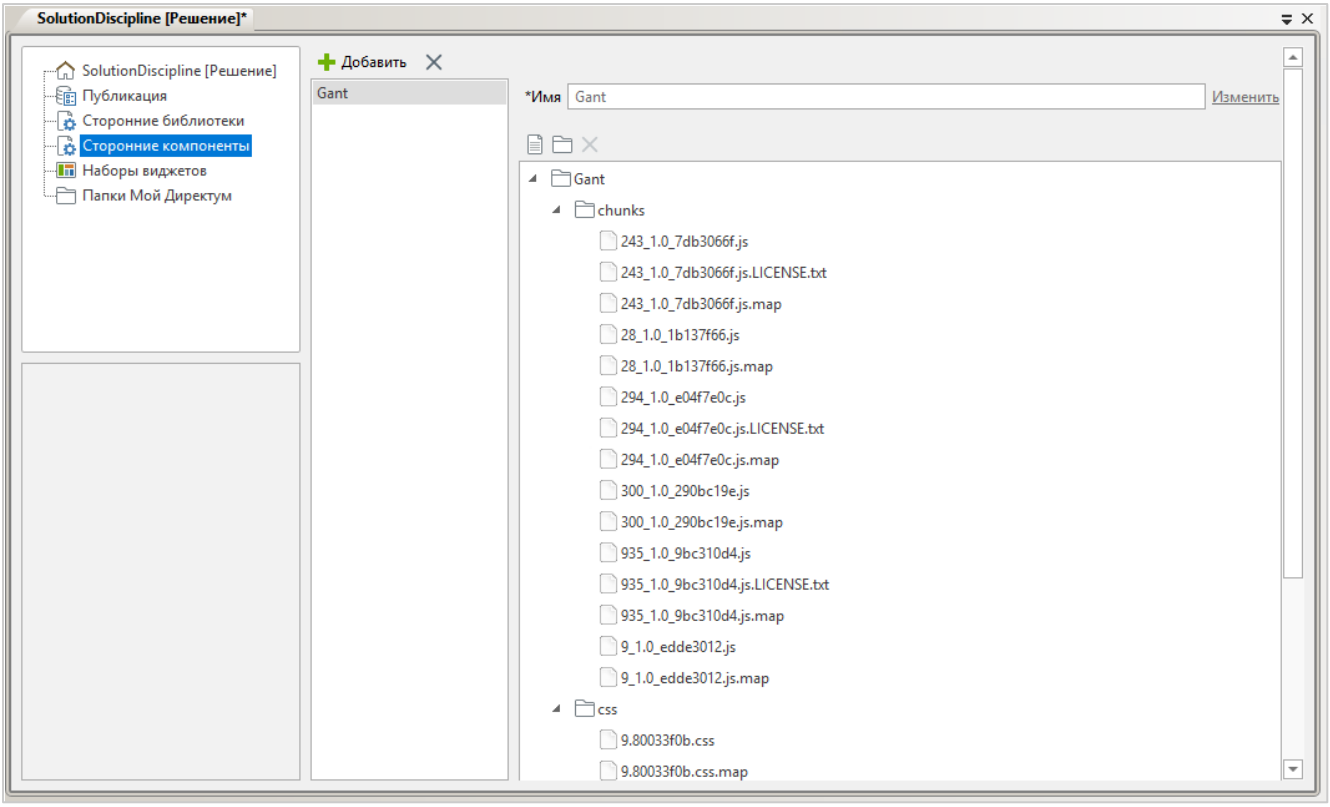
1. В редакторе решения перейдите в узел «Сторонние компоненты»:



2. Нажмите на кнопку **Добавить**.
3. В открывшемся окне укажите имя компонента. Оно должно быть уникальным, состоять из букв латинского алфавита и цифр.
4. Нажмите на кнопку **ОК**.
5. Добавьте содержимое компонента. Для этого воспользуйтесь кнопками:
   - – добавить содержимое всей папки на локальном диске.

     ВАЖНО. В папке для загрузки должно быть не более 1000 файлов;
   - – добавить определенный файл в выбранную папку компонента.

После загрузки в области предпросмотра отображаются все файлы стороннего компонента:



В результате сторонние контролы можно добавить:

- в среде разработки в узле «Форма»;
- в проводнике веб-клиента в редакторах форм и обложек.

В среде разработки сторонний контрол может использоваться только в том решении, в котором он добавлен. Чтобы использовать контрол в нескольких решениях, добавьте сторонний компонент с ним в каждое и заполните одинаковым набором файлов одной версии.

Версионирование контрола

В веб-клиенте можно одновременно работать с несколькими версиями одного контрола. Например, чтобы критичные доработки стороннего компонента не повлияли на перекрытые решения, рекомендуется вносить изменения в новую версию.

Версия стороннего контрола соответствует версии стороннего компонента и задается в скачанном файле component.manifest.js в параметре **componentVersion**.

Чтобы создать новую версию стороннего контрола:

1. В скачанном файле component.manifest.js стороннего компонента в параметре **componentVersion** измените номер версии.
2. Соберите проект стороннего компонента.
3. В среде разработки в редакторе решения добавьте новый сторонний компонент. Рекомендуется указать в названии номер версии, чтобы отличать от предыдущей.
4. Опубликуйте решение.

В результате в редакторе форм и обложек можно использовать и созданную ранее версию контрола, и новую.

Наборы виджетов

В проводнике системы есть область виджетов, которая предназначена для удобного отображения данных о процессах работы в системе и быстрого доступа к документам, заданиям, папкам и т.д. В области виджетов отображаются:

- виджеты, для которых в среде разработки задается логика для отображения информации, перехода в списки или формирования отчетов. Например, в виджете может содержаться информация о количестве задач в работе, действие для формирования отчета. Разрабатываются в редакторах модулей в узле «Виджеты»;
- виджеты типов сущностей и папок. При нажатии на действия в виджетах открываются соответствующие списки сущностей и папки. Например, при нажатии в проводнике системы на виджет папки «Избранное» откроется данная папка текущего пользователя. Добавляются в редакторе решения в узле «Наборы виджетов».

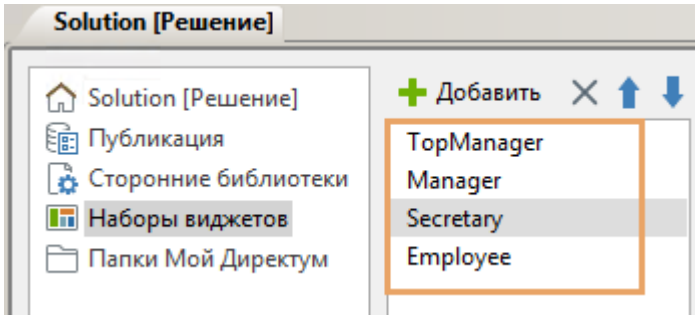
В *базовом решении* есть готовые наборы виджетов для работы пользователей. Наборы доступны в зависимости от ролей, в которые входит пользователь. Пользователь может использовать один из доступных ему наборов и изменить состав виджетов.

С помощью среды разработки вы можете:

- создавать свои наборы, содержащие существующие виджеты базовых решений, виджеты типов сущностей и папок, а также виджеты, разработанные вами в новом решении;
- настроить доступность набора виджетов для пользователей. Например, чтобы набор был доступен только пользователям, которые входят в определенную роль или занимают определенную должность;
- скрывать в редакторе текущего решения наборы виджетов базового решения из списка доступных пользователю.

Подробнее см. раздел «Пример создания набора виджетов».

При первом открытии проводника системы автоматически применяется набор виджетов, доступный для пользователя. Если доступно несколько наборов, то применится тот из них, который идет первым в списке в редакторе решения в узле «Наборы виджетов»:



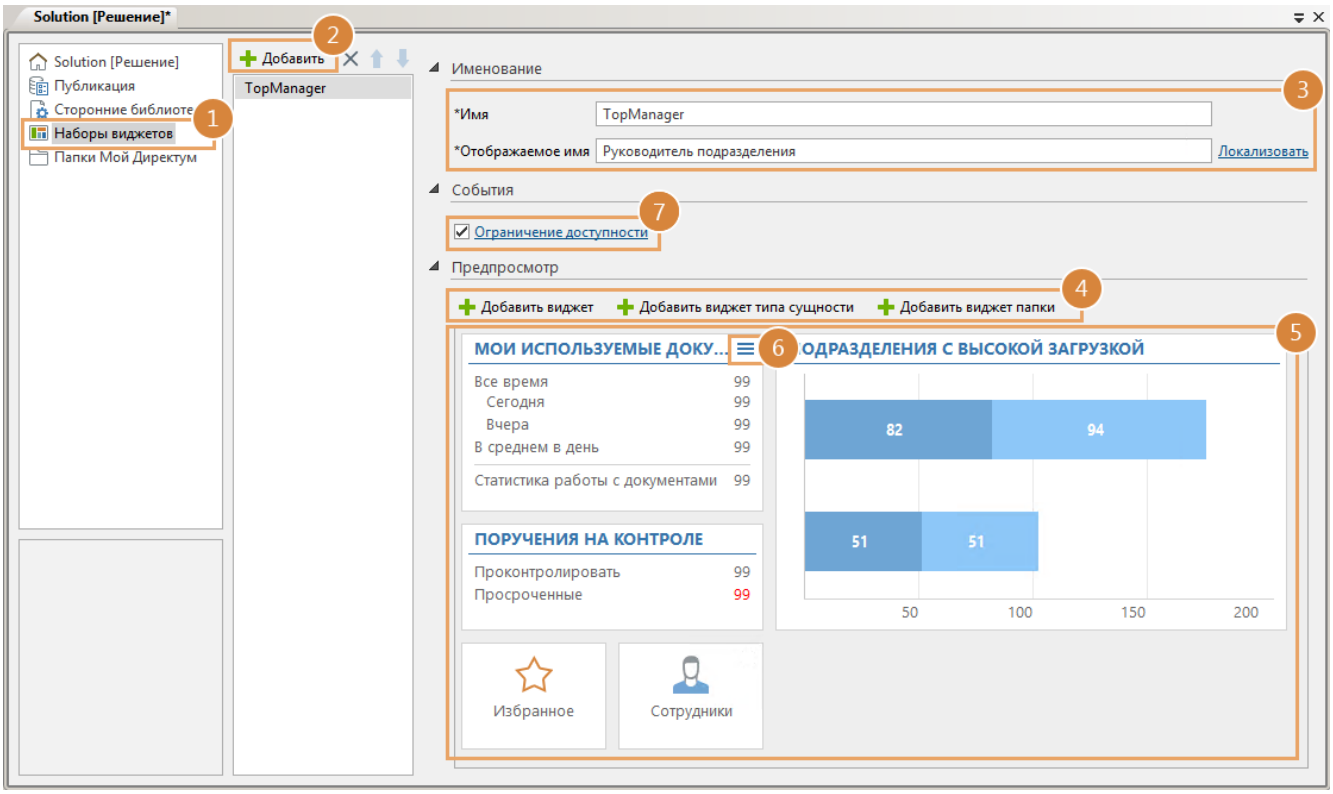
Пример создания набора виджетов

Предположим, необходимо создать набор виджетов, который в проводнике системы доступен только руководителям подразделений. В набор нужно добавить виджеты:

- «Мои используемые документы», разработан в новом решении. У виджета есть параметр-перечисление **Показывать документы**. Возможные значения параметра: **Все**, **Измененные**, **Просмотренные**. По умолчанию отображается информация по всем документам. Необходимо, чтобы в новом наборе в виджете отображалась информация только по документам, которые были открыты для изменения;
- «Подразделения с высокой загрузкой» и «Поручения на контроле», которые уже есть в базовом решении;
- для перехода в список сотрудников;
- для открытия папки «Избранное».

Для этого:

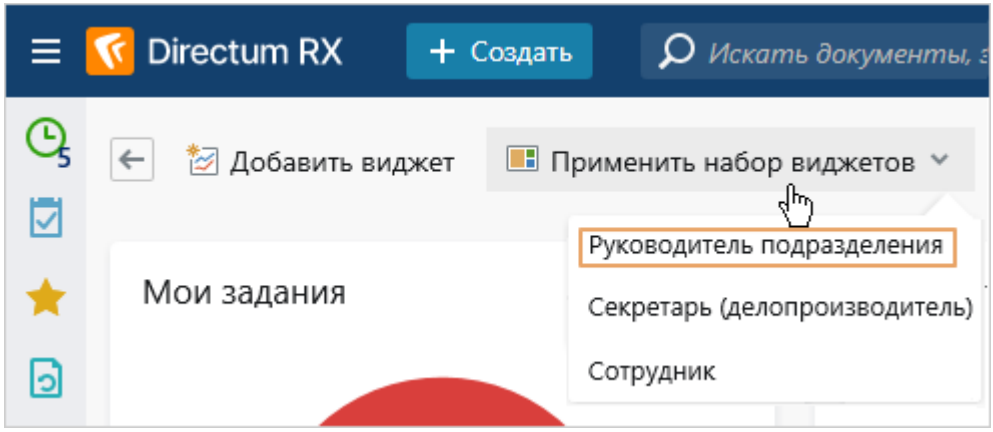
1. В редакторе решения перейдите в узел «Наборы виджетов».



2. Нажмите на кнопку **Добавить**.
3. В списке элементов выделите добавленный элемент и задайте настройки:

   ***Имя** набора виджетов. Должно быть уникальным в рамках одного решения, состоять из букв латинского алфавита и цифр.

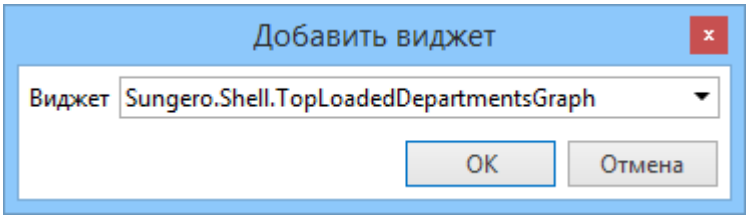
***Отображаемое имя** набора виджетов, которое будет отображаться в качестве названия набора в проводнике системы.



Для *локализации* отображаемого имени перейдите по ссылке **Локализовать**.

4. Добавьте в набор необходимые виджеты. Для этого используйте кнопки:

   Добавить виджет. Один из разработанных виджетов. При нажатии на кнопку откроется окно:



   В выпадающем списке **Виджет** для выбора доступны виджеты базового решения и виджеты, разработанные в новом решении. Выберите нужный и нажмите на кнопку **ОК**.

   Добавить виджет типа сущности. Виджет, который представляет собой ссылку на один из список типов сущности. При нажатии на кнопку откроется окно для выбора типа сущности. В выпадающем списке выберите тип сущности «Сотрудники» (Sungero.Company.Employee) и нажмите на кнопку **ОК**.

   Добавить виджет папки. Виджет, который представляет собой ссылку на одну из папок. При нажатии на кнопку откроется окно для выбора вычисляемой папки. В выпадающем списке выберите папку «Избранное» (Favorites) и нажмите на кнопку **ОК**.

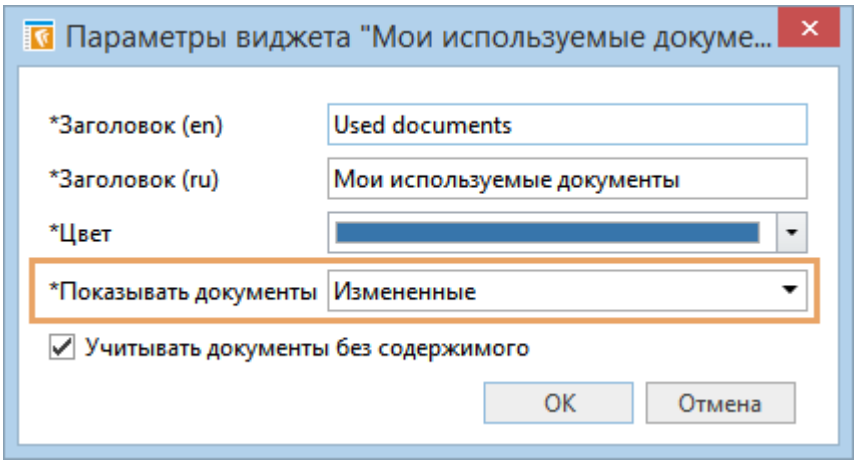
   Добавленные виджеты отобразятся в области предпросмотра набора виджетов.

5. Настройте расположение виджетов в наборе. Для этого перемещайте виджеты в области предпросмотра с помощью мыши.

6. По умолчанию в виджете «Мои используемые документы» отображается информация по всем документам. Необходимо, чтобы в нем отображалась информация только по документам, которые были открыты для изменения. Для этого измените значение параметра виджета **Показывать документы**.

   ВАЖНО. Состав параметров зависит от виджета и настраивается в редакторе модуля в узле «Виджеты».

Чтобы изменить значение параметра, в области предпросмотра в виджете нажмите на кнопку **Параметры**. Откроется окно настройки:



В выпадающем списке **Показывать документы** выберите значение **Измененные** и нажмите на кнопку **OK**.

7. Задайте обработчик события **Ограничение доступности**, чтобы определить состав пользователей, которым доступен данный набор виджетов. Для этого установите флажок **Ограничение доступности** и перейдите по ссылке. Откроется редактор программного кода. Задайте ограничение по вхождению в роль «Руководители подразделений».

   ВАЖНО. Если обработчик события **Ограничение доступности** не задан, то набор виджетов будет доступен для выбора всем пользователям.

   Пример. Обработчик события **Ограничение доступности** для набора виджетов руководителя:

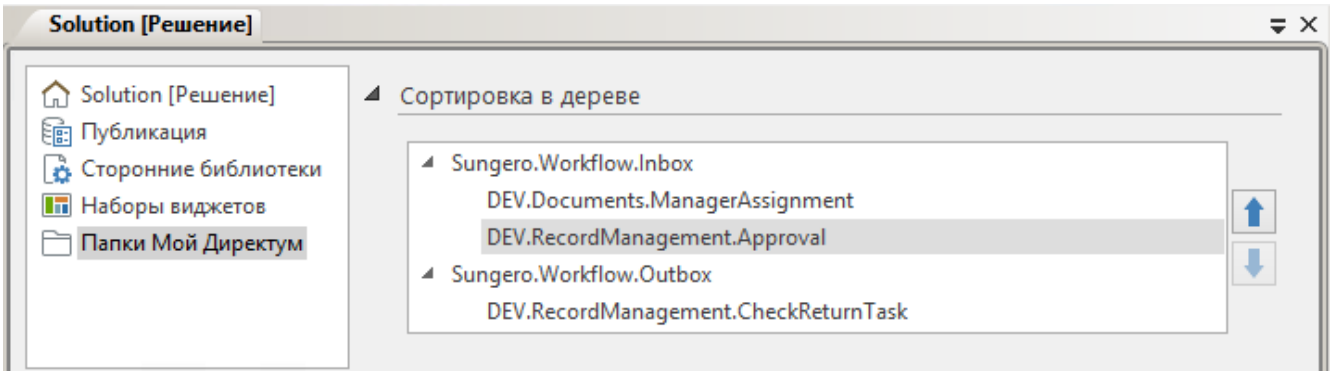
```
internal static class DesktopViewHandlers
  {
    /// <summary>
    /// Проверить вхождение текущего пользователя
    /// в роль руководителей подразделений.
    /// </summary>
    /// <returns> True, если входит, иначе False.</returns>
    private static bool CanApplyTopManager()
    {
      return IncludedInRole("Руководители подразделений");
    }
    /// <summary>
    /// Проверить вхождение текущего пользователя в роль.
    /// </summary>
    /// <param name="roleName"> Имя роли.</param>
    /// <returns> True, если входит, иначе False.</returns>
    public static bool IncludedInRole(string roleName)
    {
      var ceoRole = Roles.GetAll(r => r.Name == roleName).FirstOrDefault();
      if (ceoRole == null)
        return false;
      var employee = Users.Current;
      return employee.IncludedIn(ceoRole);
    }
  }
```

В результате в проводнике системы появится новый набор виджетов, который будет доступен для пользователей, входящих в роль «Руководители подразделений».

Папки Мой Директум

Для настройки порядка расположения подпапок в папках «Входящие» и «Исходящие» в проводнике системы используйте узел «Папки Мой Директум». В нем отображаются все вычисляемые папки, созданные в модулях данного решения и перекрытиях модулей, для которых в редакторе в поле ***Область отображения** указаны значения **Папка «Входящие»** или **Папка «Исходящие»**.

Чтобы изменить расположение подпапок, используйте кнопки ⬆ и ⬇. Изменять расположение подпапок можно только в рамках папок «Входящие» или «Исходящие».



В результате при открытии проводника системы пользователь увидит только те папки, которые подходят ему по событию Ограничение доступности и в том порядке, который задан в редакторе решения. Если ограничений нет, но папки не отображаются у пользователя, выполните дополнительные действия.

Если у пользователя не отображается папка «Входящие» или «Исходящие», очистите настройки видимости папок с помощью запроса:

- для конкретного пользователя:

```
delete from sungero_system_setting where name like
'SpecialFoldersVisibility' and owner = <ИД_сотрудника>
```

- для всех пользователей:

```
delete from sungero_system_setting where name like
'SpecialFoldersVisibility'
```

Если изменения не подхватились, перезапустите пул веб-сервера Directum RX.

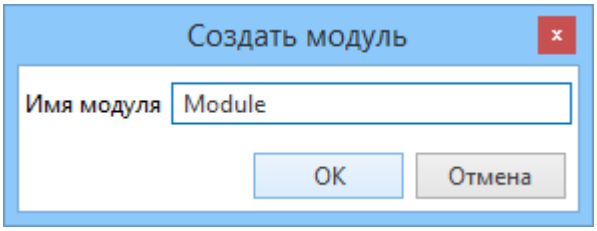
Модуль

Модуль – самостоятельная часть системы, решающая бизнес-задачи некоторой предметной области. Например, модуль «Договоры» позволяет автоматизировать работу с договорными документами. Модуль состоит из типов сущностей, вычисляемых папок и отчетов. В своем решении можно создать новый модуль или перекрыть существующий.

ПРИМЕЧАНИЕ. При создании и перекрытии модуля необходимо учитывать схему и связи модулей стандартного решения Directum RX.

Создание модуля

1. В дереве выберите ваше решение, в котором будет находиться модуль. В контекстном меню решения последовательно выберите пункты **Создать** и **Модуль**.
2. В открывшемся окне укажите имя модуля. Оно используется, например, при обращении к функциям, типам сущностей. Имя модуля должно быть уникальным, состоять из букв латинского алфавита и цифр.



3. Нажмите на кнопку **ОК**. В дереве решений появится новый модуль.
4. Откройте редактор модуля и заполните необходимые параметры.
5. Создайте типы сущностей и вычисляемые папки модуля. Настройте состав и порядок их расположения в проводнике системы.
6. Создайте блоки бизнес-процессов, если схему задачи планируется настраивать в проводнике веб-клиента. Блоки будут доступны в библиотеке блоков для настройки схем процессов.
7. Настройте обложку модуля и виджеты.
8. Настройте фоновые процессы.

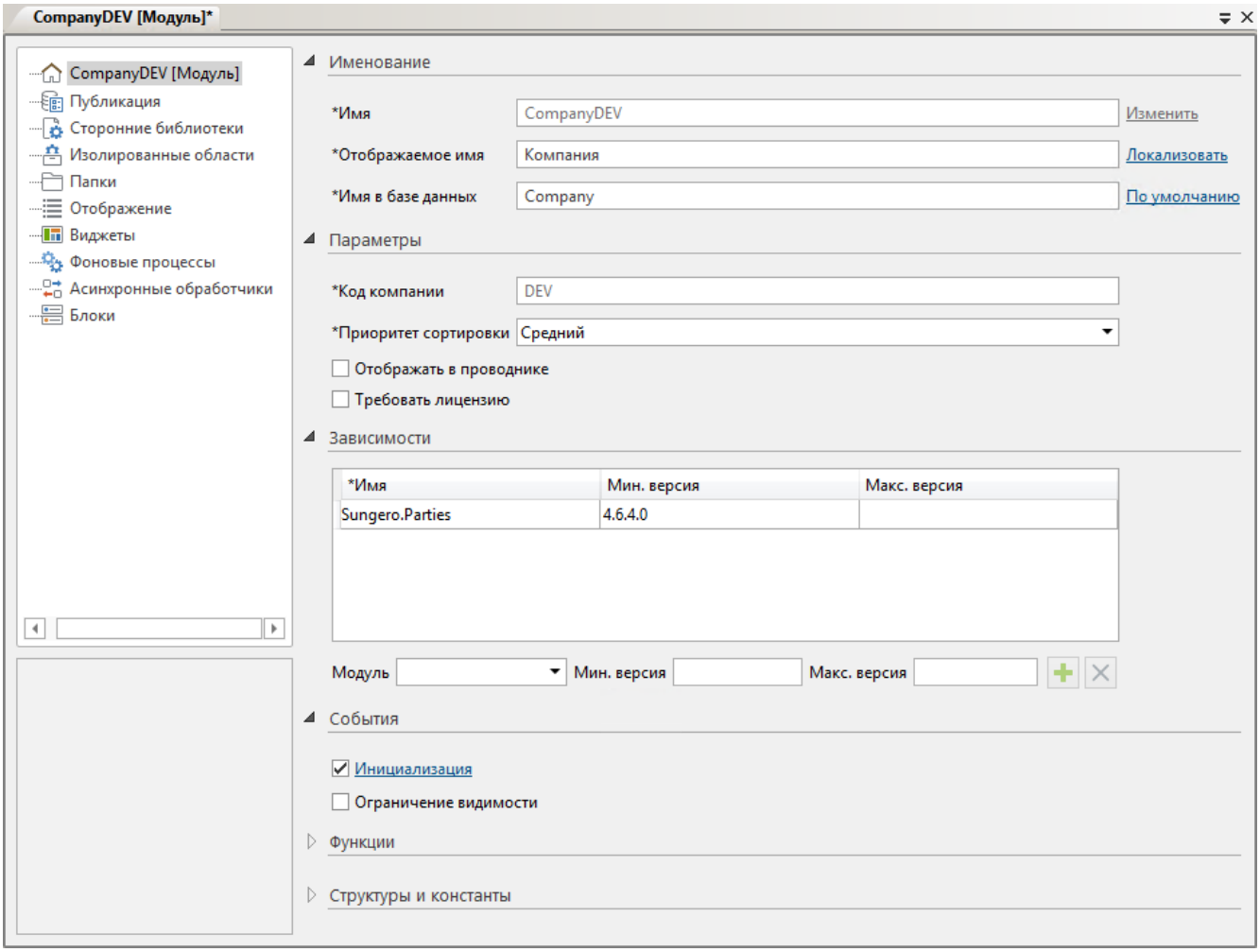
При сборке решения в него включаются созданные модули. Если в среде разработки один или несколько модулей удалить из состава решения, то при публикации они удалятся и из решения в системе.

Редактор модуля

Чтобы задать или изменить настройки модуля, щелкните два раза левой кнопкой мыши по его названию в дереве решений. Откроется редактор, в котором:

- задаются свойства модуля, настройки публикации;
- подключаются сторонние библиотеки и создаются изолированные области;
- добавляются папки;
- настраивается отображение элементов модуля в дереве;
- разрабатываются виджеты, фоновые процессы и асинхронные обработчики;
- добавляются функции.

При открытии редактора отображается основной узел «Модуль», в котором задаются параметры, зависимости от других модулей, функции, структуры и константы.



Именование

***Имя**. Указывается при создании модуля. Используется, например, при обращении к функциям, типам сущностей. Имя модуля должно быть уникальным в рамках кода компании, состоять из букв латинского алфавита и цифр. При необходимости имя модуля можно изменить. Для этого сохраните изменения в редакторе модуля, перейдите по ссылке **Изменить** и задайте новое имя. После изменения имени доработайте программный код, в котором оно используется.

***Отображаемое имя**. Имя модуля, которое будет отображаться в проводнике системы. Для *локализации* имени перейдите по ссылке **Локализовать**.

СОВЕТ. Для удобной работы в системе рекомендуется задавать короткое имя модуля.

***Имя в базе данных**. Используется при генерации таблиц типов сущностей модуля в базе данных. Заполняется автоматически. При необходимости имя в базе данных можно изменить. Для этого введите нужное значение вручную. Имя модуля должно быть уникальным в рамках кода компании и состоять из букв латинского алфавита. Чтобы вернуть первоначальное значение, нажмите **По умолчанию**. После изменения имени модуля в базе данных, доработайте программный код и SQL-запросы, в которых он используется.

Параметры

***Код компании**. Используется при написании программного кода и генерации уникальных имен таблиц в базе данных. По коду можно определить, какая компания разработала модуль. Заполняется автоматически и доступен только для чтения.

***Приоритет сортировки**. Позволяет задать расположение модуля в проводнике системы. Возможные значения:

- **Высокий**. Модуль будет находиться вверху списка модулей. Рекомендуется указывать для часто используемых модулей;
- **Средний**. Модуль будет находиться в середине списка модулей;
- **Низкий**. Модуль будет находиться внизу списка модулей. Рекомендуется указывать для редко используемых модулей.

Модули с одинаковым приоритетом отображаются в алфавитном порядке.

ПРИМЕЧАНИЕ. Приоритет учитывается только для обложек, созданных в среде разработки. Новые обложки рекомендуется настраивать в веб-клиенте с помощью справочника **Представления модулей**. Подробнее см. раздел «Обложка».

Отображать в проводнике. Не рекомендуется вручную устанавливать флажок. Он автоматически снимается при создании нового модуля. Чтобы отобразить модуль на панели навигации, в веб-клиенте настройте представление модуля и обложку. Флажок установлен только для модулей, обложки которых были ранее созданы в среде разработки. В этом случае перенесите обложки в веб-клиент, подробнее см. инструкцию по обновлению, раздел «Конвертация обложек модулей».

ПРИМЕЧАНИЕ. Обложку нельзя настраивать одновременно в веб-клиенте и в среде разработки, так как в этом случае на панели навигации появятся дубли модулей и обложек.

Требовать лицензию. Если флажок установлен, для работы с модулем необходимо приобрести лицензию на использование. Подробнее см. описание архитектуры системы, раздел «Лицензирование».

При разработке решения с лицензируемым модулем учитывайте особенности доступа к вычисляемым папкам и действиям:

- если в лицензируемом модуле есть папки, которые нужно показывать на панели навигации и без лицензии, то их рекомендуется вынести в нелицензируемый модуль. В этом случае доступ к папкам настраивается через роли. Подробнее см. в руководстве администратора, раздел «Предопределенные роли». Если папки оставить в лицензируемом модуле, то без действующей лицензии они скрыты на панели навигации;
- на обложке отображаются все действия, которые вынесены при ее настройке. Если действие регулируется лицензией, а она отсутствует, то при попытке выполнить его возникает ошибка. Например, для массовой выгрузки документов из архива нужна лицензия на модуль «Финансовый архив». Подробнее см. в руководстве администратора, раздел «Предопределенные роли».

Зависимости

Зависимости. Список модулей, программный код или элементы разработки которых используются в текущем модуле. Например, добавьте зависимость, если в своем модуле вы:

- использовали публичную функцию модуля «Документооборот»;
- создали свойство с типом Ссылка и в поле ***Тип сущности** выбрали тип сущности «Сотрудники» из модуля «Компания».

Рекомендуется добавлять модуль в список зависимостей сразу после использования его элементов в своем модуле. Если зависимости не указаны, то при публикации возникнут ошибки.

Например, если в программном коде вашего модуля вы использовали справочник модуля «Контрагенты», добавьте модуль *базового решения* в список зависимостей. Для этого:

1. В выпадающем списке **Модуль** выберите модуль «Контрагенты» (Sungero.Parties). В поле **Мин. версия** автоматически подставится текущая версия модуля. Рекомендуется оставлять значение по умолчанию.
2. Нажмите на кнопку . В табличной части «Зависимости» отобразится добавленная зависимость.

   Если зависимость добавлена по ошибке, выделите ее в табличной части и нажмите на кнопку .

Не рекомендуется удалять зависимости из списка, даже если элементы модуля больше не используются.

СОВЕТ. После добавления зависимости запустите публикацию с выбором и проверьте, что для решений, модули которых указаны в качестве зависимых, установились синие положения ползунков. Это необходимо для корректной публикации решения.

События

Событие инициализации выполняется на веб-сервере. Добавьте обработчик события **Инициализация**, если для работы модуля нужно:

- заполнить справочники значениями по умолчанию или создать сущности, для которых в параметрах типа сущности установлен флажок **Создавать только программно**;
- выдать необходимым пользователям права доступа по умолчанию на вычисляемые папки модуля или типы сущностей;
- создать предопределенные группы пользователей и ролей.

Каждое действие рекомендуется писать в отдельной функции инициализации, после чего вызывать данную функцию в событии.

Если в редакторе модуля установлен флажок **Отображать в проводнике**, то можно настроить отображение модуля на панели навигации. Для этого установите флажок **Ограничение видимости**. При этом генерируется программный код, который включает отображение модуля для всех пользователей. Доработайте код, если нужно задать условия видимости. Например, модуль «Компания» можно сделать видимым для текущего пользователя, если он входит в роль «Ответственные за настройку оргструктуры» или замещает пользователя, входящего в эту роль.

Пример кода:

```
   public override bool IsModuleVisible()
{
    return Users.Current.IncludedIn(Constants.Module.DepartmentManagersRole) ||
    Substitutions.ActiveSubstitutedUsers.Any(substitutedUser =>
substitutedUser.IncludedIn(Constants.Module.DepartmentManagersRole));
}
```

Чтобы удалить логику событий **Инициализация** или **Отображать в проводнике**, достаточно снять соответствующий флажок. При этом удалится код события, но функции останутся. При необходимости удалите функции вручную.

Функции

Добавление клиентских, серверных, разделяемых функций модуля и функций инициализации.

Структуры и константы

Добавление структур и констант, которые в последующем можно использовать в функциях и обработчиках событий.

Обложка

Обложка содержит общую информацию о модуле, часто используемые действия и их описание. Отображается при выборе модуля в проводнике системы.

ВАЖНО. Обложки рекомендуется настраивать в веб-клиенте. Если ранее вы создавали или изменяли обложки в среде разработки, перенесите их в веб-клиент. Подробнее см. инструкцию по обновлению среды разработки, раздел «Конвертация обложек модулей». Редактирование обложек в среде разработки оставлено для совместимости.

Создание обложки в веб-клиенте

Обложки новых модулей рекомендуется настраивать в веб-клиенте. Для этого:

1. Из среды разработки запустите веб-клиент *локальной системы*.
2. В веб-клиенте создайте запись справочника **Представления модулей** и настройте обложку. Подробнее см. пример настройки модуля и стартовой страницы.

   ПРИМЕЧАНИЕ. При добавлении действий на обложку доступны типы сущностей, отчеты, функции и вычисляемые папки всех модулей системы.

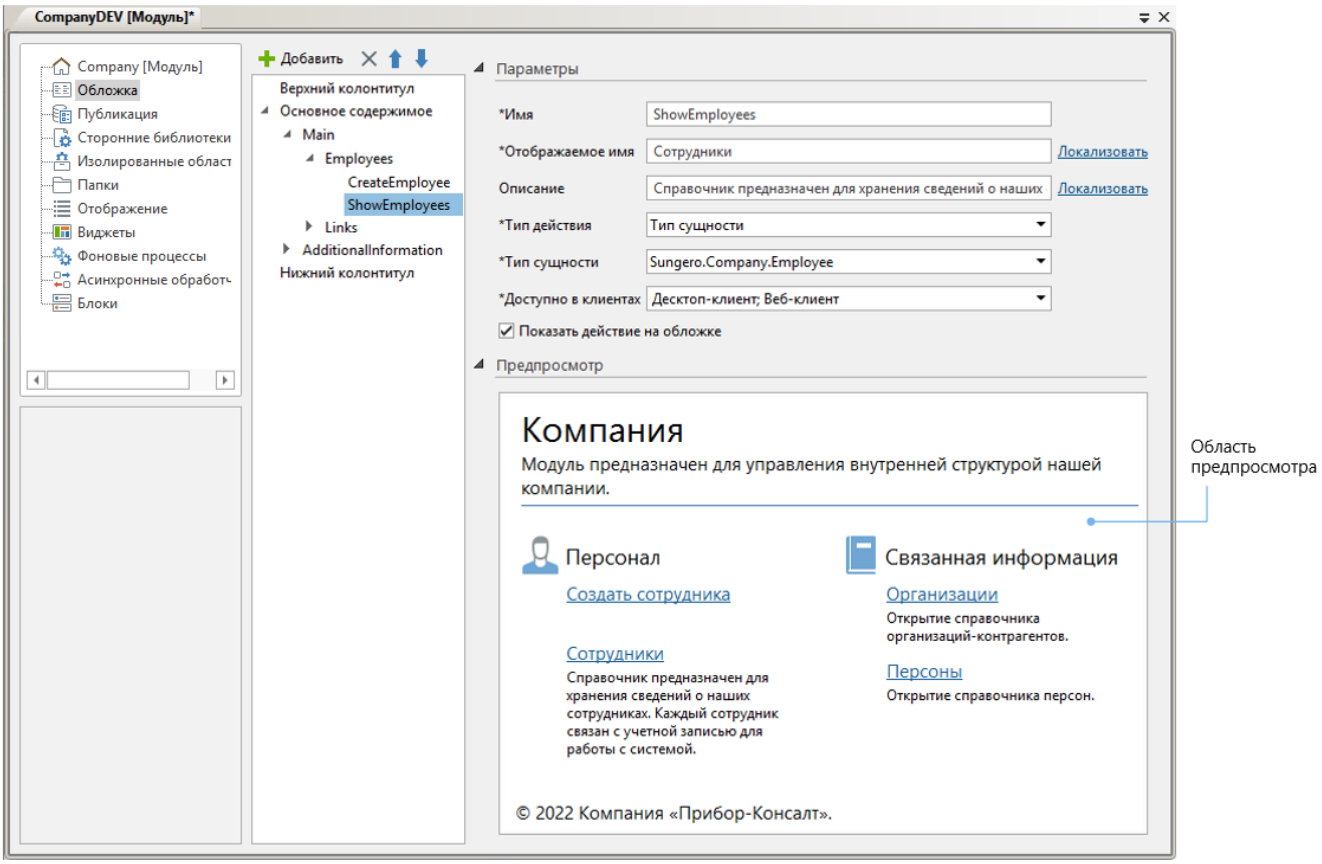
3. Представление модуля сохраните как настройку по умолчанию. Для этого в карточке представления перейдите на вкладку «Хранение» и установите флажок **Настройка по умолчанию**. В поле **Сохранить в модуле системы** выберите модуль, к которому относится настройка. Подробнее см. в руководстве администратора, раздел «Жизненный цикл настроек бизнес-процессов».
4. Сохраните карточку представления и обновите страницу браузера. В результате в веб-клиенте локальной системы на панели навигации появляется созданный модуль, при переходе на него отображается настроенная обложка.
5. Когда обложка настроена и готова к работе, создайте пакет разработки и опубликуйте его в *продуктивной системе*.

Редактирование обложки в среде разработки (устар.)

ПРИМЕЧАНИЕ. Узел «Обложка» отображается, если в редакторе модуля установлен флажок **Отображать в проводнике**. При создании новых модулей флажок снят, так как их обложки настраиваются в веб-клиенте.

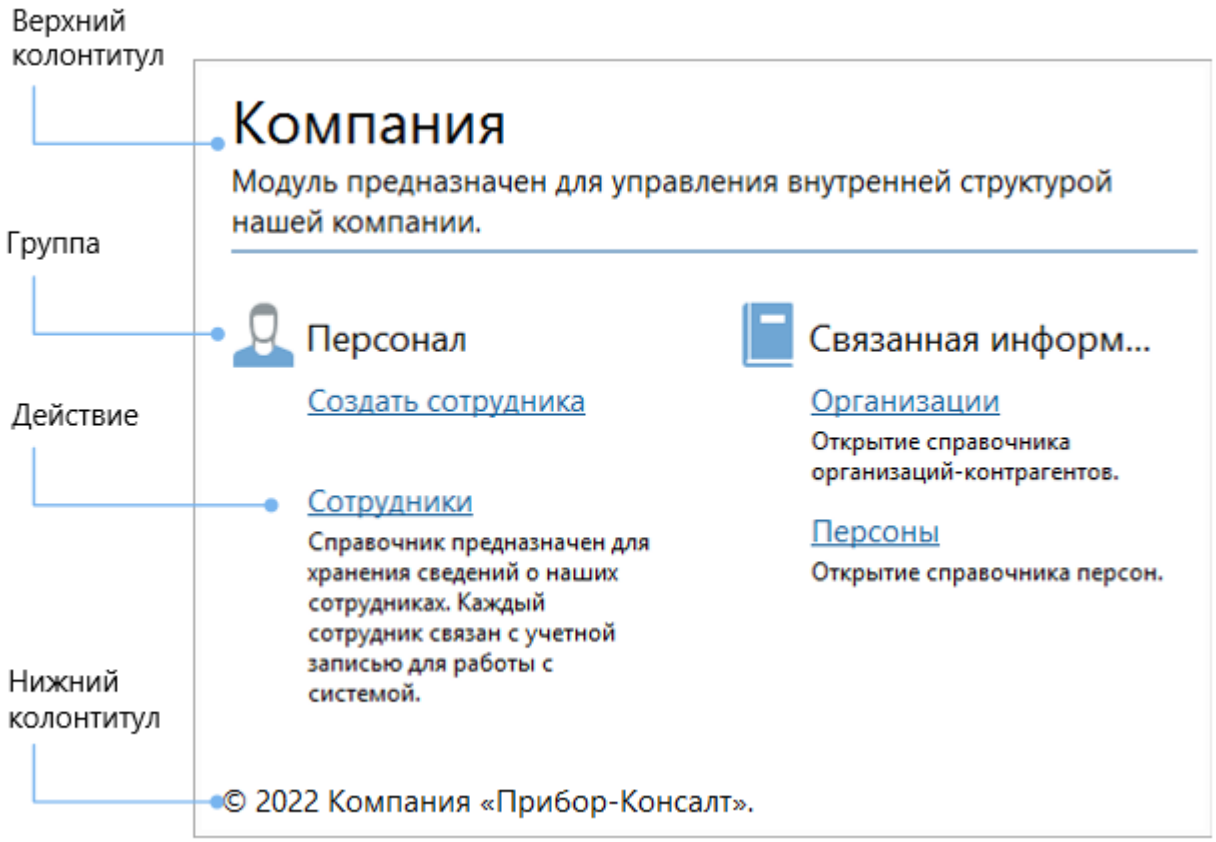
Перед редактированием обложки проверьте, что:

1. В узле «Отображение» для модуля настроен хотя бы один отображаемый элемент, иначе модуль не будет отображаться на панели навигации.
2. У пользователей есть права на создание и чтение отображаемых элементов модуля. Права можно выдать, например, с помощью функций инициализации.



Обложка модуля имеет предопределенную структуру и состоит из элементов:

- верхний и нижний колонтитулы. В верхнем колонтитуле содержится заголовок модуля, его краткое описание и назначение. В нижнем колонтитуле содержится дополнительная информация;
- вкладки, группы и действия. Часто используемые действия в модуле рекомендуется добавлять на обложку модуля. Для каждого действия указывается тип: функция, тип сущности, вычисляемая папка или отчет. Действия объединяются в группы. Группы располагаются на вкладках. В предпросмотре отображается редактируемая вкладка.

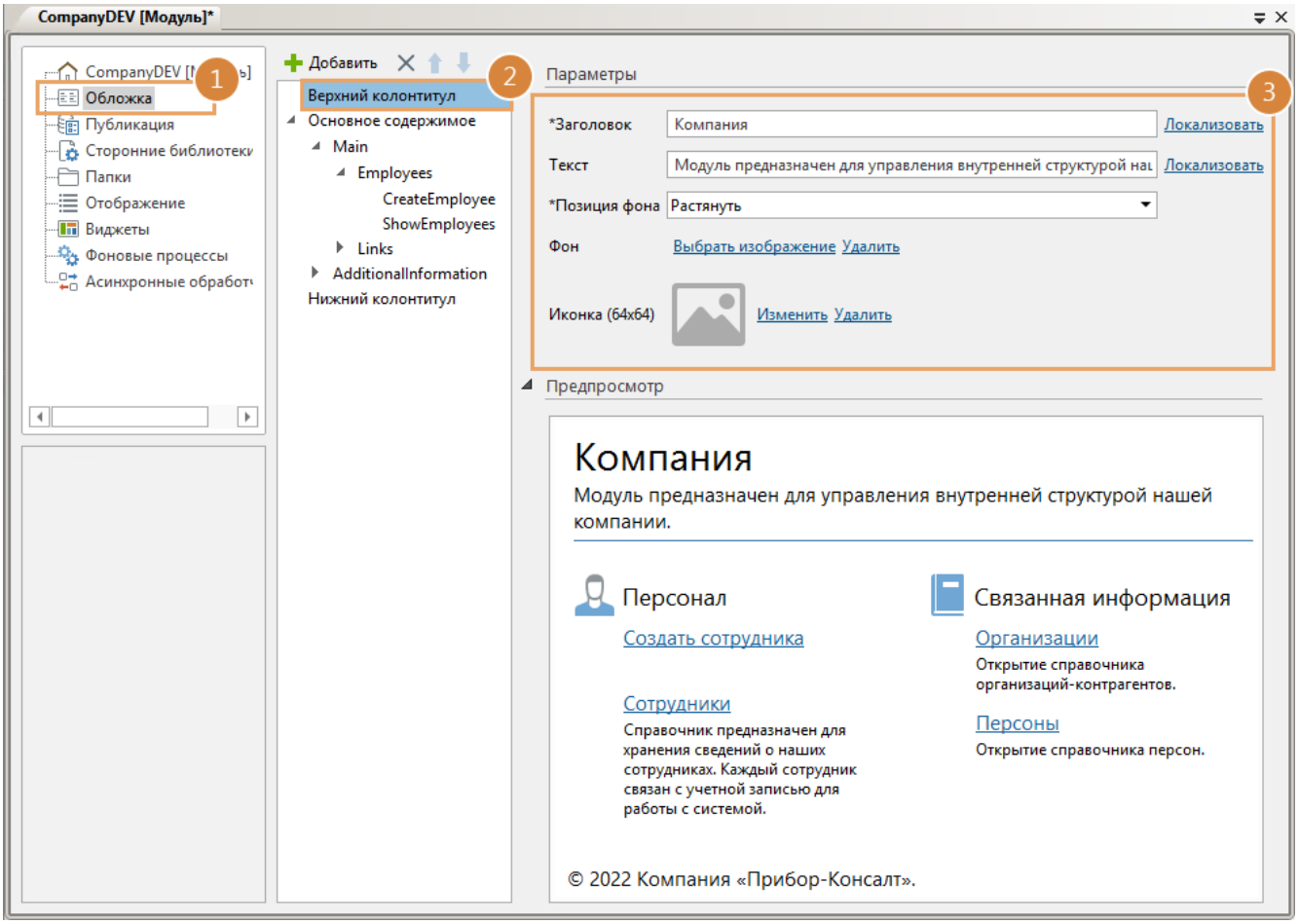


При настройке обложки используйте рекомендации. Подробнее см. в руководстве администратора, раздел «Рекомендации по оформлению обложки».

Колонтитулы и фон

Чтобы настроить верхний и нижний колонтитулы и задать фон основного содержимого обложки:

1. В редакторе модуля перейдите в узел «Обложка».



2. В списке элементов выберите **Верхний колонтитул**.
3. Задайте настройки:

   ***Заголовок**. Имя модуля.

   Текст. Краткое описание и назначение модуля.

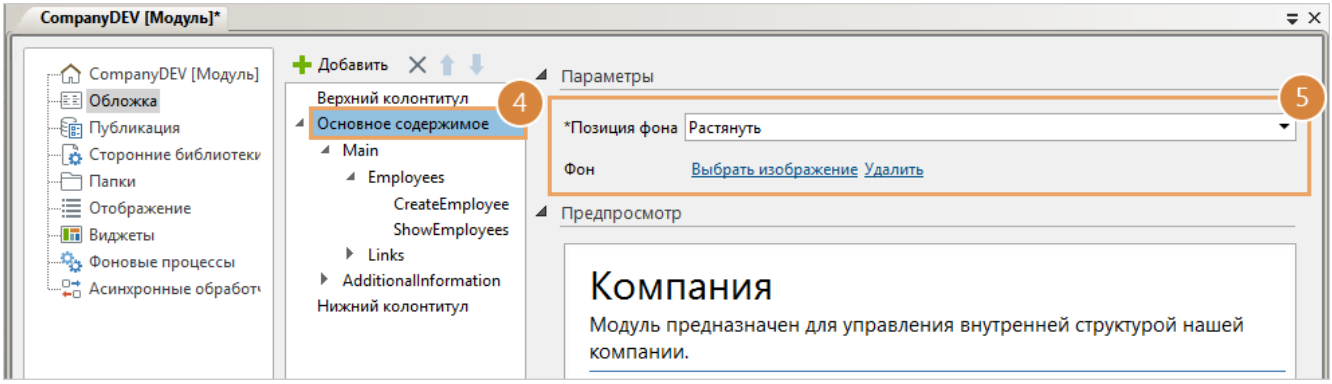
   Для *локализации* заголовка и описания модуля перейдите по ссылке **Локализовать**.

   ***Позиция фона**. Способ размещения фона в области заголовка. Возможные значения: **По центру**, **Растянуть**, **Замостить**.

   Фон. Фоновое изображение. Чтобы установить фон, перейдите по ссылке **Выбрать изображение**.

   Иконка. Будет отображаться слева от заголовка и описания модуля. Иконка должна быть в формате SVG и в размере 64x64 пикселя. Иконки, используемые в *базовом решении*, можно найти в локальной папке исходных кодов. Расположение папки задается при установке системы. Например, в папке C:\DEVRX\Base\Sungero.Company\Sungero.Company.ClientBase\Icons можно найти файлы иконок модуля «Компания».

4. В списке элементов выберите **Основное содержимое**.



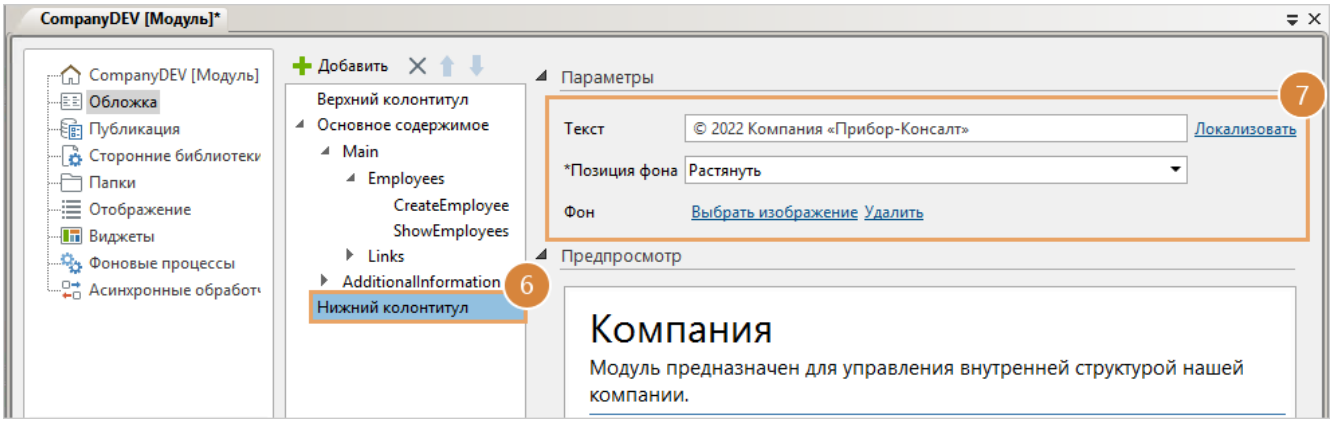
5. Задайте настройки:

   ***Позиция фона**. Способ размещения фона в основном содержимом. Возможные значения: **По центру**, **Растянуть**, **Замостить**.

   Фон. Фоновое изображение. Чтобы установить фон, перейдите по ссылке **Выбрать изображение**.

   ПРИМЕЧАНИЕ. Выбранный фон отобразится на всех вкладках.

6. В списке элементов выберите **Нижний колонтитул**.



7. Задайте настройки:

   Текст, который будет отображаться в нижнем колонтитуле. Может содержать, например, информацию о компании, в которой разработан модуль.

   ***Позиция фона**. Способ размещения фона в нижнем колонтитуле. Возможные значения: **По центру**, **Растянуть**, **Замостить**.

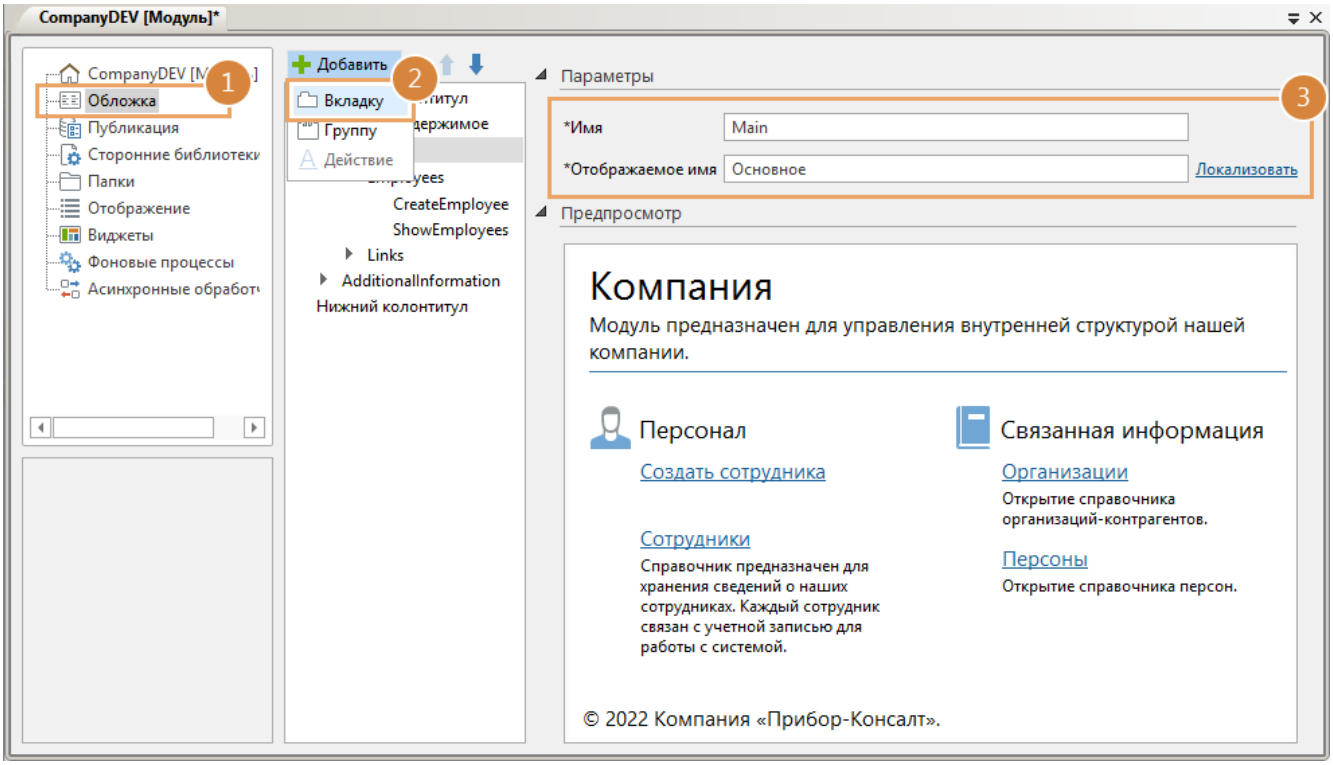
   Фон. Фоновое изображение. Чтобы установить фон, перейдите по ссылке **Выбрать изображение**.

Результат настройки можно посмотреть в области предпросмотра обложки.

Вкладки, группы и действия

Чтобы добавить вкладки, группы и действия на обложку модуля:

1. В редакторе модуля перейдите в узел «Обложка».

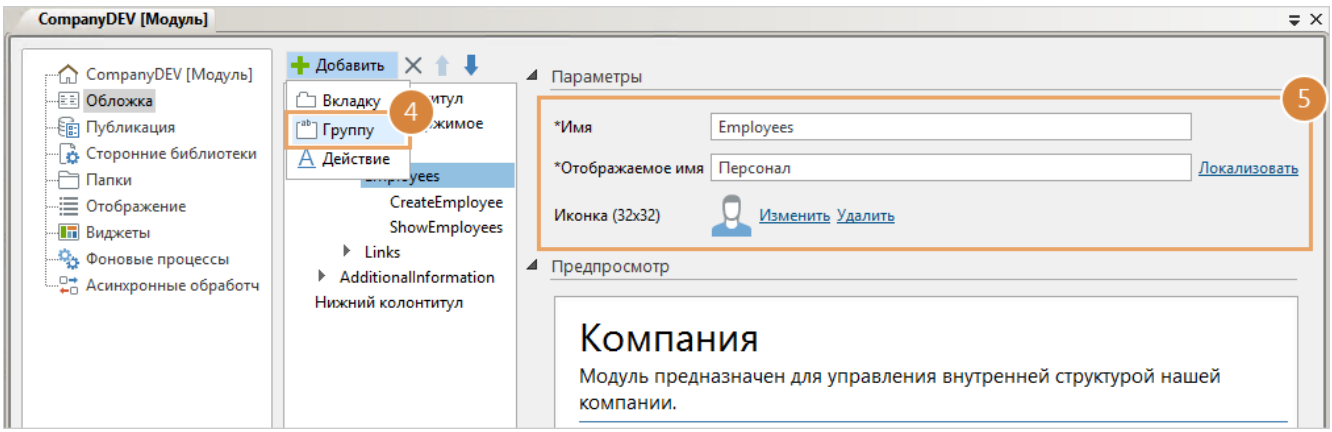


2. Добавьте вкладку. Для этого на панели инструментов в выпадающем списке **Добавить** выберите пункт **Вкладку**.
3. Задайте настройки:

   ***Имя** вкладки. Должно быть уникальным в рамках модуля, состоять из букв латинского алфавита и цифр.

   ***Отображаемое имя**. Название вкладки, которое будет отображаться на обложке. Для *локализации* отображаемого имени вкладки перейдите по ссылке **Локализовать**.
4. На вкладку добавьте группу. Для этого на панели инструментов в выпадающем списке **Добавить** выберите пункт **Группу**.

   Группы на вкладке размещаются в две колонки. Нечетные группы располагаются в левой колонке, четные – в правой.



5. Задайте настройки:

   ***Имя** группы. Должно быть уникальным в рамках одного модуля, состоять из букв латинского алфавита и цифр.

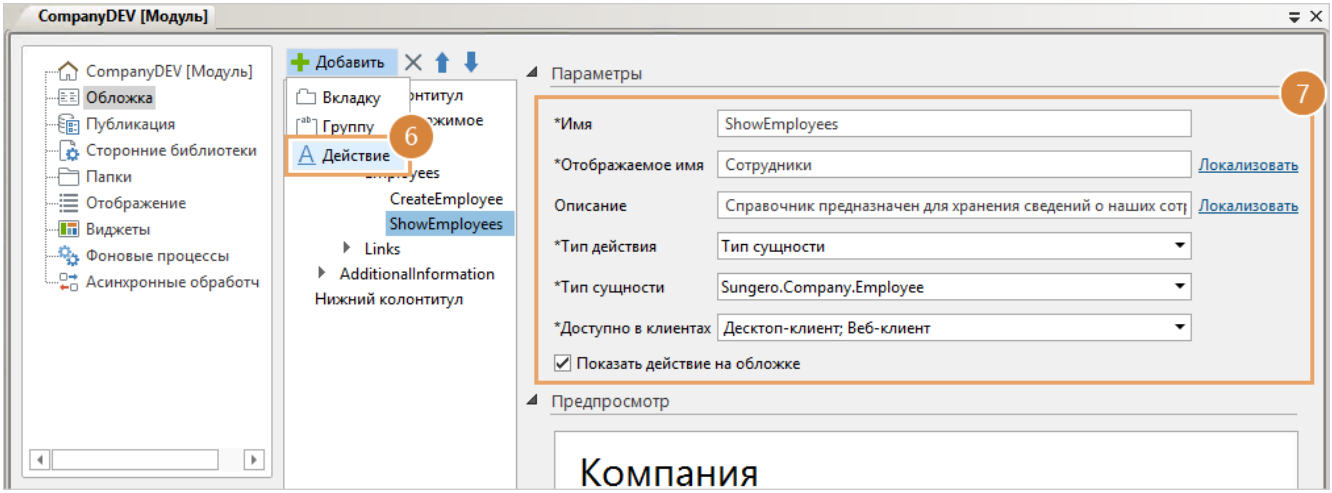
   ***Отображаемое имя**. Название группы, которое будет отображаться на обложке. Для локализации имени группы перейдите по ссылке **Локализовать**.

   Иконка. Будет отображаться слева от наименования группы. Иконка должна быть в формате SVG и в размере 32x32 пикселя. Иконки, используемые в *базовом решении*, можно найти в локальной папке исходных кодов. Расположение папки задается при установке системы. Например, в папке C:\DEVRX\Base\Sungero.Company\Sungero.Company.ClientBase\Icons можно найти файлы иконок модуля «Компания».

   ПРИМЕЧАНИЕ. Группа отображается на обложке модуля только после того, как в нее добавлены действия.

6. В группу добавьте действия. Для этого на панели инструментов в выпадающем списке **Добавить** выберите пункт **Действие**.

   Действия в группе размещаются в том порядке, в котором они следуют в списке элементов.



7. Задайте настройки:

   ***Имя** действия. Должно быть уникальным в рамках модуля, состоять из букв латинского алфавита и цифр.

   ***Отображаемое имя**. Название действия, которое будет отображаться на обложке.

   Описание. Краткое описание действия.

   Для локализации отображаемого имени действия и описания перейдите по ссылке **Локализовать**.

   ***Тип действия**. Способ выполнения действия. Возможные значения:

   - **Тип сущности**. Будет открываться список сущностей. Для этого в выпадающем списке ***Тип сущности** выберите необходимый тип;
   - **Функция**. Будет выполняться действие, которое задано в функции. Для этого в выпадающем списке ***Функции** выберите необходимую функцию. Если в списке нет нужной функции, создайте новую. Для этого нажмите на кнопку **Добавить функцию**. Для выполнения действий на обложке модуля используются только клиентские функции с модификатором virtual;

- **Вычисляемая папка**. Будет открываться вычисляемая папка. Для этого в выпадающем списке ***Вычисляемая папка** выберите необходимую папку;
- **Отчет**. Будет формироваться отчет текущего модуля в формате, заданном по умолчанию. Для этого в выпадающем списке ***Отчет** выберите необходимый отчет.

ВАЖНО. Поля ***Тип сущности**, ***Функции**, ***Вычисляемая папка** и ***Отчет** отображаются, если в выпадающем списке ***Тип действия** выбрано соответствующее значение.

***Доступно в клиентах**. По умолчанию добавленное действие доступно в десктоп-клиенте и в веб-клиенте. Если действие не поддерживается в одном из клиентов, то в выпадающем списке снимите флажок.

Показать действие на обложке. Добавленное действие отображается на обложке модуля. Снимите флажок, если необходимо скрыть действие с обложки.

8. Добавьте нужное количество вкладок, групп и действий. Чтобы изменить расположение, в списке элементов выберите вкладку, группу или действие и переместите при помощи кнопок ⬆ и ⬇ на панели инструментов.

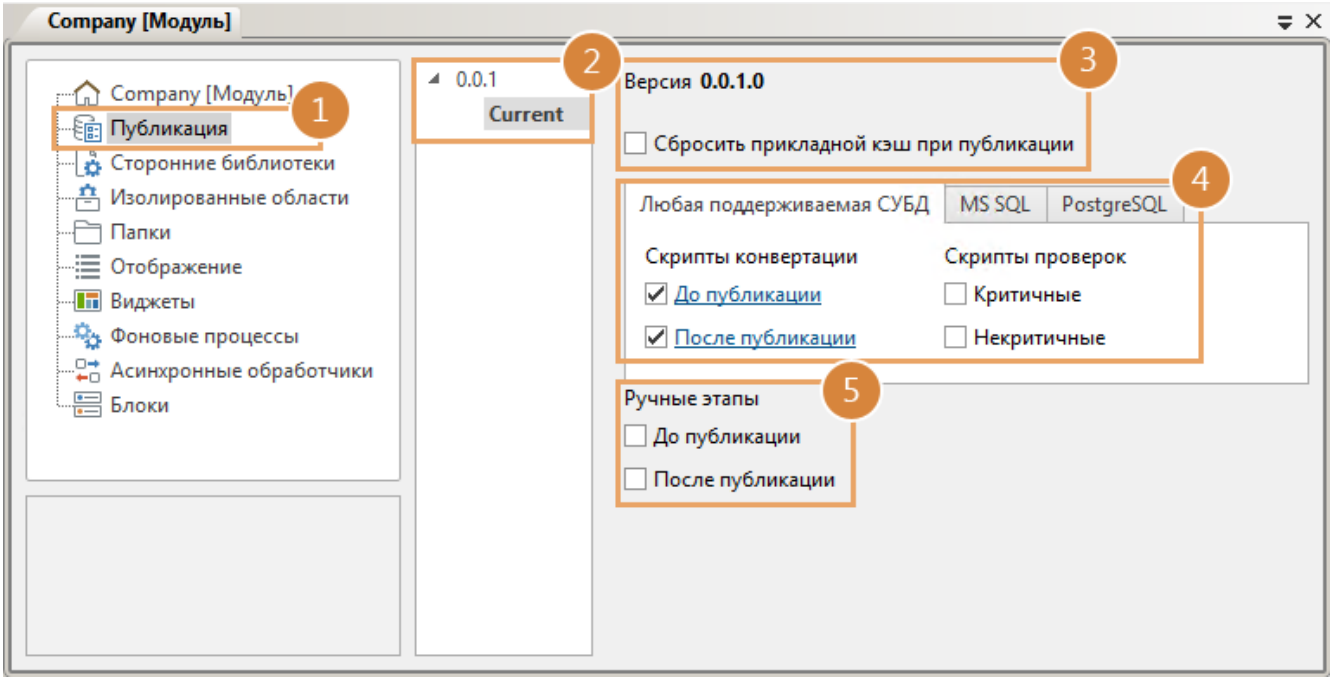
Результат настройки можно посмотреть в области предпросмотра обложки.

Публикация

Публикация представляет собой обновление разработки на веб-сервере. Настройки публикации задаются в редакторе модуля. Включают добавление скриптов конвертации, проверок и ручных этапов, которые нужно выполнить до или после публикации. Если изменился перекрытый модуль, то задайте настройки в редакторе решения. Если изменения произошли в разработке, которую сделали вы сами, задайте настройки в редакторе соответствующего модуля.

Чтобы задать настройки публикации:

1. В редакторе модуля перейдите в узел «Публикация».



2. В узле **Current**, который соответствует текущей версии, укажите настройки. Если нужно логически отделить скрипт конвертации по версии, увеличьте номер версии решения и после этого добавьте необходимый скрипт.

   При необходимости можно посмотреть скрипты предыдущих версий. Для этого раскройте узел версии, выше текущей.

3. При необходимости установите флажок **Сбросить прикладной кэш при публикации**, если в программном коде изменилась работа с кэшем. Например, изменилось вычисление для виджетов модуля.

4. Добавьте скрипты. Для этого выберите вкладку:

   - «Любая поддерживаемая СУБД» – если синтаксис запросов подходит для Microsoft SQL Server и СУБД на основе PostgreSQL;
   - «MS SQL» и «PostgreSQL» – если синтаксис запросов отличается. Добавьте на каждой вкладке свой скрипт запроса, в дальнейшем система сама определит нужный скрипт в зависимости от того, какая СУБД используется.

   На вкладках рядом с нужным названием скрипта установите флажок. Станет доступна ссылка, перейдите по ней. Откроется редактор SQL-запросов. В редакторе напишите код SQL-запроса.

   Чтобы удалить скрипт, снимите флажок.

5. Если у вас настроена синхронизация с другими базами данных, добавьте ручные этапы. В тексте запроса укажите скрипт для конвертации другой базы данных.

6. Сохраните изменения.

Скрипты конвертации, проверок и ручных этапов

В среде разработки Directum RX используются скрипты конвертации, проверок и ручных этапов. Скрипты добавляются на вкладках модуля или решения в узле «Публикация».

Скрипты конвертации и проверок могут быть общими для всех СУБД и специфичными для конкретной поддерживаемой СУБД. Если есть СУБД-зависимый скрипт, и он соответствует текущей СУБД, то при публикации он будет выполнен. Иначе выполнится скрипт, общий для всех СУБД, если он есть. Имена файлов отличаются постфиксами. Например, скрипты конвертации могут называться так:

- convert_before.sql – общий скрипт для всех СУБД;
- convert_before_**mssql**.sql – скрипт для Microsoft SQL Server;
- convert_before_**postgres**.sql – скрипт для СУБД на основе PostgreSQL.

Скрипты конвертации

Скрипты конвертации делятся по времени выполнения: **до публикации** (convert_before.sql) и **после публикации** (convert_after.sql). Они предназначены для настройки конвертации данных модуля или решения с одной версии на другую.

Напишите скрипты конвертации, если нужно:

- в типе сущности заполнить свойство значением. Например, когда при добавлении нового логического свойства в справочнике с миллионом записей необходимо в половине записей указать значение **True** или **False** по определенному алгоритму;
- изменить тип свойства с сохранением данных, например, со строки на перечисление. Если данные сохранять не требуется, скрипт конвертации можно не писать;
- добавить новое обязательное свойство любого типа в тип сущности;
- изменить обязательность свойства для типа сущности;
- для свойства-ссылки изменить тип сущности, на который он ссылается. При этом новый тип является наследником старого абстрактного или другим типом сущности.

Подробнее см. примеры скриптов конвертации: появилось свойство логического типа, свойство стало обязательным, изменен тип свойства.

Добавлять скрипты конвертации не требуется, если:

- добавлен, переименован или удален тип сущности;
- увеличена длина строкового свойства;
- изменен тип сущности, на который ссылается свойство-ссылка, при этом новый тип меняется на предка. Пример: Contract изменился на ContractBase;
- изменен тип свойства с **Целое** на **Идентификатор**, с **Целое** на **Вещественное**, со **Строка** на **Текст**. Данные перечисленных типов конвертируются автоматически. Если используется СУБД Microsoft SQL Server и для колонки был создан индекс, то его нужно восстанавливать вручную;
- добавлено необязательное свойство или свойство удалено.

Рекомендуется писать многопроходные скрипты конвертации. Многопроходность означает, что скрипт может быть запущен множество раз, но конечный результат будет таким, как и при одном прогоне. Если скрипт не многопроходный, то при его повторном выполнении могут появиться ошибки и тогда *публикация* не выполнится.

Многопроходность достигается за счет фильтрации записей. Подробнее см. пример скрипта конвертации «Появилось свойство логического типа».

ПРИМЕЧАНИЕ. При необходимости можно логически отделить один скрипт конвертации от другого или разделить структурные изменения данных (добавление, переименование, удаление свойств). В этом случае предварительно увеличьте последнюю цифру в номере версии решения вручную.

Скрипты проверок

Предназначены для проверки того, что данные модуля или решения можно конвертировать.

Среди скриптов проверок выделяют:

- **Критичные** (critical_check.sql). Выполняются перед публикацией модуля. Если при проверке возникают ошибки, то публикация прерывается;
- **Некритичные** (noncritical_check.sql). Выполняются перед публикацией модуля, после критичных проверок. Если при проверке возникают ошибки, то выводится соответствующее предупреждение. При этом публикация продолжается.

В одном файле скриптов с проверками можно разместить несколько отдельных проверок, при этом каждая проверка должна быть записана в следующем виде:

```
/*#script#
Заголовок проверки, который появится при возникновении ошибки.*/

select * from Recipient
```

Скрипты ручных этапов

Предназначены для поддержки совместимости между разными базами данных. Например, в разработке для типа сущности добавили новое необязательное свойство и есть функция, которая синхронизирует его значение между базами данных. В базе данных Directum RX свойство появится после публикации, а в базе данных сторонней системы требуется создать свойство вручную. Для этого нужно написать соответствующий скрипт ручного этапа по созданию этого свойства в сторонней базе данных.

Скрипты ручных этапов делятся по времени выполнения – **До публикации** и **После публикации**, где:

- manual_before.txt – файл с описанием ручного этапа, выполняемого до публикации версии модуля;
- manual_after.txt – файл с описанием ручного этапа, выполняемого после публикации версии модуля.

Порядок выполнения скриптов

При публикации скрипты выполняются последовательно в следующем порядке:

При публикации решения в нужной последовательности выполняются все скрипты публикации. При повторной публикации модуля выполняются только скрипты текущей версии (**Current**).

Пример скрипта конвертации. Появилось свойство логического типа

Предположим, в справочник «Персональные настройки» (PersonalSetting) добавлено новое свойство **Показывать подписи с результатом «Не согласовано»**. Имя свойства в базе данных – ShowNotApprove.

Конвертация необходима, чтобы:

- быстро и массово заполнить свойство значением по умолчанию. При необходимости пользователи могут изменить значение вручную;
- система работала стабильно и не требовала от пользователя дополнительных действий.

Добавьте скрипт **После публикации (MS SQL)** в модуль «Документооборот» (Docflow):

```
update
  Sungero_Docflow_PersonSetting
set
  ShowNotApprove = 0
where
  ShowNotApprove is null
```

Многопроходность достигается за счет фильтрации записей, в которых поле ShowNotApprove уже заполнено.

Пример скрипта конвертации. Появилось новое обязательное свойство

Предположим, в справочнике «Право подписи» (SignatureSetting) появилось новое обязательное свойство **Документопоток**. Имя свойства в базе данных – DocumentFlow.

Конвертация необходима, чтобы:

- быстро и массово заполнить свойство **Документопоток** значением, которое вычисляется по алгоритму;
- оптимизировать работу фоновых процессов, бизнес-процессов, интеграцию с сервисами обмена, механизм синхронизации с 1С.

Добавьте скрипт **После публикации (MS SQL)** в модуль «Документооборот» (Docflow):

```
  update
  signSettings
set
  signSettings.DocumentFlow =
    case
      when
        signSettings.Discriminator = '99B2BA4B-8861-42CB-8BF6-3040E9995C11'
      then
        'Contracts'
      when
        (select
```

```
            count(*)
          from
            (select
               docKinds.DocumentFlow
             from
               Sungero_Docflow_SignKinds as signKinds,
               Sungero_Docflow_DocumentKind as docKinds
             where
               signKinds.SignSettings = signSettings.Id and signKinds.DocumentKind
= docKinds.Id
             group by
               docKinds.DocumentFlow) as t) = 1
      then
        (select
           docKinds.DocumentFlow
         from
           Sungero_Docflow_SignKinds as signKinds,
           Sungero_Docflow_DocumentKind as docKinds
         where
           signKinds.SignSettings = signSettings.Id and signKinds.DocumentKind =
docKinds.Id
         group by
           docKinds.DocumentFlow)
      else
        'All'
    end
from
  Sungero_Docflow_SignSettings signSettings
where
  signSettings.DocumentFlow is null
```

Пример скрипта конвертации. Свойство стало обязательным

Предположим, в справочнике «Право подписи» (SignatureSetting) свойство **Приоритет** стало обязательным. Имя свойства в базе данных – Priority.

Конвертация необходима, чтобы:

- быстро и массово заполнить свойство **Приоритет** значением по умолчанию;
- оптимизировать работу фоновых процессов, бизнес-процессов, интеграцию с сервисами обмена, механизм синхронизации с 1С.

Добавьте скрипт **После публикации (MS SQL)** в модуль «Документооборот» (Docflow):

```
update

  Sungero_Docflow_SignSettings

set

  Priority = '0'

from

  Sungero_Docflow_SignSettings

where

  Priority is null
```

Пример скрипта конвертации. Изменился тип данных свойства

Предположим, в справочнике «Право подписи» (SignatureSetting) у свойства **Основание** меняется тип со строки на перечисление. Имя свойства в базе данных – Reason.

В процессе публикации модуля будет удалено свойство типа **Строка** и создастся свойство типа **Перечисление**.

Конвертация необходима, чтобы перенести данные сначала из старого поля с типом **Строка** во временную таблицу (скрипт до публикации), а потом из временной таблицы в новое поле с типом **Перечисление** (скрипт после публикации). Таким образом исключается потеря данных, и они преобразуются по заданному алгоритму.

Добавьте скрипт **До публикации (MS SQL)** в модуль «Документооборот» (Docflow):

```
   if exists (select * from information_schema.tables where table_name =
      'SignatureSettingsReason')

  drop table SignatureSettingsReason

if exists (select * from information_schema.COLUMNS where table_name =
'Sungero_Docflow_SignSettings' and COLUMN_NAME = 'Reason')

   and not exists (select * from information_schema.COLUMNS where table_name =
'Sungero_Docflow_SignSettings' and COLUMN_NAME = 'DocumentInfo')

begin

  execute('create table SignatureSettingsReason(Id bigint, Reason nvarchar(250))

           insert into SignatureSettingsReason

           select s.Id, s.Reason from Sungero_Docflow_SignSettings s')

end
```

Добавьте скрипт **После публикации (MS SQL)** в модуль «Документооборот» (Docflow):

```
   if exists (select * from information_schema.tables where table_name =
      'SignatureSettingsReason')

    and exists (select * from information_schema.COLUMNS where table_name =
'Sungero_Docflow_SignSettings'

and COLUMN_NAME = 'DocumentInfo')

    and exists (select * from information_schema.COLUMNS where table_name =
'Sungero_Docflow_SignSettings'

 and COLUMN_NAME = 'Reason')

begin

  update

    s

  set

    Reason =

      case
```

```
        when lower(tmp.Reason) in ('устав', 'charter')
        then 'Duties'
        else 'Other'
      end,
    DocumentInfo =
      case
        when lower(tmp.Reason) in ('устав', 'charter')
        then ''
        else tmp.Reason
      end
  from SignatureSettingsReason tmp
    join Sungero_Docflow_SignSettings s on tmp.Id = s.Id
  drop table SignatureSettingsReason
end
```

Сторонние библиотеки

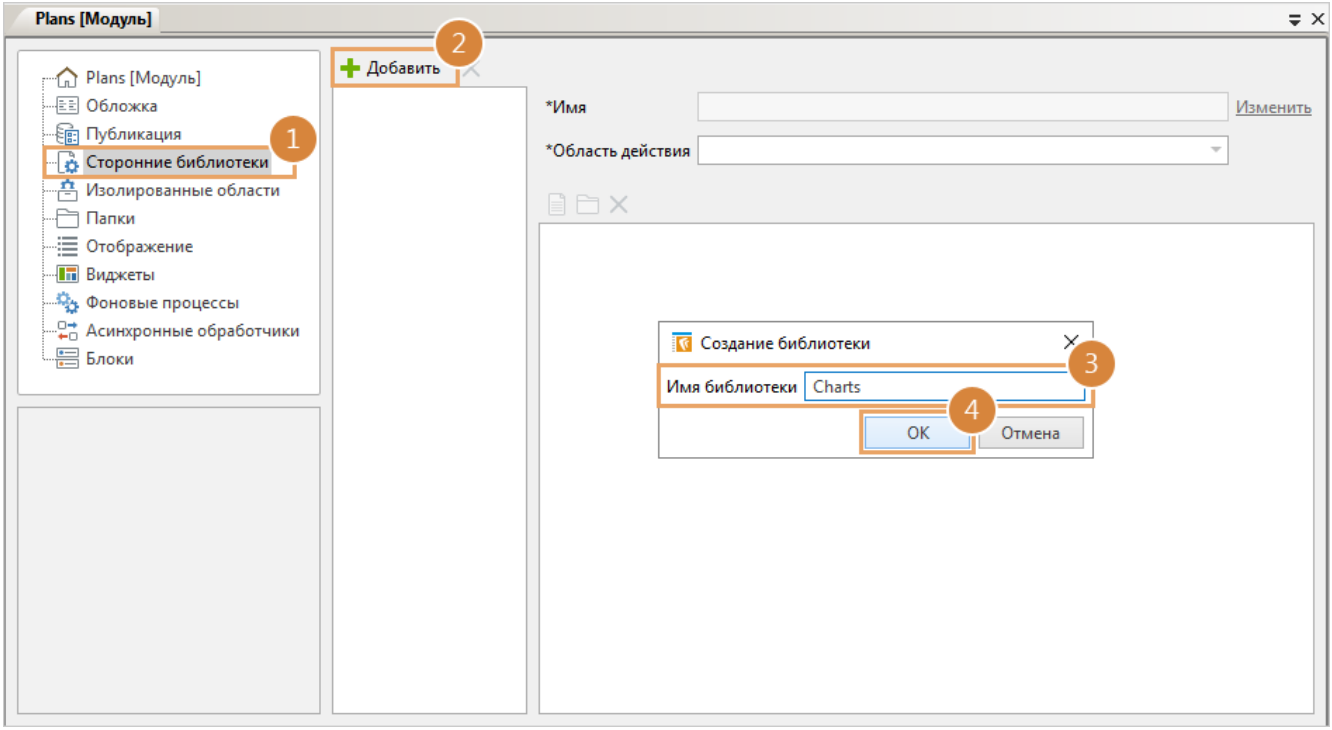
Для расширения возможностей системы доступно добавление *сторонних библиотек*. Вы можете реализовать необходимую функциональность в виде отдельной .NET-библиотеки и подключить ее к нужному решению или модулю. После добавления библиотеки *сторонние сборки*, которые входят в ее состав, можно использовать в программном коде.

Выполнение кода сторонней сборки можно перенести на сервис изолированного кода. Для этого нужно подключить стороннюю библиотеку к изолированной области. Тогда код сторонней сборки будет выполняться в отдельном процессе. Работа других сервисов продолжится, даже если в стороннем коде возникнут ошибки.

ВАЖНО. Сторонние библиотеки для прикладной разработки Directum RX должны быть собраны под .NET Standard 2.0.

Чтобы добавить стороннюю библиотеку:

1. В редакторе решения или модуля перейдите в узел «Сторонние библиотеки».



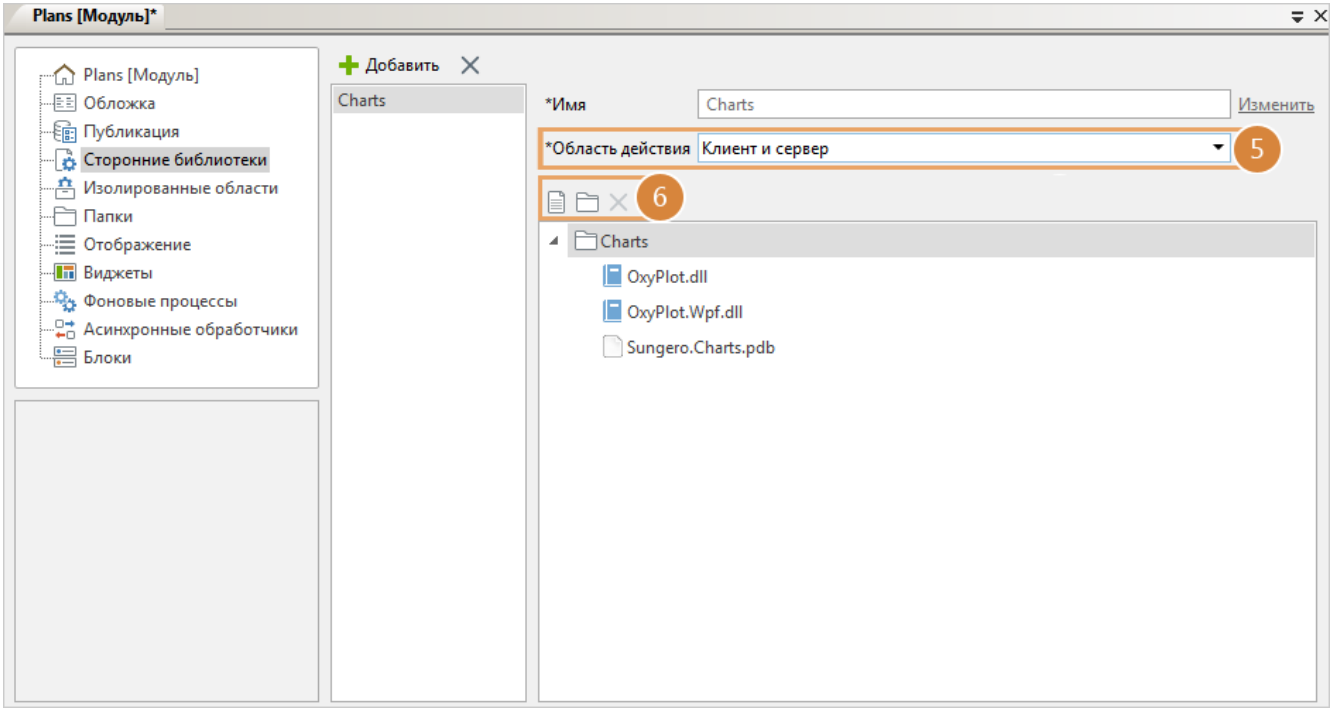
2. Нажмите на кнопку **Добавить**.
3. В открывшемся окне укажите имя библиотеки. Оно должно быть уникальным, состоять из букв латинского алфавита и цифр.
4. Нажмите на кнопку **ОК**.
5. Заполните поле ***Область действия**, по которому определяется, в каком программном коде будут доступны сборки библиотеки. Возможные значения:

   WPF Клиент. Настройка является устаревшей и оставлена для совместимости.

   Базовый клиент. Сборки доступны в клиентском коде, предназначенном для веб-клиента.

   Сервер и изолированные области. Сборки доступны в серверном коде, например в событиях сущности. Также сборки доступны в коде изолированной области.

Клиент и сервер. Сборки доступны в любом программном коде.



6. Добавьте содержимое библиотеки. Для этого воспользуйтесь кнопками:
   - для добавления содержимого всей папки, выбранной с локального диска;
   - для добавления определенного файла в выбранную папку библиотеки.

   ПРИМЕЧАНИЕ. Запрещается использование сторонних сборок размером более 15 МБ.

После загрузки в области предпросмотра отображаются все добавленные сборки и файлы библиотеки. Сборки, которые можно использовать в программном коде, отмечены значком . Остальные файлы библиотеки отмечены значком .

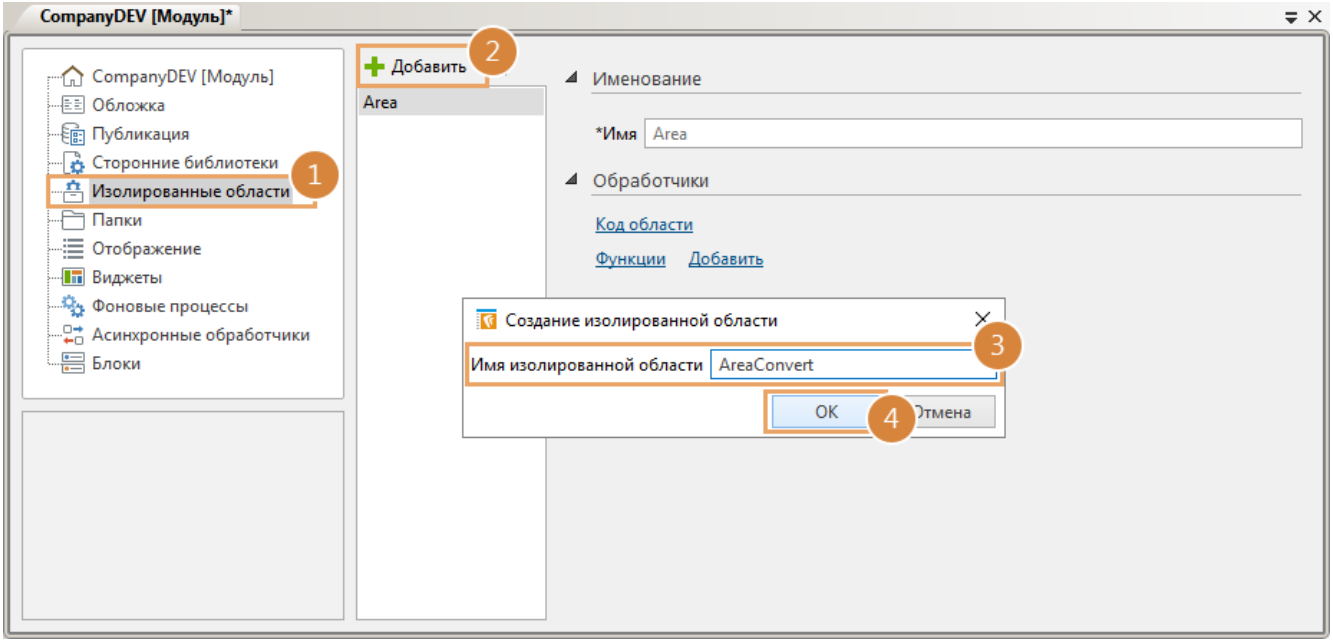
ВАЖНО. Сторонняя библиотека может использоваться только в рамках того решения или модуля, в который она добавлена. Чтобы использовать библиотеку в нескольких модулях, добавьте ее в каждый модуль и заполните одинаковым набором файлов и сборок одной версии.

Изолированные области

Чтобы выполнять код сторонней сборки в отдельном процессе независимо от других сервисов, его можно перенести на сервис изолированного кода (IsolatedArea). В этом случае работа других сервисов продолжится, даже если при выполнении изолированного кода возникнут ошибки. Также в изолированную область можно добавлять код, который содержит ресурсоемкие вычисления, например конвертация больших документов.

Чтобы добавить изолированную область:

1. В редакторе модуля перейдите в узел «Изолированные области».



2. Нажмите на кнопку **Добавить**.
3. В открывшемся окне укажите имя изолированной области. Оно должно быть уникальным, состоять из букв латинского алфавита и цифр.
4. Нажмите на кнопку **ОК**.
5. По ссылке Функции создайте функции, которые будут вызываться в изолированной области.
6. По ссылке Код области добавьте код, который нужно выполнять на сервисе изолированного кода.

При создании изолированной области учитывайте особенности и ограничения.

После сборки решения для каждого модуля создаётся сборка **<Имя модуля>.Isolated.dll**. При необходимости администратор может изменить настройки запуска сервиса изолированного кода.

Функции

В качестве аргументов и результатов выполнения функции используются простые типы, потоки данных базового типа Stream(System.IO.Stream), списки, словари и изолированные структуры модуля.

При создании функции можно использовать только изолированные структуры, добавленные в этом модуле. Чтобы структура стала доступной в изолированной области, перед ее описанием нужно добавить атрибут **Public** с параметром **Isolated** со значением **true**:

```
   [Public(Isolated=true)]
partial class DpadSignaturInfo
{
  public int SignIcon { get; set; }
  public string Status { get; set; }

}
```

Чтобы функция стала доступна в клиентском, серверном и разделяемом коде, перед ее описанием нужно добавить атрибут **Public**:

```
[Public]
public void Function()
{
  ...
}
```

Формат вызова функции:

Sungero.<Имя модуля>.IsolatedFunctions.<Имя области>.<Имя метода>(<список аргументов>)

Если в функции используются потоки данных, то учитывайте их особенности. В качестве возвращаемого значения и аргументов функции можно использовать только потоки данных базового типа Stream(System.IO.Stream). При этом в коде функции можно использовать любые потоки, например FileStream или MemoryStream. Передавать можно несколько потоков, а возвращать только один. Возвращенный поток из изолированной функции может быть прочитан один раз, повторная попытка чтения приведет к ошибке.

Пример функции с использованием потоков:

```
   public class IsolatedFunctions
{
  [Public]
  public Stream AddStamp(Stream pdfDocumentStream, string stamp)
  {
     var pdfStamper = new PdfConverter.PdfStamper();
     Stream result = pdfStamper.AddStamp(pdfDocumentStream, stamp);
     return result;
  }
}
```

Пример вызова функции в коде изолированной области:

```
   public class ModuleFunctions
{
  public virtual void LoadVersionFromFileAndAddStamp(IElectronicDocumentVersions
version, string path, string stamp)
  {
    using (FileStream fileStream = File.OpenRead(path))
    using (Stream documentWithStamp =
Sungero.StampModule.IsolatedFunctions.StampArea.AddStamp(fileStream, stamp))
    {
      version.Body.Write(documentWithStamp);
    }
  }
}
```

Код области

Код выполняется в изолированной области. В нем можно обращаться к функциям из сторонних сборок и функциям, созданным в изолированной области.

На код изолированной области не распространяются стандартные ограничения среды разработки. Например, вы можете добавить свои интерфейсы и их реализацию. Также в среде разработки недоступна отладка кода изолированных областей. Вы можете использовать стороннюю программу, например Microsoft Visual Studio.

Особенности изолированных областей

Модули с изолированными областями можно перекрывать. При этом в изолированной области перекрытого модуля доступны только его изолированные структуры или изолированные структуры предка.

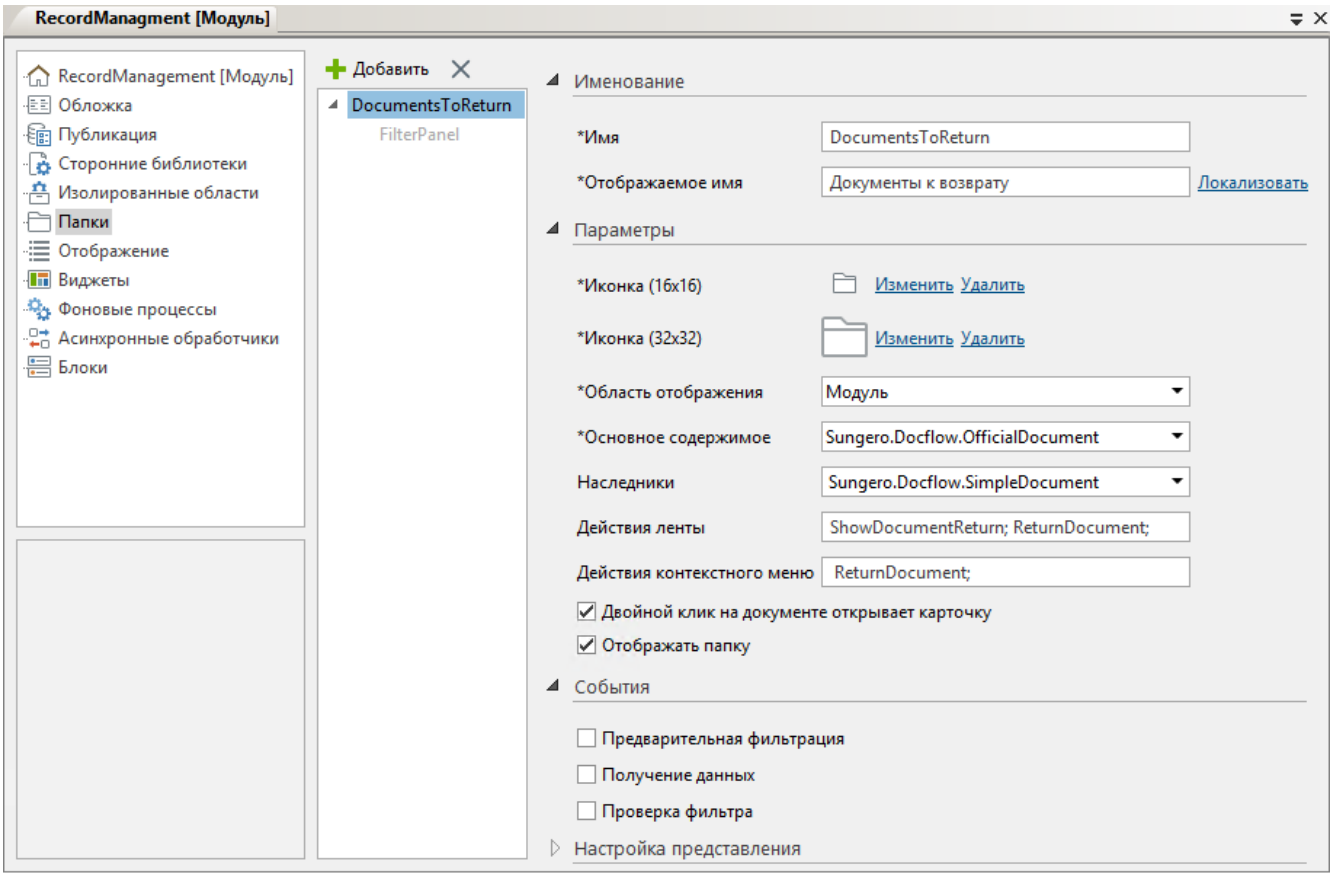
Для изолированных областей есть ограничения:

- в коде изолированной области нельзя работать с сущностями, репозиториями, базой данных. Данные передаются через параметры функций изолированной области;
- в перекрытых модулях нельзя добавлять новые изолированные области;
- при изменении кода области публикация разработки без перезапуска веб-сервера недоступна;
- сервис асинхронных событий (Worker), настроенный на работу под .NET Framework, не может работать с изолированными областями. При попытке вызвать функцию изолированной области возникает ошибка «Isolated area service is not supported». Чтобы вызывать такие функции, настройте выполнение фоновых процессов и асинхронных обработчиков на сервисе Worker, который запущен под .NET Core. Сервис асинхронных событий, настроенный под .NET Framework, рекомендуется использовать только для фоновых процессов и асинхронных обработчиков, которые не работают под .NET Core. Например, это может потребоваться, если подключены устаревшие сторонние библиотеки.

В изолированных областях можно использовать сторонние библиотеки, у которых указана область действия **Сервер и изолированные области**. По умолчанию доступны библиотеки Aspose, Newtonsoft.Json и другие, подключенные в стандартной разработке. Чтобы использовать их возможности, достаточно в программном коде добавить нужные пространства имен.

Папки

Узел «Папки» предназначен для создания и настройки папок, содержимое которых вычисляется только программно.



Вычисляемые папки могут находиться:

- **в модуле**. Например, в модуле «Делопроизводство» есть папка «Документы к возврату», в которой отображаются делопроизводственные документы, бумажные экземпляры которых выданы сотрудникам и подлежат возврату;
- **в папке «Входящие»**. Например, папка «На согласование», в которой отображаются только задания на согласование, полученные пользователем;
- **в папке «Исходящие»**. Например, папка «Поручения», в которой отображаются только поручения, отправленные пользователем, и поручения, которые отправлены от его имени, например, помощником.

Вы можете создавать свои вычисляемые папки, а также изменять папки в перекрытиях модулей базового решения. Используйте узел «FilterPanel», если для папки нужно настроить панель фильтрации.

Именование

***Имя**. Должно быть уникальным в рамках модуля, состоять из букв латинского алфавита и цифр.

***Отображаемое имя**. Имя папки, которое будет отображаться в проводнике системы. Для *локализации* имени перейдите по ссылке **Локализовать**.

Параметры

***Иконка**. Настройка является устаревшей и оставлена для совместимости.

***Область отображения** вычисляемой папки в проводнике системы. Возможные значения: **Модуль**, **Папка «Входящие»**, **Папка «Исходящие»**.

СОВЕТ. Если настраивается иерархический список, то в поле ***Область** отображения укажите значение **Модуль**.

***Основное содержимое**. *Тип сущности*, по которому для содержимого папки определяется состав видимых колонок, кнопки быстрого создания, действия на ленте и в контекстном меню. Основным содержимым могут быть:

- для папки модуля – любые типы сущностей;
- для папки, которая находится в папке «Входящие» – только типы заданий, заданий на приемку и уведомлений;
- для папки, которая находится в папке «Исходящие» – только типы задач.

Наследники. Список типов сущностей, которые являются наследниками типа сущности, указанного в поле ***Основное содержимое**. По указанным типам сущностей дополнится состав видимых колонок, кнопки быстрого создания, действия на ленте и в контекстном меню, которые будут доступны в содержимом папки.

Действия ленты и **Действия контекстного меню**. Список действий, которые скрыты на ленте или в контекстном меню основного содержимого или наследников, но в вычисляемой папке должны быть доступны. Действия перечисляются через «;». Чтобы добавить действие на ленту или в контекстное меню вычисляемой папки:

1. Откройте редактор типа сущности и перейдите в узел «Действия».
2. Скопируйте имя скрытого действия и добавьте через «;» в поле **Действия ленты** или **Действия контекстного меню**.

Двойной клик на документе открывает карточку. При открытии документа из папки двойным щелчком левой кнопкой мыши или с помощью клавиши ENTER будет открываться его карточка. Флажок отображается, если в поле ***Область отображения** указано значение **Модуль**, а в качестве основного содержимого тип документа.

Отображать папку в проводнике системы. Используется в перекрытиях модулей. Снимите флажок, чтобы скрыть папку в перекрытии.

Иерархический список. Установите флажок, чтобы содержимое папки отображалось в виде иерархии.

Отображать количество непрочитанных заданий. Справа от названия папки указывается количество непрочитанных заданий. Флажок отображается, если в поле ***Область отображения** указано значение **Папка «Входящие»**.

Использовать настройки представления от родительской папки. Внешний вид папки (состав видимых колонок, настройка группировки и т.д.) по умолчанию дублирует внешний вид той папки, в которой она находится. Если флажок установлен, то настройки внешнего вида в области предпросмотра станут доступны только для просмотра. Флажок отображается, если в поле ***Область отображения** указано значение **Папка «Входящие»** или **Папка «Исходящие»**.

События

Предварительная фильтрация. Обработчик события формирует SQL-запрос для фильтрации содержимого по основным критериям. Рекомендуется использовать при получении выборки данных из больших списков, так как событие позволяет значительно сократить список по основным критериям фильтрации. Затем с полученным списком отрабатывает событие **Получение данных**.

Получение данных. Обработчик события для вычисления содержимого папки. В коде обработчика пишут:

- что должно отображаться в папке. Например, только задания на подписание или документы, отправленные контрагентам;
- условия фильтрации содержимого папки, если настроена панель фильтрации.

Событие доступно, если заполнено поле ***Основное содержимое**.

Ограничение доступности. Обработчик события для определения состава пользователей, у которых папка по умолчанию отображается в проводнике системы. При этом остальные пользователи при необходимости смогут добавить папку в проводник. Например, создана папка потока для заданий на печать документов, с которой будут работать помощники руководителей. Добавьте обработчик, чтобы папка по умолчанию отображалась у них в проводнике. При желании помощники смогут скрыть папку, а другие пользователи добавить с помощью контекстного меню. Если обработчик не задан, то папка по умолчанию отображается в проводнике у всех пользователей. Событие доступно, если в поле ***Область отображения** указано значение **Папка «Входящие»** или **Папка «Исходящие»**.

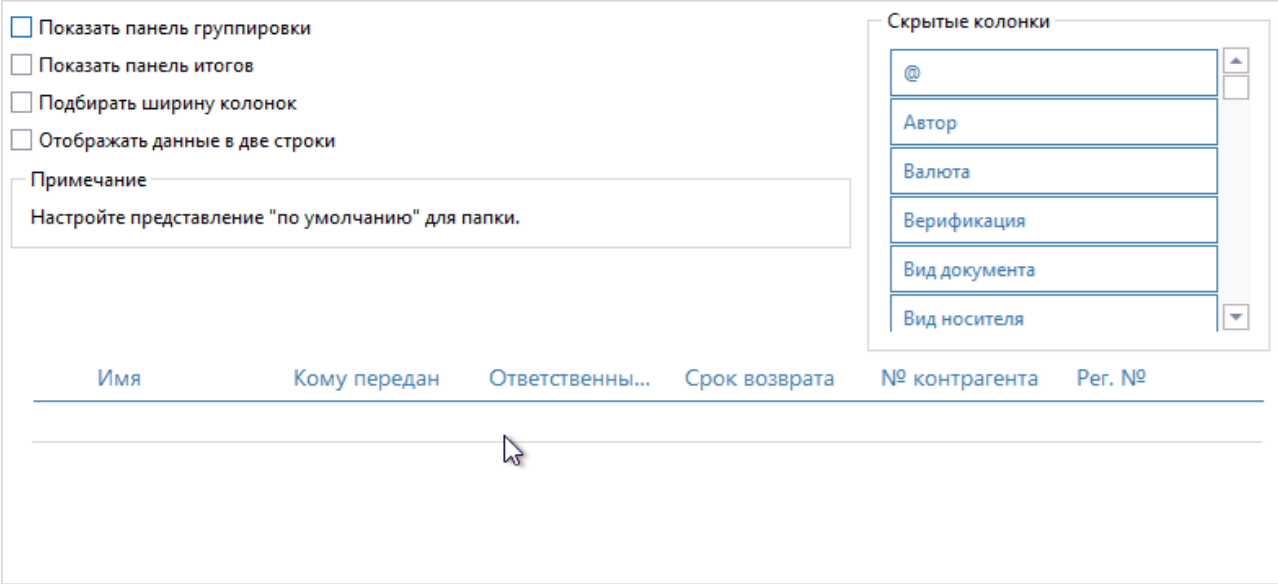
ВАЖНО. Чтобы пользователи увидели папку в системе и могли работать с ней, в обработчике события инициализации нужно программно выдать права доступа на папку.

Проверка фильтра. Обработчик события для *валидации* панели фильтрации. Проверка фильтра работает только в веб-клиенте. Подробнее см. «Проверка фильтров в списках и папках» и пример, как настроить валидацию панели фильтрации для входящих документов.

Обработчики событий **Получение данных** и **Проверка фильтра** используют аргумент _filter для обращения к панели фильтрации и ее элементам. Подробнее см. пример настройки фильтрации записей в папке.

Настройка внешнего вида

По умолчанию вычисляемая папка содержит колонки **ИД** и **Имя**. В области «Настройка представления» можно изменить ее внешний вид:



Показать панель группировки. По умолчанию будет отображаться панель для группировки записей.

При необходимости настройте группировку записей по значению колонки. Для этого перетащите заголовок колонки на панель группировки. Например, для удобной работы со списком договорных документов можно сгруппировать список по ответственным за возврат.

Отображать данные в две строки. Записи будут отображаться в две строки. Рекомендуется указывать, если содержимое отображаемой колонки не вмещается в одну строку.

ПРИМЕЧАНИЕ. Настройки **Показать панель итогов** и **Подбирать ширину колонок** устаревшие, оставлены для совместимости.

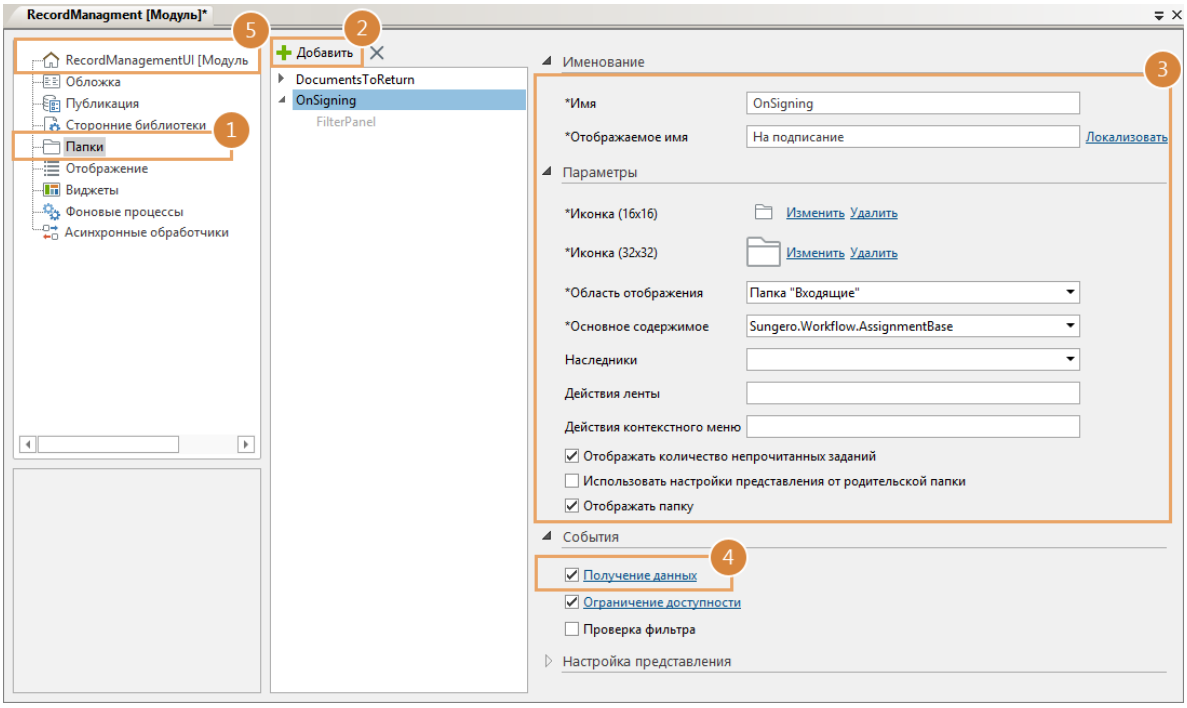
Скрытые колонки. В области находятся колонки, которые не будут отображаться в содержимом папки. Чтобы скрыть колонку, перетащите ее заголовок из табличной части в область **Скрытые колонки**.

Порядок отображения колонок в содержимом папки можно изменить. Для этого переместите их заголовки в области предпросмотра.

Пример создания вычисляемой папки

Предположим, необходимо создать папку «На подписание», которая доступна только руководителям подразделений. Папка должна содержать задания на подписание документов, полученные пользователем, и находиться в папке «Входящие». Для этого:

1. В редакторе модуля перейдите в узел «Папки».



2. Нажмите на кнопку **Добавить**.
3. В списке выделите добавленный элемент. Заполните поля:

   ***Имя** – OnSigning.

   ***Отображаемое имя** – На подписание.

   ***Область отображения** – Папка «Входящие».

   ***Основное содержимое** – Sungero.Workflow.AssignmentBase.

   Добавьте иконки и установите флажок **Отображать количество непрочитанных заданий**. Проверьте, что установлен флажок **Отображать папку**. Подробнее описание параметров см. в разделе «Папки».

4. Чтобы в папку попадали только задания на подписание, добавьте код в обработчик события **Получение данных**. Для этого установите флажок рядом с событием и перейдите по ссылке. В открывшемся редакторе напишите код для вычисления содержимого:

```
/// <summary>
    /// Выбор из списка всех заданий пользователя только заданий на подписание.
    /// </summary>
    /// <param name="query">Список заданий для фильтрации.</param>
    /// <returns>Отфильтрованный список заданий.</returns>
    public virtual IQueryable<Sungero.Workflow.IAssignmentBase>
    OnSigningDataQuery(IQueryable<Sungero.Workflow.IAssignmentBase> query)
    {
        // Фильтр по типу задания.
```

```
    return query.Where(x => Docflow.ApprovalSigningAssignments.Is(x) &&
Docflow.ApprovalSigningAssignments.As(x).IsConfirmSigning != true);
}
```

5. Чтобы у руководителей подразделений папка «На подписание» по умолчанию отображалась в списке папок в проводнике Directum RX, в узле «Модуль» добавьте серверную функцию:

```
/// <summary>
/// Проверка вхождения текущего пользователя в произвольную роль.
/// </summary>
/// <param name="roleSid">Sid роли.</param>
/// <returns> Функция возвращает значение True,
/// если пользователь входит в роль, иначе возвращает False.</returns>
public static bool IncludedInRole(Guid roleSid)
{
  var role = Roles.GetAll(r => r.Sid == roleSid).SingleOrDefault();
  return role != null && Users.Current.IncludedIn(role);
}
```

6. В узле «Модуль» укажите зависимость от модуля:
   - Sungero.Workflow, так как в узле «Папки» в параметре ***Основное содержимое** указано значение Sungero.Workflow.AssignmentBase;
   - Sungero.Docflow, так как в добавленном программном коде использована константа и типы сущностей из данного модуля.

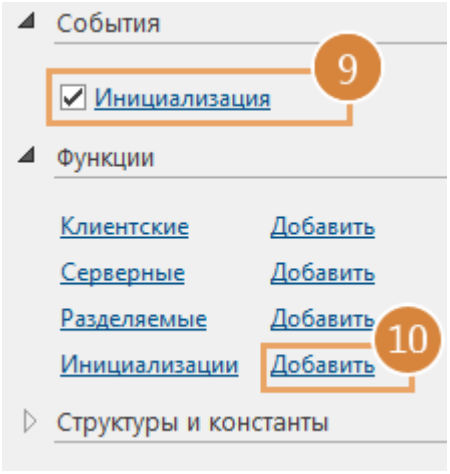
   Если зависимости не указаны, то при публикации могут возникнуть ошибки.

7. В узле «Папки» добавьте обработчик события **Ограничение доступности**. В коде обработчика вызовите серверную функцию:

```
public virtual bool IsOnSigningVisible()
{
  return
Functions
.Module
.IncludedInRole
(Sungero.Docflow.PublicConstants.Module.RoleGuid.BusinessUnitHeadsRole);
}
```

8. При необходимости настройте внешний вид папки по умолчанию и панель фильтрации.

9. Чтобы папка появилась в проводнике системы, при инициализации модуля выдайте на нее права доступа. Для этого в узле «Модуль», в группе «События» установите флажок рядом с событием **Инициализация**.



10. В группе «Функции» перейдите по ссылке **Добавить** рядом с функциями инициализации. В редакторе программного кода измените имя **Function1()** на **GrantRightsOnFolder()** и напишите код для выдачи прав на вычисляемую папку. Добавьте также вызов созданной функции в обработчике события **Initializing()**:

```
public partial class ModuleInitializer
    {
      /// <summary>
      /// Обработчик события инициализации.
      /// </summary>
      public override void
    Initializing(Sungero.Domain.ModuleInitializingEventArgs e)
      {
      /// Вызов функции инициализации.
        GrantRightsOnFolder();
      }
      /// <summary>
      /// Функция инициализации для выдачи прав на вычисляемую папку.
      /// </summary>
      public static void GrantRightsOnFolder()
      {
        var allUsers = Roles.AllUsers;
        Sungero
    .RecordManagement.SpecialFolders.OnSigning.AccessRights.Grant(allUsers,
    DefaultAccessRightsTypes.Read);
        Sungero.RecordManagement.SpecialFolders.OnSigning.AccessRights.Save();
        InitializationLogger.Debug("Выданы права на вычисляемую папку 'На
    подписание'");
      }
    }
```

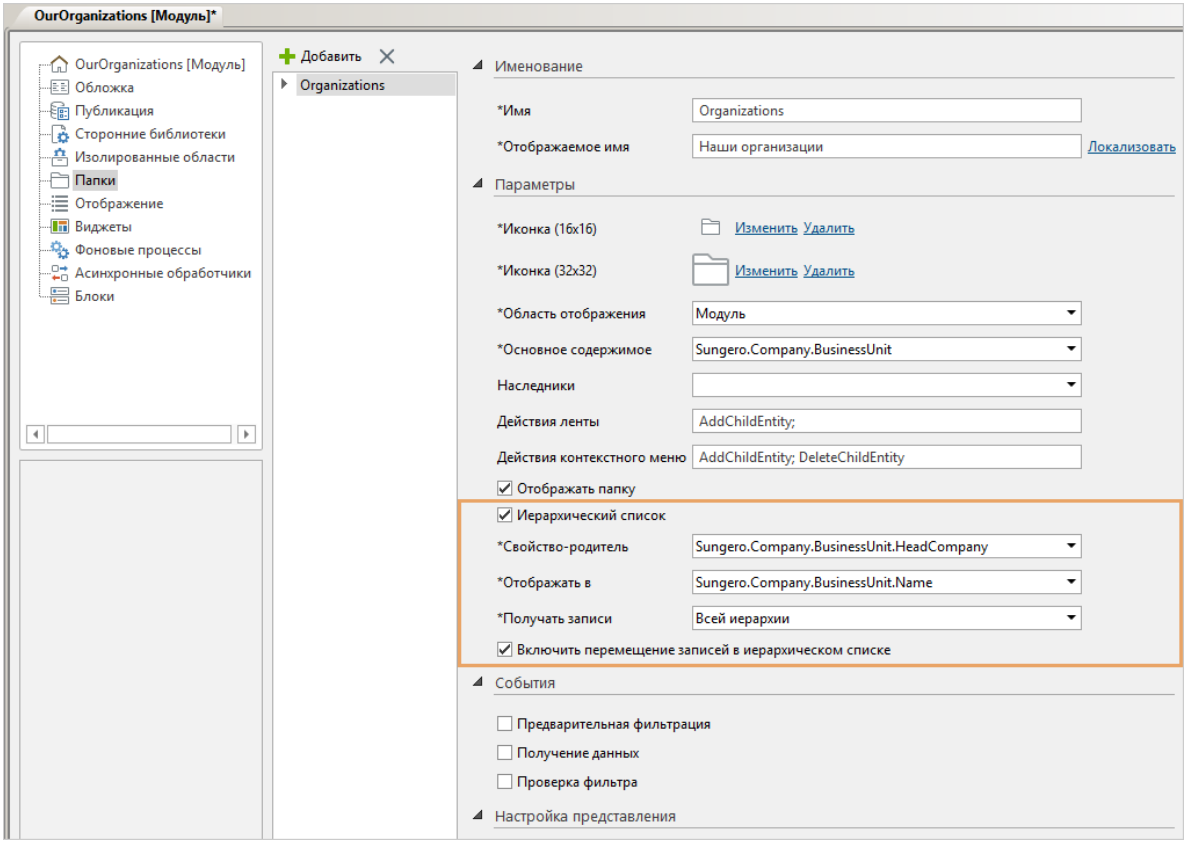
ПРИМЕЧАНИЕ. Для обращения к вычисляемым папкам в коде используется формат: **<Код компании>.<Имя модуля>.SpecialFolders.<Имя вычисляемой папки>**. Например, Sungero.RecordManagement.SpecialFolders.OnSigning. При необходимости подставьте в код примера свои значения.

В результате после *публикации* решения и инициализации в папке «Входящие» у руководителей подразделений будет отображаться папка потока «На подписание». В папке будут задания на подписание документов, полученные пользователем. Другие сотрудники могут добавить папку в дерево с помощью контекстного меню.

СОВЕТ. Для настройки порядка расположения подпапок в папках «Входящие» и «Исходящие» используйте узел «Папки Мой Директум» в редакторе решения.

Настройка иерархического списка

В папке можно отображать объекты в виде иерархического списка. Например, для справочника **Наши организации**, в котором есть головные и дочерние организации. Для этого задайте параметры:



Иерархический список. Если флажок установлен, содержимое папки отображается в виде древовидной структуры. Для настройки иерархии заполните поля:

- ***Свойство-родитель**, по которому строится иерархия. Возможные значения: свойства-ссылки типа сущности, указанного в поле ***Основное содержимое**.

  Например, чтобы настроить иерархию для записей справочника **Наши организации**, в выпадающем списке укажите свойство **HeadCompany** (Головная орг.).

- ***Отображать в**. Столбец, в котором отображается иерархия. Возможные значения: свойства-ссылки или свойства-строки типа сущности, указанного в поле ***Основное содержимое**.

  СОВЕТ. Иерархию удобно отслеживать по названию, в таком случае можно выбрать свойство **Name** (Наименование);

- ***Получать записи**. Способ выполнения события Получение данных. Возможные значения:

- **Верхнего уровня**. Событие применяется только при первоначальной загрузке списка для родительских записей. Например, условия события применяются только для головной организации в справочнике **Наши организации**.
- **Всей иерархии**. Событие применяется на всех уровнях иерархического списка: родителях и их потомках. В этом случае условия применяются и для головной, и для дочерних организаций справочника **Наши организации**.

При первоначальной загрузке иерархического списка также применяется событие Предварительная фильтрация для обоих типов значений поля ***Получать записи**.

Включать перемещение записей в иерархическом списке. Если флажок установлен, то можно перемещать записи иерархического списка между узлами с помощью механизма Drag&Drop. При этом ее свойство-родитель изменяется и сохраняется с отработкой событий, в том числе выполняется *валидация*. Если по ее результатам сохранение записи невозможно, перемещение не выполняется и в проводнике отображается соответствующее сообщение.

ПРИМЕЧАНИЕ. При необходимости вы можете добавить собственные ограничения работы Drag&Drop в иерархическом списке, прописав код в событии До сохранения.

Особенности работы с иерархическими списками

При настройке иерархического списка в папке необходимо учитывать особенности:

- в параметрах папки в поле ***Область отображения** необходимо указывать значение **Модуль**, т.к. в папках потока отображаются только те записи, в которых пользователь является исполнителем или инициатором;
- событие Предварительная фильтрация применяется для обоих типов значений поля ***Получать записи** только при первоначальной загрузке иерархического списка;
- если в поле ***Получать записи** выбрано значение **Верхнего уровня**, список экспортируется только для родительских записей. Для значения **Всей иерархии** выгружается весь список без учета иерархии;
- нет события UI-фильтрации, в котором накладываются дополнительные фильтры на запросы выборки данных;
- недоступна группировка записей в списке;
- недоступно компактное представление списка;
- в программном коде необходим контроль цикличности сущностей. Например, ввести запрет на сохранение записи при работе со списком наших организаций, если дочерняя организация становится родительской записью для головной.

Пример ограничения работы Drag&Drop

Если в иерархических списках нужно добавить собственные ограничения работы механизма Drag&Drop, дополните логику *валидации*. Для этого в событии сущности До сохранения воспользуйтесь параметром **AfterDnD** и пропишите необходимый код. Например, чтобы отключить Drag&Drop в иерархическом списке:

```
public override void BeforeSave(Sungero.Domain.BeforeSaveEventArgs e)
{
  if (e.Params.Contains("AfterDnD"))
  {
     e.AddError("Нельзя перемещать запись с помощью механизма Drag&Drop");
```

```
  }
}
```

Пример создания дочерних записей в иерархическом списке

Чтобы в иерархическом списке при создании запись автоматически становилась дочерней для текущей выделенной записи, необходимо в событии сущности Создание написать код, задающий значение свойства-родителя создаваемой сущности. Для этого доступны дополнительные параметры:

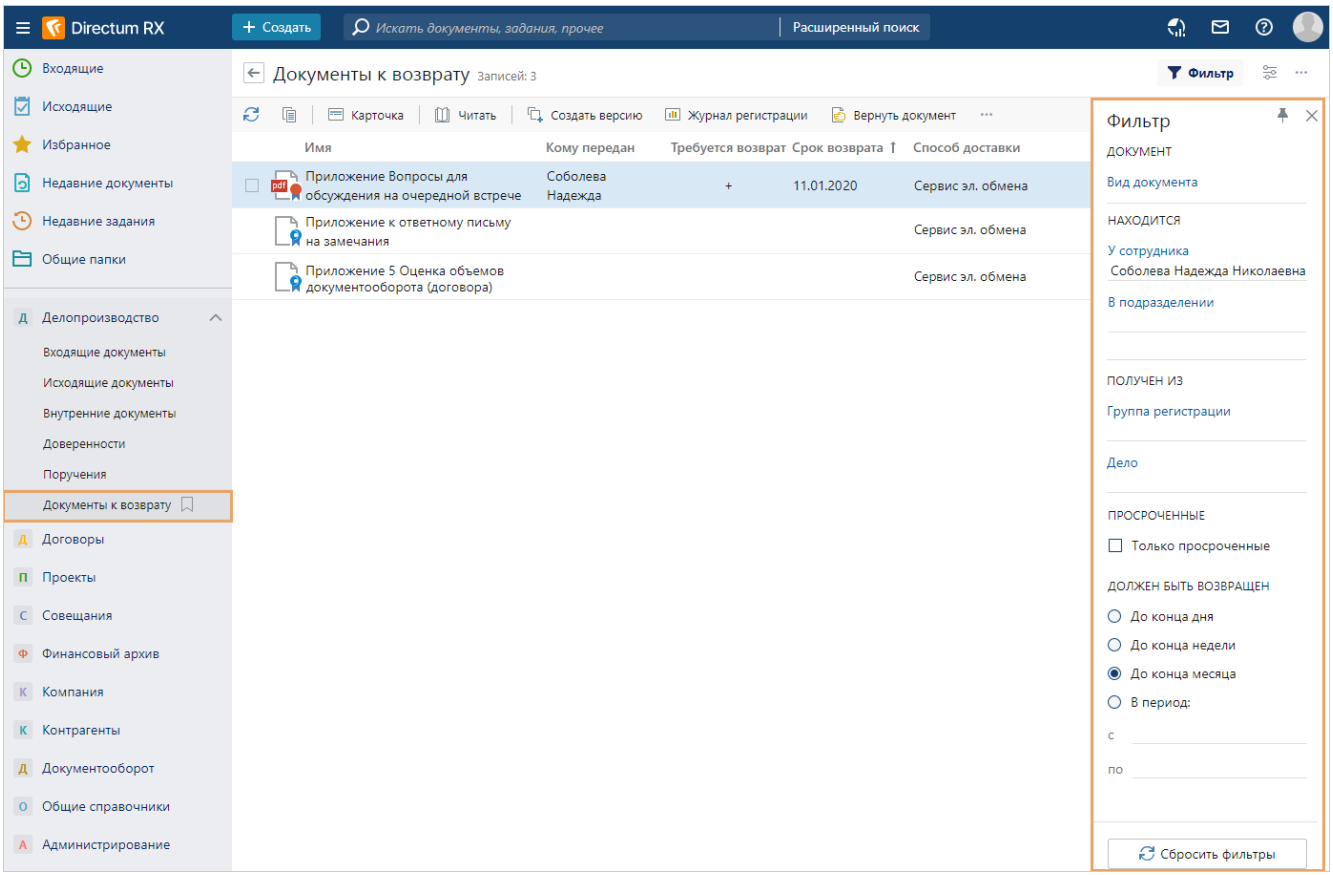
- **ActiveEntityId** – идентификатор выделенной записи;
- **ActiveEntityType** – тип выделенной записи.

Например, в справочнике **Наши организации** для иерархии у новых создаваемых записей можно реализовать автоматическое заполнение свойства **HeadCompany** (Головная орг.):

```
public override void Created(Sungero.Domain.CreatedEventArgs e)
{
  var parentEntityId = 0;
  if (e.Params.TryGetValue("ActiveEntityId", out parentEntityId))
  {
    _obj.HeadCompany = BusinessUnits.Get(parentEntityId);
  }
  base.Created(e);
}
```

Пример настройки фильтрации записей в папке

Создана вычисляемая папка «Документы к возврату» и для нее добавлена панель фильтрации:



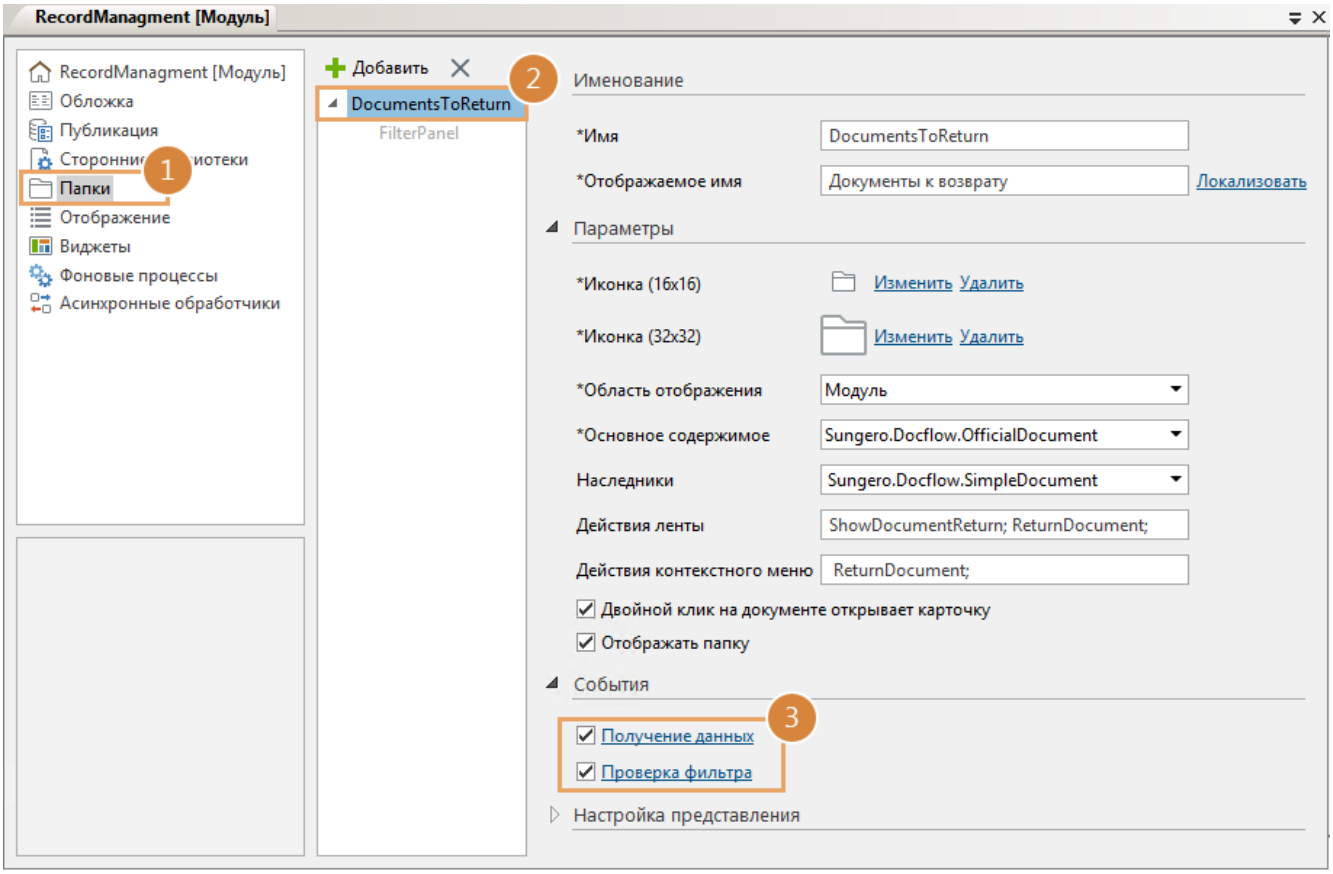
Необходимо настроить фильтрацию записей в папке. Для этого добавьте обработчики событий:

- **Получение данных**. В событии напишите логику вычисления содержимого папки: в ней должны отображаться регистрируемые и нумеруемые документы, которые подлежат возврату. В этом же событии задайте условия для фильтрации содержимого папки по критериям на панели фильтрации;
- **Проверка фильтра**. В событии напишите код для *валидации* панели фильтрации. В системе должно появляться сообщение, если при фильтрации списка пользователь выбрал произвольный период возврата документов и не заполнил критерии «с», «по», «В подразделении» и «У сотрудника». При таком сочетании критериев запрос на веб-сервере будет выполняться долго и вернет большое количество записей. Т.е. пользователю все равно придется уточнять критерии фильтрации.

  ПРИМЕЧАНИЕ. Проверка фильтра работает только в веб-клиенте.

Чтобы добавить логику работы панели фильтрации:

1. В редакторе модуля перейдите в узел «Папки» и в списке элементов выделите вычисляемую папку «Документы к возврату»:



2. Добавьте логику вычисления содержимого папки и условия фильтрации. Для этого в группе «События» установите флажок **Получение данных** и перейдите по ссылке. В открывшемся редакторе напишите код обработчика.

   ВАЖНО. В начало обработчика добавьте проверку, что панель фильтрации включена. Для этого используйте аргумент _filter.

```
/// <summary>
  /// Получить выданные документы.
  /// </summary>
  /// <param name="_filter">Фильтр.</param>
  /// <returns>Список выданных документов.</returns>
  public virtual IQueryable<Sungero.Docflow.IOfficialDocument>
  DocumentsToReturnDataQuery(IQueryable<Sungero.Docflow.IOfficialDocument>
  query)
  {
    // Проверка того, что панель фильтрации включена.
    if (_filter == null)
      return documents;

    // Получение списка сущностей, которые должны отображаться
    // в вычисляемой папке (регистрируемые и нумеруемые документы,
    // которые подлежат возврату).

    var today = Calendar.UserToday;
    var documents = query.Where(l => l.IsReturnRequired == true ||
  l.IsHeldByCounterParty == true);
```

```
  documents = documents.Where(d => !Docflow.ContractualDocumentBases.Is(d) &&
 !Docflow.AccountingDocumentBases.Is(d));

  // Фильтр по статусу.
  if (_filter.Overdue)
  {
    documents = documents.Where(l => l.Tracking.Any(d => d.ReturnDate >
d.ReturnDeadline || (!d.ReturnDate.HasValue && d.ReturnDeadline < today)));
  }

  // Фильтр по виду документа.
  if (_filter.DocumentKind != null)
  {
    documents = documents.Where(l => Equals(l.DocumentKind,
_filter.DocumentKind));
  }

  // Фильтр по сотруднику.
  if (_filter.Employee != null)
  {
    documents = documents.Where(l => l.Tracking.Any(d =>
Equals(d.DeliveredTo, _filter.Employee)));
  }

  // Фильтр по подразделению.
  if (_filter.Department != null)
  {
    documents = documents.Where(l => l.Tracking.Any(d =>
Equals(d.DeliveredTo.Department, _filter.Department)));
  }

  // Фильтр по группе регистрации.
  if (_filter.RegistrationGroup != null)
  {
    documents = documents.Where(l => l.DocumentRegister != null &&
Equals(l.DocumentRegister.RegistrationGroup, _filter.RegistrationGroup));
  }

  // Фильтр по делу.
  if (_filter.Filelist != null)
  {
    documents = documents.Where(l => Equals(l.CaseFile, _filter.Filelist));
  }

  // Исключить строки с Результатом возврата: "Возвращен".
  var returned = Docflow.OfficialDocumentTracking.ReturnResult.Returned;

  // Фильтр по сроку возврата: до конца дня.
  if (_filter.EndDay)
    documents = documents.Where(l => l.Tracking.Any(p =>
!Equals(p.ReturnResult, returned) && p.ReturnDeadline < today.AddDays(1)));

  // Фильтр по сроку возврата: до конца недели.
  if (_filter.EndWeek)
    documents = documents.Where(l => l.Tracking.Any(p =>
!Equals(p.ReturnResult, returned) && p.ReturnDeadline <= today.EndOfWeek()));
```

```
    // Фильтр по сроку возврата: до конца месяца.
    if (_filter.EndMonth)
      documents = documents.Where(l => l.Tracking.Any(p =>
  !Equals(p.ReturnResult, returned) && p.ReturnDeadline <=
  today.EndOfMonth()));

    // Фильтр по сроку возврата: в период с, по.
    if (_filter.Manual)
    {
      var dateFrom = _filter.ReturnPeriodDataRangeFrom;
      var dateTo = _filter.ReturnPeriodDataRangeTo;

      if (dateFrom.HasValue && !dateTo.HasValue)
        documents = documents.Where(l => l.Tracking.Any(p =>
  !Equals(p.ReturnResult, returned) && p.ReturnDeadline >= dateFrom.Value));

      if (dateTo.HasValue && !dateFrom.HasValue)
        documents = documents.Where(l => l.Tracking.Any(p =>
  !Equals(p.ReturnResult, returned) && p.ReturnDeadline <= dateTo.Value));

      if (dateFrom.HasValue && dateTo.HasValue)
        documents = documents.Where(l => l.Tracking.Any(p =>
  !Equals(p.ReturnResult, returned) && (p.ReturnDeadline >= dateFrom.Value &&
  p.ReturnDeadline <= dateTo.Value)));
    }

    return documents;
  }
```

3. Добавьте валидацию панели фильтрации по критериям «с», «по», «В подразделении» и «У сотрудника». Для этого в группе «События» установите флажок **Проверка фильтра**, перейдите по ссылке и напишите код:

```
partial class DocumentsToReturnFolderHandlers
  {
    /// <summary>
    /// Проверка фильтра для списка выданных документов.
    /// </summary>
    public override void
  DocumentsToReturnValidateFilterPanel
  (Sungero.Domain.Client.ValidateFilterPanelEventArgs e)
    {
      // Не выполнять фильтрацию, если установлен
      // произвольный период возврата документов
      // и не заполнены критерии "с", "по" "В подразделении" и "У сотрудника".
      if (_filter.Manual &&
          _filter.Department== null &&
          _filter.Employee == null &&
          _filter.Info.ReturnPeriodDataRangeFrom == null &&
          _filter.Info.ReturnPeriodDataRangeTo == null)
      e.AddError("Заполните параметры, чтобы сократить число результатов
  поиска", _filter.Info.Department, _filter.Info.Employee,
  _filter.Info.ReturnPeriodDataRangeFrom,
  _filter.Info.ReturnPeriodDataRangeTo);
    }
  }
```

4. Опубликуйте и проверьте разработку.

В результате в папке будут отображаться регистрируемые и нумеруемые документы, которые подлежат возврату.

Отображение

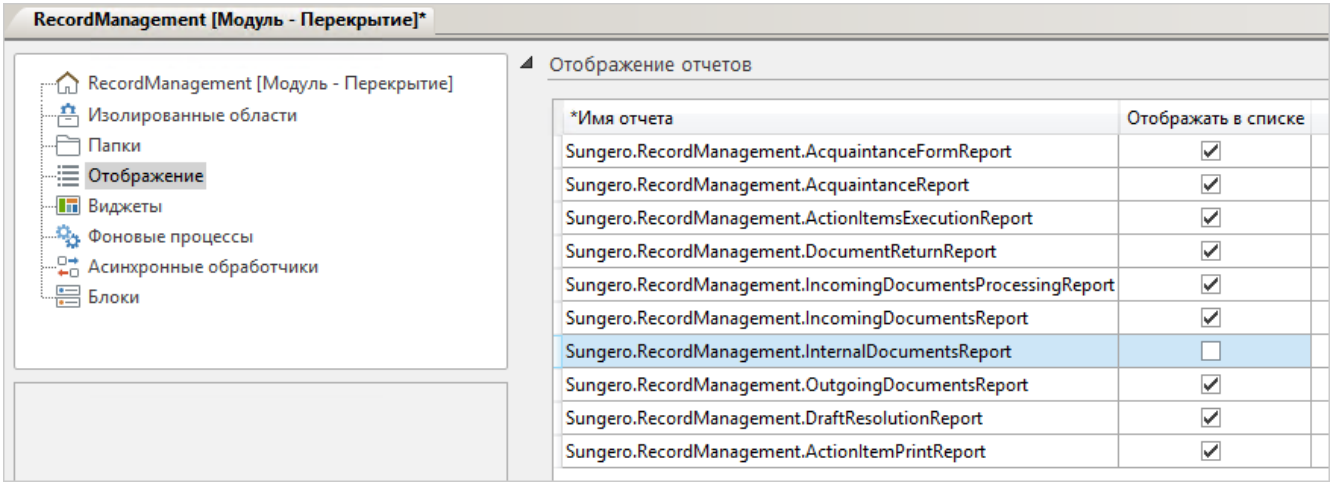
Узел «Отображение» предназначен для настройки:

- отображения отчетов модуля
- списка элементов модуля (устар.)

Настройка отображения отчетов

В табличной части «Отображение отчетов» настраивается список отчетов, которые доступны в модуле по ссылке **Все отчеты**. Чтобы скрыть отчет, перейдите в табличную часть и в столбце **Отображать в списке** снимите флажок рядом с нужным отчетом. И наоборот, если нужно отобразить отчет, установите флажок.

Например, чтобы в перекрытом модуле Sungero.RecordManagement скрыть отчет **Журнал внутренних документов**, снимите флажок рядом с отчетом Sungero.RecordManagement.InternalDocumentsReport:



Настройка элементов модуля (устар.)

Элементы модуля – типы сущностей и вычисляемые папки, которые отображаются на панели навигации при раскрытии папки модуля. Например, элементами могут быть реестры документов и справочники.

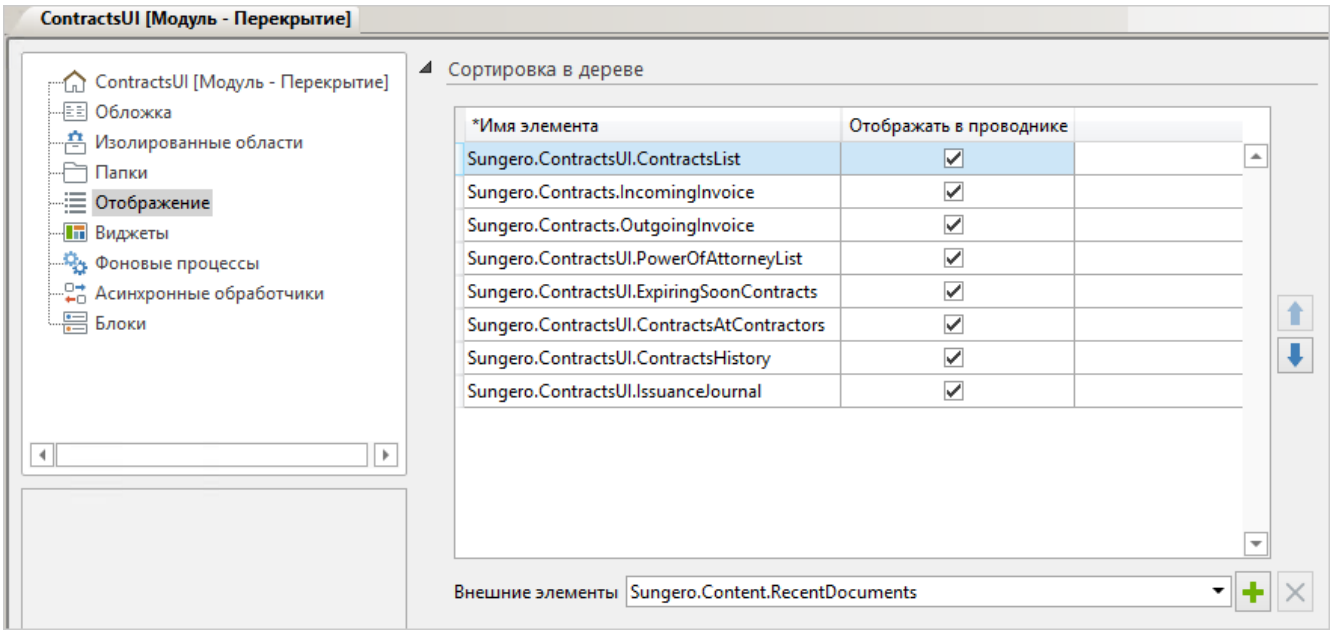
ПРИМЕЧАНИЕ. Обложки рекомендуется настраивать в веб-клиенте. Их редактирование в среде разработке оставлено для совместимости.

Чтобы настроить список элементов через среду разработки:

1. В узле «Отображение» перейдите в табличную часть «Сортировка в дереве». Табличная часть отображается, если в основном узле «Модуль» установлен флажок **Отображать в проводнике**.

2. Чтобы скрыть элемент, в столбце **Отображать в проводнике** снимите флажок рядом с нужным типом сущности или вычисляемой папкой. И наоборот, если нужно отобразить элемент, установите флажок.

   ПРИМЕЧАНИЕ. Чтобы модуль был видимым в системе, в нем должен быть хотя бы один отображаемый элемент.



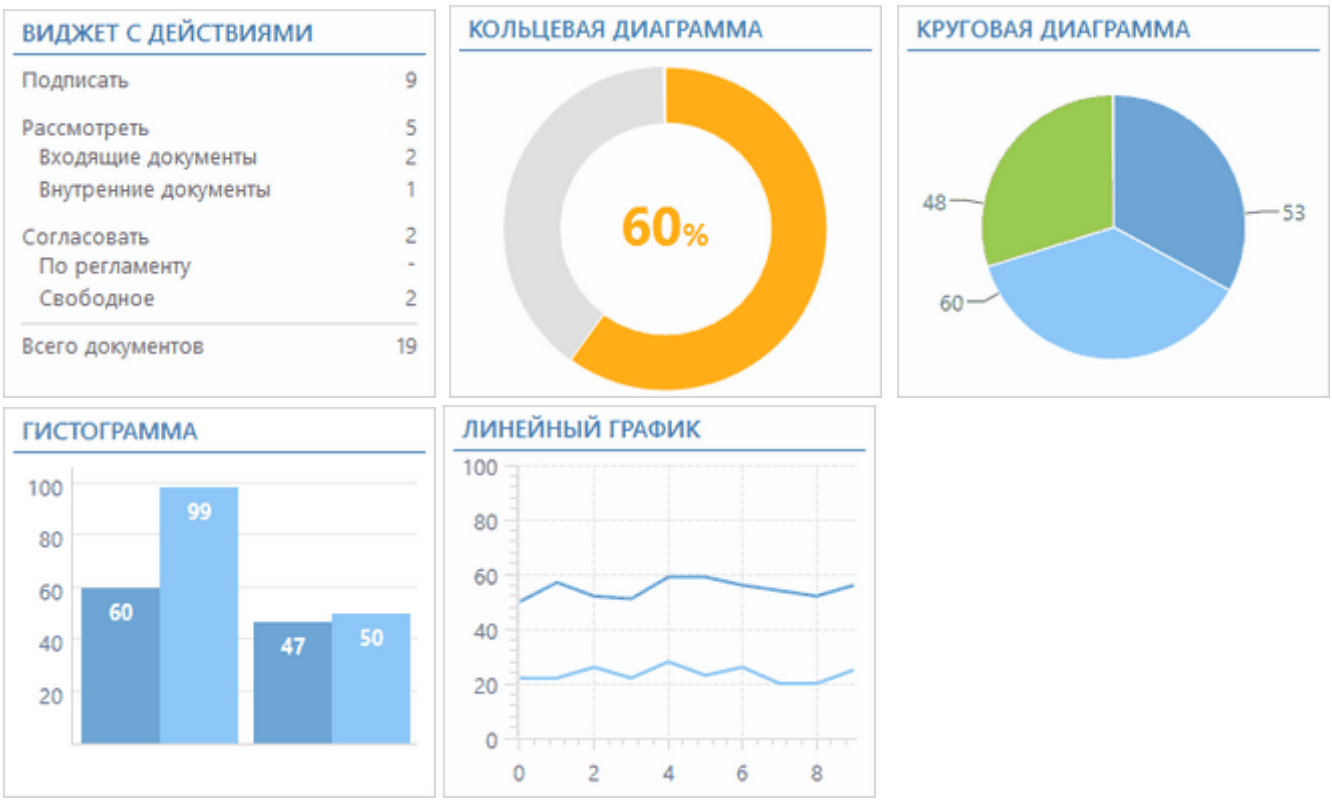
3. В выпадающем списке **Внешние элементы** для выбора доступны типы сущностей и вычисляемые папки системных модулей и модулей, с которыми установлены зависимости. Выберите элементы, которые нужно отображать в текущем модуле, затем нажмите на кнопку . Выбранный элемент добавится в табличную часть.
4. Чтобы изменить порядок отображаемых элементов, используйте кнопки и .

Виджеты

Виджеты предназначены для отображения информации по исполнительской дисциплине, загрузке сотрудников с целью мониторинга их работы и повышения эффективности бизнес-процессов компании. В проводнике системы или веб-клиента они располагаются в области виджетов. Могут входить в состав предопределенного набора виджетов, созданного разработчиком.

На виджетах могут отображаться действия (в виде ссылок) и диаграммы, при нажатии на которые открывается список сущностей, формируется отчет или выполняется другой программный код, написанный разработчиком.

В среде разработки виджеты выглядят следующим образом:



В *базовом решении* уже есть готовые виджеты, каждый из которых содержит свой набор действий и диаграмм. С помощью среды разработки вы можете:

- создавать свои виджеты в редакторах модулей;
- изменять виджеты базового решения в редакторах *перекрытий модулей*;
- скрывать отображение виджетов из списка доступных пользователю.

Для настройки того, что будет происходить при нажатии на элементы виджета, в среде разработки указывается тип действия или диаграммы:

- **Показ списка сущностей**. При нажатии на ссылку или диаграмму открывается список сущностей. Для разработки используются обработчики событий:

  «Фильтрация» для фильтрации списка сущностей.

  «Получение значения» для вычисления значения, которое отображается в виджете.

- **Разрабатываемое**. При нажатии на ссылку или диаграмму выполняется заданная логика, например, формируется отчет. Для разработки используются обработчики событий:

  «Получение значения» для вычисления значения, которое отображается в виджете.

  «Выполнение» для задания логики, которая выполнится при нажатии на ссылку или диаграмму в виджете.

Для виджетов можно добавлять параметры, с помощью которых пользователи смогут настраивать отображение данных на виджете.

Виджет с действиями

Действие – элемент виджета, при нажатии на который открывается список сущностей, папка, запускается отчет или выполняется другой программный код, написанный разработчиком. Справа от названий действий могут отображаться соответствующие им значения. Например, количество заданий, которые находятся в работе у пользователя, сумма договоров за прошлый месяц.

Для логической группировки действий создаются вложенные действия, которые отображаются с небольшим отступом от главного действия. Чтобы визуально отделить действия, используется разделитель, который отображается в виде линии или отступа.

Создание виджета с действиями

Предположим, создается виджет для отображения данных об использовании документов. Необходимо, чтобы в виджете:

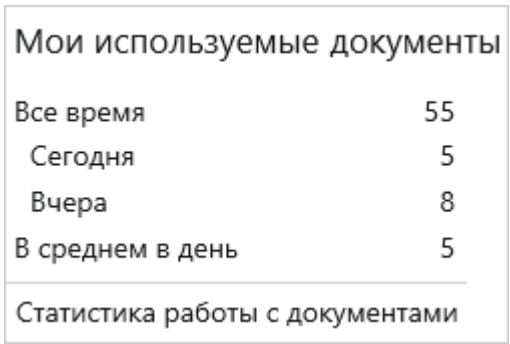
- для главного действия «Все время» и вложенных действий «Сегодня», «Вчера» отображалось количество документов, с которыми текущий пользователь работал за все время, за сегодня и за прошлый день. При нажатии на действия должны открываться соответствующие списки документов;
- для главного действия «В среднем в день» отображалось количество документов, с которым в среднем работает пользователь в день;
- при нажатии на главное действие «Статистика работы с документами» формировался соответствующий отчет.

Также у пользователя должна быть возможность выбирать, какие данные будут отображаться в действиях «Все время», «Сегодня», «Вчера», «В среднем в день»:

- по всем документам, только по документам, которые были открыты на просмотр или на редактирование;
- по всем документам или только по документам, у которых есть содержимое.

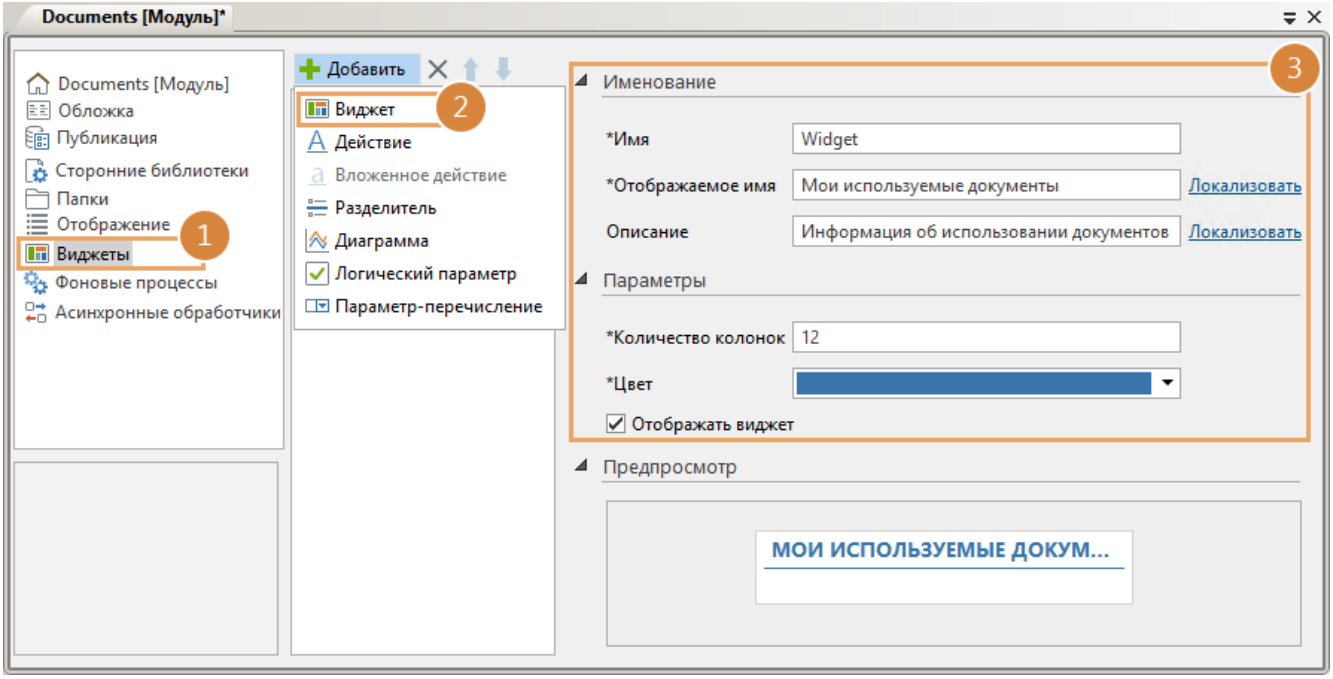
Для этого при создании виджета необходимо добавить параметры, в зависимости от значений которых будут изменяться данные в виджете.

Созданный виджет должен выглядеть следующим образом:



Чтобы создать виджет:

1. В редакторе модуля перейдите в узел «Виджеты».

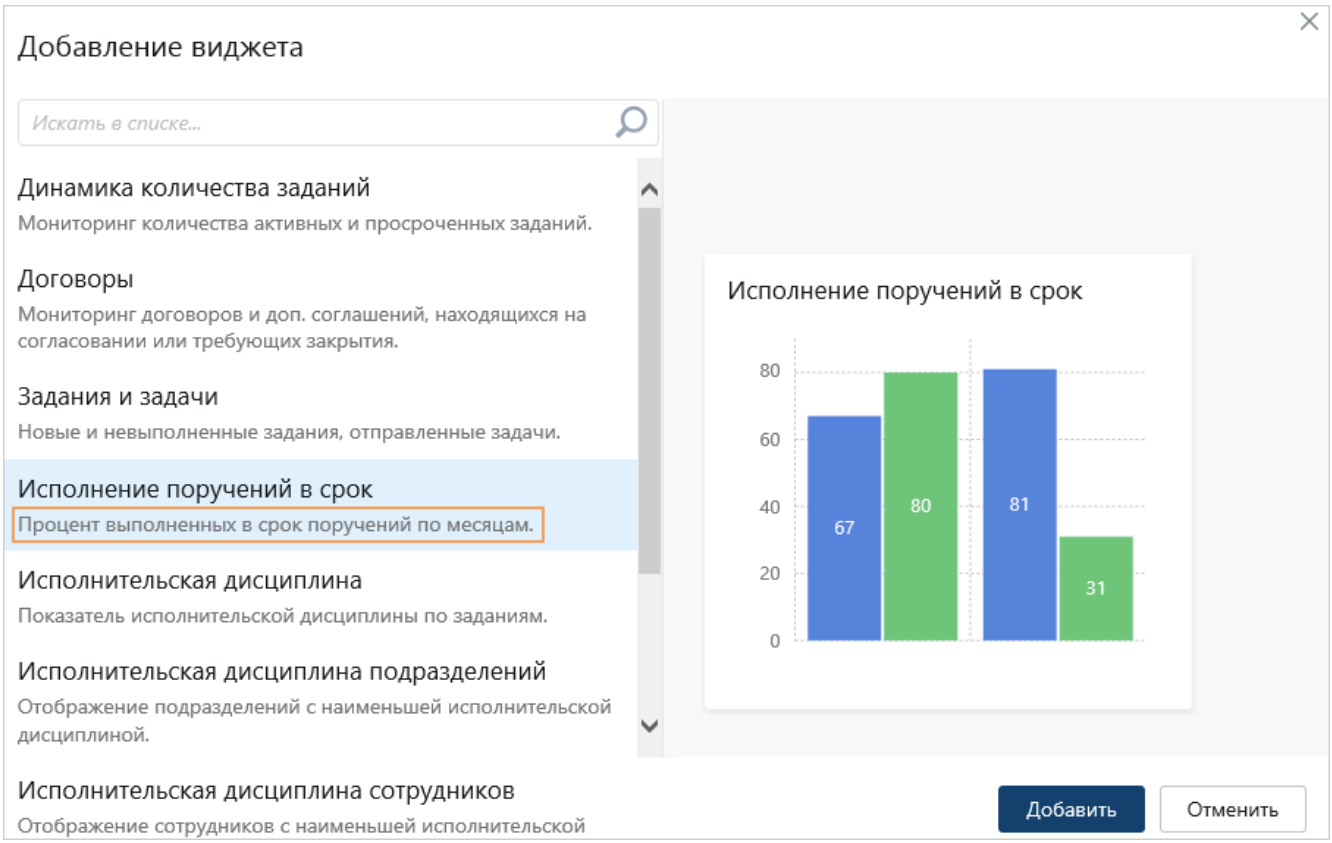


2. Нажмите на кнопку **Добавить** и в выпадающем списке выберите пункт **Виджет**.
3. В списке элементов выделите добавленный виджет и задайте настройки:

   ***Имя** виджета. Должно быть уникальным в рамках одного модуля, состоять из букв латинского алфавита и цифр.

   ***Отображаемое имя**. Заголовок виджета в проводнике системы.

   Описание. Краткое описание виджета. Отображается в проводнике системы в окне добавления виджета.



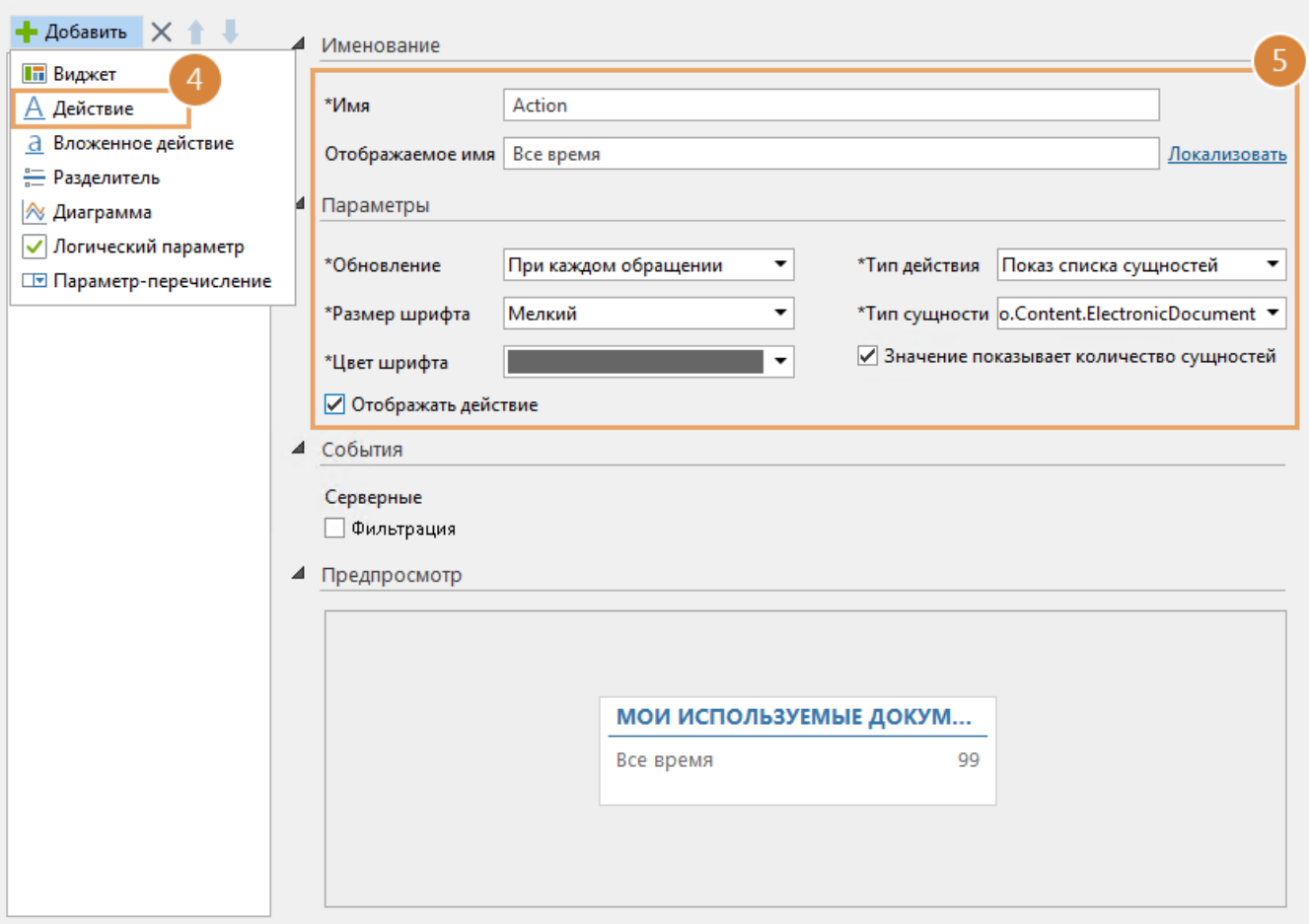
Для *локализации* отображаемого имени и описания перейдите по ссылке **Локализовать**.

***Количество колонок**. Ширина виджета. Возможные значения: целые числа от 1 до 100.

***Цвет** виджета по умолчанию. Для выбора доступно несколько предопределенных цветов.

Отображать виджет. Добавленный виджет доступен для выбора в окне добавления виджетов в Directum RX. Снимите флажок, если необходимо скрыть виджет.

4. Добавьте в виджет главное действие «Все время». Для этого нажмите на кнопку **Добавить** и в выпадающем списке выберите пункт **Действие**.



5. В списке элементов выберите добавленный элемент и задайте настройки:

***Имя** действия. Должно быть уникальным в рамках одного виджета и состоять из букв латинского алфавита.

Отображаемое имя. Название действия виджета в проводнике системы. Для *локализации* отображаемого имени перейдите по ссылке **Локализовать**.

***Обновление**. Период, в котором обновляются данные в действии. Возможные значения: **При каждом обращении**, **В начале дня**, **В начале недели**, **В начале месяца**, **В начале квартала**, **В начале года**. Укажите значение **При каждом обращении**, чтобы данные обновлялись при каждом открытии области виджетов и при добавлении виджета.

***Размер шрифта** названия и значения действия в виджете. Возможные значения: **Мелкий**, **Средний**, **Крупный**.

***Цвет шрифта** для значения действия в виджете. Возможные значения: **серый** и **красный**.

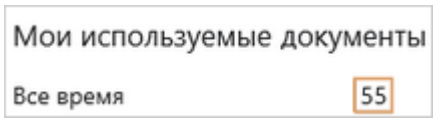
Отображать действие. Добавленное действие отображается в виджете. Снимите флажок, если необходимо скрыть действие.

***Тип действия** виджета. Возможные значения:

- **Показ списка сущностей**. При нажатии на действие в виджете откроется список сущностей того типа, который указан в поле ***Тип сущности**. При необходимости список сущностей можно отфильтровать с помощью события Фильтрация;
- **Разрабатываемое**. Логика, которая будет выполняться при нажатии на действие в виджете, задается в обработчике события Выполнение.

Так как при нажатии на действие «Все время» необходимо открывать список документов, в поле ***Тип действия** выберите значение **Показ списка сущностей**, а в поле ***Тип сущности** укажите тип сущности «Электронный документ» (Sungero.Content.ElectronicDocument).

Установите флажок **Значение показывает количество сущностей**, чтобы справа от действия отображалось количество сущностей в соответствующем списке.

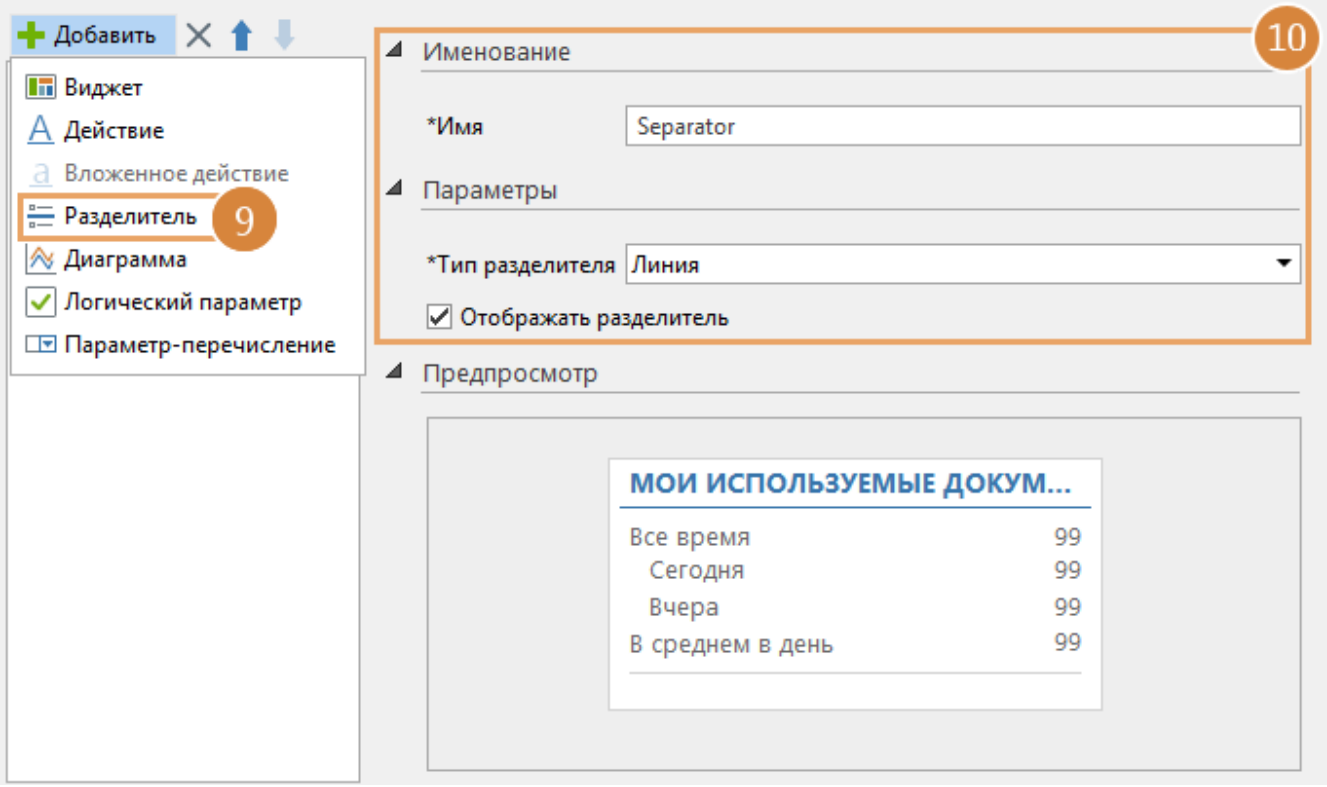


Флажок отображается, если в поле ***Тип действия** указано значение **Показ списка сущностей**.

6. Добавьте вложенные действия «Сегодня» и «Вчера». Для этого в списке элементов выделите главное действие «Все время» и нажмите на кнопку **Добавить**. В выпадающем списке выберите пункт **Вложенное действие**.
7. В списке элементов выберите добавленный элемент и задайте настройки для вложенного действия аналогично настройкам главного действия виджета.

   ПРИМЕЧАНИЕ. Действия в виджете отображаются в том порядке, в котором они расположены в списке элементов. Чтобы изменить порядок, используйте кнопки ⬆ и ⬇ на панели инструментов. Вложенные действия можно перемещать только в пределах главного действия, в котором они созданы.
8. Добавьте главное действие «В среднем в день». В выпадающем списке ***Тип действия** выберите значение **Разрабатываемое**. Задайте остальные настройки аналогично действию «Все время».

9. Чтобы отделить действия «Все время», «Сегодня», «Вчера» и «В среднем в день» от действия «Статистика работы с документами», добавьте разделитель. Для этого в списке элементов нажмите на кнопку **Добавить** и выберите пункт **Разделитель**.



10. В списке элементов выберите добавленный элемент и задайте настройки:

    ***Имя** разделителя. Должно быть уникальным в рамках одного виджета, состоять из букв латинского алфавита и цифр.

    ***Тип разделителя**. Способ разделения действий в виджете. Возможные значения: **Отступ**, **Линия**.

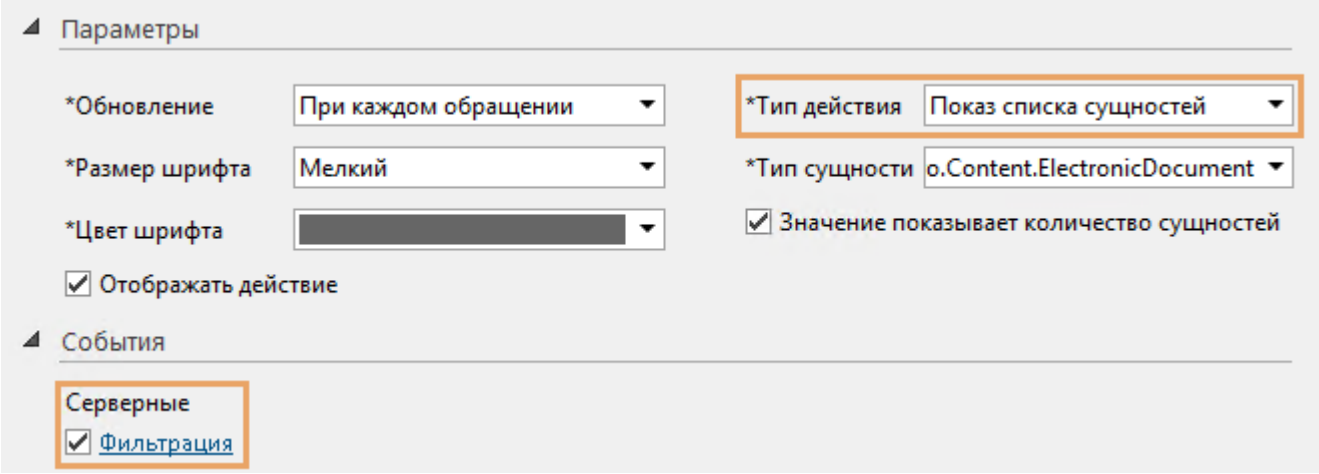
    Отображать разделитель. Добавленный разделитель отображается в виджете. Снимите флажок, если необходимо скрыть разделитель.

11. Добавьте главное действие «Статистика работы с документами». В выпадающем списке ***Тип действия** выберите значение **Разрабатываемое**. Задайте остальные настройки аналогично действию «Все время».

12. Чтобы у пользователя была возможность выбирать, какие данные будут отображаться в действиях «Все время», «Сегодня», «Вчера» и «В среднем в день», добавьте:

    - логический параметр **Учитывать документы без содержимого**. В зависимости от значения параметра в виджете будут отображаться данные по всем документам или только по документам, у которых есть содержимое;
    - параметр-перечисление **Показывать документы**. Возможные значения: **Все**, **Просмотренные**, **Измененные**. В зависимости от значения параметра в виджете будут отображаться данные по всем документам, только по документам, которые были открыты на просмотр или на редактирование.

13. Задайте обработчики событий для действий. Для этого в редакторе свойств действия установите флажок, который соответствует нужному событию, и перейдите по ссылке:
    - «Фильтрация» для фильтрации списка сущностей. Задайте обработчик события для действий «Все время», «Сегодня», «Вчера». Событие доступно, если в настройках действия в поле ***Тип действия** указано значение **Показ списка сущностей**:

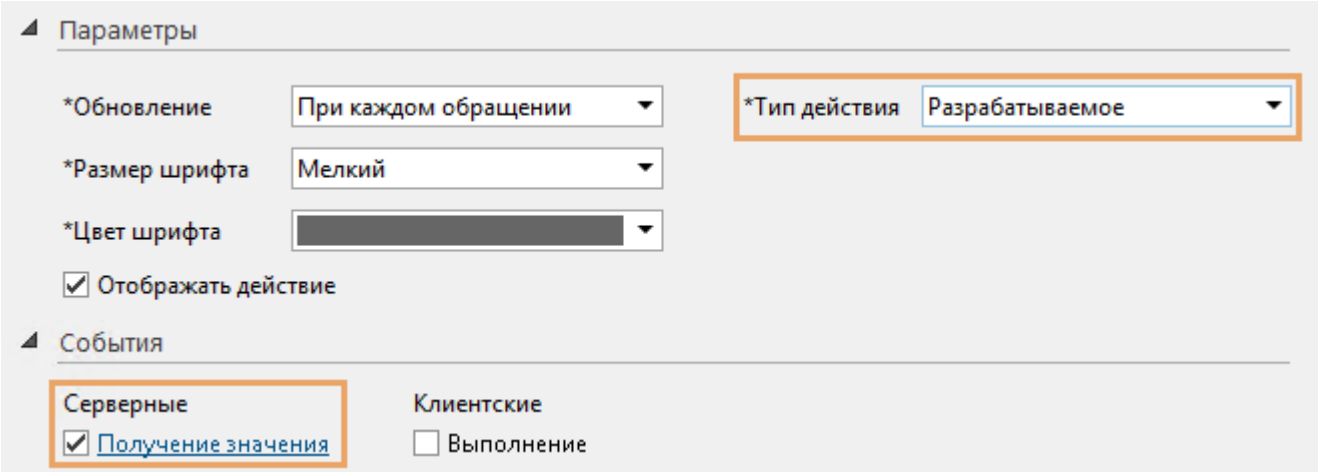


Обработчик события «Фильтрация» для действия «Сегодня» с учетом значений параметров виджета

```
   /// <summary>
/// Формирование списка документов, который открывается
/// при нажатии на действие "Сегодня".
/// </summary>
public virtual IQueryable<Sungero.Content.IElectronicDocument>
DocumentsTodayFiltering(IQueryable<Sungero.Content.IElectronicDocument> query)
{
  // Если для параметра-перечисления "Показывать документы"
  // установлено значение "Просмотренные", то в виджете
  // отображается информация по документам, которые
  // в течение дня которые пользователь открывал на просмотр.
  if (_parameters.ShowDocuments == Docflow.Widgets.Documents.ShowDocuments.Read)
    query = query.WhereDocumentHistory(h => h.Action ==
Sungero.CoreEntities.History.Action.Read);
  // Если для параметра-перечисления "Показывать документы"
  // установлено значение "Измененные", то в виджете отображается
  // информация по документам, которые в течение дня пользователь
  // открывал на редактирование.
  if (_parameters.ShowDocuments == Docflow.Widgets.Documents.ShowDocuments.Update)
    query = query.WhereDocumentHistory(h => h.Action ==
Sungero.CoreEntities.History.Action.Update);
  // Если для логического параметра
  // "Учитывать документы без содержимого" установлено значение
  // True, то в виджете отображается информация по документам,
  // у которых есть версии.
  if (!_parameters.AllowDocumentWithoutBody)
    query = query.Where(h => h.HasVersions);
  // Если для параметра-перечисления "Показывать документы"
  // установлено значение "Все" и для логического параметра
  // «Учитывать документы без содержимого» установлено значение
  // False,то в виджете отображается информация по всем документам
  // которые в течение дня открывал пользователь.
```

```
  return query
    .WhereDocumentHistory(h => Equals(Users.Current, h.User) && h.HistoryDate >=
Calendar.Today)
}
```

- «Получение значения» для вычисления значения, которое отображается в виджете справа от действия. Задайте обработчик события для действия «В среднем в день». Событие доступно, если в поле ***Тип действия** указано значение **Разрабатываемое** или значение **Показ списка сущностей** и снят флажок **Значение показывает количество сущностей**:

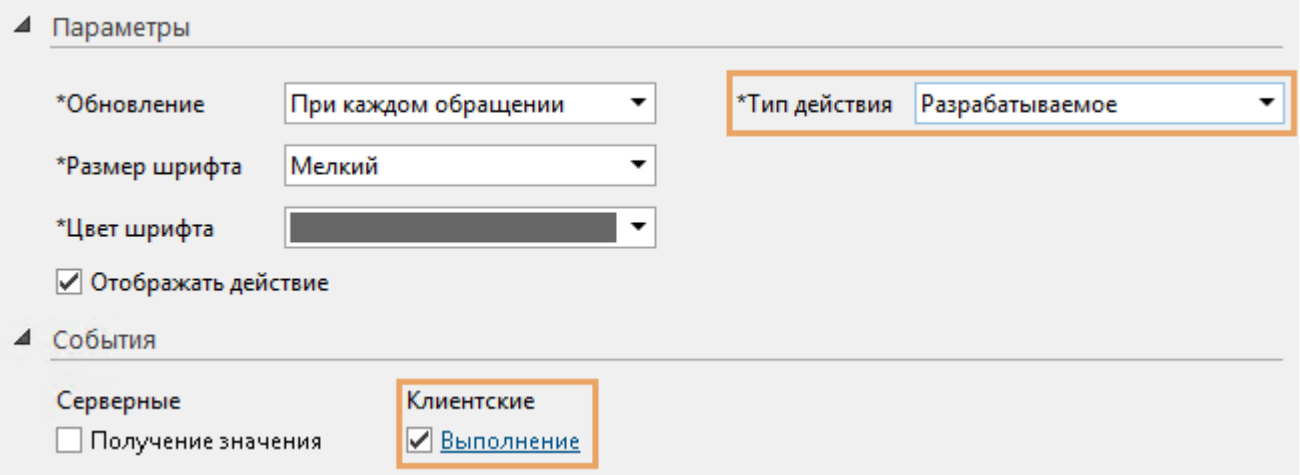


Обработчик события «Получение значения» для действия «В среднем в день» с учетом значений параметров виджета

```
   /// <summary>
/// Расчет среднего количества документов в день, с которым работает
/// пользователь.
/// </summary>
public virtual string GetDocumentsPerDayValue()
{
  var documents = Sungero.Content.ElectronicDocuments.GetAll();
  // Если для параметра-перечисления "Показывать документы"
  // установлено значение "Просмотренные", то в виджете
  // отображается информация по документам, которые пользователь
  // открывал на просмотр.
  if (_parameters.Show == Docflow.Widgets. Documents.Show.Read)
    documents = documents.WhereDocumentHistory(h => h.Action ==
Sungero.CoreEntities.History.Action.Read);
  // Если для параметра-перечисления "Показывать документы"
  // установлено значение "Измененные", то в виджете отображается
  // информация по документам, которые пользователь открывал на
  // редактирование.
  if (_parameters.Show == Docflow.Widgets. Documents.Show.Update)
    documents = documents.WhereDocumentHistory(h => h.Action ==
Sungero.CoreEntities.History.Action.Update);
  // Если для логического параметра
  // "Учитывать документы без содержимого" установлено значение
  // True,то в виджете отображается информация по документам,
  // у которых есть версии.
  if (!_parameters.AllowDocumentWithoutBody)
    documents = documents.Where(h => h.HasVersions);
  // Вызов функции для расчета среднего количества документов,
```

```
  //с которым работает пользователь в день.
  var averageDocumentsPerDay = CalculateAverageDocumentsPerDay(documents);
  return averageDocumentsPerDay;
}
```

- «Выполнение» для задания логики, которая будет выполнена при нажатии на действие виджета. Задайте обработчик события для действия «Статистика работы с документами». Например, для открытия отчета. Событие доступно, если в поле ***Тип действия** указано значение **Разрабатываемое**.



Обработчик события «Выполнение» для действия «Статистика работы с документами»

```
   /// <summary>
/// Вызов отчета "Статистика работы с документами".
/// </summary>
public virtual void ExecuteDocumentsStatisticAction()
{
  var report = Docflow.Reports.GetDocumentStatisticReport();
  report.Open();
}
```

ВАЖНО. Для событий доступен аргумент **_parameters**, с помощью которого можно обратиться к параметрам и их значениям.

В результате будет создан виджет для отображения данных об использовании документов пользователем.

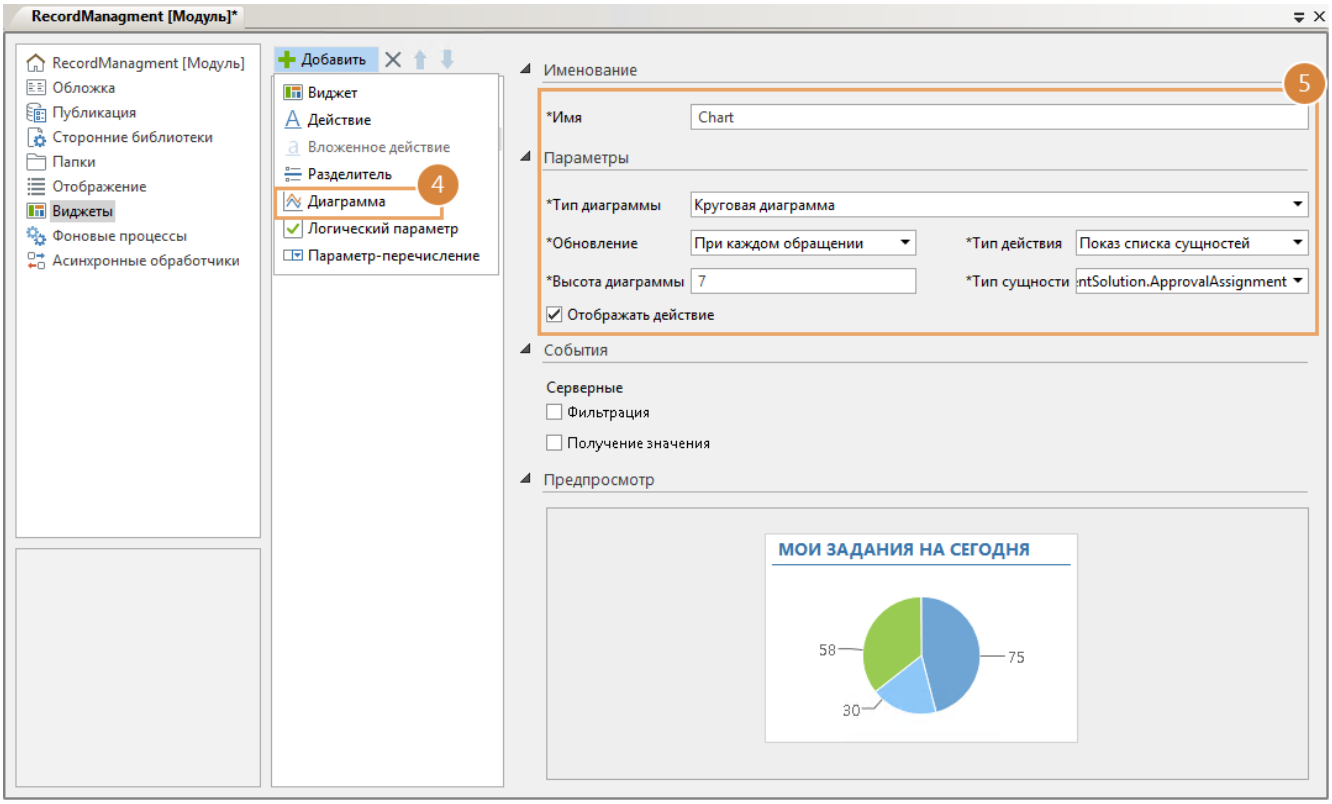
Виджет с диаграммой

Диаграмма – это элемент виджета, предназначенный для отображения аналитической информации в графическом виде, например, для отображения исполнительской дисциплины, показателей эффективности сотрудника. Есть несколько типов диаграмм, например круговая диаграмма:

- кольцевая диаграмма
- круговая диаграмма
- гистограмма
- линейный график

Создание виджета с диаграммой

1. В редакторе модуля перейдите в узел «Виджеты».
2. Нажмите на кнопку **Добавить** и в выпадающем списке выберите пункт **Виджет**.
3. В списке элементов выделите добавленный виджет и задайте настройки, как при создании виджета с действиями.
4. Добавьте в виджет диаграмму. Для этого нажмите на кнопку **Добавить** и в выпадающем списке выберите пункт **Диаграмма**.



5. В списке элементов выберите добавленный элемент и задайте настройки:

   ***Имя** диаграммы. Должно быть уникальным в рамках одного виджета, состоять из букв латинского алфавита и цифр.

   ***Тип диаграммы**, например, «Кольцевая диаграмма», «Линейный график».

***Высота диаграммы** с учетом заданной ширины виджета. Возможные значения: целые числа от 1 до 100. Если диаграмма с указанной высотой не вмещается в ширину виджета, то она увеличится только до того размера, который вмещается. В виджете добавятся отступы по вертикали. Чтобы диаграмма вмещалась, в настройках виджета в поле ***Количество колонок** увеличьте ширину виджета, или уменьшите высоту диаграммы.

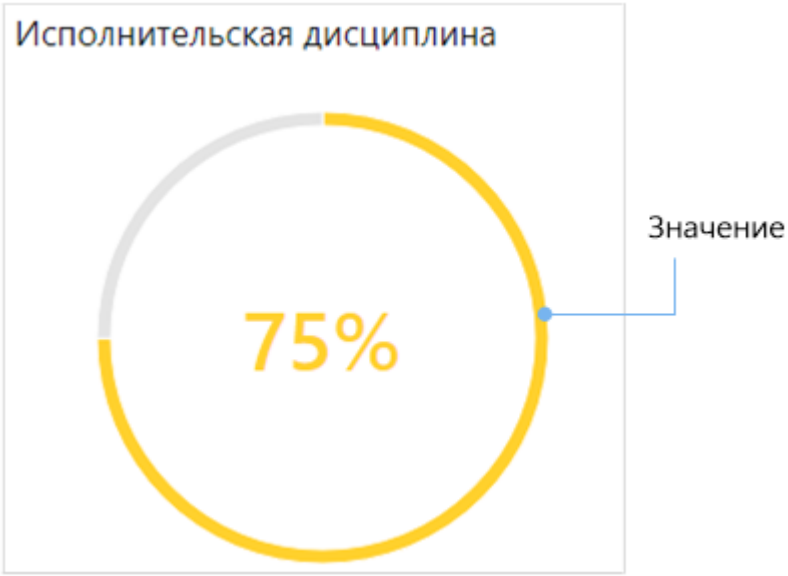
Остальные поля заполняются также, как при разработке виджета с действиями.

6. Чтобы у пользователя была возможность выбирать, какие данные будут отображаться на диаграмме, добавьте параметры.
7. Задайте обработчики событий. Если в поле ***Тип действия** выбрано значение:
   - **Показ списка сущностей**, то задайте обработчики событий **Фильтрация** и **Получение значения;**
   - **Разрабатываемое**, то задайте обработчики событий **Получение значения** и **Выполнение**.

   Для линейного графика задается только обработчик события **Получение значения**.

Кольцевая диаграмма

Используется для отображения одного показателя в процентах. Например, для отображения исполнительской дисциплины.



Если в программном коде вычислено значение больше 100%, то на диаграмме отобразится 100%, если меньше 0%, то отобразится 0%.

Событие «Фильтрация»

Если в настройках диаграммы в поле ***Тип действия** выбрано значение **Показ списка сущностей**, то задайте обработчик события **Фильтрация**.

Событие предназначено для фильтрации списка сущностей, который открывается при нажатии на значение диаграммы.

Обработчик события «Фильтрация» для списка заданий на исполнение поручений

```
var userNow = Calendar.UserNow;
var serverNow = userNow.FromUserTime();
var userBeginOfToday = userNow.BeginningOfDay();
var serverBeginOfToday = userBeginOfToday.FromUserTime();
var serverEndOfDay = userNow.EndOfDay().FromUserTime();
query = query.Where(a => !Equals(a.Deadline, serverBeginOfToday) &&
                    (a.Deadline < serverNow && a.Deadline !=
a.Deadline.Value.Date ||
                     a.Deadline < serverEndOfDay.Date && a.Deadline ==
a.Deadline.Value.Date));
```

В результате при нажатии на значение виджета откроется список просроченных заданий текущего сотрудника.

Событие «Получение значения»

Событие предназначено для вычисления значения диаграммы и настройки внешнего вида.

Аргументы события:

- e.Chart.AddValue() – добавление значения кольцевой диаграммы.

```
public                                  virtual                                  void
   GetWidgetChartValue(Sungero.Domain.GetWidgetGaugeChartValueEventArgs e)
   {
     // Добавление значения кольцевой диаграммы.
     e.Chart.AddValue(label, value, color);
   }
```

Содержит параметры:

label – текст подсказки, которая появляется при наведении курсора мыши на значение диаграммы. Текст подсказки рекомендуется задавать в редакторе ресурсов. Тип параметра – строка.

value – значение, которое отображается внутри диаграммы, указывается в процентах. Тип параметра – число.

color – настройка цвета для значения кольцевой диаграммы. Параметр необязательный. Тип параметра – цвет.

Обработчик события «Получение значения» виджета «Исполнительская дисциплина»

```
   public                                  virtual                                  void
      GetAssignmentCompletionGraphAllAssignmentChartValue
      (Sungero.Domain.GetWidgetGaugeChartValueEventArgs e)
{
  if (!Equals(_parameters.Performer,
Widgets.AssignmentCompletionGraph.Performer.All) && Users.Current.IsSystem ==
true)
    return;

  // Получение значения кольцевой диаграммы.
  var value =
Docflow
.PublicFunctions
.Module.GetPercentageOfAssignmentsCompletedInTime(businessUnitIds,
```

```
departmentsIds, employee, periodBegin, Calendar.UserToday.EndOfDay(), unwrap,
false, needFilter);
  // Если значение вычислить не удалось,
  // в виджете появится сообщение «Нет данных».
  if (value == null)
    return;

  // Получение цвета виджета.
  var color =
Functions.Module.GetAssignmentCompletionWidgetValueColor(value.Value);
  // Добавление значения кольцевой диаграммы.
  // Текст подсказки берется из ресурсов модуля.
  e.Chart.AddValue(string.Format("{0}",
Sungero.RecordManagement.Resources.AssignmentCompletion), value.Value, color);
}
```

В результате в диаграмме виджета «Исполнительская дисциплина» будет отображаться значение – процент исполнительской дисциплины. Цвет виджета будет меняться в зависимости от процента выполненных заданий в срок по функции **GetAssignmentCompletionWidgetValueColor()**: выше 75% – зеленый, выше 50% – желтый, ниже 50% – красный.

Событие «Выполнение»

Если в настройках диаграммы в поле ***Тип действия** выбрано значение **Разрабатываемое**, то задайте обработчик события **Выполнение**.

Событие предназначено для задания логики, которая выполняется при нажатии на значение кольцевой диаграммы.

Обработчик события «Выполнение» виджета «Исполнительская дисциплина»

```
   public                                virtual                                void
      ExecuteAssignmentCompletionGraphAllAssignmentChartAction()
{
  // Открытие отчета по исполнительской дисциплине.
  PublicFunctions.Module.OpenAssignmentCompletionReport(_parameters.Performer,
_parameters.Period);
}
```

В результате при нажатии на значение виджета откроется отчет по исполнительской дисциплине подразделения, сотрудников или выбранного сотрудника. То, какой отчет будет формироваться из виджета, зависит от настроенных параметров.

Круговая диаграмма

Используется для отображения отношения частей к целому. Например, для отображения выполненных и просроченных заданий по отношению к общему количеству заданий.

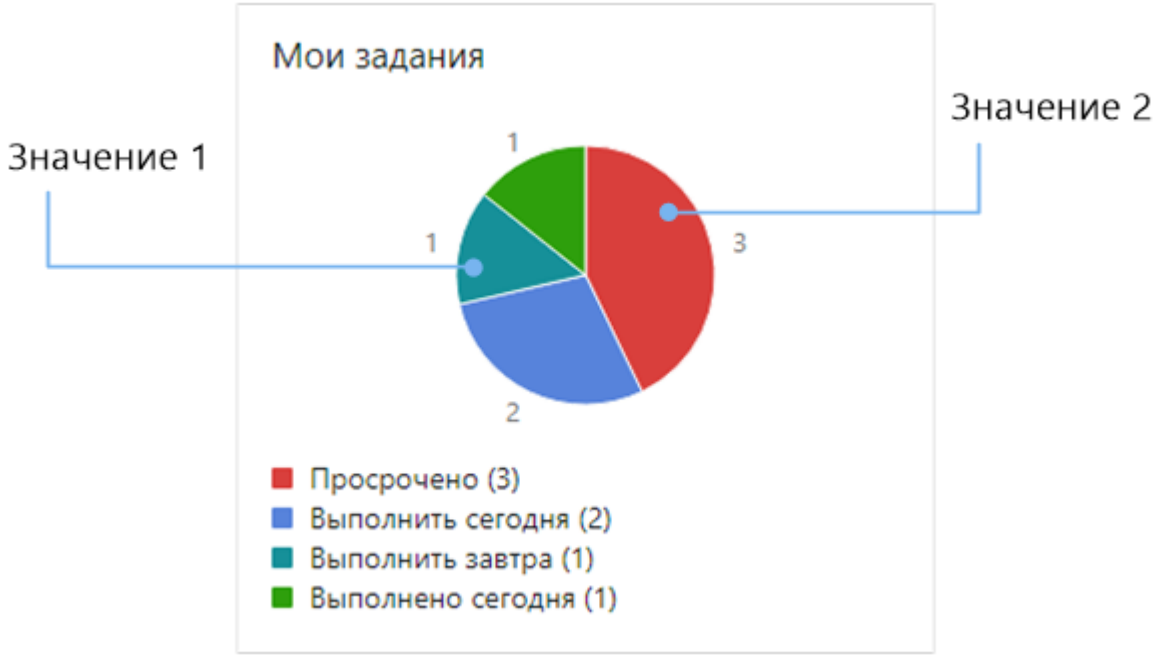


Диаграмма состоит из значений – отдельных секторов. Для нее настраивается отображение легенды.

Событие «Фильтрация»

Если в настройках диаграммы в поле ***Тип действия** выбрано значение **Показ списка сущностей**, то задайте обработчик события **Фильтрация**. Событие предназначено для фильтрации списка сущностей, который открывается при нажатии на значение диаграммы.

Аргументы события:

- e.ValueId – уникальный идентификатор значения круговой диаграммы. Задается в обработчике события Получение значения.

Обработчик события «Фильтрация» для списка заданий

```
    // При нажатии на сектор открывать список просроченных заданий
       // текущего сотрудника.
public virtual IQueryable<Sungero.Workflow.IAssignment>
MyTodayAssignmentsChartFiltering(System.Linq.IQueryable
<Sungero.Workflow.IAssignment> query,
Sungero.Domain.WidgetPieChartFilteringEventArgs e)
{
  // ValueId – идентификатор сектора значения круговой диаграммы.
  if (e.ValueId == OverdueToday)
     return query.Where(a => a.Performer.Equals(Users.Current) && a.Deadline <
Calendar.UserNow);
  return query;
```

```
}
```

В результате при нажатии на сектор с идентификатором OverdueToday будет открываться список просроченных заданий текущего сотрудника.

Событие «Получение значения»

Событие предназначено для вычисления значений диаграммы и настройки внешнего вида.

Аргументы события:

- e.Chart.AddValue() – добавление значений круговой диаграммы.

```
public virtual void GetWidgetChartValue(Sungero.Domain.GetWidgetPieChartValueEventArgs e)
{
  // Добавление значений круговой диаграммы.
  e.Chart.AddValue(valueId1, label1, value1, color1);
  e.Chart.AddValue(valueId2, label2, value2, color2);
}
```

Содержит параметры:

valueId – уникальный идентификатор значения круговой диаграммы. Используется в событиях Фильтрация и Выполнение. Рекомендуется задавать в константах модуля. Тип параметра – строка.

label – текст, который отображается в легенде и в подсказке, появляющейся при наведении курсора мыши на значение. Текст рекомендуется задавать в редакторе ресурсов. Тип параметра – строка.

value – значение, которое отображается рядом с сектором. Тип параметра – число.

color – настройка цвета для сектора круговой диаграммы. Параметр необязательный. Тип параметра – цвет.

- e.Chart.DisplayValueFormat – формат значения, которое отображается рядом сектором. Например, единицы измерения, проценты.

```
// Числовое значение рядом с сектором будет отображаться в процентах.
  e.Chart.DisplayValueFormat = {"0:P"};
```

Подробнее см. в статьях Microsoft «Строки стандартных числовых форматов» и «Строки настраиваемых числовых форматов».

- e.Chart.IsLegendVisible – отображение легенды для диаграммы. По умолчанию отображается.

```
// Отключить отображение легенды.
  e.Chart.IsLegendVisible = false;
```

Обработчик события «Получение значения» виджета «Мои задания»

```
public virtual void GetMyTodayAssignmentsChartValue
  (Sungero.Domain.GetWidgetPieChartValueEventArgs e)
{
  var assignments = Workflow.Assignments.GetAll();
  // Добавление сектора с просроченными заданиями.
      var overdue = Functions.Module.GetMyTodayAssignments(assignments,
   _parameters.Substitution, Constants.Module.OverdueToday).Count();
  if (overdue != 0)
```

```
    e.Chart.AddValue(Constants.Module.OverdueToday, Resources.WidgetMTAOverdue,
 overdue, Colors.Charts.Red);
// Добавление сектора с заданиями, которые нужно выполнить сегодня.
    var deadline = Functions.Module.GetMyTodayAssignments(assignments,
 _parameters.Substitution, Constants.Module.DeadlineToday).Count();
if (deadline != 0)
                                e.Chart.AddValue(Constants.Module.DeadlineToday,
 Resources.WidgetMTADeadline, deadline, Colors.Charts.Color1);
// Добавление сектора с заданиями, которые нужно выполнить до послезавтра.
    var after = Functions.Module.GetMyTodayAssignments(assignments,
 _parameters.Substitution, Constants.Module.AfterToday).Count();
if (after != 0)
    e.Chart.AddValue(Constants.Module.AfterToday, Resources.WidgetMTAAfter,
 after, Colors.Charts.Color6);
// Добавление сектора с заданиями, которые выполнены сегодня.
    var completed = Functions.Module.GetMyTodayAssignments(assignments,
 _parameters.Substitution, Constants.Module.CompletedToday).Count();
if (completed != 0)
                               e.Chart.AddValue(Constants.Module.CompletedToday,
 Resources.WidgetMTACompleted, completed, Colors.Charts.Green);
}
```

В результате диаграмма виджета «Мои задания» будет состоять из четырех секторов, для каждого из которых отображается свое значение и цвет. При нажатии на сектор откроется соответствующий список заданий.

Событие «Выполнение»

Если в настройках диаграммы в поле ***Тип действия** выбрано значение **Разрабатываемое**, то задайте обработчик события **Выполнение**. Событие предназначено для задания логики, которая выполняется при нажатии на значение диаграммы.

Аргументы события:

- e.ValueId – уникальный идентификатор значения круговой диаграммы. Задается в обработчике события Получение значения.

Обработчик события «Выполнение»

```
public virtual void
   ExecuteMyTodayAssignmentsChart(Sungero.Domain.Client.
   ExecuteWidgetPieChartActionEventArgs e)
{
  // При нажатии на сектор «Выполнить сегодня»
  // выполнится функция MyTodayAssignments()
  if (e.ValueId == Constants.Module.CompletedToday)
    Functions.Module.MyTodayAssignments(_parameters.Performer);

}
```

Гистограммы

В виджеты можно добавлять вертикальные и горизонтальные гистограммы:

- с накоплением – для просмотра отношения отдельных частей к целому за определенный период. Например, для отображения загрузки сотрудников за последний месяц: сколько всего задач у каждого сотрудника, сколько из них просрочено, а сколько без нарушения срока;
- с группировкой – для сравнения значений. Например, для сравнения исполнительской дисциплины сотрудников.

ПРИМЕЧАНИЕ. При выборе типа гистограмм учитывайте, что в гистограммах с группировкой, в отличие от гистограмм с накоплением, отображаются нулевые значения. Например, если исполнительская дисциплина сотрудника составляет 0%, то можно настроить открытие списка просроченных заданий при нажатии на это значение.

Гистограмма состоит из нескольких серий и значений. Для нее настраивается отображение легенды и подпись к оси.

Событие «Фильтрация»

Если в настройках гистограммы в поле ***Тип действия** выбрано значение **Показ списка сущностей**, то задайте обработчик события **Фильтрация**.

Событие предназначено для фильтрации списка сущностей, который открывается при нажатии на значение гистограммы.

Аргументы события:

- e.SeriesId – уникальный идентификатор серии гистограммы;
- e.ValueId – уникальный идентификатор значения серии.

Идентификаторы задаются в обработчике события Получение значения.

Обработчик события «Выполнение»

```
public virtual IQueryable<Sungero.Workflow.IAssignment>
MyAssignmentsChartFiltering(IQueryable<Sungero.Workflow.IAssignment> query,
Sungero.Domain.WidgetBarChartFilteringEventArgs e)
{
  long performerId;
  // Использование идентификатора серии гистограммы.
  if(long.TryParse(e.SeriesId, out performerId))
    // Использование идентификатора значения серии.
    if(e.ValueId == MyModule.Constants.Module.Overdue)
    {
      var userNow = Calendar.UserNow;
      var serverNow = userNow.FromUserTime();
      var userBeginOfToday = userNow.BeginningOfDay();
      var serverBeginOfToday = userBeginOfToday.FromUserTime();
      var serverEndOfDay = userNow.EndOfDay().FromUserTime();
      return query.Where(a => a.Performer.Id == performerId &&
!Equals(a.Deadline, serverBeginOfToday) && (a.Deadline < serverNow && a.Deadline
!= a.Deadline.Value.Date || a.Deadline < serverEndOfDay.Date && a.Deadline ==
a.Deadline.Value.Date));
    }
  else
    return query.Where(a => a.Performer.Id == performerId);
}
```

Событие «Получение значения»

Событие предназначено для добавления серий, вычисления значений гистограммы и настройки внешнего вида.

Аргументы события:

- e.Chart.AddNewSeries() – добавление серии гистограммы.

```
public virtual void
   GetWidgetChartValue(Sungero.Domain.GetWidgetBarChartValueEventArgs e)
   {
```

```
    // Добавление новой серии.
    var series1 = e.Chart.AddNewSeries(seriesId1, title1);
  }
```

Содержит параметры:

seriesId – уникальный идентификатор серии. Используется в событиях Фильтрация и Выполнение. Тип параметра – строка.

title – подпись к серии, например, ФИО сотрудников, названия подразделений, месяцы. Тип параметра – строка.

- series1.AddValue() – добавление значений серии гистограммы.

```
public virtual void GetWidgetChartValue(Sungero.Domain.GetWidgetBarChartValueEventArgs e)
  {
    // Создание серии.
    var series1 = e.Chart.AddNewSeries(seriesId1, title1);
    // Заполнение серии значениями.
    series1.AddValue(valueId1, label1, value1, color1);
    series1.AddValue(valueId2, label2, value2, color2);
  }
```

Содержит параметры:

valueId – уникальный идентификатор значения. Используется в событиях Фильтрация и Выполнение. Можно указывать, например, ИД пользователя, ИД группы пользователей, Guid группы, месяц, число. Рекомендуется задавать в константах модуля. Тип параметра – строка.

label – текст, который отображается в легенде и в подсказке, появляющейся при наведении курсора мыши на значение. Текст рекомендуется задавать в редакторе ресурсов. Тип параметра – строка.

value – значение, которое отображается в колонках гистограммы. Тип параметра – число.

color – настройка цвета для значения гистограммы. Параметр необязательный. Тип параметра – цвет.

- e.Chart.AxisTitle – подпись к оси гистограммы.

```
e.Chart.AxisTitle = "Подпись к оси";
```

- e.Chart.IsLegendVisible – отображение легенды для гистограммы. По умолчанию отображается. Если из заголовка виджета или подписи к оси понятно, что означают конкретные значения, то рекомендуется отключать отображение легенды для экономии места под гистограмму.

```
// Отключить отображение легенды.
  e.Chart.IsLegendVisible = false;
```

- e.Chart.IsValuesVisible – отображение значений в колонках гистограммы. По умолчанию отображаются. Рекомендуется отключать отображение значений, если они не вмещаются в колонки гистограммы.

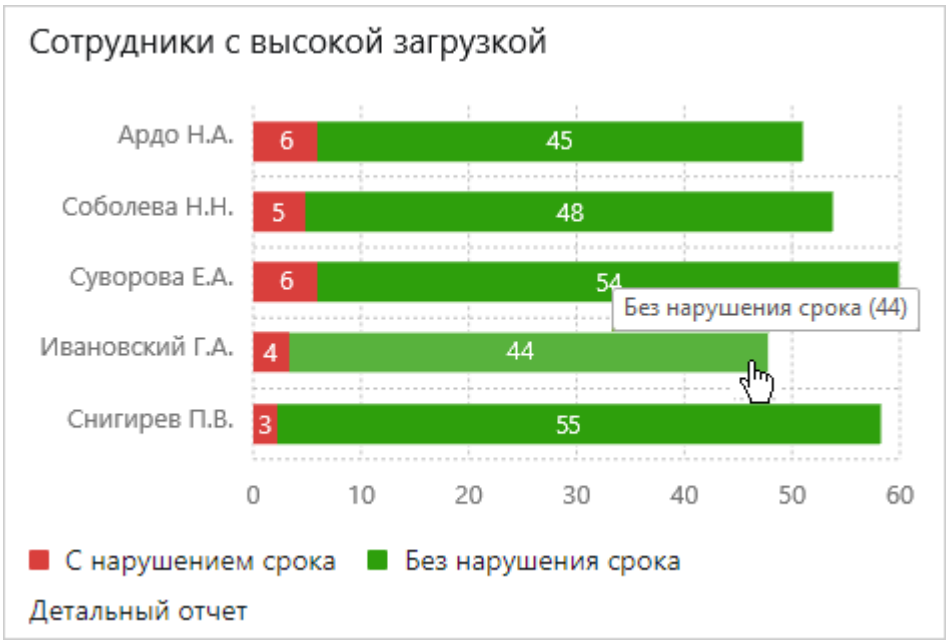
```
// Отключить отображение значений в колонках гистограммы.
  e.Chart.IsValuesVisible = false;
```

Обработчик события «Получение значения» виджета «Сотрудники с высокой загрузкой»

```
  // Используется горизонтальная гистограмма с накоплением.
public virtual void
GetTopLoadedPerformersGraphTopLoadedPerformersValue
(Sungero.Domain.GetWidgetBarChartValueEventArgs e)
```

```
{
  if (!Equals(_parameters.CarriedObjects,
Widgets.TopLoadedPerformersGraph.CarriedObjects.All) && Users.Current.IsSystem
== true)
    return;
  // Вычисление загрузки сотрудников.
  var performerLoads = items
  .Where(i => i.AssignmentCompletion != null && i.IsActiveEmployee)
  .OrderByDescending(i => i.AssignmentsCount)
  .Take(5)
  .Select(i => Structures.Module.PerformerLoad.Create(Employees.Get(i.Employee),
i.AssignmentsCount, i.OverdueAssignmentsCount));
// Получение загруженных сотрудников с уникальными именами.
// Сотрудники сортируются по убыванию количества заданий.
  var uniqueNamedPerformerLoads =
Functions.Module.SetUniquePerformerNames(performerLoads.ToList());
  foreach (var performerLoad in uniqueNamedPerformerLoads)
  {
    // Создание серии performerSeries.
    var performerSeries =
e.Chart.AddNewSeries(performerLoad.PerformerLoad.Employee.Id.ToString(),
performerLoad.UniqueName);
    // Добавление значения серии (задания с нарушением срока).
    performerSeries.AddValue(Constants.Module.OverduedAssignments,
Resources.WithOverdue,
    // Добавление значения серии (задания без нарушения срока).
    performerLoad.PerformerLoad.OverduedAssignment, Colors.Charts.Red);
    performerSeries.AddValue(Constants.Module.NotOverduedAssignments,
Resources.WithoutOverdue, performerLoad.PerformerLoad.AllAssignment,
performerLoad.PerformerLoad.OverduedAssignment, Colors.Charts.Green);
  }
}
```

В результате в гистограмме будут отображаться сотрудники с высокой загрузкой. Для каждого сотрудника показывается количество заданий с нарушением срока и без нарушения.



Событие «Выполнение»

Если в настройках гистограммы в поле ***Тип действия** выбрано значение **Разрабатываемое**, то задайте обработчик события **Выполнение**.

Событие предназначено для задания логики, которая выполняется при нажатии на значение гистограммы.

Аргументы события:

- e.SeriesId – уникальный идентификатор серии гистограммы;
- e.ValueId – уникальный идентификатор значения серии.

Идентификаторы задаются в обработчике события Получение значения.

Обработчик события «Выполнение» виджета «Сотрудники с высокой загрузкой»

```
    public virtual void ExecuteTopLoadedPerformersGraphTopLoadedPerformersAction
       (Sungero.Domain.Client.ExecuteWidgetBarChartActionEventArgs e)
{
  // Получение сотрудника по ИД.
  long employeeId;
  if (!long.TryParse(e.SeriesId, out employeeId))
    // Открытие отчета по исполнительской дисциплине сотрудника.
    PublicFunctions.Module.OpenReportFromEmployeeWidgets(employeeId,
_parameters.Period);
}
```

В результате при нажатии на колонку гистограммы откроется отчет по исполнительской дисциплине выбранного сотрудника.

Линейный график

Используется для отображения динамики изменения различных показателей. Например, для отображения динамики количества заданий, мониторинга количества просроченных и заданий в работе.

Линейный график состоит из нескольких серий и значений, которые представляют собой отдельные точки (x,y) на линии. При нажатии на значение графика появляется подсказка.

Для линейного графика задается только обработчик события **Получение значения**, в котором настраивается отображение легенды, оси и подписи к осям.

Событие «Получение значения»

Событие предназначено для добавления серий графика, заполнения серии значениями и настройки внешнего вида.

Аргументы события:

- e.Chart.AddNewSeries() – добавление серии линейного графика.

```
public virtual void GetWidgetChartValue(Sungero.Domain.GetWidgetPlotChartValueEventArgs e)
{
  // Добавление новой серии.
  var series1 = e.Chart.AddNewSeries(title, color);
}
```

  Содержит параметры:

  title – название линии графика, которое отображается в легенде и в подсказке, появляющейся при нажатии на значение графика. Текст рекомендуется задавать в редакторе ресурсов. Тип параметра – строка.

  color – настройка цвета для серии графика. Параметр необязательный. Тип параметра – цвет.

- series1.AddValue() – добавление значений серии линейного графика.

```
public virtual void GetWidgetChartValue(Sungero.Domain.GetWidgetPlotChartValueEventArgs e)
{
  // Создание серии.
  var series1 = e.Chart.AddNewSeries(title, color);
  // Заполнение серии значениями.
  series1.AddValue(valueX1, valueY1);
  series1.AddValue(valueX2, valueY2);
  series1.AddValue(valueX3, valueY3);
}
```

  Содержит параметры:

  valueX – координата точки на линии по оси X. Тип параметра – дата и время или число.

  valueY – координата точки на линии по оси Y. Тип параметра – число.

- e.Chart.Axis.X.AxisType – тип оси: дата (DateTime) или число (Double).

  ВАЖНО. Если указан тип оси DateTime, то в методе AddValue() параметр valueX необходимо задавать в том же формате DateTime.

- e.Chart.Axis.X.Title – подпись к оси X;
- e.Chart.Axis.Y.Title – подпись к оси Y;
- e.Chart.Axis.Y.MaxValue – максимальное значение по оси Y. Если значение будет больше указанного, то график обрежется;
- e.Chart.Axis.Y.MinValue – минимальное значение по оси Y. Если значение будет меньше указанного, то график обрежется.

ПРИМЕЧАНИЕ. Если значения MaxValue или MinValue не указаны, то добавляется отступ вверх от максимального значения графика или вниз от минимального значения. Значение отступа рассчитывается как 10% от разницы максимального и минимального значения.

- e.Chart.IsLegendVisible – отображение легенды для линейного графика. По умолчанию отображается.

Обработчик события «Получение значения» виджета «Динамика количества заданий»

```
    public                                virtual                                void
       GetActiveAssignmentsDynamicAssignmentsDynamicChartValue
       (Sungero.Domain.GetWidgetPlotChartValueEventArgs e)
{
  if (!Equals(_parameters.CarriedObjects,
Widgets.ActiveAssignmentsDynamic.CarriedObjects.All) && Users.Current.IsSystem
== true)
    return;
  // Получение точек на графике виджета.
  var points =
Docflow
.PublicFunctions.Module.GetActiveAssignmentsDynamicPoints(businessUnitIds,
departmentsIds, employee, periodBegin, periodEnd, unwrap, needFilter);
  if (!points.Any())
    return;
  // Тип оси X – дата.
  e.Chart.Axis.X.AxisType = AxisType.DateTime;
  // Подпись к оси Y. Берется из ресурсов модуля.
  e.Chart.Axis.Y.Title = Resources.WidgetActiveAssignmentsDynamicYAxisTitle;
  // Создание первой серии.
  var assignmentsInWork =
e.Chart.AddNewSeries(Resources.WidgetActiveAssignmentsDynamicSeriesAllTitle,
Colors.Charts.Color1);
  // Создание второй серии.
  var overdueAssignmentsInWork =
e
.Chart.AddNewSeries(Resources.WidgetActiveAssignmentsDynamicSeriesOverduedTitle,
Colors.Charts.Red);
  var maxValue = points.Select(p => p.ActiveAssignmentsCount).Max();
  var minValue = points.Select(p => p.ActiveOverdueAssignmentsCount).Min();
  // Заполнение первой и второй серии значениями.
  foreach (var point in points)
  {
    assignmentsInWork.AddValue(point.Date, point.ActiveAssignmentsCount);
    overdueAssignmentsInWork.AddValue(point.Date,
point.ActiveOverdueAssignmentsCount);
  }
  // Округление значений по оси Y.
  if (minValue > Math.Round(maxValue * 0.1))
    e.Chart.Axis.Y.MinValue = Math.Round(minValue * 0.95);
  else
    // Минимальное значение по оси Y.
    e.Chart.Axis.Y.MinValue = 0;
    // Максимальное значение по оси Y.
    e.Chart.Axis.Y.MaxValue = Math.Round(maxValue * 1.05);
}
```

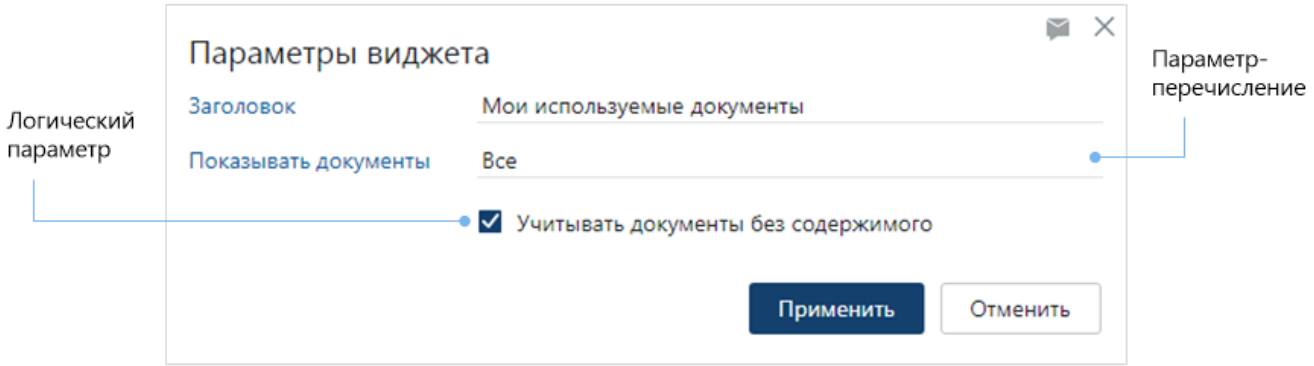
В результате на графике будут отображаться две линии, которые показывают динамику количества заданий (с нарушением срока и без нарушения). При нажатии на значение графика появится подсказка.

Параметры виджетов

С помощью параметров пользователи могут выбирать, какие данные будут отображаться в действиях и диаграммах виджета. В проводнике системы значения параметров задаются в окне настройки, которое открывается при нажатии на кнопку **Параметры** в области заголовка виджета.

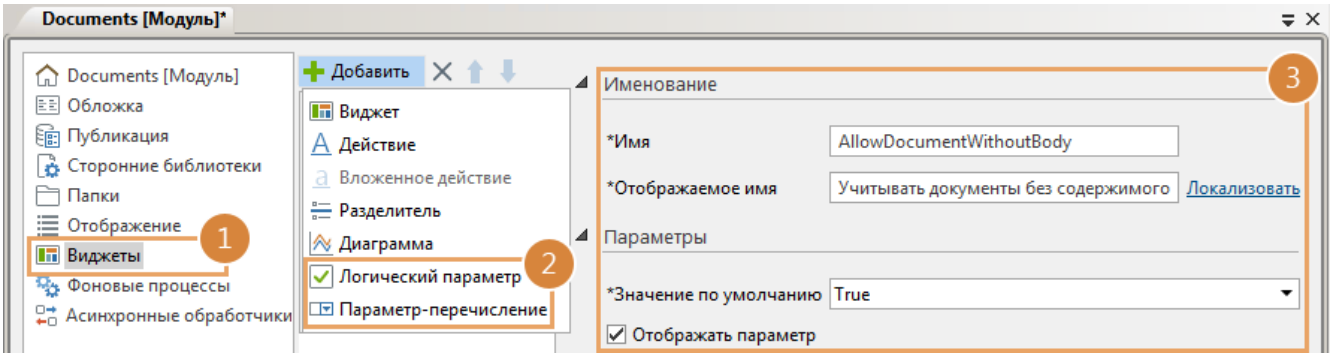
У каждого виджета есть предопределенные параметры ***Заголовок** и ***Цвет**, которые создаются автоматически. Остальные параметры разрабатываются в редакторе модуля в узле «Виджеты». Разрабатываемые параметры бывают двух типов:

- логические. Принимают значения **True** и **False**;
- перечисления. Возможные значения задаются разработчиком.



Создание параметров виджета

1. В редакторе модуля перейдите в узел «Виджеты».



2. В списке элементов выделите виджет, для которого нужно добавить параметр. Нажмите на кнопку **Добавить** и в выпадающем списке выберите пункт **Логический параметр** или **Параметр перечисление**.

   ПРИМЕЧАНИЕ. Параметры в окне настройки параметров виджета отображаются в том порядке, в котором они расположены в списке элементов. Чтобы изменить порядок используйте кнопки ⬆ и ⬇ на панели инструментов.

3. В списке элементов выделите добавленный элемент и задайте настройки:

   ***Имя** параметра. Должно быть уникальным в рамках одного виджета, состоять из букв латинского алфавита и цифр.

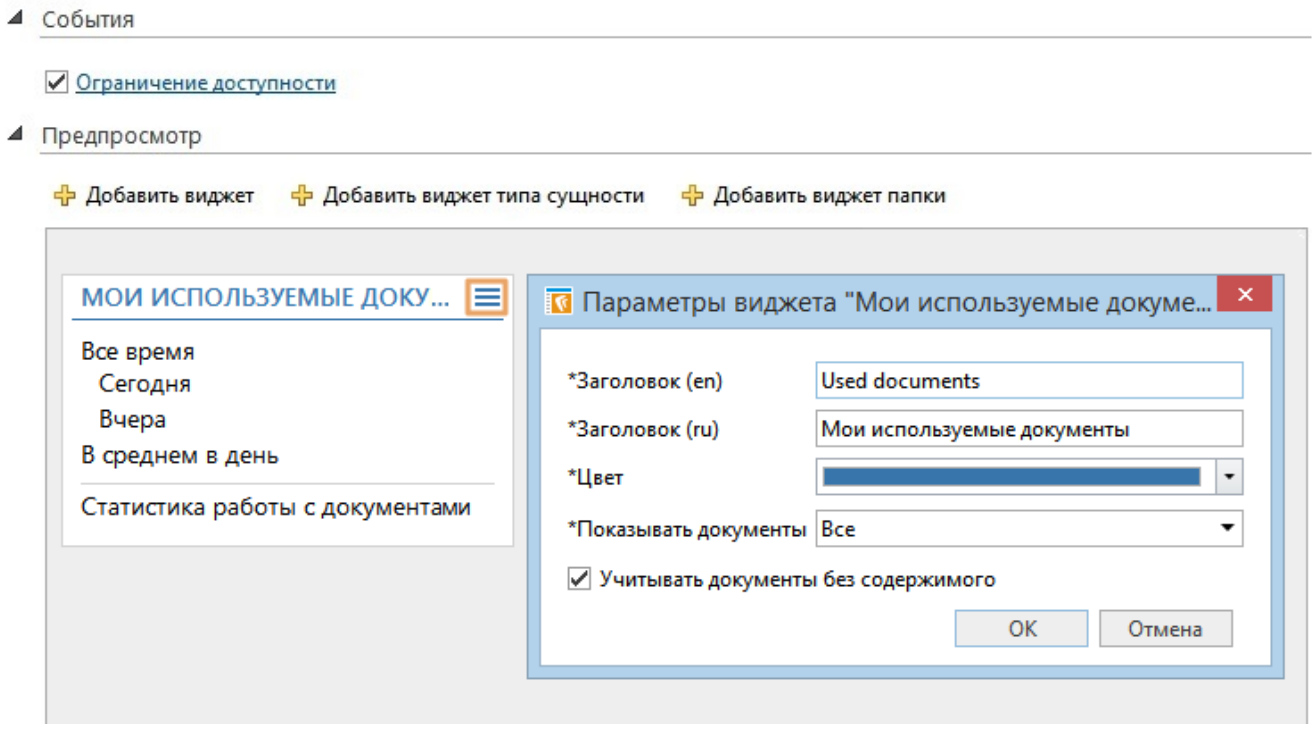
   ***Отображаемое имя**. Название параметра, которое будет отображаться в окне настройки параметров виджета в проводнике системы:

   ***Значение по умолчанию**. Значение параметра в проводнике системы по умолчанию. Возможные значения для логического параметра: **True** и **False**. Возможные значения для параметра перечисления выбираются из табличной части «Значения перечисления».

   Отображать параметр. Добавленный параметр отображается в настройках виджета. Снимите флажок, если необходимо скрыть параметр.

В результате для виджета будет добавлен параметр. После этого задайте программный код для изменения данных в действиях виджета в зависимости от значений параметра. Код задается отдельно для каждого действия в редакторе модуля в узле «Виджеты» в обработчиках событий.

СОВЕТ. Чтобы посмотреть, как будет выглядеть окно с новыми параметрами, в редакторе решения в узле «Наборы виджетов» добавьте в набор необходимый виджет. В области предпросмотра нажмите на кнопку **Параметры**. Откроется окно настройки параметров:



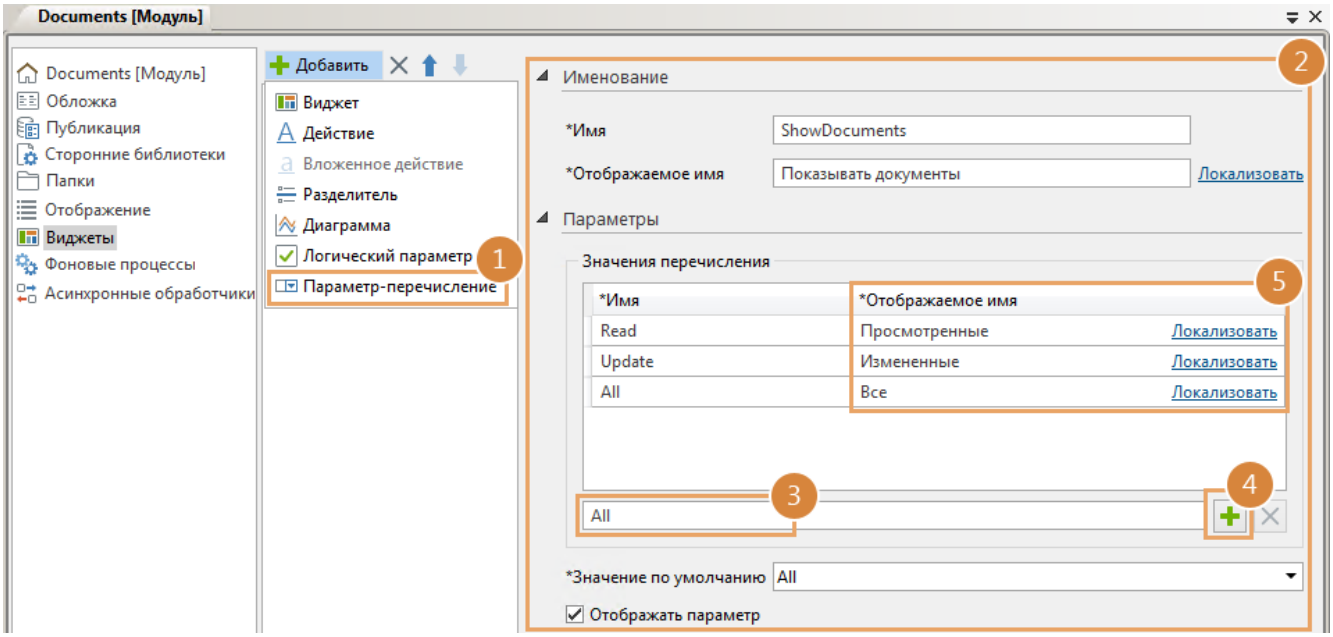
Пример добавления параметра-перечисления

Предположим, разрабатывается виджет «Мои используемые документы» с действиями «Все время», «Сегодня» и «Вчера». Необходимо, чтобы у пользователя была возможность выбирать, какие данные будут отображаться в действиях виджета:

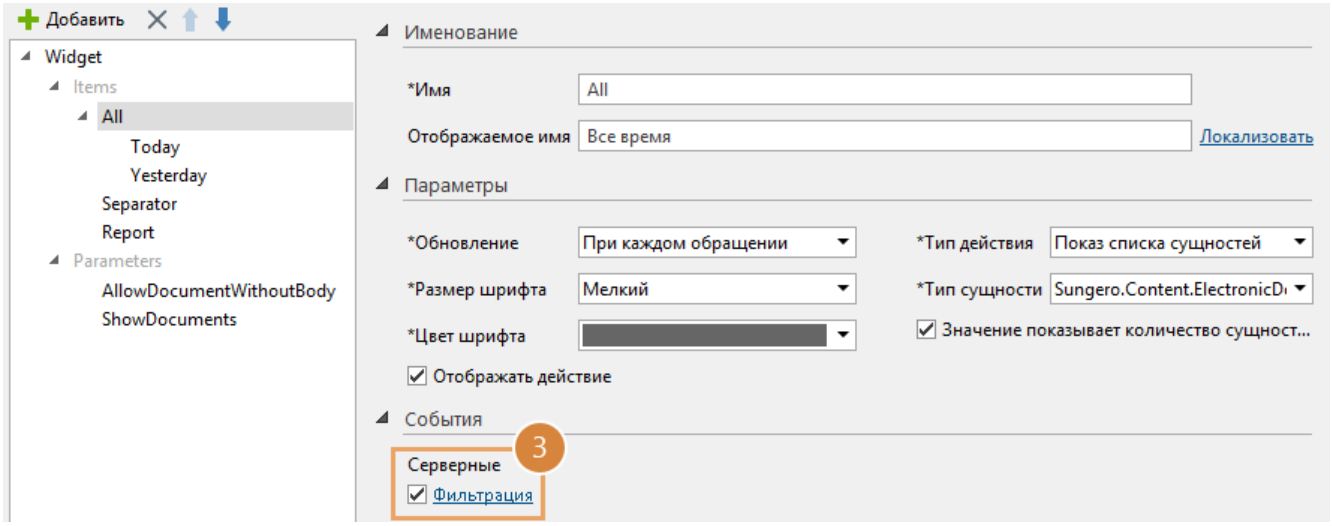
- только по документам, открытым на просмотр;
- только по документам, открытым на редактирование;
- по всем документам.

В этом случае добавьте для виджета параметр-перечисление с возможными значениями **Просмотренные**, **Измененные** и **Все**. Для этого:

1. В редакторе модуля перейдите в узел «Виджеты» и создайте параметр-перечисление.



2. В списке элементов выделите добавленный элемент и задайте настройки параметра.
3. Добавьте возможные значения параметра. Для этого в поле для ввода укажите имя значения.
4. Нажмите на кнопку . В табличной части будет создана новая запись. В колонке ***Имя** будет указанно введенное значение.
5. В колонке ***Отображаемое имя** введите имя значения, которое будет отображаться в проводнике системы. Для *локализации* отображаемого имени перейдите по ссылке **Локализовать**.
6. Чтобы данные в действиях виджета фильтровались в зависимости от значения параметра, задайте обработчик события **Фильтрация**. Для этого в списке элементов выделите элемент **Items** и выберите действие «Все время». Установите флажок **Фильтрация** и перейдите по ссылке. Откроется редактор программного кода.



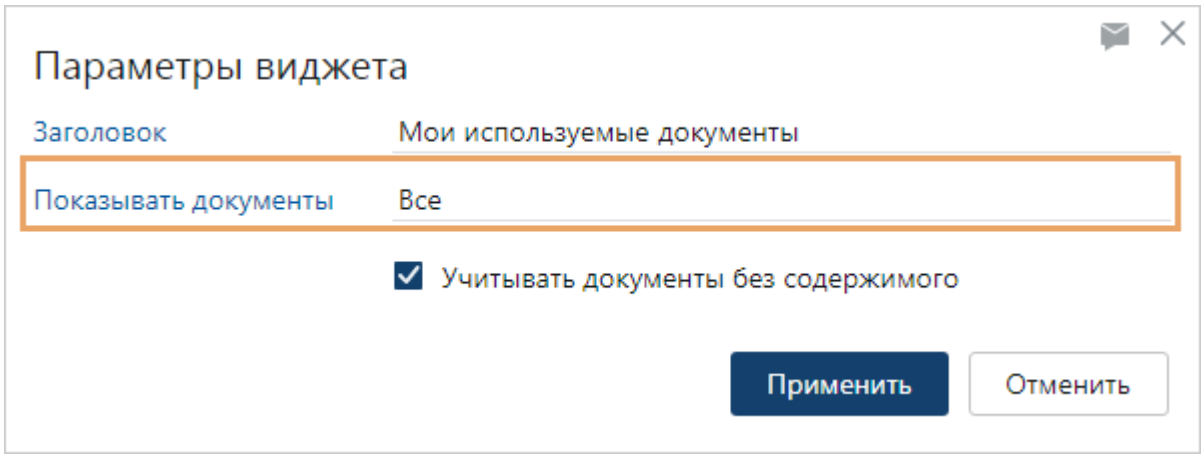
Аналогичным образом задайте обработчик события **Фильтрация** для действий «Сегодня» и «Вчера».

ПРИМЕЧАНИЕ. В зависимости от типа действия виджета для него могут быть доступны обработчики событий **Получение значения** и **Выполнение**.

Обработчик события «Фильтрация» для действия «Все время» с учетом параметра «Показывать документы»

```
   /// <summary>
/// Формирование списка документов, который отображается
/// при нажатии на действие "Все время".
/// </summary>
public virtual IQueryable<Sungero.Content.IElectronicDocument>
WidgetAllTimeFiltering(IQueryable<Sungero.Content.IElectronicDocument> query)
{
  // Если для параметра-перечисления "Показывать документы" установлено
  // значение "Просмотренные", то в виджете отображается информация
  // по документам, которые пользователь открывал на просмотр.
  if (_parameters.Show == Docflow.Widgets.Widget.Show.Read)
    query = query.WhereDocumentHistory(h => h.Action ==
Sungero.CoreEntities.History.Action.Read);
  // Если для параметра-перечисления "Показывать документы" установлено
  // значение "Измененные", то в виджете отображается информация
  // по документам, которые пользователь открывал на редактирование.
  if (_parameters.Show == Docflow.Widgets.Widget.Show.Update)
    query = query.WhereDocumentHistory(h => h.Action ==
Sungero.CoreEntities.History.Action.Update);
  // Если для параметра-перечисления "Показывать документы" установлено
  // значение "Все" и для логического параметра
  // "Учитывать документы без содержимого" установлено значение False,
  // то в виджете отображается информация по всем документам
  // которые открывал пользователь.
  return query
    .WhereDocumentHistory(h => Equals(Users.Current, h.User))
    .Where(d => IsOftenUsed(d));
}
```

В результате для виджета «Мои используемые документы» будет добавлен параметр-перечисление:

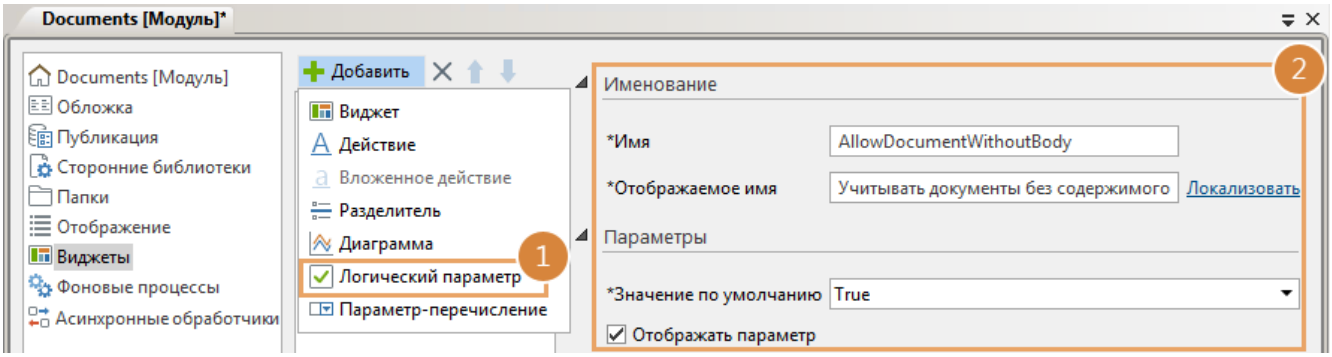


В зависимости от значения параметра, которое указал пользователь, в действиях виджета будут отображаться данные по всем документам, только по документам, которые были открыты на просмотр или на редактирование.

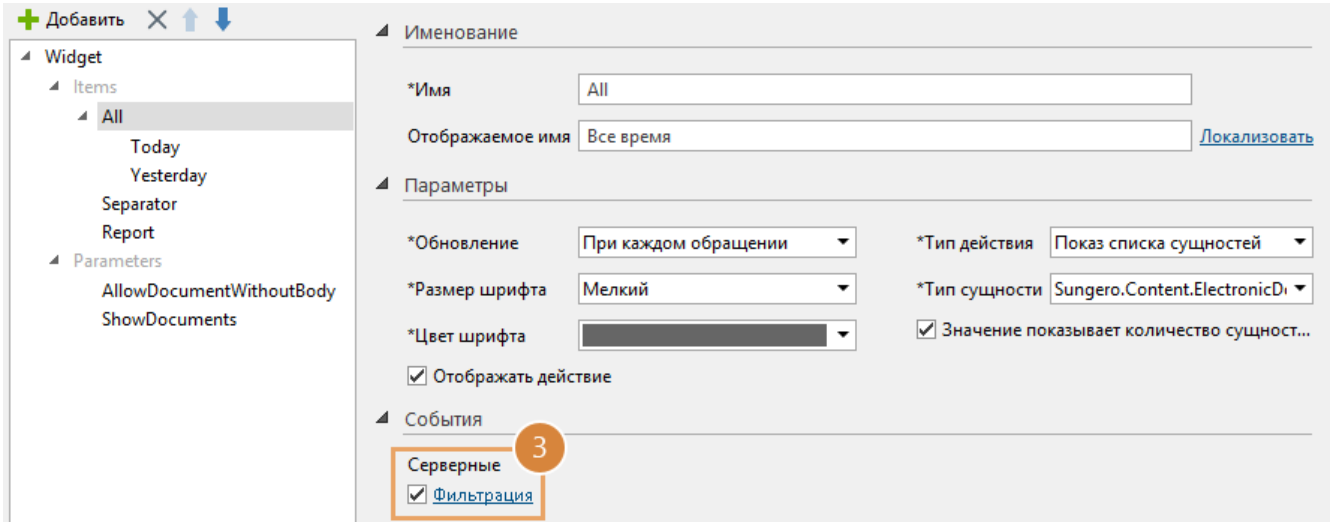
Пример добавления логического параметра

Предположим, разрабатывается виджет «Мои используемые документы» с действиями «Все время», «Сегодня» и «Вчера». Необходимо, чтобы у пользователя была возможность выбирать, какие данные будут отображаться в действиях виджета: по всем документам или только по документам, у которых есть содержимое. Для этого создайте логический параметр:

1. В редакторе модуля перейдите в узел «Виджеты» и создайте логический параметр.



2. В списке элементов выделите добавленный элемент и задайте настройки параметра.
3. Чтобы данные в действиях виджета фильтровались в зависимости от значения параметра, задайте обработчик события **Фильтрация**. Для этого в списке элементов выделите элемент **Items** в выберите действие «Все время». Установите флажок **Фильтрация** и перейдите по ссылке. Откроется редактор программного кода.



Аналогичным образом задайте обработчик события **Фильтрация** для действий «Сегодня» и «Вчера».

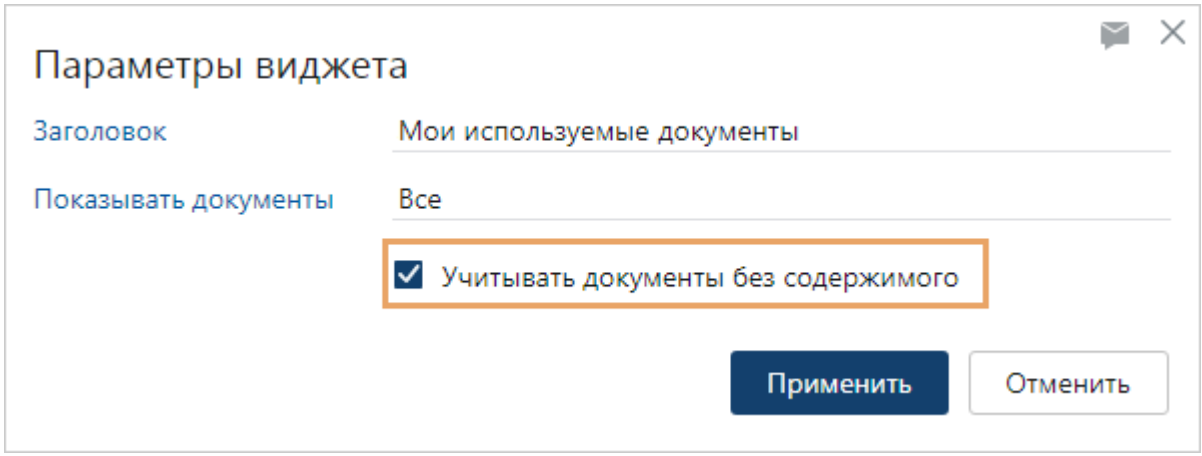
ПРИМЕЧАНИЕ. В зависимости от типа действия виджета для него могут быть доступны обработчики событий **Получение значения** и **Выполнение**.

Обработчик события «Фильтрация» для действия «Все время» с учетом параметра «Учитывать документы без содержимого»

```
   /// <summary>
/// Формирование списка документов, который отображается
/// при нажатии на действие "Все время".
/// </summary>
```

```
public virtual IQueryable<Sungero.Content.IElectronicDocument>
WidgetAllTimeFiltering(IQueryable<Sungero.Content.IElectronicDocument> query)
{
  // Если для логического параметра
  // "Учитывать документы без содержимого" установлено значение True,
  // то в виджете отображается информация только по документам,
  // у которых есть версии.
  if (!_parameters.AllowDocumentWithoutBody)
    query = query.Where(h => h.HasVersions);
  // Если для логического параметра
  // "Учитывать документы без содержимого" установлено значение False,
  // то в виджете отображается информация по всем документам
  // которые открывал пользователь.
  return query
    .WhereDocumentHistory(h => Equals(Users.Current, h.User))
    .Where(d => IsOftenUsed(d));
}
```

В результате для виджета «Мои используемые документы» будет добавлен логический параметр:



В зависимости от значения параметра, которое указал пользователь, в действиях виджета будут отображаться данные по всем документам или только по документам, у которых есть содержимое.

Фоновые процессы

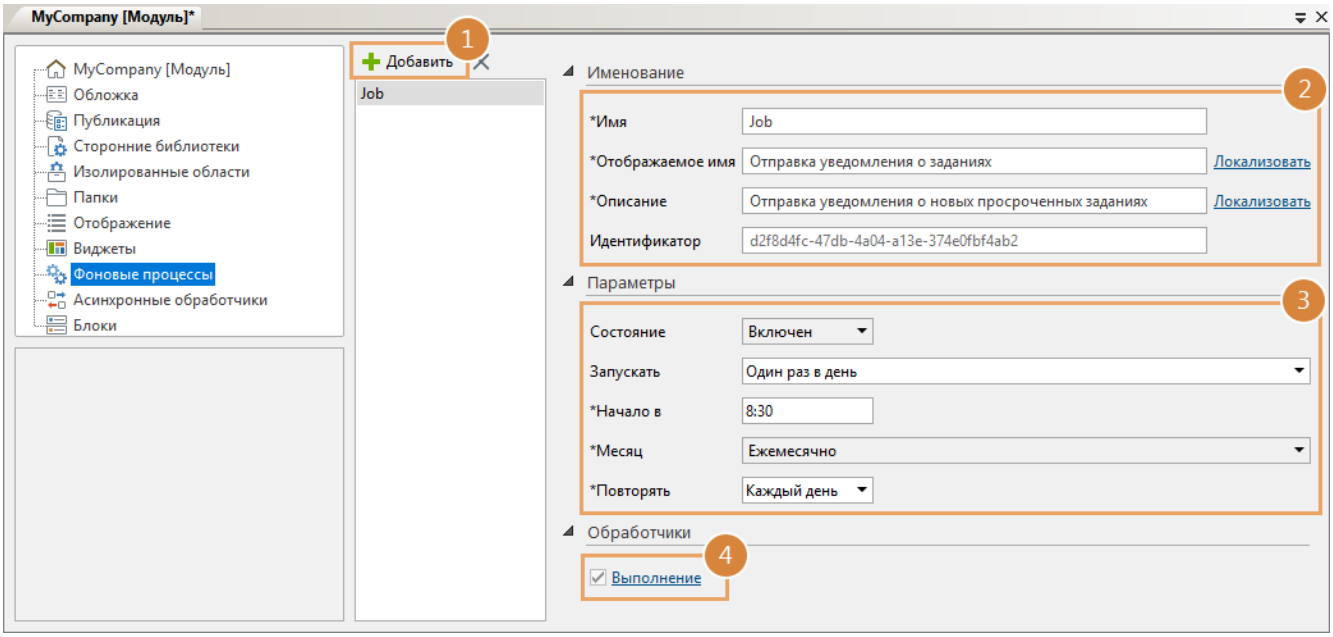
Фоновые процессы используются для выполнения работ в Directum RX, которые запускаются по заданному расписанию. Например, используются для автоматической рассылки уведомлений о завершении сроков действия договоров или периодической синхронизации данных с сервисами обмена электронными юридически значимыми документами.

Фоновые процессы настраиваются в редакторе модуля в узле «Фоновые процессы». Вы можете изменять процессы стандартного решения Directum RX, отключать и создавать свои.

ПРИМЕЧАНИЕ. Настройки фонового процесса, заданные в среде разработки, используются в Directum RX по умолчанию. При необходимости администратор сможет изменить описание процесса и расписание запуска, а также отключить его. Таким образом если администратор отключил процесс в списке «Фоновые процессы», то при запуске из программного кода процесс не выполнится.

Создание фонового процесса

1. Откройте редактор модуля, в котором будет создан процесс. В узле «Фоновые процессы» нажмите на кнопку **Добавить**.



2. В списке элементов выберите добавленный процесс и задайте настройки:

   ***Имя** фонового процесса. Должно быть уникальным в рамках одного модуля, состоять из букв латинского алфавита и цифр.

   ***Отображаемое имя**. Название фонового процесса, которое будет отображаться в Directum RX.

   Описание. Назначение фонового процесса. Для *локализации* отображаемого имени и описания перейдите по ссылке **Локализовать**.

   Идентификатор фонового процесса.

3. Настройте расписание запуска фонового процесса.

4. Задайте обработчик события «Выполнение» для выполнения логики фонового процесса.

   Все вычисления фоновых процессов выполняются на сервисе асинхронных событий. По умолчанию сервис не оборачивает вызовы обработчика в транзакцию, поэтому это можно сделать вручную с помощью класса Transactions.

   При возникновении ошибок в фоновом процессе транзакция позволяет откатить до исходного состояния все вычисления, которые выполняются в ней. Таким образом, ошибки в транзакции не влияют на выполнение всего фонового процесса. Например, можно сделать так, чтобы ошибки обработки одной сущности не влияли на обработку последующих сущностей.

   Пример. Фоновый процесс для синхронизации абонентских ящиков

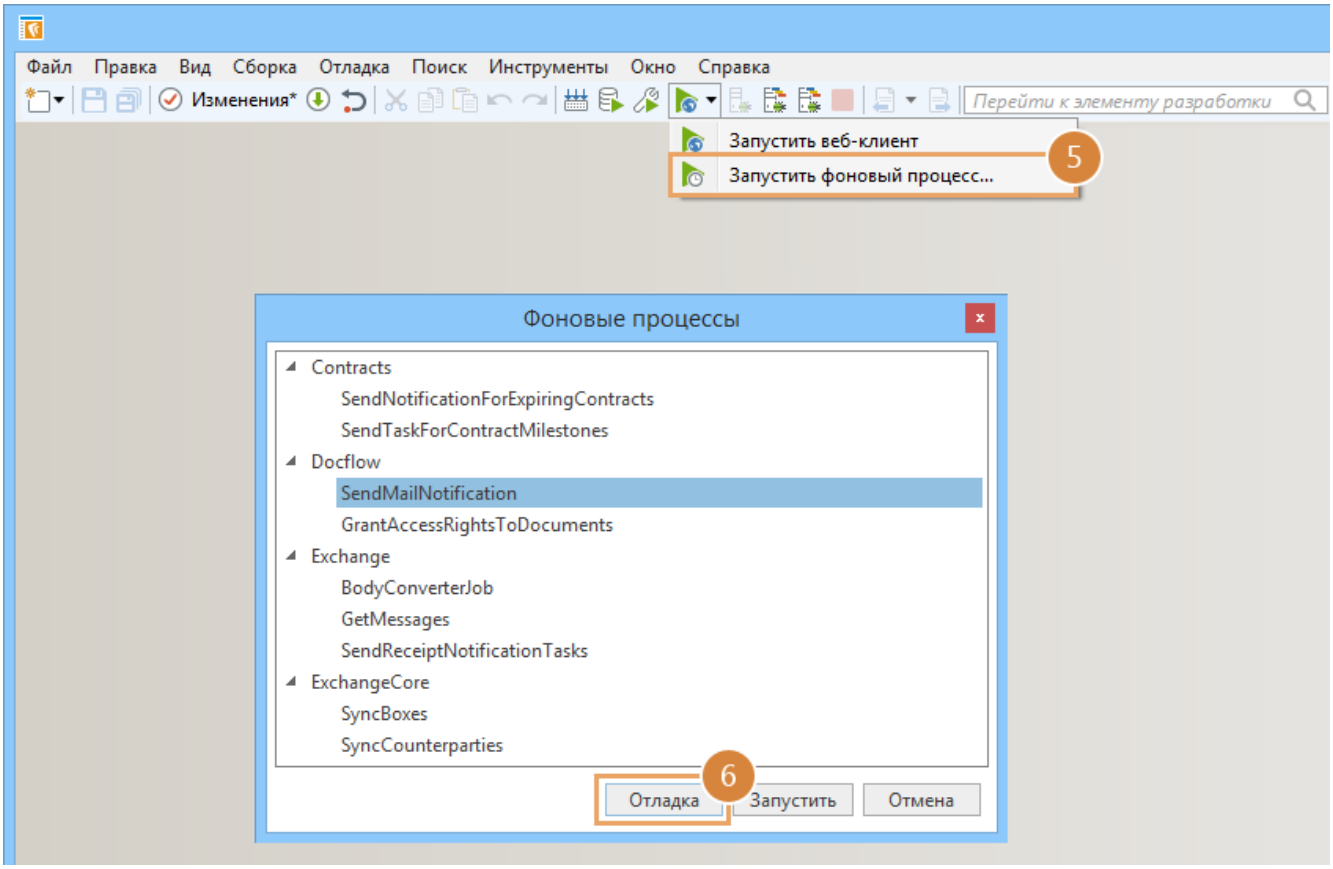
```
/// </summary>
  public virtual void SyncBoxes()
  {
    var boxes = BusinessUnitBoxes.GetAll(b => Equals(b.ConnectionStatus,
        ExchangeCore.BusinessUnitBox.ConnectionStatus.Connected) ||
        Equals(b.ConnectionStatus, ExchangeCore.BusinessUnitBox.
        ConnectionStatus.Error)).ToList();
```

```
    foreach (var box in boxes)
    {
      // Transactions.Execute() – метод для выполнения функции в транзакции.
      // Если синхронизация абонентского ящика завершится с ошибками,
      // то синхронизация следующих абонентских ящиков продолжится.
      Transactions.Execute(() =>
      {
          Functions.BusinessUnitBox.SyncBoxStatus(box);

          if (box.ConnectionStatus == ExchangeCore.BusinessUnitBox.
              ConnectionStatus.Connected)
          Functions.BusinessUnitBox.
             UpdateExchangeServiceCertificates(box);
      });
    }
}
```

ПРИМЕЧАНИЕ. Для фонового процесса можно настроить запуск в Directum RX вне заданного расписания. Например, когда нужно запустить процесс после выполнения определенного события. Для этого добавьте серверную функцию модуля, которая будет запускать фоновый процесс:

```
Jobs.<Имя фонового процесса>.Enqueue();
```

5. При необходимости выполните отладку фонового процесса. Для этого на панели инструментов в выпадающем списке **Запустить клиент** выберите пункт **Запустить фоновый процесс...**.



6. В открывшемся окне выберите фоновый процесс и нажмите на кнопку **Отладка**.

   Чтобы проверить выполнение фонового процесса в системе, в открывшемся окне нажмите на кнопку **Запустить**.

Изменение фонового процесса

Чтобы изменить фоновый процесс стандартного решения Directum RX, перекройте модуль, к которому он относится. В перекрытии модуля вы сможете изменить отображаемое имя, описание фонового процесса, расписание запуска, обработчик события **Выполнение**. Также можете отключить фоновый процесс.

В результате после публикации изменений в Directum RX:

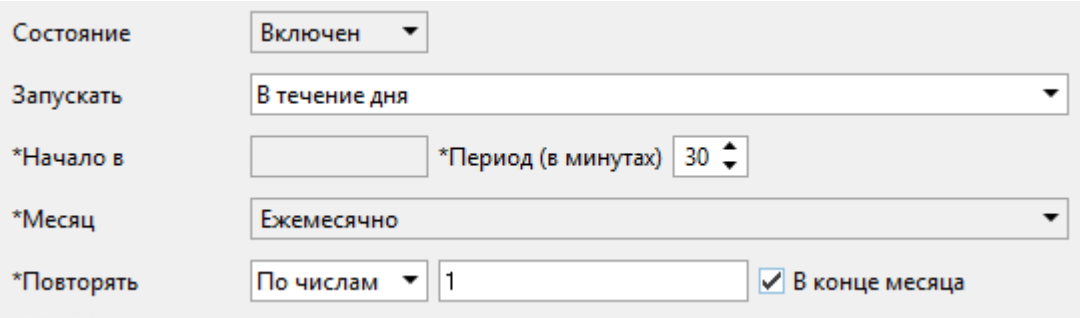
- обновится отображаемое имя фонового процесса;
- обновится расписание запуска фонового процесса, если администратор его не изменял.

  Если расписание в системе изменено, то оно не обновится. При этом администратор сможет восстановить настройки расписания по умолчанию;
- если фоновый процесс отключен администратором, то отображаемое имя и расписание обновятся, но сам процесс останется отключенным;
- если фоновый процесс отключен в среде разработки, то после публикации он больше не будет отображаться и запускаться в Directum RX.

ПРИМЕЧАНИЕ. Описание фонового процесса синхронизируется в Directum RX только один раз при публикации. Если в дальнейшем изменить описание в среде разработки, то в Directum RX оно не изменится.

Настройка расписания фонового процесса

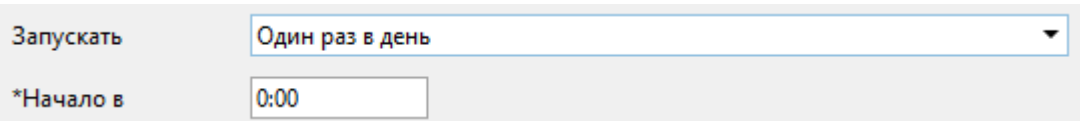
Чтобы настроить расписание запуска фонового процесса, укажите параметры:



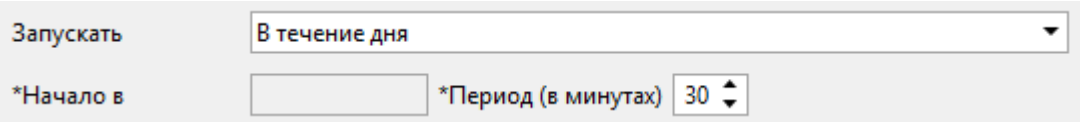
Состояние фонового процесса. Возможные значения: «Включен», «Отключен».

Запускать. Период запуска фонового процесса. Возможные значения:

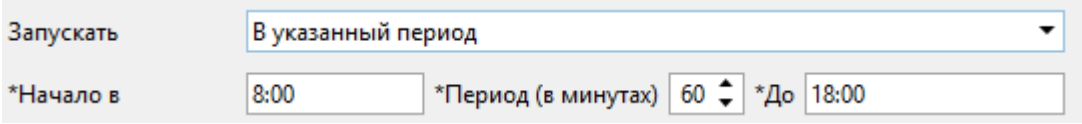
- «Один раз в день». Процесс запускается один раз в день в определенное время, которое задается в поле ***Начало в**. Например, с 6.00 один раз в день:



- «В течение дня». Процесс запускается несколько раз в течение дня через промежуток времени, заданный в поле ***Период (в минутах)**. Например, каждые 30 минут в течение дня:



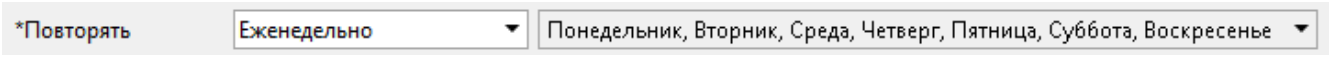
- «В указанный период». Процесс запускается в период, который указан в полях ***Начало** в и ***До**. Процесс выполнится через промежуток времени, заданный в поле ***Период (в минутах)**. Например, с 8.00 до 18.00 каждый час:



Месяц, в котором должен выполняться процесс. Выберите необходимые месяцы из списка или установите флажок **Ежемесячно**.

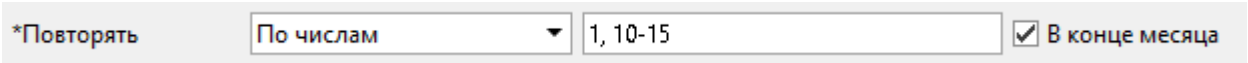
Повторять запуск процесса по заданному расписанию. Возможные значения:

- «Каждый день». Процесс запускается каждый день;
- «Еженедельно». Процесс запускается по выбранным дням недели:



- «По числам». Процесс запускается в указанные числа месяца. Числа перечисляются через запятую, если нужно указать промежуток чисел, то через дефис.

  Если нужно, чтобы процесс запускался в последний день месяца, установите флажок **В конце месяца**.

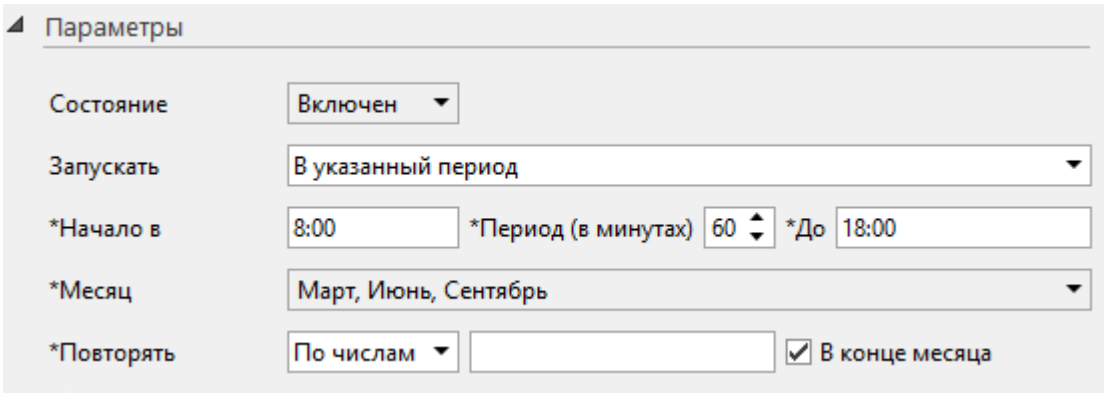


- «По рабочим дням». Процесс запускается по рабочим дням. Для правильного определения рабочих дней в Directum RX должен быть заполнен справочник «Календари рабочего времени».

Пример. Расписание для запуска фонового процесса

Необходимо, чтобы фоновый процесс запускался в период с 8.00 до 18.00 каждый час. Процесс должен повторяться в последний день каждого квартала.

В этом случае задайте следующие настройки:



Асинхронные обработчики

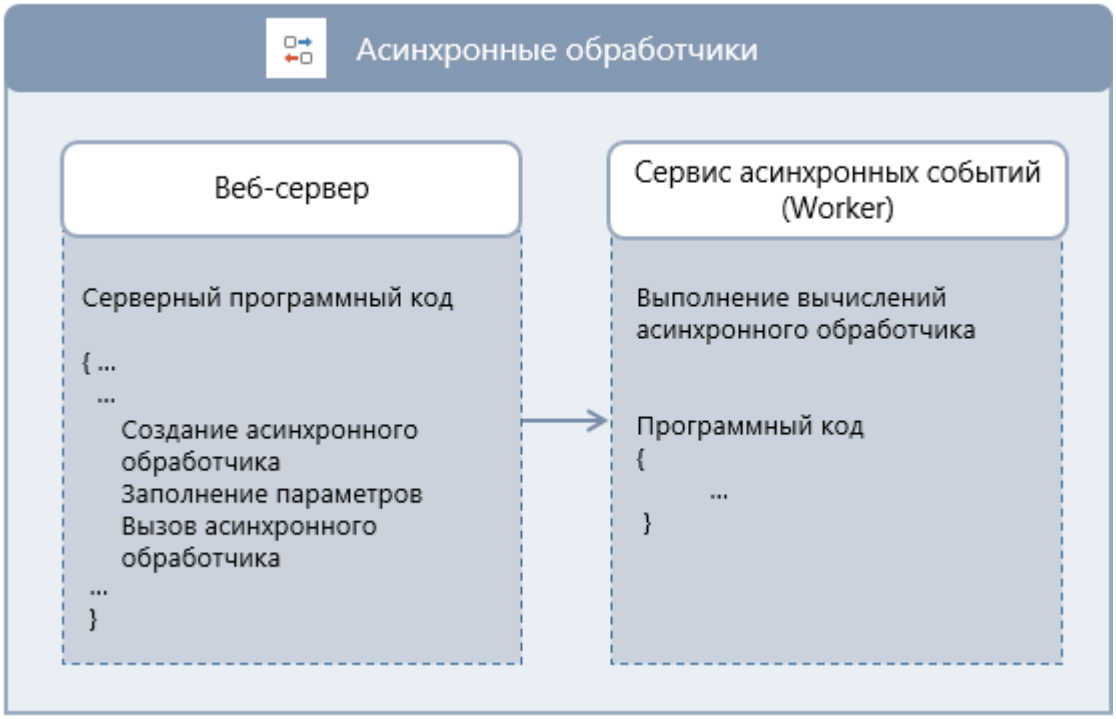
Асинхронные обработчики позволяют:

- вынести длительные, ресурсоемкие операции, не требующие немедленного возврата результата, с веб-сервера на отдельный сервис **Worker**;
- одновременно выполнить вычисления в несколько потоков (распараллеливание вычислений).

Благодаря этому можно ускорить выполнение тяжелых операций, снизить нагрузку на веб-сервер, а также ускорить отклик системы для конечного пользователя.

Например, в Directum RX асинхронные обработчики используются для автоматической выдачи прав на документы сотрудникам по настроенным правилам. В этом случае права выдаются на большое количество документов, результат не требуется получить сразу же после выполнения (в основном потоке), а сам процесс является ресурсоемким и его можно распараллелить на несколько потоков. Поэтому вычисления удобно выполнить на отдельном сервисе **Worker**.

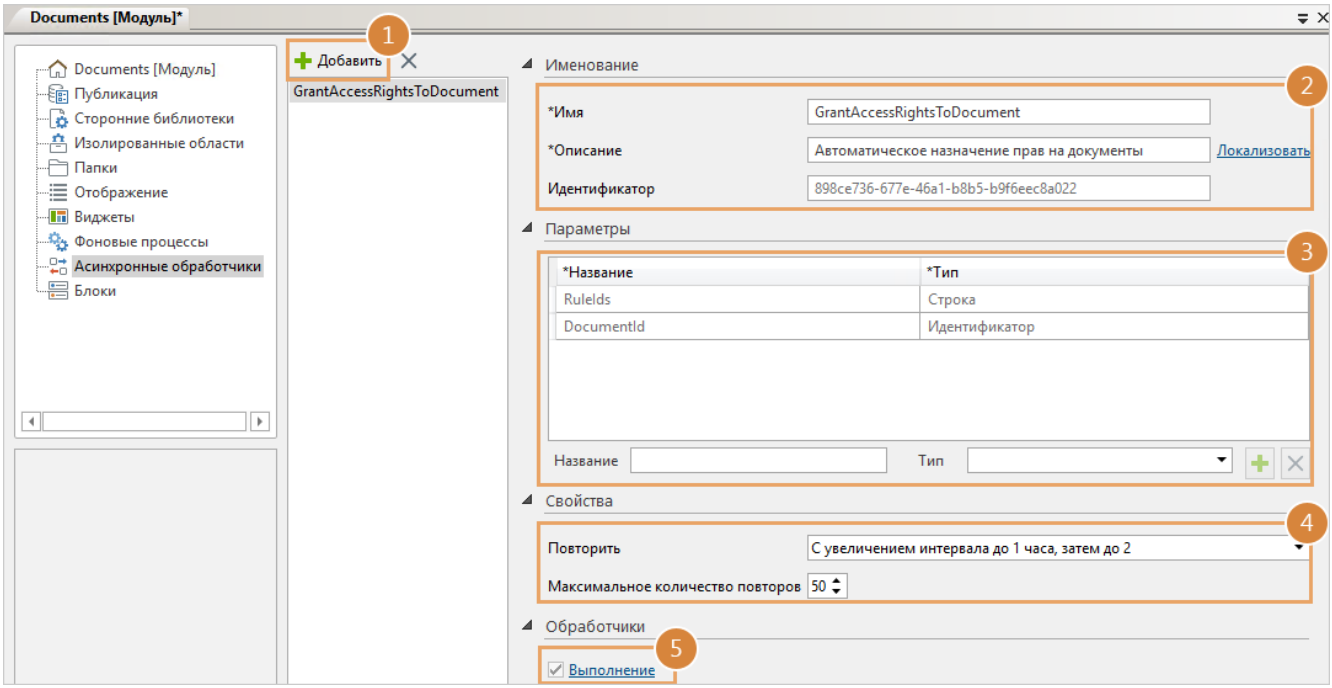
Схема работы



1. В нужном месте серверного кода прикладной разработчик создает асинхронный обработчик, заполняет параметры и вызывает его. При этом выполнение серверного кода продолжается.
2. Асинхронный обработчик отправляется на сервис **Worker**.
3. На сервисе **Worker**, то есть за рамками основного потока, асинхронный обработчик выполняет свои вычисления. Если во время выполнения асинхронного обработчика возникает ошибка, то он повторяется в соответствии с интервалом, заданным разработчиком при его создании.

Создание асинхронного обработчика

1. Откройте редактор модуля, в котором будет создан асинхронный обработчик. В узле «Асинхронные обработчики» нажмите на кнопку **Добавить**.



2. В списке элементов выберите добавленный асинхронный обработчик и задайте настройки:

   ***Имя** асинхронного обработчика. Должно быть уникальным в рамках одного модуля, может состоять из букв латинского алфавита и цифр.

   ***Описание**. Назначение асинхронного обработчика. Для *локализации* описания перейдите по ссылке **Локализовать**.

   Идентификатор асинхронного обработчика. Присваивается автоматически при создании. Используется при настройке в секции сервиса асинхронных событий конфигурационного файла.

3. Задайте параметры, которые будут передаваться из места вызова асинхронного обработчика в код его выполнения.

   Название параметра. Должно быть уникальным в рамках одного асинхронного обработчика, может состоять из букв латинского алфавита и цифр.

   Тип данных параметра. Возможные значения: **Строка**, **Целое**, **Идентификатор**, **Вещественное**, **Дата**, **Логическое**.

   ПРИМЕЧАНИЕ. Типы параметров асинхронных обработчиков осознанно ограничены простыми типами. Например, нельзя использовать тип **Бинарные данные**, так как передача файлов через массив байт приведет к большому потреблению ресурсов на сервисе **Worker**. С файлами надо работать только через потоки.

4. В свойстве **Повторить** укажите, как в случае возникновения ошибки будет происходить повторное выполнение асинхронного обработчика на сервисе **Worker**. Например, при ошибках, связанных с блокировками. Возможные значения:

   - **С увеличением интервала до 1 часа, затем до 2**. В течение первого часа обработчик выполняется с увеличивающимся по экспоненте интервалом между попытками повтора. После этого они выполняются каждый час. Через 12 часов

интервалы между повторами увеличиваются до 2 часов. По умолчанию асинхронный обработчик прекращает выполнение после 50 попыток повтора;

- **Через равные промежутки времени** с обязательным указанием интервала повтора в минутах. Если выбран этот пункт, то для указания интервала отобразится свойство ***Период (в минутах)**. По умолчанию асинхронный обработчик повторно выполняется каждые 15 минут и прекращает выполнение после 100 попыток.

При необходимости измените количество повторов в свойстве **Максимальное количество повторов**.

5. Задайте обработчик события **Выполнение** для добавления логики асинхронного обработчика.

Пример. Автоматическая выдача прав на документы

```
public                                virtual                                void
   GrantAccessRightsToDocument
   (Sungero
   .Docflow
   .Server.AsyncHandlerInvokeArgs.GrantAccessRightsToDocumentInvokeArgs
   args)
{
  long documentId = args.DocumentId;
  string stringRuleIds = args.RuleIds;

  var document = OfficialDocuments.GetAll(d => d.Id ==
documentId).FirstOrDefault();
  if (document == null)
  {
    args.RetryReason = "GrantAccessRightsToDocument: no document with id
{0}";
    return;
  }

  if (Locks.GetLockInfo(document).IsLockedByOther)
  {
    args.Retry = true;
    args.RetryReason = "GrantAccessRightsToDocument: document with id {0} is
blocked";
    return;
  }
  ...
}
```

Кроме того, в асинхронных обработчиках доступна работа с транзакциями, которые позволяют объединить несколько операций в один логический блок. При использовании транзакций сделайте программную проверку их выполнения. Если транзакция завершится с ошибкой, асинхронный обработчик повторно выполняться не будет.

6. Вызовите асинхронный обработчик в нужном месте серверного кода. Асинхронные обработчики создаются по формату <Имя модуля>.AsyncHandlers.<Имя асинхронного обработчика>.Create().

   Пример. Создание и вызов асинхронного обработчика (автоматическая выдача прав)

```
public override void AfterSave(Sungero.Domain.AfterSaveEventArgs e)
  {
    ...
  // Создать асинхронный обработчик GrantAccessRightsToDocument.
                                        var asyncRightsHandler                    =
  Docflow.AsyncHandlers.GrantAccessRightsToDocument.Create();
  // Заполнить параметры асинхронного обработчика.
    asyncRightsHandler.DocumentId = _obj.Id;
  // Вызвать асинхронный обработчик.
    asyncRightsHandler.ExecuteAsync();

    ...
  }
```

   При вызове асинхронного обработчика можно указать уведомления, которые нужно показывать пользователям при запуске и завершении выполнения обработчика.

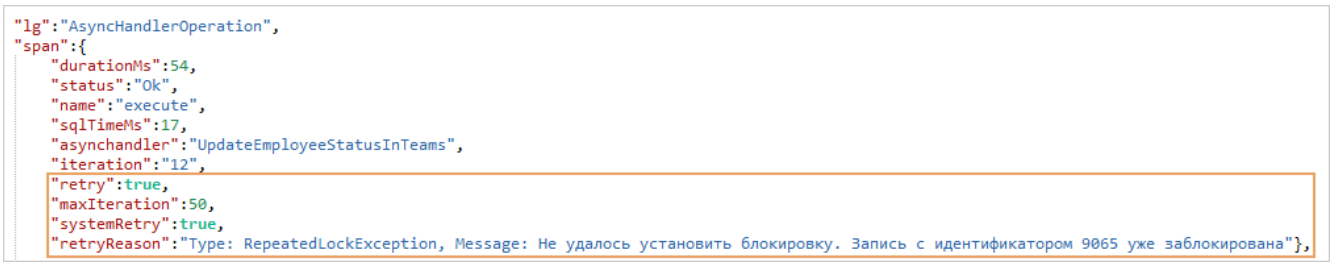
7. При необходимости выполните отладку кода асинхронного обработчика. Для этого на панели инструментов выберите **Отладить общий сервис**.

Аргументы события «Выполнение»

В событии асинхронного обработчика можно использовать аргументы:

Аргумент	Тип	Описание
args.MaxRetryCount	int	Максимальное количество повторов обработчика. Если лимит повторов превышен, то асинхронный обработчик не выполняется
args.NextRetryTime	Nullable<DateTime>	Дата и время следующего повтора обработчика
args.Retry	bool	Признак, который определяет, нужно ли повторно выполнять асинхронный обработчик после возникновения ошибки
args.RetryIteration	int	Счетчик повторов. С помощью параметра можно понять, какая попытка повтора сейчас выполняется
args.RetryReason	string	Произвольная информация, которая записывается в лог-файлы. Например, причина повтора обработчика, результат его выполнения

СОВЕТ. Используйте аргумент **RetryReason** вместо отдельного логирования с помощью метода DebugFormat(). Информация из этого аргумента фиксируется в лог-файле и ее можно будет использовать для анализа работы обработчиков с помощью решения «Мониторинг Directum RX»:



В системе для логгера **AsyncHandlerOperation** в атрибуте **span** записываются значения:

- **retry** – признак того, что выполнение обработчика отложено;
- **maxIteration** – максимальное количество итераций;
- **systemRetry** – признак того, что система отправила обработчик на переповтор;
- **retryReason** – информация, которую записывает система при возникновении необработанного исключения либо сообщение, заданное разработчиком в аргументе **RetryReason**.

ПРИМЕЧАНИЕ. Если при выполнении обработчика возникает необработанное исключение, то система отправляет его на переповтор. В поле **systemRetry** записывается значение **true**, а в поле **retryReason** система записывает класс и текст ошибки. В этой ситуации информация из аргумента **RetryReason** не записывается в лог-файле.

Изменение асинхронного обработчика

Чтобы изменить асинхронный обработчик стандартного решения Directum RX, перекройте модуль, к которому он относится. В перекрытии модуля вы сможете изменить интервал повтора, а также обработчик события **Выполнение**.

Типы сущностей

В проводнике системы пользователи работают с сущностями – объектами, вокруг которых строится процесс работы в системе. Например, сущностями являются конкретные документы, записи справочников, задачи и задания.

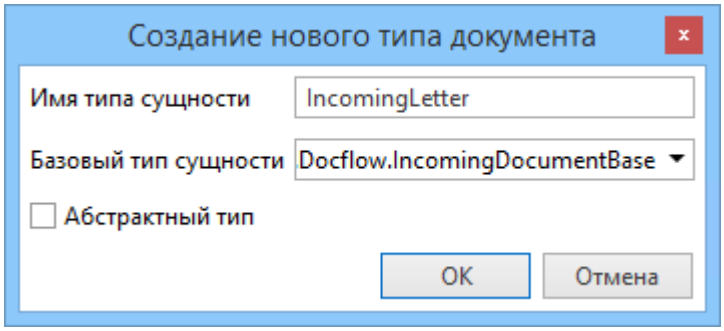
Каждая сущность относится к определенному типу сущности. Выделяют типы документов, справочников, задач, заданий, уведомлений и заданий на приемку. В своем решении можно создавать новые типы сущностей или перекрывать существующие. При создании типа сущности для него указывается *базовый тип сущности*, от которого наследуются свойства, действия, формы, ленты, операции авторизации, события и другое.

Например, в среде разработки создан тип документа «Договор» (Sungero.Contracts.Contract). При его создании в качестве базового типа сущности был указан «Базовый договор» (Sungero.Contracts.ContractBase). Пользователи работают в системе с сущностями типа документа «Договор» – конкретными договорами.

У каждого типа сущности есть свой редактор, в котором при необходимости можно задать настройки и переопределить элементы, полученные при наследовании и перекрытии.

Создание типа сущности

1. В дереве решений выделите модуль. Если создается задание, задание на приемку или уведомление, в дереве решений выделите задачу, для которой будет формироваться тип сущности.
2. В контекстном меню модуля или задачи последовательно выберите пункты **Создать** и **<Имя типа сущности>**, например, **Документ** или **Задание**.
3. В открывшемся окне заполните поля:



Имя типа сущности. Должно быть уникальным, состоять из букв латинского алфавита и цифр. Имя используется, например, при обращении к функциям.

Базовый тип сущности, от которого будут наследоваться свойства, действия, форма и т.д.

4. Установите флажок **Абстрактный тип**, если необходимо тип сущности сделать *базовым*. При создании других типов сущностей он будет доступен в выпадающем списке **Базовый тип сущности**.

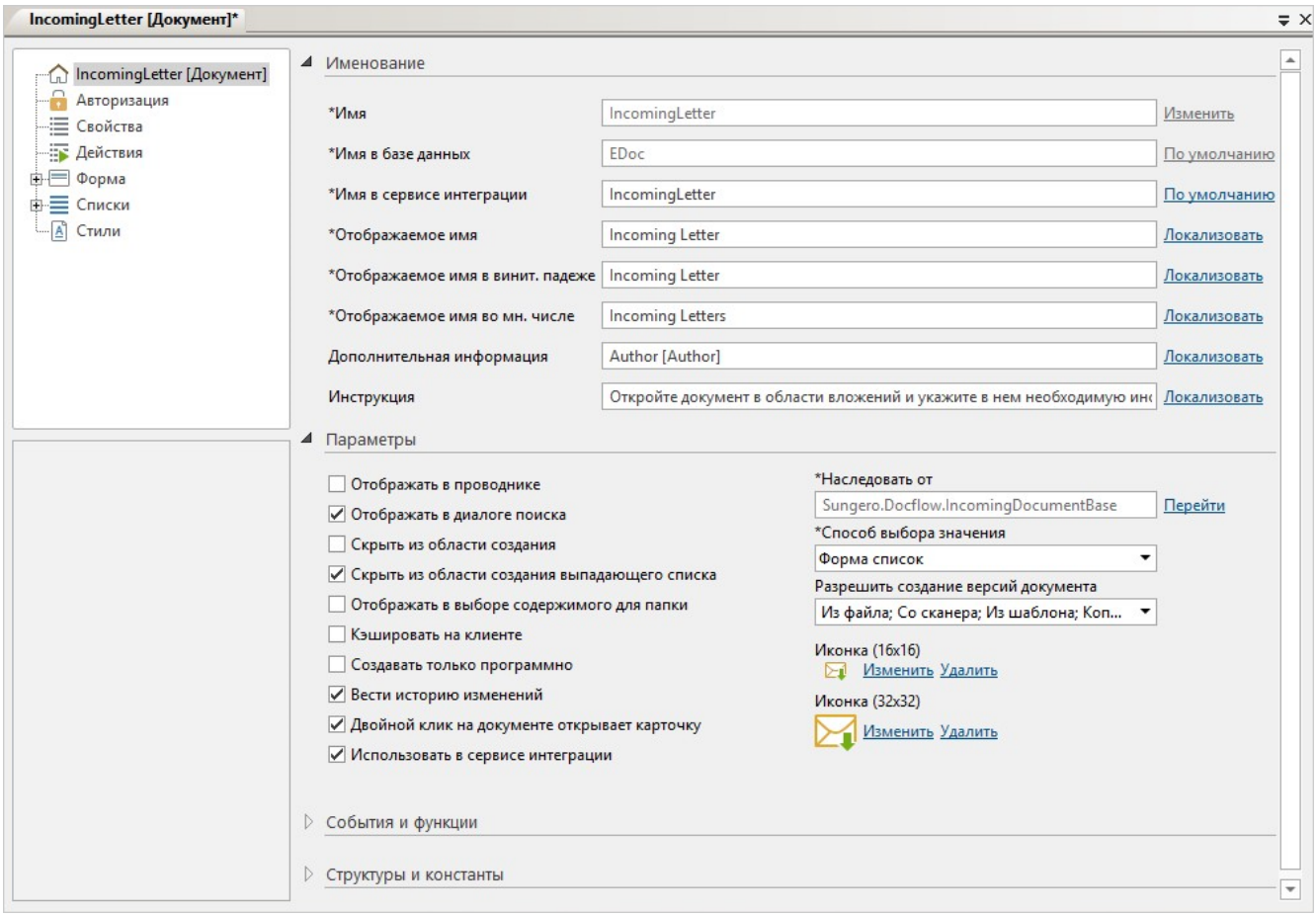
   ВАЖНО. Флажок доступен только для типов документов или справочников. Например, в среде разработки есть базовый тип справочника «Правило согласования» (ApprovalRuleBase). От него создано два наследника: «Правила согласования» (ApprovalRule) и «Правила согласования договоров» (ContractsApprovalRule). Пользователи в системе работают только с наследниками: конкретными правилами согласования.
5. Нажмите на кнопку **ОК**. Тип сущности появится в дереве решений, и откроется его редактор.
6. Задайте необходимые параметры в редакторе типа сущности.
7. Добавьте функции и обработчики событий.

   ПРИМЕЧАНИЕ. При необходимости добавьте обработчик события инициализации. Например, рекомендуется добавлять обработчик, чтобы создать сущности, для которых в параметрах типа сущности установлен флажок **Создавать только программно**, заполнить справочники значениями по умолчанию, выдать на тип сущности права доступа и т.д. Каждое действие рекомендуется писать в отдельной функции инициализации и вызывать в событии.

Редактор типа сущности

Чтобы задать или изменить настройки типа документа, справочника, задачи, задания, задания на приемку или уведомления, щелкните два раза левой кнопкой мыши по названию в дереве решений. Откроется редактор, в котором задаются параметры типа сущности, операции авторизации, свойства, действия, форма, списки, стили и другие настройки. В зависимости от типа сущности состав настроек может меняться.

При открытии редактора отображается узел с названием типа сущности. В узле задаются основные параметры, события и функции, структуры и константы.



Именование

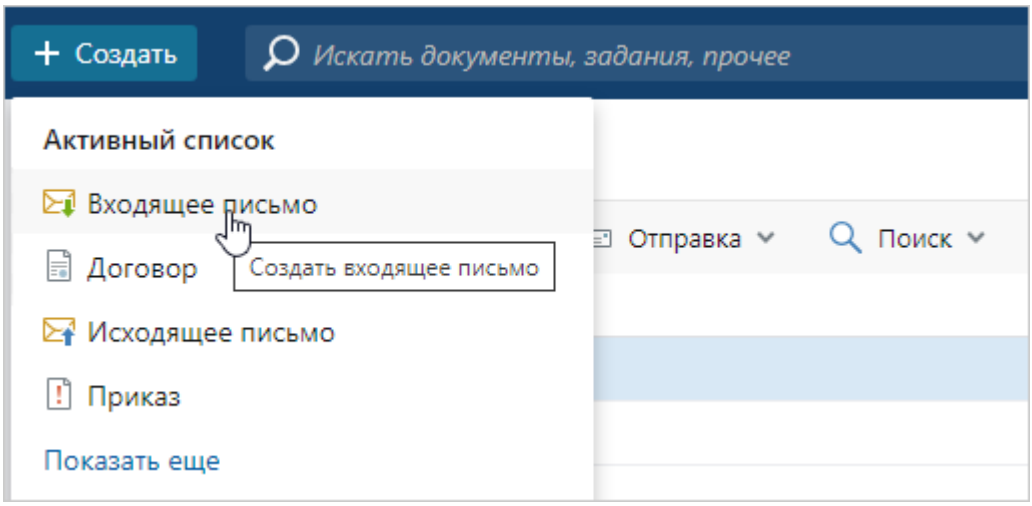
***Имя**. Указывается при создании типа сущности. Используется, например, при обращении к сущностям в программном коде. Имя должно быть уникальным в рамках одного модуля, состоять из букв латинского алфавита и цифр. При необходимости имя можно изменить. Для этого сохраните все изменения в редакторе, перейдите по ссылке **Изменить** и задайте новое имя. После изменения имени доработайте программный код, в котором оно используется.

***Имя в базе данных**. Используется при генерации таблиц для хранения данных типа сущности в базе данных. Заполняется автоматически. При необходимости имя в базе данных можно изменить. Для этого введите новое значение вручную. Имя должно быть уникальным в рамках одного модуля, состоять из букв латинского алфавита и цифр. Чтобы вернуть первоначальное значение, нажмите **По умолчанию**.

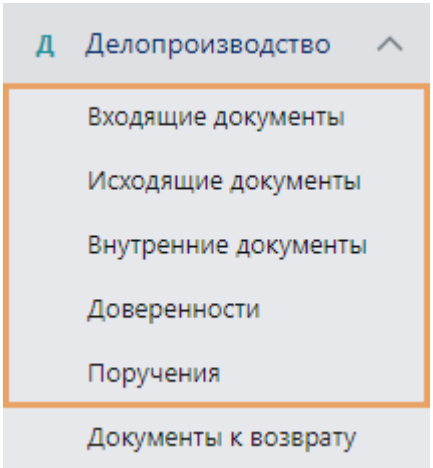
***Имя в сервисе интеграции**. Используется при генерации наборов данных **EntitySet** в метаданных OData для типов сущностей, которые используются при настройке интеграции Directum RX с внешними системами. Заполняется автоматически по шаблону <Имя модуля><Имя типа сущности>. При необходимости имя в сервисе интеграции можно изменить. Например, для ранее созданных типов сущностей с одинаковыми именами, из-за которых возникал конфликт в сервисе интеграции. Для этого введите новое значение вручную. Имя должно быть уникальным в рамках одного модуля, состоять из букв латинского алфавита и цифр. Чтобы вернуть первоначальное значение, нажмите **По умолчанию**.

***Отображаемое имя**. Имя типа сущности, которое отображается в проводнике системы.

***Отображаемое имя в винит. падеже**. Имя типа сущности, которое, например, отображается во всплывающей подсказке при наведении на значение из активного списка:

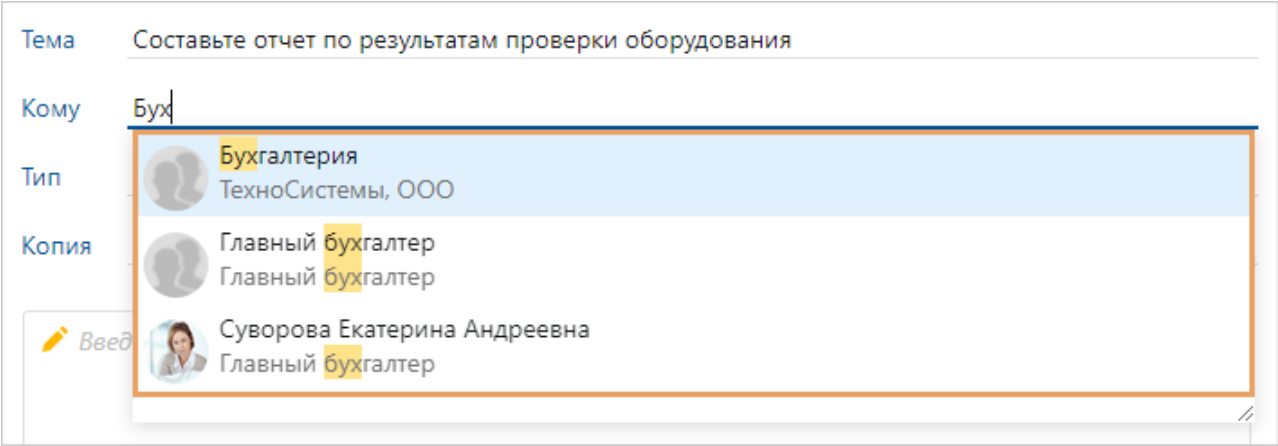


***Отображаемое имя во мн. числе**. Имя списка сущностей данного типа, которое отображается в списке папок проводника системы:



Для *локализации* отображаемых имен перейдите по ссылке **Локализовать**.

Дополнительная информация, которая отображается в веб-клиенте на панелях «Управление доступом», а также в выпадающих списках, когда пользователь заполняет значение свойства-ссылки в карточках сущностей, на панели фильтрации и в диалоговых окнах.

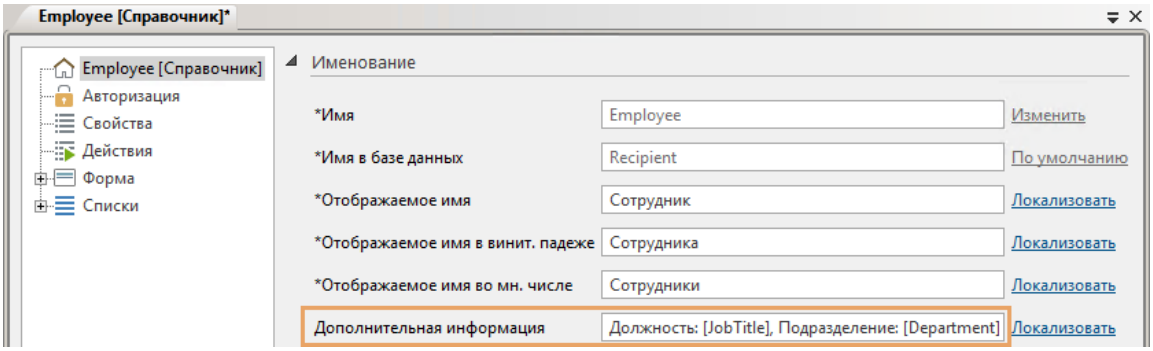


В качестве дополнительной информации указывается одно или несколько свойств типа сущности с типом Ссылка или Строка и Текст. Свойства указываются в квадратных скобках, простой текст задается в произвольном виде, например, <Текст> [<Свойство типа сущности>].

По значениям свойств, которые используются в дополнительной информации, пользователь может искать подходящее значение свойства-ссылки с помощью *like-ввода*.

Пример. Настройка дополнительной информации для справочника «Сотрудники»

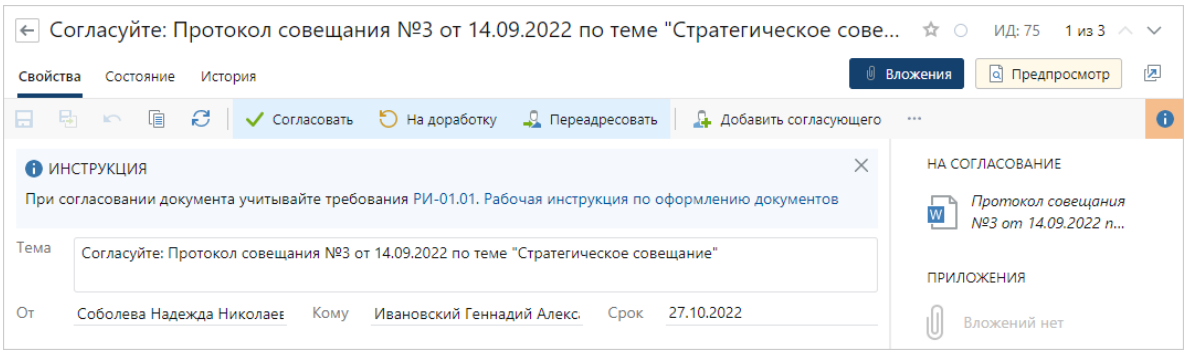
Предположим, необходимо, чтобы для сотрудника в качестве дополнительной информации отображались его должность и подразделение. Для этого в редакторе типа справочника **Сотрудники** (Employee) в поле **Дополнительная информация** введите значение «Должность: [JobTitle], Подразделение: [Department]»:



Для локализации простого текста в дополнительной информации перейдите по ссылке **Локализовать**.

В результате при выборе значений свойства-ссылки на справочник **Сотрудники** в выпадающем списке будет отображаться дополнительная информация. По названию должности или подразделения пользователь сможет искать подходящее значение.

Инструкция к сущности. Укажите информацию о работе с сущностью, например, порядок работы с заданием, подсказки по заполнению карточки или ссылки на внутренние регламенты компании. Добавленная инструкция отображается в веб-клиенте в карточке сущности под лентой:

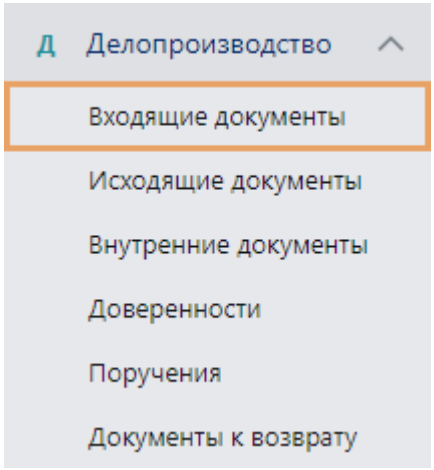


После чтения инструкцию можно скрыть по кнопке . Нажмите на подсказку на ленте, чтобы скрыть или вновь показать инструкцию, если нужно вернуться к ее чтению. В карточке сущности сохраняется состояние инструкции: скрыта или показана.

Иногда нужно менять инструкцию в зависимости от исполнителя задания, состояния сущности и т.п. Например, может потребоваться показывать разный текст на каждой стадии жизненного цикла документа. Для этого настройте инструкцию через аргумент e.Instruction в клиентском событии «Показ формы» (Showing).

Параметры

Отображать в проводнике. Тип сущности отображается в списке папок модуля в проводнике системы.



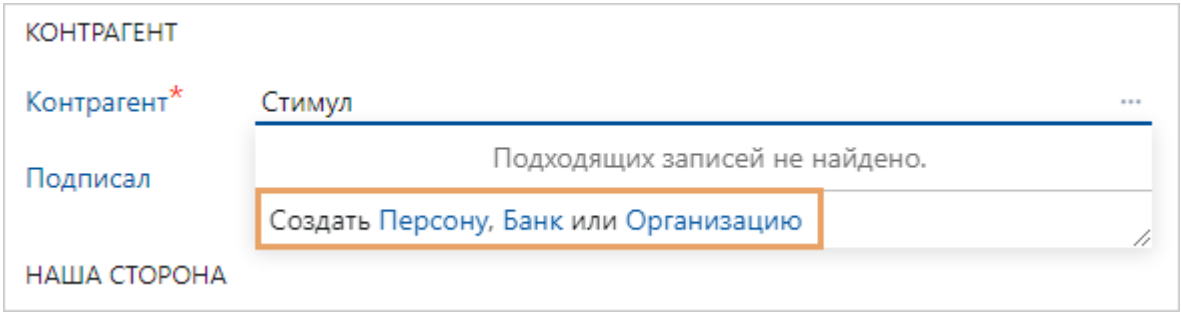
СОВЕТ. Снимите флажок, если тип сущности является системным и пользователи не будут работать со списком сущностей.

Отображать в диалоге поиска. Сущности данного типа можно найти по свойствам в проводнике системы. Для этого в узле «Свойства» для нужных свойств установите флажок **Отображать в диалоге поиска**. Например, для документов типа «Приложение к обращению» можно добавить поиск по обращению, к которому относятся документы. Рекомендуется использовать в поисках не более пяти свойств типа сущности. Флажок доступен только в редакторе типа справочника, документа и задачи.

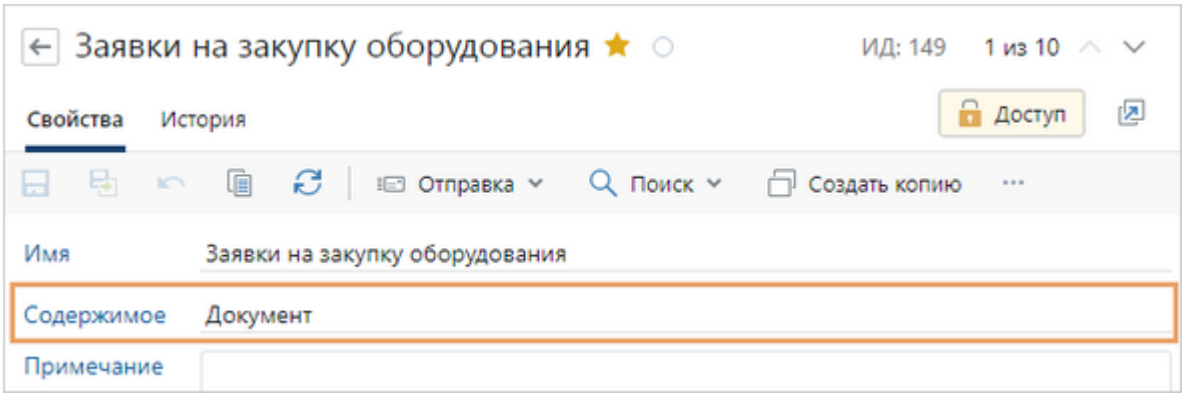
Скрыть из области создания. При нажатии на кнопку

нельзя выбрать тип сущности в группе «Активный список», а также в окне создания новой записи. При этом тип можно выбрать, если открыт список данного типа сущности. Подробнее см. руководство пользователя, раздел «Создание объектов».

Скрыть из области создания выпадающего списка. Тип сущности нельзя создать из выпадающего списка.



Отображать в выборе содержимого для папки. Тип сущности можно выбирать в качестве основного содержимого папок:



В зависимости от типа сущности определяется состав видимых колонок, кнопки быстрого создания, действия на ленте и в контекстном меню, которые будут доступны в содержимом папки.

Кэшировать на клиенте. Настройка является устаревшей и оставлена для совместимости.

Создавать только программно. Пользователи не смогут создавать сущности в клиентском приложении Directum RX. Например, флажок устанавливается для служебных справочников или задач, которые необходимо создавать автоматически при выполнении какого-либо действия. В *базовом решении* Directum RX флажок установлен для типа «Задача на продление срока» и справочника «Типы документов». Флажок доступен в редакторах типа справочника, документа и задачи.

СОВЕТ. Если флажок установлен, напишите код для программного создания сущностей в функции инициализации.

Вести историю изменений. Если флажок установлен, то в карточке сущности появится закладка «История». На ней фиксируется история работы с сущностью.

Использовать в сервисе интеграции. Этим признаком отмечаются типы сущностей, которые используются при интеграции Directum RX с внешними системами. Флажок установлен для всех типов сущностей из базового решения. Также при создании типа сущности флажок по умолчанию установлен для типов документов, задач и заданий. Особенности при:

- наследовании. Если в *базовом типе сущности* флажок установлен, то при создании наследников в дочерних типах флажок устанавливается автоматически. При этом в дочерних типах флажок можно снять.
- перекрытии. Если флажок установлен в типе сущности из *базового решения*, то после перекрытия этого типа флажок снять нельзя, но можно установить, если флажка изначально не было.

Если при модификации прикладной разработки HR Pro создается новый тип задачи, флажок **Использовать в сервисе интеграции** необходимо устанавливать всегда. Это нужно, чтобы задания и уведомления, связанные с задачей, отображались в личном кабинете.

СОВЕТ. При установке флажка проверьте, что в вашей разработке нет другого типа сущности с таким же именем, у которого уже установлен флажок **Использовать в сервисе интеграции**. Из-за особенностей *протокола OData*, когда флажок уставлен в двух типах сущностей с одинаковыми именами, после публикации *метаданные* генерируются только для одного случайного типа. Ко второму типу без метаданных не получится обратиться из внешней системы. Сообщение о возникшем конфликте появляется в лог-файле сервиса интеграции. Чтобы устранить ошибку, измените значение в поле **Имя в сервисе интеграции** для одного из типов сущностей.

Добавлять результаты выполнения в проводнике. Если флажок установлен, то в веб-клиенте в свойствах блоков типа «Задание» можно создавать результаты выполнения задания. Флажок доступен только в редакторе типа задания.

Запретить создавать версии схемы задачи в перекрытиях. Если флажок установлен, то в перекрытии типа задачи не получится изменить схему. Например, установите флажок, если схема задачи сложная и при доработке на новом *слое* ее можно сломать. Флажок доступен только в редакторе типа задачи. В перекрытиях параметр нельзя изменить.

Двойной клик на документе открывает карточку. Карточка документа открывается из вычисляемой папки или типизированного списка двойным щелчком левой кнопки мыши или нажатием клавиши ENTER. Рекомендуется устанавливать флажок, если в карточках документов данного типа много свойств, которые могут понадобиться пользователю при работе с документом. Например, свойства входящего письма с информацией от кого и кому был отправлен документ. Флажок доступен только в редакторе документа.

Действующая/Закрытая. Возможность закрытия записей справочников. Рекомендуется задавать параметр для всех типов справочников. У типа справочника появится свойство **Статус** с возможными значениями **Действующая** и **Закрытая**. В качестве значений свойств-ссылок на тип сущности доступны только действующие записи. Например, при заполнении в справочнике **Контакты** поля **Персона**, в списке для выбора отображаются только записи справочника **Персоны** со статусом **Действующая**. Флажок доступен только в редакторе типа справочника.

Только исполнители могут выполнять задания. Возможность для инициатора выполнить задание за исполнителя. Установите флажок, например, если требуется запретить инициатору задачи выполнять задания за исполнителей при согласовании официальных документов. Флажок аналогично работает в конкурентных заданиях: если он установлен, то инициатор не может взять задание в работу и выполнить за исполнителя.

При этом инициатор может выполнить задание, если замещает исполнителя, является руководителем подразделения или нашей организации. По умолчанию флажок не установлен. Флажок доступен только в редакторе типа задачи.

ПРИМЕЧАНИЕ. Ограничение на выполнение распространяется и на программную работу с заданием.

Создавать задание на приемку. По умолчанию задание на приемку не создается. Установите флажок, если после выполнения задачи нужно автоматически создавать задание на приемку. Флажок доступен только в редакторе типа задачи.

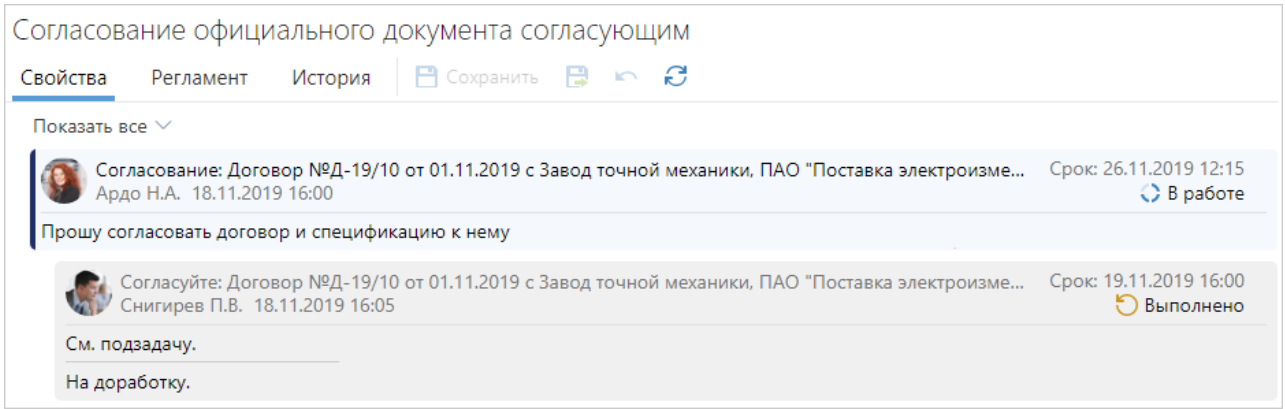
ПРИМЕЧАНИЕ. Определить логику выполнения действия можно через свойство Задание на приемку (NeedsReview), даже если флажок снят.

В переписке показывать тип вместо темы. По умолчанию в переписке отображается тема задачи. Она хранится в свойстве **Subject**. Как правило, тема включает в себя название документа, которое может быть достаточно длинным. Например: «Согласование: <Название документа>», «Распечатайте: <Название документа>» и т.д. Если флажок установлен, то вместо темы выводится тип задачи, задания или уведомления: «Задача на согласование по регламенту», «Печать официального документа» и т.д. Это упрощает чтение переписки. Флажок доступен только в редакторе типа задачи.

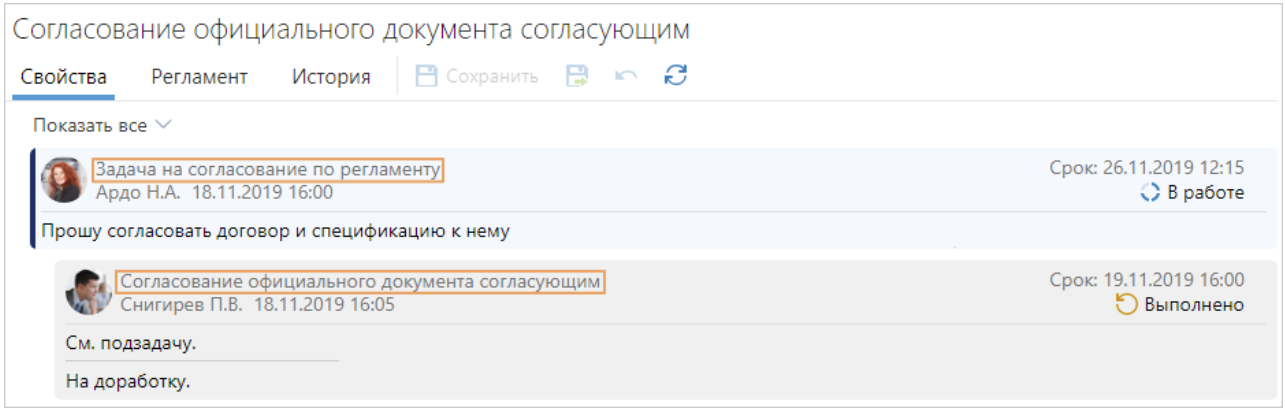
ПРИМЕЧАНИЕ. Тема изменяется только в переписке. В папках «Входящие» и «Исходящие» она остается прежней.

Пример:

- флажок не установлен:



- флажок установлен:

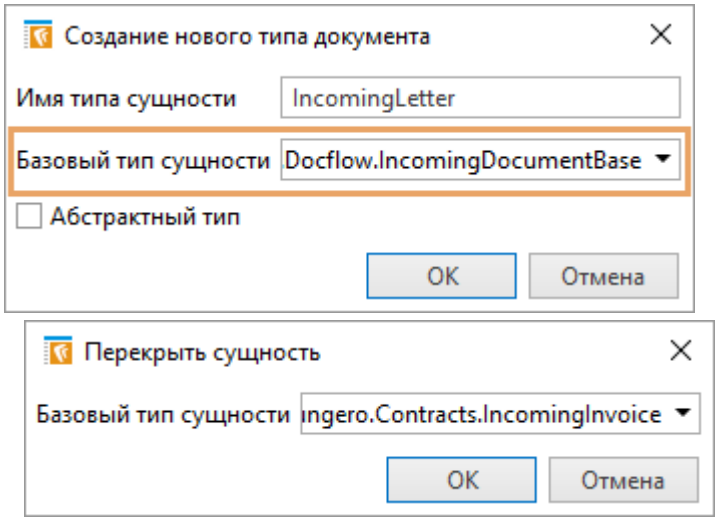


Чтобы в качестве темы задать собственное значение, используйте свойство **ThreadSubject** объектной модели. В этом случае параметр **В переписке показывать тип вместо темы** и свойство **Subject** не учитываются.

Схема настраивается в проводнике. Если флажок установлен, то для выбранного типа задачи в веб-клиенте нужно создать вариант процесса и настроить схему. В этом случае блоки для схемы создаются в среде разработки в узле «Блоки». В узле «Схема» отображается ссылка для открытия схемы в веб-клиенте. Если флажок снят, то схема задачи вместе с блоками создается в среде разработке в редакторе типа задачи. В этом случае узел «Схема» отображается, а узел «Блоки» скрыт. При создании нового типа задачи флажок установлен по умолчанию. Флажок доступен только в редакторе типа задачи.

ВАЖНО. Если флажок установлен, то для типа задачи должна быть хотя бы одна настроенная схема варианта процесса. Схему нужно настроить в *локальной системе*. Если это не сделать, в *продуктивной системе* задача не отправится. Подробнее см. раздел «Создание и публикация настроек по умолчанию».

***Наследовать от**. Имя типа сущности, от которого наследуются свойства, действия, форма, логика работы и другое. Указывается в поле **Базовый тип сущности** при создании и перекрытии типа сущности:



В редакторе типа сущности поле доступно только для просмотра.

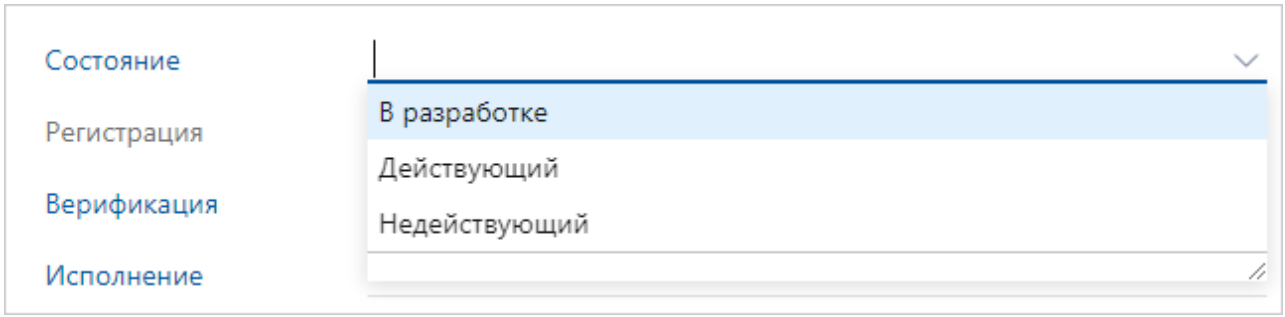
ПРИМЕЧАНИЕ. Если в среду разработки импортировано несколько *базовых решений* с одинаковыми перекрытиями, то при перекрытии этого же типа сущности на своем слое в

редакторе перекрытого типа в поле **Наследовать от** отображается последний тип сущности в цепочке перекрытий. Подробнее см. раздел «Объединение изменений в перекрытиях».

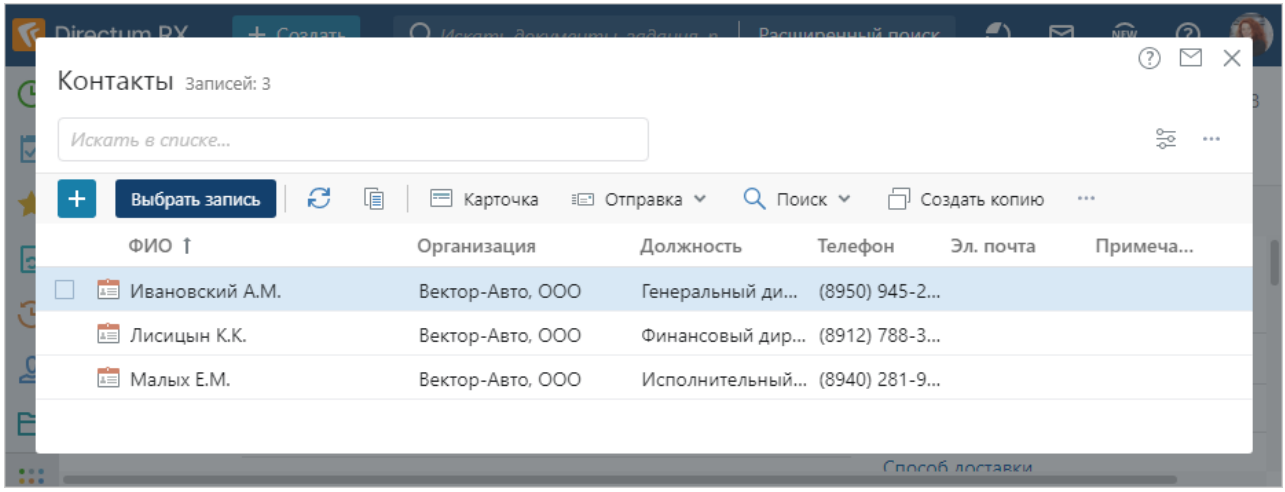
Иногда нужно посмотреть, как в типе сущности реализовано, например, событие. Если тип сущности унаследован, то в его редакторе не отображается код этого события. Чтобы открыть редактор базового типа сущности, нажмите на ссылку **Перейти**. Если базовый тип тоже является наследником, таким же способом можно перейти к его предку. Ссылка доступна, если базовый тип сущности находится на прикладном уровне.

***Способ выбора значения** свойства-ссылки на тип сущности. Возможные значения:

- **Выпадающий список**. Значения свойства-ссылки отображаются в виде выпадающего списка. Рекомендуется устанавливать, если сущностей данного типа немного (до 20), их количество не будет меняться и у типа сущности мало свойств.



- **Форма-список**. Значения свойства-ссылки отображаются в отдельном окне при нажатии на кнопку ···. Рекомендуется устанавливать, если сущностей данного типа много, их количество будет расти и у типа сущности много свойств.



Разрешить создание версий документа. Создание версии документа доступно одним или несколькими способами. Возможные значения:

- **Из файла**. При выборе этого способа можно создавать документ или его новую версию из файла, в том числе с помощью механизма Drag&Drop. Также документ можно импортировать;
- **Со сканера**. В этом случае документ или его версию можно создавать сканированием;
- **Из шаблона**. Разрешается создавать документ или его версию с использованием шаблона;

- **Копированием**. При выборе этого значения документ и его версию можно создавать на основе выбранной или последней версии, а также через действие **Создать копию** контекстного меню документа.

ПРИМЕЧАНИЕ. Определить логику доступности действия можно в обработчике события Выполнение/Возможность выполнения, даже если флажок снят.

Иконка. Иконка типа сущности, которая отображается в проводнике системы, например, в списке сущностей или в заголовке карточки сущности. Добавьте две иконки в формате SVG и в размерах 16x16 и 32x32. Иконки, используемые в *базовом решении*, можно найти в локальной папке исходных кодов. Расположение папки задается при установке системы. Например, в папке C:\DEVRX\Base\Sungero.Company\Sungero.Company.ClientBase\Icons можно найти файлы иконок модуля «Компания».

События и функции

Добавление обработчиков для:

- серверных событий. Код обработчиков выполняется на сервере, когда происходит соответствующее событие;
- клиентских событий. Код обработчиков выполняется на стороне клиентского приложения, когда происходит соответствующее событие.

Добавление функций типа сущности. Могут быть клиентскими, серверными и разделяемыми. Функции типа сущности предназначены для повторного использования кода в обработчиках событий сущности либо для вынесения вычислений на сторону сервера.

Структуры и константы

Добавление структур и констант, которые в последующем можно использовать в функциях и обработчиках событий.

Авторизация

Авторизация – это процесс проверки прав пользователя перед выполнением действий в системе. Права могут быть выданы пользователю, группе или роли, в которую он входит. Права настраивает администратор в списке «Права доступа на типы объектов» или на панели «Управление разрешениями» в карточке пользователя, группы или роли. Также администратор может создать новый тип прав, если существующих в системе недостаточно для решения задач.

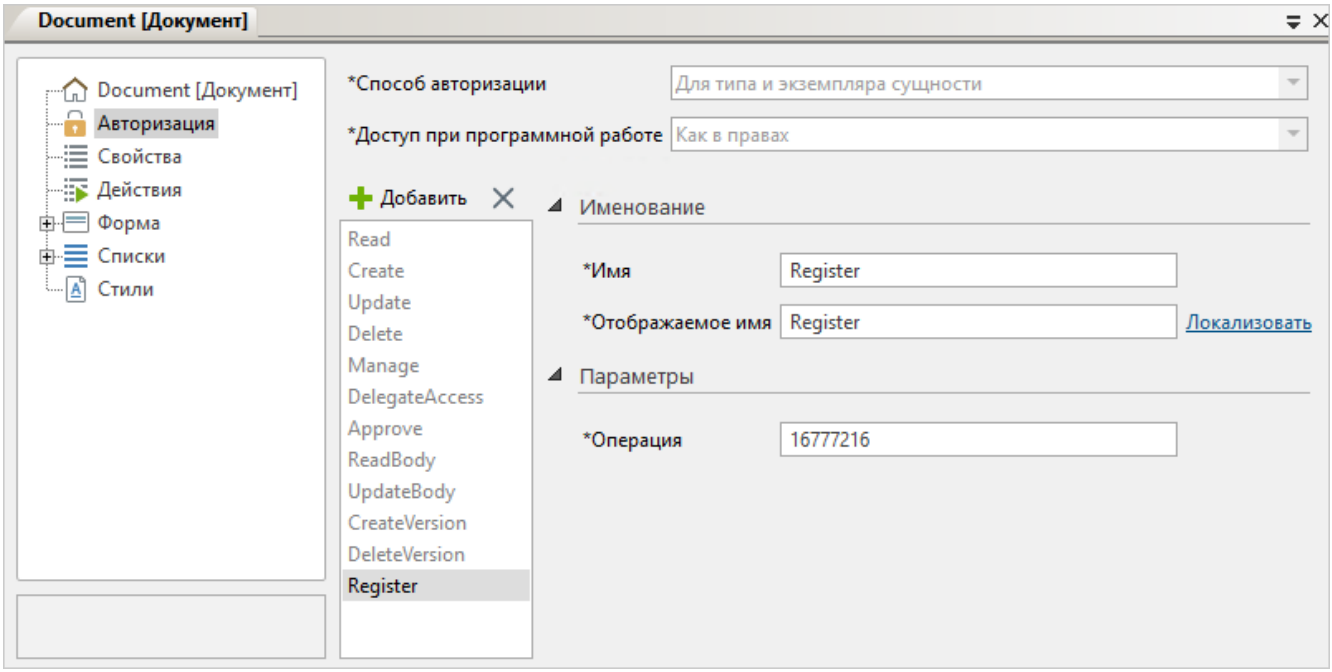
При необходимости создать новый тип прав может разработчик. При этом используются понятия:

- **объект** – то, над чем выполняется действие. Например, запись справочника **Сотрудники**, тип документа «Договор», функция регистрации или публикации разработки. С точки зрения объекта при подсчете результирующих прав для проверки учитываются:
  - права, которые выданы непосредственно на объект;
  - права на тип объекта;
  - права *базовых типов*, которые *наследуются* объектом;

- **субъект** – тот, кто выполняет действие. В Directum RX все субъекты прав являются наследниками IRecipient. Полный список всех субъектов прав в стандартном решении Directum RX: пользователи (IUser), сотрудники (IEmployee), роли (IRole), прикладные группы пользователей (наши организации, подразделения, группы регистрации). С точки зрения субъекта при подсчете результирующих прав для проверки учитываются:
  - права, выданные лично субъекту;
  - права, выданные всем группам, в которые входит субъект;
  - права, полученные по замещению и прочие специальные права;
- **операция авторизации** отображаются в проводнике системы в карточках записей списка **Типы прав**. Определяют, какие операции субъект может выполнять над объектом. Например, для записи справочника **Сотрудники** это может быть операция удаления, для типа документа «Договор» – операция создания нового документа, для функций регистрации или публикации разработки – операция выполнения.

Используйте узел «Авторизация», чтобы создать операции авторизации и задать способы проверки прав.

При создании типа сущности некоторые операции авторизации наследуются от *базового типа*. В списке элементов они выделены серым цветом. У наследуемых операций авторизации для изменения доступно только отображаемое имя.



***Способ авторизации**, по которому определяется, что для выполнения операций с сущностью у пользователя должны быть права:

- **Для экземпляра сущности**. Например, права на конкретный документ в системе;
- **Для типа сущности**. Например, права на договорные документы;
- **Для типа и экземпляра сущности**. Права на тип сущности и на каждую сущность в отдельности. Выбираются максимальные из выданных прав. Например, если для договорных документов выданы права на просмотр, а на конкретный договор – права на изменение, то у пользователя будут права на изменение договора.

***Доступ при программной работе**. Уровень доступа к данным, которые используются системой при выполнении программного кода. Возможные значения:

- **Как в правах** – набор доступных операций определяется правами пользователя. Значение по умолчанию;
- **Чтение** – доступна только операция чтения;
- **Полный** – доступны все операции.

Если выбрано значение **Чтение** или **Полный**, то система программно может читать или изменять данные, даже если пользователю явно не выданы права доступа на сущность. Например, при загрузке веб-клиента в системе проверяется, есть ли у пользователя замещения. Для этого программно выполняется операция «Чтение» из справочника **Замещения**, при этом прав на справочник у пользователя нет.

Доступность полей ***Способ авторизации** и ***Доступ при программной работе**, а также набор их значений зависит от типа сущности, для которого настраиваются операции авторизации:

Тип сущности	Доступность полей	*Способ авторизации	*Доступ при программной работе
Наследники Sungero.CoreEntities.DatabookEntry	Изменение	Доступны для выбора все возможные значения. Значение по умолчанию **Для типа сущности**	Доступны для выбора все возможные значения. Значение по умолчанию **Как в правах**
Наследники Sungero.CoreEntities.User	Просмотр	**Для типа сущности**	**Чтение**
Наследники Sungero.CoreEntities.Group	Просмотр	**Для типа и экземпляра сущности**	**Чтение**
Все типы документов, задач, заданий, уведомлений и заданий на приемку	Просмотр	**Для типа и экземпляра сущности**	**Как в правах**
Справочники, которые являются наследниками прикладных базовых типов	Просмотр	Такое же, как в базовом типе	Такое же, как в базовом типе

***Имя** операции авторизации. Должно быть уникальным в рамках типа сущности, состоять из букв латинского алфавита и цифр.

***Отображаемое имя**. Имя операции авторизации, которое будет отображаться в списке **Типы прав**. Для *локализации* перейдите по ссылке **Локализовать**.

***Операция**. Код операции авторизации. Должен быть уникальным. Поле заполняется автоматически. Рекомендуется оставить значение по умолчанию.

ВАЖНО. При разработке прав доступа важно не допускать ситуаций, когда:

- сотрудник не видит данные в системе, которые он должен видеть;
- сотрудник видит и может изменять данные в системе, которые видеть и менять ему не положено.

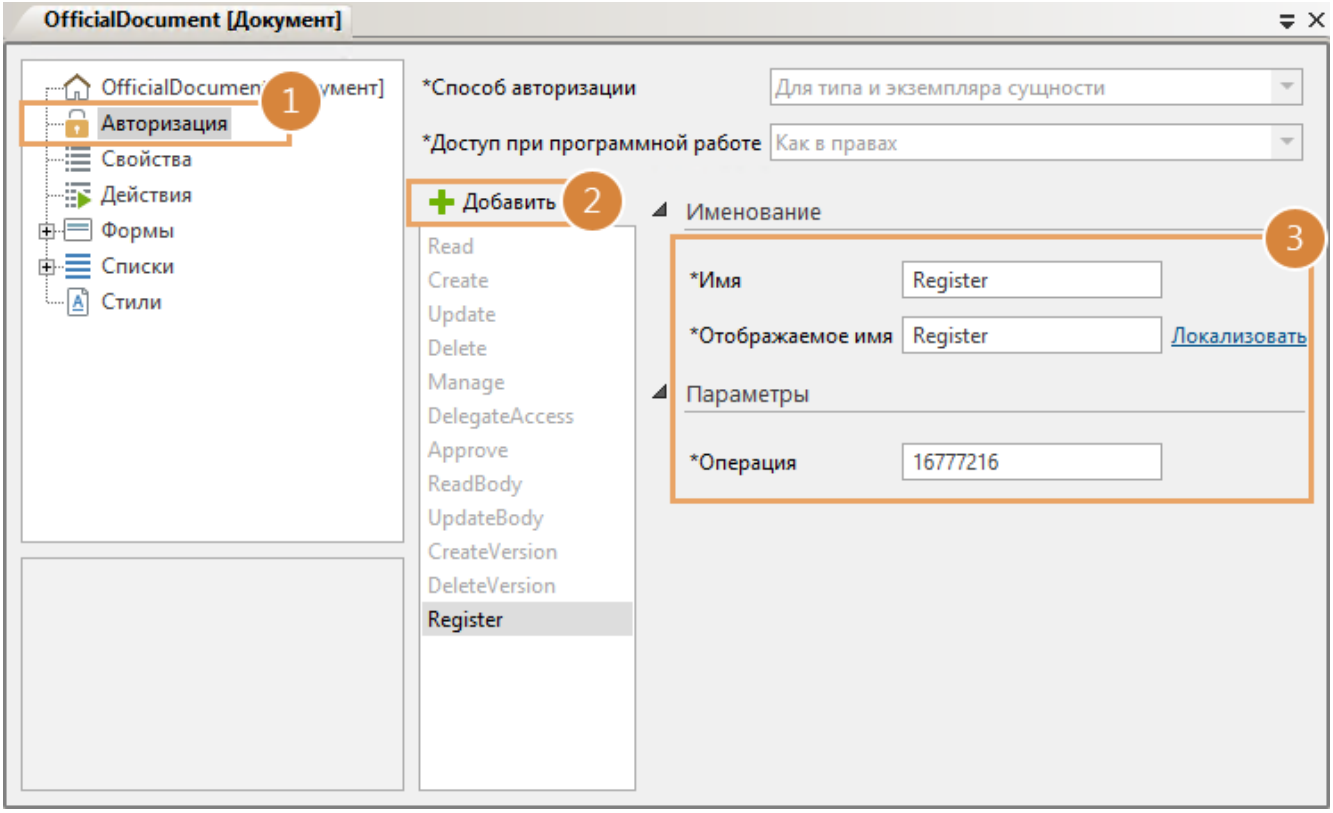
Чтобы подобные ситуации не возникали, при разработке прав доступа учитывайте ряд особенностей и следуйте рекомендациям по реализации.

Пример создания операции авторизации

В разделе описан пример создания операции авторизации для типа документа. Для остальных типов сущностей порядок создания аналогичный.

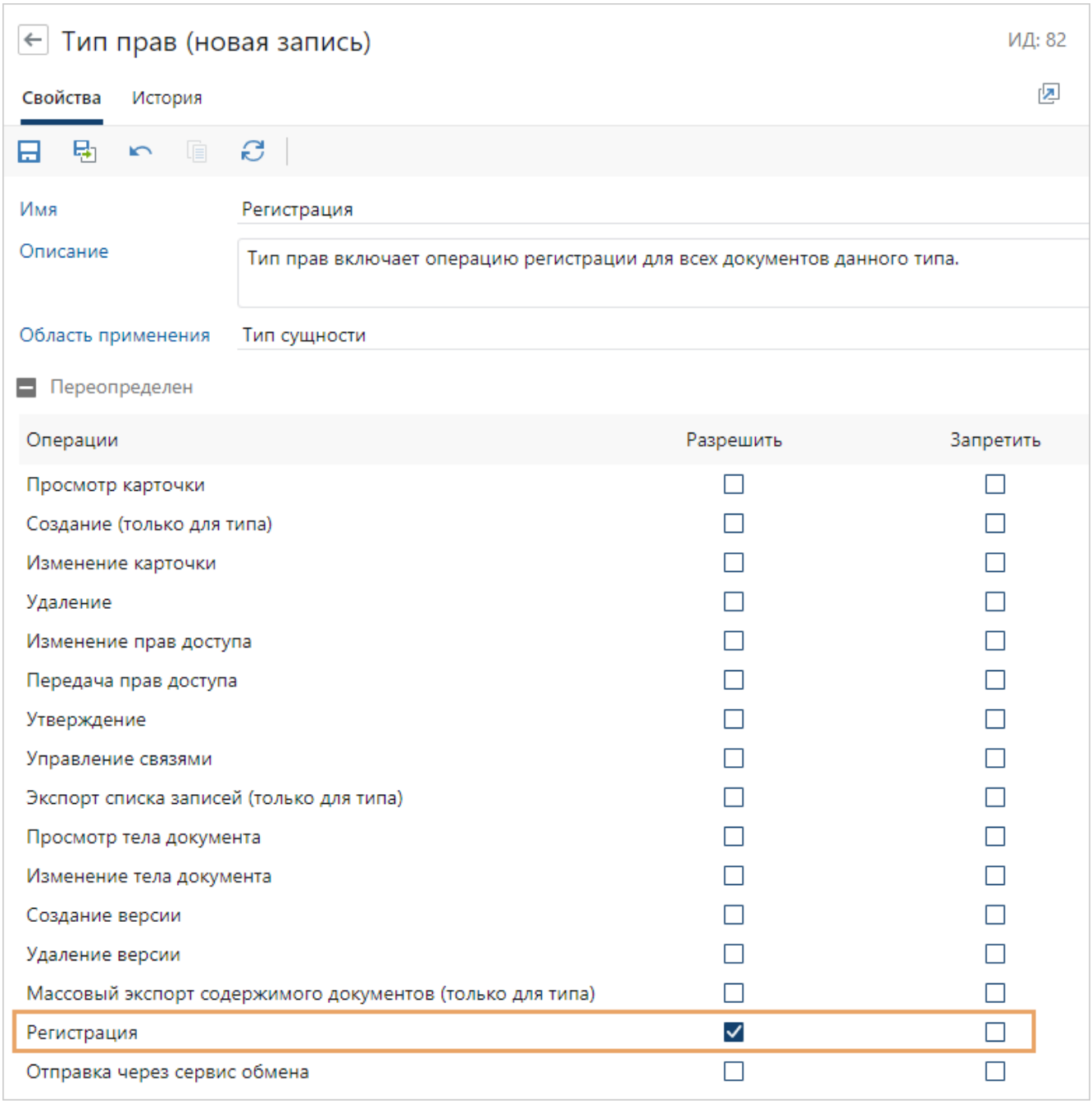
Предположим, у официального документа есть действие **Зарегистрировать**. Необходимо, чтобы выполнять действие могли пользователи, у которых есть права на изменение и регистрацию документа. Права на изменение определяются операцией авторизации **Update** (Изменение), которая наследуется от базового типа документа. Чтобы добавить проверку прав на регистрацию, создайте операцию авторизации. Для этого:

1. В редакторе типа сущности «Официальный документ» перейдите в узел «Авторизация»:

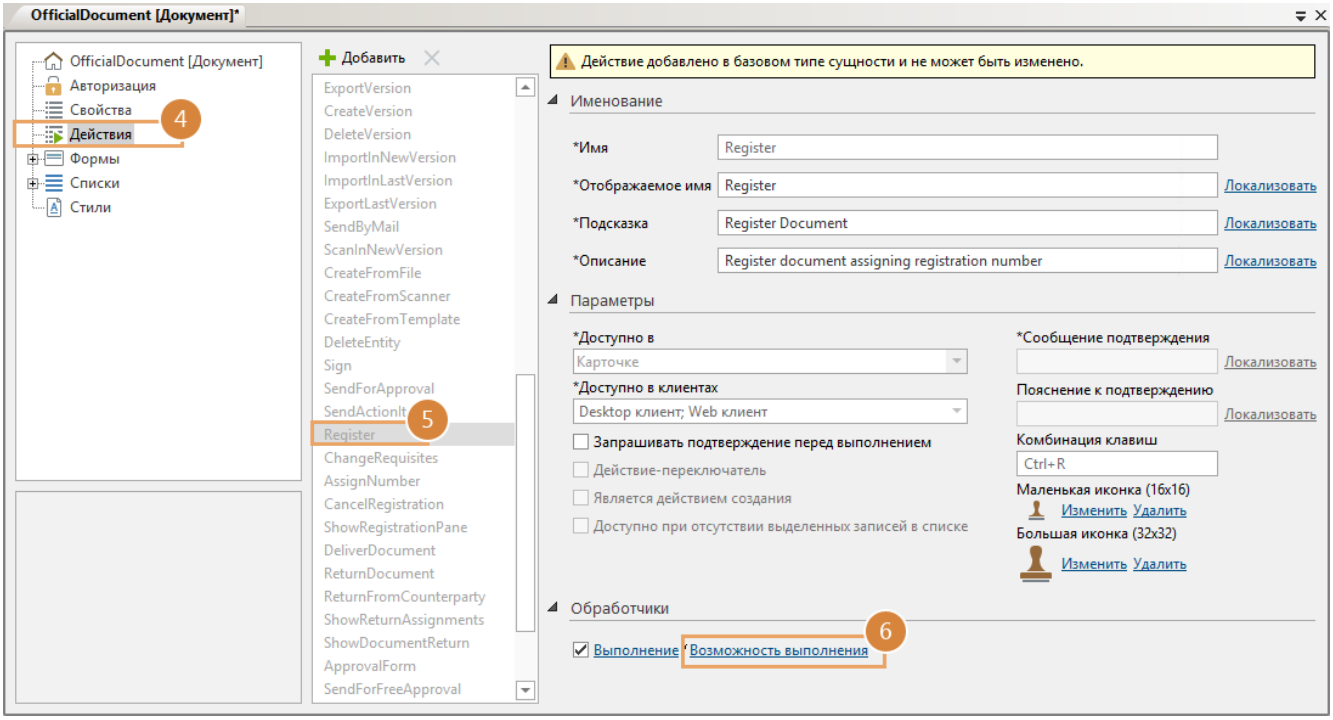


2. Нажмите на кнопку **Добавить**.

3. В списке элементов выделите добавленную операцию и в редакторе свойств задайте настройки для операции. В результате операция авторизации будет доступна в проводнике системы в списке **Типы прав** в карточках записей для всех наследников официального документа:



4. Чтобы добавить проверку прав на регистрацию документа для действия **Зарегистрировать**, перейдите в узел «Действия».



5. В списке элементов выберите действие Register (**Зарегистрировать**).
6. Добавьте код в обработчик события «Возможность выполнения». Для этого в редакторе свойств перейдите по ссылке **Возможность выполнения**. Откроется редактор программного кода:

```
public virtual bool CanRegister(Sungero.Domain.Client.CanExecuteActionArgs
e)
{
   //Действие "Регистрация" будет доступно, если у пользователя есть
   //права на изменение и регистрацию документа.
      return accessRights.CanUpdate() && accessRights.CanRegister()
}
```

В результате действие **Зарегистрировать** у официального документа будет доступно для пользователей, у которых есть права на изменение и регистрацию документа.

Рекомендации по разработке прав доступа

В разделе вы найдете ответы на вопросы:

- какой способ авторизации указать для нового типа сущности
- какой доступ задавать при программной работе
- как выдать права на экземпляр
- как проверить вхождение пользователя в роль, группу
- какие есть рекомендации по выдаче прав на разные объекты в инициализации модуля

Выбор способа авторизации для нового типа сущности

При создании нового типа сущности важно правильно указать способ авторизации:

- для типов документов и задач автоматически выбирается способ авторизации **Для типа и экземпляра сущности**, менять его запрещено;
- для большинства справочников, в которых не нужно скрывать весь список записей или разделять данные на личные и общие, рекомендуется выбирать способ авторизации **Для типа сущности**. Он проще для использования, так как права на экземпляры определяются только настройкой администрирования;
- для справочников, в которых необходимо выдавать разным пользователям разные права на записи (например, «Проекты»), рекомендуется выбирать способ авторизации **Для типа и экземпляра сущности** и назначать права на экземпляры программно.

Доступ при программной работе со справочниками

Если разрабатывается справочник, с которым пользователи напрямую не будут работать, в нем хранится конфиденциальная информация или настройки системы, то рекомендуется:

- не выдавать права на просмотр справочника для всех пользователей;
- выдать доступ при программной работе на чтение или полный.

В системе таким образом ограничен доступ к справочникам с настройками.

ПРИМЕЧАНИЕ. Если вы ограничиваете права пользователей на справочник и выдаете только программный доступ, то учитывайте, что запросы на чтение, создание, изменение и удаление сущностей через сервис интеграции система понимает как выполнение запроса:

- в программном режиме, если используются запросы к функциям модулей. Они отмечены атрибутами **[Public(WebApiRequestType = RequestType.Get)]** и **[Public(WebApiRequestType = RequestType.Post)]**;
- в визуальном режиме, если запросы отправляются с использованием OData-контроллеров. То есть для выполнения запроса у пользователя должны быть права доступа.

Например, для справочника **Подключения нашей организации к сервису доверенностей** задан доступ на чтение при программной работе. Если у пользователя забрать права на справочник, он все равно сможет отправлять доверенность на регистрацию в реестр ФНС. При этом он не увидит содержимое справочника, а также через сервис интеграции у него не получится выгрузить данные о подключениях к сервису доверенностей своей или других наших организаций.

Выдача прав на экземпляр

Возможна автоматическая выдача прав системой или программная выдача прав на экземпляр.

Примеры автоматической выдачи прав на объект сотруднику системой:

- выдача прав на вложенные в задачу документы, чтобы исполнитель задания мог их увидеть;
- выдача прав группе регистрации документа, чтобы другие делопроизводители могли увидеть зарегистрированный в их журнале документ.

Программно права выдаются при помощи метода Grant():

```
_obj.AccessRights.Grant(recipient, accessRightsType);
```

Чтобы исключить лишнюю перевыдачу прав, используйте методы:

- IsGranted() – проверяет права пользователя на объект:

```
// Проверить, выданы ли права на исходную задачу, и при необходимости выдать их.
if (!_obj.MainTask.AccessRights.IsGranted(accessRight.AccessRightsType,
accessRight.Recipient))
  _obj.MainTask.AccessRights.Grant(accessRight.Recipient,
accessRight.AccessRightsType);
```

- IsGrantedDirectly() – проверяет права пользователя на объект, выданные пользователю лично. Не учитывает, в какие группы и роли он входит:

```
// Проверить, выданы ли права секретарю, и при необходимости выдать их.
if (_obj.Secretary!= null &&
!_obj.AccessRights.IsGrantedDirectly(DefaultAccessRightsTypes.Change,
_obj.Secretary))
  _obj.AccessRights.Grant(_obj.Secretary, DefaultAccessRightsTypes.Change);
```

- IsGrantedWithoutSubstitution() – проверяет, выданы ли права пользователю, не учитывает права по замещению:

```
// Проверить, выданы ли исполнителю права на вложение, и при необходимости выдать их.
var accessRights = (IInternalEntityAccessRights)attachment.AccessRights;
if (!accessRights.IsGrantedWithoutSubstitution(Security.BasicOperations.Read,
performer))
  attachmentsWithoutAccessRight.Add(attachment);
```

В системе нет информации о том, каким образом были выданы права: программно или вручную пользователем. Не рекомендуется изымать права, если неизвестно, как они были выданы. Исключением являются ситуации, когда права однозначно выдаются или изымаются системой и не меняются вручную:

```
  // Изъять права на изменение у предыдущего ответственного.
if (responsible != null && !Equals(responsible, _obj.ResponsibleEmployee))
  _obj.AccessRights.Revoke(responsible, DefaultAccessRightsTypes.Change);
```

Проверка прав через вхождение в роль

Полный доступ, как правило, включает в себя все операции, кроме удаления. Поэтому замещающий, руководитель или администратор могут получить «чужие» права. При этом на администратора не распространяются даже запрещающие права.

Из-за этого может возникнуть потребность дополнительно проверить, входит ли пользователь в определенную группу. Например, в группу регистрации.

Чтобы проверить вхождение пользователя в группу с учетом иерархии вложенных групп, используйте методы и свойства:

- AllRecipientIds() – возвращает список ИД *субъектов прав* (в основном, роли и группы), правами которых обладает текущий пользователь. При этом учитываются права по замещению.

  ```
  // Получить действующие журналы регистрации, на которые есть права у текущего пользователя.
  var documentRegisters = DocumentRegisters.GetAll()
  .Where(l => l.Status == CoreEntities.DatabookEntry.Status.Active)
  .Where(l => l.RegisterType == RegisterType.Numbering ||
  Recipients.AllRecipientIds.Contains(l.RegistrationGroup.Id));
  ```

- AllRecipientIdsFor() – возвращает список ИД субъектов прав, правами которых обладает указанный пользователь. При этом учитываются права по замещению;
- DirectSubstitutionRecipientIds – возвращает список ИД пользователей, которых напрямую замещает текущий пользователь;
- DirectSubstitutionRecipientIdsFor – возвращает список ИД пользователей, которых напрямую замещает указанный пользователь;
- OwnRecipientIds – возвращает список ИД субъектов прав, правами которых обладает текущий пользователь без учета прав по замещению.

  ```
  // Проверить, является ли текущий пользователь участником совещания.
  query = query.Where(x => Equals(x.President, currentEmployee) || Equals(x.Secretary, currentEmployee) || x.Members.Any(m => Company.Employees.OwnRecipientIds.Contains(m.Member.Id)));
  ```

- OwnRecipientIdsFor – возвращает список ИД субъектов прав с правами непосредственно указанного пользователя. При этом права по замещению не учитываются.

  ```
  // Получить права подписи, на которые есть права у указанного сотрудника.
  var ids = Recipients.OwnRecipientIdsFor(_filter.Recipient).ToList();
  query = query.Where(s => ids.Contains(s.Recipient.Id));
  ```

Для единичной проверки прав можно использовать проверку через вхождение в роль. Это позволяет обойтись без добавления операции авторизации. При этом нужно помнить о производительности системы, так как при проверке вхождения пользователя в роль отправляется запрос на сервер и в базу данных.

Выдача прав на объекты в инициализации модуля

В инициализации модуля рекомендуется выдавать права на все объекты, с которыми будет работать пользователь. Это позволяет начать работу в системе без дополнительной настройки.

Документы. Рекомендуется в инициализации модуля:

- выдавать права всем пользователям на создание таких документов, с которыми будут работать все сотрудники. Например, на создание заявлений, рабочих и простых документов.

```
Docflow.SimpleDocuments.AccessRights.Grant(allUsers,
   DefaultAccessRightsTypes.Create);
Docflow.SimpleDocuments.AccessRights.Save();
```

- выдавать специально созданным ролям права на создание документов, с которыми будут работать отдельная группа сотрудников. Например, договоры, документы бухгалтерии. Зачастую эти же роли будут использоваться для лицензирования модуля.

Справочники. Рекомендуется в инициализации модуля:

- не выдавать права автоматически на справочники, в которых хранится конфиденциальная информация или настройки системы. По умолчанию при установке системы для справочников *базового решения* задано ограничение: не выдаются права на просмотр роли «Все пользователи». Нужные права конкретным сотрудникам настроит администратор системы. Чтобы система могла обращаться к данным из разработанного вами справочника, задайте для него доступ при программной работе;
- для остальных справочников, с которыми пользователи будут работать, рекомендуется выдавать права на просмотр.

```
Company.Employees.AccessRights.Grant(allUsers, DefaultAccessRightsTypes.Read);
Company.Employees.AccessRights.Save();
```

- выдавать права на создание только отдельным ролям. Например, ролям «Ответственные за настройку регистрации», «Ответственные за контрагентов». Редко всем пользователям требуется менять справочники.

Задачи. Рекомендуется в инициализации модуля для новых типов задач выдавать права на создание всем пользователям.

```
   Workflow.SimpleTasks.AccessRights.Grant(allUsers,
      DefaultAccessRightsTypes.Create);
Workflow.SimpleTasks.AccessRights.Save();
```

Роли. Рекомендуется в инициализации модуля создавать предопределенные роли с назначенными правами на нужные объекты, чтобы администратор мог быстро настроить права конкретным сотрудникам или подразделениям.

```
   // Создать роль «Ответственные за финансовый архив».
Docflow.PublicFunctions.Module.CreateRole
(Resources.RoleNameFinancialArchiveResponsible,
Resources.DescriptionFinancialArchiveResponsible,
FinancialArchive.Constants.Module.FinancialArchiveResponsibleRole);
```

Отчеты. Рекомендуется в инициализации модуля выдавать права на запуск отчета всем пользователям (исключением являются узкоспециализированные отчеты, права на которые выдаются конкретным ролям). Такой подход позволяет не выдавать каждый раз пользователям права на запуск отчета вручную.

```
   // Права на отчеты модуля "Делопроизводство" выдаются только
      делопроизводителям.
var clerks = Docflow.PublicFunctions.DocumentRegister.Remote.GetClerks();
```

```
Reports.AccessRights.Grant(Reports.GetIncomingDocumentsReport().Info,        clerks,
DefaultReportAccessRightsTypes.Execute);
```

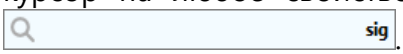
Вычисляемые папки. Требуется в инициализации модуля выдавать права на просмотр вычисляемых папок всем пользователям. Потому что у администратора нет возможности их настраивать вручную.

```
    public static void GrantRightOnFolders(IRole allUsers)
{
  Logger.Debug("Init: Grant right on shell special folders to all users.");
  Sungero.Shell.SpecialFolders.Approval.AccessRights.Grant(allUsers,
DefaultAccessRightsTypes.Read);
  Sungero.Shell.SpecialFolders.Approval.AccessRights.Save();
}
```

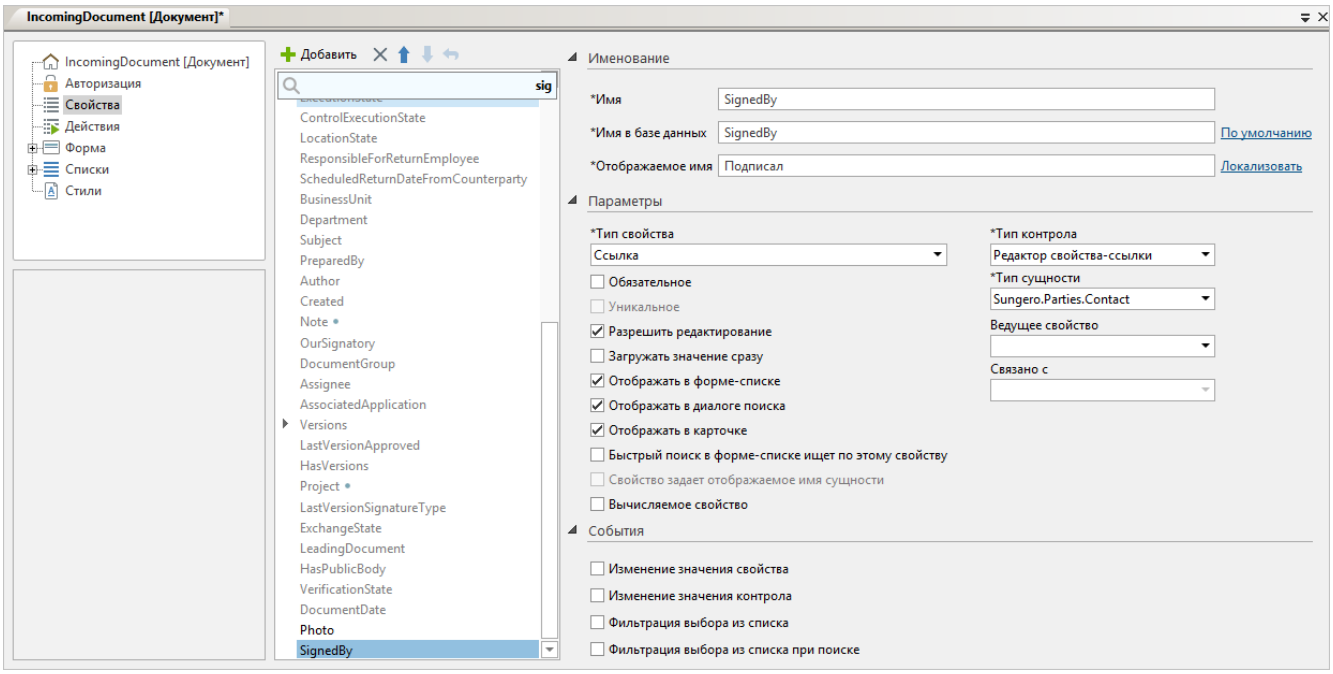
Свойства

Используйте узел «Свойства», чтобы создавать, изменять или копировать свойства типа сущности. Чтобы пользователи системы могли задавать значения свойства, в узле «Форма» добавьте свойства на карточку сущности.

При создании типа сущности некоторые свойства наследуются от *базового типа*. В списке элементов они выделены серым цветом. У наследуемых свойств можно изменять только часть настроек, например, видимость, доступность, отображаемое имя.

Чтобы быстро найти нужное свойство и доработать его, воспользуйтесь поиском: установите курсор на любое свойство и начните вводить имя. Появится поле ввода, например, 

. Далее выполняется поиск вниз от выделенного свойства и подсвечивается первое найденное совпадение. Если повторно ввести то же самое значение, находится следующее совпадение и т.д. При достижении конца списка поиск начинается с начала.



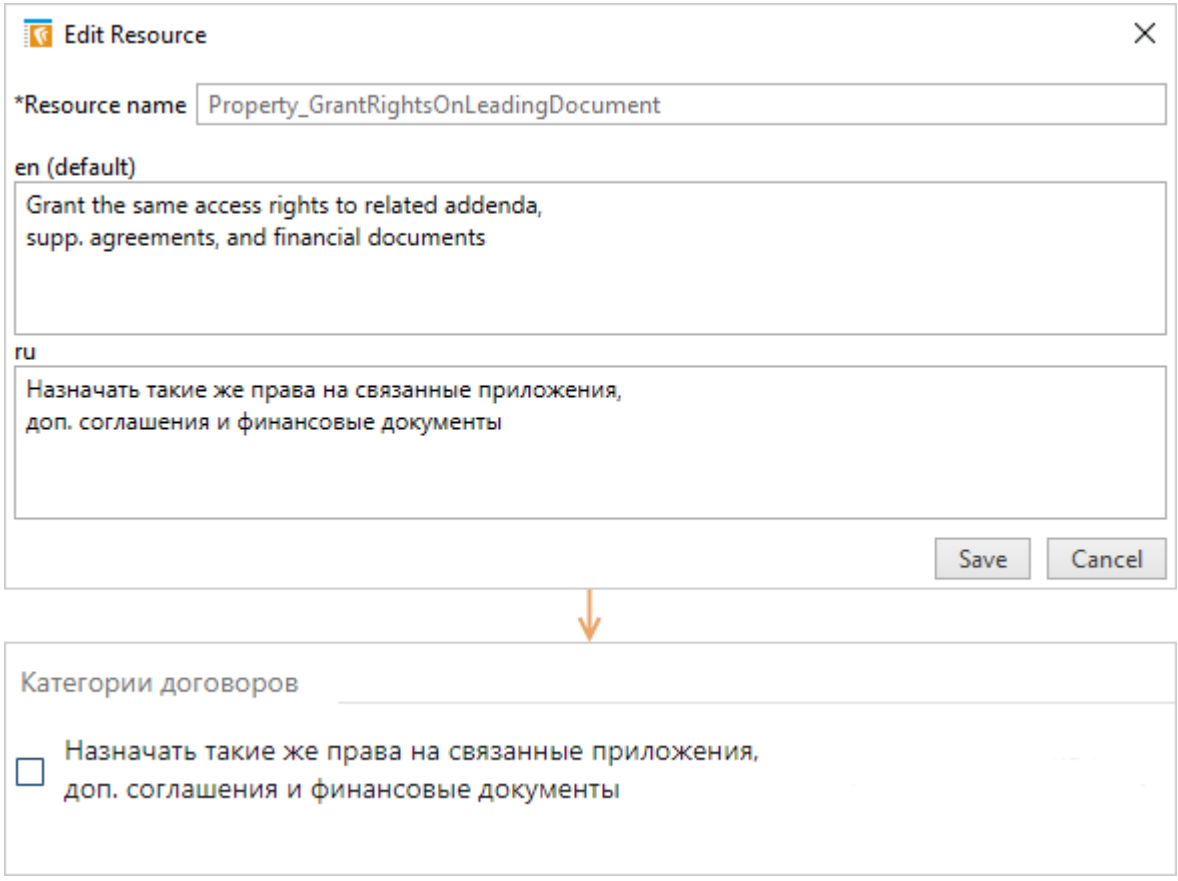
Именование

***Имя**. Должно быть уникальным в рамках типа сущности, состоять из букв латинского алфавита и цифр.

***Имя в базе данных**. Используется при генерации в базе данных имени колонки в таблице типа сущности. Заполняется автоматически. При необходимости имя в базе данных можно изменить. Для этого введите нужное значение вручную. Имя должно быть уникальным в рамках типа сущности, состоять из букв латинского алфавита и цифр. Чтобы вернуть первоначальное значение, нажмите **По умолчанию**.

***Отображаемое имя**. Имя свойства, которое будет отображаться в списке сущностей и карточке сущности. Для *локализации* имени перейдите по ссылке **Локализовать**.

ПРИМЕЧАНИЕ. При необходимости отображаемое имя контрола можно показать в несколько строк. Например, если оно длинное. Для этого в редакторе ресурсов введите имя контрола в том виде, в котором оно должно отображаться в веб-клиенте.



Параметры

***Тип свойства**. В зависимости от типа для свойства определяется:

- формат, в котором будет задаваться и храниться значение свойства;
- тип контрола, в котором будет отображаться значение свойства;
- список дополнительных параметров свойства.

***Тип контрола**. *Контрол* для отображения данных свойства. Заполняется автоматически в зависимости от выбранного типа свойства.

Обязательное. Свойство будет обязательным для заполнения. В карточке сущности перед названием свойства добавится символ *.

ПРИМЕЧАНИЕ. Обязательные поля также можно настраивать в веб-клиенте в справочнике **Представления форм**. Если в проводнике установлен флажок **Обязательно для заполнения**, то его нельзя изменить с помощью программного кода.

Уникальное. Значение свойства должно быть уникальным в рамках типа сущности. Например, уникальным должно быть значение свойства **Id** входящего документа. Доступно для свойств типа **Строка**, **Целое** и **Идентификатор**. Если флажок установлен в родительском типе, то при наследовании и перекрытии его нельзя снять.

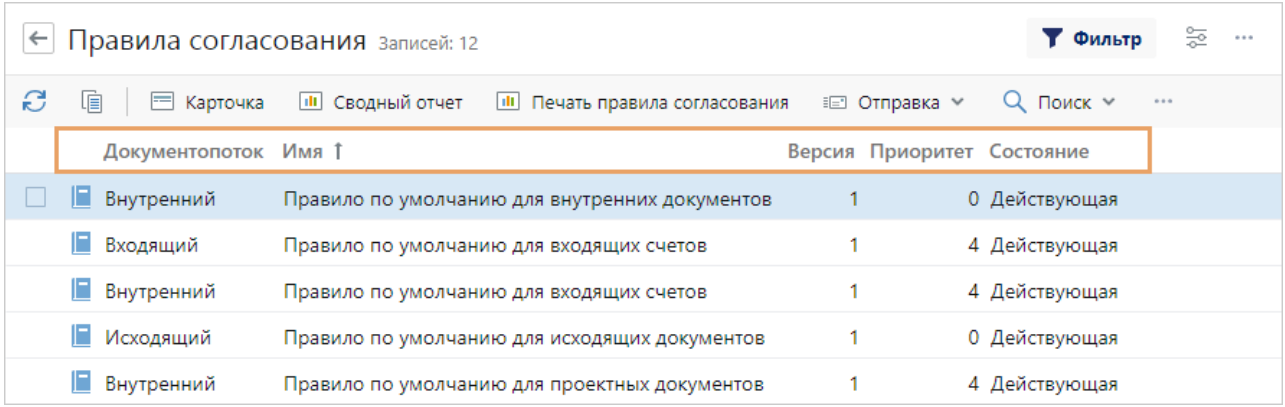
ПРИМЕЧАНИЕ. Значения свойств с параметром **Уникальное** при наследовании будут уникальными в рамках свойства-предка. Например, можно создать справочник и присвоить свойству **Id** типа **Идентификатор** параметр **Уникальное**, а потом создать справочник-наследник. Если добавить две записи в справочник-родитель и одну запись в справочник-наследник, то значения свойств **Id** будут, соответственно, **Id** 1 и **Id** 2 для справочника-родителя и **Id** 3 для справочника-наследника.

Разрешить редактирование. Свойство будет доступно для изменения в карточке сущности.

Загружать значение сразу. При получении сущности с веб-сервера свойство будет загружаться вместе с сущностью (*«жадная» загрузка*). Используется для оптимизации программного кода, так как позволяет сократить количество запросов к веб-серверу. Вместе с сущностью рекомендуется загружать свойства простых типов, например, число, строка. Загрузка всех свойств сущности может увеличить время выполнения запросов и привести к росту сетевого трафика.

СОВЕТ. Снимите флажок, если необходимо, чтобы свойство загружалось с веб-сервером только при непосредственном обращении к свойству. Например, чтобы содержимое версии загружалось отдельно от карточки документа.

Отображать в форме-списке. Свойство можно будет использовать в качестве колонки в списке и папке сущностей. Доступно для свойств типа **Дата**, **Перечисление**, **Строка**, **Ссылка**, **Логическое**, **Целое**, **Идентификатор**, **Вещественное** и **Текст**. Например, для справочника «Правила согласования» флажок установлен для свойств **Документопоток**, **Имя**, **Версия**, **Приоритет**, **Состояние**. При настройке внешнего вида списка правил согласования эти свойства можно отобразить в качестве колонок списка:



ПРИМЕЧАНИЕ. Если у свойства дочерней коллекции флажок **Отображать в форме-списке** не установлен, то флажок нельзя изменять с помощью программного кода.

Отображать в диалоге поиска. Будет возможен поиск по значению свойства в проводнике системы, если в настройках редактора типа сущности установлен флажок **Отображать в диалоге поиска**. Например, для документов типа «Приложение к обращению» можно добавить поиск по обращению, к которому относится документ. Доступно для свойств типа **Дата**, **Перечисление**, **Строка**, **Ссылка**, **Логическое**, **Целое**, **Идентификатор**, **Вещественное**, **Текст** и **Бинарные данные**. Рекомендуется использовать в поисках не более пяти свойств типа сущности. В окне поиска свойства будут отображаться в том порядке, в котором они расположены в узле «Свойства» в списке элементов. Чтобы изменить порядок, используйте кнопки и .

ПРИМЕЧАНИЕ. С помощью параметра можно настроить поиск для сущностей типа справочник, документ и задача. В редакторе типа задания параметр оставлен для совместимости.

Отображать в карточке. Свойство будет отображаться в карточке сущности.

ПРИМЕЧАНИЕ. Если в среде разработки элемент скрыт программно, то в веб-клиенте он не отобразится, даже если добавить его с помощью редактора форм в проводнике.
Свойство дочерней коллекции может быть скрыто из табличной части с помощью программного кода. Чтобы отобразить это свойство в табличной части, в списке всех столбцов установите рядом с ним флажок .

Быстрый поиск в форме-списке ищет по этому свойству. По значениям свойства можно будет выполнять быстрый поиск в списке сущностей в проводнике системы. Доступно для свойств типа **Строка** и **Ссылка**.

Свойство задает отображаемое имя сущности. Значение свойства будет отображаться в контроле свойства-ссылки на данный тип сущности. Например, если свойство является ссылкой на тип сущности **Обращение**, то в контроле может отображаться название обращения. Для типа сущности можно задать одно свойство с данным признаком. Доступно для свойств типа **Строка** и **Ссылка**.

Разрешить сортировку в списке. Записи табличной части карточки можно будет сортировать по столбцу, который соответствует дочернему свойству коллекции. Снимите флажок, если нужно отключить сортировку по свойству. Если флажок установлен в родительском типе, то при наследовании и перекрытии его можно снять. Доступно для дочерних свойств коллекции.

Вычисляемое свойство. Значение свойства вычисляется автоматически. Для этого необходимо выбрать тип вычисления и дополнительно заполнить поля либо прописать код. Подробнее см. в разделе «Вычисляемые свойства».

События

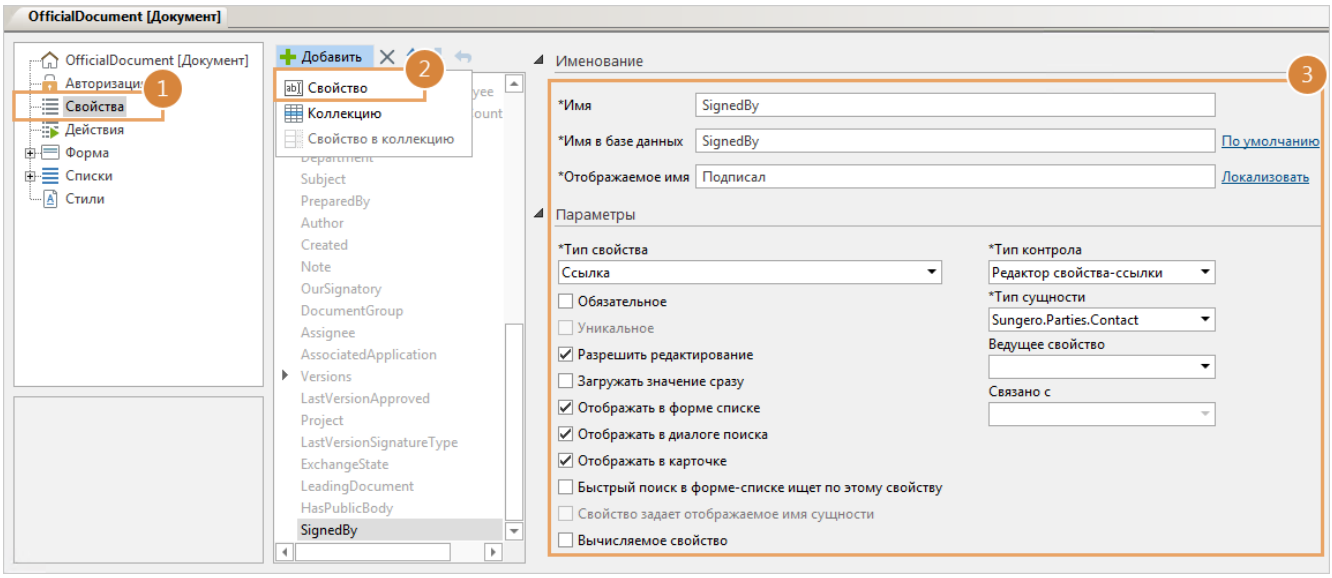
События. Добавление обработчиков событий свойств сущности. Код обработчиков событий выполняется на веб-сервере и/или в клиентском приложении, когда происходит соответствующее событие. Список доступных событий зависит от типа свойства.

Создание свойства

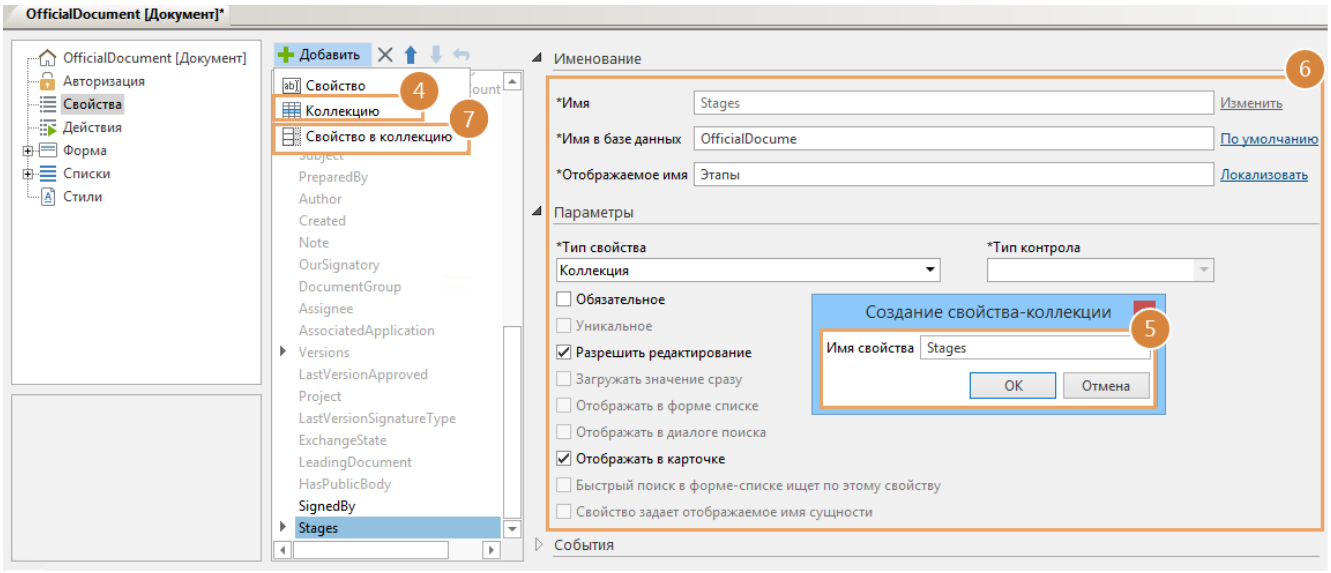
Если требуется создать свойство, похожее на уже существующее, создайте его копированием.

В разделе описан пример создания свойства для типа документа. Для остальных типов сущностей порядок создания аналогичный.

1. В редакторе типа сущности перейдите в узел «Свойства».



2. Чтобы добавить свойство, нажмите на кнопку **Добавить** и в выпадающем списке выберите пункт **Свойство**.
3. В списке элементов выделите добавленное свойство и задайте настройки.
4. Чтобы добавить *свойство-коллекцию*, нажмите на кнопку **Добавить**. В выпадающем списке выберите пункт **Коллекцию**.



5. В открывшемся окне введите имя свойства-коллекции и нажмите на кнопку **OK**.
6. В списке элементов выделите добавленное свойство-коллекцию и задайте настройки.

7. Добавьте свойства в коллекцию. Для этого выделите свойство-коллекцию и нажмите на кнопку **Добавить**. В выпадающем списке выберите пункт **Свойство в коллекцию**.
8. В списке элементов выделите добавленное свойство, задайте для него параметры и добавьте обработчики событий.

При необходимости добавьте свойства на карточку сущности. Для этого используйте узел «Форма» в редакторе типа сущности.

Типы свойств

Тип свойства определяет, какие данные могут быть значением свойства. Также тип задает способ отображения значения и дополнительные параметры свойства.

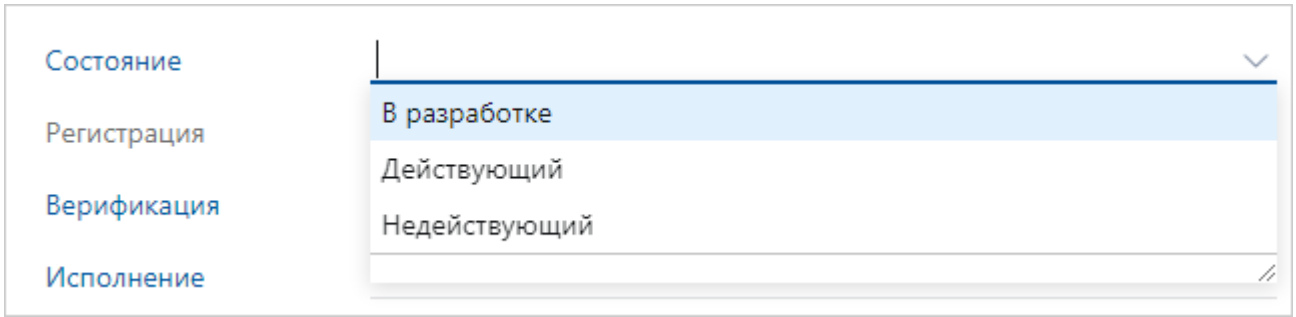
Свойства могут быть следующих типов:

- Перечисление
- Ссылка
- Коллекция
- Дата
- Логическое
- Вещественное
- Целое
- Идентификатор
- Строка
- Бинарные данные
- Текст
- Картинка
- Бинарные данные в хранилище

Перечисление

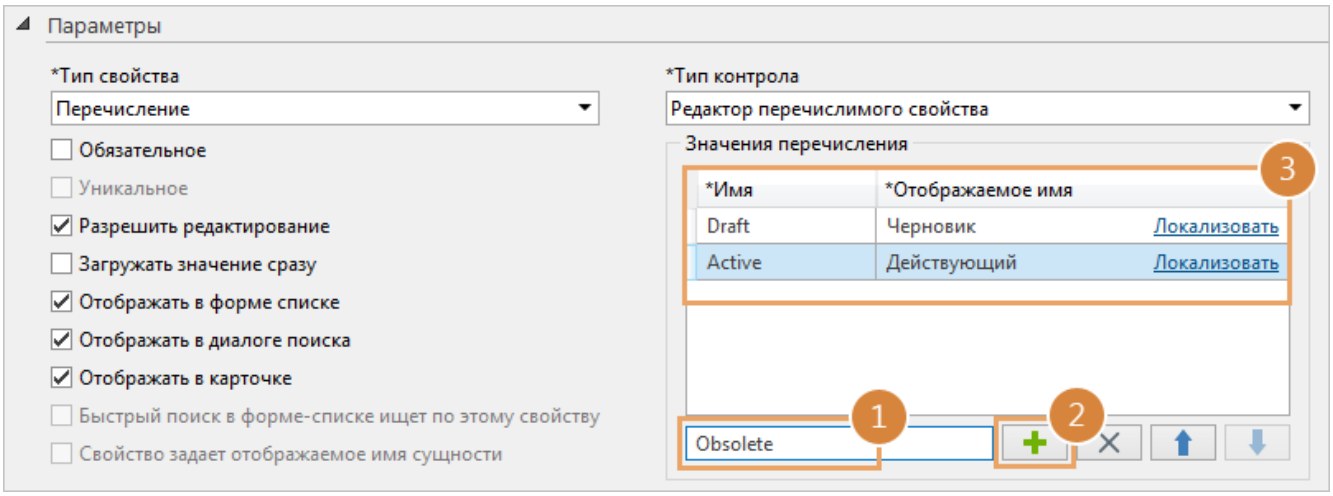
Используйте, если возможных значений у свойства немного и они заранее известны. Например, для указания стадии жизненного цикла документа с возможными значениями: **В разработке**, **Действующий**, **Устаревший**.

Значения свойства будут выбираться из выпадающего списка.



Чтобы задать список возможных значений свойства:

1. В редакторе типа сущности в узле «Свойства» в поле для ввода укажите имя значения.



2. Нажмите на кнопку . В табличной части появится новая запись, а в колонке ***Имя** – введенное значение. Чтобы удалить значение, выделите его в табличной части и нажмите на кнопку . Чтобы изменить порядок значений, используйте кнопки и .
3. В колонке ***Отображаемое имя** введите имя значения, которое будет отображаться в проводнике системы. Для *локализации* отображаемого имени перейдите по ссылке **Локализовать**.

В результате значения, указанные в табличной части, будут доступны для выбора в качестве возможных значений свойства.

К значению свойства типа «Перечисление» можно обратиться в серверном и клиентском коде:
<Имя модуля>.<Имя сущности>.<Имя перечисления>.<Значение>.

Пример 1. Установка исходного статуса обработки документа

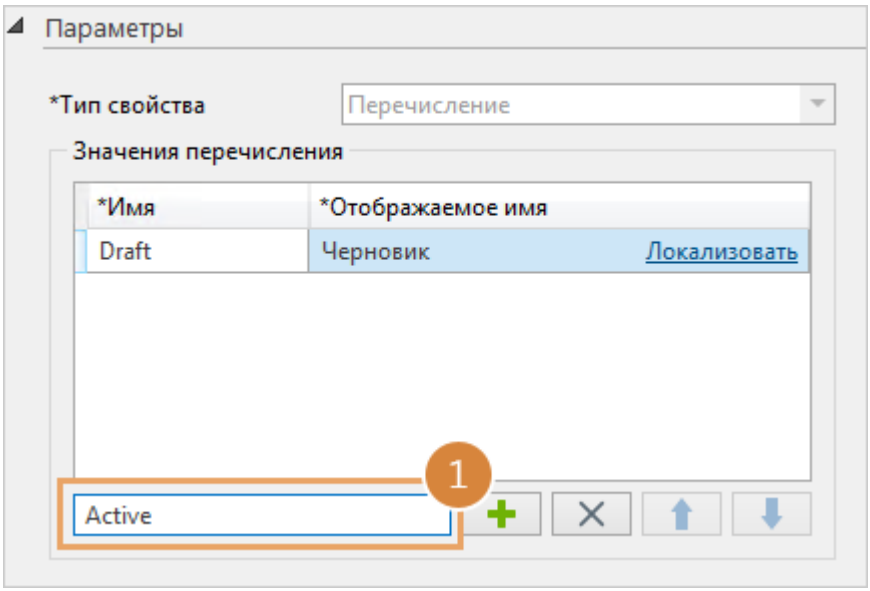
```
    // Установить исходный статус обработки документа «Сравнение начато».
comparisonInfo.ProcessingStatus =
Sungero.Docflow.DocumentComparisonInfo.ProcessingStatus.Started;
```

Пример 2. Получение результата сравнения документов

```
    // Показать результат, если сравнение документов было завершено успешно или с
       ошибкой.
if (comparisonInfo.ProcessingStatus ==
Docflow.DocumentComparisonInfo.ProcessingStatus.Compared ||
comparisonInfo.ProcessingStatus ==
Docflow.DocumentComparisonInfo.ProcessingStatus.Error)
{
  Functions.Module.ShowDocumentComparisonResults(comparisonInfo);
}
```

Чтобы задать список возможных значений свойства блока бизнес-процесса:

1. В редакторе блока в группе «Параметры» в поле для ввода укажите имя значения:



2. Нажмите на кнопку . В табличной части появится новая запись, а в колонке ***Имя** – введенное значение. Чтобы удалить значение, выделите его в табличной части и нажмите на кнопку . Чтобы изменить порядок значений, используйте кнопки и .
3. В колонке ***Отображаемое** имя введите имя значения, которое будет отображаться в проводнике системы. Для локализации отображаемого имени перейдите по ссылке **Локализовать**.

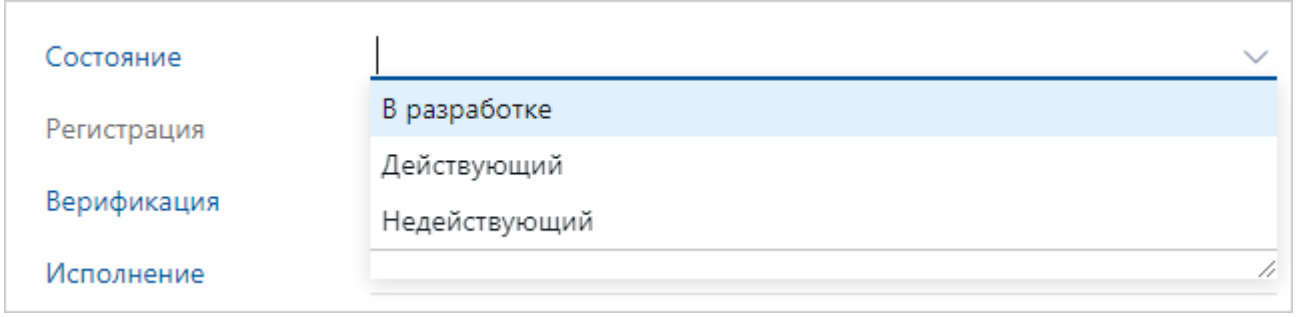
В результате указанные значения будут доступны в проводнике веб-клиента при настройке блоков.

Ссылка

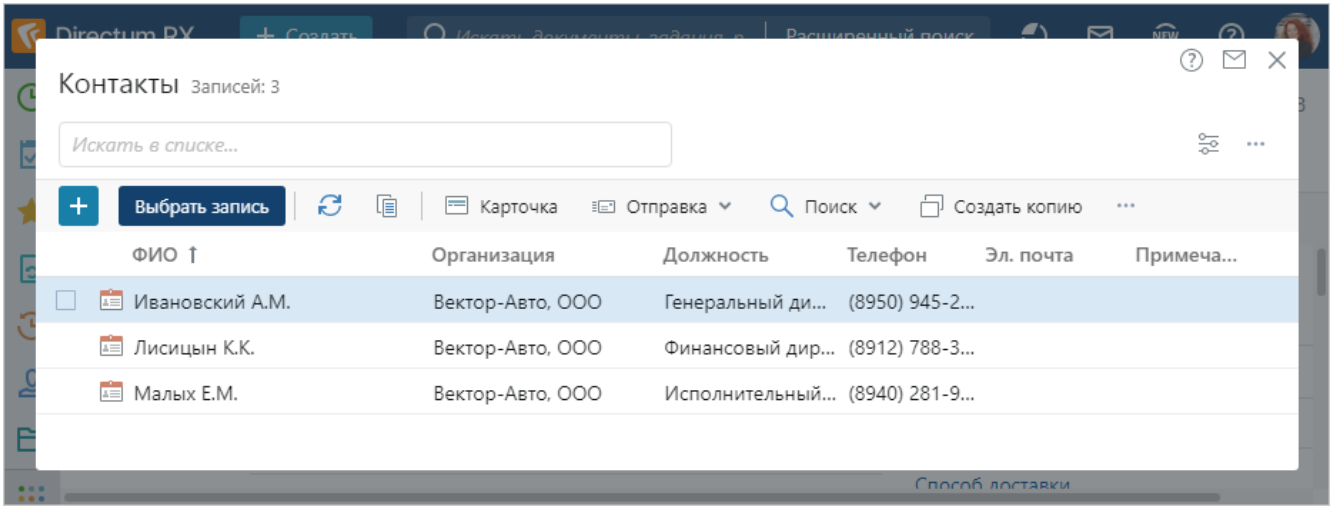
Используйте для работы со ссылками на типы сущностей, возможными значениями свойства будут сущности указанного типа. Например, если свойство является ссылкой на справочник **Сотрудники**, то возможными значениями будут записи этого справочника.

Значения свойства будут выбираться в контроле **Редактор свойства-ссылки**. Отображение контрола зависит от способа выбора значения для типа сущности, на который настроена ссылка. Способ выбора значения указывается на основной вкладке редактора типа сущности. Может отображаться в виде:

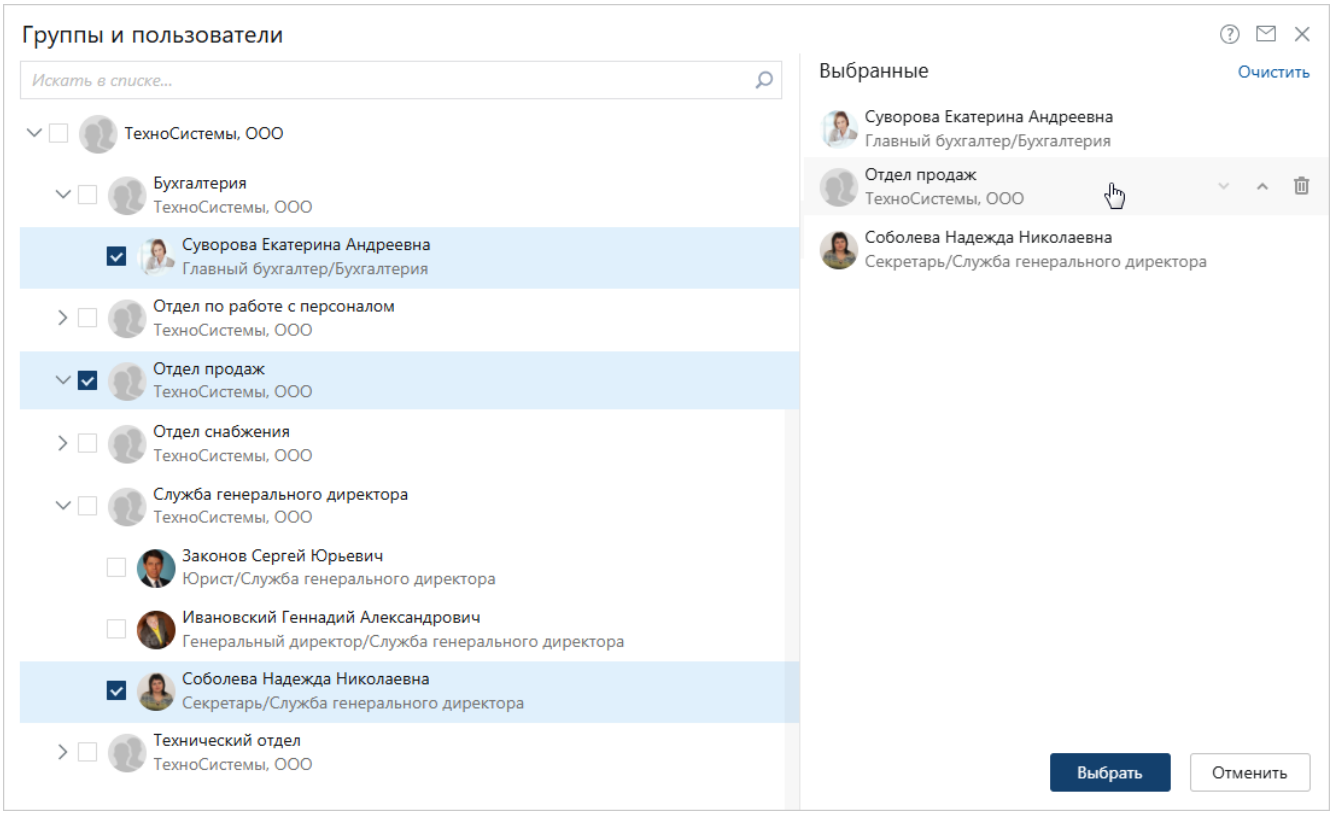
- выпадающего списка:



- отдельной формы-списка, которая открывается при нажатии на кнопку ⋯:



- формы с древовидным списком, которая используется для выбора сотрудников из организационной структуры:



Чтобы задать тип сущности, записи которого будут возможными значениями свойства, в редакторе типа сущности в узле «Свойства» заполните поля:

- ***Тип свойства**. В выпадающем списке выберите значение **Ссылка**;
- ***Тип сущности**. Сущности данного типа будут значениями свойства. Например, в справочнике **Город** для свойства **Регион** можно настроить заполнение с помощью типа сущности **Регион**;

- **Отображать как список при выборе**. Флажок доступен только для типа сущности **Субъект прав** (Sungero.CoreEntities.Recipients) и его наследников. Если флажок установлен, то для выбора значения будет использоваться форма-список;
- **Отображать как выпадающий список при выборе**. Флажок доступен только для типа сущности **Субъект прав** (Sungero.CoreEntities.Recipients) и его наследников. Если флажок установлен, то в веб-клиенте для выбора значения используется выпадающий список, даже если в редакторе типа сущности в поле **Способ выбора значения** указано значение **Форма-список**. При этом выбор значений из формы с древовидным списком недоступен.

Если нужно использовать форму с древовидным списком, то снимите флажки **Отображать как список при выборе** и **Отображать как выпадающий список при выборе**, а также проверьте, что в редакторе типа сущности в поле **Способ выбора значения** указано значение **Форма-список**.

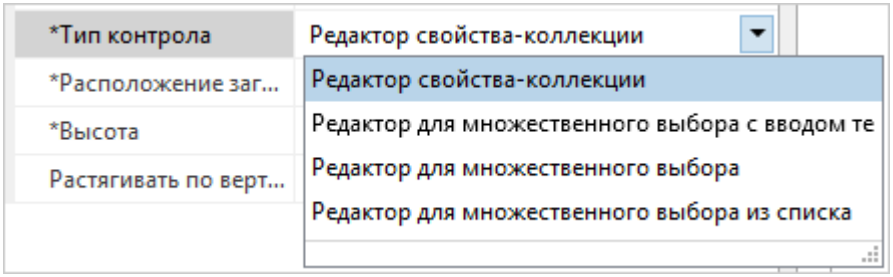
Чтобы можно было в зависимости от других полей заполнить свойство только определенными значениями, настройте логику «Ведущий – ведомый» в свойствах.

Также настроить фильтрацию списка можно с помощью обработчика события Фильтрация выбора из списка. При этом настройка логики «Ведущий – ведомый» не мешает фильтрации с помощью обработчика, и наоборот, они могут действовать совместно: сначала отрабатывает настройка, а затем обработчик события.

Коллекция

Используйте для работы с набором записей, у которых есть свои свойства. Например, для хранения информации о выдаче и возврате документов. Тип свойства «Коллекция» устанавливается автоматически при создании *свойства-коллекции*.

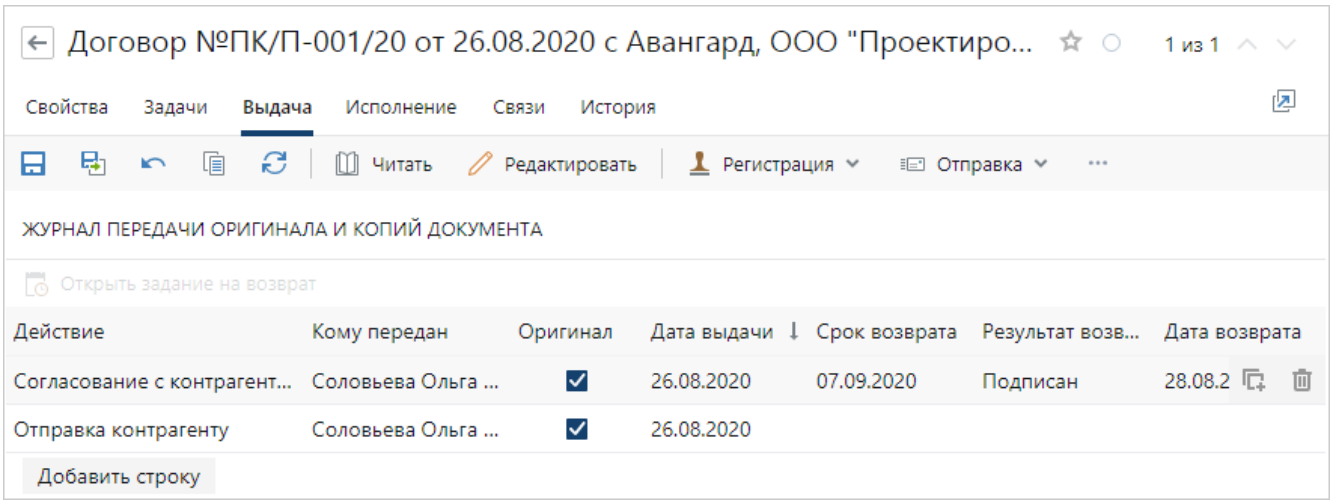
Отображение свойства в карточке сущности зависит от типа контрола. Тип задается в редакторе типа сущности в узле «Форма». Доступные типы контролов:



Редактор свойства-коллекции

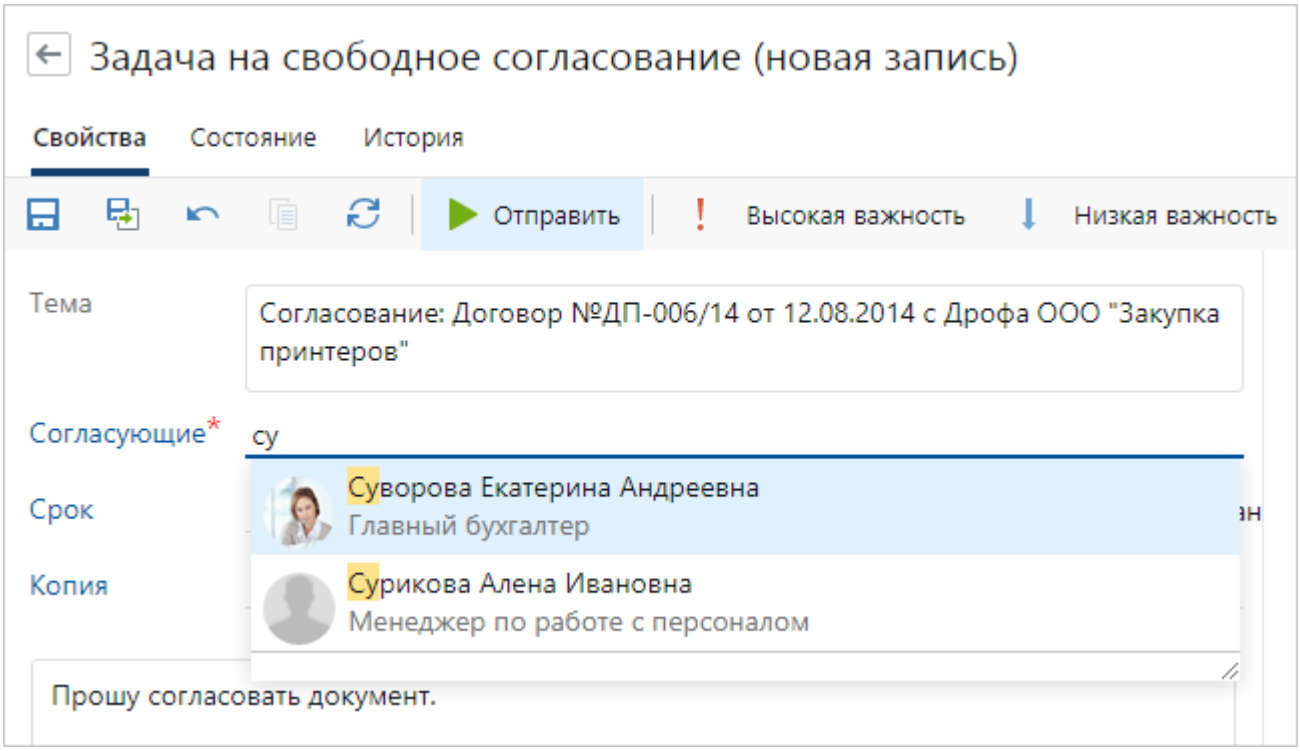
Отображается в виде таблицы, в которой столбцам соответствуют дочерние свойства коллекции. Состав и порядок отображения столбцов табличной части, а также кнопок на панели действий можно настраивать в проводнике.

ВАЖНО. Во время сохранения изменений все пустые записи удаляются. Если при добавлении строк настроено автоматическое заполнение и пользователь их не редактирует, они также удаляются при сохранении.



Редактор для множественного выбора с вводом

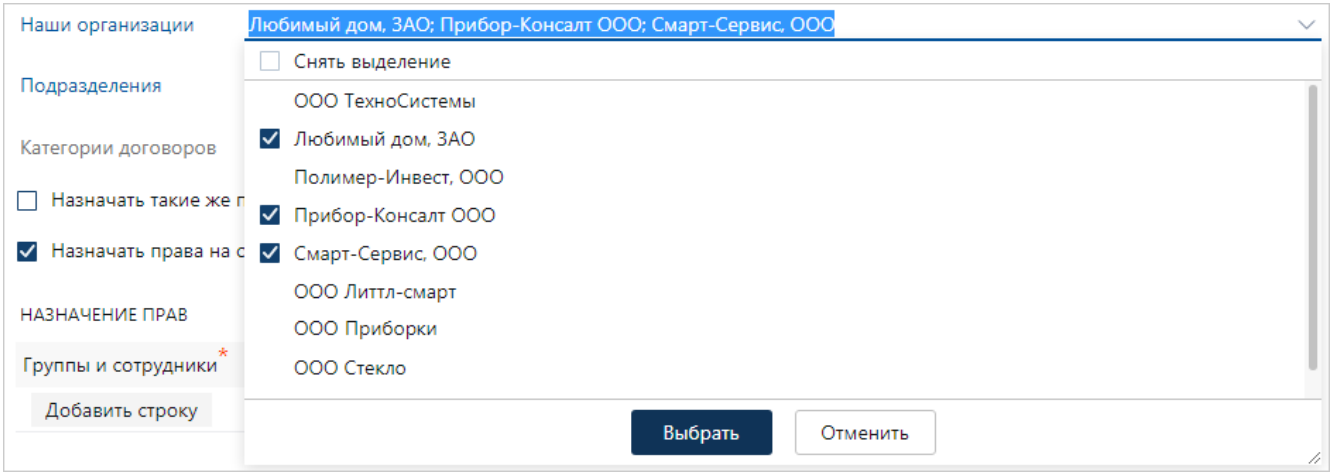
Отображается в виде выпадающего списка. Пользователь может выбрать одну запись из списка:



Редактор для множественного выбора

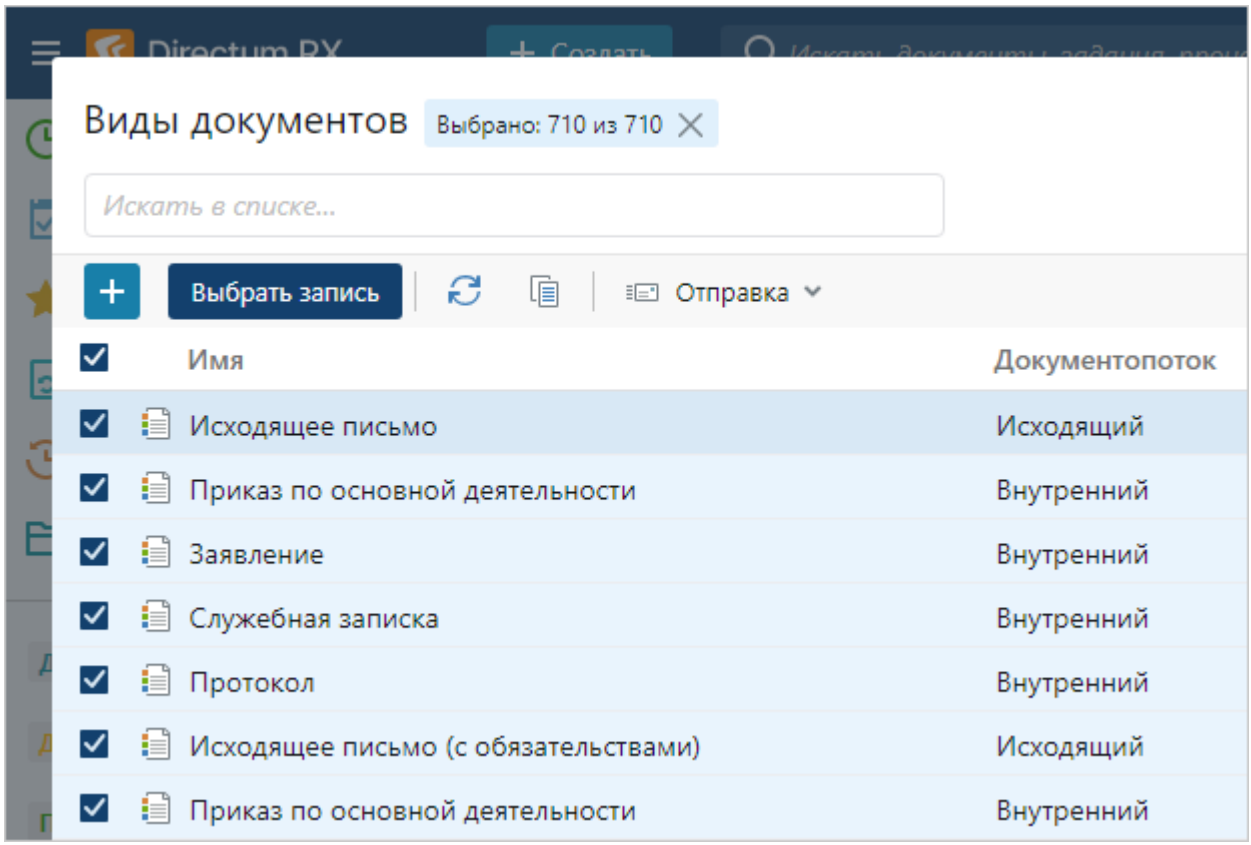
Отображается в виде выпадающего списка. Пользователь может выбрать до 100 записей. В этом типе контрола список отображается не полностью, если в нем больше 100 записей.

Рекомендуется устанавливать этот тип для справочников, содержимое которых со временем значительно не увеличится, например, для выбора наших организаций в карточке правила назначения прав доступа на документы:



Редактор для множественного выбора из списка

Отображается в виде модального выпадающего списка. При нажатии на кнопку ••• открывается отдельная форма-список. В ней можно выбрать любое количество записей:



ПРИМЕЧАНИЕ. При необходимости в параметрах контрола можно настроить, чтобы в списке отображались ссылки на записи разных источников.

Чтобы при добавлении или удалении записи из коллекции дополнительно выполнялась некоторая логика, в редакторе типа сущности в узле «Свойства» добавьте обработчики событий Добавление в коллекцию и Удаление из коллекции.

Дата

Используйте для работы с датой и временем.

В редакторе типа сущности в узле «Свойства» в поле ***Формат даты** дополнительно укажите формат отображения даты и времени. Возможные значения: **Только дата**, **Дата и время**, **Год и месяц**, **Только год**.

Чтобы программно проверить, в каком формате отображаются значения свойств типа «Дата», используйте перечисление CommonLibrary.DateTimeFormat.

Логическое

Используйте для свойств, которые могут иметь значения **True** и **False**. Например, используется для свойства поручения **На контроле**.

Вещественное

Используйте для работы с вещественными числами.

Целое

Используйте для работы с целыми числами.

Идентификатор

Идентификатор – число типа long. Предназначен только для хранения идентификаторов сущностей. В остальных случаях для хранения чисел достаточно меньшего диапазона значений, поэтому используйте тип «Целое» или «Вещественное».

Строка

Используйте для работы со строками. Например, для указания краткого содержания документа.

В редакторе типа сущности в узле «Свойства» в поле ***Длина строки** дополнительно укажите максимально возможное количество символов в строке в диапазоне от 1 до 1000.

Бинарные данные

Используйте для работы с данными небольшого размера. Например, для работы с текстовыми файлами. Значение свойства задается только программно.

Текст

Используйте для работы с текстовыми данными. Например, для хранения примечаний к документам и записям справочников. В отличие от типа **Бинарные данные**, для задания и отображения значения свойства используется контрол **Многострочный текстовый редактор**.

Картинка

Используйте для работы с графическими данными небольшого размера. Например, для хранения фотографии в карточке сотрудника. В отличие от типа **Бинарные данные**, для задания и отображения значения используется контрол **Контроль для изображения**. Контрол поддерживает следующие форматы изображений: **BMP**, **GIF**, **JPEG**, **PNG**, **TIFF**, **WMP**, **ICO**.

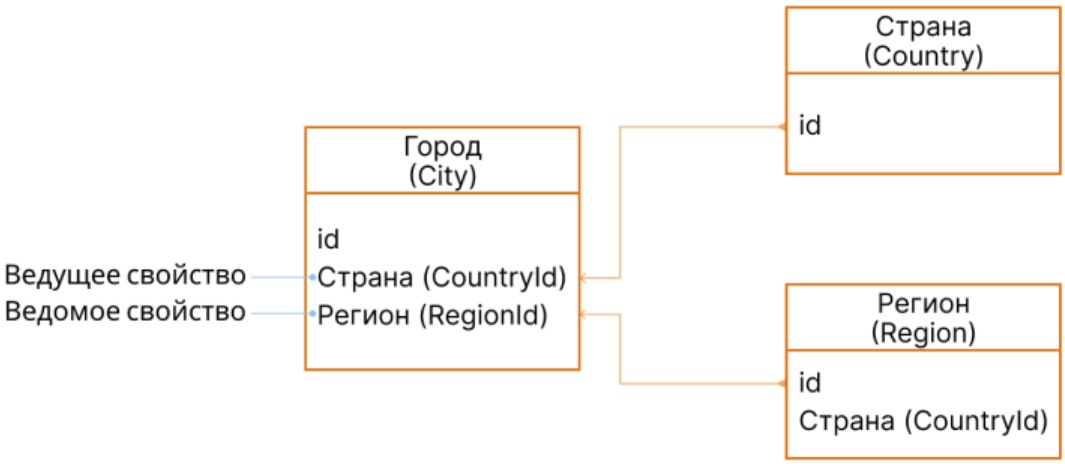
Бинарные данные в хранилище

Используйте для работы с данными больших размеров. Например, для хранения содержимого электронных документов. Значение свойства задается только программно.

Логика «Ведущий – ведомый» в свойствах

В среде разработки для отражения логических связей между свойствами типа **Ссылка** предусмотрена возможность связывания свойств с помощью логики «Ведущий – ведомый». Например, в типе сущности **Город** свойство **Страна** является ведущим для свойства **Регион**, а свойство **Регион** – ведомым для **Страна**.

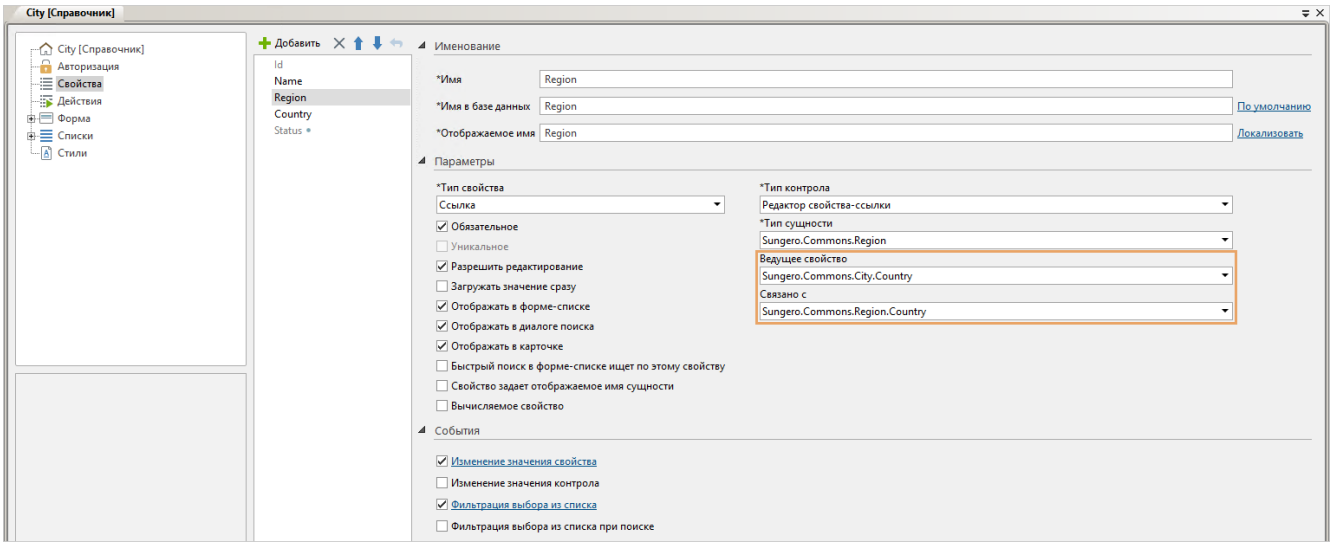
Ведущее свойство определяет доступность значений ведомого свойства. Например, при заполнении свойства **Регион** можно выбрать только регион страны, указанной в поле **Страна**:



ПРИМЕЧАНИЕ. Настройка логики «Ведущий – ведомый» доступна только для свойств типа **Ссылка**.

Чтобы настроить логику «Ведущий – ведомый» в свойствах:

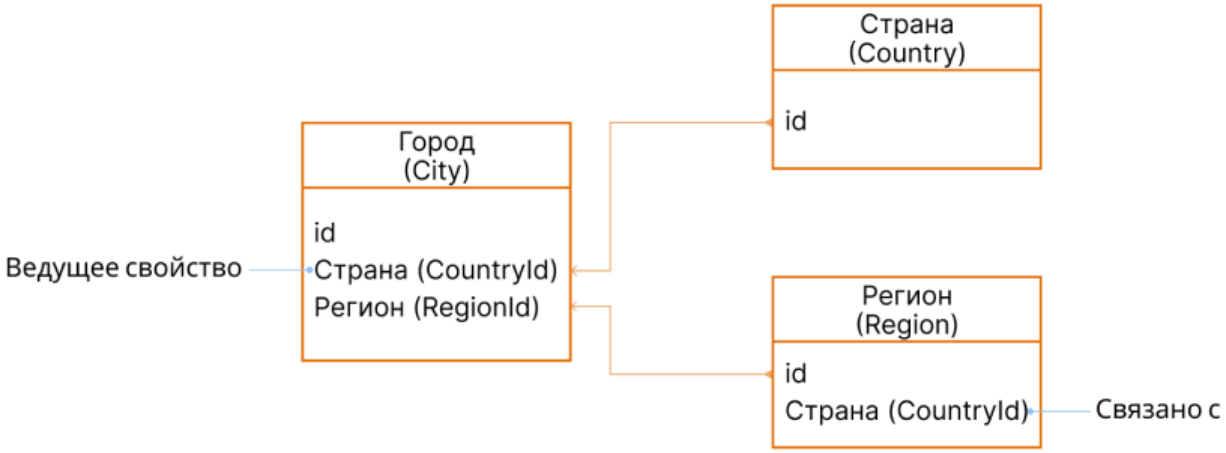
1. Откройте редактор типа сущности, например тип справочника **Город**.
2. В узле «Свойства» перейдите к свойству, список доступных значений которого нужно фильтровать, например **Регион**. Оно будет ведомым:



3. Заполните поля:
   - **Ведущее свойство**, по которому будет фильтроваться список доступных значений, выбирается из выпадающего списка. Оно будет ведущим. Например, для свойства **Регион** ведущим может являться свойство **Страна**, и при заполнении поля **Регион** для выбора будут доступны только регионы страны, указанной в соответствующем поле.

- **Связано с**. Свойство типа сущности, указанного в поле ***Тип сущности**, которое идентично свойству в поле **Ведущее свойство**. Значение выбирается из выпадающего списка. Устанавливает связь между ведущим и ведомым свойствами.

ПРИМЕЧАНИЕ. Поля **Ведущее свойство** и **Связано с** можно очистить с помощью горячих клавиш CTRL+DELETE. Также их значения обнуляются при изменении поля ***Тип свойства**.



Свойство может иметь одно ведущее свойство и несколько ведомых. При изменении ведущего его связь с ведомыми сбрасывается, а список доступных к выбору значений для каждого ведомого свойства фильтруется.

В результате свойства окажутся связанными. Для фильтрации списка доступных значений ведомого свойства будет использоваться свойство, указанное в поле **Связано с**. Например, при выборе региона в карточке типа сущности **Город** можно будет выбрать только регионы страны, указанной в поле **Страна**.

Также, если настроена логика «Ведущий – ведомое», при заполнении ведомого свойства, ведущее автоматически подставится.

С помощью логики можно выстроить цепочку свойств, например **Страна** – **Регион** – **Город**. При этом каждое ведомое свойство фильтруется только по его непосредственному ведущему. В таком случае, если заполнить свойство **Страна**, а **Регион** оставить пустым, список доступных значений свойства **Город** не будет фильтроваться.

Вычисляемые свойства

Значения вычисляемых свойств определяются автоматически на основании других данных. Благодаря использованию таких свойств:

- снижается стоимость разработки, так как уменьшается количество ситуаций, когда разработчику необходимо писать код;
- можно максимально быстро актуализировать необходимые данные. Например, после устранения ошибок соединения абонентского ящика с сервисом обмена значение статуса соединения сразу обновляется. Это происходит даже если карточка ящика заблокирована другим пользователем;

- оптимизируется процесс сохранения сущности, так как при изменении таких свойств сохраняются только они, а не вся сущность. То есть не вызываются обработчики событий, которые затрагивают другие свойства сущности;
- уменьшается нагрузка на систему, поскольку количество выполняемого кода снижается.

В среде разработки можно настроить два типа вычисления:

- ссылка на свойство
- внешнее вычисление

ПРИМЕЧАНИЕ. Вычисляемые свойства нельзя редактировать в карточке сущности.

Настройка доступна для свойств типа Строка, Ссылка, Целое, Вещественное, Дата, Логическое и Перечисление.

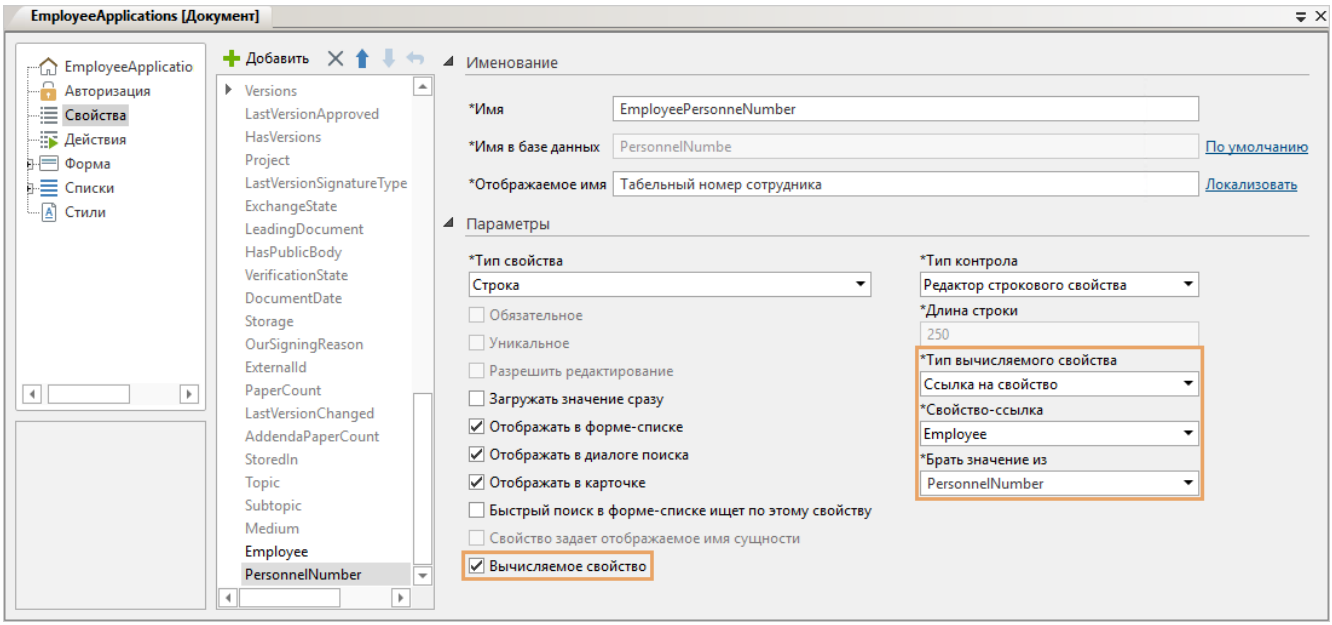
Ссылка на свойство

Значение свойства определяется на основе других свойств типа **Ссылка**. Его нельзя редактировать программно, но к нему можно обращаться в коде.

ПРИМЕЧАНИЕ. Вычисляемое свойство хранит не значение, а ссылку на него в базе данных. Благодаря этому в карточке сущности значение актуализируется автоматически.

Предположим, в карточке созданного типа документа **Заявки сотрудников** после заполнения поля **Сотрудник** необходимо чтобы свойство **Табельный номер сотрудника** автоматически заполнялось информацией из справочника **Сотрудники**. Для этого:

1. Откройте редактор типа сущности **Заявки сотрудников**.
2. В узле «Свойства» перейдите к свойству, значение которого нужно вычислять автоматически. В нашем случае **Табельный номер сотрудника** (EmployeePersonnelNumber):



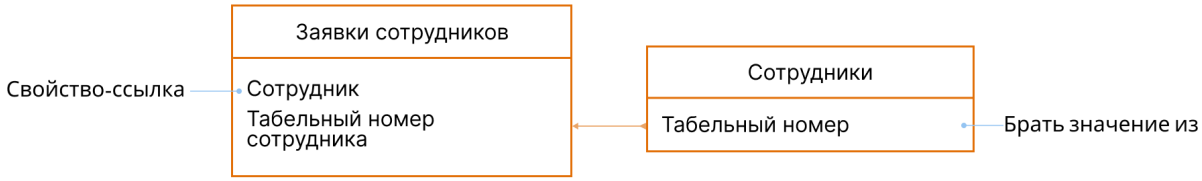
3. Установите флажок **Вычисляемое свойство** и заполните поля:

   Тип вычисляемого свойства. В выпадающем списке выберите значение **Ссылка на свойство**.

   Свойство-ссылка. В выпадающем списке выберите свойство типа **Ссылка**, откуда нужно брать данные. В примере это свойство **Сотрудник** (Employee), которое ссылается на одноименный справочник.

   Брать значение из. В выпадающем списке выберите свойство, значение которого нужно брать. В примере это свойство **Табельный номер** (PersonnelNumber) справочника **Сотрудники** (Employee).

   Далее на схеме показан механизм заполнения полей:



4. Сохраните изменения.

В результате свойству **Табельный номер сотрудника** (EmployeePersonnelNumber) будет автоматически присваиваться значение после заполнения свойства **Сотрудник** (Employee).

Внешнее вычисление

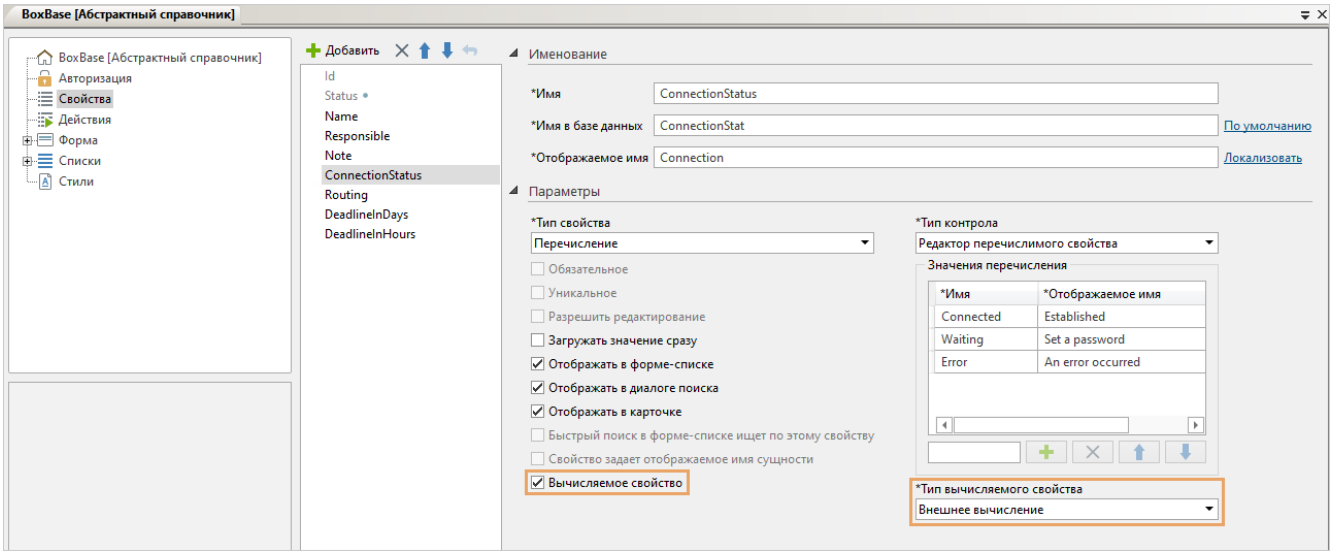
Внешнее вычисление следует использовать, когда значение свойства вычисляется по сложному алгоритму, при этом оно должно актуализироваться максимально быстро. Такое свойство можно изменять программно, при этом оно остается доступным для редактирования, даже если карточка сущности заблокирована.

Для свойства нельзя создавать собственные обработчики событий, но с ним можно работать в других функциях и обработчиках.

ПРИМЕЧАНИЕ. При сохранении изменений свойств с типом **Внешнее вычисление** не вызываются обработчики событий, которые затрагивают другие свойства сущности. Таким образом оптимизируется процесс сохранения.

Предположим, в карточке абонентского ящика подразделения необходимо поддерживать в актуальном состоянии статус соединения с сервисом обмена. Для этого:

1. Откройте редактор типа сущности **Абонентский ящик** (BoxBase).
2. В узле «Свойства» перейдите к свойству **ConnectionStatus** и установите флажок **Вычисляемое свойство**:



3. В выпадающем списке **Тип вычисляемого свойства** выберите пункт **Внешнее вычисление**.
4. Перейдите в разделяемые функции модуля **ExchangeCore** и добавьте код, который актуализирует значение свойства **ConnectionStatus**:

```
    /// <summary>
/// Проставить статус соединения в абонентских ящиках подразделений.
/// </summary>
/// <param name="box">Абонентский ящик нашей организации.</param>
public void SetDepartmentBoxConnectionStatus(IBusinessUnitBox box)
{
  var departmentBoxes = Functions.BoxBase.Remote.GetActiveChildBoxes(box);
  var closedDepartmentBoxes = Functions.BoxBase.Remote.GetClosedChildBoxes(box);
  departmentBoxes.AddRange(closedDepartmentBoxes);
  departmentBoxes = departmentBoxes.Where(x => !Equals(x.ConnectionStatus,
box.ConnectionStatus)).ToList();
  foreach (var departmentBox in departmentBoxes)
  {
    if (!Equals(departmentBox.ConnectionStatus, box.ConnectionStatus))
    {
      departmentBox.ConnectionStatus = box.ConnectionStatus;
      departmentBox.Save();
    }
  }
}
```

5. Добавьте вызов функции в обработчик события фонового процесса **SyncBoxes**.
6. Сохраните изменения.

В результате состояние соединения абонентского ящика с сервисом обмена в свойстве **ConnectionStatus** автоматически актуализируется фоновым процессом. Это выполняется, даже если пользователь блокирует карточку абонентского ящика.

В системе Directum RX помимо **Соединение абонентского ящика** таким способом реализованы свойства **Фактическая дата исполнения поручения** и **Статус поручения**.

События свойств сущности

События свойств задаются в редакторе типа сущности в узле «Свойства». Обработчики событий выполняются на веб-сервере и/или в клиентском приложении.

События свойств сущности	
Изменение значения свойства	<Имя свойства>Changed
Изменение значения контрола	<Имя свойства>ValueInput
Фильтрация выбора из списка	<Имя свойства>Filtering
Фильтрация выбора из списка при поиске	<Имя свойства>SearchDialogFiltering
Фильтрация выбора из выпадающего списка	<Имя свойства>Filtering
Добавление в коллекцию	<Имя свойства>Added
Удаление из коллекции	<Имя свойства>Deleted

Изменение значения свойства (<Имя свойства>Changed)

Выполняется при изменении значения свойства, в том числе при программном изменении. Для свойства-коллекции выполняется после изменения состава коллекции или одной из записей коллекции.Событие относится к разделяемому коду. Код обработчика выполняется на веб-сервере и/или в клиентском приложении.

Назначение:

Событие используется для изменения **других** свойств, текущее свойство изменять нельзя. Например, создано свойство для хранения **Суммы товара**. Для него добавлен обработчик события **Изменение значение свойства**, чтобы при изменении значения суммы, изменялось свойство **Сумма с учетом НДС**.

Аргументы события:

- _obj – сущность, у которой изменилось значение свойства;
- e.NewValue – новое значение свойства;
- e.OldValue – предыдущее значение свойства;
- e.OriginalValue – исходное значение свойства, которое сохранено в базе данных;
- e.Params – дополнительные параметры;
- e.State – состояние свойства.

Изменение значения контрола (<Имя свойства>ValueInput)

Выполняется только при изменении значения в контроле пользователем вручную. Доступно для свойств типа Дата, Перечисление, Строка, Ссылка, Логическое, Целое, Вещественное и Текст. Событие относится к клиентскому коду. Код обработчика выполняется в клиентском приложении.

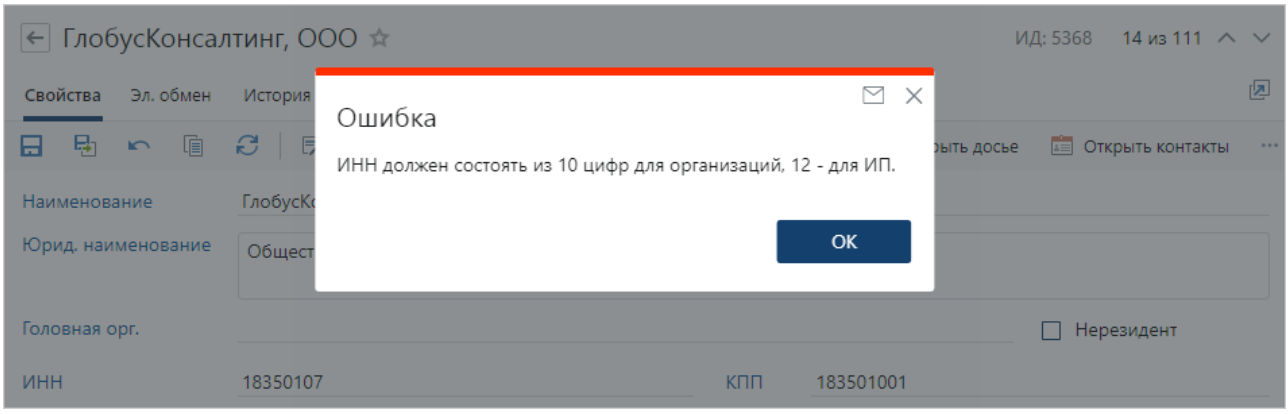
Назначение. Используется, например для показа диалогов, запросов.

ВАЖНО. В обработчике события не рекомендуется вызывать серверные функции (с атрибутом **Remote**) и использовать логику, которая будет долго обрабатываться. Данный обработчик выполняется часто. Поэтому, если в нем есть серверные функции, то на веб-сервер отправляется большое количество запросов, что, в свою очередь, влияет на быстродействие системы.

Аргументы события:

- _obj – сущность, у которой изменилось значение свойства;
- e.AddError() – добавление сообщения об ошибке. Выполняемые операции будут прерваны;
- e.AddInformation() – добавление информационного сообщения. Выполняемые операции не будут прерваны;
- e.AddWarning() – добавление предупреждения. Выполняемые операции не будут прерваны.

  ПРИМЕЧАНИЕ. Сообщение об ошибке, информационное сообщение и предупреждение отображаются с соответствующей иконкой в верхней части карточки сущности или диалоговым окном. Например, сообщение об ошибке выглядит следующим образом:



- e.NewValue – новое, введенное в контрол значение свойства;
- e.OldValue – предыдущее значение свойства в контроле;
- e.Params – дополнительные параметры.

Фильтрация выбора из списка (<Имя свойства>Filtering)

Доступно только для свойств типа Ссылка. Событие относится к серверному коду. Код обработчика выполняется на веб-сервере.

Назначение. Фильтрация значений типа сущности при выборе значения свойства-ссылки. Например, чтобы при выборе контакта в карточке входящего письма отображать только контактные лица организации, указанной в поле ***Корреспондент**.

ПРИМЕЧАНИЕ. При выборе значения пользователь может создать новую запись, если нет подходящей. Если созданная запись не подходит под условия фильтрации для выбора из списка, то появляется сообщение об этом, и запись нельзя выбрать.

Аргументы события:

- _obj – сущность, для свойства которой выполняется фильтрация;
- query – исходный запрос на получение возможных значений для свойства-ссылки;
- e.Params – дополнительные параметры;
- _root – доступ к другим свойствам сущности, для свойства которой выполняется фильтрация. Аргумент доступен только для свойств-ссылок дочерней коллекции.

  СОВЕТ. Используйте аргумент **_root**, чтобы изменить сущность в зависимости от результатов фильтрации свойства. Например, в одной из наших организаций существует подразделение **Отдел закупок**, а в другой – нет. В записи справочника «Право подписи» в поле **Подразделения** по умолчанию будут доступны все записи из справочника «Подразделения». Чтобы при настройке права подписи документов для сотрудника из второй нашей организации запись несуществующего подразделения не появлялась в списке, в карточке справочника «Право подписи» можно настроить фильтрацию свойства-коллекции **Подразделения** в зависимости от значения свойства-ссылки **Наши организации**. В таком случае с помощью **_root** можно обратиться к свойствам карточки сущности (справочника).

Пример:

```
/// <summary>
/// Функция для фильтрации справочника "Контакты" при выборе значения
свойства-ссылки.
/// В окне выбора будут отображаться только контактные лица организации,
/// указанной в свойстве Correspondent текущей сущности.
/// </summary>
public virtual IQueryable<T> ContactFiltering(IQueryable<T> query,
Sungero.Domain.PropertyFilteringEventArgs e)
{
  if (_obj.Correspondent != null)
    query = query.Where(c => Equals(c.Company, _obj.Correspondent));
  return query;
}
```

Фильтрация выбора из списка при поиске (<Имя свойства>SearchDialogFiltering)

Доступно только для свойств типа Ссылка. Событие относится к серверному коду. Код обработчика выполняется на веб-сервере.

Назначение. Фильтрация значений типа сущности при выборе значения свойства-ссылки в диалоге поиска. Например, при выборе типа документа в диалоге поиска в критерии **Вид документа** отображаются только те виды, которые созданы для выбранного типа: для типа «Договор», отображаются только виды договоров, для типа «Входящее письмо» только виды писем, которые поступают в компанию и т.д. Это возможно, так как типы созданы на основе базового типа документа **Официальный документ** (OfficialDocument). В редакторе базового типа для свойства **Вид документа** (DocumentKind) добавлен обработчик события фильтрации выбора при поиске:

```
public virtual IQueryable<T> DocumentKindSearchDialogFiltering(IQueryable<T>
query, Sungero.Domain.PropertySearchDialogFilteringEventArgs e)
```

```
{
  if (e.EntityType != null)
    query = query.Where(k => k.DocumentType.DocumentTypeGuid ==
e.EntityType);
  return query;
}
```

Все типы, которые созданы на основе OfficialDocument, наследуют данное событие.

Аргументы события:

- query – исходный запрос на получение возможных значений для свойства-ссылки;
- e.EntityType – уникальный идентификатор (GUID) типа сущности, выбранного в диалоге поиска.

Фильтрация выбора из выпадающего списка (<Имя свойства>Filtering)

Доступно только для свойств типа Перечисление. Событие относится к клиентскому коду. Код обработчика выполняется в клиентском приложении.

Назначение. Фильтрация значений в свойстве-перечислении. Например, в типе сущности «Настройки регистрации» для свойства-перечисления **Тип настройки** указаны значения «Регистрация», «Нумерация» и «Резервирование». Для входящего документопотока не предусмотрены настройки резервирования. Чтобы скрыть значение «Резервирование» в карточке, настраивается фильтрация.

Аргументы события:

- _obj – сущность, для свойства которой выполняется фильтрация;
- query – исходный запрос на получение возможных значений для свойства-перечисления.

Добавление в коллекцию (<Имя свойства>Added)

Выполняется при добавлении новой записи в свойство-коллекцию. Событие относится к разделяемому коду. Код обработчика выполняется на веб-сервере и/или в клиентском приложении.

Назначение:

- заполнение значений по умолчанию для новой записи свойства-коллекции;

  ПРИМЕЧАНИЕ. Если пользователь не редактирует добавленные записи, они удаляются при сохранении изменений свойства-коллекции.
- написание дополнительной логики, которая выполняется при добавлении новой записи в свойство-коллекцию.

Аргументы события:

- _obj – сущность, у которой в свойство-коллекцию добавлена запись;
- _source – если новая запись свойства-коллекции была добавлена копированием существующей, то значением свойства будет запись, из которой создана новая. В противном случае значением свойства будет **null**;

- _added – запись, добавленная в свойство-коллекцию;
- e.Params – дополнительные параметры.

Удаление из коллекции (<Имя свойства>Deleted)

Выполняется при удалении записи из свойства-коллекции. Событие относится к разделяемому коду. Код обработчика выполняется на веб-сервере и/или в клиентском приложении.

Назначение. Написание дополнительной логики, которая будет обрабатываться при удалении записи.

Аргументы события:

- _obj – сущность, у которой из свойства-коллекции удаляется запись;
- _deleted – запись, удаленная из свойства-коллекции;
- e.Params – дополнительные параметры.

Действия

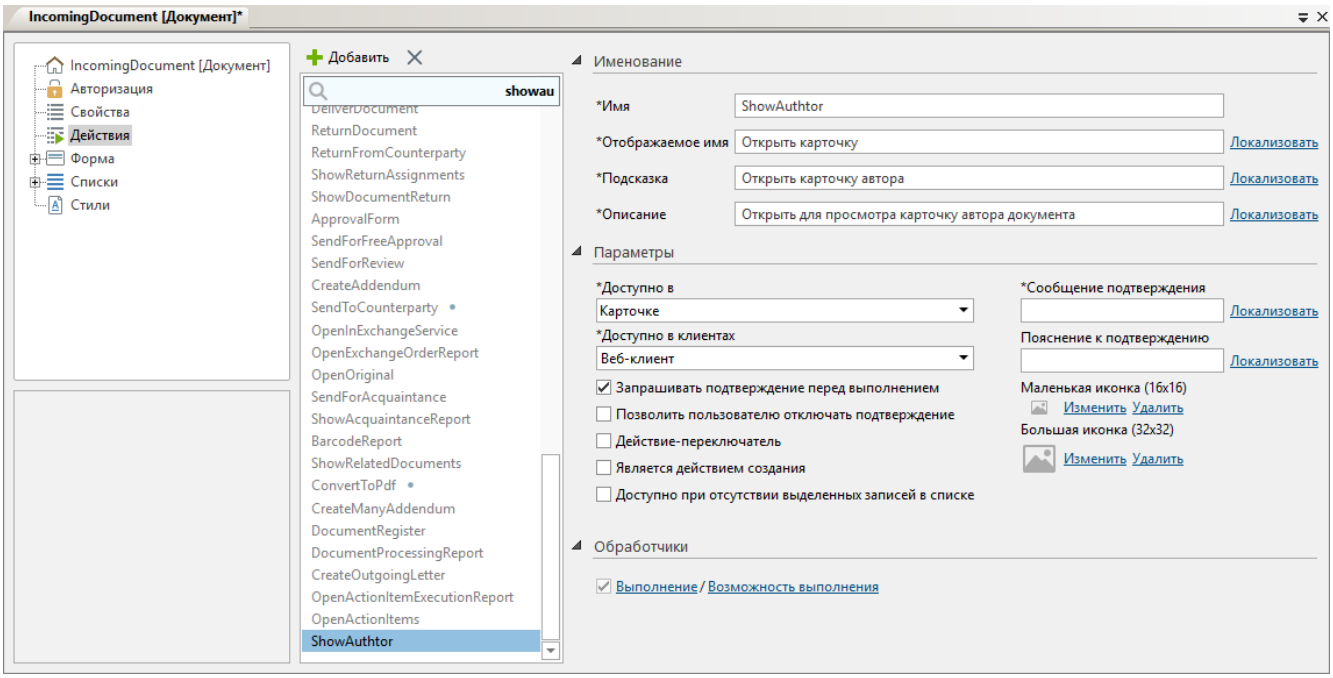
Используйте узел «Действия», чтобы создавать, изменять или копировать действия типа сущности. Чтобы действия стали доступны пользователям, в узле «Лента» добавьте их на ленту карточки или списка сущностей.

ПРИМЕЧАНИЕ. Если в среде разработки элемент скрыт программно, то в веб-клиенте он не отобразится, даже если добавить его с помощью редактора форм в проводнике.

При создании типа сущности некоторые действия наследуются от *базового типа*. Например, при создании справочника на основе Sungero.CoreEntities.DatabookEntry от него наследуются действия **Save**, **Refresh** и др. В списке элементов они выделены серым цветом. У наследуемых действий можно изменять только часть настроек, например, отображаемое имя, описание, подсказку. Кроме этого, можно переопределить обработчики наследуемого действия.

Чтобы быстро найти нужное действие и доработать его, воспользуйтесь поиском: установите курсор на любое действие и начните вводить имя. Появится поле ввода, например,

. Далее выполняется поиск вниз от выделенного действия и подсвечивается первое найденное совпадение. Если повторно ввести то же самое значение, находится следующее совпадение и т.д. При достижении конца списка поиск начинается с начала.



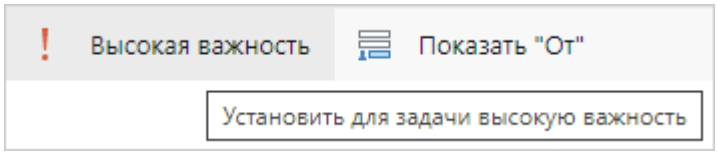
Именование

***Имя**. Должно быть уникальным в рамках одного типа сущности, состоять из букв латинского алфавита и цифр.

***Отображаемое имя**. Имя действия, которое будет отображаться в качестве названия кнопки на ленте.

***Подсказка**. Краткое описание действия. Отображается в заголовке всплывающей подсказки в десктоп-клиенте.

***Описание**. Подробное описание действия. Отображается во всплывающей подсказке:



Для *локализации* имени, описания и подсказки перейдите по ссылке **Локализовать**.

Параметры

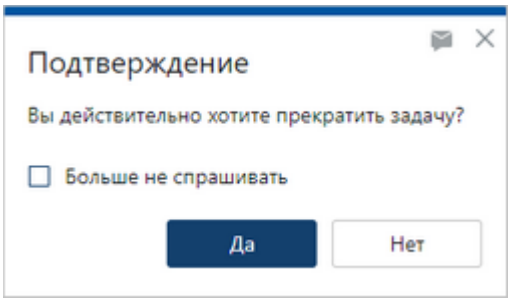
***Доступно в**. Форма, на которой будет доступно действие. Возможные значения:

- **Списке**. Действие списка сущностей;
- **Карточке**. Действие карточки сущности;
- **Карточке и списке**. Действие карточки и списка сущностей;
- **Дочерней коллекции**. Действие *свойства-коллекции* сущности.

В зависимости от выбранной формы у действия могут появляться дополнительные настройки.

***Доступно в клиентах**. По умолчанию добавленное действие доступно в веб-клиенте и в десктоп-клиенте. Если действие не поддерживается в одном из клиентов, то в выпадающем списке снимите нужный флажок.

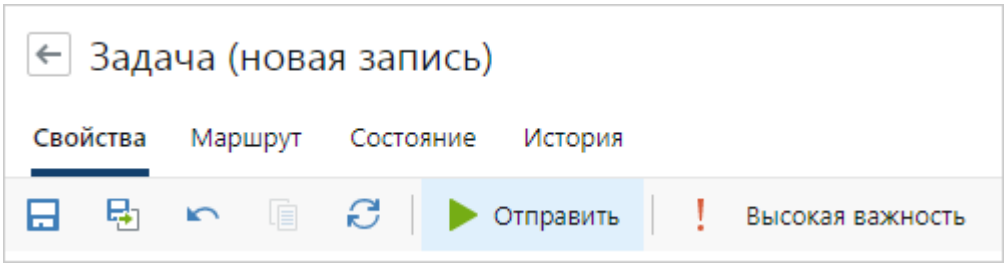
Запрашивать подтверждение перед выполнением. При выполнении действия будет появляться диалог подтверждения с текстом, который указан в полях ***Сообщение подтверждения** и **Пояснение к подтверждению**. Для локализации сообщения перейдите по ссылке **Локализовать**.:



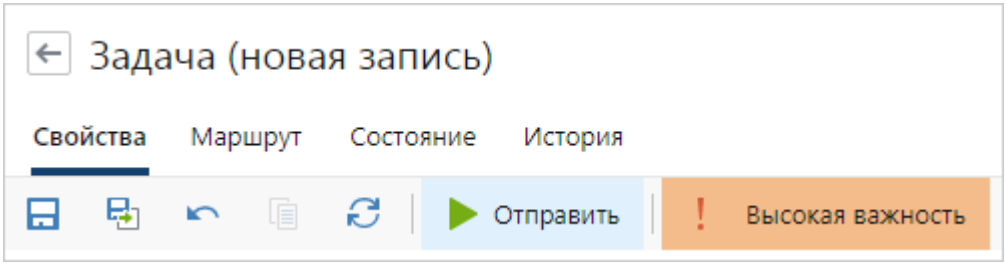
Позволить пользователю отключать подтверждение. Флажок доступен, если установлен флажок **Запрашивать подтверждение перед выполнением**. Если в среде разработки флажок установлен, в веб-клиенте в диалоге отображается флажок **Больше не спрашивать**. Таким образом пользователь может по собственному желанию отключить появление диалога подтверждения.

Действие-переключатель. Кнопка, соответствующая действию, будет иметь два состояния: не нажата и нажата.

Кнопка не нажата



Кнопка нажата



Например, переключатели добавляются, если в карточке сущности нужно скрыть часть контролов в зависимости от того, нажата ли кнопка, соответствующая действию. Подробнее см. в разделе «Переключатель». Для задания начального состояния кнопки (нажата или не нажата) можно в обработчике события Выполнение/Возможность выполнения использовать аргумент события e.IsToggled.

Является действием создания. Код действия будет вызываться вместо обычного создания сущности. Например, если для создания сущности нужно запросить у пользователя информацию о ней.

Доступно при отсутствии выделенных записей в списке. Действие доступно в том числе, если в списке сущностей не выделен ни один элемент.

Действие доступно, если выделено до 100 записей в списке. Действие доступно для нескольких выбранных сущностей, если их не более 100. Флажок отображается, если в поле ***Доступно в** выбрано значение **Списке**, **Карточке и списке** или **Дочерней коллекции**.

Действие доступно, если выделено любое количество записей в списке. Действие доступно для любого количества выбранных сущностей. Флажок отображается, если в поле ***Доступно в** выбрано значение **Списке**, а в поле ***Доступно в клиентах** указано значение **Веб-клиент**.

Обновить список после выполнения действия. После выполнения действия автоматически обновляется содержимое списка. Например, после подписания договора автоматически обновляется реестр договоров, чтобы на документе в списке появился значок . Флажок отображается, если в поле ***Доступно в** выбрано значение **Списке** или **Карточке и списке**.

Общее для всех коллекций. Действие доступно для всех свойств-коллекций типа сущности. Флажок отображается, если в поле ***Доступно в** выбрано значение **Дочерней коллекции**.

Проверять возможность выполнения один раз для коллекции. Событие Возможность выполнения (CanExecute) вычисляется один раз для всей коллекции, а не для каждого элемента по отдельности. По умолчанию флажок установлен и отображается, если в поле ***Доступно в** выбрано значение **Дочерней коллекции**. Подробнее см. в разделе «Оптимизация вычисления возможности выполнения действия».

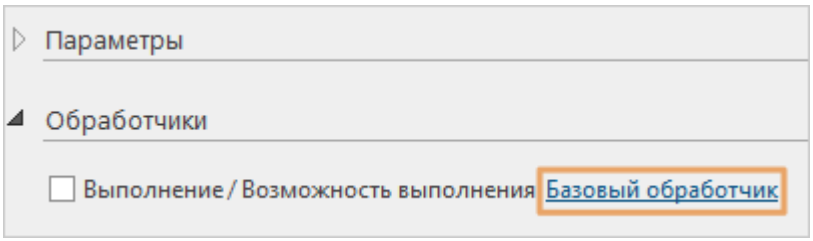
Маленькая иконка и **Большая иконка**. Иконки для маленькой и большой кнопок на ленте или пункта контекстного меню. Иконки должны быть в формате SVG и в размерах 16x16 и 32x32 соответственно. Иконки, используемые в *базовом решении*, можно найти в локальной папке исходных кодов. Расположение папки задается при установке системы. Например, в папке C:\DEVRX\Base\Sungero.Company\Sungero.Company.ClientBase\Icons можно найти файлы иконок модуля «Компания».

***Свойство-коллекция**, для которого будет доступно действие. Флажок отображается, если в поле ***Доступно в** выбрано значение **Дочерней коллекции**.

Обработчики

Обработчики. Добавление обработчиков событий действий сущности. Код обработчиков событий выполняется в клиентском приложении, когда происходит соответствующее событие.

Иногда нужно посмотреть прикладной код интересующего действия. Если обработчик унаследован, то в его редакторе не отображается код этого действия. Чтобы открыть базовый обработчик, нажмите на ссылку **Базовый обработчик**:



ПРИМЕЧАНИЕ. Ссылка доступна, если одновременно выполнены условия:

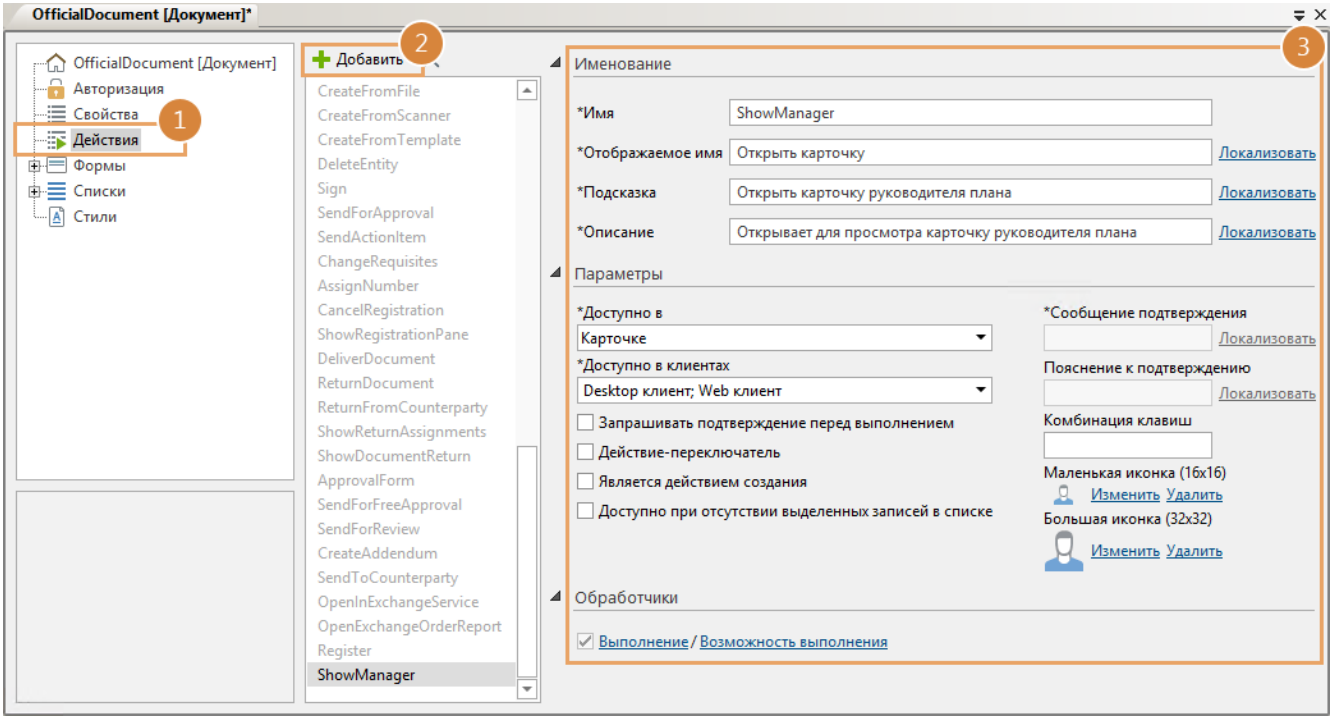
- у выбранной сущности не переопределен обработчик;
- базовый обработчик находится на прикладном уровне.

Создание действия

Если требуется создать действие, похожее на уже существующее, создайте его копированием.

В разделе описан пример создания действия для типа документа. Для остальных типов сущностей порядок создания аналогичный.

1. В редакторе типа сущности перейдите в узел «Действия».



2. Нажмите на кнопку **Добавить**.
3. В списке элементов выделите добавленное действие, задайте для него параметры и добавьте обработчики событий.

При необходимости добавьте действия на ленты карточки сущности и списка сущностей. Для этого используйте узел «Лента» в редакторе типа сущности.

События действий сущности

События действий задаются в редакторе типа сущности в узле «Действия». Обработчики событий выполняются в клиентском приложении.

ПРИМЕЧАНИЕ. Обработчики событий «Выполнение» и «Возможность выполнения» также задается для результатов выполнения задания.

Выполнение (<Имя действия>)

Назначение:

Событие определяет логику, которая выполняется при вызове действия.

Аргументы событий:

- _obj – определяет:
  - сущность, карточка которой открыта, если действие относится к карточке сущности;
  - выбранную сущность, если действие относится к списку сущностей;
  - выбранную запись, если действие относится к свойству-коллекции.
- e.Entity – сущность, для которой выполняется действие. Если действие относится к свойству-коллекции, то это дочерняя сущность;
- e.RootEntity – главная сущность, доступна для действий, у которых в поле ***Доступно в** выбрано значение **Дочерней коллекции**;
- e.CloseFormAfterAction – определяет необходимость закрытия окна после выполнения действия;
- e.FormType – определяет тип формы, из которой вызывается действие. Может принимать значения FormType.Card (карточка сущности) или FormType.Collection (список сущностей);
- e.IsToggled – определяет текущее состояние кнопки-переключателя. Возможные значения:
  - **True**. Кнопка нажата;
  - **False**. Кнопка не нажата;
- e.Action – информация о действии. Позволяет получить имя, иконку и описание действия;
- e.ClearMessageAfterAction – определение необходимости очистки сообщений валидации после выполнения действия;
- e.Validate – признак того, что сущность валидная. Проверяется заполненность обязательных полей, длина строк и т.д. Возможные значения:
  - **True**. Сущность валидная (доступно выполнение действия);
  - **False**. Сущность не валидная. При выполнении действия появится сообщение, заданное в **AddError**;
- e.AddError() – добавление сообщения об ошибке. Выполняемые операции не будут прерваны, их нужно прекратить вручную. Для этого в коде после валидации необходимо проверить состояние сущности с помощью метода **Validate()**;

- e.AddInformation() – добавление информационного сообщения. Выполняемые операции не будут прерваны;
- e.AddWarning() – добавление предупреждения. Выполняемые операции не будут прерваны.
- e.Params – дополнительные параметры.

В зависимости от установленных настроек действия, могут быть сгенерированы дополнительные параметры и свойства обработчиков:

- если для действия установлен флажок **Действие доступно, если выделено до 100 записей в списке**, то имя свойства изменяется на **_objs**;
- если для действия установлен флажок **Действие доступно, если выделено любое количество записей в списке**, то имя свойства изменяется на **_objIds**;
- если для действия установлен флажок **Доступно при отсутствии выделенных сущностей**, то из обработчика не будет доступа к сущности и списку сущностей;
- если действие относится к свойству-коллекции, то добавляется **_all** для обращения ко всем значениям коллекции.

Возможность выполнения (Can<Имя действия>)

Назначение:

Событие определяет, при каких условиях действие доступно. Возвращаемые значения:

- **True**. Действие доступно. Значение по умолчанию;
- **False**. Действие недоступно.

ВАЖНО. В обработчике события **Возможность выполнения** не рекомендуется вызывать серверные функции (с атрибутом **Remote**) и использовать логику, которая будет долго обрабатываться. Данный обработчик выполняется часто. Поэтому, если в нем есть серверные функции, то на веб-сервер отправляется большое количество запросов, что, в свою очередь, влияет на быстродействие системы.

Аргументы событий:

- _obj – определяет:
    - сущность, карточка которой открыта, если действие относится к карточке сущности;
    - выбранную сущность, если действие относится к списку сущностей;
    - выбранную запись, если действие относится к свойству-коллекции.
- e.Entity – сущность, для которой выполняется действие. Если действие относится к свойству-коллекции, то это дочерняя сущность;
- e.FormType – определяет тип формы, из которой вызывается действие. Может принимать значения FormType.Card (карточка сущности) или FormType.Collection (список сущностей);
- e.Params – дополнительные параметры.

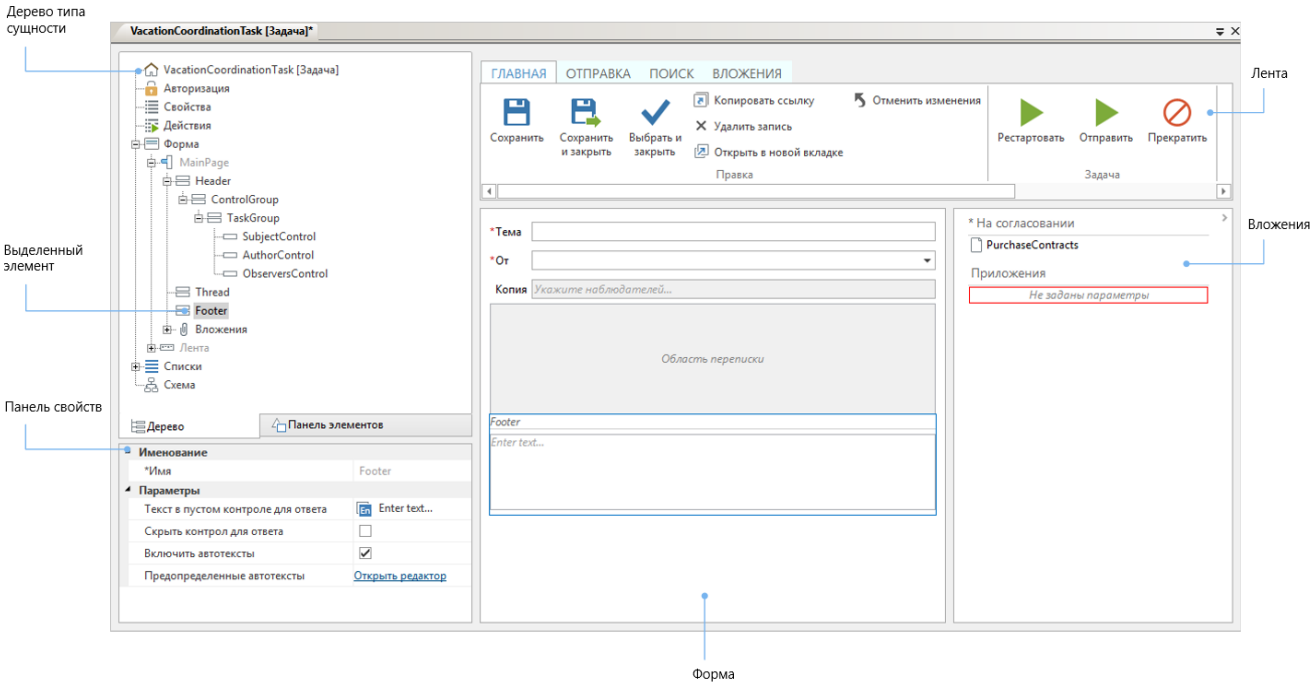
В зависимости от установленных настроек действия, могут быть сгенерированы дополнительные параметры и свойства обработчиков:

- если для действия установлен флажок **Действие доступно, если выделено до 100 записей в списке**, то имя свойства изменяется на **_objs**;
- если для действия установлен флажок **Действие доступно, если выделено любое количество записей в списке**, то имя свойства изменяется на **_objIds**;
- если для действия установлен флажок **Доступно при отсутствии выделенных сущностей**, то из обработчика не будет доступа к сущности и списку сущностей;
- если действие относится к свойству-коллекции, то добавляется **_all** для обращения ко всем значениям коллекции.

Форма

Прикладной разработчик в среде разработки задает внешний вид формы. В дальнейшем *аналитик* может в проводнике системы изменить эти настройки.

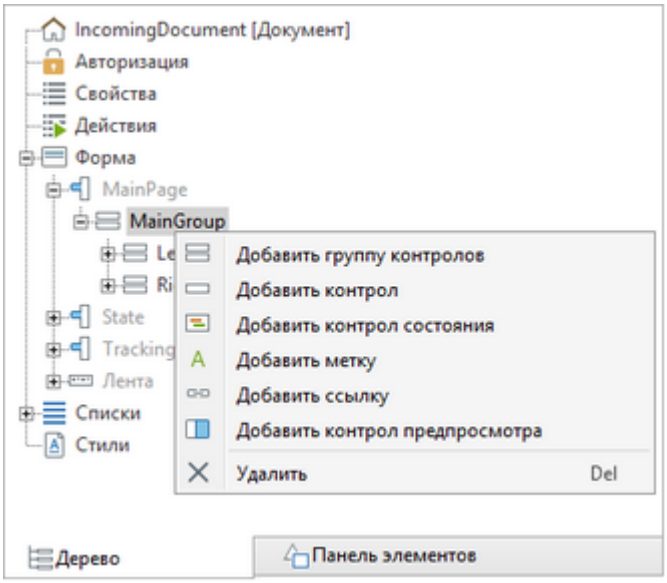
Используйте редактор форм, чтобы настроить вкладки и ленту карточки типа сущности документа, справочника, задачи, задания, задания на приемку или уведомления. Дополнительно в редакторе форм доступна настройка области вложений для задач, заданий и уведомлений.



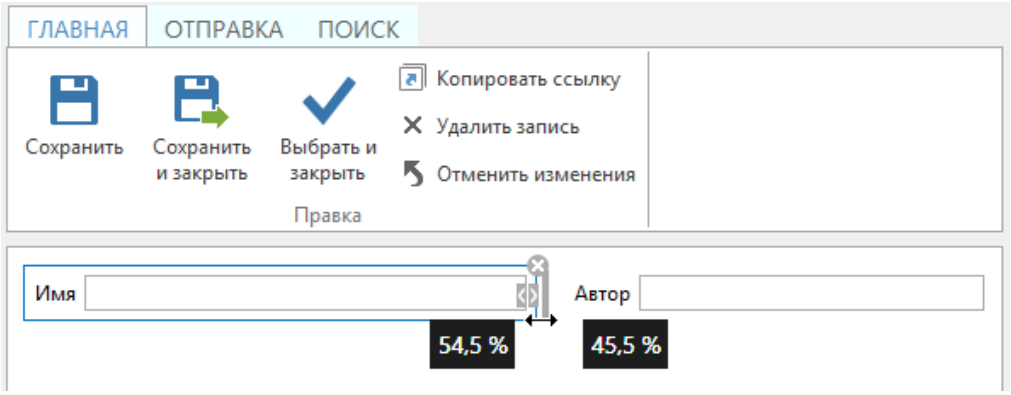
Элементы на форму можно добавлять следующими способами:

- перетаскиванием с панели элементов. Места для вставки подсветятся желтым цветом. При добавлении контрола и метки на форму появится выпадающий список для выбора связанного свойства. А при добавлении кнопки и переключателя на ленту – список для выбора действия.

- с помощью контекстного меню в дереве типа сущности:



Для элементов, добавленных на форму, можно настроить ширину с помощью кнопок или растягивая элемент с помощью мыши. При растягивании элемента изменяется размер колонки, в которой находится элемент, а при нажатии на кнопки изменяется количество колонок, которое он занимает.



При растягивании справа и слева от курсора отображается ширина колонок в процентах. Это поможет настроить одинаковую ширину колонок в разных карточках типов сущностей.

Элемент, который выбран на форме или ленте, выделяется синей рамкой. Если элемент выделен красной рамкой, проверьте его имя и параметры на панели свойств.

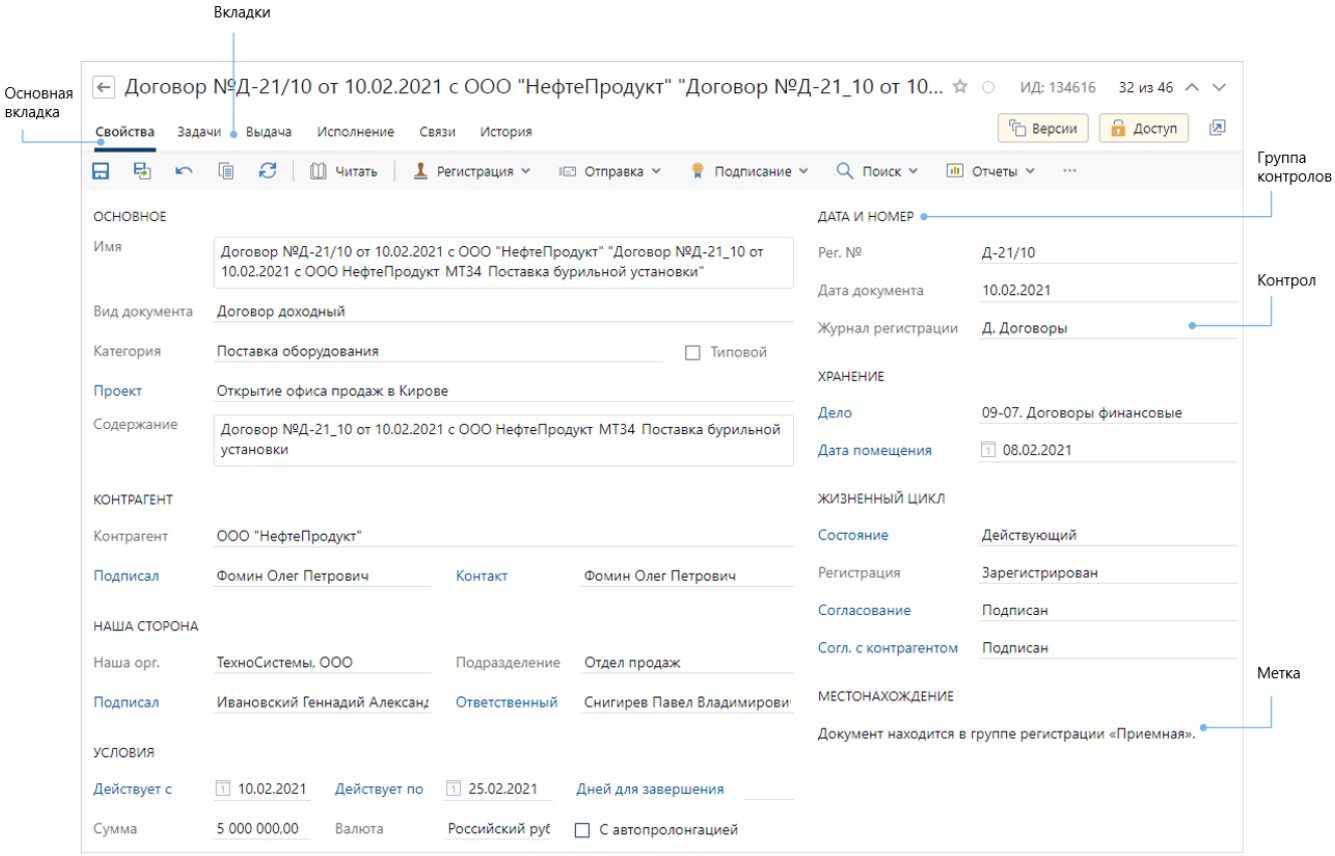
ПРИМЕЧАНИЕ. Чтобы быстро перейти из формы к соответствующему пункту в узле **Действия** или **Свойства**, щелкните два раза левой кнопкой мыши по кнопке или свойству.

Вкладки

В карточке сущности есть основная вкладка (MainPage) и дополнительные. На вкладки можно добавлять:

- контролы для просмотра и редактирования значений свойств типов сущностей в карточках;
- метки – контролы для просмотра значений свойств типов сущности;
- ссылки – контролы, при нажатии на которые выполняются клиентские функции или открываются внешние ссылки;

- контролы состояния для отображения информации, которая понадобится при работе с сущностью. Например, с помощью контрола состояния в карточке поручения отображается информация об этапах работы с поручением, а в карточке задачи на согласование по регламенту – информация об этапах согласования документа;
- предпросмотр для удобства работы и быстрого ознакомления с документами. Контрол позволяет отобразить содержимое, не загружая сам документ и не открывая его в приложении-редакторе;
- группы контролов для группировки контролов и настройки их расположения на карточке сущности.
- сторонние контролы – для отображения разработанных контролов.



Контролы рекомендуется располагать на основной вкладке (MainPage). Она обязательная, ее нельзя скрыть или удалить. По умолчанию называется «Свойства». Это имя можно изменить для типов справочников и документов.

ПРИМЕЧАНИЕ. Системные вкладки («История», «Связи», «Переписка» и т.д.) при необходимости можно скрывать в веб-клиенте. Через программный код это делать не рекомендуется, т.к. в таком случае вкладки будут отображаться в редакторе форм, но скрыты от пользователя.

При *перекрытии* типов сущностей учитывайте особенности доработки вкладок.

Добавление вкладки

Добавляйте на форму вкладки, чтобы размещать на них *контролы*, необходимые для отдельных сценариев работы. Например, на вкладке «Выдача» в карточке документа находятся контролы для фиксирования информации о выдаче и возврате документа.

Чтобы добавить вкладку:

1. В дереве типа сущности перейдите в узел «Форма» и в контекстном меню узла выберите пункт **Добавить вкладку**.
2. При необходимости измените расположение вкладки в карточке. Для этого перетащите вкладку выше или ниже в дереве сущности. Места для вставки подсвечиваются желтым цветом.

3. Заполните поля:

   ***Имя**. Должно быть уникальным в рамках карточки, состоять из букв латинского алфавита и цифр.

   ***Отображаемое имя**. Имя, которое будет отображаться в качестве названия вкладки в карточке сущности. Для *локализации* имени нажмите на кнопку **Локализовать**.

   ПРИМЕЧАНИЕ. Для основной вкладки это поле необязательное. Если отображаемое имя не указано, используется значение по умолчанию – «Свойства».

   Отображать в форме. Вкладка будет отображаться на карточке сущности. При необходимости в дальнейшем можно скрыть или отобразить вкладку программно.
4. Добавьте на вкладку необходимые группы контролов, контролы, метки, ссылки, контролы состояния, предпросмотр.

В результате в карточке типа сущности появится вкладка. После публикации решения новая вкладка будет расположена справа от системных.

Группа контролов

Группы контролов добавляются на вкладки карточки и могут быть вложены в другие группы. Группа представляет собой таблицу, в ячейках которой находятся контролы, ссылки, метки, контролы состояния, сторонние контролы и другие вложенные группы.

Чтобы добавить группу контролов:

1. В дереве типа сущности выделите узел «Форма» и перейдите на вкладку «Панель элементов».
2. Добавьте на форму элемент **Группа контролов**.

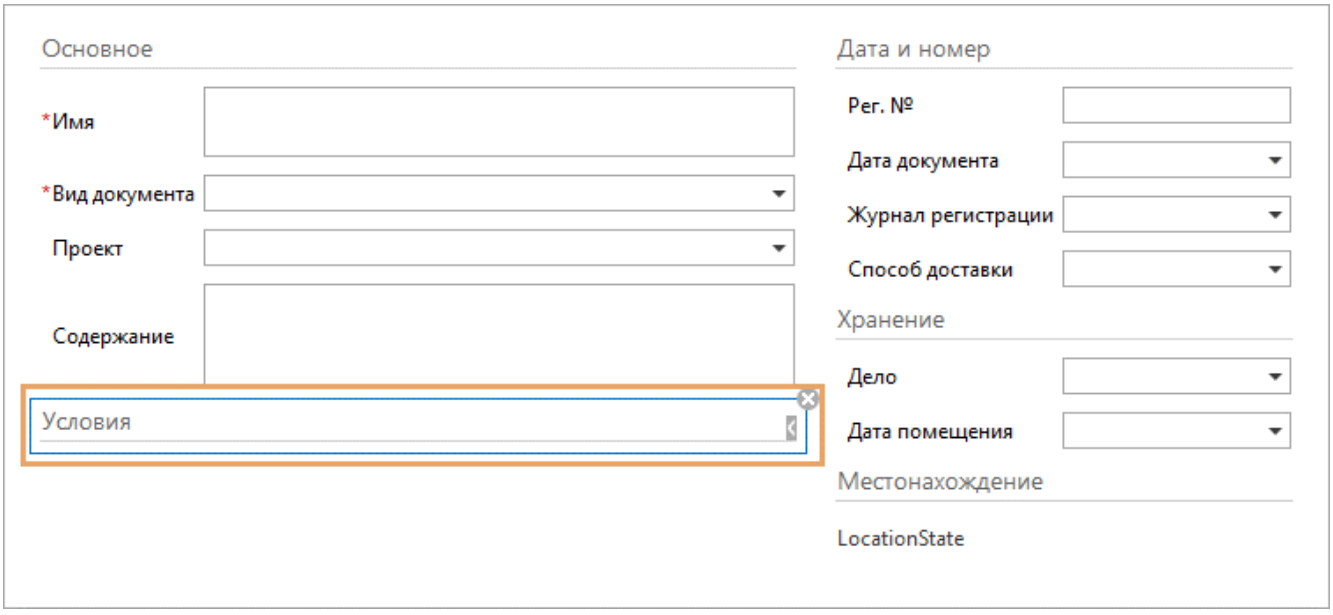
   ПРИМЕЧАНИЕ. Чтобы добавить группу контролов на другую вкладку, выберите ее в дереве типа сущности.
3. Измените имя и параметры группы на панели свойств:

Именование	
***Имя**	Должно быть уникальным в рамках вкладки, состоять из букв латинского алфавита и цифр
Отображаемое имя	Заголовок группы в карточке сущности. Для *локализации* имени нажмите на кнопку **Локализовать**
Параметры	
Использовать контролы предка	После перекрытия типа сущности снимите флажок, чтобы контролы в группе стали доступны для изменения

Именование	
Выравнивать размеры по группам	Установите флажок, если в группу будут вложены другие группы. Свойство выравнивает расстояния между именами контролов и контролами во вложенных группах. Подробнее...
Отображать в форме	Группа контролов будет отображаться в карточке сущности
***Тип группы**	Возможные значения: «Панель без границ», «Панель с заголовком», «Сворачиваемая панель», «Панель с заголовком без границ». Чтобы не перегружать интерфейс большим количеством линий, рекомендуется использовать панели без границ и панели с заголовком без границ
Изменять ширину вложенных групп	Установите флажок, чтобы пользователи могли растягивать дочерние группы, вложенные в родительскую группу. Подробнее...
***Количество колонок в группе (1-10)**	Группа контролов будет содержать указанное количество колонок. Может принимать значение от 1 до 10
***Номер колонки** ***Номер строки** ***Количество колонок** ***Количество строк**	Свойства задаются и изменяются автоматически при добавлении, перемещении или удалении групп контролов на форме. Изменять свойства вручную не рекомендуется

4. Добавьте в группу необходимое количество контролов.

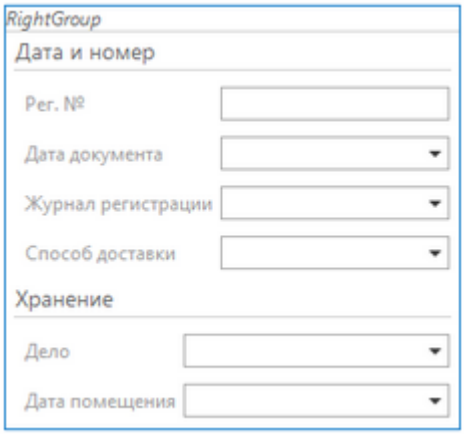
В результате на основной вкладке карточки типа сущности появится группа контролов, например:



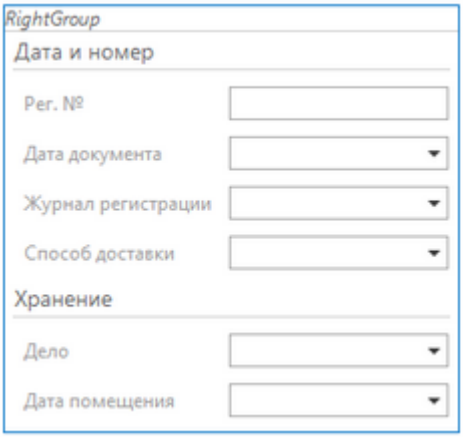
Группу можно удалить по кнопке . При этом в веб-клиенте вместе с ней также удаляются все контролы, которые добавлены на форму в представлении формы.

Использование параметра «Выравнивать размеры по группам»

Флажок «Выравнивать размеры по группам» для группы RightGroup не установлен

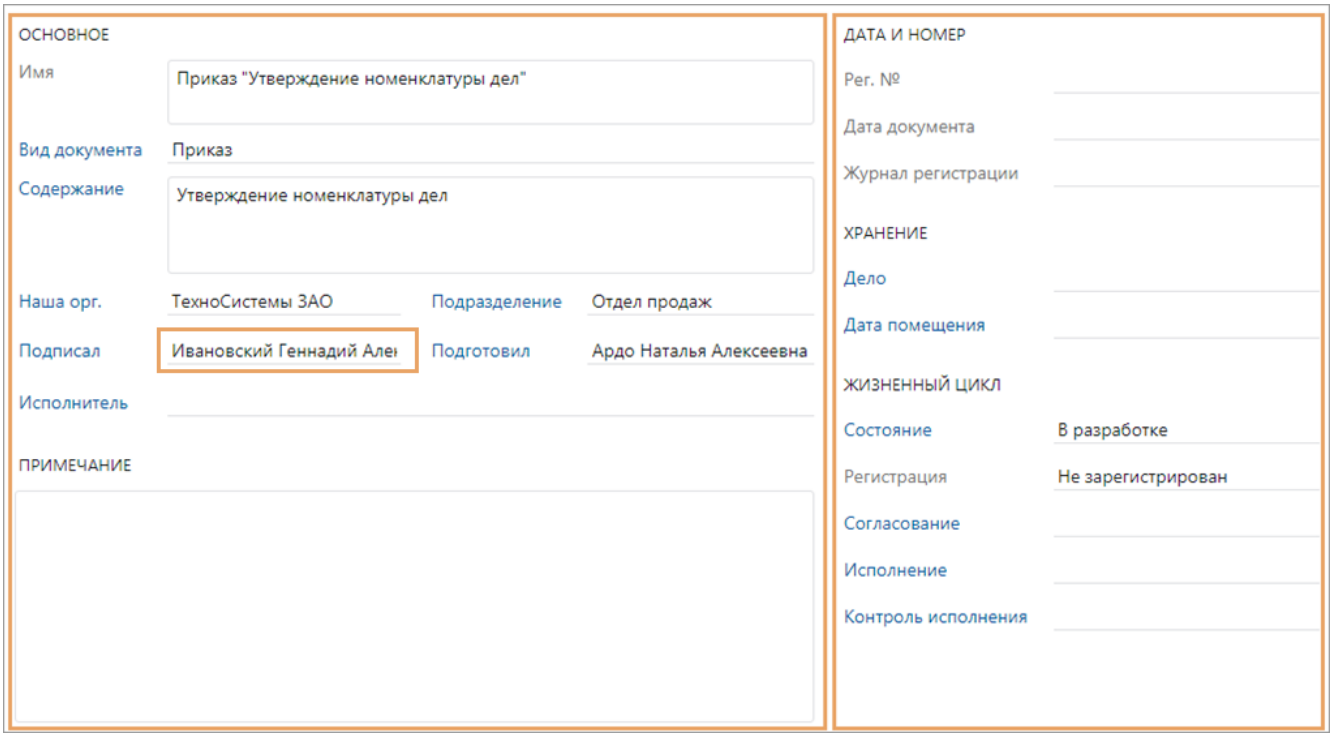


Флажок «Выравнивать размеры по группам» для группы RightGroup установлен



Использование параметра «Изменять ширину вложенных групп»

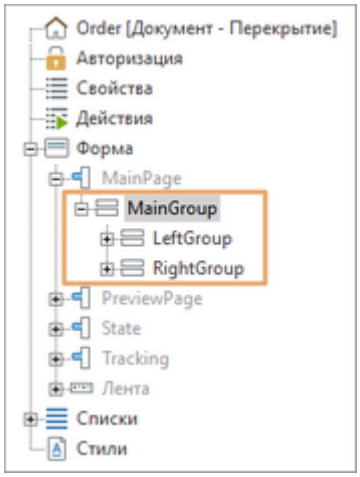
Форма типа сущности часто состоит из двух групп:



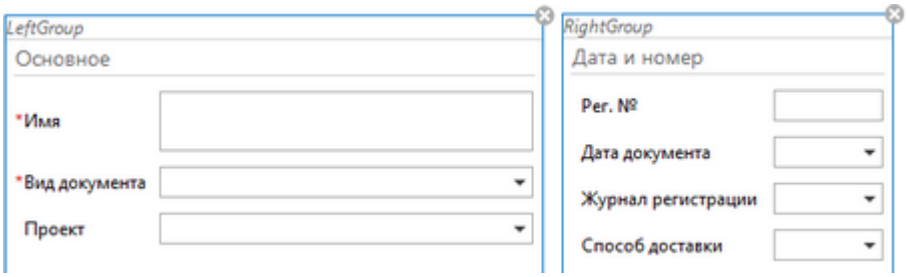
Когда экран небольшой, а данных в полях карточек много, то часть информации обрезается. В примере обрезана информация в поле **Подписал**. Разработчик может настроить форму так, чтобы пользователи могли растягивать ширину двух групп. Это поможет пользователям быстро просматривать интересующие данные.

Чтобы настроить растягивание групп, на панели свойств для родительской группы установите флажок **Изменять ширину вложенных групп**. Флажок доступен, если выполнены условия:

- в родительскую группу вложены две дочерние группы и нет других элементов:



- на форме вложенные дочерние группы расположены по горизонтали:



- у родительской группы снят флажок **Использовать контролы предка**.

Флажок автоматически снимается, если группа перестает удовлетворять условиям.

Если флажок установлен, в веб-клиенте между дочерними группами появляется разделитель, с помощью которого можно настроить нужную ширину. Если в дочерние группы добавить две другие группы и выполнить остальные условия, то между ними также появится разделитель.

Настройка применяется для всех карточек одного типа сущности. Например, если настроить ширину в карточке одного договора, то при открытии другого подбирать ширину заново не придется.

Минимальный размер, который пользователь можно настроить для группы, составляет 20% от общей ширины.

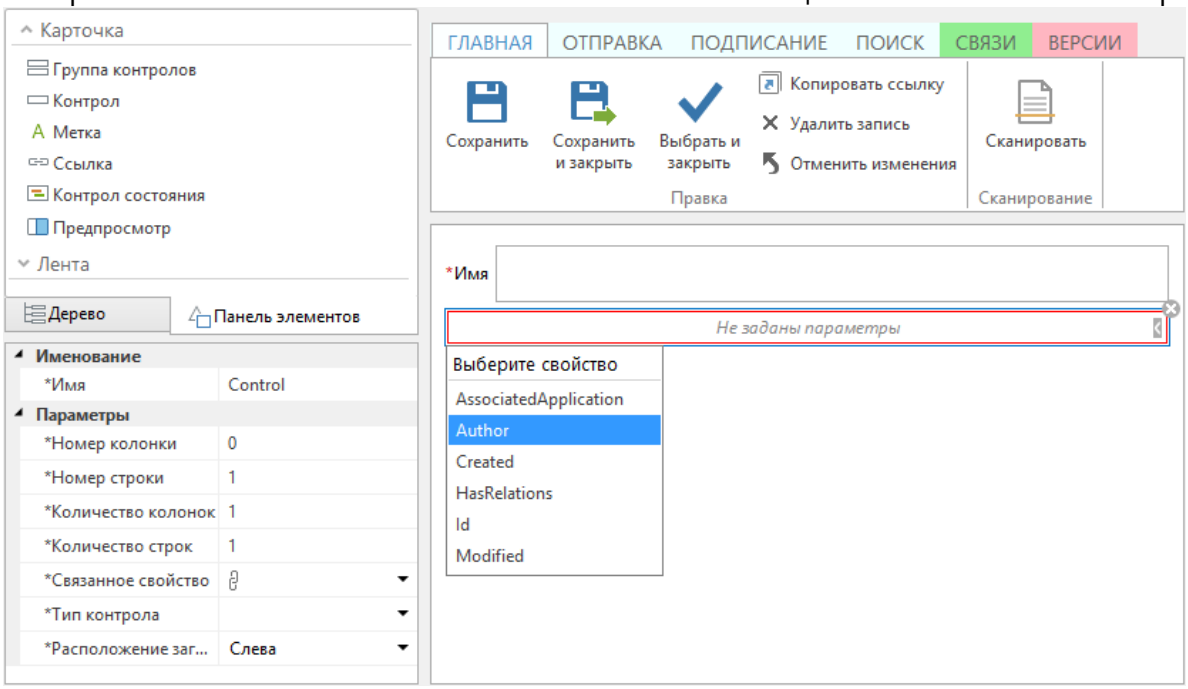
Контрол

Контрол добавляется в группу и предназначен для визуального отображения и редактирования значений свойств типов сущностей в карточках. Чтобы добавить контрол:

1. В дереве типа сущности выделите узел «Форма» и перейдите на вкладку «Панель элементов».
2. Добавьте на форму элемент **Контрол**.

   ПРИМЕЧАНИЕ. Чтобы добавить контрол на другую вкладку, выберите ее в дереве типа сущности.

3. В выпадающем списке выберите свойство типа сущности, значение которого будет отображаться с помощью контрола:



ПРИМЕЧАНИЕ. В выпадающем списке отображаются свойства, которых еще нет на форме.

4. Измените имя и параметры контрола на панели свойств. Параметры зависят от типа свойства, с которым связан контрол.

Основные параметры

Именование

***Имя**	Имя формируется по формату Control<порядковый номер>. Для удобства обращения к контролу в программном коде измените имя на максимально емкое и понятное. Имя должно быть уникальным в рамках группы контролов, состоять из букв латинского алфавита и цифр. СОВЕТ. Если с помощью мыши перетащить контрол на форму и выбрать связанное свойство в выпадающем списке, то имя контрола заполнится автоматически и будет совпадать с именем свойства

Параметры

***Номер колонки** ***Номер строки** ***Количество колонок** ***Количество строк**	Свойства задаются и изменяются автоматически при добавлении, перемещении или удалении контролов на форме. Изменять свойства вручную не рекомендуется
***Связанное свойство**	Свойство типа сущности, значение которого будет отображаться с помощью контрола. Чтобы быстро перейти к параметрам связанного свойства, нажмите на кнопку или в редакторе форм дважды щелкните по контролу. Откроется узел «Свойства», связанное свойство будет выделено в списке элементов

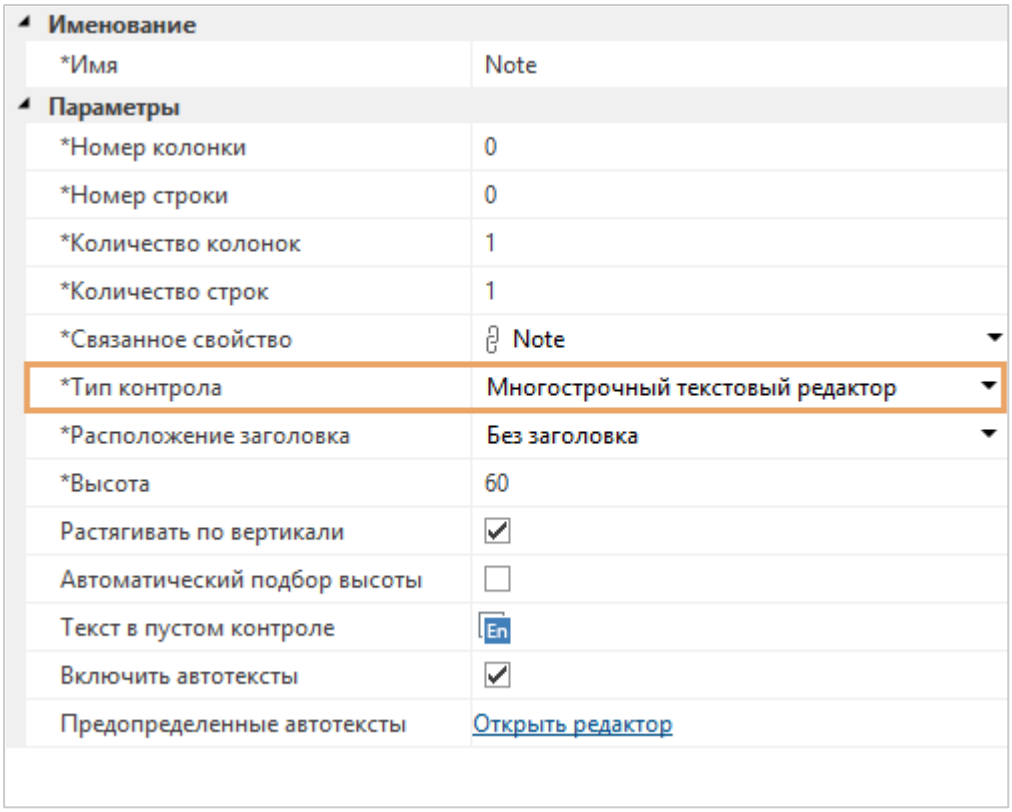
***Тип контрола**	Тип контрола, в котором будет отображаться значение свойства на карточке. Заполняется автоматически при выборе связанного свойства в зависимости от его типа
***Расположение заголовка**	Возможные значения: **Сверху**, **Слева**, **Без заголовка**. Рекомендуется скрывать заголовок контрола, если: • он связан со свойством-коллекцией и отображается в виде таблицы; • в группе с говорящим названием располагается только один контрол. В остальных случаях рекомендуется располагать заголовок контрола слева. Параметр не отображается для контролов, которые связаны со свойством типа **Логическое**

Параметры для свойства типа Текст или Строка

Параметры

***Высота**	Высота контрола в карточке
Растягивать по вертикали	Контрол будет автоматически растягиваться по вертикали при изменении размера карточки в проводнике системы. Флажок автоматически снимается, если установлен **Автоматический подбор высоты**
Автоматический подбор высоты	Контрол будет автоматически растягиваться по вертикали при изменении размера текста в контроле. Флажок автоматически снимается, если установлен **Растягивать по вертикали**
***Высота в строках**	Высота контрола по умолчанию, задается в строках. Параметр отображается, если установлен флажок **Автоматический подбор высоты**
Макс. высота в строках	Максимальная высота контрола, задается в строках. Параметр отображается, если установлен флажок **Автоматический подбор высоты**. Чтобы максимальная высота контрола была неограниченна, укажите значение **0**
Текст в пустом контроле	Текст с подсказкой для пользователя, которая будет отображаться, если в контроле не указано значение. Для *локализации* текста нажмите на кнопку **Локализовать**
Включить автотексты	При заполнении контрола пользователи смогут написать свой текст или использовать готовые часто используемые автотексты
Предопределенные автотексты	Варианты автотекстов для заполнения контрола. Параметр отображается, если установлен флажок **Включить автотексты**

ПРИМЕЧАНИЕ. Для свойства типа «Строка» дополнительные свойства появляются, если в поле ***Тип контрола** выбрано значение «Многострочный текстовый редактор».



Параметры для свойства типа Картинка

Параметры

***Ширина изображения** ***Высота изображения**	Ширина и высота изображения в карточке
Изображение по умолчанию	Изображение, которое будет отображаться в карточке до тех пор, пока пользователь не загрузит другое изображение. Чтобы добавить изображение по умолчанию, перейдите по ссылке **Изменить**

Параметры для свойства типа Ссылка

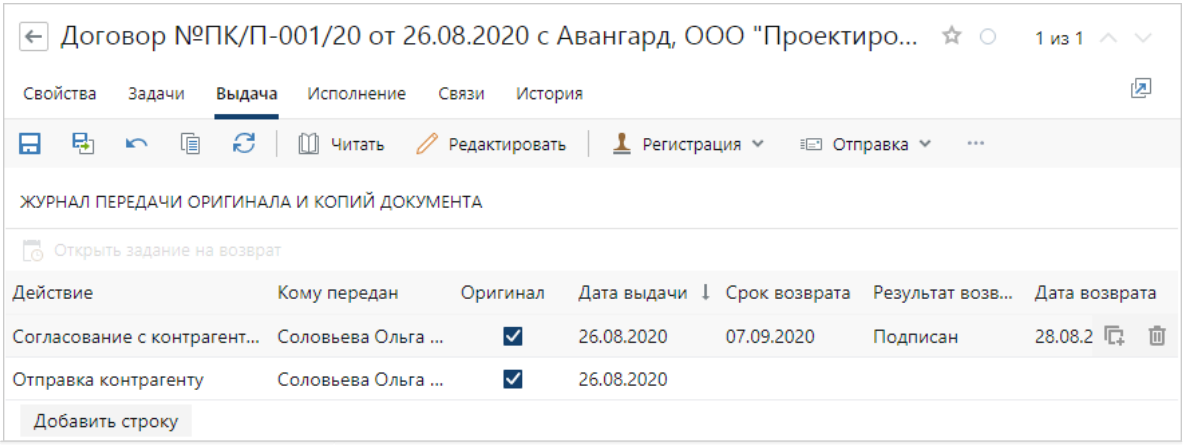
Параметры

Дополнительные свойства	Дополнительные свойства типа сущности, значения которых будут отображаться в контроле. Параметр доступен, если заполнен параметр ***Связанное свойство**. Указать можно только свойства типа Ссылка. Заполняется, если в списке нужно отображать ссылки на записи разных справочников. Например, в карточке договора для поля **Подписал** нужно настроить возможность выбирать сотрудников нашей организации и сотрудников контрагентов. Для этого добавьте свойства Employee (Сотрудник), Contact (Контакт) и Person (Персона). Затем в настройках контрола в поле ***Связанное свойство** укажите Employee, а в поле **Дополнительные свойства** – Contact и Person. В результате в веб-клиенте при заполнении поля **Подписал** в выпадающем списке доступны для выбора записи из справочников **Сотрудники**, **Контакты** и **Персоны**
Текст в пустом контроле	Текст с подсказкой для пользователя, которая будет отображаться, если в контроле не указано значение. Для *локализации* текста нажмите на кнопку **Локализовать**

Параметры для свойства типа Коллекция

Параметры

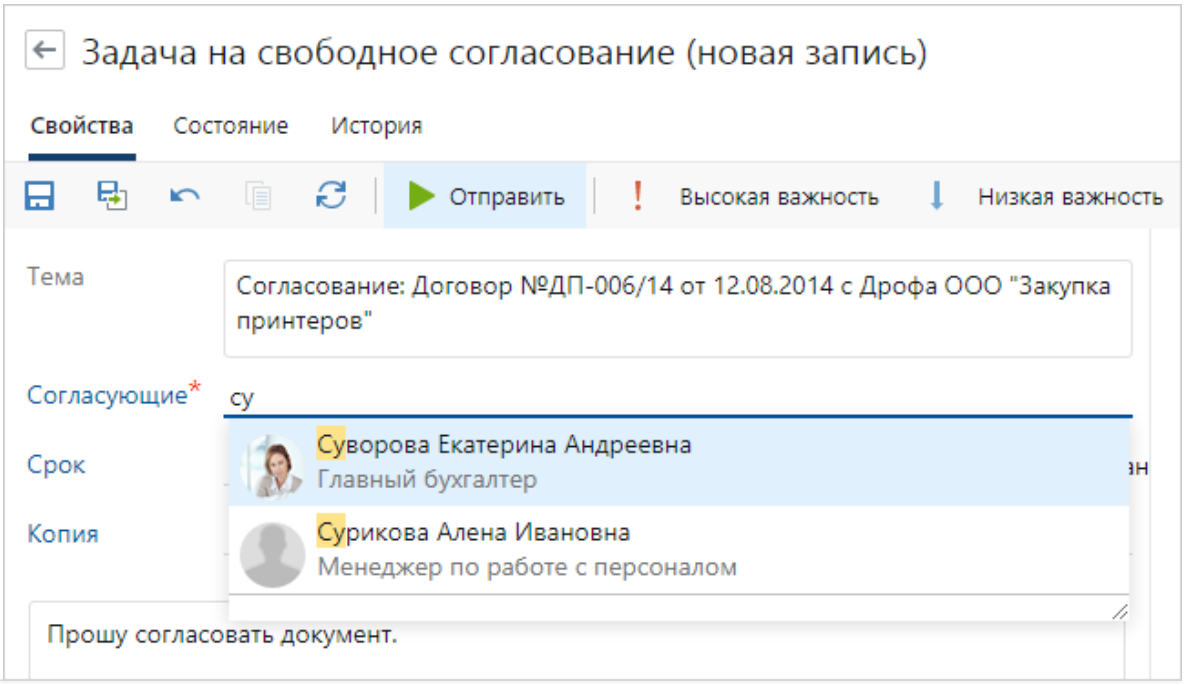
Тип контрола «Редактор свойства-коллекции»



***Высота**	Высота контрола в карточке
Растягивать по вертикали	Контрол будет автоматически растягиваться по вертикали при изменении размера карточки в проводнике системы

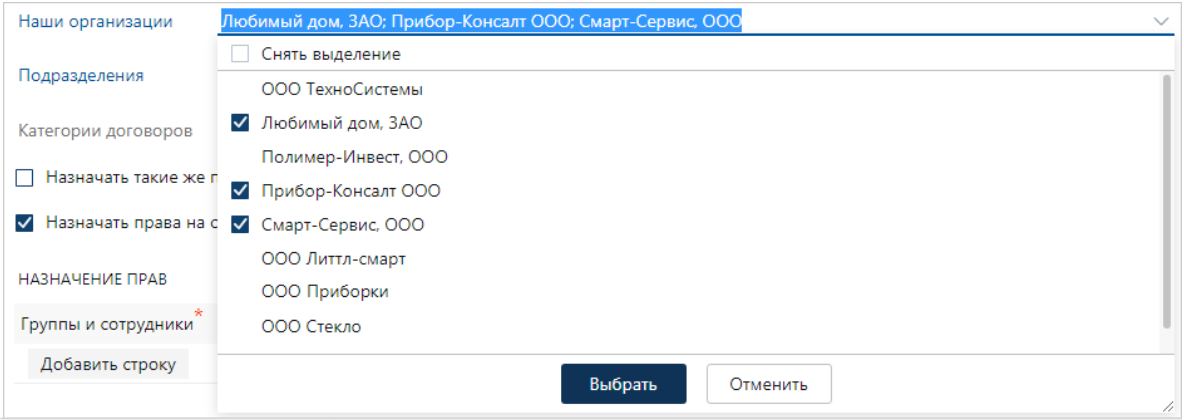
Параметры

Тип контрола «Редактор для множественного выбора с вводом»



***Связанное свойство в коллекции**	Дочернее свойство коллекции, значение которого будет отображаться с помощью контрола. Для выбора доступны только свойства типа **Ссылка**
***Разрешить повторяющиеся значения**	Значения, введенные в контрол, могут повторяться
Текст в пустом контроле	Текст с подсказкой для пользователя, которая будет отображаться, если в контроле не указано значение. Для *локализации* текста нажмите на кнопку **Локализовать**

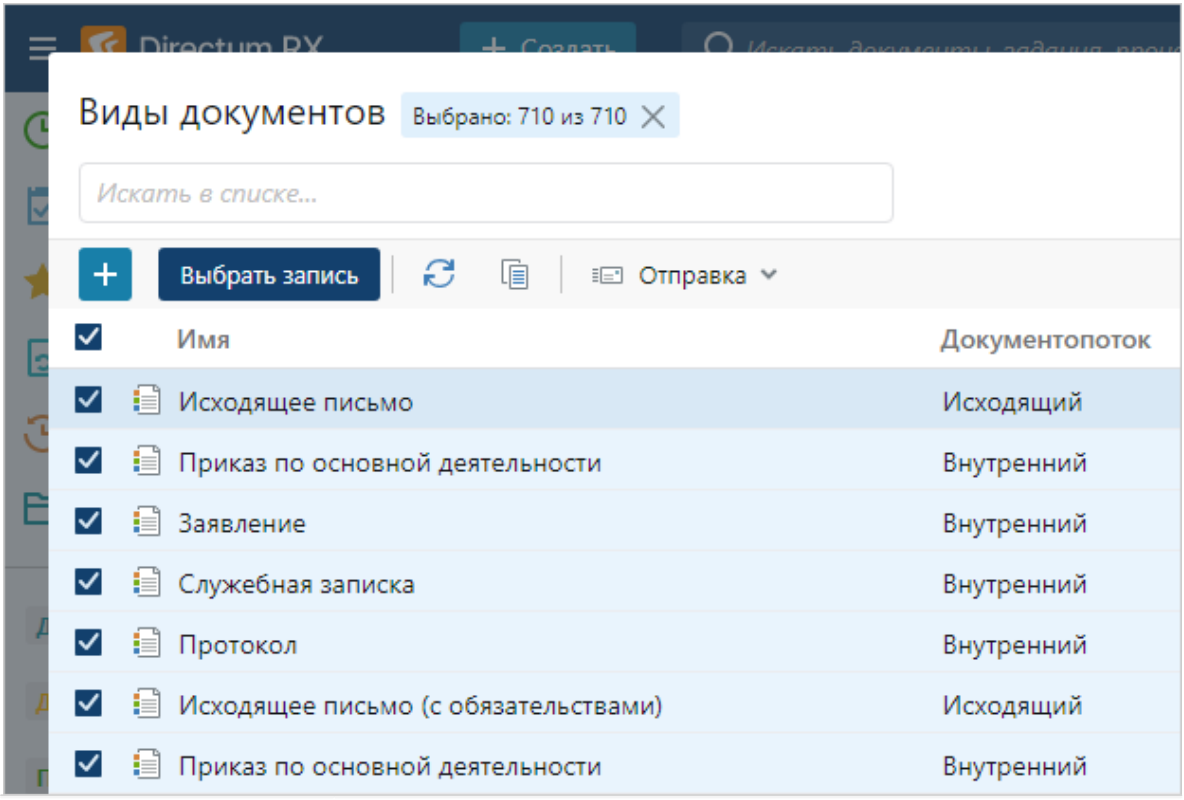
Тип контрола «Редактор для множественного выбора»



***Связанное свойство в коллекции**	Дочернее свойство коллекции, значение которого будет отображаться с помощью контрола. Для выбора доступны только свойства типа **Ссылка**

Тип контрола «Редактор для множественного выбора из списка»

Параметры

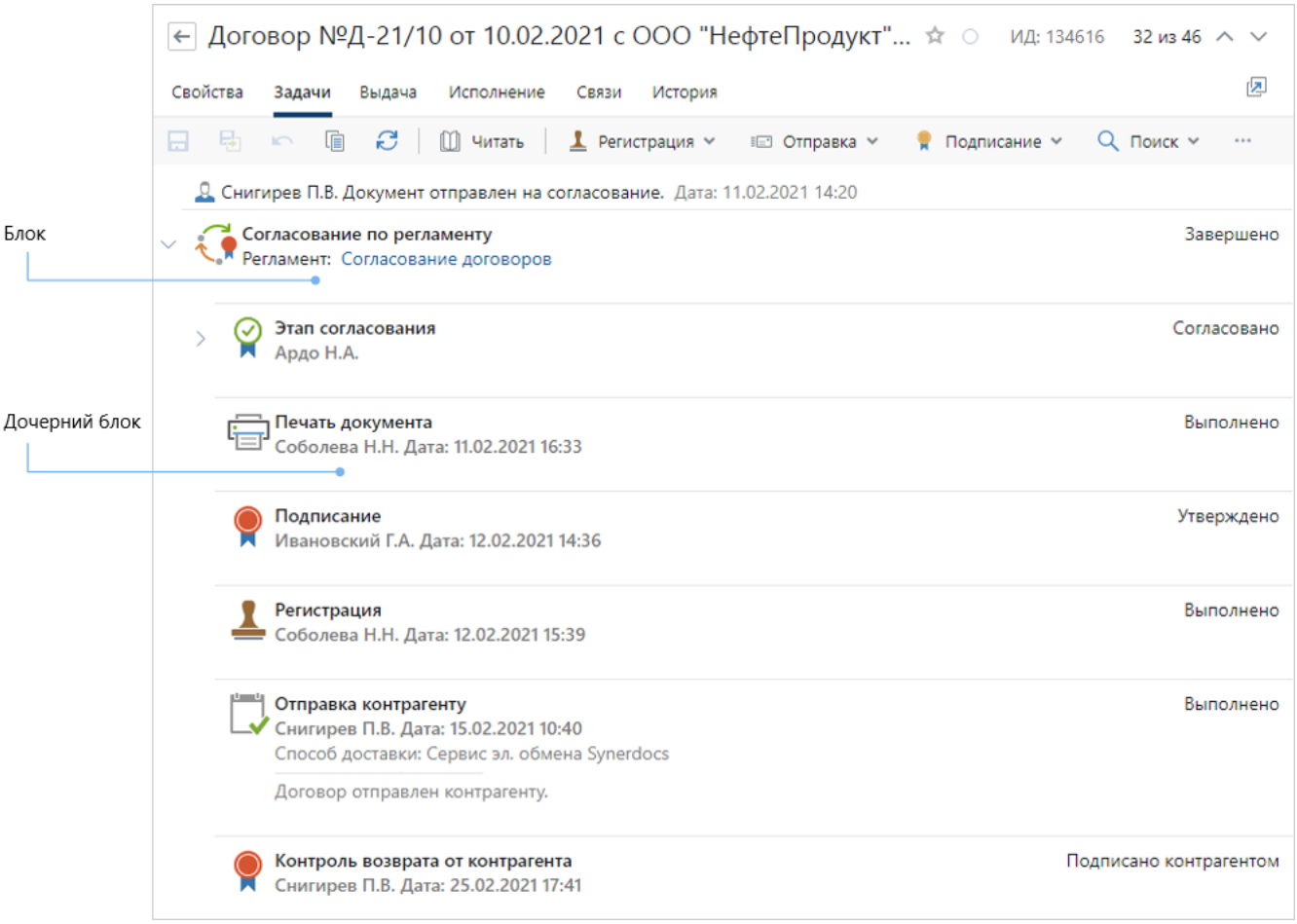


***Связанное свойство в коллекции**	Дочернее свойство коллекции, значение которого будет отображаться с помощью контрола. Для выбора доступны только свойства типа **Ссылка**
Дополнительные свойства	Дополнительные дочерние свойства коллекции, значения которых будут отображаться в контроле. Поле доступно, если заполнено поле ***Связанное свойство в коллекции**. Указать можно только свойства типа Ссылка. Заполняется, если в списке для множественного выбора нужно отображать ссылки на записи разных справочников. Например, в карточке договора добавляется поле **Подписали**, для которого нужно настроить возможность выбирать сотрудников нашей организации и сотрудников контрагентов. Для этого в коллекцию добавьте свойства Employee (Сотрудник), Contact (Контакт) и Person (Персона). Затем в настройках контрола в поле ***Связанное свойство в коллекции** укажите Employee, а в поле **Дополнительные свойства** – Contact и Person. В результате в веб-клиенте при заполнении поля **Подписали** в списке для множественного выбора доступны записи из справочников **Сотрудники**, **Контакты** и **Персоны**

Контрол состояния

Контрол состояния предназначен для отображения информации, которая может понадобиться при работе с сущностью. Например, с помощью контрола состояния в карточке поручения отображается информация об этапах работы с поручением, а в карточке задачи на согласование по регламенту – информация об этапах согласования документа.

Например, контрол состояния добавлен в карточку договора на вкладку «Задачи». В контроле отображается информация о задачах и заданиях, в которые вложен документ:

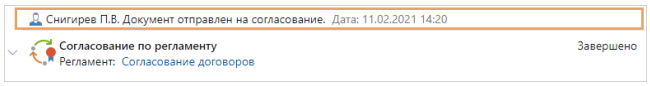


Информация в контроле состояния отображается в виде блоков. Каждый блок соответствует одному этапу работы. В содержимом блока находится текст с информацией об этапе работы с сущностью.

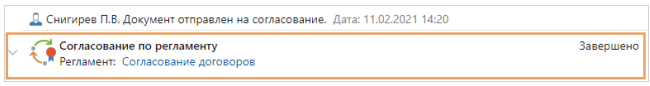
В программном коде для блоков задаются следующие настройки:

- отображение блока с рамкой или без рамки. Например, в карточке договора в виде блока без рамки отображается краткая информация о действии по отправке документа:

Блок без рамки с информацией о действии

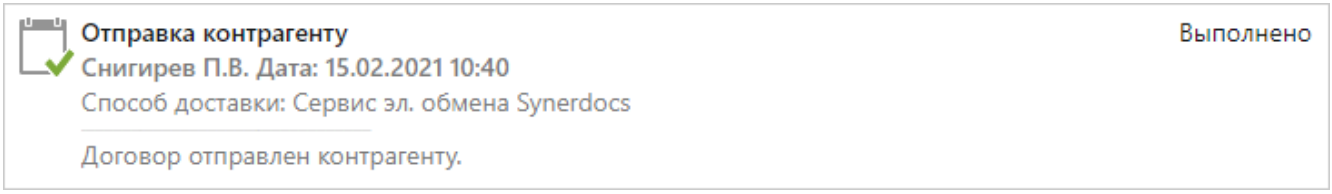


Блок с рамкой с информацией о задаче



- присоединение блока к блоку, который следует ниже. Рекомендуется задавать для блоков без рамки, это позволит отображать данные в контроле более компактно;
- отображение блоков в виде дерева. Например, в карточке договора на первом уровне находится задача на согласование документа. На втором – задания, созданные по задаче. Если у блока существуют дочерние блоки, то рядом с ним появится значок ◢. При нажатии на значок дочерние блоки сворачиваются и разворачиваются;

- количество колонок, в которых отображается текст блока. Например, для блока задания используются две колонки с данными:



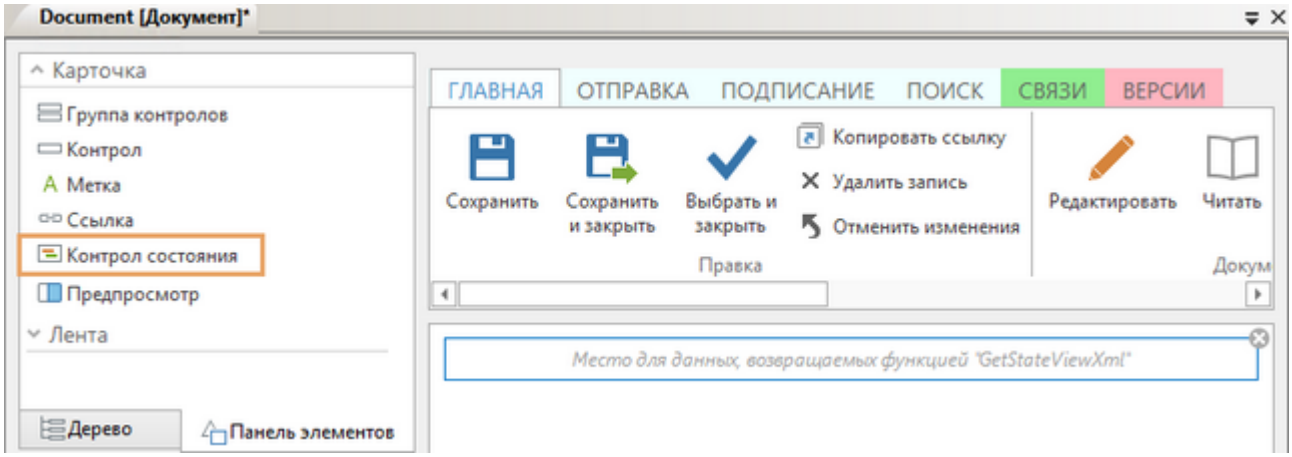
- форматирование текста блока. Например, чтобы отображать название задачи жирным шрифтом или количество дней задержки красным цветом;
- ссылка на сущность, карточка которой откроется при двойном щелчке по блоку левой кнопкой мыши;
- добавление иконки для идентификации блока:

СОВЕТ. Для блоков без рамки рекомендуется задавать иконку маленького размера.

Пример добавления контрола состояния

Предположим, необходимо, чтобы в карточке официального документа отображалась краткая информация о задачах, созданных по документу. Для этого:

1. В редакторе типа документа «Официальный документ» перейдите в узел «Форма».
2. Создайте вкладку и группу, где будет находиться контрол состояния.
3. Перейдите на вкладку «Панель элементов» и добавьте элемент **Контрол состояния** на форму.



4. Измените имя и параметры контрола состояния на панели свойств:

Именование	
***Имя**	Должно быть уникальным в рамках группы, состоять из букв латинского алфавита и цифр
Параметры	

Именование	
***Номер колонки** ***Номер строки** ***Количество колонок** ***Количество строк**	Свойства задаются и изменяются автоматически при добавлении, перемещении или удалении контролов на форме. Изменять свойства вручную не рекомендуется
Отображать в форме	Контрол состояния будет отображаться в карточке сущности. По умолчанию флажок установлен

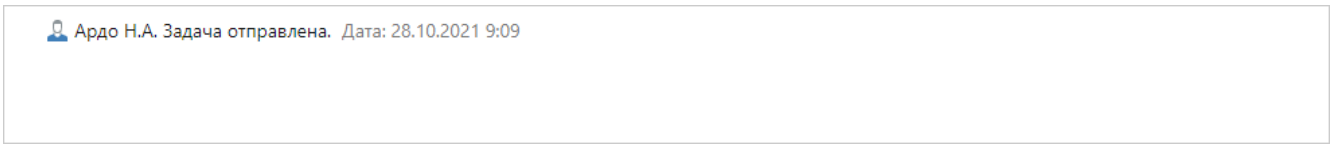
5. Добавьте серверную функцию, для этого нажмите на кнопку **Добавить функцию**. Откроется редактор программного кода.
6. Доработайте имя функции и описание, добавьте методы для создания блоков.

   ПРИМЕЧАНИЕ. Чтобы локализовать имя функции, которое отображается в редакторе форм для контрола состояния, используйте атрибут **LocalizeFunction**.

   Добавление блока без рамки с информацией о задаче:

```
/// <param name="stateView"> Схема представления.</param>
   /// <param name="user"> Пользователь, который выполнил действие.</param>
   /// <param name="text"> Текст действия.</param>
   /// <param name="substituted">Замещающий.</param>
   private static void AddUserActionBlock(StateView stateView, IUser user,
   string text, IUser substituted)
   {
     var block = stateView.AddBlock();
     // Присоединить блок без рамки к блоку, который следует ниже.
     block.DockType = DockType.Bottom;
     // Добавить в блок иконку типа сущности.
     block.AssignIcon(StateBlockIconType.User, StateBlockIconSize.Small);
     // Отключить отображение рамки блока.
     block.ShowBorder = false;
     // Добавить простой текст в блок.
     block.AddLabel(GetUserActionText(user, text, substituted) + ".");
   }
```

   Результат после публикации:

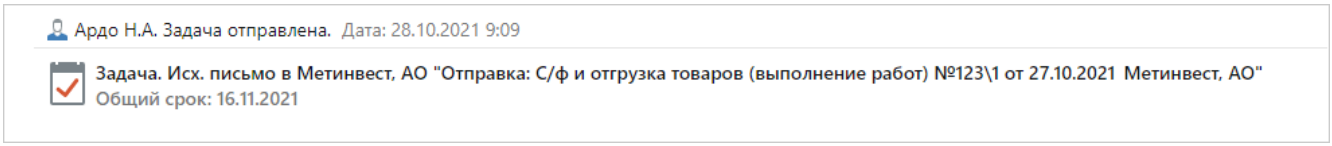


   Создание блока для отображения простой задачи:

```
/// <param name="stateView"> Схема представления.</param>
   /// <param name="task"> Задача, для которой формируется блок.
   /// При двойном нажатии левой кнопкой мыши по блоку откроется карточка
   задачи.
   private static void AddSimpleTaskView(StateView stateView, ISimpleTask task)
   {
     // Добавить блок информации о действии.
     AddUserActionBlock(stateView, task.Author,
   OfficialDocuments.Resources.StateViewTaskSent, task.StartedBy);
     var block = stateView.AddBlock();
     // Привязать задачу к блоку.
     block.Entity = task;
```

```
// Добавить в блок иконку типа сущности.
block.AssignIcon(StateBlockIconType.OfEntity, StateBlockIconSize.Large);
// Добавить название задачи жирным шрифтом.
block.AddLabel(string.Format("Задача. {0}", task.Subject),
  Functions.Module.GetHeaderStyle());
// Добавить срок задачи в новой строке.
block.AddLineBreak();
var deadline = task.MaxDeadline.HasValue ?
  task.MaxDeadline.Value.ToUserTime().ToShortDateShortTime() :
  "без срока";
block.AddLabel(string.Format("Общий срок: {0}", deadline),
  Functions.Module.GetDeadlineStyle());
```

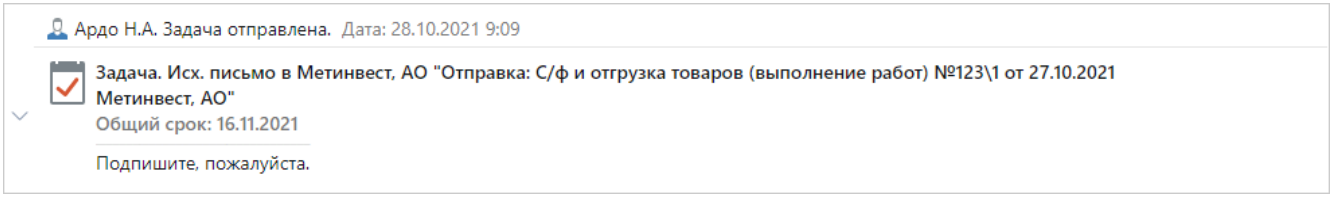
Результат после публикации:



Добавление текста задачи:

```
var taskText = Functions.Module.GetTaskText(task);
  block.AddLineBreak();
  block.AddLabel("_____",
  Functions.Module.GetSeparatorStyle());
  block.AddLineBreak();
  block.AddLabel(taskText);
```

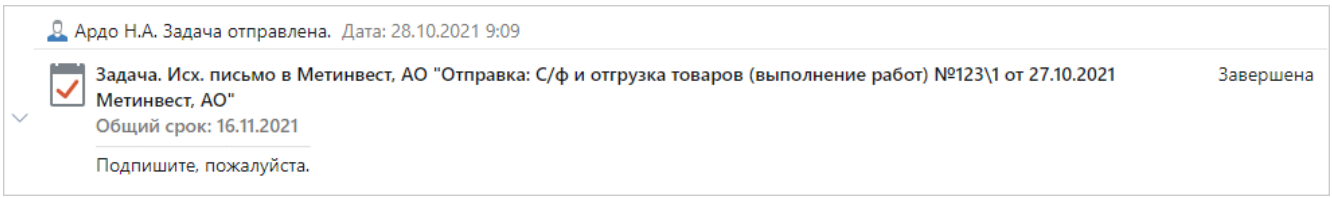
Результат после публикации:



Добавление статуса задачи в правую колонку блока:

```
var rightContent = block.AddContent();
  var status =
  Workflow.SimpleTasks.Info.Properties.Status.GetLocalizedValue(task.Status);
  rightContent.AddLabel(status);
```

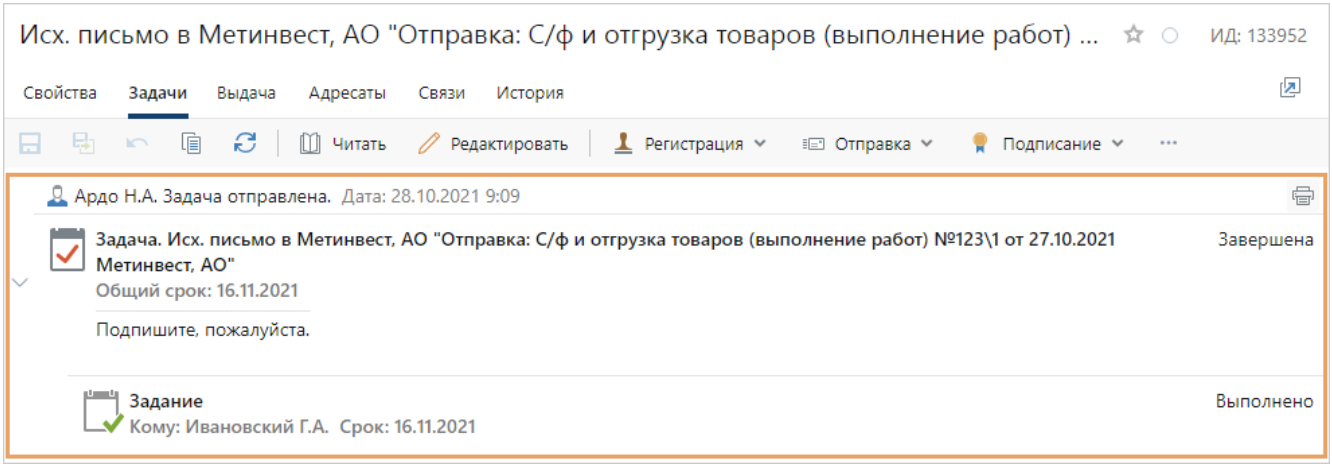
Результат после публикации:



Если по задаче сформированы задания, то добавить дочерние блоки для их отображения:

```
/// <param name="assignments"> Задания, сформированные по задаче.</param>
  var assignmentBlocks =
  GetSimpleAssignmentsView(assignments.ToList()).Blocks);
  if (assignmentBlocks.Any())
  block.AddChildBlocks(assignmentBlocks);
  }
```

В результате в карточке официального документа и его наследников на отдельной закладке будет содержаться информация о простых задачах, в которые вложен документ. Задания, сформированные по задаче, будут отображаться в виде дочерних блоков:

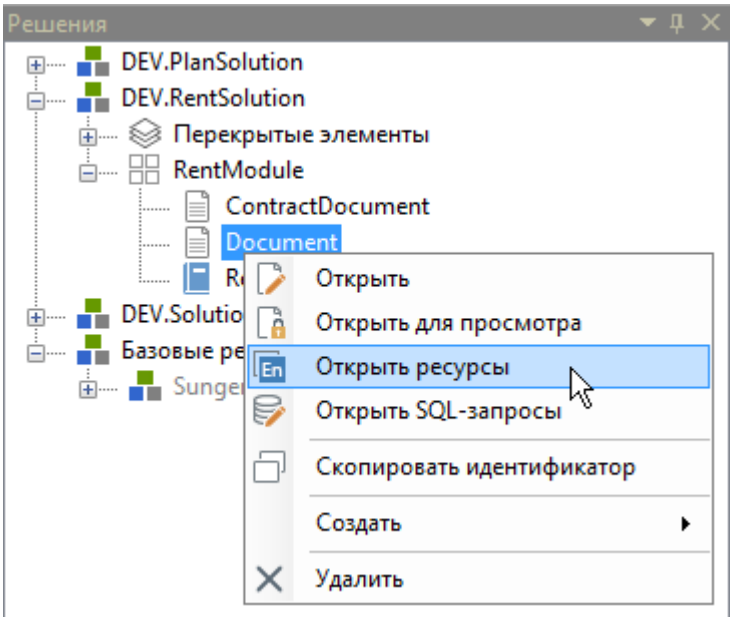


Добавление иконок

При разработке контрола состояния вы можете использовать предопределенные иконки или добавлять свои.

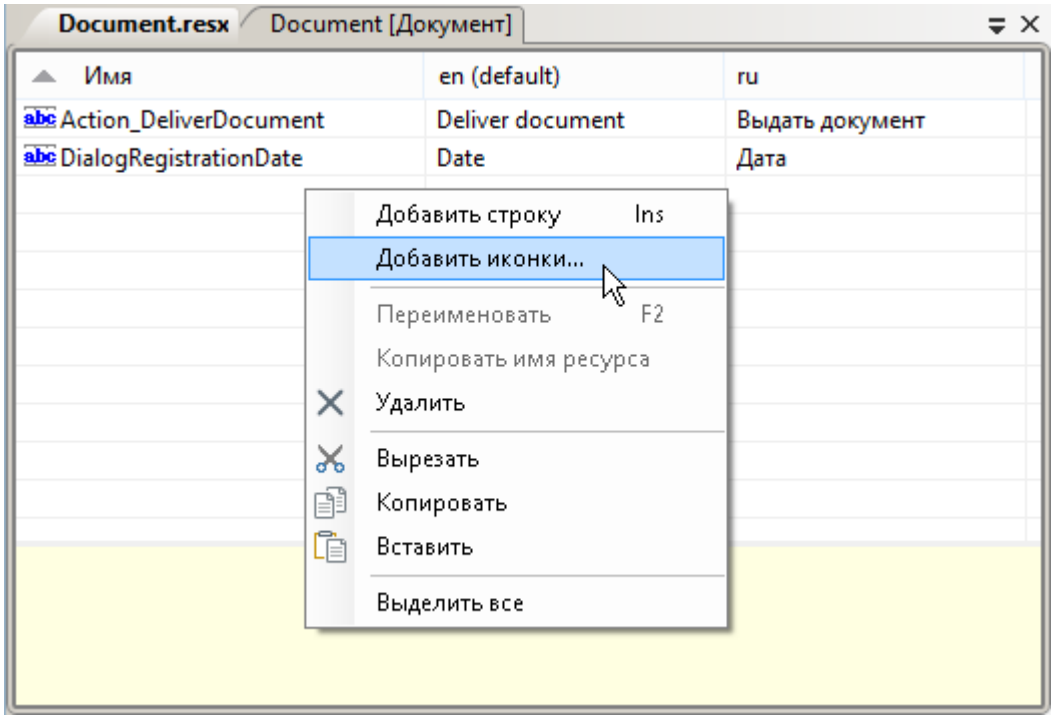
Чтобы добавить свою иконку:

1. В дереве решений в контекстном меню типа сущности, для которого разрабатывается контрол состояния, выберите пункт **Открыть ресурсы**.



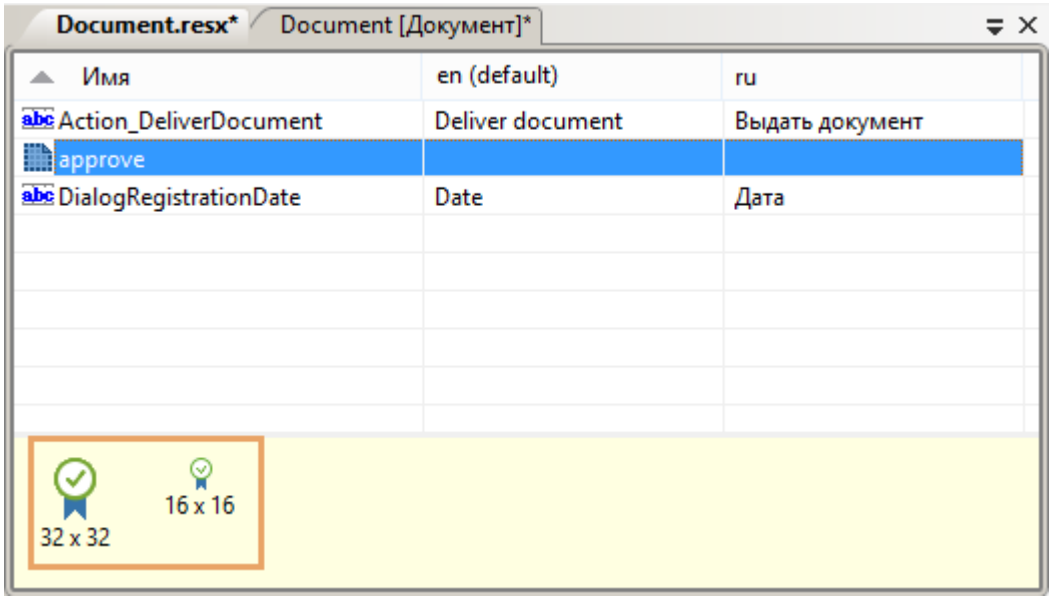
Откроется редактор ресурсов типа сущности.

2. В контекстном меню табличной части редактора ресурсов выберите пункт **Добавить иконки...**.



3. Выберите файл формата **ICO** с изображениями иконок. Рекомендуется, чтобы в файле содержались два изображения размеров 32x32 и 16x16.

   После добавления файла изображения иконок появятся в редакторе:



4. В серверную функцию контрола состояния добавьте код в формате:

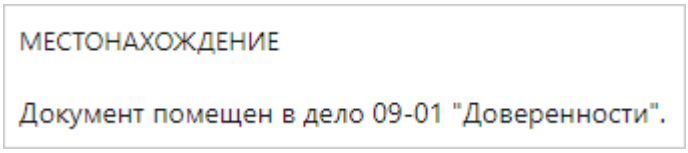
```
block.AssignIcon(<Тип сущности во мн.ч>.Resources.<Имя иконки>,
   StateBlockIconSize.<Размер иконки (Large, Small)>);
```

   Пример:

```
block.AssignIcon(ApprovalRuleBases.Resources.Assignment,
   StateBlockIconSize.Large);
```

Метка

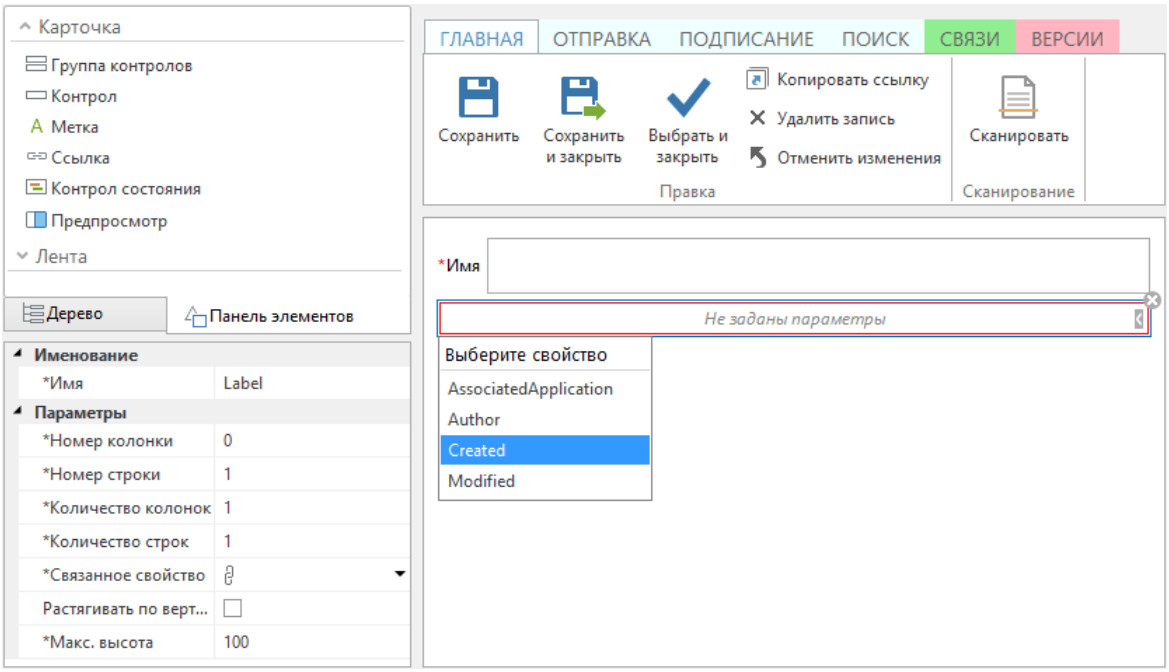
Добавляйте на форму метку, если в карточке сущности нужно показать справочную информацию, например, местонахождение документа:



Чтобы добавить метку:

1. В дереве типа сущности выделите узел «Форма» и перейдите на вкладку «Панель элементов».
2. Добавьте на форму элемент **Метка**.

   ПРИМЕЧАНИЕ. Чтобы добавить метку на другую вкладку, выберите ее в дереве типа сущности.
3. В выпадающем списке выберите свойство типа сущности, значение которого будет отображаться с помощью метки:



   ПРИМЕЧАНИЕ. В выпадающем списке отображаются свойства, которых еще нет на форме.
4. Измените имя и параметры метки на панели свойств:

Именование

***Имя**	Имя формируется по формату Label<порядковый номер>. Для удобства обращения к метке в программном коде измените имя на максимально емкое и понятное. Имя должно быть уникальным в рамках группы, состоять из букв латинского алфавита и цифр. СОВЕТ. Если с помощью мыши перетащить метку на форму и выбрать связанное свойство в выпадающем списке, то имя метки заполнится автоматически и будет совпадать с именем свойства

Именование	
Параметры	
***Номер колонки** ***Номер строки** ***Количество колонок** ***Количество строк**	Свойства задаются и изменяются автоматически при добавлении, перемещении или удалении контролов на форме. Изменять свойства вручную не рекомендуется
***Связанное свойство**	Свойство типа сущности, значение которого будет отображаться с помощью контрола. Для выбора доступны свойства типа **Дата**, **Строка**, **Текст** и **Ссылка**. Чтобы быстро перейти к параметрам связанного свойства, нажмите на кнопку или в редакторе форм дважды щелкните по метке. Откроется узел «Свойства», связанное свойство будет выделено в списке элементов
Растягивать по вертикали	Контрол будет автоматически растягиваться по вертикали при изменении размера карточки в проводнике системы
***Макс. высота**	Максимальная высота контрола для отображения значений свойства

В результате на основной вкладке карточки типа сущности появится метка.

Ссылка

Добавляйте на форму ссылку, если при работе с группой *контролов* нужно выполнить клиентскую функцию или открыть сайт. Перед добавлением проверьте, что:

- ссылку по каким-либо причинам не удобно размещать на ленте. Например, на вкладку ленты уже добавлено много действий, а создавать отдельную вкладку для одного действия не рекомендуется;
- ссылка связана с использованием определенной группы контролов на форме.

Например, перед заполнением полей **Сумма** и **Валюта** пользователю нужно посмотреть информацию о курсе валют на сайте Центрального банка России. Чтобы пользователь мог быстро перейти на сайт, разместите ссылку рядом с контролами на форме.

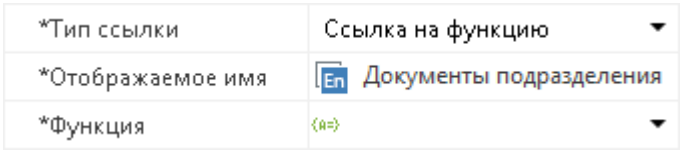
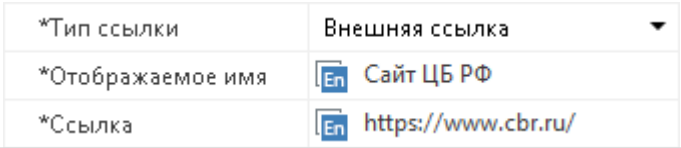
Чтобы добавить ссылку:

1. В дереве типа сущности выделите узел «Форма» и перейдите на вкладку «Панель элементов».
2. Добавьте на форму элемент **Ссылка**.

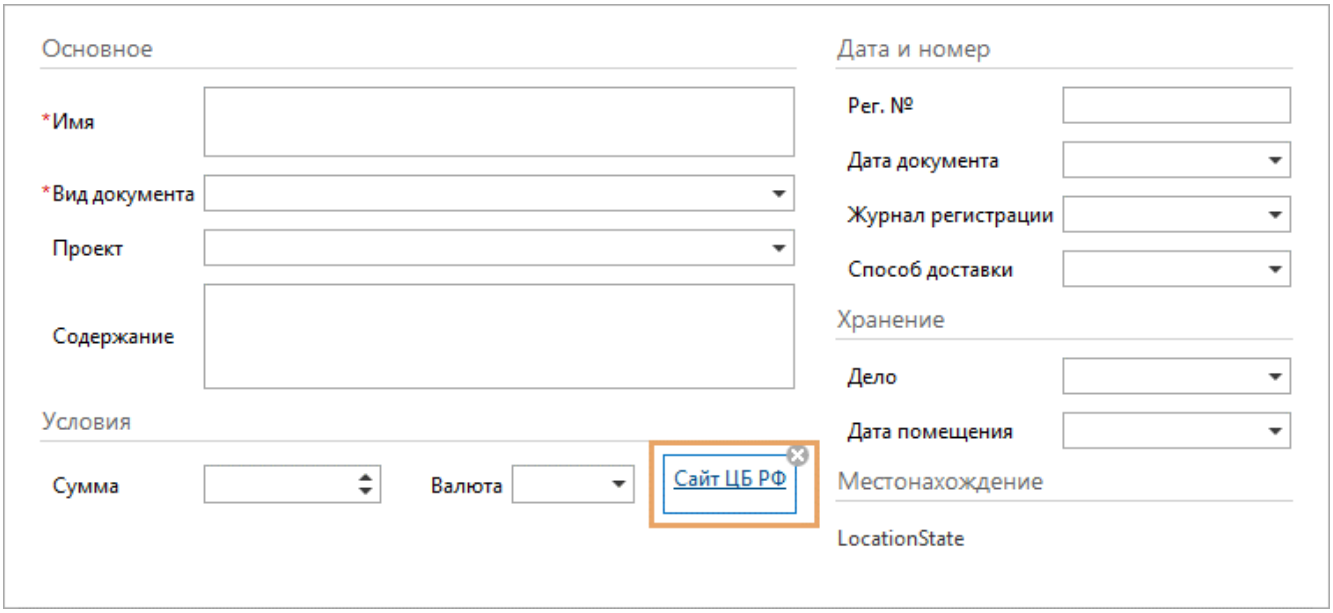
   ПРИМЕЧАНИЕ. Чтобы добавить ссылку на другую вкладку, выберите ее в дереве типа сущности.
3. Измените имя и параметры ссылки на панели свойств:

Именование	
***Имя**	Должно быть уникальным в рамках группы, состоять из букв латинского алфавита и цифр
Параметры	

Именование

***Номер колонки** ***Номер строки** ***Количество колонок** ***Количество строк**	Свойства задаются и изменяются автоматически при добавлении, перемещении или удалении контролов на форме. Изменять свойства вручную не рекомендуется
***Тип ссылки**	Выберите одно из значений: • «Ссылка на функцию». При нажатии на ссылку выполнится клиентская функция. ВАЖНО. Функция не должна возвращать значений. Чтобы создать функцию, нажмите на кнопку Добавить функцию:  Кроме этого в поле ***Функция*** можно выбрать существующую клиентскую функцию данного типа сущности или его предка (базового типа сущности). Открыть код функции можно по той же кнопке . • «Внешняя ссылка». В поле ***Ссылка*** укажите адрес внешней ссылки: 
***Отображаемое имя**	Название ссылки, которое отображается на форме. Для *локализации* нажмите на кнопку **Локализовать**

В результате на основной вкладке карточки типа сущности появится ссылка, например:

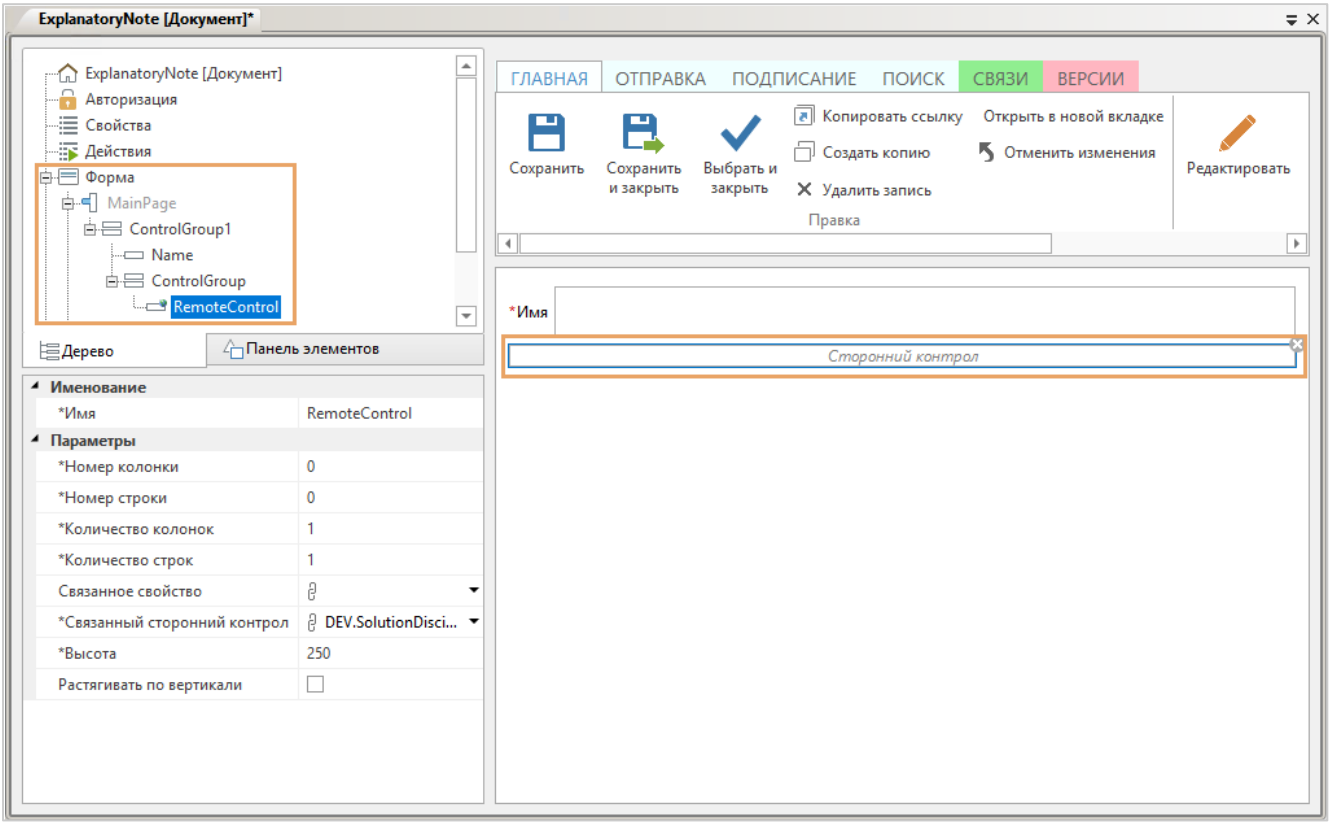


Сторонний контрол

Чтобы доработать карточку объекта под потребности компании, можно использовать самостоятельно разработанные контролы. Например, в карточку документа добавить контрол, который отображает состояние жизненного цикла.

Чтобы использовать сторонний контрол на форме:

1. Добавьте сторонние компоненты с одним или несколькими сторонними контролами.
2. В дереве типа сущности выделите узел «Форма» и перейдите на вкладку «Панель элементов».
3. Добавьте в группу на форме элемент **Сторонний контрол**:



ПРИМЕЧАНИЕ. Чтобы добавить сторонний контрол на другую вкладку, выберите ее в дереве типа сущности.

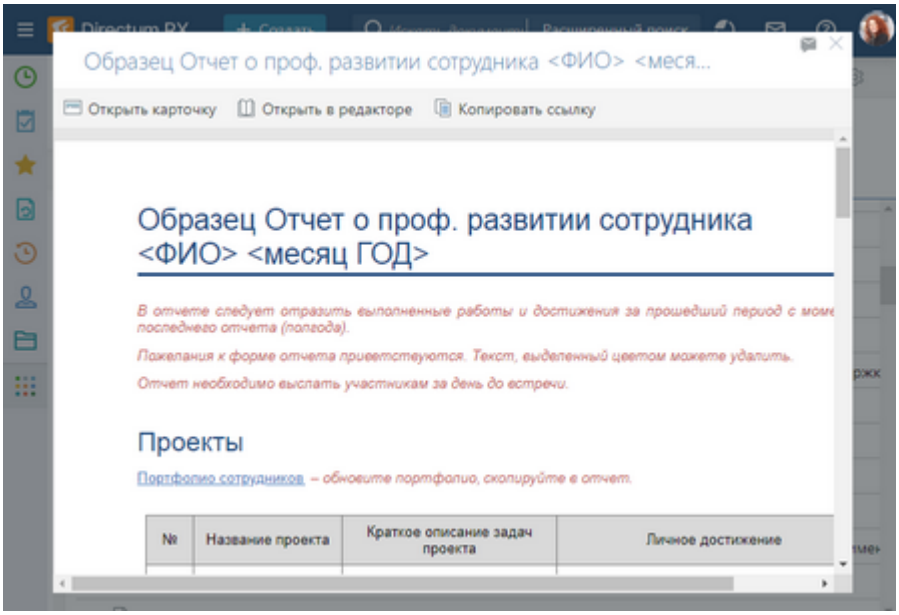
4. В выпадающем списке ***Связанный сторонний контрол** выберите, какой сторонний контрол нужно отображать в карточке.
5. Измените имя и параметры стороннего контрола на панели свойств:

Именование	
***Имя**	Имя формируется по формату RemoteControl<порядковый номер>
Параметры	
***Номер колонки** ***Номер строки** ***Количество колонок** ***Количество строк**	Свойства задаются и изменяются автоматически при добавлении, перемещении или удалении сторонних контролов на форме. Изменять свойства вручную не рекомендуется

Именование	
Связанное свойство	Свойство типа сущности, значение которого отображается с помощью контроля. Если сторонний контрол не предназначен для отображения или редактирования связанного свойства, то указывать его не нужно. Чтобы быстро перейти к параметрам связанного свойства, нажмите на кнопку или в редакторе форм дважды щелкните по стороннему контролу. Откроется узел «Свойства», связанное свойство будет выделено в списке элементов
***Связанный сторонний контрол**	Сторонний контрол, который будет отображаться в карточке
***Высота**	Высота стороннего контрола в карточке
Растягивать по вертикали	Сторонний контрол будет автоматически растягиваться по вертикали при изменении размера карточки в проводнике системы

Предпросмотр

Предпросмотр можно добавить только для типов документов. Контрол предназначен для предварительного просмотра содержимого документа. Он позволяет отобразить текст документа, не загружая его в приложении-редакторе. Например, в стандартном решении Directum RX предпросмотр используется для отображения модального окна. Такое окно появляется при открытии пользователем документа на чтение в веб-клиенте, если включена настройка **Использовать предпросмотр документов**:



Размещение контрола на форме настраивается для разных ситуаций использования. Например, можно добавить предпросмотр на форму карточки документа, чтобы заполнять карточку копированием текста из области предпросмотра, не открывая документ.
ВАЖНО. Контрол отображается только в веб-клиенте.

Чтобы добавить предпросмотр:

1. В дереве типа документа выделите узел «Форма», выберите необходимую вкладку или создайте новую. Перейдите на «Панель элементов» и разместите на вкладке группу контролов.
2. Добавьте в группу элемент **Предпросмотр**.

3. Измените имя и параметры контрола на панели свойств:

Именование	
***Имя**	Имя формируется по формату Preview<порядковый номер>. Для удобства обращения к контролу в программном коде измените имя на максимально емкое и понятное. Оно может состоять из букв латинского алфавита и цифр. Контрол можно разместить на вкладке только один раз
Параметры	
***Номер колонки** ***Номер строки** ***Количество колонок** ***Количество строк**	Свойства задаются и изменяются автоматически при добавлении, перемещении или удалении групп контролов на форме. Изменять свойства вручную не рекомендуется

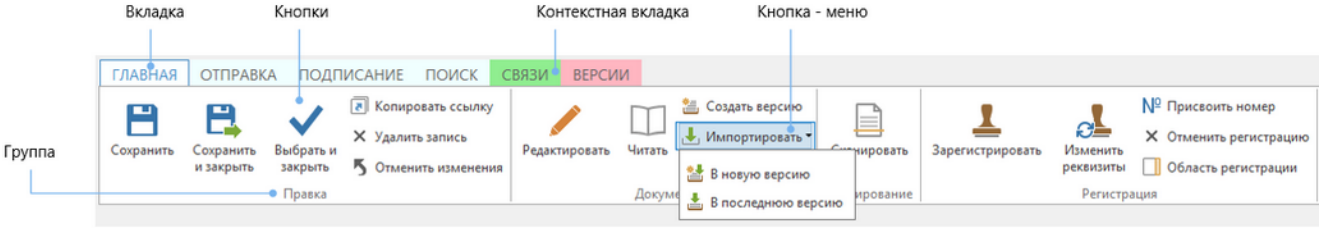
Лента

Для настройки ленты карточки используйте узел «Лента» в редакторе форм.

Лента представляет собой набор действий для работы с сущностью. На ленту можно добавить:

- вкладки для объединения групп действий, которые используются для выполнения схожих сценариев работы. Например, в карточке документа на вкладке **Отправка** находятся группы с действиями для отправки документа другим сотрудникам и для добавления ссылки на документ в папку «Избранное»;
- контекстные вкладки для отображения действий, которые нужны пользователю только при заполнении свойства-коллекции или при работе с определенной вкладкой в карточке типа сущности. Например, если в карточке документа перейти на вкладку «Версии», то на ленте появится контекстная вкладка с набором действий для работы с версиями документа;
- группы для объединения схожих действий и настройки их расположения на ленте. Например, в карточке документа все действия для регистрации объединены на ленте в группу **Регистрация**;
- кнопки для выполнения действий в проводнике системы;
- переключатели для скрытия части контролов в карточке сущности в зависимости от того, нажата кнопка или нет. Например, при заполнении карточки документа с помощью кнопки-переключателя пользователь может скрыть поля, которые в данный момент заполнять не нужно. При следующем открытии карточки скрытые контролы снова отобразятся. Кнопку можно связать с действиями, у которых в параметрах установлен флажок **Действие-переключатель**;
- переключатели состояния для изменения значения свойства сущности с типом **Логическое**. Например, если кнопка-переключатель состояния нажата, то значение свойства **True** и в карточке отображается один набор полей. Если кнопка не нажата, то значение свойства **False** и в карточке отображается другой набор полей. Значение свойства сохраняется, при следующем открытии карточки отображается последнее положение кнопки-переключателя состояния (нажата или не нажата) и соответствующий набор полей в карточке;

- кнопки-меню для отображения однотипных действий в виде выпадающего списка. Например, при нажатии на кнопку **Импортировать** в выпадающем списке доступны действия для импорта файла в новую и в последнюю версию документа;
- разделители, если в выпадающем списке кнопки-меню необходимо отделить одни действия от других.



ПРИМЕЧАНИЕ. Для настройки ленты списка используйте узел «Лента» в редакторе списка.

При создании типа сущности от базового типа автоматически наследуется лента. В дереве типа сущности наследуемые элементы ленты выделены серым цветом. Их нельзя удалить, но можно скрыть с помощью свойства **Отображать на ленте**. В наследуемых элементах можно изменить все свойства, кроме свойств ***Имя** и ***Действие**.

ПРИМЕЧАНИЕ. Если в среде разработки элемент скрыт программно, то в веб-клиенте он не отобразится, даже если добавить его с помощью редактора форм в проводнике.

Чтобы изменить расположение элемента, перетащите его в нужное место. Места для вставки подсвечиваются желтым цветом.

Перемещать элементы можно только в пределах той вкладки, где они созданы.

ПРИМЕЧАНИЕ. Перед настройкой ленты ознакомьтесь с рекомендациями.

Вкладка

Добавляйте вкладки, чтобы объединить группы действий, которые используются для выполнения схожих сценариев работы. Например, в карточке документа на вкладке **Отправка** находятся группы с действиями для отправки документа другим сотрудникам и для добавления ссылки на документ в папку «Избранное».

ВАЖНО. Рекомендуется добавлять вкладки, только если на ленте нет подходящей вкладки для нового действия.

Предположим, для документа созданы действия, с помощью которых можно открыть карточку документа в другой системе и найти в ней связанные документы. При этом на ленте нет подходящих вкладок или групп для новых действий. В этом случае создайте вкладку:

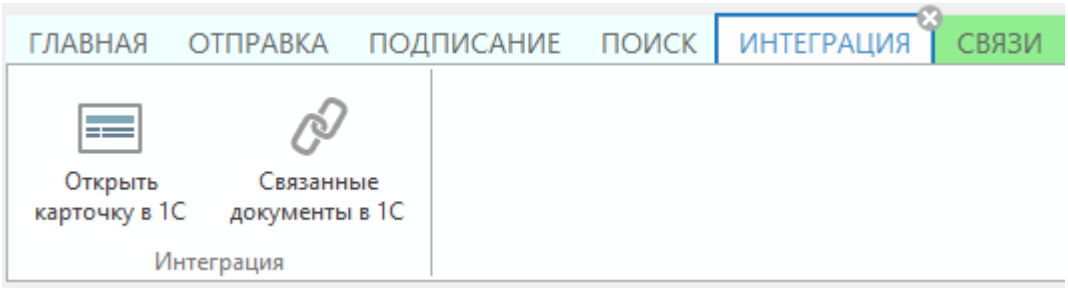
1. В редакторе форм выделите узел «Лента» и перейдите на вкладку «Панель элементов».
2. Добавьте на ленту элемент **Вкладка**. При этом на вкладку автоматически добавится группа (Group).
3. Измените имя и параметры вкладки на панели свойств:

Именование	
***Имя**	Должно быть уникальным в рамках ленты, состоять из букв латинского алфавита и цифр

Именование	
Параметры	
***Отображаемое имя**	Название вкладки, которое будет отображаться на ленте. Для *локализации* отображаемого имени нажмите на кнопку **Локализовать**
Отображать на ленте	Вкладка будет отображаться на ленте

4. Задайте свойства группы и добавьте кнопки.

В результате на ленту карточки документа будет добавлена вкладка с двумя действиями. При необходимости на нее можно добавить другие группы и кнопки.



Контекстная вкладка

Добавляйте контекстные вкладки, чтобы отображать определенные действия на ленте только когда в них есть необходимость. Например, если в карточке документа перейти на вкладку «Версии», то на ленте появляется контекстная вкладка с набором действий для работы с версиями документа. При других условиях контекстная вкладка будет скрыта.

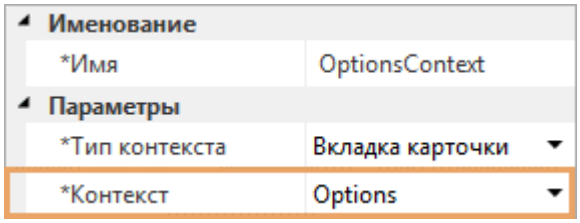
Появление контекстных вкладок можно связать:

- с заполнением значений свойства-коллекции;
- с работой на определенной вкладке карточки.

Предположим, в карточку документа добавлена вкладка «Параметры». Для работы с параметрами разработаны действия, которые нужно добавить на ленту. Действия понадобятся пользователю только при переходе на вкладку «Параметры», поэтому создайте для этих действий контекстную вкладку:

1. В редакторе форм выделите узел «Лента» и перейдите на вкладку «Панель элементов».
2. Добавьте на ленту элемент **Контекстная вкладка**. На контекстную вкладку автоматически добавятся вкладка (Page) с группой (Group).
3. Измените имя и параметры контекстной вкладки на панели свойств:

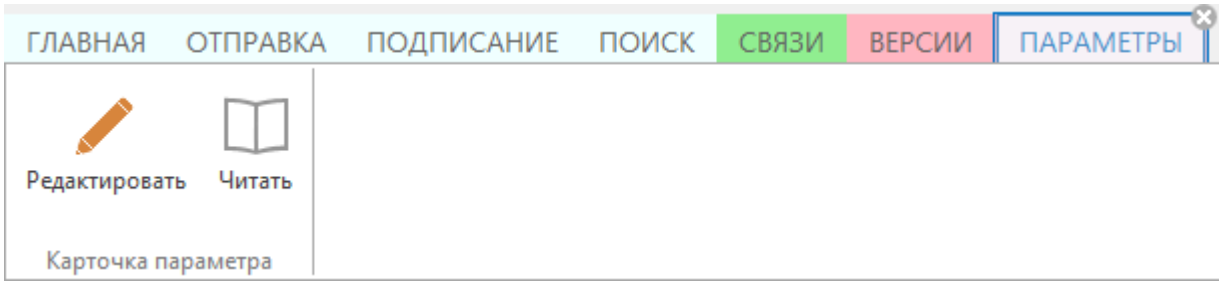
Именование	
***Имя**	Должно быть уникальным в рамках ленты, состоять из букв латинского алфавита и цифр
Параметры	

Именование	
***Тип контекста**	От типа контекста зависят значения в поле ***Контекст**.  Возможные значения: • **Вкладка карточки**. В поле ***Контекст** выберите вкладку карточки типа сущности. Контекстная вкладка появится после перехода на указанную вкладку в карточке. Появление контекстной вкладки можно связать с переходом на любую вкладку карточки, кроме основной вкладки (MainPage). • **Контрол**. В поле ***Контекст** выберите *контрол* типа «Редактор свойства-коллекции». Контекстная вкладка появится при заполнении значений свойства-коллекции. ВАЖНО. Для разных контекстных вкладок в поле ***Контекст** должны быть указаны разные значения

4. Задайте свойства вкладки (Page).
5. Задайте свойства группы (Group) и добавьте на вкладку необходимые кнопки.

В результате при переходе на вкладку «Параметры» в карточке документа на ленте появится контекстная вкладка «Параметры». При необходимости на контекстную вкладку можно добавить другие вкладки, группы и кнопки.

ВАЖНО. Не добавляйте действия сущностей на вкладку «Вложения» или контекстные вкладки, иначе действия не будут отображаться.



Группа

Добавляйте группы, чтобы объединить схожие действия и настроить их расположение на ленте. Например, в карточке документа все действия для регистрации объединены на ленте в группу **Регистрация**.

Предположим, что для справочника **Контрагенты** созданы действия **Написать письмо** и **Перейти на сайт**. Действия будут использоваться часто, поэтому их нужно добавить на вкладку **Главная**. На вкладке нет группы с похожими действиями, поэтому добавьте новую группу:

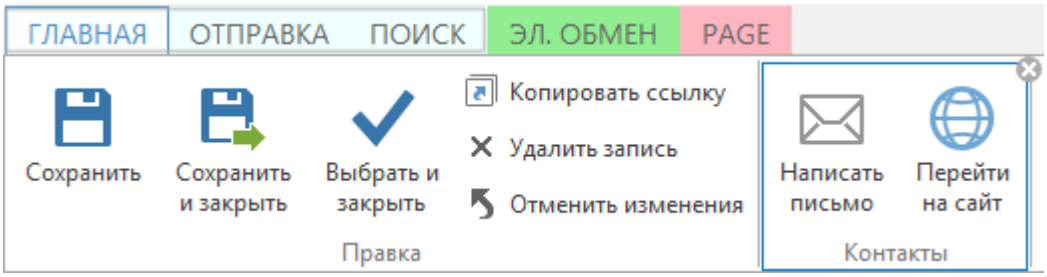
1. В редакторе форм выделите узел «Лента» и перейдите на вкладку «Панель элементов».
2. Добавьте на ленту элемент **Группа**.

3. Измените имя и параметры группы на панели свойств:

Именование	
***Имя**	Должно быть уникальным в рамках одной вкладки, состоять из букв латинского алфавита и цифр
Параметры	
***Отображаемое имя**	Название группы, которое будет отображаться на ленте. Для *локализации* отображаемого имени нажмите на кнопку **Локализовать**
Отображать на ленте	Группа будет отображаться на ленте
Иконка (32x32)	Отображается, если лента свернута до размеров, не позволяющих отобразить все действия группы. Если иконка не задана, то отображается иконка первого из действий в группе

4. Добавьте в группу необходимые кнопки.

В результате на ленту карточки контрагента на вкладку **Главная** будет добавлена группа **Контакты** с кнопками **Написать письмо** и **Перейти на сайт**. При необходимости в группу можно добавить другие кнопки.



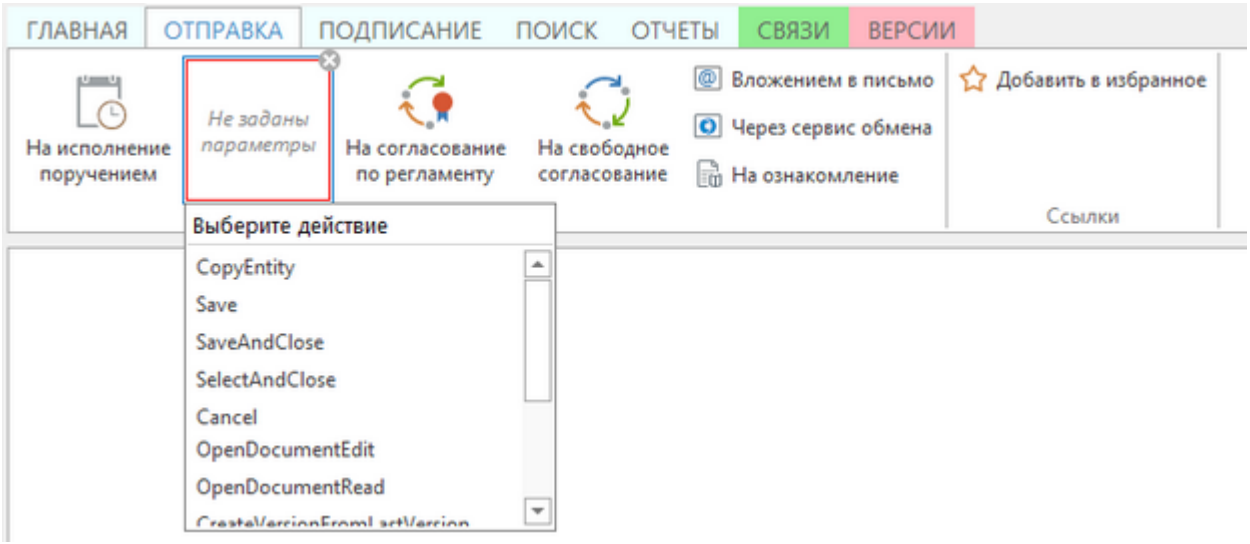
Кнопка

Добавляйте на ленту кнопки для выполнения действий в проводнике системы.

Предположим, что для входящего документа создано действие **Отправка на рассмотрение руководителю**. Действие нужно вынести на ленту карточки документа. На ленте уже есть вкладка **Отправка** и группа **Отправить** с однотипными действиями. Добавьте новое действие в данную группу:

1. В редакторе форм выделите узел «Лента» и перейдите на вкладку «Панель элементов».
2. На ленте перейдите на вкладку **Отправка** и добавьте в группу **Отправить** элемент **Кнопка**.

3. В выпадающем списке выберите действие, которое будет вызываться при нажатии на кнопку:



ПРИМЕЧАНИЕ. В выпадающем списке отображаются действия, которых еще нет на ленте.

4. Измените имя и параметры кнопки на панели свойств:

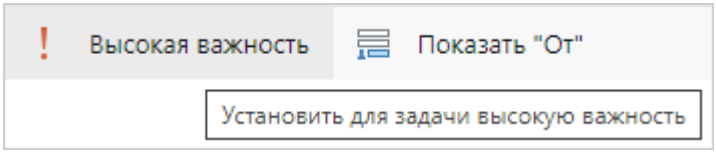
Именование

***Имя**	Имя формируется по формату ActionButton<порядковый номер>. Для удобства обращения к кнопке в программном коде измените имя на максимальное емкое и понятное. Имя должно быть уникальным в рамках группы, состоять из букв латинского алфавита и цифр. СОВЕТ. Если с помощью мыши перетащить кнопку на ленту и выбрать связанное действие в выпадающем списке, то имя кнопки заполнится автоматически и будет совпадать с именем действия

Параметры

***Действие**	Действие, которое будет вызываться при нажатии на кнопку. При настройке ленты формы для выбора доступны действия, в параметрах которых в поле ***Доступно** в указано значение **Карточке**, **Карточке и списке** или **Дочерней коллекции**. При настройке ленты списка для выбора доступны действия, в параметрах которых в поле ***Доступно** в указано значение **Списке** или **Карточке и списке**. Чтобы быстро перейти к параметрам связанного действия, нажмите на кнопку или в редакторе форм дважды щелкните по кнопке. Откроется узел «Действие», связанное действие будет выделено в списке элементов
***Размер кнопки**	Размер кнопки для действия на ленте. Возможные значения: **Большая**, **Маленькая**. При необходимости измените размер кнопки
Отображать на ленте	Кнопка будет отображаться на ленте

Для добавленной кнопки будет использоваться описание действия, которое она вызывает:



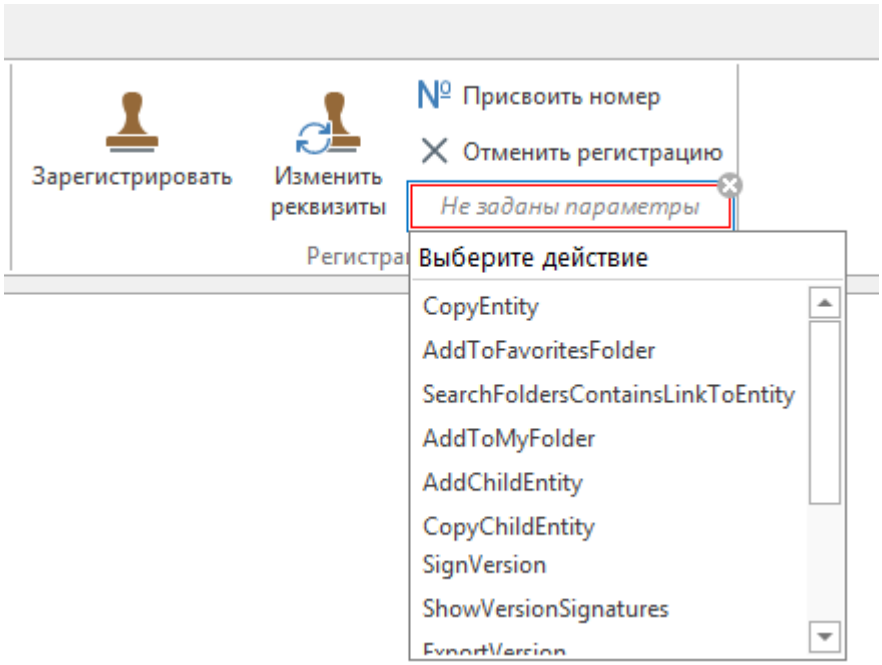
Переключатель

Переключатель можно связать с действиями сущности, у которых в параметрах установлен флажок **Действие-переключатель**.

Добавляйте переключатель, если нужно скрыть часть контролов в карточке типа сущности в зависимости от того, нажата кнопка или нет. Например, по умолчанию в карточке отображается группа контролов. При первом нажатии на кнопку группа контролов скрывается. При втором нажатии восстанавливается вид карточки по умолчанию. После закрытия карточка также принимает вид по умолчанию.

Предположим, для документа создано действие **Область регистрации**. При нажатии на кнопку в карточке должна отображаться или скрываться группа контролов (область регистрации). Для этого добавьте на ленту кнопку-переключатель:

1. В редакторе форм выделите узел «Лента» и перейдите на вкладку «Панель элементов».
2. Добавьте на ленту элемент **Переключатель**.
3. В выпадающем списке выберите действие, которое будет вызываться при нажатии на кнопку:

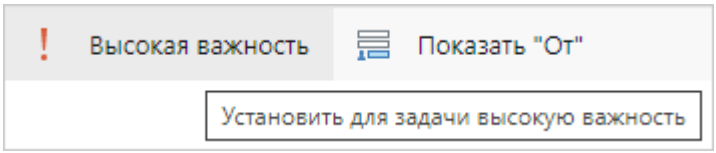


ПРИМЕЧАНИЕ. В выпадающем списке отображаются действия, которых еще нет на ленте.

4. Измените имя и параметры переключателя на панели свойств:

Именование	
***Имя**	Имя формируется по формату ToggleButton<порядковый номер>. Для удобства обращения к переключателю в программном коде измените имя на максимально емкое и понятное. Имя должно быть уникальным в рамках группы, состоять из букв латинского алфавита и цифр. СОВЕТ. Если с помощью мыши перетащить переключатель на ленту и выбрать связанное действие в выпадающем списке, то имя переключателя заполнится автоматически и будет совпадать с именем действия
Параметры	
***Действие**	Действие, которое будет вызываться при нажатии на кнопку. Для выбора доступны только действия, в параметрах которых установлен флажок **Действие-переключатель**. Чтобы быстро перейти к параметрам связанного действия, нажмите на кнопку или в редакторе форм дважды щелкните по переключателю. Откроется узел «Действие», связанное действие будет выделено в списке элементов
***Размер кнопки**	Размер кнопки для действия на ленте. Возможные значения: **Большая**, **Маленькая**. При необходимости измените размер кнопки
Отображать на ленте	Кнопка будет отображаться на ленте

Для добавленной кнопки будет использоваться описание действия, которое она вызывает:



В результате на ленту карточки документа будет добавлена кнопка-переключатель.

Переключатель состояния

Добавляйте переключатель состояния, чтобы изменять значение свойства сущности с типом **Логическое**. Например, если кнопка-переключатель состояния («залипающая кнопка») нажата, то значение свойства **True** и в карточке отображается один набор полей. Если кнопка не нажата, то значение свойства **False** и в карточке отображается другой набор полей. Значение свойства сохраняется. Это значит, что при следующем открытии карточки отображается последнее состояние кнопки (нажата или не нажата) и соответствующий набор полей в карточке.

Предположим, у поручения есть логическое свойство **Составное поручение**. При этом:

- если значение свойства **False**, то в карточке должен отображаться набор полей для создания простого поручения, у которого срок и состав работ для исполнителя и соисполнителей одинаков;

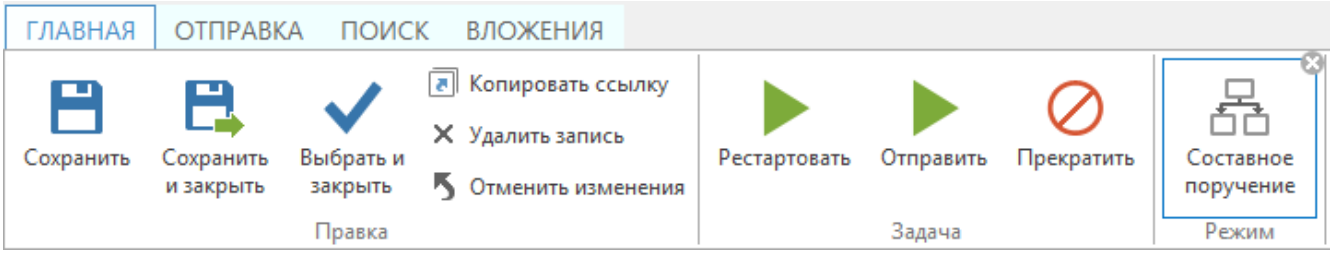
- если значение свойства **True**, то в карточке должен отображаться набор полей для создания составного поручения, у которого несколько равноправных исполнителей. При этом у каждого исполнителя может быть свой срок поручения и индивидуальное описание работ.

Чтобы настроить отображение полей, добавьте на ленту переключатель состояния:

1. В редакторе форм выделите узел «Лента» и перейдите на вкладку «Панель элементов».
2. Добавьте на ленту элемент **Переключатель состояния**.
3. Измените имя и параметры переключателя состояния на панели свойств:

Именование	
***Имя**	Должно быть уникальным в рамках группы, состоять из букв латинского алфавита и цифр
***Отображаемое имя**	Название кнопки, которое будет отображаться на ленте
Подсказка	Краткое описание кнопки. Отображается в заголовке всплывающей подсказки
Описание	Подробное описание кнопки. Отображается во всплывающей подсказке. ПРИМЕЧАНИЕ. Для *локализации* отображаемого имени, описания и подсказки нажмите на кнопку **Локализовать**
Параметры	
Действие	Действие, которое будет вызываться при нажатии на кнопку. При настройке ленты для выбора доступны действия, в параметрах которых в поле ***Доступно в** указано значение **Карточке**, **Карточке и списке** или **Дочерней коллекции**. Чтобы быстро перейти к параметрам связанного действия, нажмите на кнопку или в редакторе форм дважды щелкните по переключателю состояния. Откроется узел «Действие», связанное действие будет выделено в списке элементов
***Размер кнопки**	Размер кнопки для действия на ленте. Возможные значения: **Большая**, **Маленькая**. При необходимости измените размер кнопки
***Свойство состояния**	Свойство состояния, которое будет изменяться при нажатии на кнопку. Для выбора доступны только свойства с типом **Логическое**. ВАЖНО. Если поле **Действие** заполнено, то значение свойства не будет изменяться при нажатии на кнопку. В этом случае изменять значение свойства нужно программно в коде действия
Маленькая иконка **Большая иконка**	Иконки для маленькой и большой кнопок на ленте. Иконки должны быть в формате SVG и в размерах 16x16 и 32x32 соответственно
Отображать на ленте	Кнопка будет отображаться на ленте

В результате на ленту карточки поручения будет добавлена кнопка-переключатель состояния.



Кнопка-меню

Добавляйте кнопку-меню, чтобы отображать однотипные действия в виде выпадающего списка.

Предположим, что у документа есть однотипные действия **Импортировать в новую версию** и **Импортировать в последнюю версию**. В этом случае можно использовать кнопку-меню, чтобы при нажатии на кнопку в выпадающем списке были доступны оба действия. Действие будет использоваться часто, поэтому добавьте кнопку-меню на вкладку **Главная**:

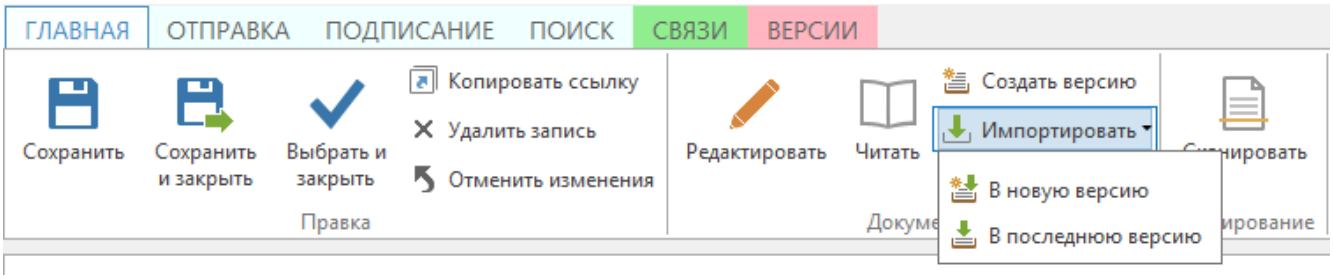
1. В редакторе форм выделите узел «Лента» и перейдите на вкладку «Панель элементов».
2. Добавьте на ленту элемент **Кнопка-меню**.
3. Измените имя и параметры кнопки-меню на панели свойств:

Именование	
***Имя**	Должно быть уникальным в рамках группы, состоять из букв латинского алфавита и цифр
Параметры	
***Отображаемое имя**	Название кнопки, которое будет отображаться на ленте
Всплывающая подсказка	Краткое описание кнопки. Отображается в заголовке всплывающей подсказки
Описание	Подробное описание кнопки. Отображается во всплывающей подсказке. ПРИМЕЧАНИЕ. Для *локализации* отображаемого имени, описания и подсказки нажмите на кнопку **Локализовать**
***Размер кнопки**	Размер кнопки для действия на ленте. Возможные значения: **Большая**, **Маленькая**. При необходимости измените размер кнопки
Маленькая иконка **Большая иконка**	Иконки для маленькой и большой кнопок на ленте. Иконки должны быть в формате SVG и в размерах 16x16 и 32x32 соответственно
Отображать на ленте	Кнопка будет отображаться на ленте

3. Настройте пункты меню. Для этого добавьте кнопки для импорта в новую и последнюю версию документа.
4. Если необходимо отделить несколько пунктов меню, добавьте между кнопками элемент **Разделитель**. На панели свойств задайте имя и параметры для разделителя:

Именование	
***Имя**	Должно быть уникальным в рамках кнопки-меню, состоять букв латинского алфавита и цифр
Параметры	
Отображать на ленте	Разделитель будет отображаться в выпадающем списке кнопки-меню

В результате на ленту карточки документа будет добавлена кнопка-меню.



Рекомендации по настройке ленты

Для удобного использования и поддержания единого стиля системы при настройке ленты соблюдайте рекомендации.

Общие рекомендации

- однотипные элементы ленты карточек разных типов сущностей располагайте одинаково. Например, группы **Регистрация** в карточках входящего и исходящего писем выглядят однотипно и располагаются в одном и том же месте на лентах;
- однотипные элементы карточки и списка сущностей располагайте одинаково. Например, вкладки **Отправка** в карточке и в списке исходящих писем выглядят одинаково и располагаются в одном и том же месте на лентах;
- не перемещайте вкладки, группы и кнопки, которые были получены при наследовании;
- в заголовках не используйте сокращения и знаки препинания;
- используйте flat-стиль иконок;
- для иконок используйте цветовую гамму, которая сочетается с имеющимися в системе иконками.

Настройка кнопок

- если на ленте уже есть группа с однотипными действиями, то новое действие добавляйте в эту группу;
- кнопки на ленте располагайте по приоритетам. Наиболее часто используемые действия должны находиться в центре ленты, так как это наиболее заметная область. Редко используемые действия должны быть в правой части;

- не изменяйте левую часть ленты, чтобы при открытии разных карточек или списков сохранялась статичность ленты;
- заголовки кнопок делайте примерно одинаковой длины и максимально короткими;
- в качестве заголовка кнопки используйте глаголы и/или существительные, например **Создать**, **Выбрать запись**, **Список версий**.

Настройка групп

- не изменяйте системные группы действий на ленте;
- не добавляйте большое количество групп и кнопок, чтобы на ленте группы не отображались в свернутом виде;
- если на ленте уже есть вкладка с однотипными группами действий, то новую группу добавляйте на эту вкладку. Например, если на ленте уже есть вкладка **Поиск**, то группу с действиями для выполнения поиска стоит добавить на эту вкладку;
- в качестве заголовка группы используйте существительные или отглагольные существительные. Например, **Создание**, **Документ**, **Действия**.

Настройка вкладок

- создавайте новые вкладки для групп действий, предназначенных для выполнения отдельных сценариев работы. Например, для групп с действиями для подписания документов выделяется вкладка **Подписание**;
- в качестве заголовка вкладки используйте существительные в единственном числе или отглагольные существительные. Например, **Отправка**, **Поиск**.

Настройка контекстного меню

- добавляйте в контекстное меню только часто используемые действия;
- не добавляйте в контекстное меню больше 10 действий, чтобы оно не было очень большим;
- в контекстном меню не должно быть больше одного уровня вложенности действий.

Автотексты для заполнения контролов

В Directum RX сотрудники могут использовать автотексты, чтобы в веб-клиенте быстро написать текст задачи, отчитаться о выполнении задания, заполнить поле ***Содержание** в приказе и т.д.

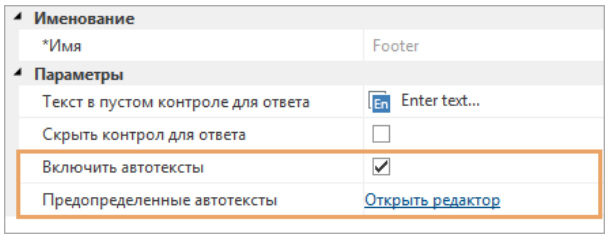
В своей разработке вы также можете использовать автотексты для заполнения контролов на формах типов сущностей. Для этого:

1. Установите флажок рядом с параметром **Включить автотексты**, он отображается в свойствах:
   - группы «Footer» в редакторе форм задач, заданий и заданий на приемку;
   - контролов, которые связаны со свойствами типа Текст или Строка. Для свойства типа **Строка** параметр появляется, если в поле ***Тип контрола** выбрано значение «Многострочный текстовый редактор».

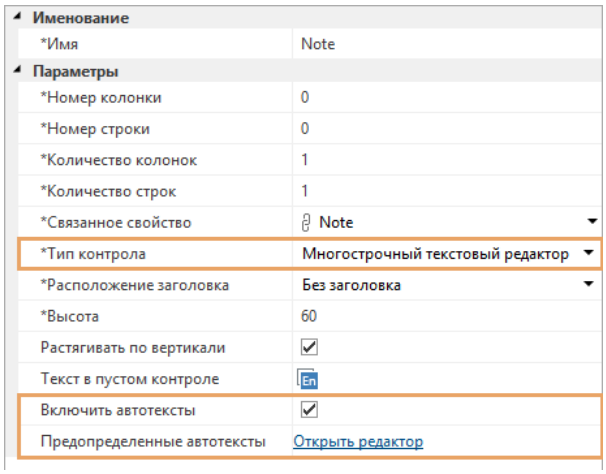
В задания и заданиях на приемку для групп «Footer» флажок **Включить автотекст** по умолчанию установлен. Для контролов и задач – снят.

2. В параметре **Предопределенные автотексты** перейдите по ссылке **Открыть редактор**. Параметр отображается, если установлен флажок **Включить автотексты**. Не обязателен для заполнения.

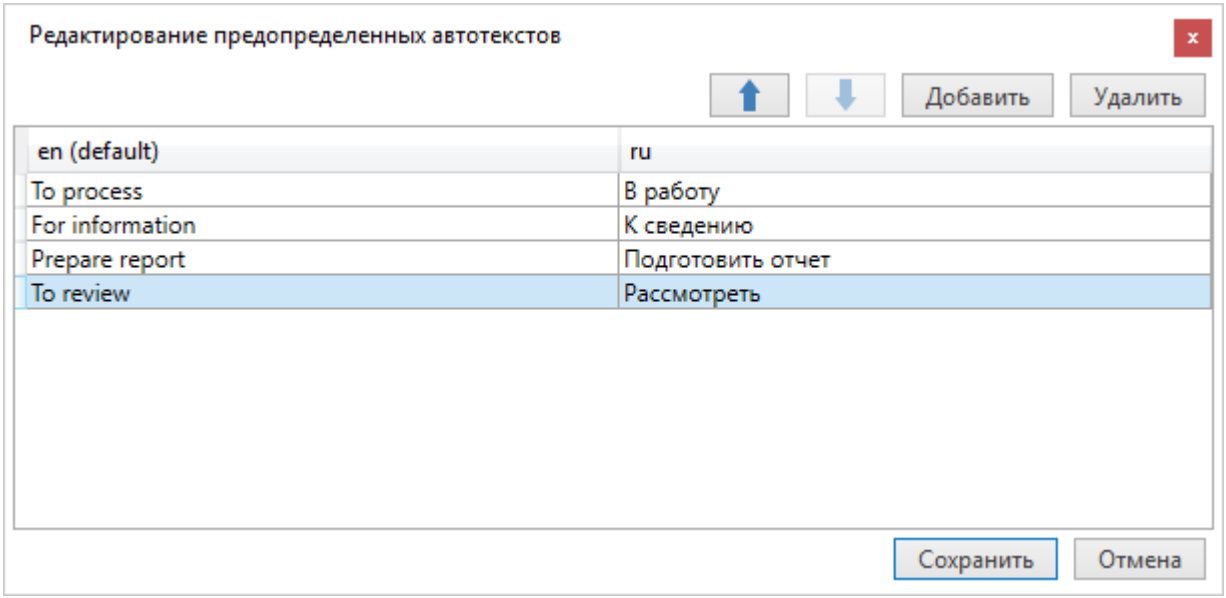
Пример редактора свойств для группы «Footer»



Пример редактора свойств контрола, связанного со свойством типа Строка



3. В появившемся окне добавьте варианты автотекстов:



Допустимая длина одного автотекста – до 100 символов. Рекомендуется вводить максимум 10 фраз. Чтобы изменить порядок значений, используйте кнопки ⬆ и ⬇.

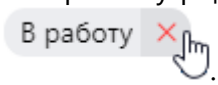
4. Нажмите на кнопку **Сохранить**. Введенные автотексты сохранятся в системные ресурсы типа сущности.

   Чтобы удалить все предопределенные автотексты, достаточно снять флажок **Включить автотексты**.

При первом вводе текста в поле сотрудники видят фразы, которые заданы в параметре **Предопределенные автотексты**. В процессе работы пользователь может использовать эти автотексты или вводить свой текст. Если сотрудник использует одну и ту же свою фразу 2 раза, она добавляется в базу данных как пользовательский автотекст и начинает отображаться в системе. Таким образом собирается статистика и для каждого сотрудника формируется

уникальный набор готовых автотекстов. Максимум в поле отображается 10 пользовательских и предопределенных автотекстов в соответствии с частотой использования.

Если флажок **Включить автотексты** установлен, но предопределенные автотексты не добавлены, то сперва сотрудник не видит фраз для заполнения поля. Но через некоторое время он начнет видеть фразы, которые 2 раза вводил сам.

По крестику рядом с автотекстом сотрудники могут удалять фразы, которыми не пользуются: 

. Предопределенные автотексты просто перестают отображаться в системе, а пользовательские, которые не используются 3 месяца, удаляются из базы данных с помощью фонового процесса «Удаление неиспользуемых автотекстов». Так уменьшается размер базы данных.

Если сотрудники не хотят использовать автотексты совсем, им достаточно в настройках проводника отключить их отображение. Поэтому перед добавлением автотекстов оцените, сколько пользователей будут использовать автотексты, нужно ли тратить время на их добавление.

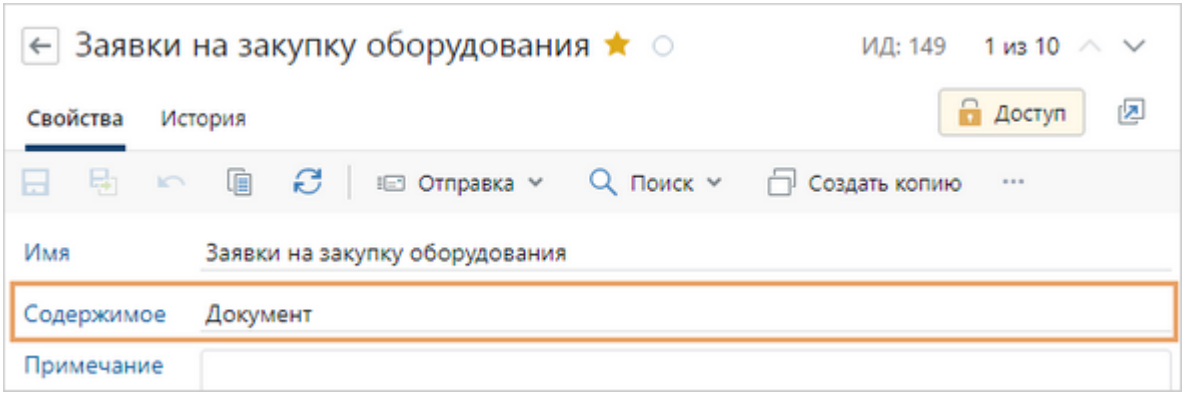
На своем *слое разработки* можно перекрывать типы сущностей базовых решений и дорабатывать автотексты. Но, если в базовом типе сущности добавят новые автотексты или удалят существующие, изменения не появятся в перекрытом типе.

Списки

Используйте редактор списка, чтобы настроить:

- внешний вид списка сущностей (Collection);
- панель фильтрации в списке;
- ленту списка;
- контекстное меню;
- внешний вид списка для содержимого папки (Folder).

  В проводнике системы содержимое папок представлено в виде списков. Для каждой папки указан тип содержимого:

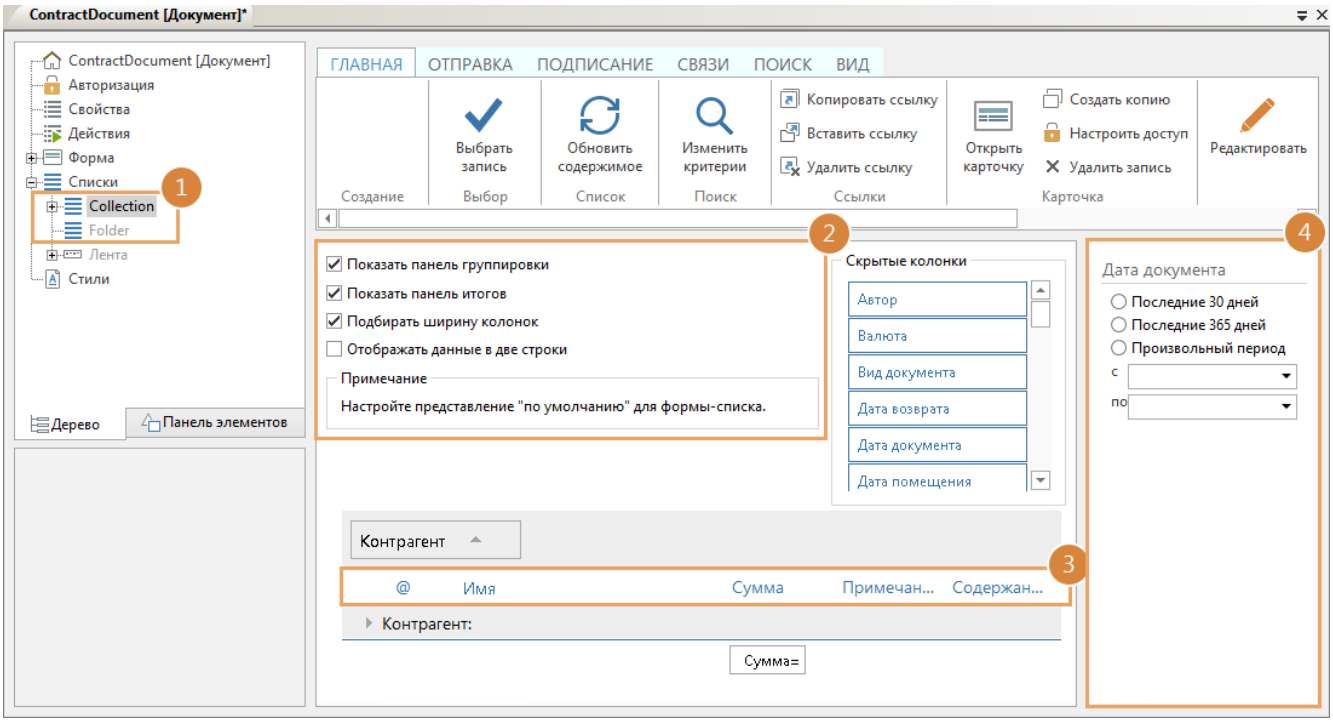
  В зависимости от типа изменяется вид списка по умолчанию для содержимого папки, состав действий на ленте и кнопки быстрого создания.

  Чтобы пользователь мог выбрать тип сущности как основное содержимое папки, в параметрах типа сущности установите флажок **Отображать в выборе содержимого для папки.**

Чтобы настроить внешний вид списка по умолчанию:

1. В редакторе списка перейдите в узел Collection.

   ПРИМЕЧАНИЕ. Чтобы настроить внешний вид списка для содержимого папки, перейдите в узел Folder.



2. Задайте настройки:

   Показать панель группировки. По умолчанию будет отображаться панель для группировки записей. При установке флажка рекомендуется настроить группировку записей по значению одной или нескольких колонок. Настроить группировку можно максимум по пяти колонкам. Для этого перетащите заголовки колонок на панель группировки. Например, для удобной работы со списком договорных документов можно сгруппировать список по контрагентам.

   Отображать данные в две строки. Записи будут отображаться в две строки. Рекомендуется указывать, если содержимое колонки не вмещается в одну строку.

   ПРИМЕЧАНИЕ. Настройки **Показать панель итогов** и **Подбирать ширину колонок** устаревшие, оставлены для совместимости.

3. Настройте видимые колонки. Для этого из списка перетащите заголовки колонок, которые нужно скрыть, в область **Скрытые колонки**. Колонки, которые нужно отобразить, перетащите в список. Чтобы изменить расположение колонки в списке, переместите ее в нужное место.

4. При необходимости настройте панель фильтрации для списка сущностей (Collection).

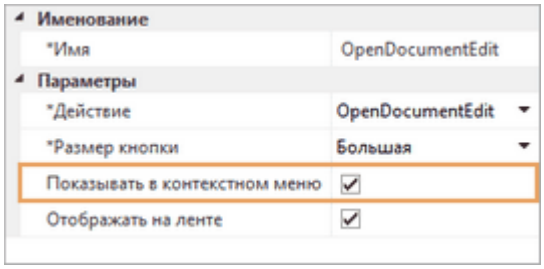
Лента

Для настройки ленты списка сущностей используйте узел «Лента» в редакторе списка. Лента списка настраивается аналогично ленте карточки. При этом на нее нельзя добавить контекстную вкладку и кнопку-переключатель состояния.

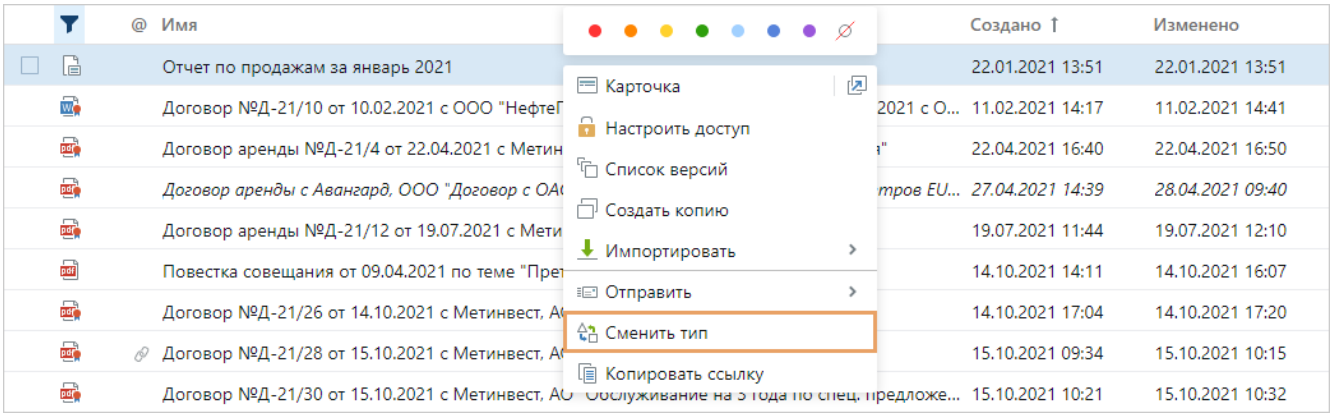
Для удобства работы пользователей кнопки, кнопки-меню или переключатели, которые добавлены на ленту, можно разместить в контекстном меню.

Например, для типа сущности «Простой документ» создано действие **Сменить тип**. Действие добавлено на ленту списка. Чтобы добавить действие в контекстное меню:

1. Выделите кнопку на ленте списка типа сущности.
2. В свойствах установите флажок **Показывать в контекстном меню**.



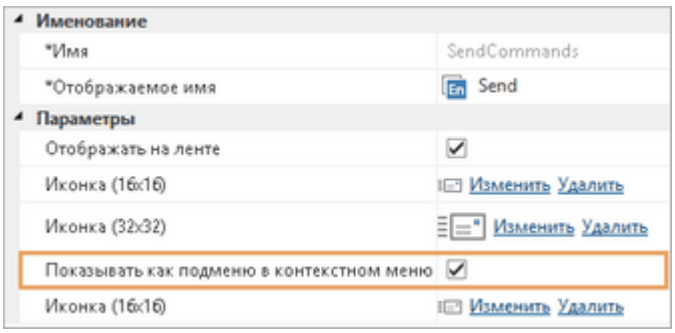
В результате в Directum RX контекстное меню будет выглядеть следующим образом:



Также в контекстное меню можно добавить группу действий.

Например, на ленте списка типа сущности «Входящий документ» на вкладке **Отправка** есть группа **Отправить**. Чтобы добавить группу действий в контекстное меню:

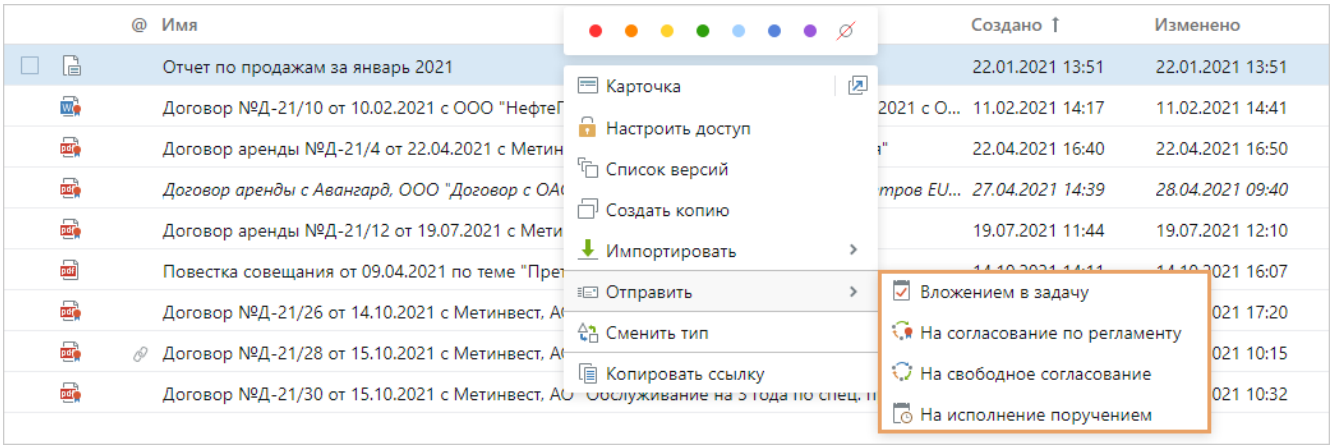
1. Выделите группу на ленте списка типа сущности.
2. В свойствах группы установите флажок **Показывать как подменю в контекстном меню**. Появится свойство **Иконка (16x16)**.



3. Выберите иконку в формате SVG и в размере 16x16 для отображения в контекстном меню.

При необходимости часть действий из группы можно скрыть в контекстном меню. Для этого выделите элемент группы на ленте и в свойствах элемента снимите флажок **Показывать в контекстном меню**.

В результате в Directum RX контекстное меню будет выглядеть следующим образом:



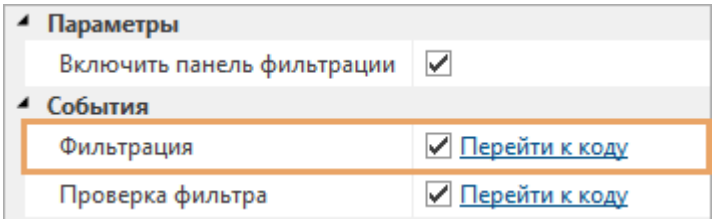
ПРИМЕЧАНИЕ. Перед настройкой ленты и контекстного меню ознакомьтесь с рекомендациями.

Пример настройки фильтрации записей в списке

Создан список «Входящие документы» и для него добавлена панель фильтрации. Необходимо добавить обработчик события **Фильтрация** и настроить фильтрацию записей в списке по критериям журнал регистрации, статус, вид документа и интервал времени, в котором документ был зарегистрирован или создан.

Чтобы добавить логику работы панели фильтрации:

1. В редакторе типа документа «Входящее письмо» (IncomingLetter) последовательно раскройте узлы «Списки», «Collection» и выделите узел «FilterPanel».
2. На панели свойств установите флажок **Фильтрация** и перейдите по ссылке. Откроется редактор программного кода.



3. Напишите код для фильтрации сущностей в списке. Обработчик события выполняется на веб-сервере.

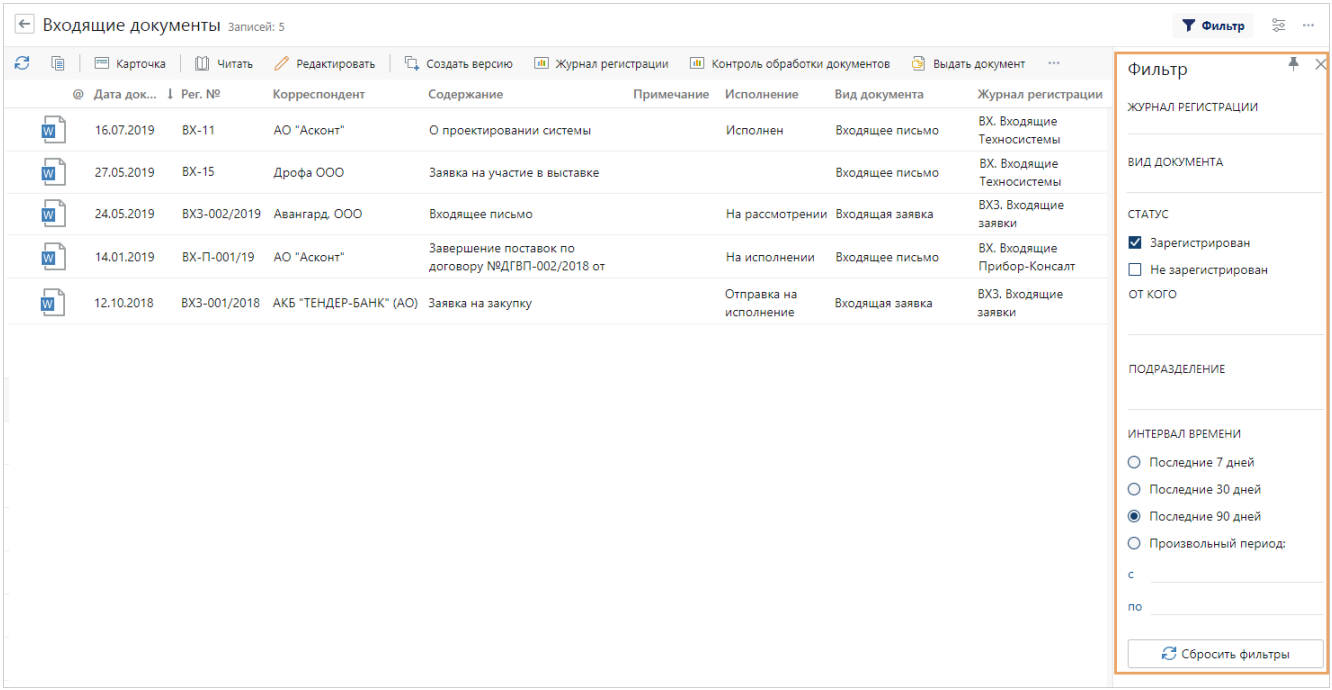
   ВАЖНО. В начало обработчика с помощью аргумента _filter добавьте проверку, что панель фильтрации включена.

```
/// <summary>
  /// Фильтрация списка входящих документов.
  /// </summary>
  public override IQueryable<T> Filtering(IQueryable<T> query,
  Sungero.Domain.FilteringEventArgs e)
  {
    // Проверка того, что панель фильтрации включена.
    if (_filter == null)
      return query;
    // Контрол "Ссылка на сущность". Фильтр по журналу регистрации.
    // Значение выбирается из справочника "Журналы регистрации".
```

```
  if (_filter.DocumentRegister != null)
    query = query.Where(d => d.DocumentRegister == _filter.DocumentRegister);
  // Контрол "Набор флажков". Фильтр по статусу документа.
  // Возможные значения: Зарегистрирован, Не зарегистрирован.
  if (_filter.Registered || _filter.NotRegistered)
    query = query
      .Where(d => (_filter.Registered && d.RegistrationState ==
OfficialDocument.RegistrationState.Registered) ||
           (_filter.NotRegistered && d.RegistrationState ==
OfficialDocument.RegistrationState.NotRegistered));
  // Контролы "Набор флажков" и "Период дат".
  // Фильтр по интервалу времени в котором документы были зарегистрированы
  // или созданы.
  var periodBegin =
Functions.Module.GetPeriodBeginFromFilter(_filter.LastWeek,
_filter.LastMonth, _filter.Last90Days, _filter.DateRangeFrom);
  var periodEnd = Functions.Module.GetPeriodEndFromFilter(_filter.LastWeek,
_filter.LastMonth, _filter.Last90Days, _filter.DateRangeTo);
  query = query.Where(j => (j.RegistrationDate ?? j.Created) >= periodBegin
&& (j.RegistrationDate ?? j.Created) <= periodEnd);
  return query;
}
```

4. Опубликуйте и проверьте разработку.

В результате в списке будут отображаться входящие документы организации с учетом фильтров:

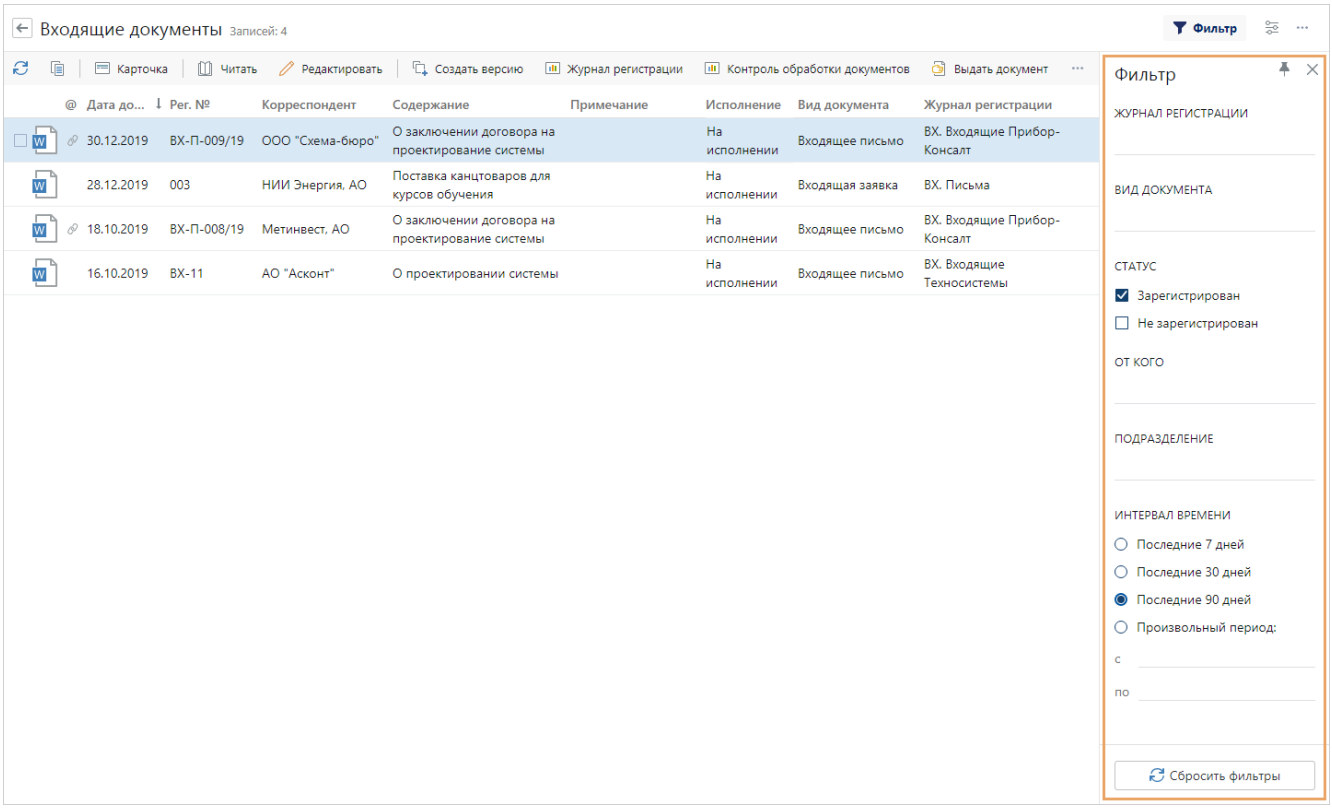


Аргумент _filter

Текущее состояние панели фильтрации хранится в аргументе **_filter**. Поэтому в обработчиках событий аргумент используется для обращения к панели фильтрации и ее элементам. Например, аргумент применяется в обработчиках событий **Фильтрация** для списков, а также в обработчиках событий **Получение данных** и **Проверка фильтра** для папок.

С аргументом **_filter** связана настройка фильтрации записей по критериям. Критериям соответствуют свойства, которые связаны с добавленными на панель фильтрации контролами. Поэтому, например, при фильтрации списка входящих документов список доступных свойств будет один, при фильтрации договоров – другой и т.д.

Критерии фильтрации для списка «Входящие документы»



Свойства

Имя	Тип	Описание
Counterparty	ICounterparty	Контрагент
DateRangeFrom	Nullable<DateTime>	Период дат с...
DateRangeTo	Nullable<DateTime>	Период дат по...
Department	IDepartment	Подразделение
DocumentKind	IDocumentKind	Вид документа
DocumentRegister	IDocumentRegister	Журнал регистрации
Last90Days	bool	Последние 90 дней
LastMonth	bool	Последние 30 дней
LastWeek	bool	Последние 7 дней
ManualPeriod	bool	Произвольный период, введенный вручную
NotRegistred	bool	Статус документа «Зарегистрирован»
Registred	bool	Статус документа «Не зарегистрирован»

Чтобы проверить, включена ли панель фильтрации, в начало обработчика события добавляется проверка вида:

```
if (_filter == null)
return query;
```

Пример. Фильтрация списка входящих документов по критериям

```
public override IQueryable<T> Filtering(IQueryable<T> query,
    Sungero.Domain.FilteringEventArgs e)
{
  // Проверить, что панель фильтрации включена.
  if (_filter == null)
    return query;
  // В контроле "Ссылка на сущность" отфильтровать по журналу регистрации.
  // Выбрать значение из справочника "Журналы регистрации".
  if (_filter.DocumentRegister != null)
    query = query.Where(d => d.DocumentRegister == _filter.DocumentRegister);
  // В контроле "Набор флажков" отфильтровать по статусу документа
«Зарегистрирован».
  if (_filter.Registered)
    query = query
      .Where(d => (_filter.Registered && d.RegistrationState ==
OfficialDocument.RegistrationState.Registered);
      return query;
}
```

Стили

Используйте узел «Стили», чтобы обратить внимание пользователя на определенную запись в списке или отдельную ячейку. Настроенные стили отображаются в:

- списках;
- вложениях;
- связях.

Создание стилей доступно для типов сущностей: справочник, документ, задание и уведомление Для настройки стилей используются свойства с типом Логическое, Дата, Перечисление и Ссылка. При этом можно использовать сразу несколько свойств.

Наследование и перекрытие стилей

При наследовании и перекрытии нельзя изменять приоритет и настройки стилей, созданных на предметном слое. Например, изменить цвет просроченного задания с красного на розовый не получится. Для стилей, созданных на прикладном уровне, при наследовании и перекрытии можно изменять все, кроме имени.

Настройка стиля для записи в списке

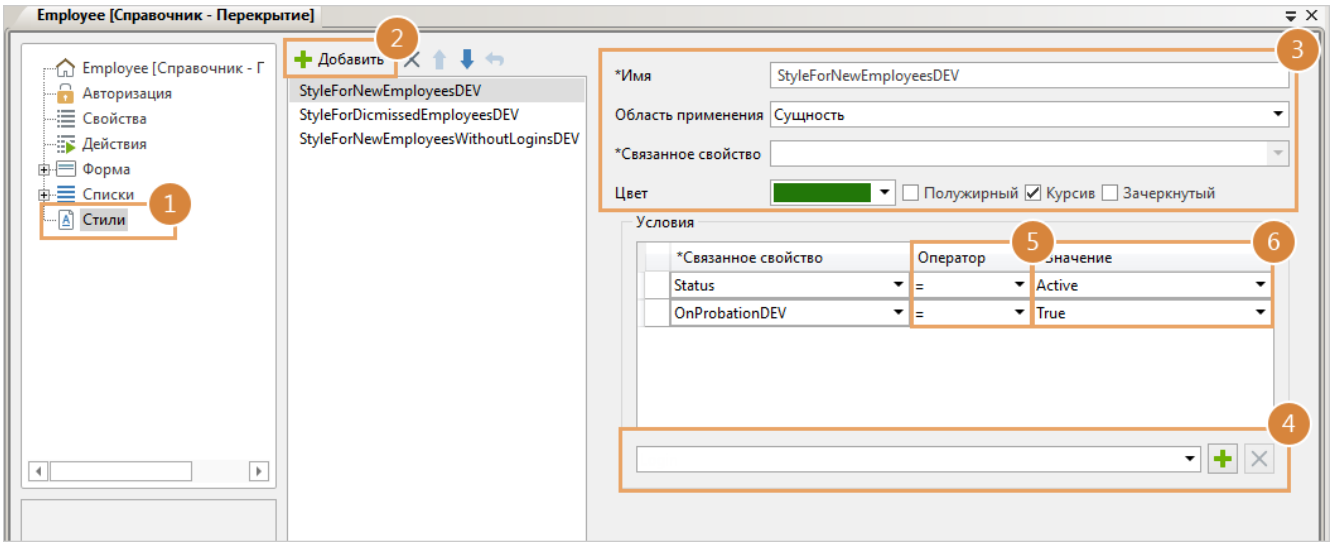
Предположим, в справочнике **Сотрудники** необходимо настроить стиль отображения записей в зависимости от текущего статуса сотрудника:

- уволившийся сотрудник отображается ~~зачеркнутым шрифтом~~;

- сотрудник на испытательном сроке отображается *зеленым курсивом*;
- неавтоматизированный сотрудник, т.е. тот, для которого не создана учетная запись, отображается фиолетовым цветом.

Для этого:

1. Перекройте тип справочника «Сотрудники» (Employee) и в дереве типа сущности перейдите в узел «Стили»:



2. Нажмите на кнопку **Добавить**.
3. Заполните поля:

   ***Имя** стиля. Должно быть уникальным в рамках типа сущности, состоять из букв латинского алфавита и цифр.

   Область применения. В выпадающем списке выберите значение **Сущность** для настройки стиля записи.

   Цвет. В выпадающем списке выберите необходимый цвет.

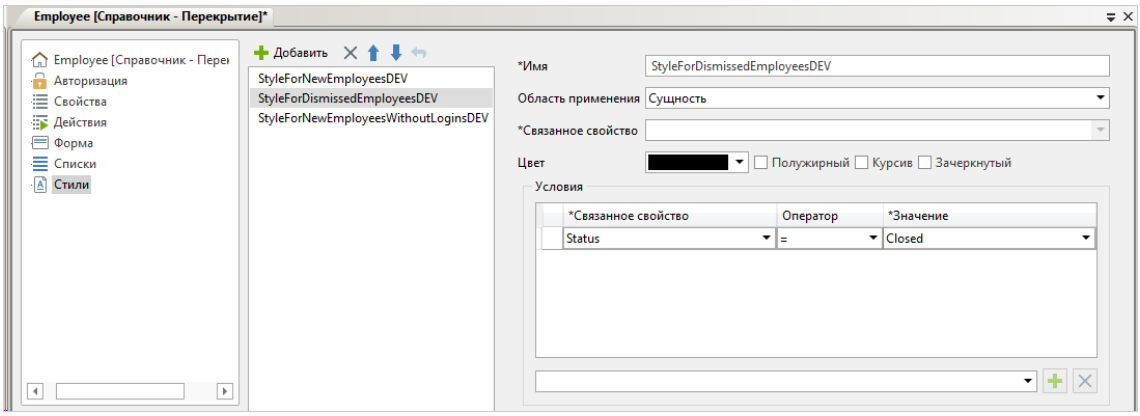
   Полужирный, **Курсив**, **Зачеркнутый стиль** начертания.
4. Под табличной частью «Условия» в выпадающем списке выберите связанное свойство. По нему будет определяться оформление записи. Нажмите на кнопку **Добавить критерий**. Значение добавится в табличную часть «Условия».
5. В табличной части «Условия» для каждого добавленного свойства укажите **Оператор**. В зависимости от типа свойства доступны операторы =, <>, >, >=, <, <=, **Null** и **Not Null**.
6. Если был выбран оператор сравнения, то выберите из выпадающего списка ***Значение** подходящее значение свойства.

Настройка стиля для сотрудника, который уволился

Чтобы настроить стиль для сотрудника, который уволился:

1. Укажите имя StyleForDismissedEmployeesDEV.
2. В выпадающем списке **Область применения** выберите значение **Сущность**.
3. Выберите черный цвет и зачеркнутое начертание.

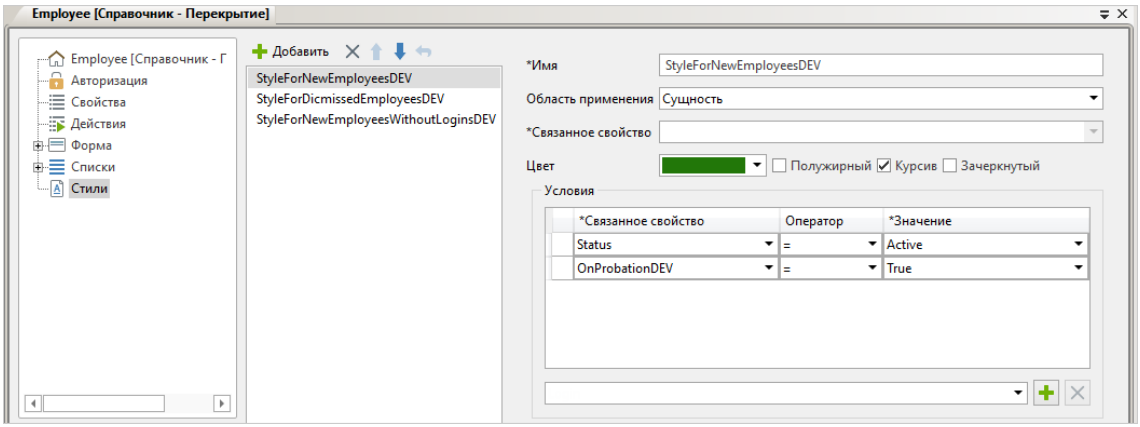
4. Для настройки стиля используйте свойство-перечисление **Состояние** (Status). Запись сотрудника должна быть зачеркнутой, когда свойство **Состояние** имеет значение **Закрытая** (Closed):



Настройка стиля для сотрудника на испытательном сроке

Чтобы настроить стиль для сотрудника на испытательном сроке:

1. Укажите имя StyleForNewEmployeesDEV.
2. В выпадающем списке Область применения выберите значение Сущность.
3. Выберите зеленый цвет и начертание курсивом.
4. Для настройки стиля используйте свойство-перечисление Состояние (Status) и добавленное разработчиком логическое свойство На испытательном сроке (OnProbationDEV). Запись сотрудника должна быть выделена зеленым курсивом, когда свойство Состояние имеет значение действующая, а свойство На испытательном сроке – Истинно (True):

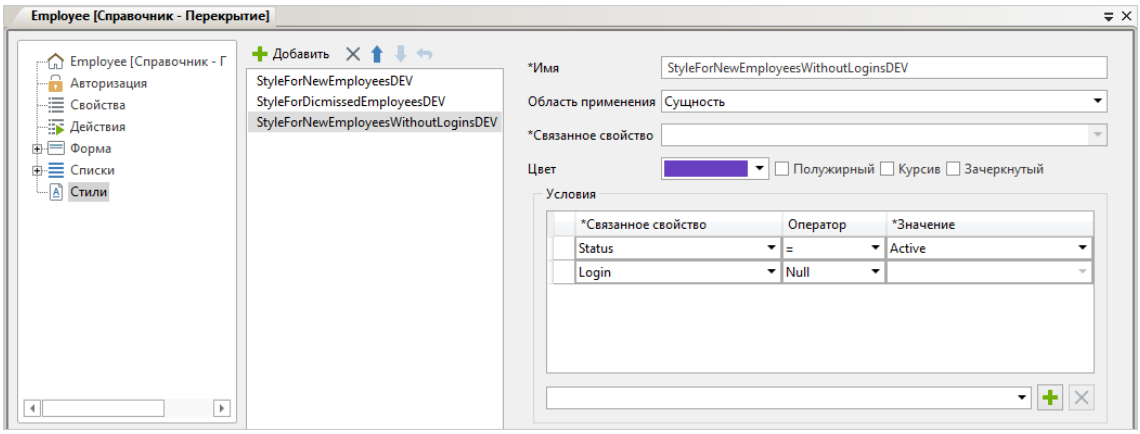


Настройка стиля для неавтоматизированного сотрудника

Чтобы настроить стиль для неавтоматизированного сотрудника:

1. Укажите имя StyleForEmployeesWithoutLoginsDEV.
2. Выберите фиолетовый цвет.
3. В выпадающем списке **Область применения** выберите значение **Сущность**.

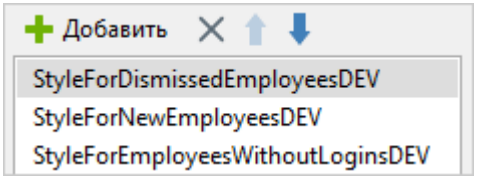
4. Для настройки стиля используйте свойство-перечисление Состояние (Status) и свойство-ссылку **Учетная запись** (Login). Запись сотрудника должна быть фиолетового цвета, когда свойство **Состояние** имеет значение действующая, а свойство **Учетная запись** не содержит значение:



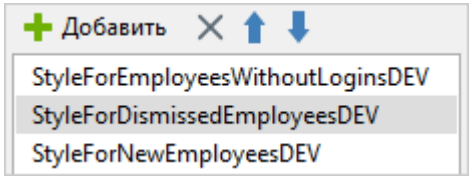
7. Расставьте стили в порядке приоритета с помощью кнопок и . Если для записи подходят сразу несколько стилей, то применится тот, который находится выше в списке стилей.

   Например, если неавтоматизированный сотрудник уволится, то цвет и начертание его записи в справочнике зависит от расположения StyleForDismissedEmployeesDEV и StyleForEmployeesWithoutLoginsDEV в списке стилей.

   Черный цвет, зачеркнутое начертание

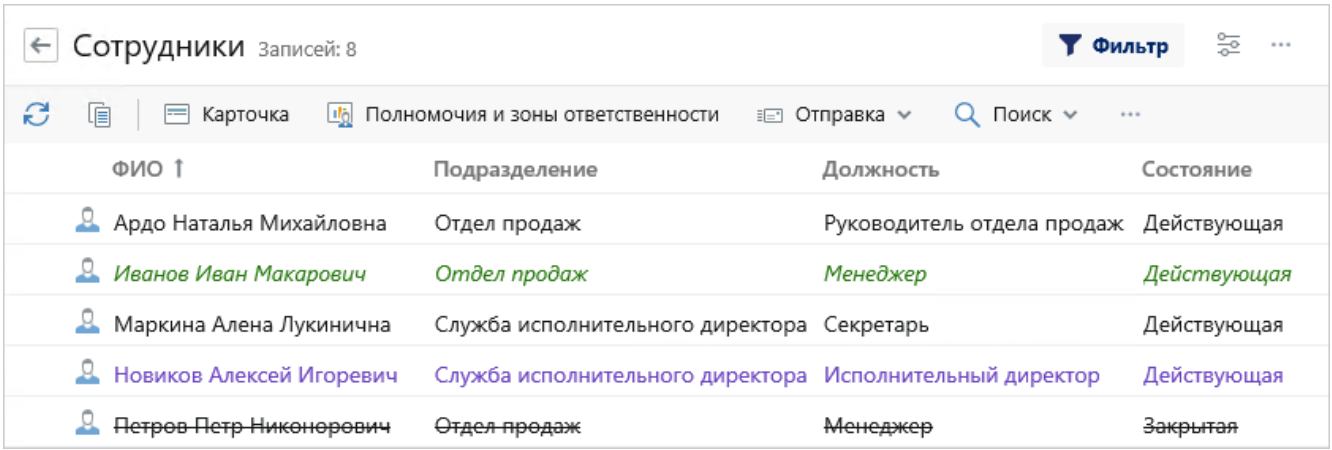
   

   Фиолетовый цвет без начертания

   ПРИМЕЧАНИЕ. При определении применяемого стиля также учитываются стили ячеек. Если один из них приоритетнее стилей записей, то применяются стили ячеек, иначе только стили записей.

В результате стиль отображения записей справочника **Сотрудники** будет изменяться в зависимости от указанных условий:



При определении применяемого стиля учитываются приоритет и выполнение условий обоих типов стилей – и записей, и ячеек. Если первым подходящим типом является:

- стиль записи, то применяется только стиль записи. При этом на одну запись может применяться только один стиль;
- стиль ячейки, то применяются только стили ячеек. При этом на одну запись может применяться несколько стилей ячеек, но только по одному стилю на свойство.

ПРИМЕЧАНИЕ. Для каждой записи применяемый стиль определяется отдельно.

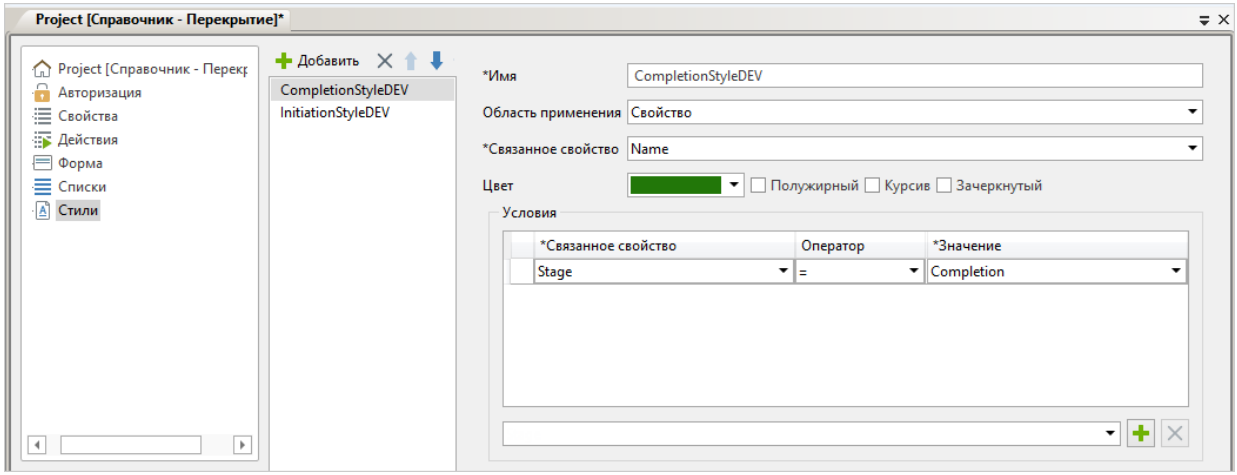
Настройка стиля для ячейки в списке

Предположим, что в списке «Реестр проектов» необходимо настроить стиль ячейки. Имя проекта и процент выполнения будут меняться в зависимости от его состояния:

- **Завершение** – имя отображается зеленым цветом, процент выполнения **полужирным зеленым цветом**;
- **Инициация** – имя отображается *оранжевым курсивом*.

Для этого:

1. Перекройте тип справочника «Проекты» (Project) и в дереве типа сущности перейдите в узел «Стили».
2. Нажмите на кнопку **Добавить** и настройте стиль для имени проекта в состоянии **Завершение**:



3. Заполните поля:

   ***Имя**. Укажите имя CompletionStyleDEV.

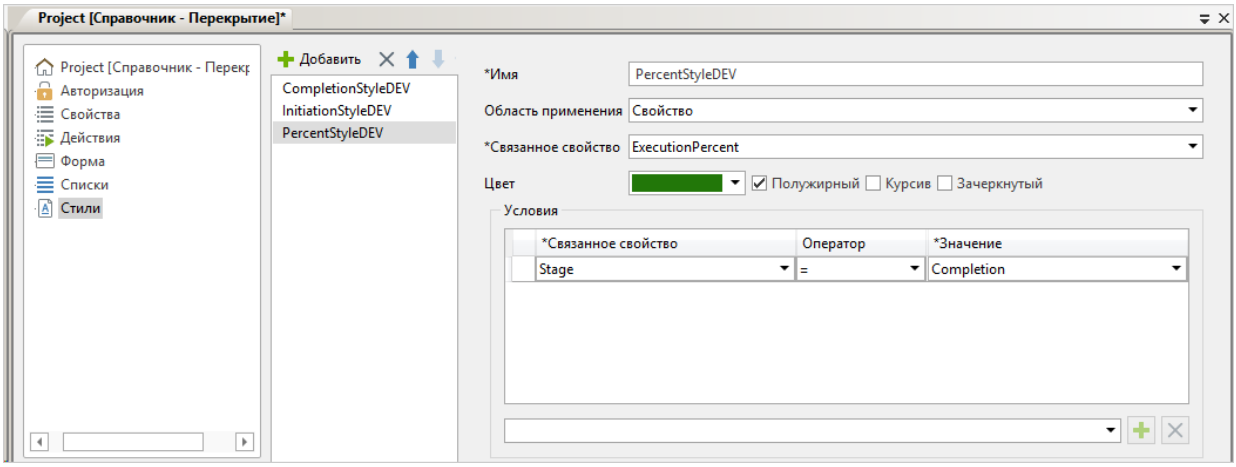
   Область применения. В выпадающем списке выберите значение **Свойство**. Поле ниже ***Связанное свойство** станет доступным.

   ***Связанное свойство**, на которое применяется стиль. Укажите **Name**.

   Цвет. Выберите зеленый.

4. Под табличной частью «Условия» в выпадающем списке выберите связанное свойство **Состояние** (Stage). Нажмите на кнопку **Добавить критерий** . Имя должно быть выделено зеленым цветом, когда **Состояние** имеет значение **Завершение** (Completion).

5. Нажмите на кнопку **Добавить** и настройте стиль для процента выполнения проекта в состоянии **Инициация**:



6. Заполните поля:

   ***Имя**. Укажите имя PercentStyleDEV.

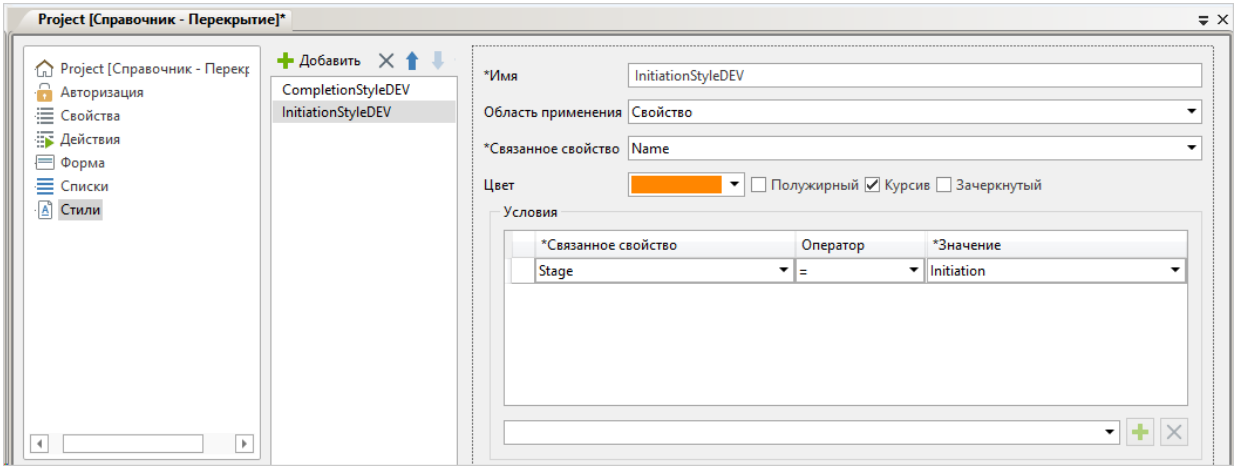
   Область применения. В выпадающем списке выберите значение **Свойство**. Поле ниже ***Связанное свойство** станет доступным.

   ***Связанное свойство**. Укажите **ExecutionPercent**.

   Цвет. Выберите зеленый цвет и полужирное начертание.

7. Под табличной частью «Условия» в выпадающем списке выберите связанное свойство **Состояние** (Stage). Нажмите на кнопку **Добавить критерий**. Процент завершения должен быть выделен зеленым полужирным шрифтом, когда **Состояние** имеет значение **Завершение** (Completion).

8. Нажмите на кнопку **Добавить** и настройте стиль для имени проекта в состоянии **Инициация**:



9. Заполните поля:

   ***Имя**. Укажите имя InitiationStyleDEV.

   Область применения. В выпадающем списке выберите значение **Свойство**. Поле ниже ***Связанное свойство** станет доступным.

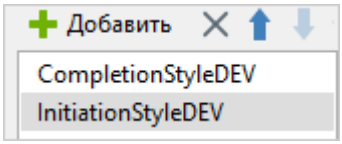
   ***Связанное свойство**. Укажите **Name**.

   Цвет. Выберите оранжевый цвет и начертание курсивом.

10. Под табличной частью «Условия» в выпадающем списке выберите связанное свойство **Состояние** (Stage). Нажмите на кнопку **Добавить критерий**. Имя должно

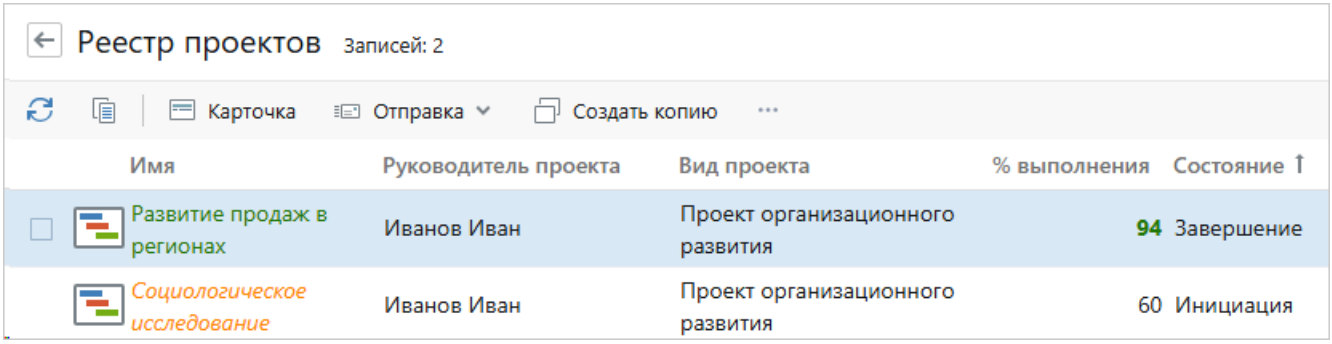
быть выделено оранжевым курсивом, когда **Состояние** имеет значение **Инициация** (Initiation).

Расставьте стили в порядке приоритета с помощью кнопок ⬆ и ⬇. Если для записи подходят сразу несколько стилей, то применится тот, который находится выше в списке стилей.



ПРИМЕЧАНИЕ. При определении применяемого стиля также учитываются стили записей. Если один из них приоритетнее стилей ячеек, то применяются стили записей, иначе только стили ячеек.

В результате стили отображения ячеек для свойств **Наименование** и **% Выполнения** «Реестр проектов» будет изменяться в зависимости от состояния проекта:



При определении применяемого стиля учитываются приоритет и выполнение условий обоих типов стилей – и записей, и ячеек. Если первым подходящим типом является:

- стиль записи, то применяется только стиль записи. При этом на одну запись может применяться только один стиль;
- стиль ячейки, то применяются только стили ячеек. При этом на одну запись может применяться несколько стилей ячеек, но только по одному стилю на свойство.

ПРИМЕЧАНИЕ. Для каждой записи применяемый стиль определяется отдельно.

События типов сущностей

Для реализации логики работы с сущностью используются:

- серверные события сущности. Выполняются на сервере, например, при сохранении, удалении, создании и копировании сущности;
- клиентские события сущности. Выполняются в клиентском приложении, например, при открытии, обновлении, закрытии карточки сущности;
- разделяемые события сущности. Выполняются на сервере или в клиентском приложении;
- события свойств сущности. Выполняются на сервере и/или в клиентском приложении, например, при изменений значения свойства сущности;
- события действий сущности. Выполняются в клиентском приложении, например, при выполнении действия с сущностью;

- события блоков схемы задачи. Выполняются на сервере и/или в клиентском приложении, например, при старте задачи. Подробнее см. примеры событий Старт задания и Выполнение задания;
- события групп вложений задачи, задания, задания на приемку, уведомления. Выполняются в разделяемом коде на сервере и/или в клиентском приложении, например, при добавлении вложения в группу.

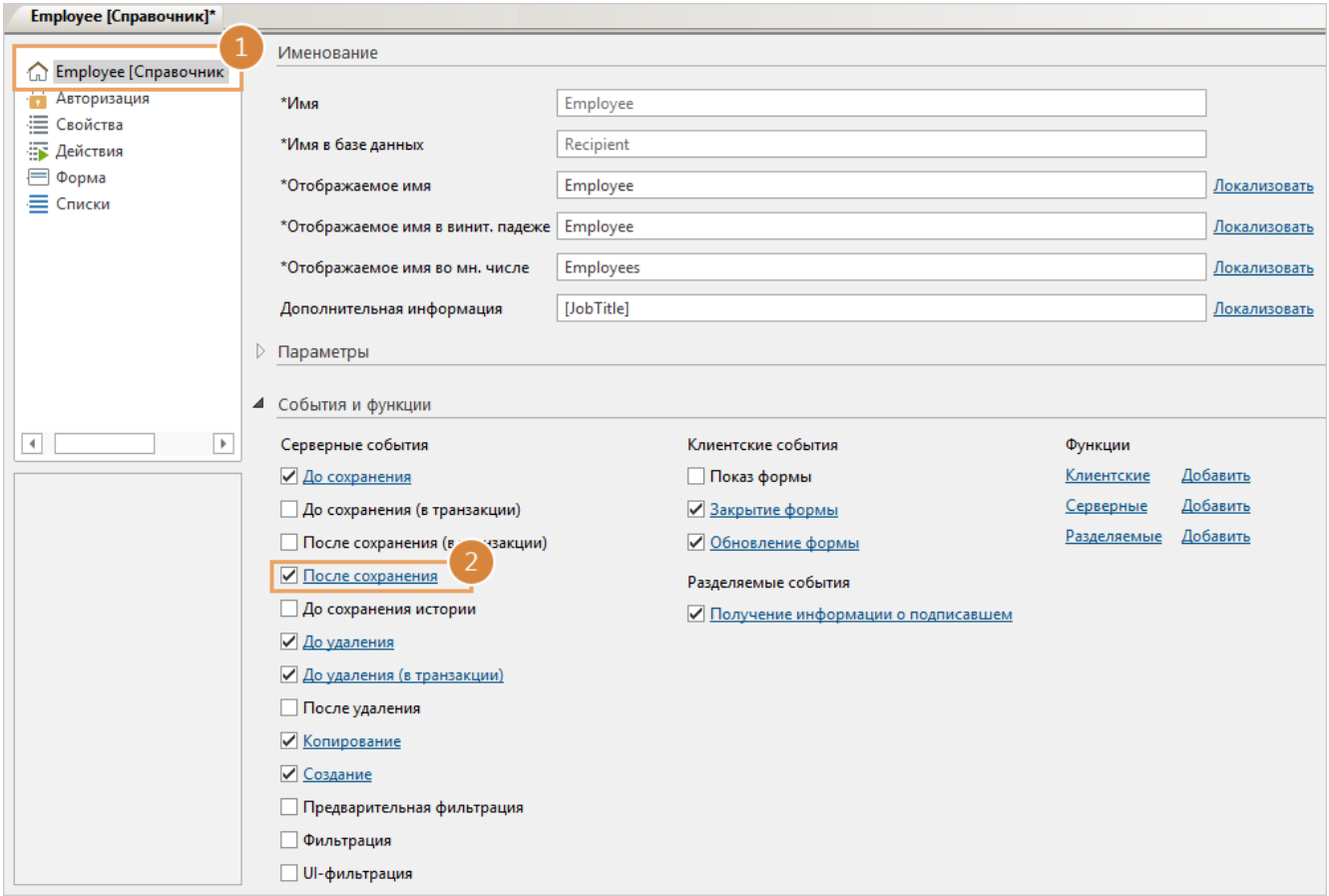
Например, если необходимо выполнить проверку заполнения свойств или изменить их при сохранении сущности, вычисления добавляются в событие До сохранения сущности.

Во всех событиях есть различные предопределенные аргументы и свойства. Например, **_obj**. позволяет получить текущую сущность, из которой это событие вызывается, а через **e.** можно обратиться к другим дополнительным свойствам и методам события. Например, через e.Params можно передать какие-либо параметры между двумя обработчиками событий одной сущности.

Для программной реализации логики события используется обработчик.

Как создать событие типа сущности

1. Откройте основной узел в редакторе типа сущности:



2. В группе «События и функции» рядом с нужным событием установите флажок. Станет доступна ссылка с названием события.
3. Перейдите по ссылке.
4. В открывшемся редакторе напишите код обработчика события.

Чтобы удалить код события, снимите флажок рядом с нужным событием.

Чтобы задать обработчики для других событий, перейдите в соответствующий узел. Например, для настройки событий свойств сущности перейдите в узел «Свойства».

ПРИМЕЧАНИЕ. События типов сущностей и свойств сущностей можно программно отключить.

Порядок выполнения событий

В разделе содержатся схемы выполнения событий сущности и ее формы:

- Сохранение
- Удаление
- Создание
- Открытие и закрытие карточки
- Подписание

При необходимости в обработчики событий можно добавить код.

Сохранение сущности

Сохранение сущности вызывается:

- в клиентском приложении. Например, при нажатии на кнопку **Сохранить** в карточке сущности;
- на сервере. Например, при вызове метода Save() в серверных функциях и обработчиках событий.

Если при сохранении сущности изменяется или создается другая сущность (далее на схеме – связанная сущность), то она тоже сохранится, даже если для нее не задан метод Save().

Порядок сохранения сущности:

Удаление сущности

Удаление сущности вызывается:

- в клиентском приложении. Например, при нажатии на кнопку **Удалить запись** в карточке или списке сущностей;
- на сервере. Например, при вызове метода Delete() в серверных функциях и обработчиках событий.

Если при удалении сущности изменяется или создается другая сущность (далее на схеме – связанная сущность), то она сохранится, даже если для нее не задан метод Save().

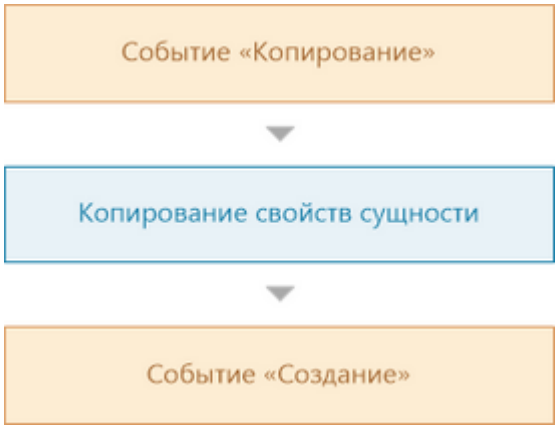
Порядок удаления сущности:

Создание сущности

Создание сущности вызывается:

- в клиентском приложении. Например, при нажатии на кнопку **Создать** или **Создать копию** в проводнике системы;
- на сервере. Например, при вызове метода Create() в серверных функциях и обработчиках событий.

Порядок создания сущности копированием:



ВАЖНО. При создании сущности, например по кнопке **Создать**, выполняется только обработчик события **Создание**.

Открытие и закрытие карточки сущности

Открытие и закрытие карточки сущности вызывается в клиентском приложении. Например, открытие карточки выполняется при нажатии на кнопку **Открыть карточку** в проводнике системы, при переходе по кнопкам в карточке сущности или при вызове метода Show() в клиентских функциях и обработчиках событий.

Порядок открытия и закрытия карточки сущности:

ПРИМЕЧАНИЕ. Обработчик события **Обновление формы** может выполняться не только после открытия карточки, но и во время работы с ней. Например, карточка исходящего письма обновится после резервирования номера по кнопке **Присвоить номер** на ленте. После обновления в карточке появится информационное сообщение о том, что номер зарезервирован.

Подписание документа

Подписание сущности вызывается в клиентском приложении. Например, при выполнении действия **Подписать** в проводнике системы или при вызове методов Approve(), Endorse(), NotEndorse() в обработчиках событий, серверных и клиентских функциях.

Порядок подписания документа:

Серверные события

События сущности задаются в редакторе типа сущности в группе «Серверные события». Выполняются на веб-сервере.

Серверные события	
События, которые есть у всех типов сущностей	
До сохранения	BeforeSave
До сохранения (в транзакции)	Saving
После сохранения (в транзакции)	Saved
После сохранения	AfterSave
До сохранения истории	BeforeSaveHistory

Серверные события	
До удаления	BeforeDelete
До удаления (в транзакции)	Deleting
После удаления	AfterDelete
Копирование	CreatingFrom
Создание	Created
Предварительная фильтрация	PreFiltering
Фильтрация	Filtering
События только для типов документов	
До подписания	BeforeSigning
Смена типа	ConvertingFrom
События только для типа задачи	
До старта	BeforeStart
До рестарта	BeforeRestart
До возобновления	BeforeResume
До прекращения	BeforeAbort
После приостановки	AfterSuspend
События только для типа задания	
До выполнения	BeforeComplete
События только для задания на приемку	
До принятия	BeforeAccept
До отправки на доработку	BeforeSendForRework
События только для конкурентного задания	
До взятия в работу	BeforeStartWork
До возврата невыполненным	BeforeReturnUncompleted
События только для типа справочника	
UI-фильтрация	UIFiltering
Показ подсказки (только для наследников Sungero.CoreEntities.User)	GetDigest

До сохранения (BeforeSave)

Выполняется до начала SQL-транзакции перед сохранением сущности в базе данных.

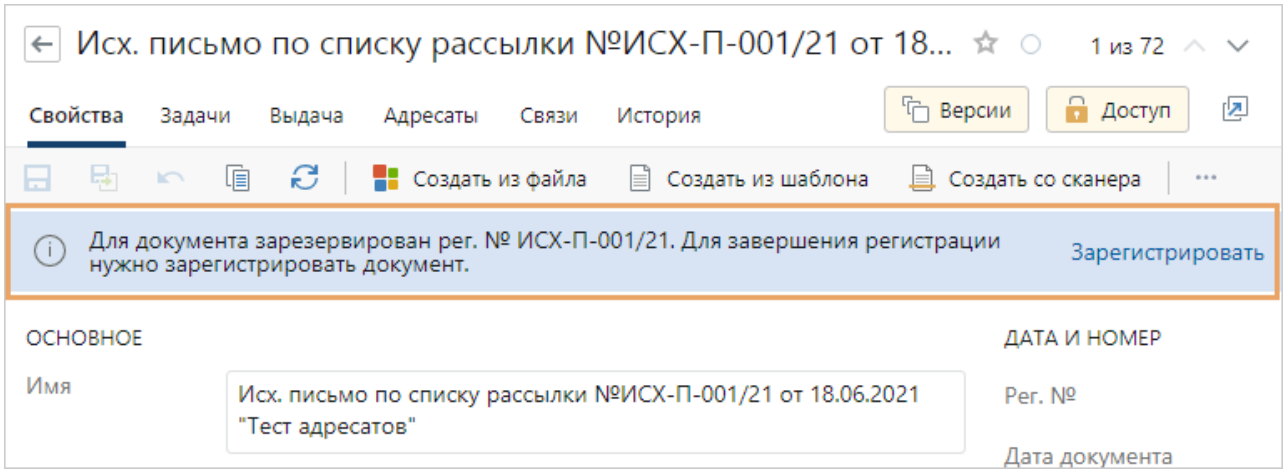
Назначение:

- *валидация* сущности (проверка заполнения свойств, наличие прав доступа);
- расчет вычисляемых свойств, например, итогов;
- изменение свойств текущей сущности, например, установка даты изменения;
- создание или изменение других сущностей.

Аргументы события:

- _obj – сущность, которая будет сохранена;
- e.AddError() – добавление сообщения об ошибке. Выполняемые операции будут прерваны;
- e.AddInformation() – добавление информационного сообщения. Выполняемые операции не будут прерваны;
- e.AddWarning() – добавление предупреждения. Выполняемые операции не будут прерваны.

  ПРИМЕЧАНИЕ. Сообщение об ошибке, информационное сообщение и предупреждение отображаются с соответствующей иконкой в верхней части карточки сущности. Например, информационное сообщение выглядит следующим образом:



- e.IsValid – признак того, что сущность валидная. Возможные значения:
  - **True**. Сущность валидная (доступна для сохранения);
  - **False**. Сущность не валидная. При сохранении сущности появится сообщение об ошибке, заданное в AddError.

  Например, если в карточке группы регистрации установить флажок **Регистрирует входящие документы**, то при сохранении карточки выполнится проверка группы. Если группа валидная, то она будет включена в роль «Регистраторы входящих документов»;
- e.Params – дополнительные параметры.

Пример:

```
   // Событие «До сохранения» группы регистрации.
public override void BeforeSave(Sungero.Domain.BeforeSaveEventArgs e)
{
  var responsible = _obj.State.Properties.ResponsibleEmployee.OriginalValue;
  var isResponsible = (responsible == null ||
Recipients.AllRecipientIds.Contains(responsible.Id)) &&
_obj.AccessRights.CanUpdate();
  // Если сотрудник не является ответственным за группу регистрации
  // и у него нет прав на выполнение операции «Управление правами доступа»,
  // то выводить сообщение "Недостаточно прав для изменения группы
регистрации".
  if (!isResponsible && !_obj.AccessRights.CanManage())
    e.AddError(RegistrationGroups.Resources.EnoughRights);
}
```

До сохранения (в транзакции) (Saving)

Выполняется в SQL-транзакции перед сохранением сущности в базе данных.

Назначение. Добавление вычислений, которые сохранятся в базе данных только в случае успешного сохранения сущности. Например, если при регистрации документа возникнет исключение, то регистрационный номер не будет сохранен в базу данных. При следующей регистрации номер будет заново вычисляться.

Аргументы события:

- _obj – сущность, которая будет сохранена;
- e.Params – дополнительные параметры.

Пример:

```
   // Выдать права участникам на повестку совещания.
public override void Saving(Sungero.Domain.SavingEventArgs e)
{
  base.Saving(e);
  PublicFunctions.Meeting.SetAccessRightsOnDocument(_obj.Meeting, _obj);
}
```

После сохранения (в транзакции) (Saved)

Выполняется в SQL-транзакции после сохранения сущности в базе данных. Сущность нельзя изменять в событии, так как она уже сохранена в базе данных.

Назначение. Выполнение дополнительной логики при сохранении сущности, которая должна выполняться в транзакции. Например, обращение к базе данных через SQL-запросы.

Аргументы события:

- _obj – сохраненная сущность;
- e.Params – дополнительные параметры.

Пример:

```
   // Если в нашей организации изменился руководитель, то у предыдущего
      руководителя
// удалить системные замещения.
public override void Saved(Sungero.Domain.SavedEventArgs e)
{
  if (_obj.State.Properties.CEO.IsChanged &&
    _obj.State.Properties.CEO.OriginalValue != null)
    Functions.Module.DeleteSystemSubstitutions(managers,
_obj.State.Properties.CEO.OriginalValue);
}
```

После сохранения (AfterSave)

Выполняется после завершения SQL-транзакции после сохранения сущности в базе данных. Сущность нельзя изменять в событии, так как она уже сохранена в базе данных.

ПРИМЕЧАНИЕ. Если в коде обработчика события возникнет исключение, пользователь увидит в системе соответствующее сообщение. При этом сама сущность все равно будет сохранена в базе данных.

Назначение. Выполнение дополнительной логики после сохранения сущности. Например, запись в лог-файл.

Аргументы события:

- _obj – сохраненная сущность;
- e.Params – дополнительные параметры.

Пример:

```
   // Сотруднику, указанному в карточке документа в поле «Ответственный»
// (Responsible), выдать права на чтение этого документа.
public override void AfterSave(Sungero.Domain.AfterSaveEventArgs e)
{
  // Проверить, есть ли у сотрудника права на чтение документа.
  if (!_obj.AccessRights.CanRead(_obj.Responsible))
    // Если нет – выдать.
    _obj.AccessRights.Grant(_obj.Responsible, DefaultAccessRightsTypes.Read);
}
```

До сохранения истории (BeforeSaveHistory)

Выполняется перед сохранением записи истории в системе. В том числе при вызове **entity.History.Write()** в серверном событии.

ПРИМЕЧАНИЕ. Если в коде обработчика события возникнет исключение, пользователь увидит в системе соответствующее сообщение. Все изменения, которые выполнялись с сущностью до выполнения события, будут отменены. Например, измененная сущность не будет сохранена.

Назначение:

- уточнение информации в истории. Например:
  - при регистрации документа вместо действия «Изменение» будет записываться действие «Регистрация» (переопределение записи истории);
  - при импорте документа в истории указывается информация, из какого файла сделан импорт;
- запись дополнительных действий в историю. Например, при регистрации официального документа изменяется его состояние. В этом случае в историю добавится две записи: регистрация и изменение состояния документа.

Аргументы события:

- _obj – сущность, для которой записывается история;
- e.Action – действие с сущностью. Например, **Создание**, **Изменение**, **Просмотр**, **Удаление**, **Настройка прав**, **Подписание**. Тип свойства: Перечисление. Доступно только для чтения;

- e.Operation – логическая операция в предметной области, выполняющаяся над сущностью. В истории работы с сущностью отображается в колонке **Действие**. Например, «Экспорт» документа. Если логической операции нет, то в колонке будет отображаться информация о действии с сущностью из e.Action. Тип свойства: **Перечисление**. Свойство можно переопределять;
- e.OperationDetailed – детальная информация о логической операции. В истории работы с сущностью отображается как локализованное значение в колонке **Комментарий**. Например, «Экспорт в файл». Тип свойства: **Перечисление**. Свойство можно переопределять;
- e.Comment – комментарий к операции. В истории работы с сущностью отображается как нелокализованная часть комментария в колонке **Комментарий**. Расположение в комментарии зависит от настроек OperationDetailed. Например, при экспорте документа в файл в комментарии указывается имя файла. Тип свойства: Строка. Свойство можно переопределять.

  ВАЖНО. Значение комментария формируется из двух свойств OperationDetailed и Comment. Подробнее о локализации см. в разделе «Локализация действий истории»;
- e.VersionNumber – номер версии документа, для которой выполняется действие. Тип свойства: Целое. Доступно только для чтения;
- e.HistoryDate – дата и время действия. Тип свойства: Дата. Доступно только для чтения;
- e.HostName – имя компьютера, с которого было выполнено действие. Тип свойства: **Строка**. Доступно только для чтения;
- e.IsSubstitute – признак того, что действие выполнялось по замещению. Возможные значения:
  - **True**. У замещающего пользователя нет прав на выполнение операции, у замещаемого пользователя есть права;
  - **False**. У замещающего пользователя есть права на выполнение операции.

  Тип свойства – Логическое. Доступно только для чтения;
- e.Write() – запись дополнительной информации в историю. При вызове метода событие **До сохранения истории** не будет вызываться повторно.

Пример:

При создании документа в историю работы добавляется отдельная запись регистрации. Для добавления записи в историю используется метод **Write()** в событии «До сохранения истории». Перед этим нужно локализовать значения операции.

```
    // Добавить отдельную запись регистрации в историю работы при создании
       документа.
public override void
BeforeSaveHistory(Sungero.Content.DocumentHistoryEventArgs e)
{
  var isCreateAction = e.Action ==
Sungero.CoreEntities.History.Action.Create;
  var operation = new
Enumeration(Constants.OfficialDocument.Operation.Registration);
  var operationDetailed = new
Enumeration(Constants.OfficialDocument.OperationDetailed.Registration);
```

```
  var comment = !string.IsNullOrWhiteSpace(_obj.RegistrationNumber) ?
    string.Join("|", _obj.RegistrationNumber, _obj.DocumentRegister) :
    string.Empty;
  if (isCreateAction && _obj.RegistrationState !=
RegistrationState.NotRegistered)
    e.Write(operation, operationDetailed, comment, null);
}
```

Другой пример добавления записи в историю см. в разделе «Как добавить действие в историю работы сущностью».

До удаления (BeforeDelete)

Выполняется до начала SQL-транзакции перед удалением сущности из базы данных.

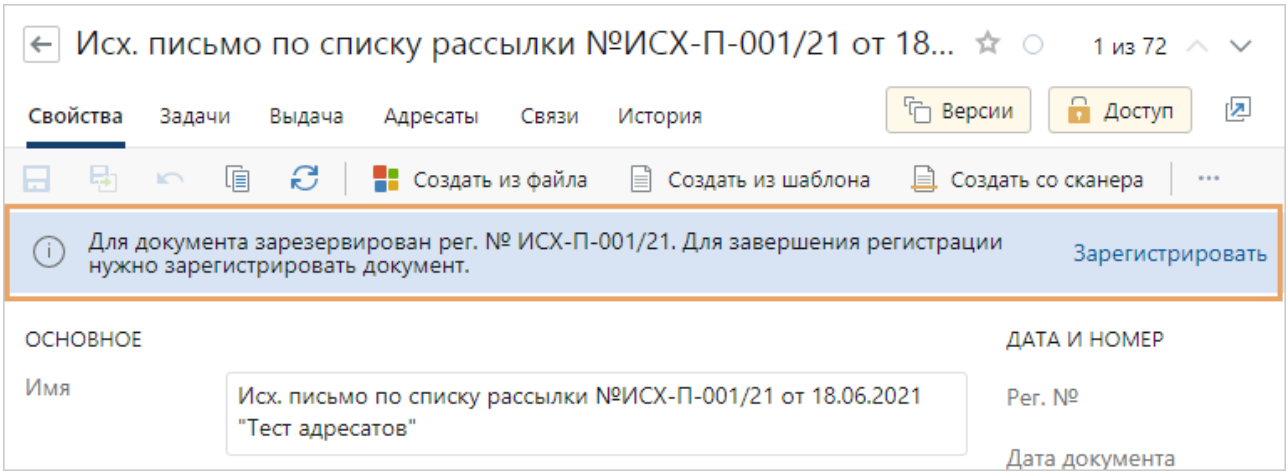
Назначение:

- *валидация* сущности (проверка выполнения условий перед удалением сущности);
- создание или изменение других сущностей.

Аргументы события:

- _obj – сущность, которая будет удалена;
- e.AddError() – добавление сообщения об ошибке. Выполняемые операции будут прерваны;
- e.AddInformation() – добавление информационного сообщения. Выполняемые операции не будут прерваны;
- e.AddWarning() – добавление предупреждения. Выполняемые операции не будут прерваны.

  ПРИМЕЧАНИЕ. Сообщение об ошибке, информационное сообщение и предупреждение отображаются с соответствующей иконкой в верхней части карточки сущности. Например, информационное сообщение выглядит следующим образом:



- e.IsValid – признак того, что сущность валидная. Возможные значения:
  - **True**. Сущность валидная (доступна для удаления);
  - **False**. Сущность не валидная. При удалении сущности появится сообщение об ошибке, заданное в AddError;
- e.Params – дополнительные параметры.

Пример:

```
   // Отобразить сообщение об ошибке, если сущность уже подписана.
public override void BeforeDelete(BeforeDeleteEventArgs e)
{
  if (_obj.IsSigned)
    e.AddError("Нельзя удалять подписанный документ");
}
```

До удаления (в транзакции) (Deleting)

Выполняется в SQL-транзакции перед удалением сущности из базы данных.

Назначение:

- внесение изменений в связанные сущности;
- выполнение дополнительной логики в транзакции при удалении сущности. Например, обращение к базе данных через SQL-запросы.

Аргументы события:

- _obj – сущность, которая будет удалена;
- e.Params – дополнительные параметры.

После удаления (AfterDelete)

Выполняется после завершения SQL-транзакции, после удаления сущности из базы данных.

ПРИМЕЧАНИЕ. Если в коде обработчика события возникнет исключение, пользователь увидит в системе соответствующее сообщение. При этом сущность будет удалена из базы данных.

Назначение. Выполнение дополнительной логики после удаления сущности. Например, запись в лог-файл.

Аргументы события:

- _obj – сущность, которая была удалена;
- e.Params – дополнительные параметры.

Пример:

```
   // В задаче на ознакомление с документом (AcquaintanceTask) удалить список
// участников ознакомления.
public override void AfterDelete(Sungero.Domain.AfterDeleteEventArgs e)
{
  // Удалить список участников ознакомления, соответствующий задаче.
  if (_obj != null)
  {
    var participants = AcquaintanceTaskParticipants.GetAll().FirstOrDefault(x
 => x.TaskId == _obj.Id);
    if (participants != null)
      AcquaintanceTaskParticipants.Delete(participants);
  }
}
```

Копирование (CreatingFrom)

Выполняется при создании сущности копированием из другой сущности.

Назначение. Переопределение состава копируемых свойств при создании новой сущности из другой. Изменять создаваемую сущность в обработчике события нельзя.

Аргументы события:

- _source – сущность, которая копируется;
- _info – информация о сущности;
- e.Map() – правило копирования значений свойств исходного типа сущности. Используется, например, когда нужно чтобы свойство скопировалось иначе;
- e.Without() – исключение свойств из списка копируемых;
- e.WithoutAccessRights() – создание сущности без копирования прав доступа;
- e.Params – дополнительные параметры.

Пример 1:

```
   // При копировании организации запретить копировать комментарий.
public override void CreatingFrom(Sungero.Domain.CreatingFromEventArgs e)
{
  base.CreatingFrom(e);
  e.Without(_info.Properties.IsCardReadOnly);

  // Организацию нельзя редактировать, если она создана как копия нашей
  // организации. При копировании такой организации, комментарий в новую
  // переноситься не должен.
  if (_source.IsCardReadOnly == true)
    e.Without(_info.Properties.Note);
}
```

Пример 2:

```
   // Создать копию документа без копирования прав доступа.
public override void CreatingFrom(Sungero.Domain.CreatingFromEventArgs e)
{
  base.CreatingFrom(e);
  e.WithoutAccessRights();
}
```

Создание (Created)

Выполняется при создании новой сущности.

Назначение. Заполнение свойств сущности значениями по умолчанию.

Аргументы события:

- _obj – создаваемая сущность;
- e.Params – дополнительные параметры.

Пример:

```
   // Запретить создание карточки справочника "Персоны"
```

```
// из любых мест, кроме карточки справочника "Контакты".
public override void Created(Sungero.Domain.CreatedEventArgs e)
{
  if (!CallContext.CalledFrom(Contacts.Info))
    throw new InvalidOperationException("Создавать персону можно только из
карточки контакта.");

  // Дальнейшие вычисления
}
```

ВАЖНО. В событии «Создание» можно выдать права доступа для роли, группы или пользователя на создаваемую сущность. При этом отключается стандартное поведение системы: автор не получит полные права на сущность. Если автору тоже нужны полные права, их нужно выдать явно, например:

```
   public override void Created(Sungero.Domain.CreatedEventArgs e)
{
  base.Created(e);
  _obj.AccessRights.Grant(Roles.AllUsers,
Constants.DocumentKind.DocumentKindChoiseAccessRightType);
  _obj.AccessRights.Grant(Users.Current,
DefaultAccessRightsTypes.FullAccess);
}
```

Предварительная фильтрация (PreFiltering)

Выполняется каждый раз при получении сущности или списка сущностей.

Назначение. Используется для оптимизации выполнения тяжелого SQL-запроса к большому списку сущностей с панелью фильтрации. Например, предварительную фильтрацию можно использовать для ограничения выборки данных, если список содержит более 10 млн записей.

Чтобы выполнить запрос к базе данных оптимально, при добавлении события соблюдайте условия:

- критерии фильтрации события PreFiltering должны значительно сократить объем данных;
- для выбранных критериев фильтрации в базе данных созданы индексы;
- критерии не должны фильтровать список по логическим свойствам. Вместо этого, например, задайте предварительную фильтрацию по подразделению или организации.

Если событие **Предварительная фильтрация** задано, то SQL-запрос выполняется в два этапа:

- событие PreFiltering формирует запрос на получение выборки данных по базовым критериям. В результате список записей ограничивается для дальнейшей фильтрации;
- событие Filtering фильтрует получившийся список по остальным критериям.

В коде события PreFiltering можно сформировать содержимое списка по часто используемым критериям, например отфильтровать реестр договоров по периоду. При этом в выборку попадают только записи, на которые у пользователя есть права доступа. Затем запрос из события Filtering выполняется к уменьшенному объему данных.

События формируют один общий запрос вместо двух, если выполняется одно из условий:

- вычисления перенесены с веб-сервера, например на сервис асинхронных событий;
- в событии PreFiltering не заданы фильтры;
- в списке нет панели фильтрации;
- используется СУБД Microsoft SQL Server.

Возвращаемое значение. Измененный запрос на получение списка сущностей. По умолчанию возвращается исходный запрос.

Аргументы события:

- query – исходный запрос на получение списка сущностей;
- e.Params – дополнительные параметры;
- _createFromTemplateContext – документ, для которого создается версия из шаблона. Аргумент отображается только в шаблонах документов, наследниках от Sungero.Content.ElectronicDocumentTemplate. Подробнее см. пример.

ПРИМЕЧАНИЕ. Не рекомендуется использовать оператор OR в коде обработчика события фильтрации, если предполагается, что запрос будет обращаться к большим спискам (более 1 млн. записей).

Фильтрация (Filtering)

Выполняется каждый раз при получении сущности или списка сущностей.

Назначение. Фильтрация списка сущностей. Например, в событии пишут код, чтобы сформировать содержимое списка по умолчанию, добавить условия фильтрации для списка с панелью фильтрации, отфильтровать список шаблонов, доступных для документа.

Возвращаемое значение. Измененный запрос на получение списка сущностей. По умолчанию возвращается исходный запрос.

Аргументы события:

- query – исходный запрос на получение списка сущностей;
- e.Params – дополнительные параметры;
- _createFromTemplateContext – документ, для которого создается версия из шаблона. Аргумент отображается только в шаблонах документов, наследниках от Sungero.Content.ElectronicDocumentTemplate. Подробнее см. пример.

ПРИМЕЧАНИЕ. Не рекомендуется использовать оператор OR в коде обработчика события фильтрации, если предполагается, что запрос будет обращаться к большим спискам (более 1 млн. записей).

Пример:

```
   // При создании документа фильтровать список шаблонов.
public override IQueryable<T> Filtering(IQueryable<T> query,
Sungero.Domain.FilteringEventArgs e)
{
  // Проверить, что фильтрация выполняется
  // при создании документа из шаблона.
  if (_createFromTemplateContext != null)
  {
    // Фильтровать шаблоны по статусу.
    query = query.Where(d => d.Status == Status.Active);

    // Фильтровать шаблоны по критериям.
    if (Docflow.OfficialDocuments.Is(_createFromTemplateContext))
    {
      var document =
Docflow.OfficialDocuments.As(_createFromTemplateContext);
      query = query.Where(template =>
template.DocumentKind.Equals(document.DocumentKind));
    }
  }
  return query;
}
```

Подробнее см. пример разработки «Как отфильтровать список шаблонов, доступных при создании документа».

До подписания (BeforeSigning)

Выполняется на веб-сервере при подписании документа.

Назначение. *Валидация* сущности (проверка выполнения условий при подписании документа определенным пользователем с одним из результатов: **Согласовать**, **Не согласовать**, **Утвердить**).

Аргументы события:

- _obj – подписываемый документ;
- e.Certificate – сертификат, которым подписан документ;
- e.Signature – подпись, которая устанавливается на документ;
- e.SignatureTargetFormat – целевой формат усовершенствования подписи;
- e.AddError() – добавление сообщения об ошибке. Выполняемые операции будут прерваны;
- e.IsValid – признак того, что сущность валидная. Возможные значения:

  True. Сущность валидная (документ доступен для подписания).

  False. Сущность не валидная. При подписании документа появится сообщение AddError.
- e.Params – дополнительные параметры.

Пример: в системе настроено усовершенствование всех подписей до формата CAdES-T, а для подписей кадровых документов необходим другой формат – CAdES-A. Для этого у нужного типа документа переопределите событие **До подписания** (BeforeSigning) и в аргументе **e.SignatureTargetFormat** укажите целевой формат усовершенствования подписи:

```
   public override void BeforeSigning(Sungero.Domain.BeforeSigningEventArgs e)
{
  e.SignatureTargetFormat = SignatureAutoImproveTargetFormat.CadesA;
}
```

Смена типа (ConvertingFrom)

Выполняется на веб-сервере при смене типа документа.

Назначение. Определение правил переноса значений свойств исходного типа документа в новый тип.

Аргументы события:

- e.Map() – правило переноса значений свойств исходного типа документа в новый тип. По умолчанию общие свойства всегда переносятся. Если свойства не общие или нужно изменить правило переноса, то укажите их в e.Map();
- e.Without() – исключение свойств, которые переносятся в новый тип документа;
- e.Params – дополнительные параметры.

ВАЖНО. Не используйте аргумент **e.WithoutAccessRights()** – смена типа без копирования прав доступа. При его выполнении в системе возникает исключение.

Пример:

```
   // Сменить тип входящего документа электронного обмена.
public override void ConvertingFrom(Sungero.Domain.ConvertingFromEventArgs e)
{
  base.ConvertingFrom(e);

  // Не копировать состояние жизненного цикла для входящего документа
эл.обмена.
  if (ExchangeDocuments.Is(_source) &&
      OfficialDocuments.As(_source).LifeCycleState !=
OfficialDocument.LifeCycleState.Obsolete)
    e.Without(_info.Properties.LifeCycleState);

  // Дальнейшие вычисления.
}
```

До старта (BeforeStart)

Выполняется на веб-сервере перед стартом задачи.

Назначение:

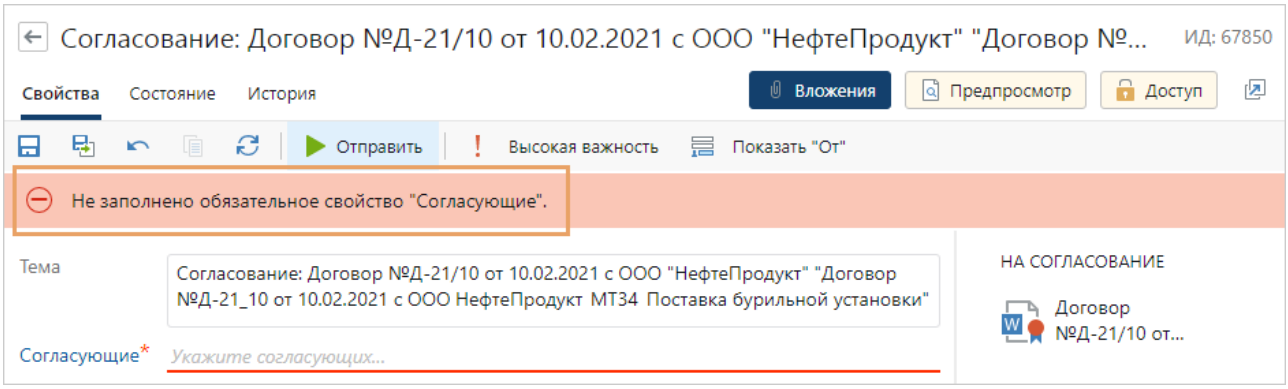
- *валидация* задачи (проверка возможности старта задачи, проверка заполнения свойств);

- заполнение текста задачи или свойств информацией из связанных сущностей или вложений.

Аргументы события:

- _obj – отправляемая задача;
- e.AddError() – добавление сообщения об ошибке. Выполняемые операции будут прерваны;
- e.AddInformation() – добавление информационного сообщения. Выполняемые операции не будут прерваны;
- e.AddWarning() – добавление текста с предупреждением. Выполняемые операции не будут прерваны.

  ПРИМЕЧАНИЕ. Сообщение об ошибке, информационное сообщение и предупреждение отображаются в верхней части формы карточки с соответствующей иконкой. Например, сообщение об ошибке выглядит следующим образом:



- e.IsValid – признак того, что задача валидная. Возможные значения:
  - **True**. Задача валидная (доступна для старта);
  - **False**. Задача не валидная. При старте задачи появится сообщение AddError.

    Например, при старте задачи выполнится проверка заполнения обязательных полей. Если все обязательные поля заполнены, то задача будет отправлена исполнителю.
- e.Params – дополнительные параметры.

Пример:

```
    // При старте задачи проверить, что документы вложены в группу «DocumentsForSign».
public static void BeforeStart(Sungero.Workflow.Server.BeforeStartEventArgs e)
{
  var attachmentsAddedHere = _obj.DocumentsForSign.Where(d =>
_obj.AttachmentsInfo.Any(i => i.Target == entity && i.LinkedTo(a));

  if (!attachmentsAddedHere.Any())
    e.AddError("Вложите документы на подписание");
}
```

До рестарта (BeforeRestart)

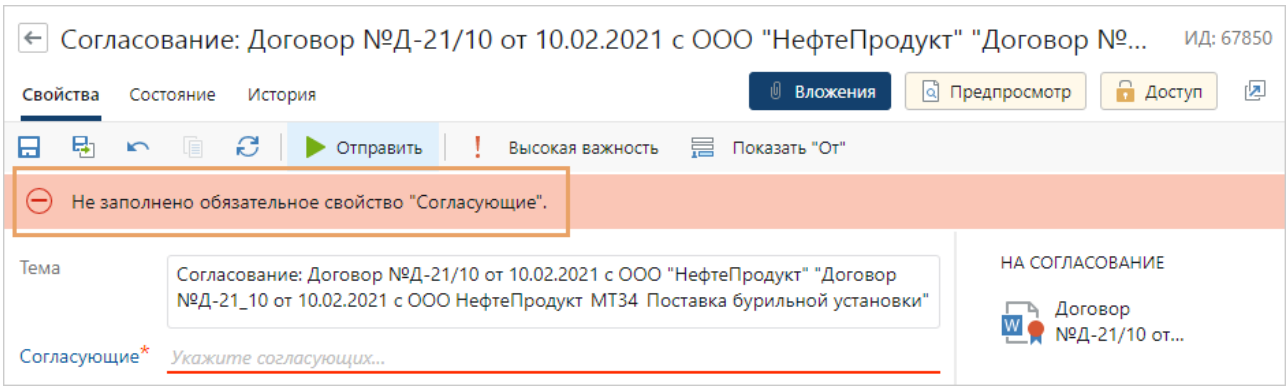
Выполняется на веб-сервере перед рестартом задачи.

Назначение. Заполнение свойств задачи перед рестартом. При этом свойства задачи становятся доступными для изменения.

Аргументы события:

- _obj – возобновляемая задача;
- e.AddError() – добавление сообщения об ошибке. Выполняемые операции будут прерваны;
- e.AddInformation() – добавление информационного сообщения. Выполняемые операции не будут прерваны;
- e.AddWarning() – добавление текста с предупреждением. Выполняемые операции не будут прерваны.

  ПРИМЕЧАНИЕ. Сообщение об ошибке, информационное сообщение и предупреждение отображаются в верхней части формы карточки с соответствующей иконкой. Например, сообщение об ошибке выглядит следующим образом:



- e.IsValid – признак того, что задача валидная. Возможные значения:
  - **True**. Задача валидная (доступна для рестарта);
  - **False**. Задача не валидная. При рестарте задачи появится сообщение AddError.
- e.Params – дополнительные параметры.

Пример:

```
    // При рестарте задачи на исполнение поручения (ActionItemExecutionTask)
       очистить
// ее причину прекращения и статус.
public override void
BeforeRestart(Sungero.Workflow.Server.BeforeRestartEventArgs e)
{
  _obj.AbortingReason = string.Empty;
  _obj.ExecutionState = null;

  // Дальнейшие вычисления.
}
```

До возобновления (BeforeResume)

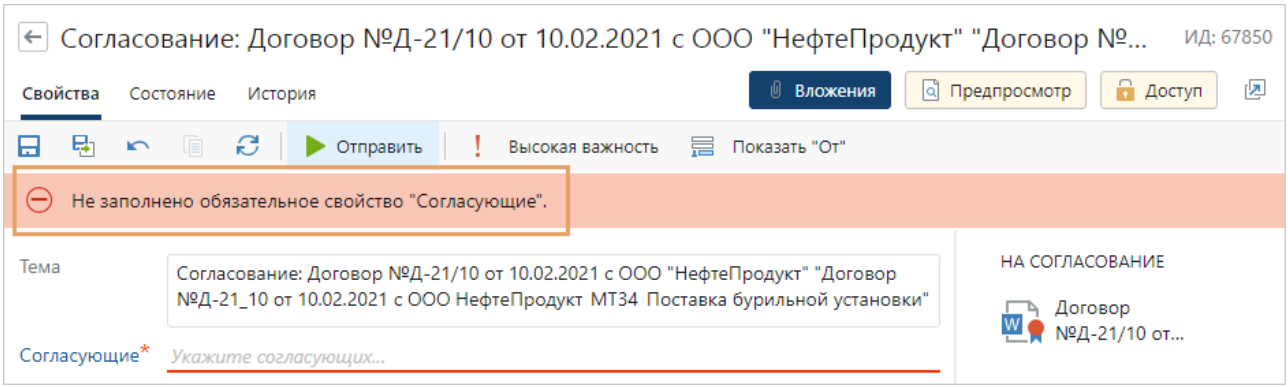
Выполняется на веб-сервере перед тем, как возобновить задачу.

Назначение. Выполнение логики, необходимой для корректного возобновления задачи.

Аргументы события:

- _obj – возобновляемая задача;
- e.AddError() – добавление сообщения об ошибке. Выполняемые операции будут прерваны;
- e.AddInformation() – добавление информационного сообщения. Выполняемые операции не будут прерваны;
- e.AddWarning() – добавление текста с предупреждением. Выполняемые операции не будут прерваны.

  ПРИМЕЧАНИЕ. Сообщение об ошибке, информационное сообщение и предупреждение отображаются в верхней части формы карточки с соответствующей иконкой. Например, сообщение об ошибке выглядит следующим образом:



- e.IsValid – признак того, что задача валидная. Возможные значения:
  - **True**. Задача валидная (доступна для возобновления);
  - **False**. Задача не валидная. При возобновлении задачи появится сообщение AddError.
- e.Params – дополнительные параметры.

Пример:

```
    // При возобновлении задачи проверить, что документы вложены в группу «DocumentsForSign».
public override void BeforeResume(Sungero.Workflow.Server.BeforeResumeEventArgs e)
{
  base.BeforeResume(e);
  Functions.ApprovalTask.RevokeAuthorAccessRights(_obj);
}
```

До прекращения (BeforeAbort)

Выполняется на веб-сервере перед тем, как прекратить задачу.

Назначение. Проверка возможности прекращения задачи на заданном этапе.

Аргументы события:

- _obj – прекращаемая задача;
- e.AddError() – добавление сообщения об ошибке. Выполняемые операции будут прерваны;

- e.AddInformation() – добавление информационного сообщения. Выполняемые операции не будут прерваны;
- e.AddWarning() – добавление текста с предупреждением. Выполняемые операции не будут прерваны.

  ПРИМЕЧАНИЕ. Сообщение об ошибке, информационное сообщение и предупреждение отображаются в верхней части формы карточки с соответствующей иконкой.
- e.IsValid – признак того, что задача валидная. Возможные значения:
  - **True**. Задача валидная (доступна для прекращения);
  - **False**. Задача не валидная. При прекращении задачи появится сообщение AddError.
- e.Params – дополнительные параметры.

Пример:

```
   // Восстановить полные права автора на документ при прекращении задачи.
public override void BeforeAbort(Sungero.Workflow.Server.BeforeAbortEventArgs
 e)
{
  base.BeforeAbort(e);
  Functions.ApprovalTask.RestoreAuthorAccessRights(_obj);
}
```

После приостановки (AfterSuspend)

Выполняется на веб-сервере после того, как задача была приостановлена из-за ошибки выполнения.

Назначение. Выполнение некоторой логики при возникновении ошибки. Например, отправка уведомления об ошибке.

Аргументы события:

- _obj – задача;
- e.Params – дополнительные параметры.

До выполнения (BeforeComplete)

Выполняется на веб-сервере перед тем, как выполнится задание.

Назначение:

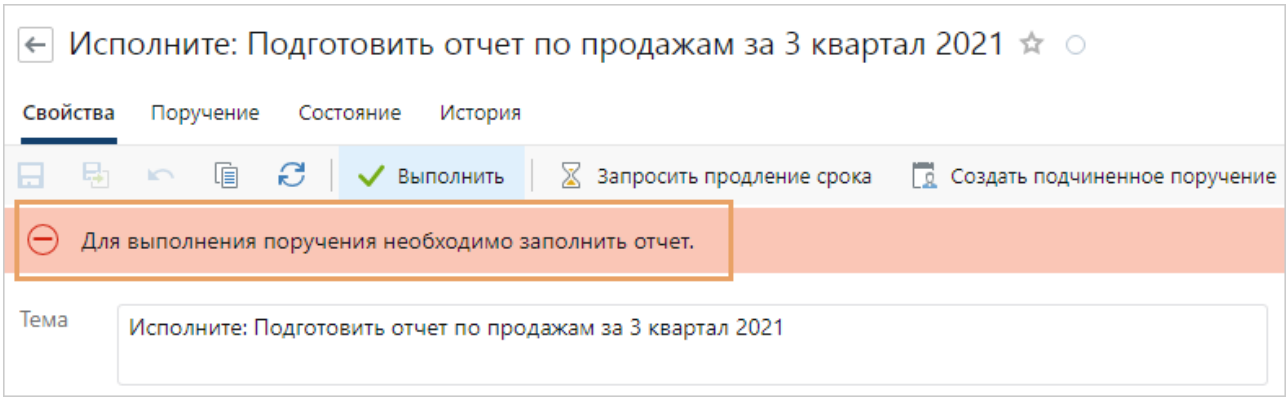
- *валидация* задания (проверка возможности выполнения, проверка заполнения полей);
- заполнение текста или свойств информацией из связанных сущностей или вложений.

Аргументы события:

- _obj – выполняемое задание;
- e.AddError() – добавление сообщения об ошибке. Выполняемые операции будут прерваны;
- e.AddInformation() – добавление информационного сообщения. Выполняемые операции не будут прерваны;

- e.AddWarning() – добавление текста с предупреждением. Выполняемые операции не будут прерваны.

  ПРИМЕЧАНИЕ. Сообщение об ошибке, информационное сообщение и предупреждение отображаются в верхней части формы карточки с соответствующей иконкой. Например, сообщение об ошибке выглядит следующим образом:



- e.IsValid – признак того, что задание валидное. Возможные значения:
  - **True**. Задание валидное (доступно для выполнения);
  - **False**. Задание не валидное. При выполнении задания появится сообщение AddError.
- e.Params – дополнительные параметры.

Пример:

```
   // Зафиксировать дополнительную информацию в тексте задания.
public override void
BeforeComplete(Sungero.Workflow.Server.BeforeCompleteEventArgs e)
{
  base.BeforeComplete(e);
  // Если задание согласовано и сформировано по этапу, в котором установлен
  // признак «Проверка полномочий», проверить полномочия сотрудника.
  // Если задание выполнено с превышением полномочий, добавить в текст задания
  // предупреждение об этом.
  if (_obj.Result == Result.Approved)
  {
    var stage = ApprovalStages.As(_obj.Stage);
    if (stage != null && stage.IsCheckAuthority == true)
    {
      var str =
Purchase
.PublicFunctions.GetInfoAboutAuthority(Sungero.Company.Employees.Current,
_obj.DocumentGroup.OfficialDocuments.FirstOrDefault());
      if (!string.IsNullOrEmpty(str))
        e.Result = Purchase.Resources.CheckAuthorityAssignmentFormat(e.Result,
str);
    }
  }
}
```

Подробнее см. пример разработки «Как зафиксировать дополнительную информацию о согласовании в тексте задания».

До принятия (BeforeAccept)

Выполняется на веб-сервере перед тем, как принять задание на приемку.

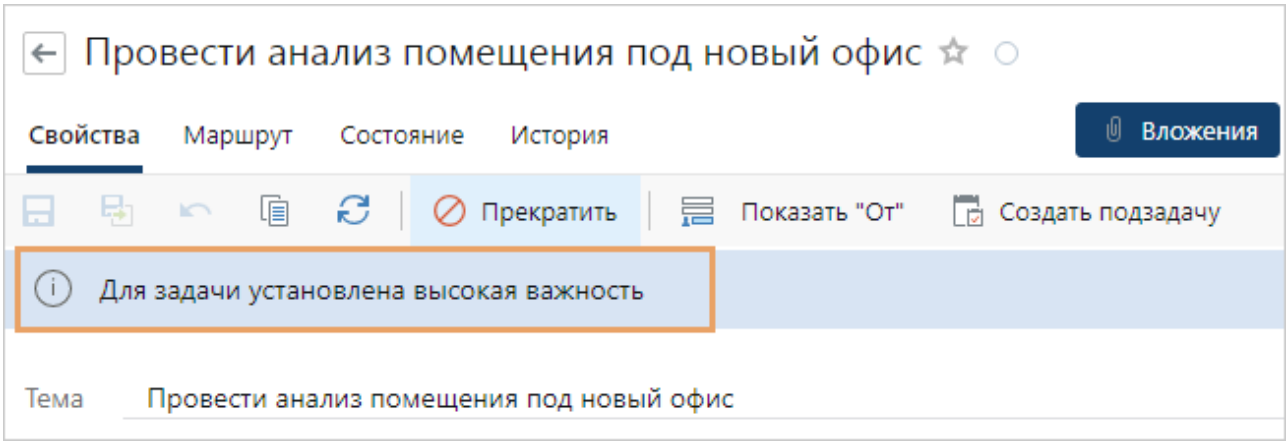
Назначение:

- *валидация* задания (проверка возможности выполнения, проверка заполнения полей);
- заполнение текста или свойств информацией из связанных сущностей или вложений.

Аргументы события:

- _obj – принимаемое задание на приемку;
- e.AddError() – добавление сообщения об ошибке. Выполняемые операции будут прерваны;
- e.AddInformation() – добавление информационного сообщения. Выполняемые операции не будут прерваны;
- e.AddWarning() – добавление текста с предупреждением. Выполняемые операции не будут прерваны.

  ПРИМЕЧАНИЕ. Сообщение об ошибке, информационное сообщение и предупреждение отображаются в верхней части формы карточки с соответствующей иконкой. Например, информационное сообщение выглядит следующим образом:



- e.IsValid – признак того, что задание на приемку валидное. Возможные значения:
  - **True**. Задание на приемку валидное (доступно для принятия);
  - **False**. Задание на приемку не валидное. При принятии задания на приемку появится сообщение AddError.
- e.Params – дополнительные параметры.

До отправки на доработку (BeforeSendForRework)

Выполняется на веб-сервере перед тем, как отправить задание на приемку на доработку.

Назначение:

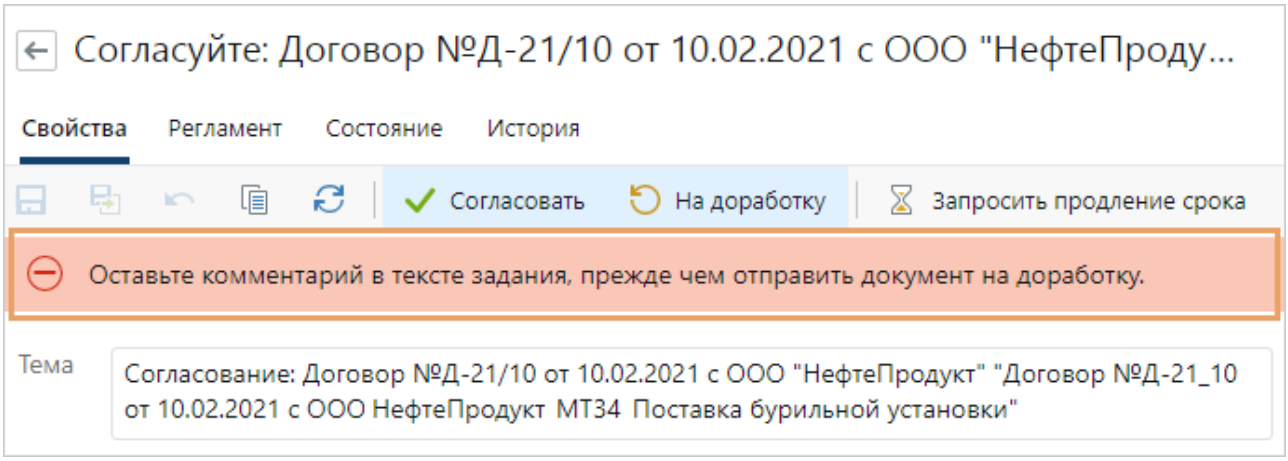
- *валидация* задания (проверка возможности выполнения, проверка заполнения полей);
- заполнение текста или свойств информацией из связанных сущностей или вложений.

Аргументы события:

- _obj – отправляемое на доработку задание на приемку;

- e.AddError() – добавление сообщения об ошибке. Выполняемые операции будут прерваны;
- e.AddInformation() – добавление информационного сообщения. Выполняемые операции не будут прерваны;
- e.AddWarning() – добавление текста с предупреждением. Выполняемые операции не будут прерваны.

  ПРИМЕЧАНИЕ. Сообщение об ошибке, информационное сообщение и предупреждение отображаются в верхней части формы карточки с соответствующей иконкой. Например, информационное сообщение выглядит следующим образом:



- e.IsValid – признак того, что задание на приемку валидное. Возможные значения:
  - **True**. Задание на приемку валидное (доступно для отправки на доработку);
  - **False**. Задание на приемку не валидное. При принятии задания на приемку появится сообщение AddError.
- e.Params – дополнительные параметры.

До взятия в работу (BeforeStartWork)

Выполняется перед тем, как конкурентное задание взято в работу.

Назначение:

- *валидация* задания (проверка возможности его выполнения, а также заполнения полей);
- заполнение текста или свойств информацией из связанных сущностей или вложений.

Особенности:

- работает при вызове действия **StartWork**;
- работает при вызове метода **StartWork()**;
- не работает при выполнении метода **Forward()**;
- работает до сохранения и изменения статусов.

Аргументы события:

- e.AddError() – добавление сообщения об ошибке. Выполняемые операции будут прерваны. Например, можно запретить взять задание в работу;

- e.AddInformation() – добавление информационного сообщения. Выполняемые операции не будут прерваны;
- e.AddWarning() – добавление текста с предупреждением. Выполняемые операции не будут прерваны.

  ПРИМЕЧАНИЕ. Сообщение об ошибке, информационное сообщение и предупреждение отображаются в верхней части формы карточки с соответствующей иконкой.
- e.Entity – сущность, для которой вызывается событие;
- e.IsExistWarnings – признак того, что в результате проверки сущности есть предупреждения;
- e.IsValid – признак того, что задание валидное. Возможные значения:
  - **True**. Задание валидное (доступно для выполнения);
  - **False**. Задание не валидное. При выполнении задания появится сообщение AddError;
- e.Params – дополнительные параметры.

Пример:

```
   // Если задание просрочено, при взятии в работу добавить к сроку 4 рабочих
      часа.
public override void
BeforeStartWork(Sungero.Workflow.Server.BeforeStartWorkEventArgs e)
{
  if (Docflow.PublicFunctions.Module.CheckDeadline(_obj.Performer,
_obj.Deadline, Calendar.Now))
  _obj.Deadline = Calendar.Now.AddWorkingHours(4);
}
```

До возврата невыполненным (BeforeReturnUncompleted)

Выполняется перед тем, как конкурентное задание вернут невыполненным.

Назначение:

- *валидация* задания (проверка возможности его выполнения, а также заполнения полей);
- заполнение текста или свойств информацией из связанных сущностей или вложений.

Особенности:

- работает при вызове действия **ReturnUncompleted**;
- работает при вызове метода **ReturnUncompleted()**;
- не работает при выполнении метода **Forward()**;
- работает до сохранения и изменения статусов.

Аргументы события:

- e.AddError() – добавление сообщения об ошибке. Выполняемые операции будут прерваны. Например, можно запретить возвращать конкурентное задание невыполненным;

- e.AddInformation() – добавление информационного сообщения. Выполняемые операции не будут прерваны;
- e.AddWarning() – добавление текста с предупреждением. Выполняемые операции не будут прерваны.

  ПРИМЕЧАНИЕ. Сообщение об ошибке, информационное сообщение и предупреждение отображаются в верхней части формы карточки с соответствующей иконкой.
- e.Entity – сущность, для которой вызывается событие;
- e.IsExistWarnings – признак того, что в результате проверки сущности есть предупреждения;
- e.IsValid – признак того, что задание валидное. Возможные значения:
  - **True**. Задание валидное (доступно для выполнения);
  - **False**. Задание не валидное. При выполнении задания появится сообщение AddError;
- e.Params – дополнительные параметры.

Пример:

```
   public override void
      ReturnUncompleted(Sungero.Domain.Client.ExecuteActionArgs e)
{
// Получить все проекты резолюции, добавленные пользователем в рамках
задания.
  var allActionItems = _obj.ResolutionGroup.ActionItemExecutionTasks.ToList();
  var actionItemsToDelete =
Functions.Module.Remote.GetActionItemsAddedToAssignment(_obj, allActionItems,
 Company.Employees.Current,
  PublicConstants.PreparingDraftResolutionAssignment.ResolutionGroupName);
  var hasActionItemsToDelete = actionItemsToDelete.Any();

// Диалог подтверждения. Если есть хотя бы один проект, сообщить об этом.
// Проекты резолюции удаляются в событии "До возврата невыполненным".
  var description = hasActionItemsToDelete ?
Resources.ConfirmDeleteDraftResolutionAssignment : null;
  var dropDialogId = hasActionItemsToDelete
  ?
Constants
.DocumentReviewTask.DocumentReviewAssignmentConfirmDialogID.ReturnUncompleted
  :
Constants
.DocumentReviewTask
.DocumentReviewAssignmentConfirmDialogID
.ReturnUncompletedWithDeletingDraftResolutions;
var dropIsConfirmed =
Docflow
.PublicFunctions
.Module
.ShowConfirmationDialog
(e.Action.ConfirmationMessage,description,null,dropDialogId);
if (!dropIsConfirmed)
  {
  e.CloseFormAfterAction = false;
```

```
  return;
  }

base.ReturnUncompleted(e);
}
```

UI-фильтрация (UIFiltering)

Выполняется только на веб-сервере после наложения всех остальных фильтров.

Назначение. Событие используют для настройки видимости оргструктуры. В обработчике события прикладной разработчик пишет код, который накладывает дополнительные фильтры на запросы выборки данных.

UI-фильтрация работает только в веб-клиенте. Событие выполняется, когда пользователи просматривают списки справочников и заполняют контролы, связанные со ссылками на справочники. При этом контролы могут быть:

- в карточках;
- на панелях фильтрации списков и папок;
- в диалогах поиска;
- в диалогах с выпадающими списками с выбором одного и нескольких значений.

ВАЖНО. Событие не выполняется:

- в диалогах, если в выпадающем списке или выпадающем списке с выбором нескольких значений явно указан источник данных либо указан список допустимых значений;
- в вычисляемых папках и списках, если их содержимое формируется с помощью IQueryable или IList.

Возвращаемое значение. Измененный запрос на получение списка сущностей. По умолчанию возвращается исходный запрос.

Аргументы

- query – исходный запрос на получение списка сущностей;
- e.Params – дополнительные параметры.

Точечное отключение выполнения обработчика. Иногда записи одного и того же справочника используются для заполнения контролов в разных местах, например, на форме типа задачи и на форме типа документа. При этом нужно, чтобы при заполнении контрола на форме документа событие UI-фильтрации срабатывало, а при заполнении контрола на форме задачи – нет, то есть показывались все записи из справочника. В этой ситуации для контрола на форме задачи выполнение события можно точечно отключить с помощью специального атрибута.

Чтобы событие не выполнялось при заполнении контрола в карточке типа сущности, в код обработчика события Фильтрация выбора из списка для связанного с контролом свойства-ссылки добавьте атрибут `e.DisableUiFiltering = true`. Например:

```
public virtual IQueryable<T> OurSignatoryFiltering(IQueryable<T> query,
    Sungero.Domain.PropertyFilteringEventArgs e)
```

```
{
  e.DisableUiFiltering = true;
  return query;
}
```

Используйте этот же атрибут, чтобы отменить выполнение события UI-фильтрации при заполнении контролов:

- на панели фильтрации в списках. Для этого добавьте атрибут в код обработчика события Фильтрация сущностей в редакторе свойства-ссылки;
- в диалоге поиска. Для этого добавьте атрибут в код обработчика события Фильтрация выбора из списка при поиске в редакторе свойства-ссылки.

Чтобы событие UI-фильтрации не выполнялось при заполнении контрола на панели фильтрации вычисляемой папки, в обработчик события Фильтрации сущностей добавьте атрибут: `_disableUiFiltering = true`.

В *базовом решении* Directum RX обработчики для событий UI-фильтрации добавлены в справочниках **Сотрудник**, **Подразделение** и **Наша организация**. Список мест, где выполнение обработчика отключено, см. в руководстве администратора, в разделе «Списки, закладки и поля, содержимое которых формируется по бизнес-правилам». См. также пример «Как ограничить видимость сотрудников в списках».

При *наследовании* и *перекрытии* справочников разработчик может:

- вызвать событие UI-фильтрации из родительского типа сущности;
- написать свою логику фильтрации;
- отключить UI-фильтрацию в полях и справочниках, где в стандартной версии она по умолчанию работает.

Пример:

```
   // В выпадающем списке отображать три записи из справочника
      «Пользователи».
// С помощью метода From() указан список допустимых значений.
var user = dialog.AddSelect("Ответственный", true,
Sungero.CoreEntities.Users.Null).From(user1, user2, user3);

/// <summary>
/// Поиск по дате регистрации.
/// </summary>
/// <returns>Список документов.</returns>
public virtual IQueryable<IOfficialDocument> SearchByRegistrationDate()
{
  // Создать диалог ввода даты регистрации.
  var dialog =
Dialogs.CreateInputDialog(Resources.SearchFromRegistrationData);
  var registrationDateFrom = dialog.AddDate(Resources.RegistrationDateFrom,
false);
  var registrationDateTo = dialog.AddDate(Resources.RegistrationDateTo,
false);
  if (dialog.Show() == DialogButtons.Ok)
    // Функция возвращает список документов (IQueryable),
    // к которому не применяется событие UI-фильтрации.
```

```
    return
Functions.Module.Remote.GetRegisteredDocuments( registrationDateFrom.Value,
registrationDateTo.Value);
  return null;
}
```

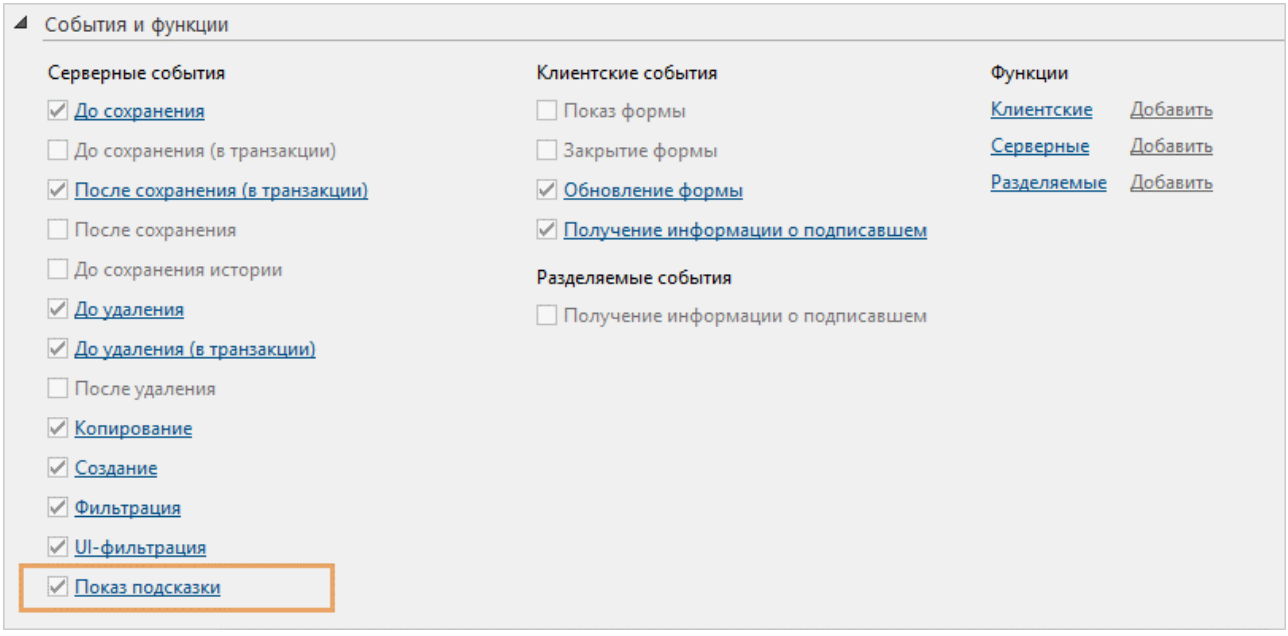
Показ подсказки (GetDigest)

Выполняется, когда в проводнике веб-клиента пользователь наводит курсор на фото или имя сотрудника:

- в списке или папке;
- в поле **Кому** в карточке задачи;
- в переписке по задаче, заданию или уведомлению;
- в свойствах-ссылках;
- в истории работы с сущностью;
- в окне о блокировке содержимого документа другим сотрудником, а также в сообщении о блокировке карточки документа.

Назначение. Показ всплывающей подсказки с информацией о сотруднике.

Событие доступно только для типа сущности **Пользователь** (Sungero.CoreEntities.User) и его наследников, например, **Сотрудник** (Employee):



Возвращаемое значение. Сформированная подсказка с информацией о сотруднике.

Аргументы события: e.Entity – сущность, для которой вызывается событие.

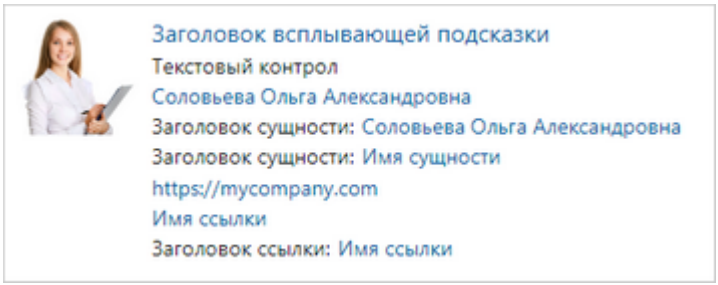
В *базовом решении* Directum RX обработчик события **Показ подсказки** задан для типа справочника **Сотрудники** (Employee). В нем вызывается функция **GetEmployeePopup()**, в которой формируется подсказка. Чтобы изменить подсказку, достаточно переопределить функцию либо написать свой код для наполнения содержимого.

Чтобы создать подсказку с нуля либо изменить существующую, используйте класс UserDigest, а также методы и свойства его экземпляра **UserDigest.Create()**. В подсказку можно добавить заголовок, текстовый контрол, ссылки на сущности или внешние ресурсы.

Пример:

```
   // Получить подсказку о сотруднике.
      public override IDigestModel GetDigest(Sungero.Domain.GetDigestEventArgs
      e)
{
  var digest = UserDigest.Create(_obj);
  digest.Header = "Заголовок всплывающей подсказки";
  digest.AddLabel("Текстовый контрол");
  // Ссылка на сущность.
  digest.AddEntity(_obj);
  // Ссылка на сущность с заголовком.
  digest.AddEntity(_obj, "Заголовок сущности:");
  // Ссылка на сущность с заголовком. Имя сущности задается в коде.
  digest.AddEntity(_obj, "Заголовок сущности:", "Имя сущности");
  // Ссылка с полным адресом.
  digest.AddHyperlink("https://mycompany.com");
  // Ссылка, имя которой задается в коде.
  digest.AddHyperlink("https://mycompany.com", "Имя ссылки");
  // Ссылка с заголовком. Имя ссылки задается в коде.
  digest.AddHyperlink("https://mycompany.com", "Имя ссылки", "Заголовок:");
  return digest;
}
```

В результате подсказка примет вид:



Клиентские события

События формы сущности и другие задаются в редакторе типа сущности в группе «Клиентские события». Выполняются в клиентском приложении.

Клиентские события	
События, которые есть у всех типов сущностей	
Показ формы	Showing
Закрытие формы	Closing
Обновление формы	Refresh
События только для типов документов	
До показа диалога подписания	ShowingSignDialog

Показ формы (Showing)

Выполняется перед показом формы карточки сущности.

Назначение. Задание первоначальных настроек формы.

ВАЖНО. В обработчике события не рекомендуется вызывать серверные функции (с атрибутом **Remote**) и использовать логику, которая будет долго обрабатываться. Данный обработчик выполняется часто. Поэтому, если в нем есть серверные функции, то на веб-сервер отправляется большое количество запросов, что, в свою очередь, влияет на быстродействие системы.

Аргументы события:

- _obj – сущность, карточка которой открывается;
- e.AddError() – добавление сообщения об ошибке. Выполняемые операции не будут прерваны, их нужно прекратить вручную. Для этого в коде после валидации необходимо проверить состояние сущности с помощью свойства **IsValid** или метода **Validate()**;
- e.AddInformation() – добавление информационного сообщения. Выполняемые операции не будут прерваны;
- e.AddWarning() – добавление предупреждения. Выполняемые операции не будут прерваны;
- e.Instruction – инструкция к сущности. С помощью аргумента можно:
  - полностью заменить текст инструкции;
  - добавить текст в конец инструкции;
  - отключить отображение инструкции в веб-клиенте. Для этого задайте в свойстве значение **null**, пробелы или пустую строку;
  - добавить ссылку на сущность, например на документ;
- e.IsValid – признак того, что сущность валидная. Возможные значения:
  - **True**. Сущность валидная (можно открыть форму карточки сущности);
  - **False**. Сущность не валидная. При открытии формы появится сообщение об ошибке, заданное в **AddError**;
- e.Params – дополнительные параметры;
- e.Title – заголовок открываемой формы;
- e.HideAction(<информация о действии>) – скрытие действия или результата выполнения на ленте в карточке сущности. Например, метод e.HideAction(_obj.Info.Actions.DeleteEntity) скроет действие удаления сущности на ленте. Подробнее см. «Как в задании скрыть результат выполнения».

Пример 1:

```
   public override void Showing(Sungero.Presentation.FormShowingEventArgs e)
{
  // Не показывать свойство Department, если карточка справочника
"Сотрудники"
  // была открыта из карточки справочника "Подразделения".
  base.Showing(e);
  if (CallContext.CalledFrom(Departments.Info))
    _obj.State.Properties.Department.IsVisible = false;
}
```

Пример 2:

```
   public override void Showing(Sungero.Presentation.FormShowingEventArgs e)
```

```
{
  // Добавить текст в конец инструкции.
  if (e.Instruction != null)
    e.Instruction += "Экземпляр согласованного договора передайте
контрагенту.";
}
```

Пример 3:

```
   public override void Showing(Sungero.Presentation.FormShowingEventArgs e)
{
  // Отключить отображение инструкции.
  e.Instruction = null;
}
```

Пример 4:

```
   public override void Showing(Sungero.Presentation.FormShowingEventArgs e)
{
  // Вставить ссылку на документ в текст инструкции.
  var hyperlink = Hyperlinks.Get(Sungero.Docflow.OfficialDocuments, 334);
  e.Instruction = string.Format("При согласовании документа учитывайте
требования {0}", hyperlink);
}
```

Закрытие формы (Closing)

Выполняется перед закрытием формы карточки сущности.

Назначение. Задание дополнительной логики.

Аргументы события:

- _obj – сущность, карточка которой закрывается;
- Params – дополнительные параметры.

Пример:

```
   // При закрытии карточки входящего счета сделать необязательными для
      заполнения
// свойства Номер счета, Дата счета, Сумма и Валюта.
public override void Closing(Sungero.Presentation.FormClosingEventArgs e)
{
  base.Closing(e);

  _obj.State.Properties.Number.IsRequired = false;
  _obj.State.Properties.Date.IsRequired = false;
  _obj.State.Properties.TotalAmount.IsRequired = false;
  _obj.State.Properties.Currency.IsRequired = false;
}
```

Обновление формы (Refresh)

Вызывается при открытии и сохранении карточки сущности, выполнении обработчика клиентского события или действия на ленте, в котором идет обращение к серверной функции.

Назначение:

- задание логики для обновления видимости и доступности свойств на форме;
- *валидация* сущности после выполнения серверной функции.

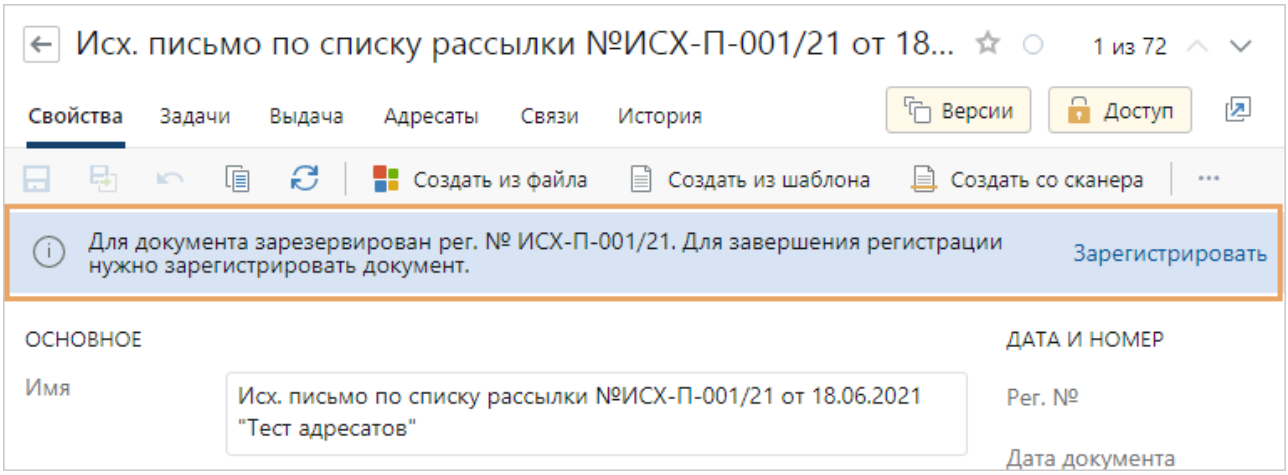
ВАЖНО. В обработчике события не рекомендуется вызывать серверные функции (с атрибутом **Remote**) и использовать логику, которая будет долго обрабатываться. Данный обработчик выполняется часто. Поэтому, если в нем есть серверные функции, на веб-сервер отправляется большое количество запросов, что, в свою очередь, влияет на быстродействие системы.

Не рекомендуется добавлять обязательное свойство в тип сущности, если есть заполненные карточки ее экземпляров. Например, если в карточку договора требуется добавить свойство **Сумма с НДС** и уже есть ранее заполненные карточки договоров, то в редакторе типа документа это свойство нужно сделать необязательным. Иначе код в событии «Обновлении формы» не отработает при невизуальных вычислениях. Например, при программной выдаче прав, согласовании документа или интеграции с другими системами возникнет ошибка.

Аргументы события:

- _obj – сущность, которая будет обновлена;
- e.AddError() – добавление сообщения об ошибке. Выполняемые операции не будут прерваны, их нужно прекратить вручную. Для этого в коде после валидации необходимо проверить состояние сущности с помощью свойства **IsValid** или метода **Validate()**;
- e.AddInformation() – добавление информационного сообщения. Выполняемые операции не будут прерваны;
- e.AddWarning() – добавление предупреждения. Выполняемые операции не будут прерваны.

  ПРИМЕЧАНИЕ. Сообщение об ошибке, информационное сообщение и предупреждение отображаются с соответствующей иконкой в верхней части карточки сущности. Например, информационное сообщение выглядит следующим образом:



- e.IsValid – признак того, что сущность валидная. Возможные значения:
  - **True**. Сущность валидная (доступна для обновления);
  - **False**. Сущность не валидная. При сохранении сущности появится сообщение об ошибке, заданное в AddError.

Например, карточка документа обновится после резервирования номера по кнопке **Присвоить номер** на ленте. Если номер зарезервирован успешно, то карточка документа обновится и в ней появится информационное сообщение о том, что номер зарезервирован.

- e.Params – дополнительные параметры;
- e.Title – заголовок формы после обновления. Например, после регистрации документа в заголовок добавится его регистрационный номер.

Пример 1:

```
    public override void Refresh(Sungero.Presentation.FormRefreshEventArgs e)
{
  // Изменить заголовок окна карточки в соответствии с именем документа.
  e.Title = _obj.Name == Docflow.Resources.DocumentNameAutotext ? null :
_obj.Name;
}
```

Пример 2:

```
    // Если повестка совещания ранее была создана из карточки совещания,
// то в карточке документа поле "Совещание" должно быть недоступно для
изменения.
public override void Refresh(Sungero.Presentation.FormRefreshEventArgs e)
{
  var isCalledFromMeeting = CallContext.CalledFrom(Meetings.Info);
  if (isCalledFromMeeting)
    _obj.State.Properties.Meeting.IsEnabled = false;

  base.Refresh(e);
}
```

До показа диалога подписания (ShowingSignDialog)

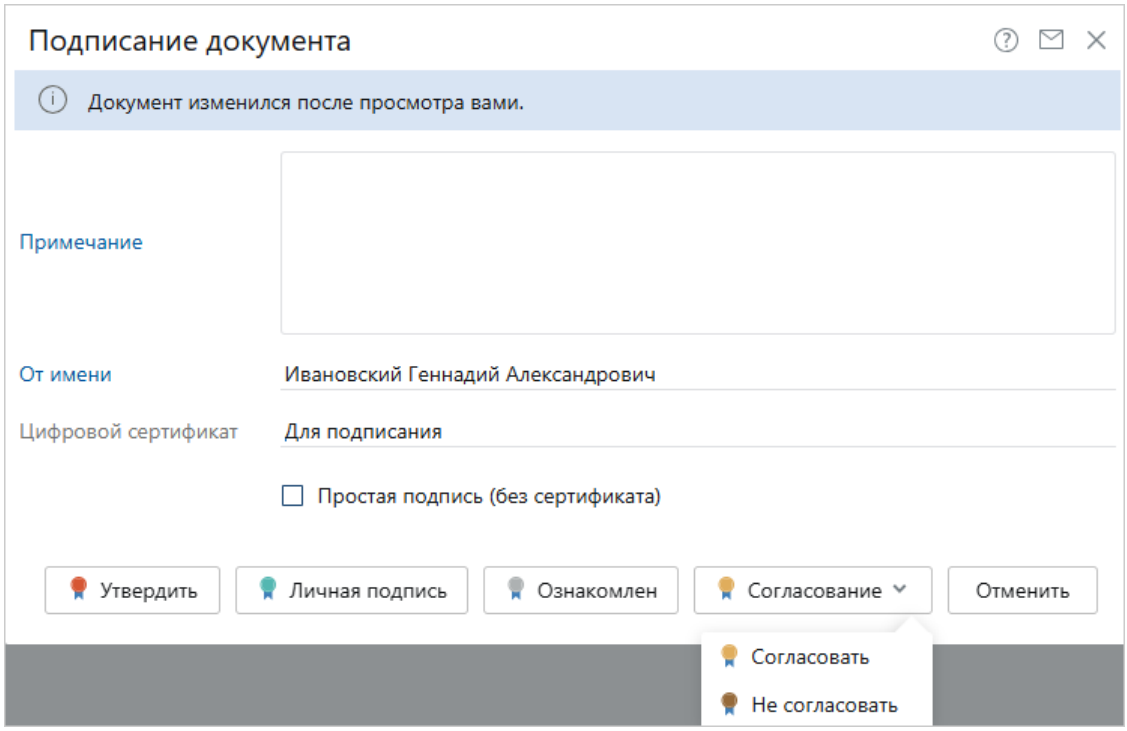
Выполняется на стороне клиентского приложения при подписании документа.

Назначение. Настройка диалога подписания.

Аргументы события:

- e.CanAcquaint – признак того, что в диалоге можно поставить подпись с результатом **Ознакомлен**;
- e.CanApprove – признак того, что в диалоге можно поставить подпись с результатом **Утвердить**;
- e.CanEndorse – признак того, что в диалоге можно поставить подпись с результатом **Согласовать** или **Не согласовать**;
- e.CanPersonalSign – признак того, что в диалоге можно поставить подпись с результатом **Личная подпись**;

- e.Hint – подсказка в диалоге подписания. Используется, например, для предупреждения о том, что содержимое документа изменилось после последнего просмотра.



Пример:

```
   // До показа диалога подписания проверить ошибки валидации.
public override void
ShowingSignDialog(Sungero.Domain.Client.ShowingSignDialogEventArgs e)
{
  var errors =
Functions.OfficialDocument.Remote.GetApprovalValidationErrors(_obj, true);
  foreach (var error in errors)
    // Отобразить подсказку, если есть ошибки.
    e.Hint.Add(error);
  // Если нет ошибок, будет разрешена утверждающая подпись.
  e.CanApprove = !errors.Any();
}
```

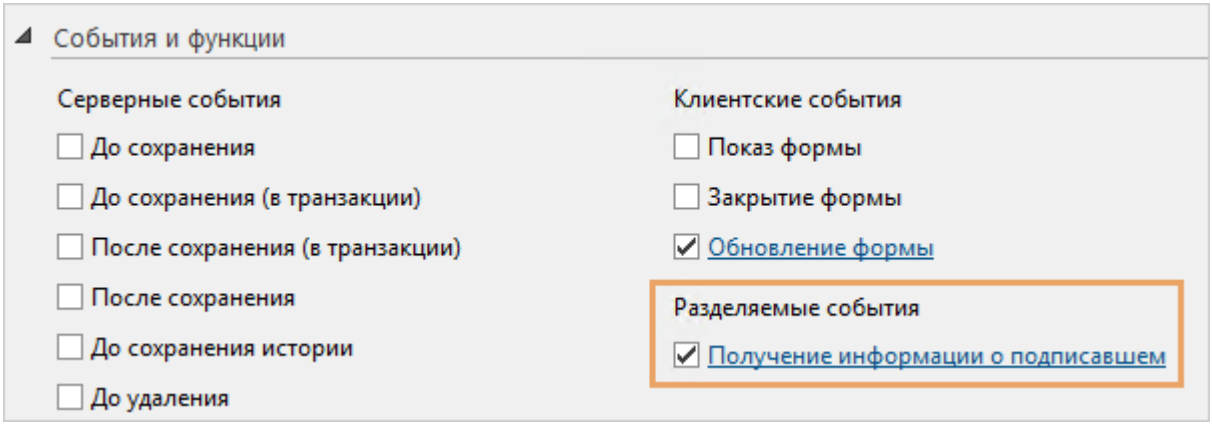
Разделяемые события

У наследников Sungero.CoreEntities.User помимо основных серверных и клиентских событий есть разделяемое событие «Получение информации о подписавшем».

Получение информации о подписавшем (SignatoryInfoObtaining)

Выполняется, когда пользователь подписывает документ.

Назначение. Событие используется, чтобы при подписании документа сохранить информацию о подписавшем в классе ISignature.AdditionalInfo. Полученную информацию можно использовать, например, для отображения в отчете.



ПРИМЕЧАНИЕ. В предыдущих версиях Directum RX событие было клиентским. Если тогда для него был написан обработчик, то в редакторе типа справочника событие по-прежнему отображается в группе «Клиентские события». Например, оно является клиентским в *базовом решении* Directum RX в справочнике **Сотрудники** (Employee). Такое поведение оставлено для совместимости кода. После перекрытия справочника событие появится в группе «Разделяемые события» и его можно будет переопределить.

Аргументы события:

- _obj – пользователь, для которого вызывается событие;
- e.FullName – полное имя пользователя. Автоматически заполняется из свойства **Name** сущности;
- e.AdditionalInfo – дополнительная информация о пользователе. Может содержать любую текстовую информацию. Например, должность пользователя. Эта информация нигде не отображается, но ее можно получить через интерфейс подписи **ISignature**.

Возвращаемое значение: нет. Информация о подписавшем записывается в аргументы **e.FullName** и **e.AdditionalInfo**.

Пример обработчика события для справочника «Сотрудники» (Employee):

```
   public                                override                                void
      SignatoryInfoObtaining(Sungero.Domain.Client.SignatoryInfoObtainingEventArgs
       e)
{
  // Получить должность сотрудника.
  var result = _obj.JobTitle != null ? _obj.JobTitle.Name.Trim() : string.Empty;
  // Если подразделение не пустое, то выполнить дальнейшие действия.
  if (_obj.Department != null)
  {
  // Получить подразделение сотрудника.
    var department = string.IsNullOrEmpty(_obj.Department.ShortName) ?
_obj.Department.Name : _obj.Department.ShortName;
    // Если должность не заполнена, то в результат записать только подразделение.
    if (string.IsNullOrEmpty(result))
      result = department;
    else
    // Иначе в результат записать должность и подразделение.
      result = string.Format("{0} ({1})", result, department);
```

```
  }
  // Результат записать в дополнительную информацию о пользователе
  // и добавить разделитель |.
  e.AdditionalInfo = result + "|";
}
```

Параметр Params

Для обработчиков событий и действий типов сущностей доступен параметр **Params**. Он предназначен для передачи дополнительных параметров между обработчиками. Например, при открытии задачи на исполнение поручения проверяется, разрешены ли в системе поручения без срока. Результат записывается в параметр. При выполнении обработчика события **Обновление формы** проверка повторно уже не выполняется. Используется ранее вычисленное значение параметра.

Значения параметра не сохраняются в базу данных и существуют только в рамках отдельной сущности.

ВАЖНО. При нажатии на кнопку **Отмена текущих изменений** (Cancel) значения всех параметров очищаются. Чтобы этого избежать, воспользуйтесь одним из способов:

- в событии Обновление формы (Refresh) проверьте и восстановите значения параметров с помощью методов **e.Params.Contains()** и **e.Params.AddOrUpdate()**;
- если параметр сущности был передан извне или требуются длительные вычисления параметра (например, вызов Remote-функций), то перекройте действие **Cancel** и до вызова базового обработчика (base.Cancel) сохраните значение параметра, а затем восстановите его.

Для работы с параметром **Params** используйте следующие методы:

Метод	Описание
Add()	Создание параметра. Если параметр уже создан, то при повторном вызове метода **Add()** появится сообщение об ошибке. `e.Params.Add("<Имя параметра>", <Значение параметра>);`
AddOrUpdate()	Создание параметра. Если параметр уже создан, то его значение будет перезаписано. `e.Params.AddOrUpdate("<Имя параметра>", <Значение параметра>);`

TryGetValue()	Проверка наличия параметра и получение значения.
	`<Тип значения> <Имя переменной>;`
	`e.Params.TryGetValue("<Имя параметра>", out <Имя переменной>);`

Contains()	Проверка наличия параметра. `e.Params.Contains("<Имя параметра>");`
Remove()	Удаление параметра. `e.Params.Remove("<Имя параметра>");`

Для параметров доступны значения только следующих типов:

- простые типы:

short, short?	Guid, Guid?	double, double?
int, int?	bool, bool?	Uri

short, short?	Guid, Guid?	double, double?
long, long?	decimal, decimal?	string
char, char?	DateTime, DateTime?	

- перечисления: Enumeration, Enumeration?;
- обобщенные списки и словари указанных типов (List<>, Dictionary<>).

Прочие типы, в том числе сущности, нельзя сохранять в **Params**.

Отключение событий сущностей и свойств

События типов сущностей и свойств сущностей можно программно отключить. Это позволяет оптимизировать вычисления при обработке большого количества объектов. К примеру, можно массово заполнить поля в документах и при этом отключить автозаполнение связанных свойств.

Какие события можно отключать	Какие события нельзя отключать
• создание типа сущности (Created); • сохранение типа сущности (BeforeSave, Saving, Saved, AfterSave); • удаление типа сущности (BeforeDelete, Deleting, AfterDelete); • до подписания (BeforeSigning); • до сохранения истории (BeforeSaveHistory); • изменение значения свойства (Changed); • изменение, добавление и удаление свойств дочерней коллекции (Changed, Added, Deleted)	• показ, закрытие и обновление формы (Showing, Closing, Refresh); • изменение значения контрола (ValueInput); • серверные события типа задачи (BeforeStart, BeforeRestart, BeforeResume, BeforeAbort, AfterSuspend); • до выполнения задания (BeforeComplete); • фильтрация всех типов сущностей (Filtering); • UI-фильтрация справочника (UIFiltering); • смена типа документа (ConvertingFrom)

Для отключения событий типов сущностей используется метод **EntityEvents.Disable()**. Например, чтобы создать документ без выполнения событий «До сохранения» и «После сохранения», укажите их при вызове метода:

```
   public static void CreateDocNoEvents(Sungero.Domain.Client.ExecuteActionArgs e)
{
  using (EntityEvents.Disable(AgendaDocuments.Info.Events.BeforeSave,
AgendaDocuments.Info.Events.AfterSave))
  {
    var doc = AgendaDocuments.Create();
    doc.Name = "Протокол совещания" + Calendar.Now;
    doc.Save();
  }
}
```

С помощью метода **EntityEvents.DisableAll()** можно отключить сразу все события сущности:

```
   using (EntityEvents.DisableAll(AgendaDocuments.Info))
{
  _obj.Author = Users.Get(1);
}
```

События свойств сущности отключаются аналогично. Далее в первом примере кода отключается событие «Изменение значения свойства» свойства **Name**, во втором –события всех свойств сущности **AgendaDocument**:

```
    using
        (EntityEvents.Disable(AgendaDocuments.Info.Properties.Name.Events.Changed))
using (EntityEvents.DisableAll(AgendaDocuments.Info.Properties))
```

Разработка справочников

На уровне платформы к базовым типам справочников относятся:

- DatabookEntry – справочник. Большинство справочников в системе создано на его основе. Примеры наследников: Населенные пункты, Журналы регистрации;
- User – пользователь системы. Справочник и его наследники используются для назначения прав доступа. Пример наследника: Сотрудники;
- Group – группа пользователей системы. Группа включает в себя пользователей и другие группы. Справочник и его наследники используются для назначения прав доступа. Примеры наследников: Подразделения, Группы регистрации.

Особенности разработки

Основные действия при создании типа справочника см. в разделе «Типы сущностей».

В редакторе типа справочника задаются:

- основные параметры;
- свойства, которые выносятся на форму;
- действия, которые добавляются на ленту;
- списки типов сущностей;
- стили отображения записей.

Рекомендации по выбору наименования для справочника:

- указывайте имя в единственном числе;
- не используйте транслитерацию. Например, используйте «Department» вместо «Otdel»;
- ориентируйтесь на названия типов сущностей в *базовом решении*. Например, если на форму добавляется свойство-ссылка на справочник «Контрагенты» (Company), то свойство лучше назвать «Company», а не «Organization».

От каких типов справочников можно создавать наследников

CoreEntities – администрирование:

- CounterpartyDepartmentBox – абонентский ящик подразделения контрагента
- DatabookEntry – справочник

ExchangeCore – электронный обмен. Настройки:

- BoxBase – абонентский ящик
- PowerOfAttorneyQueueItem – элемент очереди синхронизации эл. доверенностей

- Group – группа пользователей системы
- User – пользователь системы

Company – компания:

- UserBase – пользователь

Docflow – документооборот:

- ApprovalRoleBase – роль согласования
- ApprovalRuleBase – правило согласования
- ConditionBase – условие
- DocumentGroupBase – группа документа
- ProjectBase – проект
- StoragePolicyBase – политика хранения
- QueueItemBase – очередь электронного обмена

Parties – контрагенты:

- CompanyBase – организация
- Counterparty – контрагент

Projects – проекты:

- ProjectQueueItemBase – элемент очереди выдачи прав модуля «Проекты»

Какие типы справочников можно перекрывать

Commons – общие справочники:

- City – населенный пункт
- Country – страна
- Currency – валюта
- ClassifierTrainingSession – сессия дообучения классификатора)
- EntityRecognitionInfo – результаты распознавания сущности
- ExternalEntityLink – ссылка внешней системы
- Region – регион
- VatRate – ставка НДС

Contracts – договорные документы:

- ContractApprovalRole – роль согласования
- ContractCategory – категория договора
- ContractCondition – условие согласования

Company – компания:

- BusinessUnit – наша организация
- Department – подразделение
- Employee – сотрудник
- JobTitle – должность
- ManagersAssistant – ассистент руководителя
- VisibilityRule – правило видимости оргструктуры
- VisibilitySetting – настройки видимости организационной структуры

Docflow – документооборот:

- AccessRightsRule – правило назначения прав
- ApprovalRole – роль согласования
- ApprovalRule – правило согласования
- ApprovalStage – этап согласования
- CaptureSource – источник поступления документов

- ContractsApprovalRule – правила согласования

Exchange – электронный обмен:

- ExchangeDocumentInfo – информация о документе обмена

ExchangeCore – электронный обмен. Настройки:

- BodyConverterQueueItem – элемент очереди конвертации тел документов
- BusinessUnitBox – абонентский ящик нашей организации
- CounterpartyDepartmentBox – абонентский ящик подразделения контрагента
- CounterpartyQueueItem – элемент очереди синхронизации контрагентов
- DepartmentBox – абонентский ящик подразделения
- ExchangeService – сервис обмена
- MessageQueueItem – элемент очереди синхронизации сообщений
- HistoricalMessagesDownloadSession – сессия загрузки исторических сообщений
- MessagesFilteringSetting – настройка загрузки сообщений из сервиса обмена
- ExchangeDocumentType – тип документа в сервисе обмена

Parties – контрагенты:

- Bank – банк
- Company – организация
- Contact – контакт
- CounterpartyKind – вид контрагента
- DueDiligenceWebsite – сайт проверки контрагента
- Person – персона
- CaseFile – дело
- Condition – условие
- DistributionList – список рассылки
- DocumentKind – вид документа
- DocumentRegister – журнал регистрации
- DocumentSendAction – справочник
- DocumentType – тип документа
- FileRetentionPeriod – срок хранения дела
- MailDeliveryMethod – способ доставки документов
- MarkKind – вид отметки
- MediumType – вид носителя документа
- PowerOfAttorneyQueueItem – элемент очереди синхронизации эл. доверенностей
- PersonalSettings – персональные настройки
- RegistrationGroup – группа регистрации
- RegistrationSetting – настройка регистрации
- RetentionPolicy – политика перемещения
- SignatureSetting – право подписи
- SmartProcessingSetting – настройки интеллектуальной обработки
- StoragePolicy – политика хранения
- Topic – рубрика

Projects – проекты:

- ProjectCore – проект
- Project – проект
- ProjectApprovalRole – роль согласования
- ProjectDocumentRightsQueueItem – элемент очереди выдачи прав на проектные документы
- ProjectKind – вид проекта
- ProjectMemberRightsQueueItem – элемент очереди выдачи прав участникам проекта
- ProjectRightsQueueItem – элемент очереди выдачи прав на проект и папки проекта
- ProjectTeam – команда проекта

RecordManagement – делопроизводственные документы:

- AcquaintanceList – список ознакомления
- AcquaintanceTaskParticipant – участники задачи на ознакомление

SmartProcessing – интеллектуальная обработка:

- Blob – бинарный образ документа
- BlobPackage – пакет бинарных образов документов
- RepackingSession – сессия перекомплектования

Meetings – документы по совещаниям:

- Meeting – совещание

MobileApps – мобильные приложения:

- MobileAppSetting – настройка мобильных приложений

События типов справочников

Для типа справочника кроме основных серверных и клиентских событий дополнительно доступны события:

Серверные события:

- UI-фильтрация
- Показ подсказки (только для наследников Sungero.CoreEntities.User)

Разделяемые события:

- Получение информации о подписавшем

Также в каждом типе сущности можно задать события свойств и действий.

Объектная модель для работы с типами справочников

- Основные механизмы
- CoreEntities – администрирование
- Базовое решение Directum RX
- Commons – общие справочники

Примеры разработки

Используйте примеры, которые помогут изучить возможности среды разработки и подскажут идеи для развития модификации системы:

- Доступность, обязательность, видимость свойств

- Папки и списки

Наследуемые операции авторизации

От базовых типов справочников наследуются следующие операции авторизации:

Операция	Описание
Read	Чтение
Create	Создание
Update	Изменение
Delete	Удаление
Manage	Изменение прав доступа
DelegateAccess	Передача прав доступа

Наследуемые действия

От базовых типов справочников Sungero.CoreEntities.User, Sungero.CoreEntities.Group, Sungero.CoreEntities.DatabookEntry наследуются:

Действие	Описание	Где доступно?
CopyEntity	Создать запись справочника копированием и открыть карточку новой записи	Карточка и список
DeleteEntity	Удалить запись справочника без возможности восстановления	Карточка и список
Save	Сохранить изменения в карточке	Карточка
SaveAndClose	Сохранить изменения и закрыть карточку	Карточка
SelectAndClose	Сохранить изменения, выбрать запись справочника и закрыть карточку. Например, действие **Выбрать и закрыть** на ленте в карточке справочника **Контакты** при создании контакта из карточки входящего документа	Карточка
Cancel	Отменить текущие изменения в карточке	Карточка
ShowSelectEntity	Открыть карточку записи справочника из списка	Список
AddChildEntity	Добавить новую строку в текущую коллекцию	Дочерняя коллекция
DeleteChildEntity	Удалить строку из текущей коллекции	Дочерняя коллекция
CopyChildEntity	Копировать строку и добавить новую строку в текущую коллекцию	Дочерняя коллекция
AddToFavoritesFolder	Копировать ссылку на запись справочника в папку «Избранное»	Карточка и список
AddToMyFolder	Копировать ссылку на запись справочника в **Мой Directum**	Карточка и список

Действие	Описание	Где доступно?
SearchFoldersContainsLinkToEntity	Найти папки, в которые вложена ссылка на запись справочника	Карточка и список
CopyToClipboard	Копировать в буфер обмена ссылку на запись справочника, выделенную в списке	Список
CopyToClipboardFromCard	Копировать в буфер обмена ссылку на запись справочника, в карточке которой вызвано действие	Карточка
ChangeSearchCriteria	Изменить критерии поиска	Список
ChooseEntityAndCloseCollection	Выбрать записи справочника для свойства-ссылки и закрыть список записей. Например, действие **Выбрать запись** на ленте в списке записей справочника **Регионы** при создании записи справочника **Населенные пункты**	Список
Refresh	Обновить содержимое списка в режиме выбора записей и в результатах поиска	Список
SetAccessRights	Открыть карточку записи справочника для настройки прав доступа. При выполнении действия открывается закладка «Доступ» в карточке сущности	Список
SetRecipientAccessRights	Настроить прав доступа. Доступно в карточках пользователей и групп пользователей. Например, действие **Разрешения** на ленте в карточке справочника **Сотрудники**	Карточка
AddPersonalPhoto	Загрузить фотографию. Это должно быть квадратное изображение PNG или JPEG с минимальным размером 300x300 пикселей. Размер файла не должен превышать 300 КБ. Доступно в карточках пользователей	Карточка
DeletePersonalPhoto	Удалить фотографию пользователя. Доступно в карточках пользователей	Карточка

Разработка документов

Документ в системе Directum RX представляет собой карточку и версии, в которых находится содержимое документа. Документы можно создавать, отправлять в задачах на согласование, рассмотрение, подписывать, настраивать права доступа, связывать между собой и многое другое.

Особенности разработки

Основные действия при создании нового типа документа см. в разделе «Типы сущностей».

Порядок создания типа документа похож на создание типов справочников. При этом есть свои особенности:

- у документа может быть содержимое (Body) с версиями (Versions);

- документы можно связывать между собой (Relations);
- содержимое документов можно подписывать (Signatures);
- для дополнительной защиты документы можно шифровать, включать для них строгий доступ, а также ограничивать работу с ними вне системы;
- при создании типа документа от базового типа Sungero.Content.ElectronicDocuments наследуются основные свойства и методы, действия и операции авторизации;
- в редакторе типа документа есть закладка «Стили». Она предназначена для настройки отображения документов в списке. Пример использования стилей см. в списке «Реестр договоров».

Так как при создании типов документов есть особенности, то для них появляются дополнительные настройки в карточке и API, которое обеспечивает возможность программного экспорта, импорта документов, работу с версиями, электронными подписями и т.д.

От каких типов документов можно создавать наследников

При создании типа документа правильно выбирайте *базовый тип*, от которого наследуется логика работы. Например, нужно создать «Договор закупки», в нем должен поддерживаться механизм регистрации документов и особенности по работе с договорами. В этом случае в качестве базового типа сущности укажите ContractBase – базовый договор. В нем есть необходимые для договора свойства и логика, а также он наследует механизм регистрации документов от OfficialDocument. Подробнее см. схему наследования договорных документов.

ВАЖНО. Задачей на согласование по регламенту можно отправлять только документы, которые являются наследниками OfficialDocument.

В таблице перечислены типы документов, на основе которых вы можете в своем решении создавать наследников:

Content – управление документами:

- ElectronicDocuments – документы
- ElectronicDocumentTemplates – шаблоны документов

Contracts – договорные документы:

- ContractBase – базовый договор
- ContractualDocument – договорной документ

Docflow – документооборот:

- AccountingDocumentBase – базовый финансово-хозяйственный документ
- ContractualDocumentBase – базовый договорной документ
- IncomingDocumentBase – входящий документ
- InternalDocumentBase – внутренний документ

RecordManagement – делопроизводственные документы:

- OrderBase – базовый приказ

FinancialArchive – финансовые документы:

- InternalAccountingDocumentBase – внутренний учетный документ
- MinutesBase – базовый протокол
- OfficialDocument – официальный документ
- OutgoingDocumentBase – исходящий документ
- PowerOfAttorneyRevocation – заявление на отзыв доверенности

Какие типы документов можно перекрывать

Contracts – договорные документы:

- Contract – договор
- IncomingInvoice – входящий счет
- SupAgreement – дополнительное соглашение

RecordManagement – делопроизводственные документы:

- CompanyDirective – распоряжения
- IncomingLetter – входящее письмо
- Order – приказ
- OutgoingLetter – исходящее письмо

Docflow – документооборот:

- Addendum – приложение к документу
- CounterpartyDocument – сведения о контрагенте
- DocumentTemplate – шаблон документа
- ExchangeDocument – входящий документ эл. обмена
- Memo – служебная записка
- PowerOfAttorney – доверенность
- PowerOfAttorneyRevocation – заявление на отзыв доверенности
- OfficialDocument – официальный документ
- SimpleDocument – простой документ

FinancialArchive – финансовые документы:

- ContractStatement – акт выполненных работ
- IncomingTaxInvoice – счет-фактура полученный
- OutgoingTaxInvoice – счет-фактура выставленный
- UniversalTransferDocument – универсальный передаточный документ
- Waybill – накладная
- ReconciliationStatement – акт сверки
- AssetTransferDocument – учетный документ по перемещению ОС/ТМЦ

Meetings – документы по совещаниям:

- Agenda – повестка совещания
- Minutes – протокол совещания

- FixedAssetDocument – учетный документ по ОС
- InventoryDocument – учетный документ по ТМЦ
- GeneralAccountingDocument – прочий учетный документ
- IncomingPaymentOrder – входящее платежное поручение
- OutgoingPaymentOrder – исходящее платежное поручение

Projects – проекты:

- ProjectDocument – проектный документ

События типа документа

Для типа документа кроме основных серверных и клиентских событий дополнительно доступны события:

Серверные события:

- До подписания
- Смена типа

Клиентские события:

- До показа диалога подписания

Порядок выполнения событий при подписании смотрите на схеме в разделе «Подписание документа».

Также в каждом типе сущности можно задать события свойств и действий.

Объектная модель для работы с типами документов

- ElectronicDocuments – документы
- Versions – работа с версиями
- Relations – связывание документов
- ElectronicDocumentTemplates – шаблоны документов
- FilesTypes – типы файлов
- AssociatedApplications – приложения-обработчики
- Signatures – подписи
- Базовое решение Directum RX
- Иерархия документов
- Схема наследования договорных документов

Примеры разработки

Используйте примеры разработки, которые помогут изучить возможности среды разработки и подскажут идеи для развития модификации системы:

- Документы

- Разработка модуля и типа документа
- Интеллектуальная обработка документов
- Доступность, обязательность, видимость свойств

Наследуемые операции авторизации

От базового типа сущности Sungero.Content.ElectronicDocument наследуются следующие операции авторизации:

Операция	Описание
Read	Чтение
Create	Создание
Update	Изменение
Delete	Удаление
Manage	Изменение прав доступа
DelegateAccess	Передача прав доступа
Approve	Утверждение документа
ReadBody	Чтение содержимого
UpdateBody	Изменение содержимого
CreateVersion	Создание версии
DeleteVersion	Удаление версии
ExportList	Экспорт данных из списка
BulkExportContent	Массовый экспорт содержимого документов

Наследуемые действия

От базового типа документа Sungero.Content.ElectronicDocument наследуются:

действия с содержимым

Действие	Описание	Где доступно?
OpenDocumentRead	Открыть содержимое последней версии документа на чтение	Карточка и список
OpenDocumentEdit	Открыть содержимое последней версии документа на редактирование	Карточка и список
SendByMail	Отправить содержимое последней версии документа по электронной почте	Карточка и список
OpenVersions	Открыть список версий	Список
CreateVersion	Создать версию документа на основе версии, выделенной в списке версий	Дочерняя коллекция
CreateVersionFromLastVersion	Создать новую версию на основе последней версии	Карточка и список

Действие	Описание	Где доступно?
CreateFromTemplate	Создать версию на основе шаблона	Карточка и список
CreateFromFile	Создать версию документа из файла	Карточка и список
CreateFromScanner	Создать версию документа со сканера	Карточка и список
DeleteVersion	Удалить выделенную версию	Дочерняя коллекция
ReadVersion	Открыть содержимое выделенной версии для чтения	Дочерняя коллекция
EditVersion	Открыть содержимое выделенной версии на редактирование	Дочерняя коллекция
ExportLastVersion	Экспортировать содержимое последней версии в файл	Карточка и список
ExportVersion	Экспортировать содержимое выделенной версии в файл	Дочерняя коллекция
ImportInLastVersion	Импортировать файл в последнюю версию	Карточка и список
ImportInNewVersion	Импортировать файл в новую версию	Карточка и список
ImportVersion	Импортировать файл в выделенную версию	Дочерняя коллекция
ScanInNewVersion	Сканировать образ бумажного документа в новую версию	Карточка и список
ToggleEncryption	Зашифровать или отключить шифрование документа	Карточка и список
ToggleStrictRights	Включить или отключить строгий доступ	Карточка и список
ToggleStrictRightsAndEncryption	Включить или отключить строгий доступ и шифрование документа	Карточка и список

действия для подписания

Действие	Описание	Где доступно?
Sign	Подписать содержимое последней версии	Карточка и список
SignVersion	Подписать содержимое выделенной версии	Дочерняя коллекция
ShowSignatures	Показать подписи документа. По умолчанию будут показаны актуальные подписи (последние по дате установки от каждого пользователя, подписавшего документ). При необходимости состав отображаемых подписей можно изменить и выбрать:	Карточка и список

Действие	Описание	Где доступно?
	• подписи, установленные на последнюю версию документа; • все подписи, поставленные на документ	
ShowVersionSignatures	Показать подписи выделенной версии	Дочерняя коллекция

прочие действия

Действие	Описание	Где доступно?
CopyEntity	Создать документ копированием и открыть карточку нового документа. В документ копируется содержимое последней версии исходного документа	Карточка и список
CreateDocumentFromVersion	Создать документ копированием и открыть карточку нового документа. В документ копируется содержимое выбранной версии исходного документа	Карточка
DeleteEntity	Удалить документ без возможности восстановления	Карточка и список
ShowSelectEntity	Открыть карточку выделенного документа. При этом откроется карточка документа на закладке «Свойства»	Список
SetAccessRights	Открыть карточку выделенного документа для настройки прав доступа. При этом откроется карточка документа на закладке «Доступ»	Список
Cancel	Отменить текущие изменения в карточке	Карточка
Save	Сохранить изменения в карточке	Карточка
SaveAndClose	Сохранить изменения и закрыть карточку	Карточка
AddChildEntity	Добавить новую строку в текущую коллекцию. Например, действие **Добавить строку** на ленте карточки документа, открытой на закладке «Выдача». При вызове действия будет добавлена новая строка в таблицу журнала выдачи бумажных экземпляров документа	Дочерняя коллекция
CopyChildEntity	Копировать выделенную строку и добавить новую строку в текущую коллекцию. Например, действие **Копировать строку** на ленте карточки документа, открытой на закладке «Выдача». При вызове действия будет скопирована выделенная и добавлена новая строка в таблицу журнала выдачи бумажных экземпляров документа	Дочерняя коллекция
DeleteChildEntity	Удалить выделенную строку из текущей коллекции. Например, действие **Удалить строку** на ленте карточки документа, открытой на закладке «Выдача»	Дочерняя коллекция

Действие	Описание	Где доступно?
OpenRelations	Показать список связанных документов	Список
AddToMyFolder	Копировать в **Мой Directum** ссылку на список документов	Карточка и список
AddToFavorutesFolder	Копировать ссылку на документ в папку «Избранное»	Карточка и список
CopyToClipboard	Копировать в буфер обмена ссылку на документ, выделенный в списке	Список
CopyToClipboardFromCard	Копировать в буфер обмена ссылку на документ, в карточке которого вызвано действие	Карточка
SearchFoldersContainsLinkToEntity	Найти папки, в которые вложена ссылка на документ	Карточка и список
ChangeSearchCriteria	Изменить критерии поиска	Список
Refresh	Обновить содержимое списка в режиме выбора записей и в результатах поиска	Список
ChooseEntityAndCloseCollection	Выбрать документ для свойства-ссылки и закрыть список записей. Например, действие **Выбрать запись** на ленте списка для выбора при указании ведущего документа в карточке акта к договору	Список
SelectAndClose	Сохранить изменения, выбрать документ и закрыть карточку. Например, действие **Выбрать и закрыть** на ленте карточки нового договора, который создается при указании ведущего документа в карточке акта к договору	Карточка

Строгий доступ к документам

По умолчанию в Directum RX не настроена возможность включать строгий доступ к документам. Если система установлена локально, строгий доступ настраивает администратор или разработчик системы.

С помощью среды разработки вы можете настроить:

- программное включение строгого доступа;
- включение по кнопке из карточки документа;
- совместный режим шифрования и строгого доступа;
- передача прав доступа замещающему.

При разработке учитывайте особенности по работе с правами для документов со строгим доступом.

ВАЖНО. При модификации системы и настройке интеграции необходимо отслеживать, чтобы документы со строгим доступом не уходили в другие системы, так как в них данные могут храниться в незащищенном виде. Если от имени пользователя, входящего в роль «Служебные пользователи», выполняется чтение или запись документов, для которых

установлен строгий доступ, то в лог-файле соответствующего сервиса Directum RX фиксируется ошибка (Error).

Программное включение строгого доступа

В этом варианте настройки строгий доступ включается системой автоматически, в зависимости от настроек в программном коде. При этом пользователь вручную включать строгий доступ не может.

Для программной настройки доступны:

- метод AccessRights.SetStrictMode() – установить режим работы строгого доступа;
- метод AccessRights.CanSetStrictMode() – проверить возможность изменения режима строгого доступа;
- свойство AccessRights.StrictMode – режим работы строгого доступа. Возможные значения: **None** – строгий доступ выключен; **Strict** – включен строгий доступ; **Enhanced** – включен усиленный строгий доступ.

Пример1. Программное включение строгого доступа

Если необходимо, чтобы при создании приказа для него автоматически включался строгий доступ, то перекройте тип документа «Приказ» (Sungero.RecordManagement.Order) и добавьте код в событии сущности «Создание»:

```
    public override void Created(Sungero.Domain.CreatedEventArgs e)
{
  _obj.AccessRights.SetStrictMode(Sungero.Core.AccessRightsStrictMode.Strict);
}
```

Пример 2. Выдача прав доступа в зависимости от режима строгого доступа

```
    // Если для документа выключен строгий доступ, то выдать права на изменение
       документа.
if(document.AccessRights.StrictMode == Sungero.Core.AccessRightsStrictMode.None)
{
  document.AccessRights.Grant(user, DefaultAccessRightsTypes.Change)
}
```

Включение строгого доступа из карточки документа

В этом варианте настройки пользователь с полными правами сам включает строгий доступ по кнопке в карточке нужного ему документа. Это можно настроить следующими способами:

- администратор в конфигурационном файле добавляет параметры и перечисляет в них идентификаторы типов документов, для которых доступно включение строгого доступа;
- разработчик переопределяет действие **ToggleStrictRights** – включить или отключить строгий доступ. Например, разработан новый тип документа и нужно, чтобы пользователи могли устанавливать строгий доступ для документов этого типа. Для этого в редакторе типа документа в событии **Выполнение/Возможность выполнения** переопределите логику работы действия. Подробнее пример разработки см. в разделе «Как настроить строгий доступ к документу».

Совместный режим шифрования и строгого доступа

Если для типа документа настроены шифрование и строгий доступ через интерфейс системы, то по умолчанию в карточке документа они объединяются в одно действие .

При необходимости можно отключить совместный режим. Тогда в карточке документа будут два разных действия. Это можно сделать следующими способами:

- администратор добавляет параметр в конфигурационный файл;
- разработчик переопределяет действие **ToggleStrictRightsAndEncryption** – включить или отключить строгий доступ и шифрование документа. Например, разработан тип документа и нужно, чтобы пользователи сами определяли, когда нужно зашифровать документ, а когда включить строгий доступ. Для этого в редакторе типа документа в событии **Выполнение/Возможность выполнения** переопределите логику работы действия.

Передача прав доступа замещающему

По умолчанию права на документ с включенным строгим доступом не передаются замещающему. Но могут возникнуть ситуации, когда для конкретного замещения все же необходимо передавать права. Например, если помощник замещает руководителя и выполняет действия в системе от его имени. В этой ситуации вы можете программно настроить передачу прав замещающему. Для этого используйте свойство DelegateStrictRights – признак того, что нужно делегировать строгий доступ.

Чтобы применить программную настройку, в конфигураторе Directum Launcher добавьте параметр **ALLOW_STRICT_ACCESS_DELEGATION_IN_SUBSTITUTION** – разрешить делегирование строгого доступа в замещениях. Для этого в параметре установите флажок (true).

Пример. Создание системного замещения и передача прав на документы со строгим доступом

```
   /// </summary>
/// <param name="substitutedUser">Отсутствующий пользователь.</param>
/// <param name="substitute">Замещающий пользователь.</param>
public static void CreateSystemSubstitution(IUser substitutedUser, IUser
substitute)
{
  if (Equals(substitutedUser, substitute))
    return;
  var substitution = Substitutions.Create();
  substitution.User = substitutedUser;
  substitution.Substitute = substitute;
  substitution.IsSystem = true;
  // Передавать права на документы со строгим доступом.
  substitution.DelegateStrictRights = true;
  substitution.Save();
}
```

ПРИМЕЧАНИЕ. В истории работы с замещением фиксируется информация о том, что строгий доступ делегирован.

Особенности работы с правами доступа

Если задается программный код для обработки документов сервисами Directum RX, то учитывайте особенности:

- программно со стороны сервисов можно работать с документом, для которого включен строгий доступ;
- строгий доступ можно повышать до усиленного со стороны сервисов, но нельзя обратно понизить до строгого доступа. Иначе при обращении к документам возникнет исключение.

При выполнении методов Grant(), Revoke() и RevokeAll() права доступа не меняются, если действие выполняется от пользователя:

- у которого нет полных прав на документ;
- входящего в роль «Служебные пользователи». При этом программный код выполняется на сервисе, и для документа включен усиленный строгий доступ;
- входящего в роль «Служебные пользователи», и у него нет полных прав на документ. При этом программный код выполняется не на сервисе.

В этой ситуации в лог-файл записывается информация, что права не могут быть выданы или изъяты.

Шифрование документов

По умолчанию в Directum RX не настроена возможность шифровать документы. Для настройки администратор разворачивает сервис ключей и определяет типы документов, которые можно будет шифровать.

Разработчику тоже доступна настройка шифрования документов. С помощью среды разработки вы можете настроить:

- программное шифрование документа;
- шифрование по кнопке из карточки документа;
- совместный режим шифрования и строгого доступа.

ВАЖНО. При модификации системы и настройке интеграции необходимо отслеживать, чтобы зашифрованные документы не уходили в другие системы, так как в них данные могут храниться в незащищенном виде. Если от имени пользователя, входящего в роль «Служебные пользователи», выполняется чтение или запись зашифрованных документов, то в лог-файле соответствующего сервиса Directum RX фиксируется ошибка (Error).

Программное шифрование документа

В этом варианте шифрование включается системой автоматически, в зависимости от логики, добавленной в программном коде. При этом пользователь вручную зашифровать документ не может. Для программной настройки используйте свойство IsEncrypted – признак того, что документ зашифрован.

Кроме того, в программном коде доступно свойство Encryption.IsEncryptionAvailable – признак того, что доступно шифрование документов. Возможные значения: **true** –

шифрование доступно, **false** – шифрование недоступно. Свойство используется, если нужно предусмотреть поведение работы при развернутом шифровании и при отсутствии шифрования документов. Например, чтобы документ после согласования программно шифровался, если в системе настроено шифрование, и не шифровался, если возможность отсутствует. После того, как разработчик напишет программный код, необходимо в конфигурационный файл config.yml в секцию **common_config** добавить параметр **DATA_ENCRYPTION_ENABLED** и указать в нем значение **true**.

Пример. Программное шифрование приказов

Предположим, необходимо, чтобы при создании приказа он автоматически шифровался. Для этого перекройте тип документа «Приказ» (Sungero.RecordManagement.Order) и добавьте код в событии сущности Создание.

ВАЖНО. Шифруется и содержимое версии документа (Body), и его представление (PublicBody).

```
    public override void Created(Sungero.Domain.CreatedEventArgs e)
{
  base.Created(e);
  _obj.IsEncrypted = true;
}
```

Шифрование по кнопке из карточки документа

В этом варианте настройки пользователь с полными правами сам шифрует нужный ему документ по кнопке в карточке.

Настроить шифрование по кнопке из карточки можно следующими способами:

- администратор в конфигурационном файле добавляет параметр, и перечисляет в нем идентификаторы типов документов, для которых доступно шифрование;
- разработчик переопределяет действие **ToggleEncryption** – зашифровать или отключить шифрование документа. Например, разработан новый тип документа и нужно, чтобы пользователи могли шифровать документы этого типа. Для этого в редакторе типа документа в событии **Выполнение/Возможность выполнения** переопределите логику работы действия. Подробнее пример разработки см. в разделе «Как настроить шифрование договоров».

Ограничение на работу с документами вне системы

В системе Directum RX для защиты конфиденциальной информации сотрудники могут шифровать документы и включать для них строгий доступ. Кроме этого, с помощью среды разработки можно дополнительно ограничить действия на работу с документом вне системы: открытие в локальном редакторе, экспорт из системы, отправка электронной почтой и на печать.

После включения ограничения сотрудники работают с документами только в системе, не получая локальной копии. Для этого с последней или определенной версией документа:

- ограничиваются действия: экспорт из системы (ExportVersion и ExportLastVersion), отправка по почте (SendByMail и SendVersionByMail), отправка на печать (SendToPrint и SendVersionToPrint);

- переопределяется логика в действиях: открытие (OpenDocument), открытие на чтение (OpenDocumentRead и ReadVersion) и открытие на изменение (OpenDocumentEdit и EditVersion). Документы принудительно открываются на чтение в окне предпросмотра, а на изменение – в онлайн-редакторе. Для этого должны быть развернуты сервис предпросмотра и онлайн-редактирования. Иначе при открытии документов возникает ошибка.

Чтобы ограничить работу с документами вне системы:

1. Перекройте нужный тип документа.
2. Переопределите клиентскую функцию **GetDocumentPrivacyOptions()**. По умолчанию она вызывается в событии **Возможность выполнения** (CanExecute).

   ВАЖНО. В обработчике события **Возможность выполнения** не рекомендуется вызывать серверные функции (с атрибутом **Remote**) и использовать логику, которая будет долго обрабатываться. Это влияет на быстродействие системы, так как обработчик выполняется часто.
3. В функции создайте ограничение с помощью метода **ElectronicDocumentPrivacyOptions.Create()**, указав в нем значение **true**:

```
public override ElectronicDocumentPrivacyOptions GetDocumentPrivacyOptions()
    {
      // Создать ограничения на работу с документами вне системы
      return ElectronicDocumentPrivacyOptions.Create(true);
    }
```

   Функция возвращает значение ElectronicDocumentPrivacyOptions – класс, в котором определены признаки ограничения работы с документом. Класс имеет свойство **IsConfidential** – признак того, что нужно ограничить работу с документом вне системы, так как он содержит конфиденциальные данные. По умолчанию свойство имеет значение **false**.

Не используйте функцию **GetDocumentPrivacyOptions()**, если:

- нужно ограничить не все перечисленные выше действия с документом;
- написан код для выгрузки содержимого документа и для этого не используются стандартные методы. Например, если содержимое документа Body запаковывается в архив и выгружается на компьютер пользователя.

В таких случаях, чтобы ограничить действия с документом, в своем программном коде доработайте нужные действия: в перекрытом типе документа скройте их или переопределите логику.

Разработка задач и заданий

Задачи предназначены для автоматизации бизнес-процессов компании. В задаче указываются исполнители, сроки выполнения и другие данные. Для каждого типа задачи разрабатывается схема, при выполнении которой создаются другие задачи, задания, уведомления и задание на приемку.

Карточка задачи содержит:

- закладки для просмотра и редактирования свойств;
- текст задачи. После старта задачи текст появится в области переписки в задании. Переписка позволяет проследить ход работ по задаче;

- область вложений. Содержит ссылки на сущности, с которыми работают пользователи. Например, во вложения добавляется документ, который необходимо согласовать и зарегистрировать.

Разработка типа задачи

1. Создайте тип задачи. Подробнее см. раздел «Типы сущностей».
2. Настройте схему задачи одним из способов:
   - в проводнике веб-клиента. Для этого убедитесь, что в типе задачи установлен флажок **Схема настраивается в проводнике** в редакторе типа сущности, создайте свои блоки и настройте схему бизнес-процесса. Способ используется по умолчанию;

     ПРИМЕЧАНИЕ. Если для схемы задачи разрабатывается несколько вариантов процессов, подходящих по критериям, и нужно дать возможность пользователю вручную указывать процесс, свойство «Процесс» (ProcessKind) можно вынести на форму с помощью инструмента разработки;
   - в среде разработки. Для этого в новом типе задачи снимите флажок **Схема настраивается в проводнике** в редакторе типа сущности. Схему задачи настраивает разработчик.

Особенности разработки

У типа задачи есть подчиненные типы сущностей:

- задание для выполнения работ по задаче. Основные характеристики: описание необходимых работ, срок задания и результаты выполнения. По одной задаче может быть создано несколько заданий;
- задание на приемку выполненных работ. Создается только в рамках задачи и не существует за ее пределами. Основные характеристики: срок и результат приемки работ. Задание на приемку выполняется с одним из результатов:

  «Принять», если результаты выполнения устраивают. В этом случае задача завершается.

  «Отправить на доработку», если есть пожелания по доработке результатов. В этом случае задание вернется исполнителю.

  Задача не завершится до тех пор, пока задание на приемку не будет выполнено с результатом «Принять». В рамках одной задачи создается одно задание на приемку;
- уведомление для оповещения пользователей о тех или иных работах. В уведомлении нет результатов выполнения. В рамках одной задачи может быть создано несколько уведомлений.

При разработке типа задачи есть особенности:

- для каждого типа задачи настраивается схема бизнес-процесса в проводнике веб-клиента либо в среде разработки;
- у задач и заданий есть вложения, в которые добавляются ссылки на документы, записи справочников, другие задачи. Также у задач и заданий есть панель предпросмотра вложений, на которой отображается содержимое вложенного документа.

  ПРИМЕЧАНИЕ. В типах заданий и типах уведомлений, созданных в рамках модуля, недоступны группы вложений, реализованные для конкретного типа задачи;

- в отличие от типов справочников и документов в карточке есть область переписки, которая позволяет проследить ход работ по задаче;
- для типов заданий можно настроить стили отображения, для типов задач – нет;
- при создании типа задачи от базового типа Sungero.Workflow.Task наследуются основные свойства и методы, действия и операции авторизации;
- в системе нет возможности создавать *базовые типы* задач.

От каких типов задач можно создавать наследников

В своем решении разработчик может создавать типы задач на основе базового типа Sungero.Workflow.Task, а также связанные с задачей типы заданий, заданий на приемку и уведомлений.

Какие типы задач можно перекрывать

Docflow – документооборот:

- ApprovalTask – задача на согласование по регламенту
- CheckReturnTask – контроль возврата документа
- DeadlineExtensionTask – запрос на продление срока
- FreeApprovalTask – задача на свободное согласование

Exchange – электронный обмен:

- ExchangeDocumentProcessingTask – задача на обработку входящих документов эл. обмена
- ReceiptNotificationSendingTask – задача на отправку извещений о получении документов

ExchangeCore – электронный обмен. Настройки:

- CounterpartyConflictProcessingTask – обработка конфликтов синхронизации контрагентов
- IncomingInvitationTask – обработка приглашения к эл. обмену от контрагента

RecordManagement – делопроизводственные документы:

- AcquaintanceTask – задача на ознакомление с документом
- ActionItemExecutionTask – задача на исполнение поручения
- DeadlineExtensionTask – продление срока
- DocumentReviewTask – задача на рассмотрение документа
- StatusReportRequestTask – запрос отчета по поручению

SmartProcessing – интеллектуальная обработка

- VerificationTask – задача на верификацию

Описание типов заданий, которые можно перекрывать, см. в разделе «Базовое решение Directum RX».

События типа задачи

Для типа задачи кроме клиентских и серверных событий, которые есть у всех типов сущностей, дополнительно доступны события:

Серверные события типа задачи:

- До старта
- До рестарта
- До возобновления
- До прекращения
- После приостановки

События групп вложений задачи:

- Создание вложения
- Добавление вложения
- Удаление вложения
- Заполнение вложений

Серверные события типа задания, задания на приемку, уведомления

- До выполнения
- До принятия
- До отправки на доработку

События блоков схемы задачи:

- см. описание блоков схемы

События результатов выполнения заданий:

- Выполнение/Возможность выполнения

Также в каждом типе сущности можно задать события свойств и действий.

Объектная модель для работы с задачами и заданиями

- Task – задачи
- SimpleTask – простые задачи
- Assignment, Notice – задания и уведомления
- Attachments – работа со вложениями
- Scheme – схема варианта процесса
- Blocks – блоки схемы
- Базовое решение Directum RX

Примеры разработки

Используйте примеры разработки, которые помогут изучить возможности среды разработки и подскажут идеи для развития модификации системы:

- Задачи и задания
- Согласование по регламенту
- Доступность, обязательность, видимость свойств

Настройка блоков бизнес-процессов

Схема бизнес-процесса состоит из блоков и переходов между ними. Если возможностей блоков из стандартной поставки системы недостаточно, то в среде разработки можно создать *прикладные блоки* следующих типов:

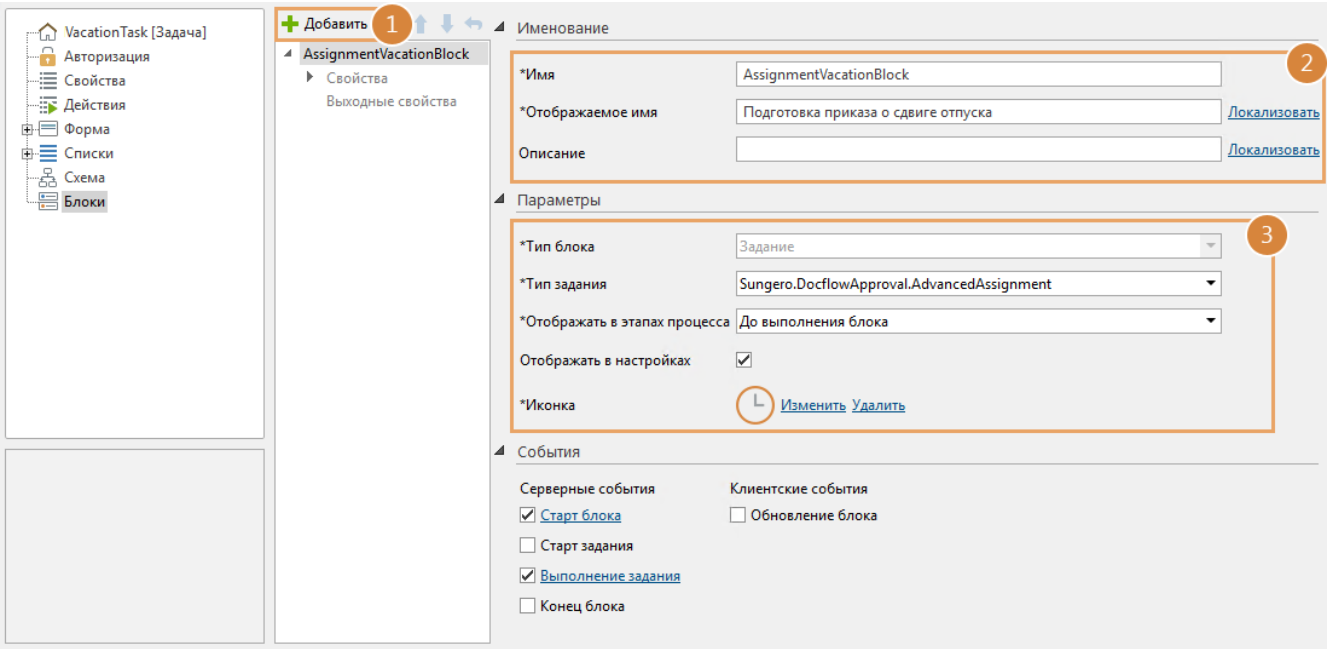
- Скрипт (ScriptBlock)
- Задача (TaskBlock)
- Задание (AssignmentBlock)
- Уведомление (NoticeBlock)
- Мониторинг (MonitoringBlock)

Разработанные прикладные блоки отображаются в веб-клиенте на панели «Библиотека блоков». Блок, созданный в редакторе типа задачи, доступен в любом варианте процесса, но только для задачи конкретного типа. При создании в редакторе модуля блок доступен для любых задач.

ПРИМЕЧАНИЕ. Настройка блоков для типа задачи доступна, если в редакторе типа задачи установлен флажок **Схема настраивается в проводнике**.

Создание блока

1. Откройте редактор модуля или типа задачи, в котором будет создан блок. В узле «Блоки» нажмите на кнопку **Добавить**:



2. В списке элементов выберите блок и задайте настройки:

   ***Имя блока**. Должно быть уникальным в рамках одного модуля, может состоять из букв латинского алфавита и цифр от 2 до 50 символов.

   ***Отображаемое имя**. Имя блока, которое отображается в редакторе схемы. Для локализации отображаемого имени перейдите по ссылке **Локализовать**.

   Описание. Поясняющий текст, который отображается в редакторе схемы. Для локализации описания блока перейдите по ссылке **Локализовать**.

ПРИМЕЧАНИЕ. При добавлении нового блока на схему варианта процесса отображается описание, заданное в среде разработки. В веб-клиенте можно изменить поясняющий текст только для конкретного экземпляра блока.

3. Задайте параметры блока:

***Тип задачи**. Для блока «Задача» укажите тип задачи;

Несколько задач. Если флажок установлен, в блоке «Задача» будет создано несколько задач.

Создать из коллекции. В выпадающем списке выберите свойство-коллекцию основной задачи. В таком случае по блоку будет создаваться такое количество задач, которое соответствует числу элементов в выбранной коллекции. Список доступен, если установлен флажок **Несколько задач**.

***Тип задания**, созданный в рамках задачи или модуля. Поле отображается для блоков «Задание» и «Уведомление».

***Тип уведомления**, созданный в рамках задачи или модуля. Поле отображается для блока «Уведомление».

***Отображать в этапах процесса**. Укажите, отображается ли по умолчанию блок на вкладке «Этапы процесса» в карточке задачи, задания или уведомления. Для этого в выпадающем списке выберите значение:

- **Пока не выполнен**. Блок отображается, если он выполняется в текущий момент или будет выполняться в дальнейшем;
- **В работе**. Блок отображается, если он выполняется в текущий момент;
- **Никогда**. Блок скрыт.

На вкладке рекомендуется отображать только значимые блоки: согласование, регистрация, подписание и т.д. Остальные можно скрыть.

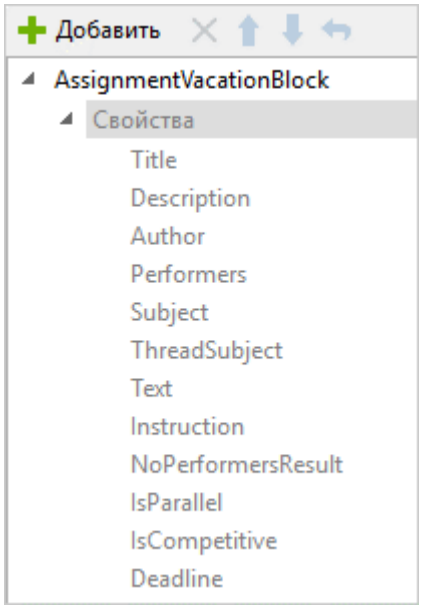
Выбранное значение используется в проводнике веб-клиента для аналогичного параметра **Отображать в этапах процесса**. Значение в веб-клиенте можно изменить, если в среде разработки для блока установлен флажок **Отображать в настройках**.

Отображать в настройках. Установите флажок, чтобы свойство стало доступно для настройки схемы процесса в проводнике веб-клиента.

ПРИМЕЧАНИЕ. Если блок устарел, то снимите флажок. В этом случае блок продолжит работать в уже настроенных схемах процессов, но исчезнет из библиотеки блоков, и его нельзя будет добавить на новые схемы.

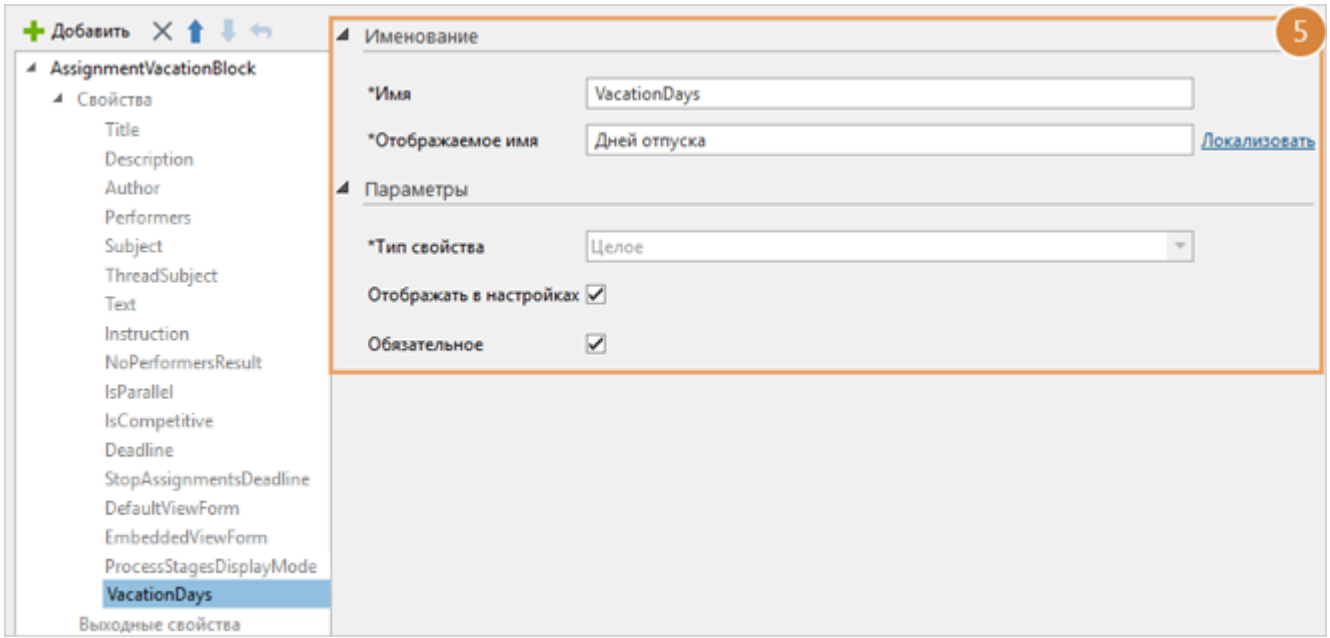
***Иконка**. Отображается в библиотеке блоков и на вкладке «Этапы процесса» слева от наименования блока. Иконка должна быть в формате SVG и в размере 32x32 пикселя. Иконки, используемые в *базовом решении*, можно найти в локальной папке исходных кодов. Расположение папки задается при установке системы.

4. Добавьте свойства блока. Существуют стандартные свойства, например **Заголовок** (Title) или **Исполнители** (Performers) для блоков «Задание» и «Уведомление». Такие свойства можно отображать или скрывать в настройках. В дополнение к стандартным можно создавать новые свойства. Для этого в списке элементов выделите узел **Свойства** и нажмите на кнопку **Добавить**:



Доступны следующие типы свойств: строка, целое, логическое, вещественное, дата и время, перечисление, ссылка, коллекция ссылок.

5. В списке элементов выделите свойство блока и задайте настройки в зависимости от его типа:



***Имя**. Должно быть уникальным в рамках одного модуля, может состоять из букв латинского алфавита и цифр.

***Отображаемое имя**. Имя свойства, которое отображается в редакторе схемы. Для локализации отображаемого имени перейдите по ссылке **Локализовать**.

***Тип сущности**. Сущности указанного типа будут значениями для параметра типа «ссылка» или «коллекция ссылок».

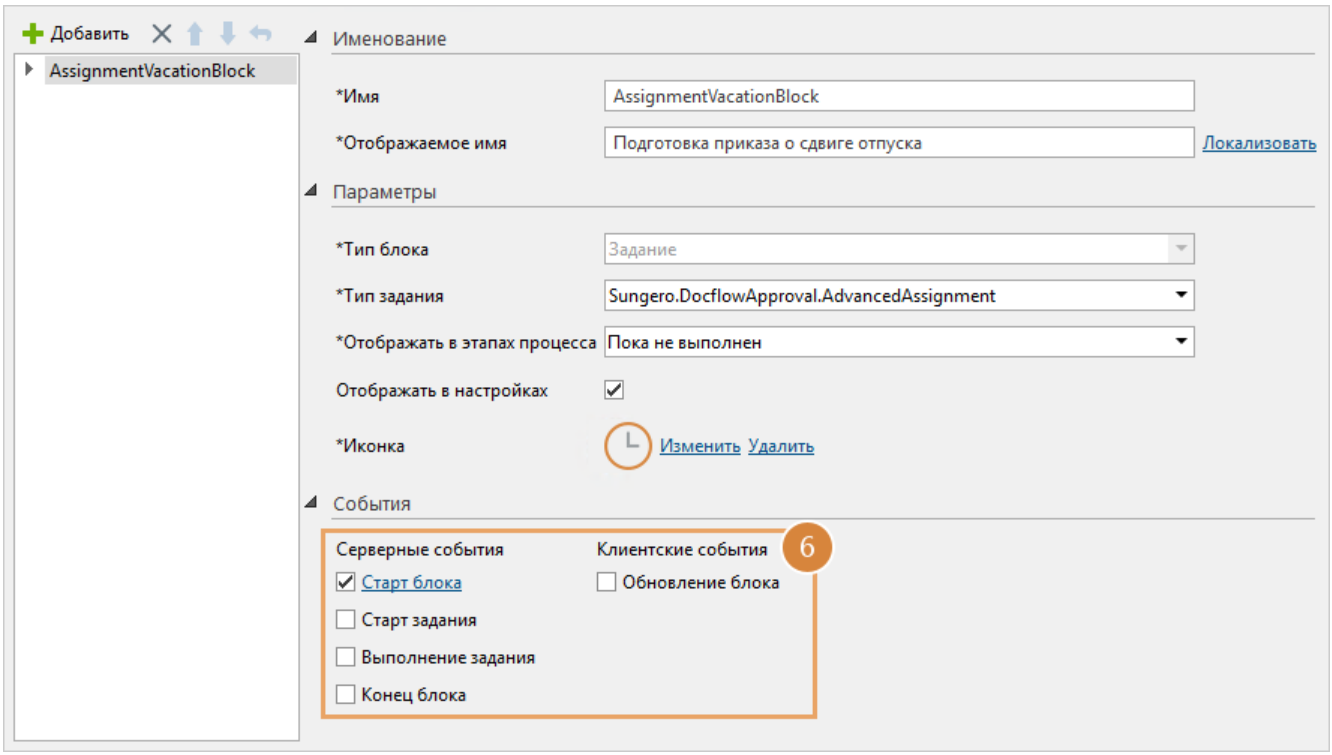
Значение перечисления. Список возможных значений перечисления.

Отображать в настройках. Свойство будет доступно для настройки схемы процесса в проводнике веб-клиента.

ПРИМЕЧАНИЕ. Если свойство устарело, то снимите флажок. Блоки, в которых задано свойство, продолжат его учитывать. При этом *аналитик* не сможет изменить свойство в существующем блоке и задать его в новом.

Обязательное. Установите флажок, если свойство нужно сделать обязательным для заполнения. В этом случае в редакторе схемы рядом с названием свойства отображается символ *.

6. Задайте события блока. Для этого выберите блок, в группе «События» рядом с нужным событием установите флажок. Станет доступна ссылка с названием события:



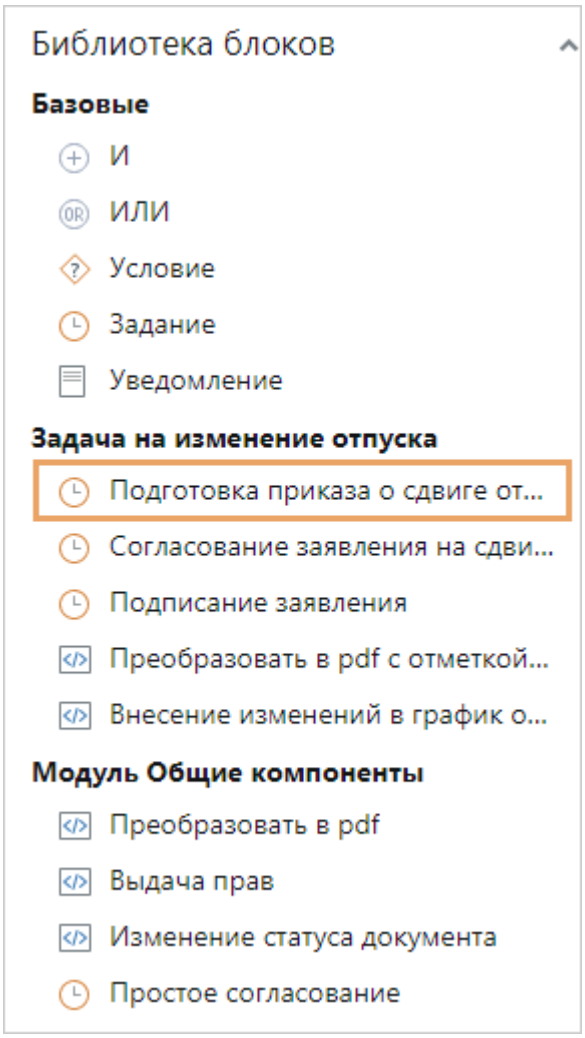
Перейдите по ссылке и в открывшемся редакторе напишите код обработчика события:

Пример. Добавить исполнителей и срок выполнения задания:

```
public virtual void AssignmentBlockStart()
{
  _block.Performers.Add(_obj.Performer);
  _block.RelativeDeadlineHours = 4.5;
}
```

7. Добавьте выходные свойства в блок, если нужно передавать данные из событий блока в параметры процесса.
8. Добавьте новые переходы для блоков типов «Скрипт» и «Мониторинг», если нужны дополнительные результаты выполнения.
9. Опубликуйте разработку.

В результате созданный блок появится в библиотеке блоков. Его можно вынести на схему бизнес-процесса и настроить. Блок можно использовать в рамках одной задачи или разных типов задач. Например, если требуется досканировать документ и в задаче на подготовку приказа, и в задаче на согласование заявления.



Блок «Скрипт»

«Скрипт» (ScriptBlock). Блок предназначен для выполнения произвольного программного кода.

Свойства блока

Свойство	Описание
Заголовок (Title)	Отображается в схеме процесса
Описание (Description)	Поясняющий текст, который отображается на панели настроек блока на схеме варианта процесса. ПРИМЕЧАНИЕ. В прикладном коде свойство доступно только для чтения

Выходные свойства блока

Свойство	Описание
Результат выполнения (ExecutionResult)	Результаты выполнения блока. Значение по умолчанию: **Success** (Успешно). **<Результат выполнения>** – произвольный результат, добавленный разработчиком

Серверные события блока

Событие	Назначение	Аргументы события
Выполнение (<Имя блока>Execute)	Выполнение произвольного программного кода. Событие выполняется на сервисе выполнения блоков схем задач (WorkflowBlockService) при обработке блока	**_obj** – задача, в рамках которой обрабатывается блок. **_block** – блок, в рамках которого выполняется событие. **_block.ProcessStagesDisplayMode** – режим отображения блока в этапах процесса. Возможные значения: • **Show** – пока не выполнен. Блок отображается, если он выполняется в текущий момент или будет выполняться в дальнейшем; • **ShowInWork** – в работе. Блок отображается, если он выполняется в текущий момент; • **Hide** – никогда. Блок скрыт. Изменять режим отображения рекомендуется в веб-клиенте через свойство блока **Отображать в этапах процесса** **_block.RetrySettings** – повторение обработчика блока

Пример. Выдача прав на вложения исполнителю

```
public virtual void MyScriptBlockExecute()
{
  // Выдать права на вложения исполнителю.
  var documents = _obj.Attachments.ToList();
  Functions.MyTask.GrantAccess(documents, _obj.Performer);
}
```

Клиентские события блока

Событие	Назначение	Аргументы события
Обновление блока (<Имя блока>RefreshBlock)	Задание логики для отображения свойств блока в зависимости от условий. Выполняется при открытии схемы процесса в редакторе и изменении свойств блока	**_block** – блок, в рамках которого выполняется событие. **_block.<название свойства>.Value** – значение свойства. **_block.<название свойства>.IsEmpty()** – проверить, заполнено ли свойство. **_block.<название свойства>.IsExpression()** – проверить, является ли свойство вычисляемым выражением. **_block.State.Properties.<название свойства>.<состояние>** – состояние свойства. Возможные значения:

Событие	Назначение	Аргументы события
		• **isVisible** – видимое; • **isEnabled** – доступное; • **isRequired** – обязательное. **_block.ProcessStagesDisplayMode** – режим отображения блока в этапах процесса. Возможные значения: • **Show** – пока не выполнен. Блок отображается, если он выполняется в текущий момент или будет выполняться в дальнейшем; • **ShowInWork** – в работе. Блок отображается, если он выполняется в текущий момент; • **Hide** – никогда. Блок скрыт. Изменять режим отображения рекомендуется в веб-клиенте через свойство блока **Отображать в этапах процесса** **_block.RetrySettings** – повторение обработчика блока

Пример. Отображение выпадающего списка в зависимости от того, установлен ли флажок

```
public virtual void AssignmentBlockRefresh()
{
  // Если флажок PropertyBool установлен, то отобразить выпадающий список
PropertyEnum
  if (_block.PropertyBool.Value == true)
  {
    _block.State.Properties.PropertyEnum.IsVisible = true;
  }
  // Если флажок PropertyBool снят, то скрыть выпадающий список PropertyEnum
  if (_block.PropertyBool.Value == false)
  {
    _block.State.Properties.PropertyEnum.IsVisible = false;
  }
  Logger.Error("AssignmentBlockRefresh");
}
```

Блок «Задача»

«Задача» (TaskBlock). Блок предназначен для создания задачи или подзадачи определенного типа в рамках варианта процесса.

Свойства блока

Свойство	Описание
Заголовок (Title)	Заголовок блока, который видит разработчик в схеме задачи
Описание (Description)	Поясняющий текст, который отображается на панели настроек блока на схеме варианта процесса. ПРИМЕЧАНИЕ. В прикладном коде свойство доступно только для чтения
Инициатор (Author)	Инициатор задачи
Наблюдатели (Observers)	Список наблюдателей задачи.
Тема (Subject)	Тема задачи, которую видят пользователи в системе
Тема в переписке (Subject in thread)	Тема задачи в переписке, которую видят пользователи
Текст (Text)	Текст задачи, который видят пользователи в системе
Важность (Importance)	Важность задачи. Возможные значения: **Нормальная**, **Высокая**
Конечный срок (MaxDeadline)	Конечный срок задачи
Подзадача (IsSubtask)	Признак того, что создается подзадача. Возможные значения: **True**, **False**. Значение по умолчанию **False** (флажок не установлен)
Задание на приемку (NeedsReview)	Признак того, что после выполнения всех блоков схемы задачи нужно создать задание на приемку. Возможные значения: **True**, **False**. Значение по умолчанию **True** (флажок установлен)
Ожидать завершения (WaitForCompletion)	Признак, определяющий переход к следующему блоку. Если установлен, переход выполнится, если все задачи в рамках текущего блока завершены. Возможные значения: **True**, **False**. Значение по умолчанию **False** (флажок не установлен)
Процесс (ProcessKind)	Вариант процесса, по которому нужно стартовать задачи, созданные в блоке
Представление по умолчанию (DefaultViewForm)	Представление формы, которое отображается при открытии карточек отдельно от списка. Если свойство не указано, подбирается наиболее подходящее представление
Представление в списке (EmbeddedViewForm)	Представление формы для карточки в списке. Если свойство не указано, отображается представление по умолчанию

Серверные события блока

Событие	Назначение	Аргументы события
Старт блока (<Имя блока>Start)	Заполнение свойств выполняемого блока. Событие выполняется на сервисе выполнения блоков схем задач (WorkflowBlockService) при старте обработки блока	**_obj** – задача, в рамках которой обрабатывается блок. **_block** – блок, в рамках которого выполняется событие.
Старт задачи (<Имя блока>StartTask)	Заполнение свойств задачи, которые нельзя заполнить через свойства блока. Например, свойства типа Текст, Картинка, Бинарные данные. Событие выполняется на сервисе выполнения блоков схем задач (WorkflowBlockService) до старта задачи (после создания и заполнения свойств задачи) и до сохранения в базу данных	**_obj** – задача, в рамках которой обрабатывается блок. **createdTask** – задача, созданная в рамках блока. Если установлен флажок **Несколько задач**, то в аргументе **item** задается элемент коллекции. **_block** – блок, в рамках которого выполняется событие.
Конец блока (<Имя блока>End)	Актуализация свойств задачи после выполнения блока. Событие выполняется на сервисе выполнения блоков схем задач (WorkflowBlockService) при завершении обработки блока ПРИМЕЧАНИЕ. Изменение свойств объектов, заданное в веб-клиенте при завершении блока, происходит после выполнения события «Конец блока»	**_obj** – задача, в рамках которой обрабатывается блок. **_block** – блок, в рамках которого выполняется событие. **createdTasks** – список задач, созданных в рамках блока
Все серверные события блока		**_block.ProcessStagesDisplayMode** – режим отображения блока в этапах процесса. Возможные значения: • **Show** – пока не выполнен. Блок отображается, если он выполняется в текущий момент или будет выполняться в дальнейшем; • **ShowInWork** – в работе. Блок отображается, если он выполняется в текущий момент; • **Hide** – никогда. Блок скрыт.

Событие	Назначение	Аргументы события
		Изменять режим отображения рекомендуется в веб-клиенте через свойство блока **Отображать в этапах процесса**

Клиентские события блока

Событие	Назначение	Аргументы события
Обновление блока (<Имя блока>RefreshBlock)	Задание логики для отображения свойств блока в зависимости от условий. Выполняется при открытии схемы процесса в редакторе и изменении свойств блока	**_block** – блок, в рамках которого выполняется событие. **_block.<название свойства>.Value** – значение свойства. **_block.<название свойства>.IsEmpty()** – проверить, заполнено ли свойство. **_block.<название свойства>.IsExpression()** – проверить, является ли свойство вычисляемым выражением. **_block.State.Properties.<название свойства>.<состояние>** – состояние свойства. Возможные значения: • **isVisible** – видимое; • **isEnabled** – доступное; • **isRequired** – обязательное **_block.ProcessStagesDisplayMode** – режим отображения блока в этапах процесса. Возможные значения: • **Show** – пока не выполнен. Блок отображается, если он выполняется в текущий момент или будет выполняться в дальнейшем; • **ShowInWork** – в работе. Блок отображается, если он выполняется в текущий момент; • **Hide** – никогда. Блок скрыт. Изменять режим отображения рекомендуется в веб-клиенте через свойство блока **Отображать в этапах процесса**

Пример. Отображение выпадающего списка в зависимости от того, установлен ли флажок

```
public virtual void AssignmentBlockRefresh()
{
  // Если флажок PropertyBool установлен, то отобразить выпадающий список
PropertyEnum
```

```
  if (_block.PropertyBool.Value == true)
  {
    _block.State.Properties.PropertyEnum.IsVisible = true;
  }
  // Если флажок PropertyBool снят, то скрыть выпадающий список PropertyEnum
  if (_block.PropertyBool.Value == false)
  {
    _block.State.Properties.PropertyEnum.IsVisible = false;
  }
  Logger.Error("AssignmentBlockRefresh");
}
```

Блок «Задание»

«Задание» (AssignmentBlock). Блок предназначен для создания заданий. Выход из блока происходит при выполнении или прекращении всех заданий, созданных в рамках блока.

Свойства блока

Свойство	Описание
Заголовок (Title)	Заголовок блока, который видит разработчик в схеме задачи
Описание (Description)	Поясняющий текст, который отображается на панели настроек блока на схеме варианта процесса. ПРИМЕЧАНИЕ. В прикладном коде свойство доступно только для чтения
Инициатор (Author)	Инициатор задания
Исполнители (Performers)	Список исполнителей. Для выбора доступны роли и наследники от групп. Sid (идентификатор субъекта прав) можно узнать в таблице Sungero_Core_Recipient
Тема (Subject)	Тема задания, которую видят пользователи в системе
Тема в переписке (ThreadSubject)	Тема задания в переписке
Текст (Text)	Текст задания, который видят пользователи в системе
Инструкция (Instruction)	Инструкция к заданию
Результат отсутствия исполнителей (NoPerformersResult)	Результат отсутствия исполнителей – это результат выполнения блока, если по нему не было создано ни одного задания. Возможные значения: • **Не задан**; • результат выполнения для задания, которое указанно в параметре ***Тип задания**. Например, **Выполнить**, **На доработку**, **Отложить**; • **Результат: Нет исполнителей**. Используйте, если нужно добавить переход из блока в виде отдельной стрелки в случае отсутствия исполнителей. На схеме появится возможность указать значение Нет исполнителей для исходящей из блока стрелки; • **Иначе**

Свойство	Описание
Параллельные задания (IsParallel)	Признак, того, что задания по блоку будут создаваться одновременно или друг за другом. Возможные значения: • **True** – параллельно; • **False** – последовательно. Значение по умолчанию **False** (флажок не установлен)
Конкурентные задания (IsCompetitive)	Признак, того, что в блоке будут создаваться конкурентные задания. Возможные значения: • **True** – создаются; • **False** – не создаются. Значение по умолчанию **False** (флажок не установлен)
Срок выполнения (RelativeDeadline)	Срок выполнения задания. При необходимости с помощью свойств RelativeDeadlineDays и RelativeDeadlineHours в программном коде можно указать срок выполнения в рабочих днях или в часах
Учитывать отсутствия (IsWithAbsences)	Признак того, что срок выполнения сдвигается на период отсутствия сотрудников. Возможные значения: • **True** – отсутствия учитываются; • **False** – не учитываются. Значение по умолчанию **False** (флажок не установлен). Если срок задается с помощью вычисляемого выражения, флажок скрывается. В этом случае для учета отсутствий необходимо использовать соответствующие функции
Срок прекращения (RelativeStopAssignmentsDeadline)	Срок принудительного прекращения задания. При необходимости с помощью свойств RelativeStopDeadlineDays и RelativeStopDeadlineHours в программном коде можно задать срок прекращения в днях или в часах
Представление по умолчанию (DefaultViewForm)	Представление формы, которое отображается при открытии карточек отдельно от списка. Если свойство не указано, подбирается наиболее подходящее представление
Представление в списке (EmbeddedViewForm)	Представление формы для карточки в списке. Если свойство не указано, отображается представление по умолчанию

Серверные события блока

Событие	Назначение	Аргументы события
Старт блока (<Имя блока>Start)	Заполнение свойств выполняемого блока. Событие выполняется на сервисе выполнения блоков схем задач (WorkflowBlockService) при старте обработки блока	**_obj** – задача, в рамках которой обрабатывается блок; **_block** – блок, в рамках которого выполняется событие
Старт задания (<Имя блока>Start<Тип задания>)	• заполнение свойств, которые должны иметь разные значения у разных заданий. Например, разные сроки выполнения задания у разных исполнителей в составном поручении;	**_obj** – задача, в рамках которой обрабатывается блок; **assignment** – задание, созданное в рамках блока; **_block** – блок, в рамках которого выполняется событие

Событие	Назначение	Аргументы события
	• заполнение свойств задания, которые нельзя заполнить через свойства блока. Например, свойства типа Текст, Картинка, Бинарные данные. Событие выполняется на сервисе выполнения блоков схем задач (WorkflowBlockService) до старта задания (после создания и заполнения свойств задания) и до сохранения в базу данных. Событие выполняется для каждого задания, созданного в рамках блока.	
Выполнение задания (<Имя блока>CompleteAssignment)	Актуализация свойств задачи или других заданий в зависимости от свойств выполненного задания. При выполнении каждого из заданий в рамках блока код обработчика события выполняет сервис выполнения блоков схем задач (WorkflowBlockService).	**_obj** – задача, в рамках которой обрабатывается блок; **assignment** – выполненное задание; **_block** – блок, в рамках которого выполняется событие
Конец блока (<Имя блока>End)	Актуализация свойств задачи при завершении всех заданий, созданных по блоку. Событие выполняется на сервисе выполнения блоков схем задач (WorkflowBlockService) при завершении обработки блока ПРИМЕЧАНИЕ. Изменение свойств объектов, заданное в веб-клиенте при завершении блока, происходит после выполнения события «Конец блока»	**_obj** – задача, в рамках которой обрабатывается блок; **_block** – блок, в рамках которого выполняется событие; **createdAssignments** – список заданий, созданных в рамках блока
Все серверные события блока		**_block.ProcessStagesDisplayMode** – режим отображения блока в этапах процесса. Возможные значения: • **Show** – пока не выполнен. Блок отображается, если он выполняется в текущий момент или будет выполняться в дальнейшем; • **ShowInWork** – в работе. Блок отображается, если он выполняется в текущий момент; • **Hide** – никогда. Блок скрыт. Изменять режим отображения рекомендуется в веб-клиенте через свойство блока **Отображать в этапах процесса**

Пример. Актуализация свойств задания в зависимости от результата его выполнения

```
   public virtual void
      MyAssignmentBlockCompleteAssignment(Sungero.Docflow.IMyAssignment assignment)
{
  // Переадресовать задание, если оно выполнено с результатом «Переадресовать».
  if (assignment.Result == Sungero.MyModule.MyAssignment.Result.Forward)
  {
```

```
    assignment.Forward(assignment.ForwardPerformer, ForwardingLocation.Next);
  }
  else if (assignment.Result == Sungero.MyModule.MyAssignment.Result.Approve)
  {
    // В список исполнителей добавить сотрудников, которые выполнили задание
    // результатом «Утверждено».
    _obj.ApprovedPerformers.Add(assignment.Performer)
  }
}
```

Клиентские события блока

Событие	Назначение	Аргументы события
Обновление блока (<Имя блока>RefreshBlock)	Задание логики для отображения свойств блока в зависимости от условий. Выполняется при открытии схемы процесса в редакторе и изменении свойств блока	**_block** – блок, в рамках которого выполняется событие **_block.<название свойства>.Value** – значение свойства; **_block.<название свойства>.IsEmpty()** – проверить, заполнено ли свойство; **_block.<название свойства>.IsExpression()** – проверить, является ли свойство вычисляемым выражением; **_block.State.Properties.<название свойства>.<состояние>** – состояние свойства. Возможные значения: • **isVisible** – видимое; • **isEnabled** – доступное; • **isRequired** – обязательное **_block.ProcessStagesDisplayMode** – режим отображения блока в этапах процесса. Возможные значения: • **Show** – пока не выполнен. Блок отображается, если он выполняется в текущий момент или будет выполняться в дальнейшем; • **ShowInWork** – в работе. Блок отображается, если он выполняется в текущий момент; • **Hide** – никогда. Блок скрыт. Изменять режим отображения рекомендуется в веб-клиенте через свойство блока **Отображать в этапах процесса**

Пример. Отображение выпадающего списка в зависимости от того, установлен ли флажок

```
public virtual void AssignmentBlockRefresh()
{
```

```
  // Если флажок PropertyBool установлен, то отобразить выпадающий список
PropertyEnum
  if (_block.PropertyBool.Value == true)
  {
    _block.State.Properties.PropertyEnum.IsVisible = true;
  }
  // Если флажок PropertyBool снят, то скрыть выпадающий список PropertyEnum
  if (_block.PropertyBool.Value == false)
  {
    _block.State.Properties.PropertyEnum.IsVisible = false;
  }
  Logger.Error("AssignmentBlockRefresh");
}
```

Блок «Уведомление»

«Уведомление» (NoticeBlock). Блок предназначен для создания уведомлений. После создания уведомлений выполняется выход из блока.

Свойства блока

Свойство	Описание
Заголовок (Title)	Отображается в схеме задачи
Описание (Description)	Поясняющий текст, который отображается на панели настроек блока на схеме варианта процесса. ПРИМЕЧАНИЕ. В прикладном коде свойство доступно только для чтения
Инициатор (Author)	Инициатор уведомления
Исполнители (Performers)	Список исполнителей. Для выбора доступны роли и наследники от групп. Sid (идентификатор субъекта прав) можно узнать в таблице Sungero_Core_Recipient
Тема (Subject)	Тема создаваемого уведомления
Тема в переписке (ThreadSubject)	Тема уведомления в переписке
Представление по умолчанию (DefaultViewForm)	Представление формы, которое отображается при открытии карточек отдельно от списка. Если свойство не указано, подбирается наиболее подходящее представление
Представление в списке (EmbeddedViewForm)	Представление формы для карточки в списке. Если свойство не указано, отображается представление по умолчанию

Серверные события блока

Событие	Назначение	Аргументы события
Старт блока (<Имя блока>Start)	Заполнение свойств выполняемого блока. Событие выполняется на веб-сервере при старте обработки блока	**_obj** – задача, в рамках которой обрабатывается блок. **_block** – блок, в рамках которого выполняется событие

Событие	Назначение	Аргументы события
Старт уведомления (<Имя блока>StartNotice)	Заполнение свойств уведомления, которые нельзя заполнить через свойства блока. Например, свойства типа Текст, Картинка, Бинарные данные. Событие выполняется на сервисе выполнения блоков схем задач (WorkflowBlockService) до старта уведомления (после создания и заполнения свойств уведомления) и до сохранения в базу данных	**_obj** – задача, в рамках которой обрабатывается блок. **notice** – уведомление, созданное в рамках блока. **_block** – блок, в рамках которого выполняется событие
Конец блока (<Имя блока>End)	Актуализация свойств задачи после создания уведомлений. Событие выполняется на сервисе выполнения блоков схем задач (WorkflowBlockService) при завершении обработки блока ПРИМЕЧАНИЕ. Изменение свойств объектов, заданное в веб-клиенте при завершении блока, происходит после выполнения события «Конец блока»	**_obj** – задача, в рамках которой обрабатывается блок. **_block** – блок, в рамках которого выполняется событие. **createdNotices** – уведомления, созданные в рамках блока
Все серверные события блока		**_block.ProcessStagesDisplayMode** – режим отображения блока в этапах процесса. Возможные значения: • **Show** – пока не выполнен. Блок отображается, если он выполняется в текущий момент или будет выполняться в дальнейшем; • **ShowInWork** – в работе. Блок отображается, если он выполняется в текущий момент; • **Hide** – никогда. Блок скрыт. Изменять режим отображения рекомендуется в веб-клиенте через свойство блока **Отображать в этапах процесса**

Пример. Назначение исполнителя текущей задачи получателем уведомления

```
   public virtual void MyNoticeBlockStart()
{
  // Назначить исполнителя текущей задачи получателем уведомления.
  _block.Performers.Add(_obj.Performer);
}
```

Клиентские события блока

Событие	Назначение	Аргументы события
Обновление блока (<Имя блока>RefreshBlock)	Задание логики для отображения свойств блока в зависимости от условий.	**_block** – блок, в рамках которого выполняется событие **_block.<название свойства>.Value** – значение свойства;

Событие	Назначение	Аргументы события
	Выполняется при открытии схемы процесса в редакторе и изменении свойств блока	**_block.<название свойства>.IsEmpty()** – проверить, заполнено ли свойство; **_block.<название свойства>.IsExpression()** – проверить, является ли свойство вычисляемым выражением; **_block.State.Properties.<название свойства>.<состояние>** – состояние свойства. Возможные значения: • **isVisible** – видимое; • **isEnabled** – доступное; • **isRequired** – обязательное **_block.ProcessStagesDisplayMode** – режим отображения блока в этапах процесса. Возможные значения: • **Show** – пока не выполнен. Блок отображается, если он выполняется в текущий момент или будет выполняться в дальнейшем; • **ShowInWork** – в работе. Блок отображается, если он выполняется в текущий момент; • **Hide** – никогда. Блок скрыт. Изменять режим отображения рекомендуется в веб-клиенте через свойство блока **Отображать в этапах процесса**

Пример. Отображение выпадающего списка в зависимости от того, установлен ли флажок

```
public virtual void AssignmentBlockRefresh()
{
  // Если флажок PropertyBool установлен, то отобразить выпадающий список
PropertyEnum
  if (_block.PropertyBool.Value == true)
  {
    _block.State.Properties.PropertyEnum.IsVisible = true;
  }
  // Если флажок PropertyBool снят, то скрыть выпадающий список PropertyEnum
  if (_block.PropertyBool.Value == false)
  {
    _block.State.Properties.PropertyEnum.IsVisible = false;
  }
  Logger.Error("AssignmentBlockRefresh");
}
```

Блок «Мониторинг»

«Мониторинг» (MonitoringBlock). Блок предназначен для контроля наступления определенного события. Например, блок добавляют, чтобы отправить уведомление, когда у документа изменится свойство.

Свойства блока

Свойство	Описание
Заголовок (Title)	Отображается в схеме задачи
Описание (Description)	Поясняющий текст, который отображается на панели настроек блока на схеме варианта процесса. ПРИМЕЧАНИЕ. В прикладном коде свойство доступно только для чтения
Период (Period)	Интервал времени, через который выполняется мониторинг
Срок прекращения (Deadline)	Срок принудительного завершения мониторинга относительно даты и времени старта

Выходные свойства блока

Свойство	Описание
Результат выполнения (ExecutionResult)	Результаты выполнения блока. Значения по умолчанию: **Success** (Успешно), **Deadline** (Срок истек). **<Результат выполнения>** – произвольный результат, добавленный разработчиком

Серверные события блока

Событие	Назначение	Аргументы события	Возвращаемое значение
Старт блока (<Имя блока>Start)	Заполнение свойств блока. Событие выполняется на сервисе выполнения блоков схем задач (WorkflowBlockService) при старте обработки блока	**_obj** – задача, в рамках которой обрабатывается блок. **_block** – блок, в рамках которого выполняется событие	
Результат мониторинга (<Имя блока>Result)	Определение завершения мониторинга.	**_obj** – задача, в рамках которой обрабатывается блок	Результат мониторинга. Возможные значения: **True**. Ожидаемое событие наступило, переход по стрелке **Успешно**. Мониторинг завершится. **False**. Ожидаемое событие не наступило. В этом случае проверяется свойство **Срок прекращения** (RelativeDeadline):

Событие	Назначение	Аргументы события	Возвращаемое значение
	Событие выполняется на сервисе выполнения блоков схем задач (WorkflowBlockService) при каждом выполнении мониторинга		• если время, указанное в свойстве, еще не наступило, то вычисляется, что ближе по времени: следующий период мониторинга или срок прекращения. Сообщение с этим временем отправляется в DelayOperationService. Событие будет запущено еще раз через заданный период, при наступлении срока прекращения или при вызове события из кода; • если время, указанное в свойстве, наступило, то происходит переход по стрелке **Срок истек**. Мониторинг завершится
Конец блока (<Имя блока>End)	Асинхронное выполнение операции при завершении блока. Событие выполняется на сервисе выполнения блоков схем задач (WorkflowBlockService) при завершении обработки блока ПРИМЕЧАНИЕ. Изменение свойств объектов, заданное в веб-клиенте при завершении блока, происходит после выполнения события «Конец блока»	**_obj** – задача, в рамках которой обрабатывается блок. **_block** – блок, в рамках которого выполняется событие	
Все серверные события блока		**_block.ProcessStagesDisplayMode** – режим отображения блока в этапах процесса. Возможные значения: • **Show** – пока не выполнен. Блок отображается, если он выполняется в текущий момент или будет выполняться в дальнейшем; • **ShowInWork** – в работе. Блок отображается, если он выполняется в текущий момент; • **Hide** – никогда. Блок скрыт.	

Событие	Назначение	Аргументы события	Возвращаемое значение
		Изменять режим отображения рекомендуется в веб-клиенте через свойство блока **Отображать в этапах процесса**	

Пример. Результат мониторинга в зависимости от выполнения всех заданий по текущей задаче

```
   // Результат мониторинга. Если все задания по текущей задаче выполнены, то
      // мониторинг завершится с результатом true, иначе – false.
public virtual bool MyMonitoringBlockResult()
{
  // Поиск заданий по текущей задаче.
  var assignments = Sungero.Workflow.Assignments.GetAll()
  .Where(a => a.MainTask == _obj.Id);
  // Если все задания выполнены, создать уведомление автору задачи.
  if (assignments.All(a => a.Status ==
  Sungero.Workflow.Assignment.Status.Completed))
  {
    var notice = Sungero.Workflow.Notices.Create(_obj);
    notice.Performer = _obj.Author;
    notice.Subject = _block.Subjects;
    notice.Task = _obj;
     // Ожидаемое событие наступило. Завершение мониторинга.
    return true;
  }
  // Ожидаемое событие не наступило. Завершение мониторинга.
  return false;
}
```

Клиентские события блока

Событие	Назначение	Аргументы события
Обновление блока (<Имя блока>RefreshBlock)	Задание логики для отображения свойств блока в зависимости от условий. Выполняется при открытии схемы процесса в редакторе и изменении свойств блока	**_block** – блок, в рамках которого выполняется событие. **_block.<название свойства>.Value** – значение свойства. **_block.<название свойства>.IsEmpty()** – проверить, заполнено ли свойство. **_block.<название свойства>.IsExpression()** – проверить, является ли свойство вычисляемым выражением. **_block.State.Properties.<название свойства>.<состояние>** – состояние свойства. Возможные значения: • **isVisible** – видимое; • **isEnabled** – доступное;

Событие	Назначение	Аргументы события
		• **isRequired** – обязательное **_block.ProcessStagesDisplayMode** – режим отображения блока в этапах процесса. Возможные значения: • **Show** – пока не выполнен. Блок отображается, если он выполняется в текущий момент или будет выполняться в дальнейшем; • **ShowInWork** – в работе. Блок отображается, если он выполняется в текущий момент; • **Hide** – никогда. Блок скрыт. Изменять режим отображения рекомендуется в веб-клиенте через свойство блока **Отображать в этапах процесса**

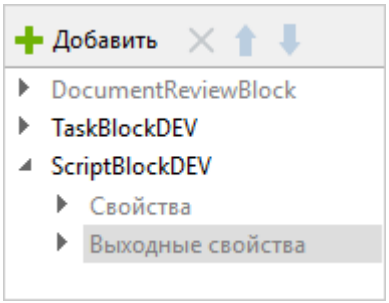
Пример. Отображение выпадающего списка в зависимости от того, установлен ли флажок

```
public virtual void AssignmentBlockRefresh()
{
  // Если флажок PropertyBool установлен, то отобразить выпадающий список
PropertyEnum
  if (_block.PropertyBool.Value == true)
  {
    _block.State.Properties.PropertyEnum.IsVisible = true;
  }
  // Если флажок PropertyBool снят, то скрыть выпадающий список PropertyEnum
  if (_block.PropertyBool.Value == false)
  {
    _block.State.Properties.PropertyEnum.IsVisible = false;
  }
  Logger.Error("AssignmentBlockRefresh");
}
```

Выходные свойства

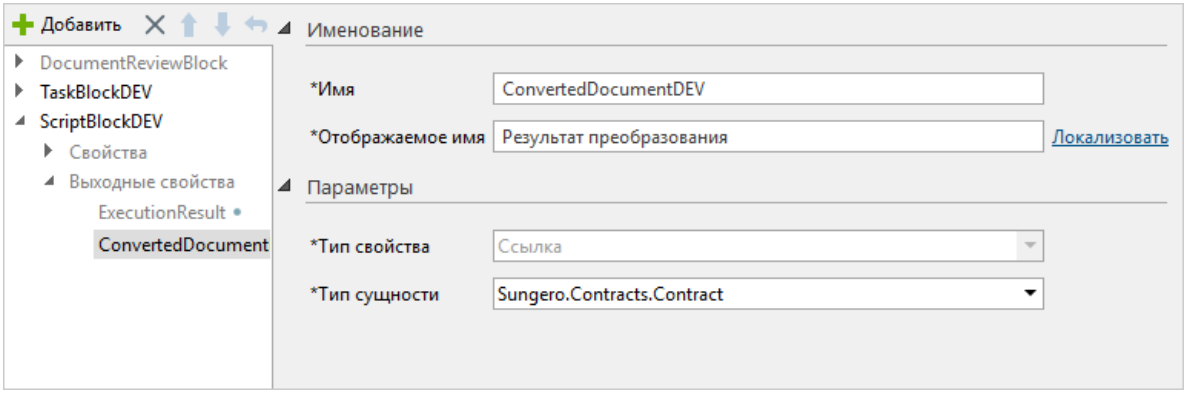
Для передачи данных из событий блока в параметры процесса можно добавить выходные свойства. Например, в них можно записывать результат преобразования приказа в формат PDF: ссылку на документ или текст ошибки. Для этого:

1. В редакторе блока списке элементов выделите узел **Выходные свойства** и нажмите на кнопку **Добавить**:



Доступные типы выходных свойств: строка, целое, логическое, вещественное, дата и время, ссылка, коллекция ссылок.

2. В списке элементов выделите выходное свойство блока и задайте настройки в зависимости от его типа:



***Имя**. Должно быть уникальным в рамках одного модуля, может состоять из букв латинского алфавита и цифр.

***Отображаемое имя**. Имя выходного свойства, которое отображается в редакторе схемы. Для локализации отображаемого имени перейдите по ссылке **Локализовать**.

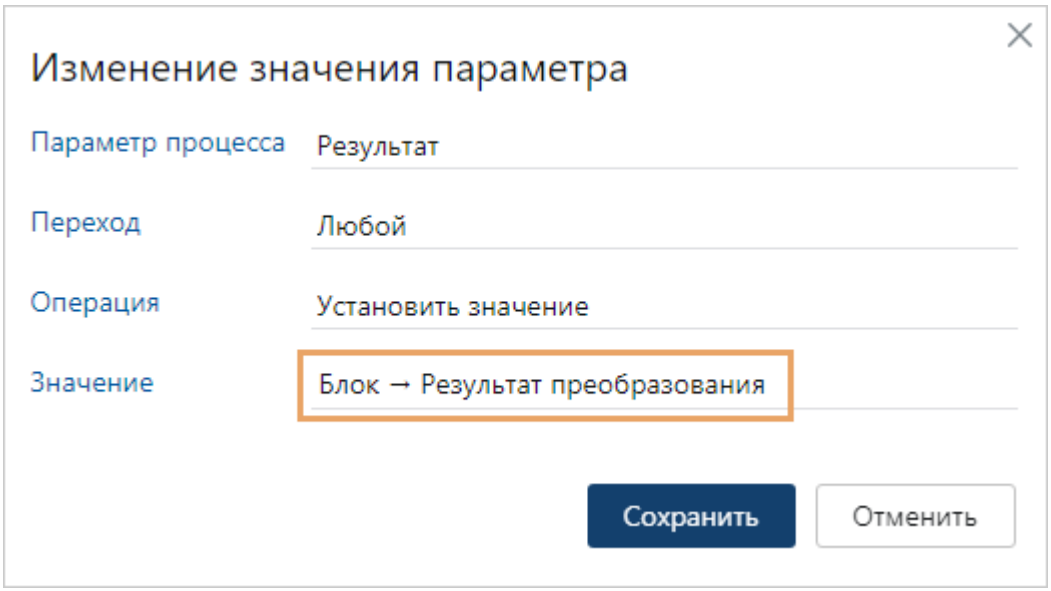
***Тип сущности**. Сущности указанного типа будут значениями для выходного свойства типа «ссылка» или «коллекция ссылок».

3. Напишите код в событии блока. Для этого используется свойство **OutProperties** переменной **_block**:

Пример. Преобразовать документ в PDF и записать результат в выходное свойство

```
public virtual void ScriptBlockExecute()
{
  var document = Sungero.Docflow.OfficialDocuments.GetAll(x => x.Id == 1);
  try
  {
    var convertedDocument = document.ConvertToPDF();
    // Записать в выходное свойство ссылку на преобразованный документ.
    _block.OutProperties.ConvertedDocument = convertedDocument;
  }
  catch
  {
    // Записать в выходное свойство текст ошибки.
    _block.OutProperties.ErrorMessage = "Не удалось выполнить преобразование
в PDF";
  }
}
```

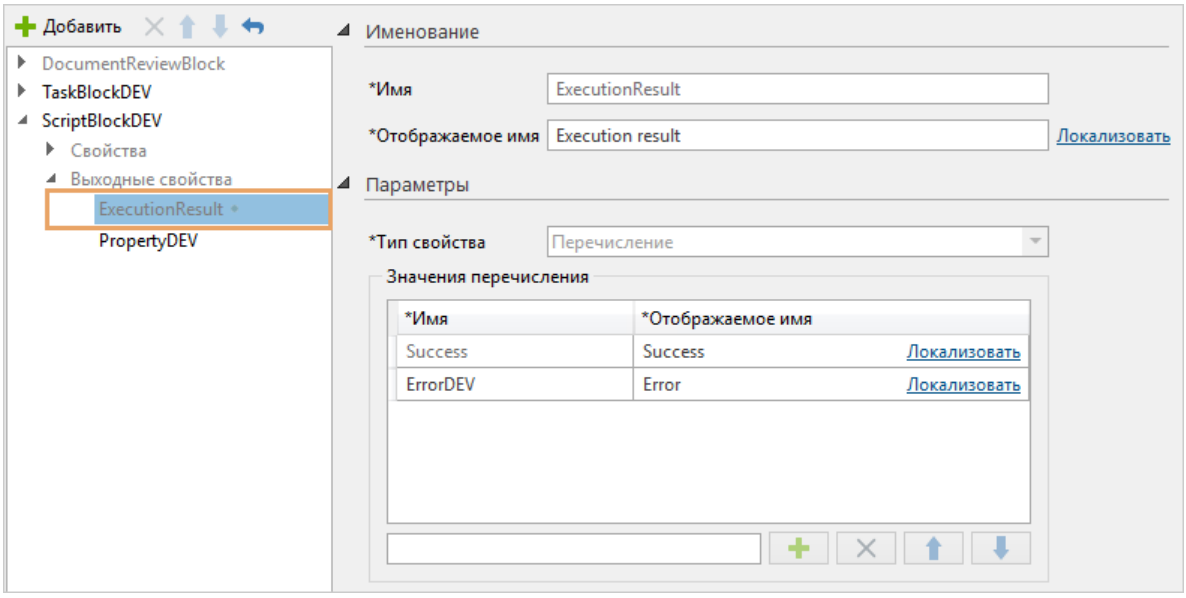
К выходным свойствам можно обратиться через вычисляемое выражение при настройке схемы процесса. Для этого в параметрах процесса при завершении блока, в вычисляемом выражении укажите элемент **Блок** и выберите нужное свойство:



Результаты выполнения в блоках «Скрипт» и «Мониторинг»

В блоках «Скрипт» или «Мониторинг» можно добавить дополнительные результаты выполнения, чтобы настроить переход по разным веткам схемы. Для этого:

1. В списке элементов блока перейдите в узел **Выходные свойства**, **ExecutionResult** и в параметрах добавьте новые результаты в виде значений перечисления. Например, для преобразования в PDF добавьте результат с ошибкой (**Error**):



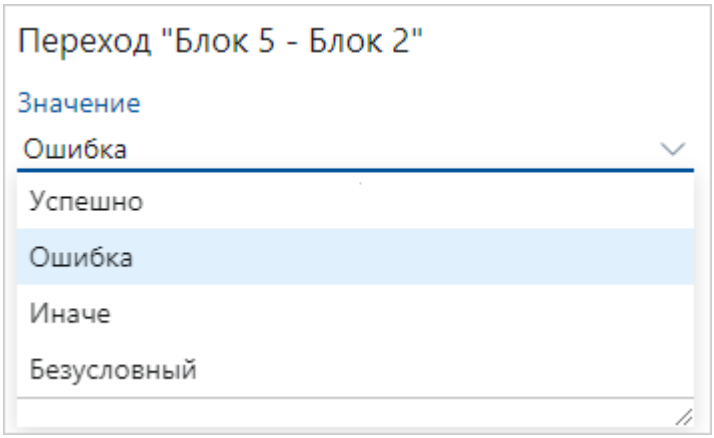
2. Для результатов напишите код в событии блока. Для этого используется свойство **OutProperties.ExecutionResult** переменной **_block**.

   Пример. Переход в зависимости от результата преобразования документа в PDF

```
public virtual void ScriptBlockExecute()
{
  var document = Sungero.Docflow.OfficialDocuments.GetAll(x => x.Id == 1);
  // Преобразовать документ в формат PDF.
```

```
    try
    {
      var convertedDocument = document.ConvertToPDF();
      // Записать результат выполнения "Успешно".
      _block.OutProperties.ExecutionResult = ExecutionResult.Success;
    }
    catch
    {
      // Записать результат выполнения "Ошибка".
      _block.OutProperties.ExecutionResult = ExecutionResult.Error;
    }
  }
```

Новые результаты выполнения отображаются в выпадающем списке при выборе переходов:



Настройка схемы задачи

Схема задачи – последовательность этапов работ, которая описывает бизнес-процессы компании. Согласно схеме в рамках задачи с помощью сервисов WorkflowProcessService и WorkflowBlockService могут создаваться другие задачи, задания, уведомления и задания на приемку. Для каждого типа задачи разрабатывается своя схема.

Для разработки схемы используется графический редактор. С его помощью вы можете перемещать *блоки* на схеме, задавать параметры, обработчики событий блоков и сразу видеть результат настройки. Чтобы открыть редактор схем, в редакторе типа задачи перейдите в узел «Схема».

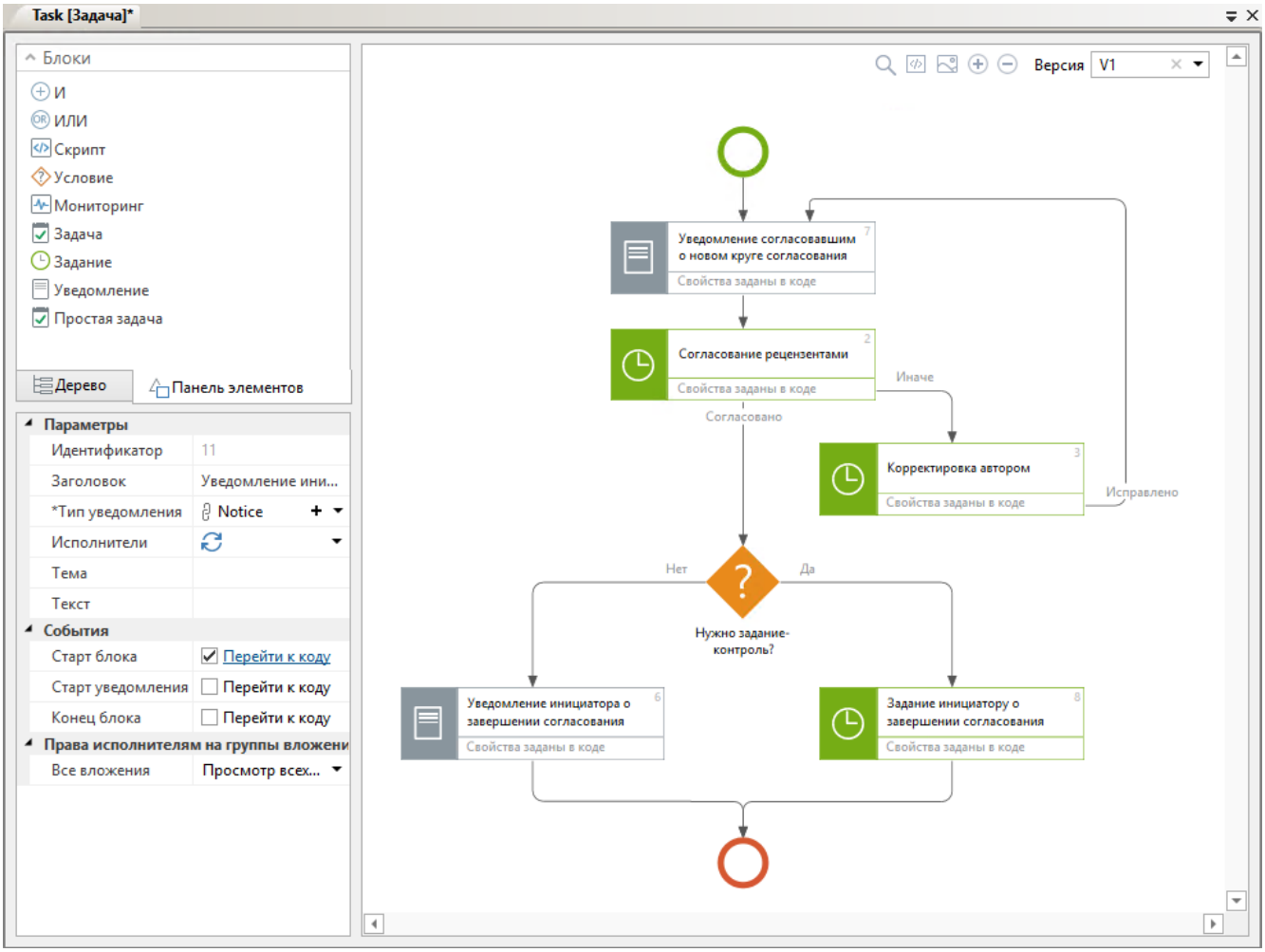


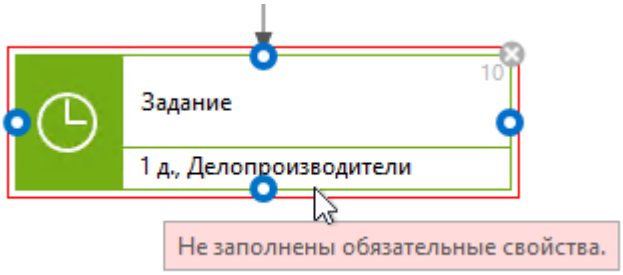
Схема размещается в основной области редактора, состоит из блоков и переходов между блоками. В левой части редактора расположены «Панель элементов», на которой находятся блоки для перемещения на схему, и «Панель свойств», на которой задаются свойства и обработчики событий блоков.

В правом верхнем углу редактора доступны кнопки для поиска блока по заголовку и идентификатору, перехода к коду обработчиков событий блоков, экспорта схемы в изображение, открытия схемы в программе для просмотра изображений и кнопки для изменения масштаба. Также в редакторе отображается контрол для работы с версиями схемы. Подробнее см. раздел «Изменение схемы».

На схеме у каждого блока отображается идентификатор и заголовок. В зависимости от типа блока могут отображаться дополнительные свойства, заданные на панели свойств. Например, срок, исполнители или количество этапов маршрута. Подробнее см. описание блоков схемы.

Чтобы посмотреть или изменить свойства добавленного блока, выберите его на схеме. Блок выделится синей рамкой.

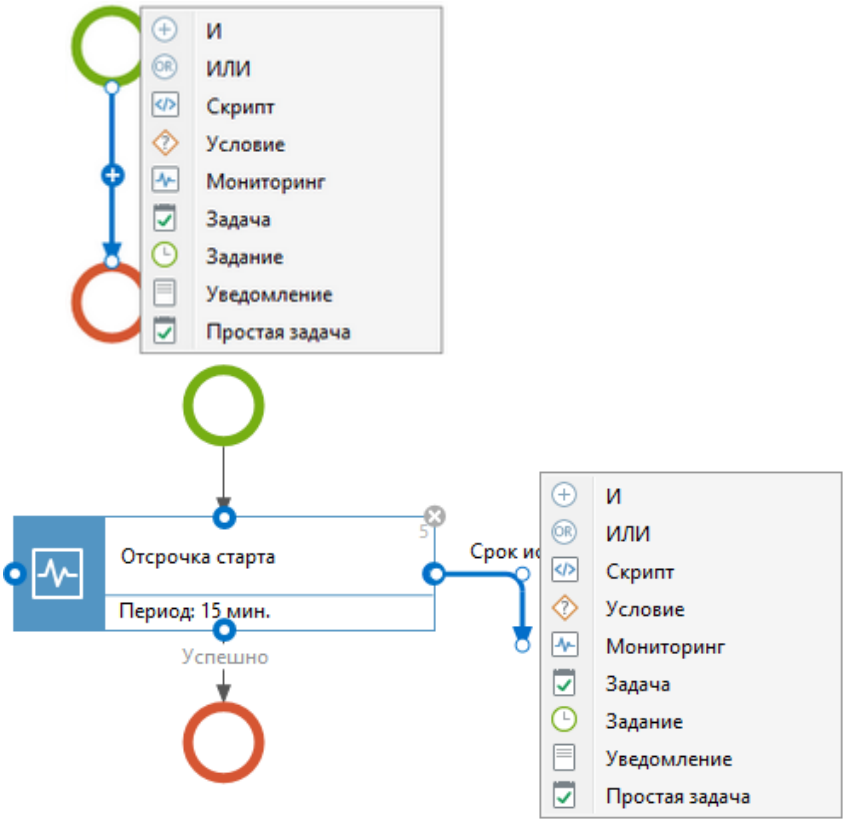
При добавлении блока и сохранении схемы выполняется валидация. Например, если для блока не заданы свойства или неверно заданы стрелки, то блок выделится красной рамкой. При наведении курсора на блок появится сообщение с описанием проблемы.



Для корректной настройки схемы используйте рекомендации.

Создание схемы

1. Добавьте блок нажав на значок в стрелке или нажав на значок из другого блока (ведите из блока стрелку до нужного места). В открывшемся меню выберите блок.

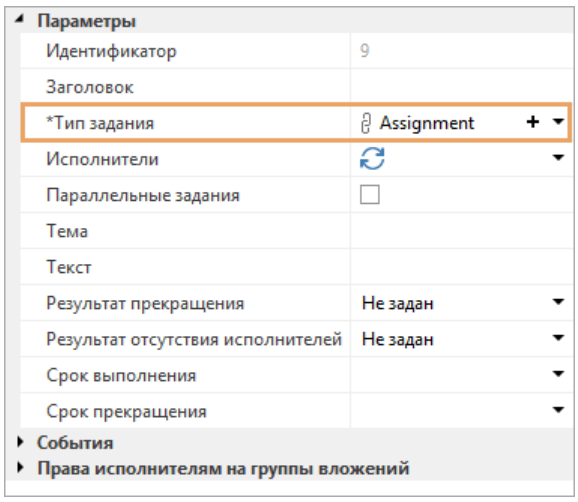


Либо перетащите нужный блок на схему с панели элементов.

2. Задайте заголовок блока. Заголовок отображается в схеме задачи и лог-файлах сервиса WorkflowBlockService при обработке блока.

   Если в дальнейшем понадобится изменить заголовок, то щелкните два раза левой кнопкой мыши в области заголовка и измените его.

3. Для блока «Задание» в свойствах укажите ***Тип задания**.



Если в выпадающем списке нет подходящего типа задания, создайте его. Для этого нажмите на кнопку **+ Добавить тип**.

Чтобы быстро перейти к редактору выбранного типа задания, нажмите на кнопку **Перейти к типу**.

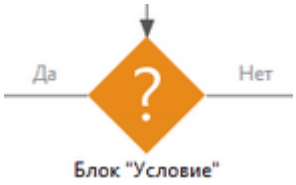
Аналогично укажите тип уведомления для блока «Уведомление».

4. Если на схему добавлены блоки «Условие», «Мониторинг» и «Задание», то укажите для них результаты выполнения. Для этого нажмите на стрелку и в выпадающем списке выберите нужное значение. Если значение уже указано, то нажмите на него, откроется выпадающий список с другими значениями.

   Для каждого блока выбираются свои результаты выполнения:

   «Мониторинг». Возможные значения: **Успешно** – блок выполнен; **Срок истек** – выполнение блока прервано (возможно при заполнении свойства блока RelativeDeadline):

   «Условие». Возможные значения: **Да** – условие истинно, **Нет** – условие ложно:

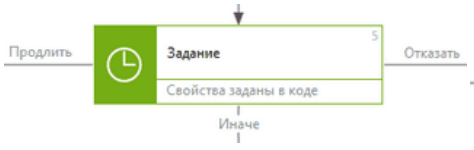
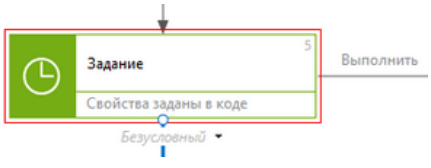


   «Задание». Возможные значения:

   - один из результатов выполнения блока. Переход из блока выполнится, когда все задания выполнятся с данным результатом.

     Результаты выполнения задаются в редакторе типа задания в узле «Результаты выполнения»;
   - **Иначе**. Переход из блока выполнится, если не сработали остальные переходы по результатам выполнения, задания выполнены с разными результатами или результат выполнения совпадает со свойством блока **Результаты прекращения** (StopResults);
   - **Безусловный**. Независимо от результатов выполнения блок всегда пойдет по данному переходу.

   ВАЖНО. Из одного блока «Задание» не могут выходить одновременно условные и безусловные переходы:

Блок с условными переходами	Блок с безусловными переходами	Неверное указание результатов
		

5. Настройте свойства для добавленных блоков. Свойства задаются на панели свойств или в коде обработчиков событий. Подробнее см. описание блоков схемы.

   ВАЖНО. Свойства **Наблюдатели** (Observers) и **Исполнители** (Performers) рекомендуется задавать в программном коде. Используйте для заполнения панель свойств, только если Sid наблюдателей и исполнителей в разрабатываемой системе совпадает с Sid в продуктовой системе.

6. Добавьте переходы между блоками, если их нет. Для этого наведите курсор мыши на блок и из значка ведите стрелку до другого блока.

7. Добавьте обработчики событий блоков. Для этого на панели свойств установите флажок рядом с названием события и перейдите по ссылке **Перейти к коду**. Откроется редактор программного кода.

   События блоков выполняются на сервисе WorkflowBlockService от имени служебного пользователя. Если при выполнении события сущность, которую нужно изменить, заблокирована, то выполнение обработчика будет отменено. Через некоторое время обработчик попробует снова внести изменения.

   ВАЖНО. Передавать какие-либо данные между блоками рекомендуется через свойства задачи или задания (уведомления).

   Для блока «Задание» необходимо вычислить исполнителя и выдать права на чтение документа. Для этого добавьте код в обработчик события **Старт блока**:

```
// Задание на возврат документа.
    public static void
    StartBlock2(Sungero.Docflow.Server.CheckReturnAssignmentArguments e)
    {
        // Вычисление исполнителя задания. Определяется через свойство
    Assignee
        // в типе задачи CheckReturnTask.
        e.Block.Performers.Add(_obj.Assignee);

        // Получение первого документа в группе вложений,
        // на который будут выданы права доступа.
        var document = _obj.DocumentGroup.OfficialDocuments.First();

        // Назначение исполнителю прав на чтение документа.
        document.AccessRights.Grant(_obj.Assignee,
    DefaultAccessRightsTypes.Read);
    }
```

8. При необходимости выполните отладку кода обработчиков.

9. Сохраните изменения.

Чтобы удалить блок или стрелку, выделите элемент на схеме и нажмите на клавишу DELETE или нажмите на значок на блоке.

Блоки схемы

Схема задачи может содержать следующие блоки:

- Начало процесса (StartBlock)
- Конец процесса (FinishBlock)
- И (AndBlock)

- ИЛИ (OrBlock)
- Скрипт (ScriptBlock)
- Условие (DecisionBlock)
- Мониторинг (MonitoringBlock)
- Задача (TaskBlock)
- Задание (AssignmentBlock)
- Уведомление (NoticeBlock)
- Простая задача (SimpleTaskBlock)
- Задание на приемку

У каждого блока есть набор свойств и событий. Свойства задаются на панели свойств в редакторе схем или в коде обработчиков событий блоков.

Начало процесса, Конец процесса, И, ИЛИ

«Начало процесса» (StartBlock). Блок предназначен для указания начала выполнения схемы задачи. Добавляется на схему автоматически при создании типа задачи. Используется в схеме только один раз и не может быть удален. Выполняется при старте задачи. Не может иметь входящие переходы.

«Конец процесса» (FinishBlock). Блок предназначен для указания завершения выполнения схемы задачи. Добавляется на схему автоматически при создании типа задачи. Используется в схеме только один раз и не может быть удален. Выполняется при завершении задачи. Не может иметь исходящие переходы.

Все переходы в схеме должны сводиться к данному блоку. Исключением являются «тупиковые ветки», которые выходят из блока «Начало процесса». Подробнее см. рекомендации по настройке.

ВАЖНО. При выполнении блока «Конец процесса» все незавершенные задания и ветки схемы задачи будут прекращены.

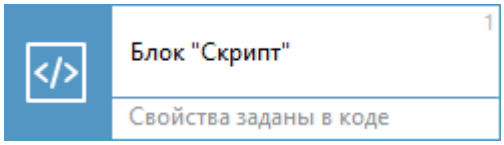
«И» (AndBlock). Блок предназначен для объединения нескольких переходов через логическое условие «И». Выполняется только если выполнены все блоки, от которых идут переходы в текущий блок.

«ИЛИ» (OrBlock). Блок предназначен для объединения нескольких переходов через логическое условие «ИЛИ». Выполняется, если выполнен хотя бы один из блоков, от которых идут переходы в текущий блок. Все задания, созданные в рамках других переходов, будут прекращены.

Свойства блоков

Свойство	Описание
Идентификатор (Id)	Присваивается блоку автоматически при создании. Свойство доступно только для просмотра

Скрипт



«Скрипт» (ScriptBlock). Блок предназначен для выполнения произвольного программного кода.

Свойства блока

Свойство	Описание
Наследуемые от BlockBase	
Идентификатор (Id)	Присваивается блоку автоматически при создании. Свойство доступно только для просмотра. Отображается в лог-файлах сервиса WorkflowBlockService при обработке блока
Заголовок (Title)	Отображается в схеме задачи

События блока

Событие	Назначение	Аргументы события
Выполнение (Script<ИД Блока>Execute)	Выполнение произвольного программного кода. Событие выполняется на сервисе выполнения блоков схем задач (WorkflowBlockService) при обработке блока	**_obj** – задача, в рамках которой обрабатывается блок

Условие

«Условие» (DecisionBlock). Блок предназначен для добавления условных ветвлений на схему.

Свойства блока

Свойство	Описание
Наследуемые от BlockBase	
Идентификатор (Id)	Присваивается блоку автоматически при создании. Свойство доступно только для просмотра. Отображается в лог-файлах сервиса WorkflowBlockService при обработке блока
Заголовок (Title)	Отображается в схеме задачи

События блока

Событие	Назначение	Аргументы события
Результат условия (Decision<ИД Блока>Result)	Расчет условия для выполнения перехода из блока. Возвращаемые значения: • **True** – условие выполнено, переход по стрелке **Да**; • **False** – условие не выполнено, переход по стрелке **Нет**. Событие выполняется на сервисе выполнения блоков схем задач (WorkflowBlockService) при обработке блока	**_obj** – задача, в рамках которой обрабатывается блок

Мониторинг

«Мониторинг» (MonitoringBlock). Блок предназначен для контроля наступления определенного события. Например, блок добавляют, чтобы отправить уведомление, когда у документа изменится свойство.

На схеме в блоке отображается период мониторинга, если он указан на панели свойств.

Свойства блока

Свойство	Описание
Период (Period)	Интервал времени, через который выполняется мониторинг
Срок прекращения (RelativeDeadline)	Срок принудительного завершения мониторинга относительно даты и времени старта
Наследуемые от BlockBase	
Идентификатор (Id)	Присваивается блоку автоматически при создании. Свойство доступно только для просмотра. Отображается в лог-файлах сервиса выполнения блоков схем задач (WorkflowBlockService) при обработке блока
Заголовок (Title)	Отображается в схеме задачи

События блока

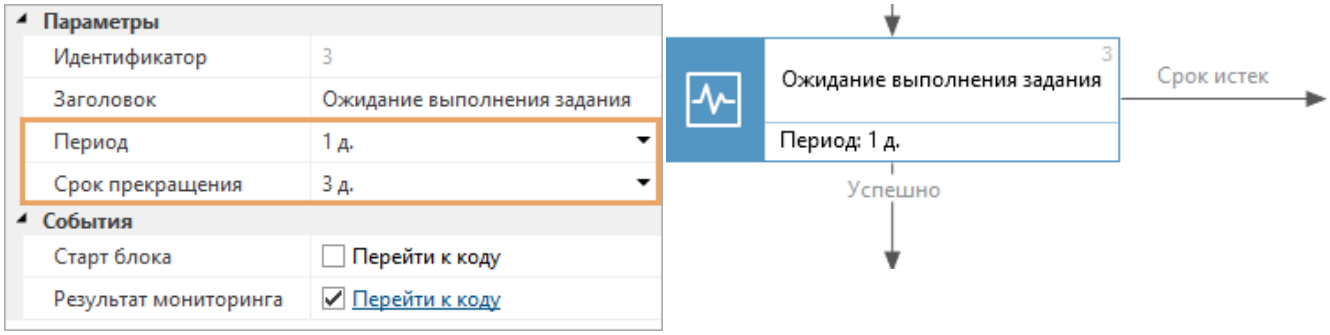
Событие	Назначение	Аргументы события	Возвращаемое значение
Старт блока (StartBlock<ИД блока>)	Заполнение свойств блока.	**_obj** – задача, в рамках которой обрабатывается блок;	

Событие	Назначение	Аргументы события	Возвращаемое значение
	Событие выполняется на сервисе выполнения блоков схем задач (WorkflowBlockService) при старте обработки блока	e.Block – блок, в рамках которого выполняется событие	
Результат мониторинга (Monitoring<ИД блока>Result)	Определение завершения мониторинга. Событие выполняется на сервисе выполнения блоков схем задач (WorkflowBlockService) при каждом выполнении мониторинга	**_obj** – задача, в рамках которой обрабатывается блок	Результат мониторинга. Возможные значения: **True**. Ожидаемое событие наступило, переход по стрелке **Успешно**. Мониторинг завершится. **False**. Ожидаемое событие не наступило. В этом случае проверяется свойство **Срок прекращения** (RelativeDeadline): • если время, указанное в свойстве, еще не наступило, то вычисляется, что ближе по времени: следующий период мониторинга или срок прекращения. Сообщение с этим временем отправляется в DelayedOperationsService. Событие будет запущено еще раз через заданный период, при наступлении срока прекращения или при вызове события из кода; • если время, указанное в свойстве, наступило, то происходит переход по стрелке **Срок истек**. Мониторинг завершится

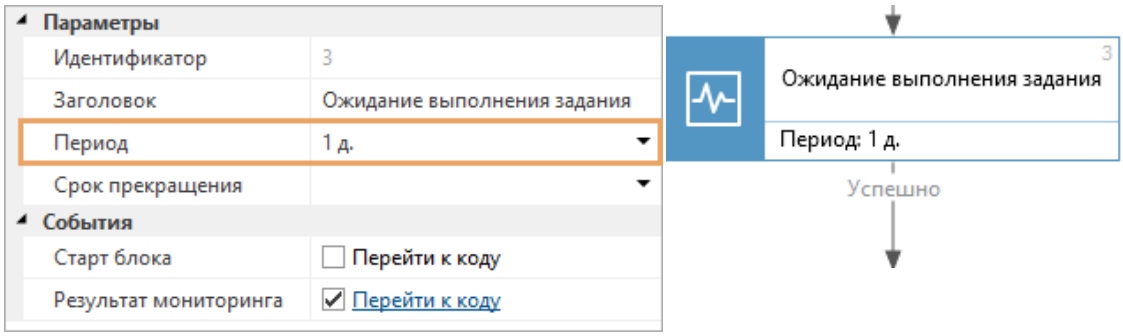
Принцип работы блока

На работу блока «Мониторинг» влияет то, как заполнены свойства **Период** и **Срок прекращения**:

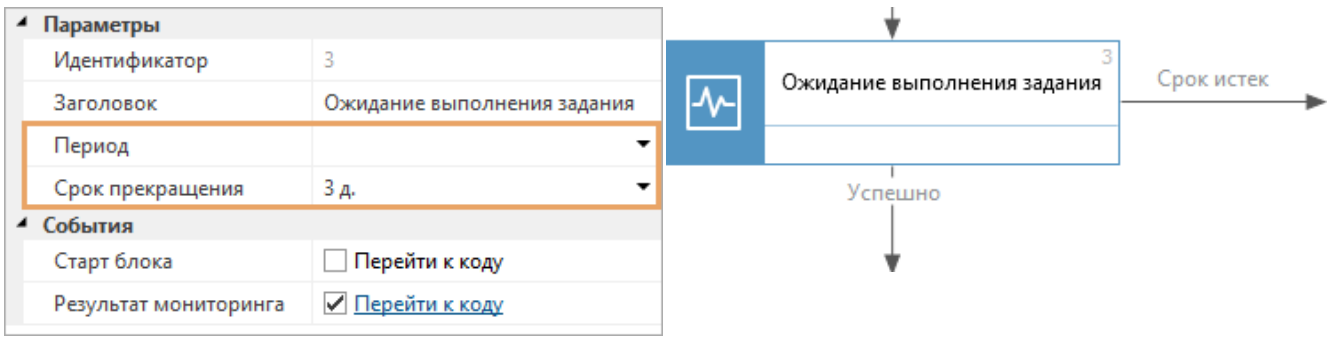
- если оба свойства **Период** и **Срок прекращения** заполнены, мониторинг выполняется через равные промежутки времени и прекращается по истечении указанного срока. Например, каждый день проверяется, наступило ли событие. Если да, то происходит выход из блока и процесс идет по ветке **Успешно**. Если нет, ожидается следующий период проверки. Через 3 дня, когда подошел срок прекращения, опять проверяется, наступило ли событие. Если да, то процесс идет по ветке **Успешно**. Если нет, то по ветке **Срок истек**. То есть, когда наступает срок прекращения, выход из блока происходит в любом случае:



- если заполнено только свойство **Период**, мониторинг выполняется через указанный период времени, например каждый день. Выход из блока происходит, если событие наступило, то есть по ветке **Успешно**. Мониторинг повторяется до тех пор, пока не наступит событие:



- если заполнено только свойство **Срок прекращения**, мониторинг выполняется один раз. Используйте этот вариант, если нужна пауза в бизнес-процессе. Например, сотруднику отправлено поручение. Нужно подождать 3 дня. Только после этого, если поручение выполнено, отправить контролеру уведомление (выход по ветке **Успешно**), а если не выполнено – задачу (выход по ветке **Срок истек**):



- если свойства **Период** и **Срок прекращения** не указаны, мониторинг выполняется только, если из кода вызвано событие блока «Результат мониторинга».

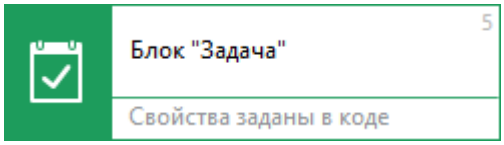
Как вызвать выполнение блока из кода

Вызвать из кода выполнение события «Результат мониторинга» можно при любом варианте заполнения свойств. Например, у блока «Мониторинг» заполнен **Период** и **Срок прекращения** проверки. Можно не дожидаться, когда наступит указное время, а запустить проверку принудительно. Для этого в серверном или разделяемом коде достаточно вызвать метод:

- **Sungero.Workflow.Blocks.Block<Идентификатор блока мониторинг>**.Execute(), чтобы выполнить событие конкретного блока;
- ExecuteAllMonitoringBlocks(), чтобы выполнить события всех активных блоков «Мониторинг» в схеме задачи.

Если при вызове метода событие наступило, то процесс идет по ветке **Успешно**, а если нет, то блок ожидает, когда наступит период проверки, срок прекращения проверки или следующий вызов метода из кода.

Задача



«Задача» (TaskBlock). Блок предназначен для создания задачи или подзадачи определенного типа в рамках какого-либо процесса. При обработке блока создается и отправляется задача, после чего выполняется выход из блока. Ожидания завершения задачи в этом блоке не происходит.

Свойства блока

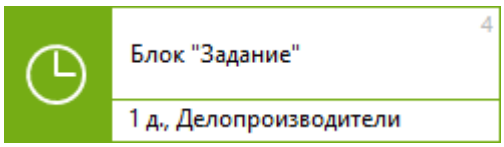
Свойство	Описание
Тип задачи	Имя типа задачи
Наблюдатели (Observers)	Список наблюдателей задачи. Для выбора доступны роли и наследники от групп. При заполнении наблюдателей через панель свойств данные берутся с локального веб-сервера. ВАЖНО. Свойство рекомендуется задавать в программном коде. Используйте для заполнения панель свойств, только если Sid наблюдателей в разрабатываемой системе совпадает с Sid в продуктовой системе. Sid (идентификатор субъекта прав) можно узнать в таблице Sungero_Core_Recipient
Подзадача (IsSubTask)	Признак того, что создается подзадача. Возможные значения: **True**, **False**. Значение по умолчанию **False** (флажок не установлен)
Задание на приемку (NeedsReview)	Признак того, что после выполнения всех блоков схемы задачи нужно создать задание на приемку. Возможные значения: **True**, **False**. Значение по умолчанию **True** (флажок установлен)
Тема (Subject)	Тема задачи, которую видят пользователи в системе

Свойство	Описание
	ВАЖНО. Свойство обязательно для заполнения. Задайте значение на панели свойств или в программном коде. Если свойство не задано, то схема выполнится с ошибкой
Текст (Text)	Текст задачи, который видят пользователи в системе
Важность (Importance)	Важность задачи. Возможные значения: **Нормальная**, **Высокая**
Наследуемые от DatabookEntry	
Идентификатор (Id)	Присваивается блоку автоматически при создании. Свойство доступно только для просмотра. Отображается в лог-файлах сервиса WorkflowBlockService при обработке блока
Заголовок (Title)	Заголовок блока, который видит разработчик в схеме задачи

События блока

Событие	Назначение	Аргументы события
Старт блока (StartBlock<ИД блока>)	Заполнение свойств выполняемого блока. Событие выполняется на сервисе выполнения блоков схем задач (WorkflowBlockService) при старте обработки блока	**_obj** – задача, в рамках которой обрабатывается блок; e.Block – блок, в рамках которого выполняется событие
Старт задачи (StartTask<ИД блока>)	Заполнение свойств задачи, которые нельзя заполнить через свойства блока. Например, свойства типа Текст, Картинка, Бинарные данные. Событие выполняется на сервисе выполнения блоков схем задач (WorkflowBlockService) до старта задачи (после создания и заполнения свойств задачи) и до сохранения в базу данных	**_obj** – задача, в рамках которой обрабатывается блок; **createdTask** – задача, созданная в рамках блока; e.Block – блок, в рамках которого выполняется событие
Конец блока (EndBlock<ИД блока>)	Актуализация свойств задачи после выполнения блока. Событие выполняется на сервисе выполнения блоков схем задач (WorkflowBlockService) при завершении обработки блока	**_obj** – задача, в рамках которой обрабатывается блок; e.Block – блок, в рамках которого выполняется событие; **e.CreatedTask** – задача, созданная в рамках блока

Задание



«Задание» (AssignmentBlock). Блок предназначен для создания заданий. Выход из блока происходит при выполнении или прекращении всех заданий, созданных в рамках блока.

На схеме в блоке отображается срок выполнения задания и исполнители, если они заданы на панели свойств.

Свойства блока

Свойство	Описание
Тип задания	Имя типа задания. В списке доступны все типы заданий, созданные в модуле и в задаче
Исполнители (Performers)	Список исполнителей. Для выбора доступны роли и наследники от групп. При заполнении наблюдателей через панель свойств данные берутся с локального веб-сервера. ВАЖНО. Свойство рекомендуется задавать в программном коде. Используйте для заполнения панель свойств, только если Sid исполнителей в разрабатываемой системе совпадает с Sid в продуктовой системе. Sid (идентификатор субъекта прав) можно узнать в таблице Sungero_Core_Recipient
Параллельные задания (IsParallel)	Признак,того, что задания по блоку будут создаваться одновременно или друг за другом. Возможные значения: • **True** – параллельно; • **False –** последовательно. Значение по умолчанию **False** (флажок не установлен)
Конкурентные задания (IsCompetitive)	Признак того, что в блоке будут создаваться конкурентные задания. Возможные значения: • **True** – создаются; • **False** – не создаются. Значение по умолчанию **False** ВАЖНО. Свойство можно использовать только для работы с блоками бизнес-процессов. Изменять свойство для блоков схем задач не рекомендуется, иначе задача прервется с ошибкой
Тема (Subject)	Тема задания, которую видят пользователи в системе
Текст (Text)	Текст задания, который видят пользователи в системе
Результаты прекращения (StopResults)	Результаты выполнения задания, при которых прекращается выполнение блока. В качестве значения можно указать один или несколько возможных результатов выполнения задания. При прекращении выполнения блока: • если задания по блоку создавались последовательно, то все последующие задания не будут создаваться; • если задания по блоку создавались параллельно, то параллельные задания будут прекращены
Результат прекращения (StopResult, устар.)	Результат выполнения задания, при котором прекращается выполнение блока. Свойство устарело и оставлено для совместимости. Вместо него рекомендуется использовать **StopResults**
Результат отсутствия исполнителей (NoPerformersResult)	Результат отсутствия исполнителей – это результат выполнения блока, если по нему не было создано ни одного задания. Возможные значения:

Свойство	Описание
	• **Не задан**; • результат выполнения для задания, которое указанно в параметре ***Тип задания**. Например, **Выполнить**, **На доработку**, **Отложить**; • **Результат: Нет исполнителей**. Используйте, если нужно добавить переход из блока в виде отдельной стрелки в случае отсутствия исполнителей. На схеме появится возможность указать значение Нет исполнителей для исходящей из блока стрелки; • **Иначе**
Срок выполнения (RelativeDeadline)	Срок выполнения задания. При необходимости с помощью свойств RelativeDeadlineDays и RelativeDeadlineHours в программном коде можно указать срок выполнения в рабочих днях или в часах
Срок прекращения (RelativeStopAssignmentsDeadline)	Срок принудительного прекращения задания. При необходимости с помощью свойств RelativeStopDeadlineDays и RelativeStopDeadlineHours в программном коде можно задать срок прекращения в днях или в часах
Наследуемые от BlockBase	
Идентификатор (Id)	Присваивается блоку автоматически при создании. Свойство доступно только для просмотра. Отображается в лог-файлах сервиса WorkflowBlockService при обработке блока
Заголовок (Title)	Заголовок блока, который видит разработчик в схеме задачи

События блока

Событие	Назначение	Аргументы события
Старт блока (StartBlock<ИД блока>)	Заполнение свойств выполняемого блока. Событие выполняется на сервисе выполнения блоков схем задач (WorkflowBlockService) при старте обработки блока	**_obj** – задача, в рамках которой обрабатывается блок; e.Block – блок, в рамках которого выполняется событие
Старт задания (StartAssignment<ИД блока>)	• заполнение свойств, которые должны иметь разные значения у разных заданий. Например, разные сроки выполнения задания у разных исполнителей в составном поручении; • заполнение свойств задания, которые нельзя заполнить через свойства блока. Например, свойства типа Текст, Картинка, Бинарные данные. Событие выполняется на сервисе выполнения блоков схем задач (WorkflowBlockService) до старта задания (после создания и заполнения свойств задания) и до сохранения в базу данных. Событие выполняется для каждого задания, созданного в рамках блока. Подробнее см. пример	**_obj** – задача, в рамках которой обрабатывается блок; **assignment** – задание, созданное в рамках блока; e.Block – блок, в рамках которого выполняется событие
Выполнение задания	Актуализация свойств задачи или других заданий в зависимости от свойств выполненного задания.	**_obj** – задача, в рамках которой обрабатывается блок;

Событие	Назначение	Аргументы события
(CompleteAssignment<ИД блока>)	При выполнении каждого из заданий в рамках блока код обработчика события выполняет сервис выполнения блоков схем задач (WorkflowBlockService). Подробнее см. пример	**assignment** – выполненное задание; e.Block – блок, в рамках которого выполняется событие
Конец блока (EndBlock<ИД блока>)	Актуализация свойств задачи при завершении всех заданий, созданных по блоку. Событие выполняется на сервисе выполнения блоков схем задач (WorkflowBlockService) при завершении обработки блока	**_obj** – задача, в рамках которой обрабатывается блок; e.Block – блок, в рамках которого выполняется событие; **e.CreatedAssignments** – список заданий, созданных в рамках блока

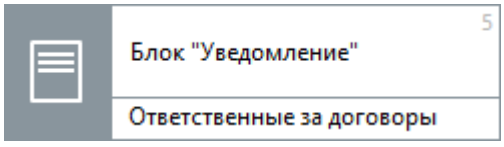
Права исполнителям на группы вложений

На панели свойств отображаются все группы вложений, которые созданы в типе задания, а также группа **Все вложения**.

Для каждой группы вложений укажите права доступа. Возможные значения:

- «Не выдавать»;
- «Просмотр»;
- «Изменение»;
- «Полный доступ».

Уведомление



«Уведомление» (NoticeBlock). Блок предназначен для создания уведомлений. После создания уведомлений выполняется выход из блока.

На схеме в блоке отображаются исполнители, если они заданы на панели свойств.

Свойства блока

Свойство	Описание
Тип уведомления	Имя типа уведомления. В списке доступны все типы уведомлений, созданные в модуле и в задаче
Исполнители (Performers)	Список исполнителей. Для выбора доступны роли и наследники от групп. При заполнении наблюдателей через панель свойств данные берутся с локального веб-сервера. ВАЖНО. Свойство рекомендуется задавать в программном коде. Используйте для заполнения панель свойств, только если Sid исполнителей в разрабатываемой системе совпадает с Sid в продуктовой системе.

Свойство	Описание
	Sid (идентификатор субъекта прав) можно узнать в таблице Sungero_Core_Recipient
Тема (Subject)	Тема создаваемого уведомления
Текст (Text)	Текст создаваемого уведомления
Наследуемые от BlockBase	
Идентификатор (Id)	Присваивается блоку автоматически при создании. Свойство доступно только для просмотра. Отображается в лог-файлах сервиса WorkflowBlockService при обработке блока
Заголовок (Title)	Отображается в схеме задачи

События блока

Событие	Назначение	Аргументы события
Старт блока (StartBlock<ИД блока>)	Заполнение свойств выполняемого блока. Событие выполняется на веб-сервере при старте обработки блока	**_obj** – задача, в рамках которой обрабатывается блок; e.Block – блок, в рамках которого выполняется событие
Старт уведомления (StartNotice<ИД блока>)	Заполнение свойств уведомления, которые нельзя заполнить через свойства блока. Например, свойства типа Текст, Картинка, Бинарные данные. Событие выполняется на сервисе выполнения блоков схем задач (WorkflowBlockService) до старта уведомления (после создания и заполнения свойств уведомления) и до сохранения в базу данных	**_obj** – задача, в рамках которой обрабатывается блок; **notice** – уведомление, созданное в рамках блока; e.Block – блок, в рамках которого выполняется событие
Конец блока (EndBlock<ИД блока>)	Актуализация свойств задачи после создания уведомлений. Событие выполняется на сервисе выполнения блоков схем задач (WorkflowBlockService) при завершении обработки блока	**_obj** – задача, в рамках которой обрабатывается блок; e.Block – блок, в рамках которого выполняется событие; **e.CreatedNotices** – уведомления, созданные в рамках блока

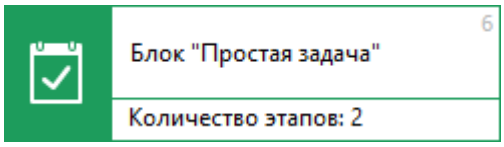
Права исполнителям на группы вложений

На панели свойств отображаются все группы вложений, которые созданы в типе уведомления, а также группа **Все вложения**.

Для каждой группы вложений укажите права доступа. Возможные значения:

- «Не выдавать»;
- «Просмотр»;
- «Изменение»;
- «Полный доступ».

Простая задача



«Простая задача» (SimpleTaskBlock). Блок предназначен для создания простой задачи или подзадачи. При обработке блока создается и отправляется задача, после чего выполняется выход из блока. Ожидание завершения задачи в этом блоке не происходит.

На схеме в блоке отображается количество этапов маршрута задачи.

Свойства блока

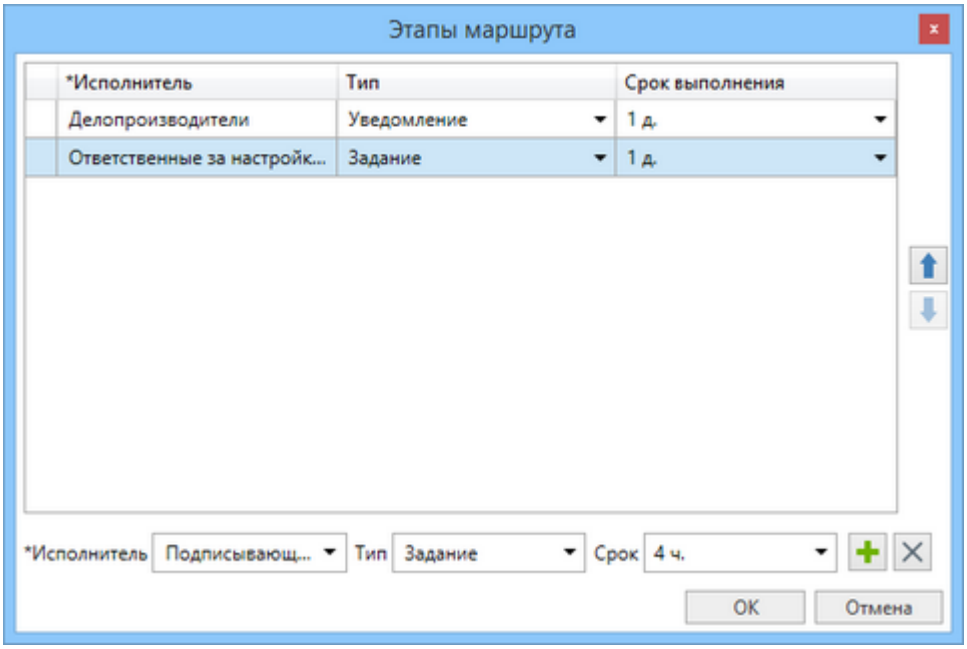
Свойство	Описание
Наблюдатели (Observers)	Список наблюдателей задачи. Для выбора доступны роли и наследники от групп. При заполнении наблюдателей через панель свойств данные берутся с локального веб-сервера. ВАЖНО. Свойство рекомендуется задавать в программном коде. Используйте для заполнения панель свойств, только если Sid наблюдателей в разрабатываемой системе совпадает с Sid в продуктовой системе. Sid (идентификатор субъекта прав) можно узнать в таблице Sungero_Core_Recipient
Старт (RouteType)	Тип маршрута задачи. Возможные значения: • **Одновременно** (**Parallel**). Задания отправляются параллельно всем исполнителям; • **Друг за другом** (**Sequential**). Задания отправляются последовательно
Маршрут (RouteSteps)	Настройка этапов маршрута задачи. ВАЖНО. Свойство рекомендуется задавать в программном коде. Используйте для заполнения панель свойств, только если Sid исполнителей этапов маршрута в разрабатываемой системе совпадает с Sid в продуктовой системе. Sid (идентификатор субъекта прав) можно узнать в таблице Sungero_Core_Recipient
Подзадача (IsSubtask)	Признак того, что создается подзадача. Возможные значения: **True**, **False**. Значение по умолчанию **False** (флажок не установлен)
Задание на приемку (NeedsReview)	Признак того, что после выполнения всех блоков схемы задачи нужно создать задание на приемку. Возможные значения: **True**, **False**. Значение по умолчанию **True** (флажок установлен)
Тема (Subject)	Тема задачи, которую видят пользователи в системе
Текст (Text)	Текст задачи, который видят пользователи в системе
Важность (Importance)	Важность задачи. Возможные значения: **Нормальная**, **Высокая**
Автор задачи (Author)	Свойство задается в программном коде
Наследуемые от BlockBase	

Свойство	Описание
Идентификатор (Id)	Присваивается блоку автоматически при создании. Свойство доступно только для просмотра. Отображается в лог-файлах сервиса WorkflowBlockService при обработке блока
Заголовок (Title)	Заголовок блока, который видит разработчик в схеме задачи

Настройка маршрута

Чтобы настроить этапы маршрута простой задачи:

1. Для свойства **Маршрут** установите флажок **Настроить**.
2. В открывшемся окне заполните поля:



***Исполнитель**. Для выбора доступны роли и наследники от групп. При заполнении данные берутся с локального веб-сервера.

ВАЖНО. Свойство рекомендуется задавать в программном коде. Используйте для заполнения панель свойств, только если Sid исполнителей в разрабатываемой системе совпадает с Sid в продуктовой системе.

Тип этапа. Возможные значения: «Задание», «Уведомление».

Срок выполнения задания. Указывается в минутах, часах или днях.

3. Нажмите на кнопку .
4. Добавьте остальные этапы и нажмите на кнопку **ОК**.

События блока

Событие	Назначение	Аргументы события
Старт блока (StartBlock<ИД блока>)	Заполнение свойств выполняемого блока. Событие выполняется на сервисе выполнения блоков схем задач (WorkflowBlockService) при старте обработки блока	**_obj** – задача, в рамках которой обрабатывается блок; e.Block – блок, в рамках которого выполняется событие

Событие	Назначение	Аргументы события
Старт задачи (StartTask<ИД блока>)	Заполнение свойств задачи, которые нельзя заполнить через свойства блока. Например, свойства типа Текст, Картинка, Бинарные данные. Событие выполняется на сервисе выполнения блоков схем задач (WorkflowBlockService) до старта простой задачи (после создания и заполнения свойств задачи) и до сохранения в базу данных	**_obj** – задача, в рамках которой обрабатывается блок; **createdTask** – задача, созданная в рамках блока; e.Block – блок, в рамках которого выполняется событие
Конец блока (EndBlock<ИД блока>)	Актуализация свойств задачи после выполнения блока. Событие выполняется на сервисе выполнения блоков схем задач (WorkflowBlockService) при завершении обработки блока	**_obj** – задача, в рамках которой обрабатывается блок; e.Block – блок, в рамках которого выполняется событие; **e.CreatedTask** – простая задача, созданная в рамках блока

Задание на приемку

Задание используется для приемки работ по задаче. Создается после выполнения всех блоков схемы, если для свойства **Задание на приемку** (NeedsReview) установлено значение **True** или в редакторе типа задачи установлен флажок **Создавать задание на приемку**. Задание на приемку не отображается на схеме.

Если задание на приемку выполнено с результатом **Принять**, задача завершается. Иначе выполняется переход в блок, который следует за блоком «Начало процесса», и начинается новая итерация процесса.

ВАЖНО. Заполнение свойств задания на приемку доступно только в обработчике события блока «Конец процесса».

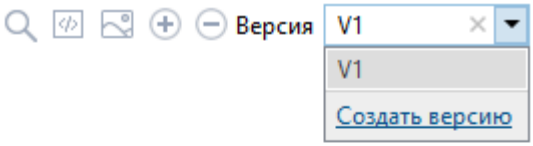
События блока

Событие	Назначение	Аргументы события
Старт задания на приемку (StartReviewAssignment<ИД блока «Конец процесса»>)	Заполнение свойств задания на приемку. Событие выполняется на сервисе выполнения блоков схем задач (WorkflowBlockService) до старта задания на приемку, если в текущей задаче для свойства **Задание на приемку** (NeedsReview) установлен значение **True**	**_obj** – задача, в рамках которой создается задание на приемку; **reviewAssignment** – задание на приемку

ПРИМЕЧАНИЕ. Если для задачи создан тип сущности задание на приемку, то пользователь получит экземпляр данного задания. Иначе пользователю придет задание на приемку базового типа сущности Sungero.Workflow.IReviewAssignment.

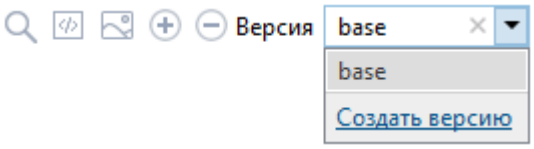
Изменение схемы

В правом верхнем углу редактора схем отображается выпадающий список для работы с версиями. При создании типа задачи у схемы одна версия (V1), и на ней отображаются только элементы начала и завершения схемы.

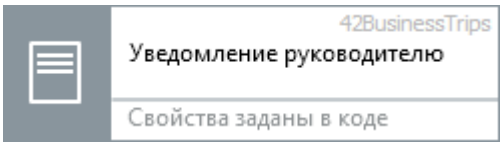


Первую версию схемы можно редактировать, но нельзя удалить.

При перекрытии типа задачи в узле «Схема» отображается базовая версия схемы – это последняя версия схемы в базовом слое. В списке версий она отмечена как **base**.



Базовую версию схемы нельзя редактировать и удалять. Чтобы доработать схему, нужно создать новую версию схемы. В нее можно вносить любые изменения. Чтобы идентификаторы блоков при обновлении схемы не пересекались, в перекрытых типах задач идентификатор блока состоит из номера блока и названия решения, например, 42BusinessTrips.



При первом перекрытии у блоков, унаследованных от базовой версии схемы, идентификатор состоит только из номера. При следующих перекрытиях на схеме могут быть блоки с идентификаторами, которые состоят только из номера, и с идентификаторами, состоящими из номера и названия решения. Например, разработчик в своем решении перекрыл тип задачи из базового решения Directum RX, создал версию схемы и добавил блоки. После чего передал свое решение в качестве базового разработчику в другую компанию. Второй разработчик после перекрытия типа задачи видит на схеме блоки с номерами и блоки с номерами и названием решения первого разработчика. Блоки, которые добавит второй разработчик, будут с номером и именем его решения. Таким образом по идентификатору можно понять, какие блоки добавлены на вашем слое.

Когда нужно создавать версию схемы

До публикации в продуктивную систему можно как угодно изменять последнюю версию схемы в новой и перекрытой задаче: добавлять и удалять блоки, создавать ветки, изменять свойства блоков и обработчики событий. После публикации в продуктивную систему, чтобы доработать обработчики событий, нужно создать новую версию схемы.

ВАЖНО. Если не создать версию задачи и внести в обработчики событий изменения, то после публикации в продуктивную систему ранее стартованные задачи могут выполняться с ошибками.

Как создать версию схемы

Чтобы создать версию, в выпадающем списке версий нажмите на ссылку **Создать версию**. Перед созданием учитывайте следующее:

- в *перекрытии* не получится создать версию схемы, если в родительском типе задачи в параметрах установлен флажок **Запретить создавать версии схемы задачи в перекрытиях**;
- каждая новая версия создается на основе предыдущей. То есть третья версия создается на основе второй, ее нельзя создать на основе первой версии;
- редактировать можно только последнюю версию схемы. Предыдущие версии доступны для просмотра и остаются для поддержки работоспособности уже существующих стартованных задач;
- обработчики событий блоков схемы не версионируются и хранятся в пределах *слоя*. Код обработчика, который должен выполняться в определенной версии схемы, необходимо заключать в условия:

  ```
  if (_obj.GetStartedSchemeVersion() == LayerSchemeVersions.V1)
  {
    //...
  }
  if (_obj.GetStartedSchemeVersion() == LayerSchemeVersions.V2)
  {
    //...
  }
  ```

  Проверить, что задача стартована по схеме ниже или выше определенной версии можно так:

  ```
  if (_obj.GetStartedSchemeVersion() < LayerSchemeVersions.V2)
  {
    //...
  }
  ```

- с помощью метода GetStartedSchemeVersion() получить информацию о версии схемы можно в обработчиках событий блоков, типов задач, заданий и уведомлений, а также в серверных, клиентских и разделяемых функциях типов задач, заданий и уведомлений. Если код пишется в обработчике для типа задания или уведомления, то получить версию схемы можно только через саму задачу (Task). Т.е. код должен выглядеть следующим образом:

  ```
  if (_obj.Task.GetStartedSchemeVersion() == LayerSchemeVersions.V1) {}
  ```

- для проверки работы предыдущей версии схемы, необходимо выбрать версию и опубликовать в локальной системе Directum RX. Версия станет текущей для схемы.

В продуктивную систему по умолчанию публикуется последняя версия схемы. После публикации уже запущенные задачи продолжают работу по схеме, которая была до обновления. Новые задачи стартуют по доработанной схеме.

Подробнее см. «Пример создания версии схемы».

Как удалить версию схемы

ВАЖНО. Не рекомендуется удалять версии схем, по которым в продуктивной системе Directum RX стартованы задачи, так как они перестанут работать.

Удалите версию схемы, например, если она была тестовой. Для этого в выпадающем списке **Версия** выберите нужную версию и нажмите ✕.

В результате после удаления:

- предыдущая версия станет доступна для изменения;
- номер удаленной версии не будет использоваться при создании следующих версий.

Обновление схемы в перекрытии

Базовые решения постоянно развиваются. Поэтому в новых версиях схемы задач, которые вы перекрыли, могут измениться: стать удобнее для использования или расшириться. При необходимости в перекрытом типе задачи можно обновить версию до новой базовой версии и объединить ее с уже добавленными изменениями. Для этого в редакторе схемы нажмите на кнопку **Обновить схему из предка с сохранением внесенных изменений**. Кнопка отображается, если в типе задачи на базовом слое появилась новая версия схемы или изменилась существующая.

После нажатия на кнопку:

- создается новая версия схемы на основании обновленной базовой версии, и к ней применяются ранее внесенные в схему изменения;
- информация о внесенных изменениях отображается в окне вывода. Она необходима, чтобы быстро разобраться при возникновении проблем;
- код обработчиков блоков при обновлении не изменяется;
- происходит *валидация* схемы. Блоки с ошибками выделяются красной рамкой. Разработчик может:
    - исправить ошибки и сохранить схему;
    - выбрать в качестве используемой предыдущую версию схемы;
    - закрыть редактор типа задачи без сохранения изменений и таким образом вернуться к версии схемы, которая была до обновления.

ПРИМЕЧАНИЕ. Иногда кнопка отображается в *базовом решении* в редакторе типа задачи. При этом, если нажать на кнопку, схема не обновляется, так как базовые решения не доступны для изменения. Ситуация возникает при одновременном выполнении условий:

- в среде разработки несколько базовых решений с перекрытием одной и той же схемы;

- импортируется новое базовое решение с перекрытием этой же схемы или обновляется существующее базовое решение.

Пример. Создание версии схемы

ПРИМЕЧАНИЕ. Пример актуален для схем задач, созданных ранее в среде разработки. Новые схемы рекомендуется настраивать через варианты процессов с помощью механизма no-code. Подробнее см. в руководстве администратора, раздел «Настройка схем бизнес-процессов».

Ситуация

Предположим, в решении создан тип задачи. Решение уже опубликовано в продуктивную систему, и сотрудники создавали задачи нового типа. В схему задачи необходимо внести изменения:

- удалить блок **Мониторинг**, в котором при наступлении определенного события создавалось уведомление;
- логику создания уведомления перенести в обработчик блока **Задание**.

Решение

1. В редакторе типа задачи перейдите в узел «Схема» и создайте версию.
2. Удалите со схемы блок **Мониторинг**. Для этого выделите блок и нажмите на клавишу DELETE. Вместе с блоком удалятся стрелки перехода. Добавьте стрелки между оставшимися блоками.
3. Выделите блок **Задание** на схеме и на панели свойств рядом с событием **Выполнение задания** перейдите по ссылке **Перейти к коду**.
4. Добавьте логику создания уведомления в обработчик события. С помощью условия укажите, что добавленный код относится ко второй версии схемы:

```
public virtual void CompleteAssignment2(Sungero.Docflow.
     IFreeApprovalAssignment assignment,
     Sungero.Docflow.Server.FreeApprovalAssignmentArguments e)
   {
       // Код обработчика события блока «Задание».
       // ...

       // Код для создания уведомления.
       // V2- вторая версия схемы, в которой используется данный код.
         if (assignment.Result == Sungero.Docflow.FreeApprovalAssignment.
           Result.ForRework && _obj.ReceiveNotice == true
           && _obj.GetStartedSchemeVersion() == LayerSchemeVersions.V2 &&
assignmentsInWork.Any())

         {
           /// ...
         }
```

```
}
```

ПРИМЕЧАНИЕ. Код обработчика события блока **Мониторинг** удалять не нужно, так как ранее стартованные задачи будут выполняться по первой версии схемы, в которой используется код обработчика.

5. Сохраните изменения.
6. Опубликуйте разработку, проверьте изменения и перенести в продуктивную систему.

В результате:

- новые и рестартованные задачи будут выполняться по второй версии схемы;
- первая версия останется для поддержки работоспособности существующих стартованных задач.

Пример. Изменение срока задания

ПРИМЕЧАНИЕ. Пример актуален для схем задач, созданных ранее в среде разработки. Новые схемы рекомендуется настраивать через варианты процессов с помощью механизма no-code. Подробнее см. в руководстве администратора, раздел «Настройка схем бизнес-процессов».

Ситуация

Предположим, создан тип задачи. В схеме задачи нужно изменить срок задания на контроль возврата документа так, чтобы срок выполнения задания стал 2 рабочих дня (ранее был 1 день).

Решение

1. В редакторе типа задачи перейдите в узел «Схема».
2. Если дорабатывается задача, опубликованная в продуктивную систему, создайте версию схемы. Если нет, решите, куда вносить доработки: в последнюю версию схемы или в новую.
3. На схеме выберите блок с заданием на контроль возврата.
4. На панели свойств рядом с событием **Старт задания** перейдите по ссылке **Перейти к коду**.
5. В обработчике события поправьте срок:

```
#region Контроль возврата
   public virtual void
   StartAssignment4(Sungero.Docflow.ICheckReturnCheckAssignment assignment,
     Sungero.Docflow.Server.CheckReturnCheckAssignmentArguments e)
   {
   // С помощью метода AddWorkingDays укажите нужное количество рабочих дней
   // для выполнения задания на контроль.
       assignment.Deadline = Calendar.UserToday.AddWorkingDays(2);
   }
```

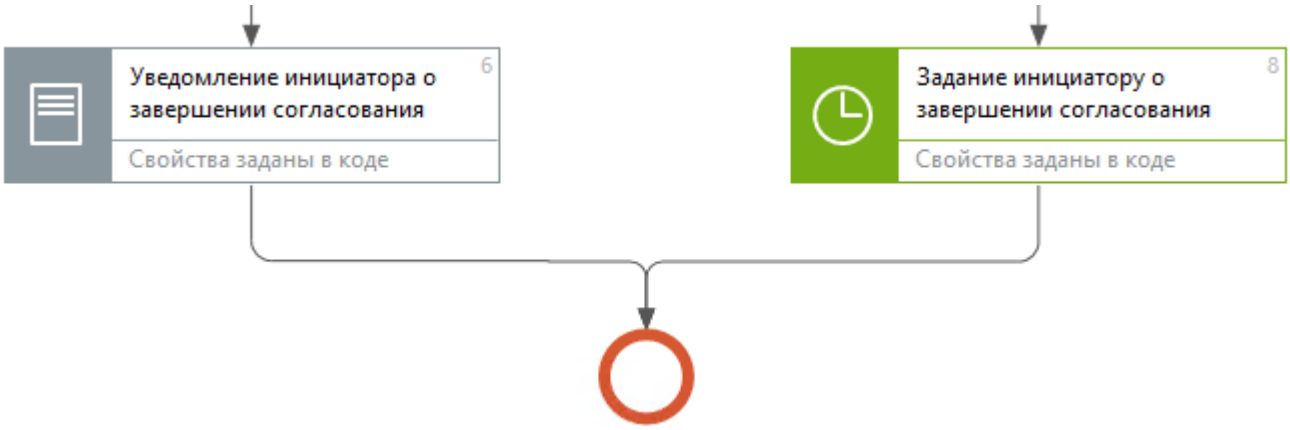
6. Сохраните изменения.
7. Опубликуйте разработку, проверьте изменения и перенесите в продуктивную систему.

В результате все новые задания на контроль возврата будут приходить со сроком 2 рабочих дня. У заданий, которые уже сформированы на момент обновления схемы, срок останется 1 день.

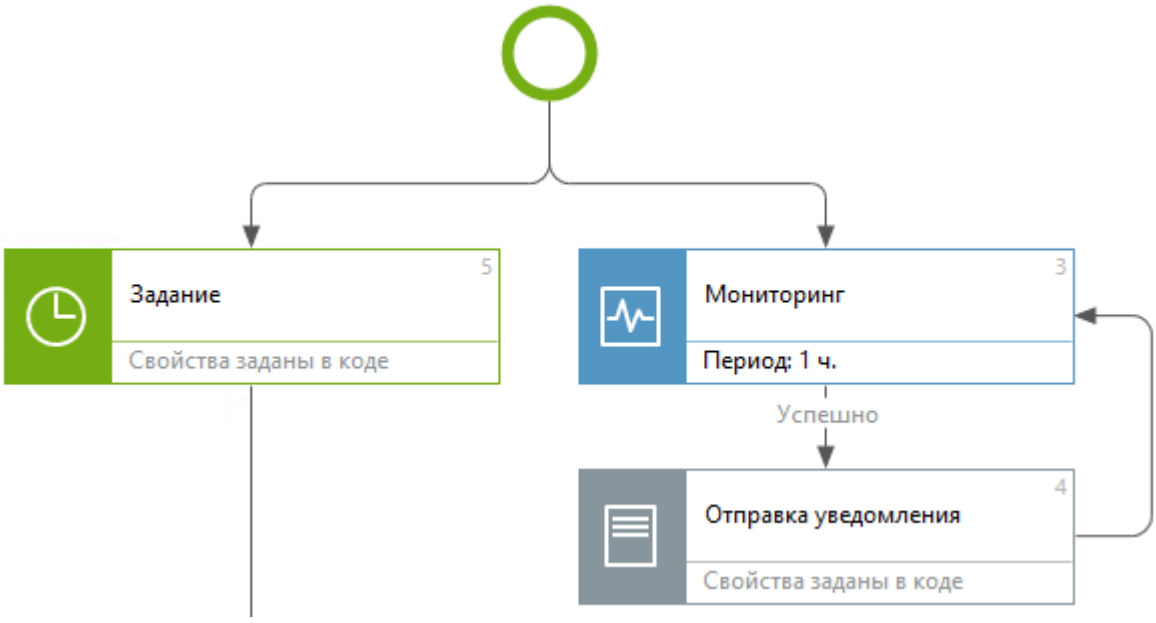
Рекомендации по настройке

При разработке схемы задачи необходимо учитывать следующее:

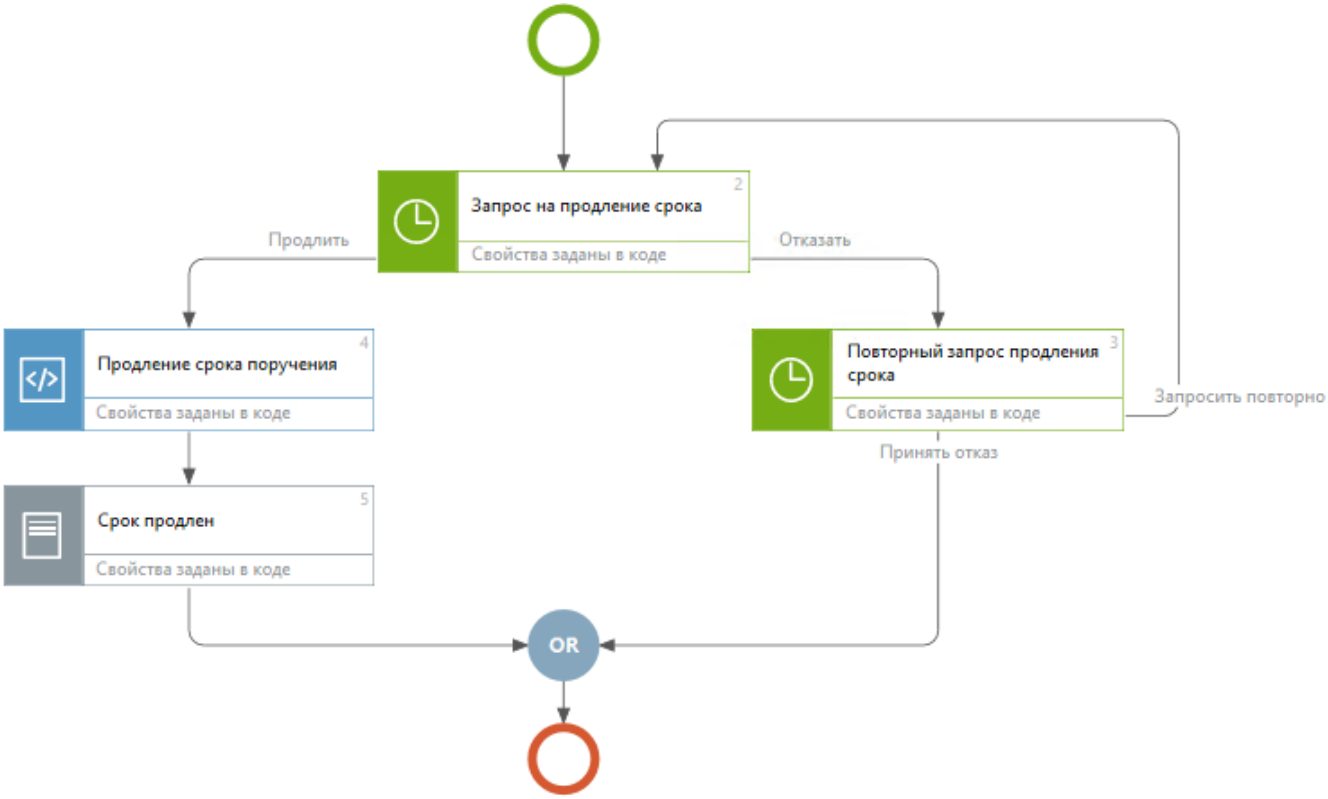
- переходы между блоками должны сводиться к блоку Конец процесса:



При необходимости можно добавлять «тупиковые ветки». Например, когда нужно отправлять уведомления с напоминанием до тех пор, пока задание не будет выполнено:



- если в схеме есть две или более параллельные ветки, то каждая пара веток должна сходиться в одной точке. Переходить из одной параллельной ветки в другую нельзя:

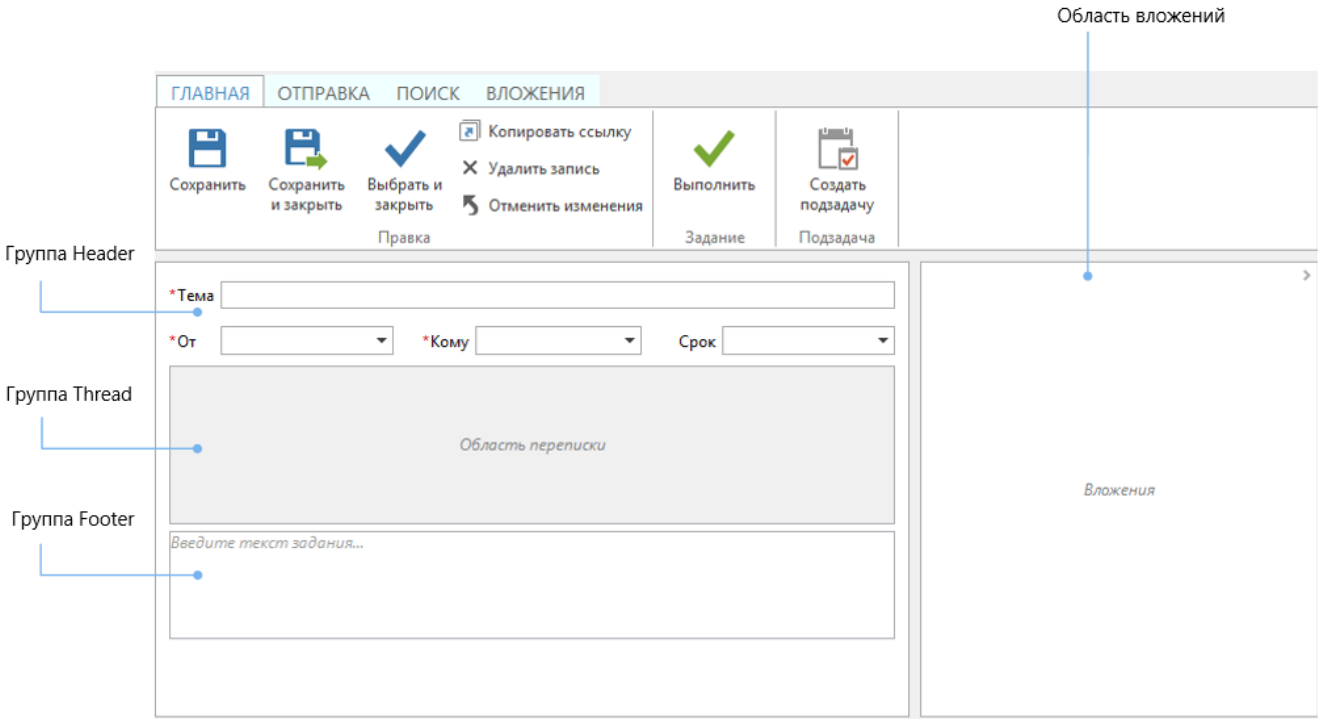


Настройка формы для задач и заданий

Используйте редактор форм, чтобы настроить ленту и вкладки карточки задачи, а также задания, задания на приемку или уведомления.

По умолчанию для данных типов сущностей настроена основная вкладка карточки (MainPage) с предопределенными группами Header, Thread, Footer и областью вложений.

ПРИМЕЧАНИЕ. У уведомлений нет группы Footer, так как они знакомят пользователей с какой-либо информацией и не требуют ответа.



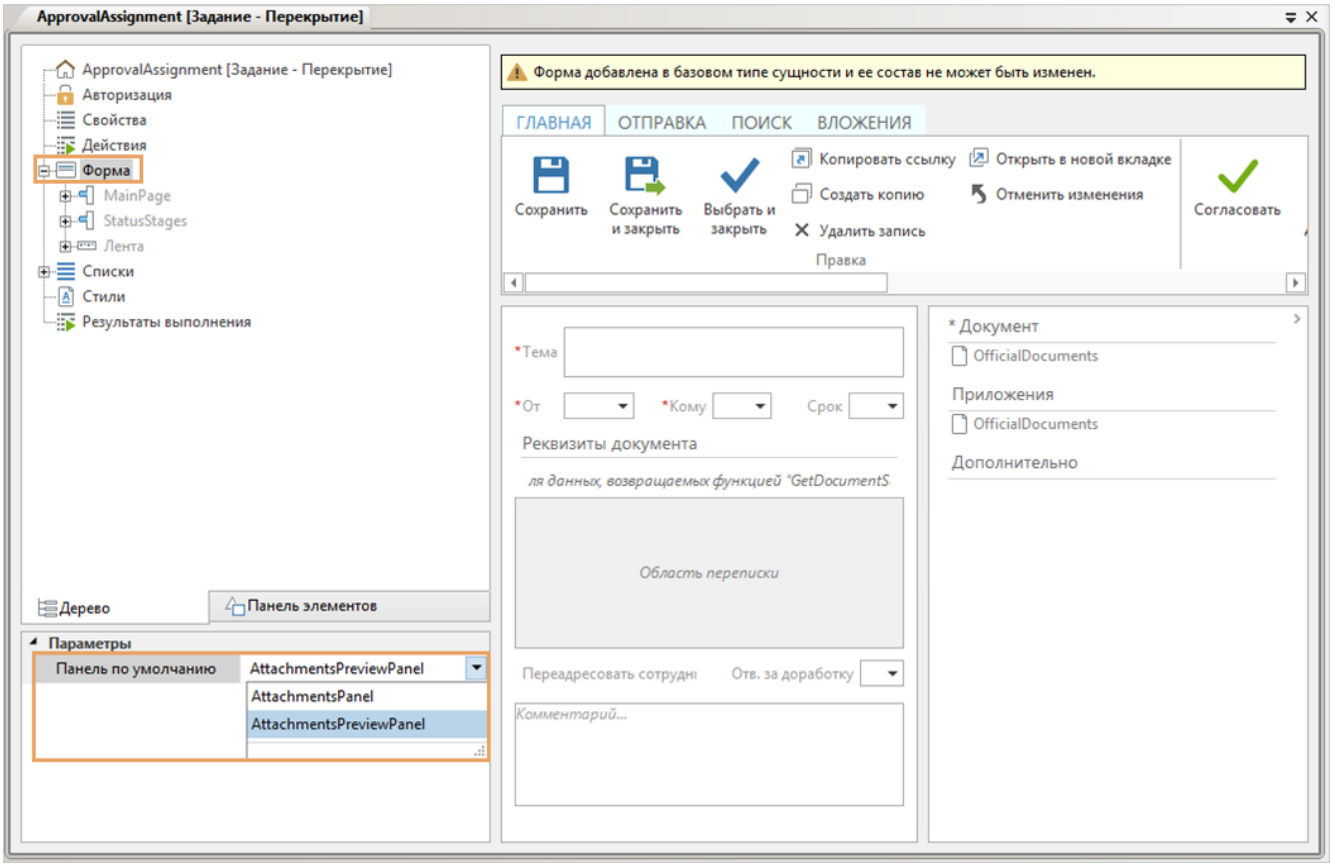
ВАЖНО. Для поддержания единого стиля системы состав групп на основной вкладке недоступен для изменения. Добавлять новые элементы можно только в группу Header и Footer.

Вложения

Для всех типов задач и заданий есть область вложений, которую вы можете настраивать. Например, настроить отображение вложений, групп вложений или включить возможность удаления. Также в карточках задач и заданий есть панель предпросмотра вложений, ее состав недоступен для изменения.

Когда пользователь в системе открывает карточку задачи или задания, в ней отображается одна из панелей: «Вложения» или «Предпросмотр». В среде разработки можно настроить, какая панель будет открыта по умолчанию. Для этого:

1. В дереве типа задачи или задания выделите узел «Форма».



2. На панели свойств для параметра **Панель по умолчанию** выберите нужное значение:
   - **AttachmentsPanel** – панель вложений;
   - **AttachmentsPreviewPanel** – панель предпросмотра вложений.

Выбранная панель будет отображаться в карточке типа задачи или задания при первом ее открытии. В дальнейшем у сотрудника открывается та панель, которую он выбрал последней. Если активация панели настроена программно, то выбор панели пользователем игнорируется.

Группа Header

Содержит контролы:

- для задач – в них пользователи указывают тему, сроки выполнения, исполнителей задачи;
- для заданий – в них отображается информация о теме, инициаторе, исполнителях и сроках.

У группы есть свойства ***Имя**, **Отображаемое имя**, **Выравнивать размеры по группам**, **Отображать на форме**, **Типы группы**, **Количество колонок в группе (1-10)**, которые совпадают со свойствами обычной группы.

Группа Thread

Содержит область переписки.

Чтобы показывать группу на другой вкладке карточки, в свойствах группы установите флажок **Отображать на отдельной вкладке**. Тогда на главной вкладке отображаются только группы Header и Footer, а контрол **Область переписки** скрыт. При этом на форме добавляется вкладка «Переписка».

ПРИМЕЧАНИЕ. Отображение переписки также можно настраивать в веб-клиенте в справочнике «Представления форм». Если в среде разработки установить флажок **Отображать на отдельной вкладке**, то в веб-клиенте автоматически устанавливается флажок **Скрыть область переписки**. При этом аналитик может изменить положение флажка в веб-клиенте.

Группа Footer

Содержит контрол, в котором пользователи:

- для задач – указывают список работ по задаче;
- для заданий – вводят информацию о результатах выполнения задания.

При необходимости в свойстве **Текст в пустом контроле для ответа** укажите текст подсказки. Текст отображается, если контрол не заполнен. Для *локализации* подсказки нажмите на кнопку **Локализовать**.

Снимите флажок **Включить автотексты**, если в тексте задачи или задания будут редко использоваться типовые формулировки. По умолчанию флажок установлен, поэтому при заполнении текста пользователи смогут писать свой текст или использовать готовые часто используемые фразы. Если флажок **Включить автотексты** установлен, в параметре **Предопределенные автотексты** задайте варианты фраз для заполнения контрола. Подробнее см. раздел «Автотексты для заполнения контролов».

Чтобы скрыть контрол для ввода текста, в свойствах группы установите флажок **Скрыть контрол для ответа**. В заданиях на приемку контрол по умолчанию скрыт.

Работа с вложениями

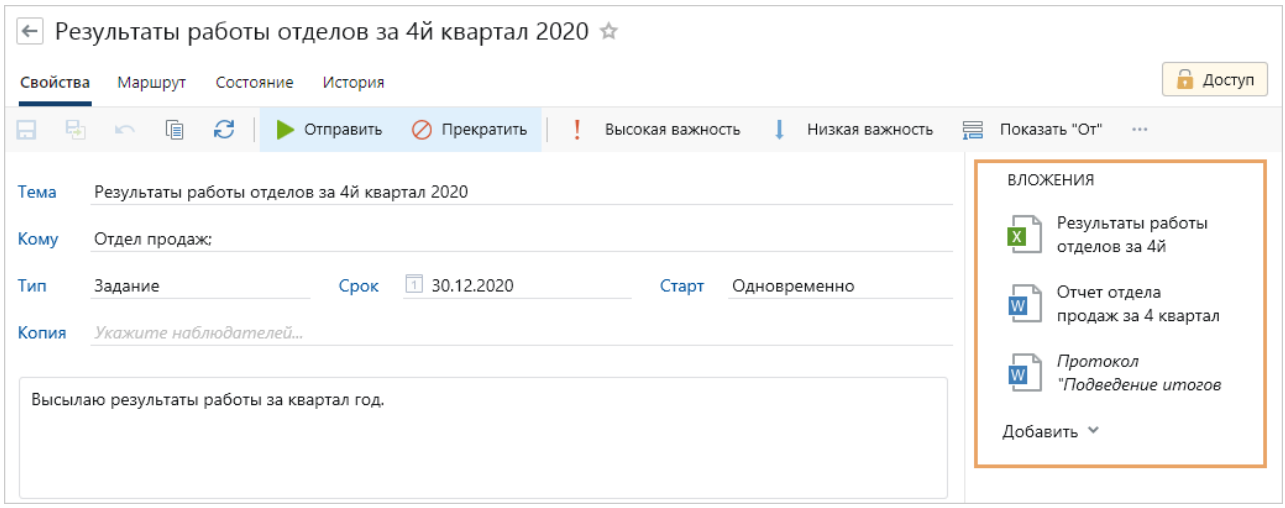
Вложение – это ссылка на документ или любой другой объект системы, которая добавляется в карточку задачи или задания:

- в область вложений задачи инициатор добавляет ссылки на объекты, которые понадобятся исполнителям при выполнении работ;
- во вложениях заданий исполнители видят материалы для выполнения работ и добавляют ссылки на результаты.

Отображение вложений

В карточке задачи и задания вложения могут отображаться в виде обычного списка или в виде групп вложений.

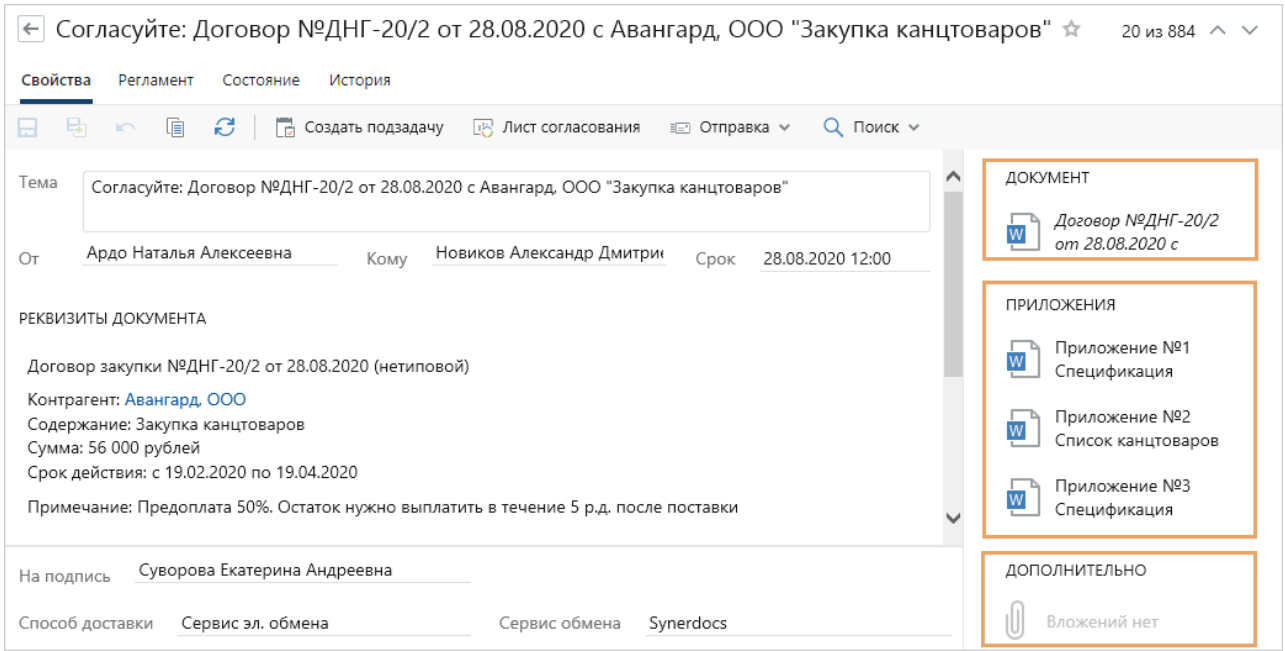
Вложения в виде обычного списка выглядят следующим образом:



В этом случае инициатор и исполнители видят вложения всего семейства задач:

- в карточке задания отображаются вложения, которые добавил инициатор в задачу, и вложения, которые добавили другие исполнители в своих заданиях;
- в карточке задачи отображаются все вложения.

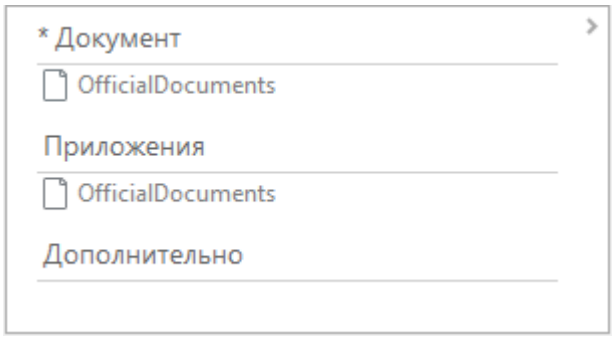
Строго формализованные бизнес-процессы имеют ограничения на количество вложений, тип вложений или их видимость на том или ином этапе процесса. Для настройки ограничений выделяются группы вложений:



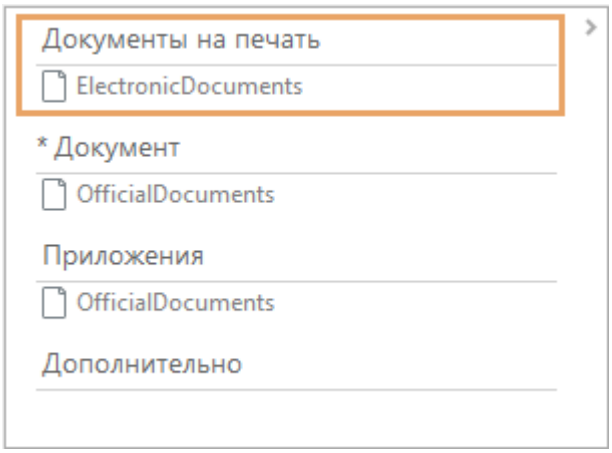
В этом случае исполнитель видит только нужные вложения на каждом этапе бизнес-процесса:

- в карточке задачи отображаются только группы вложений задачи, и не отображаются группы вложений заданий;
- в карточке задания отображаются группы вложений задачи, если разработчик явно не указал, что их надо скрывать. Также отображаются группы вложений заданий того же типа. Группы вложений заданий других типов в карточке задания не отображаются.

Например, в задаче на согласование по регламенту ApprovalTask отображаются группы вложений:



После старта задачи согласующие получают задания. В них отображаются те же группы вложений, что и в задаче, так как разработчик не указал, что их надо скрыть. После согласования формируется задание на печать документов (ApprovalPrintingAssignment), в его карточку добавляется группа «Документы на печать». Она видна только исполнителю задания и не отображается во вложениях задачи и других заданиях:



У подзадач тоже могут быть вложения. Они не отображаются в карточках заданий основной задачи. Таким образом, вложения основного процесса и подпроцесса не смешиваются. При этом в карточку подзадачи вкладываются вложения всего семейства задачи, а в переписке самой задачи отображаются и процессы, и подпроцессы.

ВАЖНО. При разработке типа задачи или задания обратите внимание, что пользователь в системе видит только те вложения, на которые есть права доступа. Наличие у него прав на задачу или задание не дает автоматические права на просмотр вложений. Права на вложения необходимо задавать явно, напоминание об этом выдается пользователю при старте задачи или выполнении задания. Разработчик также может программно выдать или забрать права на вложения.

Не добавляйте действия сущностей на вкладку «Вложения» и контекстные вкладки, иначе действия не будут отображаться.

События групп вложений

Для группы вложений задачи задания, задания на приемку, уведомления можно добавить обработчики событий:

- создание вложения
- добавление вложения

- удаление вложения
- заполнение вложений

Примеры использования обработчиков событий:

- добавление и удаление групп вложений
- заполнение группы вложений

Объектная модель для работы с вложениями

- Attachments – работа со вложениями
- Attachments – состояние групп вложений задач, заданий, заданий на приемку, уведомлений

Также см. описание наследуемых действий при работе с вложениями.

Добавление и удаление вложений

По умолчанию сотрудники могут добавлять или удалять вложения только до старта задачи или до выполнения задания. При этом сотрудник может удалять только свои вложения, хотя видит чужие. Если вложения поделены на группы и изменять их состав разрешено разработчиком, то удалять вложения других сотрудников и добавлять свои можно в любой отображаемой группе.

ВАЖНО. Тип задания, задания на приемку и уведомления наследует группы вложений типа задачи, в которой он создан. Группы, полученные при наследовании, отображаются серым цветом в редакторе форм. Их можно скрыть или изменить доступность. Тип задания или тип уведомления, реализованный в рамках модуля, не может получить группу вложений, созданную для конкретного типа задачи.

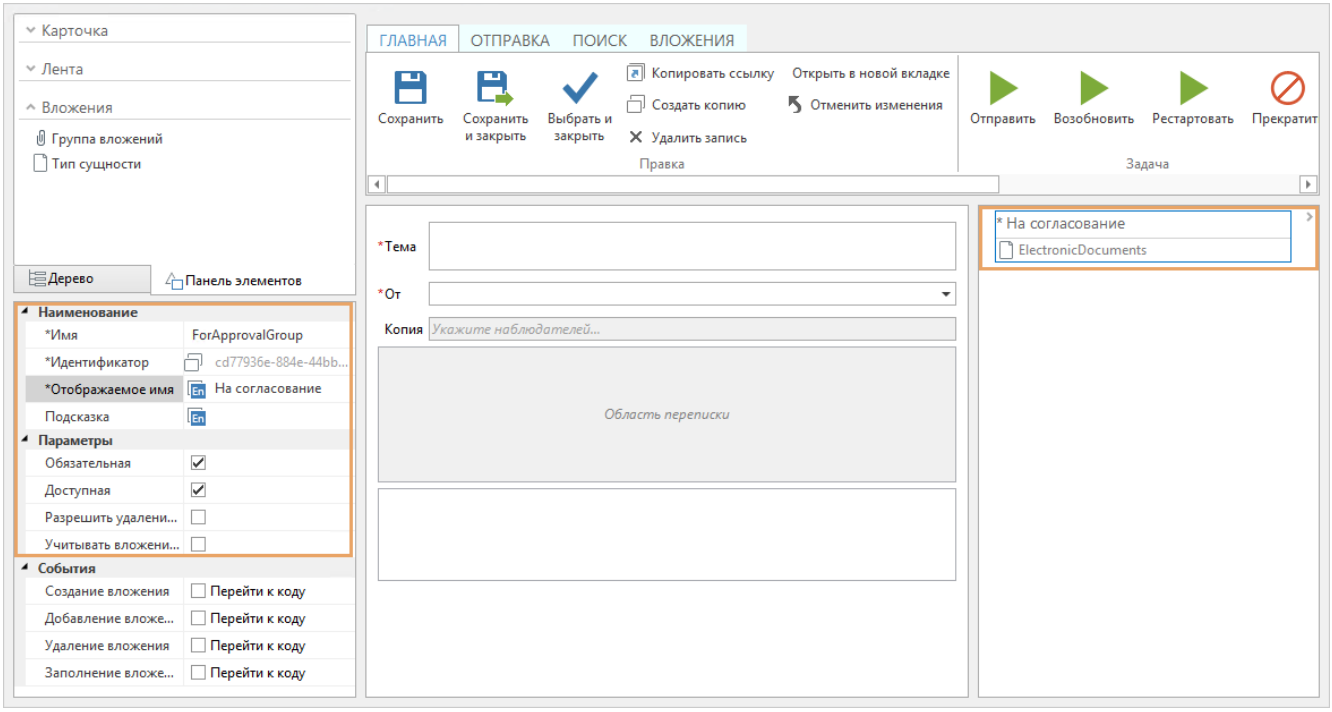
Рассмотрим, как добавить вложения в задачу на свободное согласование. Вложения в задания добавляются аналогичным образом. Предположим, нужно выделить группы вложений:

- «На согласование». В группе будет один документ для согласования типа ElectronicDocument;
- «Приложения». В группе будут документы, которые связаны с согласуемым документом типом связи **Приложение**. Группа недоступна для изменения;
- «Дополнительно». В группе будут ссылки на любые объекты системы.

Чтобы добавить вложения:

1. В дереве типа задачи выделите узел «Форма» и перейдите на вкладку «Панель элементов».

2. Добавьте на форму элемент **Группа вложений**.



3. Задайте свойства для группы вложений:

   ***Имя**. Должно быть уникальным в рамках типа сущности, состоять из букв латинского алфавита и цифр.

   ***Идентификатор**. Используется для настройки прав доступа на вложения при разработке схемы в блоке «Задача». Поле доступно только для просмотра. Чтобы скопировать идентификатор, нажмите на кнопку **Скопировать**.

   ***Отображаемое имя**. Заголовок группы вложений в карточке задачи.

   Подсказка, которая отображается, если в группе нет вложений. Чтобы в подсказке отображался текст «Вложений нет», оставьте поле незаполненным.

   Для *локализации* отображаемого имени и подсказки нажмите на кнопку **Локализовать**.

   Обязательная. Пользователям обязательно нужно добавить вложения в группу. Например, установите флажок для группы «На согласование», чтобы в задачу обязательно вложили согласуемый документ.

   Доступная. Пользователи смогут изменять состав вложений в группе. Например, снимите флажок для группы «Приложения», чтобы пользователи вручную не меняли состав вложений в группе, которая заполняется автоматически.

   Разрешить удаление вложений. Можно изменить, если установлен флажок в поле **Доступная**. Пользователи смогут удалять лишние или случайно добавленные вложения. Параметр доступен в задачах, если в область вложений добавлены группы. В заданиях отображается для групп, унаследованных от типа «Задача». Например, в процессе согласования документа дополнительно добавлены ссылки на другие документы. Если флажок установлен, инициатор задачи или участники согласования могут удалять любые лишние ссылки в задании до его выполнения или в прекращенной задаче до ее рестарта. При этом вложения удаляются и из задачи. Для этого у участников должны быть права на изменение. Если флажок не установлен, удалять можно только свои вложения. Удаление вложений записывается в историю. По умолчанию флажок не установлен.

Особенности:

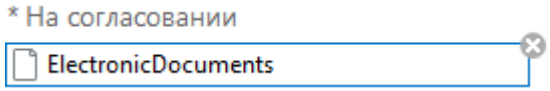
- при установке флажка в типе задачи он автоматически устанавливается в связанных заданиях. При снятии в задаче аналогично снимается в задании;
- при установке в типе задания флажок в задаче не ставится;
- если в базовом решении в типе задачи установлен флажок, то при перекрытии его можно снять. При этом автоматически снимется флажок и в связанном типе задания. Если в базовом решении в типе задачи флажок не установлен, то его можно установить. В этом случае в типе задания флажок автоматически не подставится;
- в перекрытом типе задания можно снимать и устанавливать флажки;
- если вложение добавлено в задаче и удаляется из задания, то событие удаления генерируется только после сохранения карточки задания;
- флажок не отображается, если задано событие Заполнение вложений, так как в нем разрешено удаление вложений.

Учитывать вложения от родительских сущностей. Если флажок установлен, при создании из задания подзадачи того же типа, в ней автоматически отображаются те же вложения. Например, установите флажок для группы «Дополнительно», чтобы в ней при создании подчиненного поручения отображались вложения из поручения.

Использование флажка позволяет оптимизировать заполнение групп вложений, так как разработчику не нужно писать код для их копирования из задания в подзадачу. В результате снижается нагрузка на базу данных, так как в ней не создаются дубли вложений.

ВАЖНО. Если флажок установлен, то вложения группы привязываются к родительской задаче, а в подзадачах – вычисляются. Из-за этого при поиске задач, в которые вложен документ, в результатах родительские задачи отображаются, а подзадачи – нет.

4. Чтобы пользователи добавляли в группу «На согласование» ссылки только на определенный тип сущности, добавьте в группу элемент **Тип сущности**:



5. Задайте свойства:

   Наименование. Недоступно для изменения. Задается автоматически при указании типа сущности.

   Тип сущности. Пользователи смогут добавлять в группу ссылки на сущности указанного типа. Для группы «На согласование» укажите тип ElectronicDocument.

   Количество ссылок, которое должно быть в группе. Если значение не указано, в группу можно добавить любое количество ссылок. Для группы «На согласование» укажите значение «1», чтобы пользователи добавляли в группу только один согласуемый документ.

   Отключить. Установите флажок, если группа получена при перекрытии задачи и нужно снять все ограничения на вложения в группе. Флажок можно установить для типов сущностей, которые добавлены в группу вложений на рабочем слое. Для типов сущностей, добавленных на базовом слое, флажок недоступен.

6. Добавьте обработчики для событий создания, добавления, удаления и заполнения вложений. Например, задайте обработчик, чтобы в группу «Приложения»

автоматически попадали ссылки на документы, которые связаны с согласуемым документом типом связи **Приложение**.

Для этого в дереве выделите группу «На согласование». В свойствах группы установите флажок рядом с событием **Добавление вложения** и нажмите на ссылку **Перейти к коду**. В открывшемся редакторе добавьте код обработчика:

```
public virtual void ForApprovalGroupAdded(Sungero.Workflow.Interfaces.AttachmentAddedEventArgs e)
{
  var document = _obj.ForApprovalGroup.ElectronicDocuments.First();

  // Убрать лишние кавычки и переносы строк в имени документа.
  using (TenantInfo.Culture.SwitchTo())
    _obj.Subject = Functions.Module.TrimSpecialSymbols(FreeApprovalTasks.Resources.TaskSubject, document.Name);

  if (!_obj.State.IsCopied)
  {
    // Синхронизировать приложения документа и группы вложения.
    // AddendaGroup - группа вложений "Приложения".
    Functions.Module.SynchronizeAddendaAndAttachmentsGroup(_obj.AddendaGroup, document);
    if (OfficialDocuments.Is(document))
        // Добавить связанные документы в группу вложения.
        Functions.OfficialDocument.AddRelatedDocumentsToAttachmentGroup(OfficialDocuments.As(document), _obj.OtherGroup);
  }

  // Обработать добавление документа как основного вложения в задачу.
  if (OfficialDocuments.Is(document))
    Functions.OfficialDocument.DocumentAttachedInMainGroup(OfficialDocuments.As(document), _obj);
}
```

7. Добавьте нужное количество групп вложений и задайте их свойства.

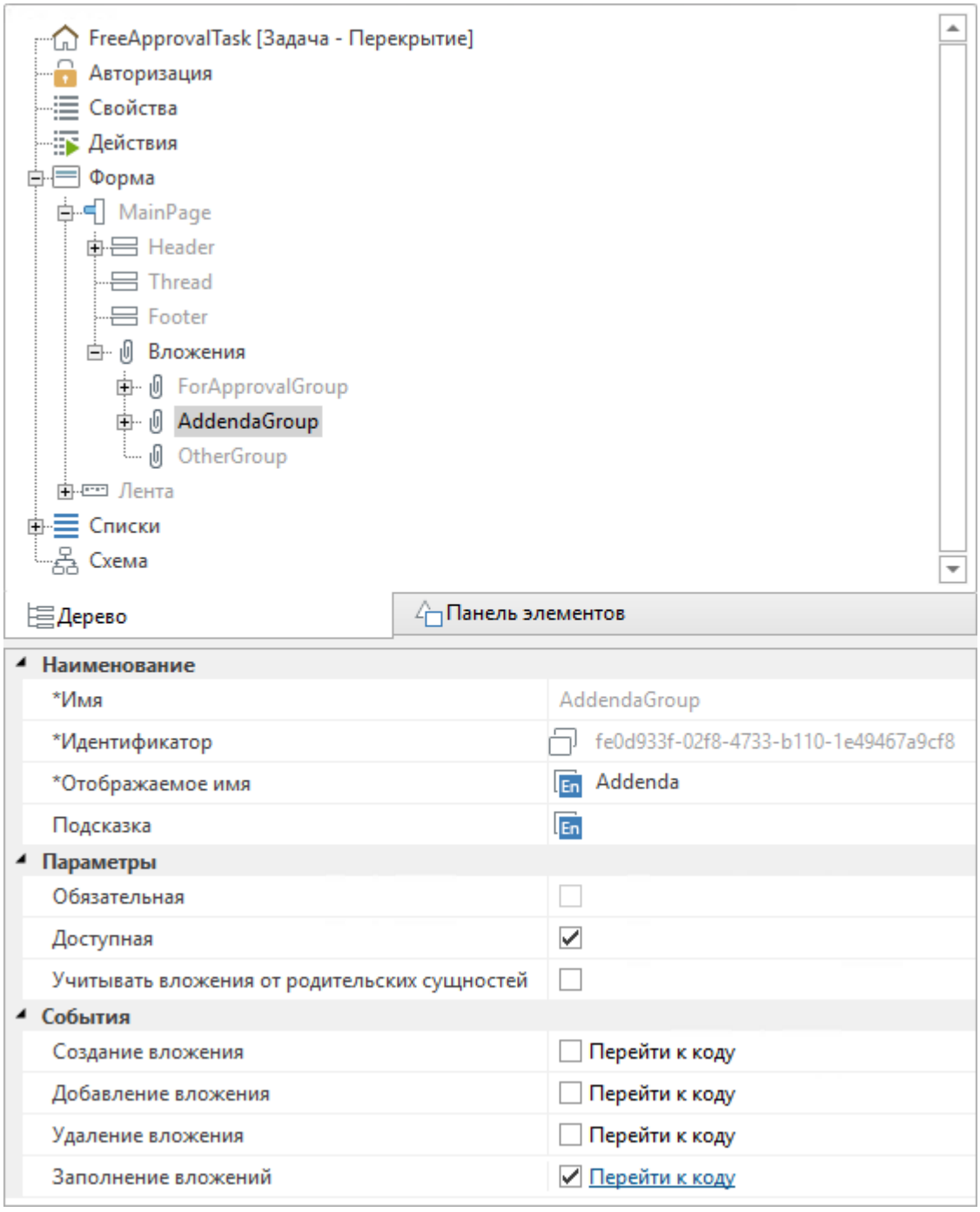
СОВЕТ. Чтобы изменить расположение группы в области вложений, перетащите ее в нужное место.

В результате в карточке задачи появятся группы вложений «На согласовании», «Приложения» и «Дополнительно».

Пример заполнения группы вложений

Рассмотрим, как в задаче на свободное согласование настроить группу «Приложения» таким образом, чтобы в ней отображались приложения к документу из группы «На согласование». Группа «Приложения» будет обновляться при добавлении и удалении вложения в группе «На согласование», при старте и рестарте задачи, создании заданий, добавлении исполнителей. Для этого:

1. В дереве типа задачи выделите узел «Форма» и выберите группу вложений AddendaGroup (Приложения):



2. Задайте свойства для группы вложений.

3. Добавьте код для получения приложений вложения из группы ForApprovalGroup (На согласование). В текущем примере можно использовать функцию базового решения GetAddenda.
4. Установите флажок рядом с событием Заполнение вложений и нажмите на ссылку **Перейти к коду**.
5. В открывшемся редакторе добавьте код обработчика для заполнения группы AddendaGroup (Приложения):

```
public override void
   AddendaGroupPopulating
   (Sungero.Workflow.Interfaces.AttachmentGroupPopulatingEventArgs e)
{
  /// С помощью обработчика GetAddenda получить
  /// приложения вложений из группы ForApprovalGroup (На согласование)
  /// и заполнить ими группу Addenda (Приложения).
  e.PopulateFrom(_obj.ForApprovalGroup, GetAddenda);
}
```

В результате в группе «Приложения» вложения автоматически обновляются при изменении состава вложений в группе «На согласование».

События групп вложений задачи, задания, задания на приемку, уведомления

События групп вложений задаются в редакторе типа задачи, задания, задания на приемку, уведомления на панели свойств в узле «Форма».

Создание вложения (GroupCreated)

Выполняется в разделяемом коде после создания вложения в группе.

Назначение. Изменение свойств вложения перед добавлением в группу.

Аргументы события:

- _obj – задача (задание, задание на приемку, уведомление), в которую добавляется вложение;
- e.Attachment – сущность, добавленная в группу вложений.

Добавление вложения (GroupAdded)

Выполняется в разделяемом коде при добавлении вложения в группу. Подробнее см. в разделе «Добавление и удаление вложений».

Назначение. Изменение свойств задачи (задания, задания на приемку, уведомления), в которую добавляется вложение. Например, можно изменить тему задачи.

Аргументы события:

- _obj – задача (задание, задание на приемку, уведомление), в которую добавляется вложение;
- e.Attachment – сущность, добавленная в группу вложений;

- e.AttachmentInfo – информация о вложении. Содержит аргументы:
  - AttachedAt – дата и время добавления вложения;
  - AttachedBy – пользователь, который добавил вложение;
  - GroupName – имя группы, в которую добавлено вложение;
  - Owner – задача (задание, задание на приемку, уведомление), в которую добавлено вложение.

Удаление вложения (GroupDeleted)

Выполняется в разделяемом коде при удалении вложения из группы.

Назначение. Изменение свойств задачи (задания, задания на приемку, уведомления), из которой удаляется вложение. Например, можно изменить тему задачи.

Аргументы события:

- _obj – задача (задания, задания на приемку, уведомления), из которой удаляется вложение;
- e.Attachment – сущность, удаленная из группы вложений;
- e.AttachmentInfo – информация о вложении. Содержит аргументы:
  - AttachedAt – дата и время удаления вложения;
  - AttachedBy – пользователь, который удалил вложение;
  - GroupName – имя группы, из которой удалено вложение;
  - Owner – задача (задания, задания на приемку, уведомления), из которой удалено вложение.

Заполнение вложений (GroupPopulating)

Событие вызывается:

- при добавлении и удалении вложений в группе, которая указана в аргументе e.PopulateFrom. Текущая группа полностью очищается и заполняется полученными вложениями;
- при старте и рестарте задачи, создании заданий, а также при добавлении исполнителей. В этом случае в текущей группе добавленные вручную вложения остаются.

ПРИМЕЧАНИЕ. При отправке заданий параллельно всем исполнителям обработчик вызывается один раз для первого задания, а при отправке последовательно – на каждое новое задание.

Подробнее см. в разделе «Пример заполнения группы вложений».

Назначение. Получение вложений, связанных с вложениями из указанной группы, и заполнение ими текущей группы вложений.

Аргументы события:

- _obj – задача (задание, задание на приемку, уведомление), в которой заполняется группа вложений;

- e.PopulateFrom – вложения, которыми нужно заполнить группу:
  - Group – группа, при изменении содержимого которой перезаполняется связанными вложениями текущая группа;
  - GetAttachments – обработчик для получения вложений, связанных с документами из группы Group.

Если один из аргументов имеет значение null, то при выполнении события генерируется исключение.

ВАЖНО. Группу, в которой вызывается обработчик, нельзя указывать в аргументе Group.

Результаты выполнения заданий

Для настройки результатов выполнения и их приоритетов используйте одноименный узел – «Результаты выполнения». При добавлении результата выполнения автоматически появляется кнопка с тем же именем на панели действий.

ВАЖНО. Чтобы в веб-клиенте аналитик мог добавлять результаты выполнения, в редакторе типа задания установите флажок «Добавлять результаты выполнения в проводнике».

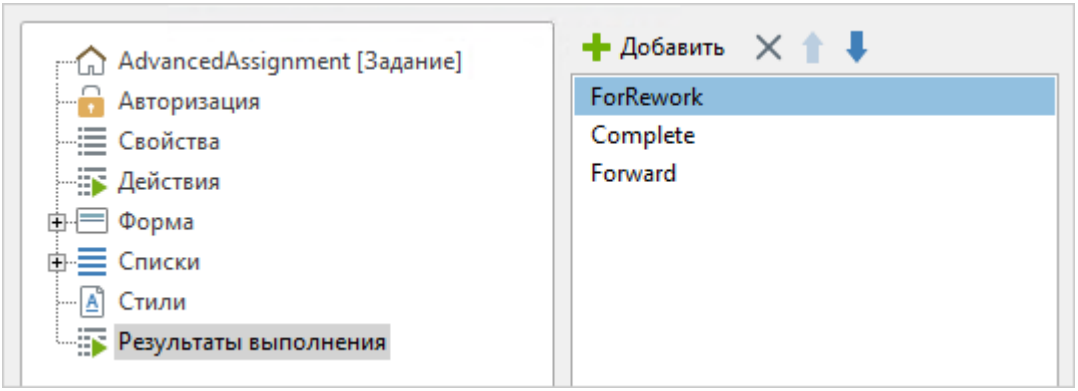
В перекрытом типе задания:

- чтобы использовать новый результат выполнения, нужно на своем *слое* разработки перекрыть родительский тип задачи, создать версию схемы и использовать добавленный результат выполнения;
- нельзя удалять результаты выполнения, добавленные в родительском типе задания, чтобы не нарушить логику работы *базовых решений*;
- нельзя добавлять результаты выполнения и изменять приоритет, если в типе задачи, где используется задание, установлен флажок **Запретить создавать версии схемы задачи в перекрытиях**.

Настройка результатов выполнения

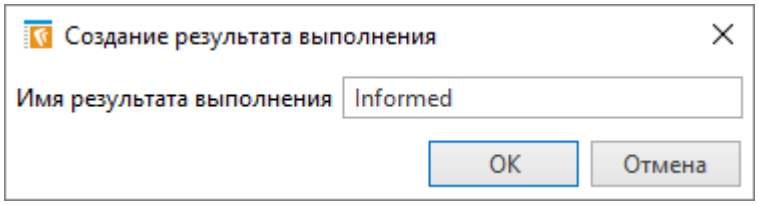
Предположим, при выполнении задания на рассмотрение документа исполнитель может вынести резолюцию, принять к сведению информацию в задаче или создать поручение по документу. В этом случае для типа задания создайте три результата выполнения: **Вынесена резолюция**, **Принято к сведению**, **Отправлено на исполнение**. Для этого:

1. Создайте тип сущности «Задание» (Assignment) и перейдите в узел «Результаты выполнения».

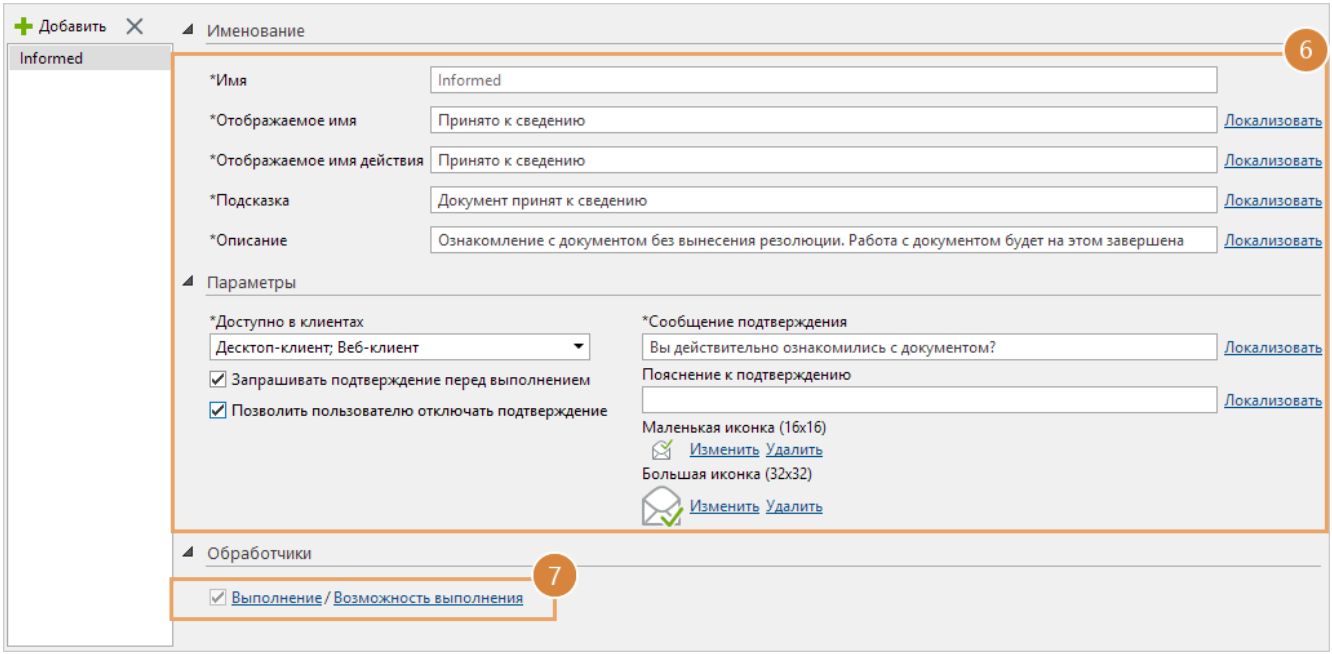


2. Удалите результат выполнения **Выполнить** (Complete), добавленный по умолчанию. Для этого выделите его в списке элементов и нажмите на кнопку .
3. Добавьте новые результаты выполнения. Для этого нажмите на кнопку **Добавить**.
4. В открывшемся окне заполните поле **Имя результата выполнения**. Имя должно быть уникальным в рамках задания, состоять из букв латинского алфавита и цифр.

   ПРИМЕЧАНИЕ. После создания результата выполнения имя будет доступно только для просмотра.



5. Нажмите на кнопку **OK**. В списке элементов появится добавленный результат.
6. Задайте настройки:

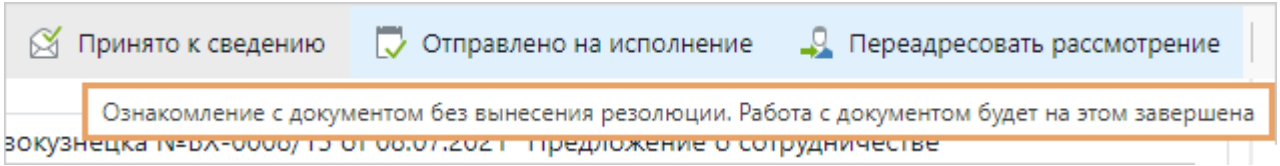


***Отображаемое имя** результата выполнения, которое отображается в области переписки после выполнения задания.

***Отображаемое имя действия**, которое отображается на панели действий в карточке задания.

***Подсказка**. Краткое описание результата выполнения. Отображается в заголовке всплывающей подсказки в десктоп-клиенте.

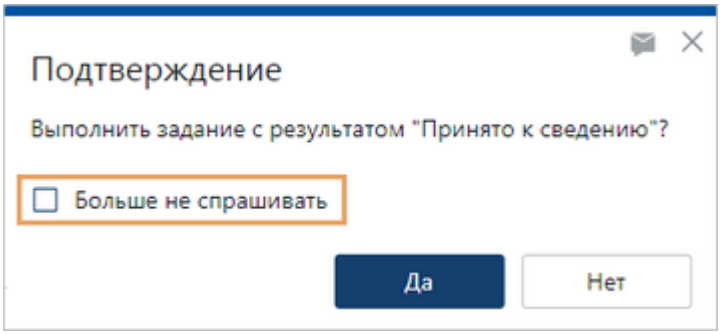
***Описание**. Подробное описание результата выполнения. Отображается во всплывающей подсказке к действию на панели действий:



***Доступно в клиентах**. По умолчанию добавленный результат выполнения задания доступен в десктоп-клиенте и в веб-клиенте. Если результат выполнения не поддерживается в одном из клиентов, то в выпадающем списке снимите флажок.

Запрашивать подтверждение перед выполнением. При выполнении задания будет появляться диалог подтверждения с текстом, который указан в полях ***Сообщение подтверждения** и **Пояснение к подтверждению**. Для локализации сообщения перейдите по ссылке **Локализовать**.

Позволить пользователю отключать подтверждение. Флажок доступен, если установлен флажок **Запрашивать подтверждение перед выполнением**. Если в среде разработки флажок установлен, в веб-клиенте в диалоге отображается флажок **Больше не спрашивать**. Таким образом пользователь может по собственному желанию отключить появление диалога подтверждения.



***Сообщение подтверждения**. Текст сообщения, которое пользователь увидит при выполнении задания, если установлен флажок **Запрашивать подтверждение перед выполнением**.

Пояснение к подтверждению. Дополнительная информация к тексту сообщения.

Для *локализации* отображаемых имен, подсказки, описания, сообщения подтверждения и пояснения к нему перейдите по ссылке **Локализовать**.

Маленькая иконка и **Большая иконка**. Маленькая иконка отображается в области переписки при выполнении задания с данным результатом. Большая иконка отображается на панели действий в карточке задания. Иконки должны быть в формате SVG и в размерах 16x16 и 32x32 соответственно. Иконки, используемые в *базовом решении*, можно найти в локальной папке исходных кодов. Расположение папки задается при установке системы. Например, в папке C:\DEVRX\Base\Sungero.Company\Sungero.Company.ClientBase\Icons можно найти файлы иконок модуля «Компания».

7. Добавьте обработчик события Выполнение/Возможность выполнения для результатов выполнения.

В результате на панели действий в карточке задания появятся кнопки для выполнения задания:

Приоритет результатов выполнения

В зависимости от приоритета результатов выполнения задания определяется результат выполнения блока. Если задания завершены с разными результатами, то итоговым считается тот, который имеет наивысший приоритет. Например, несколько сотрудников согласовали договор, а один – вернул на доработку. В таком случае блок завершается с результатом «На доработку», так как для него задан приоритет выше, чем для результата «Согласовать».

Чтобы повысить или понизить приоритет результата, используйте кнопки и . В результате порядок результатов выполнения применяется для новых блоков, добавленных на схему. В ранее настроенных блоках приоритет не изменяется. В дальнейшем аналитик может изменить приоритет при настройке схем бизнес-процессов в свойствах блока «Задание».

Наследуемые операции авторизации

От всех базовых типов задач, заданий, заданий на приемку, уведомлений наследуются следующие операции авторизации:

Операция	Описание
Read	Чтение
Create	Создание
Update	Изменение
Delete	Удаление
Manage	Изменение прав доступа
DelegateAccess	Передача прав доступа

Дополнительно для заданий и заданий на приемку наследуются операции авторизации:

Операция	Описание
Complete	Выполнение задания
Change text	Изменение текста задания

Наследуемые действия

От базового типа задачи Sungero.Workflow.Task наследуются:

действия с задачей

Действие	Описание	Где доступно?
CopyEntity	Создать задачу копированием и открыть карточку новой задачи	Карточка и список
DeleteEntity	Удалить задачу без возможности восстановления	Карточка и список
Save	Сохранить изменения в карточке	Карточка
SaveAndClose	Сохранить изменения и закрыть карточку	Карточка
SelectAndClose	Сохранить изменения, выбрать задачу и закрыть карточку	Карточка
Cancel	Отменить текущие изменения в карточке	Карточка
ShowSelectedEntity	Открыть карточку задачи из списка	Список
AddChildEntity	Добавить строку в текущую коллекцию	Дочерняя коллекция

Действие	Описание	Где доступно?
CopyChildEntity	Копировать строку и добавить новую в текущую коллекцию	Дочерняя коллекция
DeleteChildEntity	Удалить строки из текущей коллекции	Дочерняя коллекция
Abort	Прекратить задачу и остановить все задания по ней	Карточка
CreateSubtask	Создать подзадачу	Карточка и список
Restart	Рестартовать задачу и переоткрыть карточку	Карточка
Resume	Возобновить задачу и закрыть карточку. Выполнение работ начнется с того места, где они были оставлены	Карточка
Start	Отправить задачу и закрыть карточку	Карточка
StartedBy	Выбрать пользователя, от имени которого будет отправлена задача. Используется при замещении. В карточке задачи появится поле ***От**	Карточка
AddToFavoritesFolder	Копировать ссылку на задачу в папку «Избранное»	Карточка и список
AddToMyFolder	Копировать ссылку на задачу в **Мой Directum**	Карточка и список
SearchFolderConainsLinkToEntity	Найти папки, в которые вложена ссылка на задачу	Карточка и список
CopyToClipboard	Копировать ссылку на задачу в буфер обмена	Список
CopyToClipboardFromCard	Копировать ссылку на задачу в буфер обмена	Карточка
ChangeSearchCriteria	Изменить критерии поиска	Карточка и список
ChooseEntityAndCloseCollection	Выбрать задачу для свойства-ссылки и закрыть список записей	Список
CreateMeeting	Добавить в календарь. Создание файла в формате iCalendar для добавления встречи в календарь	Карточка
Refresh	Обновить содержимое списка в режиме выбора записей и в результатах поиска	Список
SetAccessRights	Открыть карточку задачи для настройки прав доступа. При выполнении действия открывается закладка «Доступ» в карточке сущности	Список
EditActiveText	Изменить текст в отправленной задаче. В перекрытых задачах можно запретить изменять текст. Например, если текст, введенный инициатором, используется в темах заданий. Для этого нужно скрыть кнопку **Изменить текст** на ленте	Карточка

действия с вложениями

Действие	Описание	Где доступно?
CreateAttachment	Создать сущность и добавить ссылку в область вложений	Карточка
PasteAttachment	Вставить ссылку из буфера обмена в область вложений	Карточка
DeleteAttachment	Удалить ссылку из области вложений	Карточка
CopyAttachmentLink	Копировать ссылку на вложение в буфер обмена	Карточка
SelectAllAttachments	Выделить все ссылки в области вложений	Карточка
OpenAttachments	Открыть карточку выделенного вложения	Карточка

От базовых типов задания Sungero.Workflow.Assignment, уведомления Sungero.Workflow.Notice и задания на приемку Sungero.Workflow.ReviewAssignment наследуются:

действия с заданием (заданием на приемку, уведомлением)

Действие	Описание	Где доступно?
CopyEntity	Создать задачу копированием и открыть карточку новой задачи. Действие не выполняется для задания в клиентском приложении	Карточка и список
DeleteEntity	Удалить задание без возможности восстановления. Действие не выполняется для задания в клиентском приложении	Карточка и список
Save	Сохранить изменения в карточке	Карточка
SaveAndClose	Сохранить изменения и закрыть карточку	Карточка
SelectAndClose	Сохранить изменения, выбрать задание и закрыть карточку	Карточка
Cancel	Отменить текущие изменения в карточке	Карточка
ShowSelectedEntity	Открыть карточку задания из списка	Список
AddChildEntity	Добавить строку в текущую коллекцию	Дочерняя коллекция
DeleteChildEntity	Удалить строку из текущей коллекции	Дочерняя коллекция
CopyChildEntity	Копировать строку и добавить новую в текущую коллекцию	Дочерняя коллекция
Accepted	Принять задание на приемку и закрыть карточку. Действие выполняется только в задании на приемку в клиентском приложении	Карточка

Действие	Описание	Где доступно?
CompleteByForce	Выполнить задание принудительно. Например, если в задании на свободное согласование документ во вложении открыт на редактирование, то при выполнении задания появится сообщение о том, что документ рекомендуется закрыть. При необходимости пользователь сможет выполнить задание не закрывая документ. Для этого нужно перейти по соответствующей ссылке, например **Согласовать**, в тексте сообщения. Действие доступно для задания и задания на приемку	Карточка
CompleteLeadingAssignment	Выполнить задание, из которого создана текущая подзадача (выполнение Ведущего задания). Действие доступно для задания на приемку. Например, в карточке задания на приемку работ по подзадаче может отображаться сообщение о том, что есть невыполненное ведущее задание. Чтобы выполнить задание, необходимо перейти по ссылке Выполнить в тексте сообщения	Карточка
ShowLeadingAssignment	Открыть карточку ведущего задания. Действие доступно для задания на приемку. Например, в карточке задания на приемку работ по подзадаче может отображаться сообщение о том, что есть невыполненное ведущее задание. Чтобы открыть задание, необходимо перейти по ссылке **Открыть** в тексте сообщения	Карточка
CreateSubtask	Создать подзадачу	Карточка и список
CreateMeeting	Добавить встречу в календаре Microsoft Outlook	Карточка
ForRework	Отправить на доработку. При этом карточка задания закроется и откроется карточка исходной задачи, где можно будет написать новый текст задачи, уточнить тему и исполнителей и отправить заново. Действие доступно для задания на приемку	Карточка
MarkRead	Пометить выбранные задания как прочитанные. Действие доступно в контекстном меню выбранного задания	Список
MarkUnread	Пометить выбранные задания как непрочитанные. Действие доступно в контекстном меню выбранного задания	Список
MarkReadUnread	Пометить выбранные непрочитанные задания как прочитанные и наоборот. Действие доступно на ленте проводника системы	Список

Действие	Описание	Где доступно?
MarkUnreadAndClose	Закрыть карточку и пометить задание (задание на приемку, уведомление) непрочитанным	Карточка
AddToFavoritesFolder	Копировать ссылку на задание в папку «Избранное»	Карточка и список
AddToMyFolder	Копировать ссылку на задание в **Мой Directum**	Карточка и список
SearchFoldersContainsLinkToEntity	Найти папки, в которые вложена ссылка на задание	Карточка и список
CopyToClipboard	Копировать ссылку на задание в буфер обмена из списка	Список
CopyToClipboardFromCard	Копировать ссылку на задание в буфер обмена из карточки задания	Карточка
ChangeSearchCriteria	Изменить критерии поиска	Список
ChooseEntityAndCloseCollection	Выбрать задание для свойства-ссылки и закрыть список записей	Список
Refresh	Обновить содержимое списка в режиме выбора записей и в результатах поиска	Список
SetAccessRights	Открыть карточку задания для настройки прав доступа. При выполнении действия открывается закладка «Доступ» в карточке сущности. Действие не выполняется в задании	Список
EditActiveText	Изменить текст в выполненном задании. В перекрытых заданиях можно запретить изменять текст. Например, если текст, введенный исполнителем, используется в темах уведомлений. Для этого нужно скрыть кнопку **Изменить текст** на ленте.	Карточка
StartWork	Взять конкурентное задание в работу	Карточка
ReturnUncompleted	Вернуть конкурентное задание невыполненным	Карточка

Копирование свойств и действий

Копирование позволяет быстро создать свойство или действие на основе существующего. Это полезно, если новый элемент отличается только одним параметром.

ПРИМЕЧАНИЕ. Копировать можно только свойства и действия у типов сущностей прикладного уровня. Например, тип сущности «Договор» (Contract) создан на прикладном уровне, можно скопировать его свойство **Дата документа** (DocumentDate), действие **Открыть оригинал** (OpenOriginal) и др.

Чтобы скопировать свойство или действие:

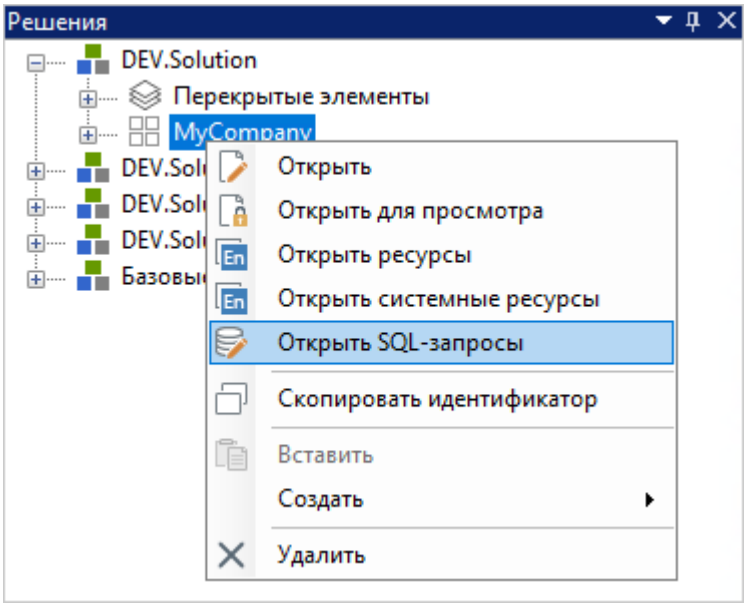
1. В редакторе типа сущности перейдите в узел «Свойства» или «Действия».
2. Выделите нужный элемент и в контекстном меню выберите пункт **Копировать**.
3. В контекстном меню узла «Свойства» или «Действия» выберите пункт **Вставить**.
4. Скорректируйте параметры и именование скопированного элемента.
5. При необходимости добавьте:
   - обработчики событий свойств или действий сущности, т.к. они не копируются;
   - свойства на карточку сущности. Для этого используйте узел «Форма» в редакторе типа сущности.
   - действия на ленту карточки сущности и списка сущностей. Для этого используйте узел «Лента» в редакторе типа сущности.

Добавление SQL-запроса

В среде разработки можно добавлять SQL-запросы при разработке модулей, типов сущностей и отчетов. Например, когда необходимо обратиться к базе данных.

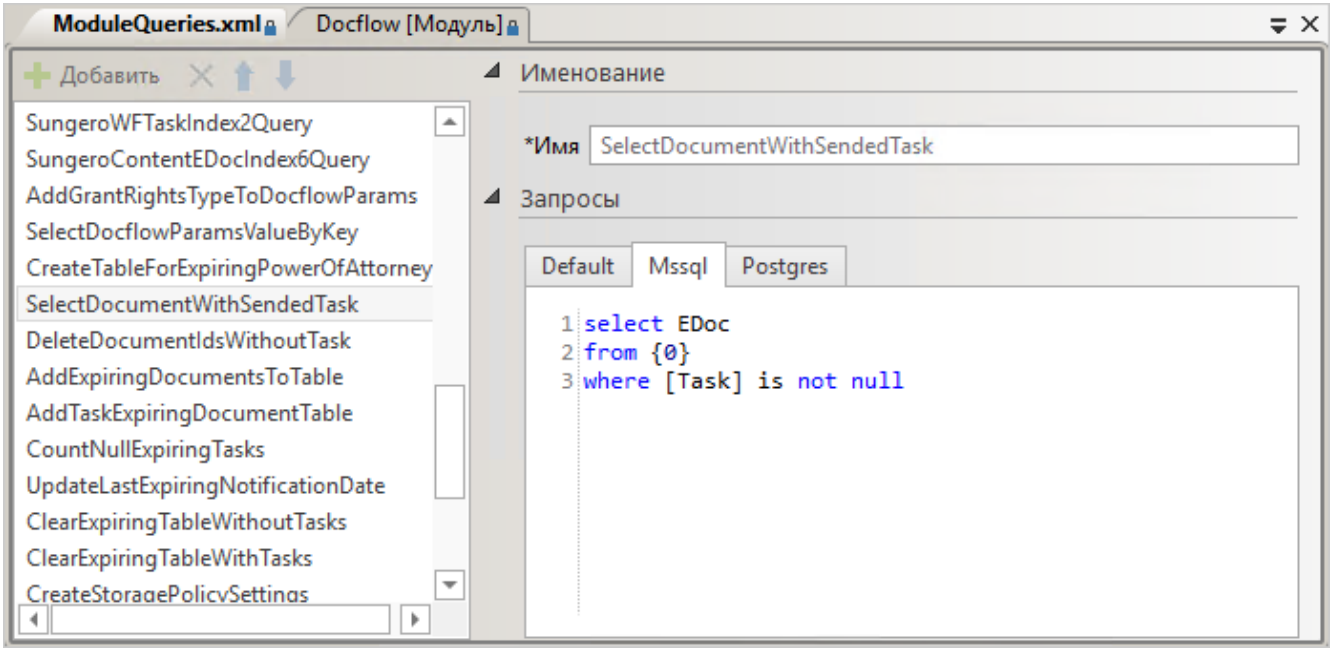
Чтобы добавить SQL-запрос:

1. В дереве решений выберите модуль, тип сущности или отчет и в контекстном меню выберите пункт **Открыть SQL-запросы**.



   ПРИМЕЧАНИЕ. Пункт **Открыть SQL-запросы** доступен также в контекстном меню для элементов *базового решения*. Например, таким образом вы можете посмотреть, какие SQL-запросы есть в модуле Docflow.
2. В открывшемся окне нажмите на кнопку **Добавить**.
3. Укажите имя SQL-запроса. Имя должно быть уникальным, состоять из букв латинского алфавита и цифр. Первый символ должен начинаться с буквы.
4. Напишите SQL-запрос на вкладке:
   - «Mssql» – если используется СУБД Microsoft SQL Server;
   - «Postgres» – если используется СУБД на основе PostgreSQL;
   - «Default» – если запрос подходит для всех СУБД.

Вы можете написать запрос сразу на нескольких вкладках. В последующем система сама определит нужный запрос, в зависимости от того, какая СУБД у вас используется.



Например, в модуле Docflow добавлен запрос **SelectDocumentWithSendedTask** для получения документов, по которым отправлены задачи:

Mssql

```
   select EDoc
from {0}
where [Task] is not null
```

Postgres

```
   select EDoc
from {0}
where Task is not null
```

В дальнейшем к добавленному запросу можно обратиться в серверном коде с помощью класса:

- **Queries.Module** – если запрос добавлен для модуля;
- **Queries.<Имя типа сущности>** – если запрос добавлен для типа сущности;
- **Queries.<Имя отчета>** – если запрос добавлен для отчета.

Например, в модуле Docflow добавлена серверная функция, в которой есть обращение к запросу **SelectDocumentWithSendedTask**:

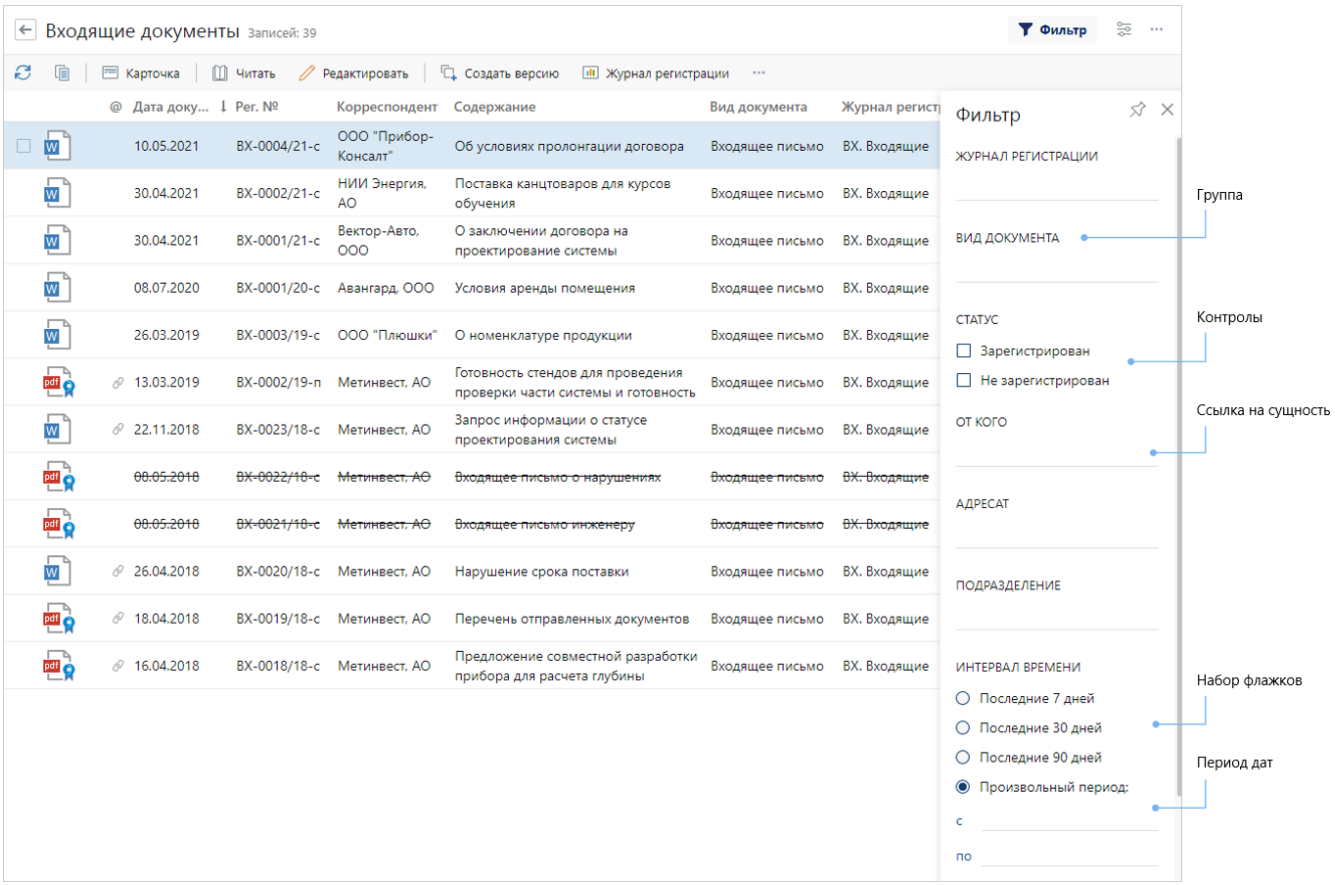
```
   /// <summary>
/// Получить документы, по которым уже отправлены задачи.
/// </summary>
/// <param name="expiringDocumentTableName">Имя таблицы, в которой хранятся ID
документов.</param>
/// <returns>Список ID документов, по которым задачи уже отправлены.</returns>
[Public]
public static List<int> GetDocumentsWithSendedTask(string
expiringDocumentTableName)
{
  var result = new List<int>();
  var commandText = string.Format(Queries.Module.SelectDocumentWithSendedTask,
expiringDocumentTableName);
  using (var command = SQL.GetCurrentConnection().CreateCommand())
  {
    try
```

```
    {
      command.CommandText = commandText;
      using (var rdr = command.ExecuteReader())
      {
        while (rdr.Read())
        result.Add(rdr.GetInt32(0));
      }
      return result;
    }
    catch (Exception ex)
    {
      Logger.Error("Error while getting array of docs with sent tasks", ex);
      return result;
    }
  }
}
```

Настройка панели фильтрации

Панель фильтрации используют для поиска записей по определенным критериям в вычисляемых папках или списках типов сущностей. Например, панель фильтрации есть в списке входящих документов. С ее помощью можно отфильтровать записи в списке по журналу регистрации, виду документа, дате регистрации или создания документа.

Панель состоит из групп, в каждой из которых находятся *контролы*. По значениям, указанным в контролах, записи фильтруются в папке или списке.



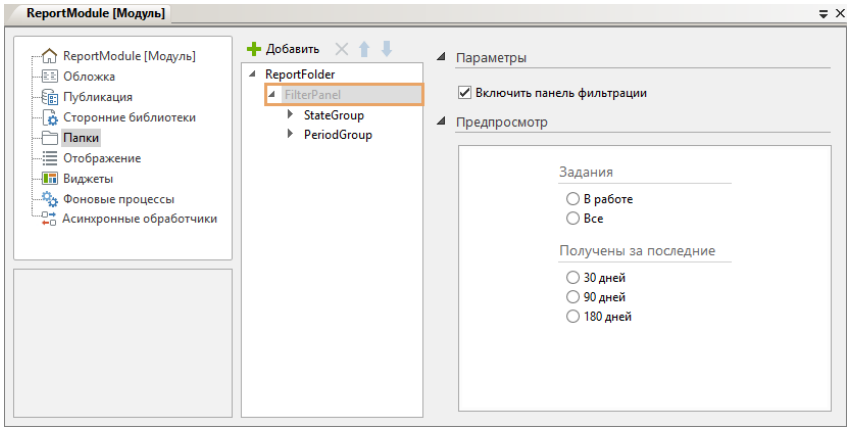
Выделяют следующие типы контролов:

- набор флажков. Используется для выбора одного или нескольких значений из фиксированного набора возможных значений. Может отображаться в виде набора флажков или переключателя;
- ссылка на сущность. Используется для выбора сущности заданного типа. Может отображаться в виде выпадающего списка или формы-списка;
- период дат. Используется для выбора интервала времени.

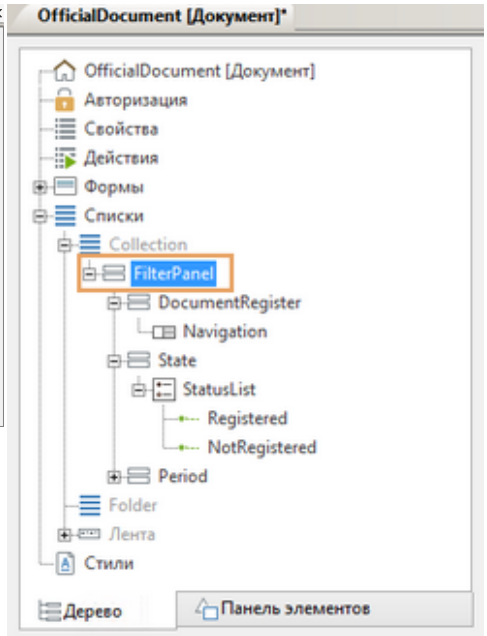
СОВЕТ. При выборе типа контрола используйте рекомендации.

Внешний вид панели фильтрации настраивается в узле «FilterPanel», который находится в редакторе модуля для папок и в редакторе типа сущности для списков:

Панель фильтрации для папок



Панель фильтрации для списков



По умолчанию узел «FilterPanel» пуст, если оставить его таким, то панель фильтрации не будет отображаться в списке.

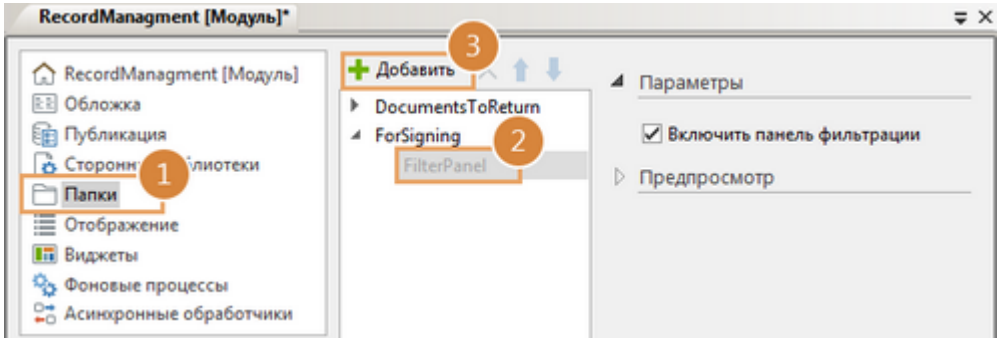
Панель фильтрации можно настроить с нуля или быстро создать копированием существующей. При необходимости отображение панели можно отключить.

Если при фильтрации записей на веб-сервер отправляется «тяжелый» запрос, который возвращает тысячи записей и влияет на быстродействие системы, добавляется проверка фильтра.

Добавление панели фильтрации

Чтобы добавить панель фильтрации для вычисляемой папки:

1. В редакторе модуля перейдите в узел «Папки».



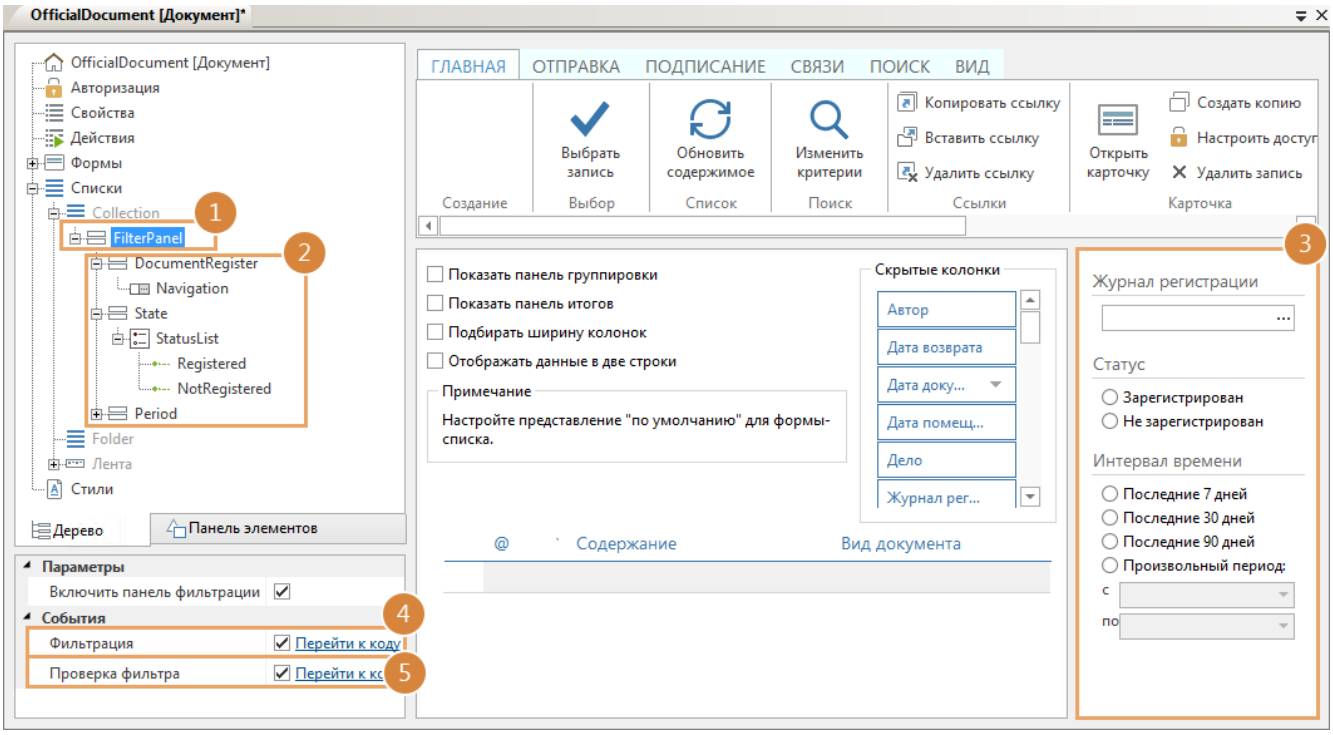
2. В списке элементов выделите узел «FilterPanel».
3. Добавьте необходимые группы и *контролы* с помощью кнопки **Добавить** и настройте их отображение.

Результат настройки панели фильтрации отобразится в области предпросмотра.

4. В событии Получение данных напишите условия фильтрации записей в вычисляемой папке. Подробнее см. «Пример настройки фильтрации записей в папке».

Чтобы добавить панель фильтрации для списка:

1. В дереве типа сущности последовательно раскройте узлы «Списки», «Collection» и выделите узел «FilterPanel».



2. С помощью контекстного меню добавьте необходимые группы и *контролы*, настройте их отображение.
3. Задайте свойства групп и контролов. Результат настройки панели фильтрации отобразится в области предпросмотра.
4. На панели свойств добавьте условия фильтрации записей в списке.
5. В событии «Фильтрация» напишите условия фильтрации записей в вычисляемой папке. При необходимости добавьте обработчик события для проверки фильтра. Подробнее см. «Пример настройки фильтрации записей в списке».

Копирование панели фильтрации

Можно быстро настроить внешний вид панели фильтрации с помощью ее копирования из другого списка типа сущности или вычисляемой папки. Например, можно скопировать панель фильтрации из вычисляемой папки модуля *базового решения* Directum RX или из списка типа сущности, который вы разработали ранее.

ВАЖНО. Копируется только внешний вид панели фильтрации: группы и контролы. Программный код условий фильтрации нужно задавать вручную.

Чтобы скопировать панель фильтрации:

1. В вычисляемой папке или в списке типа сущности выделите узел «FilterPanel» с настроенной панелью фильтрации.
2. В контекстном меню выберите пункт **Копировать**.

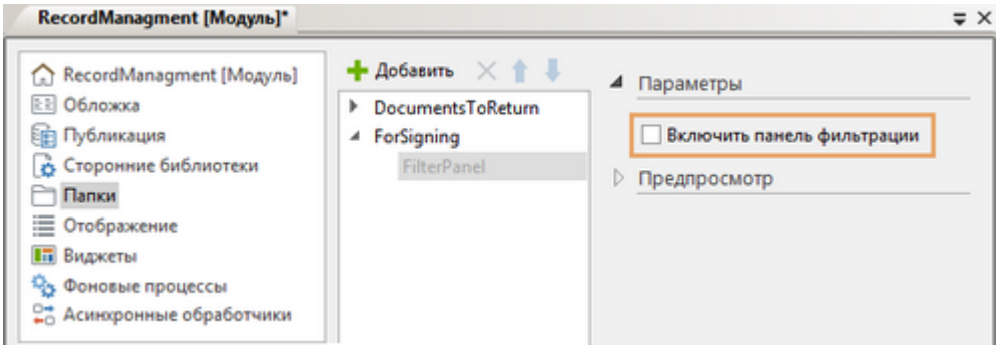
3. Перейдите в вычисляемую папку или список, для которого хотите добавить панель фильтрации:
   - для папки выделите узел «FilterPanel» с пустой панелью и в контекстном меню выберите пункт **Вставить**;
   - для списка в узле «Collection» выделите узел «FilterPanel» с пустой панелью фильтрации и в контекстном меню выберите пункт **Вставить**.

   В результате скопируется внешний вид панели.
4. При необходимости удалите ненужные группы и контролы или добавьте новые.
5. Добавьте условия фильтрации, которые задаются в событиях: для папок – Получение данных, для списков – Фильтрация.

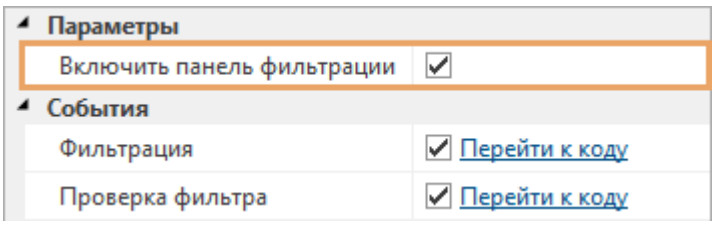
Скрытие панели фильтрации

Если панель фильтрации не нужна, ее можно отключить:

- для папок. Перейдите в узел «Папки» в списке элементов, для нужной вычисляемой папки выделите узел «FilterPanel» и снимите флажок **Включить панель фильтрации**:



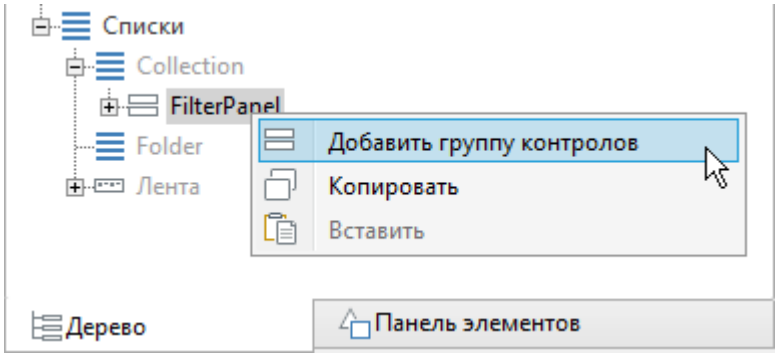
- для списков. В узле «Collection» выделите «FilterPanel» и на панели свойств снимите флажок **Включить панель фильтрации**:



Создание группы контролов

1. В дереве типа сущности последовательно раскройте узлы «Списки», «Collection» и выделите узел «FilterPanel».

2. В контекстном меню узла выберите пункт **Добавить группу контролов**.



3. Задайте свойства группы:

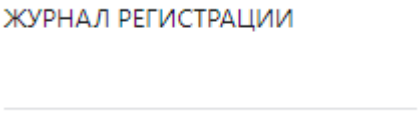
   ***Имя** группы. Должно быть уникальным в рамках панели фильтрации, состоять из букв латинского алфавита и цифр.

   Отображаемое имя. Название группы, которое отображается на панели фильтрации. Для *локализации* отображаемого имени нажмите на кнопку **Локализовать**.

   Отображать в панели фильтрации. Группа отображается на панели фильтрации. Если необходимо скрыть группу на панели фильтрации списка в перекрытии модуля, снимите флажок.

4. Добавьте в группу необходимые *контролы*.
5. Задайте условия фильтрации записей в списке.

В результате на панели фильтрации появится группа, например:



При необходимости измените расположение группы на панели. Для этого перетащите ее выше или ниже в дереве сущности. Места для вставки подсвечиваются желтым цветом. Результат настройки панели фильтрации отобразятся в области предпросмотра.

СОВЕТ. При группировке контролов используйте рекомендации.

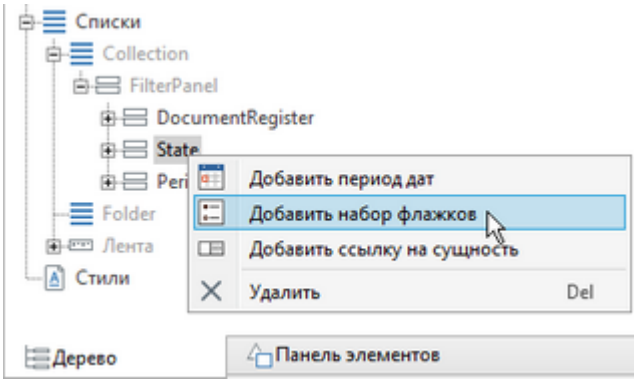
Добавление набора флажков

Предположим, что у сущностей в списке есть свойство **Состояние** с фиксированным набором значений: **Зарегистрирован**, **Не зарегистрирован** и **Зарезервирован**. Необходимо настроить фильтрацию записей в списке по значению свойства. Также может потребоваться фильтровать записи сразу по нескольким значениям. Для этого используйте *контрол* «Набор флажков» с возможностью выбора нескольких флажков одновременно.

Чтобы добавить набор флажков:

1. В дереве типа сущности последовательно раскройте узлы «Списки», «Collection» и «FilterPanel».

2. В контекстном меню группы выберите пункт **Добавить набор флажков**.



3. Задайте свойства набора флажков:

   ***Имя** набора флажков. Должно быть уникальным в рамках группы, состоять из букв латинского алфавита и цифр.

   Отображаемое имя. Название набора флажков, которое будет отображаться на панели фильтрации. Для *локализации* отображаемого имени нажмите на кнопку **Локализовать**.

   ПРИМЕЧАНИЕ. Чтобы в качестве отображаемого имени набора флажков использовалось название группы, оставьте поле **Отображаемое имя** незаполненным.

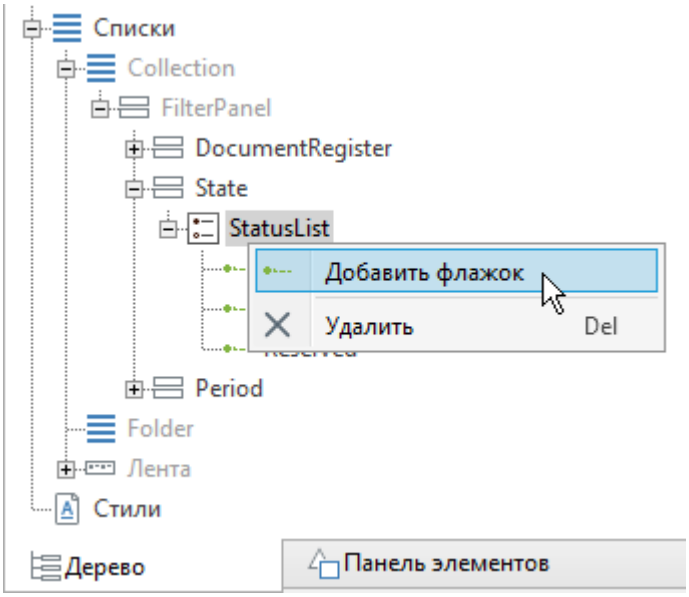
   Разрешить множественное выделение флажков. Для фильтрации списка можно будет установить несколько флажков одновременно. Например, таким образом можно будет фильтровать список по статусам **Зарегистрирован** и **Зарезервирован**.

   Контрол «флажок» для управления доступностью. При настройке панели фильтрации возможны варианты, когда нужно, чтобы контрол с набором флажков был:

   - доступен всегда. В этом случае оставьте поле незаполненным;
   - доступен только если установлен флажок из другого набора флажков. В этом случае укажите в поле имя флажка, который нужно будет установить на панели фильтрации. В выпадающем списке для выбора доступны флажки из всех наборов, которые добавлены на панель фильтрации, кроме флажков текущего набора. Например, аналогично настраивается контрол «Период дат», который становится доступным, когда установлен один из флажков для фильтрации записей по наиболее частым периодам.

   Отображать в панели фильтрации. Набор флажков будет отображаться на панели фильтрации. Если необходимо скрыть набор флажков на панели фильтрации списка в перекрытии модуля, снимите флажок.

4. Добавьте в контрол «Набор флажков» необходимые флажки. Для этого в контекстном меню контрола выберите пункт **Добавить флажок**.



5. Задайте свойства флажка:

   ***Имя** флажка. Должно быть уникальным в рамках панели фильтрации, состоять из букв латинского алфавита и цифр.

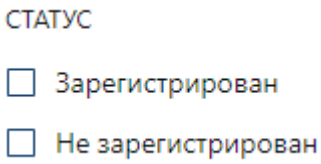
   ***Отображаемое имя**. Название флажка, которое будет отображаться на панели фильтрации. Для *локализации* отображаемого имени нажмите на кнопку **Локализовать**.

   Выделен по умолчанию. На панели фильтрации флажок будет установлен по умолчанию. Рекомендуется устанавливать для значений, по которым фильтрация будет выполняться наиболее часто. Например, в данном случае можно настроить отображение только записей со статусом **Зарегистрирован**.

   Отображать в панели фильтрации. Флажок будет отображаться на панели фильтрации.

6. Задайте условия фильтрации записей в списке.

В результате на панели фильтрации появится контрол с набором флажков, например:



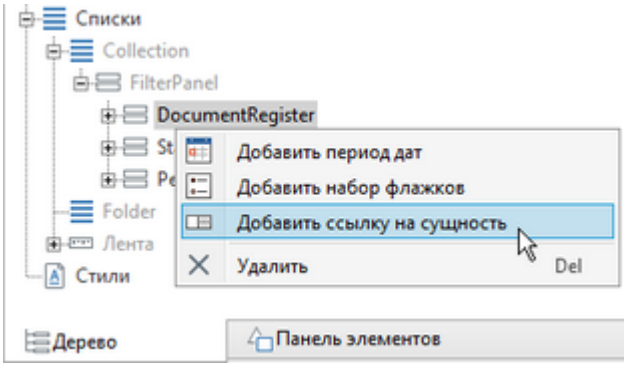
При необходимости измените расположение контрола на панели фильтрации. Для этого перетащите контрол выше или ниже в дереве сущности. Места для вставки подсвечиваются желтым цветом. Перемещать контрол можно только в пределах группы, где он был создан. Аналогично можно изменять расположение флажков в пределах набора флажков.

Добавление ссылки на сущность

Предположим, что у сущностей в списке есть свойство **Журнал регистрации**. Возможные значения свойства выбираются из соответствующего справочника. Необходимо настроить фильтрацию записей в списке по журналу регистрации. Для этого используйте *контрол* «Ссылка на сущность».

Чтобы добавить контрол со ссылкой на сущность:

1. В дереве типа сущности последовательно раскройте узлы «Списки», «Collection» и «FilterPanel».
2. В контекстном меню группы выберите пункт **Добавить ссылку на сущность**.



3. Задайте свойства контрол со ссылкой на сущность:

   ***Имя** контрола. Должно быть уникальным в рамках группы, состоять из букв латинского алфавита и цифр.

   Отображаемое имя. Название контрола, которое будет отображаться на панели фильтрации. Для *локализации* отображаемого имени нажмите на кнопку **Локализовать**.

   ПРИМЕЧАНИЕ. Чтобы в качестве отображаемого имени контрола использовалось название группы, оставьте поле **Отображаемое имя** незаполненным.

   ***Тип сущности**. Сущности указанного типа будут использоваться в качестве возможных значений контрола.

   ***Способ выбора значения**. Возможные значения:

   - **Выпадающий список**. Значения будут отображаться в виде выпадающего списка. Рекомендуется устанавливать, если сущностей нужного типа немного (до 20), их количество не будет меняться и у типа сущности мало свойств;
   - **Форма список**. Значения свойства-ссылки будут отображаться в отдельном окне. Рекомендуется устанавливать, если сущностей нужного типа много, их количество будет расти и у типа сущности много свойств.

   ***Количество строк**, которое будет использоваться для отображения контрола Возможные значения: **Одна строка** (значение по умолчанию), **Две строки**.

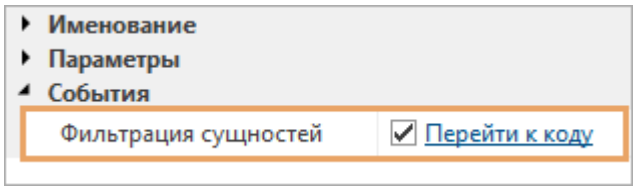
   Контрол «флажок» для управления доступностью. При настройке панели фильтрации возможны варианты, когда нужно, чтобы контрол со ссылкой на сущность был:

   - доступен всегда. В этом случае оставьте поле незаполненным;

- доступен только если установлен флажок из некоторого набора флажков. В этом случае укажите в поле имя флажка, который нужно будет установить на панели фильтрации. В выпадающем списке для выбора доступны флажки из всех наборов, которые добавлены на панель фильтрации. Например, аналогично настраивается контрол «Период дат», который становится доступным, когда установлен один из флажков для фильтрации записей по наиболее частым периодам.

Отображать в панели фильтрации. Контрол будет отображаться на панели фильтрации. Если необходимо скрыть контрол на панели фильтрации списка в перекрытии модуля, снимите флажок.

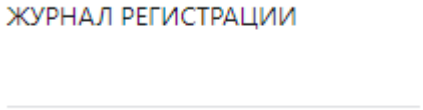
4. Если необходимо фильтровать возможные значения контрола «Ссылка на сущность», задайте обработчик события **Фильтрация сущностей**. Для этого установите флажок рядом с событием и перейдите по ссылке **Перейти к коду**.



Например, в событии можно задать обработку, чтобы при выборе журнала регистрации отображались только те журналы, в которых регистрируются документы определенного типа.

5. Задайте условия фильтрации записей в списке.

В результате на панели фильтрации появится контрол, в котором можно будет выбрать определенный тип сущности, например:



При необходимости измените расположение контрола на панели фильтрации. Для этого перетащите контрол выше или ниже в дереве сущности. Места для вставки подсвечиваются желтым цветом. Перемещать контрол можно только в пределах группы, где он был создан.

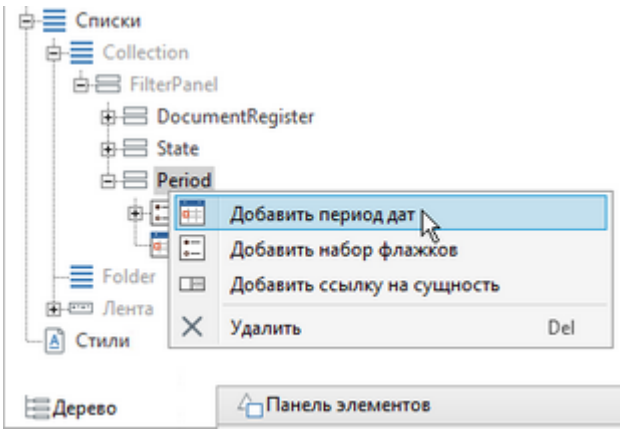
Добавление периода дат

Предположим, что у сущностей в списке есть свойства **Дата регистрации** и **Дата создания**. Необходимо настроить фильтрацию записей в списке по значениям этих свойств. При этом есть наиболее частые периоды, по которым может быть отфильтрован список: **Последние 7 дней**, **Последние 30 дней** и **Последние 90 дней**. Используйте *контрол* «Набор флажков» для указания одного из предопределенных периодов. Для указания произвольного периода используйте контрол «Период дат».

Чтобы добавить контролы:

1. В дереве типа сущности последовательно раскройте узлы «Списки», «Collection» и «FilterPanel».
2. Добавьте в необходимую группу набор флажков и создайте флажки, соответствующие наиболее частым периодам фильтрации.

3. В контекстном меню группы выберите пункт **Добавить период дат**.



4. Задайте свойства контрола с периодом дат:

   ***Имя** контрола. Должно быть уникальным в рамках группы, состоять из букв латинского алфавита и цифр.

   Отображаемое имя. Название контрола, которое будет отображаться на панели фильтрации. Для *локализации* отображаемого имени нажмите на кнопку **Локализовать**.

   ПРИМЕЧАНИЕ. Чтобы в качестве отображаемого имени контрола использовалось название флажка из набора, оставьте поле **Отображаемое имя** незаполненным.

   ***Формат даты**, отображаемой в контроле. Возможные значения: **Дата и время**, **Только время**, **Только дата**, **Год и месяц**, **Только год**.

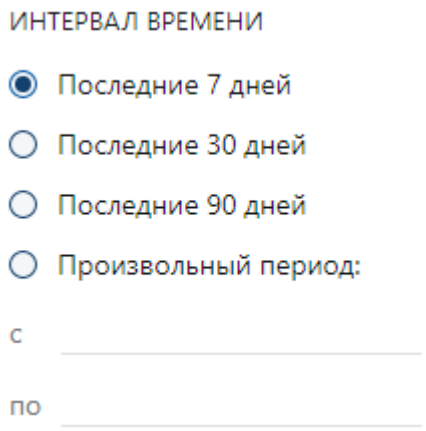
   Контрол «флажок» для управления доступностью данного контрола. При настройке панели фильтрации возможны варианты, когда нужно, чтобы данный контрол был:

   - доступен всегда. В этом случае оставьте поле незаполненным;
   - доступен только если установлен флажок из некоторого набора флажков. В этом случае укажите в поле имя флажка, который нужно будет установить на панели фильтрации. В выпадающем списке для выбора доступны флажки из всех наборов, которые добавлены на панель фильтрации. Например, укажите в поле имя флажка, соответствующего интервалу **Произвольный период**.

   Отображать в панели фильтрации. Контрол будет отображаться на панели фильтрации. Если необходимо скрыть контрол на панели фильтрации списка в перекрытии модуля, снимите флажок.

5. Задайте условия фильтрации записей в списке.

В результате на панели фильтрации появится группа фильтров по периоду дат, например:



При необходимости измените расположение контролов на панели фильтрации. Для этого перетащите один из контролов выше или ниже в дереве сущности. Места для вставки подсвечиваются желтым цветом. Перемещать контрол можно только в пределах группы, где он был создан.

Рекомендации по настройке панели фильтрации

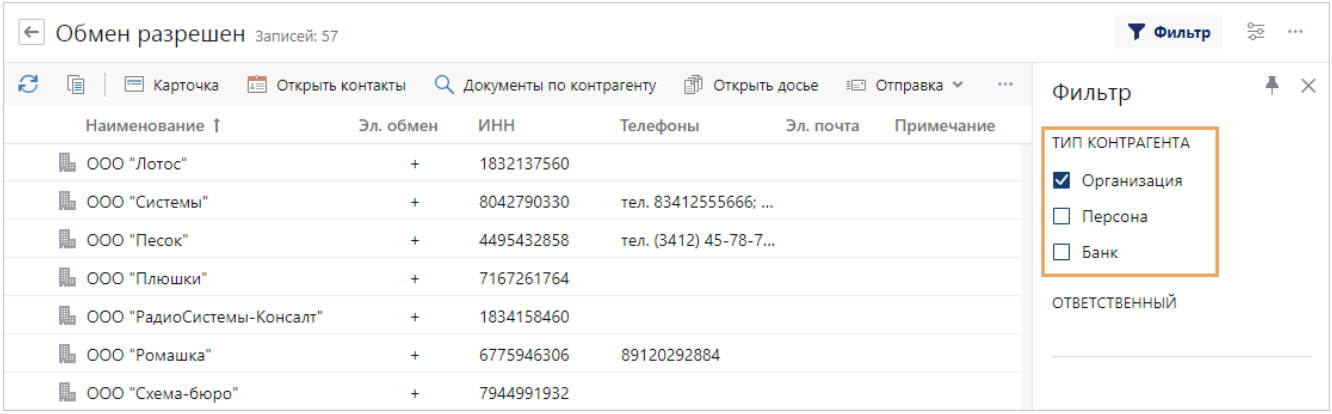
Для удобного использования и поддержания единого стиля системы при настройке панели фильтрации соблюдайте рекомендации:

- выбирайте типы контролов в зависимости от решаемой задачи;
- используйте стандартную группировку контролов;
- выносите на панель значения, по которым записи фильтруются наиболее часто при выполнении типовых сценариев работы.

Панель фильтрации рекомендуется добавлять, когда:

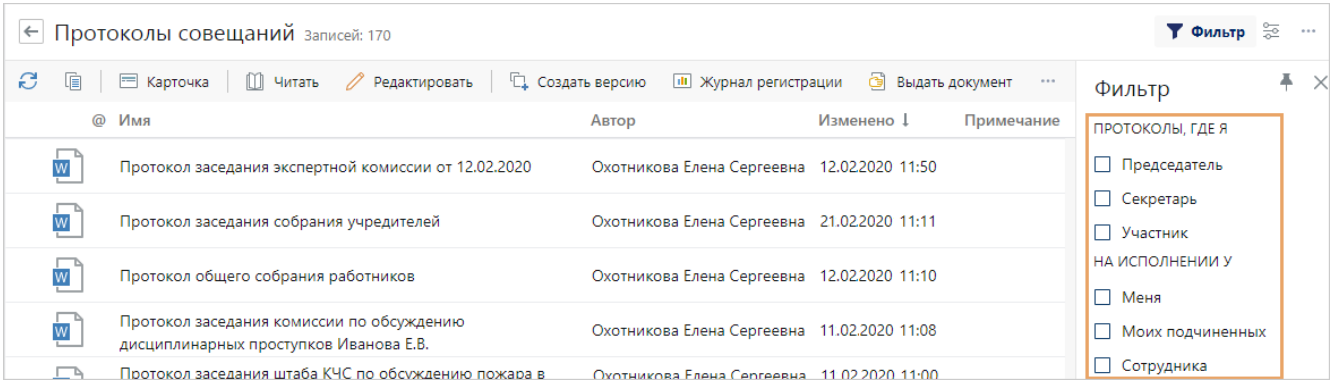
- используется список, в котором много сущностей и есть типовые сценарии работы с фильтрацией записей. Например, типовой сценарий для списка входящих документов – оставить только документы, зарегистрированные в конкретном журнале регистрации в определенный период времени;
- нужны такие выборки записей, которые сотрудник не может настроить через интерфейс: просроченные задания, фильтрация по типам сущностей и т. д.

  Пример. Фильтрация по типу контрагента в списке «Обмен разрешен»:



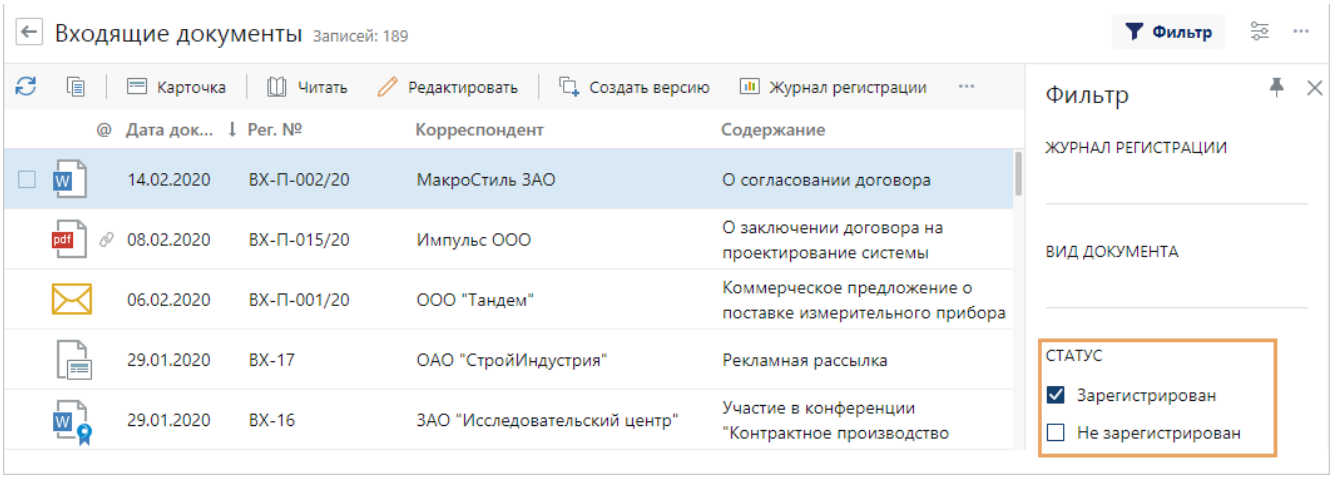
- в зависимости от ролей пользователей есть различные сценарии работы и нужно использовать несколько типовых представлений одного списка.

  Пример. Фильтрация в списке «Протоколы совещаний» позволяет найти протоколы, где текущий пользователь выступает в роли секретаря или исполнителя:



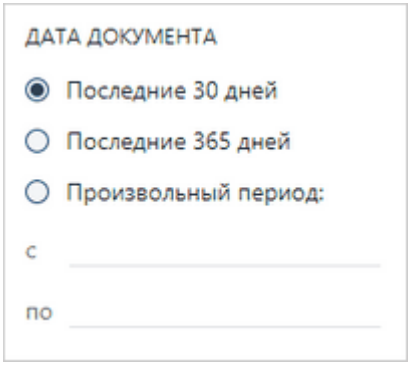
- нужна фильтрация по свойству, которое не вынесено в столбцы списка.

  Пример. В списке «Входящие документы» нет столбца со статусом документа, свойство вынесено на панель фильтрации:



- нужна фильтрация по временному периоду, который сложно настроить с помощью фильтрации дат в столбце.

  Пример. Фильтрация по дате документа в списке «Реестр договоров»:



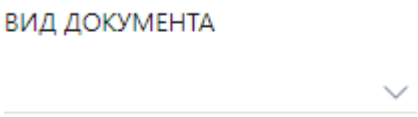
Типы контролов

Ссылка на сущность

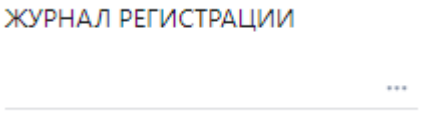
Контрол используется для выбора сущности заданного типа. Может отображаться в виде:

- выпадающего списка;
- отдельной формы-списка, которая открывается при нажатии на кнопку ⋯.

Выпадающий список



Форма-список



Рекомендуется скрывать значения, которые неактуальны для текущего списка. Например, в списке «Реестр договоров» можно выбрать только договорные виды документов:

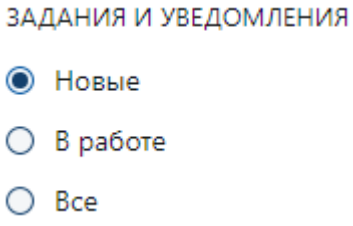
```
   // Показать только договорные виды документов.
var query = query
  .Where(d => d.Status == CoreEntities.DatabookEntry.Status.Active)
  .Where(d => d.DocumentFlow == Docflow.DocumentKind.DocumentFlow.Contracts);
```

Набор флажков

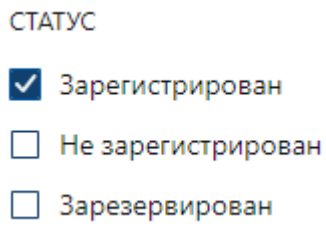
Контрол используется для выбора одного или нескольких значений из фиксированного набора возможных значений. Может отображаться в виде:

- переключателя – для выбора одного значения;
- набора флажков – для выбора нескольких значений одновременно.

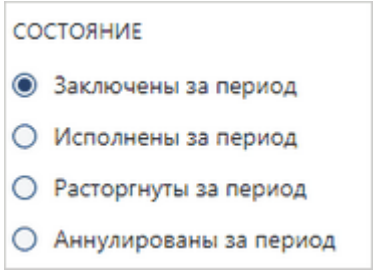
Переключатель



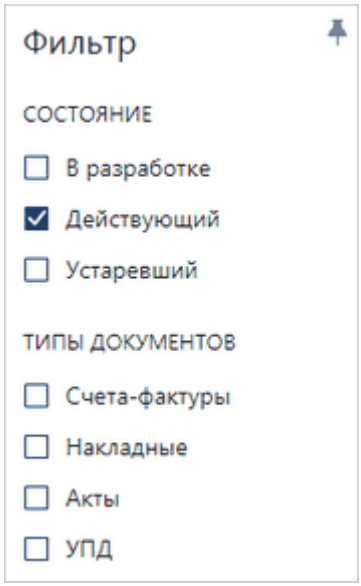
Набор флажков



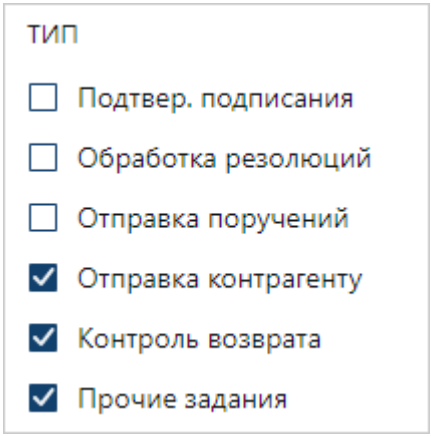
Переключатель рекомендуется использовать, если критерии фильтрации взаимно исключают друг друга:



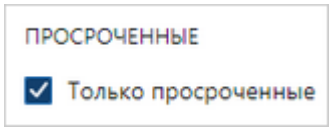
Набор флажков рекомендуется использовать, если у свойства сущности есть несколько значений и нужно выбирать их в различных комбинациях. Например, в списке «Финансовые документы» используется фильтрация по состоянию и типам документов:



Для флажков рекомендуется выбирать такие значения, чтобы они покрывали все возможные значения связанного критерия. Если значений критерия слишком много, то выделите 3-4 наиболее популярных варианта, а остальные отнесите к флажку «Прочие»:



Набор флажков также можно использовать для свойства типа Логическое. Например, нарушение срока возврата – свойство логического типа. Панель фильтрации в списке «Документы к возврату» позволяет найти выданные документы, у которых истек возврата:



При написании обработчика фильтрации рекомендуется учитывать выбранные флажки по логическому условию «ИЛИ». При этом, если не выбран ни один флажок, условие по этому набору флажков накладывать не нужно. Например:

```
// Если выбрано хоть одно значение, то искать записи, в которых встречается
```

```
// хотя бы один выбранный статус.
if (_filter.Registered || _filter.Reserved || _filter.NotRegistered)
  query = query.Where(l =>
    (_filter.Registered && l.RegistrationState ==
        OfficialDocument.RegistrationState.Registered) ||
    (_filter.Reserved && l.RegistrationState ==
        OfficialDocument.RegistrationState.Reserved) ||
    (_filter.NotRegistered && l.RegistrationState ==
        OfficialDocument.RegistrationState.NotRegistered));
```

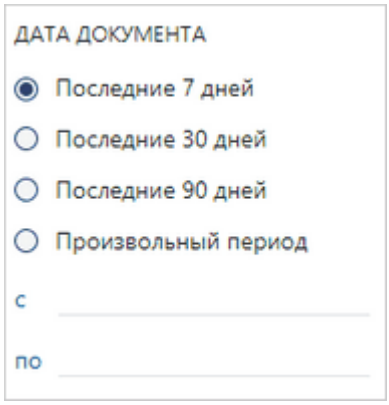
Период дат

Контрол используется для выбора интервала времени. Для удобной фильтрации по датам рекомендуется добавить предопределенные периоды с учетом контекста списка и возможность ввода произвольного периода.

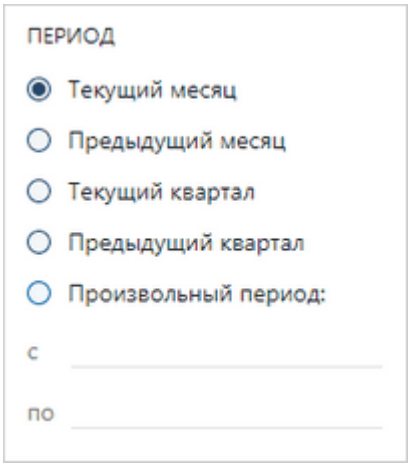
Используйте контрол «Период дат» вместе с контролом «Набор флажков». В этом случае флажки набора соответствуют наиболее частым периодам, по которым фильтруются сущности в списке: последние 7/30/90 дней и др. При этом «Период дат» становится доступным, если установлен определенный флажок, например, **Произвольный период**.

Рекомендуемые предопределенные периоды:

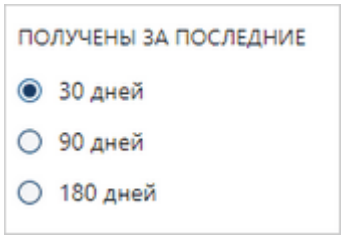
- для большинства типов документов в качестве периодов используются последние 7/30/90 дней. Подобные относительные периоды удобны тем, что позволяют вывести недавние объекты за постоянный период.



- для учетных документов обычно настраиваются календарные периоды: текущий месяц, предыдущий месяц, текущий квартал, предыдущий квартал и т. п. Такие периоды часто используются в бухгалтерском учете.



- для задач и заданий обычно используются периоды 30/90/180 дней.



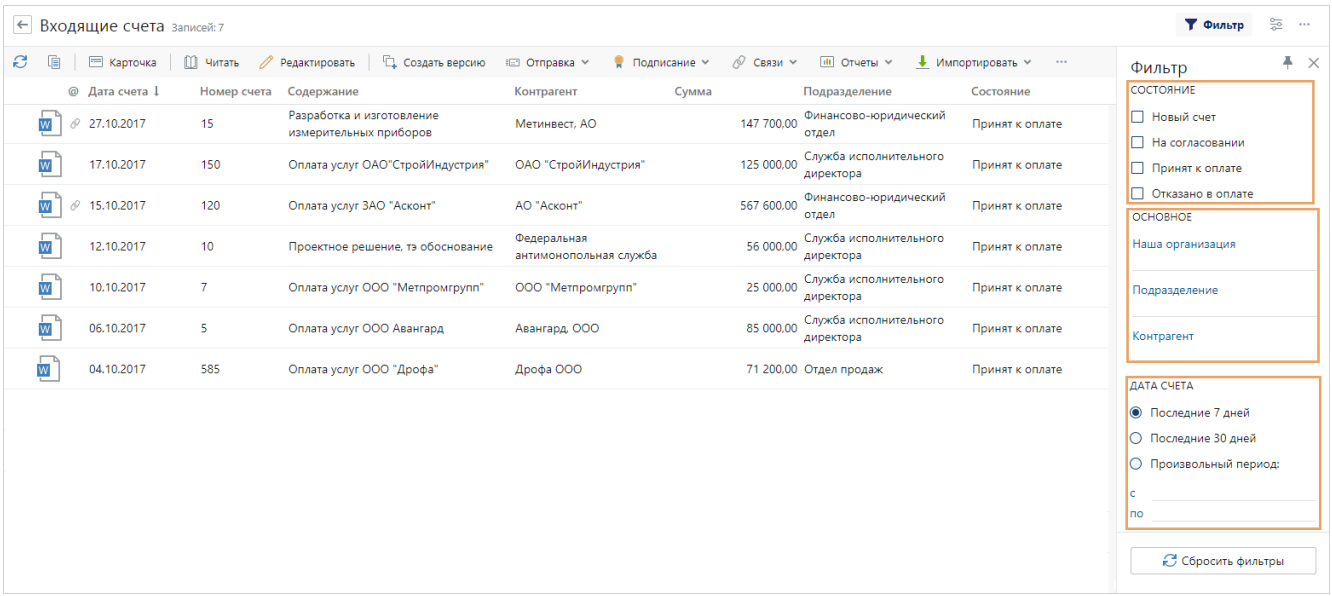
Рекомендуется оставить возможность указать произвольный период дат, если это не критично для скорости открытия списка. Например, для ускорения открытия такая фильтрация отключена в папках потоков входящих заданий.

Группировка контролов

Для панелей фильтрации рекомендуется использовать стандартную группировку контролов:

- в верхней части размещайте фильтры по состоянию или по типам сущностей списка;
- в нижней части размещайте фильтры по датам;
- в средней части размещайте остальные контролы, при этом самые важные – сверху.

На панели размещайте только часто используемые фильтры, чтобы не появлялась вертикальная полоса прокрутки.



Располагайте однотипные элементы панели фильтрации одинаково. Например, в списках входящих и исходящих документов есть группа для фильтрации по интервалу времени. В этом случае группа должна выглядеть однотипно и находиться в одном и том же месте панели фильтрации в обоих списках.

При написании обработчика фильтрации объединяйте группы между собой по логическому условию «И», т.е. складывайте условия. Например, наложение фильтров используется в списке «Входящие счета»:

```
// Фильтрация по состояниям.
```

```
if ((_filter.Draft || _filter.OnApproval || _filter.PayAccepted ||
_filter.PayRejected) &&
    !(_filter.Draft && _filter.OnApproval && _filter.PayAccepted &&
_filter.PayRejected))
...
// Фильтрация по группе "Основные" накладывается к предыдущим фильтрам.
if (_filter.Counterparty != null)
  query = query.Where(x => Equals(x.Counterparty, _filter.Counterparty));
...
// Фильтрация по датам накладывается к предыдущим фильтрам.
if (_filter.Last7daysInvoice)
  query = query.Where(c => (c.Date ?? c.Created) >= Calendar.Today.AddDays(-7));
```

Значения критериев

Выносите на панель значения, по которым записи фильтруются наиболее часто при выполнении типовых сценариев работы. Например, типовой сценарий – оставить только входящие письма, зарегистрированные в конкретном журнале регистрации. В этом случае на панель фильтрации выносятся свойства **Вид документа** и **Журнал регистрации**.

Для панели фильтрации рекомендуется выставлять значения по умолчанию:

- период – минимальный период среди предопределенных;
- состояние записей – **Действующие**, если есть фильтрация по действующим и закрытым записям;
- флажки в группах – в большинстве случаев сняты;
- представление списка – основное, редко используемые записи скрыты. Например, по умолчанию в списке заданий показываются только задания в работе.

Рекомендации для панели фильтрации в папке потока:

- добавьте группу с фильтрацией заданий по состоянию: **В работе**, **Все**. Значение по умолчанию **В работе**;
- добавьте фильтрацию по дате создания с тремя вариантами: **30 дней**, **90 дней**, **180 дней**. Значение по умолчанию **90 дней**. Это уменьшает размер выборки и исключает из нее старые задания. Такое ограничение ускоряет открытие списка и обновление количества входящих заданий;
- если выбрана фильтрация по состоянию **В работе**, то период дат сделайте недоступным для изменения и не учитывайте его при формировании выборки.

Отчеты

Отчеты позволяют получить данные из системы и отобразить их в удобном виде во внешних приложениях.

Отчеты формируются по данным:

- конкретной сущности (отчеты типов сущностей). Например, отчет **Лист согласования** является отчетом типа сущности «Официальный документ». Формируется по данным только того документа, в карточке которого он был вызван;

- по всем данным системы (отчеты модулей). Например, отчет **Контроль возврата документов** является отчетом модуля «Делопроизводство». Формируется по данным всех документов системы.

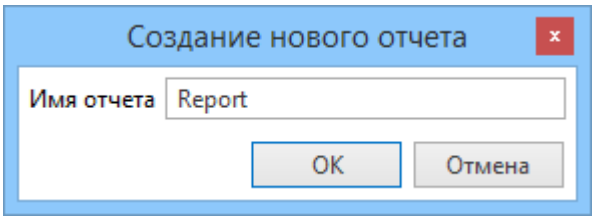
Данные для отчета берутся из источников данных, которые формируются на основе типов сущностей или SQL-запросов.

Для настройки внешнего вида отчета в редакторе настраивается макет отчета. Макет разрабатывается с помощью дизайнера отчетов FastReport.Net.

ВАЖНО. Некоторые возможности FastReport.Net отключены в среде разработки Directum RX, например возможность предварительного просмотра, использование скриптов, печать, экспорт и отправка отчета по e-mail непосредственно из дизайнера.

Если требуется разработать отчет, похожий на уже существующий в решении, создайте его копированием. Чтобы создать отчет:

1. Изучите рекомендации на сайте Directum Club. Изучите рекомендации по разработке отчетов в Linux, если Directum RX используется на компьютерах с этой операционной системой.
2. В дереве решений выделите модуль, к которому будет относиться отчет, или тип сущности, данные которого будут использованы в отчете. В контекстном меню последовательно выберите пункты **Создать** и **Отчет**.



3. В открывшемся окне укажите имя отчета. Имя должно быть уникальным, состоять из букв латинского алфавита и цифр.
4. Нажмите на кнопку **ОК**. Отчет появится в дереве решений, и откроется его редактор.
5. Заполните необходимые параметры в редакторе отчета.
6. Добавьте параметры отчета.
7. Добавьте источник данных, из которого будут получены данные для отчета.
8. Создайте макет внешнего вида страниц отчета.
9. Настройте отображение отчета или на обложку модуля добавьте действие для его запуска.
10. Чтобы пользователи смогли сформировать созданный отчет, при инициализации модуля выдайте на него права доступа. Для этого в узле «Модуль», в группе «События» установите флажок рядом с событием **Инициализация** и добавьте функцию инициализации. Например, при инициализации модуля Docflow права на отчеты выдаются следующим образом:

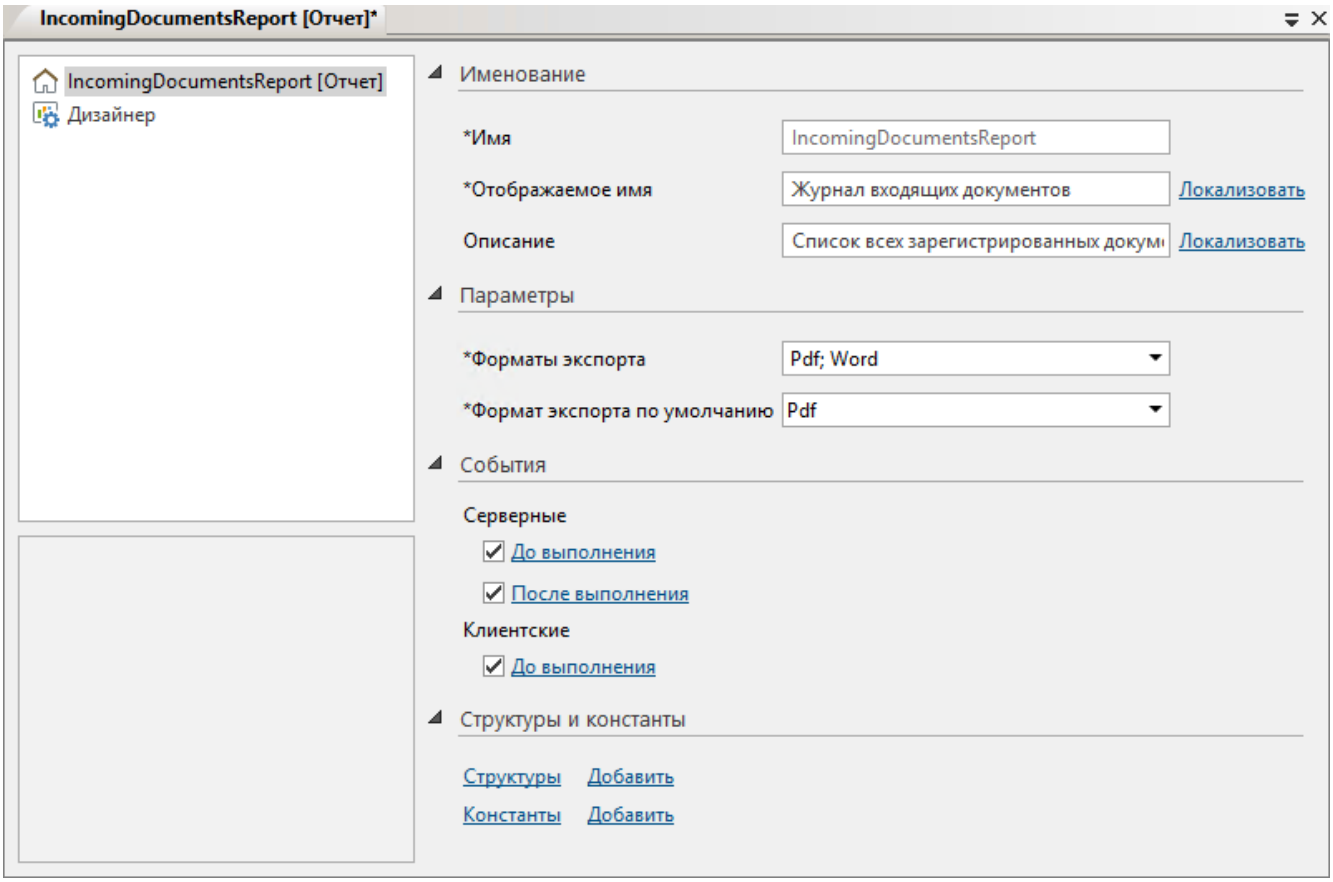
```
var allUsers = Roles.AllUsers;
  Reports.AccessRights.Grant(Reports.GetApprovalRulesConsolidatedReport().Info,
   allUsers, DefaultReportAccessRightsTypes.Execute);
```

Редактор отчета

Чтобы задать или изменить настройки отчета, щелкните два раза левой кнопкой мыши его название в дереве решений. Откроется редактор отчета.

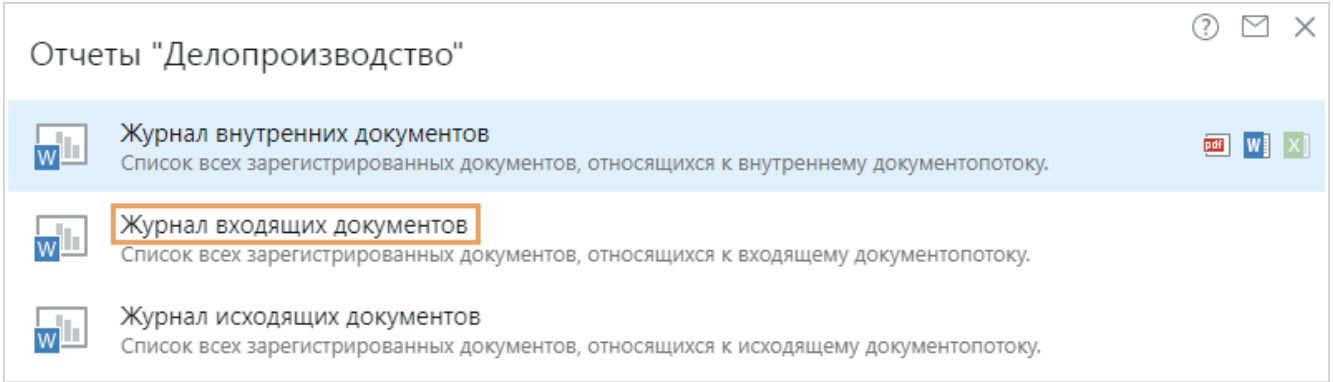
В редакторе задаются свойства и настраивается макет отчета. Редактор состоит из узлов «Отчет» и «Дизайнер».

При открытии редактора отображается узел «Отчет», в котором задаются именование, параметры, обработчики событий, структуры и константы для отчета.

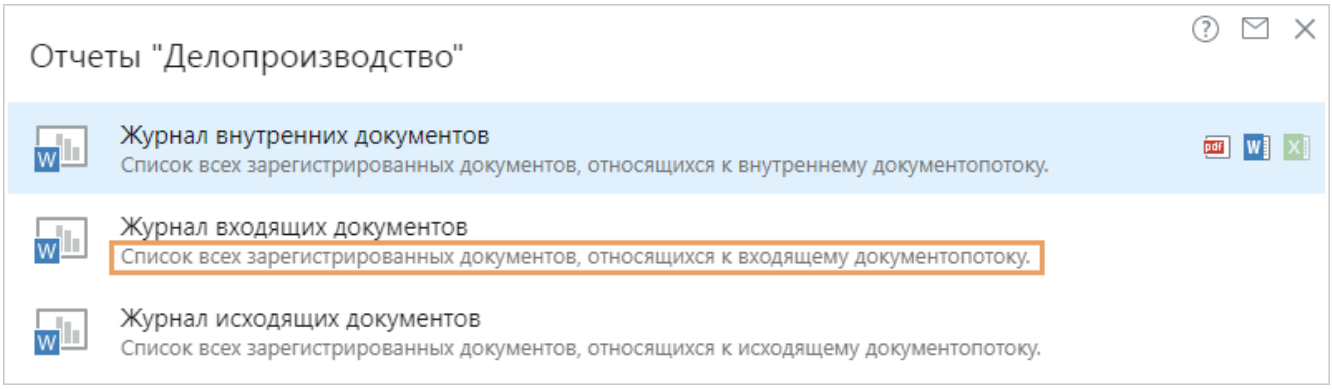


***Имя**. Указывается при создании отчета. Имя должно быть уникальным, состоять из букв латинского алфавита и цифр.

***Отображаемое имя**. Имя отчета, которое будет отображаться в проводнике системы.



Описание. Текст, который будет отображаться в проводнике системы в качестве описания отчета.



Для *локализации* отображаемого имени и описания перейдите по ссылке **Локализовать**.

***Форматы экспорта**. Список форматов, в которых будет формироваться отчет. Возможные значения:

- **Pdf**. Отчет создается в формате PDF;
- **Word**. Отчет создается в формате DOCX;
- **Excel**. Отчет создается в формате XLSX.

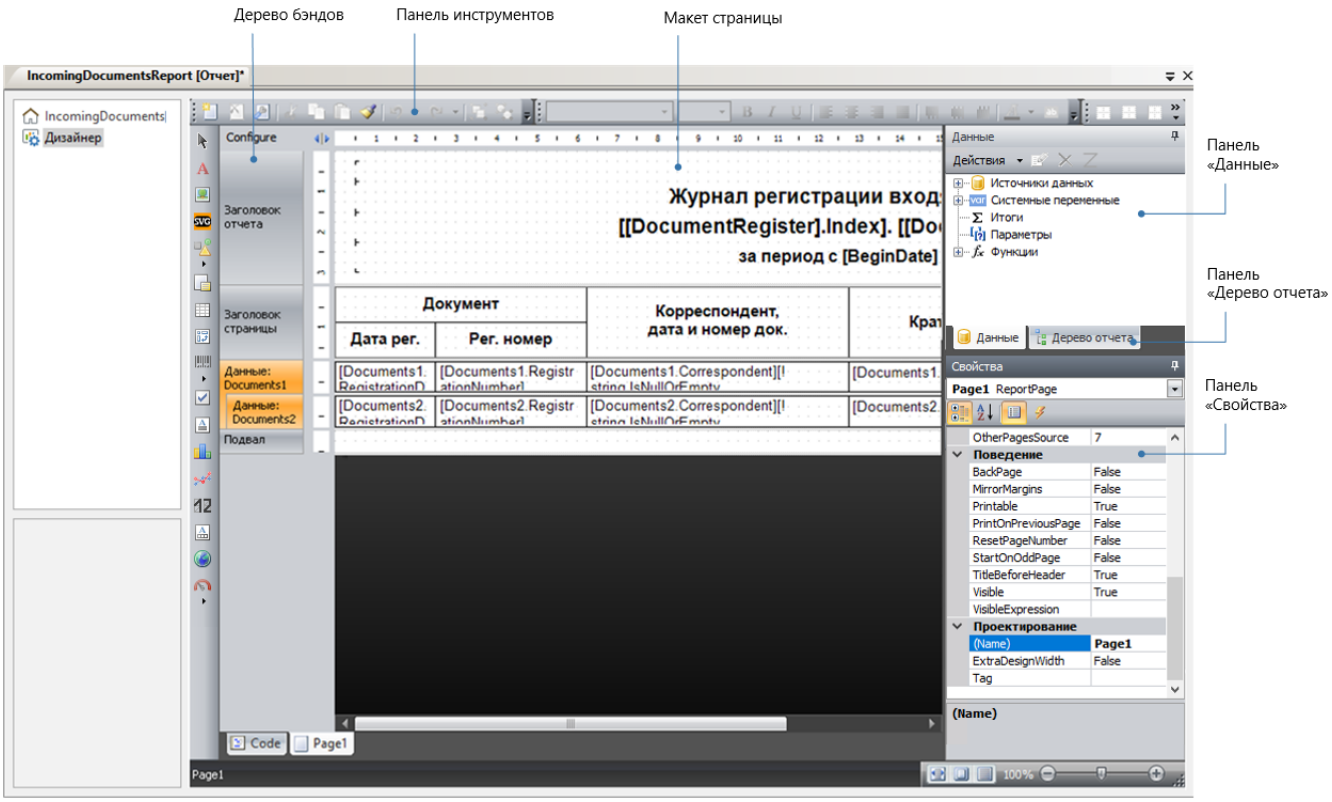
***Формат экспорта по умолчанию**. Формат, в котором будет сформирован отчет, если пользователь не выбрал другой.

События. Добавление обработчиков событий отчета.

Структуры и константы. Добавление структур и констант, которые в последующем можно использовать в функциях и обработчиках событий.

Дизайнер отчета

Используйте узел «Дизайнер», чтобы создать макет отчета, управлять источниками данных и параметрами отчета. Макет отчета разрабатывается с помощью FastReport.Net.



Панель инструментов. Содержит действия для форматирования элементов макета отчета. Например, действия для настройки стиля текста, границ и заливки ячеек таблицы.

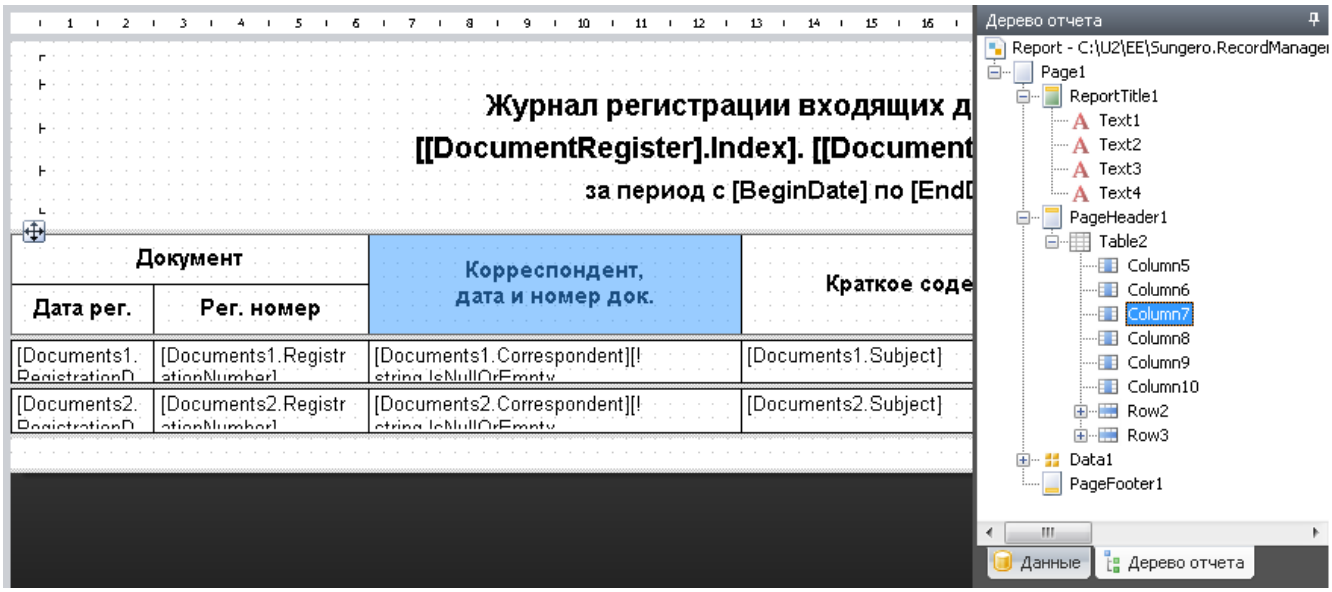
Дерево бэндов. Предназначено для настройки и отображения структуры бэндов макета. Бэнд – область страницы отчета, является контейнером для объектов отчета. Например, на бэнде «Данные» располагают объекты «Таблица» и «Текст» для отображения данных отчета.

Макет страницы. Позволяет настроить внешний вид отчета. Макет страницы состоит из бэндов и объектов. В зависимости от типа бэнда объекты, которые на него добавлены, будут отображаться в определенном месте отчета.

В узле «Дизайнер» также расположены панели для работы с элементами отчета:

- **Данные** для управления источниками данных, параметрами, системными переменными и другими данными отчета. Из панели можно быстро добавить данные на бэнды. Для этого переместите из панели «Данные» нужное поле, параметр или переменную на бэнд в макете страницы. При этом автоматически будет создан объект «Текст»;

- **Дерево отчета** для навигации по элементам макета отчета: страницам, бэндам и объектам. Если элемент выделен на панели «Дерево отчета», то он также будет выделен на макете страницы:



- **Свойства** для изменения свойств отчета и его элементов. Чтобы отобразить на панели свойства конкретного элемента, выделите его в макете страницы, на панели «Данные», «Дерево отчета» или выберите в выпадающем списке на панели «Свойства».

Подробнее о дизайнере отчетов см. в документации FastReport.Net.

ВАЖНО. Некоторые возможности FastReport.Net отключены в среде разработки Directum RX.

Добавление параметров

Параметры отчета – переменные, на основании которых формируется отчет. Значения параметров запрашиваются у пользователя с помощью диалоговых окон или вычисляются на основе других параметров и данных отчета.

Используются для:

- фильтрации данных по условию, заданному в параметре;
- отображения значения параметра в отчете.

Например, в отчете **Журнал регистрации исходящих документов** в качестве параметров используется интервал дат. В отчет попадают документы, зарегистрированные в указанный период.

К параметрам отчета можно обращаться в обработчиках событий, при формировании источников данных и при настройке макета.

Существуют предопределенные параметры отчета, создаваемые автоматически:

- **ExportFormat**. Формат, в котором будет сформирован отчет;
- **Entity**. Сущность, для которой строится отчет. Параметр создается только для отчетов типа сущности. Параметр обязателен к заполнению, по умолчанию не заполняется.

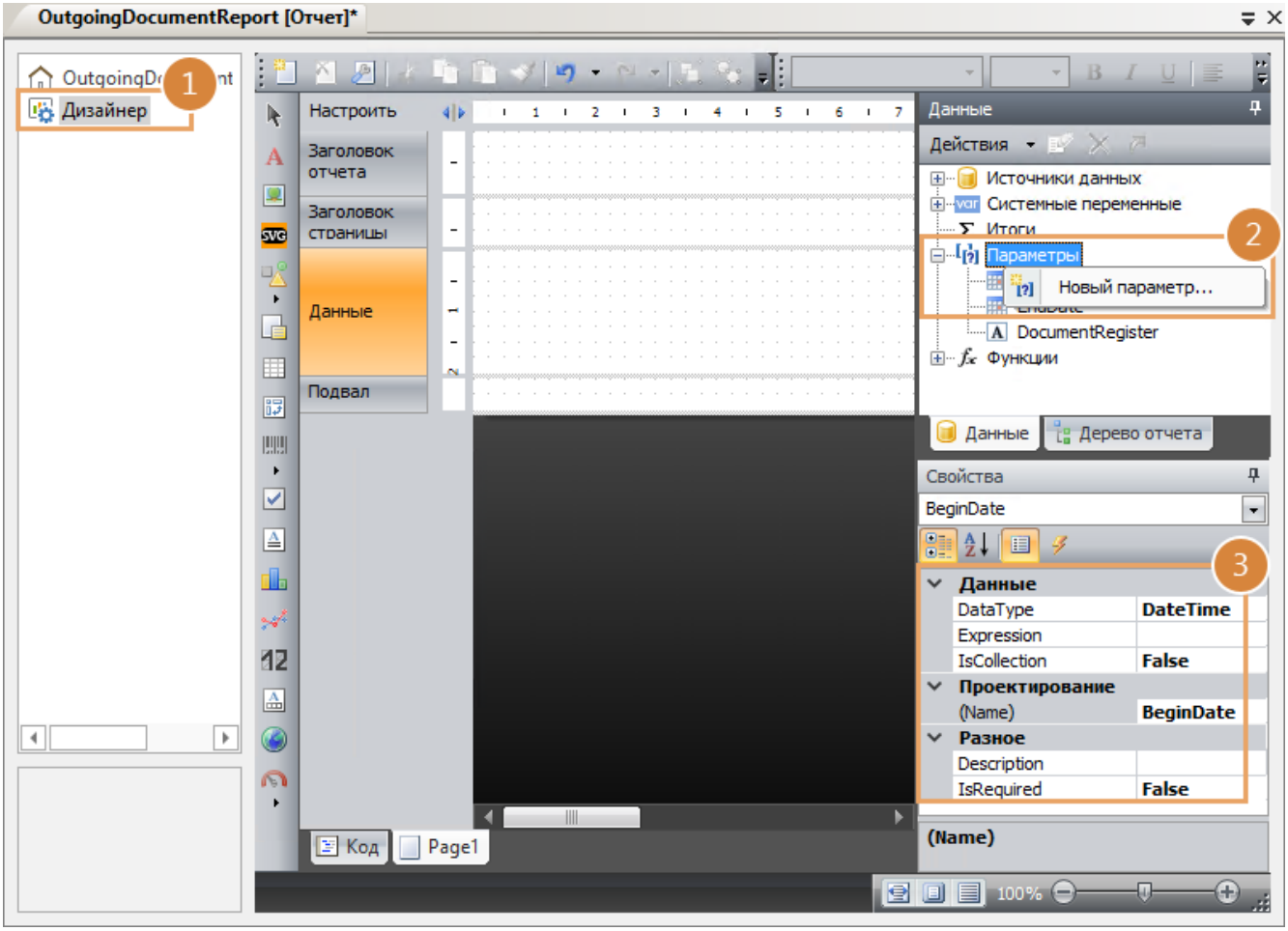
Также для каждого отчета можно создавать дополнительные параметры.

Предположим, необходимо разработать отчет, в котором будут содержаться исходящие документы, зарегистрированные в определенном журнале регистрации в некоторый период времени. Перед формированием отчета пользователь в диалоговом окне указывает значения параметров:

- даты начала и завершения периода формирования отчета;
- журнал регистрации, по документам которого нужно построить отчет.

Для этого:

1. В редакторе отчета перейдите в узел «Дизайнер».



2. На панели «Данные» в контекстном меню узла «Параметры» выберите пункт **Новый параметр...**.
3. На панели «Данные» выделите добавленный параметр и на панели «Свойства» заполните свойства параметра:

   DataType. Тип данных параметра. Значение по умолчанию **String**. Для параметров, соответствующих датам начала и завершения периода формирования отчета, укажите значение DataTime. Для параметра, соответствующего журналу регистрации, укажите значение **Sungero.Docflow.IDocumentRegister** (тип справочника «Журнал регистрации»).

   Expression. Выражение, которое возвращает значение параметра. Значение выражения вычисляется при обращении к параметру. Например, используется для указания значения по умолчанию. Так как значения параметров будут запрашиваться с помощью диалогового окна, оставьте свойство незаполненным для всех параметров.

IsCollection. Признак того, что свойство является коллекцией значений. Возможные значения:

- **True**. Значением параметра будет коллекция элементов типа, который указан в поле **DataType**. В этом случае для добавления элементов в коллекцию используйте метод **AddRange()**;
- **False**. Значением параметра будет один элемент. Значение по умолчанию.

Оставьте значение **False** для всех параметров.

Description. Текстовое описание параметра.

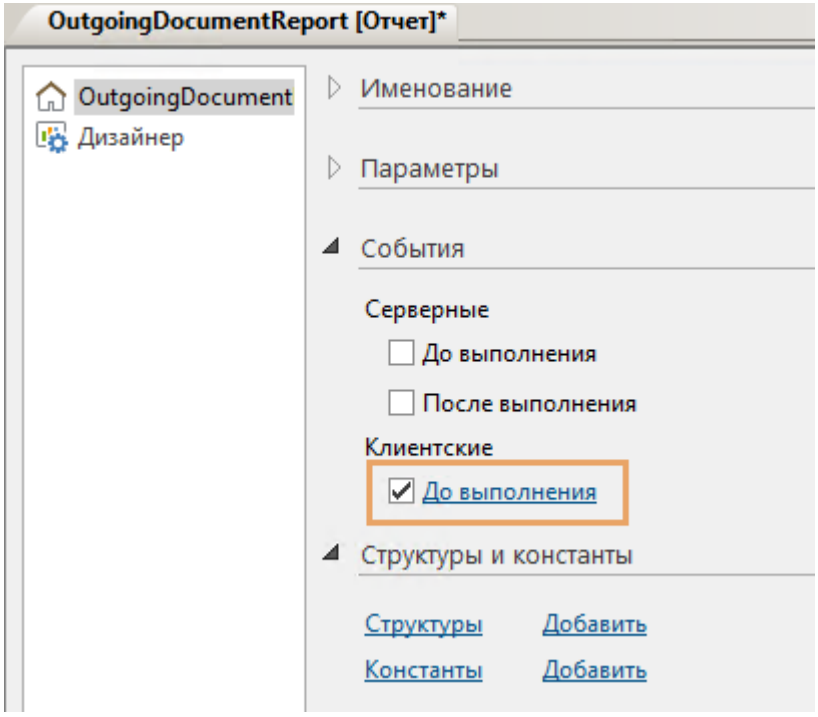
IsRequired. Признак обязательности заполнения параметра. Возможные значения:

- **True**. Параметр обязательный для заполнения. Перед выполнением отчета будет проверяться заполнение параметра;
- **False**. Параметр не обязательный для заполнения. Значение по умолчанию.

Установите значение **True** для всех параметров.

Name. Имя параметра. Должно быть уникальным, состоять из букв латинского алфавита и цифр. Для параметров, соответствующих датам начала и завершения периода формирования отчета, укажите имена **BeginDate** и **EndDate**. Для параметра, соответствующего журналу регистрации, укажите имя **DocumentRegister**.

4. Настройте создание диалогового окна для запроса значений параметров. Для этого перейдите в узел «Отчет», установите флажок напротив клиентского события «До выполнения» и перейдите по ссылке:

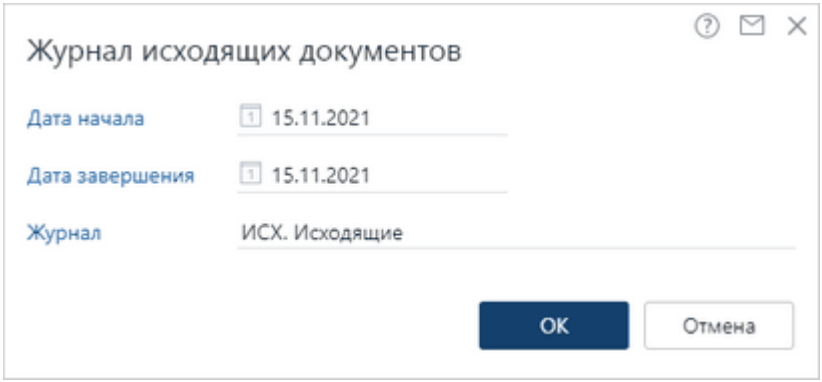


5. В открывшемся редакторе введите код:

```
public override void
    BeforeExecute(Sungero.Reporting.Client.BeforeExecuteEventArgs e)
{
  if (!OutgoingDocumentsReport.BeginDate.HasValue &&
!OutgoingDocumentsReport.EndDate.HasValue)
  {
    // Создание диалогового окна для запроса значений параметров
    // BeginDate, EndDate и DocumentRegister.
```

```
    var dialog = Dialogs.CreateInputDialog("Журнал исходящих документов");
    var beginDate = dialog.AddDate("Дата начала", true,
OutgoingDocumentsReport.BeginDate);
    var endDate = dialog.AddDate("Дата завершения", true,
OutgoingDocumentsReport.EndDate);
    var documentRegister = dialog.AddSelect("Журнал", true,
OutgoingDocumentsReport.DocumentRegister);

    if (dialog.Show() == DialogButtons.Ok)
    {
      // Передача введенных значений в параметры BeginDate, EndDate
      // и DocumentRegister.
      OutgoingDocumentsReport.BeginDate = beginDate.Value.Value;
      OutgoingDocumentsReport.EndDate = endDate.Value.Value;
      OutgoingDocumentsReport.DocumentRegister = documentRegister.Value;
    }
    else
      e.Cancel = true;
  }
}
```

В результате параметры отчета будут заполнены значениями, которые указал пользователь в диалоговом окне:



Источники данных

Источник данных – объект, из которого можно получить данные для формирования отчета. Источник данных описывает тип и структуру предоставляемых в отчете данных.

Источники данных для отчета бывают двух видов:

- источник данных на основании типа сущности позволяет получить список сущностей выбранного типа. Рекомендуется использовать для простых отчетов, где не требуется сложных связей между данными. Например, для отчета **Журнал регистрации исходящих документов** используется источник данных на основании типа сущности «Исходящее письмо»;
- SQL-источник данных позволяет получить данные из произвольных SQL-выборок, таблиц модулей платформы и прикладных модулей, системных таблиц базы данных. Рекомендуется использовать в сложных отчетах, где необходима выборка данных из большого количества источников. Например, для отчета **Контроль возврата документов** используется SQL-запрос для выборки данных из всех типов документов.

Для отчета задается один или несколько источников данных. Например, для отчета **Настройка регистрации и нумерации документов** используются:

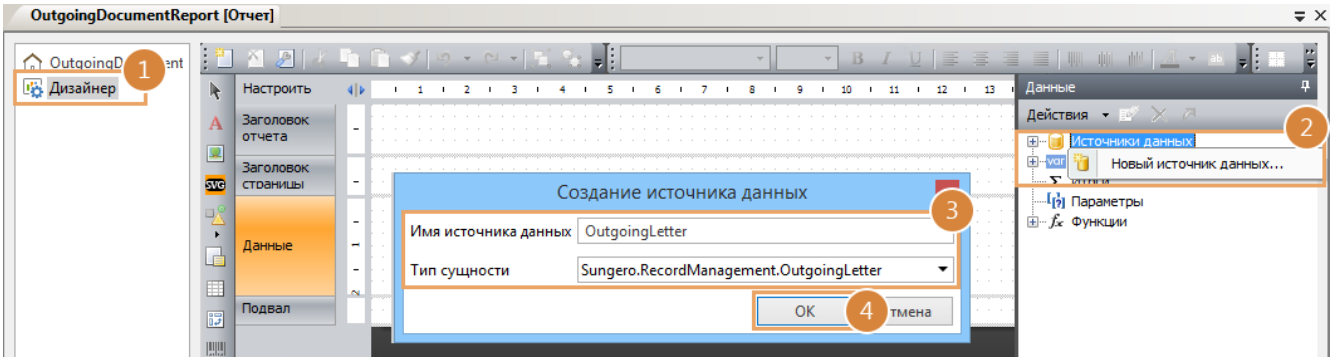
- источник данных на основании типа сущности «Наша организация» (Sungero.Company.BusinessUnit);
- SQL-источник данных, который содержит поля данных с информацией о настройках регистрации и нумерации документов.

Источники данных связаны между собой по идентификатору организации. В результате для формирования отчета используются данные из обоих источников. В отчете отображаются настройки регистрации и нумерации в разрезе организаций.

Добавление источника данных на основании типа сущности

Предположим, необходимо разработать отчет, в котором будет содержаться информация об исходящих документах, зарегистрированных в определенном журнале регистрации в некоторый период времени. Так как для отчета потребуются данные только типа сущности «Исходящее письмо», то добавьте соответствующий источник данных. Для этого:

1. В редакторе отчета перейдите в узел «Дизайнер».



2. На панели «Данные» в контекстном меню узла «Источники данных» выберите пункт **Новый источник данных...**.
3. В открывшемся окне заполните поля:

   ***Имя источника данных**. Должно быть уникальным и состоять из букв латинского алфавита.

   ***Тип сущности**, который будет использоваться в качестве источника данных. В качестве источника данных укажите тип сущности «Исходящее письмо» (Sungero.RecordManagement.OutgoingLetter).

4. Нажмите на кнопку **ОК**. Источник данных и его свойства добавятся в дерево на панель «Данные». Откроется редактор кода источника данных, в котором сгенерируется код обработчика. По умолчанию при выполнении отчета возвращаются все сущности типа «Исходящее письмо», на которые у пользователя есть права.

```
public virtual IQueryable<Sungero.RecordManagement.IOutgoingLetter>
  GetOutgoingLetter()
  {
    return Sungero.RecordManagement.OutgoingLetters.GetAll();
  }
```

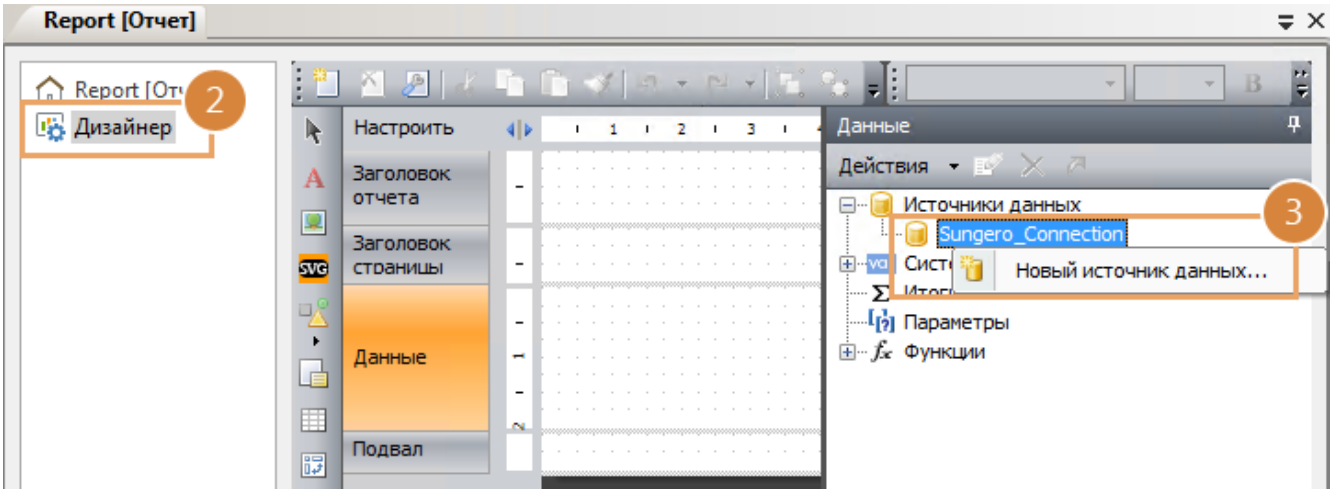
5. Чтобы в отчете отображались только исходящие документы, зарегистрированные в определенном журнале регистрации в некоторый период времени, настройте фильтрацию данных в источнике. В качестве критериев фильтрации используйте запрашиваемые параметры отчета. Для этого дополните код в обработчике:

```
public virtual IQueryable<Sungero.RecordManagement.IOutgoingLetter>
    GetOutgoingLetters()
    {
      // Получить все исходящие письма.
      return Sungero.RecordManagement.OutgoingLetters.GetAll()
        // Оставить только зарегистрированные исходящие письма.
        .Where(l => l.RegistrationState ==
    Sungero.Docflow.OfficialDocument.RegistrationState.Registered)
        // Фильтрация исходящих писем по журналу регистрации
        // (по параметру DocumentRegister).
        .Where(l => Equals(l.DocumentRegister,
    OutgoingDocumentsReport.DocumentRegister))
        // Фильтрация исходящих писем в отчете по периоду дат
        // (по параметрам BeginDate и EndDate).
        .Where(l => l.RegistrationDate >= OutgoingDocumentsReport.BeginDate)
        .Where(l => l.RegistrationDate <= OutgoingDocumentsReport.EndDate)
        // Сортировка исходящих писем в отчете по регистрационному номеру
        // и дате регистрации.
        .OrderBy(l => l.RegistrationNumber)
        .ThenBy(l => l.RegistrationDate);
    }
```

Добавление SQL-источника данных

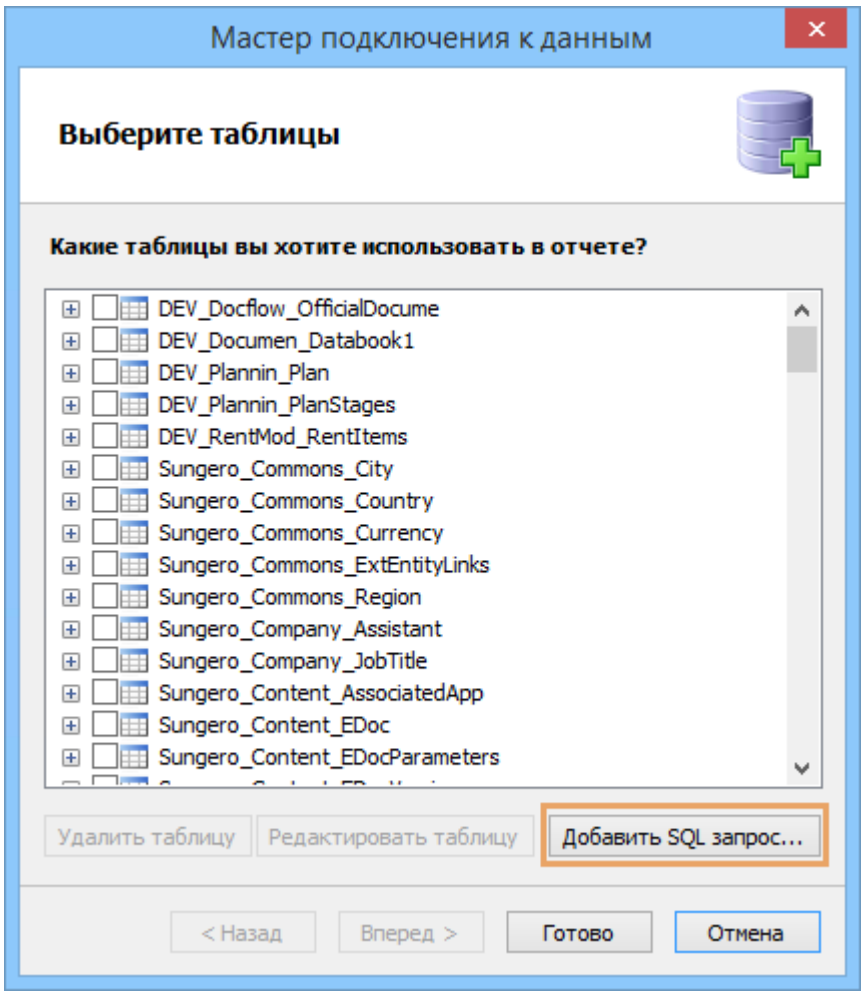
Предположим, необходимо разработать отчет, в котором содержится статистика работы сотрудников с документами. Для отчета необходима информация из нескольких таблиц базы данных, поэтому добавьте SQL-источник. Для этого:

1. Добавьте запрос к SQL-серверу.
2. В редакторе отчета перейдите в узел «Дизайнер».



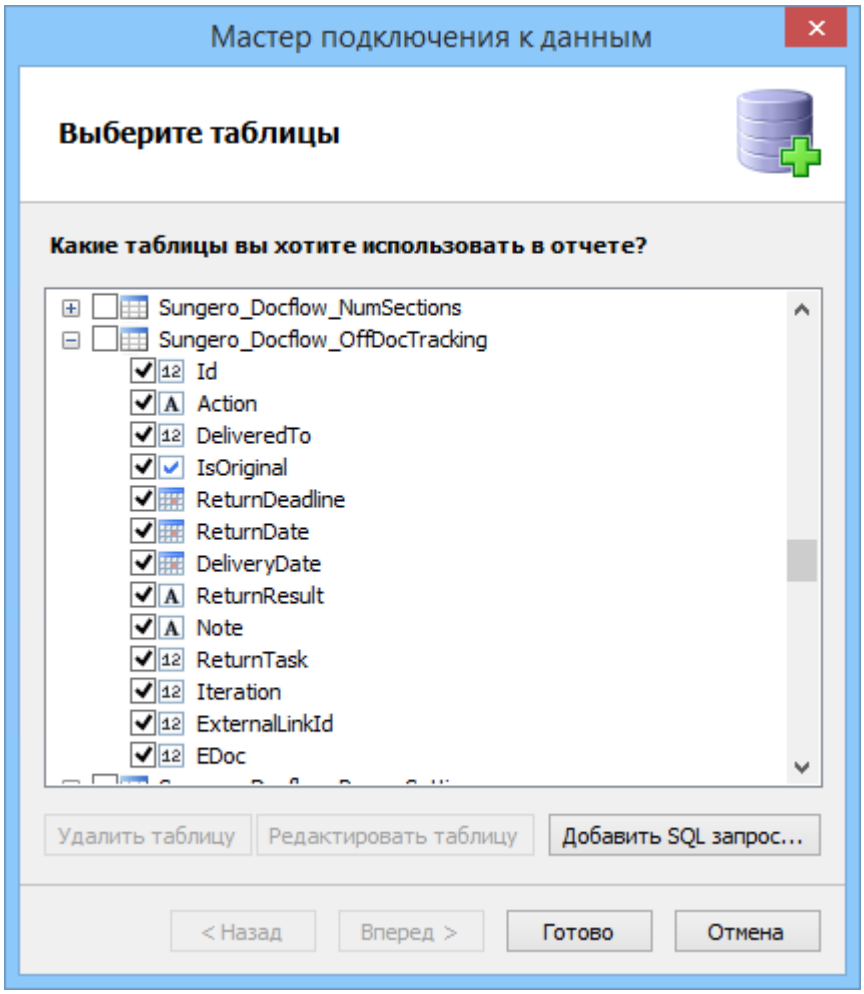
3. На панели «Данные» в контекстном меню узла «Sungero_Connection» выберите пункт **Новый источник данных...**. Откроется окно мастера подключения к данным.

4. На шаге «Выберите таблицы» нажмите на кнопку **Добавить SQL запрос...**.



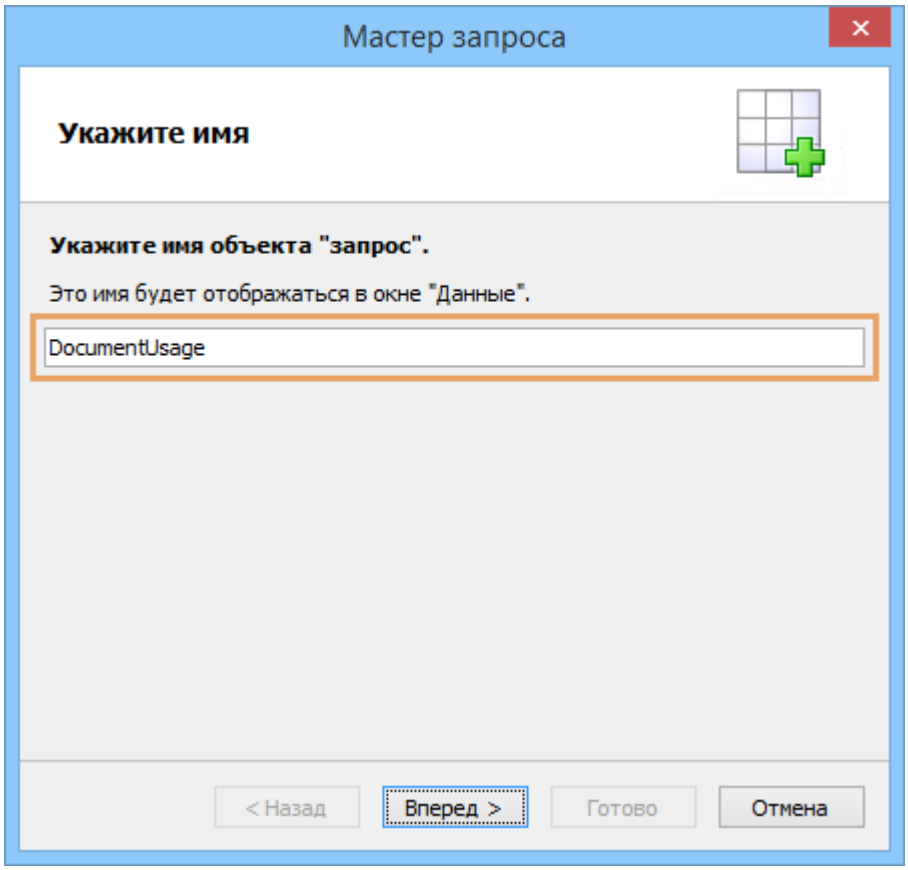
СОВЕТ. Если для выполнения отчета нужны данные таблиц базы данных, но не требуется делать сложную выборку, то SQL-запрос можно не добавлять.

Если для отчета не используется SQL-запрос, установите флажки рядом с нужными таблицами и их полями. Чтобы выбрать поля таблицы, нажмите на кнопку ⊞ рядом с таблицей и в развернувшемся списке отметьте флажками нужные поля.

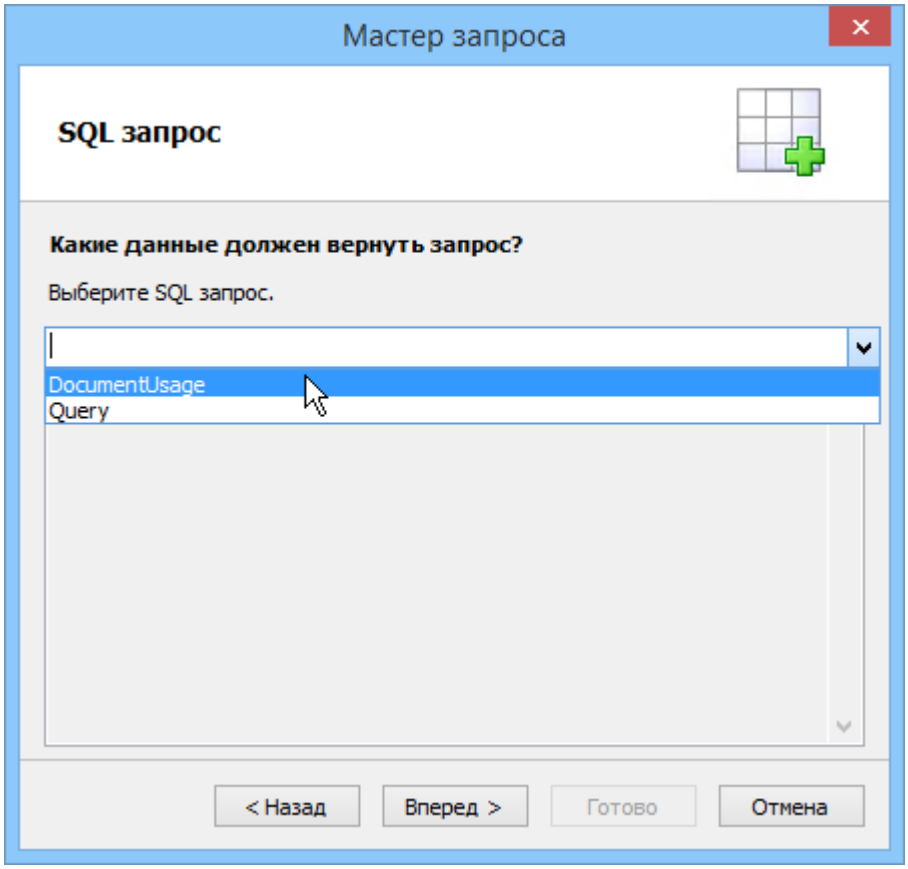


После выбора таблиц нажмите на кнопку **Готово**. Шаги 5-14 делать не нужно.

5. На шаге «Укажите имя» введите имя источника данных. Имя должно быть уникальным, состоять из букв латинского алфавита и цифр.

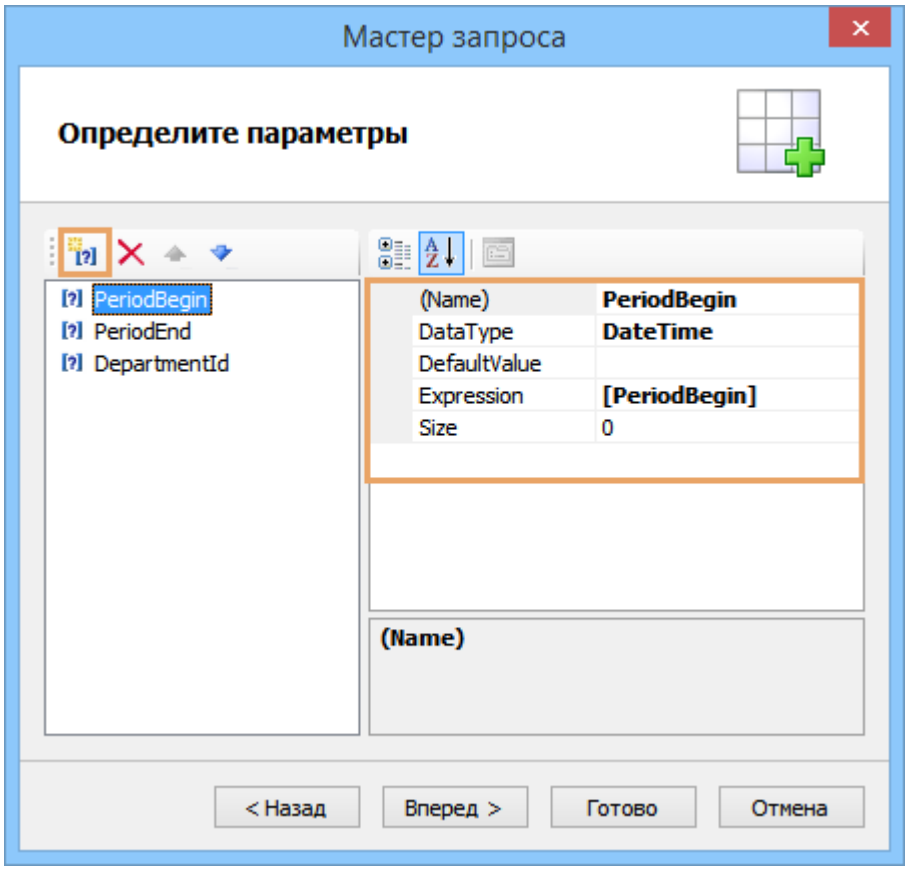


6. Нажмите на кнопку **Вперед >**.
7. На шаге «SQL запрос» в выпадающем списке выберите ранее добавленный запрос.



8. Нажмите на кнопку **Вперед >**.

9. На шаге «Определите параметры» укажите параметры, которые использовались в запросе. По данным параметрам будет формироваться отчет. Чтобы создать параметр запроса, нажмите на кнопку **Добавить параметр**:



Для данного отчета добавляются следующие параметры: PeriodBegin и PeriodEnd – период, за который будет сформирован отчет, DepartmentId – подразделение, по результатам работы сотрудников которого будет сформирован отчет.

10. Заполните свойства параметров запроса:

    Name. Имя параметра, который использовался в тексте SQL-запроса.

    DataType. Тип данных параметра.

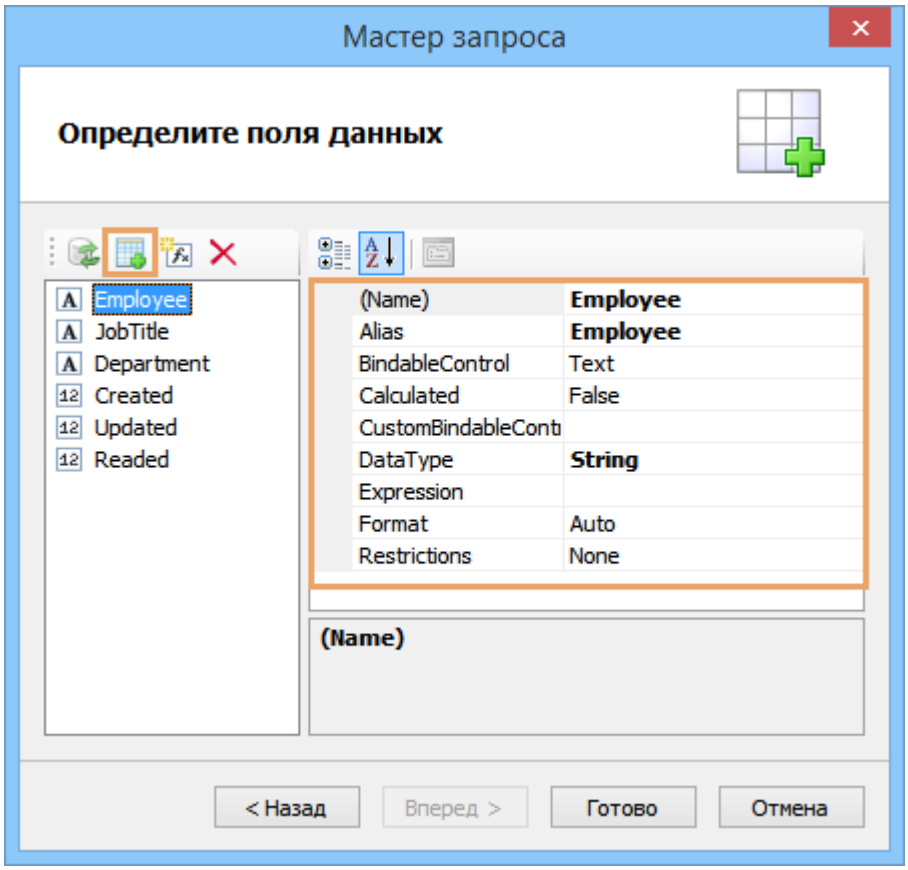
    DefaultValue. Значение по умолчанию, которое будет присвоено параметру, если отсутствует значение в свойстве **Expression**.

    Expression. Выражение, которое возвращает значение параметра. Значение будет вычислено при выполнении отчета.

    Size. Размер данных параметра. Указывается, если параметр имеет тип данных **String**.

11. Нажмите на кнопку **Вперед >**.

12. На шаге «Определите поля данных» укажите поля данных, возвращаемые в запросе. Чтобы создать поле, нажмите на кнопку **Добавить поле**:



Для данного отчета добавляются следующие поля данных: Employee – ФИО сотрудника, JobTitle – должность, Department – подразделение, Created – количество созданных документов, Updated – количество измененных документов, Readed – количество просмотренных документов.

ПРИМЕЧАНИЕ. При необходимости в отчет добавляются поля, значения которых будут вычисляться не в SQL-запросе, а на основе других данных отчета. Например, на основе параметров полей источника данных. В этом случае добавьте вычисляемое поле. Для этого нажмите на кнопку **Добавить вычисляемое поле**.

13. Заполните свойства полей данных:

Name. Имя поля данных.

Alias. Псевдоним поля. Используется для уточнения или изменения имени. Если псевдоним задан, то он используется для обращения к полю вместо оригинального имени.

BindableControl. Объект, связанный с полем. При вынесении поля на макет отчета будет создан указанный объект. Возможные значения:

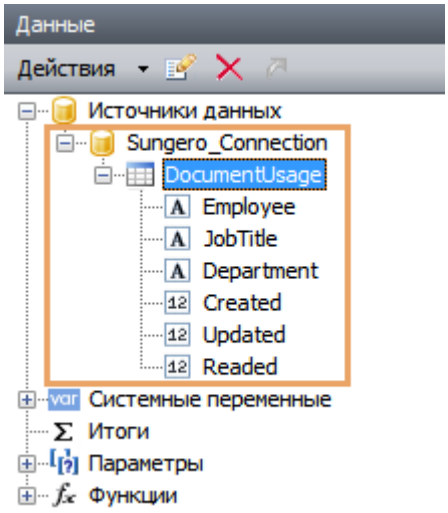
- **Text**. Объект «Текст»;
- **RichText**. Объект «Форматированный текст». Подробнее описание объекта см. в документации FastReport.Net, в статье «Объект «Форматированный текст»;
- **Picture**. Объект «Рисунок»;
- **CheckBox**. Объект «Флажок». Подробнее описание объекта см. в документации FastReport.Net, в статье «Объект «Флажок»;
- **Custom**. Соответствующий объект будет подобран автоматически.

Calculated. Признак того, что поле является вычисляемым.

DataType. Тип данных поля.

Expression. Выражение, которое возвращает значение вычисляемого поля данных. Значение поля будет вычислено при выполнении отчета.

14. Нажмите на кнопку **Готово**. Источник данных и его поля добавятся в дерево на панели «Данные»:

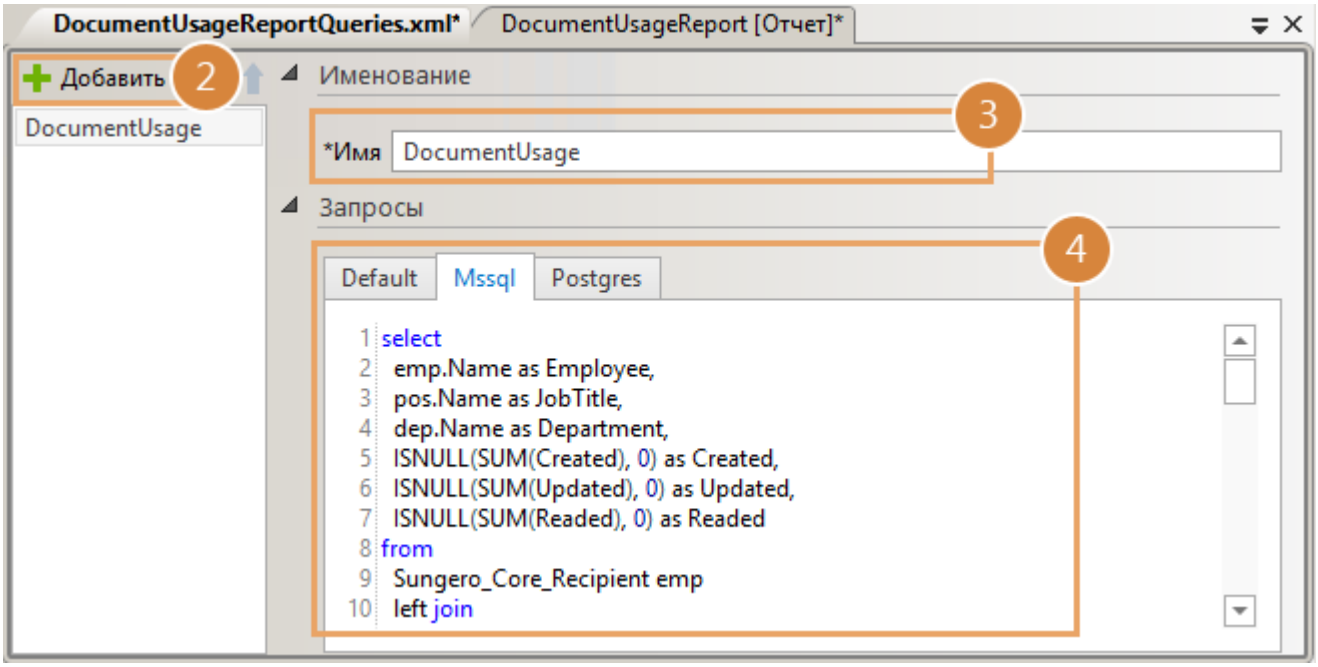


В результате для отчета добавится SQL-источник данных, содержащий информацию из нескольких таблиц базы данных.

Добавление SQL-запроса

Если для формирования отчета нужны данные таблиц базы данных, из которых необходимо делать сложную выборку, то добавьте SQL-запрос. Для этого:

1. В дереве решений выберите отчет и в контекстном меню выберите пункт **Открыть SQL-запросы**.
2. В открывшемся окне нажмите на кнопку **Добавить**.



3. Укажите имя SQL-запроса. Имя должно быть уникальным, состоять из букв латинского алфавита и цифр. Первый символ должен начинаться с буквы.
4. Напишите SQL-запрос на вкладке:
   - «Mssql» – если используется СУБД Microsoft SQL Server;
   - «Postgres» – если используется СУБД PostgreSQL/Postgres Pro/Jatoba;
   - «Default» – если запрос подходит для всех СУБД.

   Вы можете написать запрос сразу на нескольких вкладках. В последующем система сама определит нужный запрос, в зависимости от того, какая СУБД у вас используется.

   Например, для формирования отчета «Статистика работы с документами» добавляются следующие запросы:

Mssql

```
   select
  emp.Name as Employee,
  pos.Name as JobTitle,
  dep.Name as Department,
  ISNULL(SUM(Created), 0) as Created,
  ISNULL(SUM(Updated), 0) as Updated,
  ISNULL(SUM(Readed), 0) as Readed
from
  Sungero_Core_Recipient emp
  left join
  (select
     Employee,
     DocId,
     case when ActionType = 'Create' then 1
else 0
     end as Created,
     case when ActionType = 'Update' then 1
else 0
     end as Updated,
     case when ActionType = 'Read' then 1 else
 0
     end as Readed
   from
     (select
        distinct
        his.[User] as Employee,
        EntityId as DocId,
        case
          when [Action] = 'Create'
            then 'Create'
          when [Action] = 'Update' and
(Operation = 'UpdateVerBody' or Operation =
'CreateVersion')
            then 'Update'
          when [Action] = 'Read' and Operation
= 'ReadVerBody'
            then 'Read'
          end as ActionType
      from
        [dbo].[Sungero_Content_DocHistory] his
      where
        EntityId <> 0
        and his.HistoryDate between
@PeriodBegin and @PeriodEnd) hist
        ) history on
    emp.Id = history.Employee
  left join Sungero_Company_JobTitle pos on
    emp.JobTitle_Company_Sungero = pos.Id
```

Postgres

```
   select
  emp.Name as Employee,
  pos.Name as JobTitle,
  dep.Name as Department,
  COALESCE(SUM(Created), 0) as Created,
  COALESCE(SUM(Updated), 0) as Updated,
  COALESCE(SUM(Readed), 0) as Readed
from
  Sungero_Core_Recipient emp
  left join
  (select
     Employee,
     DocId,
     case when ActionType = 'Create' then 1
else 0
     end as Created,
     case when ActionType = 'Update' then 1
else 0
     end as Updated,
     case when ActionType = 'Read' then 1 else
0
     end as Readed
   from
     (select
        distinct
        his.User as Employee,
        EntityId as DocId,
        case
          when Action = 'Create'
            then 'Create'
          when Action = 'Update' and (Operation
= 'UpdateVerBody' or Operation =
'CreateVersion')
            then 'Update'
          when Action = 'Read' and Operation =
'ReadVerBody'
            then 'Read'
          end as ActionType
      from
        Sungero_Content_DocHistory his
      where
        EntityId <> 0
        and his.HistoryDate between
@PeriodBegin and @PeriodEnd) hist
        ) history on
    emp.Id = history.Employee
  left join Sungero_Company_JobTitle pos on
    emp.JobTitle_Company_Sungero = pos.Id
```

Mssql

```
  join Sungero_Core_Login logins on
    emp.Login = logins.Id
  join Sungero_Core_Recipient dep on
    emp.Department_Company_Sungero = dep.Id
where
  emp.Discriminator = 'b7905516-2be5-4931-
961c-cb38d5677565'
  and emp.Status = 'Active'
  and (@DepartmentId = 0 or
emp.Department_Company_Sungero =
@DepartmentId)
group by
  emp.Name, pos.Name, dep.Name
order by
  ISNULL(SUM(Created), 0) +
ISNULL(SUM(Updated), 0) + ISNULL(SUM(Readed),
0), dep.Name, emp.Name
```

Postgres

```
  join Sungero_Core_Login logins on
    emp.Login = logins.Id
  join Sungero_Core_Recipient dep on
    emp.Department_Company_Sungero = dep.Id
where
  emp.Discriminator = 'b7905516-2be5-4931-961c-
cb38d5677565'
  and emp.Status = 'Active'
  and (@DepartmentId = 0 or
emp.Department_Company_Sungero = @DepartmentId)
group by
  emp.Name, pos.Name, dep.Name
order by
  COALESCE(SUM(Created), 0) +
COALESCE(SUM(Updated), 0) +
COALESCE(SUM(Readed), 0), dep.Name, emp.Name
```

5. Сохраните изменения.

В дальнейшем к добавленному запросу можно обратиться в серверном коде с помощью класса **Queries.<Имя отчета>**.

ПРИМЕЧАНИЕ. В среде разработки можно добавлять SQL-запросы при разработке модулей и типов сущностей. Например, когда нужно обратиться к базе данных для получения документов, которые были отправлены в задаче.

Добавление связи между источниками данных

Чтобы в отчете отображалась информация из нескольких источников данных, между ними добавляется связь. Между связанными источниками устанавливается отношение «главный-подчиненный». Таким образом, одной строке данных главного источника соответствует несколько строк подчиненного источника.

Связанные источники данных используются, например, для создания отчета типа «Главный – подчиненный».

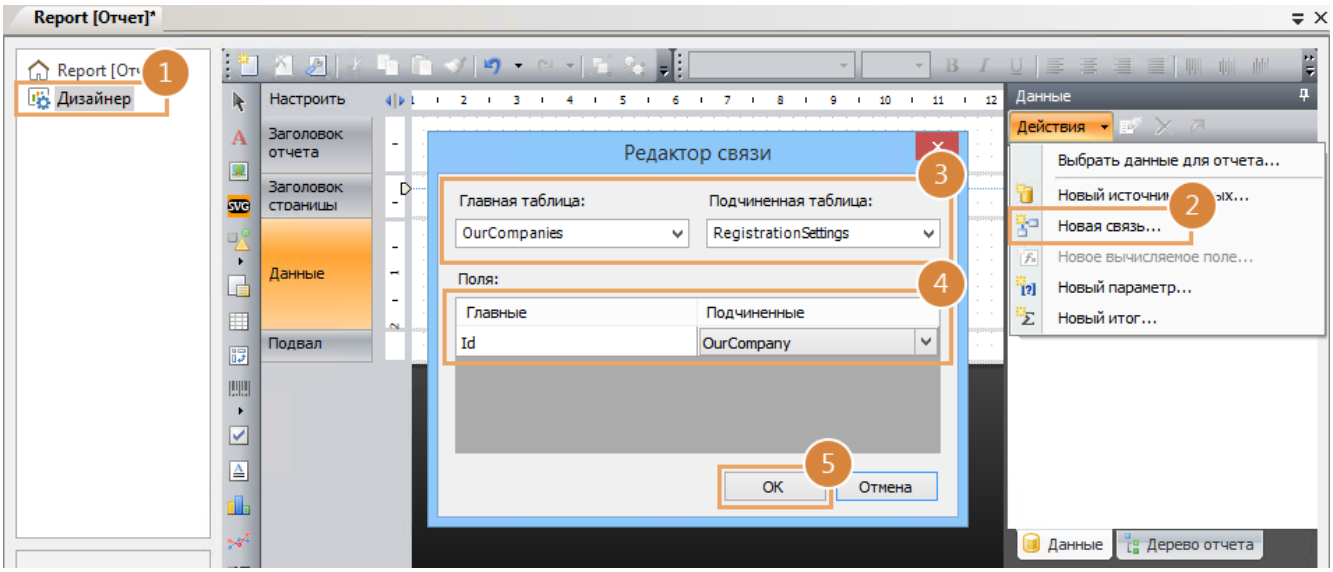
Предположим, разрабатывается отчет, в котором содержится информация о настройках регистрации и нумерации для видов документов. При этом к одной организации может относиться несколько настроек.

Для отчета созданы источники данных:

- OurCcompanies – источник данных на основании типа сущности **Наша организация** (Sungero.Company.BusinessUnit);
- RegistrationSettings – SQL-источник данных, который содержит поля данных с информацией о настройках регистрации и нумерации видов документов.

Необходимо, чтобы в отчете сначала отображалась строка с названием организации, а потом все соответствующие ей настройки. Для этого добавьте связь между источниками данных. Так как к одной организации может относиться несколько настроек, то главным сделайте источник с информацией об организациях (OurCompanies), а подчиненным – с информацией о настройках (RegistrationSettings). Для этого:

1. В редакторе отчета перейдите в узел «Дизайнер».



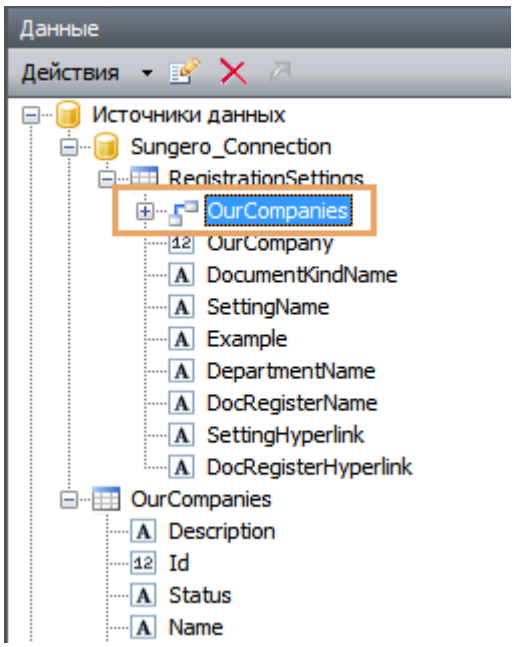
2. На панели «Данные» в выпадающем списке «Действия» выберите пункт **Новая связь...**.
3. В открывшемся редакторе связи заполните поля:

   Главная таблица. Главный источник данных. В выпадающем списке выберите значение OurCompanies.

   Подчиненная таблица. Подчиненный источник данных. В выпадающем списке выберите значение RegistrationSettings.
4. В табличной части «Поля» укажите ключевые поля в главной и подчиненной таблице. Таблицы могут быть связаны с помощью одного или нескольких полей.

   В столбце **Главные** в выпадающем списке выберите поле, которое соответствует идентификатору организации в главном источнике. В столбце **Подчиненные** в выпадающем списке выберите поле, которое соответствует идентификатору организации в подчиненном источнике.
5. Нажмите на кнопку **ОК**.

В результате источники данных будут связаны. Созданная связь отображается на панели «Данные» в списке полей подчиненного источника данных RegistrationSettings в виде значка . В качестве названия связи используется имя главного источника OurCompanies:



При нажатии на значок ⊞ рядом со связью раскроется список полей главного источника данных.

ВАЖНО. Для отображения в отчете значений полей главного источника данных используется формат [подчиненный_источник.главный_источник.поле]. Например, запись [RegistrationSettings. OurCompanies.Name] возвращает значение поля **Name** главного источника OurCompanies.

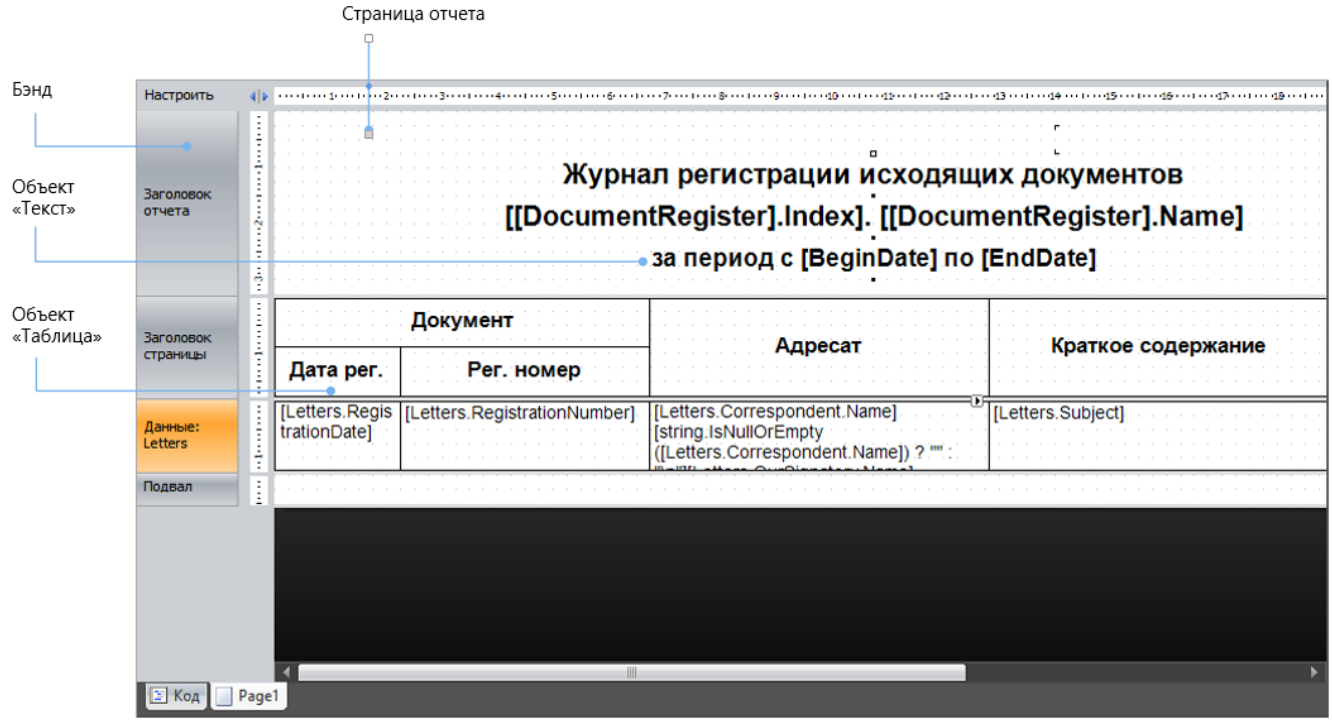
После добавления связи настройте макет отчета типа «Главный – подчиненный».

Настройка макета отчета

Для настройки внешнего вида страницы отчета в редакторе разрабатывается макет, который содержит:

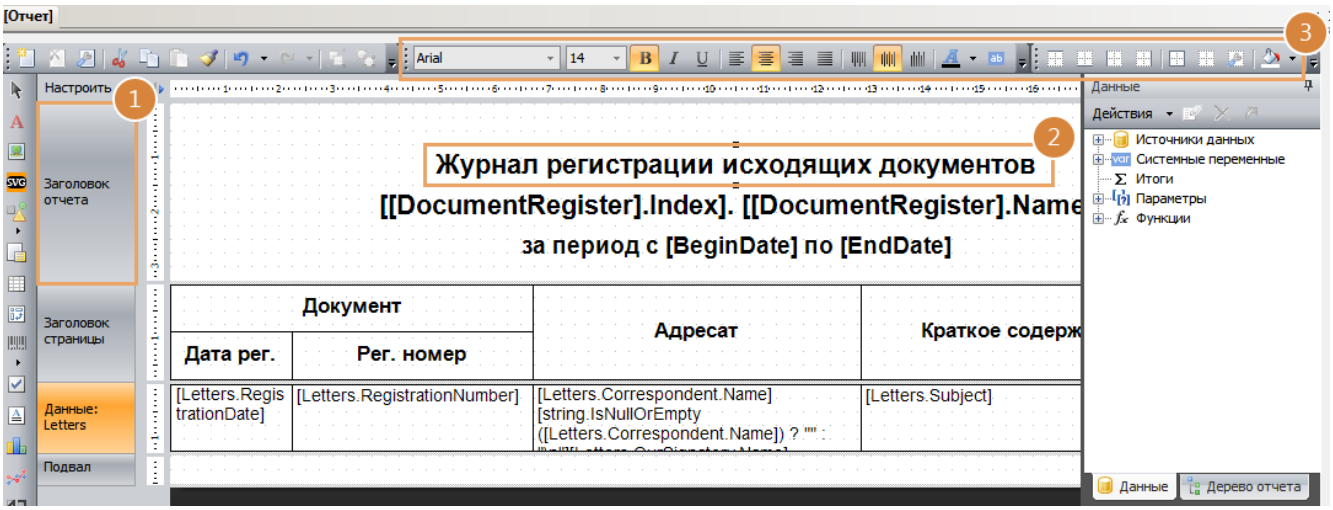
- одну или несколько разных по оформлению страниц. Например, в отчете может быть титульная страница и страницы с данными. Каждая страница макета может генерировать одну или несколько страниц отчета, в зависимости от содержащихся на ней данных. Подробнее о создании многостраничного отчета см. в документации FastReport.Net, в статье «Многостраничный отчет типа «Буклет»;
- *бэнды* для размещения объектов отчета. Располагаются на страницах отчета. В редакторе отчета в узле «Дизайнер» бэнды отображаются в виде прямоугольной области. При создании отчета в макет автоматически добавляются бэнды «Заголовок отчета», «Заголовок страницы», «Данные» и «Подвал»;

- объекты для отображения данных и оформления отчета. Располагаются на бэндах. Например, на бэнде «Заголовок отчета» могут находиться объекты для отображения текста, а на бэнде «Заголовок страницы» объект для отображения таблицы.



Чтобы настроить макет отчета:

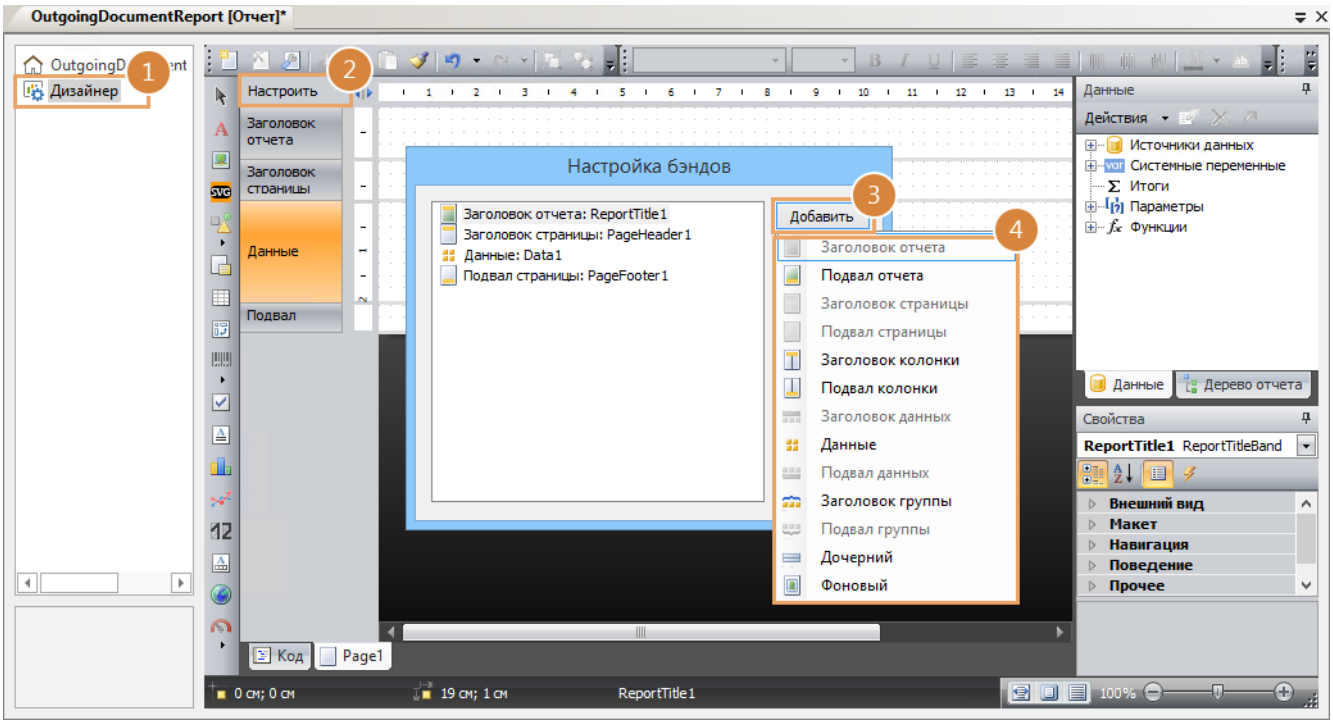
1. Задайте структуру отчета. Для этого задайте свойства существующих бэндов и при необходимости добавьте новые бэнды.



2. Добавьте на бэнды объекты для отображения информации и оформления отчета. Например, добавьте объекты «Текст» на бэнд «Заголовок отчета» для отображения названия отчета.
3. Задайте форматирование объектов для настройки внешнего вида отчета. Например, укажите формат текстовых данных, настройте оформление таблиц.

Добавление бэндов

1. В редакторе отчета перейдите в узел «Дизайнер».



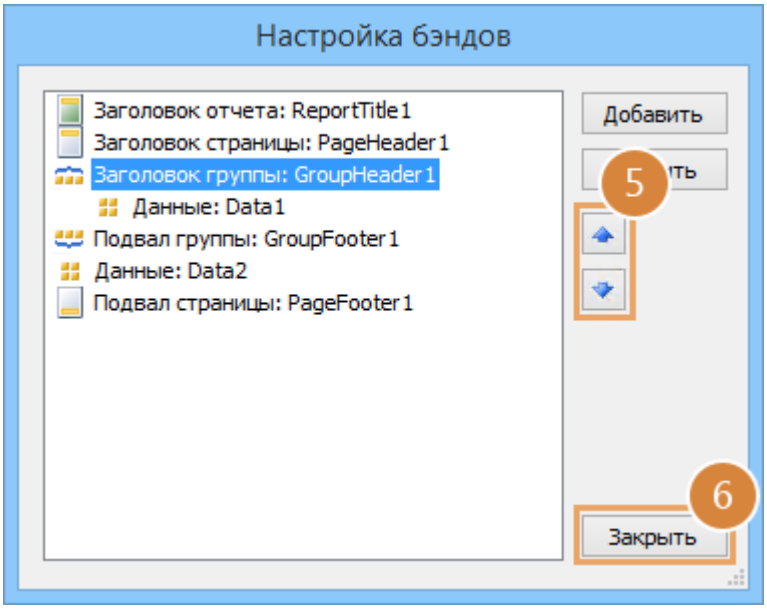
2. Нажмите на кнопку **Настроить** в дереве бэндов.
3. В окне настройки бэндов нажмите на кнопку **Добавить**.
4. В выпадающем списке выберите пункт, соответствующий нужному типу бэнда. В зависимости от типа бэнд отображается в определенном месте отчета:

Бэнд	Способ отображения	Пример содержимого
Заголовок отчета	Отображается один раз в начале отчета	Название отчета, дата формирования
Подвал	Отображается один раз после последней строки данных, но перед бэндом «Подвал страницы»	Место для подписания отчета вручную
Заголовок страницы	Отображается вверху на каждой странице отчета	Заголовок таблицы с данными отчета
Подвал страницы	Отображается внизу на каждой странице отчета, аналог нижнего колонтитула	Номер страницы
Заголовок колонки	Используется для отчетов с несколькими колонками. Отображается вверху каждой колонки, после заголовка страницы	Заголовок колонки таблицы с данными
Подвал колонки	Отображается внизу каждой колонки, перед подвалом страницы	Итоговые значения, вычисленные по данным в колонке
Данные	Подключается к источнику данных и отображается столько раз, сколько строк в источнике	Значения полей источников данных, оформленные в виде таблицы
Заголовок данных	Подключается к бэнду «Данные» и отображается перед первой строкой данных	Подзаголовок таблицы с данными

Бэнд	Способ отображения	Пример содержимого
Подвал данных	Подключается к бэнду «Данные» и отображается после последней строки данных	Итоговые значения, вычисленные по всем данным таблицы
Заголовок группы	Отображается в начале каждой группы, когда значение условия группировки меняется	Признак группировки данных
Подвал группы	Отображается в конце каждой группы	Итоговые значения, вычисленные по каждой группе данных
Дочерний	Подключается к любому бэнду и отображается сразу после него	Дополнительная информация о данных отчета
Фоновый	Отображается в виде фона на каждой странице отчета	Фоновое изображение

5. Чтобы изменить порядок следования бэндов, используйте кнопки и .

ПРИМЕЧАНИЕ. Перемещать можно только бэнды «Данные» и «Заголовок группы», если в макет добавлено несколько бэндов «Данные».

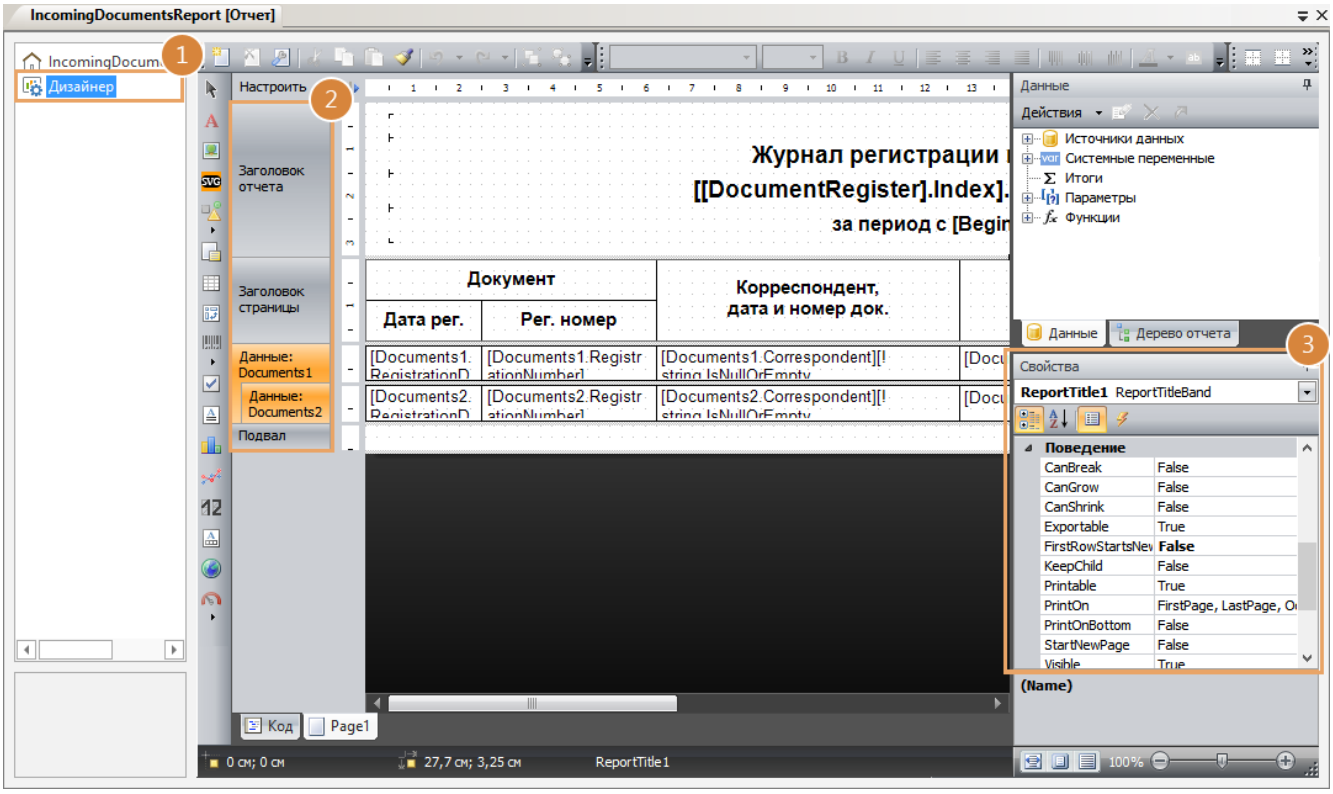


6. Нажмите на кнопку **Закрыть**.

После настройки макета задайте свойства бэндов и добавьте объекты для отображения данных отчета.

Изменение свойств бэндов

1. В редакторе отчета перейдите в узел «Дизайнер».



2. Щелкните левой кнопкой мыши по названию нужного бэнда в дереве бэндов.
3. Задайте свойства бэнда в окне «Свойства».

   ПРИМЕЧАНИЕ. В таблице перечислены только наиболее часто используемые свойства. Описание других свойств см. в документации FastReport.Net.

Свойство	Описание
Внешний вид	
Border	Настройка рамки бэнда. Рекомендуется изменять настройки рамки с помощью кнопок на панели инструментов
Columns	Количество колонок в бэнде. Доступно для бэнда «Данные»
Fill	Настройка заливки бэнда. Рекомендуется изменять настройки рамки с помощью кнопок на панели инструментов
Макет	
Height	Высота бэнда. Указывается в сантиметрах
Поведение	
CanBreak	Способ отображения содержимого бэнда при переходе на другую страницу. Возможные значения: **True**. При переходе на другую страницу часть объектов переносится на новую страницу. **False**. При переходе на другую страницу все содержимое бэнда переносится на новую страницу

Свойство	Описание
CanGrow	Увеличение размера бэнда. Возможные значения: **True**. Высота бэнда автоматически увеличивается под размер объектов на нем. **False**. Размер бэнда фиксирован
CanShrink	Уменьшение размера бэнда. Возможные значения: **True**. Высота бэнда автоматически уменьшается под размер объектов на нем. **False**. Размер бэнда фиксирован
KeepChild	Способ отображения дочернего бэнда при переходе на новую страницу. Возможные значения: **True**. Дочерний бэнд не отрывается от родительского и не переносится на другую страницу. **False**. Дочерний бэнд отрывается от родительского и переносится на другую страницу
Printable	Способ отображения бэнда в сформированном отчете. Возможные значения: **True**. Бэнд отображается. **False**. Бэнд не отображается
PrintOn	Страницы, на которых отображается бэнд. Возможные значения: **FirstPage**. Первая страница. **LastPage**. Последняя страница. **OddPages**. Нечетные страницы. **EvenPages**. Четные страницы. Свойство может принимать несколько значений одновременно. По умолчанию равно сочетанию **FirstPage**, **LastPage**, **OddPages**, **EvenPages**
PrintOnBottom	Отображение бэнда внизу страницы. Возможные значения: **True**. Бэнд отображается внизу страницы, перед бэндом «Подвал страницы». Используется, например, для итоговых значений. **False**. Бэнд отображается в том месте, где он расположен на макете отчета
RepeatOnEveryPage	Повтор бэнда на каждой странице. Доступно для бэндов «Заголовок группы», «Заголовок данных», «Подвал группы» и «Подвал данных». Возможные значения: **True**. При переносе содержимого бэнда «Данные» на новую страницу, бэнд повторяется на каждой странице. **False**. Бэнд добавлен в отчет один раз
StartNewPage	Отображение бэнда на новой странице. Возможные значения: **True**. Бэнд отображается на новой странице. **False**. Бэнд отображается на странице, на которой закончился предыдущий бэнд
Visible	Отображение бэнда в отчете. Возможные значения: **True**. Бэнд отображается. **False**. Бэнд не отображается

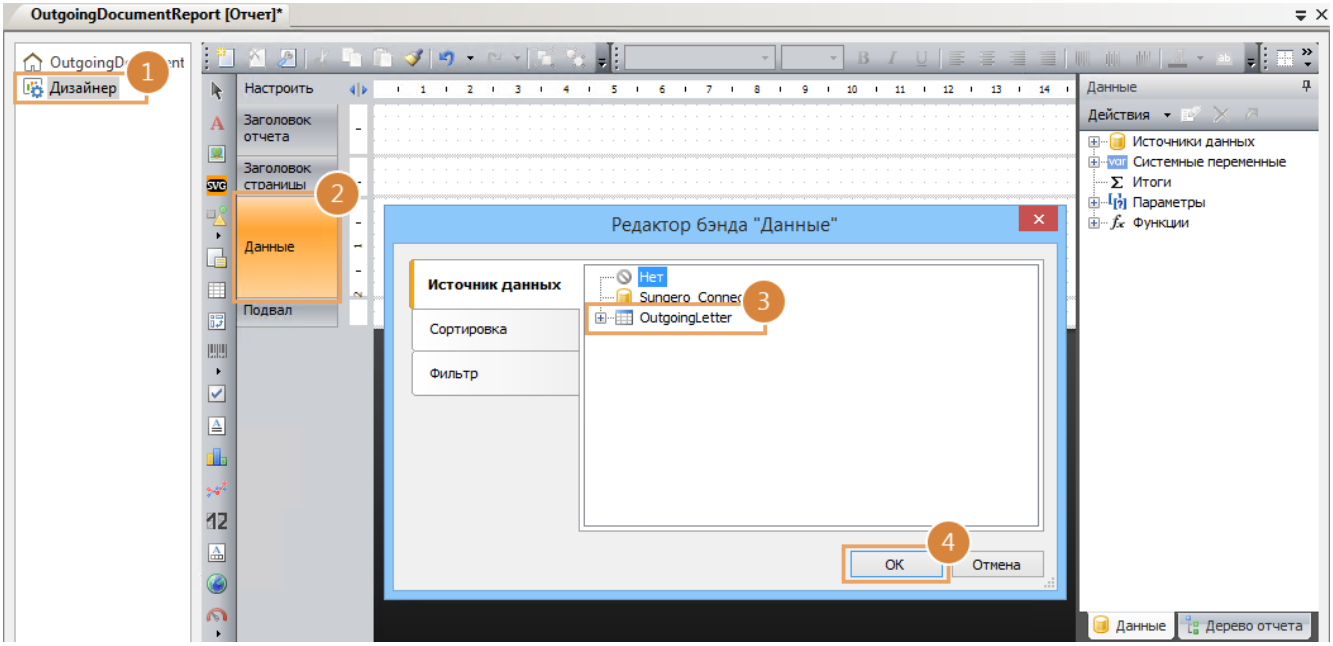
Подключение бэнда к источнику данных

Чтобы в отчете данные отображались в виде списка, *бэнд* «Данные» необходимо подключить к источнику данных. В этом случае в отчете отображаются все строки с данными, которые содержатся в источнике. Например, если для бэнда указан источник данных на основании типа сущности «Исходящие документы», то в отчете отобразятся все исходящие документы в порядке их создания в системе.

ВАЖНО. Если бэнд «Данные» не подключен к источнику, отчет будет содержать только одну строку с данными.

Чтобы подключить бэнд к источнику данных:

1. В редакторе отчета перейдите в узел «Дизайнер».



2. Дважды щелкните левой кнопкой мыши на бэнде «Данные».
3. В открывшемся окне редактора бэнда на закладке «Источник данных» выберите источник, данные которого необходимо отобразить в отчете.
4. Нажмите кнопку **OK**.

После того, как бэнд подключен к данным, разместите на нем объекты, которые будут показывать информацию из полей данных.

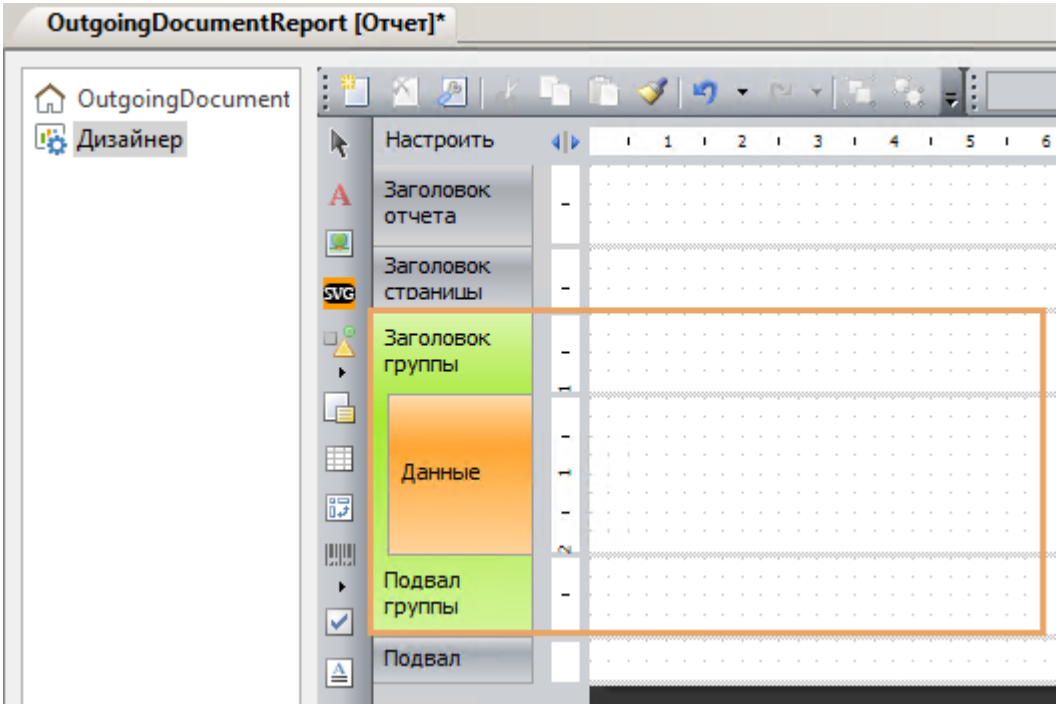
Группировка данных

Для работы с отчетами, в которых содержится большое количество записей, данные в отчете можно группировать. Например, группировать документы по подразделениям, к которым они относятся.

Группа представляет собой набор из трех *бэндов*:

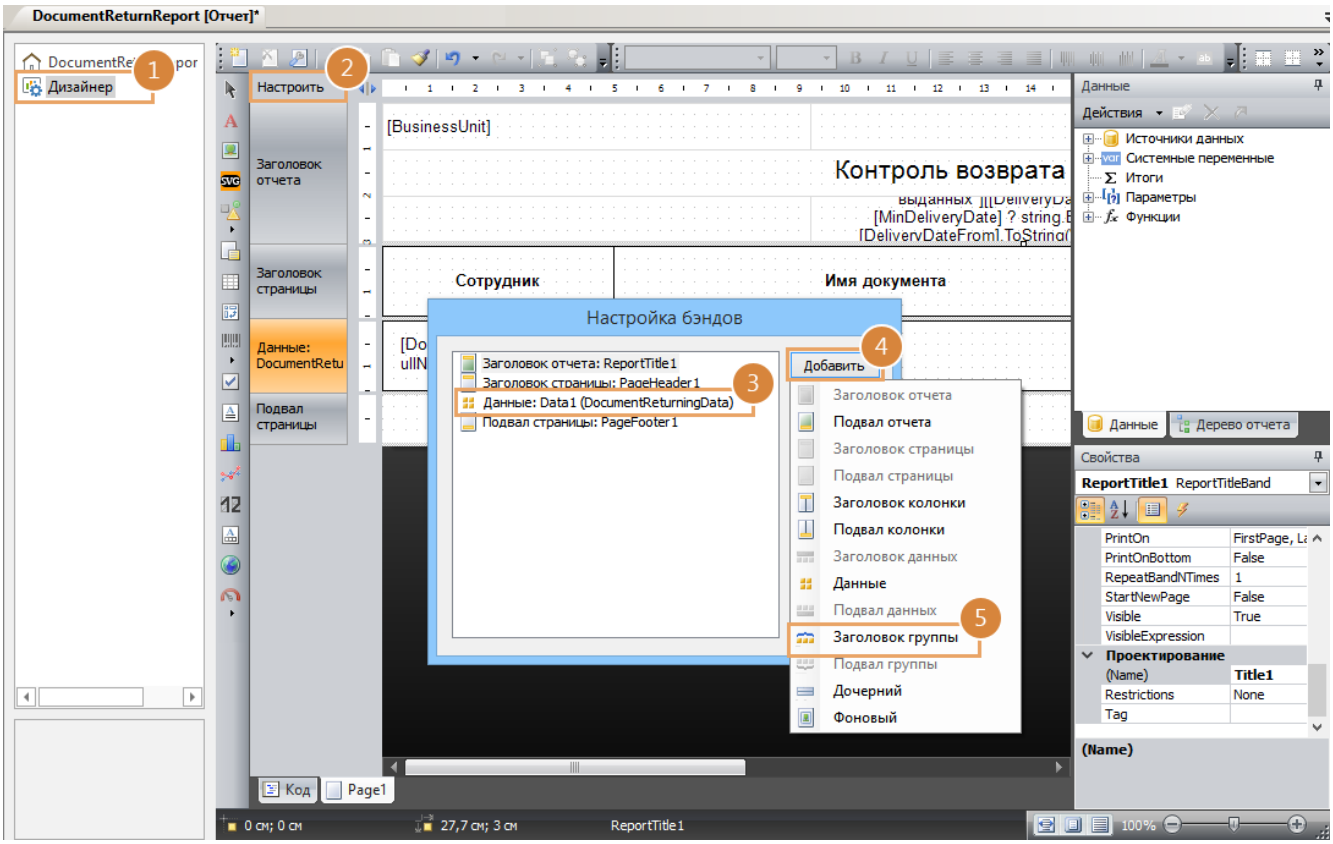
- «Заголовок группы», содержит условие группировки;
- «Данные», содержит данные отчета, удовлетворяющие условию группировки;

- «Подвал группы», содержит, например, итоговые значения, вычисленные по группе данных. Бэнд не является обязательным, при необходимости его можно удалить.



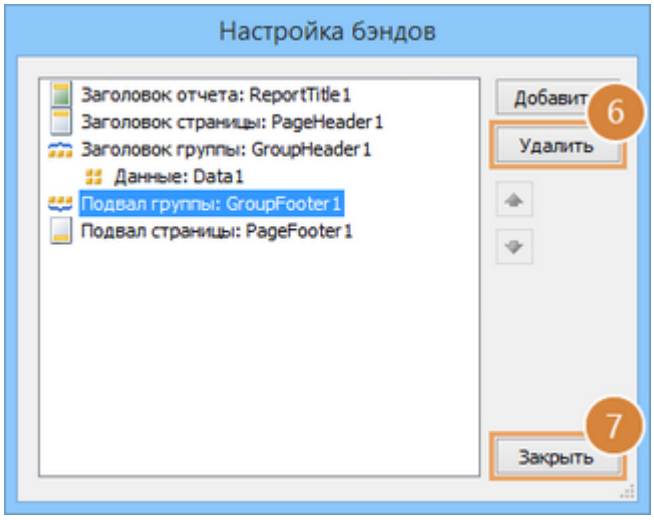
Предположим, разработан отчет с информацией о документах, которые были выданы сотрудникам, но еще не возвращены. Документы в отчете должны быть сгруппированы по подразделениям. Для этого добавьте в макет отчета бэнд «Заголовок группы» и настройте группировку. Для этого:

1. В редакторе отчета перейдите в узел «Дизайнер».

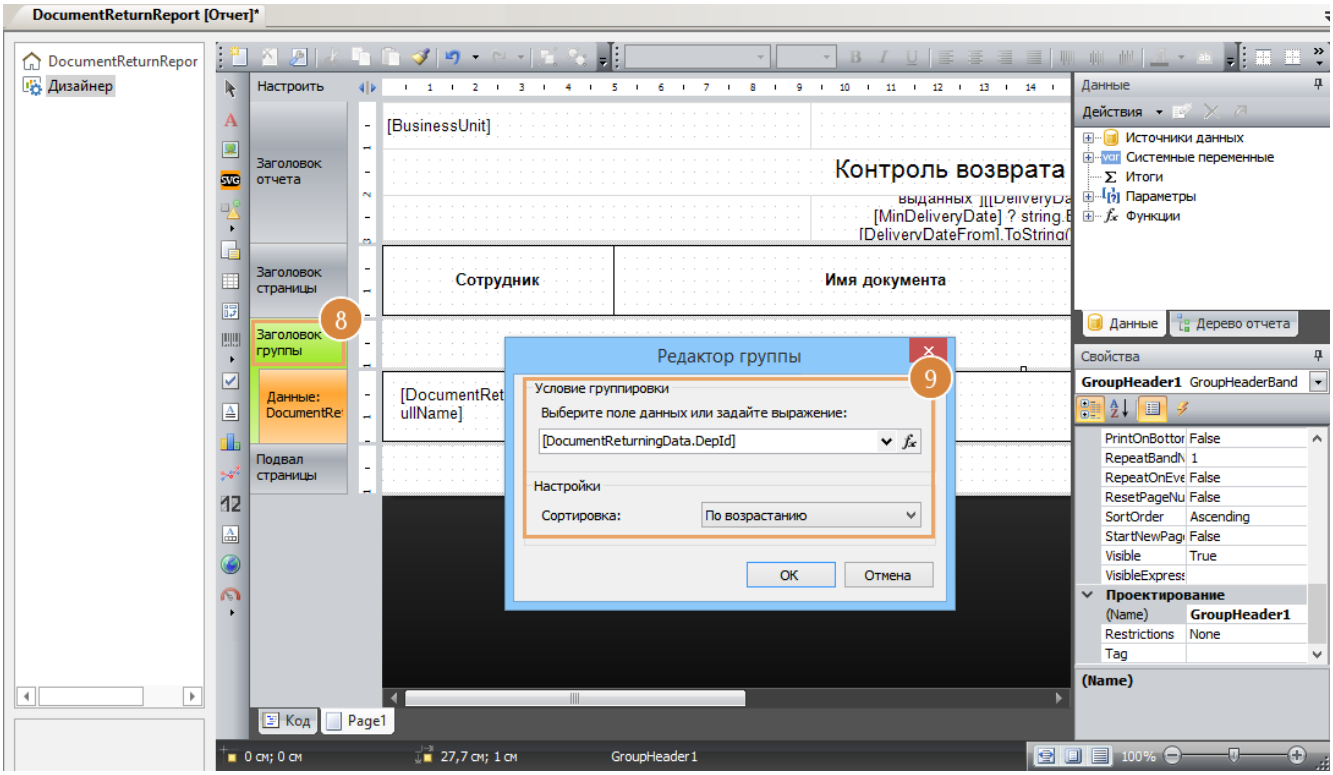


2. Нажмите на кнопку **Настроить** в дереве бэндов.

3. В окне настройки выделите бэнд «Данные», для которого нужно настроить группировку записей.
4. Нажмите на кнопку **Добавить**.
5. В выпадающем списке выберите пункт **Заголовок группы**. В результате перед бэндом «Данные» добавится бэнд «Заголовок группы», а после – бэнд «Подвал группы».
6. Если бэнд «Подвал группы» не должен отображаться в отчете, то удалите его. Для этого окне настройки выделите бэнд и нажмите на кнопку **Удалить**.



7. Нажмите на кнопку **Закрыть**.
8. Чтобы настроить группировку по подразделениям, в дереве бэндов щелкните два раза левой кнопкой мыши на бэнде «Заголовок группы».

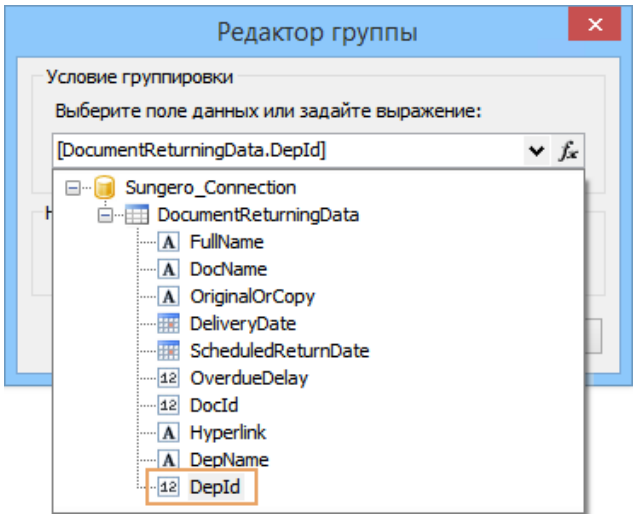


9. Заполните поля в редакторе группы:

   Выберите поле данных или задайте выражение. Условие, по которому группируются записи в отчете. В качестве условия используется поле источника данных или выражение. Чтобы настроить группировку по полю источника данных, в выпадающем списке последовательно выберите:

   - **имя источника > имя поля**, если используется источник данных на основании типа сущности;
   - **Sungero_Connection > имя источника > имя поля**, если используется SQL-источник данных.

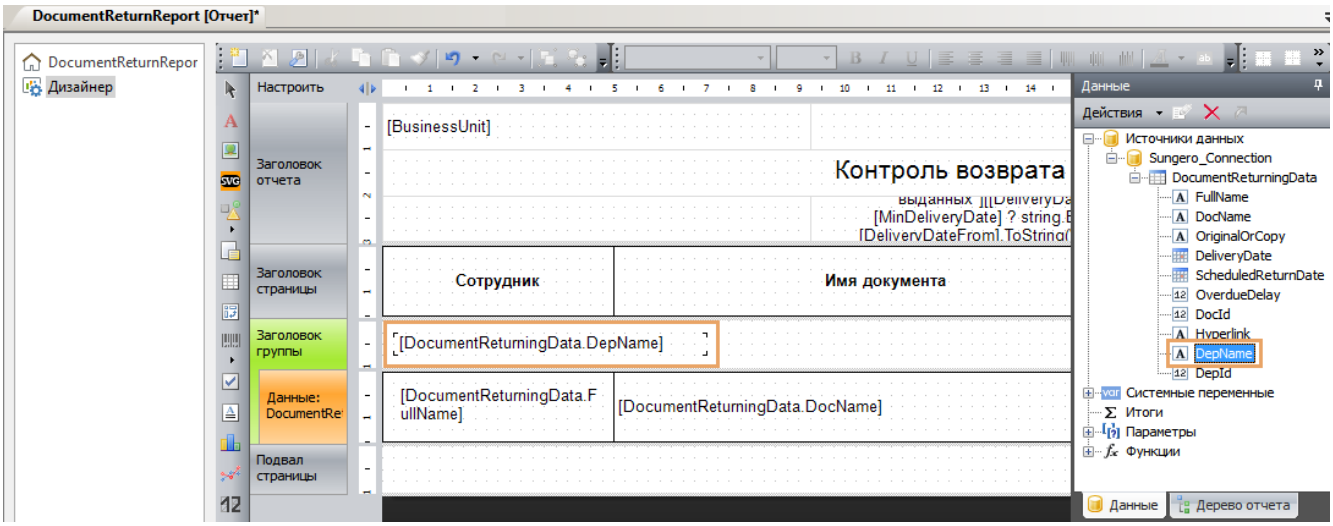
   Для группировки документов по подразделениям укажите поле источника данных, соответствующее идентификатору подразделения. Например, в выпадающем списке последовательно выберите Sungero_Connection > DocumentReturningData > DepId.



   Сортировка. Способ сортировки значений внутри группы. Возможные значения: **Не сортировать**, **По возрастанию**, **По убыванию**.

10. Нажмите на кнопку **ОК**.

11. Чтобы в отчете отображалась строка с названием подразделения, по которому сгруппированы записи, из панели «Данные» переместите поле источника данных DepName на бэнд «Заголовок данных». При этом будет создан объект «Текст», который подключен к выбранному полю.



12. Настройте форматирование объекта «Текст» на бэнде «Заголовок группы».

В результате данные в отчете будут сгруппированы по подразделениям:

10.08.2015

Контроль возврата документов

выданных с 08.09.2014 по 31.08.2015

Сотрудник	Имя документа	ИД	Оригинал/ копия	Дата выдачи	Срок возврата	Количество дней просрочки
Отдел снабжения						
Кондратьев Р. Г.	Договор №2014/1 от 15.10.2014 с Импульс, ООО "Поставка компьютерной техники"	560	Оригинал	15.10.2014	-	-
Кондратьев Р. Г.	Договор №2014/3-с от 25.11.2014 с ЗАО "Асконт" "Приобретение компьютерной техники"	619	Оригинал	25.11.2014	-	-
Кондратьев Р. Г.	Договор №2014/2-с от 25.11.2014 с М.А.Д. "Инжиниринг" "Аренда монтажного стенда"	617	Оригинал	25.11.2014	-	-
Кондратьев Р. Г.	Договор №2014/5-с от 26.11.2014 с Мебельсон, ООО "Офисные кресла"	622	Оригинал	26.11.2014	-	-
Финансово-юридический отдел						
Законов С. Ю.	Договор с Liebert HIROSS S.p.A "Проверка замещениц предметном отображении"	1311	Оригинал	13.07.2015	15.07.2015	19
Законов С. Ю.	Договор с Liebert HIROSS S.p.A "Проверка замещениц предметном отображении"	1311	Оригинал	13.07.2015	15.07.2015	19

1

Сортировка и фильтрация данных

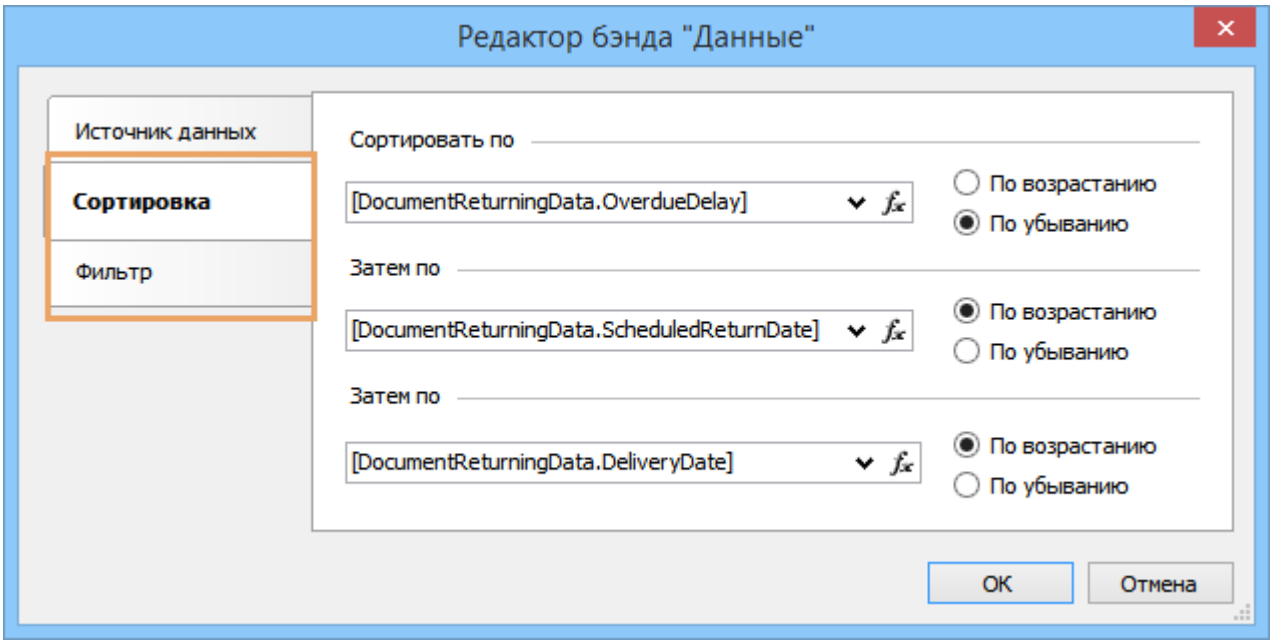
По умолчанию данные в отчете отображаются в том порядке, в котором они расположены в источнике. Для удобной работы с отчетом данные сортируются и фильтруются. Например, если для отчета используется источник данных на основании типа сущности «Исходящие документы», то по умолчанию в отчете отображаются все исходящие документы в порядке их создания в системе. При необходимости в отчете можно оставить только документы определенного журнала регистрации и отсортировать их по регистрационному номеру.

В качестве критериев сортировки и фильтрации используется любое выражение, например, поле источника данных.

Сортировка и фильтрация настраиваются следующими способами:

- критерии сортировки и фильтрации указываются в SQL-запросе на получение данных при добавлении SQL-источника данных. Запрос выполняется на сервере данных и вернет только те записи, которые нужны в отчете;
- для источников данных на основании типа сущности фильтрация и сортировка настраиваются в редакторе кода источника данных с помощью методов Where() и OrderBy();

- критерии сортировки и фильтрации указываются в редакторе бэнда «Данные» на закладках «Сортировка» и «Фильтр»:



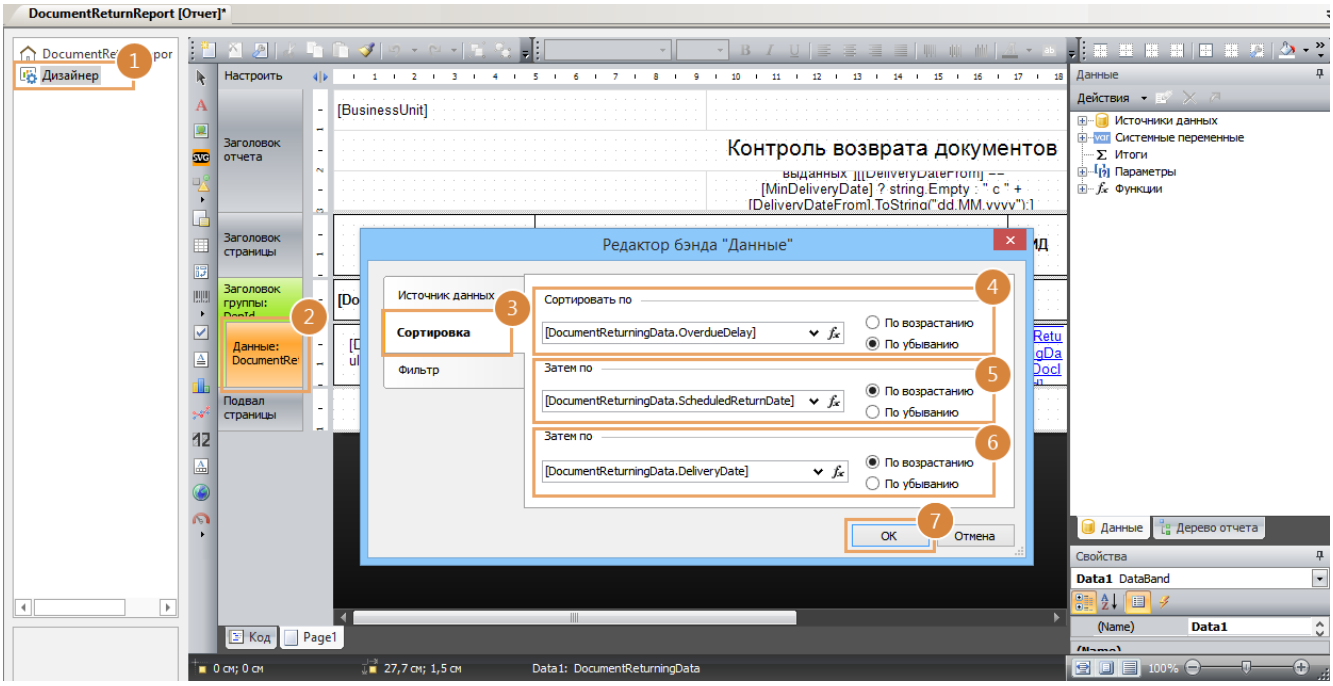
ПРИМЕЧАНИЕ. Фильтрацию данных рекомендуется выполнять при добавлении источника данных.

Предположим, необходимо разработать отчет с информацией о документах, которые выданы сотрудникам, но еще не возвращены или не подлежат возврату. Документы в отчете должны быть отсортированы следующим образом:

- по убыванию количества дней просрочки возврата;
- по возрастанию плановой даты возврата документа;
- по возрастанию даты выдачи документа сотруднику.

Для этого:

1. В редакторе отчета перейдите в узел «Дизайнер».



2. Дважды щелкните левой кнопкой мыши на бэнде «Данные».

3. В открывшемся окне редактора бэнда перейдите на закладку «Сортировка».
4. В группе полей «Сортировать по» в выпадающем списке выберите поле источника данных, соответствующее количеству дней просрочки возврата (OverdueDelay) и установите переключатель **По убыванию**.
5. В группе полей «Затем по» в выпадающем списке выберите поле источника данных, соответствующее плановой дате возврата (ScheduledReturnDate) и установите переключатель **По возрастанию**.
6. Во второй группе полей «Затем по» в выпадающем списке выберите поле источника данных, соответствующее дате выдачи документа (DeliveryDate) и установите переключатель **По возрастанию**.
7. Нажмите на кнопку **ОК**.

В результате данные в отчете будут отсортированы в соответствии с указанными условиями:

Контроль возврата документов

выданных с 01.08.2015

Сотрудник	Имя документа	ИД	Оригинал/ копия	Дата выдачи	Срок возврата	Количество дней просрочки
Отдел продаж						
Ардо Н. А.	Протокол №1-07.15 от 06.07.2015 "Совещания о планах на 3-й квартал"	1166	Оригинал	18.08.2015	24.08.2015	2
Ардо Н. А.	Договор №ТС/15/10 от 22.07.2015 с ГУ Российская академия медицинских наук Российский научный центр хирургии имени академика Б.В. Петровского "Аренда станков"	1367	Оригинал	21.08.2015	01.09.2015	-
Финансово-юридический отдел						
Законов С. Ю.	Договор №ТС/15/12 от 07.08.2015 с ЗАО "Асконт" "Аренда помещения"	1360	Оригинал	18.08.2015	21.08.2015	3
Законов С. Ю.	Приказ №КАДР-2 от 11.07.2015 "Проект приказа"	1268	Оригинал	20.08.2015	25.08.2015	1
Законов С. Ю.	Доп. соглашение №12/2 от 22.06.2015 к договор №ТС/14/016/пр с ООО "Сатурн" "Новый объем работ"	1046	Оригинал	11.08.2015	-	-
Законов С. Ю.	Приказ №ОРД-П-3 от 30.01.2015 "О тиражировании системы электронного документооборота и управления взаимодействием DIRECTUM в УАВР №2 - филиале ООО «Газпром трансгаз Чайковский»"	844	Оригинал	25.08.2015	08.09.2015	-

Создание отчета типа «Главный – подчиненный»

Отчет типа «Главный – подчиненный» позволяет отображать информацию из нескольких связанных источников данных.

Предположим, разрабатывается отчет, в котором содержится информация о настройках регистрации и нумерации видов документов для разных организаций. При этом к одной организации может относиться несколько настроек.

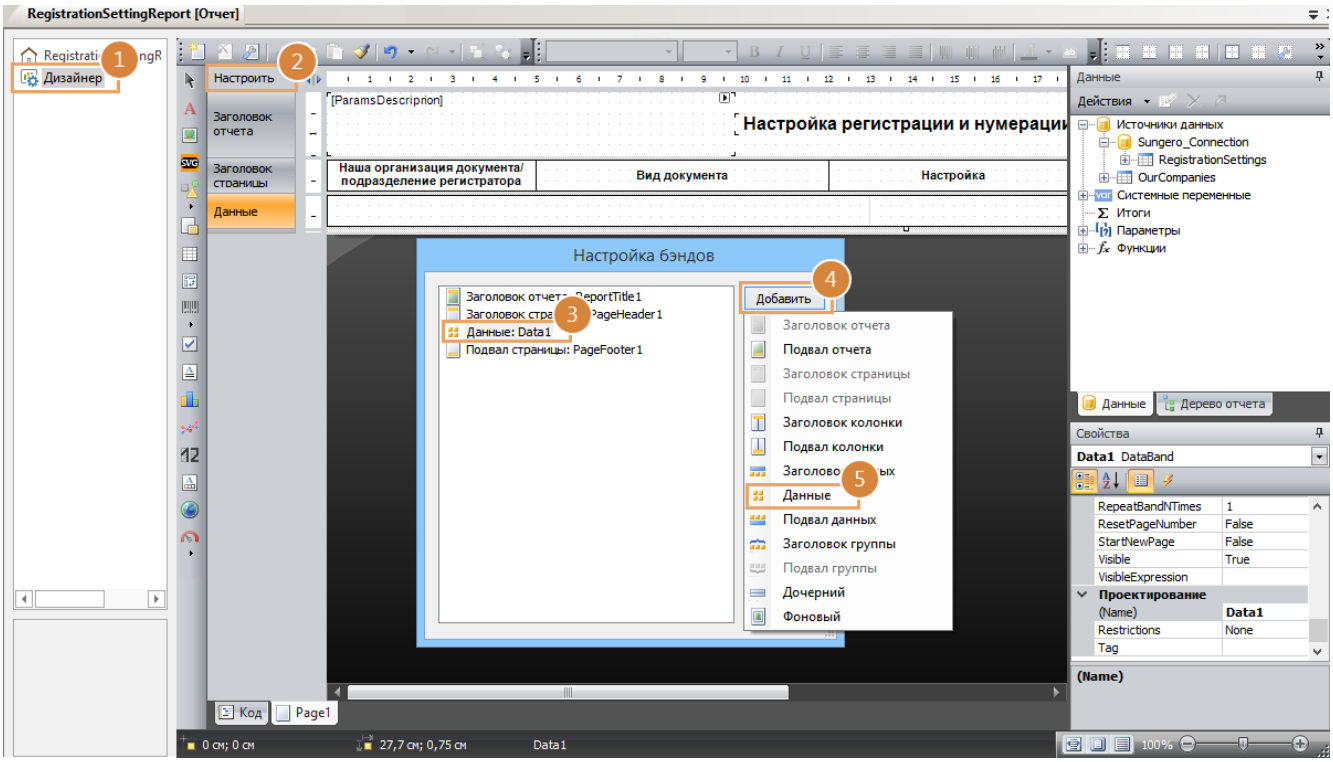
Для отчета созданы источники данных:

- OurCompanies – источник данных на основании типа сущности **Наша организация** (Sungero.Company.BusinessUnit);
- RegistrationSettings – SQL-источник данных, который содержит поля данных с информацией о настройках регистрации и нумерации видов документов.

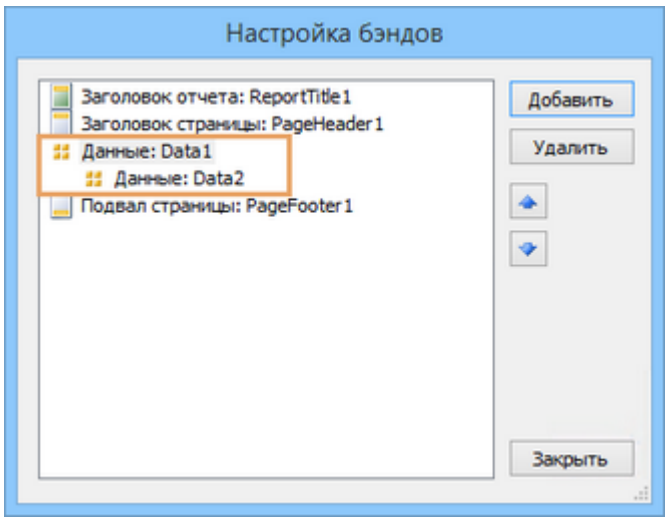
Источники связаны между собой отношением «главный – подчиненный» по идентификатору организации. Главным источником является OurCompanies, подчиненным – RegistrationSettings.

Необходимо, чтобы в отчете сначала отображалась строка с названием организации, а потом все соответствующие ей настройки. Для этого настройте макет отчета:

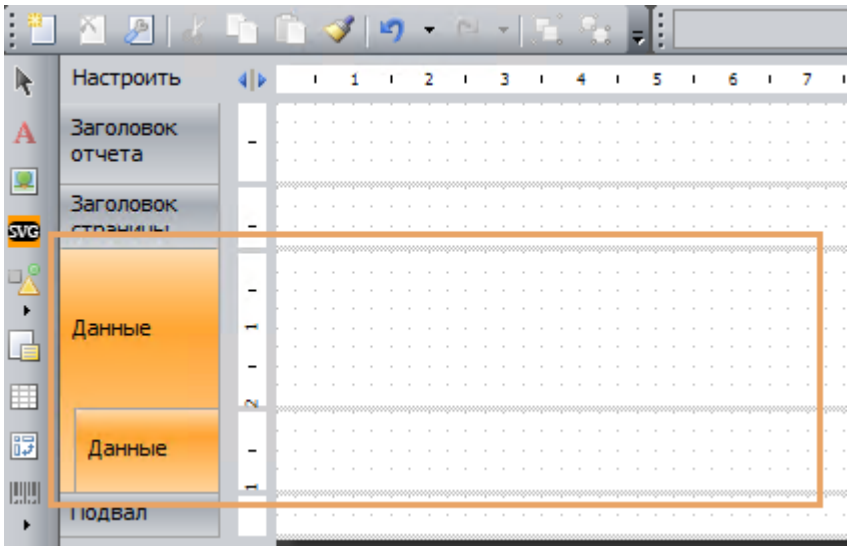
1. В редакторе отчета перейдите в узел «Дизайнер».



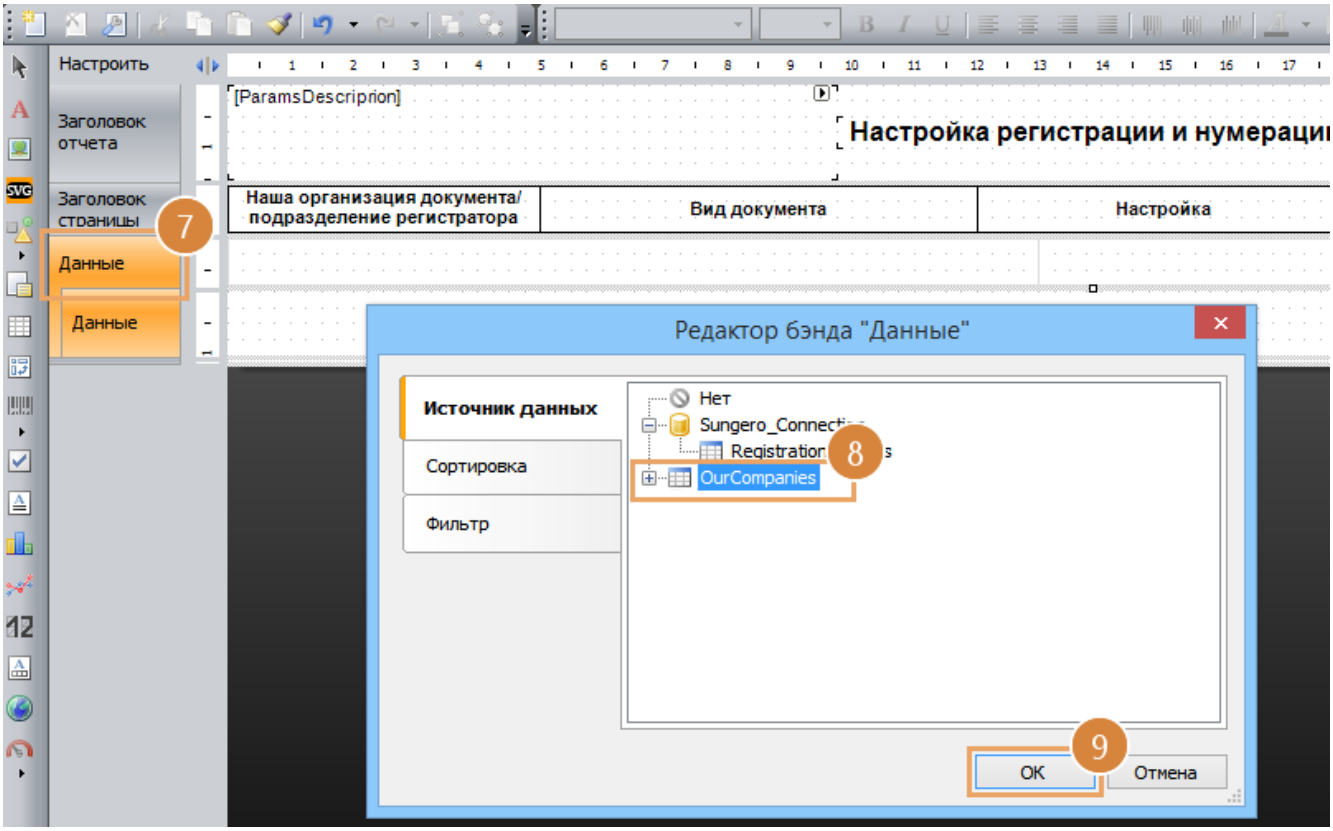
2. Нажмите на кнопку **Настроить** в дереве бэндов.
3. В окне настройки выделите бэнд «Данные».
4. Нажмите на кнопку **Добавить**.
5. В выпадающем списке выберите пункт **Данные**. В результате к выделенному бэнду добавится подчиненный бэнд «Данные»:



6. Нажмите на кнопку **Закрыть**. В макет отчета добавятся два бэнда «Данные». Бэнды расположены так, что главный бэнд содержит внутри себя подчиненный:



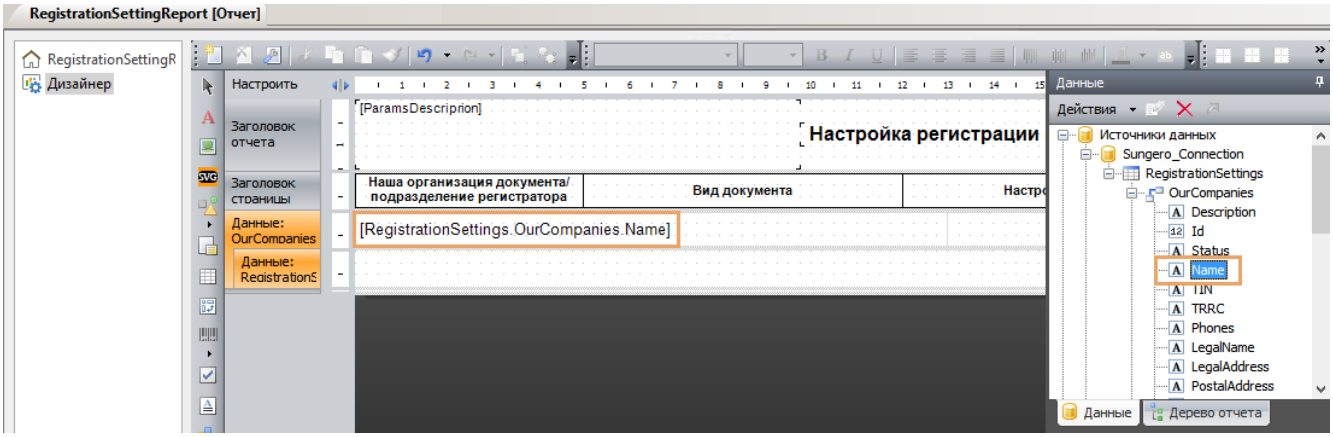
7. Подключите главный бэнд «Данные» к источнику данных. Для этого щелкните два раза левой кнопкой мыши на бэнде.



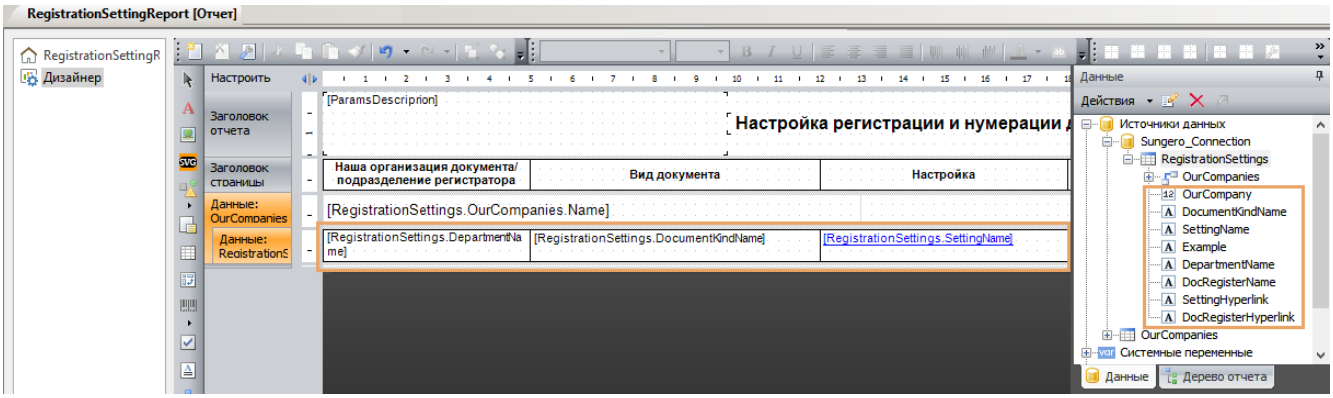
8. В открывшемся окне редактора бэнда на закладке «Источник данных» выберите источник OurCompanies.
9. Нажмите на кнопку **OK**.
10. Аналогично подключите подчиненный бэнд «Данные» к источнику данных RegistrationSettings.

11. Чтобы в отчете отображалась строка с названием организации, для которой созданы настройки регистрации и нумерации, добавьте на главный бэнд соответствующее поле данных. Для этого на панели «Данные» последовательно выберите RegistrationSettings > OurCompanies. > Name и переместите выделенное поле на главный бэнд «Данные».

    В результате будет создан объект «Текст», который подключен к выбранному полю.



12. Добавьте на подчиненный бэнд поля источника данных RegistrationSettings, значения которых нужно отображать в отчете.



В результате выполнения в отчете сначала отображается строка с названием организации, а потом все соответствующие ей настройки регистрации и нумерации видов документов:

Документопоток: Входящий
Фильтровать подразделения по нашим организациям: Да

02.09.2015

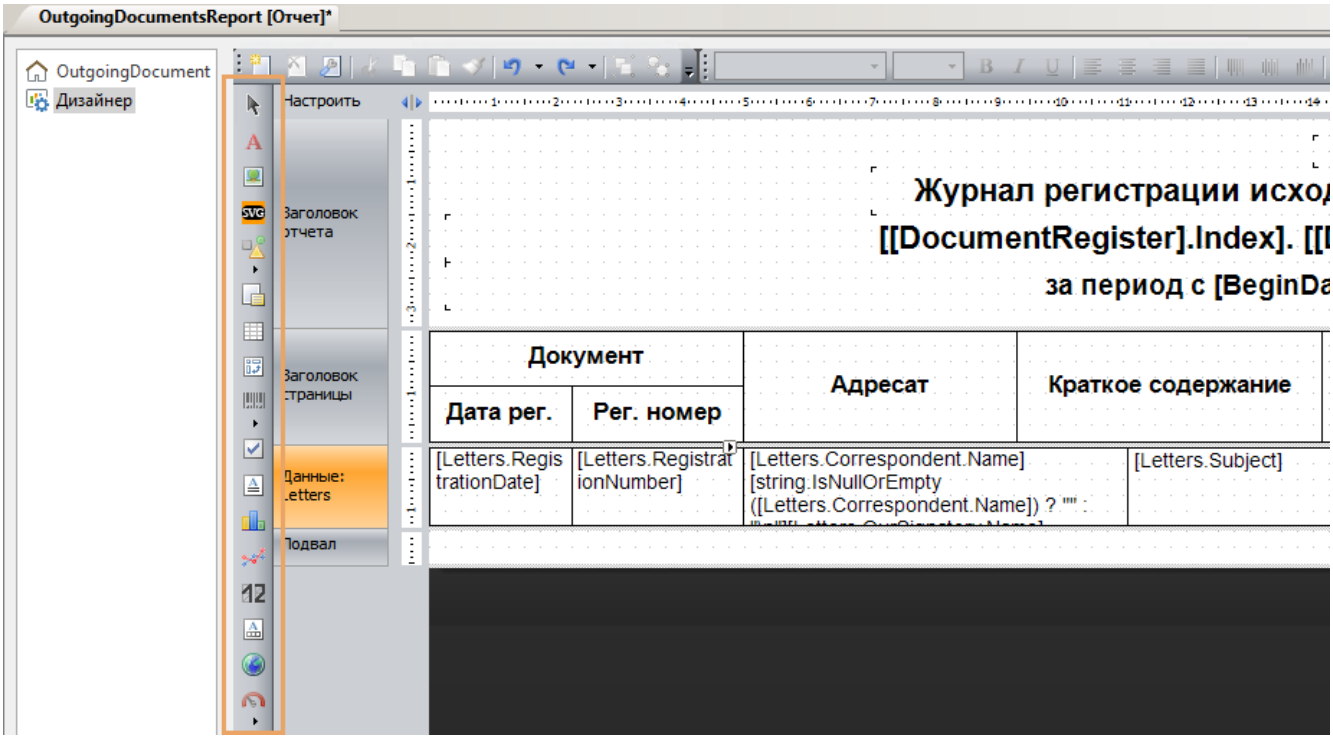
Настройка регистрации и нумерации документов

Наша организация документа/ подразделение регистратора	Вид документа	Настройка	Журнал регистрации	Пример номера
ЗАО "ТехноСистемы"				
Отдел снабжения	Входящее письмо; Входящие бухгалтерские документы; Входящий счет на оплату; Входящая заявка;			
	Входящие первичные учетные документы;	Входящие первичные учетные документы	Рег. вх. уч. док. Регистрация входящих первичных учетных документов	0001
Служба генерального директора	Входящее письмо;	Входящие Техносистемы	ВХ. Входящие	ВХ-0001/15
	Входящие бухгалтерские документы; Входящий счет на оплату; Входящая заявка;			
	Входящие первичные учетные документы;	Входящие первичные учетные документы	Рег. вх. уч. док. Регистрация входящих первичных учетных документов	0001
ООО "Прибор-Консалт"				
Отдел продаж	Входящая заявка;	Входящие заявки	ВХ3. Входящие заявки	ВХ3-001/2015
	Входящее письмо;	Входящие письма	ВХ. Входящие	ВХ-001/2015
	Входящие бухгалтерские документы; Входящий счет на оплату;			
	Входящие первичные учетные документы;	Входящие первичные учетные документы	Рег. вх. уч. док. Регистрация входящих первичных учетных документов	0001
Служба исполнительного директора	Входящее письмо;	Входящие письма	ВХ. Входящие	ВХ-001/2015
	Входящие бухгалтерские документы; Входящий счет на оплату; Входящая заявка;			
	Входящие первичные учетные документы;	Входящие первичные учетные документы	Рег. вх. уч. док. Регистрация входящих первичных учетных документов	0001

Добавление объектов отчета

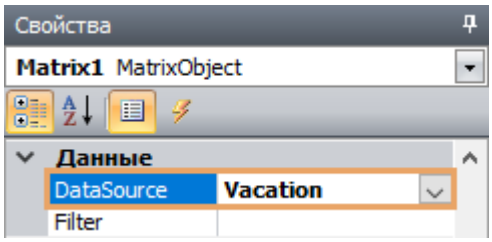
Объекты используются для отображения данных и оформления отчета. К объектам отчетов относятся, например, текст или таблица. Добавляются на бэнды в макете отчета.

Для добавления объектов используются кнопки панели объектов, расположенной в левой части редактора отчета в узле «Дизайнер»:



Для отчетов доступны следующие основные объекты:

Объект	Назначение	Пример использования
Текст	Отображение любых текстовых данных	Название и данные отчета
Рисунок	Добавление графического изображения	Логотип компании, фотографии сотрудников
Фигуры	Добавление линии, прямоугольника, эллипса, треугольника, ромба, многоугольника и т.д.	Оформление отчета
Вложенный отчет	Создание дополнительного макета отчета, имеющего свою структуру	Отображение двух независимых списков данных, расположенных рядом
Таблица	Оформление данных отчета в виде таблицы	Заголовок страницы с данными отчета
Матрица	Оформление данных отчета в виде таблицы. В отличие от объекта «Таблица», количество строк и столбцов матрицы заранее не известно и зависит от источника данных, к которым подключен объект. После добавления матрицы на бэнд: • выделите матрицу мышью и в свойстве DateSourse укажите источник данных, с которым будет связана матрица:	Заголовок страницы с данными отчета

Объект	Назначение	Пример использования
	 • перетащите нужные поля из области «Данные» в матрицу. Подробнее см. в документации FastReport.Net статью «Матричные (сводные) отчеты»	
Штрихкод	Отображение различных типов штрихкодов. СОВЕТ. Если штрих-код не считывает сканер: • убедитесь, что данный формат штрихкода поддерживает ваш сканер. Подробнее см. документацию сканера; • распечатайте отчет со штрихкодом и повторите попытку сканирования. При считывании штрихкода с экрана монитора иногда возникают проблемы; • если отчет формируется в формате PDF, попробуйте преобразовать его в формат Word; • измените расположение штрихкода в отчете: переместите его чуть левее, правее, вверх, вниз.	Штрихкод для идентификации отчета
Флажок	Отображение данных логического типа (Boolean). В отчете отображается в виде флажка с состояниями «включен» и «выключен	Значения параметров, которые указал пользователь при выполнении отчета
Форматированный текст	Добавление текстовых данных и быстрой настройки форматирования	Оформление данных в виде маркированного или нумерованного списка
Диаграмма MS Chart	Построение различных видов диаграмм на основе данных отчета. Подробнее см. в документации FastReport.Net статью «Отчеты с диаграммами»	Представление данных в виде диаграмм
Почтовый индекс	Отображение почтового индекса	Индекс для создания конвертов
Текст в ячейках	Добавление текста, каждый символ которого выводится в отдельной ячейке	Элементы отчета, которые после печати должны быть заполнены вручную печатными буквами
Карта	Отображение двумерных графических карт в формате ESRI shapefile.	Оформление данных в виде графических карт

Наиболее часто используются объекты «Текст», «Рисунок», «Таблица». Подробнее описание других объектов см. в документации FastReport.Net, статью «Объекты отчета».

ВАЖНО. Некоторые возможности FastReport.Net отключены в среде разработки Directum RX.

Свойства объектов

У каждого объекта есть свои свойства, которые определяют способ отображения и оформление объекта в готовом отчете:

- Свойства объекта «Текст»
- Свойства объекта «Рисунок»

Также все объекты отчета имеют ряд общих свойств.

ПРИМЕЧАНИЕ. В таблице перечислены только наиболее часто используемые свойства. Описание других свойств см. в документации FastReport.Net.

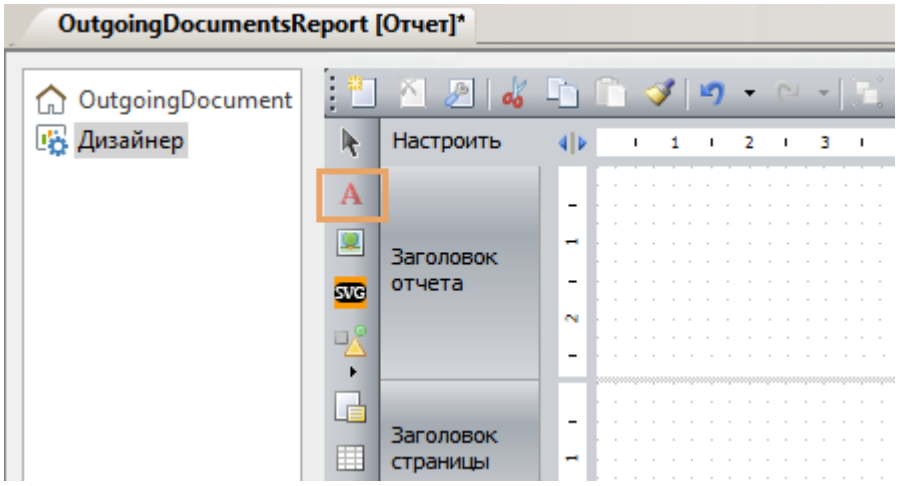
Свойство	Описание
Внешний вид	
Border	Настройка рамки объекта
Fill	Настройка заливки объекта
Макет	
Anchor	Способ изменения позиции объекта и/или его размеров при изменении размеров бэнда, на котором он расположен
Height	Высота объекта в сантиметрах
Left	Расстояние от левого края страницы до объекта в сантиметрах
Top	Расстояние от верхнего края страницы до объекта в сантиметрах
Width	Ширина объекта в сантиметрах
Поведение	
CanGrow	Увеличение размера объекта. Возможные значения: **True**. Размер объекта автоматически увеличивается под размер данных. **False**. Размер объекта фиксирован. Значение по умолчанию
CanShrink	Уменьшение размера бэнда. Возможные значения: **True**. Размер объекта автоматически уменьшается под размер данных. **False**. Размер объекта фиксирован. Значение по умолчанию
GrowToBottom	Растягивание объекта до нижней границы бэнда. Возможные значения: **True**. Объект растягивается до нижней границы бэнда, независимо от того, сколько в нем данных. **False**. Размер объекта зависит от размера содержимого. Значение по умолчанию
Printable	Способ отображения объекта. Возможные значения: **True**. Объект отображается в отчете. Значение по умолчанию. **False**. Объект не отображается в отчете

Свойство	Описание
PrintOn	Страницы, на которых отображается объект. Возможные значения: **FirstPage**. Первая страница. **LastPage**. Последняя страница. **OddPages**. Нечетные страницы. **EvenPages**. Четные страницы. **RepeatedBand**. Только на бэндах, которые повторяются на каждой странице. **SinglePage**. Единственная страница. Объект отображается, если отчет состоит из одной страницы. Свойство может принимать несколько значений одновременно. По умолчанию равно сочетанию **FirstPage**, **LastPage**, **OddPages**, **EvenPages**, **RepeatedBand**, **SinglePage**. Подробнее о работе со свойством см. в документации FastReport.Net
ShiftMode	Объект сдвигается вниз или вверх, если над ним имеется объект, который расширяется или сжимается. Возможные значения: **Always**. Объект сдвигается всегда, даже если между объектами есть свободное пространство. Значение по умолчанию. **Never**. Объект не сдвигается. **WhenOverlapped**. Объект сдвигается в том случае, если расширяющийся объект примыкает к нему сверху

Текст

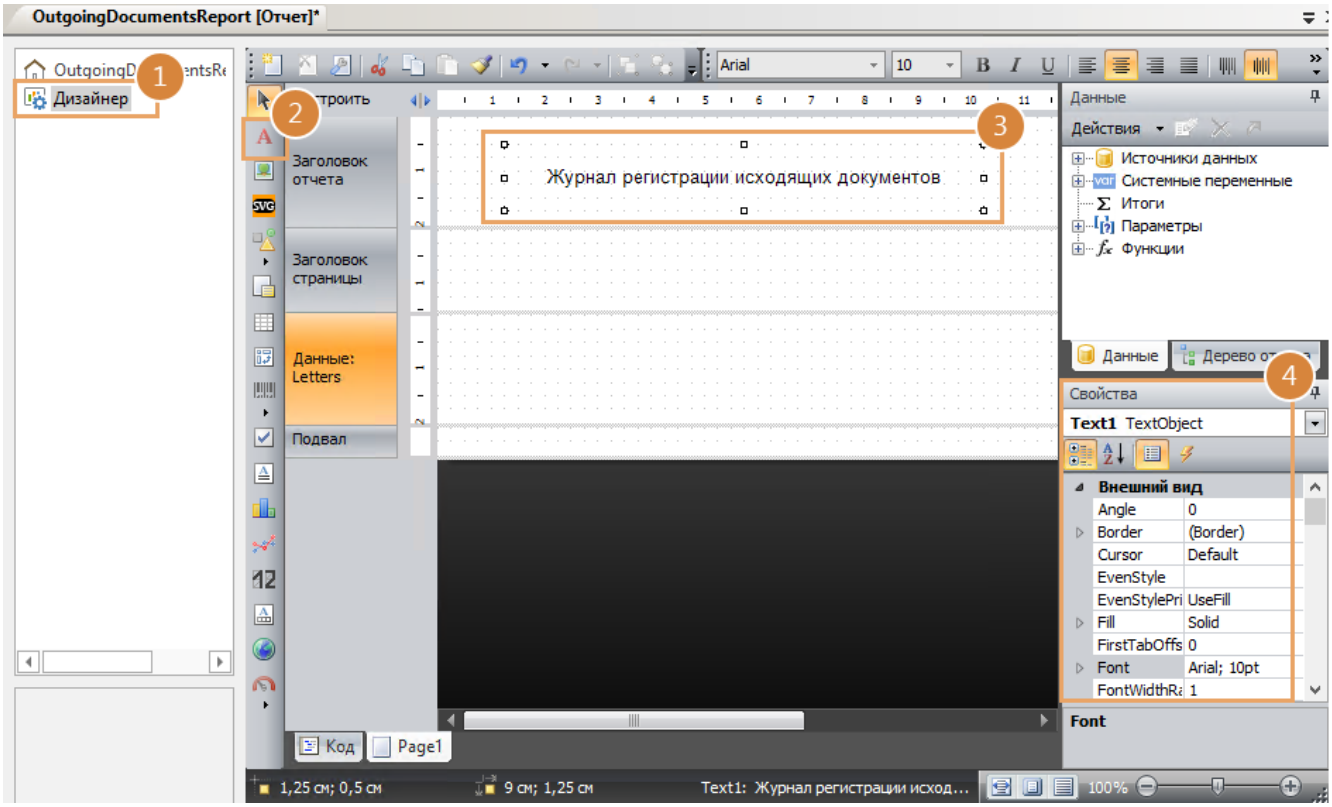
Объект «Текст» предназначен для отображения любых текстовых данных, например, простого текста или значений выражений.

Объект создается автоматически при перемещении на макет страницы системных переменных, полей источников данных, параметров отчета и функций из окна «Данные». Также объект можно создать с помощью кнопки **Текст** на панели объектов в редакторе отчета:



Предположим, разрабатывается отчет, содержащий исходящие документы, зарегистрированные в определенном журнале регистрации в некоторый период времени. Необходимо добавить заголовок отчета. Для этого:

1. В редакторе отчета перейдите в узел «Дизайнер».



2. На панели объектов нажмите на кнопку **Текст**.
3. Выберите нужную позицию в макете отчета на *бэнде* «Заголовок отчета» и нажмите левую кнопку мыши для вставки объекта.
4. При необходимости задайте свойства объекта. Для этого щелкните левой кнопкой мыши на объекте и задайте значения свойств в окне «Свойства».

   Кроме общих свойств для всех объектов, объект «Текст» имеет собственный набор свойств.

   ПРИМЕЧАНИЕ. В таблице перечислены только наиболее часто используемые свойства. Описание других свойств см. в документации FastReport.Net.

Свойство	Описание
Внешний вид	
Angle	Угол поворота текста. Значение задается в градусах
FirstTabOffset	Количество пикселей, на которое сдвинут первый символ табуляции
FontWidthRatio	Коэффициент масштабирования ширины шрифта. Значение по умолчанию 1
HorzAlign	Выравнивание текста по горизонтали. Возможные значения: **Left**. Выравнивание по левому краю. Значение по умолчанию. **Center**. Выравнивание по центу. **Right**. Выравнивание по правому краю. **Justify**. Выравнивание по ширине
LineHeight	Межстрочный интервал. Значение задается в сантиметрах

Свойство	Описание
TabWidth	Ширина символа табуляции. Значение задается в сантиметрах
TextFill	Цвет и заливка текста
Underlines	Подчеркивание текста. Возможные значения **True** и **False**. Значение по умолчанию **False**
VertAlign	Выравнивание текста по вертикали и горизонтали. Возможные значения: **Top**, **Center**, Bottom. Значение по умолчанию **Top**
Данные	
AllowExpressions	Использование выражений в тексте. Возможные значения: **True**. Выражения обрабатываются в тексте. Значение по умолчанию. **False**. В тексте отключена обработка выражений
Brackets	Пара символов, которые используются для обозначения выражений в текстовом поле. По умолчанию используются квадратные скобки
Highlight	Задание настроек условного выделения
Text	Текст объекта
Макет	
Padding	Отступы текста от краев объекта
Поведение	
AutoShrink	Автоматический подбор размера шрифта под размер поля для текста. Возможные значения: **None**. Размер и ширина шрифта не изменяются. Значение по умолчанию. **FontSize**. Размер шрифта автоматически подбирается так, чтобы вместить весь текст. **FontWidthRatio**. Ширина шрифта автоматически подбирается так, чтобы вместить весь текст
AutoShrinkMinSize	Минимальный размер, до которого может уменьшаться шрифт текста, если в свойстве **AutoShrink** установлено значение **FontWidthRatio**
AutoWidth	Автоматический подбор ширины. Возможные значения: **True**. Ширина объекта подбирается так, чтобы поместить самую длинную строку текста целиком. **False**. Ширина объекта фиксирована. Значение по умолчанию
BreakTo	Перенос текста по двум объектам. В качестве параметра указывается объект, в котором отображается текст, не поместившийся в первом объекте
CanBreak	Отображение текста объекта при переходе на другую страницу. Возможные значения: **True**. При переходе на другую страницу часть текста объекта переносится на новую страницу. У бэнда, на котором находится объект, значение аналогичного свойства также должно быть **True**. Значение по умолчанию. **False**. При переходе на другую страницу весь текст объекта переносится на новую страницу
Clip	Обрезание текста объекта. Возможные значения: **True**. Текст, который не поместился в объекте, обрезается. Значение по умолчанию. **False**. Текст, который не поместился в объекте, отображается полностью

Свойство	Описание
Duplicates	Указание способа отображения повторяющихся данных отчета. Повторяющимися считаются одинаковые значения, напечатанные в соседних строках данных. Возможные значения: **Show**. Повторяющиеся значения отображаются в отчете. Значение по умолчанию. **Hide**. Объект с повторяющимися значениями скрыт в отчете. **Clear**. Текст в объекте с повторяющимся значением очищен, но сам объект отображается в отчете. **Merge**. Объекты с одинаковыми значениями объединяются
HideValue	Значения выражений, которые нужно скрывать в отчете
HideZeros	Скрытие нулевых значений выражений. Возможные значения: **True**. Нулевые значения скрыты в отчете. **False**. Нулевые значения отображаются в отчете. Значение по умолчанию
NullValue	Строка, которая выводится в отчете вместо значения null
RightToLeft	Указание способа отображения текста. Возможные значения: **True**. Текста отображается справа налево. **False**. Текста отображается слева направо. Значение по умолчанию
WordWrap	Перенос текста по словам. Возможные значения: **True**. Текст объекта может переноситься по словам. **False**. В тексте отключен перенос текста по словам. Значение по умолчанию

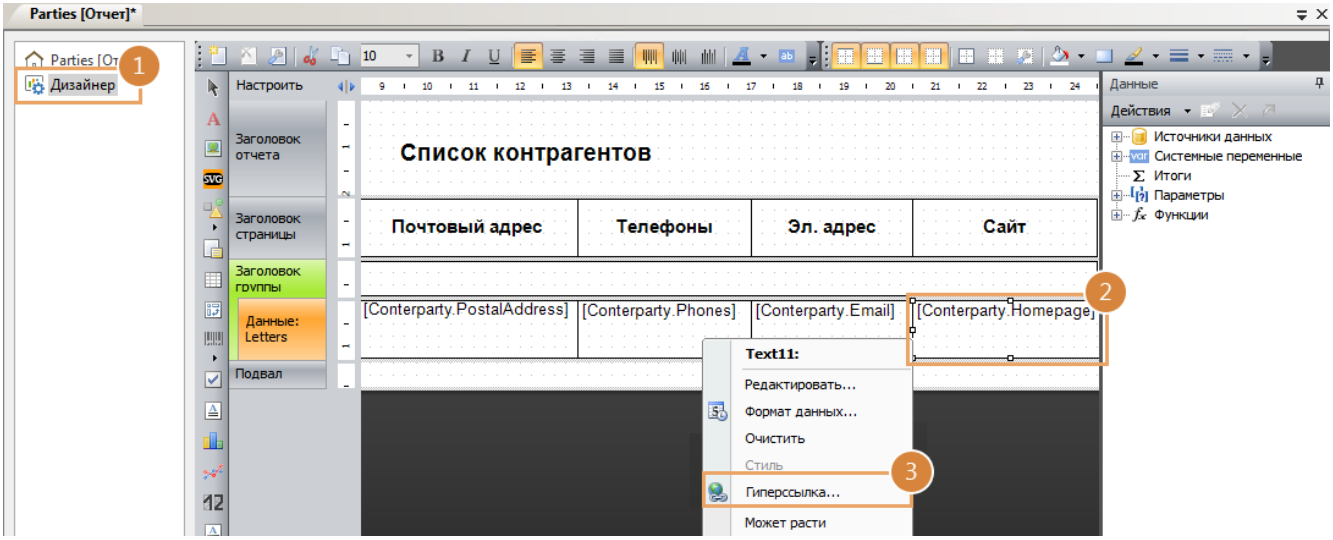
5. Задайте текст объекта с помощью редактора текста.

Настройка гиперссылок

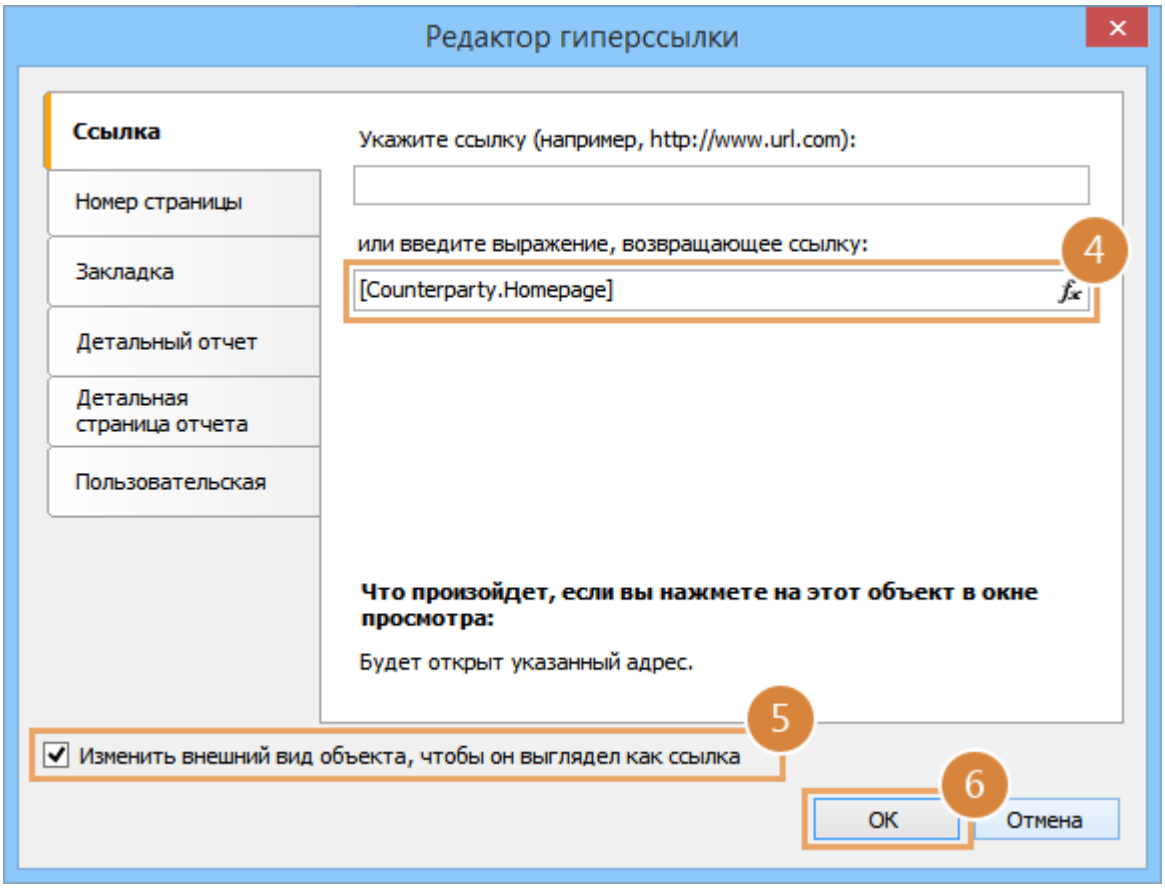
Гиперссылка позволяет настроить действие, которое будет происходить при переходе по ссылке в готовом отчете. Например, отправка e-mail, открытие веб-страницы.

Предположим, разрабатывается отчет, в котором содержится информация о контрагентах компании. В отчете содержатся адреса сайтов контрагентов. Необходимо, чтобы при переходе по ссылке открывался указанный сайт. Для этого:

1. В редакторе отчета перейдите в узел «Дизайнер».



2. Выделите объект «Текст», соответствующий полю источника данных **Сайт** с адресом сайта контрагента.
3. В контекстном меню объекта выберите пункт **Гиперссылка...**.
4. В редакторе гиперссылки на закладке «Ссылка» укажите выражение, которое возвращает ссылку на сайт контрагента. Для этого нажмите на кнопку f_x и в редакторе выражений добавьте выражение [Counterparty.Homepage] (поле источника данных **Сайт**).



5. Установите флажок **Изменить внешний вид объекта**, чтобы гиперссылка отображалась в отчете синим цветом с подчеркиванием. В готовом отчете при наведении на гиперссылку будет отображаться курсор .
6. Нажмите на кнопку **ОК**.

В результате выполнения отчета адрес сайта контрагента отображается в виде гиперссылки. При переходе по гиперссылке откроется указанный сайт.

24.08.2015

Список контрагентов

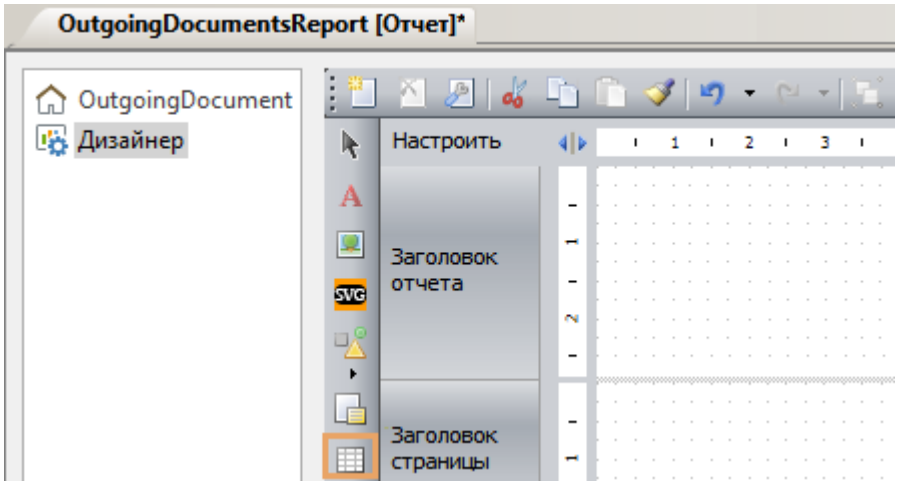
Наименование	Юридический адрес	Почтовый адрес	Телефоны	Эл. адрес	Сайт
Населенный пункт: г. Москва					
Выставочный центр "ЭКСПО-БИЗНЕС"	110020, Россия, г. Москва, ул. Вострякова, 118	110020, Россия, г. Москва, ул. Вострякова, 118	(495) 59-56-98, факс: 467-99-76		www.ekspo-bizn.ru
ООО "Дрофа"	127018, Россия, г. Москва, Сущевский вал. 49, стр 1	127018, Россия, г. Москва, Сущевский вал. 49, стр 1	(495) 795-05-45	info@drofa.ru	www.drofainfo.ru
ЗАО "Банк-МКБ"	121112, Россия, г. Москва, ул.Зорге, д.1	121112, Россия, г. Москва, ул.Зорге, д.1	(495) 987-62-19		www.bankmkb.ru
ООО "Русский кремний"	105062, Россия, г. Москва, Подсосенский пер., д.21, стр.2	105062, Россия, г. Москва, Подсосенский пер., д.21, стр.2	(495) 651-68-28	rus-silicium@rus-sil.ru	www.rus-silicium.ru

Таблица

Объект «Таблица» используется для оформления данных отчета в виде таблицы. Состоит из строк, колонок и ячеек. В ячейках таблицы указывается простой текст или выражения.

ПРИМЕЧАНИЕ. Таблицу не рекомендуется использовать на бэнде «Данные».

Объект создается с помощью кнопки **Таблица** на панели объектов в редакторе отчета:

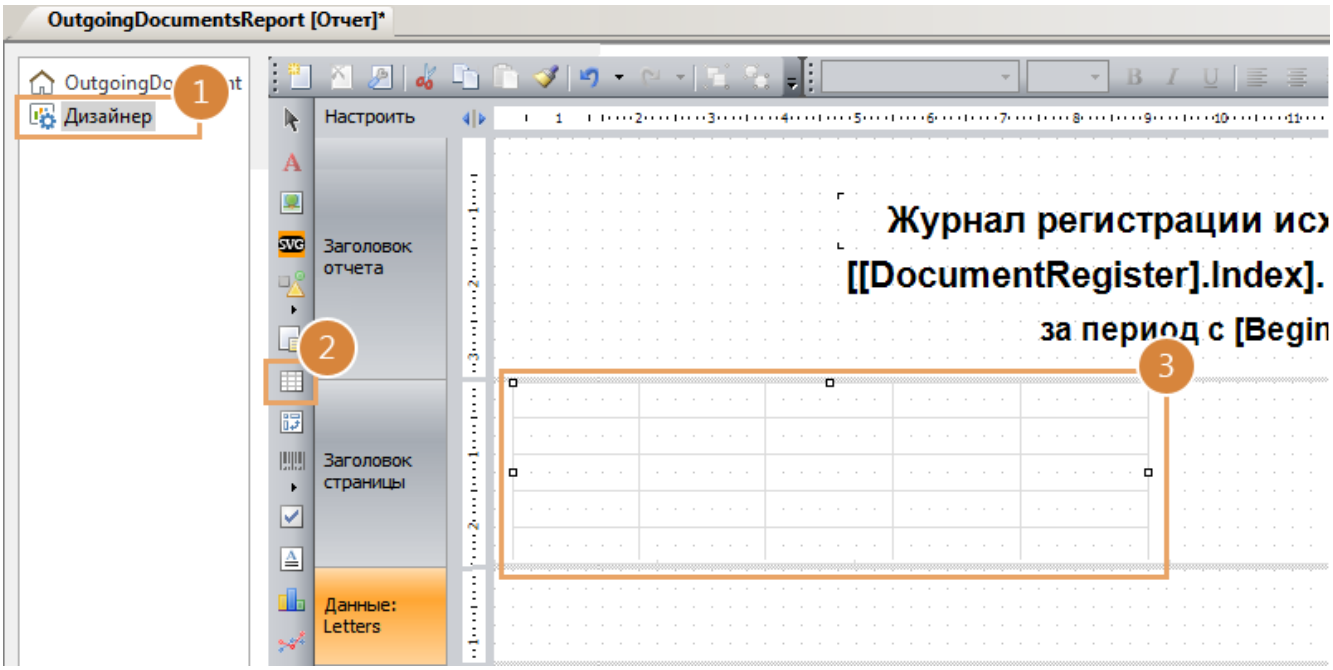


Предположим, разрабатывается отчет, в котором содержится информация об исходящих документах, зарегистрированных в определенном журнале регистрации в некоторый период времени. Необходимо добавить заголовок для таблицы данных, который будет отображаться на каждой странице отчета. Заголовок должен иметь следующий вид:

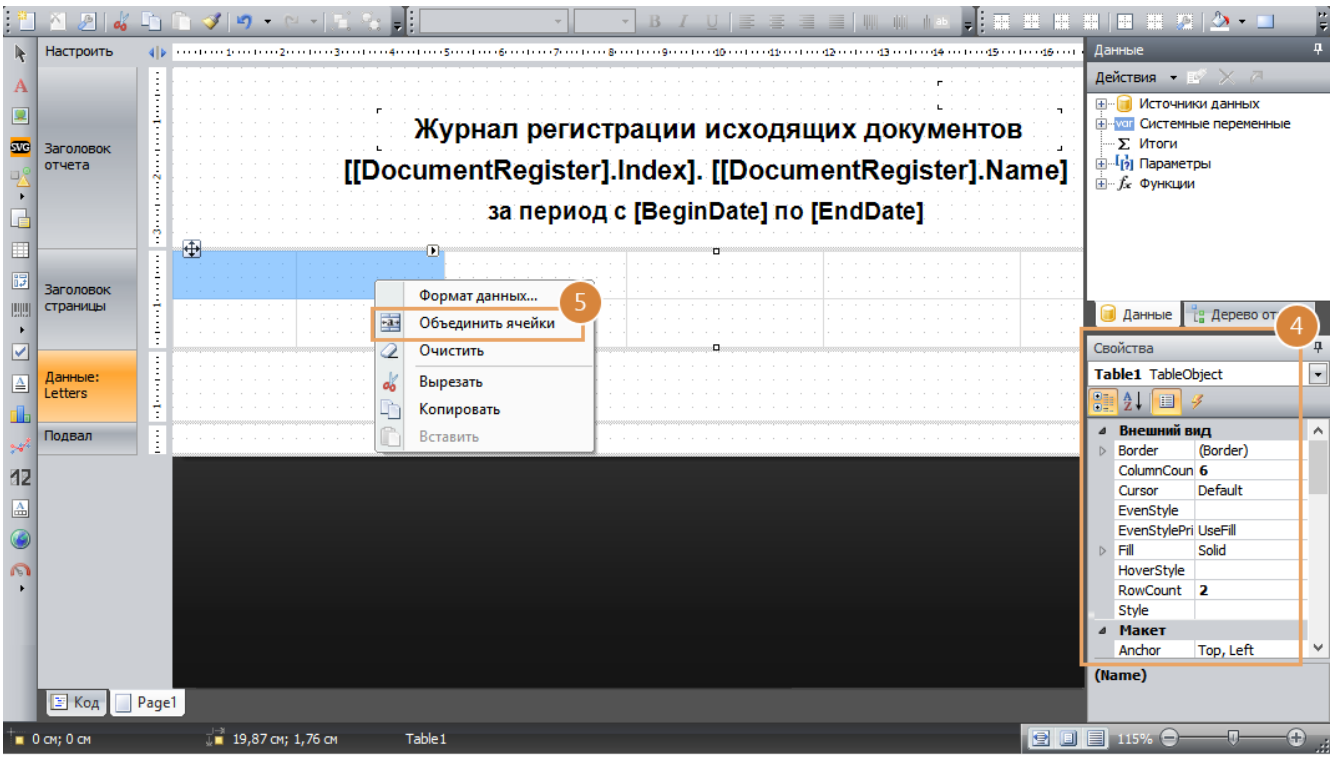
Документ		Адресат	Краткое содержание	Исполнитель	Примечание
Дата рег.	Рег. номер				

Для этого:

1. В редакторе отчета перейдите в узел «Дизайнер».



2. На панели объектов нажмите на кнопку **Таблица**.
3. Выберите нужную позицию в макете отчета на бэнде «Заголовок страницы» и нажмите левую кнопку мыши для вставки объекта. В макет отчета добавится таблица, состоящая из пяти колонок и пяти столбцов.
4. Задайте необходимое количество колонок и строк в таблице. Для этого щелкните левой кнопкой мыши на объекте и в окне «Свойства» в группе «Внешний вид» задайте для свойств **ColumnCount** (количество колонок таблицы) и **RowCount** (количество строк таблицы) значения 6 и 2 соответственно.



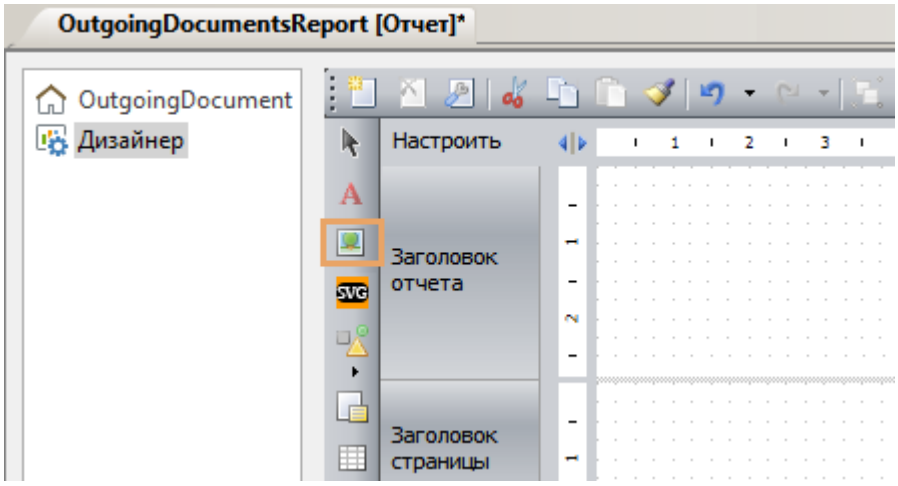
5. Чтобы объединить ячейки таблицы, выделите нужные ячейки и в контекстном меню выберите пункт **Объединить ячейки**.

6. В каждую ячейку таблицы добавьте название колонки будущей таблицы отчета. Для этого дважды щелкните по ячейке и воспользуйтесь открывшимся редактором текста.
7. Задайте форматирование для границ таблицы и текста ячеек.

Рисунок

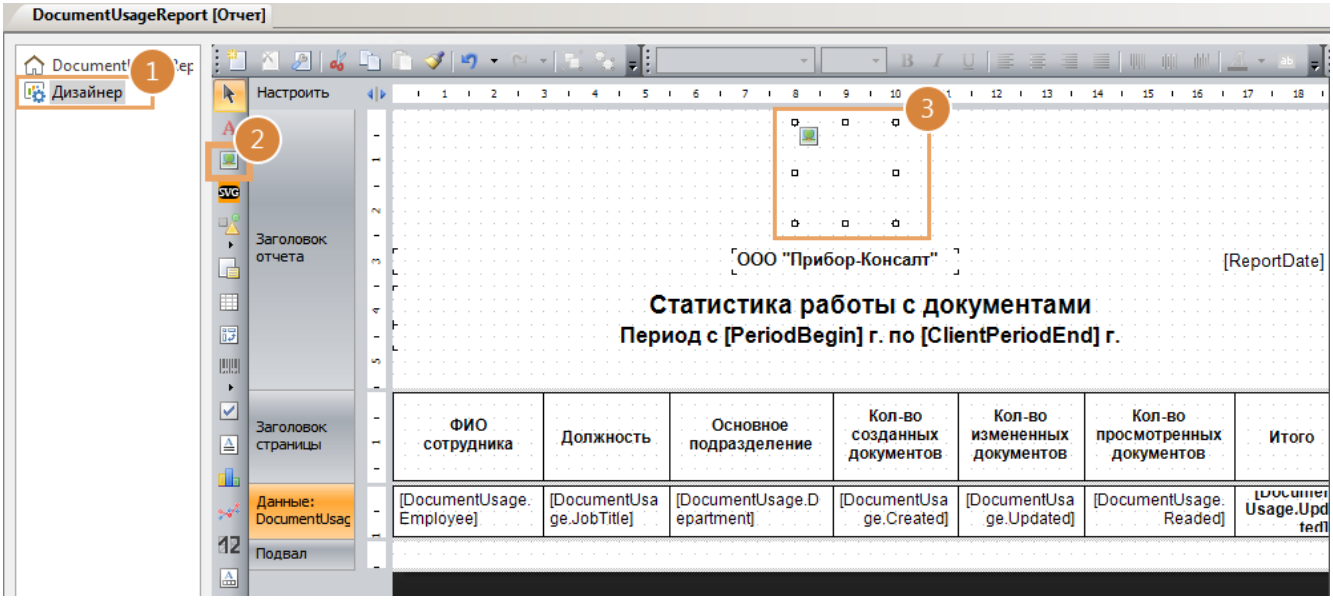
Объект «Рисунок» позволяет добавить в отчет графическое изображение в форматах **BMP**, **PNG**, **JPG**, **GIF**, **TIFF**, **ICO**, **EMF**, **WMF**. Например, логотип компании или фотографию сотрудника.

Объект создается с помощью кнопки **Рисунок** на панели объектов:



Предположим, разрабатывается отчет, в котором содержится статистика по работе с документами. В заголовок отчета необходимо добавить логотип компании. Для этого:

1. В редакторе отчета перейдите в узел «Дизайнер».



2. На панели объектов нажмите на кнопку **Рисунок**.
3. Выберите нужную позицию в макете отчета на бэнде «Заголовок отчета» и нажмите левую кнопку мыши для вставки объекта.
4. Щелкните два раза по объекту, чтобы открыть редактор рисунка.

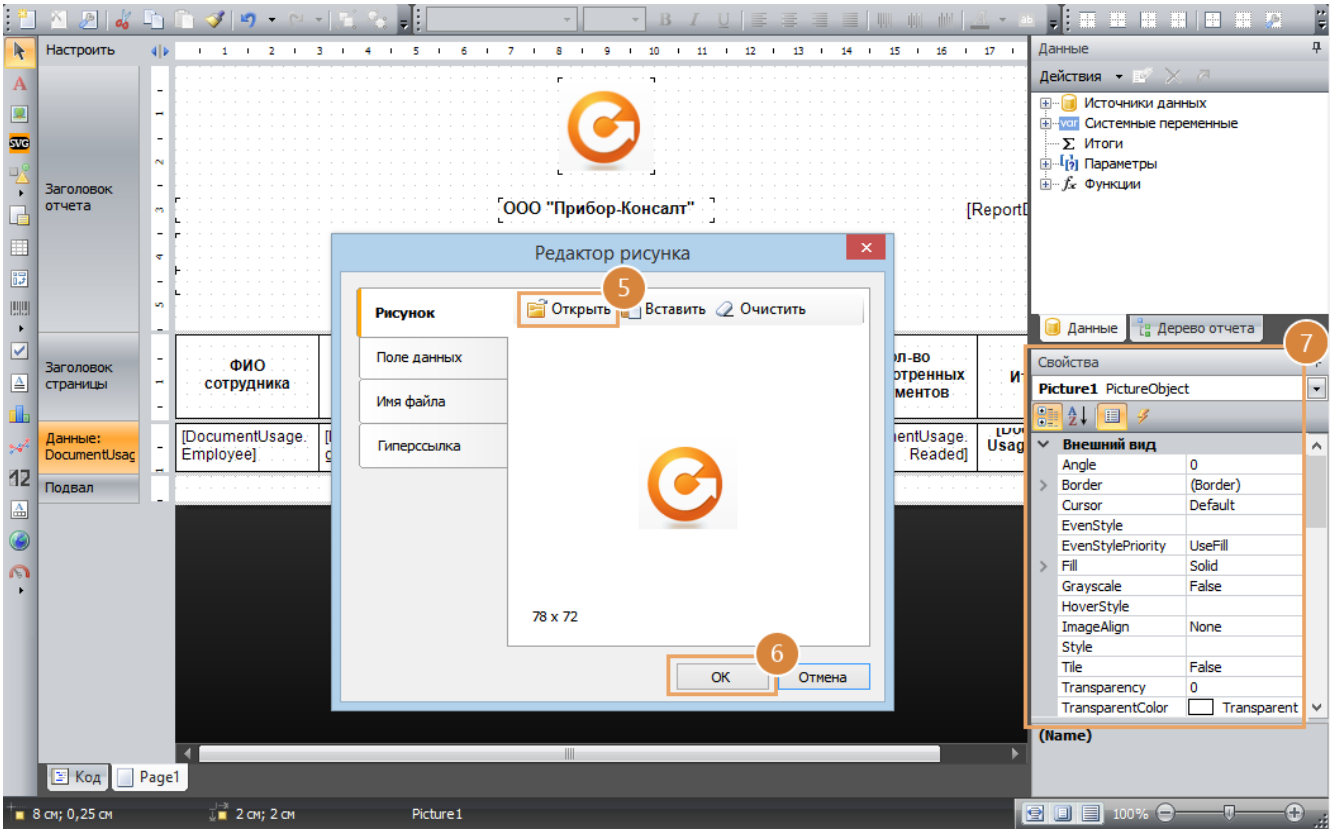
5. В открывшемся редакторе укажите рисунок, который необходимо отображать в заголовке отчета. Доступны следующие источники рисунков:

   Рисунок. Загружается из файла на компьютере и хранится в свойстве **Image** объекта «Рисунок».

   Поле данных. Рисунок загружается из поля источника данных. Имя поля хранится в свойстве **DataColumn** объекта «Рисунок».

   Гиперссылка. Рисунок загружается из Интернета каждый раз, когда отчет выполняется. Адрес хранится в свойстве **ImageLocation** объекта «Рисунок».

   Чтобы загрузить рисунок из файла, в редакторе на закладке «Рисунок» нажмите на кнопку **Открыть** и выберите нужный рисунок.



6. Нажмите на кнопку **OK**.

7. При необходимости задайте свойства объекта. Для этого щелкните левой кнопкой мыши на объекте и задайте значения свойств в окне «Свойства».

   Кроме общих свойств для всех объектов, объект «Рисунок» имеет собственный набор свойств.

   ПРИМЕЧАНИЕ. В таблице перечислены только наиболее часто используемые свойства. Описание других свойств см. в документации FastReport.Net.

Свойство	Описание
Angle	Угол поворота рисунка в градусах. Возможные значения: 0, 90, 180, 270
SizeMode	Режим отображения рисунка. Возможные значения: **AutoSize**. Объект принимает размеры рисунка. **CenterImage**. Рисунок располагается по центру объекта. **Normal**. Рисунок выводится в левом верхнем углу объекта без масштабирования.

Свойство	Описание
	StretchImage. Рисунок растягивается до размеров объекта. Пропорции рисунка не соблюдаются. **Zoom**. Рисунок растягивается до размеров объекта с соблюдением пропорций
Transparency	Степень прозрачности картинки. Свойство принимает значение от 0 до 1. Значение по умолчанию – 0, означает, что картинка непрозрачна
TransparentColor	Цвет, который будет прозрачным при отображении картинки
Image	Рисунок, который отображается в объекте
DataColumn	Поле данных, из которого загружается рисунок
Padding	Свойство позволяет задать отступы рисунка от краев объекта. Значение указывается в пикселях
ShowErrorImage	Отображение значка «Нет рисунка», если объект пустой. Используется, если рисунок загружается из Интернета

В результате выполнения отчета в заголовке будет отображаться логотип компании:

ООО "Прибор-Консалт" 27.08.2015 г.

Статистика работы с документами

Период с 28.07.2015 г. по 27.08.2015 г.

ФИО сотрудника	Должность	Основное подразделение	Кол-во созданных документов	Кол-во измененных документов	Кол-во просмотренных документов	Итого
Ивановский Геннадий Александрович	Генеральный директор	Служба генерального директора	2	7	15	**24**
Охотникова Елена Сергеевна	Делопроизводитель	Служба генерального директора	10	13	15	**38**
Соболева Надежда Николаевна	Секретарь	Служба генерального директора	17	22	14	**53**

Редактирование текстов и выражений

В тексте объектов отчета используется как обычный текст, так и выражения.

Выражение – строка кода, которая при выполнении отчета возвращает какое-либо значение. Обозначается с помощью квадратных скобок. Например, объект с текстом **2 * 2 = [2 * 2]** в отчете отображается как **2 * 2 = 4**.

В выражениях используются:

- системные переменные. Например, выражение [Page] возвращает номер текущей страницы отчета. В выражении содержится системная переменная **Page**;

- поля источников данных. Для отображения используется формат записи: [Имя источника.Имя поля]. Например, выражение [Employees.Name] возвращает имя сотрудника;
- параметры отчета. Например, выражение [BeginDate] возвращает дату формирования отчета. В выражении содержится параметр **BeginDate**;
- математические функции. Например, выражение [FormatPercent(0.15)] возвращает указанное значение в процентах, то есть 15%. В выражении содержится функция **FormatPercent()**;
- программный код. Например:

```
// Получение ссылки на сущность и приведение ссылки к нижнему регистру.
  [Hyperlinks.Get([DocumentRegister]).ToLower()]
```

ПРИМЕЧАНИЕ. Чтобы использовать серверную функцию модуля, в объявлении функции явно укажите модификатор **static**.

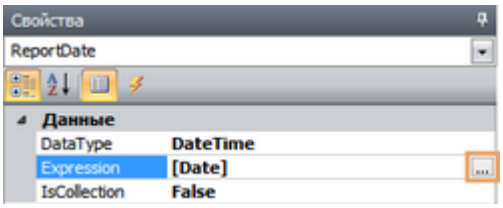
Также выражения могут использоваться:

- вместе с обычным текстом. Например, **Страница [Page]**. В результате в отчете на первой странице отобразится: **Страница 1**;
- в качестве аргументов других выражений. Например, **Следующая страница [[Page] + 1]**. В результате в отчете на первой странице отобразится: **Следующая страница 2**.

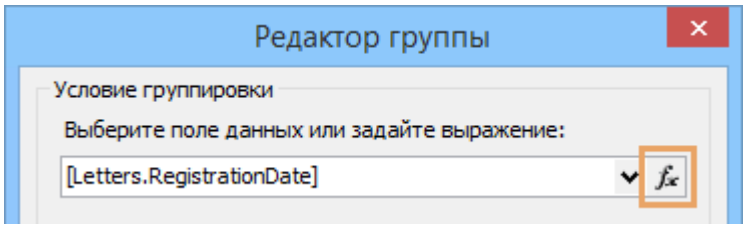
Для изменения текста и написания выражений используются соответствующие редакторы.

Редактор выражения вызывается:

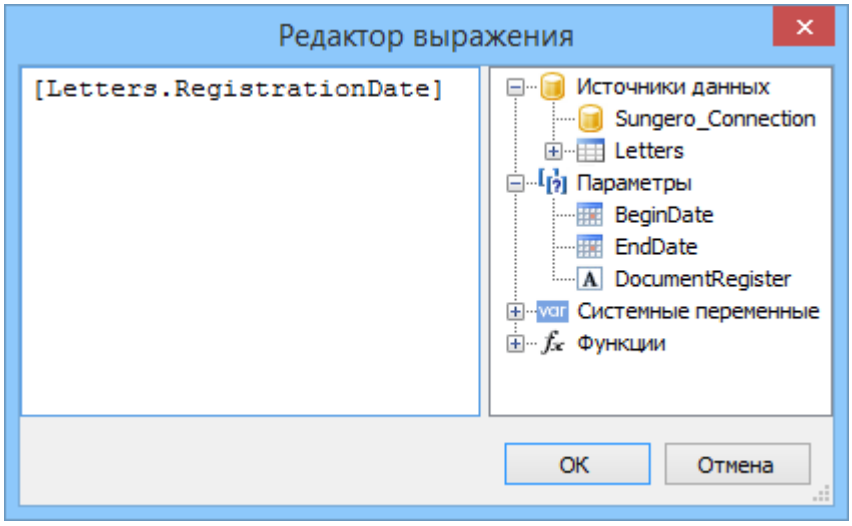
- по кнопке ..., например на панели «Свойства»:



- по кнопке fx, например в редакторе группы данных отчета:

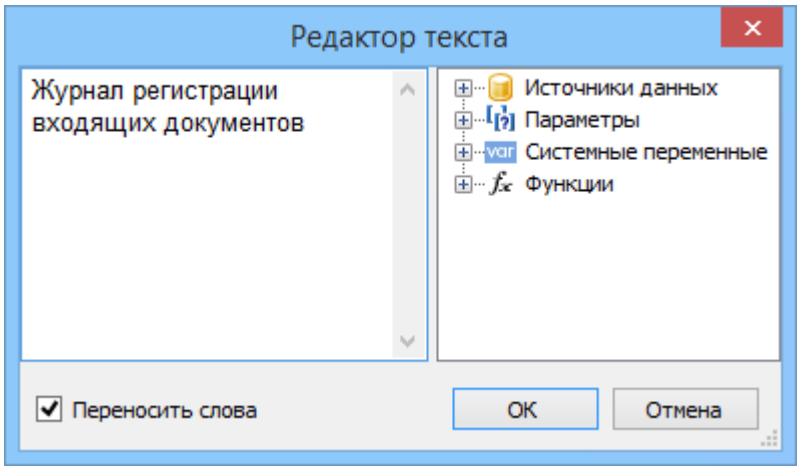


Редактор имеет вид:



В левой части редактора находится поле для задания текста и выражений. В правой части содержатся элементы, которые используются в выражениях: поля источников данных, параметры отчета, системные переменные и функции.

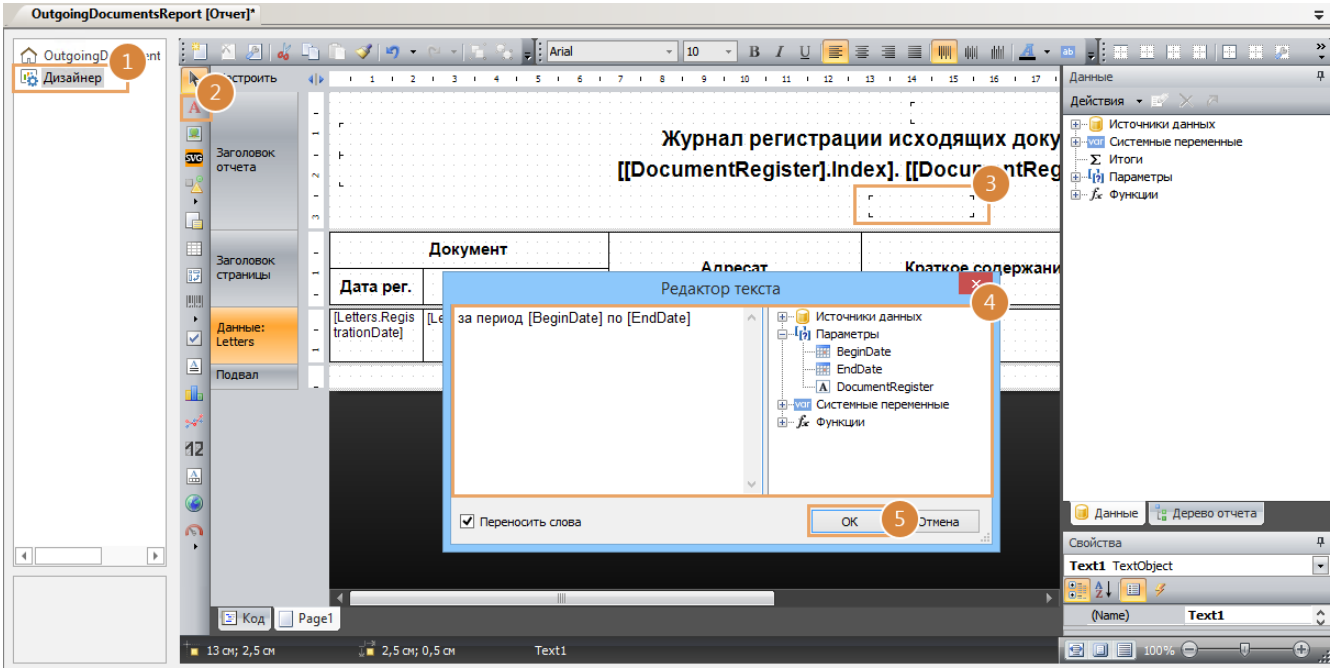
Редактор текста вызывается при двойном щелчке левой кнопкой мыши по объектам «Текст» и «Таблица». Редактор имеет вид:



Аналогично редактору выражения, в левой части находится поле для задания текста и выражений, а в правой – элементы, которые используются в выражениях. Флажок **Переносить слова** предназначен для разрешения переноса текста по словам, если он не помещается в объект одной строкой.

Предположим, разрабатывается отчет, в котором содержится информация о входящих документах, зарегистрированных в определенном журнале регистрации в некоторый период. Необходимо добавить в заголовок отчета период, за который был сформирован отчет. Для этого:

1. В редакторе отчета перейдите в узел «Дизайнер».



2. Добавьте объект «Текст» на бэнд «Заголовок отчета» с помощью кнопки **Текст** на панели объектов.
3. Щелкните два раза левой кнопкой мыши по добавленному объекту «Текст».
4. В редакторе текста добавьте текст **за период с [BeginDate] по [EndDate]**, где BeginDate и EndDate – параметры, соответствующие датам начала и завершения периода формирования отчета. Для этого воспользуйтесь одним из способов:
   - переместите параметры **BeginDate** и **EndDate** из правой части в левую часть редактора с помощью механизма Drag&Drop. Допишите остальной текст;
   - в правой части редактора дважды щелкните левой кнопкой мыши на параметрах **BeginDate** и **EndDate**. Параметры будут добавлены в левую часть редактора в текущую позицию курсора. Допишите остальной текст;
   - вручную добавьте текст и выражения в левую часть окна редактора.
5. Нажмите на кнопку **OK**.

В результате выполнения отчета в заголовке указывается период, за который формируется отчет:

13.08.2015

Журнал регистрации входящих документов
ВХ. Входящие Прибор-Консалт
за период с 01.06.2015 по 01.08.2015

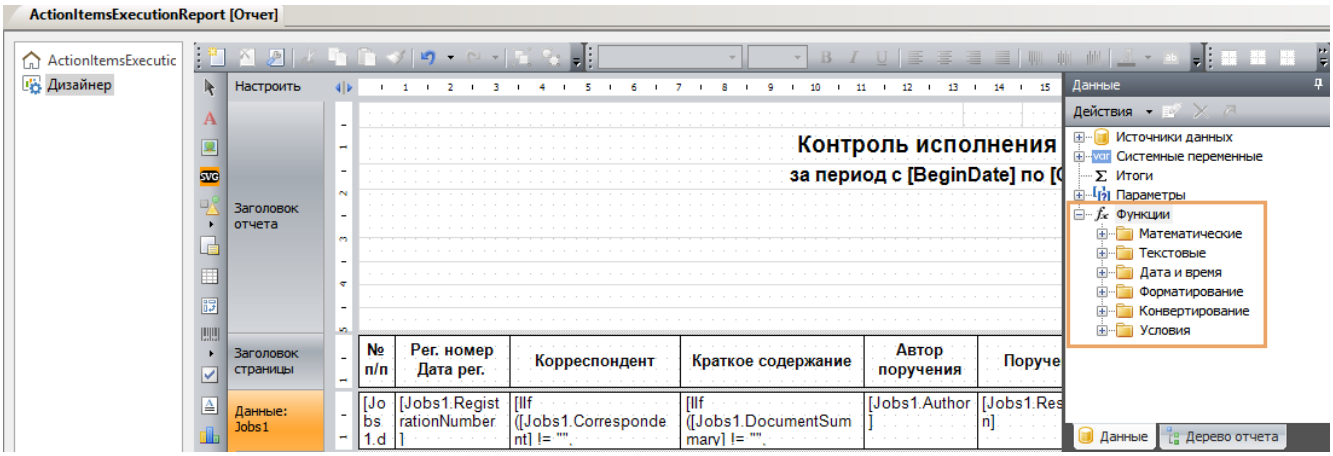
Документ		Корреспондент, дата и номер док.	Краткое содержание	Поручение	Кому направлен
Дата рег.	Рег. номер				
07.07.2015	ВХ-П-010/15	Дрофа ООО 07.07.2015 № 15-78/846	Предложение о сотрудничестве	Новиков А.Д. Передать в работу Ардо. Срок - 10.07	Ардо Н.А.
				Ардо Н.А. Подготовьте ответное письмо	Кузнецов М.С.
				Кузнецов М.С. Подготовьте ответное письмо	Мелтахова З.Р.
13.07.2015	ВХ-П-012/15	Администрация г. Новокузнецка 06.07.2015 № 0212-06/78-04	О результатах рассмотрения обращения	Новиков А.Д.	Законов С.Ю.

Использование функций

Функции позволяют выполнять ряд предопределенных команд с данными отчета. Например, сконвертировать значения поля источника данных в другой формат.

Функции вызываются в выражениях. Для вызова необходимо указать имя и параметры функции. Параметры указываются в круглых скобках и разделяются запятыми. Например, функция для отображения некоторого значения в виде процента записывается как: **FormatPercent(0.15)**.

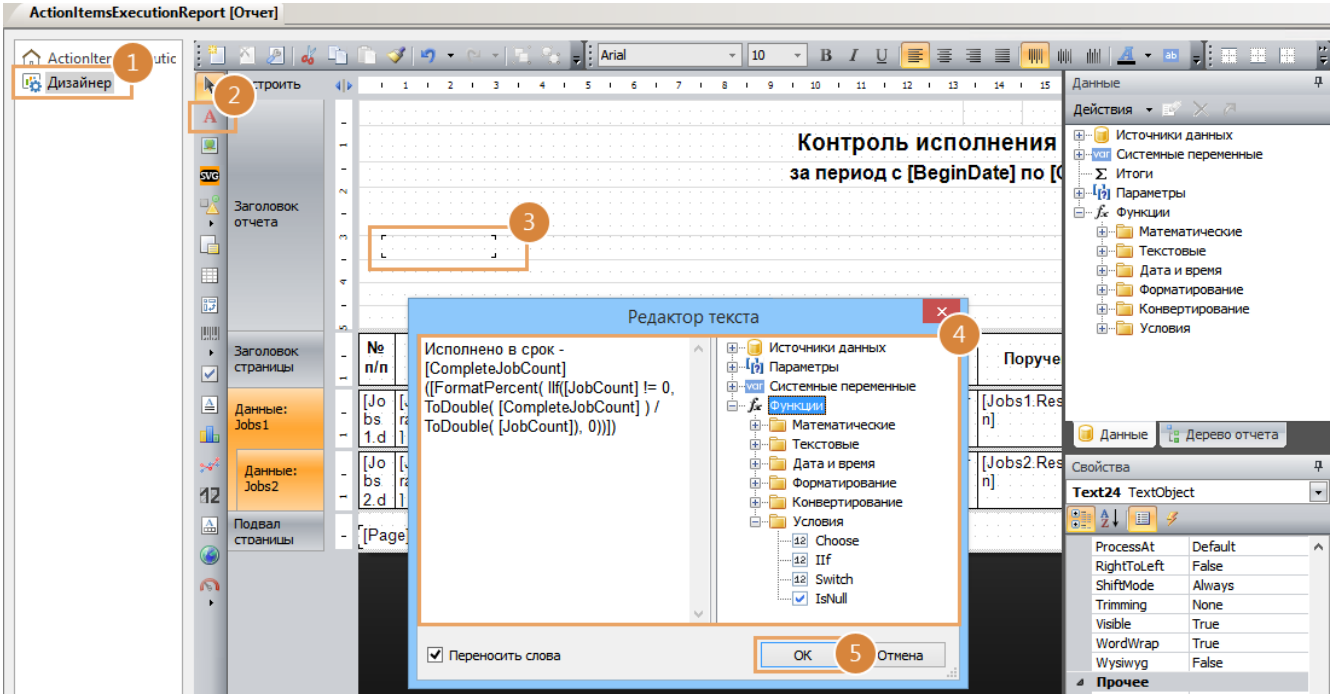
Функции доступны в окне «Данные»:



Подробнее описание функций см. в документации FastReport.Net статью «Функции».

Предположим, разрабатывается отчет, в котором содержится информация об исполнении поручений за определенный период. Необходимо, чтобы в отчете отображалось количество и процент исполненных в срок поручений в формате: Исполнено в срок – <количество исполненных в срок поручений> (<процент от общего количества поручений>). Для этого:

1. В редакторе отчета перейдите в узел «Дизайнер».



2. Добавьте объект «Текст» на бэнд «Заголовок отчета» с помощью кнопки **Текст** на панели объектов.
3. Щелкните два раза левой кнопкой мыши по добавленному объекту «Текст».
4. В открывшемся редакторе текста укажите выражение для вычисления количества и процента поручений, исполненных в срок:

   Исполнено в срок – [CompleteJobCount] ([FormatPercent(IIf([JobCount] != 0, ToDouble([CompleteJobCount]) / ToDouble([JobCount]), 0))])

   В выражении используются:

   - CompleteJobCount – параметр отчета, соответствующий количеству исполненных в срок поручений;
   - JobCount – параметр отчета, соответствующий общему количеству поручений;
   - FormatPercent() – функция для отображения значения в виде процентов. Подробнее описание функции см. в документации FastReport.Net статью FormatPercent;
   - IIf() – условная функция. Подробнее описание функции см. в документации FastReport.Net статью IIF;
   - ToDouble() – функция конвертирования значения в тип double. Подробнее описание функции см. в документации FastReport.Net статью ToDouble.

В результате при выполнении отчета указывается количество и процент поручений, исполненных в срок:

15.07.2015

Контроль исполнения поручений
за период с 10.05.2015 по 10.06.2015

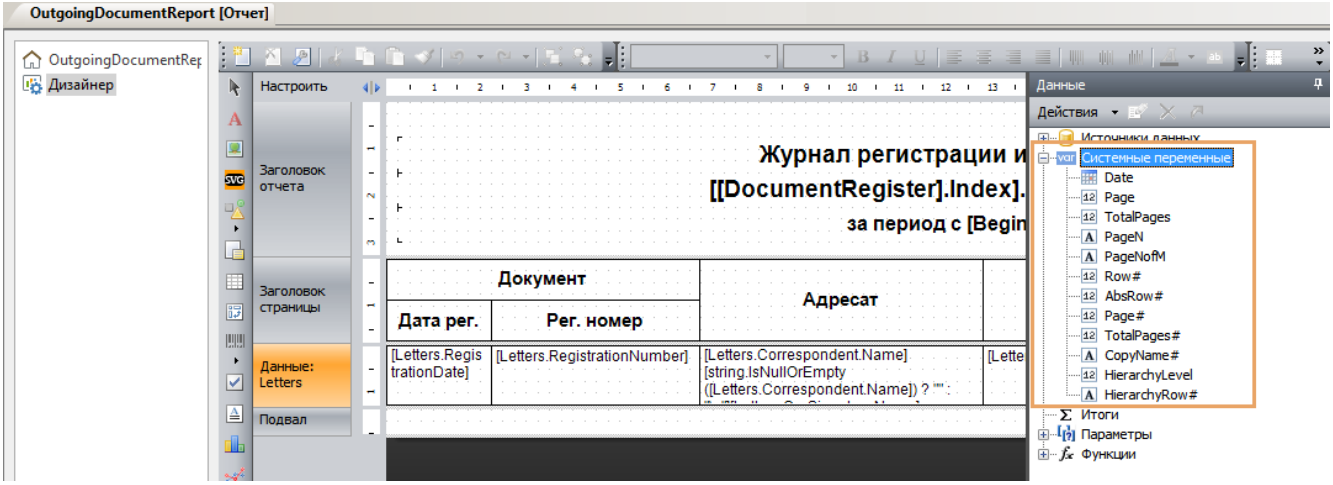
Количество поручений - 10
Исполнено в срок - 7 (70,00%)
Исполнено с нарушением срока - 1 (10,00%)
Не исполнено - 2 (20,00%)
Уровень исполнительской дисциплины - 70,00%

№ п/п	Рег. номер Дата рег.	Корреспондент	Краткое содержание	Автор поручения	Поручение	План. дата	Факт. дата	Исполнитель	Отчет	
1	ВХ-П-005/15 09.05.2015	Роспотребнадзор по Клинскому, Солнечногорсому районах Моск. об-ти	О проведении плановой проверки	Новиков А. Д.	Проверить документацию.	16.05.2015	10.05.2015	Законов С. Ю.	Проверено *Исполнено*	
2	-	-	-	Новиков А. Д.	Подготовьте план продаж на 2 полугодие	06.06.2015		Ардо Н. А.	*Не исполнено*	Не исп.
3	-	-	-	Новиков А. Д.	Подготовьте проект договора для М.А.Д. "Инжиниринг"	09.06.2015	10.06.2015 *Нарушен срок*	Кузнецов М.С.	Проект договора готов. *Исполнено*	
4	ВХ-П-010/15 07.06.2015	Дрофа ООО	Предложение о сотрудничестве	Новиков А. Д.	Подготовьте ответное письмо	10.06.2015	08.06.2015	Кузнецов М.С.	Подготовлен документ, см. во вложении *Исполнено*	

Добавление системных переменных

Системные переменные используются для отображения информации об отчете. Например, для указания даты и времени выполнения отчета, номера текущей страницы или общего количества страниц.

Системные переменные доступны в окне «Данные»:



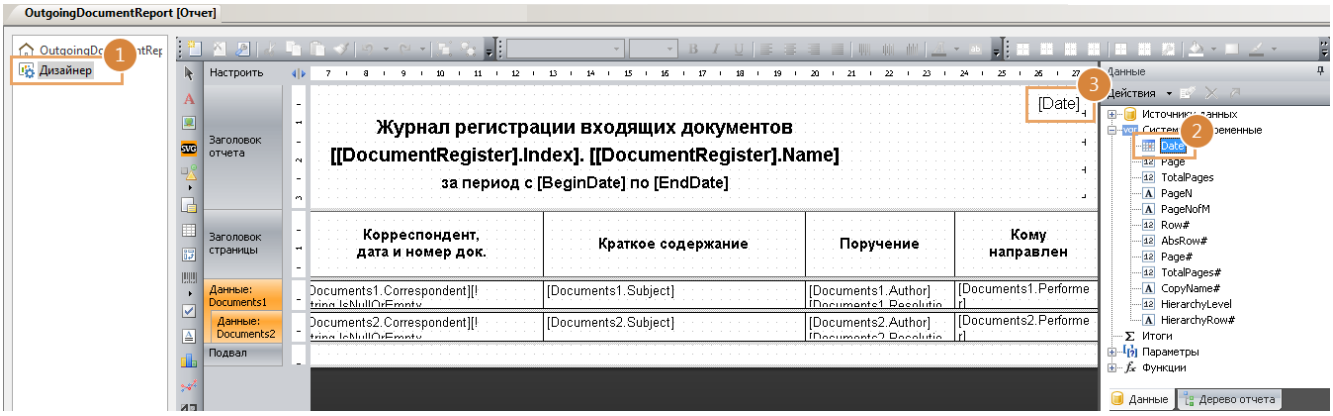
Переменная	Описание	Тип
Date	Дата и время выполнения отчета	Дата
Page	Номер текущей страницы	Число
TotalPages	Общее количество страниц в отчете	Число
PageN	Номер страницы в формате «Страница N»	Строка
PageNofM	Номер страницы в формате «Страница N из M»	Строка

Переменная	Описание	Тип
Row#	Номер строки данных внутри группы. Значение переменной обнуляется в начале новой группы	Число
AbsRow#	Номер строки данных. Значение переменной не обнуляется в начале новой группы	Число
CopyName#	Номер текущей копии отчета	Строка
HierarchyLevel	Текущий уровень в иерархическом отчете. Верхний уровень имеет значение 1. Подробнее см. в документации FastReport, статью «Печать иерархии»	Число
HierarchyRow#	Полный номер строки в иерархическом отчете. Имеет вид, например «1.2.1»	Строка

Каждая переменная имеет определенный тип данных, от этого зависит, как ее можно использовать в выражении. Например, переменная **Date** имеет тип **Дата** (DateTime). Чтобы переменная возвращала только год, обратитесь к свойству **Year**: [Date].Year. Аналогично можно получить текущий месяц: [Date].Month.

Предположим, необходимо, чтобы в заголовке отчета указывалась дата формирования отчета. Для этого:

1. В редакторе отчета перейдите в узел «Дизайнер».



2. На панели «Данные» в узле **Системные переменные** выберите переменную **Date**.
3. Переместите переменную **Date** на бэнд «Заголовок отчета» с помощью механизма Drag&Drop.

В результате выполнения отчета в заголовке указывается текущая дата:

13.08.2015

Журнал регистрации входящих документов
ВХ. Входящие Прибор-Консалт
за период с 01.06.2015 по 01.08.2015

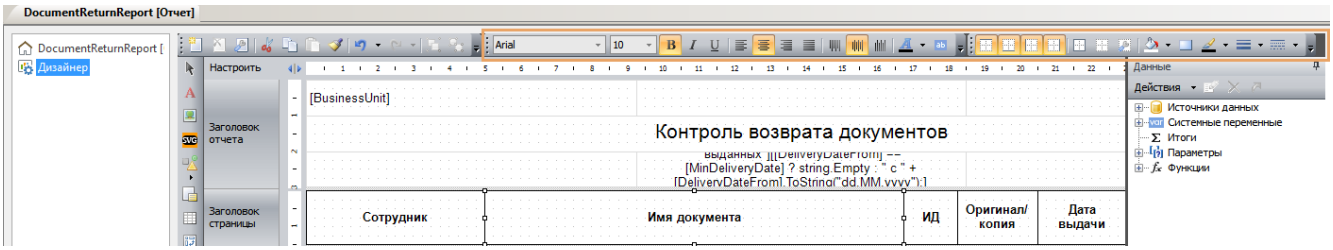
Документ		Корреспондент, дата и номер док.	Краткое содержание	Поручение	Кому направлен
Дата рег.	Рег. номер				
07.07.2015	ВХ-П-010/15	Дрофа ООО 07.07.2015 № 15-78/846	Предложение о сотрудничестве	Новиков А.Д. Передать в работу Ардо. Срок - 10.07	Ардо Н.А.
				Ардо Н.А. Подготовьте ответное письмо	Кузнецов М.С.
				Кузнецов М.С. Подготовьте ответное письмо	Мелтахова З.Р.
13.07.2015	ВХ-П-012/15	Администрация г. Новокузнецка 06.07.2015 № 0212-06/78-04	О результатах рассмотрения обращения	Новиков А.Д.	Законов С.Ю.

Форматирование

Форматирование позволяет изменить стиль бэндов и объектов отчета, например:

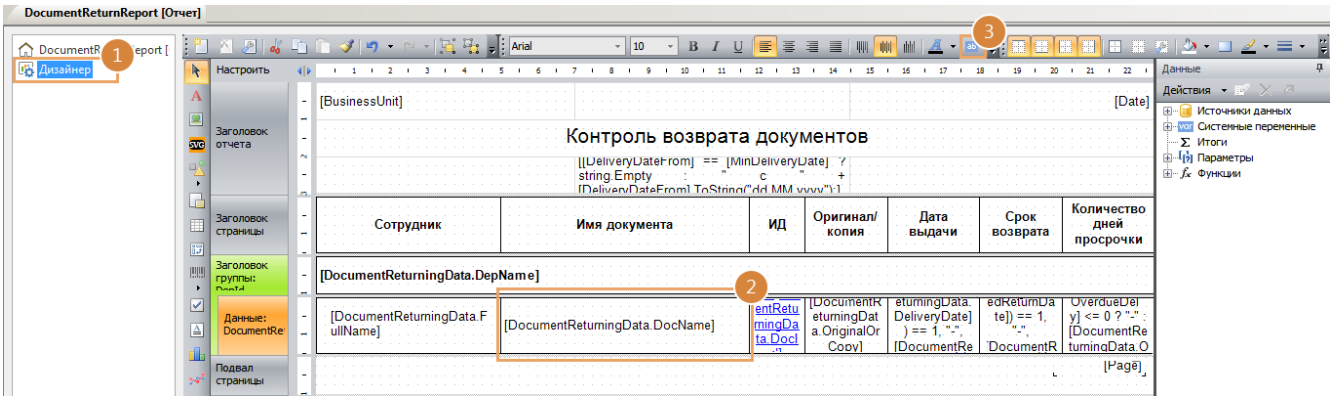
- изменить формат текста и отображаемых данных отчета;
- настроить рамки и заливку бэндов и объектов;
- автоматически изменять стиль отображения объекта при выполнении какого-либо условия.

Для форматирования объектов отчета используются действия на панели инструментов в узле «Дизайнер»:

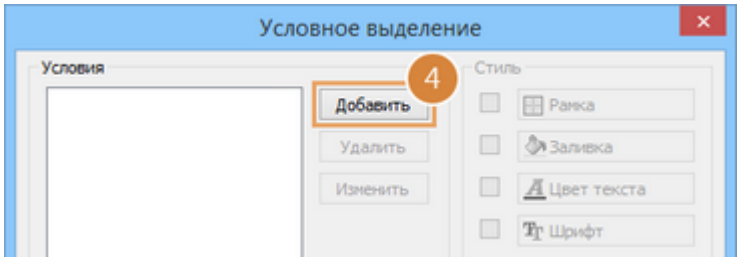


Для объектов «Текст» предусмотрена возможность изменения стиля отображения объекта при выполнении некоторых условий. Предположим, разрабатывается отчет, содержащий информацию о документах, которые были выданы сотрудникам, но еще не возвращены. Необходимо, чтобы не возвращенные в срок документы отображались в отчете красным цветом. Для этого:

1. В редакторе отчета перейдите в узел «Дизайнер».

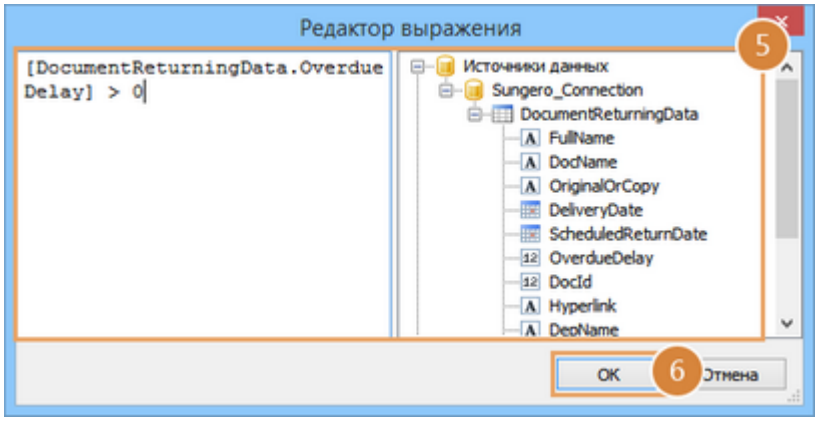


2. На бэнде «Данные» выделите объект «Текст», который должен отображаться красным цветом, если документ не возвращен в срок.
3. Нажмите кнопку **Условное выделение** на панели инструментов.
4. В редакторе условного выделения добавьте условие, при котором будет изменяться стиль текста. Для этого нажмите на кнопку **Добавить**.

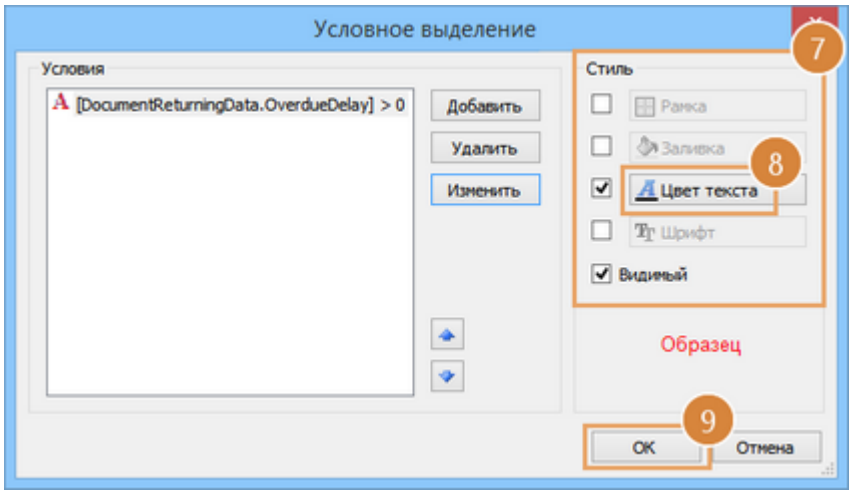


ВАЖНО. Для объекта можно добавлять несколько условий. В это случае проверяется выполнение условий, начиная с первого. Если какое-то условие выполнится, стиль применится к объекту и проверка условий завершится.

5. В открывшемся редакторе выражения введите текст условия. Условием может быть любое выражение, которое возвращает логический результат. Добавьте выражение **[DocumentReturningData.OverdueDelay] > 0** (значение поля источника данных Количество дней просрочки больше 0).



6. Нажмите на кнопку **OK**.



7. В редакторе условного выделения в группе полей «Стиль» укажите параметры стиля текста, которые нужно изменять при выполнении условия. Для этого установите флажки рядом с необходимыми параметрами:
   - **Рамка**;
   - **Заливка**;
   - **Цвет текста**. Флажок установлен по умолчанию;
   - **Шрифт**;
   - **Видимый**. Флажок установлен по умолчанию.
8. Чтобы при выполнении указанного условия цвет текста выделенных объектов становился красным, нажмите на кнопку **Цвет текста** и в выпадающем списке выберите нужный цвет.
9. Нажмите на кнопку **OK**.
10. Аналогично настройте условное выделение для всех объектов «Текст», которые должны отображаться красным цветом, если документ не возвращен в срок.

ПРИМЕЧАНИЕ. Чтобы объединить несколько ячеек по условию, например, чтобы при одном значении было три разных ячейки, а при другом – одна большая, попробуйте использовать свойство RowValues. Подробнее см. примеры в справке FastReport: «Условие выделения» и «Заполнение матрицы вручную».

В результате в отчете документы с просроченным сроком возврата будут отображаться красным цветом:

ЗАО "ТехноСистемы" 06.12.2014

Контроль возврата документов

выданных с 01.11.2014 по 01.12.2014

Сотрудник	Имя документа	ИД	Оригинал/ копия	Дата выдачи	Срок возврата	Количество дней просрочки
Финансово-юридический отдел						
Законов С. Ю.	Договор №ТС/14/015/пр от 28.11.2014 с ООО "Метпромгрупп" "Оценка применимости приборного контроля"	632	Оригинал	28.11.2014	01.12.2014	5
Законов С. Ю.	Договор №А/2014/123/ТС/14/1 от 21.11.2014 с ООО "Завод легковых автомобилей" "Аренда производственных площадей"	610	Оригинал	21.11.2014	28.11.2014	-
Законов С. Ю.	Договор №ТС/14/013/пр от 27.11.2014 с М.А.Д. "Инжиниринг" "Поставка контрольно-измерительной аппаратуры"	625	Оригинал	27.11.2014	-	-
Законов С. Ю.	Договор №ТС/14/014/пр от 28.11.2014 с ООО "Завод легковых автомобилей" "Аппаратура контроля дефектов"	627	Оригинал	28.11.2014	-	-
Законов С. Ю.	Договор №ТС/14/015/пр от 28.11.2014 с ООО "Метпромгрупп" "Оценка применимости приборного контроля"	632	Оригинал	28.11.2014	-	-

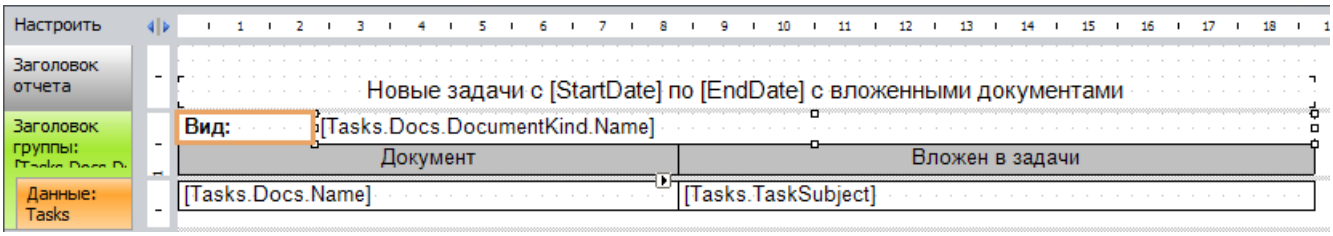
Как убрать объединение столбцов в отчете Excel

После разработки отчета и его экспорте в формат Excel некоторые данные могут занимать два столбца вместо одного. Чтобы исправить это, нужно в дизайнере выровнять ширину объектов, которые находятся над объединенными ячейками. Например, отчет в Excel выглядит следующим образом:

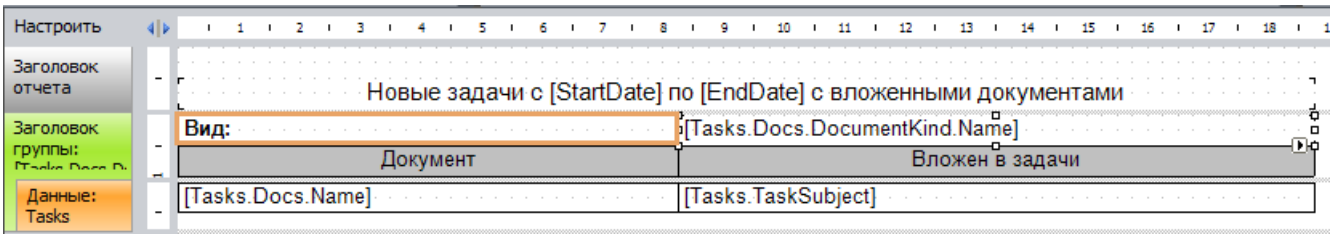
	A	B	C
1	Новые задачи с 19.03.20 по 15.09.20 с вложенными документами		
2	**Вид:**	Договор	
3	Документ		Вложен в задачи
4	Договор с Автомир "На проектирование системы противопожарной безопасности"		Согласование: Договор с Автомир "На проектирование системы противопожарной безопасности"
5			Согласование: Договор с Автомир "На проектирование системы противопожарной безопасности"
6	Договор с Автомир "О сотрудничество в сфере развития персонала"		Согласование: Договор с Автомир "О сотрудничество в сфере развития персонала"
7			Согласование: Договор с Автомир "О сотрудничество в сфере развития персонала"

В дизайнере отчета выровняйте ширину полей **Вид** и **[Tasks.Docs.DocumentKind.Name]**, а именно сделайте ширину поля **Вид** такой же, как у поля **Документы**:

Было:



Стало:



В результате отчет будет выглядеть так:

	A	B
1	Новые задачи с 19.03.20 по 15.09.20 с вложенными документами	
2	**Вид:**	Договор
3	Документ	Вложен в задачи
4	Договор с Автомир "На проектирование системы противопожарной безопасности"	Согласование: Договор с Автомир "На проектирование системы противопожарной безопасности"
5		Согласование: Договор с Автомир "На проектирование системы противопожарной безопасности"
6	Договор с Автомир "О сотрудничество в сфере развития персонала"	Согласование: Договор с Автомир "О сотрудничество в сфере развития персонала"
7		Согласование: Договор с Автомир "О сотрудничество в сфере развития персонала"

События отчета

Обработчики событий отчета выполняются в клиентском приложении и/или на веб-сервере. Существуют следующие события отчетов:

Клиентское событие «До выполнения» (BeforeExecute)

Код обработчика выполняется в клиентском приложении в начале выполнения отчета.

Назначение:

- запрос параметров отчета;
- расчет параметров на основании запроса пользователя или других параметров отчета;
- проверка корректности ввода параметров отчета.

Аргументы события:

- <Имя отчета> – выполняемый отчет. Используется для обращения к параметрам;

- e.Cancel – признак прекращения выполнения отчета после окончания работы обработчика события. Возможные значения:
    - **True**. Выполнение отчета прекратится. Например, используется, когда пользователь нажимает на кнопку **Отмена** в окне запроса параметров;
    - **False**. Выполнение отчета будет продолжено.

Серверное событие «До выполнения» (BeforeExecute)

Код обработчика выполняется на веб-сервере в начале выполнения отчета.

Назначение:

- расчет параметров отчета;
- проверка корректности параметров отчета;
- вычисление дополнительных данных, необходимых для выполнения отчета. Например, создание временных таблиц.

Аргументы события:

- <Имя отчета> – выполняемый отчет. Используется для обращения к параметрам.

Серверное событие «После выполнения» (AfterExecute)

Код обработчика выполняется на веб-сервере после выполнения отчета.

Назначение. Очистка всех дополнительных данных, созданных при выполнении отчета. Например, удаление временных таблиц.

Аргументы события:

- <Имя отчета> – выполненный отчет. Используется для обращения к параметрам;
- e.IsSuccessful – признак успешного выполнения отчета. Возможные значения:
    - **True**. Выполнение отчета прошло успешно;
    - **False**. Во время выполнения отчета возникли ошибки.

Порядок выполнения отчета

Отчет выполняется:

- при вызове методов Open() или ExportToFile(string fileName) в обработчиках клиентских событий и клиентских функциях;
- при вызове методов Export() или ExportTo(IStreamImporter streamImporter) в обработчиках серверных событий и серверных функциях;
- из списка отчетов, который открывается с помощью метода ShowAll().

Порядок выполнения отчета:

Копирование отчета

Чтобы скопировать отчет:

1. Выделите отчет в дереве решений и в контекстном меню выберите пункт **Копировать**.
2. В контекстном меню текущего или другого модуля выберите пункт **Вставить**. В результате создается копия отчета.

   ПРИМЕЧАНИЕ. Отчет типа сущности можно копировать в тип сущности или модуль. Отчет модуля нельзя копировать в тип сущности, потому что у отчета модуля не задан параметр **Entity**, обязательный для отчета типа сущности.
3. Доработайте скопированный отчет. Если в коде используются типы сущностей, которых нет в данном модуле, скорректируйте вычисления.
4. Чтобы пользователи могли запускать отчет, настройте его отображение в модуле и добавьте действие на обложку.

Рекомендации по разработке отчетов в Linux

Среда разработки устанавливается на компьютеры разработчиков с операционной системой из семейства Microsoft Windows. Соответственно разработчик тестирует разработанные отчеты в *локальной системе* на Windows. При этом отчеты в операционной системе на базе Linux могут выглядеть иначе. Возможные различия:

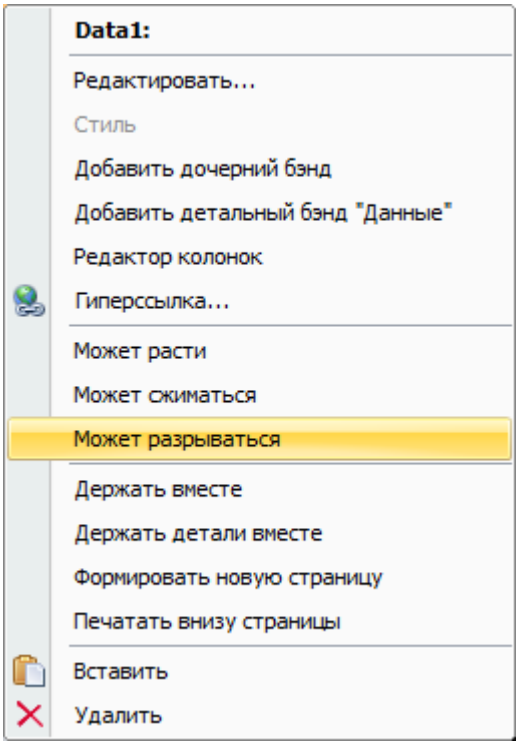
- **некорректно отображаются шрифты**.

Решение для Linux: установите на компьютер с продуктивным веб-сервером Directum RX шрифты, которые используются в организации на клиентских компьютерах пользователей и шрифты Microsoft, если их использование разрешено правилами компании.

ПРИМЕЧАНИЕ. Чтобы установить шрифты Microsoft в операционной системе Linux на основе Debian, например Ubuntu, используйте пакет основных Microsoft TrueType шрифтов. На официальном сайте https://packages.debian.org/ru/sid/ttf-mscorefonts-installer перейдите на страницу загрузки пакета и скачайте файл **ttf-mscorefonts-installer_3.8_all.deb**;

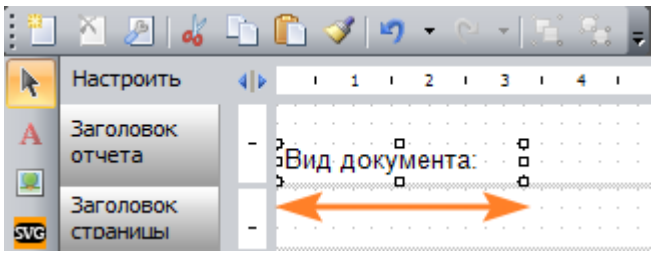
- **лишние пустые строки в таблицах**.

  Решение для Linux: в контекстном меню всех *бэндов*, на которых есть данные, снимите флажок **Может разрываться**:



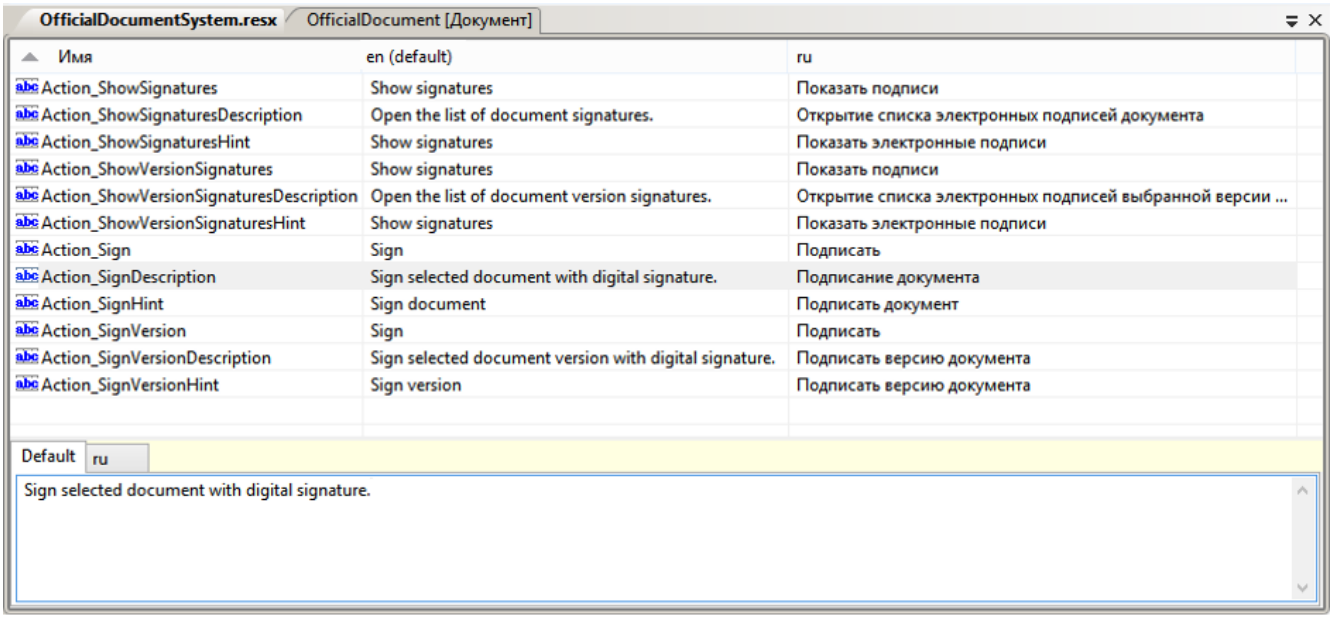
- **текст в ячейках таблицы некорректно переносится, обрезается или выходит за границы**.

  Решение для Linux: установите ширину колонок отчета с запасом примерно в 1 символ.



Ресурсы

Для *локализации* пользовательского интерфейса используются редакторы прикладных и системных ресурсов. Редакторы выглядят одинаково:



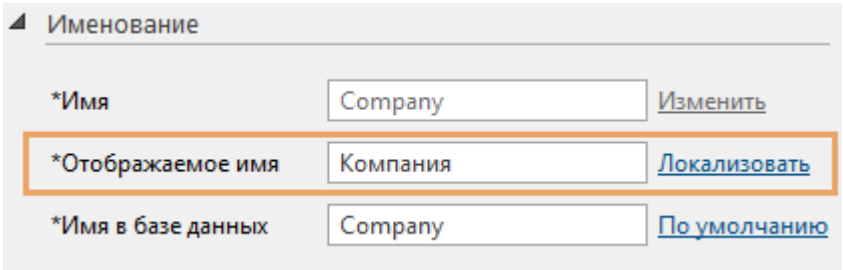
В столбце **Имя** содержатся имена *строк локализации*, которые используются в программном коде. Они имеют тип LocalizedString (локализованная строка). На закладках **Default** и **ru** задаются значения строк локализации, которые отображаются в соответствующих столбцах.

При необходимости состав поддерживаемых языков локализации можно изменить.

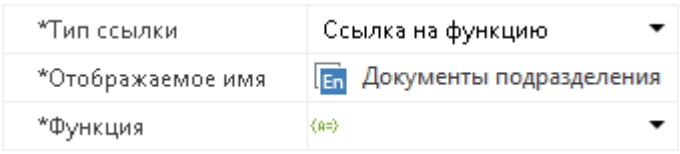
ВАЖНО. В системе не локализуются данные, добавленные пользователями. Например, названия организаций, фамилии и имена работников будут отображаться на том языке, на котором их создали пользователи.

Редактор системных ресурсов

Редактор используется для локализации названий модулей, *типов сущностей*, групп контролов, действий на ленте и т.д. Чтобы добавить ресурс, перейдите по ссылке **Локализовать** в редакторе элемента разработки:

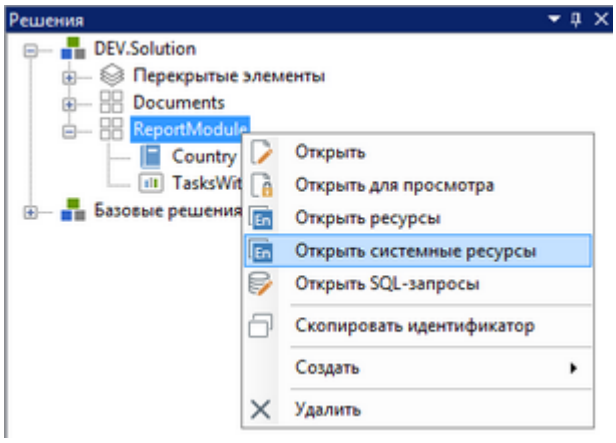


Или нажмите на кнопку **Локализовать** на панели свойств в редакторе форм типа сущности:



Откроется окно для редактирования ресурса. Подробнее см. раздел «Пример локализации действия на ленте».

Чтобы посмотреть список всех системных ресурсов элемента разработки, выделите его в дереве решений и в контекстном меню выберите пункт **Открыть системные ресурсы**:



Откроется редактор системных ресурсов, в котором:

- на закладках **Default** и **ru** можно изменить значение ресурса;
- с помощью контекстного меню можно скопировать ресурс или его имя.

ВАЖНО. Удаление и добавление ресурсов в редакторе недоступно.

Редактор прикладных ресурсов

Редактор используется для локализации диалоговых окон и сообщений *валидации*.

Добавить прикладной ресурс можно:

- прямо при написании программного кода. Подробнее см. «Пример локализации сообщения валидации»;
- в редакторе ресурсов. Чтобы его открыть, в дереве решений в контекстном меню решения, модуля, отчета или типа сущности выберите пункт **Открыть ресурсы**. Чтобы добавить ресурс, в контекстном меню табличной части выберите пункт **Добавить строку**.

Имена прикладных ресурсов должны быть уникальными в рамках элемента разработки. Они могут состоять из букв латинского алфавита, цифр и символа «_». Первым символом обязательно должна быть буква.

Формат обращения к ресурсу в программном коде зависит от того, для какого элемента разработки он создан:

Элемент разработки	Формат
Решение	<Код компании>.<Имя решения>.Resources.<Имя ресурса>

Элемент разработки	Формат
Модуль	<Код компании>.<Имя модуля>.Resources.<Имя ресурса>
Тип сущности	<Код компании>.<Имя модуля>.<Имя типа сущности во мн. ч>.Resources.<Имя ресурса>
Отчет	<Код компании>.<Имя модуля>.Reports.Resources.<Имя отчета>.<Имя ресурса>

Если в редакторе ресурсов удалить или изменить имя существующего ресурса, в программном коде значение нужно поправить вручную.

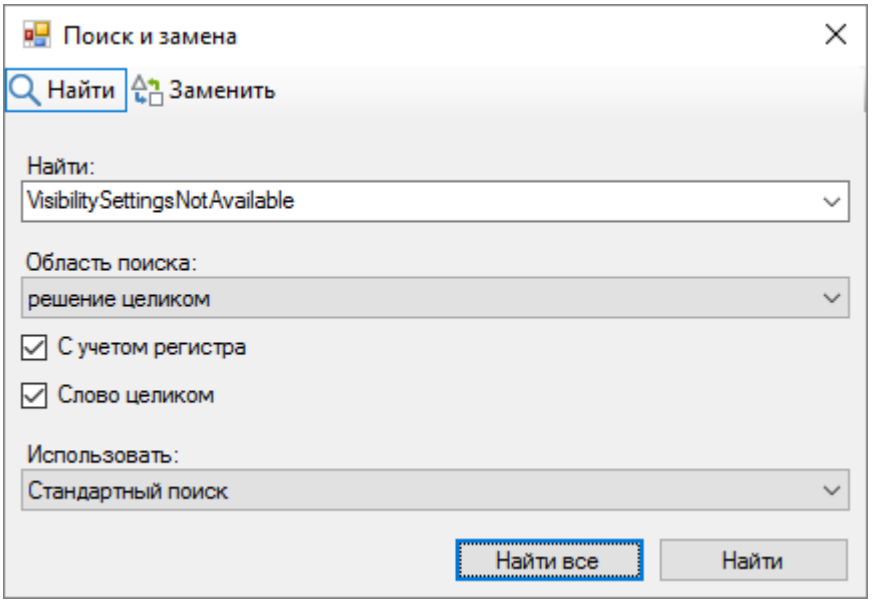
ПРИМЕЧАНИЕ. В редакторе прикладных ресурсов вы можете добавлять иконки для контрола состояния.

Поиск по ресурсам

Поиск по ресурсам позволяет найти все места в коде, где используется строка локализации. Например, это позволяет проще заменить ее на другую. Поиск выполняется по всем доступным локализациям, по умолчанию – в строках **En** и **Ru**.

Чтобы запустить поиск:

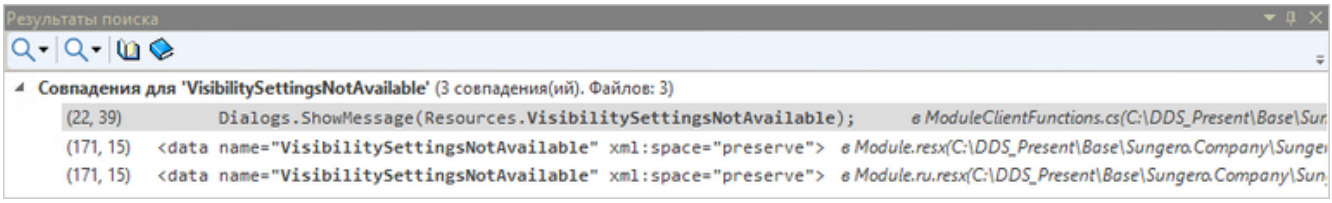
1. Нажмите на клавиши CTRL+F или CTRL+SHIFT+F. Откроется вкладка «Найти» окна поиска:



2. В поле **Найти** укажите искомую строку локализации или имя ресурса.
3. Укажите **Область поиска**. Если есть открытые редакторы или у вас выделен интересующий фрагмент кода, то в раскрывающемся списке выберите один из пунктов: **текущий документ**, **выделенный фрагмент**, **все открытые документы**. Если нет открытых редакторов или неизвестно, где задана строка, то выберите пункт **решение целиком**.
4. Если нужно найти строки по фрагменту, то снимите флажок **Слово целиком**. Например, это можно использовать для поиска строк с определенным префиксом.
5. В раскрывающемся списке **Использовать** выберите пункт **Стандартный поиск**.
6. Нажмите на кнопку **Найти все**, если нужно отобразить сразу все результаты поиска.

Если нужно приостанавливать поиск после каждого найденного результата, нажмите на кнопку **Найти**. При повторном нажатии на кнопку поиск продолжается.

Чтобы перейти к коду или к редактору ресурсов, где используется строка, в результатах поиска два раза кликните по найденному совпадению:

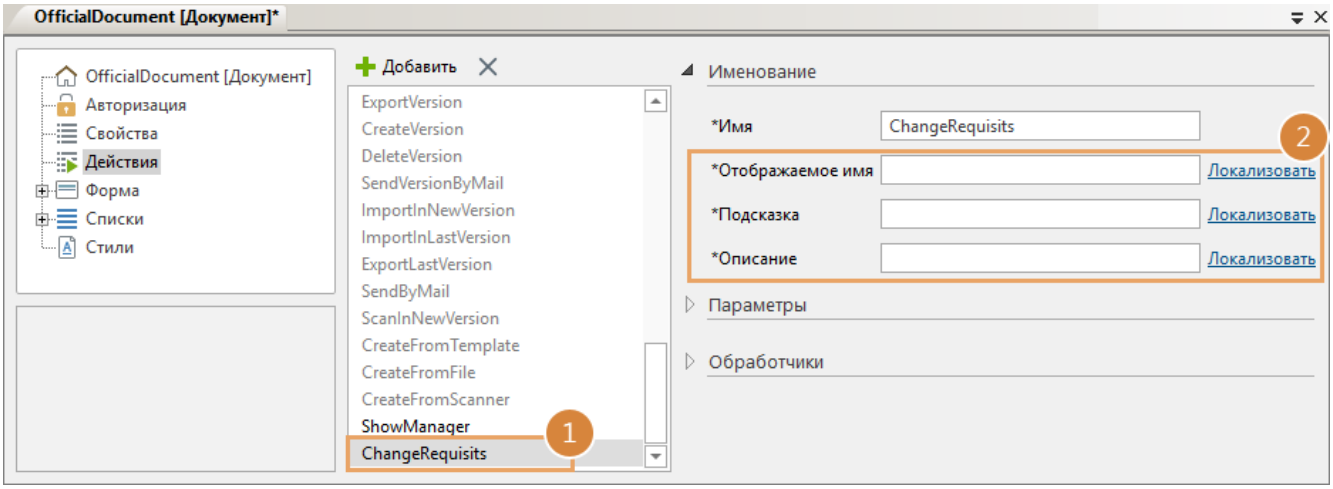


С помощью вкладки «Заменить» можно заменить текст только в коде. Если выбрана замена и найдено значение в ресурсах, то выводится предупреждение. Для замены текста в строках локализации используйте редакторы системных и прикладных ресурсов.

Пример локализации действия на ленте

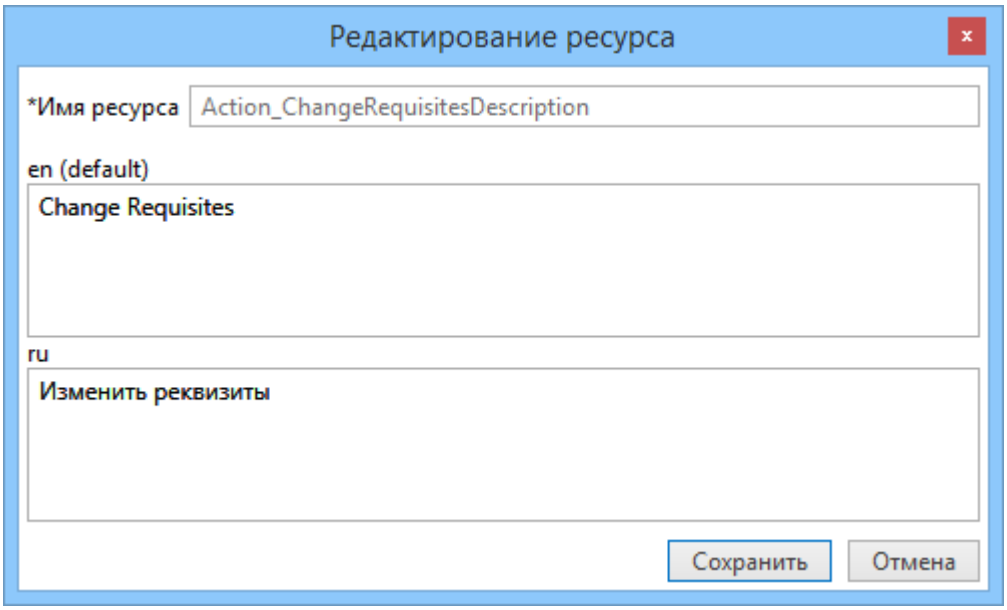
Предположим, отображаемое имя, подсказку и описание действия для официального документа необходимо указать на русском и английском языках. Для этого:

1. В редакторе типа документа в узле «Действия» выделите нужное действие.

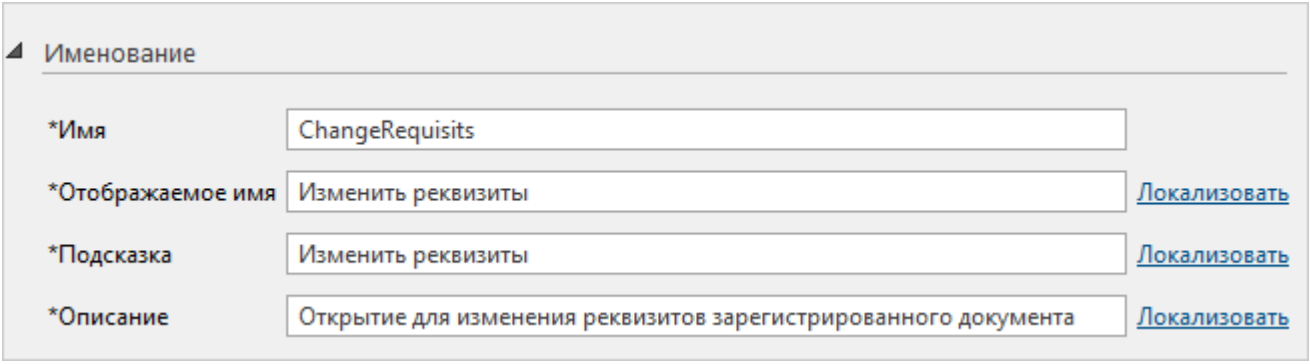


2. Для локализации имени, описания и подсказки перейдите по ссылке **Локализовать**.

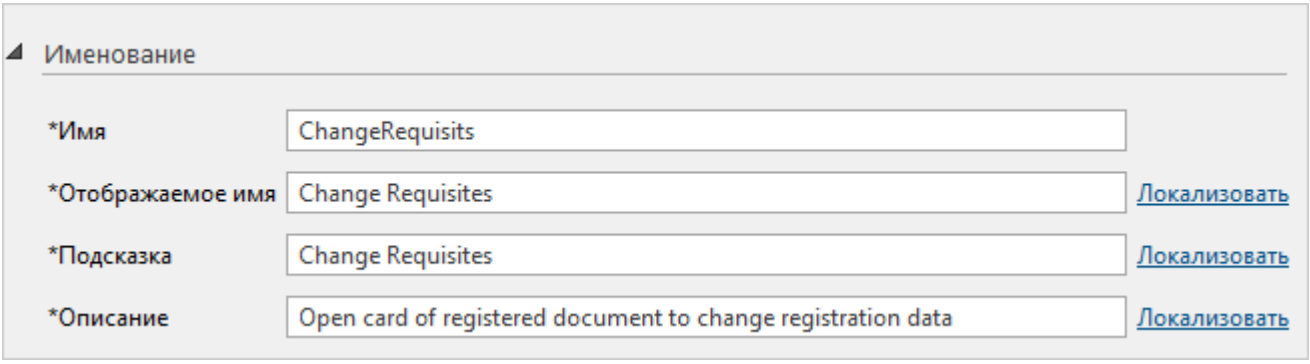
3. В открывшемся окне заполните поля **en (default)**, **ru** и нажмите на кнопку **Сохранить**. Имя ресурса генерируется автоматически и доступно только для просмотра. Например:



ВАЖНО. В среде разработки в полях ***Отображаемое имя**, ***Подсказка**, ***Описание** и т.п. отображаются значения, которые введены в поле **en (default)**. В справке на рисунках для наглядности и понятности используются русские значения:

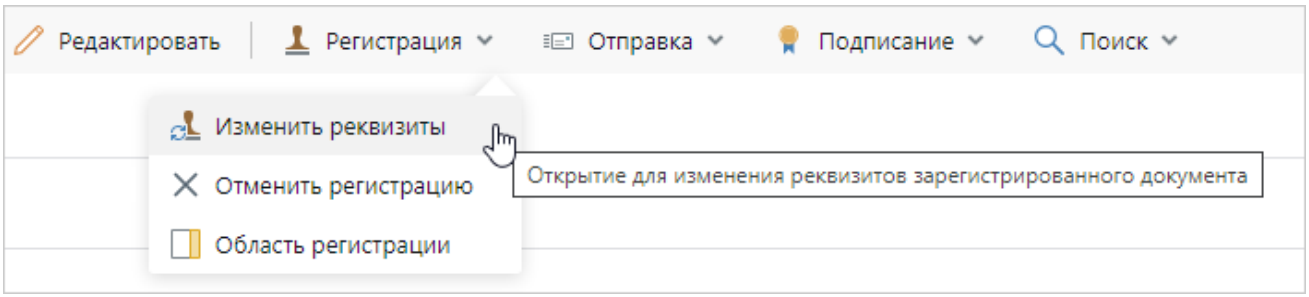


У вас при правильном заполнении полей будут отображаться английские значения:

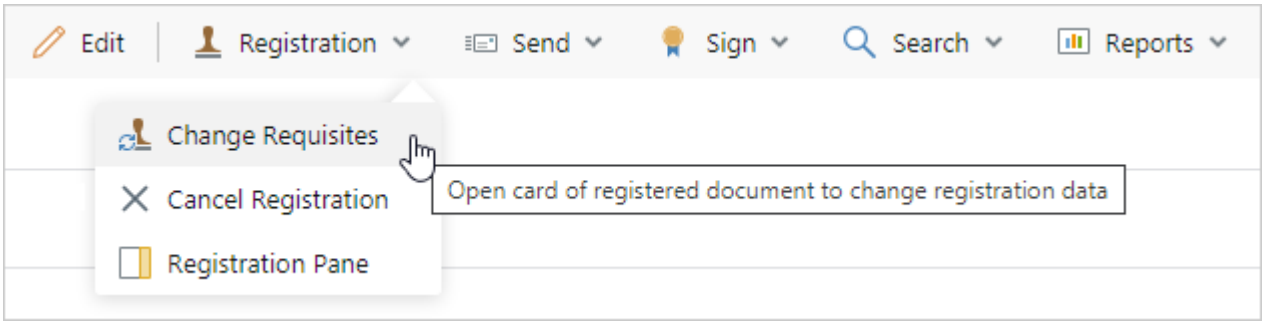


В результате имя действия, подсказка и описание будут отображаться в проводнике системы следующим образом:

Отображение на русском языке



Отображение на английском языке



ПРИМЕЧАНИЕ. Описание и отображаемое имя действия также можно локализовать на уровне пользовательских настроек без применения среды разработки. Для этого используйте редактор форм в справочнике **Представления форм**. Подробнее см. в руководстве администратора, разделы «Настройка форм карточек» и «Локализация полей».

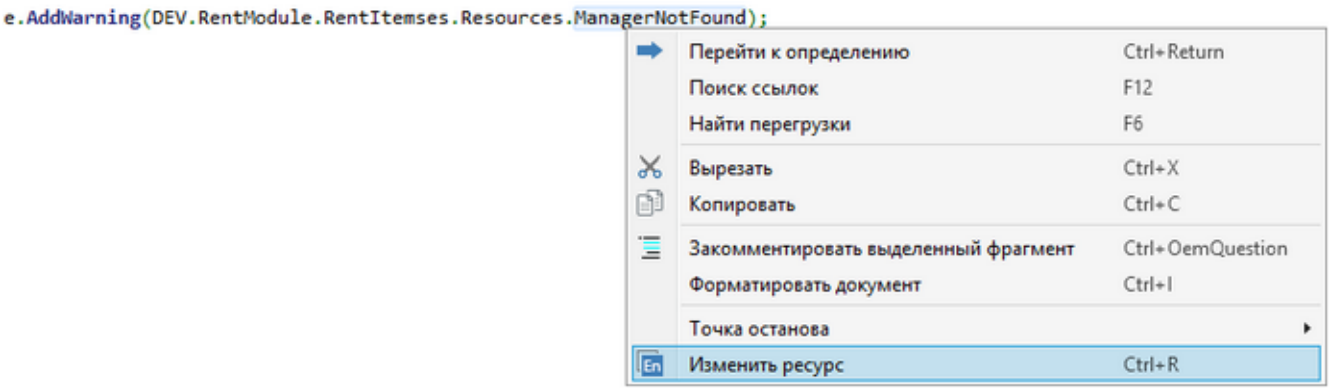
Пример локализации сообщения валидации

Предположим, для действия написан программный код, в котором проверяется, заполнено ли в карточке справочника поле «Руководитель». Если поле не заполнено, пользователю отображается сообщение валидации. Чтобы сразу добавить сообщение в *ресурсы* и локализовать его:

1. В программном коде установите курсор на сообщении валидации.
2. Нажмите сочетание клавиш CTRL+R или воспользуйтесь пунктом контекстного меню **Создать ресурс**.
3. В открывшемся окне заполните поля ***Имя ресурса**, ***en (default)**, ***ru** и нажмите на кнопку **Сохранить**.

В результате в программном коде текст сообщения заменится на полный путь до ресурса. В редакторе ресурсов элемента разработки, для которого писали программный код, появится строка с ресурсом. Добавленный ресурс будет иметь тип LocalizedString (локализованная строка).

Если вы хотите посмотреть или изменить значения ресурса, установите курсор на имени ресурса и нажмите сочетание клавиш CTRL+R или воспользуйтесь пунктом контекстного меню **Изменить ресурс**:



В открывшемся окне можно посмотреть значения ресурса и при необходимости поправить их.

Примеры изменения локализации в перекрытых элементах разработки

В перекрытых элементах разработки можно изменять значения ресурсов, которые унаследованы от элементов *базового решения*.

Изменение локализации имени для группы контролов

ПРИМЕЧАНИЕ. Отображаемое имя для группы контролов также можно локализовать на уровне пользовательских настроек без применения среды разработки. Для этого используйте редактор форм в справочнике **Представления форм**. Подробнее см. в руководстве администратора, разделы «Настройка форм карточек» и «Локализация полей».

На форме базового типа документа «Договор» есть группа контролов с отображаемым именем «Основное» (Main). Чтобы изменить локализацию отображаемого имени группы:

1. Перекройте тип документа «Договор» (Sungero.Contracts.Contract). В редакторе вашего решения добавьте зависимость от модуля Sungero.Contracts.
2. В дереве перекрытого типа документа в узле «Форма» выделите группу контролов Main. На панели свойств нажмите на кнопку **Локализовать** рядом с полем ***Отображаемое имя**. Откроется редактор ресурсов.
3. На закладках **Default** и **ru** измените прежние значения строки локализации на новые.
4. Сохраните изменения.

После публикации на форме появится новое отображаемое имя группы.

Изменение локализации сообщения валидации

В справочнике **Регион**, в серверном событии До сохранения есть программный код для проверки на уникальность кода региона. Если код не уникален, то пользователю выводится сообщение валидации. Имя ресурса в справочнике базового решения Directum RX – Regions.Resources.CodeDuplicate. Чтобы изменить локализацию сообщения:

1. Перекройте справочник **Регион** (Sungero.Commons.Region). В редакторе вашего решения добавьте зависимость от модуля Sungero.Commons.
2. В редакторе справочника **Регион** базового решения Directum RX откройте программный код серверного события **До сохранения:**

```
public override void BeforeSave(Sungero.Domain.BeforeSaveEventArgs e)
  {
    // Проверить код региона на уникальность.
    if (!string.IsNullOrWhiteSpace(_obj.Code))
    {
      var region = Regions.GetAll().FirstOrDefault(r => r.Code == _obj.Code &&

r.Country.Equals(_obj.Country) &&
                                                    r.Status !=
CoreEntities.DatabookEntry.Status.Closed);
      if (region != null && !region.Equals(_obj))
        e.AddWarning(_obj.Info.Properties.Code,
Regions.Resources.CodeDuplicate));
  }
```

3. Скопируйте код и вставьте в серверное событие **До сохранения** перекрытого справочника **Регион** вместо кода **base.BeforeSave(e)**.
4. Вместо пути обращения к ресурсу напишите желаемый текст сообщения. Например:

Было	Стало
`e.AddWarning (_obj.Info.Properties.Code, Regions.Resources.CodeDuplicate))`	`e.AddWarning(_obj.Info.Properties.Code, "Код уже существует"))`

5. С помощью горячих клавиш CTRL+R или контекстного меню **Создать ресурс** добавьте новое локализованное сообщение валидации в редактор ресурсов.

После публикации, если пользователь введет неверное значение кода региона, отобразится новое сообщение *валидации*.

Рекомендации по разработке для масштабируемых систем

В главе содержатся приемы проектирования и разработки для систем с большими объемами данных и количеством пользователей. Рекомендации помогут оптимизировать длительные процессы, обеспечить быстродействие операций, повысить скорость отклика системы и др.

Быстродействие операций

Индексы и прикладные кэши

Отключение проверки прав доступа

Передача в клиентское приложение отфильтрованных данных

Получение большого объема данных с помощью типа IQueryable

Оптимизация получения связанных документов

Оптимизация построения отчетов

Обращения к веб-серверу

Объединение небольших серверных вычислений

«Жадная» загрузка свойств

Проверка блокировки сущности в действиях

Кэширование действий с помощью параметра Params

Проверка фильтров в списках и папках

Оптимизация длительных процессов

Использование фоновых процессов или асинхронных обработчиков

Транзакции и сессии с большим количеством объектов

Запуск большого количества длительных асинхронных обработчиков

Ограничение количества ролей для пользователя

Назначение большого количества прав на типы документов

Назначение прав группе пользователей (экземплярные права)

Массовое удаление документов

Массовое удаление субъектов прав

Оптимизация работы с сущностями

Блокировка сущности перед ее изменением

Вызов асинхронного обработчика при сохранении карточки

Разработка карточек с большой табличной частью

Оптимизация события «Возможность выполнения» (CanExecute) для дочерних коллекций

Работа с большим количеством свойств типа сущности

Большое количество подписей на документе

Ускорение массового подписания документов

Оптимизация выполнения блока «Мониторинг»

Версионирование функций инициализации

Сервис интеграции

Ограничение количества объектов для передачи

Запросы на выполнение длительных операций

Быстродействие операций

Зачем измерять

Чтобы определиться, достаточно ли быстро выполняются операции в системе для комфортной работы пользователя, проанализируйте скорость выполнения операций.

Нужно учитывать, что запросы на веб-сервер автоматически прекращаются по тайм-ауту через 5 минут. При этом тайм-аут не учитывается:

- если в прикладном коде нет обращения к базе данных;
- если открыто модальное окно или диалог. В этом случае отсчет тайм-аута возобновляется после закрытия модального окна или диалога.

ПРИМЕЧАНИЕ. Длительные запросы рекомендуется передавать на выполнение сервису асинхронных событий. Если предполагается, что запрос занимает длительное время и его выполнение нельзя передать сервису асинхронных событий, то тайм-аут для него можно отключить.

Рекомендуемое время на выполнение операций – не более 3 секунд, для запуска отчета – не более 14 секунд.

Как измерять

Скорость операции в прикладном коде можно узнать тремя способами:

- проанализировать лог-файлы;
- выполнить трассировку операций с помощью специальных утилит. Например, для трассировки времени выполнения действий на веб-сервере клиенту можно использовать профилировщик dotTrace, для трассировки операций с СУБД – встроенные инструменты для анализа работы (профилировщики);
- использовать возможности технического решения «Мониторинг Directum RX».

Как ускорять

При необходимости ускорьте время выполнения операций, чтобы повысить удобство работы пользователя с системой. Для этого:

- используйте индексы и прикладные кэши;
- отключайте проверку прав доступа;
- передавайте в клиентское приложение отфильтрованные данные;
- оптимизируйте получение большого объема данных с помощью типа IQueryable, в том числе получение связанных документов;
- оптимизируйте построение отчетов.

Индексы

Если на быстродействие влияют тяжелые запросы к СУБД, вы можете ускорить доступ к данным. Для получения выборки данных из больших списков, например, состоящих из более 10 млн записей, рекомендуется формировать запрос в два этапа: событие **Предварительная фильтрация** (PreFiltering) отфильтрует данные по базовым критериям, событие **Фильтрация** (Filtering) – по остальным. Запрос выполняется в два этапа только на веб-сервере и для баз данных на основе PostgreSQL.

Если этого недостаточно для ускорения получения данных, то можно оптимизировать процесс с помощью индексов. Чаще всего их применяют для виджетов, отчетов, папок, в том числе вычисляемых.

Для быстрого доступа к данным:

1. Добавьте SQL-запрос для создания индекса. Например:

```
CREATE NONCLUSTERED INDEX {1} ON {0}
   (
     [Discriminator],
     [Performer],
     [Author],
     [MainTask],
     [CompletedBy],
     [Created]
   )
```

2. Создайте индекс при инициализации модуля. Например:

```
public static void CreateAssignmentIndices();
   {
     var tableName =
   Sungero.Docflow.Constants.Module.SungeroWFAssignmentTableName;
     var indexName =
   "idx_Asg_Discriminator_Performer_Author_MTask_ComplBy_Created";
     var indexQuery =
   string.Format(Queries.Module.SungeroWFAssignmentIndex0Query, tableName,
   indexName);

     Functions.Module.CreateIndexOnTable(tableName, indexName, indexQuery);
   }
```

Общие рекомендации по построению индекса:

- не используйте индексы для поиска по небольшим таблицам. Поиск в этом случае не будет оптимальным;
- по возможности используйте уникальные индексы, так как уникальность значений в столбце повышает производительность индекса;
- в составном индексе учитывайте порядок столбцов. Указывайте первыми столбцы, которые используются в запросах с **WHERE**. Следующие нужно перечислять с учетом уникальности их значений, то есть столбцы с самыми уникальными данными указываются в первую очередь;
- имя индекса задавайте длиной не более 63 символа. Это нужно для поддержки работы СУБД на основе PostgreSQL;
- в имени индекса обозначьте таблицу, для которой создается индекс, и основные поля, по которым строится индекс.

Прикладные кэши

Чтобы не тратить время на пересчет данных, воспользуйтесь результатами прошлого расчета, актуализируйте их и сохраните для последующего обращения. Данные, рассчитанные в предопределенной структуре прикладного кода, хранятся в отдельной SQL-таблице **Sungero_System_Cache**. Эта таблица представляет собой прикладной кэш. Используйте его для ускорения работы с данными. Например, в виджете для быстрого получения заданий подразделения с использованием кэша ранее вычисленных заданий:

```
   /// <summary>
/// Получить задания подразделения из кэша.
/// </summary>
/// <param name="department">Подразделение.</param>
/// <returns>Задания подразделения.</returns>
public static Shell.Structures.Module. GetCachedAssignment(IDepartment department)
{
  var key = string.Format(Shell.Constants.Module.DepartmentAssignmentsCacheKey,
department.Id);
  Shell.Structures.Module.EmployeeAssignmentsCache cachedAssignmentCompletion;
  if (Cache.TryGetValue(key, out cachedAssignmentCompletion))
   cachedAssignmentCompletion =
UpdateEmployeeAssignmentsCache(cachedAssignmentCompletion,
Sungero.Shell.Constants.Module.DepartmentCache);
  if (cachedAssignmentCompletion.IsChanged)
   Cache.AddOrUpdate(key, cachedAssignmentCompletion, Calendar.Today.AddDays(90));
  return cachedAssignmentCompletion;
}
```

Отключение проверки прав доступа

При получении данных с сервера выполняется проверка прав доступа. Как правило, она занимает время и при этом иногда является избыточной.

Например, при старте задачи на согласование документа выполняется поиск заданий в работе, в которые вложен этот документ. Если задания есть, то появляется предупреждение о том, что документ уже на согласовании. В такой ситуации при поиске сначала проверяются права доступа на вложения. Если задачу стартует руководитель, то проверяются еще права всех сотрудников, которых он замещает. В результате в базу данных отправляется сложный запрос, создается нагрузка на систему, и задача стартует медленно.

В этом примере проверка прав избыточна, так как пользователь напрямую не обращается к заданиям, в которые вложен документ. В системе только проверяется, можно ли отправить на согласование документ.

Чтобы упростить запрос к базе данных и ускорить выполнение операции, в такой ситуации можно отключить проверку прав. Для этого код на получение данных «оборачивается» в метод AllowRead().

ПРИМЕЧАНИЕ. Метод **AllowRead()** позволяет программно извлекать информацию, на которую у пользователя нет прав доступа. Используйте метод, только если у пользователя есть полномочия на просмотр полученной информации. Например, если он может просматривать статистику в виджетах или сумму по всем входящим счетам.

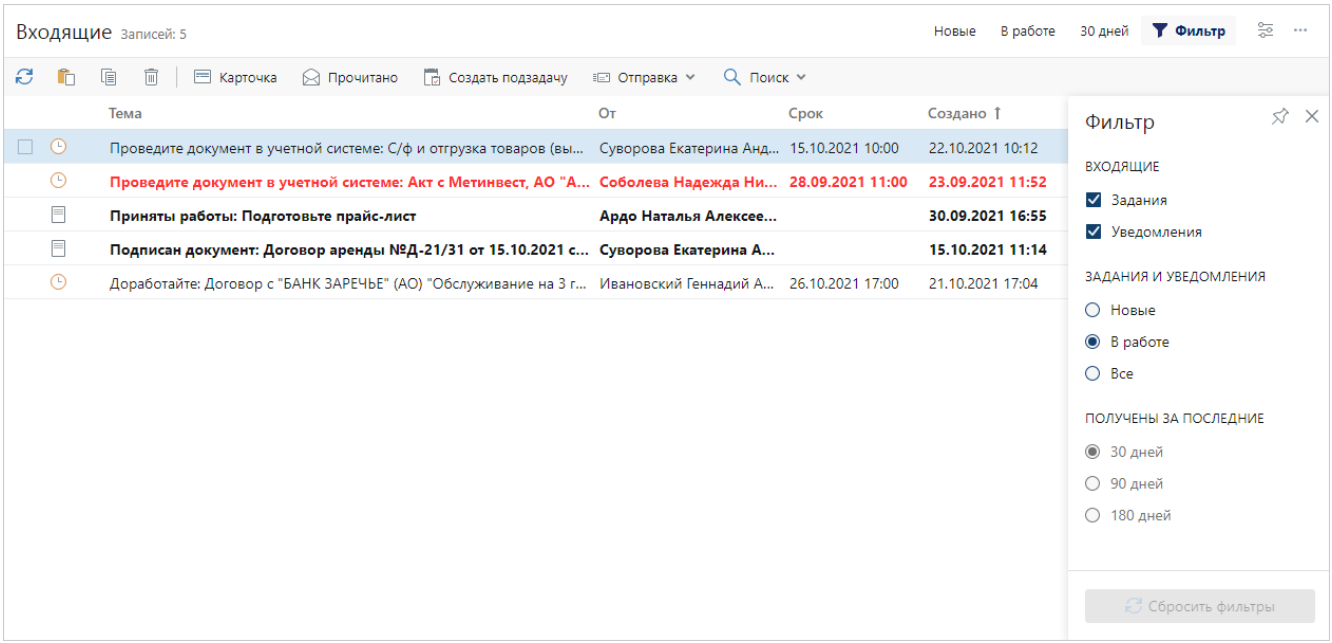
Пример

Проверить, созданы ли по документу задания на обработку. Для ускорения поиска заданий отключить проверку прав доступа.

```
/// <param name="task">Текущая задача.</param>
/// <param name="document">Документ.</param>
/// <returns>True, если по документу уже созданы задания на обработку за рамками
/// текущей задачи.</returns>
[Remote(IsPure = true)]
public virtual bool HasOtherDocumentProcessingAssignments(ITask task,
IOfficialDocument document)
{
  var hasDocumentProcessingAssignments = false;

  Sungero.Core.AccessRights.AllowRead(
    () =>
    {
      hasDocumentProcessingAssignments =
this.GetOtherDocumentProcessingAssignments(task, document).Any();
    });

  return hasDocumentProcessingAssignments;
}
```

Параметры фильтрации

Иногда количество данных в списке может быть слишком большим для вычисления и отображения. Чтобы ускорить отображение списка, можно отобрать актуальные данные, например за неделю, и на клиент передавать уже отфильтрованную информацию. Это позволит избежать лишней нагрузки на сервер. Например, такой фильтр реализован в вычисляемых папках:



Получение большого объема данных

На быстродействие может негативно влиять включение большого объема данных в список **ToList()**. При написании кода нужно помнить, что при работе с потенциально большими данными лучше использовать параметризованный тип IQueryable (запрос на получение данных). Например, IQueryable<IContractualDocument>. Тогда данные с сервера подгружаются порциями, и можно работать с ними, не ожидая полной загрузки с сервера. **ToList()** рекомендуется применять для работы с небольшим списком сущностей, который предварительно отфильтрован.

Если запрос возвращает больше 1000 записей, информация о нем по умолчанию записывается в лог-файлы веб-сервера. С помощью этого разработчик может найти неоптимальные запросы в прикладном коде и доработать их. Также время обработки запросов и количество возвращенных ими записей можно отслеживать с помощью решения «Мониторинг Directum RX».

Кроме того, не рекомендуется без необходимости получать сущность целиком, если нужны только некоторые ее свойства. Вместо этого используйте ограниченную выборку. Таким образом SQL-запросы становятся проще и выполняются быстрее.

Пример оптимального кода для запроса одного параметра

```
GetRelatedDocuments().Select(d => d.SomeProperty));
```

Пример оптимального кода для запроса нескольких параметров

```
   // Получить информацию обо всех связанных документах.
var relatedInfo =
electronicDocument.Relations.GetRelatedAndRelatedFromDocuments().Select(d => new
DocumentInfo() { Id = d.Id, Name = d.Name }).ToList();
```

Получение связанных документов

Производительность системы зависит в том числе от скорости обработки связанных документов. Чтобы этот процесс не занимал длительное время, программный код должен быть оптимальным. Для этого проверьте, что в коде не используются устаревшие методы **GetRelated()** и **GetRelatedFrom()**.

Кроме этого, определите, нагружают ли систему запросы получения связей. Для этого в решении «Мониторинг системы Directum RX» можно проанализировать длительность HTTP и SQL-запросов при обращении к базе данных и отследить, какие запросы к веб-серверу в прикладном коде нагружают систему.

Чтобы оптимизировать программный код, в котором используются методы для получения связанных документов, соблюдайте рекомендации из этого раздела.

Используйте методы GetRelatedDocuments() и GetRelatedFromDocuments()

Чтобы ускорить обработку связанных документов, вместо устаревших методов **GetRelated()** и **GetRelatedFrom()** с типом возвращаемого значения IEnumerable<T> используйте методы GetRelatedDocuments() и GetRelatedFromDocuments(). Они возвращают значение с типом IQueryable<T>.

Пример кода без оптимизации

```
.GetRelated(relationType).Where(l => OfficialDocuments.As(l)?.LeadingDocument
   != document);
```

Пример оптимального кода

```
.GetRelatedDocuments(relationType).Where(l => OfficialDocuments.Is(l) &&
   OfficialDocuments.As(l).LeadingDocument != document);
```

Некоторые запросы с методами **GetRelatedDocuments()** и **GetRelatedFromDocuments()** не поддерживаются. Например, если в запросе используется конструкция, которую нельзя преобразовать в SQL. В этом случае после вызова методов добавьте расширения **ToList()** или **ToArray()**. Такой код будет работать без оптимизации.

Пример кода, который не преобразуется в SQL

```
someDocument.GetRelated().Where(d => SomeFunction(d));
```

Пример доработки кода с использованием ToList()

```
someDocument.GetRelatedDocuments().ToList().Where(d => SomeFunction(d));
```

Используйте метод GetRelatedAndRelatedFromDocuments()

Вместо совместного вызова методов **GetRelatedDocuments()** и **GetRelatedFromDocuments()** используйте метод GetRelatedAndRelatedFromDocuments().

Пример кода без оптимизации

```
    document
```

```
    .Relations
    .GetRelated().Union(document.Relations.GetRelatedFrom()).Distinct();
```

Пример оптимального кода

```
document.Relations.GetRelatedAndRelatedFromDocuments();
```

Используйте репозиторий связей DocumentRelations

Если нужны данные о связях документа, а не сами документы, то используйте репозиторий связей **DocumentRelations**.

Пример кода без оптимизации

```
var docsWithRelations = ElectronicDocuments.GetAll().ToList().Where(d =>
    d.Relations.GetRelated().Any()).ToList();
```

Пример оптимального кода

```
var docsWithRelations = ElectronicDocuments.GetAll().Where(d =>
    DocumentRelations.GetAll().Any(r => r.Source.Equals(d)).ToList();
```

В циклах и повторяемых выражениях не используйте методы с одинаковыми параметрами и для одних и тех же документов

В циклах и повторяемых выражениях не используйте методы **GetRelatedDocuments()** и **GetRelatedFromDocuments()** с одинаковыми параметрами для одних и тех же документов. Иначе метод получает связанные документы при каждом вызове. Вместо этого рекомендуется получать список связанных документов только один раз, а затем обращаться к нему.

Сохраняйте документы до создания связей с ними

Созданные связи, которых еще нет в базе данных, учитываются при вызове методов **GetRelatedDocuments()** и **GetRelatedFromDocuments()**. Для этого в них замыкаются запросы на получение данных. Чтобы этого не происходило, методы нужно использовать после сохранения связей. В результате запросы не замыкаются, и их можно дофильтровать в прикладном коде.

Используйте метод Select(), если нужны свойства, а не вся сущность

Не рекомендуется без необходимости получать связанные документы целиком, если нужны только некоторые их свойства. Вместо этого используйте ограниченную выборку. Таким образом SQL-запросы к сущностям становятся проще и выполняются быстрее, при этом можно избежать сканирования таблиц. Также это справедливо не только для связей, но и для любых запросов к сущностям.

Пример оптимального кода для запроса одного параметра

```
GetRelatedDocuments().Select(d => d.SomeProperty));
```

Пример оптимального кода для запроса нескольких параметров

```
    // Получить информацию обо всех связанных документах.
var relatedInfo =
electronicDocument.Relations.GetRelatedAndRelatedFromDocuments().Select(d => new
DocumentInfo() { Id = d.Id, Name = d.Name }).ToList();
```

Оптимизация построения отчетов

Формирование отчетов на основе большого количества данных может нагружать систему и занимать длительное время.

Чтобы оптимизировать этот процесс, следуйте рекомендациям:

- если для построения отчета выполняется выборка из более чем 1000 записей, то вместо LINQ рекомендуется использовать оптимальные SQL-запросы. Иначе при работе с LINQ и циклами будет расходоваться большое количество ресурсов;
- поддерживайте индексы в актуальном состоянии и регулярно анализируйте необходимость создания новых.

Обращения к веб-серверу

Чтобы пользователь мог эффективно работать с системой, действия в ней должны выполняться мгновенно. Каждый запрос к веб-серверу влечет за собой ожидание времени отклика и дополнительно нагружает сервер. Большое количество запросов влияет на быстродействие системы, поэтому их нужно минимизировать.

Чтобы проверить количество запросов от веб-клиента к веб-серверу:

1. Войдите в веб-клиент. Выполните в нем доработанную или созданную в прикладном коде операцию. Например, заполните карточку разработанного типа документа.
2. В лог-файле веб-сервера проанализируйте количество записей, у которых в значении атрибута **lg** записано **WebApiCallSpan**. При поиске ориентируйтесь на примерное время выполнения операции.

ПРИМЕЧАНИЕ. Измерение количества запросов должно происходить на базе с достаточным количеством данных.

При разработке учитывайте рекомендуемое число запросов для операций:

- выделение записи в списке – 0 запросов;
- событие Обновление формы – 0 запросов;
- событие Показ формы – 0 запросов;
- событие Изменение значения контрола для свойства – 0 запросов;
- событие Возможность выполнения для действия – 0 запросов;
- создание из списка – 1 запрос;
- открытие карточки – 1 запрос;
- сохранение карточки или вызов действий – не более 5 запросов.

0 запросов означает, что в обработчиках событий не рекомендуется вызывать серверные функции (с атрибутом **Remote**) и использовать логику, которая будет долго обрабатываться. Перечисленные обработчики выполняются часто. Поэтому, если в них есть серверные функции, то на веб-сервер отправляется большое количество запросов, что, в свою очередь, влияет на быстродействие.

Если функции с атрибутом Remote используются в коде обработчиков событий **Возможность выполнения** (CanExecute) и **Обновление формы** (Refresh), то в среде разработки на этапе

сборки решения появляется предупреждение. Рекомендуется удалить функцию. Если решается ее оставить, то при работе пользователя с карточкой сущности в веб-клиенте результат выполнения кэшируется. Для снижения нагрузки на Directum RX функция повторно не выполняется, пока открыта карточка. Система получает результат из кэша. После обновления или закрытия карточки результат в кэше сбрасывается.

Если клиентское событие с вызовом Remote-функции обрабатывается на сервере больше минуты, информация об этом по умолчанию записывается в лог-файлы веб-сервера. С помощью этого разработчик может найти обработчики, которые замедляют работу сервера из-за частого вызова серверных функций. При этом, если выполнение запроса на веб-сервер превышает тайм-аут, то запрос автоматически прерывается через 5 минут. Также частоту вызова Remote-функций в клиентских событиях и время их обработки можно отслеживать с помощью технического решения «Мониторинг Directum RX».

Чтобы ускорить время выполнения операций:

- объединяйте небольшие серверные вычисления
- используйте «жадную» загрузку свойств
- проверяйте блокировки сущности в действиях
- кэшируйте действия с помощью параметра Params
- добавляйте проверку фильтров в списках и папках

Объединение вычислений

Если в коде есть несколько небольших серверных вычислений, то не стоит писать на каждое из них отдельную функцию. Для оптимизации скорости объедините их в одну с атрибутом **Remote**. В качестве параметров можно передавать структуру.

Для серверной функции добавьте к атрибуту **Remote** параметр **IsPure** со значением **True**: `[Remote(IsPure = true)]`, если функция:

- не изменяет данные, например, используется только для получения объекта или выборки данных;
- используется в клиентских событиях.

Использование параметра ускорит вызов серверных функций.

«Жадная» загрузка свойств

При получении списка сущностей с веб-сервера можно загружать их свойства следующими способами:

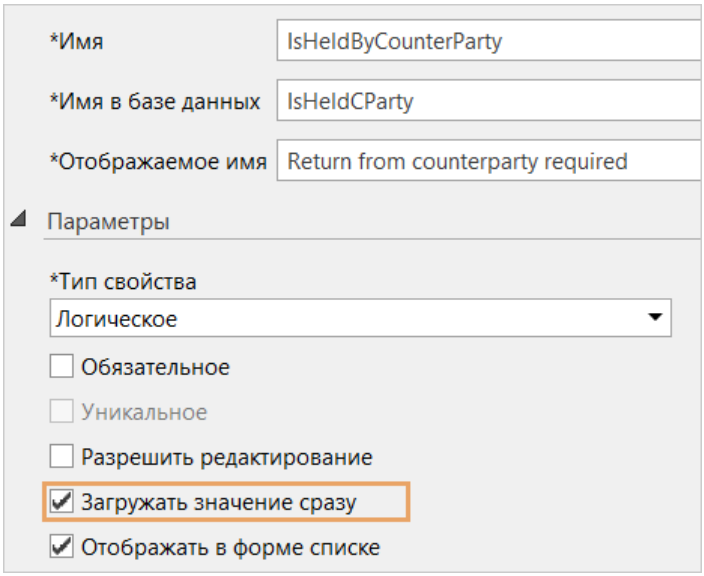
- сразу вместе с сущностями (*«жадная» загрузка*);
- отдельным обращением в каждой из сущностей к незагруженному свойству. Тогда программный код выполнится медленнее, чем в способе выше, потому что вместо одного запроса списка сущностей с нужным свойством выполнится n+1 запросов, где n – количество сущностей в списке.

Поэтому для оптимизации программного кода рекомендуем заранее предположить, какие свойства понадобятся при получении сущности, и загружать их «жадно».

Например, у официальных документов есть признак, что он находится у контрагента – **IsHeldByCounterParty**. Признак не добавлен в колонки списков, но используется в проверке доступности действия:

```
   public                                virtual                                bool
      CanReturnFromCounterparty(Sungero.Domain.Client.CanExecuteActionArgs e)
{
  return _obj.AccessRights.CanUpdate() &&
    _obj.AccessRights.CanRegister() &&
    _obj.IsHeldByCounterParty == true;
}
```

Чтобы исключить лишние запросы, для свойства установлен флажок **Загружать значение сразу** (используется «жадная» загрузка).



Проверка блокировки сущности в действиях

Действия, которые меняют сущность, доступны, только если сущность не заблокирована другим пользователем.

Рекомендуем проверять блокировку сущности в программном коде действия, а не в событии проверки доступности действия (CanExecute). Иначе отправляется лишний запрос на веб-сервер, так как обработчик CanExecute вызывается еще и при выделении записи в списке, когда информация о блокировках еще не загружена:

```
   public virtual void ReturnDocument(Sungero.Domain.Client.ExecuteActionArgs e)
{
  if (Functions.Module.IsLockedByOther(_obj, e))
    return;
  ...
}
```

Чтобы избежать рассинхронизации данных, перед изменением сущности установите блокировку.

Кэширование действий с помощью параметра Params

Чтобы уменьшить количество запросов к веб-серверу из клиентского кода, для обработчиков событий и действий типов сущностей можно сохранять результаты в параметр Params и использовать их в клиентской логике. Например, можно сохранить результат вычислений в событии обновления формы:

```
   public override void Refresh(Sungero.Presentation.FormRefreshEventArgs e)
{
      ...
      if (_obj.DocumentKinds.Any() &&
_obj.State.Properties.DocumentKinds.IsChanged)
      {
        var condition = Functions.Condition.Remote.CreateCondition();
        var possibleConditions =
Functions.ConditionBase.GetSupportedConditions(condition);

        e.Params.AddOrUpdate(Constants.ApprovalRule.IsSupportConditions,
_obj.DocumentKinds.Any(x =>
possibleConditions.ContainsKey(x.DocumentKind.DocumentType.DocumentTypeGuid)));
      }
      ...
}
```

Или использовать результат вычислений при определении возможности выполнения действия:

```
   public                                override                                bool
      CanChartAddCondition(Sungero.Domain.Client.CanExecuteActionArgs e)
{
  bool isSupportConditions;
  if
(e.Params.TryGetValue(Sungero.Docflow.Constants.ApprovalRule.IsSupportConditions,
out isSupportConditions) && !isSupportConditions)
  {
      ...
  }
}
```

ВАЖНО. При нажатии на кнопку **Отмена текущих изменений** (Cancel) значения всех параметров очищаются. Чтобы этого избежать, воспользуйтесь одним из способов:

- в событии Обновление формы (Refresh) проверьте и восстановите значения параметров с помощью методов **e.Params.Contains()** и **e.Params.AddOrUpdate()**;
- если параметр сущности был передан извне или требуются длительные вычисления параметра (например, вызов Remote-функций), то перекройте действие **Cancel** и до вызова базового обработчика (base.Cancel) сохраните значение параметра, а затем восстановите его.

Проверка фильтров в списках и папках

ВАЖНО. Проверка фильтров работает только в веб-клиенте.

При фильтрации записей на веб-сервер могут отправляться «тяжелые» запросы, которые возвращают тысячи записей и влияют на быстродействие системы. Вычислить их можно с помощью профилировщиков (встроенных инструментов анализа СУБД). У «тяжелых» запросов время CPU на сервере SQL больше 10000 мсек либо число чтения записей базы данных превышает 500000. В этом случае рекомендуется настраивать *валидацию* панели фильтрации вычисляемых папок или списков.

Валидация задается в событии **Проверка фильтра**, которое находится в редакторе вычисляемой папки и в свойствах панели фильтрации списка. В обработчике события проверьте заполнение критериев на панели фильтрации. Если при выбранных значениях запрос получается слишком «тяжелым», с помощью метода AddError() добавьте сообщение валидации. В результате «тяжелый» запрос не отправляется на веб-сервер, вместо списка пользователь увидит сообщение с просьбой уточнить критерии фильтрации. Критерии, которые нужно дозаполнить, выделяются красным цветом.

Подробнее см. «Пример настройки фильтрации записей в папке» и «Пример, как настроить валидацию панели фильтрации для входящих документов».

Оптимизация длительных процессов

Некоторые запросы клиентского приложения, которые обрабатывает веб-сервер, могут быть длительными. Например, выдача прав доступа на документы большому количеству сотрудников и подразделений. Такие длительные запросы уменьшают скорость ответа веб-сервера и увеличивают время ожидания пользователя.

Чтобы оптимизировать длительные процессы и повысить скорость отклика системы, используйте:

- асинхронные обработчики
- фоновые процессы

Оба механизма позволяют вынести вычисления с веб-сервера на отдельный сервис асинхронных событий (Worker).

Кроме этого, рекомендуется объединять в транзакции логически связанные операции в вычислениях. Например, при возникновении ошибок в фоновом процессе транзакция позволяет откатить до исходного состояния все вычисления, которые выполняются в ней. Таким образом, ошибки в транзакции не влияют на выполнение всего фонового процесса.

Если выполнение операции не может быть передано сервису асинхронных событий, то на веб-сервере она по умолчанию прерывается через 5 минут. Если в программном коде нет обращения к базе данных, в веб-клиенте открыт диалог или модальное окно, то операция выполняется без тайм-аута.

ВАЖНО. Не рекомендуется отключать тайм-аут выполнения операции, так как вместе с этим в системе выключается защита от выполнения длительных запросов и неоптимального кода. Это может привести к ошибкам и потере работоспособности.

Также систему может нагружать массовое удаление документов и субъектов прав. Поэтому рекомендуется отказаться от массового удаления в рабочее время.

Асинхронные обработчики

Предназначены для выполнения ресурсоемких операций, не требующих немедленного результата. Одновременно можно выполнить вычисления нескольких асинхронных обработчиков в несколько потоков (распараллеливание вычислений).

В Directum RX, например, асинхронный обработчик используется для автоматической выдачи прав доступа на документы.

Фоновые процессы

Предназначены для периодического выполнения работ, которые запускаются по расписанию. Их можно настроить в среде разработки Directum RX или в веб-клиенте на обложке модуля «Администрирование». Фоновые процессы имеют историю выполнения, для этого в журнале записываются и хранятся все события фоновых процессов: запуск, отключение, выполнение, изменение расписания, ошибки в работе процессов и др.

В Directum RX, например, с помощью фонового процесса создается задача для заполнения календаря рабочего времени.

Транзакции

Для работы с данными рекомендуется использовать логически объединенные последовательные операции, которые обрабатываются или отменяются целиком (транзакции). Они обеспечивают целостность выполняемых операций в фоновых процессах и асинхронных обработчиках.

В транзакцию можно передавать данные только простых типов. Чтобы передать сущность, нужно использовать ее идентификатор:

```
   var queueItemsIds = queueItems.Select(q => q.Id).ToList();
foreach (var message in messages)
{
  Transactions.Execute(
    () =>
    {
      var transactionQueueItems = ExchangeCore.MessageQueueItems.GetAll(q =>
queueItemsIds.Contains(q.Id)).ToList();
      if (this.ProcessMessage(client, transactionQueueItems, message))
      {
        var queueItem = queueItems.Single(x => x.ExternalId ==
message.ServiceMessageId);
        queueItem.ProcessingStatus =
ExchangeCore.MessageQueueItem.ProcessingStatus.Processed;
        queueItem.Save();
      }
    });
}
```

Транзакции и сессии с большим количеством объектов

ВАЖНО. Не создавайте и не меняйте в транзакции или сессии большое количество объектов. Максимальный рекомендуемый объем – не более 1000 объектов, а для задач – не более 100.

Где нужно учитывать ограничение?

- транзакции, которые используются в событиях сохранения сущностей, событиях блоков схемы задачи или добавлены в программном коде с помощью класса Transaction.

  Если в транзакции используется метод сохранения **Save()**, то это приводит к подгрузке дополнительного объема данных по каждой сущности, в том числе связанных данных по свойствам-ссылкам. Выгруженные по каждой сущности данные сохраняются в течение всей транзакции. В результате процесс сохранения в транзакциях с большим объемом данных работает медленно. Чем больше данных в обработке, тем медленнее сохранение;

- фоновые процессы и асинхронные обработчики, так как они выполняются в рамках одной сессии.

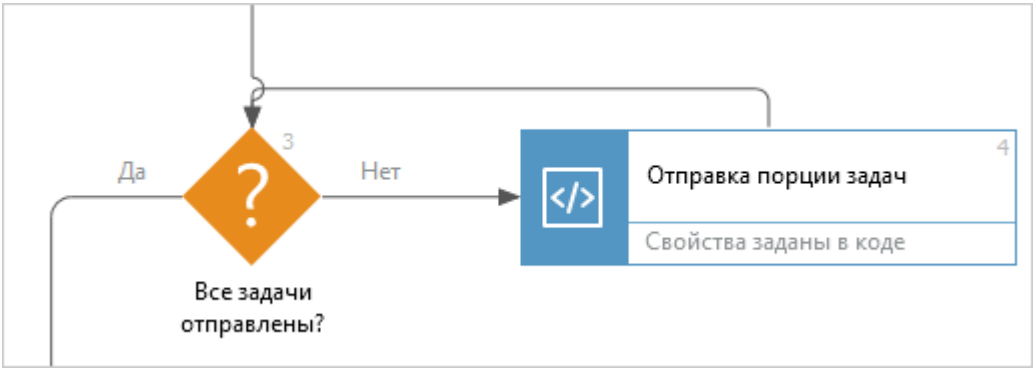
  Ограничение связано с тем, что при работе фонового процесса или асинхронного обработчика все сущности, которые были получены в вычислениях, автоматически подгружаются к сессии. Они удаляются из сессии только после завершения вычислений, даже если эти сущности больше не используются в программном коде. В результате, чем больше получено объектов, тем больше памяти необходимо сервису асинхронных событий **Worker**, и тем больше замедляется скорость сохранения даже одной сущности.

Проблемы могут возникать при сохранении и при изменении сущностей, так как в обоих случаях вызывается метод **Save()**. Также на быстродействие влияет включение большого объема данных в список ToList().

Что делать, если ограничение превышено?

Если планируется, что в транзакции или сессии объектов будет больше, то:

- создавайте объекты порциями по 100 штук и выносите это действие в асинхронный обработчик. Это позволит одновременно выполнять вычисления в несколько потоков (распараллеливать вычисления). Например, такая логика используется при загрузке исторических данных из сервисов обмена в Directum RX;
- в схему задачи добавьте блоки «Условие» и «Скрипт», с помощью которых будет проверяться, отправлены ли все подзадачи в работу. Если отправлены не все, то выполняется скрипт с отправкой следующей порции подзадач. Пример:



- используйте ограниченную выборку с помощью метода **Select()**, если нужно получить только некоторые свойства сущности. Не рекомендуется без необходимости получать сущности целиком.

Основной вариант решения – это обработка порциями. В некоторых ситуациях для ускорения работы может использоваться метод Transactions.Execute() для обработки записи в цикле. Например:

```
foreach (var id in ids)
{
  Transactions.Execute(() =>
                       {
                         // Обработка записи.
                         …
                       });
}
```

ВАЖНО. Запрещается использовать вложенные друг в друга транзакции, так как попытка выполнить или откатить одну транзакцию применяется ко всей цепочке. В связи с этим нельзя использовать **Transactions.Execute()** в вычислениях, которые уже обернуты в транзакцию на уровне платформы. К ним относятся:

- вычисления сервиса блоков схем задач (WorkflowBlockService);
- события До сохранения (в транзакции) и После сохранения (в транзакции);
- функции сервиса интеграции с атрибутом POST;
- код Remote-функций, вызванных в клиентском коде;
- код функций инициализации.

Запуск большого количества длительных асинхронных обработчиков

Не рекомендуется запускать асинхронные обработчики для выполнения длительных и ресурсоемких операций на одном экземпляре сервиса асинхронных событий Worker.

Например, из Directum RX выгружается 200 тысяч документов. Для загрузки порциями по 10 тысяч одновременно запускается 20 экземпляров одного асинхронного обработчика и каждый выполняется по 10-15 минут. Из-за большой нагрузки сервис Worker не успевает обрабатывать сообщения из очереди. В результате в системе не выполняются другие фоновые процессы и асинхронные обработчики, также наблюдаются задержки в выдаче прав доступа, отправке документов во внешнюю систему.

Чтобы таких ситуаций не возникало, рекомендуется:

- выносить асинхронный обработчик длительных и ресурсоемких операций на отдельный экземпляр сервиса асинхронных событий. Для этого необходимо настроить параметры **ASYNC_HANDLERS_PROCESSING** и **ASYNC_HANDLERS_EXCLUDED**. Подробнее см. в разделе «Обработка сообщений асинхронных обработчиков». В этом случае обработка сообщений других фоновых процессов и асинхронных обработчиков выполняется параллельно на другом сервисе;
- использовать переповтор асинхронного обработчика для обработки порциями. Например, запускать не сразу все 20 экземпляров обработчика, а на 2 меньше, чем число потоков. При этом обрабатывать только первые 10 тысяч записей в одном обработчике,

после чего проверять, остались ли еще записи для обработки. Если записи есть, то вызывать переповтор выполнения обработчика. Также можно указать время следующего повтора. В результате сообщение о переповторе попадет в очередь после того, как в нее добавятся сообщения о других фоновых процессах и асинхронных обработчиках.

Для изменения количества потоков, которые сервис может запустить параллельно, настройте параметр **ASYNC_HANDLERS_PROCESSING_PARALLEL_COUNT**. Подробнее см. в разделе «Обработка сообщений асинхронных обработчиков».

- если нет требований по быстродействию, то обработку можно выполнять порциями последовательно в рамках одного фонового процесса. При этом необходимо учитывать, что время выполнения операции увеличится.

Ограничение количества ролей для пользователя

Чтобы выдавать права доступа на документы сотрудникам из разных подразделений, используются роли. Если сотрудник входит в 200 ролей и более, то для него открытие списков документов в системе занимает длительное время. Это связано с тем, что при формировании списка запрос сначала получает все *субъекты прав* с учетом вложенности, в которые входит пользователь. Например, если пользователь включен в роль, а также во вложенные в нее роли. После этого строится запрос на получение выборки документов, на которые у пользователя есть права. Если количество ролей пользователя превышает 200, то запрос нагружает СУБД и выполняется медленно.

Чтобы оптимизировать выполнение запросов на получение списка документов:

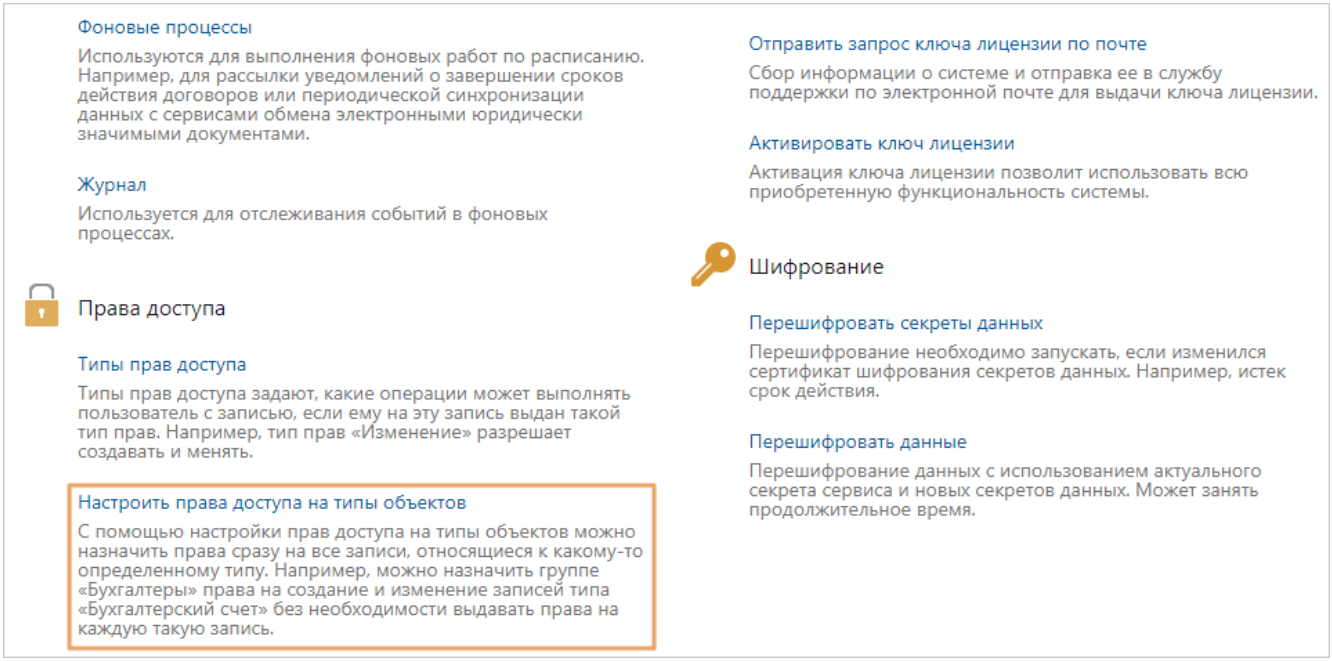
- сократите количество ролей для одного пользователя. При подсчете допустимого количества учитывайте также общее число субъектов прав для пользователя: его вхождения в роли, подразделения, наши организации, замещения, системные замещения. Например, если в роль включены другие роли, то удалите сотрудника из них;
- ограничьте список записей по свойствам, не используя права доступа. Например, по свойству «Подразделение» текущего пользователя. В этом случае для сотрудника отображается список документов с указанным значением свойства.

Назначение большого количества прав на типы документов

На некоторые типы документов требуется выдавать права большому количеству пользователей. Например, всем сотрудникам нужно выдать права на просмотр документов типов «Акт об оказании услуг (выполнении работ)» и «Товарная накладная».

В этом случае не рекомендуется использовать правила назначения прав, так как при выдаче доступа на каждый новый документ увеличивается число записей в системных таблицах с правами. При большом количестве документов, на которые выдаются права, растет и время выполнения запроса на их получение, например при открытии списков.

Чтобы оптимизировать выполнение запросов на получение списков, вместо правил назначения прав настраивайте доступ на типы объектов через обложку модуля «Администрирование»:



Аналогичные рекомендации можно использовать для справочников с экземплярными правами, например «Проекты», а также для задач и заданий.

Назначение прав группе пользователей (экземплярные права)

Если в базе данных разрастается количество записей с правами доступа, то увеличивается время на выполнение запроса к таблицам с этими записями. В зависимости от ситуации откажитесь от выдачи экземплярных прав или используйте событие **UI-фильтрация** (UIFiltering) для доступа к записям списка. Это оптимизирует работу системы, если нужно, например, массово отправить подзадачу или выдать доступ к задаче сотрудникам с правами на вложенный документ.

Пример 1. Массовая отправка подзадачи сотрудникам

Ситуация

В некоторых задачах сотрудникам автоматически назначаются права на вложенный документ.

Например, разработана задача на ознакомление. По ней массово отправляются подзадачи сотрудникам всех подразделений компании и автоматически выдаются права на организационно-распорядительный документ. По одной подзадаче может быть выдано 50-100 прав доступа в зависимости от численности сотрудников в подразделении, а документ при этом вложен в сотни подзадач (по числу подразделений). Суммарное количество прав может превышать 3000.

В результате списки документов в системе открываются длительное время. Права на панели «Управление доступом» также отображаются с замедлением. Это происходит из-за того, что

в базе данных разрастается таблица с правами доступа, и время выполнения запросов к ней увеличивается.

Решение

В такой ситуации рекомендуется назначать права на документ не каждому пользователю отдельно (экземплярные права), а сразу группе сотрудников, например роли или подразделению. Для этого можно использовать разрешения, настройку прав доступа на типы объектов и правила назначения прав.

Например, так реализована выдача прав на вложенный документ в задаче на ознакомление. При отправке задачи исполнители автоматически получают права на вложенный документ. Если в поле ***Участник** указана роль, а в поле **Кроме** – сотрудник из этой роли, то права на документ ему все равно назначаются, так как он является участником роли. При этом задание на ознакомление исключенному из задачи сотруднику не приходит.

Пример 2. Доступ к задаче для сотрудников с правами на вложенный документ

Ситуация

Предположим, разработан справочник «Сообщения». В его записях хранится информация о задачах, отправленных по документу, и заданиях по этой задаче. Так сотрудники с правами на документ могут получить информацию о задачах, даже если не являются их участниками. В качестве способа авторизации указано значение **Для типа и экземпляра сущности**.

При отправке задачи по документу в справочнике создается новая запись. На нее выдаются права тем сотрудникам, у которых есть доступ к вложенному в задачу документу.

Таким образом, для одного документа создаются десятки записей в справочнике. С увеличением количества документов разрастается число записей. Также увеличивается количество записей с правами доступа в таблице базе данных. Оно становится равным количеству задач по документу, умноженному на число прав на этот документ.

В результате в системе замедляется открытие списка сущностей. Кроме этого, наблюдаются пики потребления оперативной памяти при программной работе с сущностью.

Решение

Для оптимизации работы с сущностями рекомендуется:

- выдавать права на документ и на запись справочника не каждому сотруднику отдельно, а группе пользователей;
- вместо выдачи прав на записи справочника используйте событие UI-фильтрация.

Массовое удаление документов

В веб-клиенте сотрудник может удалять из системы ошибочно созданные документы. Также иногда нужно массово удалять документы, например десять и более. В прикладном коде для этого используется метод Delete(). Например, в системе разработчик доработал согласование документов так, что для каждого согласующего создаются рабочие копии документов. Это позволяет избежать блокировок документа для других сотрудников и из примечаний составить протокол согласования. При таком подходе количество документов увеличивается в несколько раз, поэтому после согласования нужно массово удалять рабочие копии.

В рабочее время рекомендуется отказаться от массового удаления документов, так как оно снижает быстродействие и отклик системы для всех пользователей. Это связано с медленным удалением записей из СУБД: в ней нет полного покрытия индексами ограничений (constraints), и их проверка происходит длительное время.

Если нужно массово удалить документы, то рекомендуется:

- делайте это в нерабочее время;
- удалите версии документов, а не сами документы. После этого нужно очистить файловое хранилище и изъять права на документ. Подробнее про очистку файлового хранилища см. в разделе «Сервис хранилищ» (Windows и Linux);
- переиспользуйте существующие записи. Например, при создании копии документа добавьте условие, которое будет проверять, что исходный документ можно повторно использовать. Если можно, то продолжить работу с исходным документом, а если нет, то создать копию. При реализации необходимо учитывать, что документ возможно используется в других местах: отправлен на согласование, он уже подписан, зарегистрирован и т.д. В этом случае его удалить нельзя.

Версионирование функций инициализации

Если система содержит большой объем данных и количество пользователей, то процесс инициализации может быть длительным. Например, при обновлении системы повторно задаются первоначальные настройки: заполняются справочники значениями по умолчанию, выдаются необходимые права и т.п. На эти действия требуется время.

Чтобы ускорить инициализацию, в базовом решении для модулей используется версионирование функций инициализации. Для этого добавлены функции:

- **FirstInitializing()** – задаются первоначальные настройки системы, например справочники **Организации** и **Валюты** заполняются значениями по умолчанию;
- **Initializing<номер_версии>()** – задается инициализация для конкретной версии системы, в которой была переопределена или добавлена новая логика инициализации базового модуля.

Таким образом, если система обновляется на новую версию, то первичная настройка пропускается и запускается только та логика инициализации, которая ранее не выполнялась.

Также вы можете добавить версионирование для своих функций инициализации, чтобы они не выполнялись повторно. Предположим, в версии 4.9 была добавлена своя функция инициализации. Чтобы адаптировать инициализацию версии 4.10 под версионирование:

1. Перенесите в функцию **FirstInitializing()** логику инициализации, которая была в системе до версии 4.10.
2. Создайте функцию **Initializing410()** и добавьте в нее код инициализации для версии 4.10:

```
public virtual void Initializing410()
{
  // Код инициализации для версии 4.10.0.
}
```

3. Добавьте константы для инициализации модуля.

   Пример для модуля «Документооборот» (Docflow):

```
public static class Init
{
  public static class Docflow
  {
    // Название параметра для хранения инициализированной версии модуля.
    public const string Name = "InitDocflowUpdate";
    // Номер версии первоначальной инициализации модуля.
    public const string FirstInitVersion = "4.9.0.0";
    /// Номер версии инициализации модуля на версию 4.10.
    public const string Version410 = "4.10.0.0";
  }
}
```

   Таким образом, при добавлении функций инициализации необходимо предварительно создать константы:

   - **Name** – название параметра в таблице Sungero_Docflow_Params, в который последовательно записываются инициализированные версии модуля;
   - **FirstInitVersion** – номер версии для инициализации модуля при установке системы;
   - **Version<номер_версии>** – номер версии для инициализации модуля при обновлении системы на новую версию.

4. Вызовите в событии **Initializing()** функцию **ModuleVersionInit()**. С ее помощью выполняется инициализация на определенную версию модуля.

   Функция содержит параметры:

   - **initAction** – функция, которая выполняет инициализацию указанной версии модуля;
   - **moduleInitMark** – название параметра, в котором хранится версия последней выполненной инициализации;
   - **initCodeVersion** – версия модуля, для которой предназначена указанная инициализация.

   Пример вызова функции:

```
    public override void Initializing(Sungero.Domain.ModuleInitializingEventArgs
    e)
{
                                                                        Sungero
    .Commons
    .PublicInitializationFunctions
    .Module.ModuleVersionInit(this.FirstInitializing,
    Constants.Module.Init.Commons.Name,
    Version.Parse(Constants.Module.Init.Commons.FirstInitVersion));

                                                                        Sungero
    .Commons
    .PublicInitializationFunctions
    .Module.ModuleVersionInit(this.Initializing410,
    Constants.Module.Init.Commons.Name,
    Version.Parse(Constants.Module.Init.Commons.Version410));
    }
```

5. Опубликуйте решения, для модулей которых были добавлены функции инициализации, и запустите инициализацию по кнопке **Запустить инициализацию** на панели инструментов.
6. Проверьте результат выполнения функций инициализации в локальной системе Directum RX.

Массовое удаление субъектов прав

Иногда за один раз нужно удалить множество *субъектов прав*. Вручную удалять каждую запись трудоемко, а массовое удаление снижает отклик системы. Поэтому, например, откажитесь от удаления ошибочно созданных записей или неиспользуемых ролей в пользу менее ресурсоемких способов.

Пример 1. Удаление из справочника ошибочно созданных записей

Предположим, для загрузки данных организационной структуры используется интеграция с внешней системой. Ключевым полем для поиска загруженных сотрудников в Directum RX является поле с идентификатором внешней системы. Если изменяется внешняя система, меняются и значения ключевого поля. Если не доработать настройки интеграции, то при следующей загрузке данных в систему Directum RX учетные записи сотрудников дублируются. Нужно удалить ошибочно продублированные записи.

В рабочее время рекомендуется отказаться от массового удаления учетных записей сотрудников, так как оно снижает быстродействие и отклик системы для всех пользователей. Это связано с медленным удалением записей из СУБД: в ней нет полного покрытия индексами ограничений (constraints), и их проверка происходит длительное время.

Вместо удаления рекомендуется выбрать один или несколько вариантов:

- измените состояние учетных записей на **Закрытая**;
- в редакторе типа справочника **Сотрудники** (Employee) добавьте новое свойство – признак того, что запись отмечена на удаление. В событии **UI-фильтрация** (UIFiltering) или **Фильтрация** (Filtering) напишите логику, с помощью которой в открытых списках не будут отображаться записи с признаком на удаление;
- настройте замещения на самих себя для дублирующихся записей сотрудников.

Пример 2. Удаление неиспользуемых ролей

Например, для назначения прав на регистрацию документов использовались роли. Они автоматически генерировались в разрезе журналов регистрации. В процессе работы создавались тысячи новых ролей.

После опытной эксплуатации подход к выдаче прав изменился, и для назначения прав стали использовать замещения на виртуального сотрудника. В результате в системе остались ранее созданные тысячи неиспользуемых ролей.

В этой ситуации вместо удаления ролей рекомендуется:

- переименовать роли. Вместо названия использовать символы, чтобы при like-вводе такие записи не попадали в выпадающий список. Например, можно использовать символы кодировки UTF-8;
- изменить состояние роли на **Закрытая**.

Оптимизация работы с сущностями

Учитывайте рекомендации по разработке типов сущностей:

- блокировка сущности перед изменением
- вызов асинхронного обработчика при сохранении карточки
- разработка карточек объектов с большой табличной частью
- оптимизация события «Возможность выполнения» (CanExecute)
- работа с большим количеством свойств типа сущности
- большое количество подписей на документе
- ускорение массового подписания документов
- оптимизация выполнения блока «Мониторинг»

Блокировка сущности перед изменением

ВАЖНО. Перед изменением сущности установите на нее блокировку сразу после получения из базы данных. Не рекомендуется устанавливать блокировку надолго, после сохранения изменений ее нужно сразу снять.

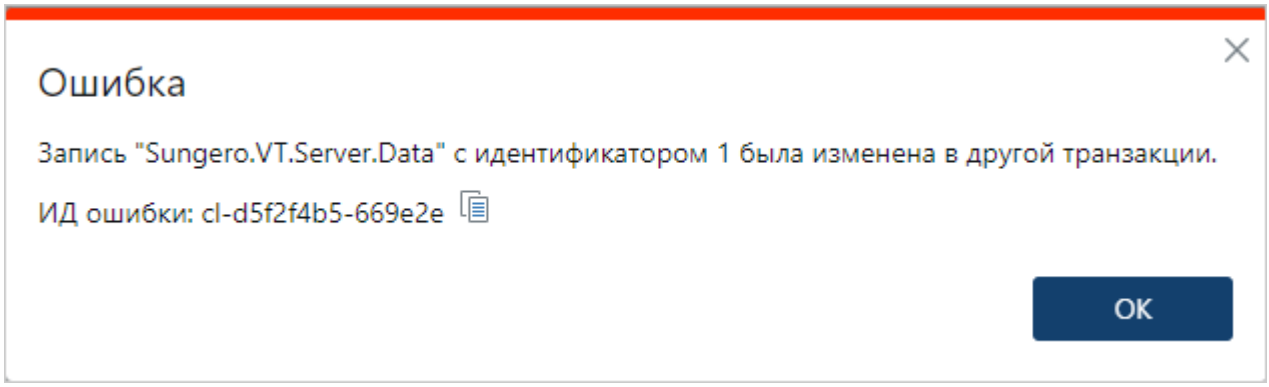
Если блокировка не установлена, то на момент сохранения отредактированных данных, сущность может быть уже изменена. При изменении одной и той же сущности в разных потоках происходит рассинхронизация данных. Также версия может устареть, если карточка открыта длительное время в модальном окне. Чтобы ошибки рассинхронизации не

возникали, после закрытия модального окна состояние сущности обновляется в базе данных, перезагружаются права доступа, несохраненные изменения теряются.

Чтобы избежать потери данных в случае отсутствия блокировок, в системе Directum RX используется версионирование сущностей:

1. Процесс 1 получает сущность из базы данных и вносит локальные изменения без установки блокировки.
2. Процесс 2 получает сущность из базы данных и вносит локальные изменения без установки блокировки.
3. Процесс 1 сравнивает изменяемую сущность с ее версией на сервере и записывает изменения в базу данных.
4. Процесс 2 сравнивает изменяемую сущность с ее версией на сервере. Если есть различия, то изменения не записываются в базу данных.

В результате, если действие выполнялось на веб-сервере, то в веб-клиенте возникает ошибка, и действие по изменению сущности нужно повторить:



Если изменения в базу данных вносил другой сервис, например Worker, то ошибка в веб-клиенте не отображается, а действие отправляется на переповтор.

В лог-файлах фиксируются исключения:

- Sungero.Domain.Exceptions.StaleEntityException – сущность изменена в параллельном потоке;
- Sungero.Domain.Exceptions.StaleEntityNotFoundException – сущность удалена в параллельном потоке.

Ошибки означают, что изменения были внесены одновременно разными сервисами, и в прикладном коде не хватает блокировок сущностей. Например, если сотрудник пытается выдать права доступа, а в это время их меняет фоновый процесс.

Как определить, где не хватает блокировки

1. Включите логирование изменений версии для всех сущностей. Для этого в конфигурационный файл config.yml в секцию **common_config** добавьте параметр **ENTITY_VERSIONING_LOG_ENABLED** и укажите в нем значение **true**.

   При возникновении конфликтов в лог-файле фиксируются исключения StaleEntityException или StaleEntityNotFoundException.

   Исключение StaleEntityException содержит информацию:

   - **ActualVersion** – актуальный номер версии в базе данных;
   - **StaleVersion** – локальный устаревший номер версии;
   - **InternalStaleVersion** – внутренний устаревший номер версии. Обычно номер совпадает с указанным в **StaleVersion**.

   Для StaleEntityNotFoundException фиксируется:

   - **EntityName** – имя типа сущности.
   - **Id** – идентификатор сущности.

   Для типа логгера VersioningLogger в атрибуте cust записывается дополнительная информация, если сущность была изменена:

   - **Type** – тип сущности;
   - **Id** – идентификатор сущности;
   - **Version** – новый актуальный номер сущности;
   - **OldVersion** – предыдущий номер сущности.

   Если сущность была удалена, то для типа логгера VersioningLogger фиксируется только номер удаляемой версии.

2. Когда ошибка возникнет, найдите ее в лог-файлах по типу исключения StaleEntityException или StaleEntityNotFoundException.

   Предположим, ошибка зафиксирована в лог-файле веб-сервера:


```
{
  "t":"2023-08-21 11:35:39.442+04:00",
  "pid":"18172+111",
  "tr":"cl-d5f2f4b5-669e2e",
  "l":"Error",
  "lg":"ExceptionPolicy",
  "mt":"Запись \"Sungero.VT.Server.Data\" с идентификатором 1 была изменена в другой
транзакции.",
  "ex":{
    "type":"Sungero.Domain.Exceptions.StaleEntityException",
    "m":"Запись \"Sungero.VT.Server.Data\" с идентификатором 1 была изменена в другой
транзакции.",
```

```
    "stack": //"---> NHibernate.StaleObjectStateException: Row was updated or deleted
by another transaction (or unsaved-value mapping was incorrect):
[Sungero.VT.Server.Data#1]
   //...
    "cust":{
      "EntityName":"Sungero.VT.Server.Data",
      "Id":1,
      "ActualVersion":11,
      "StaleVersion":10,
      "InternalStaleVersion":10
    },
    "isInternal":false
  },
  "un":"Administrator",
  "tn":"Directum RX",
  "v":"4.9.0.0000"
}
```

3. По времени возникновения ошибки найдите в лог-файлах других сервисов, где произошло параллельное изменение сущности.

   Предположим, в лог-файле сервиса асинхронных событий есть запись от логгера VersioningLogger с сообщением «Entity was updated» примерно в то же время. По дополнительной информации из сообщения можно определить, что это та сущность, с которой возникла проблема:

```
{
  "t":"2023-08-21 11:35:38.854+04:00",
  "pid":"23040+79",
  "tr":"cl-d5f2f4b5-138ae617df",
  "l":"Debug",
  "lg":"VersioningLogger",
  "mt":"Entity was updated",
  "cust":{
    "Type":"Sungero.VT.Server.Data",
    "Id":1,
    "Version":11,
    "OldVersion":10
  },
  "un":"Service User",
  "tn":"Directum RX",
  "v":"4.9.0.0000"
}
```

4. В лог-файле сервиса асинхронных событий по идентификатору трассы определите асинхронный обработчик или фоновый процесс. В примере по идентификатору трассы cl-4ebda1cc-739a4fbc07 находится асинхронный обработчик TestAsyncHandler:

```
{
  "t":"2023-08-21 11:35:38.670+04:00",
  "pid":"23040+283",
  "tr":"cl-4ebda1cc-739a4fbc07",
  "l":"Info",
  "lg":"AsyncHandlerOperation",
  "span":{
    "status":"Started",
    "name":"execute",
    "asyncHandler":"TestAsyncHandler"
  },
  "un":"Service User",
  "tn":"Directum RX",
  "v":"4.9.0.0000"
}
```

5. Проанализируйте код действия, в котором возникает ошибка, и код асинхронного обработчика, где произошло параллельное изменение.

   Пример кода выполнения действия:

```
public virtual void TestFunc()
{
  // Получить сущность для изменения без установки блокировки.
  var data = Functions.Data.Remote.GetDatas().ShowSelect();

  if (data == null)
    return;

  // Запустить АО, который будет менять ту же сущность.
  var asyncHandler = AsyncHandlers.AsyncHandler.Create();
  asyncHandler.Id = data.Id;
  asyncHandler.ExecuteAsync();

  // Показать диалог. В это время АО может успеть выполниться и изменить
сущность.
  var dialog = Dialogs.CreateConfirmDialog("?");
  if (!dialog.Show())
    return;

  data.Name = Guid.NewGuid().ToString();
  // Попытаться сохранить сущность, но она уже устарела. Возникает ошибка.
  data.Save();
}
```

   Пример кода асинхронного обработчика:

```
public                                virtual                                void
AsyncHandler(Sungero.VT.Server.AsyncHandlerInvokeArgs.AsyncHandlerInvokeArgs
args)
{
  // Получить сущность для изменения без установки блокировки.
  var data = Datas.GetAll(x => x.Id == args.Id).First();
  data.Name = Guid.NewGuid().ToString();
  data.Save();
}
```

6. Добавьте блокировку в код действия и код асинхронного обработчика сразу после получения сущности. После сохранения изменений блокировку нужно снять.

В результате при выполнении действия устанавливается блокировка на сущность, и другие процессы не могут ее изменить. Действие успешно изменяет сущность, в это время асинхронный обработчик уходит на переповтор. Он отработает после сохранения сущности в базе данных. При этом в лог-файле фиксируются исключения RepeatedLockException, связанные с блокировками.

Вызов асинхронного обработчика при сохранении карточки

При сохранении карточки сущности не рекомендуется использовать безусловный вызов асинхронного обработчика. Это связано с тем, что если сохранение карточки вызывается многократно в рамках одного процесса или вызывается в другом асинхронном обработчике

с возможностью переповтора, то очередь RabbitMQ разрастается и сообщения занимают место на диске. В результате это может привести к неработоспособности RabbitMQ.

При настройке вызова асинхронного обработчика используйте рекомендации:

- не запускайте обработчик на каждое сохранение карточки. Проанализируйте, при каких изменениях свойств это необходимо делать. Например, при изменении статуса документа не нужно вызывать обработчик для выдачи прав доступа;
- удалите вызов асинхронного обработчика из сохранения карточки или перенесите его в фоновый процесс, который будет выполняться по расписанию;
- программно отключите вызов асинхронного обработчика при сохранении карточки. Например, в Directum RX подобным образом используется отключение обработчика **GrantAccessRightsToDocument** (Асинхронная выдача прав на документ). Подробнее см. в разделе «Особенности назначения прав при миграции документов».

Разработка карточек объектов с большой табличной частью

При проектировании сущностей, в которые планируется добавлять табличную часть, рекомендуется сразу анализировать, какое количество строк предположительно будет в таблице. Это связано с тем, что при изменении карточки объекта между веб-клиентом и веб-сервером передаются данные. Если карточка содержит большую табличную часть, то процесс передачи данных нагружает систему. В результате у сотрудника при работе в веб-клиенте могут длительное время выполняться действия с карточкой объекта, а также это влияет на скорость работы в системе остальных сотрудников.

ВАЖНО. Не используйте табличную часть, если в ней ожидается:

- 100 строк (для версий Directum RX 4.6 и ниже);
- 2000 строк (для версий Directum RX 4.7 и выше).

Если планируется, что записей будет больше, то:

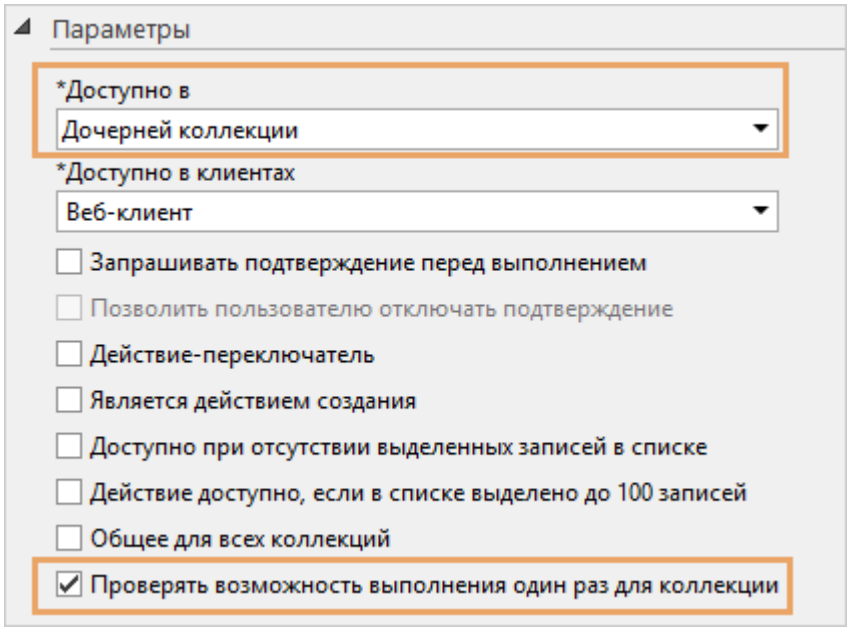
- вынесите табличную часть в отдельный справочник;
- в справочнике используйте тот же набор свойств, который предполагалось использовать в таблице;
- вместо обращения к строкам табличной части, работайте со списком записей справочника.

ПРИМЕЧАНИЕ. Учитывайте, что если данные из большого списка записей справочника будут добавляться в контрол состояния, то их отображение в контроле в любом случае нагружает систему и занимает время. Это замедляет работу сотрудников в веб-клиенте, так как при построении отображения в контроле с веб-сервера на веб-клиент загружается сразу весь объем данных. Чтобы не нагружать систему, рекомендуется вместо контрола состояния выводить список записей по кнопке в карточке объекта. Кроме того, учитывайте, что с версии Directum RX 4.7 добавлены ограничения: если данные в контроле состояния загружаются более 30 секунд или количество элементов превышает 2 000, то загрузка прерывается, и у пользователя появляется сообщение. При необходимости администратор может изменить значения ограничений.

Оптимизация события «Возможность выполнения» (CanExecute)

По умолчанию в системе оптимизировано вычисление события Возможность выполнения (CanExecute) для системных действий, которые доступны в *дочерней коллекции*: **Копирование строки** (CopyChildEntity), **Удаление строки** (DeleteChildEntity) и **Добавление строки** (AddChildEntity). Оптимизация заключается в том, что возможность выполнения действия проверяется один раз для всей коллекции, а не вызывается многократно для каждой записи. Это позволяет ускорить работу с карточками объектов, у которых в табличной части большое количество записей. Например, когда в карточке роли указано более тысячи участников.

В среде разработки новые действия для дочерних коллекций по умолчанию создаются с учетом оптимизации. Для этого в редакторе типа сущности автоматически устанавливается флажок **Проверять возможность выполнения один раз для коллекции** для действий, у которых в поле ***Доступно в** выбрано значение **Дочерней коллекции**:



Снимите этот флажок, если событие **Возможность выполнения** (CanExecute) нового действия все же необходимо вычислять отдельно для каждого элемента коллекции. Например, в системе для действий коллекции версий документа флажок снят. Возможность выполнения проверяется отдельно для каждой версии.

Как определить, что нужно оптимизировать вычисление события

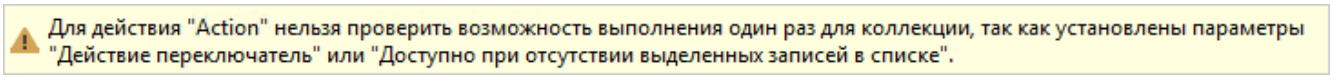
Если в карточках объектов содержатся сотни или тысячи записей и выполнение в них действий занимает время, то проверьте, как вызывается обработчик события **Возможность выполнения** (CanExecute). Скорее всего обработчик вызывается многократно для каждой записи, и это нагружает систему. В такой ситуации рекомендуется оптимизировать вычисление события:

1. С помощью информации из лог-файла веб-сервера определите, для каких действий обработчик события вызывается многократно и замедляет работу. Для этого в лог-файле запустите поиск по фразе **"method":"child"**. Для логгера с названием **CanExecutesSpan** в атрибуте **span** фиксируется информация о выполнении события (CanExecute) в дочерних коллекциях.

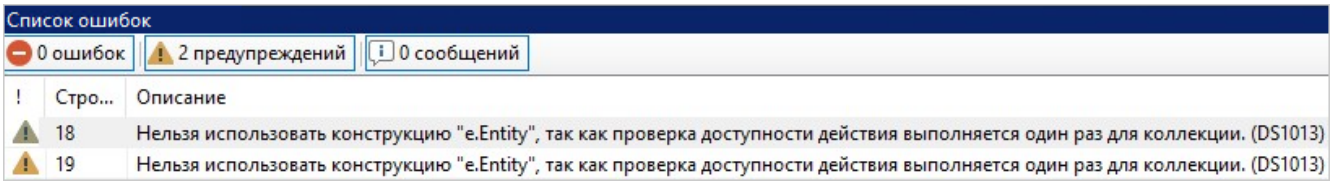
   Пример лог-файла:


```
{
    "t":"2023-09-22 11:03:01.541+04:00",
    "pid":"22436+130",
    "tr":"cl-7d941297-9b305f",
    "l":"Info",
    "lg":"CanExecutesSpan",
    "span":{
        "durationMs":707,
        "status":"Ok",
        "entitiesCount":151,
        "method":"child",
        "statCount":13,
        "totalTimeMs":1043,
        "isLong":true,
        "topTime":"[{\"NameGuid\": \"7a0d8bb0-0de6-4f37-8417-a61ede7c111d\", \"Name\":
\"OpenCard\", CanExecute: \"Sungero.Domain.Client.ChildEntityActions.CanOpenCard\"
DurationMs: 231},{\"NameGuid\": \"2cd06508-0938-4197-85ca-ce1cb0ce9750\", \"Name\":
\"CreateDoc\", CanExecute: \"Sungero.Domain.Client.ChildEntityActions.CanCreateDoc\"
DurationMs: 197},{\"NameGuid\": \"fe14d141-d127-4ebc-81bd-d6f9b1ce2265\", \"Name\":
\"ExportDoc\", CanExecute: \"Sungero.Domain.Client.ChildEntityActions.CanExportDoc\"
DurationMs: 279}]"},
    "un":"Domain\\ardo_na",
    "tn":"Directum RX",
    "v":"4.8.0.0000"
}
```


2. В атрибуте **durationMs** проверьте общую длительность выполнения всех обработчиков события. Если длительность превышает 500 миллисекунд, то добавляется атрибут **isLong** со значением **true**. Это значит, что необходимо оптимизировать вычисление обработчика.
3. Если в лог-файле есть строки с атрибутом **"isLong":true**, то в атрибуте **topTime** посмотрите имена действий, для которых нужно оптимизировать вычисление. В среде разработки для этих действий установите флажок **Проверять возможность выполнения один раз для коллекции**. Также учтите, что:
   - проверка доступна, если в редакторе типа сущности для действия сняты флажки **Действие-переключатель** или **Доступно при отсутствии выделенных записей в списке**. Если флажки установлены, то при сохранении изменений появится сообщение и изменения нельзя будет сохранить:



- в коде обработчика события **Возможность выполнения** не рекомендуется обращаться к дочерней сущности через _obj или e.Entity. Если конструкции используются, то при сборке решения в окне вывода появляется предупреждение, обработчик события выполняется один раз для всей коллекции, а не для отдельной дочерней коллекции:



ВАЖНО. Нельзя в наследниках или перекрытиях переопределить флажок **Проверять возможность выполнения один раз для коллекции**:

- для действий **Копирование строки** (CopyChildEntity), **Удаление строки** (DeleteChildEntity) и **Добавление строки** (AddChildEntity);
- для действий, добавленных в родительском типе сущности.

Работа с большим количеством свойств типа сущности

При создании свойств для типа документа или задачи в соответствующей таблице базы данных добавляется новый столбец. Когда число столбцов превышает допустимое, при добавлении нового свойства в среде разработки возникает ошибка. Ограничение зависит от используемой СУБД. Так, в Postgres Pro это число находится в пределах от 250 до 1600 в зависимости от типов колонок, в Microsoft SQL Server – до 1024.

Например, количество столбцов может быть превышено в таблицах **Sungero_Content_EDoc** (Карточки электронных документов), **Sungero_WF_Assignement** (Данные всех типов заданий, заданий на приемку и уведомлений) или **Sungero_WF_Task** (Данные всех типов задач), если:

- создано много разных типов документов, при разработке которых не используется наследование или перекрытие. Все новые поля добавляются отдельно в каждом типе документа.

  В этом случае рекомендуется спроектировать иерархию документов и выделить общих предков. Новые поля, одинаковые для нескольких типов, добавлять на уровне их абстрактных предков, например в OfficialDocument.
- для работы с заявками используются задачи и задания. Типов заявок более 10. Все свойства заявки вынесены на карточку задачи и задания.

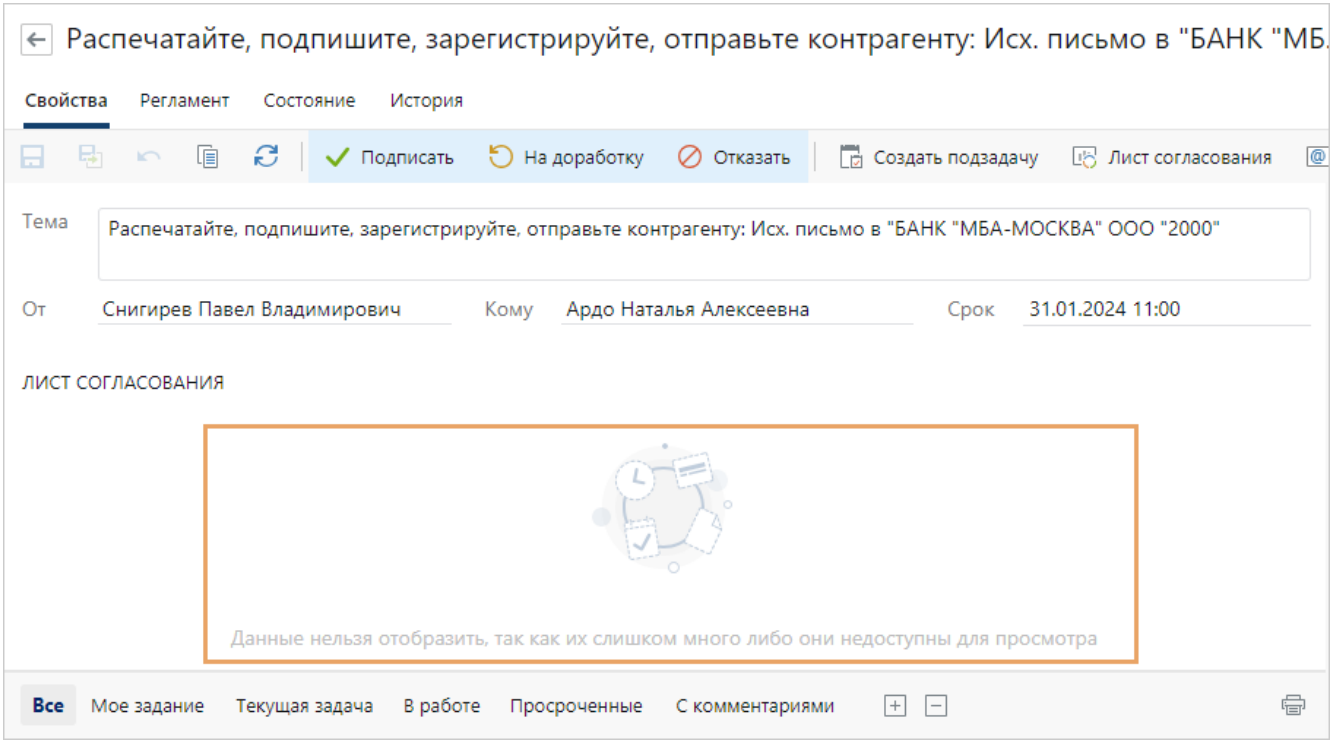
  В этом случае рекомендуется выносить заявки в отдельный справочник или тип документа и добавлять их в задачи в качестве вложений.
- свойства вложенного документа продублированы в карточках задачи и задания, чтобы выполнять по ним поиск в папках потока.

  В этом случае рекомендуется выстраивать процессы работы со списком документов, а не со списком задач. Например, в Directum RX таким образом реализован мониторинг документов на согласование с контрагентом – с помощью списка «Документы у контрагентов».

Большое количество подписей на документе

Иногда на документе может быть установлено более тысячи подписей. Например, документ отправляют на ознакомление, в ходе которого все сотрудники компании подписывают его. В результате при открытии окна с большим списком подписей создается нагрузка на систему и растет потребление памяти сервисами.

ПРИМЕЧАНИЕ. С версии 4.9 в системе ускорена работа с окном списка подписей. Также снижена нагрузка на систему при загрузке большого объема данных в контрол состояния. Например, с помощью контрола состояния в карточке задания отображается лист согласования. При этом если данные в контроле загружаются более 30 секунд или количество элементов превышает 2 000, то загрузка прерывается, и у пользователя появляется сообщение:



Тем не менее, при работе с документами, на которых установлены тысячи подписей, могут возникать проблемы при преобразовании в PDF с отметкой об электронной подписи, формировании листа согласования и др. Поэтому рекомендуется сокращать количество подписей, например:

- увеличивайте количество документов, чтобы уменьшить число подписей на одном документе. Например, можно подготовить «Согласие на работу в выходной день» отдельно для каждого подразделения, а не всей организации;
- не используйте подписи, если требуется только фиксировать сам факт подписания без сохранения юридической значимости. В этом случае подписание сотрудником можно фиксировать в отдельной сущности: коллекции в документе, отдельном справочнике или строить отчет по задачам.

СОВЕТ. Для получения списка подписей используйте перегрузку метода Signatures.Get() с фильтром. Например, с помощью нее можно получить информацию только о подписях с результатом «Утверждено». Благодаря этому, если у документа тысячи подписей, то для их получения не расходуется память сервисов Directum RX и снижается нагрузка на систему.

Ускорение массового подписания документов

Когда сотрудник подписывает большое количество документов, это может занимать время и нагружать систему. Чтобы массовое подписание документов выполнялось быстро и оптимально, используйте рекомендации из этого раздела.

Что уже оптимизировано в системе для подписания

В Directum RX для ускорения массового подписания документов уже оптимизировано:

- подписание простой электронной подписью;
- подписание усиленной электронной подписью через веб-агент;
- подписание усиленной неквалифицированной подписью (УНЭП) с использованием плагина КриптоПро DSS;
- сохранение большого количества сформированных подписей.

ПРИМЕЧАНИЕ. Оптимизация не затрагивает подписание плагинами, разработанными на основе шаблона из репозитория GitHub.

Оптимизация действует в ситуациях, когда сотрудник вручную выделяет документы в списке и через контекстное меню выбирает действие **Подписать**. Также оптимизация действует, если в системе с помощью среды разработки программно настроено подписание документов. Для этого используются перегрузки методов Signatures.Approve() и Signatures.Endorse() с параметром **entities**. В нем указывается пачка документов для подписания.

ВАЖНО. Успешно подписывается или нет сразу вся пачка документов. Поэтому не рекомендуется использовать большие пачки, например, делите их на порции по 100 штук.

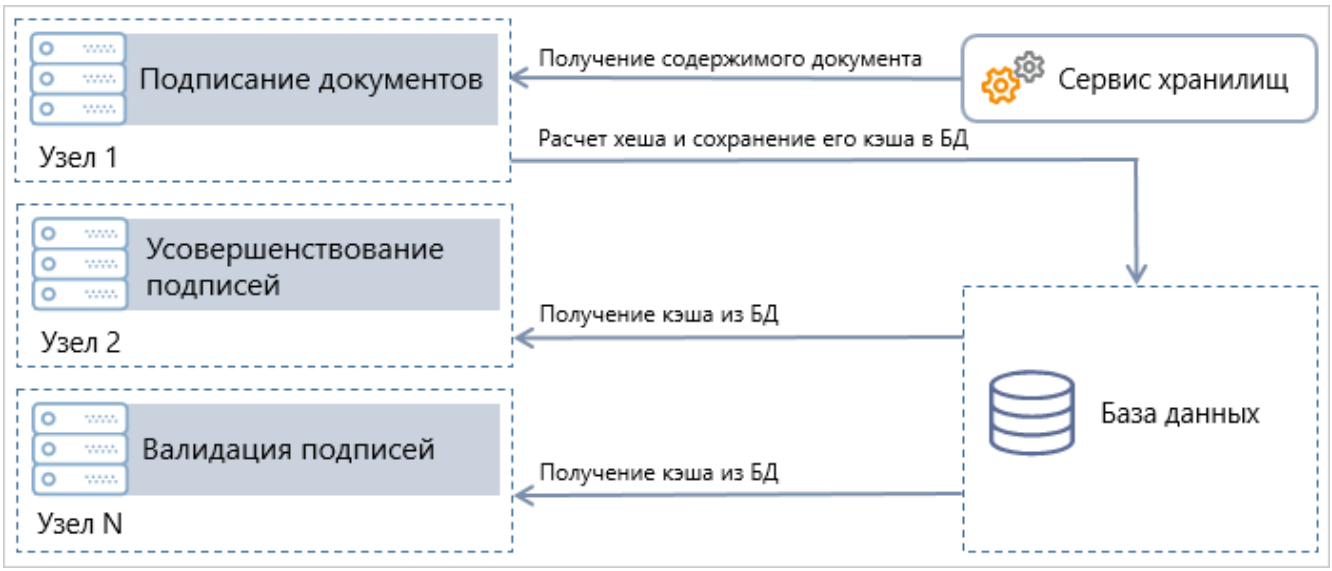
Расчет хеша документа и его кэширование

При выполнении операций с подписями рассчитывается хеш подписываемых данных. Он временно кэшируется и сохраняется в базе данных для повторного использования. Благодаря этому в системе ускоряются операции:

- подписание документа одним или несколькими сотрудниками;
- подписание нескольких документов одним сотрудником;
- валидация подписей;
- усовершенствование подписей.

Также снижается нагрузка на сервис хранилищ, так как для выполнения операций системе не нужно каждый раз получать содержимое документа и заново рассчитывать хеш. Достаточно получить его сохраненный кэш из базы данных.

Кроме того, массовые операции с подписями выполняются быстро даже когда система развернута в кластере. Например, на узле с веб-сервером выполняется подписание, а на узле с сервисом асинхронных событий – усовершенствование подписей. Системе достаточно рассчитать хеш на одном узле и получить его сохраненный кэш из базы данных на других:



По умолчанию время жизни хеша в кэше – 240 минут. Если это время истекло или подписываемые данные изменились, то хеш пересчитывается при следующем обращении к ним. Администратор может изменить значение с помощью параметра SIGNABLE_PROPERTIES_HASH_CACHE_LIFETIME_IN_MINUTES.

Как еще оптимизировать и ускорить подписание

Дополнительно ускорить массовое подписание документов можно с помощью метода Signatures.ComputeAndStoreHashes(). Например, сотруднику приходит на подписание большой поток документов, при этом их содержимое он не меняет. Чтобы подписание выполнялось быстрее, с помощью метода можно предварительно рассчитать хеши документов и задать их время жизни в кэше.

ВАЖНО. Предварительный расчет хеша необходимо выполнять на сервисе асинхронных событий (Worker) и до того, как документ придет на подписание. Например, при сохранении карточки документа добавьте запуск асинхронного обработчика для расчета его хеша.

Если используется КриптоПро DSS, то на скорость подписания влияет формат итоговой подписи: CAdES-BES – подписание выполняется быстро, каждый следующий уровень (CAdES-T, CAdES-XL, CAdES-A) замедляет время формирования подписи. Для ускорения подписания в настройках плагина КриптоПро DSS задайте формирование электронной подписи CAdES-BES. Затем задайте настройки для усовершенствования подписей до необходимого формата с помощью механизмов платформы.

Оптимизация выполнения блока «Мониторинг»

Если количество обрабатываемых блоков «Мониторинг» становится больше сотни за одну секунду, это нагружает базу данных. Очередь отложенных сообщений разрастается до такой степени, что сервис отложенных операций (**DelayedOperationsService**) не успевает их обрабатывать. Такое может случиться при одновременной работе множества задач. В результате они зависают со статусом «В обработке».

Например, в компании запущены сотни тысяч процессов, разработанных на основе задачи на исполнение поручения. В их схемах используется блок «Мониторинг», с помощью которого каждые 10 минут проверяется завершение подзадач и заданий. В результате одновременно стартованные задачи, по которым еще не сформированы задания, остаются со статусом «В обработке».

В этом случае при проектировании схем задач в среде разработки или в проводнике веб-клиента выберите один из способов:

- для оптимизации нагрузки на веб-сервер закладывайте больший период мониторинга, чем значение по умолчанию – 15 минут. Для вычисления оптимального периода учитывайте среднее количество одновременно выполняющихся задач;
- вызывайте выполнение блока «Мониторинг» программно, не указывая свойства **Период** и **Срок прекращения**. Настройте вызов события Результат мониторинга в том событии, завершение которого должен был контролировать блок мониторинга, например при выполнении задания. В этом случае мониторинг выполняется, только когда из кода вызвано событие блока **Результат мониторинга**;
- откажитесь от использования блока «Мониторинг». Вместо этого в схему задачи добавьте задание и в качестве исполнителя укажите системного пользователя. Настройте программное выполнение этого задания при достижении события, которое должен был контролировать блок «Мониторинг».

  Также не рекомендуется использовать блок «Мониторинг» для принудительного прекращения задания. Вместо мониторинга можно использовать свойство **Срок прекращения** блока «Задание».

Сервис интеграции

Учитывайте рекомендации при настройке интеграции с внешней системой:

- ограничение количества объектов для передачи
- запросы на выполнение длительных операций

Ограничение количества объектов для передачи

При проектировании интеграции необходимо сразу учитывать, какой объем информации внешняя система будет запрашивать из Directum RX. Это связано с тем, что с ростом количества передаваемых объектов увеличивается потребление памяти на сервисе интеграции из-за замыкания получившегося списка. В результате создается нагрузка, которая может привести к неработоспособности сервиса и повлиять на работу внешней системы.

Кроме того, если сервис интеграции развернут на одном компьютере с другими сервисами Directum RX, то передача данных может влиять и на их работоспособность.

Если планируется запрашивать из Directum RX большие объемы данных, то необходимо ограничивать количество объектов для передачи через методы сервиса интеграции. Для этого рекомендуется:

- передавать не более 1 000 объектов. Для этого используйте методы расширения LINQ – **Take()** и **Skip()**. Если количество объектов больше заданного в ограничении, то в ответе на запрос можно добавлять пояснение, почему переданы не все объекты, а только 1 000. Подробнее см. пример реализации;
- с версии Directum RX 4.10 используйте фильтрацию данных с помощью параметров $top и $skip. В этом случае сервис интеграции сразу получает отфильтрованные данные с учетом переданных параметров.

  В версиях 4.9 и ниже рекомендуется при передаче больших объемов данных использовать деление на порции (серверная пагинация) в запросах к СУБД вместо фильтрации данных с помощью параметров **$top** и **$skip**. Это связано с тем, что в предыдущих версиях системы при использовании параметров на сервер передаются сразу все данные, только потом они фильтруются. Из-за этого создается нагрузка на сервис интеграции.

Пример «Передавать не более 1 000 объектов»

Ситуация

Через сервис интеграции необходимо получить список договоров по контрагенту с ограниченным набором полей: ИД, наименование, дата создания. Список нужно отфильтровать по имени контрагента. При этом в ответе на запрос должно выводиться не больше 1 000 документов. Если запрошено большее количество документов, то отображать ошибку.

Решение

ВАЖНО. Далее в примерах кода замените значение **DEV** на ваш код компании и **Trade** – на имя вашего модуля, в который добавляется интеграционная функция.

1. В среде разработки в своем решении откройте редактор модуля и в группе «Структуры и константы» добавьте новую константу **GetDocumentsCountLimits**. Укажите в ней лимит по количеству объектов:

```
public static class Module
    {
      public const int GetDocumentsCountLimits = 1000;
    }
```

2. В том же редакторе модуля добавьте структуру **DocumentList**, в которой будет храниться ИД, наименование и дата создания:

```
[Public]
    partial class DocumentList
    {
      public long Id { get; set; }
```

```
  public string Name { get; set; }
  public DateTime? Created { get; set; }
}
```

3. Добавьте еще одну структуру **GetDocumentResult**, которая будет использоваться для вывода текста сообщения и списка договоров:

```
[Public]
  partial class GetDocumentResult
  {
    public string ErrorText { get; set; }
    public List<DEV.Trade.Structures.Module.IDocumentList> documentList { get;
  set; }
  }
```

4. В ресурсы модуля добавьте текст сообщения. Для этого в дереве решений в контекстном меню модуля выберите пункт **Открыть ресурсы**. Чтобы добавить ресурс, в контекстном меню табличной части выберите пункт **Добавить строку** и заполните:
   - имя – GetDocumentsCountLimitsOverload;
   - текст сообщения – «Превышено допустимое количество документов. Запрошено документов – {0}, максимальное количество – {1}».
5. В редакторе вашего решения добавьте зависимость от модуля Sungero.Contracts.
6. Добавьте интеграционную функцию для получения документов. Для этого в редакторе модуля в группе «Функции» перейдите по ссылке **Серверные** и создайте функцию **GetContractList()** с атрибутом **[Public(WebApiRequestType = RequestType.Get)]**:

```
[Public(WebApiRequestType = RequestType.Get)]
  public virtual DEV.Trade.Structures.Module.IGetDocumentResult
  GetContractList(int count, string counterpartyName)
  {
    var result = DEV.Trade.Structures.Module.GetDocumentResult.Create();

    // Получить контрагента по имени.
    var counterparty = Sungero.Parties.Companies.GetAll(x => x.Name ==
  counterpartyName).FirstOrDefault();

    // Если запрашиваемое количество документов превышает лимит,
    // показать сообщение об ошибке.
    if (count > DEV.Trade.Constants.Module.GetDocumentsCountLimits)
    {
      result.ErrorText =
  DEV.Trade.Resources.GetDocumentsCountLimitsOverloadFormat(count,
  DEV.Trade.Constants.Module.GetDocumentsCountLimits);
      return result;
    }

    // Получить список документов по контрагенту.
    result.documentList = new
  List<DEV.Trade.Structures.Module.IDocumentList>();
    result.documentList.AddRange(Sungero.Contracts.Contracts.GetAll(x =>
  x.Counterparty == counterparty).Take(count).Select(x => new
  DEV.Trade.Structures.Module.DocumentList() {Id = x.Id, Name = x.Name, Created
  = x.Created}).ToList());
    return result;
  }
```

7. Сохраните изменения и опубликуйте решение.

Тестирование

1. Если в параметрах среды разработки установлен флажок **Запускать клиент после публикации**, то после публикации автоматически запускается веб-клиент. Если флажок не установлен, на панели инструментов нажмите на кнопку **Запустить веб-клиент**.
2. В открывшейся локальной системе создайте тестовые договоры.
3. Напишите HTTP-запрос к ранее добавленной интеграционной функции. Для этого используйте URL в формате:

```
{Протокол https или http}://{имя сервера, на котором установлен сервис интеграции}/{имя сервиса интеграции}/odata/{Имя модуля}/{Имя функции (параметры)}
```

Пример 1. Запрос без превышения лимита

GET-запрос:

```
http://dev.com/integration/odata/Trade/GetContractList(count=100,counterpartyName='Техносистемы')
```

Ответ сервиса интеграции:


```
{

"@odata.context"
:
"http://dev.com/odata/$metadata#Sungero.IntegrationService.Models.Generated.Trade.IGetDocumentResultDto",
      "ErrorText":null,
      "documentList":[
        {
            "Id":36,
            "Name":"Договор с Техносистемы \"Тестовый\"",
            "Created":"2024-05-17T16:43:41+03:00"
         }
       ]
...
}
```


Пример 2. Запрос с превышением лимита

GET-запрос:

```
http://dev.com/Integration/odata/Trade/GetContractList(count=10000,counterpartyName='Test')
```

Ответ сервиса интеграции:


```
{

"@odata.context"
:
"http://dev.com/odata/$metadata#Sungero.IntegrationService.Models.Generated.Test.IGetDocumentResultDto",
       "ErrorText":"Превышено допустимое количество документов. Запрошено документов – 10000, максимальное количество – 1000",
       "documentList":[]
}
```

Запросы на выполнение длительных операций

Некоторые запросы к сервису интеграции могут возвращать ошибку с кодом «502.3 Bad Gateway» из-за длительного выполнения операции. Например, создание большого количества документов из внешней системы и их отправка на согласование могут превысить установленный в системе тайм-аут.

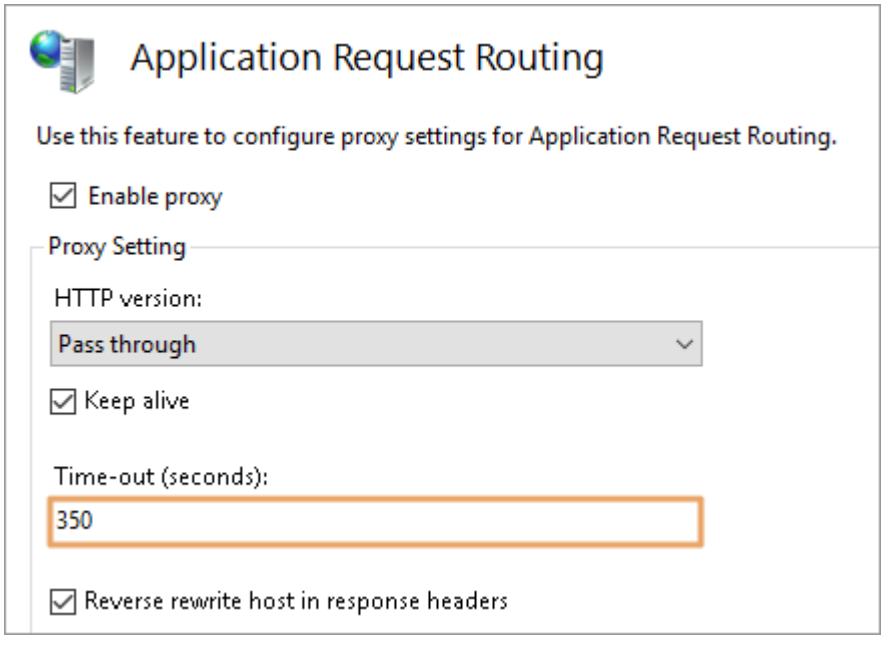
Чтобы такие запросы выполнялись без ошибок, рекомендуется перенести логику длительной операции в асинхронный обработчик, и вызвать его в интеграционной функции.

В этом случае сервис интеграции передает создание документа сервису асинхронных событий и создает запись в очереди обработки. При выполнении запроса из внешней системы сервис интеграции вернет в ответе идентификатор записи очереди. После создания документа и отправки его на согласование статус меняется на завершенный. Узнать актуальный статус выполнения запроса можно по идентификатору записи очереди. Для этого, например, нужно выполнить GET-запрос на получение сущностей или разработать свой GET-запрос.

Если по какой-то причине асинхронные обработчики использовать нельзя, увеличьте тайм-аут обработки HTTP-запросов. По умолчанию тайм-аут составляет 350 секунд. Его можно изменить только в том случае, если время выполнения операции гарантированно не превысит новое значение. Для этого:

Windows проверьте и при необходимости измените настройки IIS.

1. В диспетчере служб IIS выделите узел веб-сервера.
2. Выберите пункт **Application Request Routing Cache**.
3. В области «Действия» перейдите по ссылке **Server Proxy Settings...**.
4. В открывшемся окне в поле **Time-out(seconds)** проверьте значение и откорректируйте его, если необходимо:



Linux измените настройки в конфигурационном файле HAProxy.

Примеры разработки

В разделе описаны примеры разработки, которые помогут изучить возможности среды разработки и подскажут идеи для развития системы Directum RX.

Разработка модуля и типа документа

Ситуация

Предположим, необходимо создать модуль «Закупки» для планирования, согласования и исполнения закупок. В модуле должны быть:

- справочник «Виды закупок», который заполняется только программно и содержит виды: «Закупка у единственного поставщика», «Закупка по прямому договору», «Закупка по конкурентным переговорам»;
- справочник «Закупки», в котором хранится информация о закупках: названия, виды закупок, инициаторы, ответственные, состояния;
- тип документа «Договор закупки», который отличается от обычного договора тем, что содержит свойство со ссылкой на закупку из справочника «Закупки».

Для типа документа «Договор закупки» по умолчанию должны быть созданы виды: «Договор закупки расходных материалов», «Договор закупки оборудования», «Договор прочих закупок».

Для удобства работы с новым модулем должна быть настроена его обложка.

Дополнительно в вычисляемую папку «Реестр договоров» нужно добавить колонку «Закупка» и кнопку для быстрого создания договоров закупки. Кроме того, следует доработать панель фильтрации вычисляемой папки: добавить контрол для фильтрации договоров по закупке.

Сотрудники должны иметь возможность отправлять договор закупки на согласование, при этом:

- когда согласование завершено, инициатор закупки должен получить уведомление о подписании договора;
- если вид закупки «Закупка у единственного поставщика» и сумма договора меньше 50 тыс. рублей, договор не согласуется с финансовым директором.

Решение

Создайте решение и необходимые типы сущностей:

- Шаг 1. Создайте решение с модулем «Закупки» (Purchases) и типы справочников «Закупки» (Purchase) и «Виды закупок» (PurchaseKinds).
- Шаг 2. Создайте тип договора «Договор закупки» (PurchaseContract).
- Шаг 3. Доработайте вычисляемую папку «Реестр договоров» в перекрытии модуля «Договоры».
- Шаг 4. Создайте представление модуля «Закупки» и настройте его обложку.
- Шаг 5. Добавьте логику инициализации модуля.

Настройте согласование одним из способов:

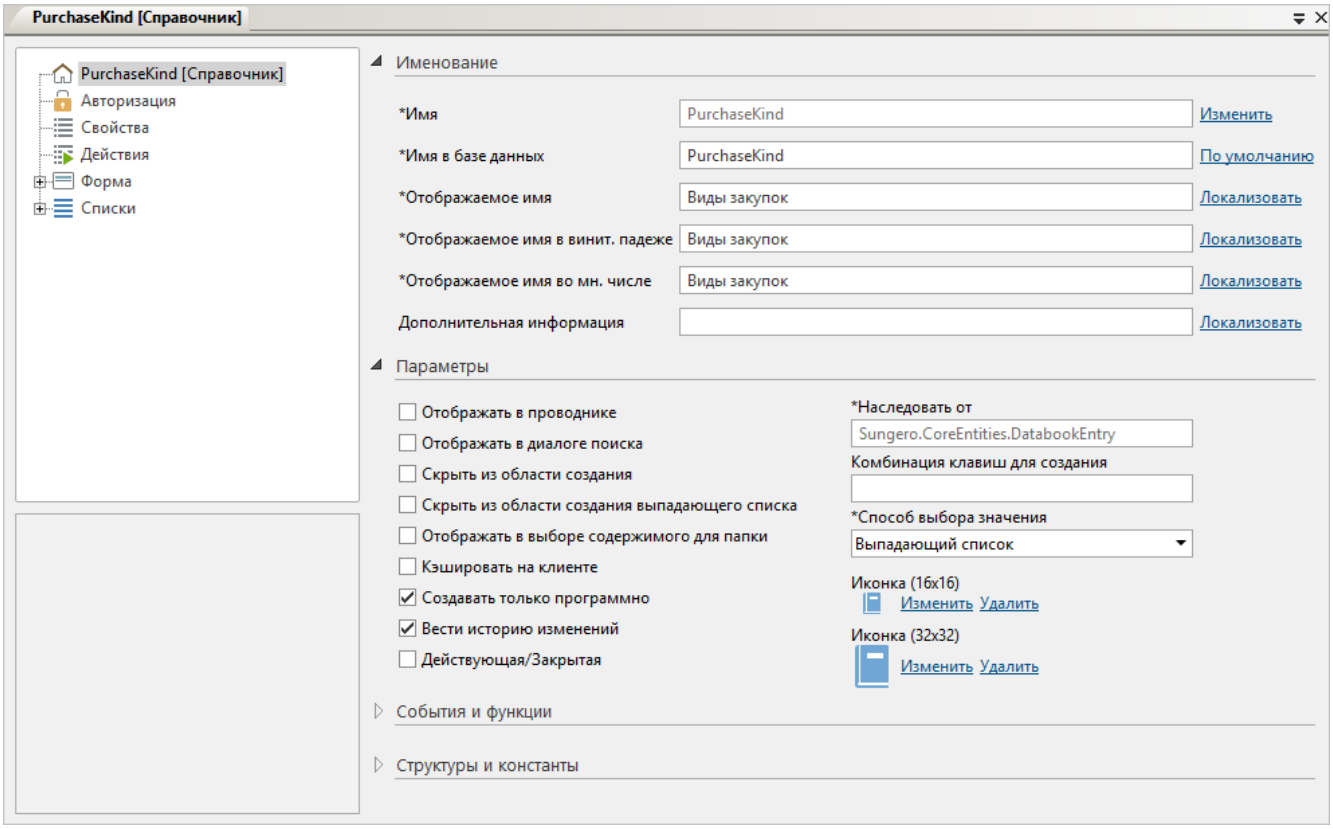
- по процессу (рекомендуется). В этом случае схема согласования настраивается с помощью механизма no-code. Подробнее см. в руководстве администратора, раздел «Настройка схем бизнес-процессов»;
- по регламенту. Для этого:

  Шаг 6. Добавьте роли согласования.

  Шаг 7. Добавьте условия согласования.

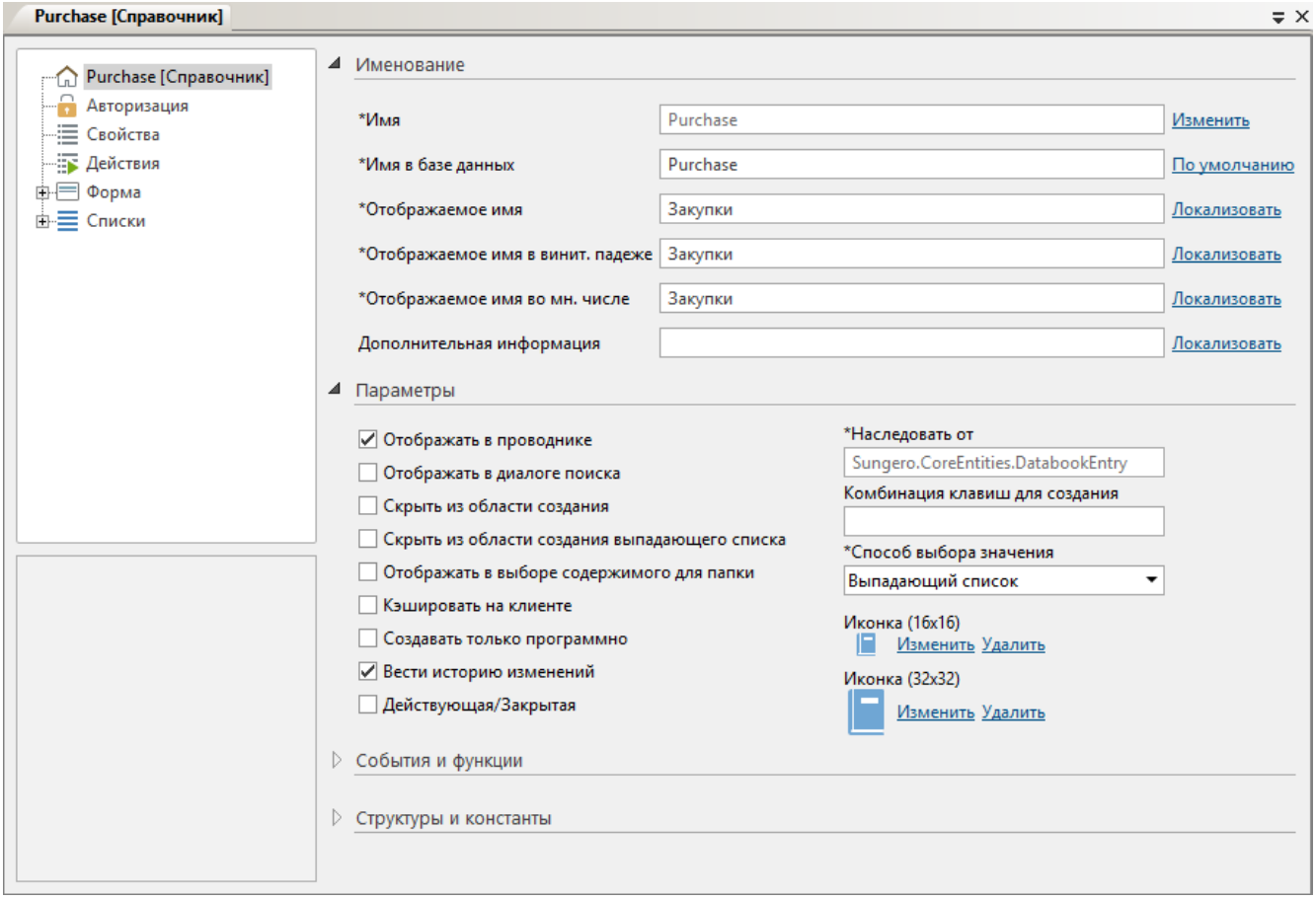
Шаг 1. Создание решения, модуля и справочников

1. Создайте решение (PurchasesSolution) и модуль «Закупки» (Purchases).
2. Создайте тип справочника «Виды закупок» (PurchaseKind). В качестве базового типа сущности укажите тип Sungero.CoreEntities.DatabookEntry. Измените именование типа справочника. Для локализации отображаемых имен перейдите по ссылке **Локализовать**.



3. Записи в справочнике будут создаваться только программно при инициализации модуля, поэтому в параметрах справочника установите флажок **Создавать только программно** и снимите флажок **Отображать в проводнике**.

4. Создайте тип справочника «Закупки» (Purchase). В качестве базового типа сущности укажите тип Sungero.CoreEntities.DatabookEntry. Измените именование типа справочника.

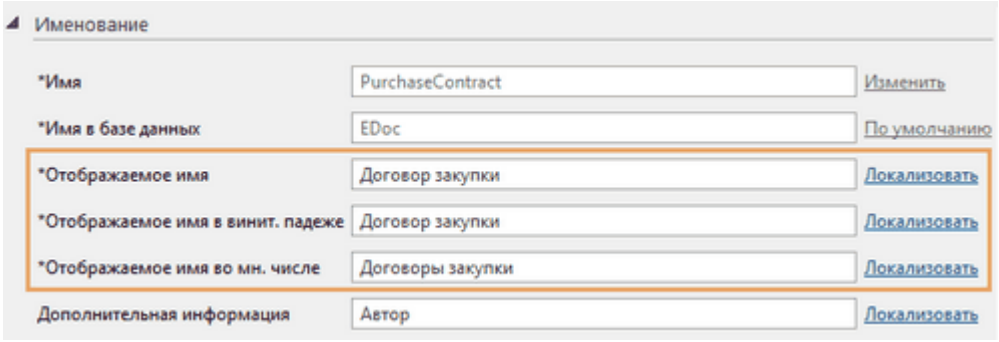


5. В редакторе типа справочника «Закупки» перейдите в узел «Свойства» и создайте свойства:
   - **Вид закупки** (PurchaseKind). Тип свойства – ссылка на справочник «Виды закупок»;
   - **Инициатор закупки** (Initiator) – сотрудник, который запросил приобретение товаров или услуг. Тип свойства – ссылка на справочник «Сотрудники» (Sungero.Company.Employee);
   - **Ответственный за закупку** (ResponsiblePurchase) – сотрудник, который обеспечивает организацию и выполнение закупки. Тип свойства – ссылка на справочник «Сотрудники» (Sungero.Company.Employee);
   - **Примечание** (Notes). Тип свойства – **Текст**.
6. Так как в справочнике «Закупки» есть свойства-ссылки на справочник «Сотрудники», перейдите в первый узел модуля «Закупки» и добавьте зависимость от модуля Sungero.Company. Если зависимости не указаны, то при публикации могут возникнуть ошибки.

Шаг 2. Создание типа договора

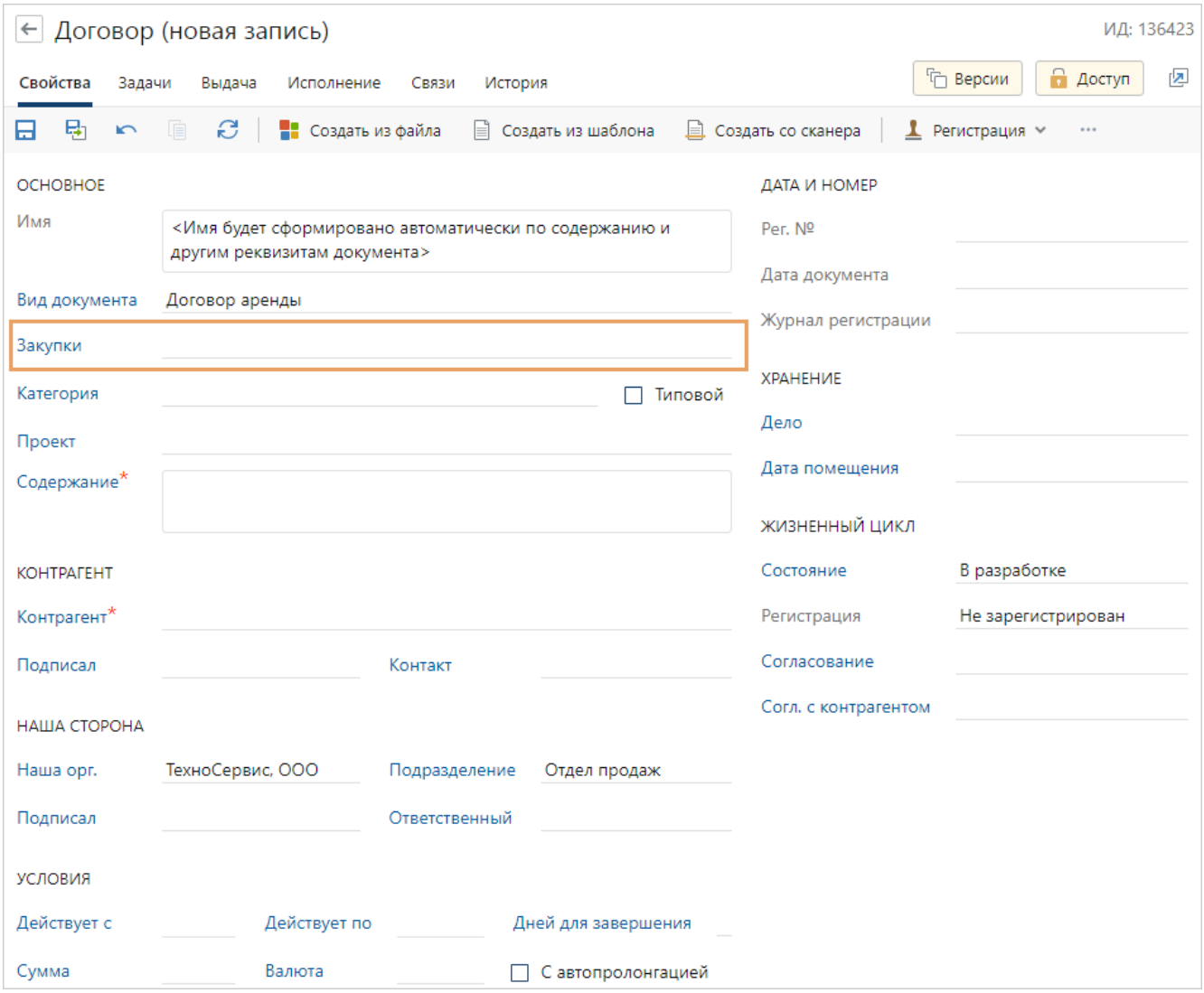
Создайте тип документа «Договор закупки», который отличается от обычного договора тем, что содержит свойство со ссылкой на закупку. Для этого:

1. Создайте тип документа «Договор закупки». Имя типа документа – PurchaseContract. В качестве базового типа сущности укажите тип Sungero.Contracts.ContractBase («Базовый договор»). Добавьте зависимость вашего модуля от модуля Sungero.Contracts. Если зависимости не указаны, то при публикации могут возникнуть ошибки. В результате:
   - от базового типа будет наследоваться стандартная логика обработки договоров;
   - изменения в созданном типе договора (наследнике) не будут затрагивать другие типы договоров. Подробнее см. схему наследования договорных документов;
   - договор закупки будет виден в реестре договоров и поддерживать всю прочую функциональность для работы с договорами.
2. Поправьте именование типа документа:

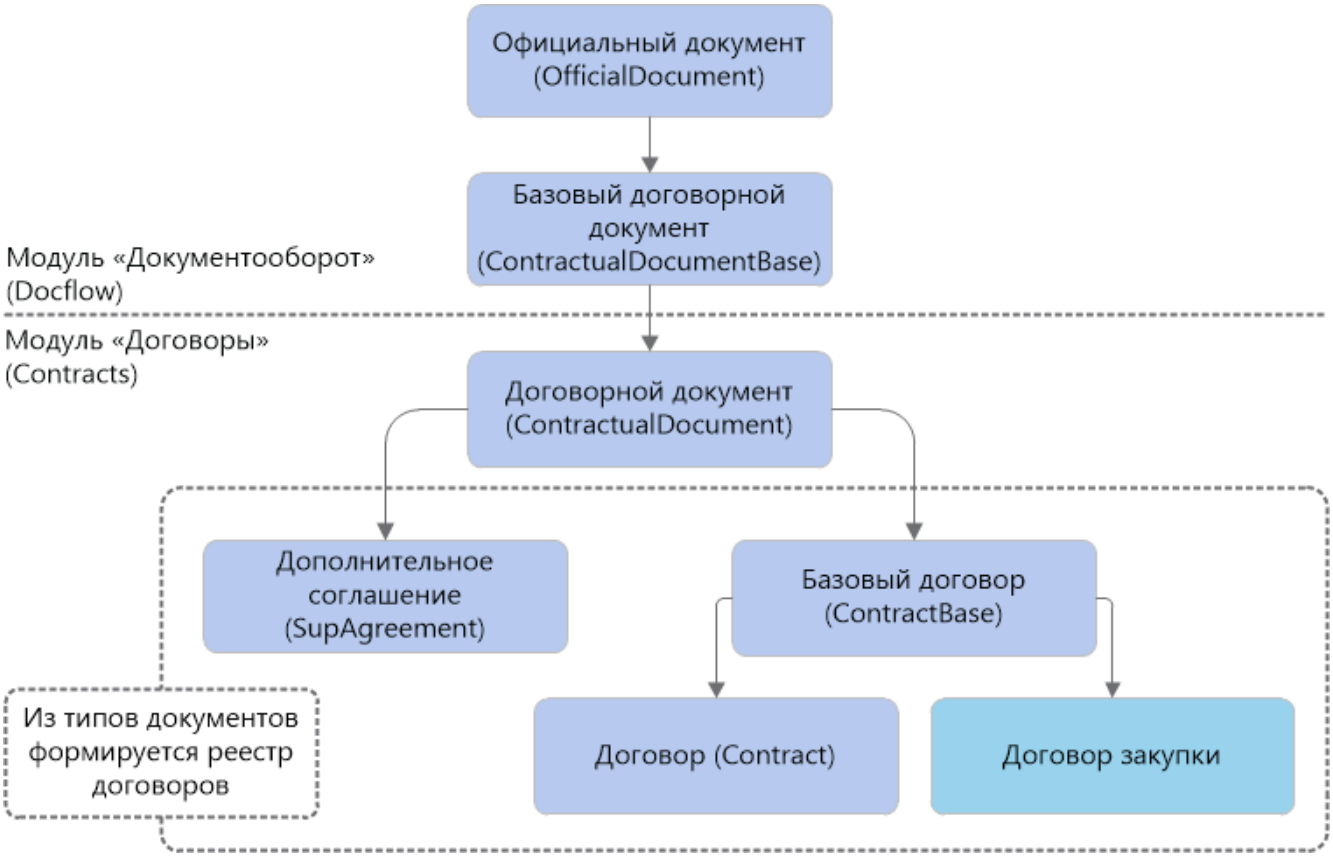


   Для локализации отображаемых имен перейдите по ссылке **Локализовать**.
3. Добавьте свойство **Закупка**. Имя свойства – Purchase, тип – ссылка на справочник «Закупки» (Purchase) и сделайте его обязательным для заполнения. Для этого в параметрах свойства установите флажок **Обязательное**.
4. В карточку договора закупки добавьте контрол и свяжите с созданным свойством. Чтобы добавить контрол на основную вкладку карточки, в узле «Форма» выделите вкладку MainPage и в свойствах элемента снимите флажок **Использовать контролы предка**.

В результате карточка договора закупки будет выглядеть следующим образом:



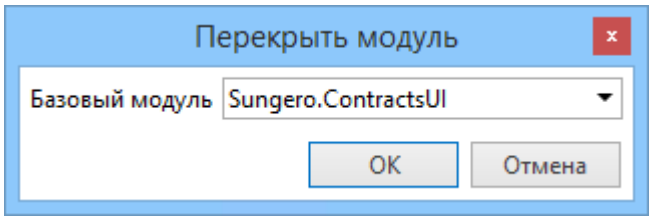
После добавления договора закупки схема наследования договорных документов будет выглядеть следующим образом:



Шаг 3. Доработка вычисляемой папки в перекрытии модуля

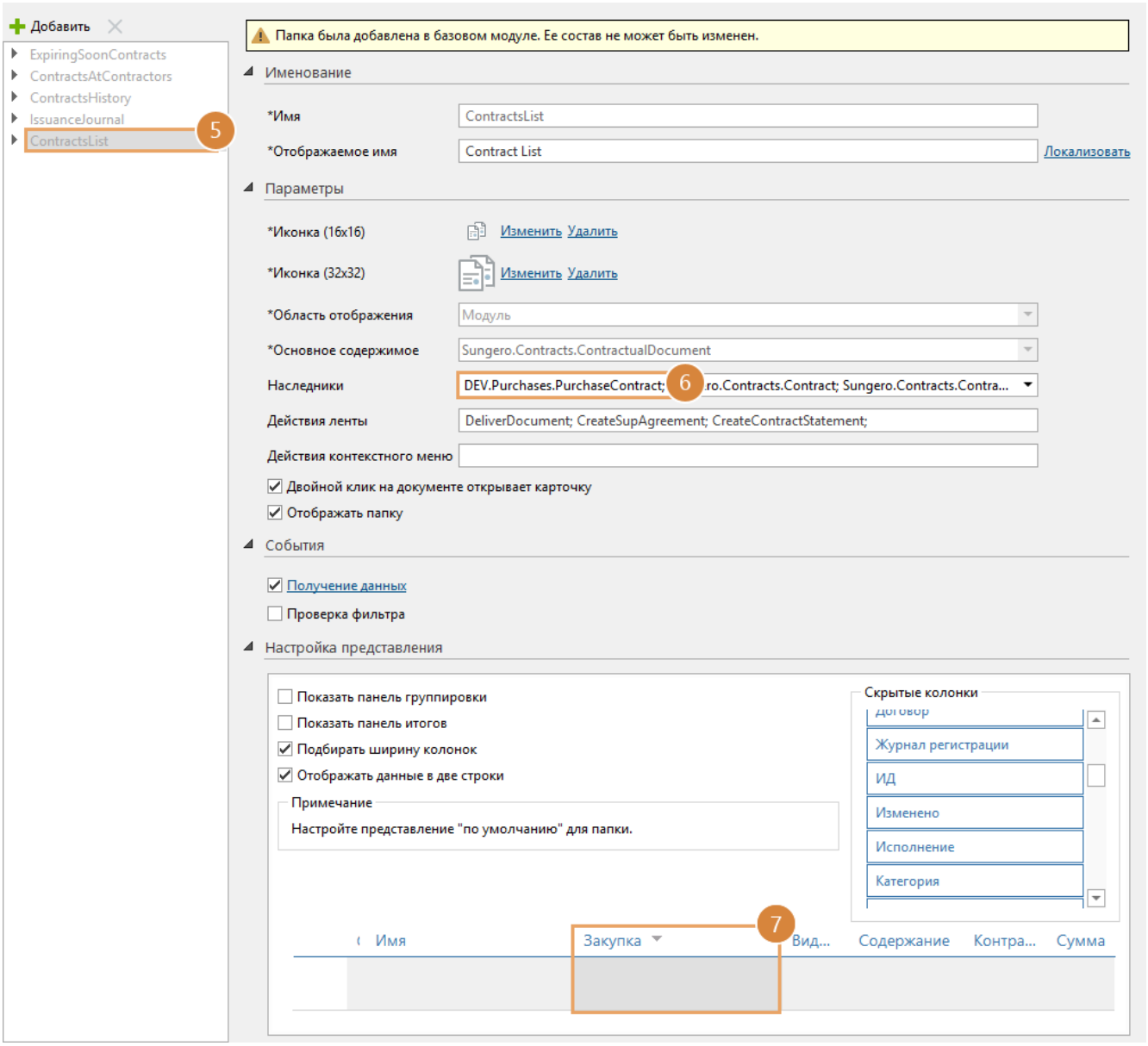
Чтобы в реестре договоров появилась колонка «Закупки» и быстрое действие для создания договора закупки:

1. В контекстном меню решения последовательно выберите пункты **Перекрыть**, **Модуль**.
2. В открывшемся окне в выпадающем списке выберите базовый модуль Sungero.ContractsUI.



3. Нажмите на кнопку **OK**. Модуль появится в дереве решений в разделе «Перекрытые элементы». Откроется его редактор.
4. В редакторе вашего решения добавьте зависимость от модуля Sungero.ContractsUI. Если зависимости не указаны, при публикации могут возникнуть ошибки.

5. В редакторе перекрытого модуля в узле «Папки» выберите вычисляемую папку «Реестр договоров» (ContractsList).

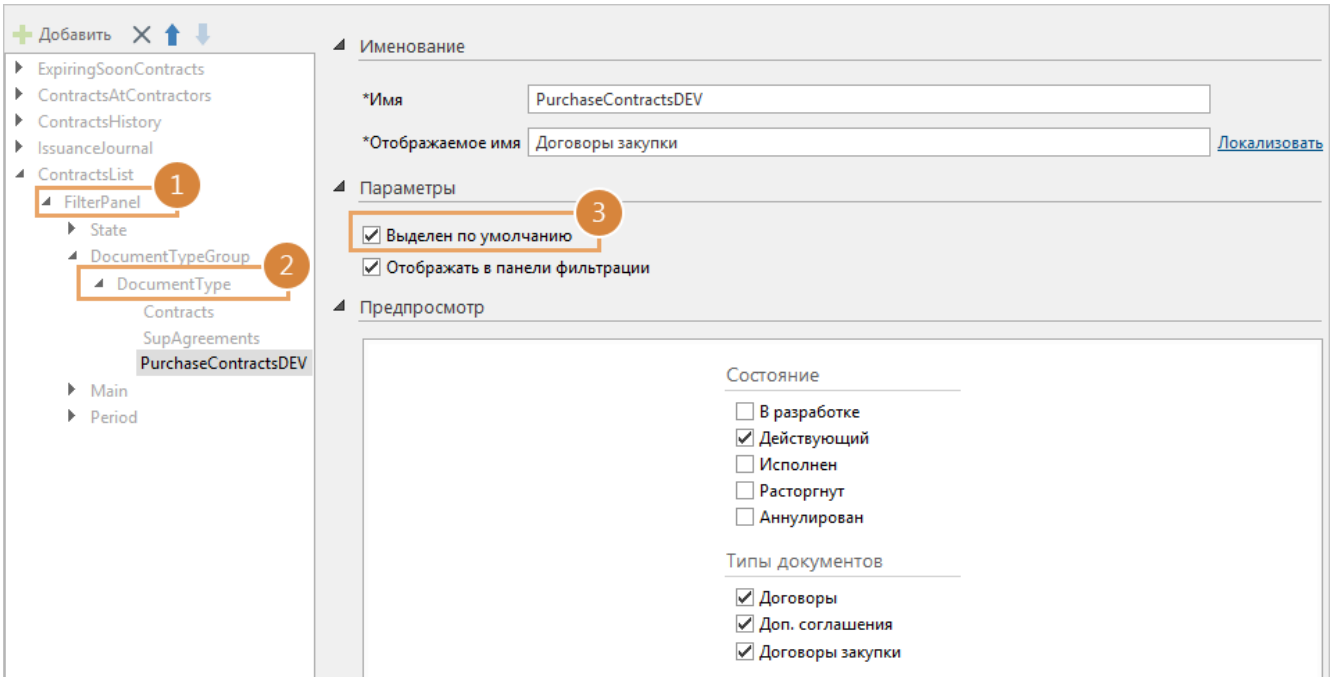


6. В выпадающем списке **Наследники** установите флажок для типа документа «Договор закупки» (PurchaseContract). В результате:
   - в области **Скрытые колонки** появится колонка **Закупка**;
   - после публикации в реестре появится быстрое действие для созданий договора закупки.
7. Настройте для папки представление по умолчанию. Для этого переместите колонку **Закупка** из области **Скрытые колонки** в табличную часть.

   СОВЕТ. Чтобы изменить порядок отображения колонок в содержимом папки, переместите их заголовки в табличной части.

Чтобы доработать панель фильтрации для вычисляемой папки «Реестр договоров»:

1. В узле ContractsList выберите элемент «FilterPanel».



2. Выделите группу **Типы документов** (DocumentTypeGroup) и добавьте флажок **Договоры закупки** (PurchaseContracts).

   ПРИМЕЧАНИЕ. В примере к имени флажка автоматически добавляется постфикс с кодом компании (DEV), так как при установке среды разработки был установлен флажок **Уникальные имена при перекрытии**.

3. В параметрах установите флажок **Выделен по умолчанию**. Результат настройки панели фильтрации будет отображаться в области предпросмотра.

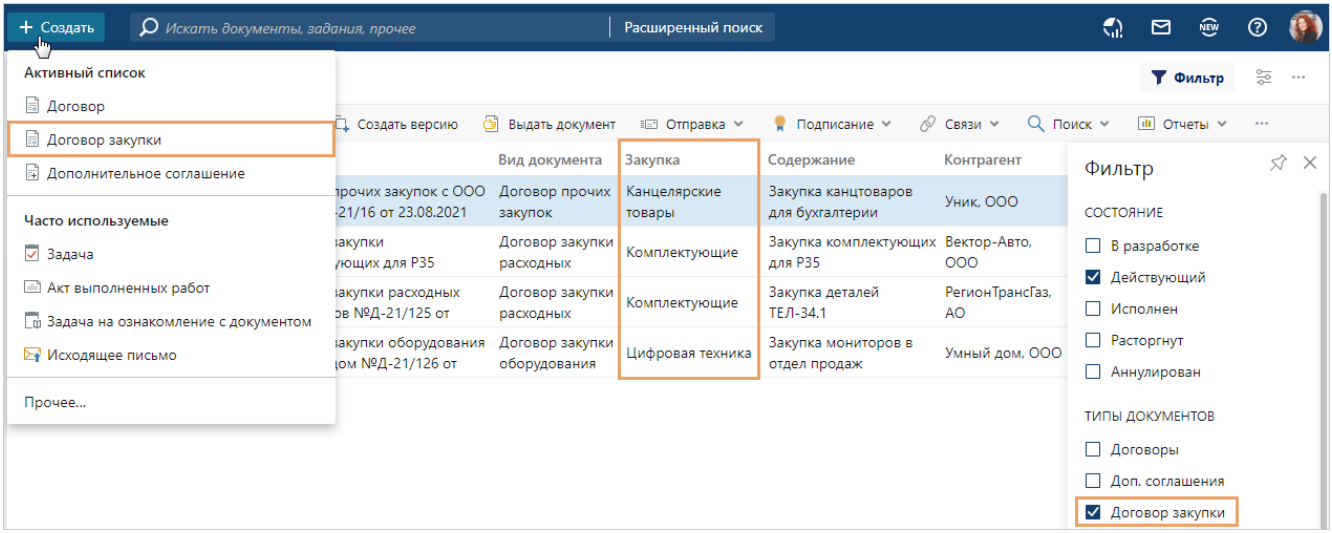
4. Вернитесь в узел ContractsList и задайте условие для фильтрации договоров по соответствующей заявке. Для этого установите флажок рядом с событием **Получение данных** и перейдите по ссылке. В открывшемся редакторе напишите код:

```
public override IQueryable<Sungero.Contracts.IContractualDocument>
  ContractsListDataQuery(IQueryable<Sungero.Contracts.IContractualDocument>
  query)
  {
    if (_filter == null)
      return query.Where(d => Sungero.Contracts.Contracts.Is(d) ||
  Sungero.Contracts.SupAgreements.Is(d) || Purchases.PurchaseContracts.Is(d));

    #region Фильтры

    // Фильтр по состоянию.
    var statuses = new List<Enumeration>();
    if (_filter.Draft)
      statuses.Add(Sungero.Contracts.ContractBase.LifeCycleState.Draft);
    if (_filter.Active)
      statuses.Add(Sungero.Contracts.ContractBase.LifeCycleState.Active);
    if (_filter.Executed)
      statuses.Add(Sungero.Contracts.ContractBase.LifeCycleState.Closed);
    if (_filter.Terminated)
      statuses.Add(Sungero.Contracts.ContractBase.LifeCycleState.Terminated);
    if (_filter.Cancelled)
```

```
    statuses.Add(Sungero.Contracts.ContractBase.LifeCycleState.Obsolete);

  // Фильтр по состоянию.
  if (statuses.Any())
    query = query.Where(q => q.LifeCycleState != null &&
statuses.Contains(q.LifeCycleState.Value));

  // Фильтр "Вид документа".
  if (_filter.DocumentKind != null)
    query = query.Where(c => Equals(c.DocumentKind, _filter.DocumentKind));

  // Фильтр "Категория".
  if (_filter.Category != null)
    query = query.Where(c => Sungero.Contracts.Contracts.Is(c) &&
Equals(c.DocumentGroup, _filter.Category));

  // Фильтр "Контрагент".
  if (_filter.Contractor != null)
    query = query.Where(c => Equals(c.Counterparty, _filter.Contractor));

  // Фильтр "Наша организация".
  if (_filter.BusinessUnit != null)
    query = query.Where(c => Equals(c.BusinessUnit, _filter.BusinessUnit));

  // Фильтр "Подразделение".
  if (_filter.Department != null)
    query = query.Where(c => Equals(c.Department, _filter.Department));

  #region Фильтрация по дате договора

  var beginDate = Calendar.UserToday.AddDays(-30);
  var endDate = Calendar.UserToday;

  if (_filter.Last365days)
    beginDate = Calendar.UserToday.AddDays(-365);

  if (_filter.ManualPeriod)
  {
    beginDate = _filter.DateRangeFrom ?? Calendar.SqlMinValue;
    endDate = _filter.DateRangeTo ?? Calendar.SqlMaxValue;
  }

  var serverPeriodBegin = Equals(Calendar.SqlMinValue, beginDate) ? beginDate
 :
Sungero
.Docflow.PublicFunctions.Module.Remote.GetTenantDateTimeFromUserDay(beginDate);
  var serverPeriodEnd = Equals(Calendar.SqlMaxValue, endDate) ? endDate :
endDate.EndOfDay().FromUserTime();
  var clientPeriodEnd = !Equals(Calendar.SqlMaxValue, endDate) ?
endDate.AddDays(1) : Calendar.SqlMaxValue;
  query = query.Where(j => (j.DocumentDate.Between(serverPeriodBegin,
serverPeriodEnd) ||
                                    j.DocumentDate == beginDate) &&
j.DocumentDate != clientPeriodEnd);

  #endregion
```

```
    // Фильтр по типу документа.
    if (!(_filter.Contracts == true && _filter.SupAgreements == true &&
_filter.PurchaseContractDEV == true) || !(_filter.Contracts == false &&
_filter.SupAgreements == false && _filter.PurchaseContractDEV == false))
    {
      if (_filter.Contracts)
        query = query.Where(d => Sungero.Contracts.Contracts.Is(d));

      if (_filter.SupAgreements)
        query = query.Where(d => Sungero.Contracts.SupAgreements.Is(d));

      if (_filter.PurchaseContractDEV)
        query = query.Where(d => Purchases.PurchaseContracts.Is(d));
    }
     else
      query = query.Where(d => Sungero.Contracts.ContractBases.Is(d) ||
Sungero.Contracts.SupAgreements.Is(d) || Purchases.PurchaseContracts.Is(d));

    #endregion

    return query;
  }
```

В результате после публикации реестр договоров будет выглядеть следующим образом:



Шаг 4. Настройка обложки модуля

ВАЖНО. Обложки рекомендуется настраивать в веб-клиенте. Их редактирование в среде разработки оставлено для совместимости.

Чтобы в веб-клиенте настроить обложку модуля:

1. Из среды разработки запустите веб-клиент *локальной системы*.
2. В справочнике **Представления модулей** создайте запись для модуля «Закупки» и настройте обложку. Подробнее см. в руководстве администратора, разделы «Настройка модулей проводника», «Пример настройки модуля и стартовой страницы».

3. Представление модуля сохраните как настройку по умолчанию. Для этого в карточке представления перейдите на вкладку «Хранение» и установите флажок **Настройка по умолчанию**. В поле **Сохранить в модуле системы** выберите модуль, к которому относится настройка. Подробнее см. в руководстве администратора, раздел «Типы настроек и их жизненный цикл».
4. Сохраните карточку представления и обновите страницу браузера.

В результате в веб-клиенте локальной системы на панели навигации появляется созданный модуль, при переходе на него отображается настроенная обложка.

Шаг 5. Инициализация модуля

Чтобы пользователь смог работать с модулем, необходимо создать предопределенные данные. В данном случае необходимо:

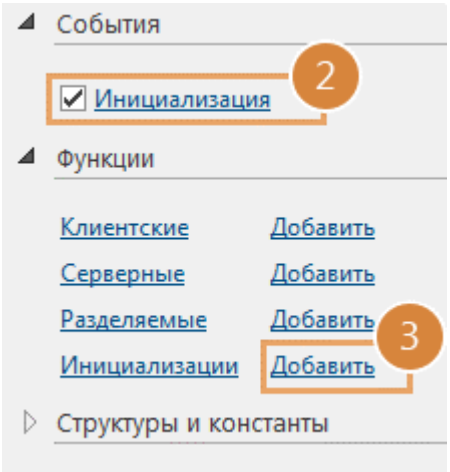
- создать виды закупок по умолчанию
- создать виды документов для договоров закупки
- выдать права на создание договоров закупки и просмотр справочников

Для этого используйте функции инициализации.

Создание видов закупок по умолчанию

Чтобы создать записи в справочнике **Виды закупок**:

1. Откройте редактор модуля «Закупки».
2. В узле «Модуль», в группе «События» установите флажок рядом с событием **Инициализация**.



3. Добавьте функцию инициализации модуля «Закупки». Для этого в группе «Функции» перейдите по ссылке **Добавить** рядом с функциями инициализации. В открывшемся редакторе измените имя **Function1()** на **CreateDefaultPurchaseKind()** и напишите код создания вида закупки. Добавьте функцию **CreateDefaultPurchaseKinds()** и вызовите в ней ранее созданную функцию с необходимыми именами видов закупки. После этого вызовите созданную функцию **CreateDefaultPurchaseKinds()** в обработчике события **Initializing()**:

```
/// <summary>
    /// Обработчик события инициализации.
    /// </summary>
```

```
public override void Initializing(Sungero.Domain.ModuleInitializingEventArgs
 e)
{
  CreateDefaultPurchaseKinds();
}
/// <summary>
/// Создание видов закупок по умолчанию.
/// </summary>
public static void CreateDefaultPurchaseKinds()
{
  InitializationLogger.Debug("Init: Create default purchase kinds.");
  CreatePurchaseKind("Закупка у единственного поставщика");
  CreatePurchaseKind("Закупка по прямому договору");
  CreatePurchaseKind("Закупка по конкурентным переговорам");
}
/// <summary>
/// Создание вида закупки.
/// </summary>
/// <param name="name">Наименование вида закупки.</param>
public static void CreatePurchaseKind(string name)
{
  // Функция выводит на экран создаваемый тип закупки.
  InitializationLogger.DebugFormat("Init: Create purchase kind {0}", name);
  var purchaseKind = PurchaseKinds.GetAll().Where(p => Equals(p.Name,
name)).FirstOrDefault();
  // Проверка, есть ли такой вид в системе.
  if (purchaseKind == null)
  {
    purchaseKind = PurchaseKinds.Create();
    purchaseKind.Name = name;
    purchaseKind.Save();
  }
}
```

При повторном запуске инициализации копии записей справочника не создадутся.

Создание видов документов для договора закупки

Для классификации документов в системе создаются виды документов. Для разработанного типа документа «Договор закупки» нужно создать три вида документа по умолчанию: договор закупки расходных материалов, договор закупки оборудования, договор прочих закупок.

Перед созданием видов договоров закупки необходимо создать тип документа «Договор закупки» в служебном справочнике «Типы документов». Запись справочника понадобится при создании видов документа. Запись в служебном справочнике можно создать только программно, поэтому необходимо использовать функции инициализации. Для этого:

1. В редакторе модуля «Закупки» перейдите по ссылке **Добавить** рядом с функциями инициализации. Измените имя **Function1()** на **CreateDocumentTypes()** и вызовите функцию **CreateDocumentType()** из модуля Sungero.Docflow *базового решения* Directum RX:

```
/// <summary>
/// Создание типа документа "Договор закупки" в служебном справочнике "Типы
документов".
```

```
/// </summary>
public static void CreateDocumentTypes()
{
  Sungero
.Docflow.PublicInitializationFunctions.Module.CreateDocumentType("Договор
закупки", PurchaseContract.ClassTypeGuid,
Sungero.Docflow.DocumentType.DocumentFlow.Contracts, true);
}
```

ПРИМЕЧАНИЕ. Так как в программном коде используется функция модуля Sungero.Docflow, добавьте данный модуль в список зависимостей вашего модуля. Если зависимости не указаны, то при публикации могут возникнуть ошибки.

2. В узле «Модуль», в группе «События» перейдите по ссылке **Инициализация**. В редакторе программного кода вызовите функцию **CreateDocumentTypes()** в обработчике события **Initializing()**:

```
/// <summary>
/// Обработчик события инициализации.
/// </summary>
public override void Initializing(Sungero.Domain.ModuleInitializingEventArgs
 e)
{
  CreateDefaultPurchaseKinds();
  CreateDocumentTypes();
}
```

Чтобы создать виды документов для договора закупки:

1. Создайте константы в модуле «Закупки», они будут использоваться в функции **CreateDocumentKind()**. Значения констант сгенерируйте случайным образом. Они должны быть уникальными.

```
public static readonly Guid PurchaseOfConsumablesKind = Guid.Parse("2B45D377-
62A8-4F5F-8710-0D55068F5FF6");
public static readonly Guid PurchaseOfEquipmentKind = Guid.Parse("0D307CCF-
F948-4B0F-B2A8-A273BB130AE5");
public static readonly Guid OtherPurchaseKind = Guid.Parse("51630A97-B502-
415F-BF0E-07DF3AE1C316");
```

2. Добавьте функцию инициализации модуля «Закупки» (**CreateDocumentKinds()**) и вызовите в ней функцию **CreateDocumentKind()** из модуля Sungero.Docflow *базового решения* Directum RX:

```
/// <summary>
/// Создание видов документов для договора закупки.
/// </summary>
public static void CreateDocumentKinds()
{
  Sungero
.Docflow.PublicInitializationFunctions.Module.CreateDocumentKind("Договор
закупки расходных материалов", " Договор закупки расходных материалов",
    Sungero.Docflow.DocumentKind.NumberingType.Registrable,
    Sungero.Docflow.DocumentType.DocumentFlow.Contracts,
    true, false, PurchaseContract.ClassTypeGuid,
    new Sungero.Domain.Shared.IActionInfo[]
{ Sungero.Docflow.OfficialDocuments.Info.Actions.SendForApproval },
    Constants.Module.PurchaseOfConsumablesKind);
```

```
  Sungero
.Docflow.PublicInitializationFunctions.Module.CreateDocumentKind("Договор
закупки оборудования", "Договор закупки оборудования",
    Sungero.Docflow.DocumentKind.NumberingType.Registrable,
    Sungero.Docflow.DocumentType.DocumentFlow.Contracts,
    true, false, PurchaseContract.ClassTypeGuid,
    new Sungero.Domain.Shared.IActionInfo[]
{ Sungero.Docflow.OfficialDocuments.Info.Actions.SendForApproval },
    Constants.Module.PurchaseOfEquipmentKind);

  Sungero
.Docflow.PublicInitializationFunctions.Module.CreateDocumentKind("Договор
прочих закупок"," Договор прочих закупок",
    Sungero.Docflow.DocumentKind.NumberingType.Registrable,
    Sungero.Docflow.DocumentType.DocumentFlow.Contracts,
    true, false, PurchaseContract.ClassTypeGuid,
    new Sungero.Domain.Shared.IActionInfo[]
{ Sungero.Docflow.OfficialDocuments.Info.Actions.SendForApproval },
    Constants.Module.OtherPurchaseKind);
    InitializationLogger.Debug("Созданы виды документов для договоров
закупки по умолчанию");
}
```

3. Вызовите добавленную функцию в обработчике события **Initializing()** модуля «Закупки»:

```
/// <summary>
/// Обработчик события инициализации.
/// </summary>
public override void Initializing(Sungero.Domain.ModuleInitializingEventArgs
 e)
{
  CreateDefaultPurchaseKinds();
  CreateDocumentTypes();
  CreateDocumentKinds();
}
```

ВАЖНО. Вызывайте функции в том порядке, в котором они должны выполняться.

Выдача прав на создание договора и просмотр справочников

Чтобы сразу после установки системы ответственные за договоры смогли создавать договоры закупки, выдайте им права на создание. Для этого:

1. Создайте функцию инициализации модуля «Закупки» (**GrantCreateRightsOnPurchaseContract()**). Напишите программный код, в котором будут выдаваться права на создание договора закупки для группы «Ответственные за договоры»:

```
/// <summary>
/// Выдача прав на создание договора закупки.
/// </summary>
public static void GrantCreateRightsOnPurchaseContract()
{
  var contractsResponsible = Roles.GetAll().Where(n => n.Sid ==
Sungero
.Docflow.Constants.Module.RoleGuid.ContractsResponsible).FirstOrDefault();
  // Проверка наличия роли.
```

```
    if (contractsResponsible != null)
    {
    // Если роль есть, то выдать права на создание договора закупки.
      PurchaseContracts.AccessRights.Grant(contractsResponsible,
DefaultAccessRightsTypes.Create);
      PurchaseContracts.AccessRights.Save();
      InitializationLogger.Debug("Выданы права на создание договоров
закупки");
    }
  }
```

2. Создайте функцию инициализации **GrantAccessRightsForDatabook()**, в которой будут выдаваться права на справочники **Закупки** и **Виды закупок**:

```
public void GrantAccessRightsForDatabook()
  {
    var allUsers = Roles.AllUsers;
    PurchaseKinds.AccessRights.Grant(allUsers, DefaultAccessRightsTypes.Read);
    PurchaseKinds.AccessRights.Save();

    var contractsResponsible = Roles.GetAll().Where(n => n.Sid ==
Sungero
.Docflow.Constants.Module.RoleGuid.ContractsResponsible).FirstOrDefault();
    // Проверка наличия роли.
    if (contractsResponsible != null)
    {
      Purchases.AccessRights.Grant(contractsResponsible,
DefaultAccessRightsTypes.Create);
      Purchases.AccessRights.Grant(contractsResponsible,
DefaultAccessRightsTypes.Change);
      Purchases.AccessRights.Save();
    }
  }
```

3. Вызовите добавленные функции в обработчике события **Initializing()** модуля «Закупки»:

```
/// <summary>
  /// Обработчик события инициализации.
  /// </summary>
  public override void Initializing(Sungero.Domain.ModuleInitializingEventArgs
   e)
  {
    CreateDefaultPurchaseKinds();
    CreateDocumentTypes();
    CreateDocumentKinds();
    GrantCreateRightsOnPurchaseContract();
    GrantAccessRightsForDatabook();
  }
```

Шаг 6. Добавление роли согласования

ПРИМЕЧАНИЕ. Шаг выполняйте, если планируете использовать согласование по регламенту. При настройке согласования по процессу этот и следующий шаг пропустите. Подробнее см. в руководстве администратора, раздел «Настройка схем бизнес-процессов».

Рассмотрим, как создать роль согласования «Инициатор закупки».

Роль согласования – справочник системы, записи которого доступны для выбора в этапах согласования. При создании роли необходимо:

- добавить в системный справочник новый тип роли «Инициатор закупки»;
- добавить логику вычисления сотрудника для роли «Инициатор закупки»;
- создать роль «Инициатор закупки» в справочнике;
- указать, для каких типов этапов согласования доступна роль;
- добавить проверку того, что роль в правиле согласования используется только с определенными видами документов.

  ВАЖНО. Создание правил согласования доступно только для тех видов документов, для которых разрешена отправка на согласование по регламенту: в карточке вида документа в поле **Действия по отправке** указано значение **Согласование по регламенту**.

Вычисление сотрудника для роли «Инициатор закупки»

1. Чтобы добавить в системный справочник новый тип роли «Инициатор закупки», создайте справочник «Роль инициатор закупки». Имя типа справочника – PurchaseApprovalRole, в качестве базового типа сущности укажите тип Sungero.Docflow.ApprovalRoleBase. Проверьте, что в параметрах типа справочника снят флажок **Отображать в проводнике**.
2. В редакторе созданного типа справочника перейдите в узел «Свойства» и для свойства-перечисления Type добавьте значение перечисления PurchInitiator (Инициатор закупки).
3. Чтобы в процессе согласования документов для роли вычислялся инициатор закупки, переопределите логику функции **GetRolePerformer()** из модуля Sungero.Docflow *базового решения* Directum RX. Для этого в серверных функциях созданного справочника (PurchaseApprovalRole) напишите код:

   ```
   public override Sungero.Company.IEmployee GetRolePerformer(Sungero.Docflow.IApprovalTask task)
   {
       if (_obj.Type == Trade.Purchases.PurchaseApprovalRole.Type.PurchInitiator)
       {
         var document = task.DocumentGroup.OfficialDocuments.FirstOrDefault();
         var purchaseContract = PurchaseContracts.As(document);
         if (purchaseContract != null)
           return purchaseContract.Purchase.Initiator;

         return null;
       }

       return base.GetRolePerformer(task);
   }
   ```

   ВАЖНО. Замените в коде Trade.Purchases на <Ваш код компании>. Purchases.
4. Так как в программном коде вы переопределили функцию модуля Sungero.Docflow, проверьте, что данный модуль указан в списке зависимостей вашего модуля. При необходимости добавьте его. Если зависимости не указаны, при публикации могут возникнуть ошибки.

Создание роли

Роль согласования – служебный справочник, его записи нельзя создать в клиентском приложении. Чтобы создать роль:

1. В редакторе модуля «Закупки» перейдите по ссылке **Добавить** рядом с функциями инициализации. Измените имя **Function1()** на **CreateApprovalRole()** и напишите программный код для создания роли:

```
/// <summary>
/// Создание роли.
/// </summary>
public static void CreateApprovalRole(Enumeration roleType, string
description)
{
    var role = PurchaseApprovalRoles.GetAll().Where(r => Equals(r.Type,
roleType)).FirstOrDefault();
    // Проверяет наличие роли.
    if (role == null)
    {
      role = PurchaseApprovalRoles.Create();
      role.Type = roleType;
    }
    role.Description = description;
    role.Save();
    InitializationLogger.Debug("Создана роль 'Инициатор закупки'");
}
```

2. В узле «Модуль» в группе «События» перейдите по ссылке **Инициализация**. В редакторе программного кода вызовите функцию **CreateApprovalRole()** в обработчике события **Initializing()**:

```
/// <summary>
/// Обработчик события инициализации.
/// </summary>
public override void Initializing(Sungero.Domain.ModuleInitializingEventArgs
 e)
{
  CreateDefaultPurchaseKinds();
  CreateDocumentTypes();
  CreateDocumentKinds();
  GrantCreateRightsOnPurchaseContract();
  GrantAccessRightsForDatabook();
  CreateApprovalRole(Trade.Purchases.PurchaseApprovalRole.Type.
                     PurchInitiator, "Инициатор закупки.");
}
```

ВАЖНО. Замените в коде Trade.Purchases на <Ваш код компании>.Purchases.

Ограничение доступности роли по типам этапов согласования

Роль «Инициатор закупки» должна быть доступной для выбора в следующих этапах согласования: «Согласование», «Задание», «Уведомление».

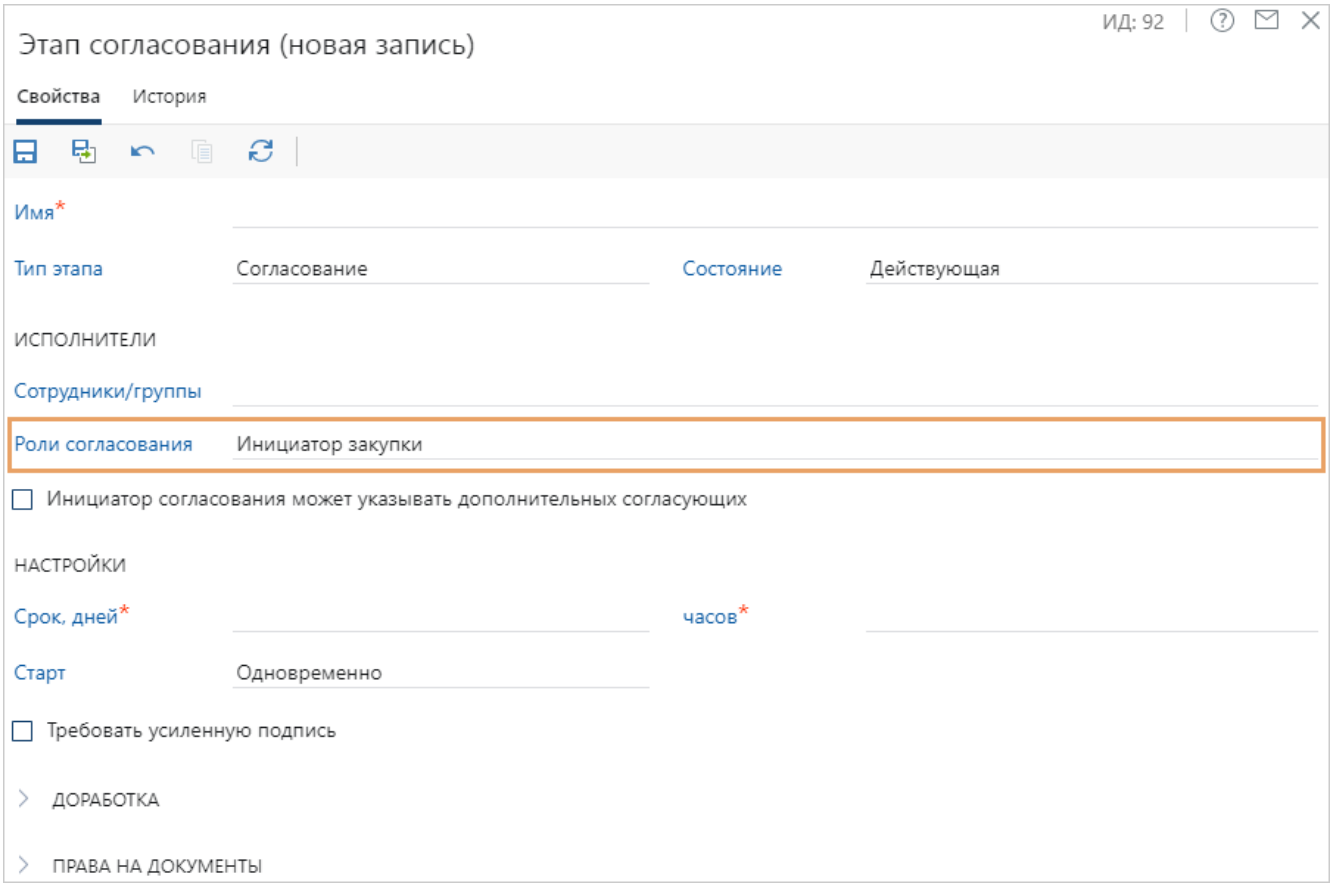
Настройте доступность роли в справочнике **Этапы согласования** (ApprovalStage). Для этого:

1. Перекройте справочник Sungero.Docflow.ApprovalStage. Проверьте, что в списке зависимостей вашего решения указан модуль Sungero.Docflow. При необходимости добавьте его. Если в решении не указаны зависимости от перекрытых модулей, то при публикации могут возникнуть ошибки.
2. Переопределите разделяемую функцию **GetPossibleRoles()**. Для этого в редакторе перекрытого справочника добавьте код разделяемой функции:

```
public override List<Enumeration?> GetPossibleRoles()
{
  var baseRoles = base.GetPossibleRoles();

  if (_obj.StageType == Sungero.Docflow.ApprovalStage.StageType.Approvers ||
      _obj.StageType == Sungero.Docflow.ApprovalStage.StageType.SimpleAgr ||
      _obj.StageType == Sungero.Docflow.ApprovalStage.StageType.Notice)
  {
    baseRoles.Add(Trade.Purchases.PurchaseApprovalRole.Type.PurchInitiator);
  }

  return baseRoles;
}
```

ВАЖНО. Замените в коде Trade.Purchases на <Ваш код компании>.Purchases.

В результате после инициализации системы новую роль можно выбирать в заданных этапах согласования:



Ограничение доступности роли по видам документов

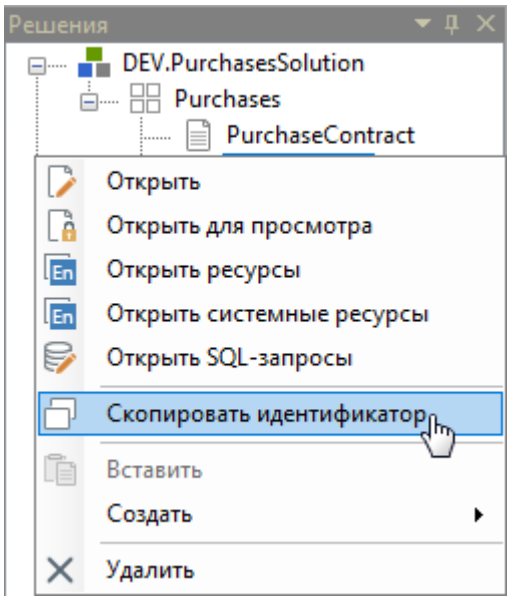
Роль «Инициатор закупки» нужна только при согласовании договоров закупки с видом документа: договор закупки расходных материалов, договор закупки оборудования, договор прочих закупок. Поэтому необходимо добавить проверку того, что роль в правиле согласования используется только с данными видами документов.

Для этого:

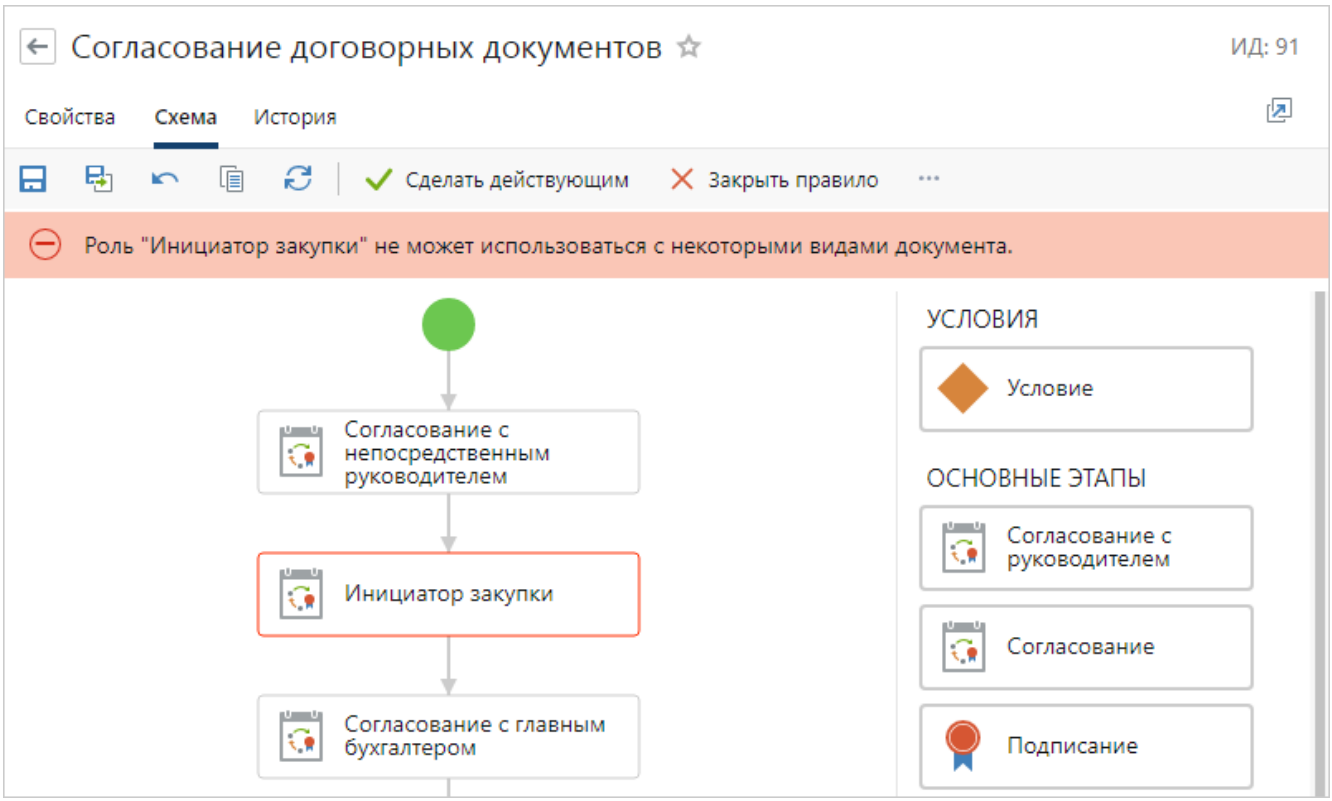
1. Переопределите разделяемую функцию **Filter()**. Для этого в редакторе справочника **Роль инициатор закупки** (PurchaseApprovalRole) добавьте код:

```
public override List<Sungero.Docflow.IDocumentKind>
Filter(List<Sungero.Docflow.IDocumentKind> kinds)
{
  var query = base.Filter(kinds);
  if (_obj.Type == Trade.Purchases.PurchaseApprovalRole.Type.PurchInitiator)
    query = query.Where(k => k.DocumentType.DocumentTypeGuid == "9159dcfe-
e779-4cca-95ce-c4c59b77c6c0").ToList();
  return query;
}
```

2. Замените:
   - код Trade.Purchases на <Ваш код компании>.Purchases;
   - идентификатор "9159dcfe-e779-4cca-95ce-c4c59b77c6c0" на идентификатор (GUID) типа документа «Договор закупки». Для этого выделите тип документа «Договор закупки» (PurchaseContract) в дереве решений и в контекстном меню выберите пункт **Скопировать идентификатор**:



В результате при сохранении правила согласования будет вызываться данная функция и проверяться доступность роли по видам документов:

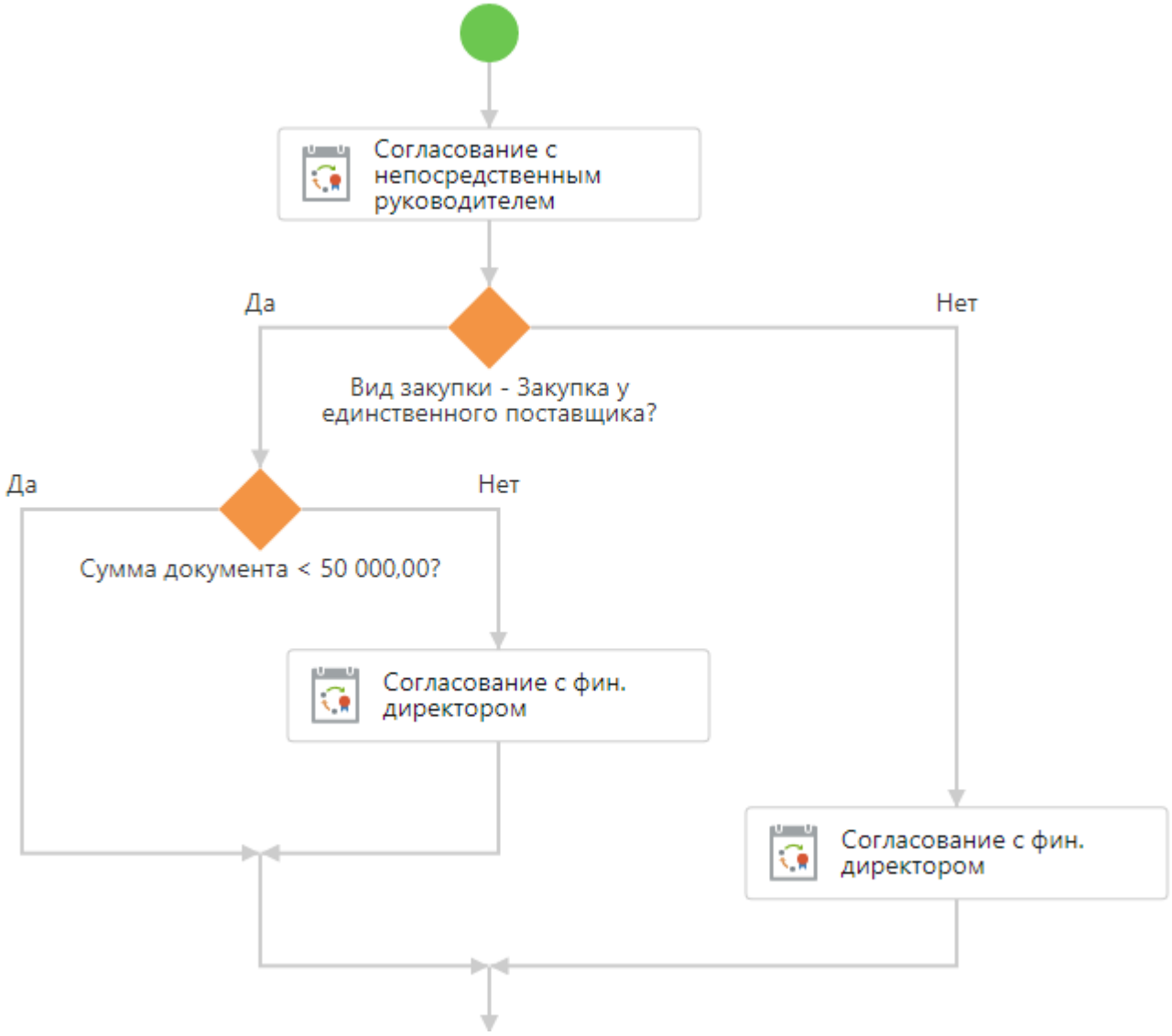


Шаг 7. Добавление условия согласования

ПРИМЕЧАНИЕ. Шаг выполняйте, если планируете использовать согласование по регламенту. При настройке согласования по процессу этот шаг пропустите. Подробнее см. в руководстве администратора, раздел «Настройка схем бизнес-процессов».

Рассмотрим, как создать условие согласования «Вид закупки равен такому-то».

В системе для договоров закупки нужно создать правило согласования и настроить схему. При этом следует учесть, что договоры закупки у единственного поставщика с суммой меньше 50 тыс. рублей не согласовывает финансовый директор:

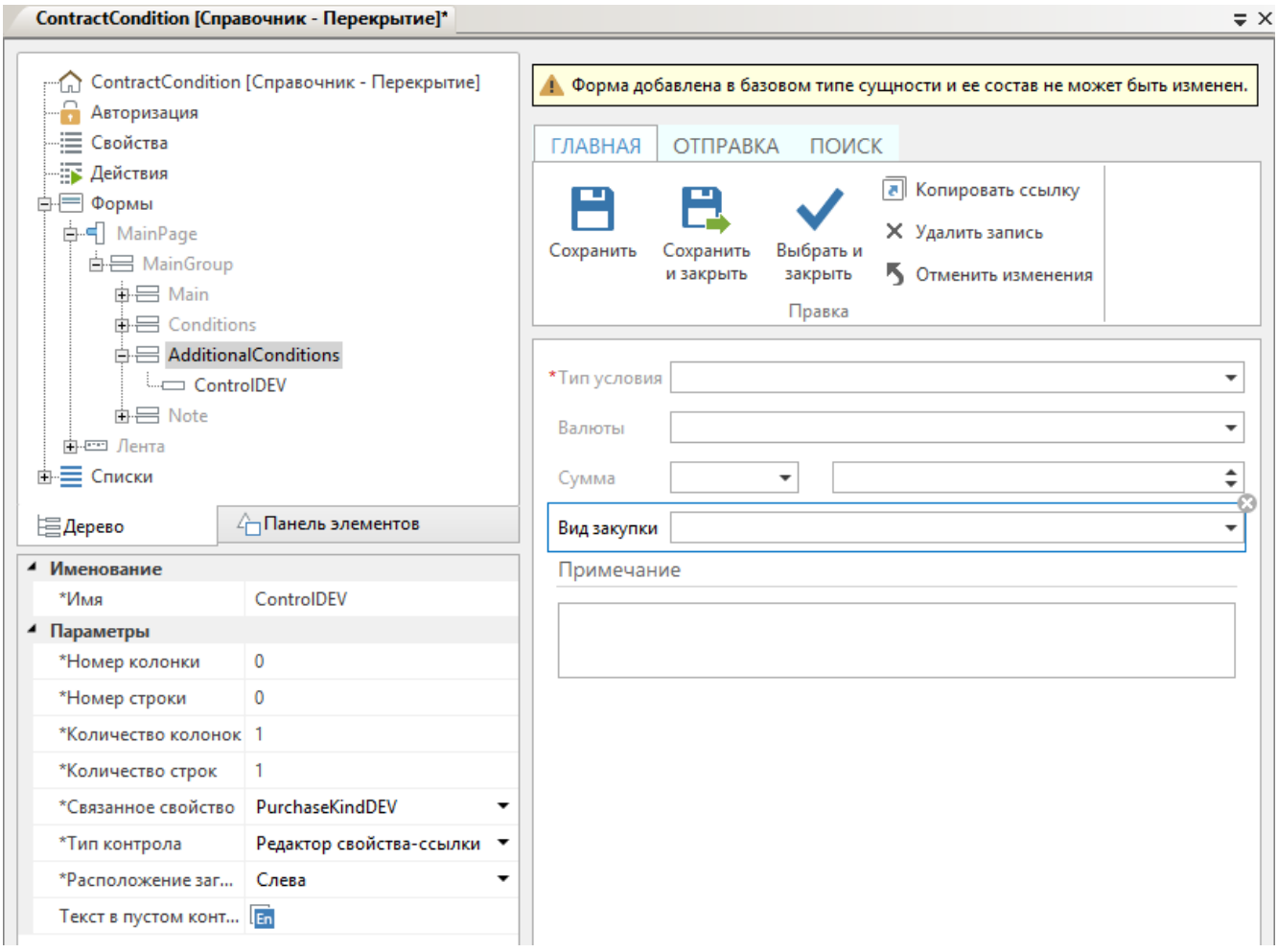


Чтобы администратор мог настроить такую схему:

1. Создайте новый тип условия «Вид закупки».
2. Привяжите условие в правиле согласования для определенных видов документов.
3. Доработайте проверку веток схемы перед ее валидацией.
4. Добавьте логику вычисления условия.
5. Настройте заполнение отображаемого имени условия в схеме.

Создание типа условия «Вид закупки»

1. Перекройте справочник «Договорное условие» (Sungero.Contracts.ContractCondition). Проверьте, что в списке зависимостей вашего решения указан модуль Sungero.Contracts. При необходимости добавьте его. Если в решении не указаны зависимости от перекрытых модулей, то при публикации могут возникнуть ошибки.
2. В редакторе перекрытого справочника перейдите в узел «Свойства» и для свойства-перечисления ConditionType добавьте значение перечисления PurchaseKind (Вид закупки).
3. Создайте свойство Вид закупки. Имя свойства – PurchaseKind. Тип свойства – ссылка на справочник «Вид закупки» (PurchaseKind).
4. В карточку справочника добавьте контрол и свяжите с созданным свойством. Чтобы добавить свойство в группу **AdditionalConditions**, выделите группу в дереве и снимите флажок **Использовать контролы предка**. В результате карточка справочника примет следующий вид:

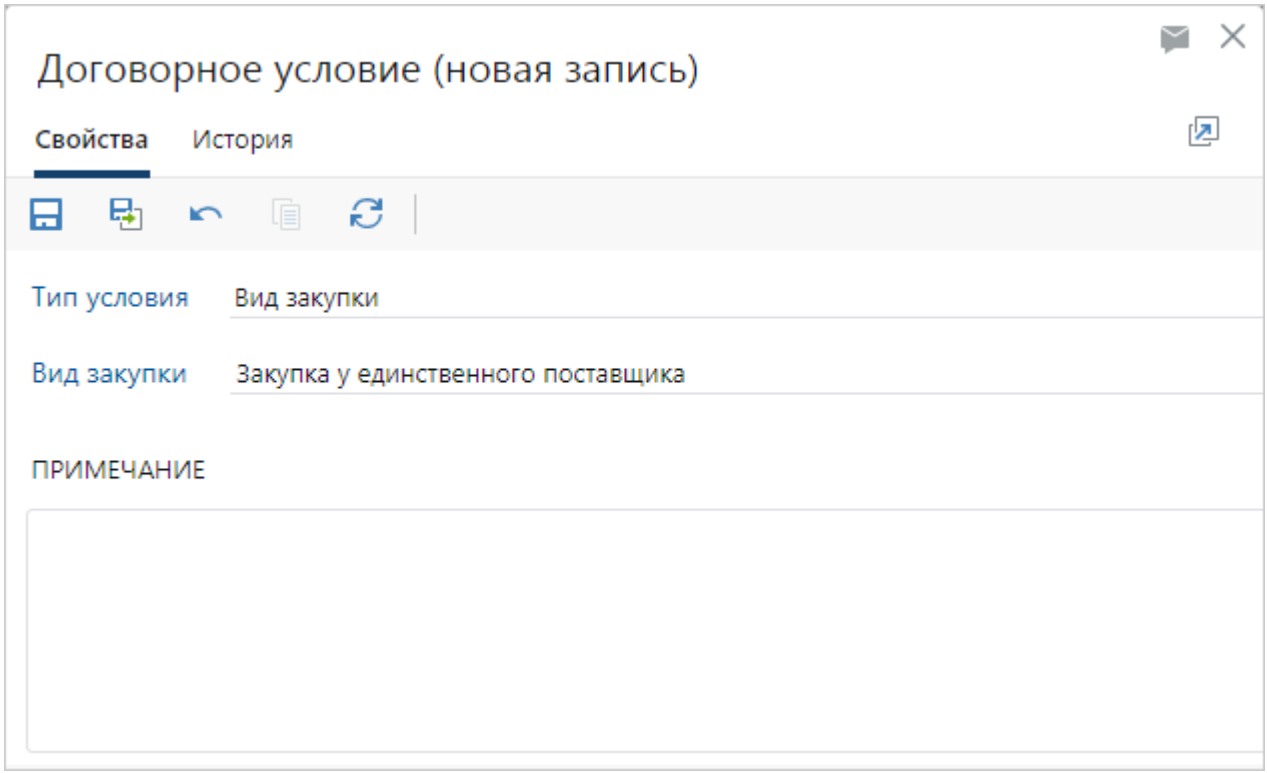


5. Видимость полей на карточке условия согласования в Directum RX изменяется динамически. В зависимости от типа условия на форме доступны поля для заполнения тех или иных свойств. Чтобы управлять доступностью и очищать значение поля **Вид закупки**, когда оно недоступно, в разделяемом коде справочника ContractCondition переопределите разделяемые функции ChangePropertiesAccess() и ClearHiddenProperties():

```
public override void ChangePropertiesAccess()
```

```
{
  base.ChangePropertiesAccess();

  var isPurchase = _obj.ConditionType == ConditionType.PurchaseKind;

  _obj.State.Properties.PurchaseKind.IsVisible = isPurchase;
  _obj.State.Properties.PurchaseKind.IsRequired = isPurchase;
}

public override void ClearHiddenProperties()
{
  base.ClearHiddenProperties();

  if (!_obj.State.Properties.PurchaseKind.IsVisible)
    _obj.PurchaseKind = null;
}
```

В результате при настройке правила согласования администратор сможет добавить тип условия «Вид закупки». Карточка условия будет выглядеть следующим образом:



Привязка условия согласования к нужным типам и видам документов

При настройке схемы согласования можно использовать несколько типов условий. Каждый тип используется для настройки согласования определенных видов документов. При создании вид документа привязывается к одному типу документа. Поэтому с помощью этого типа можно получить список доступных видов документов и настроить фильтрацию для типов условий.

Настройте привязку условия согласования к типу документа, если условие:

- создается на основании реквизита, который есть не во всех видах документов. Например, в карточке приказа не указывается контрагент, поэтому тип условия «Контрагент-нерезидент» к нему не применяется;
- вычисляется только для определенных видов документов. Например, проверять, является ли договор типовым, имеет смысл только при согласовании договоров и дополнительных соглашений. Поэтому тип условия «Типовой» можно использовать только в схемах согласования договоров и дополнительных соглашений.

Созданное условие нужно сделать доступным только в схемах согласования для видов документов: договоров закупки расходных материалов, договоров закупки оборудования и договоров прочих закупок. Для этого:

1. В разделяемом коде справочника ContractCondition переопределите функцию GetSupportedConditions(). Она проверяет, какие типы условий можно использовать в правиле согласования:

```
public override System.Collections.Generic.Dictionary<string, List<Enumeration?
    >> GetSupportedConditions()
    {
      var baseSupport = base.GetSupportedConditions();

      baseSupport["9159dcfe-e779-4cca-95ce-
    c4c59b77c6c0"].Add(ConditionType.PurchaseKind);

      return baseSupport;
    }
```

2. Замените идентификатор "9159dcfe-e779-4cca-95ce-c4c59b77c6c0" на идентификатор типа документа «Договор закупки». Для этого выделите тип документа «Договор закупки» (PurchaseContract) в дереве решений и в контекстном меню выберите пункт **Скопировать идентификатор**.

В результате правило согласования с условием по виду закупки можно добавить на схему и сохранить, только если в правиле указаны виды документов: договор закупки расходных материалов, договор закупки оборудования и договор прочих закупок.

ПРИМЕЧАНИЕ. Если функцию не переопределить, то правило с таким условием сохранить нельзя. Если функция переопределена, но при сохранении ошибка возникла, тогда проверьте, что виды документов, указанные в логике вычисления условия, совпадают с видами документов:

- в функции **GetSupportedConditions()**;
- на закладке «Свойства» в поле **Виды документов**.

Проверка веток перед валидацией

Предполагается, что на данный момент компания будет использовать простую схему согласования договоров закупок с двумя условиями. Однако в будущем в схеме появится больше условий, она усложнится. Поэтому нужно заранее подумать об оптимизации сохранения схемы, а именно доработать проверку веток схемы перед валидацией. Для этого:

1. Перекройте справочник **Правило согласования договоров** (Sungero.Contracts.ContractsApprovalRule). Проверьте, что в списке зависимостей

вашего решения указан модуль Sungero.Contracts. При необходимости добавьте его. Если в решении не указаны зависимости от перекрытых модулей, то при публикации могут возникнуть ошибки.

2. Чтобы проверить, какие ветки в правиле согласования нужно валидировать перед сохранением, переопределите функцию CheckRoutePossibility(). Для этого в серверных функциях перекрытого справочника напишите код:

```
// Проверка возможности существования ветки при добавлении в нее нового условия.
public override bool CheckRoutePossibility(List<Structures.ApprovalRuleBase.RouteStep> route,
  List<Structures.ApprovalRuleBase.ConditionRouteStep> ruleConditions,
  Structures.ApprovalRuleBase.RouteStep conditionStep)
{
  // Получить список допустимых веток по базовой логике.
  var possibleStage = base.CheckRoutePossibility(route, ruleConditions,
conditionStep);
  var conditionType = _obj.Conditions.First(c => c.Number ==
conditionStep.StepNumber).Condition.ConditionType;

  // Проверить условия по виду закупки.
  if (conditionType ==
DEV.DevelopmentExample.ContractCondition.ConditionType.PurchaseKind)
  {
    // Получить все условия по виду закупки из ветки.
    var purchaseKindConditions =
this.GetPurchaseKindConditionsInRoute(route).Where(c => c.StepNumber !=
conditionStep.StepNumber).ToList();
    // Определить допустимость ветки с точки зрения условий по виду закупки.
    possibleStage = this.CheckPurchaseKindConditions(purchaseKindConditions,
 conditionStep);
  }
  return possibleStage;
}
```

3. В редакторе перекрытого справочника добавьте серверную функцию **GetPurchaseKindConditionsInRoute()**, чтобы найти все условия по виду закупки в ветке:

```
// Получить все условия по виду закупки в данной ветке.
public List<Structures.ApprovalRuleBase.RouteStep>
GetPurchaseKindConditionsInRoute(List<Structures.ApprovalRuleBase.RouteStep>
 route)
{
  return route.Where(e => _obj.Conditions.Any(c => Equals(c.Number,
e.StepNumber) && c.Condition.ConditionType ==
DEV
.DevelopmentExample.ContractCondition.ConditionType.PurchaseKind)).ToList();
}
```

4. В редакторе перекрытого справочника добавьте серверную функцию **CheckPurchaseKindConditions()**. Когда в схеме попадется условие, функция анализирует, были ли условия такого же типа раньше. Если были, то сравнивает переход из предыдущего условия с вариантами перехода из проверяемого условия. Переходы, которые противоречат друг другу исключаются.

```
// Сравнить условия в схеме и проверить,
```

```
// какие ветки правила согласования нужно валидировать.
public bool
CheckPurchaseKindConditions
(List<Sungero.Docflow.Structures.ApprovalRuleBase.RouteStep> allConditions,
Sungero.Docflow.Structures.ApprovalRuleBase.RouteStep condition)
{
  var conditionItem = _obj.Conditions.Where(x => x.Number ==
condition.StepNumber).FirstOrDefault();
  var contractCondition =
DEV.DevelopmentExample.ContractConditions.As(conditionItem.Condition);
  var purchaseKind = contractCondition.PurchaseKind;
  // Проверить непротиворечивость условия с предыдущими условиями этого же
типа в ветке.
  foreach (var previousCondition in allConditions.TakeWhile(x => !Equals(x,
condition)))
  {
    var previousConditionItem = _obj.Conditions.Where(x => x.Number ==
previousCondition.StepNumber).FirstOrDefault();
    var previousContractCondition =
DEV
.DevelopmentExample.ContractConditions.As(previousConditionItem.Condition);
    var previousPurchaseKind = previousContractCondition.PurchaseKind;
    // Если вид закупки в предыдущем условии такой же, а переход отличается
– ветка не валидна.
    if (purchaseKind == previousPurchaseKind && previousCondition.Branch !=
condition.Branch)
      return false;
  }
  return true;
}
```

Реализация логики вычисления условия

Созданное условие должно проверять, соответствует ли вид закупки, указанный в документе, виду закупки в условии. Проверка происходит при выборе правила согласования в задаче на согласование по регламенту и при прохождении ее этапов.

В перекрытом справочнике ContractCondition переопределите разделяемую функцию **CheckCondition()**. Функция возвращает структуру, которая включает в себя:

- результат вычисления условия:

  true – условие верно.

  false – условие неверно.

  null – не удалось вычислить значение, например, не заполнено одно из свойств или используется не тот тип документа.

- строку с сообщением, в котором указывается причина, почему не вычислено условие.

  ПРИМЕЧАНИЕ. Строка с сообщением необходима, только если результат вычисления условия – **null**.

```
public override Sungero.Docflow.Structures.ConditionBase.ConditionResult
CheckCondition(Sungero.Docflow.IOfficialDocument document,
Sungero.Docflow.IApprovalTask task)
{
```

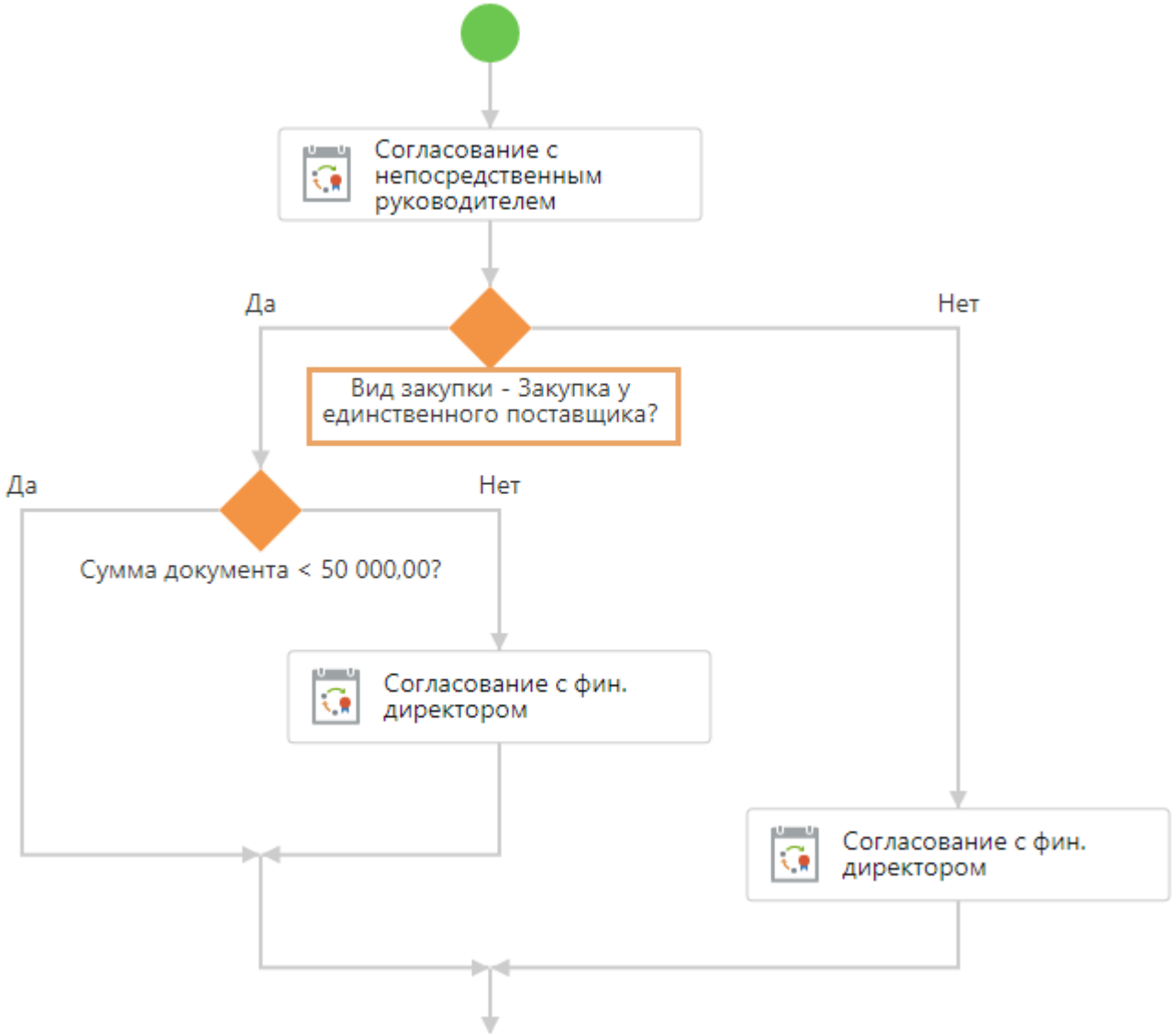
```
    if (_obj.ConditionType == ConditionType.PurchaseKind)
    {
      var contract = Purchases.PurchaseContracts.As(document);
      if (contract != null)
        return
          Sungero.Docflow.Structures.ConditionBase.ConditionResult.
          Create(contract.Purchase.PurchaseKind == _obj.PurchaseKind,
          string.Empty);
      else
        return Sungero.Docflow.Structures.ConditionBase.ConditionResult.
        Create(null, "Условие не может быть вычислено. Отправляемый документ не
  того вида.");
    }
    return base.CheckCondition(document, task);
  }
```

Заполнение отображаемого имени условия в схеме

Чтобы на схеме в правиле согласования и в отчете **Регламент согласования** отображалось имя созданного условия, переопределите серверную функцию GetConditionName(). Функция используется для заполнения свойства **Name** – наименование значения условия. Отображается в схеме правила согласования и в отчете **Регламент согласования документов**.

```
   public override string GetConditionName()
{
    using (TenantInfo.Culture.SwitchTo())
    {
        // значение ресурса PurchaseKindName – "Вид закупки – {0}?"
        if (_obj.ConditionType == ConditionType.PurchaseKind)
            return
ContractConditions.Resources.PurchaseKindNameFormat(_obj.PurchaseKind);
    }
    return base.GetConditionName();
}
```

В результате при настройке схемы правила согласования условие будет выглядеть следующим образом:



Документы

В разделе содержатся примеры:

- Как подготовить новый тип документа к работе
- Как добавить новый тип документа в окно смены типа
- Как изменить длину содержания в карточке документа
- Как изменить формирование имени документа
- Как добавить контрол состояния с информацией о задачах по документу
- Как запретить отправку договора на согласование, если не указана сумма
- Как изменить срок возврата договора при выдаче сотруднику
- Как отфильтровать список шаблонов, доступных при создании документа
- Как настроить склонение имени в шаблонах документов
- Как добавить сводку по новому типу документа
- Как изменить отметку об ЭП

- Как создать постраничную отметку для парафирования договорных документов
- Как создать отметку об оплате входящих счетов
- Как настроить работу редактора отметок
- Как получить данные без учета прав доступа
- Как создать операцию авторизации для типа документа
- Как сгенерировать отчет и записать его в версию документа
- Как настроить шифрование договоров
- Как настроить строгий доступ к документу
- Как запретить получение документов от конкретного филиала из сервиса обмена

Как подготовить новый тип документа к работе

Ситуация

В модуле компании (MyCompany) создан новый тип документа «Информационно-справочные документы» (InfoDocument) на основе типа «Внутренний документ» (Sungero.Docflow.InternalDocumentBase). Необходимо, чтобы в Directum RX сотрудники могли создавать документы нового типа, выбирать его виды, а также регистрировать эти документы.

Решение

Чтобы сотрудники могли работать с документами нового типа, в инициализации модуля необходимо программно добавить новый тип документа в справочник **Типы документов**. Затем создать виды документов для этого типа, выдать права на создание документов всем сотрудникам и запустить инициализацию модулей:

1. В редакторе своего решения добавьте зависимость от модуля «Документооборот» (Sungero.Docflow). Это необходимо, так как для работы с новыми типами и видами документов используются функции модуля Docflow.
2. Добавьте новый тип документа **InfoDocument** в справочник **Типы документов** (Sungero.Docflow.DocumentType). Добавить новый тип в справочник можно только программно с помощью функций инициализации. Для этого в редакторе модуля компании (MyCompany) в группе «Функции» перейдите по ссылке **Добавить** рядом с функциями инициализации. В открывшемся редакторе измените имя функции **Function1()** на **CreateDocumentTypes()** и вызовите в ней базовую функцию **CreateDocumentType()** модуля Sungero.Docflow:

```
/// <summary>
  /// Создание типа документа "Информационно-справочные документы"
  /// в служебном справочнике "Типы документов".
  /// </summary>
  public static void CreateDocumentTypes()
  {
    Sungero
  .Docflow
  .PublicInitializationFunctions.Module.CreateDocumentType("Информационно-
  справочные документы", InfoDocument.ClassTypeGuid,
  Sungero.Docflow.DocumentType.DocumentFlow.Inner, true);
```

```
}
```

3. Создайте новые виды документов для типа «Информационно-справочные документы». Для этого в редакторе модуля создайте константы, в которых будут храниться уникальные идентификаторы для каждого вида документа. В дальнейшем константы используются в функции **CreateDocumentKind()**. Значение констант сгенерируйте случайным образом. Они должны быть уникальными:

```
// Уникальный идентификатор для вида «Объяснительная записка».
[Public]
public static readonly Guid ExplanLettersKind = Guid.Parse("40af4030-eb63-40c3-afef-7fcd26eeceb0");

// Уникальный идентификатор для вида «Заявление».
[Public]
public static readonly Guid StatementsKind = Guid.Parse("dde67eb5-3f84-4293-aaa0-76b33de60bc4");
```

4. Создайте функцию инициализации **CreateDocumentKinds()** и вызовите в ней базовую функцию **CreateDocumentKind()** модуля Sungero.Docflow. В коде базовой функции для каждого вида документа укажите свойство **NumberingType.Registrable**, чтобы документы этих видов были доступны для регистрации:

```
/// <summary>
/// Создание видов для информационно-справочных документов.
/// </summary>
public static void CreateDocumentKinds()
{
// Создание вида документа «Объяснительная записка».
// Чтобы документы можно было регистрировать, задается свойство Registrable.
// В качестве ИД вида документа используется константа ExplanLettersKind.
Sungero
.Docflow
.PublicInitializationFunctions.Module.CreateDocumentKind("Объяснительная записка", "Объяснительная записка",
Sungero.Docflow.DocumentKind.NumberingType.Registrable,
Sungero.Docflow.DocumentType.DocumentFlow.Inner, true, false,
InfoDocument.ClassTypeGuid,
new Sungero.Domain.Shared.IActionInfo[]
{ Sungero.Docflow.OfficialDocuments.Info.Actions.SendForApproval },
Constants.Module.ExplanLettersKind);

// Создание вида документа «Заявление».
// Чтобы документы можно было регистрировать, задается свойство Registrable.
// В качестве ИД вида документа используется константа StatementsKind.
Sungero
.Docflow
.PublicInitializationFunctions.Module.CreateDocumentKind("Заявление",
"Заявление",
Sungero.Docflow.DocumentKind.NumberingType.Registrable,
Sungero.Docflow.DocumentType.DocumentFlow.Inner,
true, false, InfoDocument.ClassTypeGuid,
new Sungero.Domain.Shared.IActionInfo[]
{ Sungero.Docflow.OfficialDocuments.Info.Actions.SendForApproval },
Constants.Module.StatementsKind);
}
```

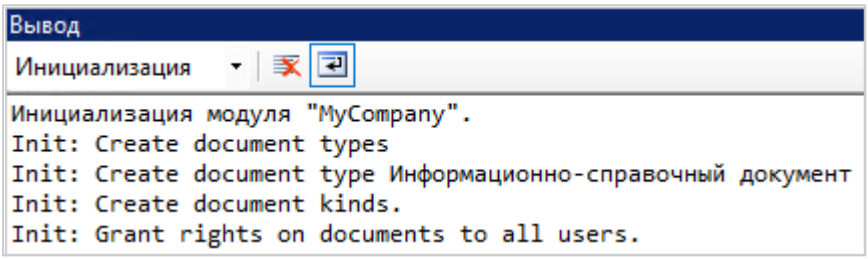
5. Выдайте всем сотрудникам компании права на создание документов нового типа «Информационно-справочные документы». Для этого в редакторе модуля создайте еще одну функцию инициализации **GrantCreateRightsOnInfoDocument()**:

```
/// <summary>
  /// Выдача прав на создание информационно-справочных документов.
  /// </summary>
  public static void GrantCreateRightsOnInfoDocument()
  {
    var allUsers = Roles.AllUsers;
    MyCompany.InfoDocuments.AccessRights.Grant(allUsers,
  DefaultAccessRightsTypes.Create);
  }
```

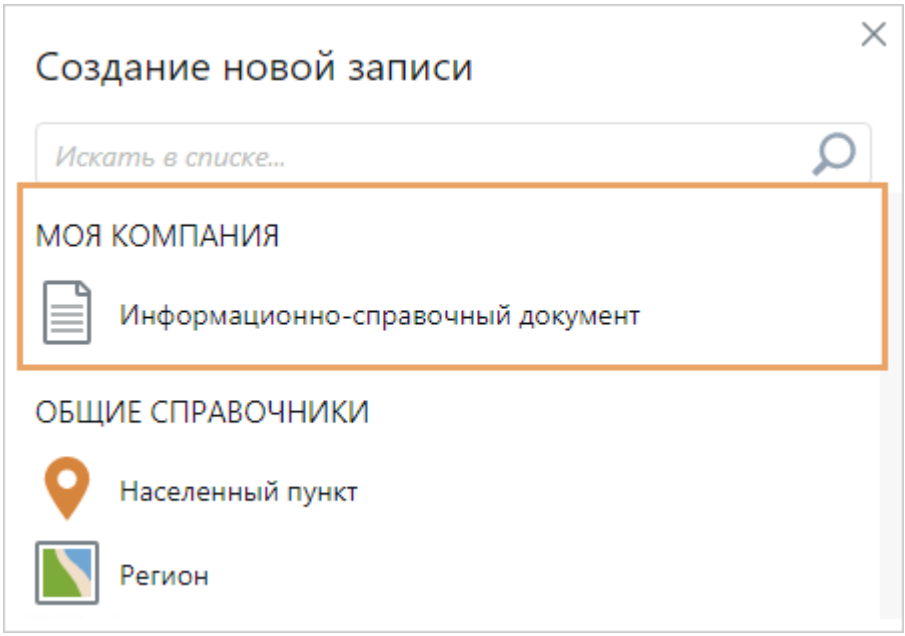
6. В группе «События» перейдите по ссылке **Инициализация**. В обработчике события **Initializing()** вызовите все добавленные ранее функции:

```
public override void Initializing(Sungero.Domain.ModuleInitializingEventArgs e)
  {
    CreateDocumentTypes();
    CreateDocumentKinds();
    GrantCreateRightsOnInfoDocument();
  }
```

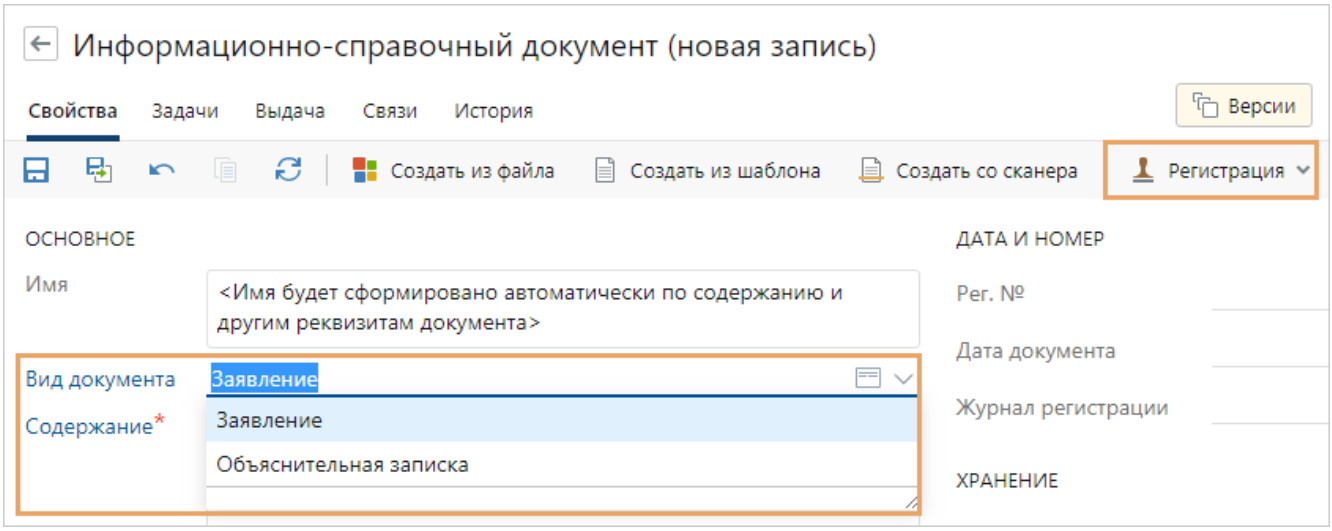
7. Запустите сборку решений, чтобы проверить ошибки в коде.
8. Сохраните изменения и опубликуйте разработку.
9. Запустите инициализацию модуля:



В результате новый тип документа станет доступен всем пользователям для создания:



В карточке документа станет доступен выбор нужных видов информационно-справочных документов и действие для регистрации:



Как добавить новый тип документа в окно смены типа

Ситуация

При смене типа простого документа открывается окно со списком доступных типов. Необходимо добавить в него свой созданный тип, например «Акт приема-передачи» (AcceptanceCertificate).

Решение

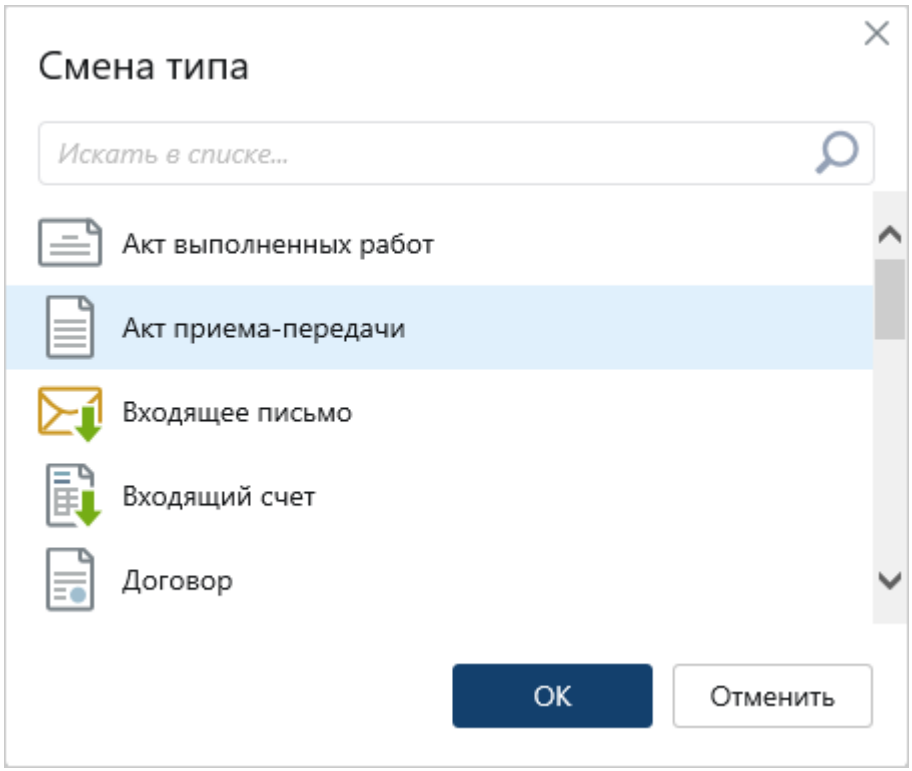
1. В своем решении перекройте тип документа «Простой документ» (Sungero.Docflow.SimpleDocument).
2. В редакторе решения добавьте зависимость от модуля «Документооборот» (Sungero.Docflow).
3. Чтобы изменить стандартную логику, переопределите клиентскую функцию GetTypesAvailableForChange(). Для этого в перекрытом типе документа создайте клиентскую функцию и напишите ее код:

```
/// <summary>
  /// Получить список типов документов, доступных для смены типа.
  /// </summary>
  /// <returns>Список типов документов, доступных для смены типа.</returns>
  public override List<Sungero.Domain.Shared.IEntityInfo>
  GetTypesAvailableForChange()
  {
    var types = new List<Sungero.Domain.Shared.IEntityInfo>();
    // Стандартный список типов документов, доступных для выбора в окне.
    types.Add(Sungero.Docflow.Addendums.Info);
    types.Add(Sungero.Docflow.Memos.Info);
    types.Add(Sungero.Meetings.Minuteses.Info);
    types.Add(Sungero.Contracts.Contracts.Info);
    types.Add(Sungero.FinancialArchive.ContractStatements.Info);
    types.Add(Sungero.Contracts.IncomingInvoices.Info);
```

```
            types.Add(Sungero.Contracts.SupAgreements.Info);
            types.Add(Sungero.Projects.ProjectDocuments.Info);
            types.Add(Sungero.RecordManagement.IncomingLetters.Info);
            types.Add(Sungero.RecordManagement.OutgoingLetters.Info);
            types.Add(Sungero.RecordManagement.Orders.Info);
            types.Add(Sungero.RecordManagement.CompanyDirectives.Info);
            types.Add(Sungero.Docflow.CounterpartyDocuments.Info);
            types.Add(Sungero.FinancialArchive.IncomingTaxInvoices.Info);
            types.Add(Sungero.FinancialArchive.OutgoingTaxInvoices.Info);
            types.Add(Sungero.FinancialArchive.UniversalTransferDocuments.Info);
            types.Add(Sungero.FinancialArchive.Waybills.Info);
            // Добавить в список свой тип документа.
            types.Add(DEV.AcceptanceCertificates.AcceptanceCertificates.Info);
            return types;
        }
```

4. Опубликуйте разработку.

В результате в окне смены типа простого документа можно выбрать добавленный тип «Акт приема-передачи»:



Как изменить длину содержания в карточке документа

Ситуация

В организации принято подробно заполнять содержание в карточке приказа. В базовом решении свойство **Содержание** (Subject) имеет ограничение на количество символов. Необходимо расширить допустимую длину до 1000 символов и более.

Решение

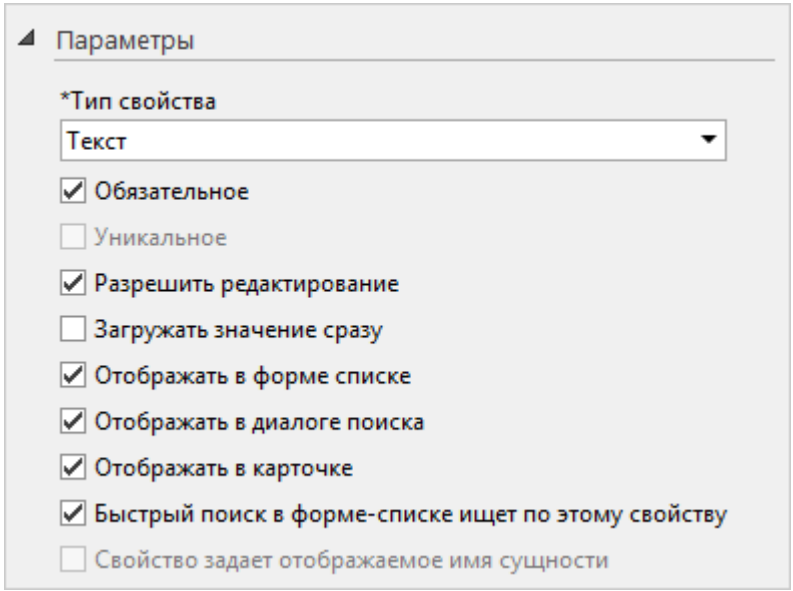
Чтобы увеличить содержание, нужно создать новое свойство с типом «Текст», добавить его на форму с заголовком **Содержание** и доработать логику использования свойства согласно требованиям вашей организации. Исходный контрол содержания потребуется удалить с формы, а его свойство сделать необязательным для заполнения. Для этого:

1. В своем решении перекройте тип документа «Приказ» (Sungero.RecordManagement.Order).
2. В редакторе решения добавьте зависимость от модуля Sungero.RecordManagement.
3. В редакторе перекрытого типа документа в узле «Форма» в параметрах основной вкладки **MainPage** снимите флажок **Использовать контролы предка**, чтобы вкладка стала доступной для изменения.
4. Перейдите в группу контролов **MainGroupDEV**, **LeftGroupDEV**, **MainDEV** и удалите контрол **SubjectDEV**. В результате с формы удалится контрол **Содержание**.
5. Сделайте базовое свойство **Subject** необязательным для заполнения.

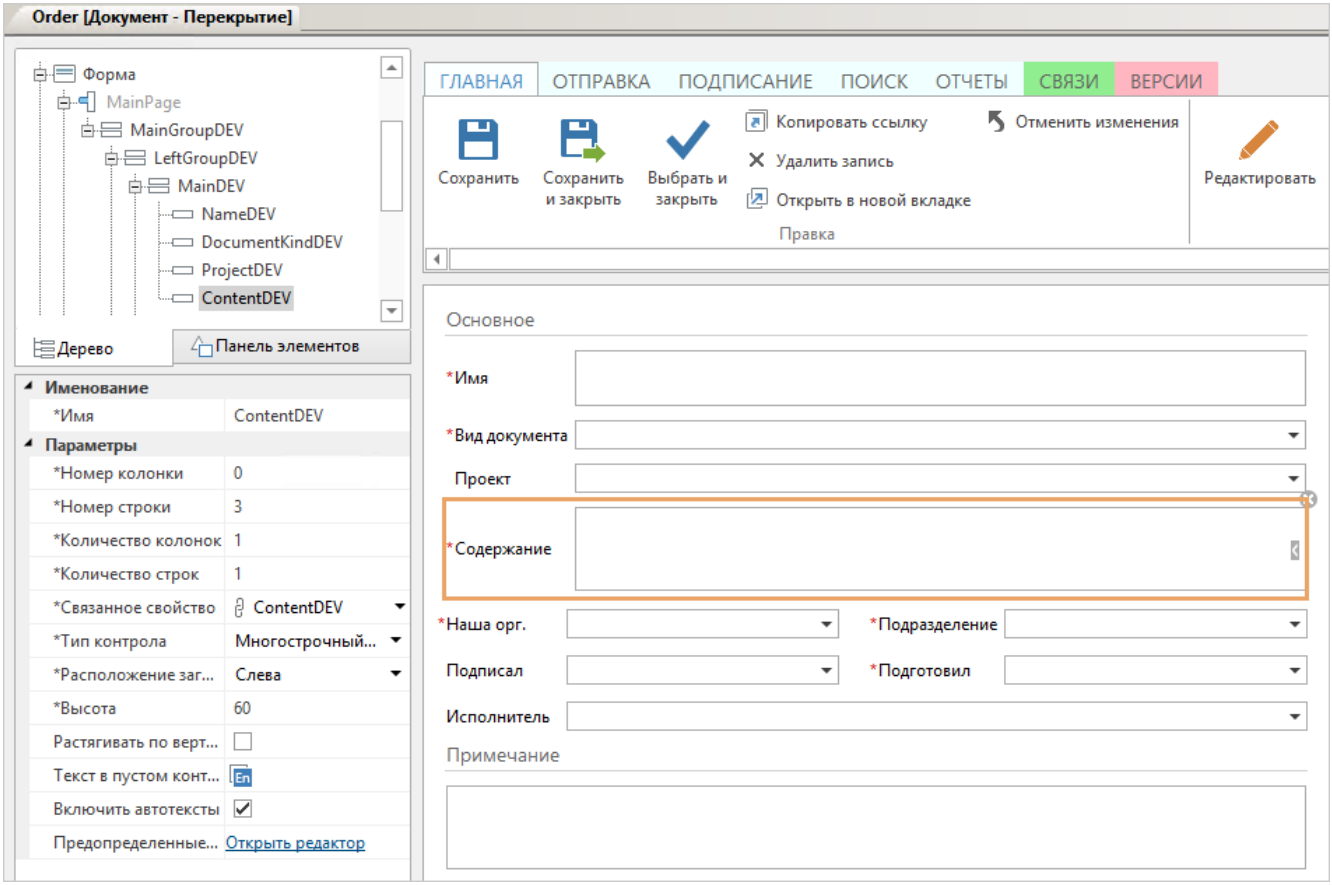
   Настройка обязательности полей наследуется от функции SetRequiredProperties() базового типа документа «Официальный документ» (OfficialDocument). Чтобы изменить обязательность свойства **Subject**, в редакторе типа документа добавьте разделяемую функцию с модификатором **override**:

```
public override void SetRequiredProperties()
  {
    base.SetRequiredProperties();
    _obj.State.Properties.Subject.IsRequired = false;
  }
```

6. В узле «Свойства» создайте свойство **Содержание** (ContentDev).
7. В параметрах свойства в поле ***Тип свойства** выберите значение «Текст». Таким образом, свойство сможет поддерживать многостраничные описания более 1000 символов. Остальные настройки сделайте такие же, как для базового свойства **Содержание** (Subject):



8. Добавьте новое свойство **ContentDev** на форму перекрытого типа документа:



9. При необходимости доработайте логику использования нового свойства согласно требованиям вашей организации.

   Например, свойство **Содержание** может использоваться при формировании имени документа. А так как новое содержание должно быть подробным (более 1000 символов), то его не надо отображать в имени документа. В этом случае следует изменить формат имени, допустим, на такой:

   <Вид документа> №<Регистрационный номер> от <Дата документа>

   Формат имени документа задается в функции FillName() базового типа документа «Официальный документ» (OfficialDocument). Чтобы изменить формат имени документа, в редакторе типа документа добавьте разделяемую функцию **FillName()** с модификатором **override**:

```
public override void FillName()
   {
     //Получить вид документа.
     var documentKind = _obj.DocumentKind;
     //Очистить имя.
     if (documentKind != null && !documentKind.GenerateDocumentName.Value &&
   _obj.Name ==
   Sungero.Docflow.OfficialDocuments.Resources.DocumentNameAutotext)
         _obj.Name = string.Empty;
     if (documentKind == null || !documentKind.GenerateDocumentName.Value)
         return;
     var name = string.Empty;
     // Заполнить имя в формате: <Вид документа> №<Рег. номер> от <Дата
   документа>.
     using (TenantInfo.Culture.SwitchTo())
```

```
    {
        // Добавить к имени №<Рег. номер>.
        if (!string.IsNullOrWhiteSpace(_obj.RegistrationNumber))
            name += Sungero.Docflow.OfficialDocuments.Resources.Number +
_obj.RegistrationNumber;
        // Добавить к имени <Дату документа>.
        if (_obj.RegistrationDate != null)
            name += " от " + _obj.RegistrationDate.Value.ToString("d");
    }
    if (string.IsNullOrWhiteSpace(name))
        name =
Sungero.Docflow.OfficialDocuments.Resources.DocumentNameAutotext;
    else if (_obj.DocumentKind != null)
        name = _obj.DocumentKind.ShortName + name;
        _obj.Name =
Sungero.Docflow.PublicFunctions.Module.TrimSpecialSymbols(name);
}
```

Затем вызовите **FillName()** в событии «Изменение значения свойства» каждого свойства, которое используется в функции:

- **Рег. №** (RegistrationNumber):

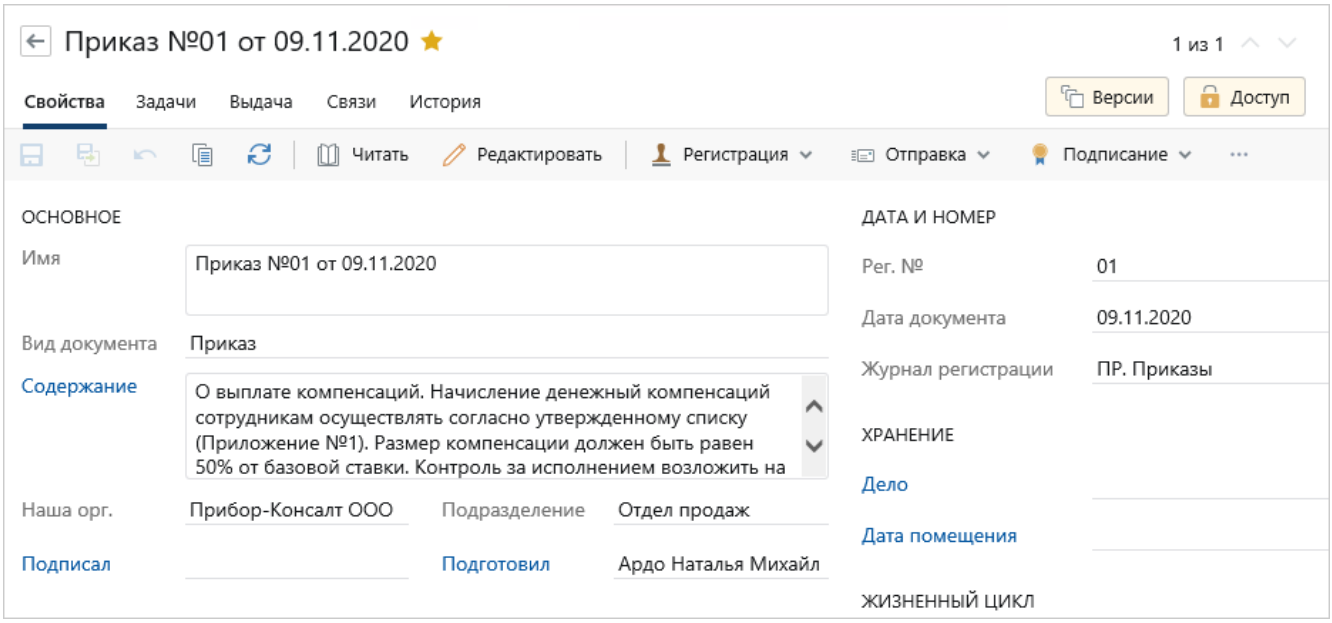
```
public override void
RegistrationNumberChanged
(Sungero.Domain.Shared.StringPropertyChangedEventArgs e)
    {
      FillName();
    }
```

- **Дата документа** (RegistrationDate):

```
public override void
RegistrationDateChanged
(Sungero.Domain.Shared.DateTimePropertyChangedEventArgs e)
    {
      FillName();
    }
```

10. Сохраните изменения и опубликуйте решение.

В результате после создания и регистрации карточка приказа примет вид:



В поле **Содержание** можно добавлять многострочный текст. Имя документа формируется по формату <Вид документа> №<Регистрационный номер> от <Дата документа>.

Как изменить формирование имени документа

Ситуация

Есть базовый тип документа «Официальный документ» (OfficialDocument), у которого в событии До сохранения вызывается функция FillName(). С помощью этой функции автоматически заполняется имя документа, если для вида документа администратор установил флажок **Формировать имя документа автоматически**.

```
// Вызов функции для формирования имени документа в событии "До сохранения".
Functions.OfficialDocument.FillName(_obj);
```

Имя документа составляется по формату <Вид документа> <ИД документа>.

```
// Код функции.
public virtual void FillName()
{
  _obj.Name = _obj.DocumentKind + " " + _obj.Id.ToString();
}
```

Предположим, у типа документа OfficialDocument есть два наследника. Необходимо, чтобы для каждого наследника имя документа формировалось следующим образом:

- <Вид документа> <ИД документа> <Номер регистрации> для первого типа документа;
- <Вид документа> №<Номер> <Имя контрагента> для второго типа документа.

Решение

Сначала рассмотрим, как изменить формат имени для первого типа документа на следующий: <Вид документа> <ИД документа> <Номер регистрации>.

Для этого необходимо переопределить логику функции **FillName()** базового типа:

1. В редакторе типа документа, который является наследником, добавьте разделяемую функцию **FillName()** с модификатором **override**:

```
public override void FillName()
  {
    base.FillName();
    _obj.Name += " " + _obj.RegistrationNumber.ToString();
  }
```

   К формату имени добавляется номер регистрации. Таким образом к логике функции базового типа сущности добавляется новая логика, поэтому добавлен вызов базового обработчика события.

2. Перейдите в узел «Свойства». В событии Изменение значения свойства вызовите функцию **FillName()** для тех свойств, значения которых влияют на имя документа.

   В данном случае только для свойства **Рег. №** (RegistrationNumber):

```
public virtual void
  RegistrationNumberChanged
  (DirRX.AppModule.Shared.StringPropertyChangedEventArgs)
  {
    FillName();
  }
```

Теперь рассмотрим, как изменить формат имени для второго типа документа на следующий: <Вид документа> №<Номер> <Имя контрагента>. С таким форматом могут быть, например, заявки на проверку контрагента, которые используются при заключении договоров. Предполагается, что для второго типа документа уже добавлено свойство **Контрагент** (Counterparty).

Для изменения формата имени второго типа документа необходимо полностью переопределить логику функции **FillName()** базового типа:

1. В редакторе типа документа, который является наследником, добавьте разделяемую функцию **FillName()** с модификатором **override**:

```
public override void FillName()
  {
    //Получить вид документа.
    var documentKind = _obj.DocumentKind;
    //Очистить имя.
    if (documentKind != null && !documentKind.GenerateDocumentName.Value &&
  _obj.Name ==
  Sungero.Docflow.OfficialDocuments.Resources.DocumentNameAutotext)
      _obj.Name = string.Empty;
    if (documentKind == null || !documentKind.GenerateDocumentName.Value)
      return;
    var name = string.Empty;
    // Заполнить имя в формате: <Вид документа> №<Номер> <Имя контрагента>.
    using (TenantInfo.Culture.SwitchTo())
    {
      // Добавить к имени №<Номер>.
      if (!string.IsNullOrWhiteSpace(_obj.RegistrationNumber))
        name += Sungero.Docflow.OfficialDocuments.Resources.Number +
  _obj.RegistrationNumber;
```

```
      // Добавить к имени <Имя контрагента>.
      if (_obj.Counterparty != null)
        name += " " + _obj.Counterparty.DisplayValue;
    }
    if (string.IsNullOrWhiteSpace(name))
      name = Sungero.Docflow.OfficialDocuments.Resources.DocumentNameAutotext;
    else if (_obj.DocumentKind != null)
      name = _obj.DocumentKind.ShortName + name;
    _obj.Name =
  Sungero.Docflow.PublicFunctions.Module.TrimSpecialSymbols(name);
  }
```

Так как логика функции базового типа сущности полностью переопределяется, то базовый обработчик события вызывать не нужно.

2. Перейдите в узел «Свойства». В событии Изменение значения свойства вызовите функцию **FillName()** для тех свойств, значения которых влияют на имя документа.

   В данном случае для двух свойств:

   - **Рег. №** (RegistrationNumber):

```
public virtual void
  RegistrationNumberChanged
  (DirRX.AppModule.Shared.StringPropertyChangedEventArgs)
    {
      FillName();
    }
```

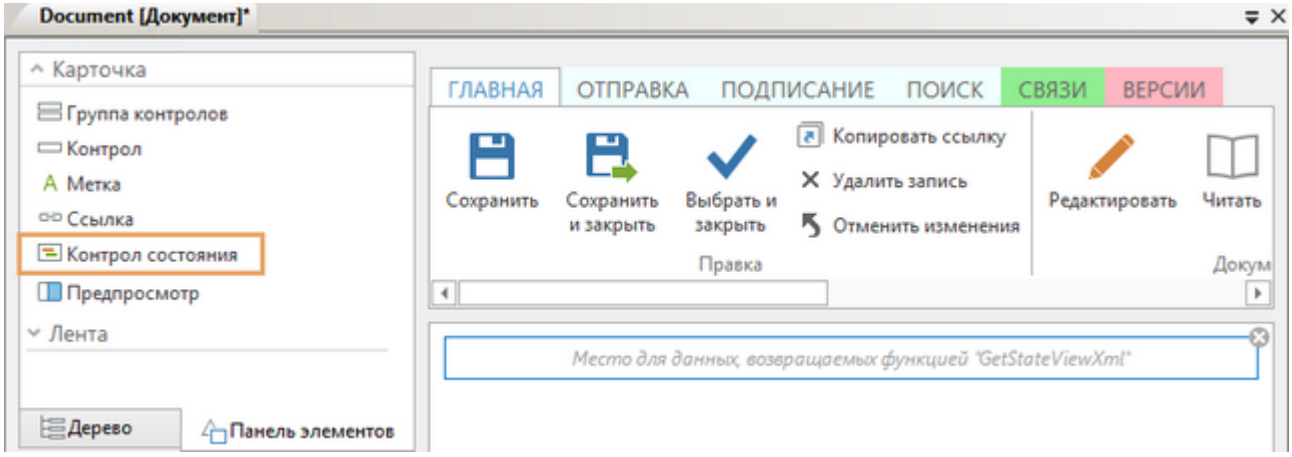
   - **Контрагент** (Counterparty):

```
public virtual void
  CounterpartyChanged
  (DirRX.AppModule.Shared.CounterpartyRequestCounterpartyChangedEventArgs e)
    {
      FillName();
    }
```

Как добавить контрол состояния с информацией о задачах по документу

Предположим, необходимо, чтобы в карточке официального документа отображалась краткая информация о задачах, созданных по документу. Для этого:

1. В редакторе типа документа «Официальный документ» перейдите в узел «Форма».
2. Создайте вкладку и группу, где будет находиться контрол состояния.
3. Перейдите на вкладку «Панель элементов» и добавьте элемент **Контрол состояния** на форму.



4. Измените имя и параметры контрола состояния на панели свойств:

Именование	
***Имя**	Должно быть уникальным в рамках группы, состоять из букв латинского алфавита и цифр
Параметры	
***Номер колонки** ***Номер строки** ***Количество колонок** ***Количество строк**	Свойства задаются и изменяются автоматически при добавлении, перемещении или удалении контролов на форме. Изменять свойства вручную не рекомендуется
Отображать в форме	Контрол состояния будет отображаться в карточке сущности. По умолчанию флажок установлен

5. Добавьте серверную функцию, для этого нажмите на кнопку **Добавить функцию**. Откроется редактор программного кода.
6. Доработайте имя функции и описание, добавьте методы для создания блоков.

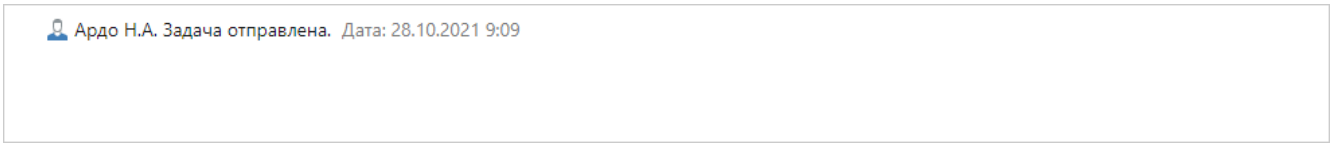
   ПРИМЕЧАНИЕ. Чтобы локализовать имя функции, которое отображается в редакторе форм для контрола состояния, используйте атрибут **LocalizeFunction**.

   Добавление блока без рамки с информацией о задаче:

```
/// <param name="stateView"> Схема представления.</param>
    /// <param name="user"> Пользователь, который выполнил действие.</param>
    /// <param name="text"> Текст действия.</param>
    /// <param name="substituted">Замещающий.</param>
    private static void AddUserActionBlock(StateView stateView, IUser user,
```

```
    string text, IUser substituted)
    {
      var block = stateView.AddBlock();
      // Присоединить блок без рамки к блоку, который следует ниже.
      block.DockType = DockType.Bottom;
      // Добавить в блок иконку типа сущности.
      block.AssignIcon(StateBlockIconType.User, StateBlockIconSize.Small);
      // Отключить отображение рамки блока.
      block.ShowBorder = false;
      // Добавить простой текст в блок.
      block.AddLabel(GetUserActionText(user, text, substituted) + ".");
    }
```

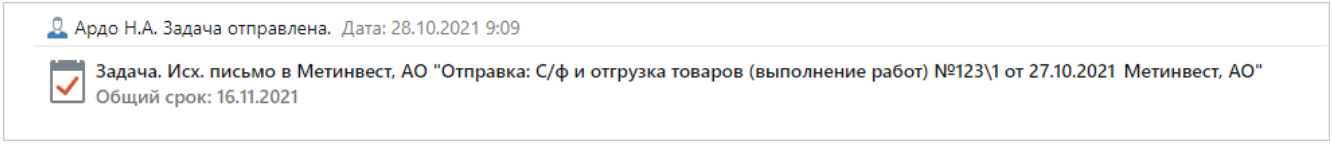
Результат после публикации:



Создание блока для отображения простой задачи:

```
/// <param name="stateView"> Схема представления.</param>
    /// <param name="task"> Задача, для которой формируется блок.
    /// При двойном нажатии левой кнопкой мыши по блоку откроется карточка
    задачи.
    private static void AddSimpleTaskView(StateView stateView, ISimpleTask task)
    {
      // Добавить блок информации о действии.
      AddUserActionBlock(stateView, task.Author,
    OfficialDocuments.Resources.StateViewTaskSent, task.StartedBy);
      var block = stateView.AddBlock();
      // Привязать задачу к блоку.
      block.Entity = task;
      // Добавить в блок иконку типа сущности.
      block.AssignIcon(StateBlockIconType.OfEntity, StateBlockIconSize.Large);
      // Добавить название задачи жирным шрифтом.
      block.AddLabel(string.Format("Задача. {0}", task.Subject),
        Functions.Module.GetHeaderStyle());
      // Добавить срок задачи в новой строке.
      block.AddLineBreak();
      var deadline = task.MaxDeadline.HasValue ?
        task.MaxDeadline.Value.ToUserTime().ToShortDateShortTime() :
        "без срока";
      block.AddLabel(string.Format("Общий срок: {0}", deadline),
        Functions.Module.GetDeadlineStyle());
```

Результат после публикации:

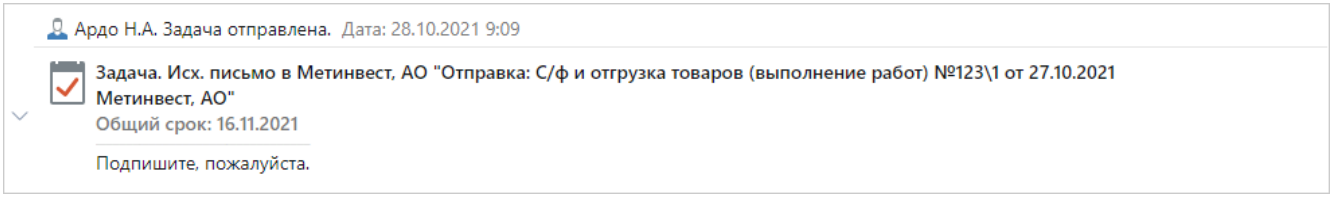


Добавление текста задачи:

```
var taskText = Functions.Module.GetTaskText(task);
    block.AddLineBreak();
    block.AddLabel("_____",
    Functions.Module.GetSeparatorStyle());
```

```
block.AddLineBreak();
block.AddLabel(taskText);
```

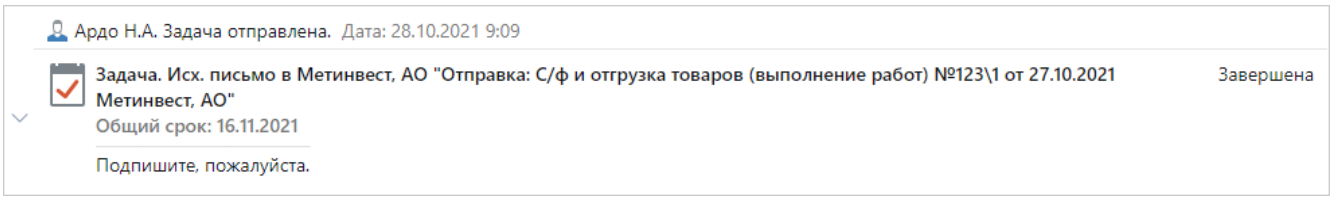
Результат после публикации:



Добавление статуса задачи в правую колонку блока:

```
var rightContent = block.AddContent();
var status =
Workflow.SimpleTasks.Info.Properties.Status.GetLocalizedValue(task.Status);
rightContent.AddLabel(status);
```

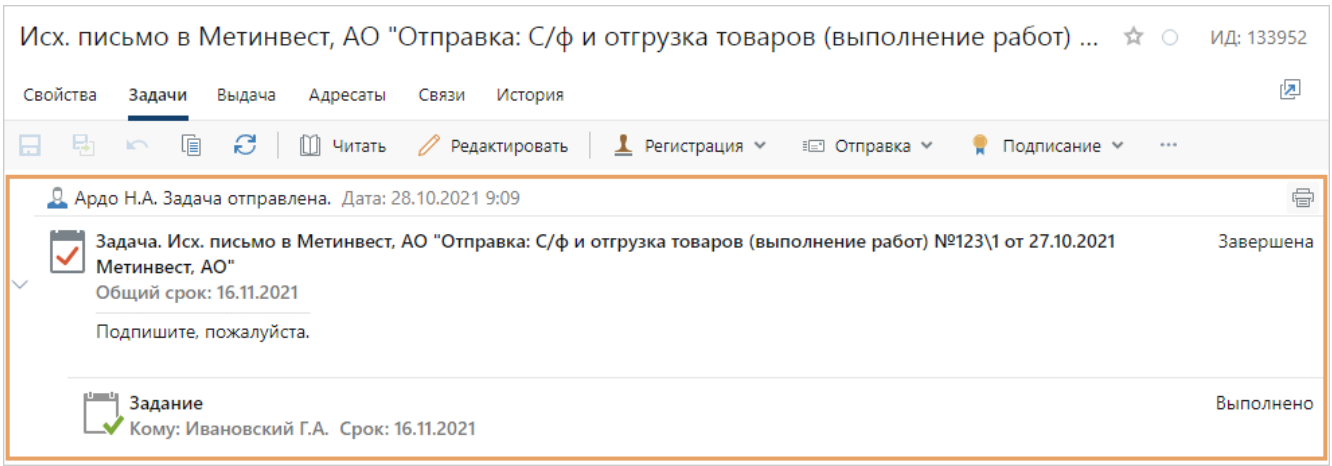
Результат после публикации:



Если по задаче сформированы задания, то добавить дочерние блоки для их отображения:

```
/// <param name="assignments"> Задания, сформированные по задаче.</param>
var assignmentBlocks =
GetSimpleAssignmentsView(assignments.ToList()).Blocks);
if (assignmentBlocks.Any())
block.AddChildBlocks(assignmentBlocks);
}
```

В результате в карточке официального документа и его наследников на отдельной закладке будет содержаться информация о простых задачах, в которые вложен документ. Задания, сформированные по задаче, будут отображаться в виде дочерних блоков:



Как запретить отправку договора на согласование, если не указана сумма

Ситуация

Предположим, необходимо запретить отправлять договор на согласование по регламенту, если в карточке договора не заполнено поле **Сумма** (TotalAmount).

ПРИМЕЧАНИЕ. С версии Directum RX 4.10 рекомендуется использовать согласование по процессу для новых задач. Их схемы настраиваются с помощью механизма no-code. Для этого создайте вариант процесса и в качестве критерия укажите, что поле **Сумма** должно быть заполнено. Подробнее см. в руководстве администратора, раздел «Настройка схем бизнес-процессов».

Решение

1. Создайте решение.
2. В контекстном меню решения последовательно выберите пункты **Перекрыть**, **Документ**.
3. В открывшемся окне в выпадающем списке выберите тип сущности «Договор» (Sungero.Contracts.Contract) из базового решения Directum RX.
4. Нажмите на кнопку **ОК**. Документ появится в дереве решений в разделе «Перекрытые элементы». Откроется его редактор.
5. Перейдите в редактор созданного решения и добавьте зависимость от модуля «Договоры» (Sungero.Contracts). Если зависимости не указаны, при публикации могут возникнуть ошибки.
6. Отправка документа на согласование по регламенту выполняется с помощью действия **SendForApproval**. Чтобы переопределить логику стандартного действия, в редакторе перекрываемого типа документа перейдите в узел «Действия». Затем выберите действие **SendForApproval** (Отправить на согласование), установите флажок рядом с событием **Выполнение** и перейдите по ссылке.
7. В открывшемся редакторе напишите код:

```
public override void SendForApproval(Sungero.Domain.Client.ExecuteActionArgs e)
    {
      // Проверить, заполнено ли поле "Сумма".
      if (!_obj.TotalAmount.HasValue)
      {
        // Если поле не заполнено, вывести сообщение.
        e.AddError(DEV.PerekrytieContract.Contracts.Resources.NeedTotalAmount);
        // Прекратить выполнение действия SendForApproval.
        return;
      }
      base.SendForApproval(e);
    }
```

DEV.PerekrytieContract.Contracts.Resources.NeedTotalAmount – строка *локализации*, которую пользователь увидит в системе, если не заполнит поле **Сумма** и попытается отправить договор на согласование по регламенту. Как добавить строку локализации прямо при написании кода, см. в примере.

ВАЖНО. Метод **e.AddError()** используется при строгой *валидации*: после появления сообщения об ошибке не выполняются следующие операции. Правило не работает, если при *перекрытии* или *наследовании* после **e.AddError()** вызывается метод, действие или событие из родительского типа сущности. Например, в коде выше после **e.AddError()** вызывается действие **SendForApproval()** из типа документа Contract. Чтобы после сообщения об ошибке не выполнялось действие **SendForApproval()**, используйте оператор **return** для принудительного завершения действия.

8. Опубликуйте разработку.

Как изменить срок возврата договора при выдаче сотруднику

Ситуация

При выдаче договора сотруднику в карточке автоматически устанавливается срок возврата 10 рабочих дней. Необходимо, чтобы к дате выдачи добавлялось не 10, а 3 рабочих дня.

Решение

Срок возврата задается в функции **TrackingActionChanged()** официального документа (OfficialDocument). Данный тип документа является базовым абстрактным типом, его нельзя перекрыть. Поэтому для решения необходимо переопределить функции и события в его наследнике.

Все типы документов в базовом решении являются наследниками OfficialDocument. Соответственно, нужно перекрыть те типы документов, у которых требуется изменить срок возврата, в данном случае – «Договор» (Contract).

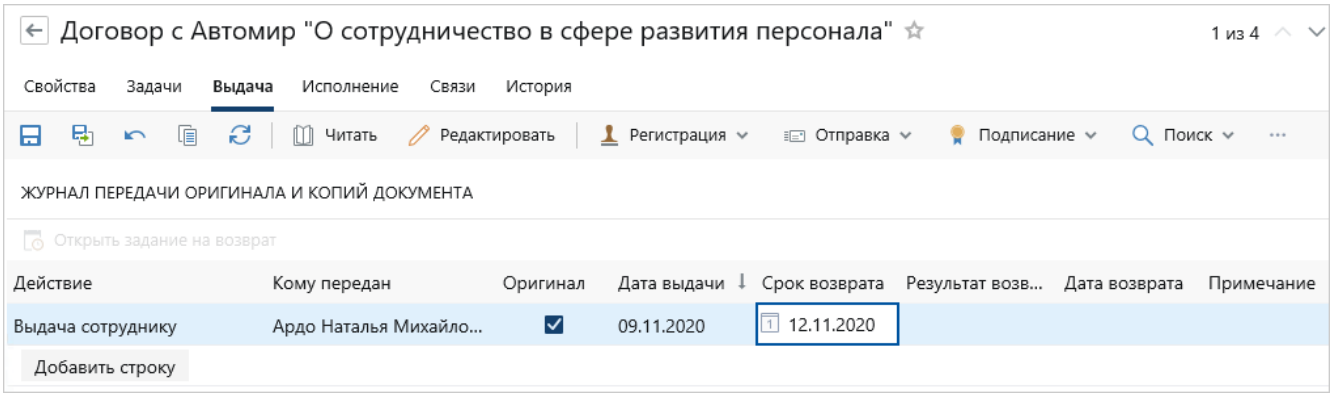
1. В своем решении перекройте тип документа «Договор» (Sungero.Contracts.Contract).
2. В редакторе решения добавьте зависимость от модуля «Договоры» (Sungero.Contracts).
3. В редакторе перекрытого типа документа перейдите в узел «Свойства». Затем разверните свойство-коллекцию Tracking и выберите свойство **Action**. Установите флажок рядом с событием Изменение значение свойства и перейдите по ссылке.
4. Чтобы изменить стандартную логику, в открывшемся редакторе переопределите функцию **TrackingActionChanged()** и в переменной **issuingScheduledDate** укажите свой срок возврата. Для этого напишите код:

```
public override void
  TrackingActionChanged
  (Sungero.Domain.Shared.EnumerationPropertyChangedEventArgs e)
  {
    if (!_obj.State.Properties.ReturnDeadline.OriginalValue.HasValue)
    {
      var user = _obj.DeliveredTo ?? Users.Current;
      // Установить свой срок возврата: 3 рабочих дня для внутренней выдачи.
```

```
var issuingScheduledDate = Calendar.Today.AddWorkingDays(user, 3).Date;
// Срок возврата от контрагента оставить без изменений.
var sendingScheduledDate = Calendar.Today.AddWorkingDays(user, 20).Date;
```

5. Опубликуйте разработку.

В результате при выдаче договора сотруднику в карточке автоматически устанавливается срок возврата 3 рабочих дня:



Как отфильтровать список шаблонов, доступных при создании документа

Ситуация

Предположим, создан тип документа OfficialDocumentTemplate («Шаблон для официальных документов») – наследник от Sungero.Content.ElectronicDocumentTemplate. В системе созданы *сущности* данного типа: шаблон объяснительной записки, шаблон заявки на оборудование, шаблон докладной записки. Нужно, чтобы каждый шаблон отображался только при создании соответствующего вида документа.

Решение

1. В редакторе типа документа OfficialDocumentTemplate добавьте свойства:
   - DocumentKind. Тип свойства – **Ссылка**, тип сущности – Sungero.Docflow.DocumentKind;
   - Status. Тип свойства – **Перечисление**, значения перечисления – Active (Действующий), Closed (Закрытый).
2. Разместите созданные свойства на форме.
3. В редакторе типа документа OfficialDocumentTemplate в первом узле установите флажок рядом с серверным событием Фильтрация и перейдите по ссылке. Откроется редактор программного кода.
4. Добавьте код:

```
public override IQueryable<T> Filtering(IQueryable<T> query,
  Sungero.Domain.FilteringEventArgs e)
  {
    // Проверить, что фильтрация выполняется
    // при создании документа из шаблона.
    if (_createFromTemplateContext != null)
    {
```

```
        // Фильтровать шаблоны по статусу.
        query = query.Where(d => d.Status == Status.Active);

        // Фильтровать шаблоны по критериям.
        if (Docflow.OfficialDocuments.Is(_createFromTemplateContext))
        {
          var document =
      Docflow.OfficialDocuments.As(_createFromTemplateContext);
          query = query.Where(template =>
      template.DocumentKind.Equals(document.DocumentKind));
        }
      }
      return query;
    }
```

ПРИМЕЧАНИЕ. По такому принципу работает фильтрация в типе документа DocumentTemplate из *базового решения* Directum RX. Поэтому в стандартной версии системы при создании документа в списке для выбора отображаются только шаблоны, которые подходят документу по типу, виду, нашей организации и подразделению. При необходимости можно перекрыть тип документа DocumentTemplate и доработать фильтрацию.

Как настроить склонение имени в шаблонах документов

Ситуация

В Directum RX сотрудники работают с шаблонами документов с автозаполняемыми полями. При создании документа из такого шаблона часть содержимого автоматически заполняется данными из системы. В том числе в документ подставляются фамилии, имена и отчества в необходимых падежах. Например, в «шапке» заявления – Александрову Александру Александровичу от Петрова Петра Петровича.

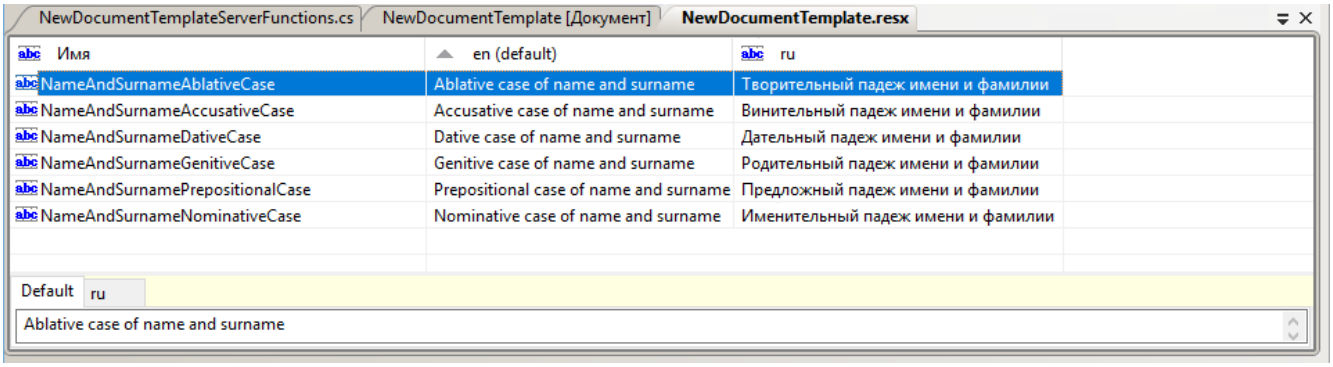
Возникают ситуации, когда у сотрудника есть только имя и фамилия. В этом случае стандартный механизм склоненияможет не работать. Для настройки склонения используйте словарь исключений. Подробнее см. в руководстве администратора, раздел «Склонение слов по падежам» (Linux, Windows). Также можно программно задавать склонение имени в шаблоне документа.

По умолчанию в системе склонение слов по падежам выполняется с помощью библиотеки Visis.dll.

Решение

1. В своем решении перекройте тип шаблона документа. В качестве базового типа выберите «Шаблон документа» (Sungero.Docflow.DocumentTemplate).
2. В редакторе решения добавьте зависимость от модуля Sungero.Docflow.

3. Добавьте ресурс для каждого из падежей имени и фамилии в шаблоне документа. Для этого в дереве решений в контекстном меню перекрытого шаблона документа выберите пункт **Открыть ресурсы**. Далее в контекстном меню табличной части выберите пункт **Добавить строку**. Заполните поля, как на картинке:



4. Чтобы просклонять имя и фамилию в родительном падеже, в шаблоне документа добавьте серверную функцию **NameAndSurnameGenitiveCase()** с атрибутом **Converter** и привязкой к добавленному ресурсу:

```
// Получить имя и фамилию в родительном падеже.
  // <param name="employee">Сотрудник.</param>
  // <returns>Имя и фамилия в родительном падеже.</returns>
  [Converter("NameAndSurnameGenitiveCase")]
  public static string NameAndSurnameGenitiveCase(Sungero.Company.IEmployee
employee)
  {
    // Задать персону и род.
    var person = employee.Person;
    var gender = CaseConverter.DefineGender(person.MiddleName);
    // Если отчество персоны не заполнено, определить род персоны
    // и получить персональные данные в родительном падеже.
    if (string.IsNullOrEmpty(person.MiddleName))
    {
      if (person.Sex == Sungero.Parties.Person.Sex.Male)
        gender = Gender.Masculine;
      if (person.Sex == Sungero.Parties.Person.Sex.Female)
        gender = Gender.Feminine;
    }
    return
  CaseConverter.ConvertPersonFullNameToTargetDeclension(person.LastName,
  person.FirstName, person.MiddleName, DeclensionCase.Genitive, gender);
  }
```

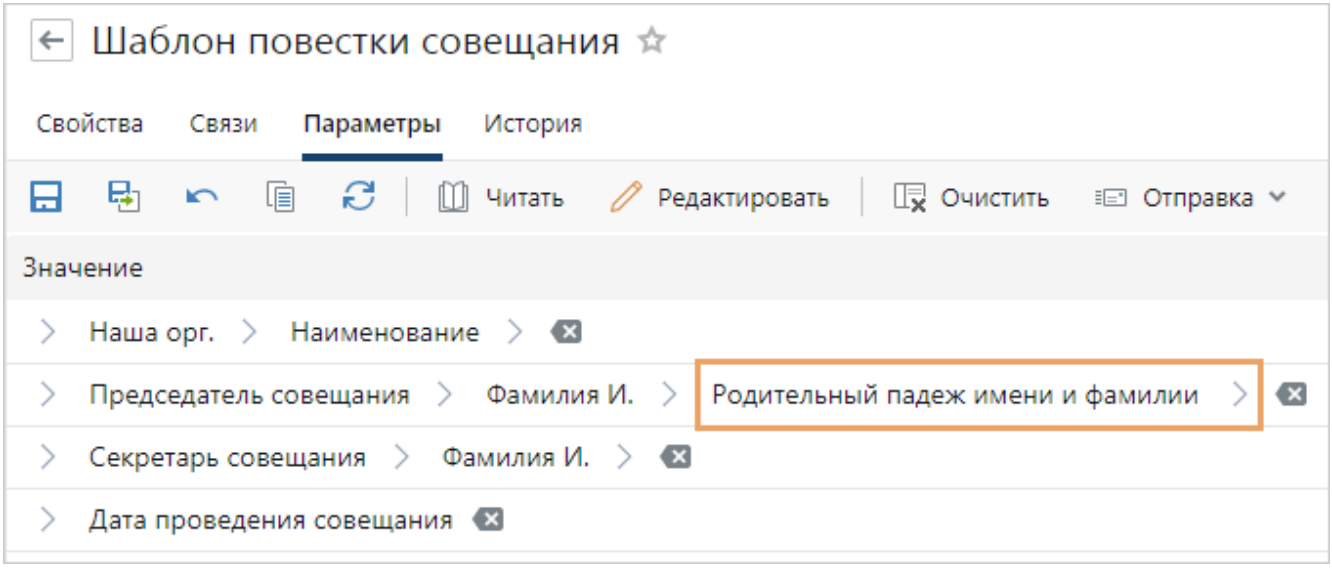
5. Чтобы просклонять имя и фамилию в дательном падеже, также добавьте серверную функцию **NameAndSurnameDativeCase()** с атрибутом **Converter** и привязкой к добавленному ресурсу:

```
// Получить имя и фамилию в дательном падеже.
  // <param name="employee">Сотрудник.</param>
  // <returns>Имя и фамилия в родительном падеже.</returns>
  [Converter("NameAndSurnameDativeCase")]
```

```
public static string NameAndSurnameDativeCase(Sungero.Company.IEmployee
employee)
{
  // Задать персону и род.
  var person = employee.Person;
  var gender = CaseConverter.DefineGender(person.MiddleName);
  // Если отчество персоны не заполнено, определить род персоны
  // и получить персональные данные в дательном падеже.
  if (string.IsNullOrEmpty(person.MiddleName))
  {
    if (person.Sex == Sungero.Parties.Person.Sex.Male)
    gender = Gender.Masculine;
    if (person.Sex == Sungero.Parties.Person.Sex.Female)
    gender = Gender.Feminine;
  }
  return
CaseConverter.ConvertPersonFullNameToTargetDeclension(person.LastName,
person.FirstName, person.MiddleName, DeclensionCase.Dative, gender);
}
```

6. Напишите серверные «функции-конвертеры» для остальных падежей по аналогии с шагами 4 и 5.
7. Сохраните изменения и опубликуйте разработку.

В результате в Directum RX в шаблоне документа фамилия и имя сотрудника без отчества будут склоняться корректно.



Как добавить сводку по новому типу документа

Ситуация

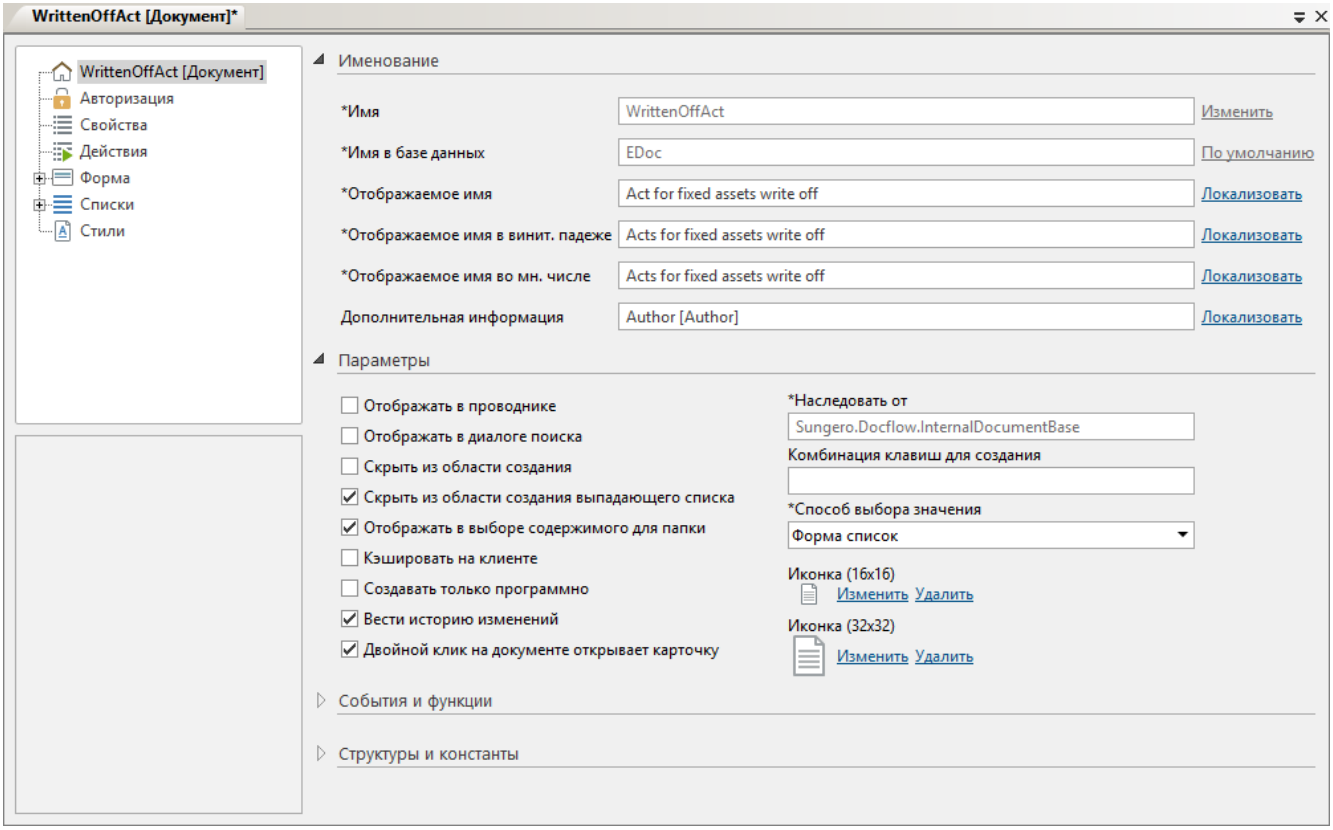
Предположим, нужно создать новый тип документа «Акт на списание основных средств» (WrittenOffAct). При согласовании документа по регламенту в заданиях на согласование и подписание, в области «Реквизиты документа» на контроле состояния

должна отображаться краткая сводка по документу с реквизитами: краткое имя, содержание, инвентарный номер, материально ответственное лицо (МОЛ), центр затрат, остаточная стоимость и причины списания.

Сводка необходима, чтобы согласующий или подписант мог ознакомиться с документом, не открывая карточку и содержимое. Зачастую данных из сводки достаточно, чтобы принять решение по документу.

Решение

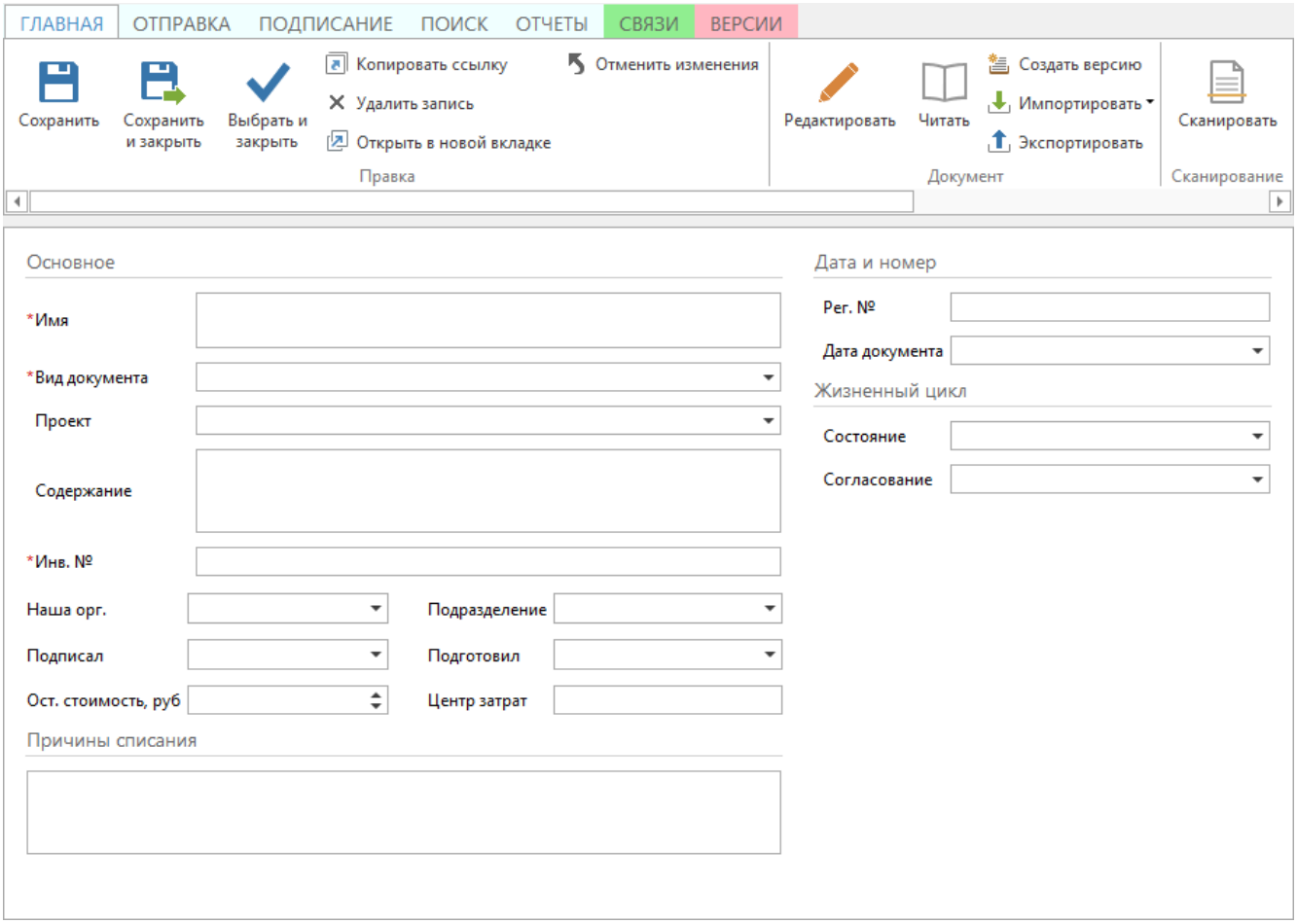
1. На основании базового типа «Внутренний документ» (InternalDocumentBase) создайте тип документа «Акт на списание основных средств» (WrittenOffAct).
2. В редакторе созданного типа заполните именование и параметры, как показано на рисунке:



3. Перейдите в узел «Свойства» и для свойства **PreparedBy** измените отображаемое имя с **Подготовил** на **МОЛ** (материально ответственное лицо).
4. Создайте свойства:

Имя и отображаемое имя свойства	Тип свойства	Пояснение
InventoryNumber (Инв. №)	**Строка**	Инвентарный номер списываемого основного средства. Например, 10/19-1569835426
ResidualValue (Ост. стоимость, руб)	**Вещественное**	Остаточная стоимость списываемого основного средства в рублях
CostCentre (Центр затрат)	**Строка**	Единица деятельности организации, которую выделяют в бухгалтерском учете для контроля за затратами

5. Настройте форму документа, как показано на скриншоте. Для этого:
   - скройте неиспользуемые группы и удалите лишние контролы;
   - добавьте созданные свойства в группы «Main» и «Additional»;
   - переименуйте группу «Примечание» на «Причины списания».

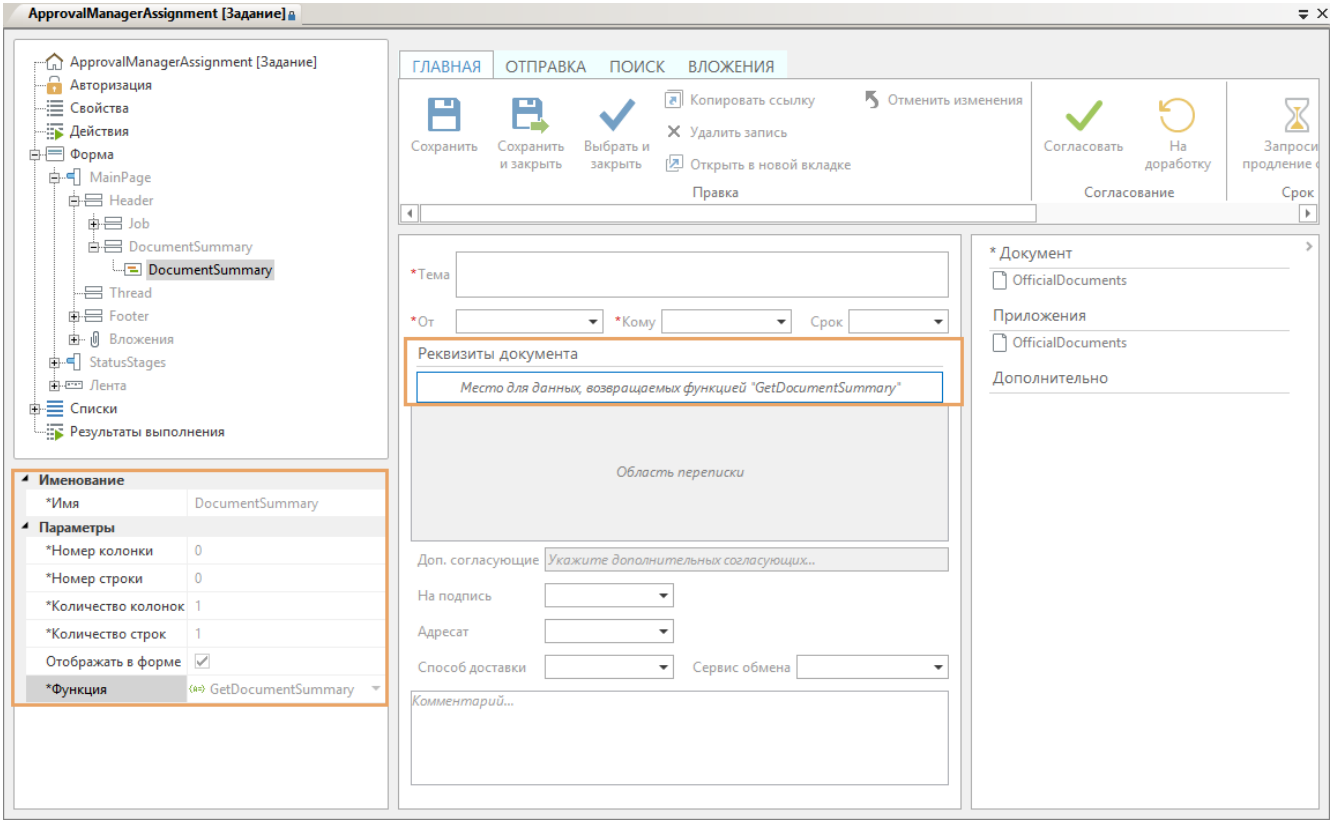


6. Чтобы построить сводку по основным реквизитам акта, в серверном коде созданного типа документа переопределите функцию GetDocumentSummary():

```
/// <summary>
  /// Построить сводку по документу.
  /// </summary>
  /// <returns>Сводка по документу.</returns>
  public override StateView GetDocumentSummary()
  {
    var documentSummary = StateView.Create();
    var block = documentSummary.AddBlock();
    const string none = "-";

    // Краткое имя документа.
    var documentName = _obj.DocumentKind.ShortName;
    if (!string.IsNullOrWhiteSpace(_obj.RegistrationNumber))
      documentName += Sungero.Docflow.OfficialDocuments.Resources.Number +
_obj.RegistrationNumber;

    if (_obj.RegistrationDate != null)
      documentName += Sungero.Docflow.OfficialDocuments.Resources.DateFrom +
_obj.RegistrationDate.Value.ToString("d");

    block.AddLabel(documentName);
```

```
  block.AddLineBreak();
  block.AddEmptyLine();

  // Содержание.
  var subject = !string.IsNullOrEmpty(_obj.Subject) ? _obj.Subject : none;
  block.AddLabel(string.Format("{0}: {1}",
_obj.Info.Properties.Subject.LocalizedName, subject));
  block.AddLineBreak();

  // Инв. №.
  block.AddLabel(string.Format("{0}: {1}",
_obj.Info.Properties.InventoryNumber.LocalizedName, _obj.InventoryNumber));
  block.AddLineBreak();

  // МОЛ – материально ответственное лицо.
  var responsible = _obj.PreparedBy != null ? _obj.PreparedBy.Name : none;
  block.AddLabel(string.Format("{0}: {1}",
_obj.Info.Properties.PreparedBy.LocalizedName, responsible));
  block.AddLineBreak();

  // Ост. стоимость.
  var residualValue = _obj.ResidualValue != null ?
_obj.ResidualValue.Value.ToString("N") : none;
  block.AddLabel(string.Format("{0}: {1}",
_obj.Info.Properties.ResidualValue.LocalizedName, residualValue));
  block.AddLineBreak();

  // Центр затрат.
  var costCentre = !string.IsNullOrEmpty(_obj.CostCentre) ? _obj.CostCentre :
 none;
  block.AddLabel(string.Format("{0}: {1}",
_obj.Info.Properties.CostCentre.LocalizedName, costCentre));
  block.AddLineBreak();

  // Причины списания.
  var note = !string.IsNullOrEmpty(_obj.Note) ? _obj.Note : none;
  block.AddLabel(string.Format("{0}: {1}",
_obj.Info.Properties.Note.LocalizedName, note));

  return documentSummary;
}
```

Область «Реквизиты документа» – это группа в карточках заданий. В ней находится контрол состояния с именем **DocummentSummary**:



Сводка по документу отображается в контроле состояния и формируется с помощью функции **GetDocumentSummary()**.

7. В редакторе модуля, в котором создали тип документа, добавьте функции инициализации:

```
public override void Initializing(Sungero.Domain.ModuleInitializingEventArgs e)
    {
      GrantRights();
      CreateDocumentTypes();
    }

    /// <summary>
    /// Выдача прав на тип документа "Акт на списание ОС".
    /// </summary>
    public static void GrantRights()
    {
    InitializationLogger.Debug("Init: Module DocSummary. Grant rights to all
    users.");

      var allUsers = Sungero.CoreEntities.Roles.AllUsers;
      DEV.DocSummary.WrittenOffActs.AccessRights.Grant(allUsers,
    DefaultAccessRightsTypes.Create);
      DEV.DocSummary.WrittenOffActs.AccessRights.Save();
    }

    /// <summary>
    /// Создание типа документа "Акт на списание ОС"
    /// в служебном справочнике "Типы документов".
    /// </summary>
    public static void CreateDocumentTypes()
```

```
{
  InitializationLogger.Debug("Init: Module DocSummary. Create document
types.");
  Sungero.Docflow.PublicInitializationFunctions.Module.
CreateDocumentType("Акт на списание ОС",
                     WrittenOffAct.ClassTypeGuid,
                     Sungero.Docflow.DocumentType.DocumentFlow.Inner, true);
}
```

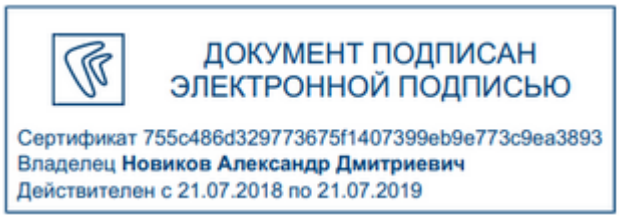
8. Опубликуйте разработку.

В результате в заданиях на подписание и согласование акта на списание основных средств в области «Реквизиты по документу» появится сводка.

Как изменить отметку об ЭП

В Directum RX есть возможность преобразовать документ в формат PDF и добавить отметку об электронной подписи (ЭП). Это позволяет наглядно подтвердить факт подписания документа ЭП. При этом формат отметки меняется в зависимости от того, какой ЭП подписан документ:

- квалифицированная ЭП. В отметке содержится информация о сертификате, его владельце и сроке действия:



- простая ЭП. В отметке содержится ФИО сотрудника, который подписал документ, и его идентификатор в системе:

Администратор может настроить размещение отметки об ЭП в шаблонах документов. Разработчик может изменить оформление отметки и состав отображаемых данных на ней для всех видов документов или конкретного вида документа, например, только для входящих счетов.

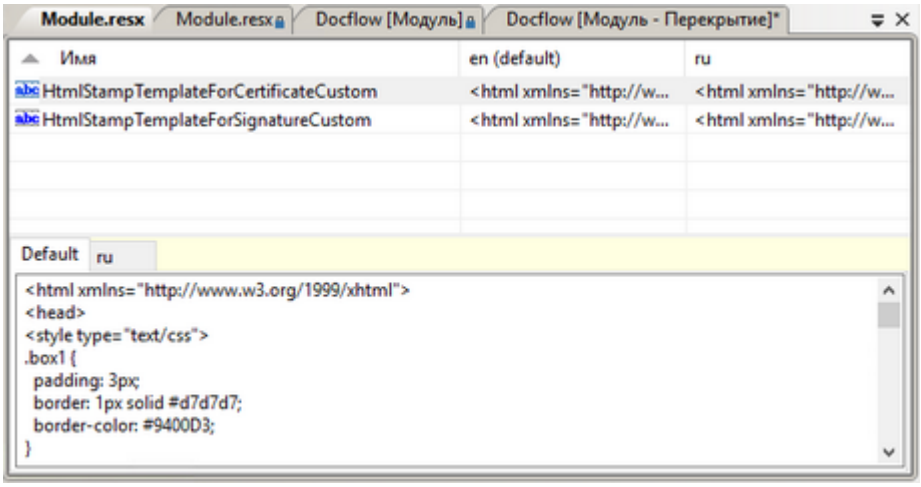
Ситуация

Для всех видов документов необходимо изменить оформление отметки об ЭП: поменять логотип, заменить цвет на фиолетовый, добавить информацию о дате и времени подписания.

Решение

1. В своем решении перекройте модуль Docflow.

2. В перекрытый модуль добавьте *ресурсы*, которые содержат HTML-код отметки об ЭП для квалифицированной и простой подписи. Для этого скопируйте существующие ресурсы из модуля Docflow:
   - в базовом решении Sungero.DirectumRX выберите модуль Docflow и в контекстном меню выберите пункт **Открыть ресурсы**;
   - в открывшемся редакторе ресурсов найдите строки с названиями **HtmlStampTemplateForCertificate** для квалифицированной ЭП и **HtmlStampTemplateForSignature** для простой ЭП;
   - в контекстном меню одного из ресурсов выберите пункт **Копировать**. Откройте ресурсы перекрытого модуля Docflow и с помощью пункта **Вставить** добавьте скопированный ресурс. Аналогично скопируйте второй ресурс. Для удобства в названия добавим приписку **Custom**.



   Отметка об ЭП формируется на основе данных, получаемых из HTML-кода. В нем вручную прописываются настройки отметки.

3. Доработайте HTML-код для каждого ресурса:
   - измените цвет #003f75 на фиолетовый #9400D3;
   - измените логотип;
   - добавьте параметр **Дата подписания {SigningDate}**.

   Пример для ресурса HtmlStampTemplateForCertificateCustom

```
<html xmlns="http://www.w3.org/1999/xhtml">
  <head>
  <style type="text/css">
  .box1 {
    padding: 3px;
    border: 1px solid #d7d7d7;
    border-color: #9400D3;
  }
  .tg1{font-size:6pt;font-family:Arial, "Segoe UI", Consolas, Calibri,
  monospace;line-height:6pt;color:#9400D3}
  .tg2{font-size:8.25pt;font-family:Arial, "Segoe UI", Consolas, Calibri,
  monospace;text-align:center;line-height:9.75pt;color: #9400D3}

  </style>
  </head>
  <body style="margin: 0">
```

```
<table class="box1">
<colgroup>
<col width = 31>
<col width = 137>
</colgroup>
<tr>
  <td width = 32 height = 39 align = "right">
    <img width = 27 src = "data:{Изображение}"/>
  </td>
  <td>
    <div class="tg2" style="margin-left:7.5px;margin-right:3.75px">ДОКУМЕНТ
ПОДПИСАН<br>ЭЛЕКТРОННОЙ ПОДПИСЬЮ</div>
  </td>
</tr>
<tr>
  <td colspan="2" style="white-space: nowrap">
    <span class="tg1">Сертификат {Thumbprint}</span>
  </td>
</tr>
<tr>
  <td colspan="2">
    <span class="tg1">Владелец <b>{SignatoryFullName}</b></span>
  </td>
</tr>
<tr>
  <td colspan="2">
    <span class="tg1">Действителен {Validity}</span>
  </td>
</tr>
<tr>
  <td colspan="2">
    <span class="tg1">Дата подписания {SigningDate}</span>
  </td>
</tr>
</table>
</body>

</html>
```

Пример для ресурса HtmlStampTemplateForSignatureCustom

```
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
<style type="text/css">
.box1 {
  padding: 3px;
  border: 1px solid #d7d7d7;
  border-color: #9400D3
}
.tg1{font-size:6pt;font-family:Arial, "Segoe UI", Consolas, Calibri,
monospace;line-height:6pt;color: #9400D3}
.tg2{font-size:8.25pt;font-family:Arial, "Segoe UI", Consolas, Calibri,
monospace;text-align:center;line-height:9.75pt;color: #9400D3}

</style>
</head>
```

```
<body style="margin: 0">

<table class="box1">
<colgroup>
<col width = 31.5>
<col width = 136>
</colgroup>
<tr>
  <td width = 31.5 height = 39 align = "right">
    <img width = 27 src = "data:{Изображение}"/>
  </td>
  <td>
    <div class="tg2" style="margin-left:7.5px;margin-right:3.75px">ДОКУМЕНТ
ПОДПИСАН<br>ЭЛЕКТРОННОЙ ПОДПИСЬЮ</div>
  </td>
</tr>
<tr>
  <td colspan="2">
    <span class="tg1">Подписал <b>{SignatoryFullName}</b></span>
  </td>
</tr>
<tr>
  <td colspan="2">
    <span class="tg1">Идентификатор пользователя {SignatoryId}</span>
  </td>
</tr>
<tr>
  <td colspan="2">
    <span class="tg1">Дата подписания {SigningDate}</span>
  </td>
</tr>
</table>
</body>

</html>
```

4. Переопределите функцию GetSignatureMarkForCertificateAsHtml() модуля Docflow, которая используется для получения отметки о квалифицированной ЭП. В функции заполняются параметры HTML-кода из ранее добавленного ресурса **HtmlStampTemplateForCertificateCustom**. Для этого в перекрытый модуль Docflow добавьте серверную функцию и напишите код:

```
/// <summary>
  /// Получить отметку об ЭП для сертификата из подписи.
  /// </summary>
  /// <param name="signature">Подпись.</param>
  /// <returns>Изображение отметки об ЭП для сертификата в виде
  HTML.</returns>
  /// <description>
  /// Пример перекрытия отметки о квалифицированной ЭП для всех документов.
  /// В отметке о квалифицированной ЭП изменены логотип и пропорции заголовка.
  /// Также в отметку добавлены дата и время подписания.
  /// Цвет отметки изменен на фиолетовый.
  /// </description>
```

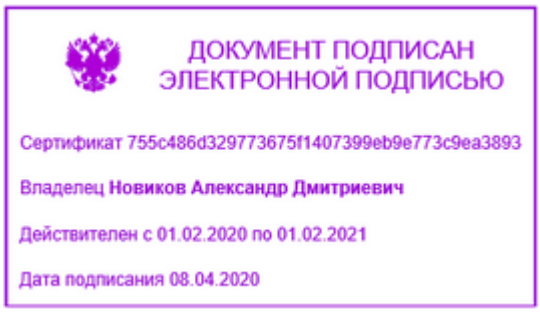
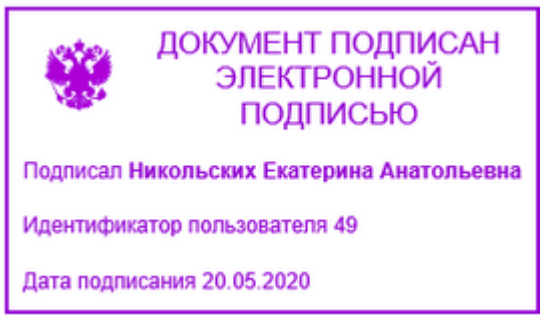
```
public override string
GetSignatureMarkForCertificateAsHtml(Sungero.Domain.Shared.ISignature
signature, ISignatureStampParams signatureStampParams)
{
  if (signature == null)
    return string.Empty;

  var certificate = signature.SignCertificate;
  if (certificate == null)
    return string.Empty;

  var certificateSubject = this.GetCertificateSubject(signature);

  var signatoryFullName = string.Format("{0} {1}",
certificateSubject.Surname, certificateSubject.GivenName).Trim();
  if (string.IsNullOrEmpty(signatoryFullName))
    signatoryFullName = certificateSubject.CounterpartyName;

  string html = Resources.HtmlStampTemplateForCertificateCustom;
  html = html.Replace("{SignatoryFullName}", signatoryFullName);
  html = html.Replace("{Thumbprint}", certificate.Thumbprint.ToLower());
  var validity = string.Format("{0} {1} {2} {3}",
                               Company.Resources.From,
                               certificate
.NotBefore.Value.ToShortDateString(),
                               Company.Resources.To,
                               certificate
.NotAfter.Value.ToShortDateString());
  html = html.Replace("{Validity}", validity);
  html = html.Replace("{SigningDate}", signature.SigningDate.ToString("g"));
  return html;

}
```

5. Аналогично переопределите функцию GetSignatureMarkForSimpleSignatureAsHtml() модуля Docflow, которая используется для получения отметки простой ЭП. В функции заполняются параметры HTML-кода из ранее добавленного ресурса **HtmlStampTemplateForSignatureCustom**.

```
/// <summary>
/// Получить отметку об ЭП для подписи.
/// </summary>
/// <param name="signature">Подпись.</param>
/// <returns>Изображение отметки об ЭП для подписи в виде HTML.</returns>
/// <description>
/// Пример перекрытия отметки о простой ЭП для всех документов.
/// В отметке о простой ЭП изменены логотип и пропорции заголовка.
/// Также в отметку добавлены дата и время подписания.
/// Цвет отметки изменен на фиолетовый.
/// </description>
public override string
GetSignatureMarkForSimpleSignatureAsHtml(Sungero.Domain.Shared.ISignature
signature, ISignatureStampParams signatureStampParams)
{
  if (signature == null)
    return string.Empty;
```

```
        var signatoryFullName = signature.SignatoryFullName;
        var signatoryId = signature.Signatory.Id;

        string html = Resources.HtmlStampTemplateForSignatureCustom;
        html = html.Replace("{SignatoryFullName}", signatoryFullName);
        html = html.Replace("{SignatoryId}", signatoryId.ToString());
        html = html.Replace("{SigningDate}", signature.SigningDate.ToString("g"));
        return html;

    }
```

В результате отметка об ЭП будет выглядеть следующим образом:

Квалифицированная ЭП	Простая ЭП
	

Как создать постраничную отметку для парафирования договорных документов

Ситуация

Необходимо создать постраничную отметку, которая будет использоваться для парафирования договорных документов. Отметка подтверждает, что каждая страница согласована нашей организацией, и обеспечивает целостность всего документа.

Требования к положению отметки:

- отступ от левого края страницы – 0,3 см;
- отступ от нижнего края страницы – 2 см;
- поворот отметки – 90 градусов против часовой стрелки.

Отметка содержит:

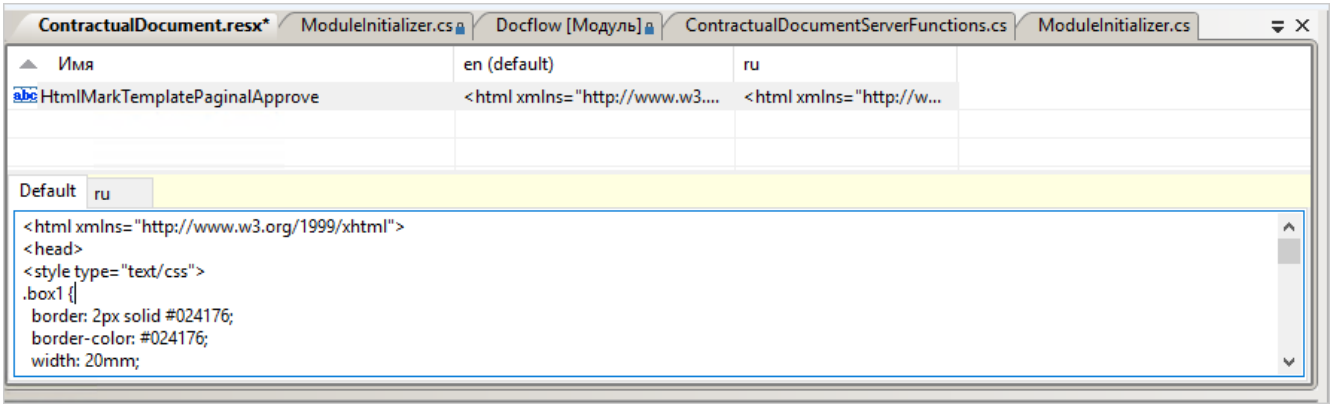
- надпись «Согласовано»;
- название нашей организации;
- версию согласованного документа.

ПРИМЕЧАНИЕ. Редактор отметок не поддерживает работу с постраничными отметками. Рекомендуется запретить открытие редактора в типах документов, где они используются.

Решение

1. Перекройте тип документа «Договорной документ» (ContractualDocument).

2. Отметка формируется на основе данных, получаемых из HTML-кода. В ресурсах типа документа «Договорной документ» добавьте HTML-код отметки. Для этого в дереве решений в контекстном меню перекрытого типа документа выберите пункт **Открыть ресурсы**. Далее в контекстном меню табличной части выберите пункт **Добавить строку**. Укажите название нового ресурса, например **HtmlMarkTemplatePaginalApprove**, и добавьте код для изображения отметки:



Пример для ресурса с кодом отметки

```
<html xmlns="http://www.w3.org/1999/xhtml">
    <head>
    <style type="text/css">
    .box1 {
      border: 2px solid #024176;
      border-color: #024176;
      width: 20mm;
      height: 15mm
    }
    .tg1{
      font-size: 14pt;
      font-family: Arial, "Segoe UI", Consolas, Calibri, monospace;
      text-align: center;
      line-height: 9.75pt;
      color: #024176;
      white-space: nowrap
    }
    .tg2{
      font-size: 12pt;
      font-family: Arial, "Segoe UI", Consolas, Calibri, monospace;
      text-align: center;
      line-height: 4pt;
      color: #024176;
      white-space: nowrap
    }

    </style>
    </head>
    <body style="margin: 0">

    <table class="box1">
      <tr>
        <td>
            <div class="tg1"><b>СОГЛАСОВАНО</b></div>
        </td>
      </tr>
    <tr>
      <td colspan="2">
        <span class="tg2">{approvedOrganization}. Версия {version}</span>
      </td>
    </tr>
```

```
</table>
</body>

</html>
```

3. В тип документа «Договорной документ»:

- добавьте серверные функции:

  GetOrCreateContractPaginalApproveMark() – создает или обновляет экземпляр постраничной отметки для утвержденных договорных документов;

  GetContractualApprovedMarkAsHtml() – получает изображение отметки в виде HTML-кода для утвержденных договорных документов.

```
    /// <summary>
    /// Создать или обновить экземпляр постраничной отметки для
парафирования договорных документов.
    /// </summary>
    [Public]
    public virtual void GetOrCreateContractPaginalApproveMark()
    {
      var markKindSid =
DEV
.Stamp
.PublicConstants.Contracts.ContractualDocument.PaginalApproveMarkKindSid;
      if (_obj.LastVersionApproved == true)
      {
        var mark = GetOrCreateMark(markKindSid);
        mark.XIndent = 0.1;
        mark.YIndent = -2;
        mark.Page = 0;
        mark.RotateAngle = 90;
        mark.Save();
      }
      else
      {
        var paginalApproveMark = GetVersionMarks(_obj.LastVersion.Id,
markKindSid).SingleOrDefault();
        Sungero.Docflow.PublicFunctions.Module.DeleteMark(_obj,
paginalApproveMark);
      }
    }

    /// <summary>
    /// Получить изображение отметки в виде HTML-кода для договорных
документов с состоянием "Утверждено".
    /// </summary>
    /// <param name="versionId">ИД версии.</param>
    /// <returns>Изображение отметки в виде HTML.</returns>
    [Public]
    public virtual string GetContractualApprovedMarkAsHtml(long versionId)
    {
      var version = _obj.Versions.Where(x => x.Id ==
versionId).FirstOrDefault();
      var signature =
Sungero.Docflow.PublicFunctions.OfficialDocument.GetSignatureForMark(_obj,
versionId, false);
```

```
        var html = string.Empty;
        using
(Sungero.Core.CultureInfoExtensions.SwitchTo(TenantInfo.Culture))
        {
          html =
Stamp.ContractualDocuments.Resources.HtmlMarkTemplatePaginalApprove;
          html = html.Replace("{version}", version.Number.ToString());
          var employee = Sungero.Company.Employees.As(signature.Signatory);
          html = html.Replace("{approvedOrganization}",
employee.Department.BusinessUnit.Name);
        }
        return html;
      }
```

- переопределите серверную функцию **ConvertToPdfWithMarks()** типа документа «Официальный документ» (OfficialDocument), которая используется для преобразования документа в PDF с добавлением отметок:

```
    /// <summary>
    /// Преобразовать документ в PDF с добавлением отметок.
    /// </summary>
    /// <param name="versionId">ИД версии документа, на который будут
проставлены отметки.</param>
    /// <returns>Результат преобразования.</returns>
    public override
Sungero.Docflow.Structures.OfficialDocument.IConversionToPdfResult
ConvertToPdfWithMarks(long versionId)
    {
      /// Пример перекрытия, в котором при выполнении действия
      /// "Создать PDF-документ с отметками" для договорных документов с
состоянием "Утверждено"
      /// добавляется отметка "Утверждено" на преобразованный в PDF
документ.
      this.GetOrCreateContractPaginalApproveMark()
      return base.ConvertToPdfWithMarks(versionId);
    }
```

4. Перекройте тип документа «Договор» (Contract) и добавьте для него серверную функцию **GetApprovedMarkAsHtml()**, которая получает отметку для договора с состоянием «Утверждено»:

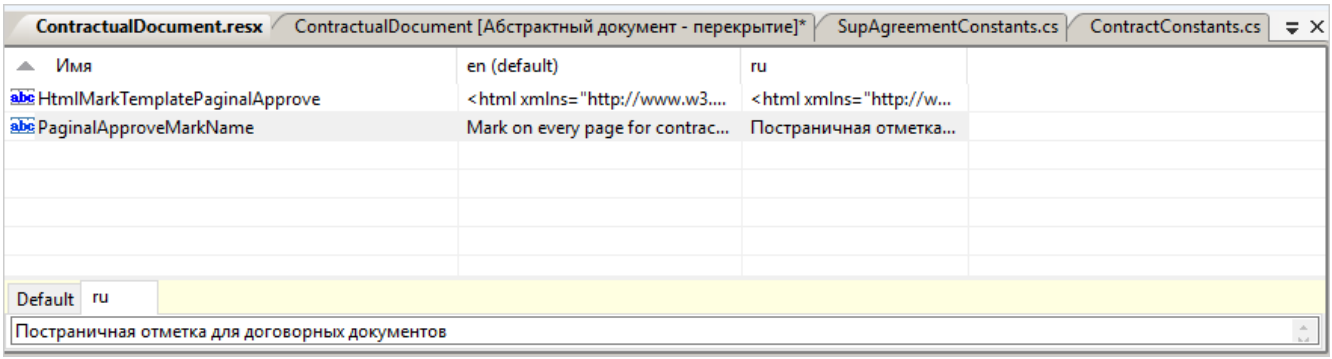
```
      /// <summary>
    /// Получить изображение отметки в виде HTML-кода для договорных
документов с состоянием "Утверждено".
    /// </summary>
    /// <param name="versionId">ИД версии.</param>
    /// <returns>Изображение отметки в виде HTML-кода.</returns>
    public virtual string GetApprovedMarkAsHtml(long versionId)
    {
      return
PublicFunctions.ContractualDocument.GetContractualApprovedMarkAsHtml(_obj,
versionId);
    }
```

5. Перекройте тип документа «Дополнительное соглашение» (SupAgreement) и добавьте для него аналогичную серверную функцию.
6. В перекрытый тип документа «Договорной документ» добавьте константы:

- **PaginalApproveMarkKindSid** – уникальный идентификатор отметки;
- **PaginalApproveMarkKindClass** – полное имя класса, из которого вызывается метод получения отметки;
- **PaginalApproveMarkKindMethod** – имя метода получения отметки.

```
public static class ContractualDocument
{// SID отметки "Утверждено".
  [Public]
  public const string PaginalApproveMarkKindSid = "3cdb9932-708f-4079-
bc50-890b700202c6";

  // Полное имя класса, из которого вызывается метод получения отметки
"Утверждено".
  [Public]
  public const string PaginalApproveMarkKindClass =
"DEV.Stamp.Functions.Contract";

  // Имя метода получения отметки "Утверждено".
  [Public]
  public const string PaginalApproveMarkKindMethod =
"GetApprovedMarkAsHtml";

 }
```

7. Задайте название для нового вида отметки (MarkKind). Для этого добавьте строку в ресурсах типа документа «Договорной документ»:

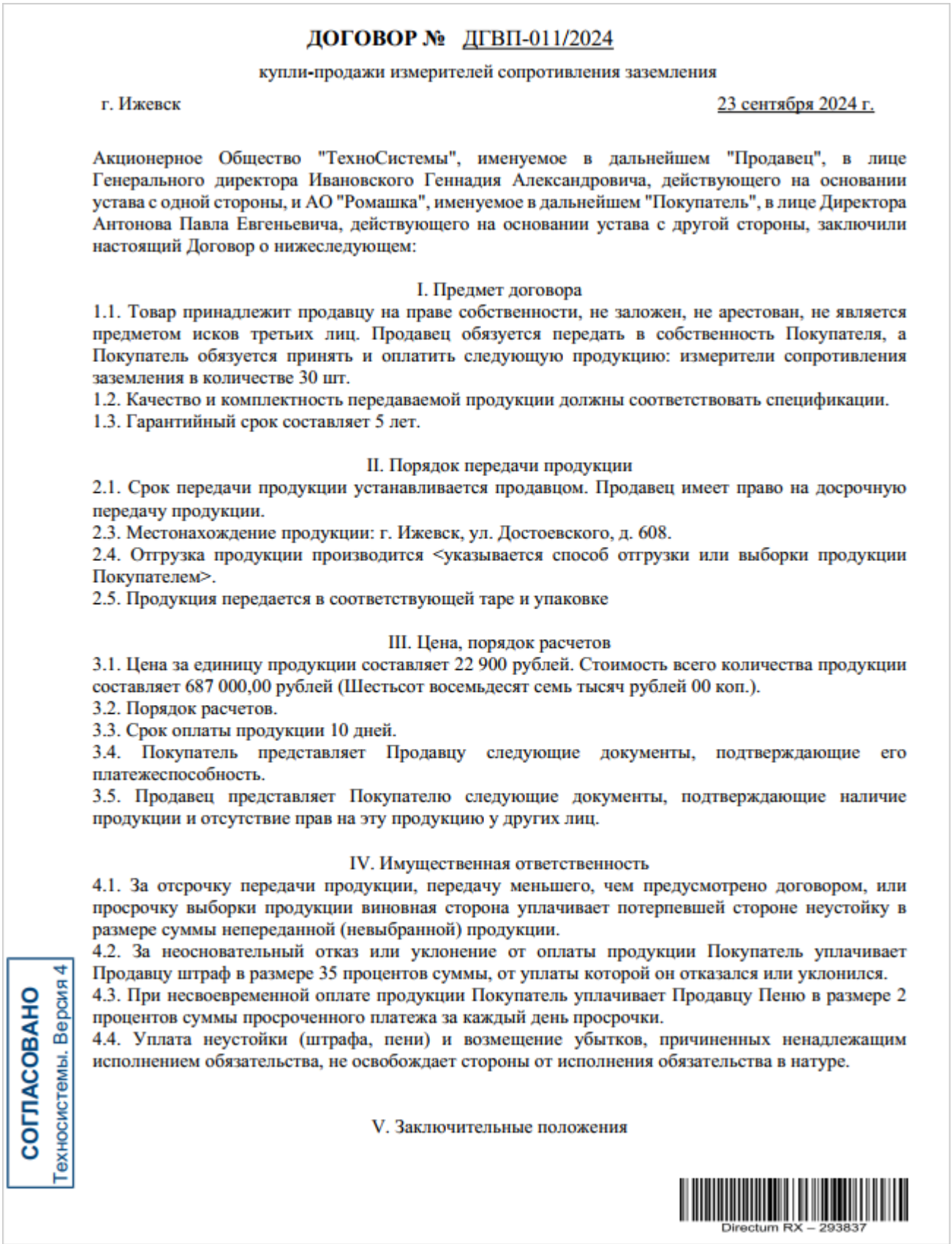


8. Перекройте модуль «Документооборот» (Docflow). В нем переопределите функцию инициализации **Initializing()** и добавьте функцию **CreateCustomMarkKinds()**, чтобы новый вид отметки создался в системе автоматически:

```
public override void
Initializing(Sungero.Domain.ModuleInitializingEventArgs e)
{
  base.Initializing(e);

  CreateCustomMarkKinds();
}

public static void CreateCustomMarkKinds()
{
  InitializationLogger.Debug("Init: Create custom mark kinds.");

        CreateMarkKind
(DEV.Stamp.ContractualDocuments.Resources.PaginalApproveMarkName,
DEV.Stamp.Constants.Contracts.ContractualDocument.PaginalApproveMarkKindSid,
                  DEV
.Stamp.Constants.Contracts.ContractualDocument.PaginalApproveMarkKindClass,
```

```
DEV
.Stamp.Constants.Contracts.ContractualDocument.PaginalApproveMarkKindMethod);
    }
```

В результате при преобразовании договоров и дополнительных соглашений в формат PDF на каждую страницу документа будет добавляться отметка:



ДОГОВОР № ДГВП-011/2024

купли-продажи измерителей сопротивления заземления

г. Ижевск 23 сентября 2024 г.

Акционерное Общество "ТехноСистемы", именуемое в дальнейшем "Продавец", в лице Генерального директора Ивановского Геннадия Александровича, действующего на основании устава с одной стороны, и АО "Ромашка", именуемое в дальнейшем "Покупатель", в лице Директора Антонова Павла Евгеньевича, действующего на основании устава с другой стороны, заключили настоящий Договор о нижеследующем:

I. Предмет договора

1.1. Товар принадлежит продавцу на праве собственности, не заложен, не арестован, не является предметом исков третьих лиц. Продавец обязуется передать в собственность Покупателя, а Покупатель обязуется принять и оплатить следующую продукцию: измерители сопротивления заземления в количестве 30 шт.
1.2. Качество и комплектность передаваемой продукции должны соответствовать спецификации.
1.3. Гарантийный срок составляет 5 лет.

II. Порядок передачи продукции

2.1. Срок передачи продукции устанавливается продавцом. Продавец имеет право на досрочную передачу продукции.
2.3. Местонахождение продукции: г. Ижевск, ул. Достоевского, д. 608.
2.4. Отгрузка продукции производится <указывается способ отгрузки или выборки продукции Покупателем>.
2.5. Продукция передается в соответствующей таре и упаковке

III. Цена, порядок расчетов

3.1. Цена за единицу продукции составляет 22 900 рублей. Стоимость всего количества продукции составляет 687 000,00 рублей (Шестьсот восемьдесят семь тысяч рублей 00 коп.).
3.2. Порядок расчетов.
3.3. Срок оплаты продукции 10 дней.
3.4. Покупатель представляет Продавцу следующие документы, подтверждающие его платежеспособность.
3.5. Продавец представляет Покупателю следующие документы, подтверждающие наличие продукции и отсутствие прав на эту продукцию у других лиц.

IV. Имущественная ответственность

4.1. За отсрочку передачи продукции, передачу меньшего, чем предусмотрено договором, или просрочку выборки продукции виновная сторона уплачивает потерпевшей стороне неустойку в размере суммы непереданной (невыбранной) продукции.
4.2. За неосновательный отказ или уклонение от оплаты продукции Покупатель уплачивает Продавцу штраф в размере 35 процентов суммы, от уплаты которой он отказался или уклонился.
4.3. При несвоевременной оплате продукции Покупатель уплачивает Продавцу Пеню в размере 2 процентов суммы просроченного платежа за каждый день просрочки.
4.4. Уплата неустойки (штрафа, пени) и возмещение убытков, причиненных ненадлежащим исполнением обязательства, не освобождает стороны от исполнения обязательства в натуре.

V. Заключительные положения

СОГЛАСОВАНО Технюсистемы. Версия 4

Directum RX – 293837

Как создать отметку об оплате входящих счетов

Ситуация

Необходимо создать отметку, подтверждающую оплату входящего счета.

Требования к положению отметки:

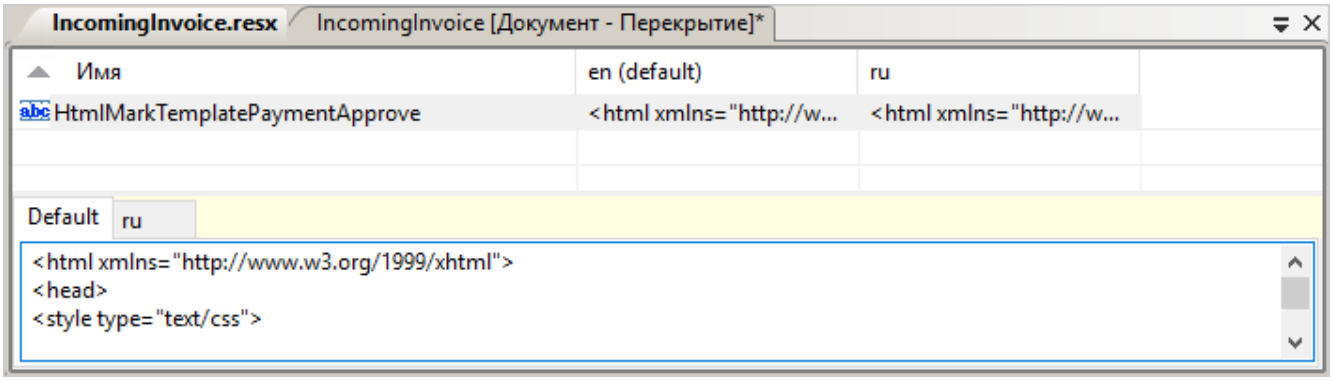
- отступ от левого края страницы – 12 см;
- отступ от верхнего края страницы – 20 см;
- номер страницы – последняя.

Отметка содержит надпись «Оплачено».

СОВЕТ. Чтобы использовать отметку об оплате в редакторе отметок, перекройте разделяемую функцию **GetAvailableMarkKindsSids()** типа документа «Официальный документ» (OfficialDocument).

Решение

1. Перекройте тип документа «Входящий счет» (IncomingInvoice).
2. Отметка формируется на основе данных, получаемых из HTML-кода. В ресурсах типа документа «Входящий счет» добавьте HTML-код отметки. Для этого в дереве решений в контекстном меню перекрытого типа документа выберите пункт **Открыть ресурсы**. Далее в контекстном меню табличной части выберите пункт **Добавить строку**. Укажите название нового ресурса, например **HtmlMarkTemplatePaymentApprove**, и добавьте код для изображения отметки:



ВАЖНО. Чтобы после преобразования документа в PDF отметка в нем отображалась корректно, необходимо использовать язык разметки HTML5 для задания ее внешнего вида.

Пример для ресурса с кодом отметки

```
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
<style type="text/css">
.box1{
  border: 2px solid #d7d7d7;
  border-color: green;
  width: 180px;
}
```

```
.tg1{
  font-size: 18pt;
  font-family: Arial, "Segoe UI", Consolas, Calibri, monospace;
  text-align: center;
  line-height: 9.75pt;
  color: green;
  white-space: nowrap
}
.small-td{
  font-size: 6pt;
  line-height: 6pt;
  white-space: break-spaces
}
</style>
</head>
<body style="margin: 0">
<table class="box1">
  <tr>
    <td class="small-td">
        <div class="tg1"><b>ОПЛАЧЕНО</b></div>
    </td>
  </tr>
</table>
</body>
</html>
```

3. В тип документа «Входящий счет» добавьте серверные функции:

   UpdateInvoicePaymentMark() – сохраняет экземпляр отметки для ее добавления в оплаченный входящий счет или удаления, если состояние документа изменилось с **Оплачено** на другое;

   GetPaymentMarkAsHtml() – получает изображение отметки в виде HTML-кода для оплаченного входящего счета.

```
   /// <summary>
/// Сохранить экземпляр отметки для ее добавления во входящий счет
/// с состоянием "Оплачено".
/// Удалить отметку, если состояние входящего счета изменено
/// с "Оплачено" на другое.
/// </summary>
 [Public]
public virtual void UpdateInvoicePaymentMark()
{
  if (_obj.LifeCycleState ==
Sungero.Contracts.IncomingInvoice.LifeCycleState.Paid)
  {
    var mark = GetOrCreateMark(IncomingInvoiceConstants.PaymentMarkKindSid);
    mark.XIndent = 12;
    mark.YIndent = 20;
    mark.Page = -1;
    mark.Save();
  }
  else
  {
    var paymentMark = GetVersionMarks(_obj.LastVersion.Id,
IncomingInvoiceConstants.PaymentMarkKindSid).SingleOrDefault();
    Docflow.PublicFunctions.Module.DeleteMark(_obj, paymentMark);
  }
}
```

```
/// <summary>
/// Получить изображение отметки в виде HTML-кода для входящего счета с
состоянием "Оплачено".
/// </summary>
/// <returns>Изображение отметки в виде HTML-кода.</returns>
public virtual string GetPaymentMarkAsHtml(long versionId)
{
  return Examples.IncomingInvoices.Resources.HtmlMarkTemplatePayment;
}
```

4. В тип документа «Входящий счет» добавьте одну из серверных функций:
   - **UpdateMarksBeforeConversion()**, если преобразование документа в PDF с добавлением отметок будет выполняться вручную, например в редакторе отметок или в прикладном блоке «Преобразование в PDF»:

     ```
     public override void UpdateMarksBeforeConversion(long versionId)
      {
        Functions.IncomingInvoice.Remote.UpdateInvoicePaymentMark(_obj);
      }
     ```

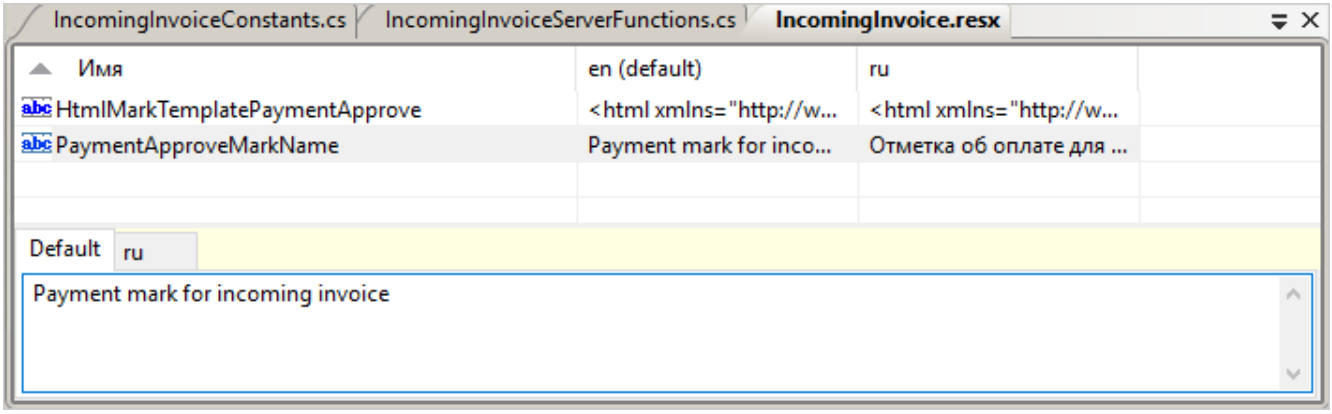
   - **ConvertToPdfWithMarks()**, если пользователь не сможет скорректировать расположение отметок при преобразовании документа в PDF с ними:

     ```
     public override
     Sungero.Docflow.Structures.OfficialDocument.IConversionToPdfResult
     ConvertToPdfWithMarks(long versionId)
      {
        this.UpdateInvoicePaymentMark();
        return base.ConvertToPdfWithMarks(versionId);
      }
     ```

5. В перекрытый тип документа «Входящий счет» добавьте константы:
   - **PaymentMarkKindSid** – уникальный идентификатор отметки;
   - **PaymentMarkKindClass** – полное имя класса, из которого вызывается метод получения отметки;
   - **PaymentMarkKindMethod** – имя метода получения отметки.

     ```
     public static class IncomingInvoice
     {
       // SID отметки "Оплачено".
       [Public]
       public const string PaymentMarkKindSid = "A89D9F46-9065-422A-B6CA-
      DED9D29EEDC7";

       // Полное имя класса, из которого вызывается метод получения отметки
     "Оплачено".
       [Public]
       public const string PaymentMarkKindClass =
     "Sungero.Examples.Functions.IncomingInvoice";

       // Имя метода получения отметки "Оплачено".
       [Public]
       public const string PaymentMarkKindMethod = "GetPaymentMarkAsHtml";
     }
     ```

6. Задайте название для нового вида отметки (MarkKind). Для этого добавьте строку в ресурсах типа документа «Входящий счет»:



7. Перекройте модуль «Документооборот» (Docflow). В нем переопределите функцию инициализации **Initializing()** и добавьте функцию **CreateCustomMarkKinds()**, чтобы новый вид отметки создался в системе автоматически:

```
public override void Initializing(Sungero.Domain.ModuleInitializingEventArgs e)
{
  base.Initializing(e);

  CreateCustomMarkKinds();
}

public static void CreateCustomMarkKinds()
{
  InitializationLogger.Debug("Init: Create custom mark kinds.");
  CreateMarkKind
(Sungero.Examples.IncomingInvoices.Resources.PaymentMarkName,
Sungero.Examples.Constants.Contracts.IncomingInvoice.PaymentMarkKindSid,
Sungero.Examples.Constants.Contracts.IncomingInvoice.PaymentMarkKindClass,
Sungero.Examples.Constants.Contracts.IncomingInvoice.PaymentMarkKindMethod);
}
```

В результате при преобразовании оплаченных входящих счетов в формат PDF добавится соответствующая отметка:

Вх. счет на оплату №112 от 04.11.2024 от Дрофа ООО "Оплата услуг ООО "Дрофа". Сумма 104 500 рублей."

		БИК	040407937
		Сч. №	30103810200000000937
Банк получателя АО "Азиатско-Тихоокеанский Банк"			
ИНН 1826484519	КПП 182945184	Сч. №	12816618391589436682
Получатель ООО «Дрофа»			

Счет № 112 от 04.11.2024 г.

Поставщик: **ООО «Дрофа»**

Покупатель:**ООО «ТехноСистемы»**

№	Наименование работ, услуг	Кол-во	Ед	Цена	Сумма
1	Оплата услуг	1	1	95 000	104 500

Итого: 104 500 руб.

В том числе НДС: 9 500 руб.

Всего к оплате: 104 500 руб.

ОПЛАЧЕНО

Руководитель Ивановский Геннадий Александрович 04.11.2024 г.

Бухгалтер Суворова Екатерина Андреевна 04.11.2024 г.

Как настроить работу редактора отметок

В среде разработки можно адаптировать работу с редактором отметок под потребности компании, например:

- добавить возможность использовать в редакторе отметку, созданную в среде разработки;
- запретить открытие редактора для определенного типа документа;
- изменить логику проверки документа перед открытием редактора.

ВАЖНО. Редактор отметок не поддерживает работу со следующими отметками: с поворотом, постраничными и у которых в записи справочника «Виды отметок» установлен флажок **На пустой странице**. Для типов документов, в которых они используются, необходимо запретить открытие редактора.

Рекомендуется тестировать работу отметок в редакторе на всех поддерживаемых браузерах.

Добавление отметки в редактор

Отметки, созданные в среде разработки, можно использовать в редакторе отметок. Для этого перекройте разделяемую функцию **GetAvailableMarkKindsSids()** типа документа «Официальный документ» (OfficialDocument).

Пример для отметки об оплате входящих счетов:

```
public override List<string> GetAvailableMarkKindsSids()
{
  var marksKinds = base.GetAvailableMarkKindsSids();

  if (_obj.LifeCycleState == Sungero.Contracts.IncomingInvoice.LifeCycleState.Paid)
    marksKinds
.Add(Sungero.Examples.Constants.Contracts.IncomingInvoice.PaymentMarkKindSid);

  return marksKinds;
}
```

Запрет на открытие редактора

Если работа с отметкой не поддерживается в редакторе, то необходимо запретить его открытие для типа документа, в который добавляется отметка. Для этого добавьте разделяемую функцию **AllowedToAddMarksManually()** типа документа «Официальный документ» (OfficialDocument):

```
public override bool AllowedToAddMarksManually()
{
  return false;
}
```

Изменение логики проверки перед открытием редактора

Перед работой в редакторе необходимо проверить, выполнены ли условия для его открытия. Например, чтобы добавить во входящий счет отметку об оплате, необходимо убедиться, что у него в поле **Состояние** указано значение **Оплачен**.

Чтобы изменить логику проверки, переопределите серверную функцию **ValidateMarksDataBeforeOpenMarkEditor()** типа документа «Официальный документ» (OfficialDocument).

Пример для отметки об оплате входящих счетов:

```
/// <summary>
/// Перед открытием редактора отметок проверить свойства документа, необходимые
для добавления отметок.
/// </summary>
/// <param name="versionId">ИД версии документа.</param>
/// <returns>Текст ошибки, если она есть. Иначе пустая строка.</returns>
 [Remote]
public override string ValidateMarksDataBeforeOpenMarkEditor(long versionId)
{
  var signature =
Docflow.PublicFunctions.OfficialDocument.GetSignatureForMark(_obj,
_obj.LastVersion.Id);
  var isPaid = _obj.LifeCycleState == LifeCycleState.Paid;

  if (signature == null && !isPaid)
    return
IncomingInvoices.Resources.DocumentShouldBeApprovedOrPaidToOpenMarkEditor;

  if (signature != null)
  {
    var separator = ". ";
    var validationErrors =
Docflow.PublicFunctions.Module.GetSignatureValidationErrorsAsString(signature,
separator);
    if (!string.IsNullOrEmpty(validationErrors))
    {
     return
Docflow
.OfficialDocuments
.Resources.SignatureNotValidErrorForMarkEditorFormat(validationErrors);
    }
  }

  return string.Empty;
}
```

Как получить данные без учета прав доступа

Ситуация

Сотрудникам необходимо просматривать сумму входящих счетов в разрезе организации. При этом у сотрудников может не быть прав на просмотр самих входящих счетов.

Решение

В карточку организации нужно добавить кнопку, при нажатии на которую выводится сумма входящих счетов, поступивших от выбранной организации. Для получения суммы без прав доступа используется метод AccessRights.AllowRead(). В клиентском коде нужно вызвать серверную функцию с этим методом и вывести сумму на экран.

1. В своем решении перекройте тип справочника «Организации» (Sungero.Parties.Company).
2. В редакторе решения добавьте зависимость от модуля «Договоры» (Sungero.Contracts).
3. Метод **AccessRights.AllowRead()** можно использовать только в серверном коде. Поэтому в перекрытом типе справочника создайте серверную функцию **InvoicesAmount()** и напишите ее код:

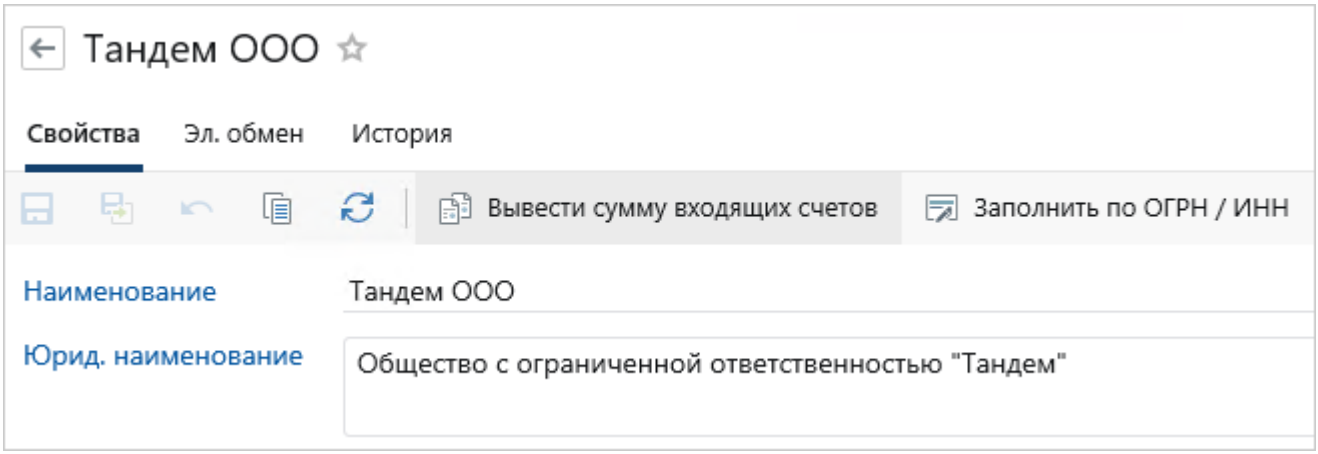
```
/// <summary>
  /// Получить сумму входящих счетов по организации.
  /// </summary>
  /// <returns> Сумма входящих счетов по организации.</returns>
  [Remote]
  public double InvoicesAmount()
  {

    // Задать переменную вещественного типа для суммы входящих счетов.
    var amount = 0.0;
    // Получить данные без учета прав доступа.
    AccessRights.AllowRead(
    () =>
    {
      // Вычислить сумму входящих счетов по организации.
      amount = (double)Sungero.Contracts.IncomingInvoices.GetAll().Where(d =>
Sungero.Parties.Counterparties.Equals(d.Counterparty, _obj)).ToList().Sum(t
=> t.TotalAmount);
    });
    return amount;
  }
```

4. В перекрытом типе справочника создайте действие **ShowAmountDEV** и в событии Выполнение вызовите ранее созданную серверную функцию. Для этого напишите код:

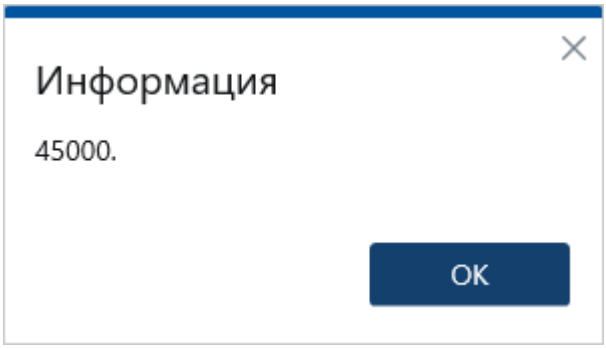
```
/// <summary>
  /// Показать сумму входящих счетов.
  /// </summary>
  /// <param name="e"></param>
  public virtual void ShowAmountDEV(Sungero.Domain.Client.ExecuteActionArgs e)
  {
    // Вызвать функцию для расчета суммы входящих счетов по организации.
    var amount = Functions.Company.Remote.InvoicesAmount(_obj);
    // Вывести сумму.
    Dialogs.ShowMessage(amount.ToString());
  }
```

5. На ленту карточки организации добавьте кнопку **Вывести сумму входящих счетов** и в качестве вызываемого действия выберите **ShowAmountDEV**.
6. Опубликуйте разработку.

В результате в карточке организации появится кнопка **Вывести сумму входящих счетов**:



При нажатии на кнопку выводится сумма входящих счетов по организации:

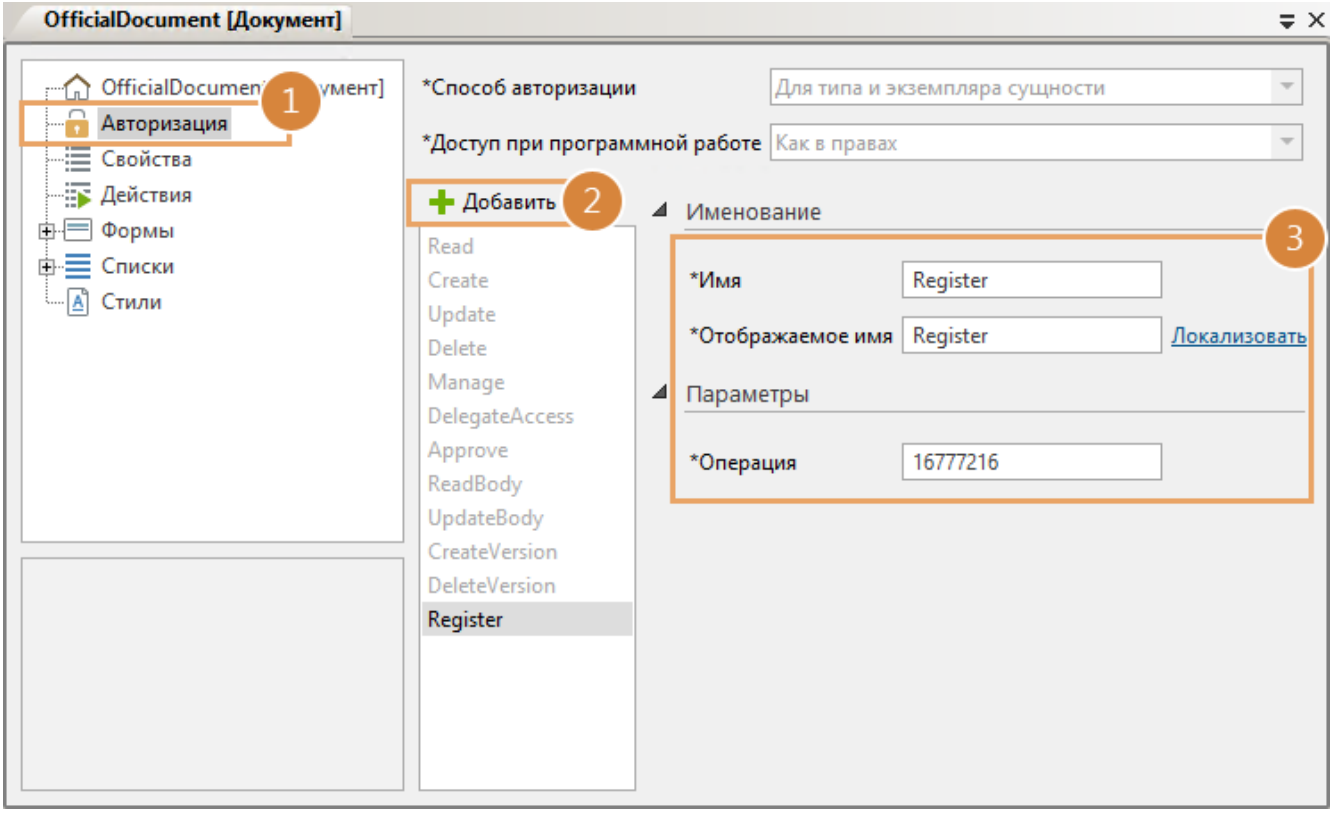


Как создать операцию авторизации для типа документа

В разделе описан пример создания операции авторизации для типа документа. Для остальных типов сущностей порядок создания аналогичный.

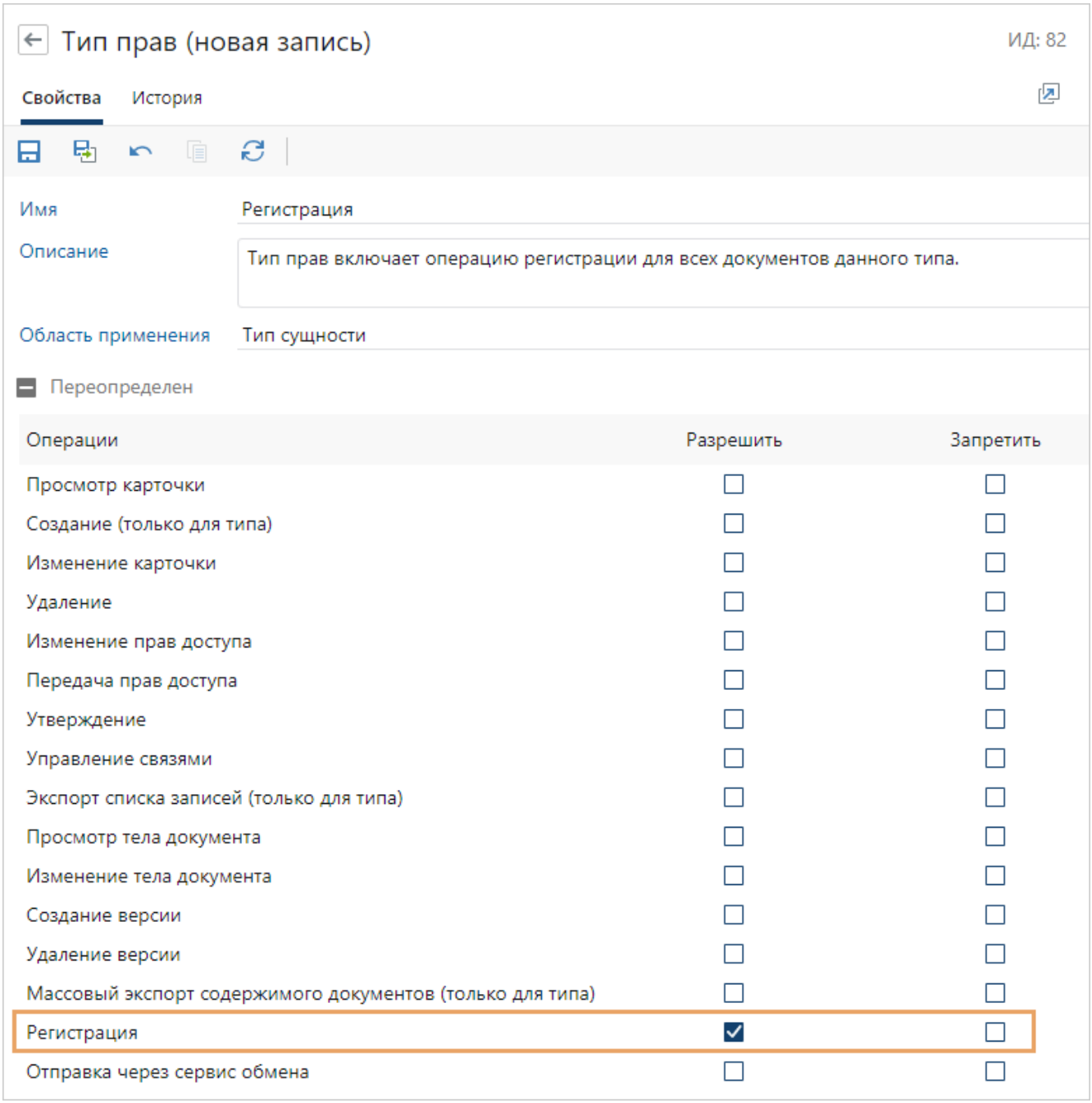
Предположим, у официального документа есть действие **Зарегистрировать**. Необходимо, чтобы выполнять действие могли пользователи, у которых есть права на изменение и регистрацию документа. Права на изменение определяются операцией авторизации **Update** (Изменение), которая наследуется от базового типа документа. Чтобы добавить проверку прав на регистрацию, создайте операцию авторизации. Для этого:

1. В редакторе типа сущности «Официальный документ» перейдите в узел «Авторизация»:

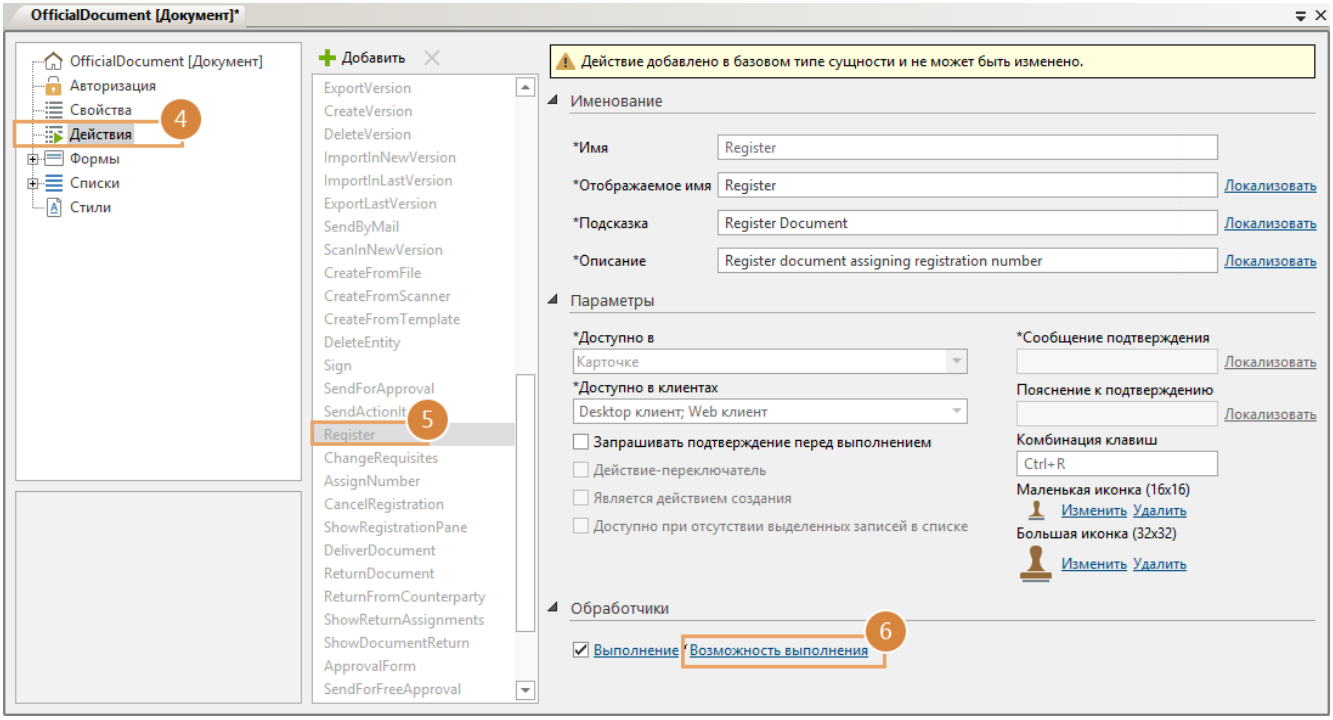


2. Нажмите на кнопку **Добавить**.

3. В списке элементов выделите добавленную операцию и в редакторе свойств задайте настройки для операции. В результате операция авторизации будет доступна в проводнике системы в списке **Типы прав** в карточках записей для всех наследников официального документа:



4. Чтобы добавить проверку прав на регистрацию документа для действия **Зарегистрировать**, перейдите в узел «Действия».



5. В списке элементов выберите действие Register (**Зарегистрировать**).
6. Добавьте код в обработчик события «Возможность выполнения». Для этого в редакторе свойств перейдите по ссылке **Возможность выполнения**. Откроется редактор программного кода:

```
public virtual bool CanRegister(Sungero.Domain.Client.CanExecuteActionArgs
e)
{
   //Действие "Регистрация" будет доступно, если у пользователя есть
   //права на изменение и регистрацию документа.
      return accessRights.CanUpdate() && accessRights.CanRegister()
}
```

В результате действие **Зарегистрировать** у официального документа будет доступно для пользователей, у которых есть права на изменение и регистрацию документа.

Как сгенерировать отчет и записать его в версию документа

Ситуация

После подписания договора необходимо в один клик формировать лист согласования и записывать его в новую версию договора.

Решение

В карточку договора нужно добавить кнопку и связать ее с действием, в событиях которого вызывается серверная функция. В функции формируется отчет «Лист согласования» и записывается в новую версию.

1. В своем решении перекройте тип документа «Договор» (Sungero.Contracts.Contract).
2. В редакторе решения добавьте зависимость от модуля «Договоры» (Sungero.Contracts) и «Документооборот» (Sungero.Docflow).
3. Метод **report.ExportTo()** можно использовать только в серверном коде. Поэтому в перекрытом типе документа создайте серверную функцию **GetApprovalSheetDEVRX()** и напишите ее код:

```
/// <summary>
  /// Сформировать лист согласования и добавить в новую версию документа.
  /// </summary>
  public virtual void GetApprovalSheetDEVRX()
  {
    // Получить отчет «Лист согласования».
    var report = Sungero.Docflow.Reports.GetApprovalSheetReport();
    report.Document = _obj;
    // Экспортировать отчет в новую версию документа.
    report.ExportTo(_obj);
    // Сохранить документ.
    _obj.Save();
  }
```

4. В перекрытом типе документа создайте действие **AddApprovalSheetDEVRX** и в событии Выполнение вызовите ранее созданную серверную функцию. Для этого напишите код:

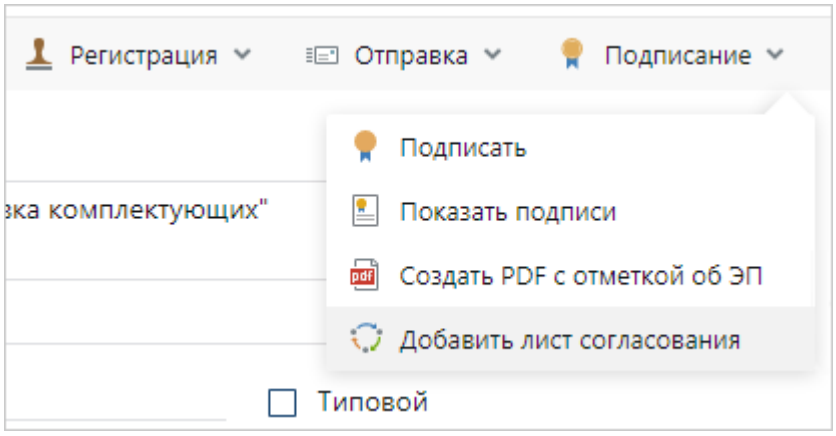
```
/// <summary>
  /// Добавить лист согласования в новую версию документа.
  /// </summary>
  /// <param name="e"></param>
  public virtual void
  AddApprovalSheetDEVRX(Sungero.Domain.Client.ExecuteActionArgs e)
  {
    Functions.Contract.Remote.GetApprovalSheetDEVRX(_obj);
  }
```

5. В карточке договора на ленту в группу «Подписание» добавьте кнопку **Добавить лист согласования** и в качестве вызываемого действия выберите **AddApprovalSheetDEVRX**.

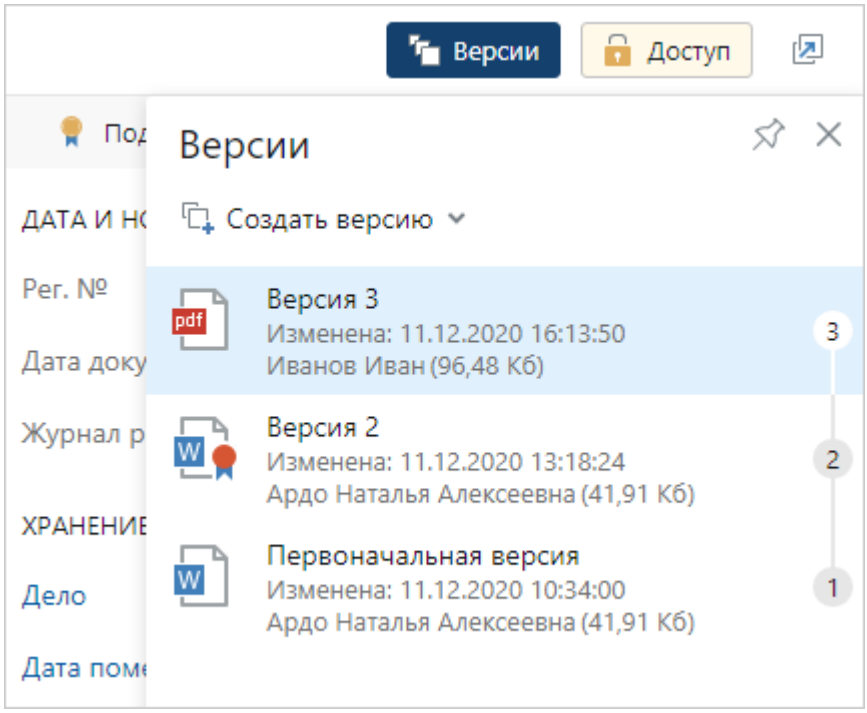
   Чтобы не возникало путаницы с названиями, рекомендуется скрыть стандартную кнопку **Лист согласования**. Она продублирована в группе «Отчеты», действие можно вызвать оттуда. Чтобы скрыть кнопку, выделите ее в редакторе форм в узле «Лента» и снимите флажок **Отображать на ленте**.

6. Опубликуйте разработку.

В результате в карточке договора появится кнопка **Добавить лист согласования**:



При нажатии на кнопку формируется лист согласования и записывается в новую версию договора:



Как настроить шифрование договоров

Ситуация

Предположим, необходимо, чтобы пользователи могли шифровать только договоры. Для всех остальных документов кнопка в карточке не должна отображаться. Кроме того, при попытке зашифровать договор должен появляться диалог подтверждения.

ВАЖНО. Шифрование документов возможно, если развернут сервис ключей.

Решение

Переопределим логику действия **ToggleEncryption**. Для этого:

1. В своем решении перекройте тип документа «Договор» (Sungero.Contracts.Contract).

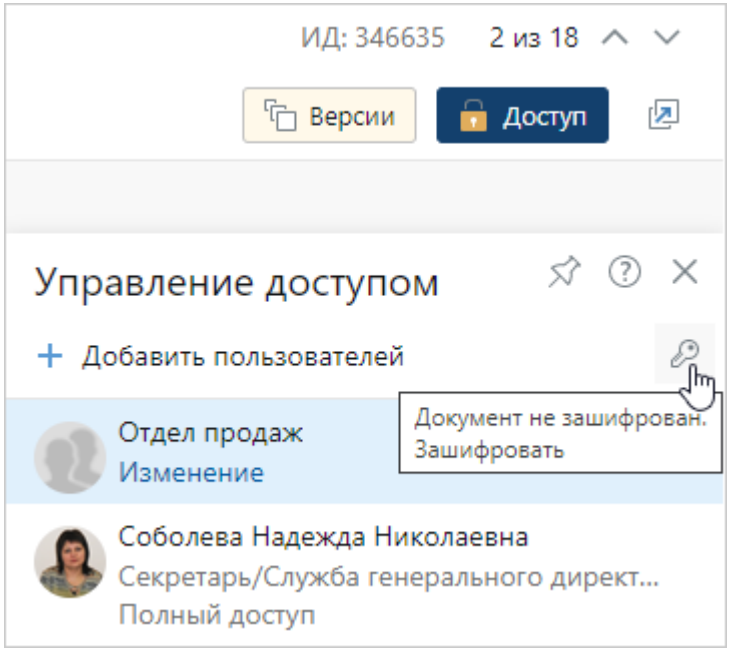
2. В редакторе решения добавьте зависимость от модуля «Договоры» (Sungero.Contracts).
3. В редакторе перекрытого типа документа перейдите в узел «Действия». В списке элементов выберите действие **ToggleEncryption**.
4. Установите флажок рядом с обработчиком **Выполнение / Возможность выполнения** и перейдите по ссылке.
5. Чтобы изменить стандартную логику, в открывшемся редакторе переопределите функции **ToggleEncryption()** и **CanToggleEncryption()**. Для этого напишите код:

```
public override void ToggleEncryption(Sungero.Domain.Client.ExecuteActionArgs e)
{
  // Создать диалог подтверждения.
  var dialog = Dialogs.CreateConfirmDialog("Вы уверены, что хотите включить шифрование?");
  // Зашифровать документ при подтверждении.
  if (dialog.Show())
  {
    base.ToggleEncryption(e);
  }
}
public override bool CanToggleEncryption(Sungero.Domain.Client.CanExecuteActionArgs e)
{
  // Показать кнопку шифрования, если документ – договор.
  return _obj.AccessRights.CanManage();
}
```

   ВАЖНО. Шифруется и содержимое версии документа (Body), и его представление (PublicBody).

6. Опубликуйте разработку.

В результате в карточках договоров появится кнопка, при нажатии появится окно подтверждения.



Также включить шифрование для нужных типов документов может администратор.

Как настроить строгий доступ к документу

Ситуация

Предположим, необходимо, чтобы пользователи могли включать строгий доступ для договоров. Для всех остальных документов кнопка в карточке не должна отображаться. Кроме того, при нажатии на кнопку должен появляться диалог подтверждения.

Решение

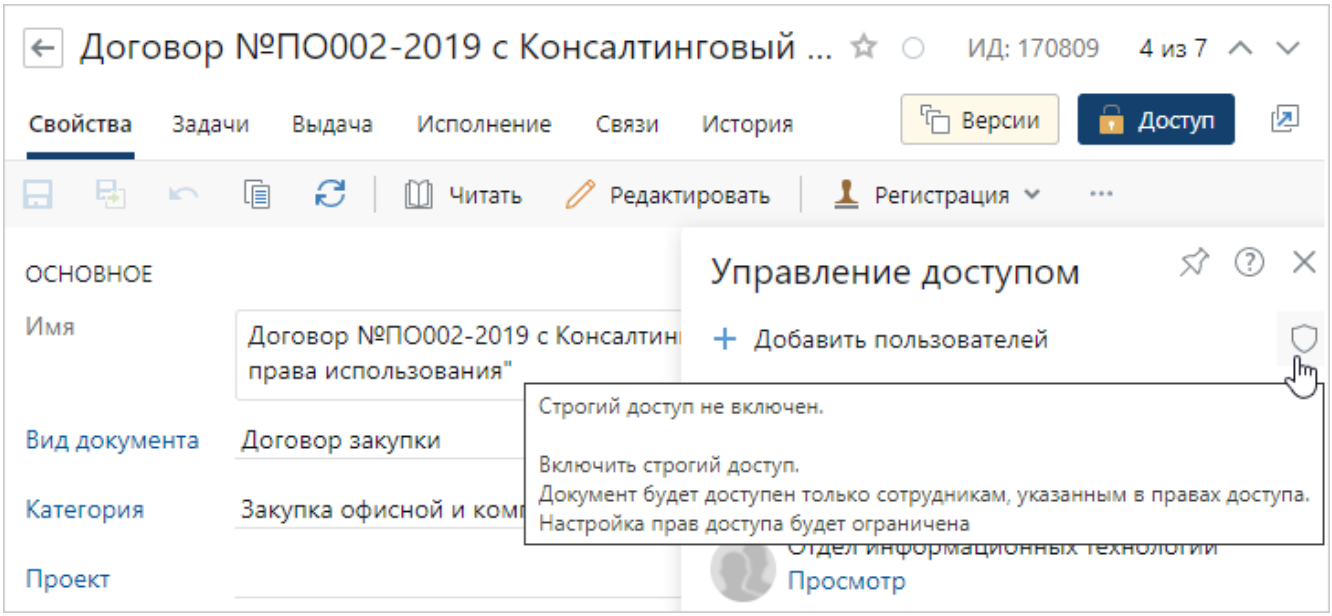
Переопределите логику действия ToggleStrictRights. Для этого:

1. В своем решении перекройте тип документа «Договор» (Sungero.Contracts.Contract).
2. В редакторе решения добавьте зависимость от модуля «Договоры» (Sungero.Contracts).
3. В редакторе перекрытого типа документа перейдите в узел «Действия». В списке элементов выберите действие **ToggleStrictRights**.
4. Установите флажок рядом с обработчиком **Выполнение / Возможность выполнения** и перейдите по ссылке.
5. Чтобы изменить стандартную логику, в открывшемся редакторе переопределите функции **ToggleEncryption()** и **CanToggleEncryption()**. Для этого напишите код:

```
public override void ToggleStrictRights(Sungero.Domain.Client.ExecuteActionArgs
    e)
   {
     // Создать диалог подтверждения.
     var dialog = Dialogs.CreateConfirmDialog("Вы уверены, что хотите включить
   строгий доступ?");
     // Включить строгий доступ при подтверждении.
     if (dialog.Show())
     {
       base.ToggleStrictRights(e);
     }
   }
   public override bool
   CanToggleStrictRights(Sungero.Domain.Client.CanExecuteActionArgs e)
   {
     // Показать кнопку щита, если документ – договор.
     return _obj.AccessRights.CanManage();
   }
```

6. Опубликуйте разработку.

В результате в карточках договоров появится кнопка, при нажатии на нее появится окно подтверждения.



ПРИМЕЧАНИЕ. Настроить строгий доступ для нужных типов документов также может администратор с помощью параметров конфигурационного файла.

Как запретить получение документов от конкретного филиала из сервиса обмена

Ситуация

В нашей организации «ООО, Юпитер» настроен обмен с контрагентом «ООО, Экспедитор» и его филиалом «ООО, Новый мир» через сервис обмена Диадок.

Необходимо настроить обмен так, чтобы документы поступали только от головной организации. Документы филиала отправляются напрямую в учетную систему, поэтому загружать их в Directum RX не нужно.

Решение

В Directum RX есть стандартная серверная функция **FilterLiteMessages()**, которая получает «легкие» сообщения с данными из сервиса обмена и фильтрует их. Подробнее см. раздел «Получение сообщений в фоновом режиме».

Чтобы задать свои настройки фильтрации данных из сервиса обмена, необходимо переопределить функцию:

1. В своем решении перекройте модуль «Электронный обмен» (Sungero.Exchange) и добавьте зависимость от него. Если зависимости не указаны, при публикации могут возникнуть ошибки.
2. Чтобы изменить стандартную логику, в редакторе перекрытого модуля переопределите серверную функцию **FilterLiteMessages()**. Для этого измените ее модификатор на **override** и доработайте код:

```
/// <summary>
    /// Отфильтровать «легкие» сообщения с информацией из сервиса обмена.
    /// </summary>
    /// <param name="businessUnitBox">Абонентский ящик нашей
    организации.</param>
    /// <param name="messages">Сообщения.</param>
    /// <returns>Список отфильтрованных сообщений.</returns>
    public override List<NpoComputer.DCX.Common.IMessage>
    FilterLiteMessages(Sungero.ExchangeCore.IBusinessUnitBox businessUnitBox,
    List<NpoComputer.DCX.Common.IMessage> messages)
    {
          var filteredMessages = base.FilterLiteMessages(businessUnitBox,
    messages);

          // ИД и код филиала контрагента в Диадок, с которым настроен обмен.
          var diadocSenderBoxId = "405b3fb0af6a4215b89bab864ff8aca0@diadoc.ru";
          var diadocSenderDepartmentCode = "6e9ab7bd-576d-4fc2-9a1e-
    9282ec8bebb3";

          // Запретить обрабатывать входящие документы от филиала «ООО, Новый
    мир» с ИД=6e9ab7bd-576d-4fc2-9a1e-9282ec8bebb3.
          filteredMessages = filteredMessages.Where(x => string.Equals(x.Sender?
    .BoxId, diadocSenderBoxId) && string.Equals(x.FromDepartment?.Id,
    diadocSenderDepartmentCode)).ToList();
          return filteredMessages;
    }
```

3. Опубликуйте разработку.

В результате из сервиса обмена Диадок в Directum RX смогут загружаться и отправляться на обработку документы только от головной организации контрагента, а от филиала документы загружаться не будут.

Согласование по регламенту

В разделе содержатся примеры:

- Как добавить результат выполнения в задание на согласование?
- Как добавить роль согласования с несколькими участниками?
- Как добавить критерий в правило согласования?
- Как проинформировать согласующего об ограничениях в полномочиях?
- Как зафиксировать дополнительную информацию о согласовании в тексте задания?

- Как в задаче по регламенту заполнить поле «Адресат» значением из нового типа документа?
- Как создать новый тип сценария для правила согласования?

Как добавить результат выполнения в задание на согласование

Ситуация

В компании настроено правило согласования документов с уменьшающимся кругом согласующих. При согласовании документа по данному правилу сотрудник может выполнить задание с одним из результатов:

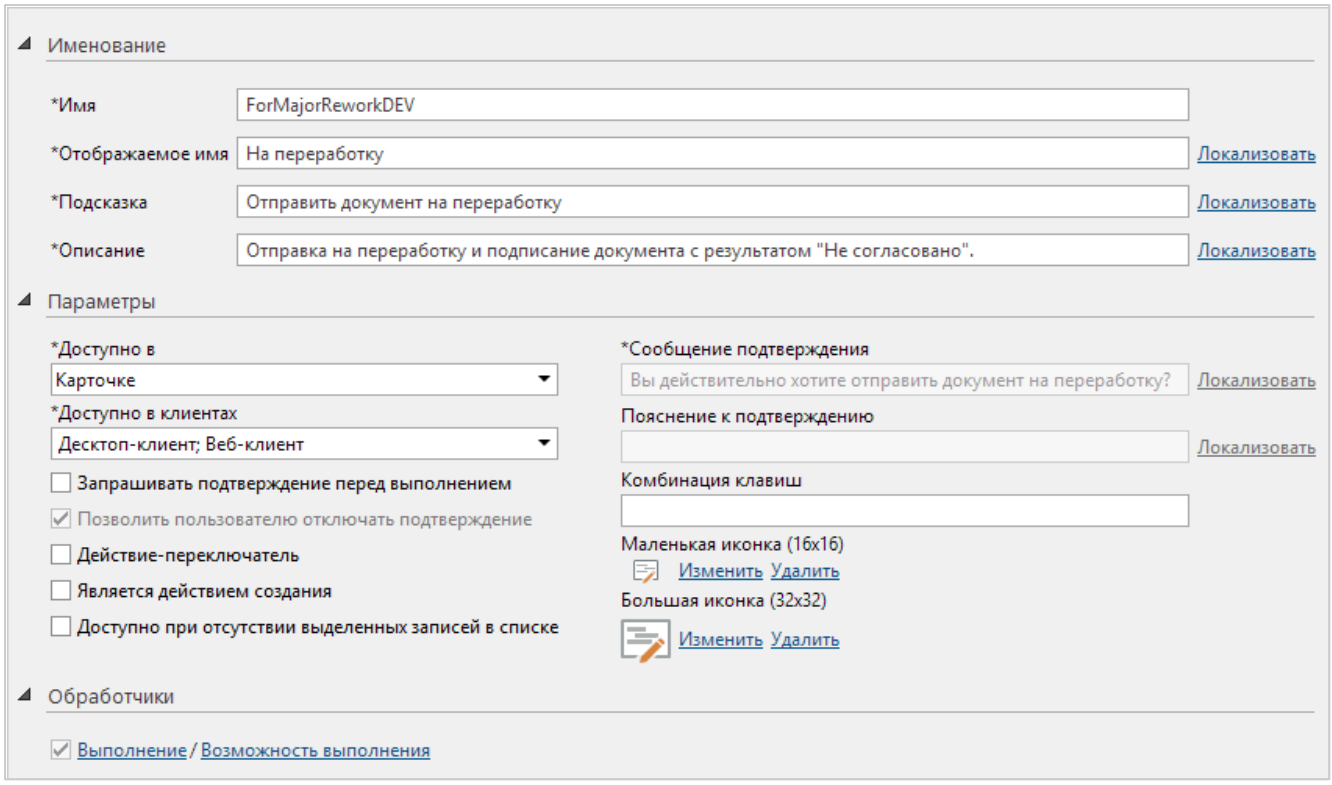
- **Согласовать**, если содержимое документа и его приложений полностью устраивает;
- **На доработку**, если есть незначительные замечания к содержимому документа или приложений. После доработки документа задание на повторное согласование придет только тем, кто выбрал этот результат выполнения;
- **Переадресовать**, если согласование документа необходимо передать другому сотруднику.

Нужно добавить результат выполнения **На переработку**. Согласующий будет использовать его вместо результата **На доработку**, если в документ нужно внести большие изменения. При выполнении задания с этим результатом после исправления документ должен прийти на следующий круг согласование всем согласующим, даже если ранее они согласовали документ.

Решение

1. Перекройте тип задания «Согласование» (ApprovalAssignment) из базового решения Directum RX.
2. В редакторе решения, в котором перекрыт тип задания, добавьте зависимость от модуля «Документооборот» (Docflow). Если зависимости не указаны, при публикации могут возникнуть ошибки.
3. В редакторе перекрытого типа задания добавьте свойство **На переработку** (ForMajorRework). Тип свойства: **Логическое**. Свойство будет принимать значение **true** при выполнении задания с результатом **На переработку**.
4. В перекрытый тип задания нельзя добавить результат выполнения **На переработку**. Кнопка **Добавить** недоступна, так как:
   - тип задания «Согласование» используется в схеме задачи на согласование по регламенту (ApprovalTask);
   - в редакторе типа «Задача на согласование по регламенту» установлен флажок **Запрет создания версий схемы в перекрытиях задачи**.

Поэтому вместо результата выполнения добавьте действие **На переработку** (ForMajorRework). Заполните поля в группах «Именование» и «Параметры», как показано на рисунке:



Чтобы заполнить поле ***Сообщение подтверждения**, установите флажок **Запрашивать подтверждение перед выполнением**. После заполнения поля снимите флажок.

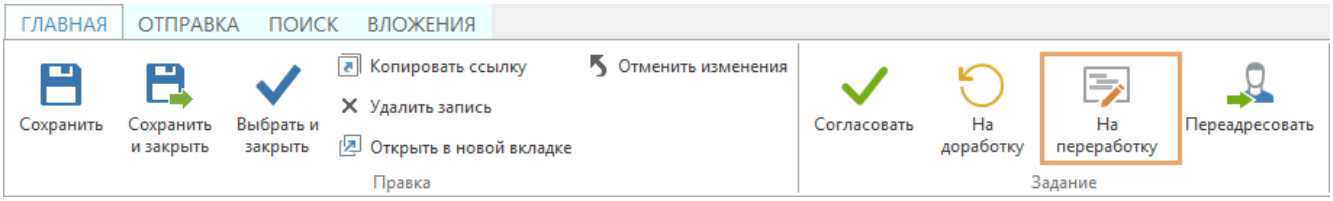
ПРИМЕЧАНИЕ. Локализуйте введенные значения. Для этого используйте ссылки **Локализовать**.

5. Для нового действия добавьте обработчик Выполнение:

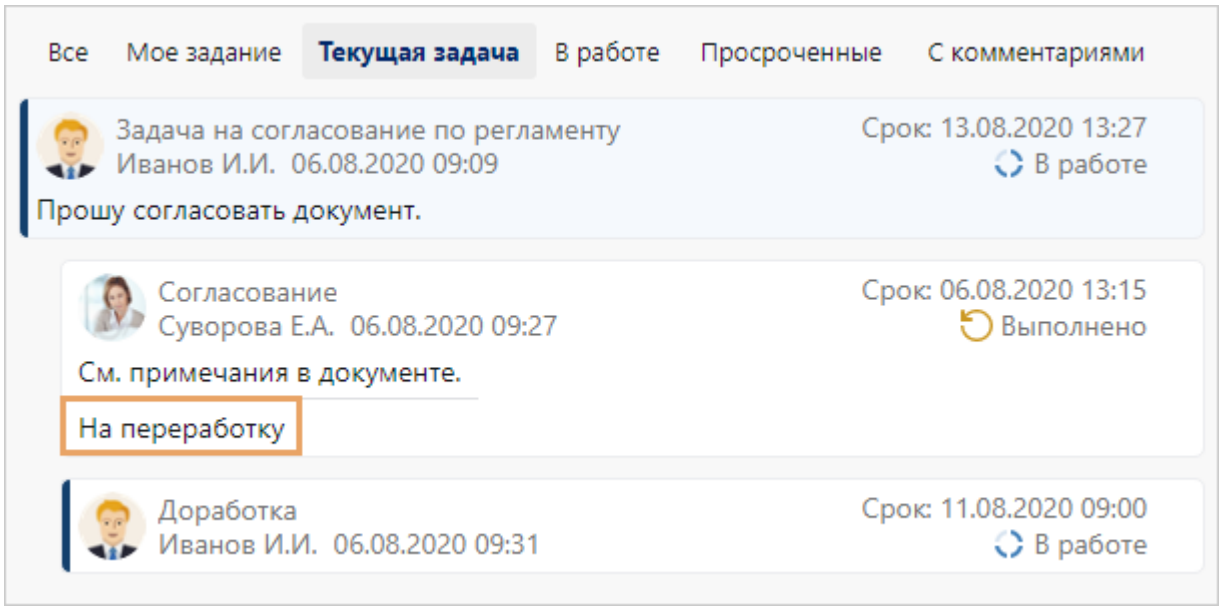
```
public virtual void RejectApprovalDEV(Sungero.Domain.Client.ExecuteActionArgs
  e)
  {
    /*При выполнении задания система пытается получить доступ на редактирование
  карточки вложенного документа, поэтому проверяем вложение на блокировку.*/
    var accessInfo =
  Locks.GetLockInfo(_obj.DocumentGroup.OfficialDocuments.FirstOrDefault());
    if (accessInfo.IsLocked)
    {
      e.AddError(accessInfo.LockedMessage);
      return;
    }

    /*Создаем объект action с типом ExecuteResultActionArgs, который нужен для
  вызова базового обработчика события "Выполнение".*/
    var action = new
  Sungero.Workflow.Client.ExecuteResultActionArgs(e.FormType, e.Entity,
  e.Action);
    /*Вызываем базовый обработчик события "Выполнение" с результатом "На
  доработку".*/
    base.ForRevision(action);
    _obj.ForMajorReworkDEV = true;
```

```
    // Закрываем карточку задания после выполнения.
    e.CloseFormAfterAction = true;
    // Выполняем задание.
    _obj.Complete(Result.ForRevision);
  }
```

6. Добавьте на ленту кнопку и свяжите с действием **На переработку**.



7. В переписке по задаче при выполнении задания с результатом **На переработку** должен отображаться соответствующий текст.



Для этого в перекрытом типе задания переопределите обработчик серверного события До выполнения:

```
public override void
  BeforeComplete(Sungero.Workflow.Server.BeforeCompleteEventArgs e)
  {
    base.BeforeComplete(e);
    if (_obj.ForMajorReworkDEV.GetValueOrDefault())
        e.Result =
DEV.PerekrytiiaExample.ApprovalAssignments.Resources.RejectApproval;
  }
```

8. Чтобы отследить результат выполнения на следующих этапах согласования, перекройте тип «Задача на согласование по регламенту» (ApprovalTask) и также добавьте свойство **На переработку** (ForMajorRework). Тип свойства: **Логическое**.

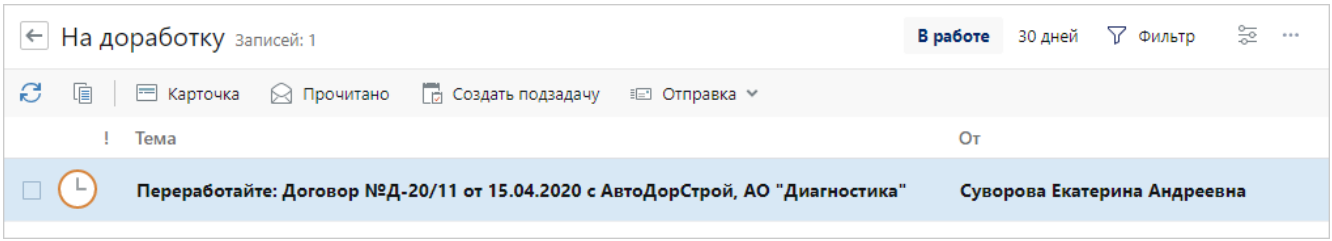
9. Перейдите в узел «Схема», выделите блок **Согласование с согласующими** и добавьте обработчик события Конец блока:

```
public override void
  EndBlock6(Sungero.Docflow.Server.ApprovalAssignmentEndBlockEventArguments e)
  {
    base.EndBlock6(e);
```

```
  if (e.CreatedAssignments.Cast<
DEV.SolutionResult.IApprovalAssignment>().Any(a =>
a.ForRemakeDEV.GetValueOrDefault()))
  _obj.ForMajorReworkDEV = true;
}
```

ПРИМЕЧАНИЕ. Нельзя заполнять свойство **На переработку** (ForMajorRework) прямо из задания при выполнении, т.к. при замещении у пользователя могут отсутствовать права на основную задачу.

10. Для блока **Доработка** добавьте обработчик события Старт блока, чтобы:
    - инициатор согласования получил задание с темой «Переработайте:<Имя документа>»:



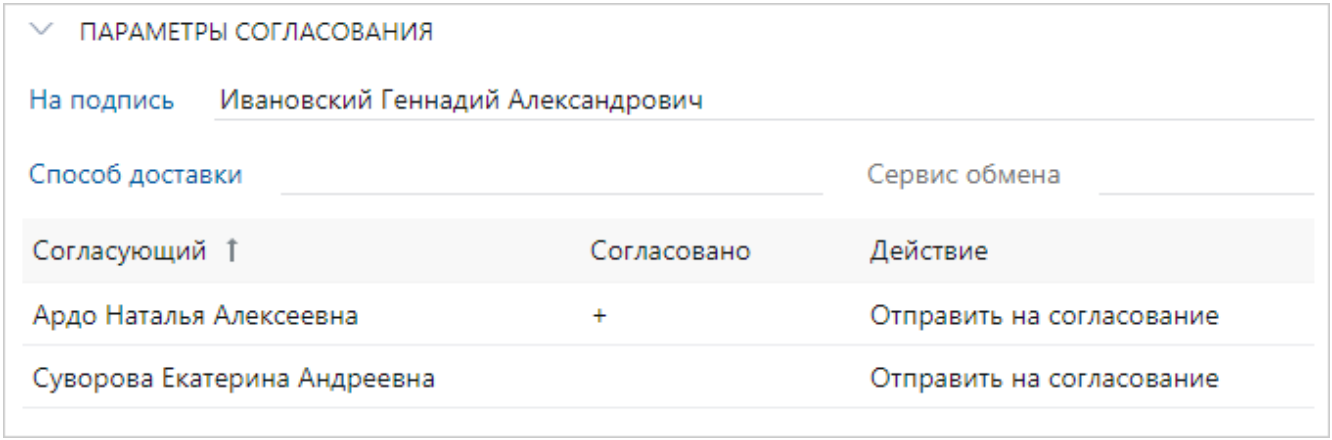
    - после выполнения задания на доработку документ пришел на повторное согласование всем согласующим.

```
public override void
StartBlock5(Sungero.Docflow.Server.ApprovalReworkAssignmentArguments e)
{
  base.StartBlock5(e);
  if (_obj.ForMajorReworkDEV.GetValueOrDefault())
  {
    // Меняем начало темы с «Доработайте» на «Переработайте».
    e.Block.Subject = e.Block.Subject.Replace("Доработайте",
"Переработайте");

    var approvalAssignments = ApprovalAssignments.GetAll(a => Equals(a.Task,
 _obj) && a.Created >= _obj.Started).ToList();
    foreach (var approver in e.Block.Approvers)
    {
      // Исключаем исполнителей, которые переадресовали задание.
      var approvalAssignment = approvalAssignments
        .Where(a => Equals(a.Performer, approver.Approver))
        .OrderByDescending(i => i.Modified)
        .FirstOrDefault();

      var forwarded = approvalAssignment != null &&
approvalAssignment.Result ==
Sungero.Docflow.ApprovalAssignment.Result.Forward;
      if (forwarded)
          approver.Action =
Sungero.Docflow.ApprovalReworkAssignmentApprovers.Action.DoNotSend;
      else
          approver.Action =
Sungero.Docflow.ApprovalReworkAssignmentApprovers.Action.SendForApproval;
    }
  }
}
```

11. Также для блока **Доработка** добавьте обработчик события Конец блока. В обработчике измените значение свойства **На переработку** (ForMajorRework) на **false**, чтобы на следующих кругах согласования была возможность доработать документ с уменьшающимся кругом согласующих.

```
public override void
  EndBlock5
  (Sungero.Docflow.Server.ApprovalReworkAssignmentEndBlockEventArguments e)
  {
    base.EndBlock5(e);
    _obj.ForMajorReworkDEV = false;
  }
```

12. Чтобы инициатор не смог изменить состав согласующих после переработки документа, сделайте недоступной табличную часть в группе «Параметры согласования».



Для этого перекройте тип задания «Доработка» (ApprovalReworkAssignment) и добавьте обработчик клиентского события Обновление формы:

```
public override void Refresh(Sungero.Presentation.FormRefreshEventArgs e)
  {
    base.Refresh(e);
    if (ApprovalTasks.As(_obj.Task).ForMajorReworkDEV.GetValueOrDefault())
      _obj.State.Properties.Approvers.IsEnabled = false;
  }
```

В результате документ можно будет отправить на доработку и на переработку.

Как добавить роль согласования с несколькими участниками

Ситуация

Предположим, договор на закупку нужно согласовать с ответственными по направлениям, по которым проводится приобретение товаров. Например, договор на закупку мониторов дополнительно согласуется с руководителем отдела ИТ. В результате сотрудников, согласующих документ, будет несколько, поэтому требуется добавить роль согласования со списком участников.

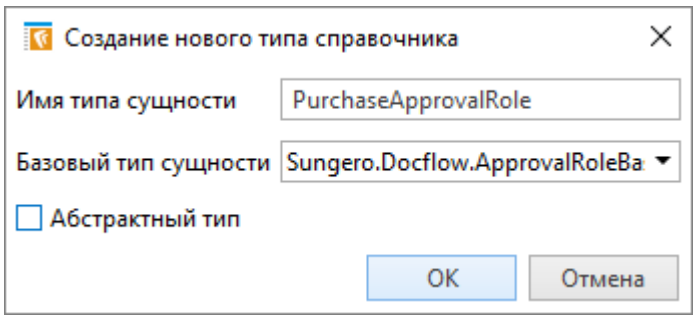
Роли согласования предназначены для вычисления исполнителей заданий, которые формируются при согласовании документа по регламенту. Используются для настройки этапов согласования.

Решение

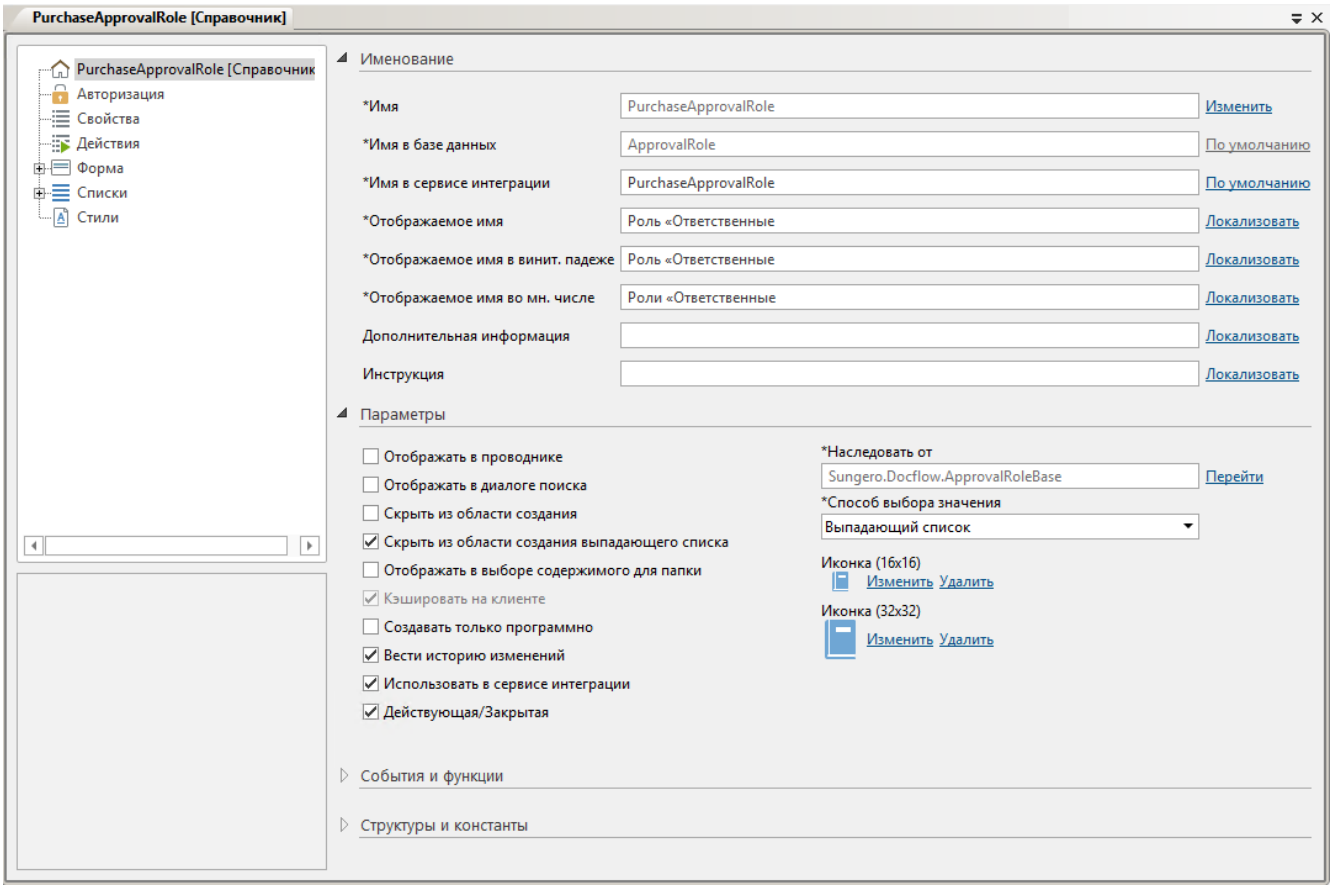
В системе роль согласования – это справочник, записи которого доступны для выбора в этапах согласования. Роль согласования с несколькими участниками создается по аналогии с созданием роли с одним сотрудником, но есть свои отличия.

Вычисление сотрудников для роли «Ответственные»

1. В своем решении, например PurchasesSolution, создайте модуль «Закупки» (Purchases).
2. Чтобы добавить тип роли **Ответственные**, создайте в модуле Purchases тип справочника «Роль «Ответственные». В качестве базового типа сущности для него укажите тип Sungero.Docflow.ApprovalRoleBase. В поле **Имя типа сущности** укажите PurchaseApprovalRole.

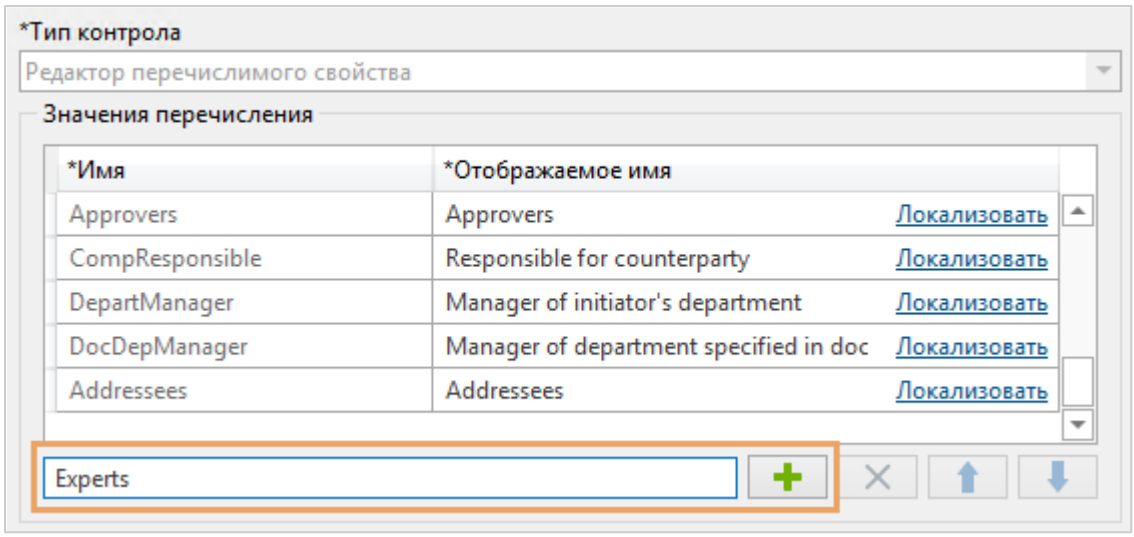


3. Откройте редактор созданного типа справочника:

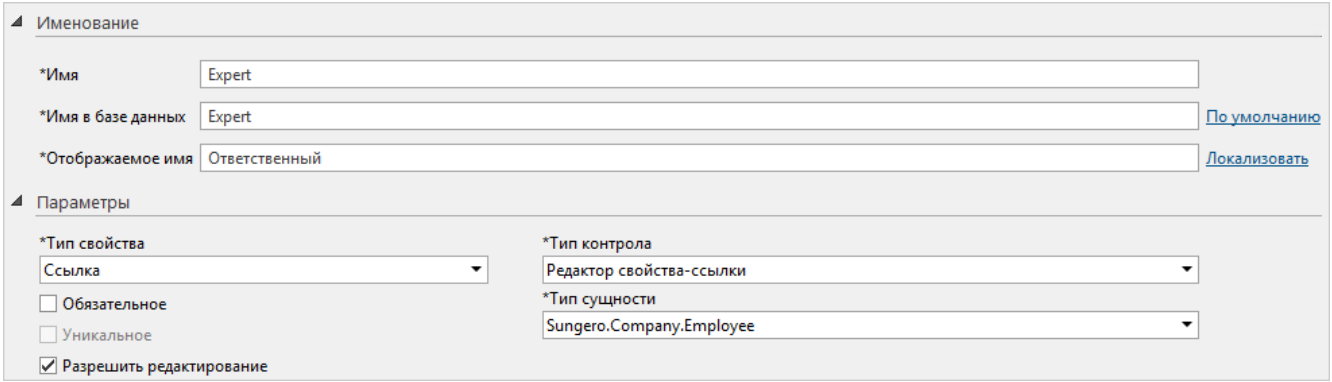


- измените именование типа справочника. Для локализации отображаемых имен перейдите по ссылке **Локализовать**;
- проверьте, что в параметрах снят флажок **Отображать в проводнике**.

4. Перейдите в узел «Свойства» и для свойства-перечисления **Type** добавьте значение перечисления Experts (Ответственные).



5. В своем решении перекройте тип договора Sungero.Contracts.Contract. Проверьте, что в списке зависимостей вашего решения указан модуль Sungero.Contracts. Если в решении не указаны зависимости от перекрытых модулей, то при публикации могут возникнуть ошибки.
6. Добавьте в договор список ответственных. Для этого перейдите в узел «Свойства» и добавьте в нем свойство-коллекцию Experts. Затем в коллекцию добавьте свойство-ссылку Expert. В поле ***Тип сущности** укажите Sungero.Company.Employee.



7. Добавьте контрол на форму перекрытого типа договора, в качестве связанного свойства укажите новое свойство-коллекцию Experts.
8. Чтобы при согласовании документов для новой роли вычислялись ответственные, заполненные в карточке договора, переопределите функцию **GetRolePerformers()**. Для этого в серверных функциях созданного типа справочника PurchaseApprovalRole напишите код:

```
// Получить список исполнителей из роли.
    // <param name="task">Задача.</param>
    // <returns>Список исполнителей.</returns>
    [Remote(IsPure = true), Public]
    public List<Sungero.CoreEntities.IRecipient>
    GetRolePerformers(Sungero.Docflow.IApprovalTask task)
    {
```

```
  var result = new List<Sungero.CoreEntities.IRecipient>();
  var document = task.DocumentGroup.OfficialDocuments.FirstOrDefault();
  var contract = PurchasesSolution.Contracts.As(document);
  if (contract!= null && _obj.Type ==
Purchases.PurchaseApprovalRole.Type.Experts)
  {
    foreach (var item in contract.Experts.Where(x => x.Expert != null))
      result.Add(item.Expert);
  }
  return result;
}
```

9. Чтобы правильно определялись исполнители этапа согласования с ответственными за направление закупок, перекройте справочник Sungero.Docflow.ApprovalStage. Проверьте, что в списке зависимостей вашего решения указан модуль Sungero.Docflow.

10. Переопределите логику серверной функции **GetStageRecipients()** справочника ApprovalStage. Функция вызывается при вычислениях исполнителей в обработчиках событий блоков схемы задачи и построении отображения на закладке «Задачи» карточки документа. Для этого в серверных функциях перекрытого типа справочника ApprovalStage напишите код:

```
// Получить исполнителей этапа без раскрытия групп и ролей.
  // <param name="task">Задача.</param>
  // <param name="additionalApprovers">Доп.согласующие.</param>
  // <returns>Исполнители.</returns>
  [Remote(IsPure = true), Public]
  public override List<IRecipient>
  GetStageRecipients(Sungero.Docflow.IApprovalTask task, List<IRecipient>
  additionalApprovers)
  {
    var recipients = base.GetStageRecipients(task, additionalApprovers);
    var role = _obj.ApprovalRoles.Where(x => x.ApprovalRole.Type ==
  Purchases.PurchaseApprovalRole.Type.Experts)
      .Select(x => Purchases.PurchaseApprovalRoles.As(x.ApprovalRole))
      .Where(x => x != null).SingleOrDefault();
    if (role!= null)
    {
      recipients
  .AddRange
  (Trade
  .Purchases
  .PublicFunctions.PurchaseApprovalRole.Remote.GetRolePerformers(role, task));
    }
    return recipients;
  }
```

ВАЖНО. В примере кода замените Trade на ваш <Код компании>.

11. Сохраните изменения и опубликуйте разработку. Далее инициализируйте роль и ограничьте ее доступность аналогично роли с одним участником.

ПРИМЕЧАНИЕ. В примере сотрудники из карточки договора, которые включаются в список участников роли «Ответственные», не отображаются в поле **Обязательные** при создании задачи на согласование по регламенту. При этом, если перейти в задаче на закладку «Регламент», то на схеме будут отображаться все исполнители. Чтобы все исполнители

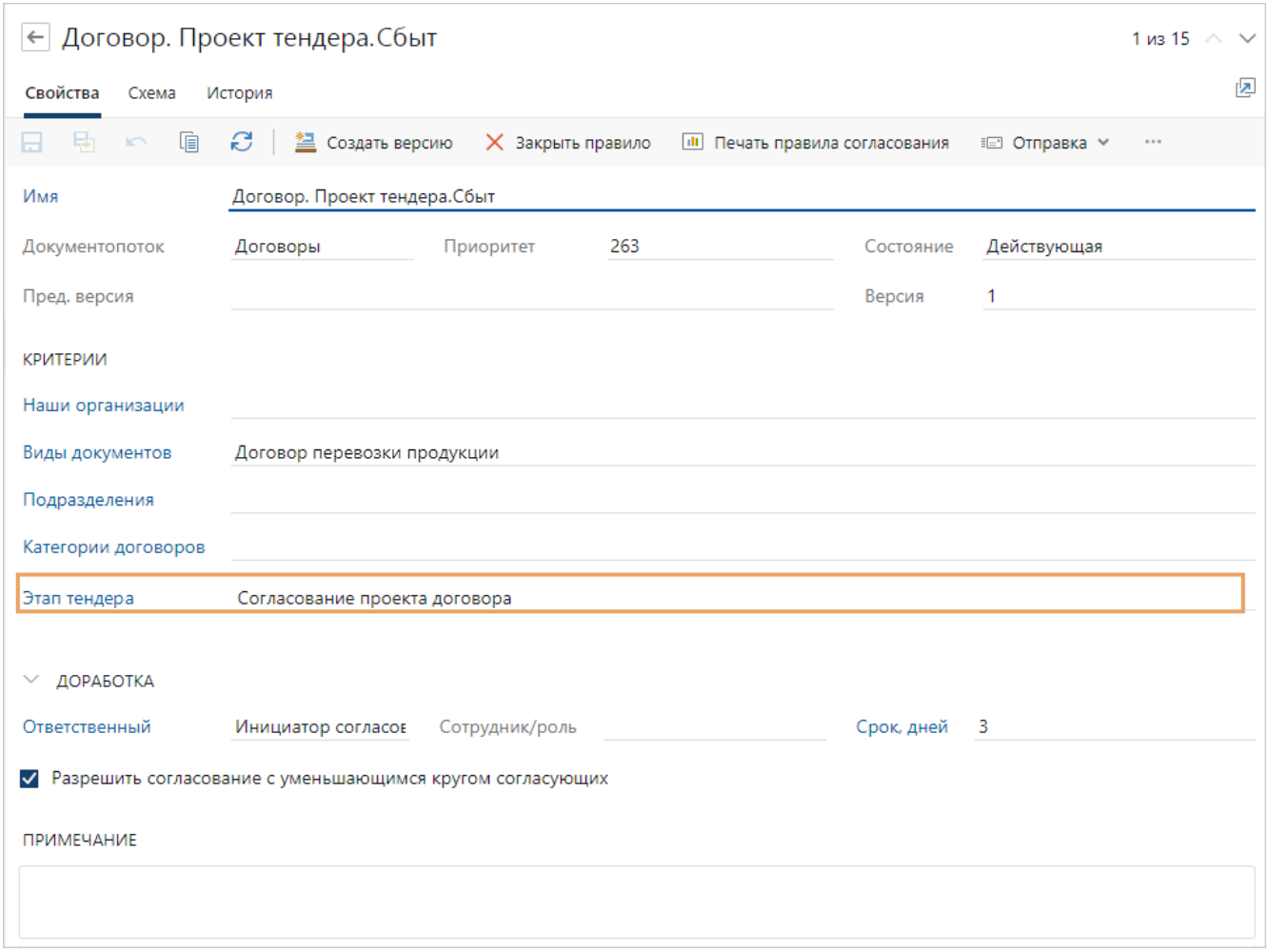
отображались и в карточке задачи, необходимо в программном коде дописать логику добавления их в поле **Обязательные**.

Как добавить критерий в правило согласования

Ситуация

В задаче на согласование по регламенту на основании критериев вычисляется наиболее подходящее правило согласования по конкретному документу. Критерии задаются в карточке правила согласования. Чем больше критериев указано, тем меньше список документов, для которых действует правило и тем выше его приоритет. Правило с максимальным приоритетом автоматически подставляется в карточку задачи на согласование по регламенту в поле ***Регламент**. Остальные подходящие правила доступны сотруднику для выбора в карточке задачи. Иногда возникают ситуации, когда критериев для подбора подходящего правила недостаточно и нужно добавить свой критерий в карточку правила согласования.

Предположим, нужно добавить критерий **Этап тендера** в правила согласования **ContractsApprovalRule** для договорных документов.

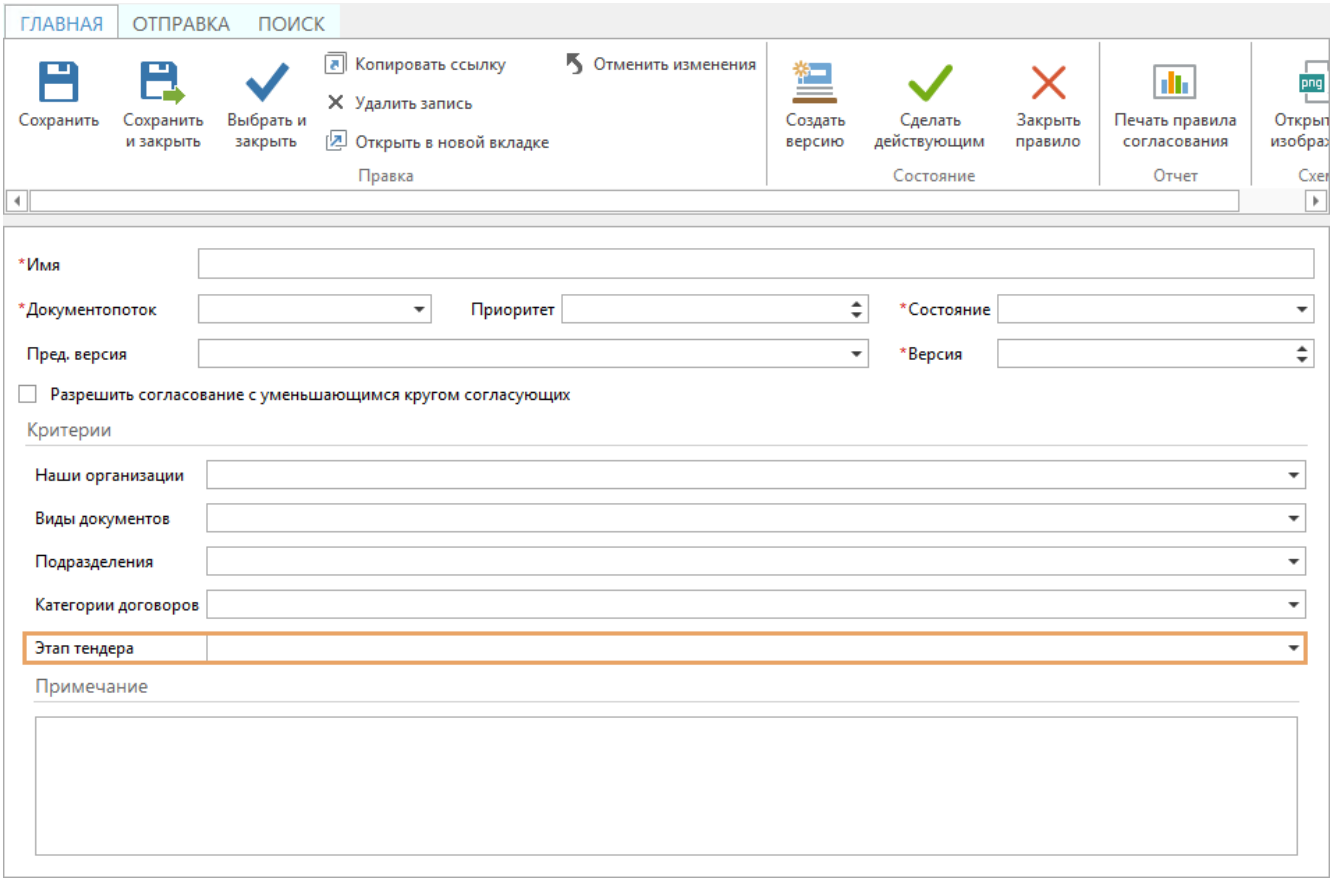


Решение

1. В своем решении перекройте тип документа «Договор» (Sungero.Contracts.Contract).

2. В редакторе решения добавьте зависимость от модуля Sungero.Contracts.
3. В редакторе перекрытого типа документа в узле «Свойства» создайте свойство **Этап тендера** (TenderStep). На основании этого свойства в следующих шагах добавляется новый критерий согласования по этапу тендера.
4. В узле «Форма» добавьте новое свойство на карточку перекрытого типа договора.
5. Перекройте справочник «Правила согласования» (ContractsApprovalRule).

   ВАЖНО. Для входящего, исходящего и внутреннего потоков документов нужно перекрывать справочник «Правило согласования» (ApprovalRule).
6. В редакторе перекрытого справочника в узле «Свойства» создайте свойство **Этап тендера** (TenderStep).
7. В узле «Форма» добавьте новое свойство на карточку перекрытого типа договора.



8. Чтобы переопределить присвоение приоритета, в серверном событии **До сохранения (в транзакции)** перекрытого справочника напишите код:

```
{
    // Присвоить правилу приоритет 0.
    _obj.Priority = 0;
    // Если заполнен критерий «Наши организации», прибавить 16.
    if (_obj.BusinessUnits.Any())
    _obj.Priority += 16;
    // Если заполнен критерий «Виды документов», прибавить 8.
    if (_obj.DocumentKinds.Any())
    _obj.Priority += 8;
    // Если заполнен критерий «Подразделения», прибавить 4.
    if (_obj.Departments.Any())
    _obj.Priority += 4;
    // Если заполнен критерий «Группы документов», прибавить 2.
    if (_obj.DocumentGroups.Any())
```

```
    _obj.Priority += 2;
    // Если заполнен критерий «Этап тендера», прибавить 1.
    if (_obj.TenderStep != null)
    _obj.Priority += 1;
    base.Saving(e);
  }
```

Для критерия, которому присваивается больший вес, указывается большая сумма. По заполненным критериям и их весу автоматически считается значение в поле **Приоритет** правила согласования. Правило с максимальным приоритетом автоматически подставляется в карточку задачи в поле **Регламент**.

9. При сохранении карточки правила согласования должно проверяться наличие дублей среди правил. Проверка выполняется по уникальности заданных критериев. Чтобы проверить наличие дублей правила согласования и учесть новый критерий, переопределите серверную функцию GetDoubleRules() в перекрытом справочнике ContractsApprovalRule:

```
public override List<Sungero.Docflow.IApprovalRuleBase> GetDoubleRules()
  {
    var conflictedRules = base.GetDoubleRules();
    return conflictedRules.Where(r => ContractsApprovalRules.Is(r) &&
                                 ContractsApprovalRules.As(r).TenderStep ==
_obj.TenderStep).ToList();
  }
```

10. Создайте серверную функцию перекрытого справочника. Вызовите в ней функцию **GetAvailableRulesByDocument()** из базового решения, которая используется для определения доступных правил по документу. Из полученных правил исключите те, которые не подходят под новый критерий. Для этого в перекрытом справочнике добавьте серверную функцию и напишите ее код:

```
[Remote(IsPure = true), Public]
  public static IQueryable<Sungero.Docflow.IApprovalRuleBase>
  GetAvailableRulesByDocumentCustom(Sungero.Docflow.IOfficialDocument
  document)
  {
    var rules =
  Sungero
  .Docflow
  .PublicFunctions
  .ApprovalRuleBase.Remote.GetAvailableRulesByDocument(document);
    var contract = Contracts.As(document);
    if (contract != null && contract.TenderStepDEV != null)
      rules = rules.Where(r => ContractsApprovalRules.Is(r) &&
            (ContractsApprovalRules.As(r).TenderStep == contract.TenderStep ||
            ContractsApprovalRules.As(r).TenderStep == null));
    return rules;
  }
```

11. Переопределите функцию получения правил согласования **GetApprovalRules()** в тех типах документов, для которых применяется новый критерий. В данном примере – в типе документа «Договор». В переопределенной функции вызовите функцию **GetAvailableRulesByDocumentCustom()**, созданную на предыдущем шаге.

```
public override List<Sungero.Docflow.IApprovalRuleBase> GetApprovalRules()
  {
    return
  Functions.ContractsApprovalRule.GetAvailableRulesByDocumentCustom(_obj)
```

```
            .OrderByDescending(r => r.Priority)
            .ToList();
    }
```

12. Опубликуйте разработку.

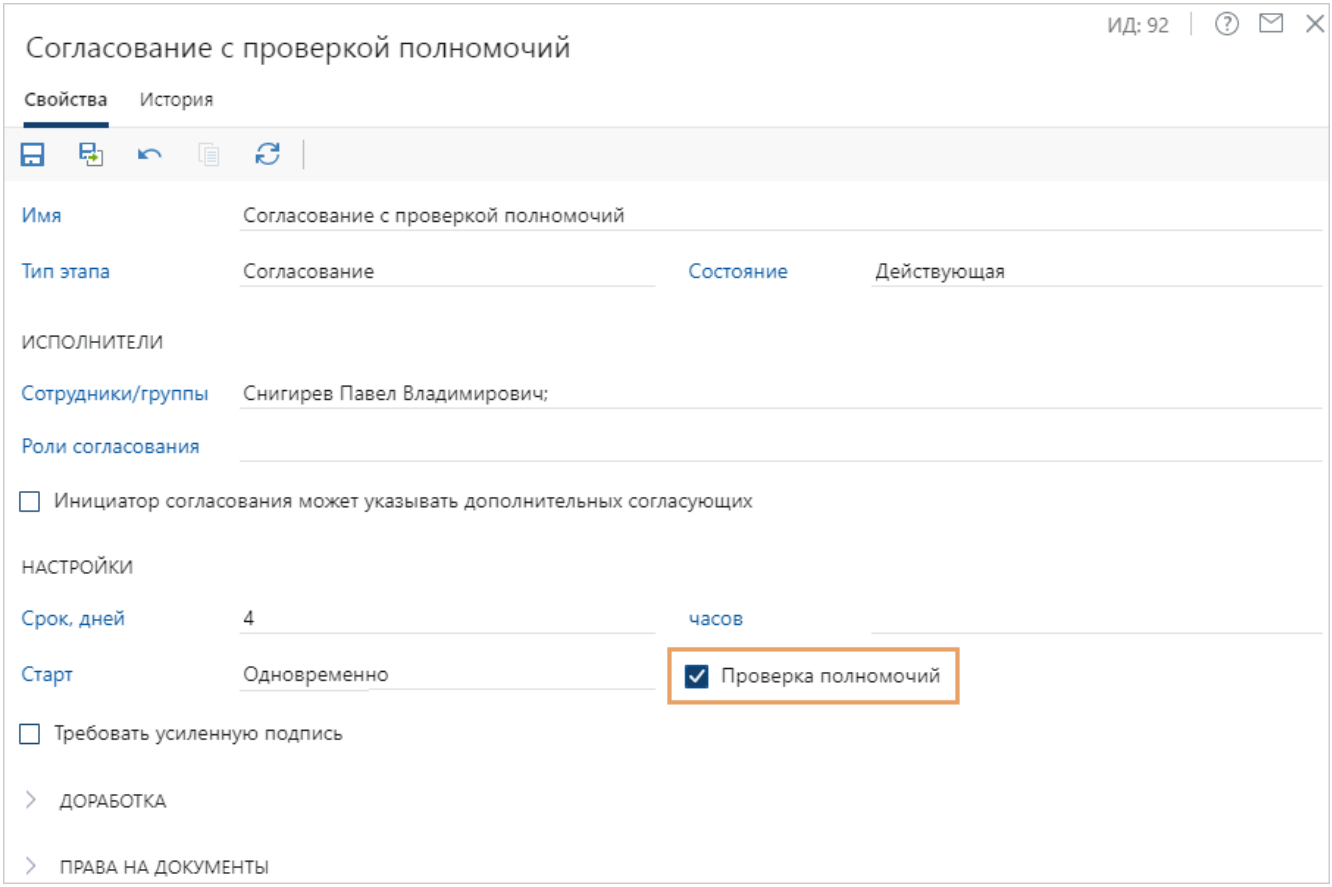
В результате новый критерий по этапу тендера добавится в карточку правила согласования для договорных документов. При отправке договорного документа, у которого в карточке заполнено поле с этапом тендера, в поле ***Регламент** подставится правило, которое предварительно настроил администратор в системе Directum RX, например, правило **Договор. Проект Тендера. Сбыт**.

Как проинформировать согласующего об ограничениях в полномочиях

Предположим, что у сотрудника есть ограничение в полномочиях: он согласует документы на сумму до 1 миллиона рублей, а для согласования документов на большую сумму ему требуется запросить подтверждение у вышестоящей организации.

Чтобы на этапе согласования вывести предупреждение, информирующее об ограничениях:

1. В карточку этапа с типом **Согласование** добавьте новое свойство логического типа **Проверка полномочий**. Для этого *перекройте* справочник Sungero.Docflow.ApprovalStage.

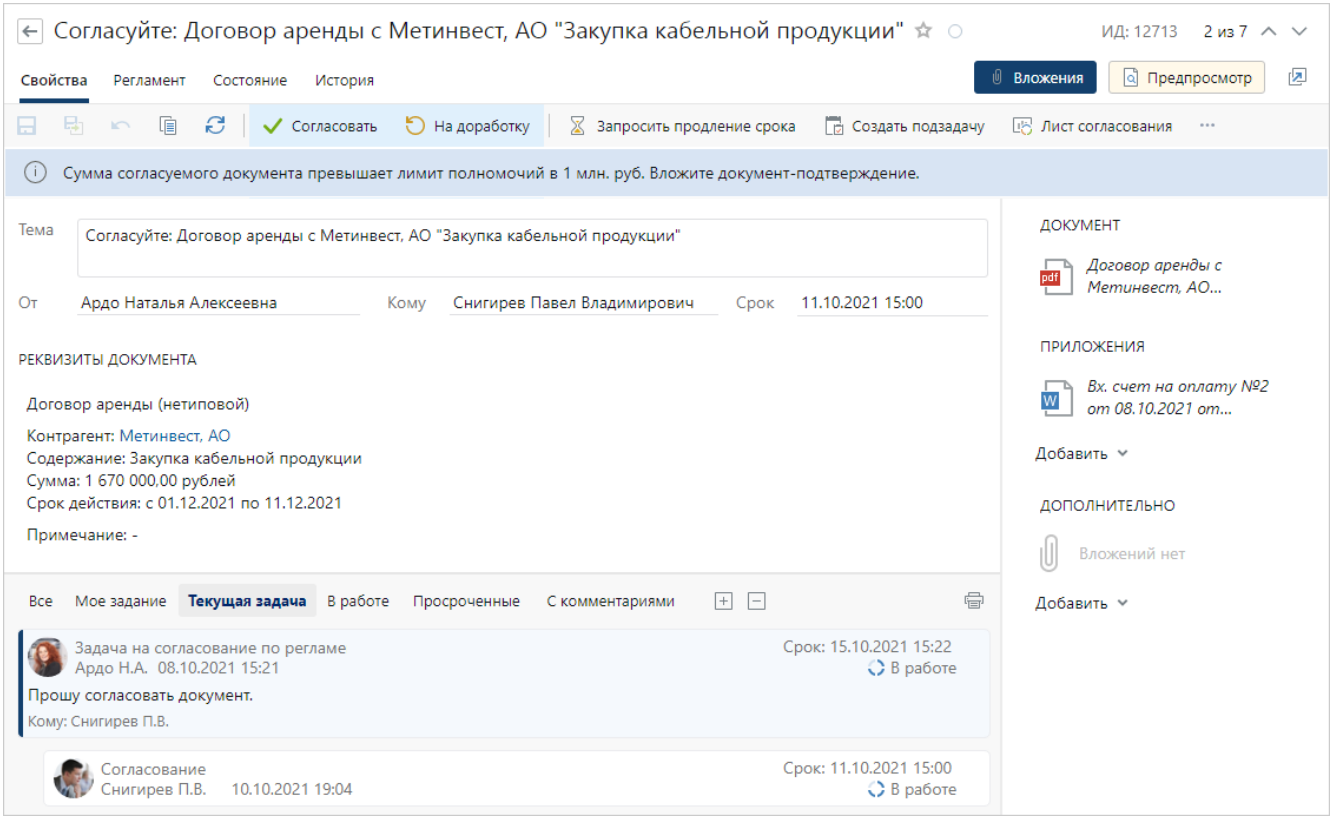


ПРИМЕЧАНИЕ. Для заполнения свойства **Проверка полномочий** в уже созданных этапах можно написать скрипт конвертации.

2. Перекройте задание на согласование документа с согласующим Sungero.Docflow.ApprovalAssignment. Если проверку полномочий необходимо реализовать на этапе согласования с руководителем, то перекрывать нужно задание Sungero.Docflow.ApprovalManagerAssignment.
3. В событии Показ формы добавьте код, в котором осуществляется проверка полномочий и вывод информационного сообщения. При этом проверка происходит, если задание сформировано по этапу, в котором установлен признак **Проверка полномочий**.

```
var stage = ApprovalStages.As(_obj.Stage);
   if (stage != null && stage.IsCheckAuthority == true)
   {
      // Проверить полномочия текущего сотрудника.
      var str =
      Purchase
      .PublicFunctions.GetInfoAboutAuthority(Sungero.Company.Employees.Current,
         _obj.DocumentGroup.OfficialDocuments.FirstOrDefault());
      // Вывести предупреждение.
      if (!string.IsNullOrEmpty(str))
        e.AddInformation(Purchase.Resources.CheckAuthorityFormat(str));
      }
```

В результате в задании на согласование сотрудник, для которого задано ограничение в полномочиях, увидит предупреждающее сообщение:



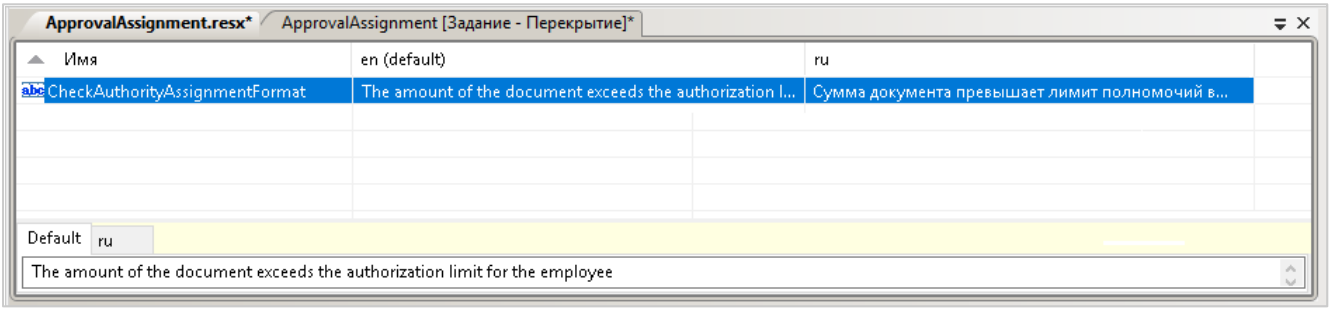
Как зафиксировать дополнительную информацию о согласовании в тексте задания

Ситуация

Предположим, на этапе согласования в задании добавлено предупреждение, которое информирует сотрудника об ограничениях в полномочиях. Чтобы отразить то, что сотрудник выполнил задание с превышением полномочий, зафиксируем информацию в тексте задания. Эти сведения могут понадобиться, например, для контролера при приемке задания.

Решение

1. Добавьте ресурс для вывода предупреждения об ограничениях сотрудника в полномочиях. Для этого в дереве решений в контекстном меню задания Sungero.Docflow.ApprovalAssignment, перекрытого при добавлении предупреждения, выберите пункт **Открыть ресурсы** и добавьте ресурс с названием **CheckAuthorityAssignmentFormat**:



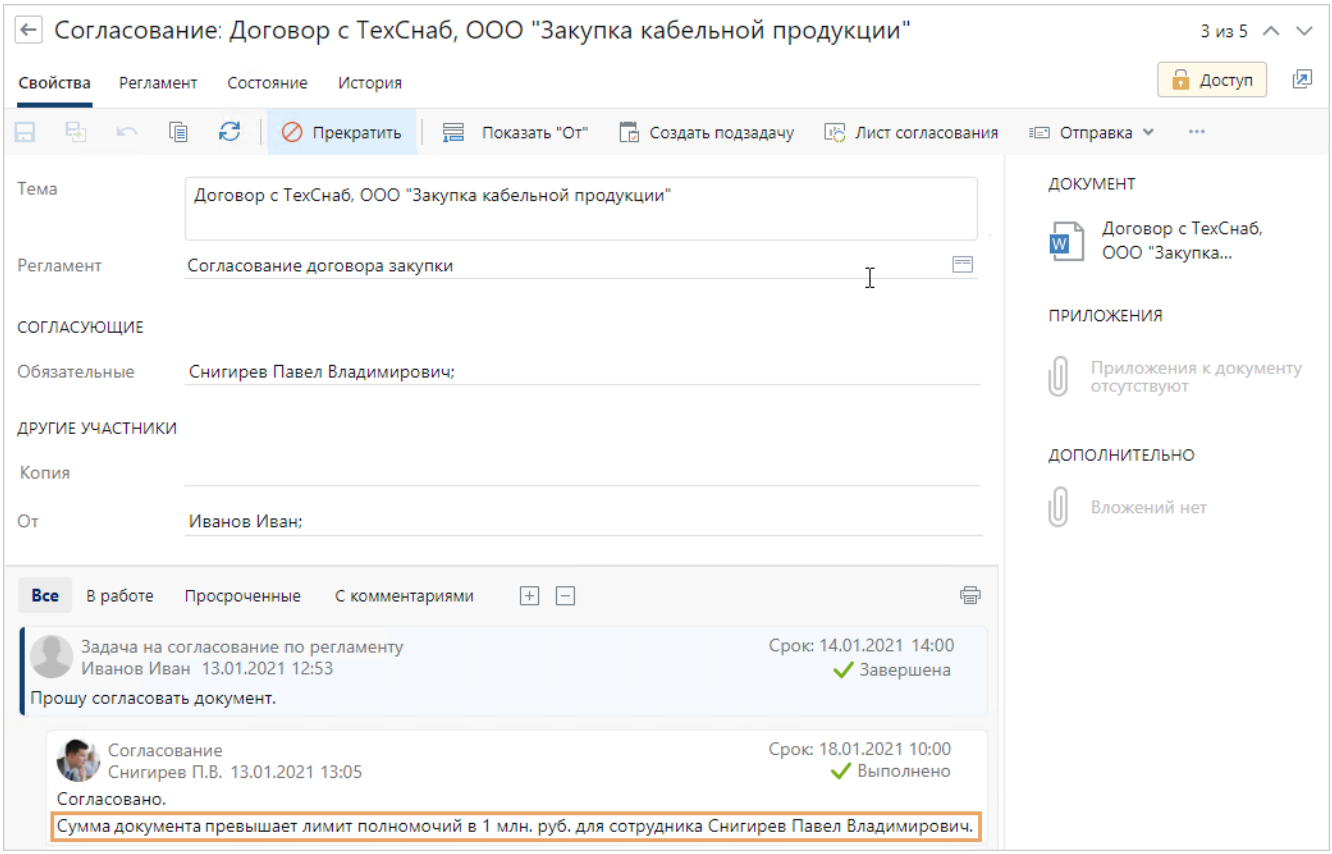
2. В редакторе перекрытого задания перейдите к событию До выполнения. В событии добавьте код, в котором дополнительная информация о превышении полномочий сотрудника записывается в результат:

```
public override void
   BeforeComplete(Sungero.Workflow.Server.BeforeCompleteEventArgs e)
   {
     base.BeforeComplete(e);
   // Если задание согласовано и сформировано по этапу, в котором установлен
   // признак «Проверка полномочий», проверить полномочия сотрудника.
   // Если задание выполнено с превышением полномочий, добавить в текст задания
   // предупреждение об этом.
     if (_obj.Result == Result.Approved)
       {
         var stage = ApprovalStages.As(_obj.Stage);
         if (stage != null && stage.IsCheckAuthority == true)
         {
           var str =
   Purchase
   .PublicFunctions.GetInfoAboutAuthority(Sungero.Company.Employees.Current,
   _obj.DocumentGroup.OfficialDocuments.FirstOrDefault());
           if (!string.IsNullOrEmpty(str))
         e.Result = Purchase.Resources.CheckAuthorityAssignmentFormat(e.Result,
   str);
```

```
            }
        }
    }
```

3. Сохраните изменения и опубликуйте разработку.

В результате в Directum RX в переписке задачи отобразится информация о выполнении задания с превышением полномочий.



При необходимости в тексте задания можно зафиксировать любую дополнительную информацию о согласовании.

Как в задаче по регламенту заполнить поле «Адресат» значением из нового типа документа

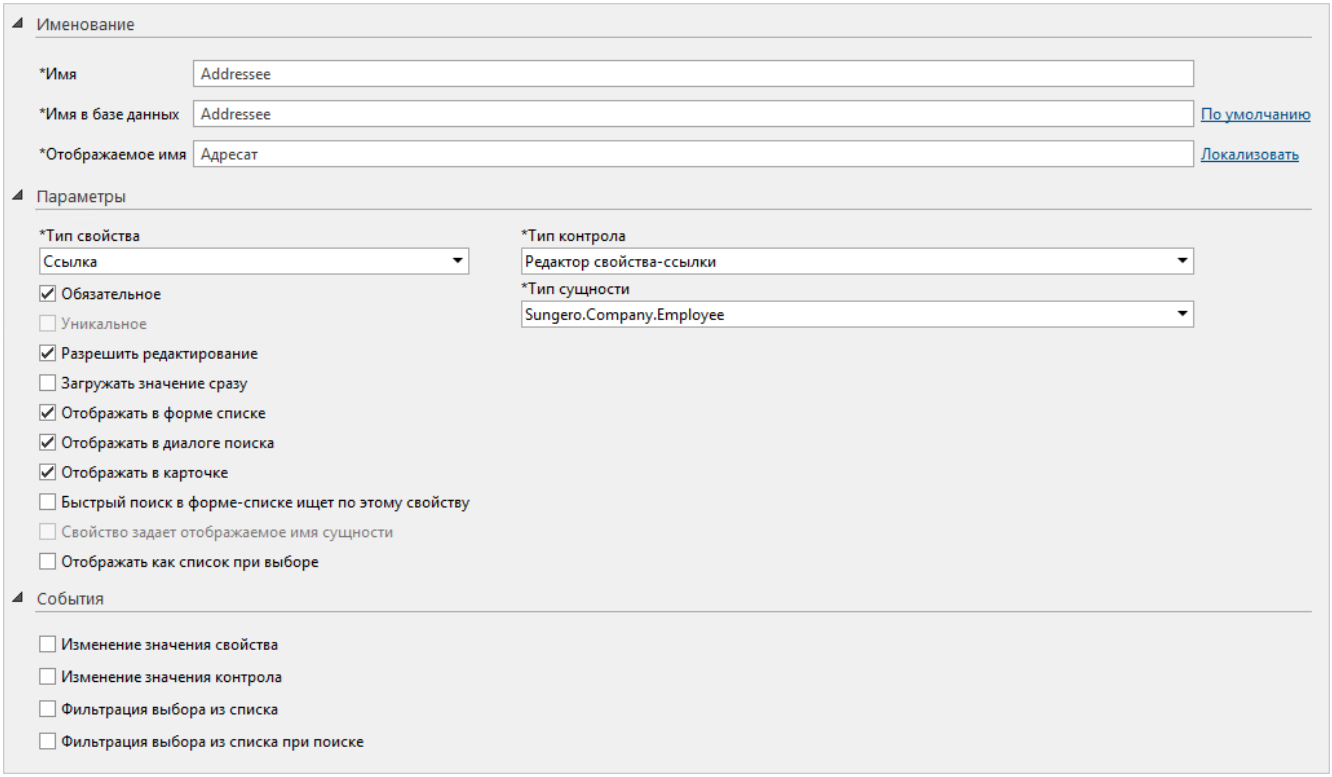
Ситуация

Предположим, нужно создать новый тип документа «Заявка» (Request), в его карточку добавить поле **Адресат**. При отправке документа данного типа на согласование по регламенту поле **Адресат** в задаче должно заполняться значением из документа.

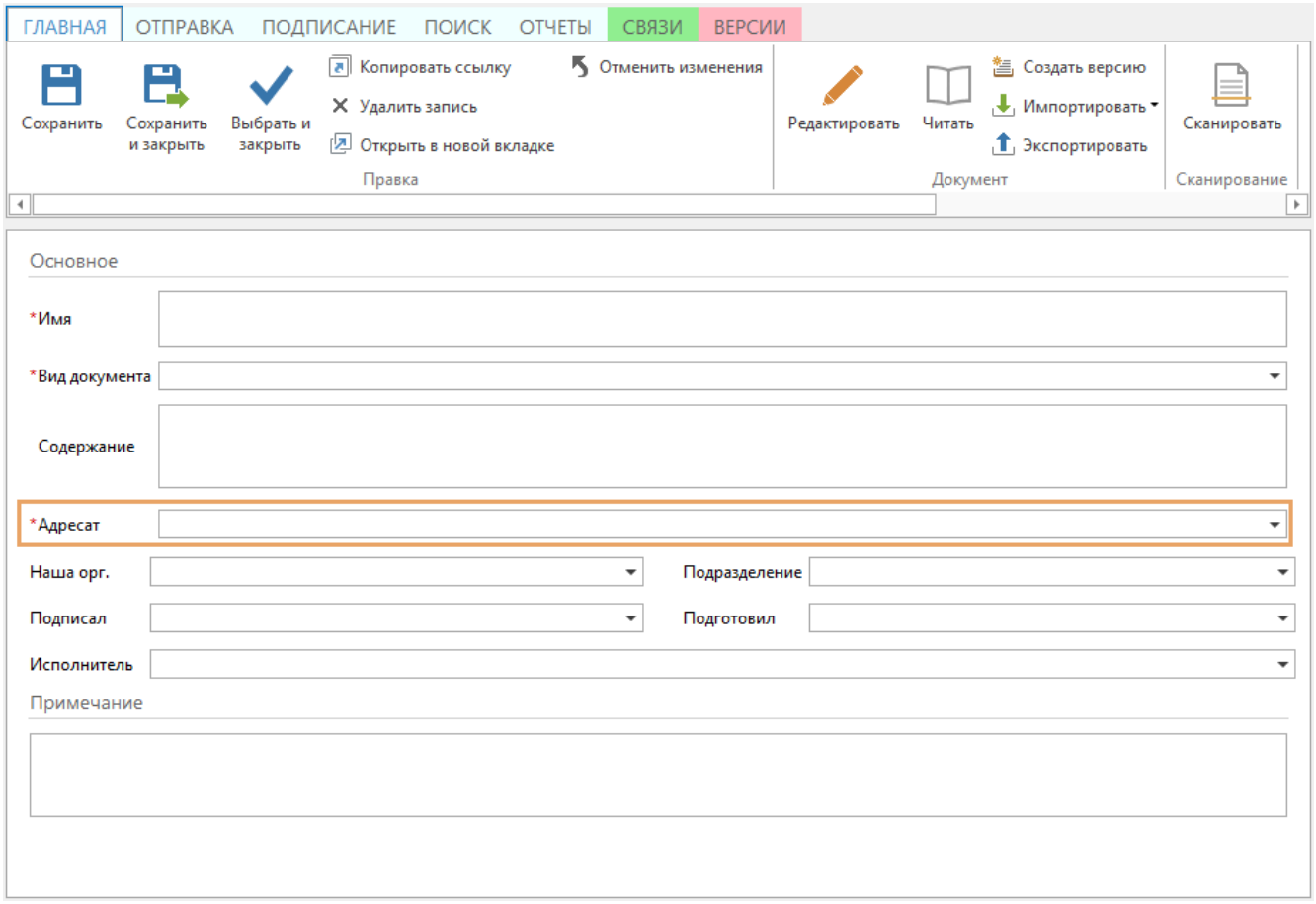
Решение

1. В своем решении создайте новый тип документа «Заявка» (Request). В качестве базового типа выберите «Внутренний документ» (InternalDocumentBase).
2. В редакторе решения добавьте зависимость от модуля Sungero.Docflow.

3. В редакторе созданного типа документа в узле «Свойства» добавьте свойство **Адресат**.



4. В узле «Форма» добавьте свойство на карточку нового типа документа. В результате форма заявки будет выглядеть следующим образом:



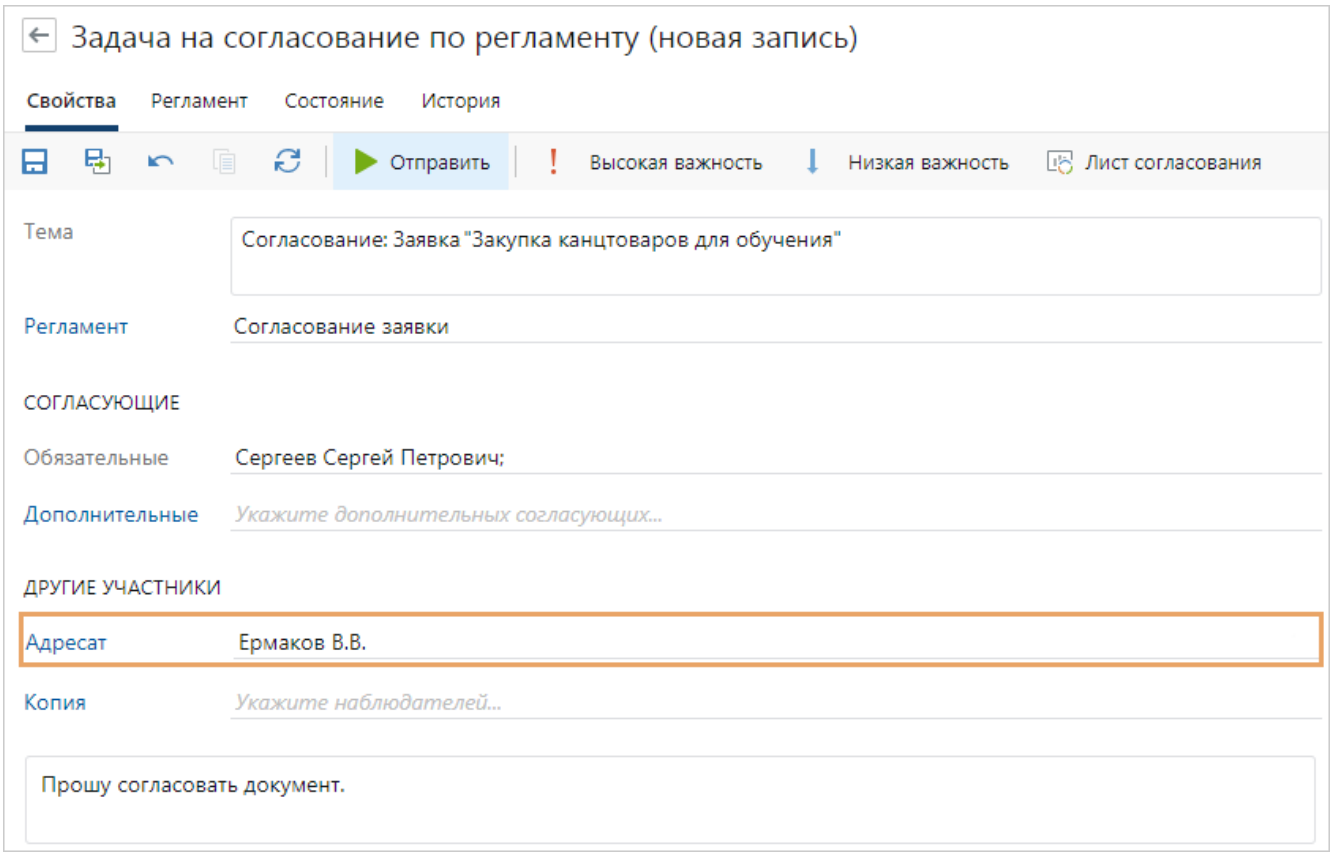
5. Чтобы значение из добавленного поля заполнялось в задаче на согласование по регламенту, перекройте задачу ApprovalTask.

6. Чтобы отразить изменения в правиле согласования, переопределите серверную функцию ApprovalRuleChanged(). Для этого в перекрытом типе задачи создайте серверную функцию и напишите ее код.

```
// <param name="rule">Новое правило.</param>
// <param name="stages">Список этапов согласования.</param>
public override void ApprovalRuleChanged(Sungero.Docflow.IApprovalRuleBase
rule, List<Sungero.Docflow.Structures.Module.DefinedApprovalBaseStageLite>
stages)
{
  // Вызвать базовый обработчик.
  base.ApprovalRuleChanged(rule, stages);
  // Ввести этап с рассмотрением адресата.
  var reviewStage = stages.Where(s => s.Stage.StageType ==
Sungero.Docflow.ApprovalStage.StageType.Review).FirstOrDefault();
  if (reviewStage != null && _obj.Addressee == null)
  {
    // Заполнить адресата из документа.
    var newDoc =
Requests.As(_obj.DocumentGroup.OfficialDocuments.FirstOrDefault());
    if (newDoc != null)
    _obj.Addressee = newDoc.Addressee;
  }
}
```

7. Сохраните изменения и опубликуйте разработку.

В Directum RX администратору нужно настроить правило согласования для нового типа документа. Чтобы поле ***Адресат** отображалось, в правиле должен быть этап типа «Рассмотрение адресатом». В результате при отправке нового типа документа на согласование по регламенту поле ***Адресат** в задаче заполнится из документа.



Как создать новый тип сценария для правила согласования

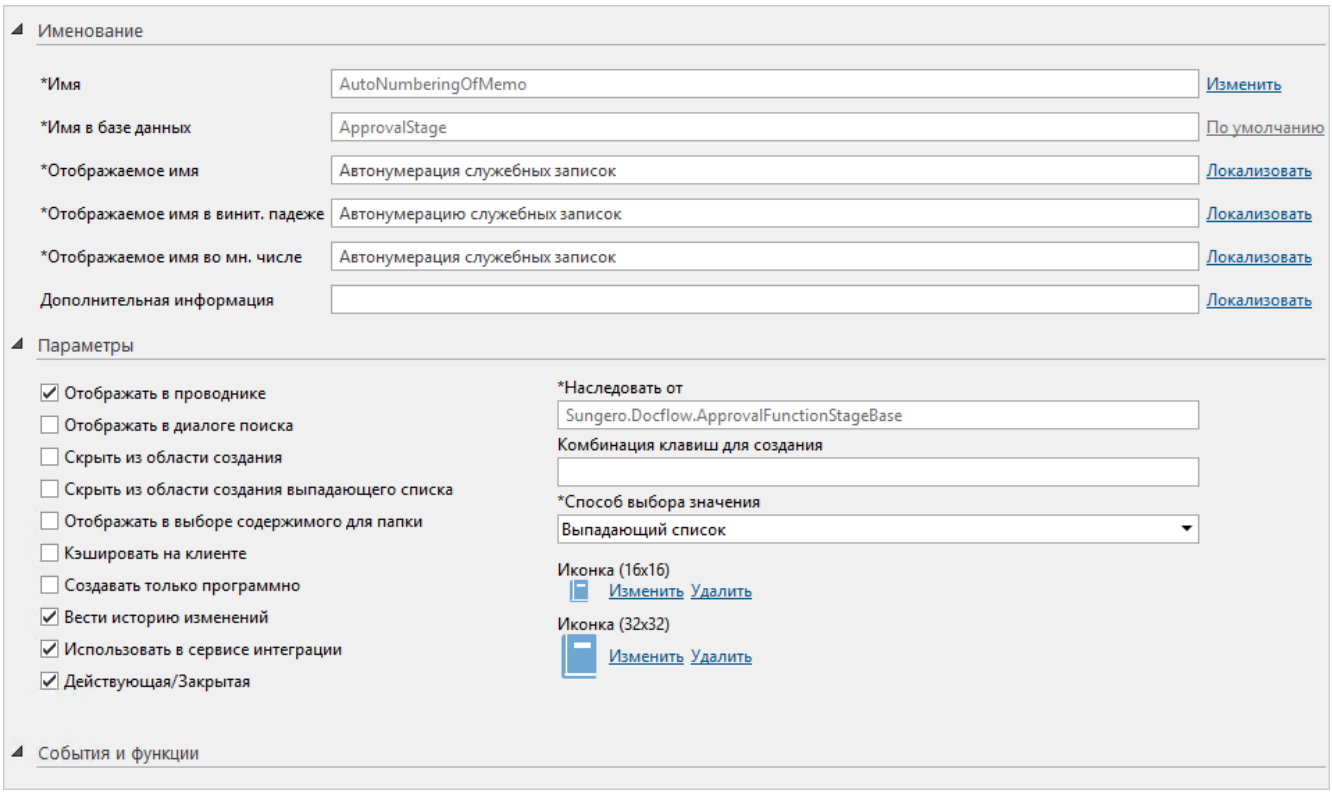
Ситуация

Кроме этапов согласования в системе Directum RX можно добавить на схему правила согласования этапы с вычислениями – *сценарии*.

Предположим, что в процессе согласования после подписания требуется автоматически присваивать номер служебным запискам. В стандартной поставке Directum RX такого этапа согласования и сценария нет. Нужно создать новый тип сценария.

Решение

1. В своем модуле создайте тип справочника путем наследования от базового справочника «Этап выполнения сценария» (ApprovalFunctionStageBase):



2. В новом типе справочнике «Автонумерация служебных записок» (AutoNumberingOfMemo) добавьте серверную функцию и напишите ее код.

```
/// <summary>
    /// Выполнить сценарий.
    /// </summary>
    /// <param name="approvalTask">Задача на согласование по регламенту.</param>
    /// <returns>Результат выполнения сценария.</returns>
    public override
    Sungero.Docflow.Structures.ApprovalFunctionStageBase.ExecutionResult
    Execute(Sungero.Docflow.IApprovalTask approvalTask)
    {
```

```
    var result = base.Execute(approvalTask);
    var document =
  approvalTask.DocumentGroup.OfficialDocuments.SingleOrDefault();
    if (document == null)
      return this.GetErrorResult("Не найден документ.");
    if (document.DocumentKind == null)
      return this.GetErrorResult("Не найден вид документа.");
    var documentRegisters =
  Sungero
  .Docflow
  .PublicFunctions
  .OfficialDocument.GetDocumentRegistersIdsByDocument(document,
  Sungero.Docflow.RegistrationSetting.SettingType.Numeration);
    if (!documentRegisters.Any())
      return this.GetErrorResult("Не найден журнал регистрации.");
    try
      //Регистрация документа.
    {
      Sungero
  .Docflow.PublicFunctions.OfficialDocument.RegisterDocument(document,
  documentRegisters.First(), Calendar.Today, null, false, false);
      document.Save();
    }
    catch (Exception ex)
    {
      result = this.GetRetryResult(string.Empty);
    }
    return result;
  }
```

3. Чтобы новый тип сценария можно было выбрать в веб-клиенте, программно создайте его запись. Для этого в модуле создайте функцию инициализации **CreateAutoNumberingOfMemoStage()** и напишите ее код.

```
/// <summary>
  /// Создание записи нового типа сценария "Автонумерация служебных записок".
  /// </summary>
  public static void CreateAutoNumberingOfMemoStage()
  {
    InitializationLogger.DebugFormat("Init: Create stage for automatic
  numbering of memos.");
    if (DEV.MyCompany.AutoNumberingOfMemos.GetAll().Any())
      return;
    var stage = DEV.MyCompany.AutoNumberingOfMemos.Create();
    stage.Name = "Автонумерация служебных записок";
    stage.TimeoutInHours = 4;
    stage.Save();
  }
```

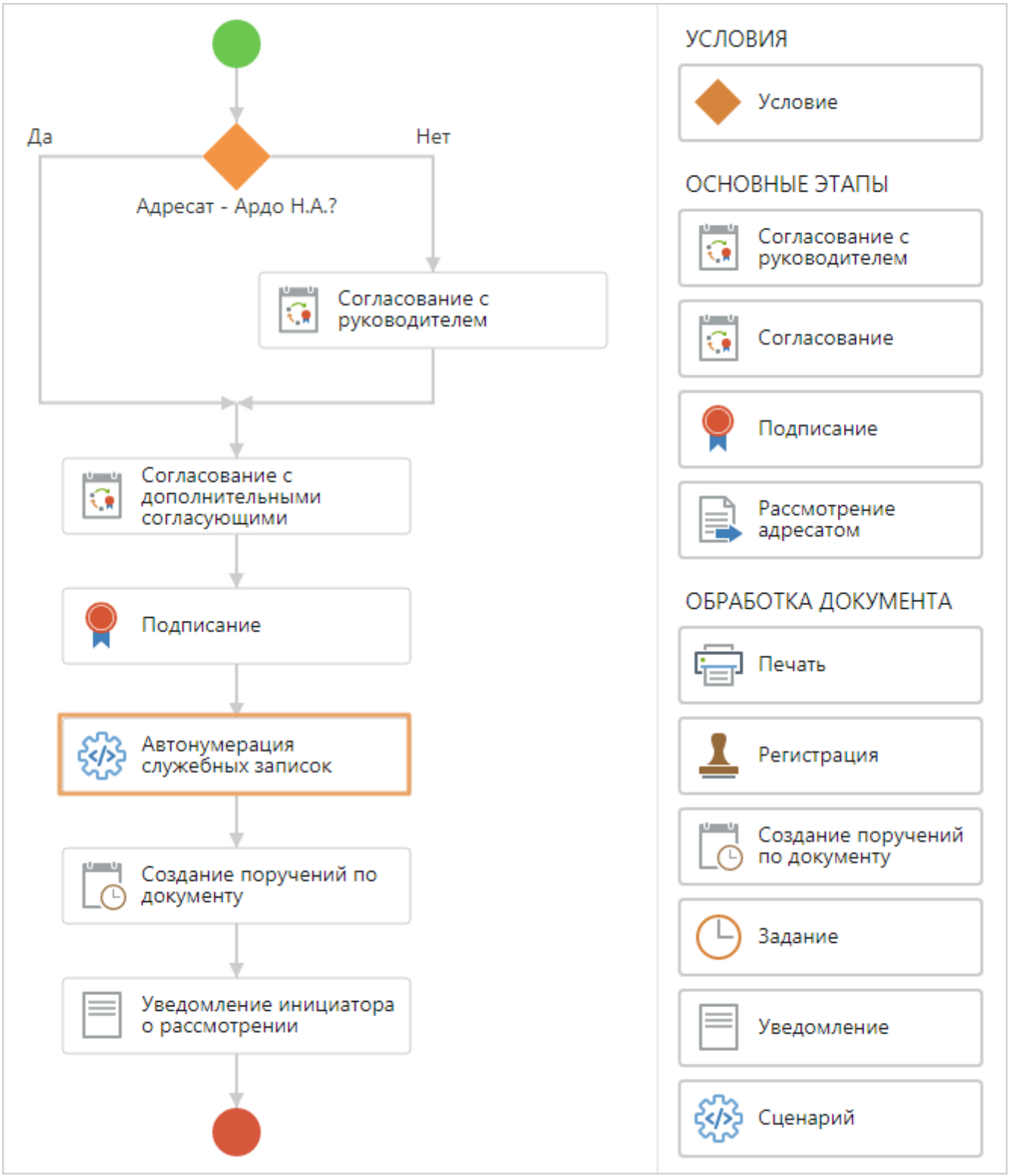
ВАЖНО. Вызовите добавленную функцию в событии **Initializing()**:

```
public override void Initializing(Sungero.Domain.ModuleInitializingEventArgs e)
  {
    CreateAutoNumberingOfMemoStage();
  }
```

4. Проверьте форму созданного справочника. Добавьте необходимые контролы или удалите ненужные. В этом примере разработки форма справочника не изменялась.

5. Сохраните изменения и опубликуйте разработку.
6. В веб-клиенте Directum RX добавьте на схему правила согласования блок «Сценарий» и нажмите на название.
7. В открывшемся окне нажмите на кнопку + и выберите из списка этап «Автонумерация служебных записок», задайте необходимые настройки.

В результате схема правила согласования будет выглядеть так:



Для добавления новых сценариев в среде разработки переопределяйте функции базового решения: **Execute()**, **Abort()**, **CheckCompletionState()**, **Validate()**, **Rework()**.

Пример использования функции Execute(). Проверка документов для преобразования в формат PDF

```
public override Docflow.Structures.ApprovalFunctionStageBase.ExecutionResult
    Execute(IApprovalTask approvalTask)
{
Logger.DebugFormat("ApprovalConvertPdfStage. Start execute convert to pdf for
task id: {0}, start id: {1}.", approvalTask.Id, approvalTask.StartId);

  var result = base.Execute(approvalTask);

  var documents = new List<IOfficialDocument>();

  var documentFromTask =
approvalTask.DocumentGroup.OfficialDocuments.SingleOrDefault();
  if (documentFromTask == null)

documents.Add(documentFromTask);
  if (_obj.ConvertWithAddenda == true)
{
  var addenda = approvalTask.AddendaGroup.OfficialDocuments.ToList();
documents.AddRange(addenda);
}

  var documentsToConvert = new List<IOfficialDocument>();
   foreach (var document in documents)
{
if (!document.HasVersions)

// Проверка документов от контрагентов.
if (Functions.OfficialDocument.IsExchangeDocument(document,
document.LastVersion.Id))

// Проверка документов с неподдерживаемым форматом.
  var versionExtension =
document.LastVersion.BodyAssociatedApplication.Extension.ToLower();
  var versionExtensionIsSupported =
AsposeExtensions.Converter.CheckIfExtensionIsSupported(versionExtension);
if (!versionExtensionIsSupported)
var lockInfo = Locks.GetLockInfo(document.LastVersion.Body);
if (lockInfo.IsLocked)
   documentsToConvert.Add(document);
}
```

Пример использования функции Abort(). Прекращение задачи на рассмотрение

```
public override void Abort(IApprovalTask task, int stageNumber)
{
  var link = GetApprovalTaskExternalLink(task, stageNumber);
  if (link != null)
  {
    var reviewTask = GetReviewTaskByExternalLink(link);
    if (reviewTask != null && reviewTask.Status ==
Workflow.Task.Status.InProcess)
```

```
    {
      reviewTask.Abort();
    }
  }
}
```

Пример использования функции CheckCompletionState(). Проверка состояния задачи на рассмотрение

```
public override Docflow.Structures.ApprovalFunctionStageBase.ExecutionResult
   CheckCompletionState(IApprovalTask approvalTask)
{
  var result = base.CheckCompletionState(approvalTask);
  var link = GetApprovalTaskExternalLink(approvalTask);
  if (link == null)

  long reviewTaskId = 0;
  long.TryParse(link.ExternalEntityId, out reviewTaskId);
  var reviewTask =
RecordManagement.DocumentReviewTasks.GetAll().SingleOrDefault(x => x.Id ==
reviewTaskId);
  if (reviewTask == null)

  var document = approvalTask.DocumentGroup.OfficialDocuments.SingleOrDefault();
  if (Locks.GetLockInfo(document).IsLockedByOther)

  var taskCompleted = reviewTask.Status == Workflow.Task.Status.Completed ||
    reviewTask.Status == Workflow.Task.Status.Aborted;
  if (taskCompleted)
  {
    this.MarkExternalLinksAsDeleted(approvalTask);
    this.SetMemoInternalApprovalStateOnDocumentReviewTaskCompleted(approvalTask,
 reviewTask);
    return this.GetSuccessResult();
  }

  return this.GetRetryResult(string.Empty);
}
```

Пример использования функции Validate(). Проверка схемы правила согласования

```
public override void Validate(IApprovalRuleBase rule,
   List<IApprovalRuleBaseStages> stagesSequence, IApprovalRuleBaseStages stage,
    Sungero.Domain.BeforeSaveEventArgs e)
{
  base.Validate(rule, stagesSequence, stage, e);

  var reviewTaskStages = stagesSequence.Where(s => s.StageType ==
Docflow.ApprovalRuleBaseStages.StageType.Function &&
ApprovalReviewTaskStages.Is(s.StageBase));

  // В одной ветке правила должно быть не более одного этапа отправки документа
на рассмотрение.
  if (reviewTaskStages.Count() > 1 && stagesSequence.Contains(stage))
  e.AddError(stage, ApprovalRuleBases.Info.Properties.Stages.Properties.Stage,
ApprovalReviewTaskStages.Resources.ManyReviewStages);
}
```

Пример использования функции Rework(). Отправка задания на доработку

```
public override void Rework(IApprovalTask task, int stageNumber)
{
  base.Rework(task, stageNumber);
  this.Abort(task, stageNumber);
}
```

Задачи и задания

В разделе содержатся примеры:

- Как создать версию схемы задачи?
- Как в задании скрыть результат выполнения?
- Как скрыть блок в контроле состояния?
- Как добавить в задание возможность запрашивать продление срока?

Как создать версию схемы задачи

ПРИМЕЧАНИЕ. Пример актуален для схем задач, созданных ранее в среде разработки. Новые схемы рекомендуется настраивать через варианты процессов с помощью механизма no-code. Подробнее см. в руководстве администратора, раздел «Настройка схем бизнес-процессов».

Ситуация

Предположим, в решении создан тип задачи. Решение уже опубликовано в продуктивную систему, и сотрудники создавали задачи нового типа. В схему задачи необходимо внести изменения:

- удалить блок **Мониторинг**, в котором при наступлении определенного события создавалось уведомление;
- логику создания уведомления перенести в обработчик блока **Задание**.

Решение

1. В редакторе типа задачи перейдите в узел «Схема» и создайте версию.
2. Удалите со схемы блок **Мониторинг**. Для этого выделите блок и нажмите на клавишу DELETE. Вместе с блоком удалятся стрелки перехода. Добавьте стрелки между оставшимися блоками.
3. Выделите блок **Задание** на схеме и на панели свойств рядом с событием **Выполнение задания** перейдите по ссылке **Перейти к коду**.
4. Добавьте логику создания уведомления в обработчик события. С помощью условия укажите, что добавленный код относится ко второй версии схемы:

```
public virtual void CompleteAssignment2(Sungero.Docflow.
    IFreeApprovalAssignment assignment,
    Sungero.Docflow.Server.FreeApprovalAssignmentArguments e)
{
    // Код обработчика события блока «Задание».
```

```
        // ...

        // Код для создания уведомления.
        // V2- вторая версия схемы, в которой используется данный код.
          if (assignment.Result == Sungero.Docflow.FreeApprovalAssignment.
            Result.ForRework && _obj.ReceiveNotice == true
            && _obj.GetStartedSchemeVersion() == LayerSchemeVersions.V2 &&
assignmentsInWork.Any())

        {
          /// ...
        }
}
```

ПРИМЕЧАНИЕ. Код обработчика события блока **Мониторинг** удалять не нужно, так как ранее стартованные задачи будут выполняться по первой версии схемы, в которой используется код обработчика.

5. Сохраните изменения.
6. Опубликуйте разработку, проверьте изменения и перенести в продуктивную систему.

В результате:

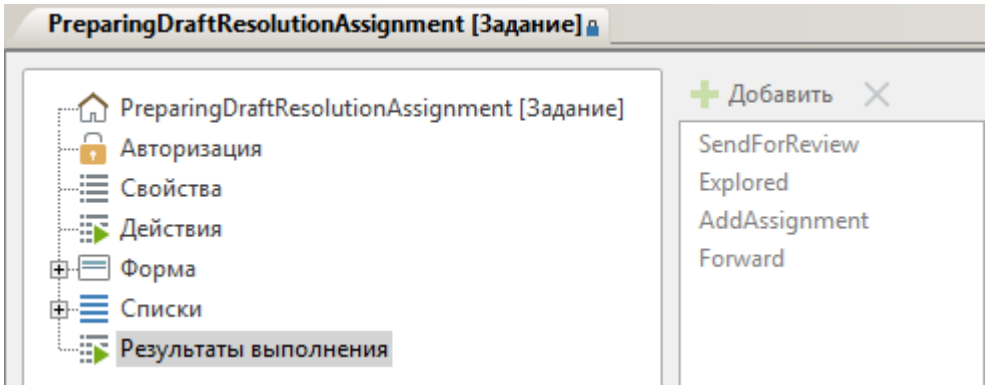
- новые и рестартованные задачи будут выполняться по второй версии схемы;
- первая версия останется для поддержки работоспособности существующих стартованных задач.

Как в задании скрыть результат выполнения

Ситуация

Нужно в задании скрыть один результат выполнения (убрать с ленты), если выполняется условие. Рассмотрим на примере базового решения Directum RX. В задании на подготовку проекта резолюции (PreparingDraftResolutionAssignment) 4 результата выполнения:

- **Отправить на рассмотрение** (SendForReview);
- **Отправить на исполнение** (AddAssignment);
- **Принято к сведению** (Explored);
- **Переадресовать рассмотрение** (Forward).



Когда руководитель отправляет проект резолюции на доработку, то в задании на подготовку проекта скрывается результат **Переадресовать рассмотрение**. Перейдем к тому, как это реализовано.

Решение

Чтобы скрыть результат выполнения в зависимости от условия, нужно в редакторе типа задания добавить обработчик для клиентского события Показ формы. За скрытие результата выполнения отвечает аргумент события e.HideAction(). В базовом решении Directum RX для задания на подготовку проекта резолюции код выглядит следующим образом:

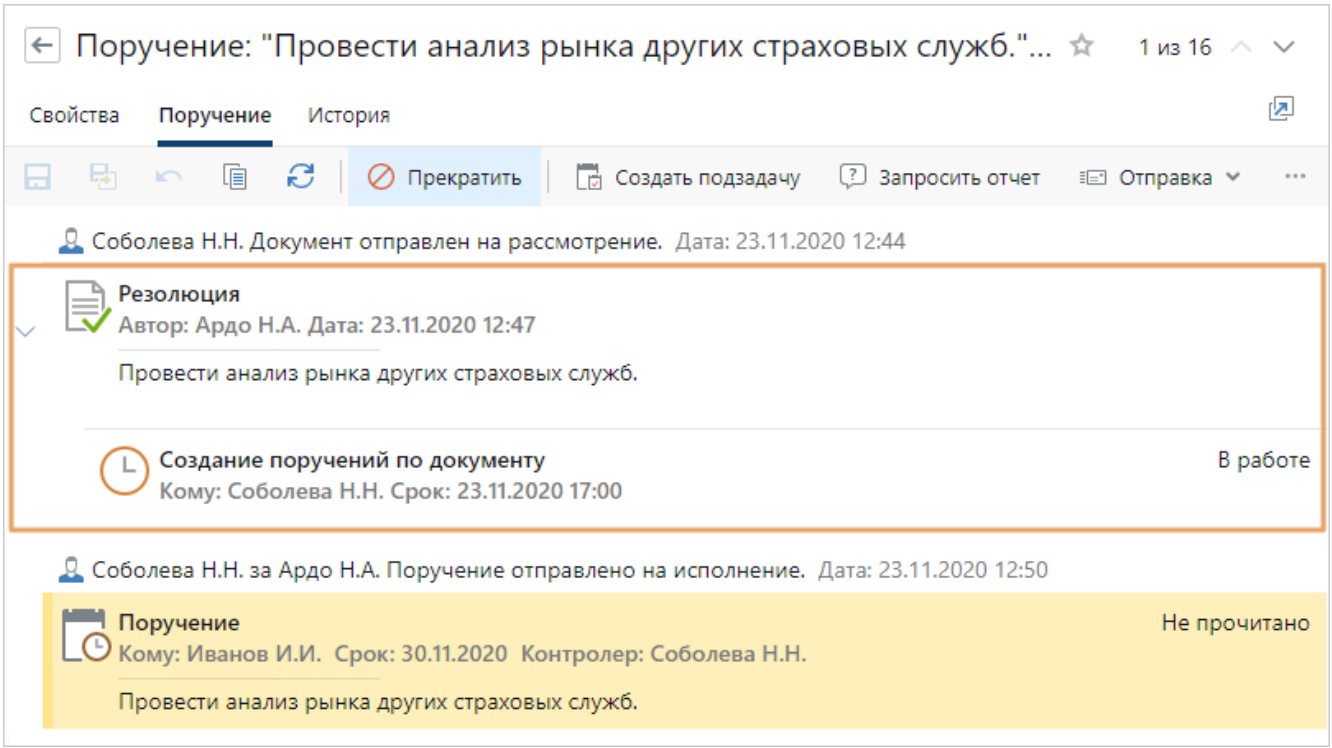
```
    public override void Showing(Sungero.Presentation.FormShowingEventArgs e)
{
  if (_obj.IsRework == true)
    e.HideAction(_obj.Info.Actions.Forward);

}
```

В своем решении вы можете перекрывать любые типы заданий и аналогично скрывать результаты выполнения по необходимым условиям.

Как скрыть блок в контроле состояния

Ситуация

В базовом решении в типе задачи «Задача на исполнение поручения» есть вкладка **Поручения** (State) с добавленным контролом состояния. В контроле отображаются блоки по работе с поручением: создание резолюции, исполнение поручения, запрос отчета и прочее. Необходимо скрыть блок с информацией о создании резолюции:



Решение

Чтобы скрыть блок с информацией о создании резолюции, нужно скрыть исходную вкладку **Поручения** и создать новую. На вкладку добавить свой контрол состояния, написать для него новый код на основе кода контрола состояния из базового решения. В коде изменить значение параметра, который отвечает за отображение блока с резолюцией. Для этого:

1. В своем решении перекройте тип задачи «Задача на исполнение поручения» (Sungero.RecordManagement.ActionItemExecutionTask).
2. В редакторе решения добавьте зависимость от модуля Sungero.RecordManagement.
3. В редакторе перекрытого типа задачи в узле «Форма» скройте вкладку **Поручения** (State), где содержится контрол состояния. Для этого в свойствах вкладки снимите флажок **Отображать в форме**. В дальнейшем вместо скрытого контрола состояния создадим новый.
4. Добавьте новую вкладку **StateNew**.
5. На вкладку добавьте группу контролов **ControlGroupDEV**, где будут отображаться все этапы работ по исполнению поручения.
6. На панели свойств в поле **Количество колонок в группе** укажите значение **1**.
7. В группу добавьте контрол состояния **ControlDEV**.

   В базовом решении в контроле состояния используются функции без модификатора **virtual**, поэтому их нельзя переопределить. Чтобы написать свою логику, нужно создать функции на основе функций контрола состояния из базового решения. Чтобы посмотреть код базовых функций, на скрытой вкладке **State** выделите контрол состояния **Control** и в свойствах контрола в поле ***Функция** нажмите на кнопку {a=}. Откроется редактор кода с функциями:

   - GetActionItemExecutionTaskStateView() – задает формат отображения контрола состояния. Ее параметр **skipResolutionBlock** отвечает за отображение этапа по созданию резолюции:

```
[Public]
public virtual Sungero.Core.StateView GetActionItemExecutionTaskStateView(IActionItemExecutionTask openedTask,
                                                                          Structures.ActionItemExecutionTask.IStateViewModel stateViewModel,
                                                                          int? draftNumber = null,
                                        //Пропустить блок резолюции?      bool skipResolutionBlock = false,
                                                                          bool withHighlight = true)
```

   - GetStateView() – отвечает за построение модели контрола состояния. Она возвращает **GetActionItemExecutionTaskStateView()** с нужными параметрами:

```
/// <summary>
/// Построить модель состояния главного поручения.
/// </summary>
/// <returns>Контрол состояния.</returns>
[Remote(IsPure = true)]
public Sungero.Core.StateView GetStateView()
{
  // Определить главное поручение и построить его состояние.
  var mainActionItemExecutionTask = this.GetMainActionItemExecutionTask();

  var stateViewModel = Structures.ActionItemExecutionTask.StateViewModel.Create();
  stateViewModel.Tasks = new List<IActionItemExecutionTask>() { mainActionItemExecutionTask };
  stateViewModel = GetAllActionItems(stateViewModel);
  return GetActionItemStateView(mainActionItemExecutionTask, _obj, stateViewModel, null, null);
}
```

- GetMainActionItemExecutionTask() – вызывается в функции **GetStateView()** и отвечает за отображение главного поручения;
- GetStateViewXml() – формирует XML-файл, на основе которого прорисовываются элементы контрола на форме.

8. Добавьте код для нового контрола состояния. Для этого на вкладке **StateNew** выделите созданный контрол состояния и на панели свойств нажмите на кнопку **Добавить функцию**.
9. В открывшемся редакторе кода скопируйте код базовых функций, переименуйте их и доработайте согласно комментариям в примерах:

GetStateViewXml()

Функцию **GetStateViewXml()** скопируйте целиком и переименуйте:

```
   [Public, Remote(IsPure = true)]
// Имя функции изменено на новое с приставкой New.
public string GetStateViewXmlNew()
{
  // Имя функции изменено на новое с приставкой New.
  return this.GetStateViewNew().ToString();
}
```

GetMainActionItemExecutionTask()

Функцию **GetMainActionItemExecutionTask()** скопируйте целиком, переименуйте и доработайте:

```
   // Имя функции изменено на новое с приставкой New.
public IActionItemExecutionTask GetMainActionItemExecutionTaskNew()
{
  var mainActionItemExecutionTask = _obj;
  // Для ITask добавлен полный путь Sungero.Workflow.ITask, чтобы задачи стали
доступны в вашем решении.
  Sungero.Workflow.ITask currentTask = _obj;
  while (currentTask.ParentTask != null || currentTask.ParentAssignment != null)
  {
    currentTask = currentTask.ParentTask ?? currentTask.ParentAssignment.Task;
    if (ActionItemExecutionTasks.Is(currentTask))
      mainActionItemExecutionTask = ActionItemExecutionTasks.As(currentTask);
  }
  return mainActionItemExecutionTask;
}
```

GetStateView()

Функцию **GetStateView()** скопируйте целиком, переименуйте и доработайте:

```
   [Remote(IsPure = true)]
// Имя функции изменено на новое с приставкой New.
public Sungero.Core.StateView GetStateViewNew()
{
  // Указано новое имя функции GetMainActionItemExecutionTask() с приставкой New.
  var mainActionItemExecutionTask =
this.GetMainActionItemExecutionTaskNew();
```

```
  var stateViewModel =
Sungero
.RecordManagement.Structures.ActionItemExecutionTask.StateViewModel.Create();
  // Для IActionItemExecutionTask добавлен полный путь
Sungero.RecordManagement.IActionItemExecutionTask, чтобы поручения стали доступны
в вашем решении.
  stateViewModel.Tasks = new
List<Sungero.RecordManagement.IActionItemExecutionTask>()
{ mainActionItemExecutionTask };
  stateViewModel = GetAllActionItems(stateViewModel);
  // Для функции GetActionItemExecutionTaskStateView() указаны все параметры.
  // Для предпоследнего параметра skipResolutionBlock установлено значение true,
чтобы скрыть блок про создание резолюции.
  return GetActionItemExecutionTaskStateView(mainActionItemExecutionTask,
stateViewModel, null, true, true);
}
```

GetActionItemExecutionTaskStateView()

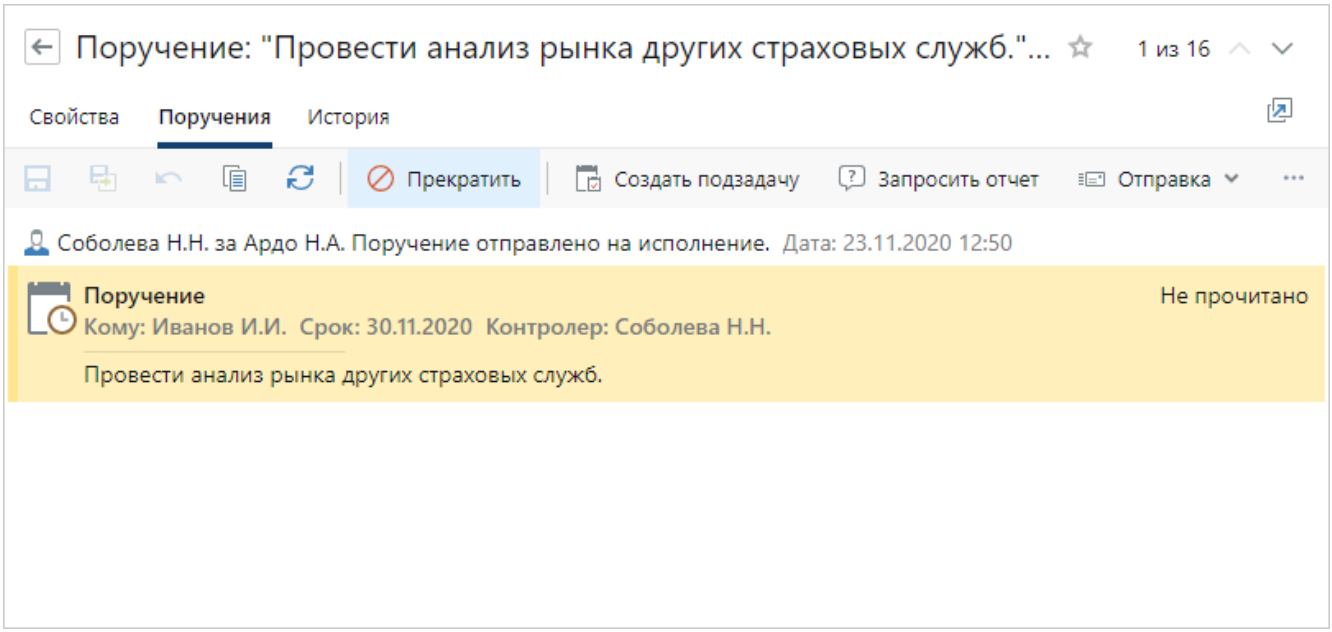
Код функции **GetActionItemExecutionTaskStateView()** копировать не требуется, так как функция доступна в любых модулях и решениях. Функция вызывается с новыми параметрами в **GetStateView()**.

10. В коде базовой функции **GetMainActionItemExecutionTask()** используется структура. Чтобы код функции **GetMainActionItemExecutionTaskNew()** корректно работал, скопируйте код структуры базового типа сущности «Задача на исполнение поручения» в редактор кода структуры перекрытого типа сущности:

```
[Public]
  partial class StateViewModel
  {
    public System.Collections.Generic.Dictionary<Sungero.Core.Enumeration?,
  string> StatusesCache { get; set; }

    public List<IActionItemExecutionTask> Tasks { get; set; }

    public List<Sungero.Workflow.IAssignment> Assignments { get; set; }
  }
```

11. Сохраните изменения и опубликуйте разработку.

В результате в карточках новых задач на исполнение поручения на вкладке «Поручения» этап по созданию резолюции отображаться не будет:



Как добавить в задание возможность запрашивать продление срока

Ситуация

На *рабочем слое* для определенного задания необходимо добавить возможность запрашивать продление срока.

Решение

1. В среде разработки в редакторе типа задания, срок которого необходимо продлить, добавьте новое действие.
2. Задайте параметры для действия и добавьте логику создания задачи на продление срока в событие «Выполнение»:

```
public virtual void ExtendDeadline(Sungero.Domain.Client.ExecuteActionArgs e)
{
  var task =
Docflow
.PublicFunctions.DeadlineExtensionTask.Remote.GetDeadlineExtension(_obj);
  task.Show();
}
```

Логика по умолчанию:

- запросить продление срока можно у инициатора задачи или его руководителя;
- продлевается срок задания, из которого запросили продление срока;
- инициатор задачи на запрос продления срока получает уведомление о продлении срока задания.

3. Если необходимо изменить стандартную логику продления срока в задании, то создайте в нем серверные функции:

   GetAssigneesForDeadlineExtension() – возвращает список сотрудников, которые могут продлить срок задания;

   ExtendAssignmentDeadline() – продлевает срок задания;

   GetPerformersForDeadlineExtensionNotification() – возвращает список сотрудников, которых необходимо уведомить о продлении срока задания;

   GetNewDeadlineForDeadlineExtensionNotification() – вычисляет новый срок для всех, кому должно прийти уведомление. Например, в поручении сроки для исполнителя и соисполнителя могут отличаться.

4. Если необходимо добавить логику продления срока в задачу, из которой формируется необходимое задание, сделайте это в функции **ExtendTaskDeadline()**. Если она не добавлена в задачу, то создайте ее предварительно.

   По умолчанию срок задачи не продлевается.

В результате в задаче для определенного задания можно запросить продление срока.

Доступность, обязательность, видимость свойств

В разделе содержатся примеры:

- Как сделать свойство обязательным
- Как сделать свойство обязательным в зависимости от значения другого свойства
- Как сделать свойство доступным в зависимости от прав доступа пользователя
- Как управлять доступностью нового свойства, если оно зависит от существующего свойства
- Как скрыть свойство из области регистрации в карточке документа
- Как выбрать закрытую запись справочника

Как сделать свойство обязательным

Ситуация

Предположим, для справочника Организации необходимо:

- свойство **ИНН** сделать обязательным для заполнения. То есть чтобы пользователь не мог сохранить карточку организации, пока не заполнит поле;
- оставить возможность программно сохранять карточку без заполнения этого поля.

Решение

Сделать свойство обязательным для заполнения можно через редактор свойств. Для этого необходимо установить флажок **Обязательное** для свойства. В результате в карточке сущности перед названием свойства добавится символ *. При этом программно сохранять карточку без заполнения этого поля будет нельзя.

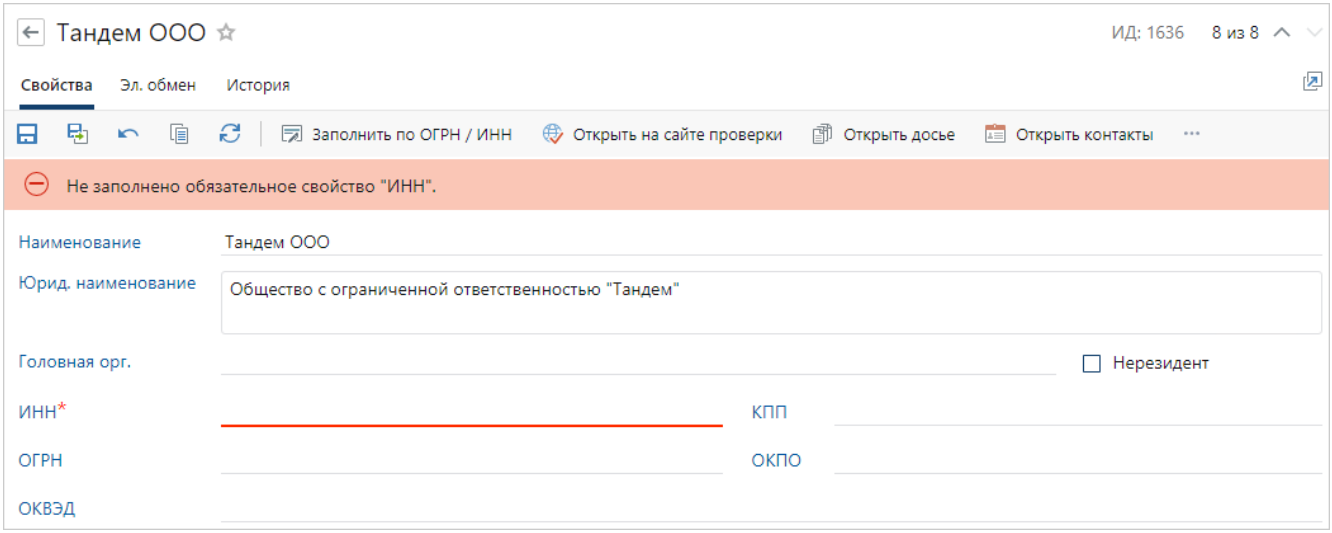
Чтобы поле **ИНН** было обязательным для пользователя, и при этом осталась возможность программно сохранять карточку справочника без заполнения этого поля:

1. В своем решении перекройте тип справочника **Организации** (Sungero.Parties.Company).
2. В редакторе решения добавьте зависимость от модуля Sungero.Parties.
3. В редакторе перекрытого типа справочника добавьте событие Показ формы и напишите код:

```
// Сделать обязательным свойство ИНН (TIN).
public override void Showing(Sungero.Presentation.FormShowingEventArgs e)
{
  base.Showing(e);
  _obj.State.Properties.TIN.IsRequired = true;

}
```

4. Сохраните изменения и опубликуйте разработку.

В результате для пользователей системы поле ИНН станет обязательным для заполнения:



При этом программно карточку справочника можно сохранять, даже если поле не будет заполнено.

Как сделать свойство обязательным в зависимости от значения другого свойства

Ситуация

Предположим, в компании есть процесс по выбору поставщика для закупки. Когда поставщик утвержден, начинается согласование договора.

Для выбора поставщика используется внутренний документ с типом «Заявка на закупку» (SourcingRequest). Карточка документа содержит свойства, схожие с договором: сумма, предмет сделки и др. С помощью флажка **Один поставщик** можно выбрать два варианта заполнения заявки: указать несколько поставщиков или только одного.

Если флажок **Один поставщик** (isSingle) установлен, то свойства **Рекомендуемый поставщик** (RecommSupplier) и **Обоснование выбора** (Justification) должны становиться видимыми и обязательными.

Решение

1. Изменения параметров свойств задаются в разделяемом коде. В редакторе типа документа «Заявка на закупку» (SourcingRequest) добавьте разделяемую функцию **SetStatePropertiesDependSingle()**, которая делает свойства **Рекомендуемый поставщик** (RecommSupplier) и **Обоснование выбора** (Justification) видимыми и обязательными.

```
public void SetStatePropertiesDependSingle()
  {
    var isSingle = _obj.IsSingle.HasValue && _obj.IsSingle.Value;
    // Свойство "Рекомендуемый поставщик" сделать обязательным и видимым.
    _obj.State.Properties.RecommSupplier.IsRequired = isSingle;
    _obj.State.Properties.RecommSupplier.IsVisible = isSingle;
    // Свойство "Обоснование выбора" сделать обязательным и видимым.
    _obj.State.Properties.Justification.IsRequired = isSingle;
    _obj.State.Properties.Justification.IsVisible = isSingle;
  }
```

   isSingle – свойство сущности с типом **Логическое**. Возможные значения: **True** – в карточке документа указан один поставщик, **False** – указано несколько поставщиков.

2. Изменение параметров свойства необходимо выполнять в событии Изменение значения свойства. Поэтому добавленную функцию вызовите в этом событии. Для этого перейдите в узел «Свойства» и выберите свойство **Один поставщик** (isSingle). Добавьте для него данное событие и в редакторе программного кода добавьте код:

```
public virtual void
  IsSingleChanged(Sungero.Domain.Shared.BooleanPropertyChangedEventArgs e)
  {
    // Свойства "Рекомендуемый поставщик" и "Обоснование выбора"
    // сделать видимыми и обязательными.
    Functions.SourcingRequest.SetStatePropertiesDependSingle(_obj);
    // Очистить значения свойств "Рекомендуемый поставщик" и "Обоснование
    // выбора", если флажок "Один поставщик" сначала был установлен,
    // а затем снят.
    if (e.NewValue.HasValue && !e.NewValue.Value)
    {
      _obj.RecommSupplier = string.Empty;
      _obj.Justification = string.Empty;
    }
  }
```

3. В редакторе типа документа добавьте событие Обновление формы и напишите код:

```
public override void Refresh(Sungero.Presentation.FormRefreshEventArgs e)
  {
    base.Refresh(e);
  ...
    Functions.SourcingRequest.SetStatePropertiesDependSingle(_obj);
  }
```

В результате, когда пользователь устанавливает флажок **Один поставщик**, свойства **Рекомендуемый поставщик** и **Обоснование выбора** становятся видимыми и обязательными.

Как сделать свойство доступным в зависимости от прав доступа пользователя

Ситуация

Предположим, в компании есть процесс по выбору поставщика для закупки. Для выбора поставщика используется внутренний документ с типом «Заявка на закупку» (SourcingRequest). Карточка документа содержит свойства, схожие с договором: сумма, предмет сделки и др. Когда поставщик определен, задание для заполнения свойства **Утвержденный поставщик** (ApprovedSupplier) в карточке заявки приходит определенному сотруднику. Это пользователь, который входит в роль «Специалист по закупкам». Для остальных сотрудников свойство недоступно для изменения. Таким образом обеспечивается прозрачность процесса и отсутствие манипуляций при выборе поставщика.

Рассмотрим, как свойство **Утвержденный поставщик** сделать доступным для изменения только специалисту по закупкам.

Решение

1. Создайте в инициализации используемого модуля предопределенную роль «Специалист по закупкам». Для этого в редакторе модуля, в группе «Структуры и константы» добавьте новую константу **ProcurementSpecialistRole**, она будет использоваться в функции **CreateRole()**.

```
public static class RoleGuid
   {
     // Указать GUID роли "Специалист по закупкам".
     public static readonly Guid ProcurementSpecialistRole = new Guid("2062022D-
   745C-4E02-AF2F-26AD229E8C61");
     ...
   }
```

2. В группе «Функции» перейдите по ссылке **Добавить** рядом с функциями инициализации. Для создания предопределенной роли «Специалист по закупкам» добавьте код, в котором вызывается функция **CreateRole()** из модуля Sungero.Docflow *базового решения* Directum RX:

```
public static void CreateRoles()
   {
     InitializationLogger.Debug("Init: Create Default Roles");
     Sungero
   .Docflow
   .PublicInitializationFunctions
   .Module.CreateRole(ModuleName.Resources.RoleNameProcurement,
   ModuleName.Resources.DescriptionProcurementSpecialistRole,
   Constants.Module.RoleGuid.ProcurementSpecialistRole);
   }
```

3. В группе «События» перейдите по ссылке **Инициализация**. В редакторе программного кода вызовите добавленную функцию **CreateRoles()** в обработчике события **Initializing()**:

```
public override void Initializing(Sungero.Domain.ModuleInitializingEventArgs e)
  {
    CreateRoles();
  }
```

   В результате в инициализации модуля будет создана предопределенная роль «Специалист по закупкам».

4. Для проверки вхождения сотрудника в роль «Специалист по закупкам» в группе «Функции» добавьте серверную функцию **IsProcurementSpecialist()**:

```
// Проверить, входит ли субъект прав в роль "Специалист по закупкам".
  // <param name="recepient">Субъект прав.</param>
  [Remote, Public]
  public bool IsProcurementSpecialist(IRecipient recipient)
  {
    var role = Roles.GetAll().FirstOrDefault(r => r.Sid ==
Constants.Module.ProcurementSpecialistRole);
    return role != null && recipient.IncludedIn(role);
  }
```

   В результате после выполнения функция вернет **True**, если сотрудник входит в роль «Специалист по закупкам», иначе – **False**.

5. Чтобы сделать свойство **Утвержденный поставщик** (ApprovedSupplier) доступным для изменения только специалисту по закупкам, в редакторе типа документа «Заявка на закупку» (SourcingRequest) добавьте событие «Показ формы». Это событие задает настройки формы карточки документа.

```
// Сделать свойство Утвержденный поставщик (ApprovedSupplier) доступным для
изменения,
  // если текущий пользователь входит в роль "Специалист по закупкам".
  public override void Showing(Sungero.Presentation.FormShowingEventArgs e)
  {
    base.Showing(e);
    _obj.State.Properties.ApprovedSupplier.IsEnabled =
Functions.Module.Remote.IsProcurementSpecialist(Users.Current);
  }
```

   В результате выполнится проверка принадлежности текущего пользователя к конкретной роли. В зависимости от этого свойство **Утвержденный поставщик** (ApprovedSupplier) будет доступным или недоступным для редактирования.

Как управлять доступностью нового свойства, если оно зависит от существующего свойства

Ситуация

Предположим, в карточку документа, который является наследником базового договора (Sungero.Contracts.ContractBase), добавлено новое свойство **Сумма с НДС**. Имя свойства – VATAmount, тип – Вещественное. Свойство необязательно для заполнения. Необходимо, чтобы новое свойство было доступным для изменения в зависимости от доступности свойства – **Сумма** (TotalAmount), т.к. свойства тесно связаны.

Решение

Для управления доступностью нового свойства, в зависимости от поведения существующего свойства, нужно переопределить логику функции **ChangeDocumentPropertiesAccess()** базового типа. Функция изменяет доступность реквизитов документа в зависимости от стадии жизненного цикла документа.

1. В редакторе типа документа, который является наследником, перейдите по ссылке к разделяемым функциям.
2. Добавьте функцию **ChangeDocumentPropertiesAccess()** с модификатором override:

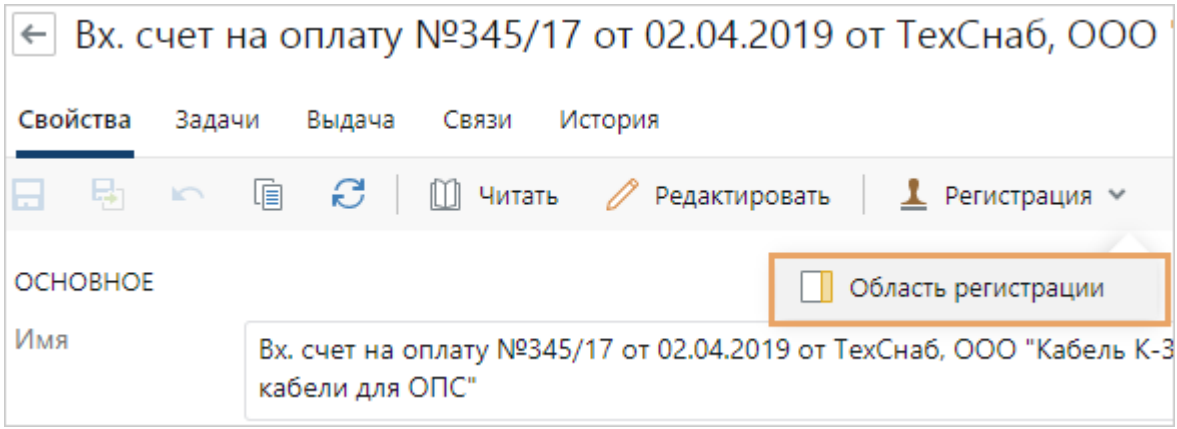
```
// Сменить доступность реквизитов документа.
  // <param name="isEnabled">True, если свойства должны быть доступны.</param>
  // <param name="isRepeatRegister">Перерегистрация.</param>
  public override void ChangeDocumentPropertiesAccess(bool isEnabled, bool isRepeatRegister)
  {
    base.ChangeDocumentPropertiesAccess(isEnabled, isRepeatRegister);
    _obj.State.Properties.VATAmount.IsEnabled = _obj.State.Properties.TotalAmount.IsEnabled;
  }
```

В результате новое свойство **Сумма НДС** в карточке документа будет доступно тогда, когда доступно свойство **Сумма**.

Как скрыть свойство из области регистрации в карточке документа

Ситуация

Предположим, в область регистрации на карточке входящего счета добавлено свойство **Оплачен** (IsPayed). Необходимо, чтобы в системе при скрытии области регистрации добавленное свойство тоже скрывалось.



Решение

С помощью функции **ChangeRegistrationPaneVisibility()** *базового решения* Directum RX изменяется отображение панели регистрации. Для скрытия добавленного свойства **Оплачен** (IsPayed) при скрытии панели регистрации, нужно переопределить логику функции **ChangeRegistrationPaneVisibility()** базового типа.

1. В редакторе типа документа, который является перекрытием входящего счета, перейдите по ссылке к разделяемым функциям.
2. Добавьте функцию **ChangeRegistrationPaneVisibility()** с модификатором override:

```
// Изменить отображение панели регистрации.
  // <param name="needShow">Признак отображения панели регистрации.</param>
  // <param name="repeatRegister">Признак повторной регистрации.</param>
  public override void ChangeRegistrationPaneVisibility(bool needShow, bool repeatRegister)
  {
    base.ChangeRegistrationPaneVisibility(needShow, repeatRegister);
    _obj.State.Properties.IsPayed.IsVisible = needShow;
  }
```

В результате свойство **Оплачен** (IsPayed) будет невидимым при скрытии панели регистрации.

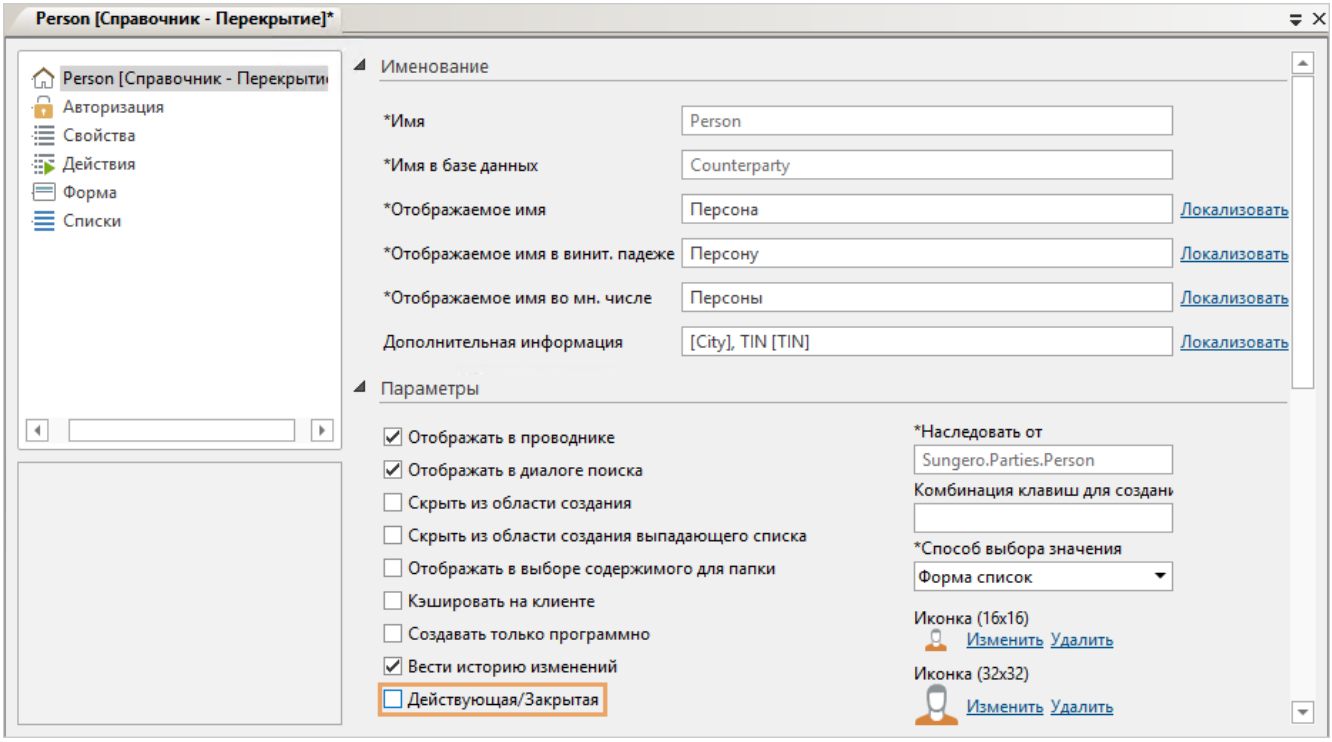
Как выбрать закрытую запись справочника

Ситуация

По умолчанию при выборе записей справочника в Directum RX доступны только действующие записи. Например, при заполнении поля **Персона** в справочнике **Контакты** в списке для выбора отображаются только записи справочника **Персоны** со статусом **Действующая**. Иногда возникают ситуации, когда нужно показывать все записи справочника, в том числе со статусом **Закрытая**.

Решение

1. В своем решении перекройте тип справочника, у которого можно будет выбирать все записи независимо от статуса. Например, перекройте справочник «Персоны» (Sungero.Parties.Person).
2. В редакторе перекрытого типа справочника снимите флажок с параметра Действующая/Закрытая.



3. Сохраните изменения и опубликуйте разработку.

В результате при заполнении поля **Персона** в списке для выбора отобразятся записи перекрытого справочника со состоянием **Действующая** и **Закрытая**.

Интеллектуальная обработка документов

Стандартная версия Directum RX содержит базовые возможности интеллектуальной обработки документов. На текущий момент реализована обработка нескольких типов документов. Модель классификации по типам документов, которая предоставляется по

запросу в службу поддержки Directum RX, также представляет собой стартовый пример работы. В зависимости от потребностей организации функциональность интеллектуальной обработки документов можно адаптировать под конкретные бизнес-задачи. Адаптация включает:

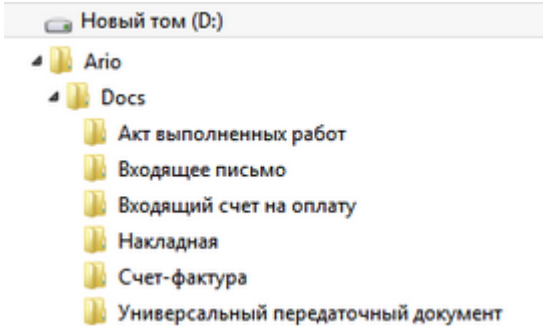
- обучение модели классификации на реальных документах организации для повышения точности классификации и извлечения фактов;
- дополнение базовых возможности интеллектуальной обработки документов. Например, есть возможность расширить состав обрабатываемых документов или изменить порядок обработки.

ВАЖНО. Перед адаптацией активируйте функциональность интеллектуальной обработки документов в своем контуре для разработки и тестирования, если еще не сделали это. Подробнее см. в справке Directum RX, раздел «Настройка интеллектуальной обработки».

Чтобы внести изменения в функциональность интеллектуальной обработки документов:

1. В среде разработки Directum RX адаптируйте разработку под необходимые бизнес-задачи. Например, добавьте интеллектуальную обработку для новых типов документов, доработайте задачу на обработку документов, измените логику заполнения полей карточек извлеченными фактами и т.д. Подробнее см. пример разработки «Как добавить обработку для нового типа документов».
2. Подготовьте документы для обучения. Потребуется минимум 100 документов каждого типа. Чем больше документов, тем выше точность классификации и дольше длится обучение.

   На сервере, где установлены сервисы Directum Ario, в сетевой папке создайте структуру папок документов каждого типа.



Требования к размещению документов для обучения:

- путь до папок с документами для обучения должен состоять из латинских символов без пробелов, использование кириллицы не допускается;
- названия папок должны быть содержательными и написаны кириллицей. Типы документов, которые создаются при обучении, получают названия соответствующих папок. Например, если папка называется «Акт выполненных работ», то соответствующий тип документов будет назван «Акт выполненных работ»;
- размещайте в папках только документы соответствующего типа. Например, в папке «Входящее письмо» должны лежать только входящие письма. Если в папке содержатся документы других типов, точность классификации документов и распознавания фактов снижается.

3. Обучите классификатор по типам документов на подготовленных документах. Подробнее см. в руководстве администратора Directum RX, раздел «Обучение классификатора».
4. Проверьте захват документов. Подробнее см. в руководстве администратора Directum RX, раздел «Проверка захвата документов».

Если точность классификации документов и распознавания фактов недостаточно высокая:

1. Проверьте, что в каждой папке с документами для обучения находятся только документы нужного типа. Например, в папке «Входящее письмо» должны находиться только входящие письма, в папке «Накладная» только накладные и т.д.
2. Увеличьте количество документов каждого типа.
3. Повторите обучение классификатора.
4. Повторно проверьте захват документов.

Если при одном из прошлых обучений точность классификации документов и распознавания фактов была выше, чем при повторном обучении, можно откатить изменения и опубликовать в классификатор модель классификации с самыми высокими показателями. В результате данная модель будет использоваться для обработки документов.

Когда захват документов прошел успешно, а точность классификации и распознавания фактов вас устраивает, подготовьте комплект для переноса разработки в продуктивный контур. Для этого:

- экспортируйте пакет разработки с исходными кодами из среды разработки Directum RX;
- экспортируйте обученную модель классификации из сервисов Directum Ario.

Передайте комплект для переноса разработки администратору системы Directum RX.

Как настроить распознавание нового типа документов

Ситуация

В системе по умолчанию поддерживается интеллектуальная обработка:

- входящих писем;
- входящих счетов;
- товарных накладных;

- актов выполненных работ;
- счетов-фактур полученных, в том числе корректировочных и исправленных;
- универсальных передаточных документов, в том числе корректировочных и исправленных;
- договоров и дополнительных соглашений.

Необходимо настроить систему, чтобы распознавались документы нового типа, например финансовые отчеты.

Решение

Чтобы настроить распознавание нового типа документов:

1. Настройте классификацию документов: дообучите базовый классификатор, проанализируйте результаты обучения и протестируйте работу классификатора, перенесите обученную модель классификации в продуктивную систему.
2. Настройте извлечение фактов из документов: создайте новую грамматику и обучите новую модель извлечения фактов, перенесите обученную модель извлечения фактов в продуктивную систему.
3. Создайте новый тип документа под задачи организации и подготовьте его к работе.
4. Разработайте функцию для создания карточки нового документа и заполнения его реквизитов. В качестве примера можно использовать функцию создания входящего письма.
5. Задайте соответствие нового класса грамматике и функции обработки результатов, созданных на втором и четвертом шаге.
6. Добавьте новый тип документа в интеллектуальные возможности системы по автоматическому дообучению классификатора. Для этого добавьте его в окно смены типа, а GUID, название модуля и название функции для создания и заполнения карточки в серверную функцию **GetEntityTypeAndProcessingFunctionMapping()**:

```
/// <summary>
    /// Получить словарь соответствий типа сущности паре модуль и имя функции
    обработки.
    /// </summary>
    /// <param name="entity">Сущность.</param>
    /// <returns>Словарь соответствий.</returns>
    [Public]
    public override System.Collections.Generic.IDictionary<string, string[]>
    GetEntityTypeAndProcessingFunctionMapping(Sungero.Domain.Shared.IEntity
    entity)
    {
      const string ModuleName = "Sungero.SmartProcessing";

      bool? isAdjustment = false;
      if (Sungero.Docflow.AccountingDocumentBases.Is(entity))
        isAdjustment =
    Sungero.Docflow.AccountingDocumentBases.As(entity).IsAdjustment;

      var taxInvoiceFunctionName = isAdjustment != true
        ?
    Sungero
    .Docflow.PublicConstants.Module.ProcessingFunctionName.CreateTaxInvoice
```

```
    :
Sungero
.Docflow
.PublicConstants.Module.ProcessingFunctionName.CreateTaxInvoiceCorrection;

  var universalTransferDocumentFunctionName = isAdjustment != true
    ?
Sungero
.Docflow
.PublicConstants
.Module.ProcessingFunctionName.CreateUniversalTransferDocument
    :
Sungero
.Docflow
.PublicConstants
.Module.ProcessingFunctionName.CreateUniversalTransferCorrectionDocument;

  var isKindAccBalance = true;
  if (Sungero.Docflow.OfficialDocuments.Is(entity))
  isKindAccBalance =
Sungero.Docflow.OfficialDocuments.As(entity).DocumentKind.Id ==
DirRX
.FinReports
.PublicFunctions
.Module
.Remote
.GetDocumentKind
(DirRX.FinReports.PublicConstants.Module.Initialize.BalanceGuid).Id;

  var finReportsDocumentFunctionName = isKindAccBalance == true
    ?
DirRX
.FinReports
.PublicConstants.Module.ProcessingFunctionName.CreateAccBalanceDocument
    :
DirRX
.FinReports
.PublicConstants.Module.ProcessingFunctionName.CreateFinanceReportsDocument;

  var mapping = new Dictionary<string, string[]>();
  mapping
.Add(Sungero.RecordManagement.PublicConstants.Module.IncomingLetterGuid,
             new[] { ModuleName,
Sungero
.Docflow
.PublicConstants.Module.ProcessingFunctionName.CreateIncomingLetter });

  mapping
.Add
(Sungero
.Docflow
.PublicConstants.AccountingDocumentBase.ContractStatementInvoiceGuid,
             new[] { ModuleName,
Sungero
.Docflow
.PublicConstants.Module.ProcessingFunctionName.CreateContractStatement });
```

```
            mapping
            .Add
            (Sungero.Docflow.PublicConstants.AccountingDocumentBase.WaybillInvoiceGuid,
                        new[] { ModuleName,
            Sungero
            .Docflow.PublicConstants.Module.ProcessingFunctionName.CreateWaybill });

            mapping
            .Add
            (Sungero
            .Docflow.PublicConstants.AccountingDocumentBase.IncomingTaxInvoiceGuid,
                        new[] { ModuleName, taxInvoiceFunctionName });

            mapping
            .Add
            (Sungero
            .Docflow.PublicConstants.AccountingDocumentBase.OutcomingTaxInvoiceGuid,
                        new[] { ModuleName, taxInvoiceFunctionName });

            mapping
            .Add
            (Sungero
            .Docflow
            .PublicConstants.AccountingDocumentBase.UniversalTransferDocumentGuid,
                        new[] { ModuleName, universalTransferDocumentFunctionName });

            mapping
            .Add
            (Sungero.Docflow.PublicConstants.AccountingDocumentBase.IncomingInvoiceGuid,
                        new[] { ModuleName,
            Sungero
            .Docflow
            .PublicConstants.Module.ProcessingFunctionName.CreateIncomingInvoice });

            mapping.Add(Sungero.Contracts.PublicConstants.Module.ContractGuid,
                        new[] { ModuleName,
            Sungero
            .Docflow.PublicConstants.Module.ProcessingFunctionName.CreateContract });

            mapping.Add(Sungero.Contracts.PublicConstants.Module.SupAgreementGuid,
                        new[] { ModuleName,
            Sungero
            .Docflow
            .PublicConstants.Module.ProcessingFunctionName.CreateSupAgreement });

            mapping.Add(DirRX.FinReports.PublicConstants.Module.FinanceReportsGuid,
                        new[] { "DirRX.FinReports", finReportsDocumentFunctionName });

            return mapping;
        }
```

7. Добавьте новый тип документа в интеллектуальные возможности системы по перекомплектованию. Для этого добавьте информацию о нем в разделяемую функцию **GetPackageDocumentTypePriorities()**:

```
/// <summary>
```

```
/// Получить приоритеты типов документов для определения ведущего документа
в комплекте.
/// </summary>
/// <returns>Словарь с приоритетами типов.</returns>
[Public]
public override System.Collections.Generic.IDictionary<System.Type, int>
GetPackageDocumentTypePriorities()
{
  var leadingDocumentPriority = new Dictionary<System.Type, int>();
  leadingDocumentPriority
.Add(typeof(DirRX.FinReports.IFinanceReports).GetFinalType(), 10);
  leadingDocumentPriority
.Add(typeof(Sungero.RecordManagement.IIncomingLetter).GetFinalType(), 9);
  leadingDocumentPriority
.Add(typeof(Sungero.Contracts.IContract).GetFinalType(), 8);
  leadingDocumentPriority
.Add(typeof(Sungero.Contracts.ISupAgreement).GetFinalType(), 7);
  leadingDocumentPriority
.Add(typeof(Sungero.FinancialArchive.IContractStatement).GetFinalType(), 6);
  leadingDocumentPriority
.Add(typeof(Sungero.FinancialArchive.IWaybill).GetFinalType(), 5);
  leadingDocumentPriority
.Add
(typeof(Sungero.FinancialArchive.IUniversalTransferDocument).GetFinalType(),
 4);
  leadingDocumentPriority
.Add(typeof(Sungero.FinancialArchive.IIncomingTaxInvoice).GetFinalType(),
3);
  leadingDocumentPriority
.Add(typeof(Sungero.Contracts.IIncomingInvoice).GetFinalType(), 2);
  leadingDocumentPriority
.Add(typeof(Sungero.FinancialArchive.IOutgoingTaxInvoice).GetFinalType(),
1);
  leadingDocumentPriority
.Add(typeof(Sungero.Docflow.ISimpleDocument).GetFinalType(), 0);

  return leadingDocumentPriority;
}
```

8. Опубликуйте изменения решения в локальную систему Directum RX.
9. Проверьте захват нового типа документов. Подробнее см. документацию Directum RX в руководстве администратора, раздел «Проверка захвата документов».

Как добавить дополнительный классификатор

Ситуация

При интеллектуальной обработке и занесении в Directum RX среди входящих писем необходимо различать обычные письма и претензии.

ВАЖНО. В данном примере рассмотрим случай, когда в карточке документа в поле **Примечание** указывается, является ли письмо претензией. Для упрощения более сложная логика обработки документов в примере не рассматривается.

Решение

В базовой конфигурации интеллектуальной обработки документов используется классификация по типу документов. Это позволяет из общего потока документов выделять входящие письма. Чтобы добавить классификацию по еще одному признаку и среди входящих писем различать обычные письма и претензии, подключите дополнительный классификатор. Для этого:

1. В сервисах Directum Ario создайте дополнительный классификатор LetterClassifier для определения обычных писем и претензий. Подробнее см. в руководстве администратора Directum RX, раздел «Создание классификатора».
2. Подготовьте документы для обучения классификатора. Документы для обучения разместите, например, в папках «Обычное письмо» и «Претензия». Потребуется минимум 100 документов каждого типа.
3. В среде разработки Directum RX создайте решение.
4. В новом решении создайте перекрытие модуля Sungero.SmartProcessing из базового решения. Подробнее см. пример перекрытия модуля.
5. В редакторе перекрытия модуля в группе «События» перейдите по ссылке **Инициализация**. В редакторе программного кода в обработчике события **Initializing()** укажите дополнительный классификатор LetterClassifier, который будет вызываться при обработке документов:

```
public override void Initializing(Sungero.Domain.ModuleInitializingEventArgs e)
  {
    base.Initializing(e);

      InitializationLogger.Debug("Init: Fill smart processing additional
classifiers.");
    FillSmartAdditionalClassifiers();
  }

  [Public]
  public virtual void FillSmartAdditionalClassifiers()
  {
                                  var smartProcessingSettings =
Sungero.Docflow.PublicFunctions.SmartProcessingSetting.GetSettings();
    if (smartProcessingSettings != null)
    {
      // Добавление классификатора LetterClassifier для определения обычных
      // писем и претензий.
      var classifierId = smartProcessingSettings.LetterClassifierId;
      var classifierName = smartProcessingSettings.LetterClassifierName;

                    if (smartProcessingSettings.AdditionalClassifiers.Where(c
=> c.ClassifierId == classifierId).Count() == 0)
      {
                                          var additionalClassifier =
smartProcessingSettings.AdditionalClassifiers.AddNew();
        additionalClassifier.ClassifierId = classifierId;
        additionalClassifier.ClassifierName = classifierName;
        smartProcessingSettings.Save();
      }
```

```
    }
  }
```

6. В перекрытом модуле в функции создания входящих писем, обработанных сервисами Directum Ario, создайте функцию, которая вызывает базовую функцию создания входящего письма и добавляет в карточку документа в поле **Примечание** информацию о том, является ли письмо претензией:

```
/// <summary>
  /// Создать входящее письмо.
  /// </summary>
  /// <param name="documentInfo" >Информация о документе.</param>
  /// <param name="responsible" >Ответственный за верификацию.</param>
  /// <returns>Входящее письмо.</returns>
  [Public]
  public virtual IOfficialDocument CreateIncomingLetter(IDocumentInfo
  documentInfo, IEmployee responsible)
  {
    // Вызов функции создания входящего письма.
    var document = RecordManagement.IncomingLetters.Create();
     Sungero.SmartProcessing.PublicFunctions.Module.FillIncomingLetterPropertie
  s(document, documentInfo, responsible);

    // Заполнение информации в карточке документа в поле "Примечание" о том,
    // является ли письмо претензией.
                                    var additionalClassifiers                 =
  documentInfo.ArioDocument.RecognitionInfo.AdditionalClassifiers;
    if (additionalClassifiers.Count > 0)
                                                                  document.Note
  =
   string
  .Format("{0}", additionalClassifiers.FirstOrDefault().PredictedClass);

    return document;
  }
```

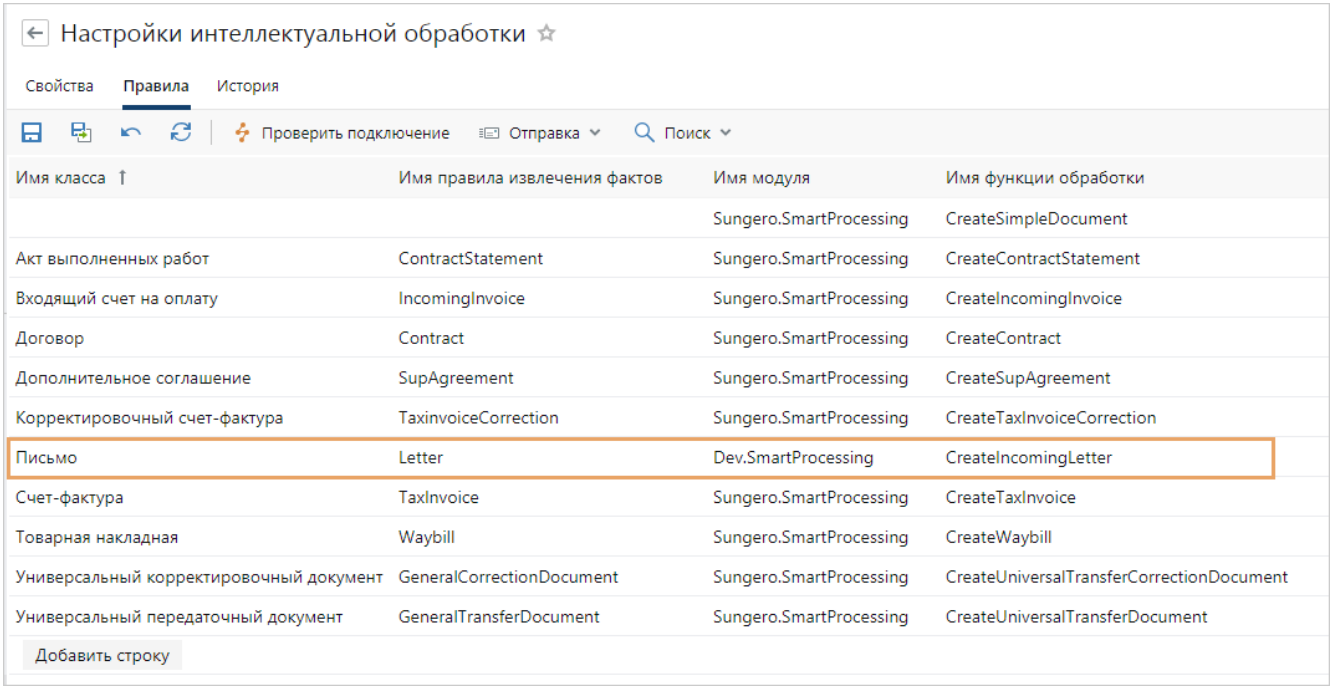
7. Опубликуйте изменения решения в локальную систему Directum RX.

8. В Directum RX на обложке модуля «Документооборот» в группе «Интеллектуальная обработка документов» выберите пункт **Настройки**. В открывшейся карточке перейдите на закладку «Правила». В табличной части в строке, которая соответствует логике обработки входящих писем, укажите:

   Имя модуля с доработанной функцией обработки результата распознавания.

   Имя функции обработки результата распознавания, которая была добавлена в п. 6.

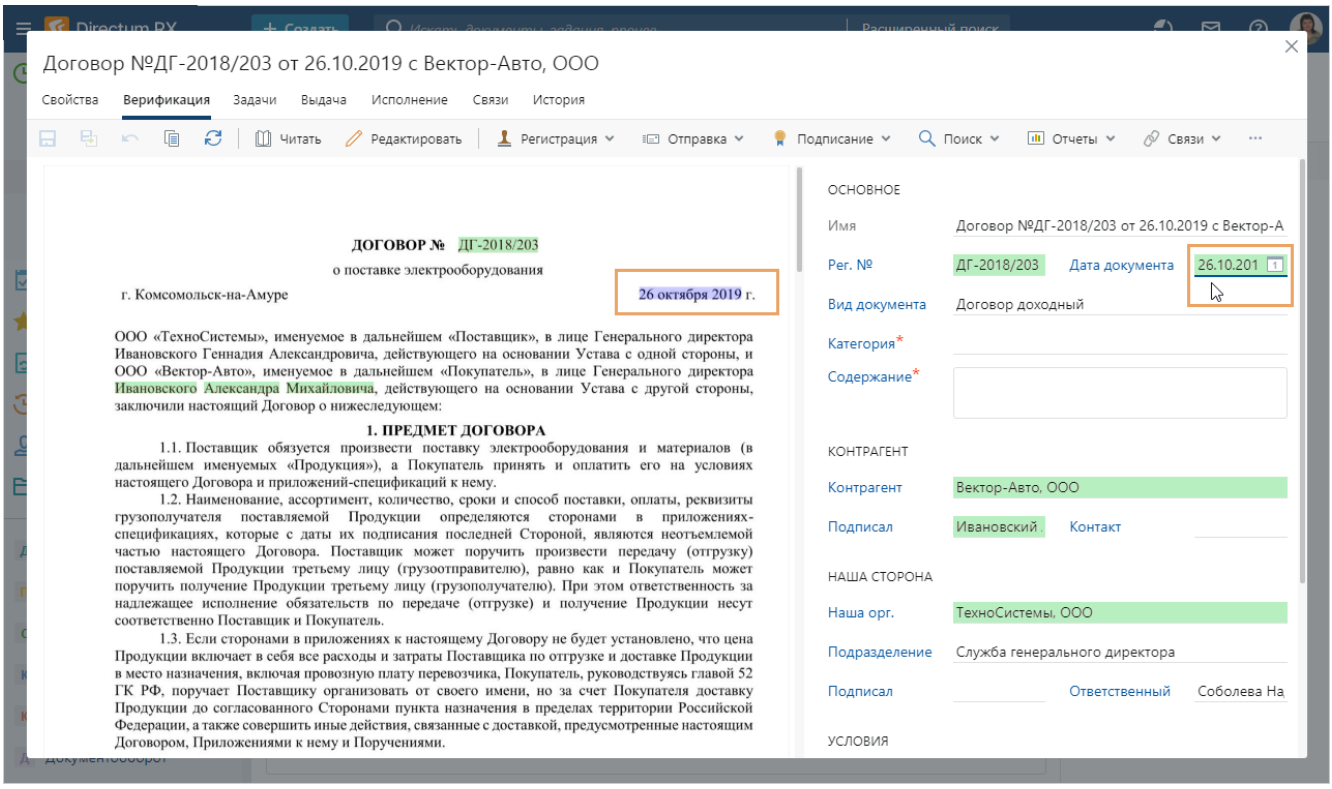
   Подробнее см. в руководстве администратора, раздел «Правила обработки документов».



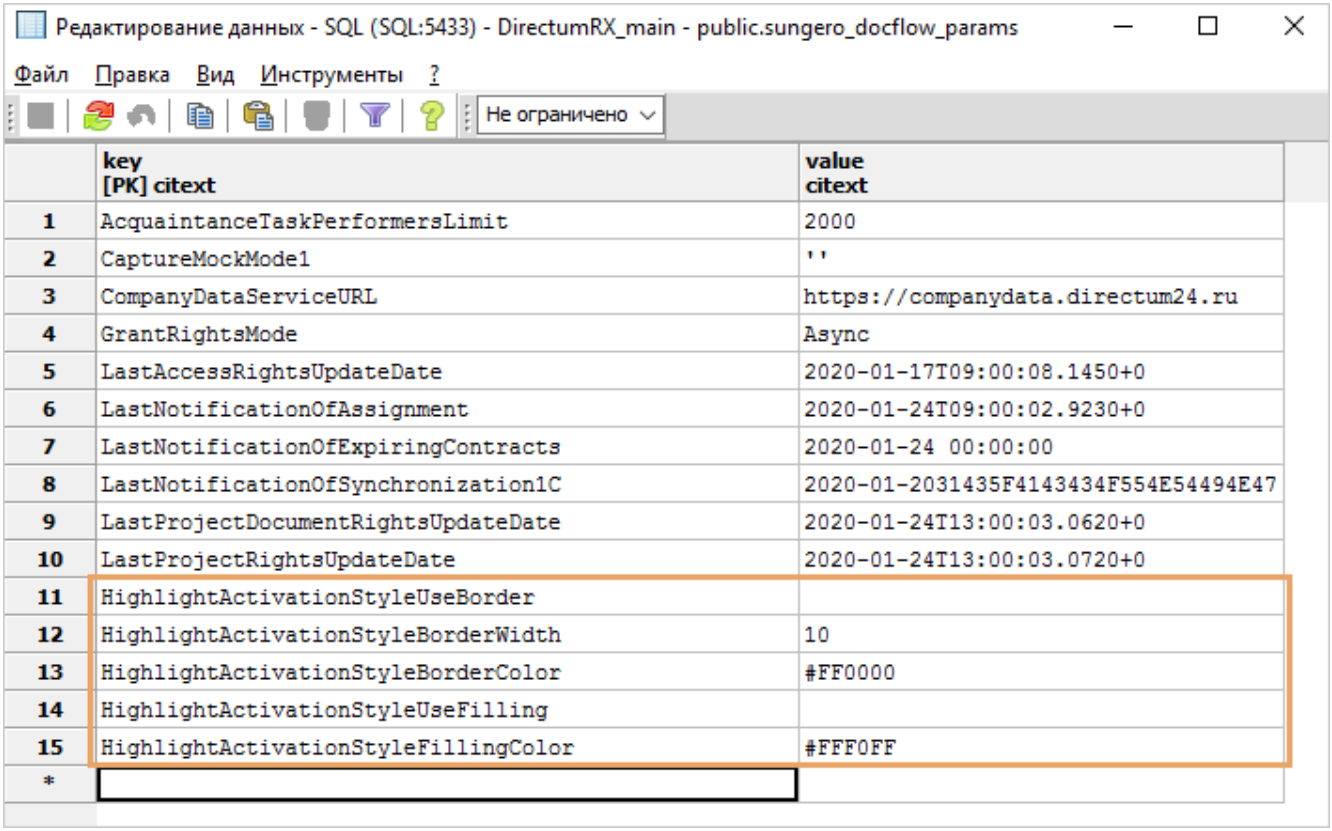
9. Обучите классификатор на подготовленных документах. Подробнее см. в руководстве администратора Directum RX, раздел «Обучение классификатора».
10. Проверьте захват документов. Подробнее см. в руководстве администратора Directum RX, раздел «Проверка захвата документов».

Как изменить стиль подсветки выделенного факта в области предпросмотра

Если в системе Directum RX используются возможности интеллектуальной обработки документов, то для упрощения проверки в карточке документа при щелчке на поле, в области предспросмотра сразу выделяется область с фактом, который соответствует значению этого поля. По умолчанию область выделяется сиреневым цветом:



При адаптации функциональности интеллектуальной обработки документов под потребности организации стиль выделения области с фактом можно изменить. Для этого в базе данных откройте таблицу sungero_docflow_params и добавьте параметры:



	key [PK] citext	value citext
1	AcquaintanceTaskPerformersLimit	2000
2	CaptureMockMode1	''
3	CompanyDataServiceURL	https://companydata.directum24.ru
4	GrantRightsMode	Async
5	LastAccessRightsUpdateDate	2020-01-17T09:00:08.1450+0
6	LastNotificationOfAssignment	2020-01-24T09:00:02.9230+0
7	LastNotificationOfExpiringContracts	2020-01-24 00:00:00
8	LastNotificationOfSynchronization1C	2020-01-2031435F4143434F554E54494E47
9	LastProjectDocumentRightsUpdateDate	2020-01-24T13:00:03.0620+0
10	LastProjectRightsUpdateDate	2020-01-24T13:00:03.0720+0
11	HighlightActivationStyleUseBorder	
12	HighlightActivationStyleBorderWidth	10
13	HighlightActivationStyleBorderColor	#FF0000
14	HighlightActivationStyleUseFilling	
15	HighlightActivationStyleFillingColor	#FFF0FF
*		

HighlightActivationStyleUseBorder. Признак выделения области с фактом с помощью рамки. Проверяется только наличие параметра в таблице, в колонке **value** может быть любое значение.

HighlightActivationStyleBorderColor. Цвет рамки. Значение задается в формате RGB в виде #RRGGBB или (R,G,B).

HighlightActivationStyleBorderWidth. Толщина рамки. Рекомендуемые значения от 0 до 50.

HighlightActivationStyleUseFilling. Признак выделения области с фактом с помощью заливки. Проверяется только наличие параметра в таблице, в колонке **value** может быть любое значение.

HighlightActivationStyleFillingColor. Цвет заливки. Значение задается в формате RGB в виде #RRGGBB или (R,G,B).

ВАЖНО. Если в таблице sungero_docflow_params нет параметров **HighlightActivationStyleUseBorder** и **HighlightActivationStyleUseFilling**, то используется стиль выделения области с фактом по умолчанию.

Пример 1. Изменение цвета для выделения области с фактом

В таблице sungero_docflow_params укажите параметры и их значения:

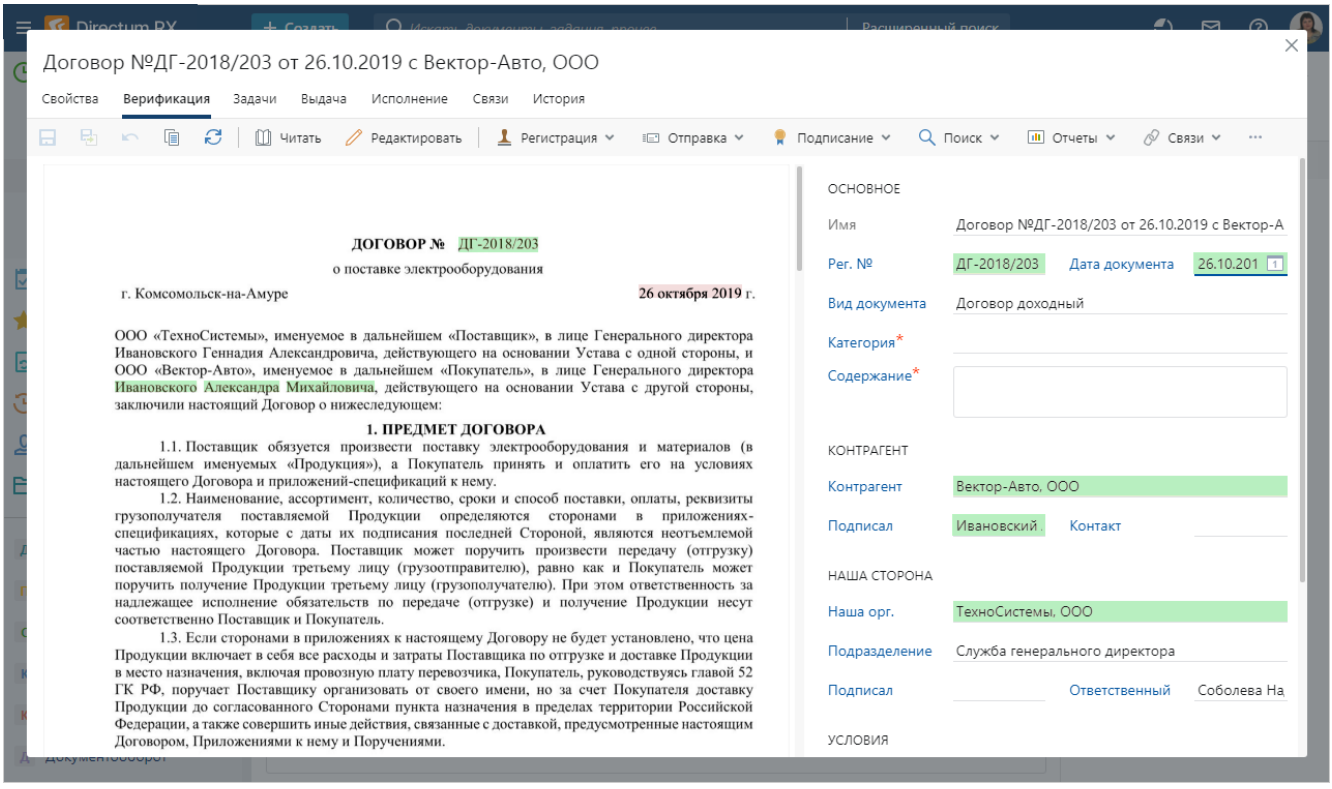
Редактирование данных - SQL (SQL:5433) - DirectumRX_main - public.sungero_docflow_params

Файл Правка Вид Инструменты ?

Не ограничено

	key [PK] citext	value citext
1	AcquaintanceTaskPerformersLimit	2000
2	CaptureMockMode1	''
3	CompanyDataServiceURL	https://companydata.directum24.ru
4	GrantRightsMode	Async
5	LastAccessRightsUpdateDate	2020-01-17T09:00:08.1450+0
6	LastNotificationOfAssignment	2020-01-24T09:00:02.9230+0
7	LastNotificationOfExpiringContracts	2020-01-24 00:00:00
8	LastNotificationOfSynchronization1C	2020-01-2031435F4143434F554E54494E47
9	LastProjectDocumentRightsUpdateDate	2020-01-24T13:00:03.0620+0
10	LastProjectRightsUpdateDate	2020-01-24T13:00:03.0720+0
11	HighlightActivationStyleUseFilling	
12	HighlightActivationStyleFillingColor	#c25353
*		

В результате при щелчке на поле справа область с фактом будет выделяться следующим образом:



Пример 2. Выделение выделения области с фактом рамкой

В таблице sungero_docflow_params укажите параметры и их значения:

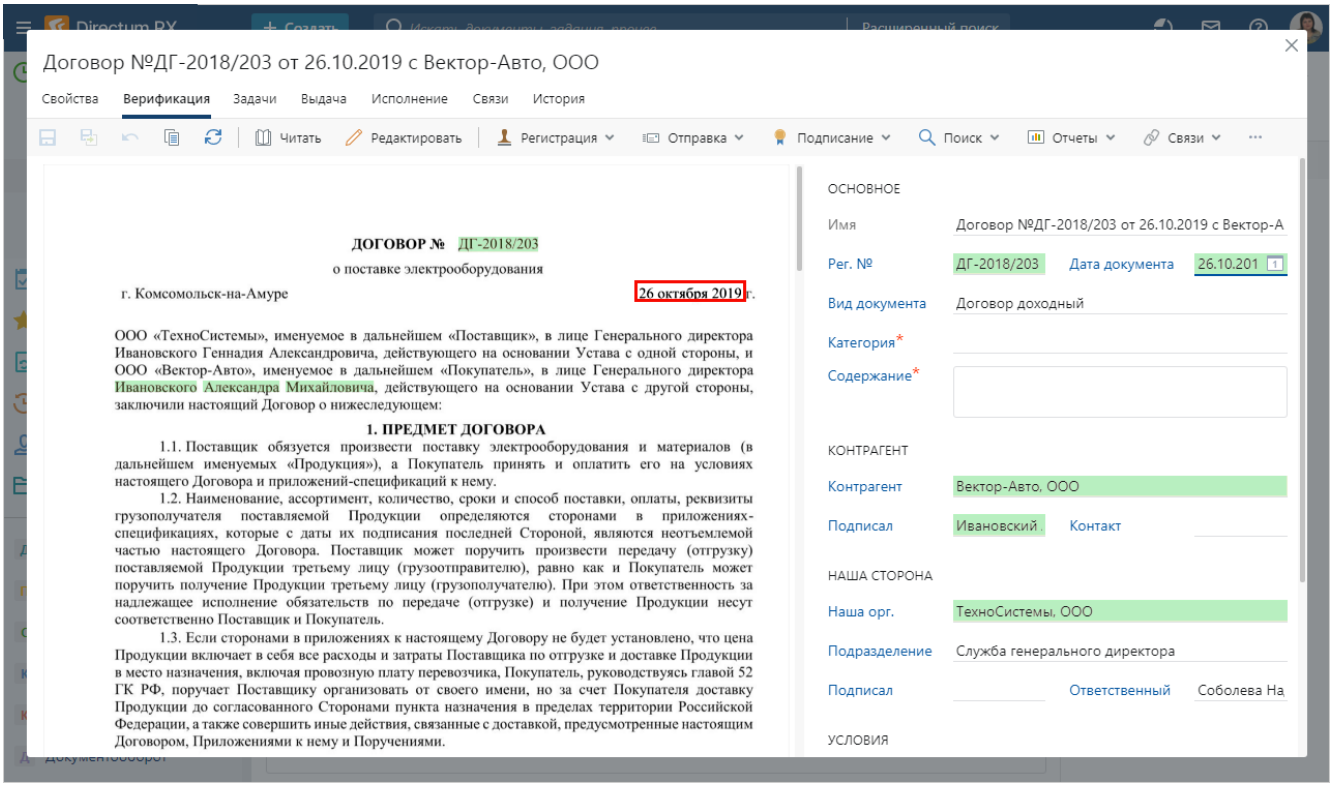
Редактирование данных - SQL (SQL:5433) - DirectumRX_main - public.sungero_docflow_params

Файл Правка Вид Инструменты ?

Не ограничено

	key [PK] citext	value citext
1	AcquaintanceTaskPerformersLimit	2000
2	CaptureMockMode1	''
3	CompanyDataServiceURL	https://companydata.directum24.ru
4	GrantRightsMode	Async
5	LastAccessRightsUpdateDate	2020-01-17T09:00:08.1450+0
6	LastNotificationOfAssignment	2020-01-24T09:00:02.9230+0
7	LastNotificationOfExpiringContracts	2020-01-24 00:00:00
8	LastNotificationOfSynchronization1C	2020-01-2031435F4143434F554E54494E47
9	LastProjectDocumentRightsUpdateDate	2020-01-24T13:00:03.0620+0
10	LastProjectRightsUpdateDate	2020-01-24T13:00:03.0720+0
11	HighlightActivationStyleUseBorder	
12	HighlightActivationStyleBorderWidth	10
13	HighlightActivationStyleBorderColor	#FF0000
*		

В результате при щелчке на поле справа область с фактом будет выделяться следующим образом:



Обработчики событий

В разделе содержатся примеры:

- Как добавить действие в историю работы с сущностью
- Передача данных между событиями одной сущности
- Передача данных между событиями разных сущностей
- Как определить, где находится код

Как добавить действие в историю работы с сущностью

Ситуация

Необходимо записывать в историю работы с сущностью выполненные действия. Например, для записей справочника ProjectCore («Проекты») фиксировать в истории изменение состояния проекта.

Состояние проекта содержится в свойстве **Stage**. Оно отражает, на какой стадии находится проект: инициация, исполнение, завершение, завершен.

Решение

В событии До сохранения истории необходимо создать перечисление с характеристикой выполняемой операции и вызвать метод **Write()**. Для этого:

1. В своем решении перекройте тип справочника ProjectCore («Проекты»).
2. В перекрытом типе справочника добавьте константу:

```
public const string StatusChange = "EStatusChange";
```

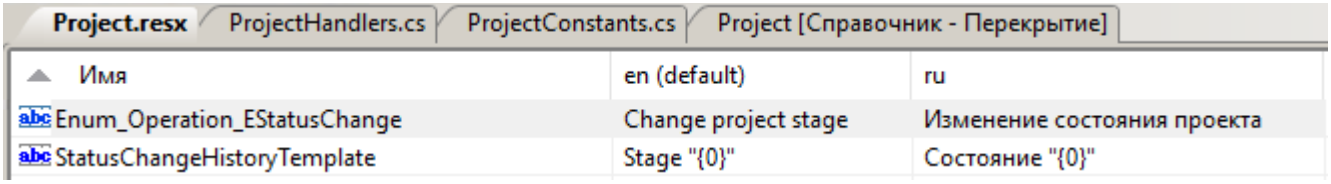
3. Добавьте обработчик события **До сохранения истории**:

```
public override void BeforeSaveHistory(Sungero.Domain.HistoryEventArgs e)
    {
      // Вызов базового обработчика события.
      base.BeforeSaveHistory(e);
      // Изменение состояния свойства Stage.
      var isStatusChanged = _obj.State.Properties.Stage.IsChanged;
      if (isStatusChanged)
      {
        // Добавление перечисления. StatusChange – ранее добавленная константа.
        var operation = new
    Enumeration(Constants.Projects.ProjectCore.StatusChange);
        var operationDetailed = operation;
        var comment = DEV.PerekrytiiaExample.Projects.Resources.
              StatusChangeHistoryTemplateFormat(Projects.Info.Properties.Stage.
              GetLocalizedValue(_obj.Stage));
        // Запись дополнительной информации в историю.
        e.Write(operation, operationDetailed, comment);
      }
    }
```

При добавлении кода обработчика события необходимо добавить *ресурсы*, значения которых будут отображаться на закладке «История»:

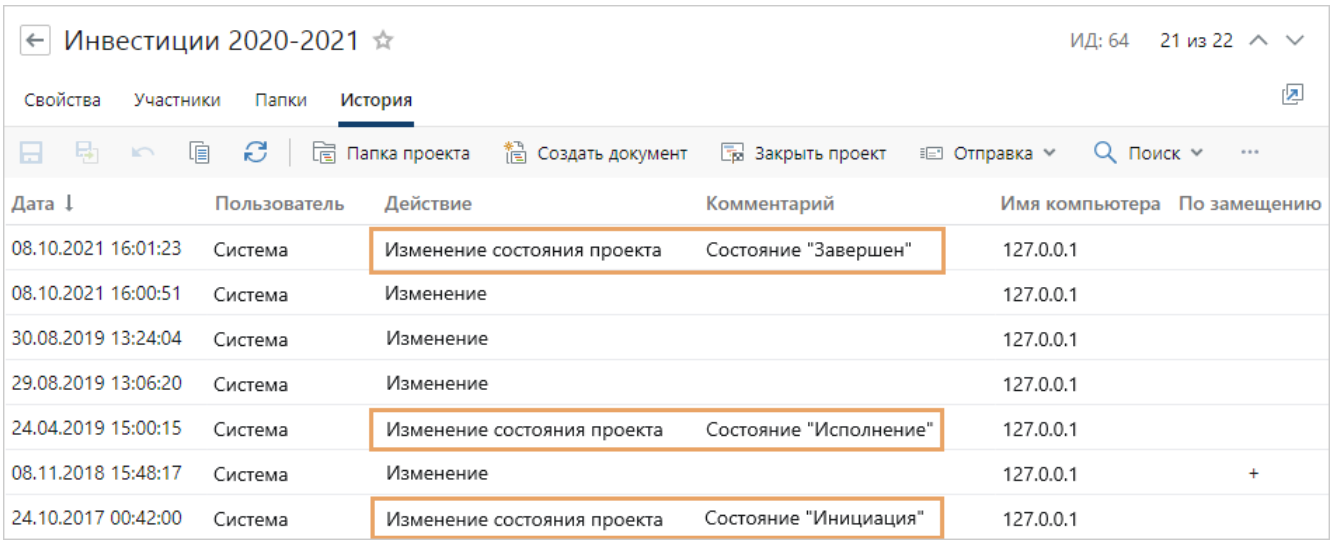
- Enum_Operation_<Ранее добавленная константа>. В данном случае это **Enum_Operation_EStatusChange**. Значение будет отображаться в столбце «Действие»;

- **StatusChangeHistoryTemplate**. Значение будет отображаться в столбце «Комментарий».



4. Сохраните изменения и опубликуйте разработку.

В результате при изменении состояния проекта в истории будет записываться информация об этом:



Передача данных между событиями одной сущности

Ситуация

Необходимо передать данные между событиями одной сущности.

Рассмотрим на примере базового решения Directum RX. При открытии карточки задачи на исполнение поручения (ActionItemExecutionTask) в событии **Показ формы** проверяется, разрешены ли в системе поручения без срока. Если разрешены, то в карточке задачи появится возможность установить флажок **Без срока**. Иначе флажок будет скрыт из карточки.

Чтобы при выполнении обработчика события **Обновление формы** проверка повторно уже не выполнялась, необходимо результат проверки в одном событии передавать в другое.

Решение

Для передачи данных между обработчиками событий используется параметр Params, поэтому необходимо:

1. В редакторе типа сущности добавить константу с названием параметра для обозначения того, что поручение без срока:

```
public const string HasIndefiniteDeadline = "HasIndefiniteDeadline";
```

2. В событии **Показ формы** результат проверки (разрешены или нет поручения без срока) записать в параметр с помощью **e.Params**. В зависимости от значения параметра, в карточке поручения появится или скроется флажок **Без срока**:

```
// Если параметра нет в карточке задачи, то проверяется,
  // разрешены ли поручения без срока.
  // Результат проверки записывается в параметр HasIndefiniteDeadline.
  public override void Showing(Sungero.Presentation.FormShowingEventArgs e)
  {
    if
  (!
  e
  .Params
  .Contains
  (RecordManagement.Constants.ActionItemExecutionTask.HasIndefiniteDeadline))

  e
  .Params
  .AddOrUpdate
  (RecordManagement.Constants.ActionItemExecutionTask.HasIndefiniteDeadline,
  Functions.Module.AllowActionItemsWithIndefiniteDeadline() ||
  _obj.HasIndefiniteDeadline == true);
  }
```

3. В событии **Обновление формы** повторно проверку уже не нужно выполнять. Необходимо использовать ранее вычисленное значение параметра **HasIndefiniteDeadline**:

```
// Проверяется существование параметра с именем HasTasksInProcess
  // и проверяется его значение.
  // TryGetValue() – проверка наличия параметра и получение значения.
  public override void Refresh(Sungero.Presentation.FormRefreshEventArgs e)
  {
    var hasIndefiniteDeadline = false;
    var indefiniteDeadlineParamExists =
  e
  .Params
  .Contains
  (RecordManagement.Constants.ActionItemExecutionTask.HasIndefiniteDeadline);

  e
  .Params
  .TryGetValue
  (RecordManagement.Constants.ActionItemExecutionTask.HasIndefiniteDeadline,
  out hasIndefiniteDeadline);
    properties.HasIndefiniteDeadline.IsVisible = indefiniteDeadlineParamExists
      ? hasIndefiniteDeadline
      : Functions.Module.AllowActionItemsWithIndefiniteDeadline() ||
  _obj.HasIndefiniteDeadline == true;
  }
```

В результате при открытии и обновлении карточки задачи на исполнение поручения флажок **Без срока** будет скрыт или показан. При этом на веб-сервер не будут постоянно отправляться лишние запросы на проверку разрешения поручений без срока.

Передача данных между событиями разных сущностей

Ситуация

Необходимо передать данные между событиями разных сущностей. Например, в событии **До сохранения** документа не выводить ошибки, если оно вызвано из задачи на согласование.

Решение

1. Добавьте константу с названием параметра для обозначения того, что сохранение документа выполняется из задачи.

```
public const string DontShowIntegrationErrors = "DontShowIntegrationErrors";
```

2. В обработчиках схемы задачи, в которых происходит сохранение документа, перед выполнением базовой логики добавьте в документ параметр:

```
var document = _obj.DocumentGroup.OfficialDocuments.First();
   ((Sungero.Domain.Shared.IExtendedEntity)
   document).Params[Constants.Module.DontShowIntegrationErrors] = true;
   // Вызов базового обработчика события.
   base.CompleteAssignment27(assignment, e);
```

3. В обработчике события **До сохранения** документа используйте ранее вычисленный параметр:

```
//Contains() – проверка наличия параметра.
   if (!e.Params.Contains(Constants.Module.DontShowIntegrationErrors))
     e.AddError(sendingResult);
```

   После сохранения параметр очищается.

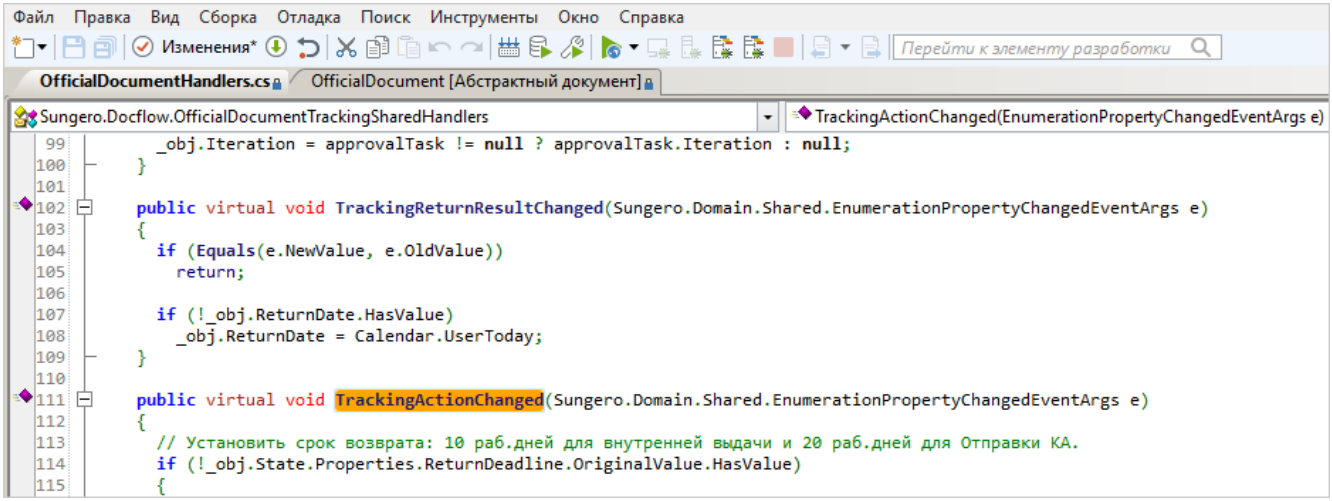
Как определить, где находится код

Ситуация

Предположим, с помощью поиска по коду в среде разработки вы нашли функцию в *базовом решении*. Если хотите ее доработать и использовать в своем решении, нужно определить, где функция используется и какой тип сущности надо перекрыть. Например, нужно определить, где используется функция **TrackingActionChanged()**.

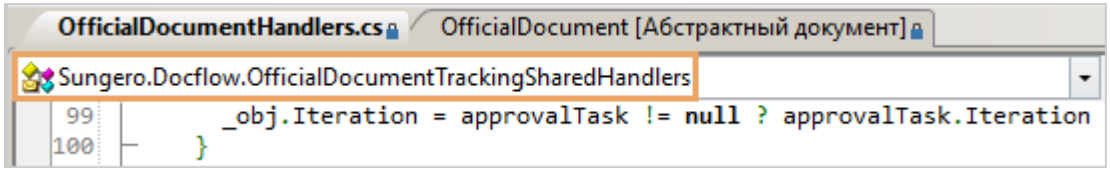
Решение

1. В результатах поиска по коду два раза щелкните по найденному совпадению. В среде разработки откроются две вкладки: редактор типа сущности и обработчик событий. В примере видно, что функция **TrackingActionChanged()** относится к официальному документу (OfficialDocument):



   СОВЕТ. Для навигации и поиска по коду удобно использовать горячие клавиши. Чтобы быстро перейти к нужному элементу разработки, используйте навигацию.

2. По названию класса, в котором находится найденная функция, определите код, к которому она относится:



   По названию класса можно определить:

   - тип сущности и модуль;
   - элемент разработки. Зависит от слова, которым заканчивается название класса:

     Functions – код функции;

     Handlers – код обработчика события типа сущности или свойства;

     Actions – код обработчика действия;
   - вид кода. Если в пути есть слова **Client**, **Server** или **Shared**, значит это клиентский, серверный или разделяемый код соответственно.

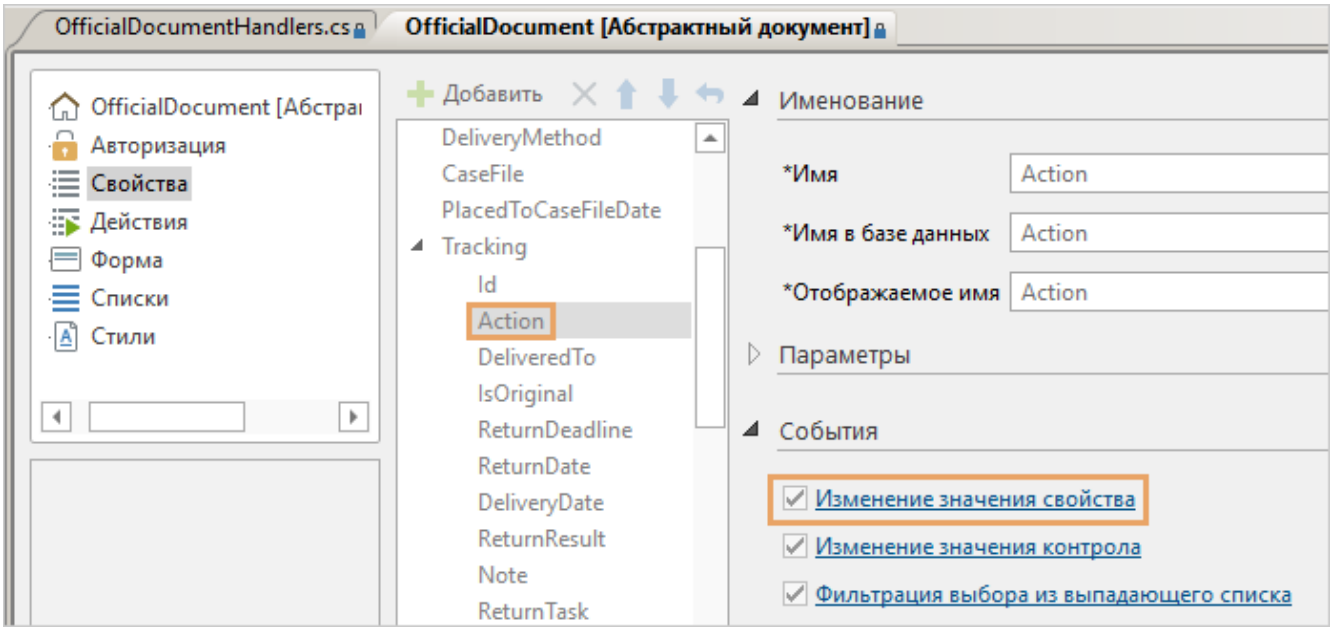
   В данном случае функция относится к классу **Sungero.Docflow.OfficialDocumentTrackingSharedHandlers**:

   - **Docflow** – тип сущности находится в модуле «Настройки документооборота»;
   - **Handlers** – код относится к обработчику событий;
   - **Shared** – используется разделяемый код.

   Дополнительно по слову **Tracking** можно понять, что функция относится к закладке «Выдача» в карточке документа.

3. Чтобы понять, в каком обработчике события находится функция, используйте ее название. Названия событий автоматически генерируются по формату. В описании событий типов сущностей приведены форматы названий автогенерируемых функций.

   По слову **Changed** можно определить, что функция названа по формату **<Имя свойства>Changed**, причем **<Имя свойства>** в данном случае содержит имя свойства-коллекции и его дочернего свойства. В результате функция относится к событию Изменение значения свойства, которое задано в свойстве **Action** свойства-коллекции **Tracking**:



Папки и списки

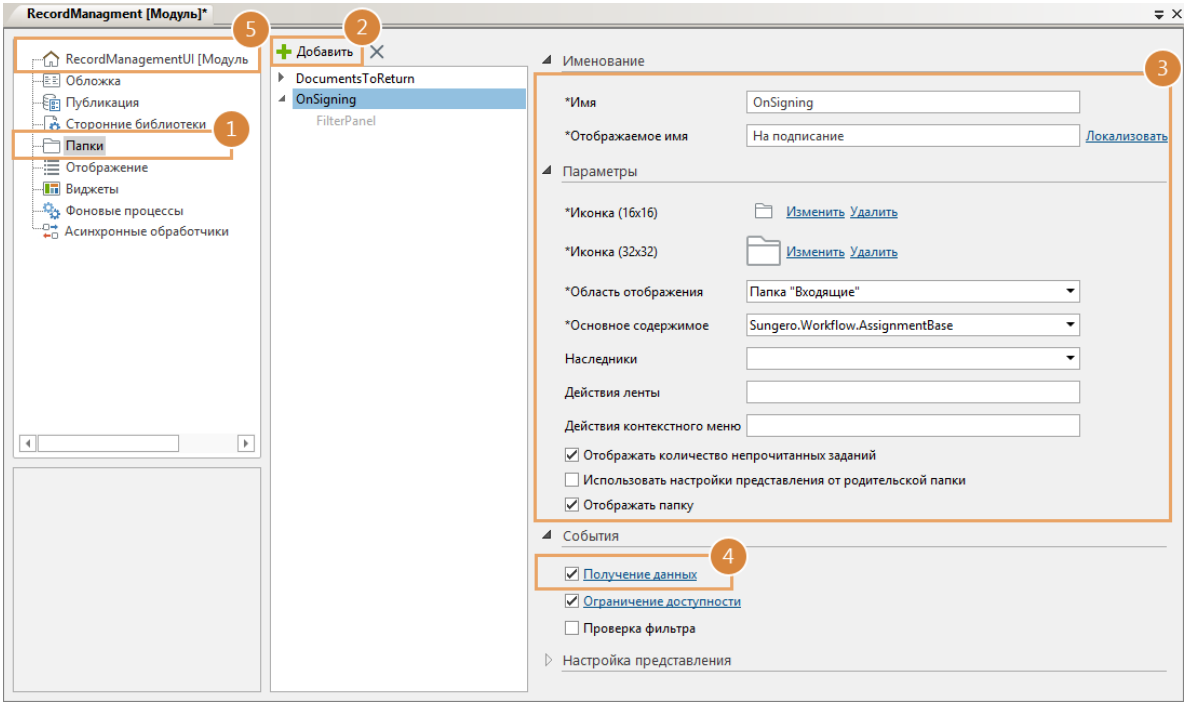
В разделе содержатся примеры:

- Как создать вычисляемую папку, которая доступна только руководителям?
- Как добавить условия фильтрации для списка «Входящие документы»?
- Как добавить условия фильтрации для папки «Документы к возврату»?
- Как настроить валидацию панели фильтрации для входящих документов?
- Как настроить список для двух типов документов с одинаковыми свойствами?
- Как ограничить видимость сотрудников в списках?

Как создать вычисляемую папку, которая доступна только руководителям

Предположим, необходимо создать папку «На подписание», которая доступна только руководителям подразделений. Папка должна содержать задания на подписание документов, полученные пользователем, и находиться в папке «Входящие». Для этого:

1. В редакторе модуля перейдите в узел «Папки».



2. Нажмите на кнопку **Добавить**.
3. В списке выделите добавленный элемент. Заполните поля:

   ***Имя** – OnSigning.

   ***Отображаемое имя** – На подписание.

   ***Область отображения** – Папка «Входящие».

   ***Основное содержимое** – Sungero.Workflow.AssignmentBase.

   Добавьте иконки и установите флажок **Отображать количество непрочитанных заданий**. Проверьте, что установлен флажок **Отображать папку**. Подробнее описание параметров см. в разделе «Папки».

4. Чтобы в папку попадали только задания на подписание, добавьте код в обработчик события **Получение данных**. Для этого установите флажок рядом с событием и перейдите по ссылке. В открывшемся редакторе напишите код для вычисления содержимого:

```
/// <summary>
    /// Выбор из списка всех заданий пользователя только заданий на подписание.
    /// </summary>
    /// <param name="query">Список заданий для фильтрации.</param>
    /// <returns>Отфильтрованный список заданий.</returns>
    public virtual IQueryable<Sungero.Workflow.IAssignmentBase>
    OnSigningDataQuery(IQueryable<Sungero.Workflow.IAssignmentBase> query)
    {
```

```
    // Фильтр по типу задания.
    return query.Where(x => Docflow.ApprovalSigningAssignments.Is(x) &&
Docflow.ApprovalSigningAssignments.As(x).IsConfirmSigning != true);
}
```

5. Чтобы у руководителей подразделений папка «На подписание» по умолчанию отображалась в списке папок в проводнике Directum RX, в узле «Модуль» добавьте серверную функцию:

```
/// <summary>
/// Проверка вхождения текущего пользователя в произвольную роль.
/// </summary>
/// <param name="roleSid">Sid роли.</param>
/// <returns> Функция возвращает значение True,
/// если пользователь входит в роль, иначе возвращает False.</returns>
public static bool IncludedInRole(Guid roleSid)
{
  var role = Roles.GetAll(r => r.Sid == roleSid).SingleOrDefault();
  return role != null && Users.Current.IncludedIn(role);
}
```

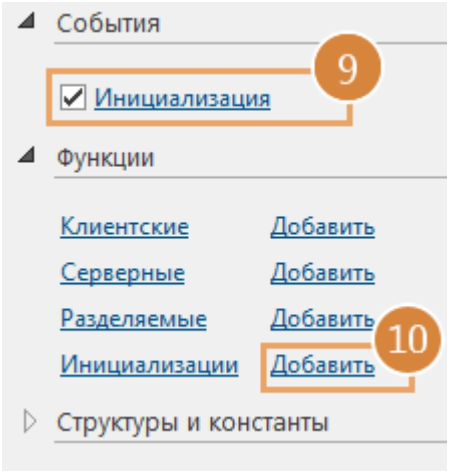
6. В узле «Модуль» укажите зависимость от модуля:
   - Sungero.Workflow, так как в узле «Папки» в параметре ***Основное содержимое** указано значение Sungero.Workflow.AssignmentBase;
   - Sungero.Docflow, так как в добавленном программном коде использована константа и типы сущностей из данного модуля.

   Если зависимости не указаны, то при публикации могут возникнуть ошибки.
7. В узле «Папки» добавьте обработчик события **Ограничение доступности**. В коде обработчика вызовите серверную функцию:

```
public virtual bool IsOnSigningVisible()
{
  return
Functions
.Module
.IncludedInRole
(Sungero.Docflow.PublicConstants.Module.RoleGuid.BusinessUnitHeadsRole);
}
```

8. При необходимости настройте внешний вид папки по умолчанию и панель фильтрации.

9. Чтобы папка появилась в проводнике системы, при инициализации модуля выдайте на нее права доступа. Для этого в узле «Модуль», в группе «События» установите флажок рядом с событием **Инициализация**.



10. В группе «Функции» перейдите по ссылке **Добавить** рядом с функциями инициализации. В редакторе программного кода измените имя **Function1()** на **GrantRightsOnFolder()** и напишите код для выдачи прав на вычисляемую папку. Добавьте также вызов созданной функции в обработчике события **Initializing()**:

```
public partial class ModuleInitializer
  {
    /// <summary>
    /// Обработчик события инициализации.
    /// </summary>
    public override void
  Initializing(Sungero.Domain.ModuleInitializingEventArgs e)
    {
    /// Вызов функции инициализации.
      GrantRightsOnFolder();
    }
    /// <summary>
    /// Функция инициализации для выдачи прав на вычисляемую папку.
    /// </summary>
    public static void GrantRightsOnFolder()
    {
      var allUsers = Roles.AllUsers;
      Sungero
  .RecordManagement.SpecialFolders.OnSigning.AccessRights.Grant(allUsers,
  DefaultAccessRightsTypes.Read);
      Sungero.RecordManagement.SpecialFolders.OnSigning.AccessRights.Save();
      InitializationLogger.Debug("Выданы права на вычисляемую папку 'На
  подписание'");
    }
  }
```

ПРИМЕЧАНИЕ. Для обращения к вычисляемым папкам в коде используется формат: **<Код компании>.<Имя модуля>.SpecialFolders.<Имя вычисляемой папки>**. Например, Sungero.RecordManagement.SpecialFolders.OnSigning. При необходимости подставьте в код примера свои значения.

В результате после *публикации* решения и инициализации в папке «Входящие» у руководителей подразделений будет отображаться папка потока «На подписание». В папке будут задания на подписание документов, полученные пользователем. Другие сотрудники могут добавить папку в дерево с помощью контекстного меню.

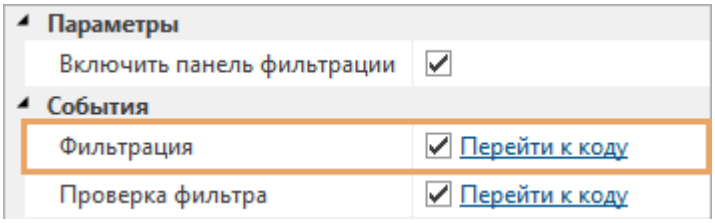
СОВЕТ. Для настройки порядка расположения подпапок в папках «Входящие» и «Исходящие» используйте узел «Папки Мой Директум» в редакторе решения.

Как добавить условия фильтрации для списка «Входящие документы»

Создан список «Входящие документы» и для него добавлена панель фильтрации. Необходимо добавить обработчик события **Фильтрация** и настроить фильтрацию записей в списке по критериям журнал регистрации, статус, вид документа и интервал времени, в котором документ был зарегистрирован или создан.

Чтобы добавить логику работы панели фильтрации:

1. В редакторе типа документа «Входящее письмо» (IncomingLetter) последовательно раскройте узлы «Списки», «Collection» и выделите узел «FilterPanel».
2. На панели свойств установите флажок **Фильтрация** и перейдите по ссылке. Откроется редактор программного кода.



3. Напишите код для фильтрации сущностей в списке. Обработчик события выполняется на веб-сервере.

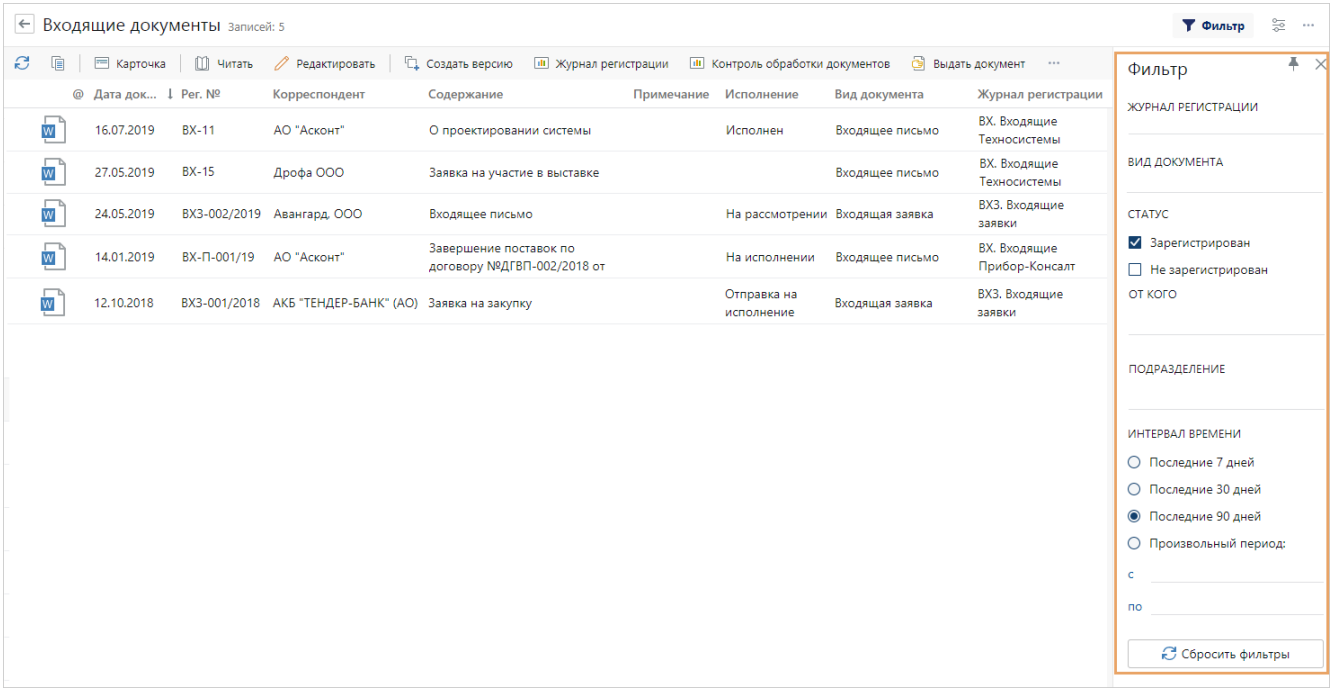
   ВАЖНО. В начало обработчика с помощью аргумента _filter добавьте проверку, что панель фильтрации включена.

```
/// <summary>
  /// Фильтрация списка входящих документов.
  /// </summary>
  public override IQueryable<T> Filtering(IQueryable<T> query,
  Sungero.Domain.FilteringEventArgs e)
  {
    // Проверка того, что панель фильтрации включена.
    if (_filter == null)
      return query;
    // Контрол "Ссылка на сущность". Фильтр по журналу регистрации.
    // Значение выбирается из справочника "Журналы регистрации".
    if (_filter.DocumentRegister != null)
      query = query.Where(d => d.DocumentRegister == _filter.DocumentRegister);
    // Контрол "Набор флажков". Фильтр по статусу документа.
    // Возможные значения: Зарегистрирован, Не зарегистрирован.
    if (_filter.Registered || _filter.NotRegistered)
      query = query
        .Where(d => (_filter.Registered && d.RegistrationState ==
  OfficialDocument.RegistrationState.Registered) ||
```

```
            (_filter.NotRegistered && d.RegistrationState ==
OfficialDocument.RegistrationState.NotRegistered));
  // Контролы "Набор флажков" и "Период дат".
  // Фильтр по интервалу времени в котором документы были зарегистрированы
  // или созданы.
  var periodBegin =
Functions.Module.GetPeriodBeginFromFilter(_filter.LastWeek,
_filter.LastMonth, _filter.Last90Days, _filter.DateRangeFrom);
  var periodEnd = Functions.Module.GetPeriodEndFromFilter(_filter.LastWeek,
_filter.LastMonth, _filter.Last90Days, _filter.DateRangeTo);
  query = query.Where(j => (j.RegistrationDate ?? j.Created) >= periodBegin
&& (j.RegistrationDate ?? j.Created) <= periodEnd);
  return query;
}
```

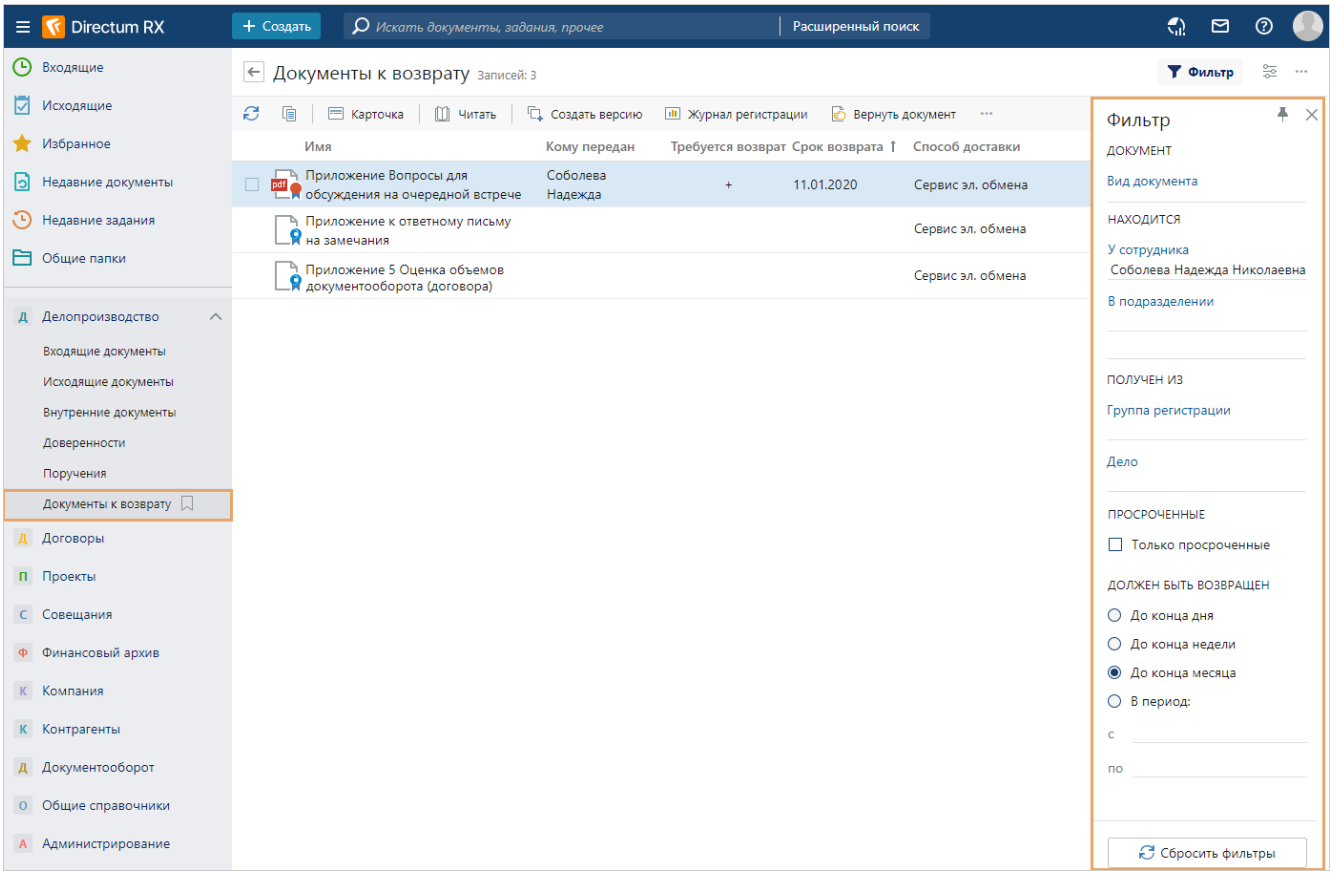
4. Опубликуйте и проверьте разработку.

В результате в списке будут отображаться входящие документы организации с учетом фильтров:



Как добавить условия фильтрации для папки «Документы к возврату»

Создана вычисляемая папка «Документы к возврату» и для нее добавлена панель фильтрации:



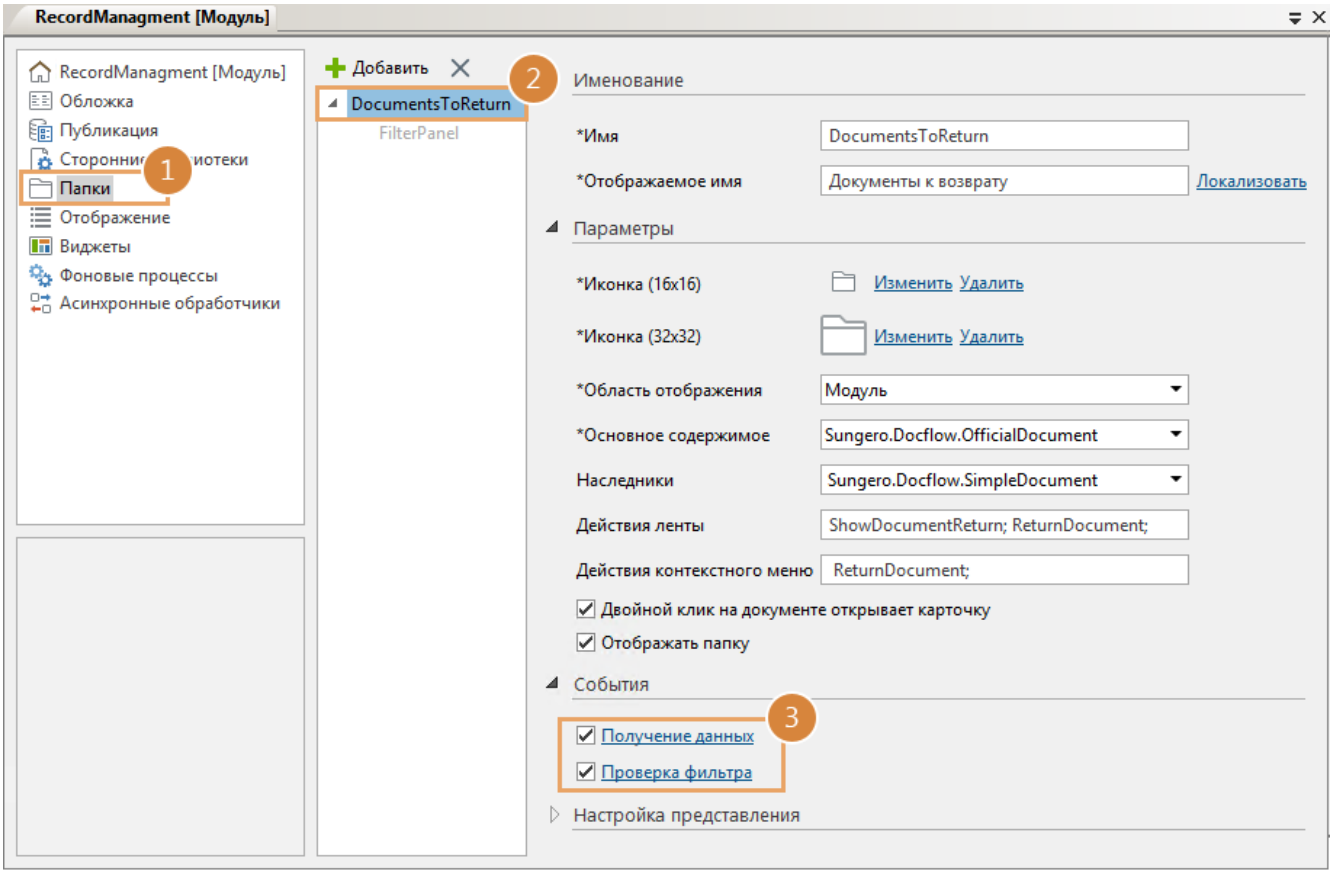
Необходимо настроить фильтрацию записей в папке. Для этого добавьте обработчики событий:

- **Получение данных**. В событии напишите логику вычисления содержимого папки: в ней должны отображаться регистрируемые и нумеруемые документы, которые подлежат возврату. В этом же событии задайте условия для фильтрации содержимого папки по критериям на панели фильтрации;
- **Проверка фильтра**. В событии напишите код для *валидации* панели фильтрации. В системе должно появляться сообщение, если при фильтрации списка пользователь выбрал произвольный период возврата документов и не заполнил критерии «с», «по», «В подразделении» и «У сотрудника». При таком сочетании критериев запрос на веб-сервере будет выполняться долго и вернет большое количество записей. Т.е. пользователю все равно придется уточнять критерии фильтрации.

  ПРИМЕЧАНИЕ. Проверка фильтра работает только в веб-клиенте.

Чтобы добавить логику работы панели фильтрации:

1. В редакторе модуля перейдите в узел «Папки» и в списке элементов выделите вычисляемую папку «Документы к возврату»:



2. Добавьте логику вычисления содержимого папки и условия фильтрации. Для этого в группе «События» установите флажок **Получение данных** и перейдите по ссылке. В открывшемся редакторе напишите код обработчика.

   ВАЖНО. В начало обработчика добавьте проверку, что панель фильтрации включена. Для этого используйте аргумент _filter.

```
/// <summary>
  /// Получить выданные документы.
  /// </summary>
  /// <param name="_filter">Фильтр.</param>
  /// <returns>Список выданных документов.</returns>
  public virtual IQueryable<Sungero.Docflow.IOfficialDocument>
  DocumentsToReturnDataQuery(IQueryable<Sungero.Docflow.IOfficialDocument>
  query)
  {
    // Проверка того, что панель фильтрации включена.
    if (_filter == null)
      return documents;

    // Получение списка сущностей, которые должны отображаться
    // в вычисляемой папке (регистрируемые и нумеруемые документы,
    // которые подлежат возврату).

    var today = Calendar.UserToday;
    var documents = query.Where(l => l.IsReturnRequired == true ||
  l.IsHeldByCounterParty == true);
```

```
  documents = documents.Where(d => !Docflow.ContractualDocumentBases.Is(d) &&
 !Docflow.AccountingDocumentBases.Is(d));

  // Фильтр по статусу.
  if (_filter.Overdue)
  {
    documents = documents.Where(l => l.Tracking.Any(d => d.ReturnDate >
d.ReturnDeadline || (!d.ReturnDate.HasValue && d.ReturnDeadline < today)));
  }

  // Фильтр по виду документа.
  if (_filter.DocumentKind != null)
  {
    documents = documents.Where(l => Equals(l.DocumentKind,
_filter.DocumentKind));
  }

  // Фильтр по сотруднику.
  if (_filter.Employee != null)
  {
    documents = documents.Where(l => l.Tracking.Any(d =>
Equals(d.DeliveredTo, _filter.Employee)));
  }

  // Фильтр по подразделению.
  if (_filter.Department != null)
  {
    documents = documents.Where(l => l.Tracking.Any(d =>
Equals(d.DeliveredTo.Department, _filter.Department)));
  }

  // Фильтр по группе регистрации.
  if (_filter.RegistrationGroup != null)
  {
    documents = documents.Where(l => l.DocumentRegister != null &&
Equals(l.DocumentRegister.RegistrationGroup, _filter.RegistrationGroup));
  }

  // Фильтр по делу.
  if (_filter.Filelist != null)
  {
    documents = documents.Where(l => Equals(l.CaseFile, _filter.Filelist));
  }

  // Исключить строки с Результатом возврата: "Возвращен".
  var returned = Docflow.OfficialDocumentTracking.ReturnResult.Returned;

  // Фильтр по сроку возврата: до конца дня.
  if (_filter.EndDay)
    documents = documents.Where(l => l.Tracking.Any(p =>
!Equals(p.ReturnResult, returned) && p.ReturnDeadline < today.AddDays(1)));

  // Фильтр по сроку возврата: до конца недели.
  if (_filter.EndWeek)
    documents = documents.Where(l => l.Tracking.Any(p =>
!Equals(p.ReturnResult, returned) && p.ReturnDeadline <= today.EndOfWeek()));
```

```
    // Фильтр по сроку возврата: до конца месяца.
    if (_filter.EndMonth)
      documents = documents.Where(l => l.Tracking.Any(p =>
  !Equals(p.ReturnResult, returned) && p.ReturnDeadline <=
  today.EndOfMonth()));

    // Фильтр по сроку возврата: в период с, по.
    if (_filter.Manual)
    {
      var dateFrom = _filter.ReturnPeriodDataRangeFrom;
      var dateTo = _filter.ReturnPeriodDataRangeTo;

      if (dateFrom.HasValue && !dateTo.HasValue)
        documents = documents.Where(l => l.Tracking.Any(p =>
  !Equals(p.ReturnResult, returned) && p.ReturnDeadline >= dateFrom.Value));

      if (dateTo.HasValue && !dateFrom.HasValue)
        documents = documents.Where(l => l.Tracking.Any(p =>
  !Equals(p.ReturnResult, returned) && p.ReturnDeadline <= dateTo.Value));

      if (dateFrom.HasValue && dateTo.HasValue)
        documents = documents.Where(l => l.Tracking.Any(p =>
  !Equals(p.ReturnResult, returned) && (p.ReturnDeadline >= dateFrom.Value &&
  p.ReturnDeadline <= dateTo.Value)));
    }

    return documents;
  }
```

3. Добавьте валидацию панели фильтрации по критериям «с», «по», «В подразделении» и «У сотрудника». Для этого в группе «События» установите флажок **Проверка фильтра**, перейдите по ссылке и напишите код:

```
partial class DocumentsToReturnFolderHandlers
  {
    /// <summary>
    /// Проверка фильтра для списка выданных документов.
    /// </summary>
    public override void
  DocumentsToReturnValidateFilterPanel
  (Sungero.Domain.Client.ValidateFilterPanelEventArgs e)
    {
      // Не выполнять фильтрацию, если установлен
      // произвольный период возврата документов
      // и не заполнены критерии "с", "по" "В подразделении" и "У сотрудника".
      if (_filter.Manual &&
          _filter.Department== null &&
          _filter.Employee == null &&
          _filter.Info.ReturnPeriodDataRangeFrom == null &&
          _filter.Info.ReturnPeriodDataRangeTo == null)
      e.AddError("Заполните параметры, чтобы сократить число результатов
  поиска", _filter.Info.Department, _filter.Info.Employee,
  _filter.Info.ReturnPeriodDataRangeFrom,
  _filter.Info.ReturnPeriodDataRangeTo);
    }
  }
```

4. Опубликуйте и проверьте разработку.

В результате в папке будут отображаться регистрируемые и нумеруемые документы, которые подлежат возврату.

Как настроить валидацию панели фильтрации для входящих документов

Ситуация

Предположим, у компании больше 10 миллионов входящих документов. Чтобы система Directum RX быстро и корректно работала со списком входящих документов, в среде разработки нужно настроить валидацию панели фильтрации для списка.

С помощью валидации система будет проверять критерии поиска, которые пользователь ввел при фильтрации списка. При необходимости система попросит пользователя уточнить критерии. Благодаря этому на веб-сервер не будут отправляться «тяжелые запросы» при фильтрации списка, а значит система будет работать быстро.

В примере для панели фильтрации входящих документов нужно добавить валидацию, если пользователь выбрал произвольный период возврата документов. Это «тяжелый запрос», поэтому нужно попросить пользователя уточнить критерии, то есть заполнить поля «От кого» или «Журнал регистрации».

ВАЖНО. Валидация панели фильтрации работает только в веб-клиенте.

Решение

Шаг 1. Создайте папку, которая заменит список входящих документов.

Шаг 2. Настройте панель фильтрации папки и *валидацию* для нее.

Шаг 3. Замените список входящих документов на созданную папку с настроенной валидацией панели фильтрации.

Шаг 4. Настройте валидацию для панели фильтрации, которая отображается в окне результатов поиска входящих писем.

Шаг 5. Доработайте тип сущности «Исходящее письмо», чтобы при заполнении поля **В ответ на** вместо списка входящих документов открывалась папка с настроенной валидацией.

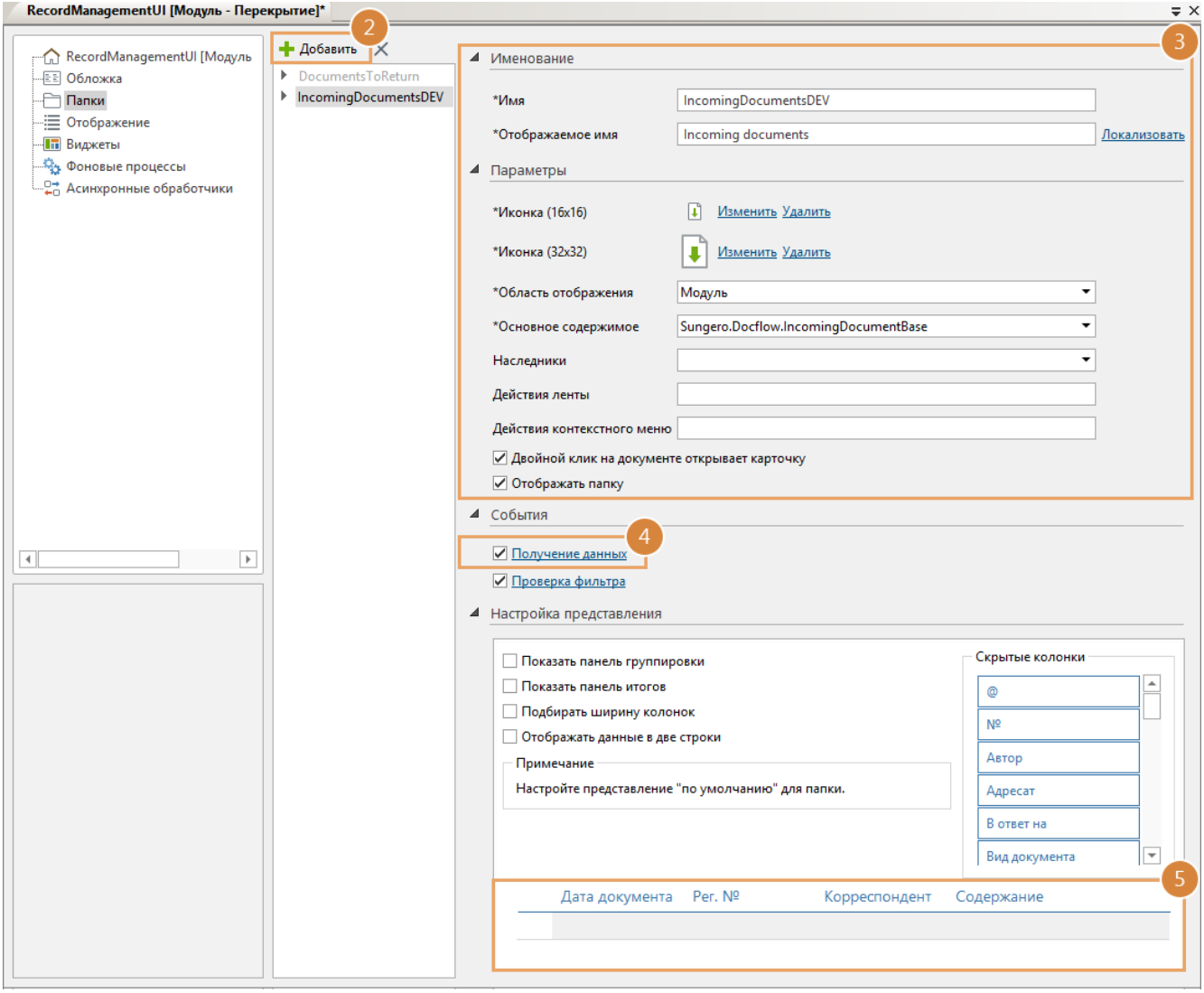
Шаг 1. Создание папки – аналог списка входящих документов

Список входящих документов настроен в базовом типе сущности «Входящий документ» (IncomingDocumentBase). *Базовые типы* нельзя перекрывать. Поэтому, чтобы настроить валидацию панели фильтрации для входящих документов, нужно создать папку, которая заменит список входящих документов.

Для этого:

1. В вашем решении перекройте модуль Sungero.RecordManagementUI и добавьте зависимость от него. Если зависимости не указаны, при публикации могут возникнуть ошибки.

2. В редакторе перекрытого модуля создайте вычисляемую папку – аналог списка входящих документов. Для этого перейдите в узел «Папки» и нажмите на кнопку **Добавить**.



3. Заполните именование и параметры папки, как показано на скриншоте.

   ПРИМЕЧАНИЕ. К имени папки автоматически добавился постфикс с кодом вашей компании, так как при установке среды разработки был установлен флажок **Уникальные имена при перекрытии**. В нашем примере код компании – DEV.

4. Добавьте логику вычисления содержимого папки и условия фильтрации. Для этого откройте редактор базового типа сущности «Входящий документ» (IncomingDocumentBase) и скопируйте код события **Фильтрация**. В редакторе созданной вычисляемой папки в группе «События» установите флажок **Получение данных** и перейдите по ссылке. В открывшемся редакторе вставьте код:

```
public virtual IQueryable<Sungero.Docflow.IIncomingDocumentBase>
  IncomingDocumentsDataQuery(IQueryable<Sungero.Docflow.IIncomingDocumentBase>
   query)
  {
    if (_filter == null)
      return query;

    // Фильтр по журналу регистрации.
    if (_filter.DocumentRegister != null)
```

```
    query = query.Where(d => Equals(d.DocumentRegister,
_filter.DocumentRegister));

  // Фильтр по виду документа.
  if (_filter.DocumentKind != null)
    query = query.Where(d => Equals(d.DocumentKind, _filter.DocumentKind));

  // Фильтр по статусу.
  // Если все галочки включены, то нет смысла добавлять фильтр.
  if (_filter.Registered != _filter.NotRegistered)
    query = query
      .Where(d => _filter.Registered && d.RegistrationState ==
Sungero.Docflow.OfficialDocument.RegistrationState.Registered ||
             _filter.NotRegistered && d.RegistrationState ==
Sungero.Docflow.OfficialDocument.RegistrationState.NotRegistered);

  // Фильтр по контрагенту.
  if (_filter.Counterparty != null)
    query = query.Where(d => Equals(d.Correspondent, _filter.Counterparty));

  // Фильтр по подразделению.
  if (_filter.Department != null)
    query = query.Where(c => Equals(c.Department, _filter.Department));

  // Фильтр по интервалу времени.
  var periodBegin = Calendar.UserToday.AddDays(-7);
  var periodEnd = Calendar.UserToday.EndOfDay();

  if (_filter.LastWeek)
    periodBegin = Calendar.UserToday.AddDays(-7);

  if (_filter.LastMonth)
    periodBegin = Calendar.UserToday.AddDays(-30);

  if (_filter.Last90Days)
    periodBegin = Calendar.UserToday.AddDays(-90);

  if (_filter.ManualPeriod)
  {
    periodBegin = _filter.DateRangeFrom ?? Calendar.SqlMinValue;
    periodEnd = _filter.DateRangeTo ?? Calendar.SqlMaxValue;
  }

  var serverPeriodBegin = Equals(Calendar.SqlMinValue, periodBegin) ?
periodBegin :
Sungero
.Docflow
.PublicFunctions.Module.Remote.GetTenantDateTimeFromUserDay(periodBegin);
  var serverPeriodEnd = Equals(Calendar.SqlMaxValue, periodEnd) ? periodEnd :
 periodEnd.EndOfDay().FromUserTime();
  var clientPeriodEnd = !Equals(Calendar.SqlMaxValue, periodEnd) ?
periodEnd.AddDays(1) : Calendar.SqlMaxValue;
  query = query.Where(j => (j.DocumentDate.Between(serverPeriodBegin,
serverPeriodEnd) ||
                            j.DocumentDate == periodBegin) && j.DocumentDate
!= clientPeriodEnd);
```

```
    return query;
}
```

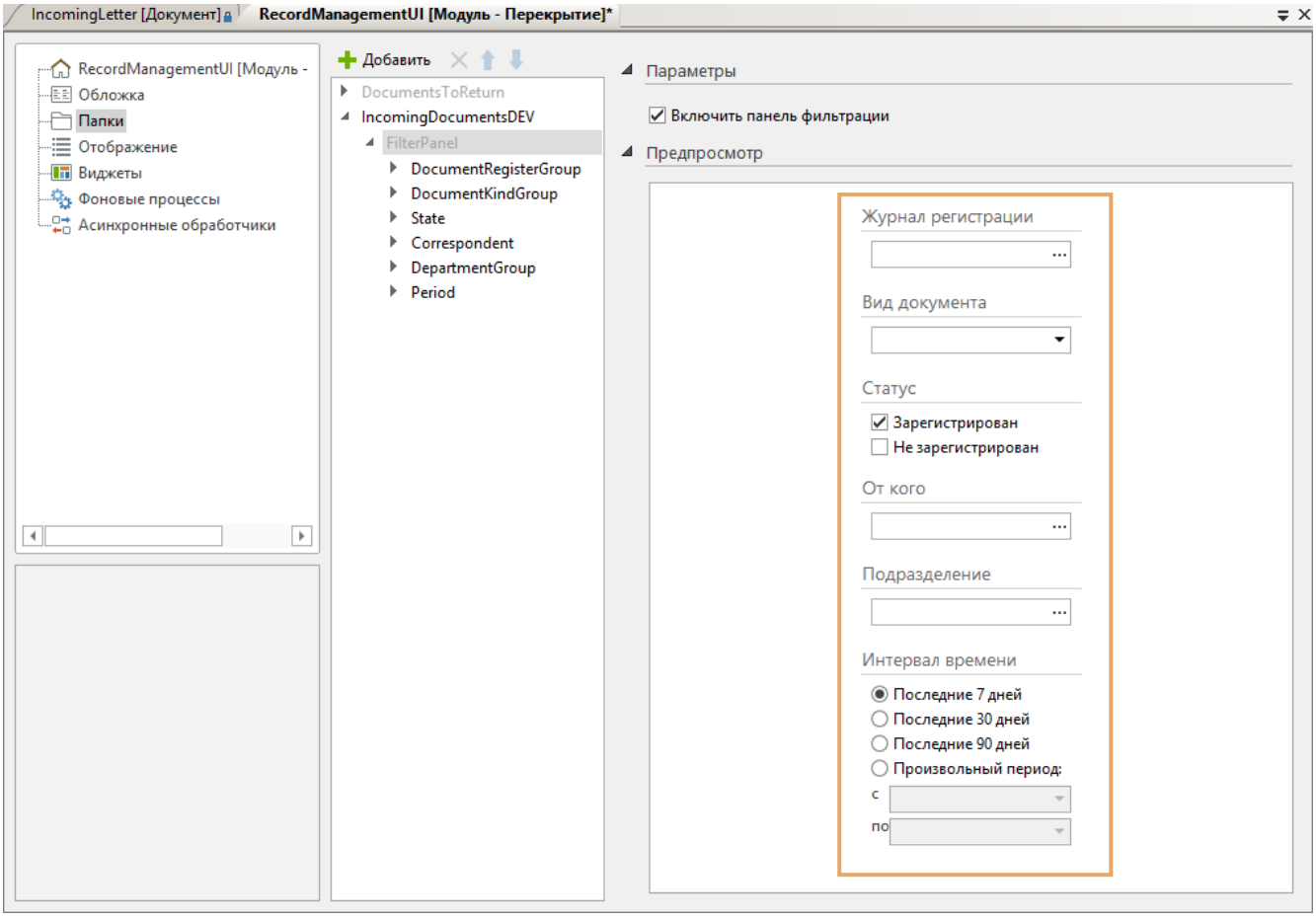
5. Настройте для папки представление по умолчанию.

Таким образом вы создали вычисляемую папку для входящих документов.

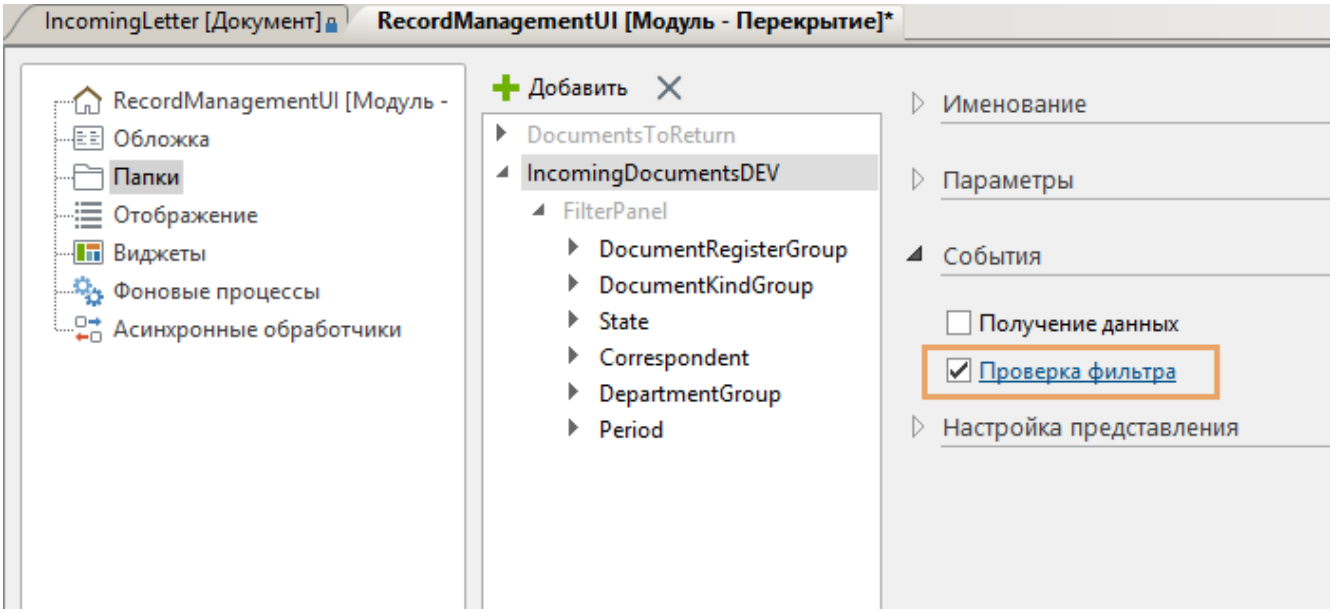
Шаг 2. Настройка панели фильтрации и валидации для папки

Чтобы в созданной папке настроить панель фильтрации, как в списке входящих документов, и добавить валидацию:

1. Откройте редактор типа документа «Входящий документ» (IncomingDocumentBase). Для этого в поле навигации по элементам разработки введите имя «Входящий документ» или IncomingDocumentBase и выберите тип документа в выпадающем списке. В результате откроется редактор типа документа.
2. Скопируйте панель фильтрации.
3. В созданной папке (IncomingDocumentsDEV) перейдите в узел «FilterPanel» и вставьте панель фильтрации.



4. Вернитесь в узел **IncomingDocumentsDEV** и добавьте валидацию панели фильтрации: если выбран произвольный период возврата документов, критерии «От кого» или «Журнал регистрации» должны быть обязательными для заполнения. Для этого в группе «События» установите флажок **Проверка фильтра**:



5. Перейдите по ссылке и напишите код:

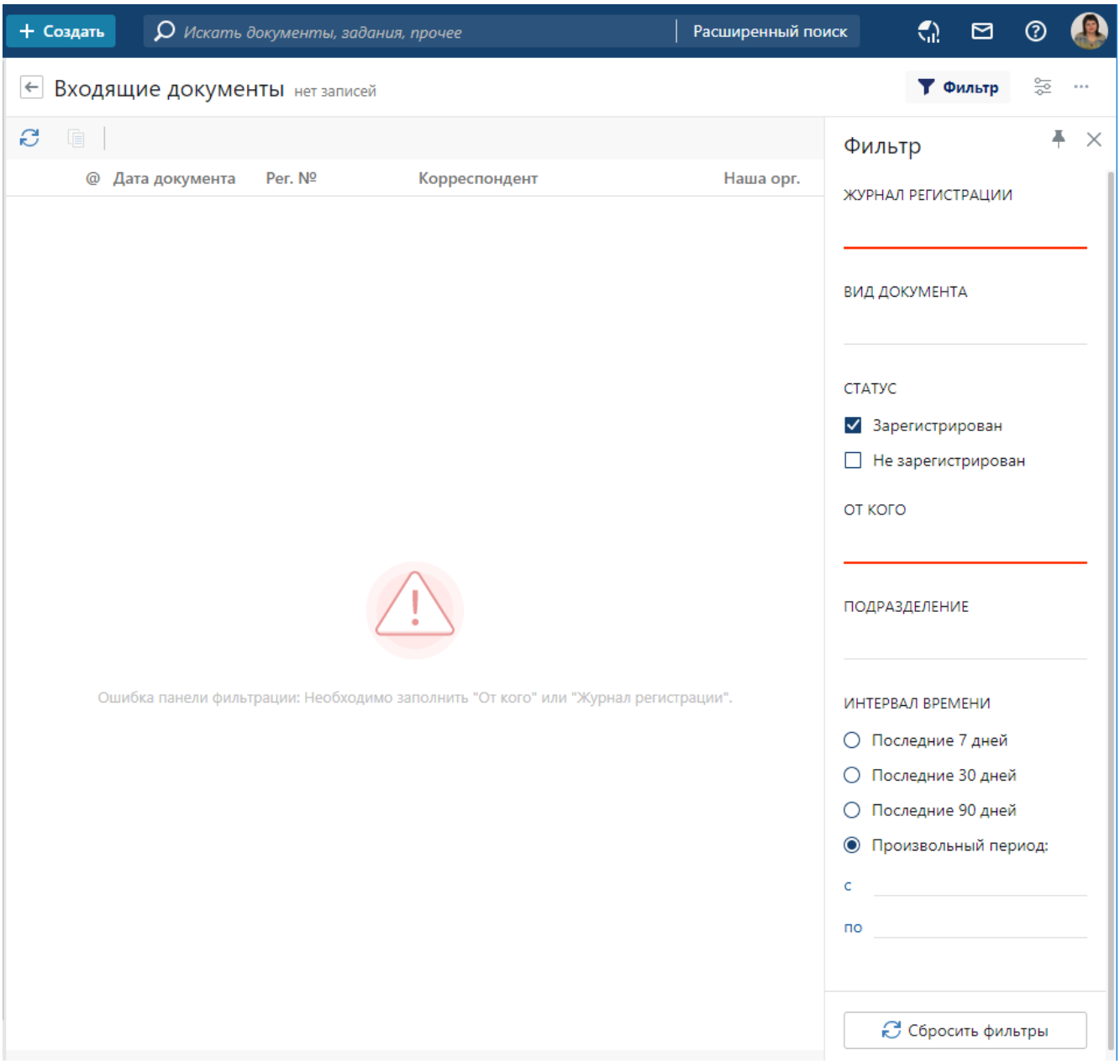
```
public                                virtual                                void
   IncomingDocumentsValidateFilterPanel
   (Sungero.Domain.Client.ValidateFilterPanelEventArgs e)
   {
     // Не выполнять фильтрацию, если установлен
     // произвольный период возврата документов
     // и не заполнены критерии "От кого" и "Журнал регистрации".
     if (_filter.ManualPeriod && (_filter.DateRangeFrom == null ||
   _filter.DateRangeTo == null) && _filter.Counterparty == null &&
   _filter.DocumentRegister == null)
       e.AddError(Resources.FilterPanelValidationError,
   _filter.Info.Counterparty, _filter.Info.DocumentRegister);
   }
```

Resources.FilterPanelValidationError – *строка локализации*, которую пользователи увидят в системе, если выберут произвольный период возврата документов и не заполнят критерии «От кого» или «Журнал регистрации». Как добавить строку локализации прямо при написании кода, см. в примере.

Параметры строки:

- имя ресурса: FilterPanelValidationError;
- en (default): Fill in the «From» or «Document Register» field;
- ru: Необходимо заполнить «От кого» или «Журнал регистрации».

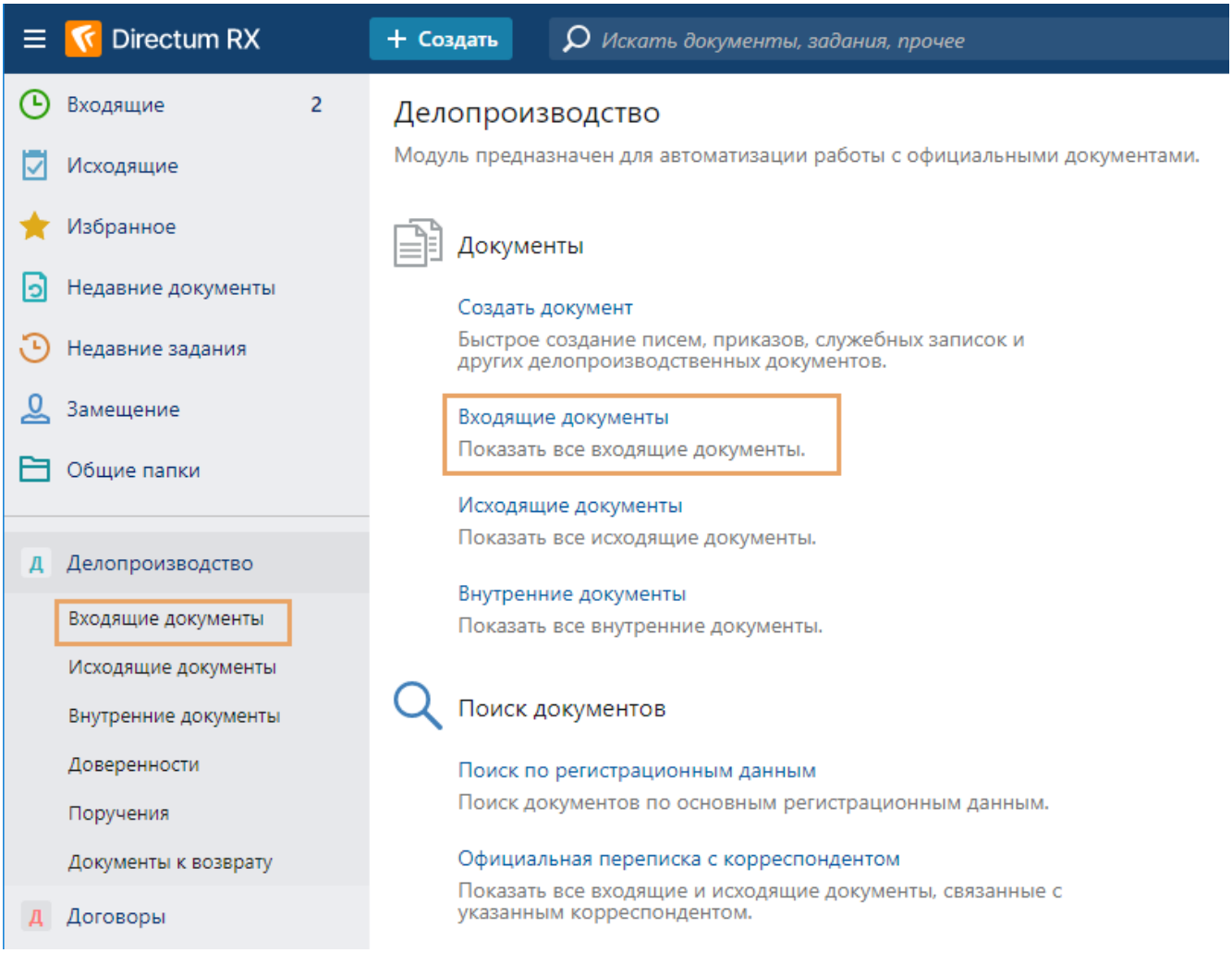
Таким образом вы настроили панель фильтрации папки и валидацию для панели фильтрации. Если пользователь выберет произвольный период возврата документов и не заполнит критерии «От кого» или «Журнал регистрации» в системе появится сообщение. Незаполненные критерии выделятся красным цветом:



Шаг 3. Замена стандартного списка входящих документов на созданную папку

ВАЖНО. Обложки рекомендуется настраивать в веб-клиенте. Пример см. в руководстве администратора, в разделе «Пример настройки модуля и стартовой страницы». Далее показана доработка обложки через среду разработки: такая возможность оставлена для совместимости.

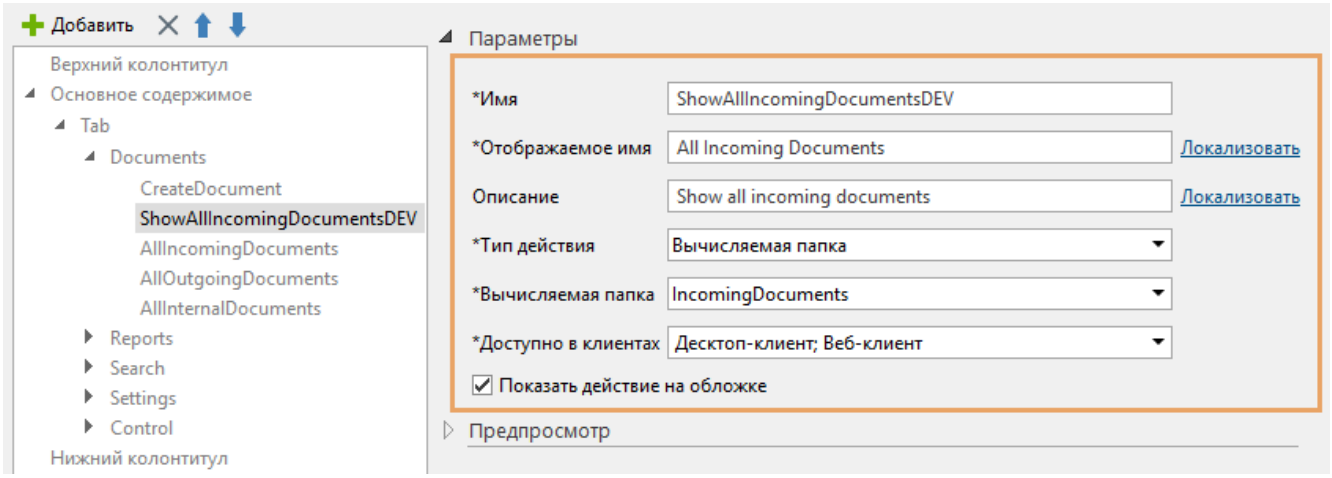
Список входящих документов открывается при переходе по ссылке на обложке модуля «Делопроизводство» и при нажатии на список в дереве элементов модуля:



Чтобы в этих местах вместо списка открывалась созданная папка:

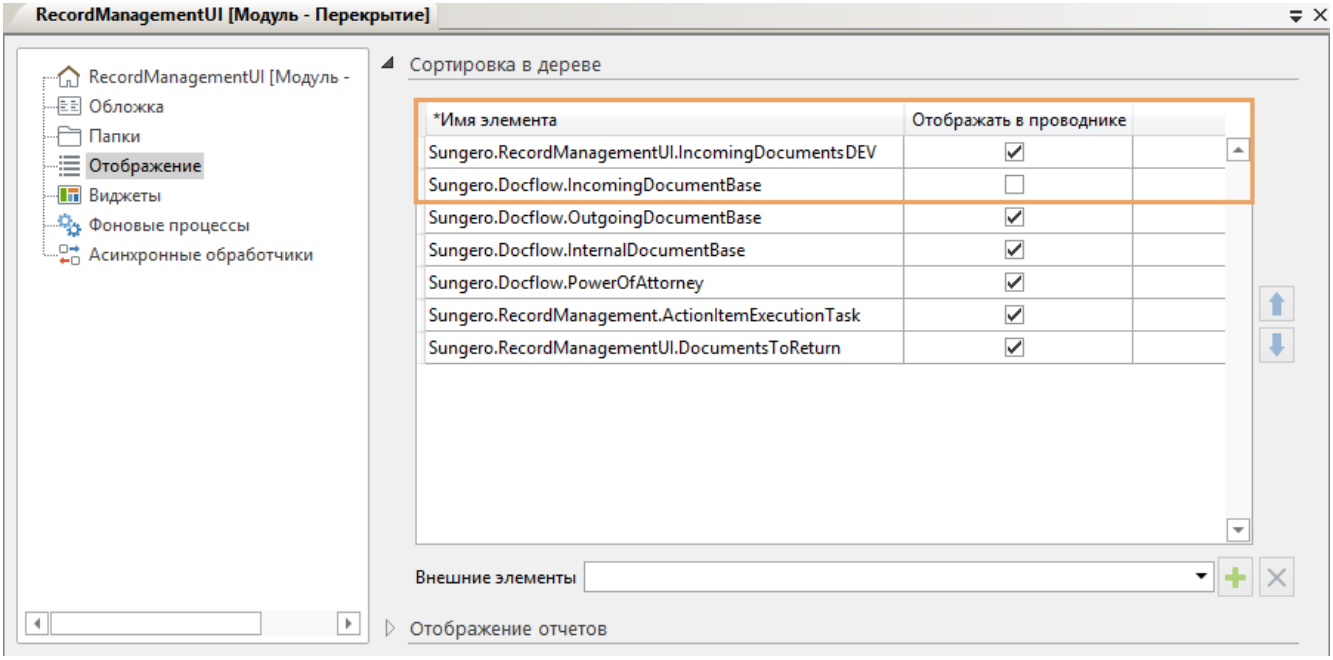
1. В редакторе перекрытого модуля RecordManagementUI перейдите в узел «Обложка».
2. В группе **Documents** выделите действие **AllIncomingDocuments** (Входящие документы) и снимите флажок **Показать действие на обложке**. Таким образом на обложке пропадет ссылка, с помощью которой пользователи открывают стандартный список входящих документов.
3. Добавьте на обложку действие для открытия созданной папки. Для этого выделите группу **Documents** и нажмите на кнопку **Добавить**.

4. Заполните параметры, как показано на скриншоте, и с помощью кнопок ⬆ и ⬇ разместите действие перед действием **AllIncomingDocuments**:



В результате на обложке вместо ссылки для открытия стандартного списка входящих документов появится ссылка для открытия созданной папки с настроенной валидацией панели фильтрации.

5. Перейдите в узел «Отображение», снимите флажок напротив элемента IncomingDocumentBase и проверьте, что установлен флажок напротив элемента IncomingDocumentsDEV.



Таким образом в проводнике системы в дереве элементов модуля вместо списка входящих документов будет отображаться созданная папка с настроенной валидацией.

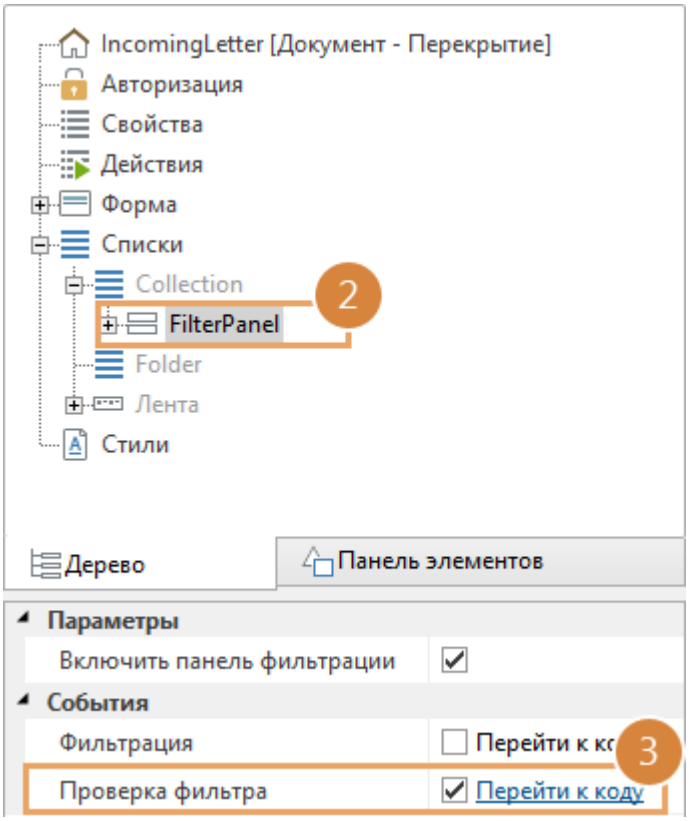
Шаг 4. Настройка валидации панели фильтрации в результатах поиска

При поиске входящих писем в списке результатов поиска также отображается панель фильтрации. Чтобы валидация работала и там:

1. В вашем решении перекройте тип документа Sungero.RecordManagement.IncomingLetter («Входящее письмо») и добавьте

зависимость от модуля Sungero.RecordManagement. Если зависимости не указаны, при публикации могут возникнуть ошибки.

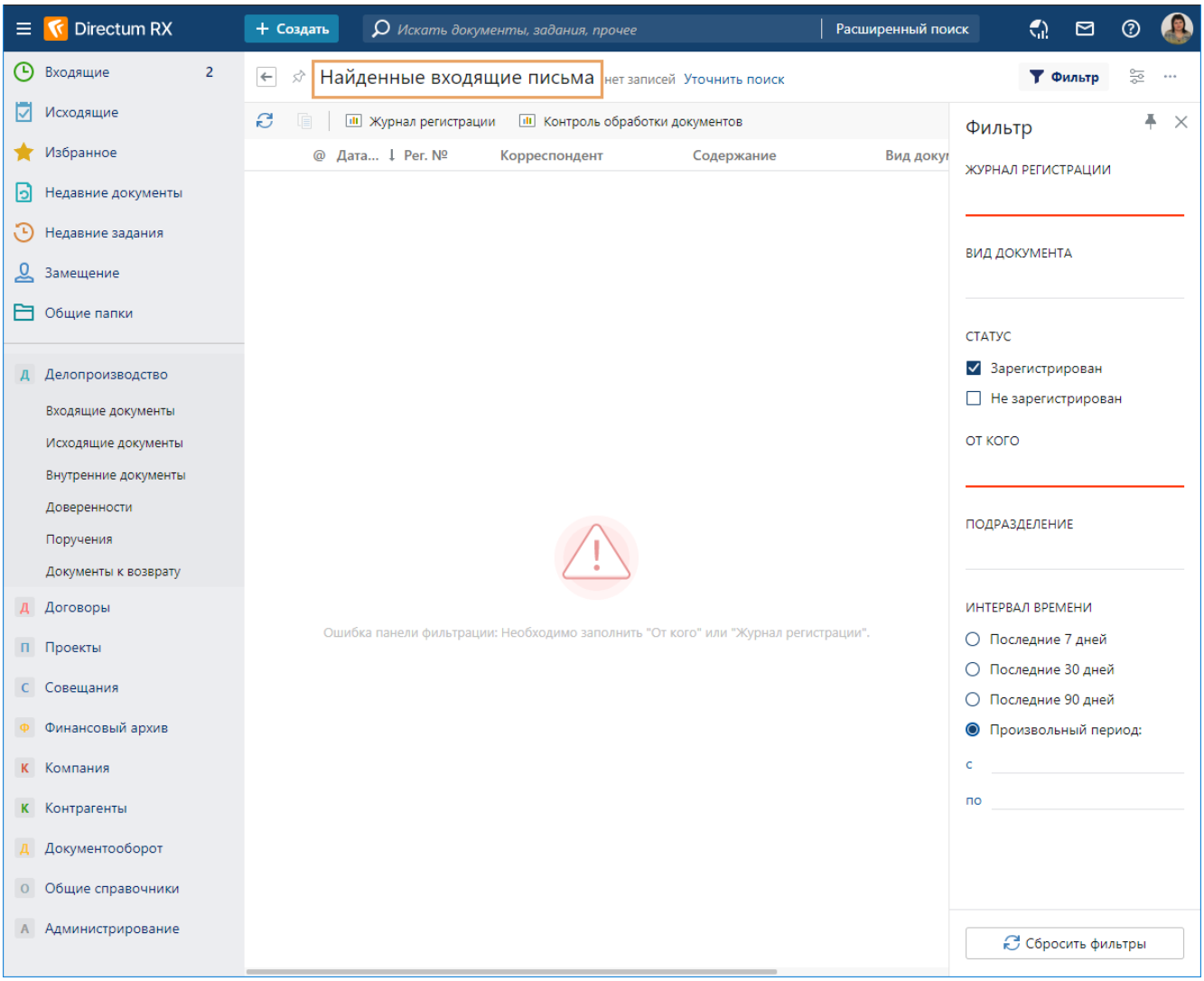
2. В редакторе перекрытого типа документа последовательно перейдите в узел «Списки», «Collection», «FilterPanel»:



3. На панели свойств установите флажок **Проверка фильтра** и перейдите по ссылке. В открывшемся редакторе добавьте код:

```
public override void
   ValidateFilterPanel(Sungero.Domain.Client.ValidateFilterPanelEventArgs e)
   {
     base.ValidateFilterPanel(e);

     // Не выполнять фильтрацию, если установлен
     // произвольный период возврата документов
     // и не заполнены критерии "От кого" и "Журнал регистрации".
     if (_filter.ManualPeriod && (_filter.DateRangeFrom == null ||
   _filter.DateRangeTo == null) && _filter.Counterparty == null &&
   _filter.DocumentRegister == null)

   e
   .AddError
   (Sungero
   .Examples.Module.RecordManagementUI.Resources.FilterPanelValidationError,
   _filter.Info.Counterparty, _filter.Info.DocumentRegister);
   }
```

Таким образом, валидация панели фильтрации будет работать и в результатах поиска входящих писем.

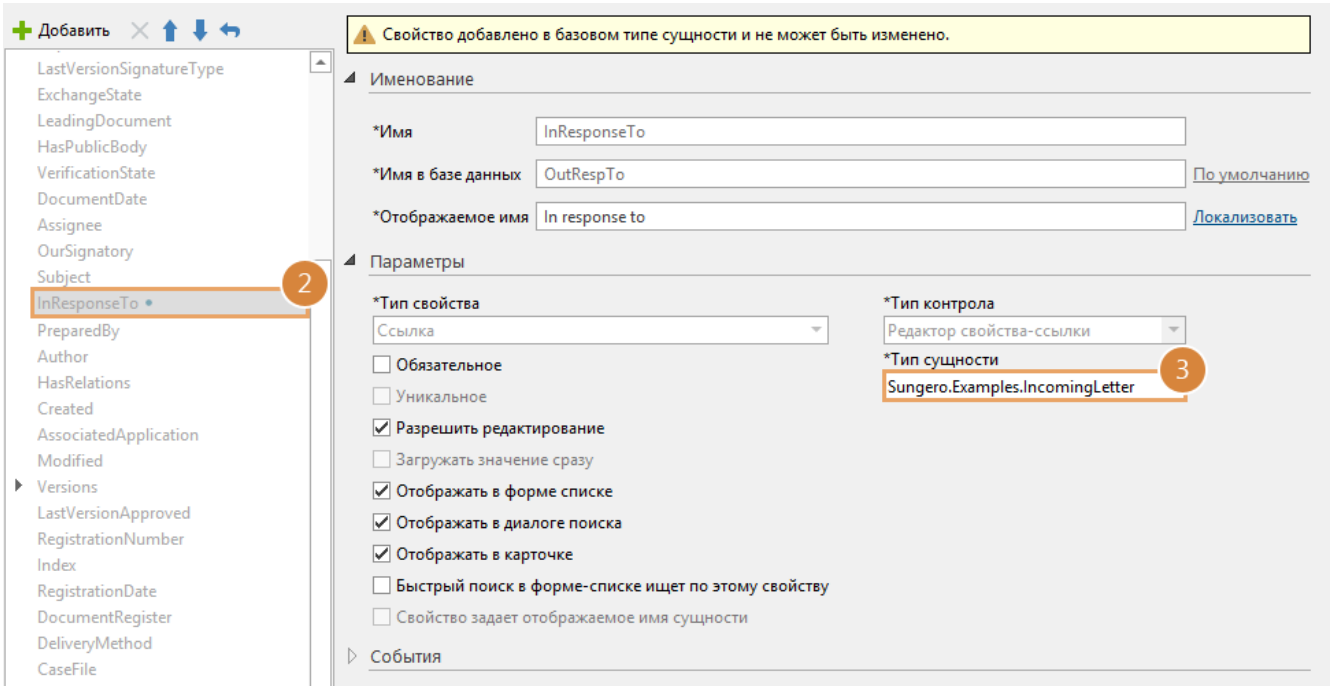


Шаг 5. Перекрытие типа документа «Исходящее письмо»

В карточке исходящего письма есть поле **В ответ на**. В нем указывают входящее письмо, в ответ на которое отправляется исходящее письмо. Для заполнения поля можно открыть список всех входящих писем от указанного контрагента. Чтобы вместо списка открывалась папка с настроенной валидацией панели фильтрации:

1. В вашем решении перекройте тип документа Sungero.RecordManagement.OutgoingLetter («Исходящее письмо») и проверьте, что модуль Sungero.RecordManagement указан в списке зависимостей. При необходимости добавьте его. Если зависимости не указаны, при публикации могут возникнуть ошибки.

2. В редакторе перекрытого типа документа OutgoingLetter перейдите в узел «Свойства» и выделите свойство «InResponceTo» (В ответ на).



3. В выпадающем списке ***Тип сущности*** вместо Sungero.Docflow.IncomingDocumentBase выберите входящее письмо, перекрытое на 4 шаге. В нашем случае это Sungero.Examples.IncomingLetter.

   ПРИМЕЧАНИЕ. Examples – имя решения, в котором перекрыт документ IncomingLetter.

Как настроить список для двух типов документов с одинаковыми свойствами

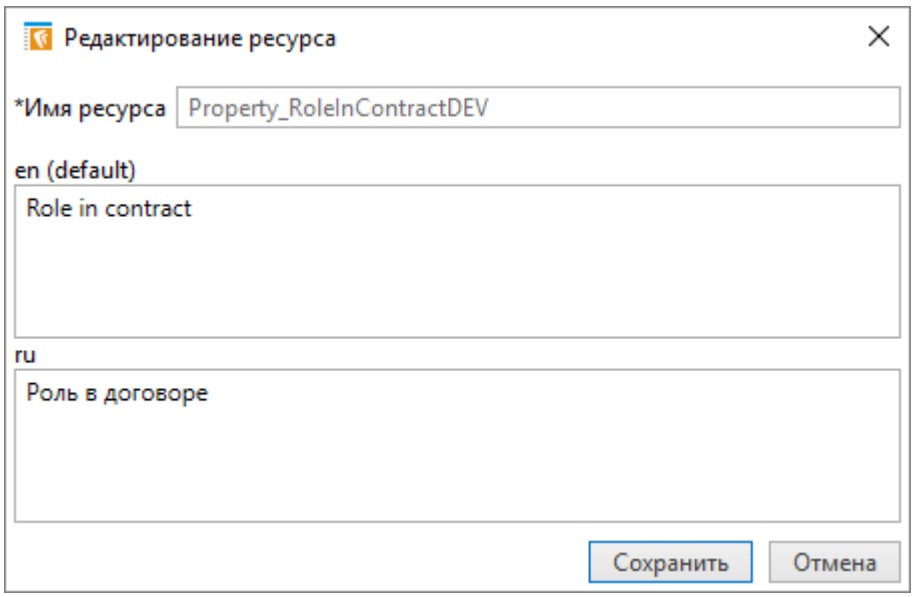
Ситуация

Предположим, перекрыты типы документов «Договор» (Contract) и «Дополнительное соглашение» (SupAgreement). Для каждого типа документа создано свойство **Роль в договоре** (RoleInContract) и добавлено на форму в группу «Условия».

В Directum RX договоры и дополнительные соглашения отображаются в реестре договоров. Нужно, чтобы в реестре появилась колонка для отображения добавленного свойства, и она была одна.

Решение

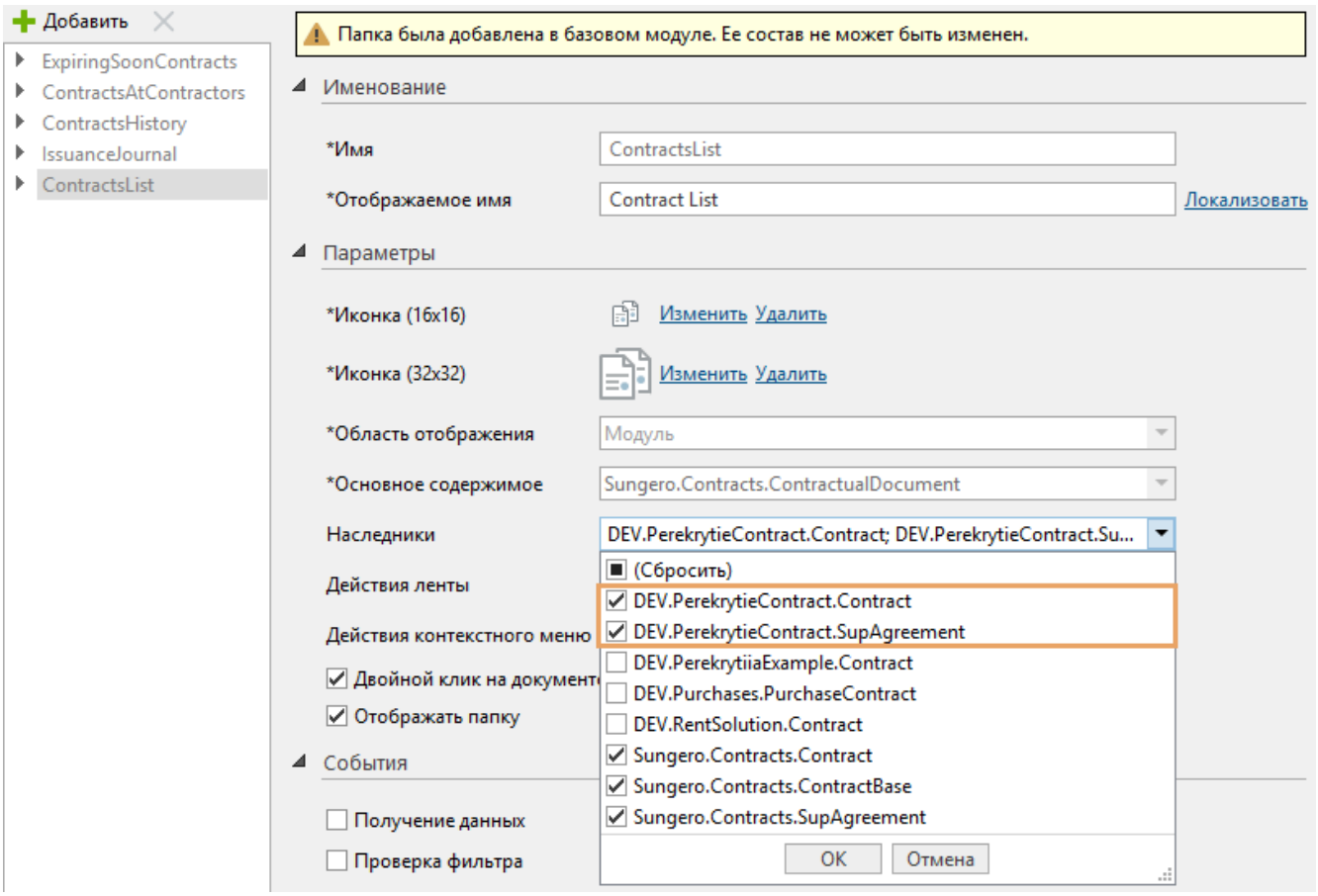
1. В редакторе типа «Договора» и «Дополнительного соглашения» проверьте, что имя, отображаемое имя и локализованные значения свойства **Роль в договоре** (RoleInContract) совпадают:



   ВАЖНО. В окне редактирования ресурса должны быть заполнены оба поля: ***en (default)***, ***ru***.

   Быстро создать одинаковые свойства можно с помощью копирования.

2. Перекройте модуль ContactsUI.
3. В редакторе вашего решения добавьте зависимость от модуля Sungero.ContractsUI. Если зависимости не указаны, при публикации могут возникнуть ошибки.
4. В редакторе перекрытого модуля в узле «Папки» выберите вычисляемую папку «Реестр договоров» (ContractsList).
5. В выпадающем списке **Наследники** установите флажок для перекрытых типов «Договор» и «Дополнительное соглашение».

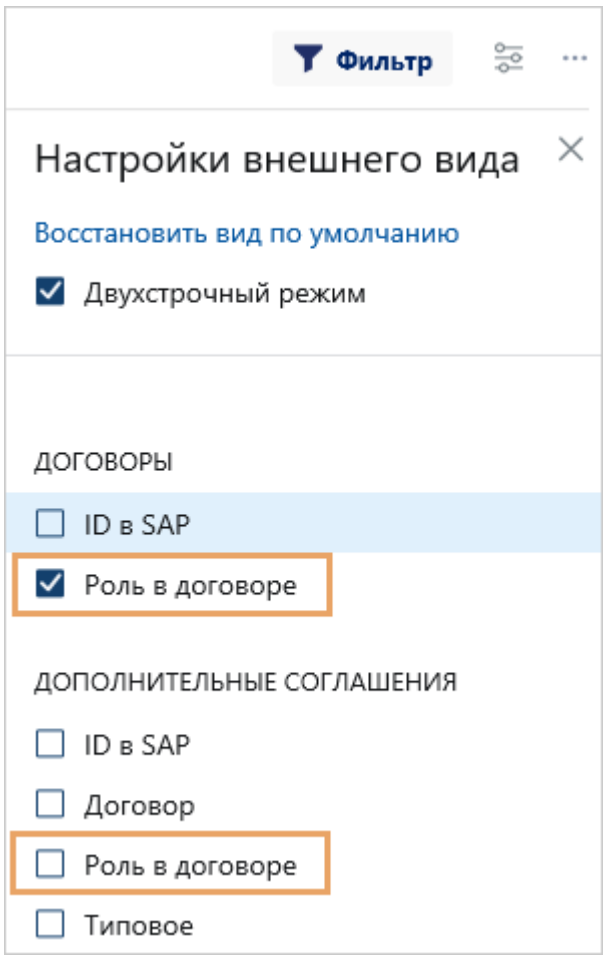


В результате в области **Скрытые колонки** появятся колонки **Роль в договоре (Договор)** и **Роль в договоре (Дополнительное соглашение)**.

6. Настройте представление по умолчанию. Для этого переместите одну из новых колонок из области **Скрытые колонки** в табличную часть, например, **Роль в договоре (Договор)**.
7. Сохраните и опубликуйте решение.

В результате в реестре договоров будет отображаться колонка **Роль в договоре** – это свойство типа документа «Договор».

В настройках внешнего вида списка колонка **Роль в договоре** будет отображаться для типа документа «Договор» и для типа документа «Дополнительное соглашение». Флажок будет стоять напротив колонки для типа «Договор».

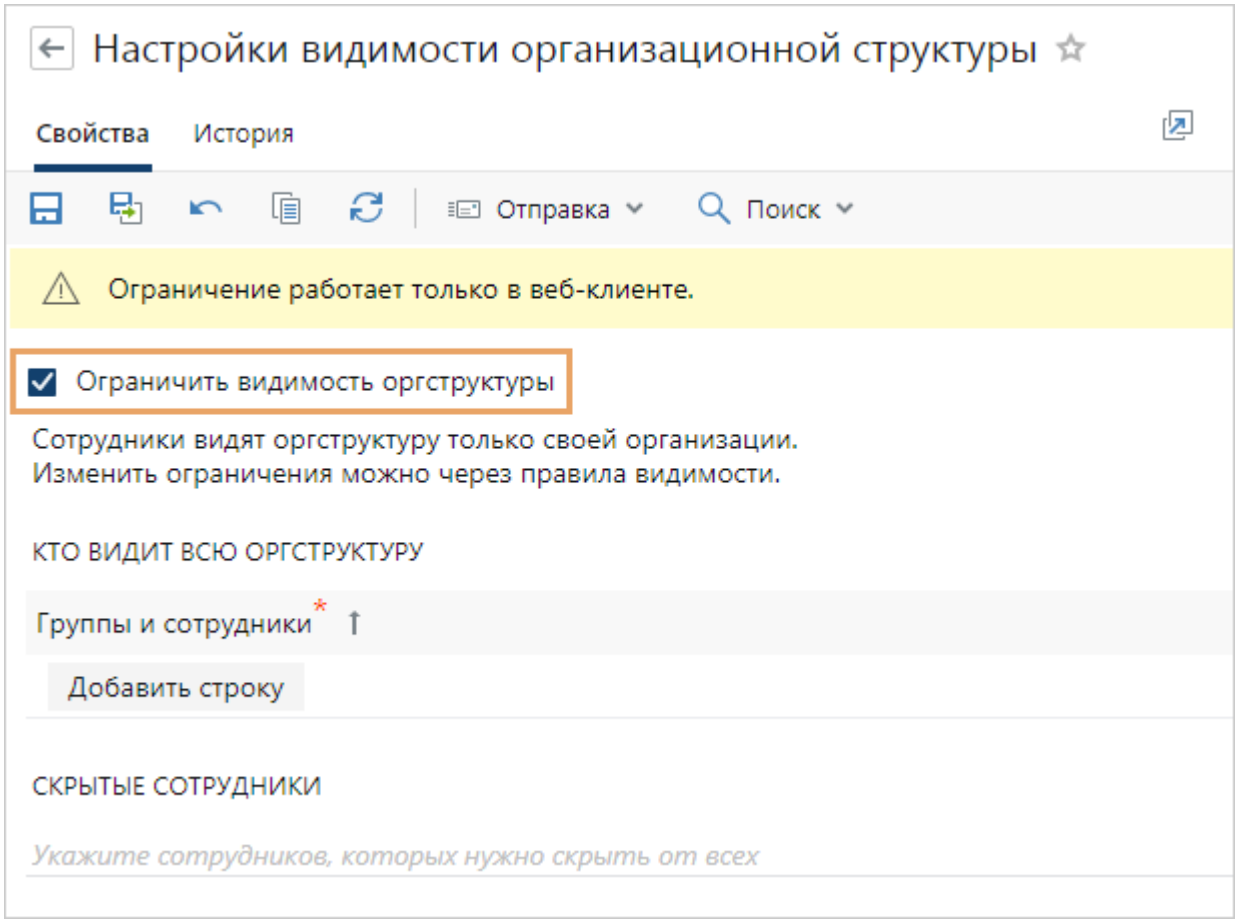


Можно снять флажок напротив колонки для типа «Договор» и поставить напротив колонки для типа «Дополнительное соглашение». В этом случае в реестре в колонке **Роль в договоре** все равно будут отображаться значения для всех документов (договоров и доп. соглашений), где оно заполнено. Это возможно благодаря механизмам платформы и тому, что имена свойств полностью совпадают. Если установить флажок для обоих типов, в реестре будет две колонки с одинаковыми именами и значениями.

Как ограничить видимость сотрудников в списках

Ситуация

Предположим, в Directum RX в справочнике **Настройки видимости оргструктуры** установили флажок **Ограничить видимость оргструктуры**:



Поэтому сотрудники видят в справочнике **Сотрудники** и в выпадающих списках при заполнении полей только коллег из той нашей организации, к которой относятся сами. При этом в карточках документов, а также в задачах и заданиях в полях **Подписал** и **На подпись** отображаются сотрудники, для которых настроено подходящее право подписи, т.е. ограничение видимости не работает.

Рассмотрим, как эта функциональность реализована в базовом решении Directum RX.

Решение

1. Чтобы сотрудники видели в справочнике **Сотрудники** и в выпадающих списках при like-вводе только коллег из своей организации, в справочник **Сотрудники** (Employee) добавлен обработчик события UI-фильтрация:

```
public override IQueryable<T> Filtering(IQueryable<T> query,
   Sungero.Domain.UiFilteringEventArgs e)
    {
      query = base.Filtering(query, e);
      if (Functions.Module.IsRecipientRestrict())
```

```
    {
      var visibleRecipientIds =
 Functions.Module.GetVisibleRecipientIds(Constants.Module.EmployeeTypeGuid);
      return query.Where(c => visibleRecipientIds.Contains(c.Id));
    }
    return query;
  }
```

Событие UI-фильтрации отрабатывает на веб-сервере после наложения всех остальных фильтров, т.е. после базовой фильтрации **base.Filtering()**. UI-фильтрация в отличии от обычной не влияет на вычисления в коде, таким образом не искажает данные, например, для отчетов. Служит для повышения удобства пользователей при работе с системой.

GetVisibleRecipientIds() – это функция, которая с помощью хранимой процедуры по настройкам определяет, кого должны видеть сотрудники, и возвращает список ИД сотрудников. На основании ИД формируется запрос query.Where(c => visibleRecipientIds.Contains(c.Id)). После чего событие UI-фильтрации возвращает список сотрудников.

2. Чтобы в карточках документов, а также в задачах и заданиях в полях **Подписал** и **На подпись** ограничение видимости не работало, выполнение события **UI-фильтрация** точечно отключено в карточке:
   - **Официального документа** (OfficialDocument) для свойства OurSignatory;
   - **Задачи на согласование по регламенту** (ApprovalTask) для свойства Signatory;
   - **Задания на доработку** (ApprovalReworkAssignment) для свойства Signatory;
   - **Задания на согласование с руководителем** (ApprovalManagerAssignment) для свойства Signatory.

   В обработчик события Фильтрация выбора из списка для перечисленных свойств добавлен код:

```
e.DisableUiFiltering = true;
  Functions.OfficialDocument.GetSignatoriesIds(document).Distinct().ToList();
  return query.Where(s => signatories.Contains(s.Id));
```

Виджеты

В разделе содержатся примеры:

- Как создать виджет с действиями?
- Примеры событий для кольцевой диаграммы
- Примеры событий для круговой диаграммы
- Примеры событий для гистограммы
- Примеры событий для линейного графика
- Как добавить параметр-перечисления?
- Как добавить параметр логического типа?
- Как создать набор виджетов, который доступен только руководителям?

Как создать виджет с действиями

Действие – элемент виджета, при нажатии на который открывается список сущностей, папка, запускается отчет или выполняется другой программный код, написанный разработчиком. Справа от названий действий могут отображаться соответствующие им значения. Например, количество заданий, которые находятся в работе у пользователя, сумма договоров за прошлый месяц.

Для логической группировки действий создаются вложенные действия, которые отображаются с небольшим отступом от главного действия. Чтобы визуально отделить действия, используется разделитель, который отображается в виде линии или отступа.

Создание виджета с действиями

Предположим, создается виджет для отображения данных об использовании документов. Необходимо, чтобы в виджете:

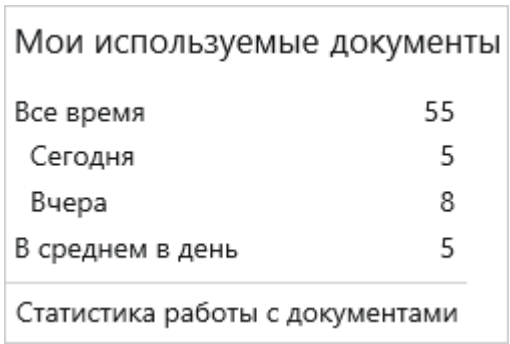
- для главного действия «Все время» и вложенных действий «Сегодня», «Вчера» отображалось количество документов, с которыми текущий пользователь работал за все время, за сегодня и за прошлый день. При нажатии на действия должны открываться соответствующие списки документов;
- для главного действия «В среднем в день» отображалось количество документов, с которым в среднем работает пользователь в день;
- при нажатии на главное действие «Статистика работы с документами» формировался соответствующий отчет.

Также у пользователя должна быть возможность выбирать, какие данные будут отображаться в действиях «Все время», «Сегодня», «Вчера», «В среднем в день»:

- по всем документам, только по документам, которые были открыты на просмотр или на редактирование;
- по всем документам или только по документам, у которых есть содержимое.

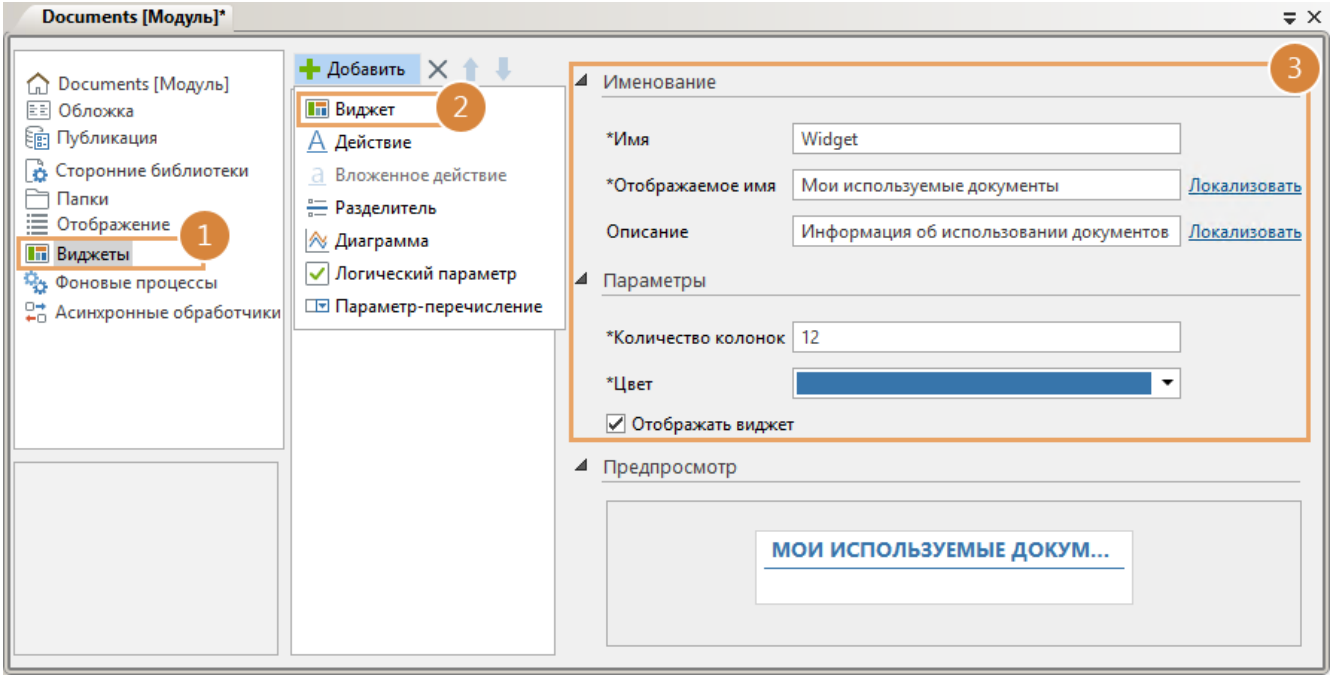
Для этого при создании виджета необходимо добавить параметры, в зависимости от значений которых будут изменяться данные в виджете.

Созданный виджет должен выглядеть следующим образом:



Чтобы создать виджет:

1. В редакторе модуля перейдите в узел «Виджеты».

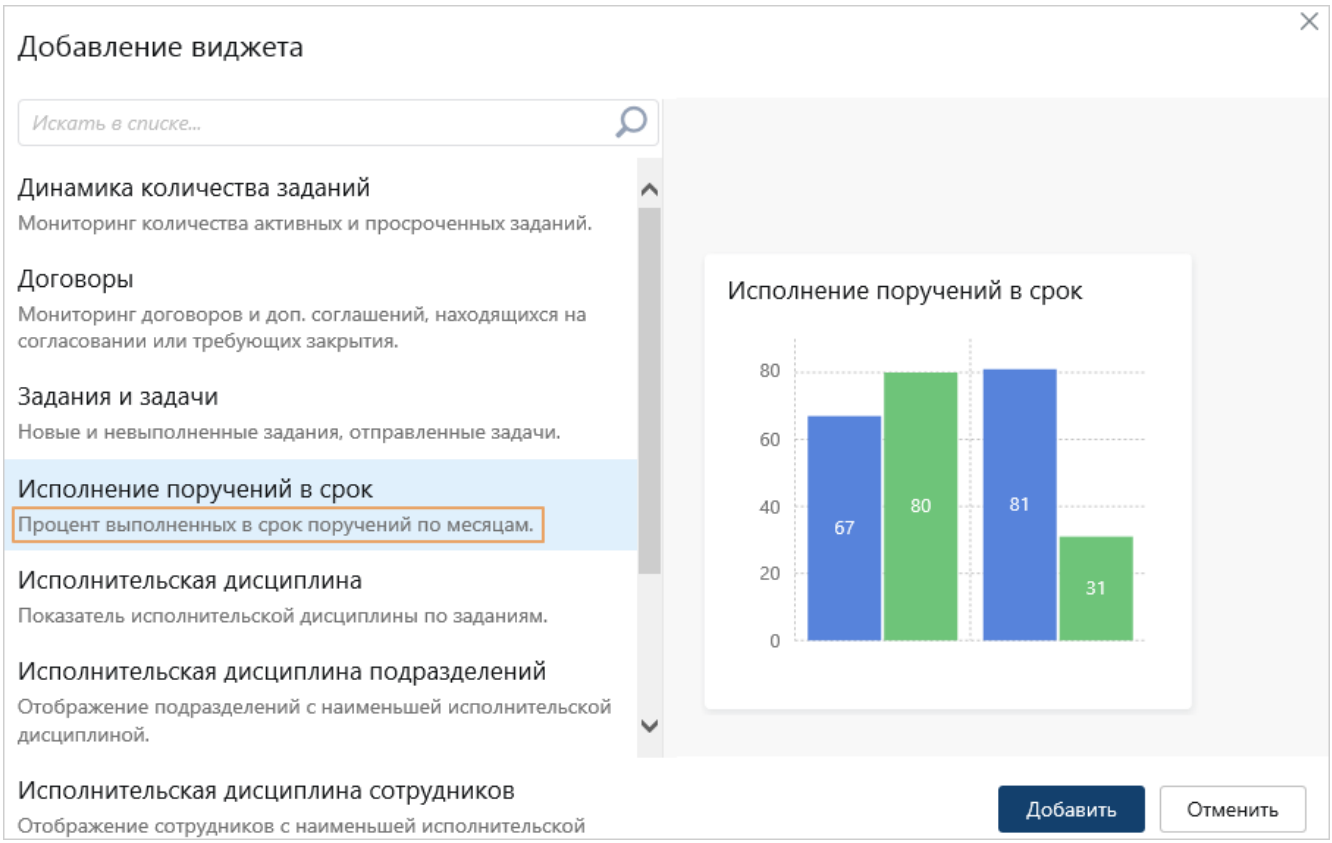


2. Нажмите на кнопку **Добавить** и в выпадающем списке выберите пункт **Виджет**.
3. В списке элементов выделите добавленный виджет и задайте настройки:

   ***Имя** виджета. Должно быть уникальным в рамках одного модуля, состоять из букв латинского алфавита и цифр.

   ***Отображаемое имя**. Заголовок виджета в проводнике системы.

   Описание. Краткое описание виджета. Отображается в проводнике системы в окне добавления виджета.



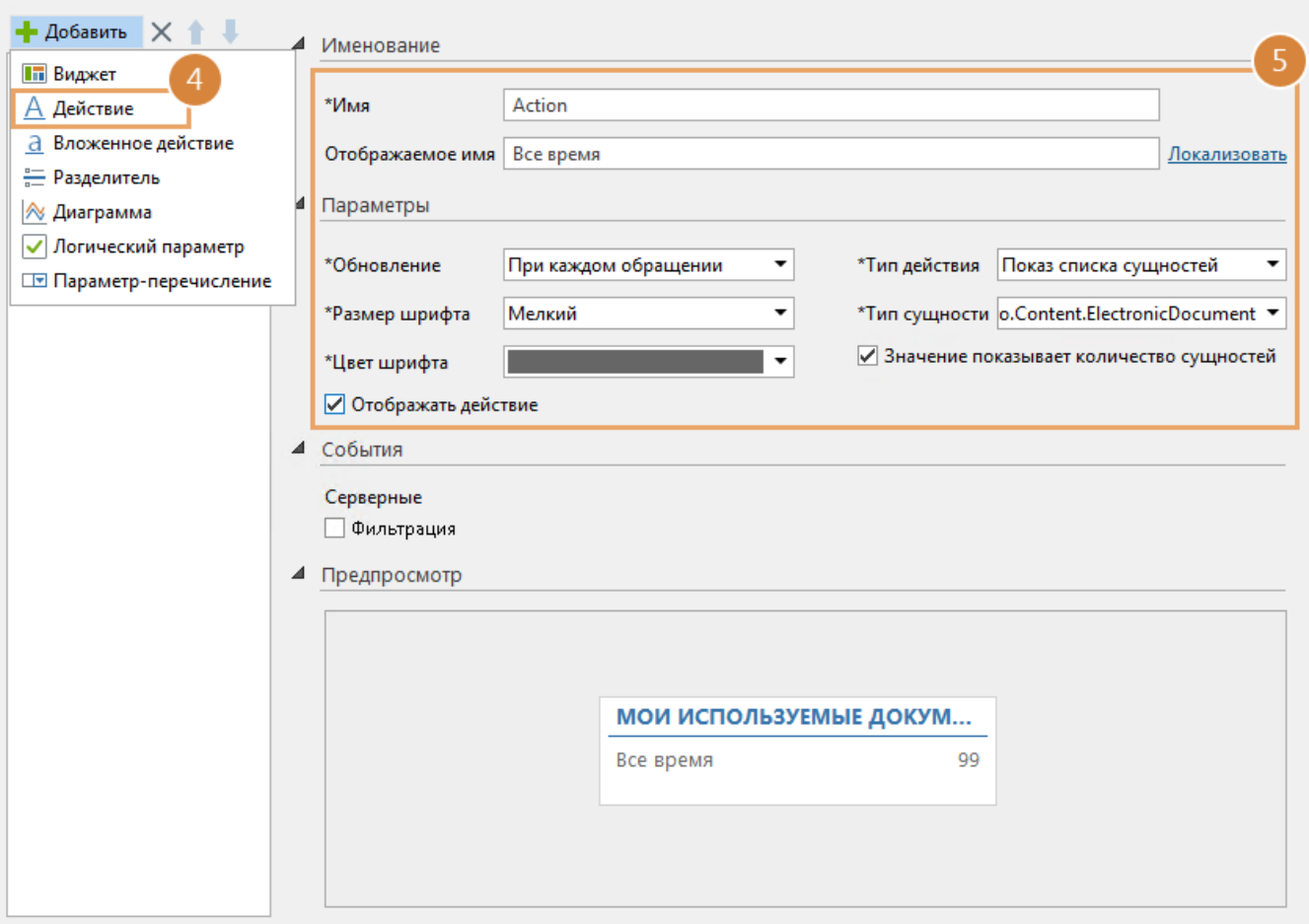
Для *локализации* отображаемого имени и описания перейдите по ссылке **Локализовать**.

***Количество колонок**. Ширина виджета. Возможные значения: целые числа от 1 до 100.

***Цвет** виджета по умолчанию. Для выбора доступно несколько предопределенных цветов.

Отображать виджет. Добавленный виджет доступен для выбора в окне добавления виджетов в Directum RX. Снимите флажок, если необходимо скрыть виджет.

4. Добавьте в виджет главное действие «Все время». Для этого нажмите на кнопку **Добавить** и в выпадающем списке выберите пункт **Действие**.



5. В списке элементов выберите добавленный элемент и задайте настройки:

***Имя** действия. Должно быть уникальным в рамках одного виджета и состоять из букв латинского алфавита.

Отображаемое имя. Название действия виджета в проводнике системы. Для *локализации* отображаемого имени перейдите по ссылке **Локализовать**.

***Обновление**. Период, в котором обновляются данные в действии. Возможные значения: **При каждом обращении**, **В начале дня**, **В начале недели**, **В начале месяца**, **В начале квартала**, **В начале года**. Укажите значение **При каждом обращении**, чтобы данные обновлялись при каждом открытии области виджетов и при добавлении виджета.

***Размер шрифта** названия и значения действия в виджете. Возможные значения: **Мелкий**, **Средний**, **Крупный**.

***Цвет шрифта** для значения действия в виджете. Возможные значения: **серый** и **красный**.

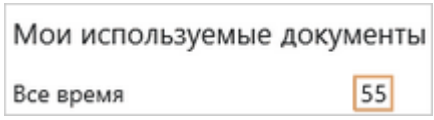
Отображать действие. Добавленное действие отображается в виджете. Снимите флажок, если необходимо скрыть действие.

***Тип действия** виджета. Возможные значения:

- **Показ списка сущностей**. При нажатии на действие в виджете откроется список сущностей того типа, который указан в поле ***Тип сущности**. При необходимости список сущностей можно отфильтровать с помощью события «Фильтрация»;
- **Разрабатываемое**. Логика, которая будет выполняться при нажатии на действие в виджете, задается в обработчике события «Выполнение».

Так как при нажатии на действие «Все время» необходимо открывать список документов, в поле ***Тип действия** выберите значение **Показ списка сущностей**, а в поле ***Тип сущности** укажите тип сущности **Электронный документ** (Sungero.Content.ElectronicDocument).

Установите флажок **Значение показывает количество сущностей**, чтобы справа от действия отображалось количество сущностей в соответствующем списке.

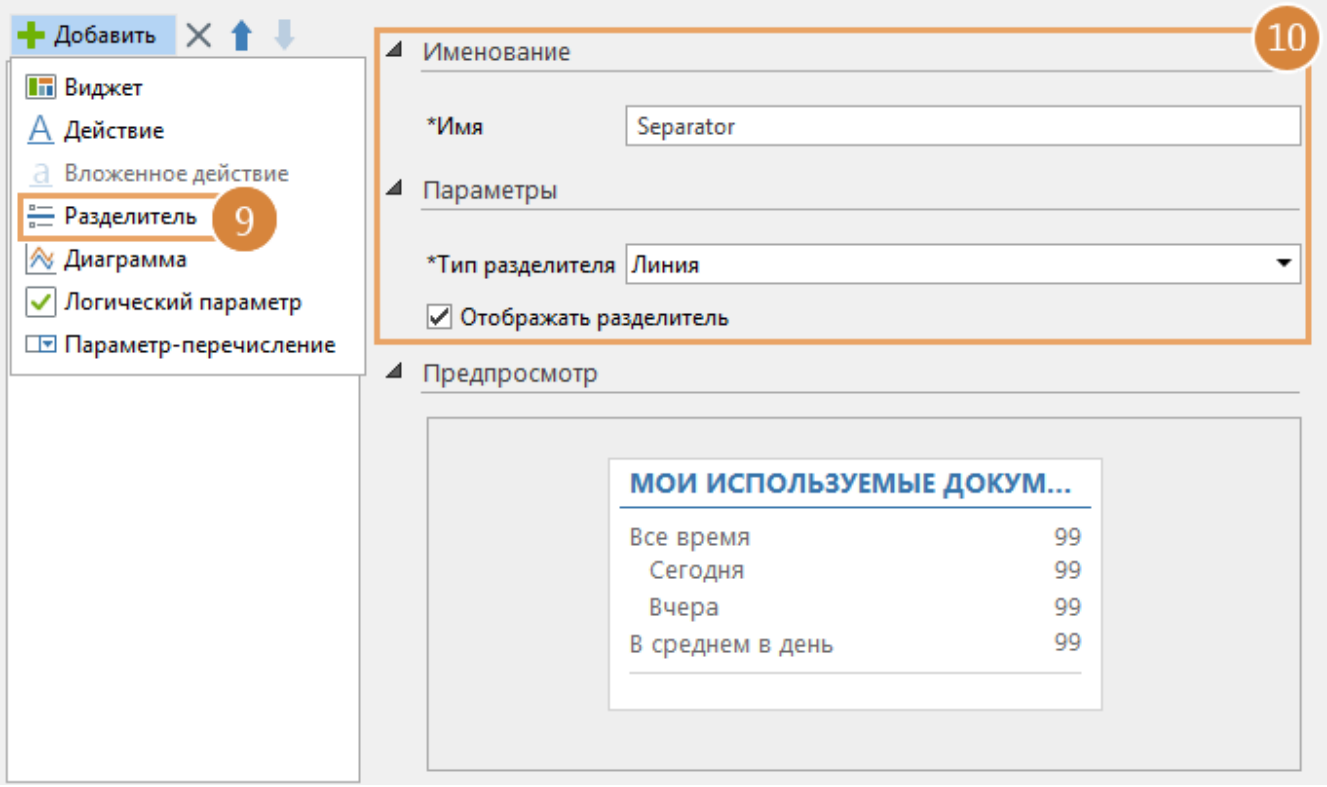


Флажок отображается, если в поле ***Тип действия** указано значение **Показ списка сущностей**.

6. Добавьте вложенные действия «Сегодня» и «Вчера». Для этого в списке элементов выделите главное действие «Все время» и нажмите на кнопку **Добавить**. В выпадающем списке выберите пункт **Вложенное действие**.
7. В списке элементов выберите добавленный элемент и задайте настройки для вложенного действия аналогично настройкам главного действия виджета.

   ПРИМЕЧАНИЕ. Действия в виджете отображаются в том порядке, в котором они расположены в списке элементов. Чтобы изменить порядок, используйте кнопки ⬆ и ⬇ на панели инструментов. Вложенные действия можно перемещать только в пределах главного действия, в котором они созданы.
8. Добавьте главное действие «В среднем в день». В выпадающем списке ***Тип действия** выберите значение **Разрабатываемое**. Задайте остальные настройки аналогично действию «Все время».

9. Чтобы отделить действия «Все время», «Сегодня», «Вчера» и «В среднем в день» от действия «Статистика работы с документами», добавьте разделитель. Для этого в списке элементов нажмите на кнопку **Добавить** и выберите пункт **Разделитель**.



10. В списке элементов выберите добавленный элемент и задайте настройки:

    ***Имя** разделителя. Должно быть уникальным в рамках одного виджета, состоять из букв латинского алфавита и цифр.

    ***Тип разделителя**. Способ разделения действий в виджете. Возможные значения: **Отступ**, **Линия**.

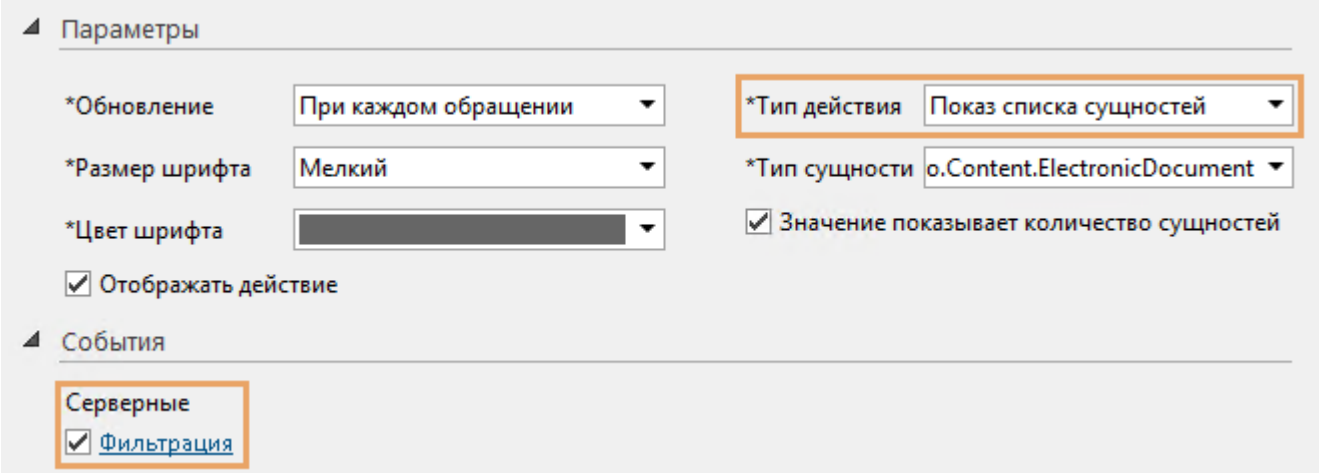
    Отображать разделитель. Добавленный разделитель отображается в виджете. Снимите флажок, если необходимо скрыть разделитель.

11. Добавьте главное действие «Статистика работы с документами». В выпадающем списке ***Тип действия** выберите значение **Разрабатываемое**. Задайте остальные настройки аналогично действию «Все время».

12. Чтобы у пользователя была возможность выбирать, какие данные будут отображаться в действиях «Все время», «Сегодня», «Вчера» и «В среднем в день», добавьте:

    - логический параметр **Учитывать документы без содержимого**. В зависимости от значения параметра в виджете будут отображаться данные по всем документам или только по документам, у которых есть содержимое;
    - параметр-перечисление **Показывать документы**. Возможные значения: **Все**, **Просмотренные**, **Измененные**. В зависимости от значения параметра в виджете будут отображаться данные по всем документам, только по документам, которые были открыты на просмотр или на редактирование.

13. Задайте обработчики событий для действий. Для этого в редакторе свойств действия установите флажок, который соответствует нужному событию, и перейдите по ссылке:
    - «Фильтрация» для фильтрации списка сущностей. Задайте обработчик события для действий «Все время», «Сегодня», «Вчера». Событие доступно, если в настройках действия в поле ***Тип действия** указано значение **Показ списка сущностей**:

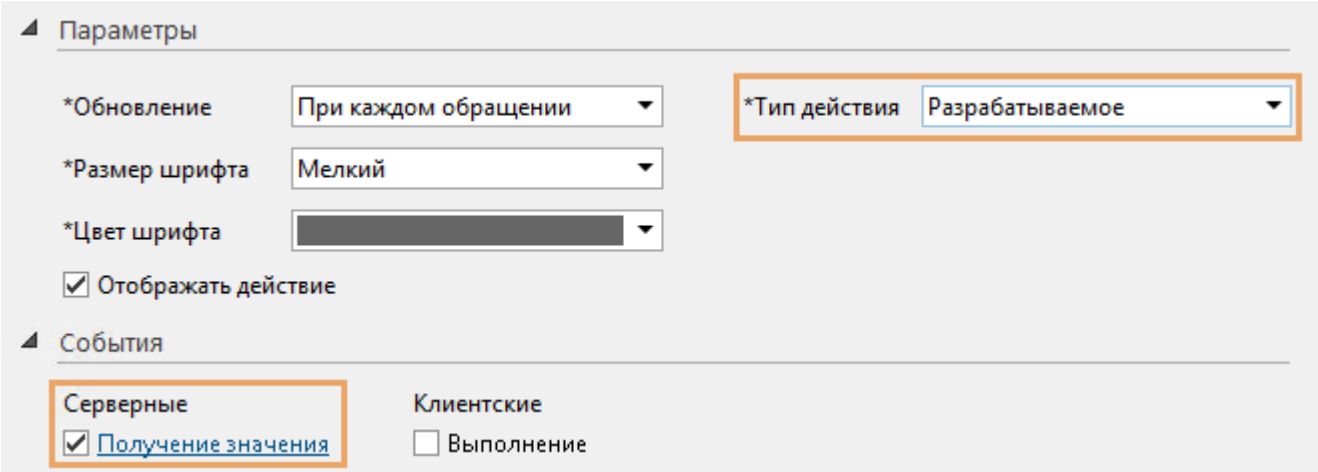


Обработчик события «Фильтрация» для действия «Сегодня» с учетом значений параметров виджета

```
   /// <summary>
/// Формирование списка документов, который открывается
/// при нажатии на действие "Сегодня".
/// </summary>
public virtual IQueryable<Sungero.Content.IElectronicDocument>
DocumentsTodayFiltering(IQueryable<Sungero.Content.IElectronicDocument> query)
{
  // Если для параметра-перечисления "Показывать документы"
  // установлено значение "Просмотренные", то в виджете
  // отображается информация по документам, которые
  // в течение дня пользователь открывал на просмотр.
  if (_parameters.ShowDocuments == Docflow.Widgets.Documents.ShowDocuments.Read)
    query = query.WhereDocumentHistory(h => h.Action ==
Sungero.CoreEntities.History.Action.Read);
  // Если для параметра-перечисления "Показывать документы"
  // установлено значение "Измененные", то в виджете отображается
  // информация по документам, которые в течение дня пользователь
  // открывал на редактирование.
  if (_parameters.ShowDocuments == Docflow.Widgets.Documents.ShowDocuments.Update)
    query = query.WhereDocumentHistory(h => h.Action ==
Sungero.CoreEntities.History.Action.Update);
  // Если для логического параметра
  // "Учитывать документы без содержимого" установлено значение
  // True, то в виджете отображается информация по документам,
  // у которых есть версии.
  if (!_parameters.AllowDocumentWithoutBody)
    query = query.Where(h => h.HasVersions);
  // Если для параметра-перечисления "Показывать документы"
  // установлено значение "Все" и для логического параметра
  // "Учитывать документы без содержимого" установлено значение
  // False,то в виджете отображается информация по всем документам,
  // которые в течение дня открывал пользователь.
```

```
  return query
    .WhereDocumentHistory(h => Equals(Users.Current, h.User) && h.HistoryDate >=
Calendar.Today)
}
```

- «Получение значения» для вычисления значения, которое отображается в виджете справа от действия. Задайте обработчик события для действия «В среднем в день». Событие доступно, если в поле ***Тип действия** указано значение **Разрабатываемое** или значение **Показ списка сущностей** и снят флажок **Значение показывает количество сущностей**:

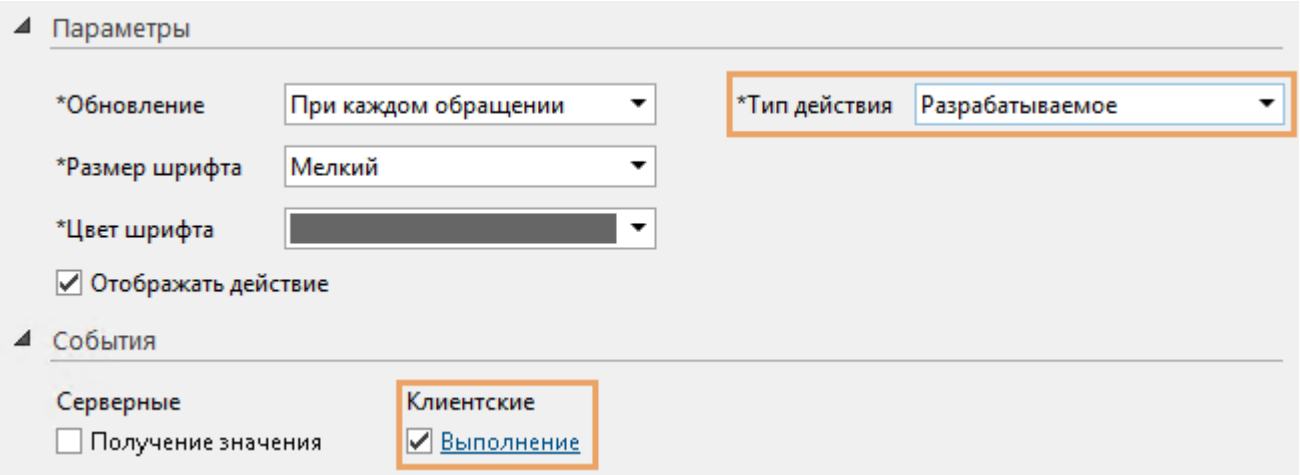


Обработчик события «Получение значения» для действия «В среднем в день» с учетом значений параметров виджета

```
   /// <summary>
/// Расчет среднего количества документов в день, с которым работает
/// пользователь.
/// </summary>
public virtual string GetDocumentsPerDayValue()
{
  var documents = Sungero.Content.ElectronicDocuments.GetAll();
  // Если для параметра-перечисления "Показывать документы"
  // установлено значение "Просмотренные", то в виджете
  // отображается информация по документам, которые пользователь
  // открывал на просмотр.
  if (_parameters.Show == Docflow.Widgets. Documents.Show.Read)
    documents = documents.WhereDocumentHistory(h => h.Action ==
Sungero.CoreEntities.History.Action.Read);
  // Если для параметра-перечисления "Показывать документы"
  // установлено значение "Измененные", то в виджете отображается
  // информация по документам, которые пользователь открывал на
  // редактирование.
  if (_parameters.Show == Docflow.Widgets. Documents.Show.Update)
    documents = documents.WhereDocumentHistory(h => h.Action ==
Sungero.CoreEntities.History.Action.Update);
  // Если для логического параметра
  // "Учитывать документы без содержимого" установлено значение
  // True,то в виджете отображается информация по документам,
  // у которых есть версии.
  if (!_parameters.AllowDocumentWithoutBody)
    documents = documents.Where(h => h.HasVersions);
  // Вызов функции для расчета среднего количества документов,
```

```
  // с которым работает пользователь в день.
  var averageDocumentsPerDay = CalculateAverageDocumentsPerDay(documents);
  return averageDocumentsPerDay;
}
```

- «Выполнение» для задания логики, которая будет выполнена при нажатии на действие виджета. Задайте обработчик события для действия «Статистика работы с документами». Например, для открытия отчета. Событие доступно, если в поле ***Тип действия** указано значение **Разрабатываемое**.



Обработчик события «Выполнение» для действия «Статистика работы с документами»

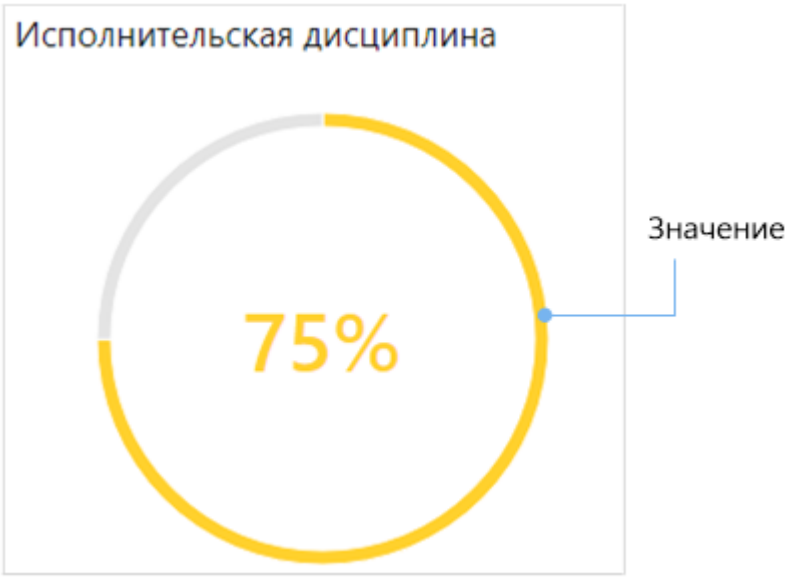
```
   /// <summary>
/// Вызов отчета "Статистика работы с документами".
/// </summary>
public virtual void ExecuteDocumentsStatisticAction()
{
  var report = Docflow.Reports.GetDocumentStatisticReport();
  report.Open();
}
```

ВАЖНО. Для событий доступен аргумент **_parameters**, с помощью которого можно обратиться к параметрам и их значениям.

В результате будет создан виджет для отображения данных об использовании документов пользователем.

Примеры событий для кольцевой диаграммы

Используется для отображения одного показателя в процентах. Например, для отображения исполнительской дисциплины.



Если в программном коде вычислено значение больше 100%, то на диаграмме отобразится 100%, если меньше 0%, то отобразится 0%.

Событие «Фильтрация»

Если в настройках диаграммы в поле ***Тип действия** выбрано значение **Показ списка сущностей**, то задайте обработчик события **Фильтрация**.

Событие предназначено для фильтрации списка сущностей, который открывается при нажатии на значение диаграммы.

Обработчик события «Фильтрация» для списка заданий на исполнение поручений

```
var userNow = Calendar.UserNow;
var serverNow = userNow.FromUserTime();
var userBeginOfToday = userNow.BeginningOfDay();
var serverBeginOfToday = userBeginOfToday.FromUserTime();
var serverEndOfDay = userNow.EndOfDay().FromUserTime();
query = query.Where(a => !Equals(a.Deadline, serverBeginOfToday) &&
                    (a.Deadline < serverNow && a.Deadline !=
a.Deadline.Value.Date ||
                     a.Deadline < serverEndOfDay.Date && a.Deadline ==
a.Deadline.Value.Date));
```

В результате при нажатии на значение виджета откроется список просроченных заданий текущего сотрудника.

Событие «Получение значения»

Событие предназначено для вычисления значения диаграммы и настройки внешнего вида.

Аргументы события:

- e.Chart.AddValue() – добавление значения кольцевой диаграммы.

```
public virtual void GetWidgetChartValue(Sungero.Domain.GetWidgetGaugeChartValueEventArgs e)
{
  // Добавление значения кольцевой диаграммы.
  e.Chart.AddValue(label, value, color);
}
```

Содержит параметры:

label – текст подсказки, которая появляется при наведении курсора мыши на значение диаграммы. Текст подсказки рекомендуется задавать в редакторе ресурсов. Тип параметра – строка.

value – значение, которое отображается внутри диаграммы, указывается в процентах. Тип параметра – число.

color – настройка цвета для значения кольцевой диаграммы. Параметр необязательный. Тип параметра – цвет.

Обработчик события «Получение значения» виджета «Исполнительская дисциплина»

```
public virtual void GetAssignmentCompletionGraphAllAssignmentChartValue
(Sungero.Domain.GetWidgetGaugeChartValueEventArgs e)
{
  if (!Equals(_parameters.Performer,
Widgets.AssignmentCompletionGraph.Performer.All) && Users.Current.IsSystem ==
true)
    return;

  // Получение значения кольцевой диаграммы.
  var value =
Docflow
.PublicFunctions
.Module.GetPercentageOfAssignmentsCompletedInTime(businessUnitIds,
departmentsIds, employee, periodBegin, Calendar.UserToday.EndOfDay(), unwrap,
false, needFilter);
  // Если значение вычислить не удалось,
  // в виджете появится сообщение «Нет данных».
  if (value == null)
    return;

  // Получение цвета виджета.
  var color =
Functions.Module.GetAssignmentCompletionWidgetValueColor(value.Value);
  // Добавление значения кольцевой диаграммы.
  // Текст подсказки берется из ресурсов модуля.
  e.Chart.AddValue(string.Format("{0}",
Sungero.RecordManagement.Resources.AssignmentCompletion), value.Value, color);
}
```

В результате в диаграмме виджета «Исполнительская дисциплина» будет отображаться значение – процент исполнительской дисциплины. Цвет виджета будет меняться в зависимости от процента выполненных заданий в срок по функции **GetAssignmentCompletionWidgetValueColor()**: выше 75% – зеленый, выше 50% – желтый, ниже 50% – красный.

Событие «Выполнение»

Если в настройках диаграммы в поле ***Тип действия** выбрано значение **Разрабатываемое**, то задайте обработчик события **Выполнение**.

Событие предназначено для задания логики, которая выполняется при нажатии на значение кольцевой диаграммы.

Обработчик события «Выполнение» виджета «Исполнительская дисциплина»

```
    public                                virtual                                void
       ExecuteAssignmentCompletionGraphAllAssignmentChartAction()
{
  // Открытие отчета по исполнительской дисциплине.
  PublicFunctions.Module.OpenAssignmentCompletionReport(_parameters.Performer,
_parameters.Period);
}
```

В результате при нажатии на значение виджета откроется отчет по исполнительской дисциплине подразделения, сотрудников или выбранного сотрудника. То, какой отчет будет формироваться из виджета, зависит от настроенных параметров.

Примеры событий для круговой диаграммы

Используется для отображения отношения частей к целому. Например, для отображения выполненных и просроченных заданий по отношению к общему количеству заданий.

Диаграмма состоит из значений – отдельных секторов. Для нее настраивается отображение легенды.

Событие «Фильтрация»

Если в настройках диаграммы в поле ***Тип действия** выбрано значение **Показ списка сущностей**, то задайте обработчик события **Фильтрация**. Событие предназначено для фильтрации списка сущностей, который открывается при нажатии на значение диаграммы.

Аргументы события:

- e.ValueId – уникальный идентификатор значения круговой диаграммы. Задается в обработчике события Получение значения.

Обработчик события «Фильтрация» для списка заданий

```
    // При нажатии на сектор открывать список просроченных заданий
       // текущего сотрудника.
public              virtual              IQueryable<Sungero.Workflow.IAssignment>
MyTodayAssignmentsChartFiltering(System.Linq.IQueryable
<Sungero.Workflow.IAssignment>                                          query,
Sungero.Domain.WidgetPieChartFilteringEventArgs e)
{
  // ValueId – идентификатор сектора значения круговой диаграммы.
  if (e.ValueId == OverdueToday)
     return query.Where(a => a.Performer.Equals(Users.Current) && a.Deadline <
Calendar.UserNow);
  return query;
}
```

В результате при нажатии на сектор с идентификатором OverdueToday будет открываться список просроченных заданий текущего сотрудника.

Событие «Получение значения»

Событие предназначено для вычисления значений диаграммы и настройки внешнего вида.

Аргументы события:

- e.Chart.AddValue() – добавление значений круговой диаграммы.

```
public                                  virtual                                  void
   GetWidgetChartValue(Sungero.Domain.GetWidgetPieChartValueEventArgs e)
   {
     // Добавление значений круговой диаграммы.
     e.Chart.AddValue(valueId1, label1, value1, color1);
     e.Chart.AddValue(valueId2, label2, value2, color2);
   }
```

Содержит параметры:

valueId – уникальный идентификатор значения круговой диаграммы. Используется в событиях Фильтрация и Выполнение. Рекомендуется задавать в константах модуля. Тип параметра – строка.

label – текст, который отображается в легенде и в подсказке, появляющейся при наведении курсора мыши на значение. Текст рекомендуется задавать в редакторе ресурсов. Тип параметра – строка.

value – значение, которое отображается рядом с сектором. Тип параметра – число.

color – настройка цвета для сектора круговой диаграммы. Параметр необязательный. Тип параметра – цвет.

- e.Chart.DisplayValueFormat – формат значения, которое отображается рядом сектором. Например, единицы измерения, проценты.

```
// Числовое значение рядом с сектором будет отображаться в процентах.
   e.Chart.DisplayValueFormat = {"0:P"};
```

Подробнее см. в статьях Microsoft «Строки стандартных числовых форматов» и «Строки настраиваемых числовых форматов».

- e.Chart.IsLegendVisible – отображение легенды для диаграммы. По умолчанию отображается.

```
// Отключить отображение легенды.
   e.Chart.IsLegendVisible = false;
```

Обработчик события «Получение значения» виджета «Мои задания»

```
public virtual void GetMyTodayAssignmentsChartValue
   (Sungero.Domain.GetWidgetPieChartValueEventArgs e)
{
  var assignments = Workflow.Assignments.GetAll();
  // Добавление сектора с просроченными заданиями.
       var overdue = Functions.Module.GetMyTodayAssignments(assignments,
   _parameters.Substitution, Constants.Module.OverdueToday).Count();
  if (overdue != 0)
     e.Chart.AddValue(Constants.Module.OverdueToday, Resources.WidgetMTAOverdue,
   overdue, Colors.Charts.Red);
  // Добавление сектора с заданиями, которые нужно выполнить сегодня.
      var deadline = Functions.Module.GetMyTodayAssignments(assignments,
   _parameters.Substitution, Constants.Module.DeadlineToday).Count();
  if (deadline != 0)
                                e.Chart.AddValue(Constants.Module.DeadlineToday,
   Resources.WidgetMTADeadline, deadline, Colors.Charts.Color1);
  // Добавление сектора с заданиями, которые нужно выполнить до послезавтра.
       var after = Functions.Module.GetMyTodayAssignments(assignments,
   _parameters.Substitution, Constants.Module.AfterToday).Count();
  if (after != 0)
        e.Chart.AddValue(Constants.Module.AfterToday, Resources.WidgetMTAAfter,
   after, Colors.Charts.Color6);
  // Добавление сектора с заданиями, которые выполнены сегодня.
      var completed = Functions.Module.GetMyTodayAssignments(assignments,
   _parameters.Substitution, Constants.Module.CompletedToday).Count();
  if (completed != 0)
                               e.Chart.AddValue(Constants.Module.CompletedToday,
   Resources.WidgetMTACompleted, completed, Colors.Charts.Green);
```

```
}
```

В результате диаграмма виджета «Мои задания» будет состоять из четырех секторов, для каждого из которых отображается свое значение и цвет. При нажатии на сектор откроется соответствующий список заданий.

Событие «Выполнение»

Если в настройках диаграммы в поле ***Тип действия** выбрано значение **Разрабатываемое**, то задайте обработчик события **Выполнение**. Событие предназначено для задания логики, которая выполняется при нажатии на значение диаграммы.

Аргументы события:

- e.ValueId – уникальный идентификатор значения круговой диаграммы. Задается в обработчике события Получение значения.

Обработчик события «Выполнение»

```
public virtual void
   ExecuteMyTodayAssignmentsChart(Sungero.Domain.Client.
   ExecuteWidgetPieChartActionEventArgs e)
{
  // При нажатии на сектор «Выполнить сегодня»
  // выполнится функция MyTodayAssignments()
  if (e.ValueId == Constants.Module.CompletedToday)
    Functions.Module.MyTodayAssignments(_parameters.Performer);

}
```

Примеры событий для гистограммы

В виджеты можно добавлять вертикальные и горизонтальные гистограммы:

- с накоплением – для просмотра отношения отдельных частей к целому за определенный период. Например, для отображения загрузки сотрудников за последний месяц: сколько всего задач у каждого сотрудника, сколько из них просрочено, а сколько без нарушения срока;
- с группировкой – для сравнения значений. Например, для сравнения исполнительской дисциплины сотрудников.

ПРИМЕЧАНИЕ. При выборе типа гистограмм учитывайте, что в гистограммах с группировкой, в отличие от гистограмм с накоплением, отображаются нулевые значения. Например, если исполнительская дисциплина сотрудника составляет 0%, то можно настроить открытие списка просроченных заданий при нажатии на это значение.

Гистограмма состоит из нескольких серий и значений. Для нее настраивается отображение легенды и подпись к оси.

Событие «Фильтрация»

Если в настройках гистограммы в поле ***Тип действия** выбрано значение **Показ списка сущностей**, то задайте обработчик события **Фильтрация**.

Событие предназначено для фильтрации списка сущностей, который открывается при нажатии на значение гистограммы.

Аргументы события:

- e.SeriesId – уникальный идентификатор серии гистограммы;
- e.ValueId – уникальный идентификатор значения серии.

Идентификаторы задаются в обработчике события Получение значения.

Обработчик события «Выполнение»

```
public virtual IQueryable<Sungero.Workflow.IAssignment>
MyAssignmentsChartFiltering(IQueryable<Sungero.Workflow.IAssignment> query,
Sungero.Domain.WidgetBarChartFilteringEventArgs e)
{
  long performerId;
  // Использование идентификатора серии гистограммы.
  if(long.TryParse(e.SeriesId, out performerId))
    // Использование идентификатора значения серии.
    if(e.ValueId == MyModule.Constants.Module.Overdue)
    {
      var userNow = Calendar.UserNow;
      var serverNow = userNow.FromUserTime();
      var userBeginOfToday = userNow.BeginningOfDay();
      var serverBeginOfToday = userBeginOfToday.FromUserTime();
      var serverEndOfDay = userNow.EndOfDay().FromUserTime();
      return query.Where(a => a.Performer.Id == performerId &&
!Equals(a.Deadline, serverBeginOfToday) && (a.Deadline < serverNow && a.Deadline
!= a.Deadline.Value.Date || a.Deadline < serverEndOfDay.Date && a.Deadline ==
a.Deadline.Value.Date));
    }
  else
    return query.Where(a => a.Performer.Id == performerId);
}
```

Событие «Получение значения»

Событие предназначено для добавления серий, вычисления значений гистограммы и настройки внешнего вида.

Аргументы события:

- e.Chart.AddNewSeries() – добавление серии гистограммы.

  ```
  public virtual void
    GetWidgetChartValue(Sungero.Domain.GetWidgetBarChartValueEventArgs e)
    {
      // Добавление новой серии.
      var series1 = e.Chart.AddNewSeries(seriesId1, title1);
    }
  ```

  Содержит параметры:

  seriesId – уникальный идентификатор серии. Используется в событиях Фильтрация и Выполнение. Тип параметра – строка.

  title – подпись к серии, например, ФИО сотрудников, названия подразделений, месяцы. Тип параметра – строка.

- series1.AddValue() – добавление значений серии гистограммы.

  ```
  public virtual void
    GetWidgetChartValue(Sungero.Domain.GetWidgetBarChartValueEventArgs e)
    {
      // Создание серии.
      var series1 = e.Chart.AddNewSeries(seriesId1, title1);
  ```

```
    // Заполнение серии значениями.
    series1.AddValue(valueId1, label1, value1, color1);
    series1.AddValue(valueId2, label2, value2, color2);
  }
```

Содержит параметры:

valueId – уникальный идентификатор значения. Используется в событиях Фильтрация и Выполнение. Можно указывать, например, ИД пользователя, ИД группы пользователей, Guid группы, месяц, число. Рекомендуется задавать в константах модуля. Тип параметра – строка.

label – текст, который отображается в легенде и в подсказке, появляющейся при наведении курсора мыши на значение. Текст рекомендуется задавать в редакторе ресурсов. Тип параметра – строка.

value – значение, которое отображается в колонках гистограммы. Тип параметра – число.

color – настройка цвета для значения гистограммы. Параметр необязательный. Тип параметра – цвет.

- e.Chart.AxisTitle – подпись к оси гистограммы.

  ```
  e.Chart.AxisTitle = "Подпись к оси";
  ```

- e.Chart.IsLegendVisible – отображение легенды для гистограммы. По умолчанию отображается. Если из заголовка виджета или подписи к оси понятно, что означают конкретные значения, то рекомендуется отключать отображение легенды для экономии места под гистограмму.

  ```
  // Отключить отображение легенды.
    e.Chart.IsLegendVisible = false;
  ```

- e.Chart.IsValuesVisible – отображение значений в колонках гистограммы. По умолчанию отображаются. Рекомендуется отключать отображение значений, если они не вмещаются в колонки гистограммы.

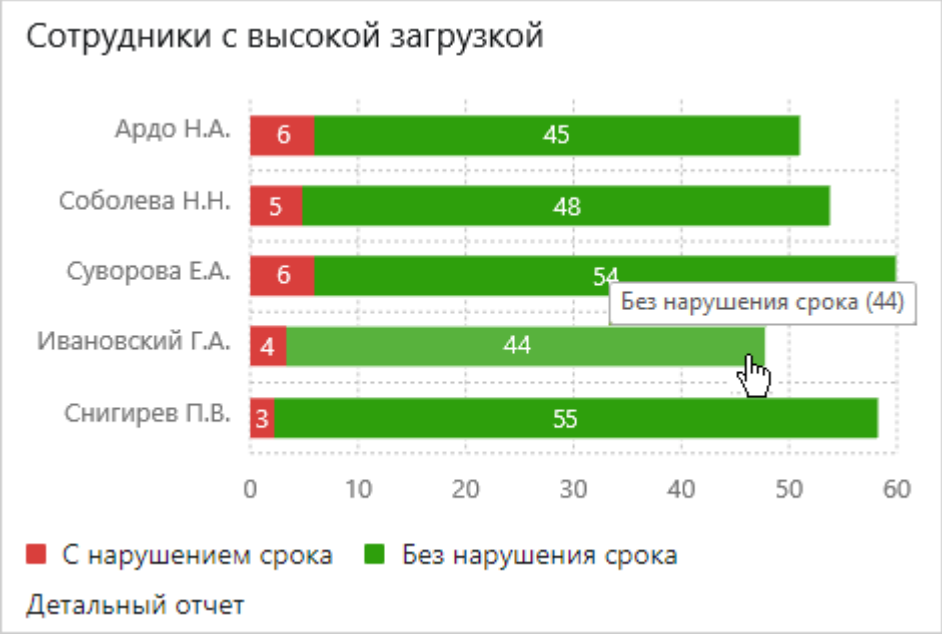
  ```
  // Отключить отображение значений в колонках гистограммы.
    e.Chart.IsValuesVisible = false;
  ```

Обработчик события «Получение значения» виджета «Сотрудники с высокой загрузкой»

```
   // Используется горизонтальная гистограмма с накоплением.
public virtual void
GetTopLoadedPerformersGraphTopLoadedPerformersValue
(Sungero.Domain.GetWidgetBarChartValueEventArgs e)
{
  if (!Equals(_parameters.CarriedObjects,
Widgets.TopLoadedPerformersGraph.CarriedObjects.All) && Users.Current.IsSystem
== true)
    return;
  // Вычисление загрузки сотрудников.
  var performerLoads = items
  .Where(i => i.AssignmentCompletion != null && i.IsActiveEmployee)
  .OrderByDescending(i => i.AssignmentsCount)
  .Take(5)
  .Select(i => Structures.Module.PerformerLoad.Create(Employees.Get(i.Employee),
i.AssignmentsCount, i.OverdueAssignmentsCount));
// Получение загруженных сотрудников с уникальными именами.
// Сотрудники сортируются по убыванию количества заданий.
  var uniqueNamedPerformerLoads =
Functions.Module.SetUniquePerformerNames(performerLoads.ToList());
```

```
    foreach (var performerLoad in uniqueNamedPerformerLoads)
    {
      // Создание серии performerSeries.
      var performerSeries =
e.Chart.AddNewSeries(performerLoad.PerformerLoad.Employee.Id.ToString(),
performerLoad.UniqueName);
      // Добавление значения серии (задания с нарушением срока).
      performerSeries.AddValue(Constants.Module.OverduedAssignments,
Resources.WithOverdue,
      // Добавление значения серии (задания без нарушения срока).
      performerLoad.PerformerLoad.OverduedAssignment, Colors.Charts.Red);
      performerSeries.AddValue(Constants.Module.NotOverduedAssignments,
Resources.WithoutOverdue, performerLoad.PerformerLoad.AllAssignment,
performerLoad.PerformerLoad.OverduedAssignment, Colors.Charts.Green);
    }
}
```

В результате в гистограмме будут отображаться сотрудники с высокой загрузкой. Для каждого сотрудника показывается количество заданий с нарушением срока и без нарушения.



Событие «Выполнение»

Если в настройках гистограммы в поле ***Тип действия** выбрано значение **Разрабатываемое**, то задайте обработчик события **Выполнение**.

Событие предназначено для задания логики, которая выполняется при нажатии на значение гистограммы.

Аргументы события:

- e.SeriesId – уникальный идентификатор серии гистограммы;
- e.ValueId – уникальный идентификатор значения серии.

Идентификаторы задаются в обработчике события Получение значения.

Обработчик события «Выполнение» виджета «Сотрудники с высокой загрузкой»

```
    public virtual void ExecuteTopLoadedPerformersGraphTopLoadedPerformersAction
      (Sungero.Domain.Client.ExecuteWidgetBarChartActionEventArgs e)
{
  // Получение сотрудника по ИД.
  long employeeId;
  if (!long.TryParse(e.SeriesId, out employeeId))
    // Открытие отчета по исполнительской дисциплине сотрудника.
    PublicFunctions.Module.OpenReportFromEmployeeWidgets(employeeId,
_parameters.Period);
}
```

В результате при нажатии на колонку гистограммы откроется отчет по исполнительской дисциплине выбранного сотрудника.

Примеры событий для линейного графика

Используется для отображения динамики изменения различных показателей. Например, для отображения динамики количества заданий, мониторинга количества просроченных и заданий в работе.

Линейный график состоит из нескольких серий и значений, которые представляют собой отдельные точки (x,y) на линии. При нажатии на значение графика появляется подсказка.

Для линейного графика задается только обработчик события **Получение значения**, в котором настраивается отображение легенды, оси и подписи к осям.

Событие «Получение значения»

Событие предназначено для добавления серий графика, заполнения серии значениями и настройки внешнего вида.

Аргументы события:

- e.Chart.AddNewSeries() – добавление серии линейного графика.

```
public                                virtual                                void
   GetWidgetChartValue(Sungero.Domain.GetWidgetPlotChartValueEventArgs e)
   {
     // Добавление новой серии.
```

```
    var series1 = e.Chart.AddNewSeries(title, color);
  }
```

Содержит параметры:

title – название линии графика, которое отображается в легенде и в подсказке, появляющейся при нажатии на значение графика. Текст рекомендуется задавать в редакторе ресурсов. Тип параметра – строка.

color – настройка цвета для серии графика. Параметр необязательный. Тип параметра – цвет.

- series1.AddValue() – добавление значений серии линейного графика.

```
public virtual void GetWidgetChartValue(Sungero.Domain.GetWidgetPlotChartValueEventArgs e)
  {
    // Создание серии.
    var series1 = e.Chart.AddNewSeries(title, color);
    // Заполнение серии значениями.
    series1.AddValue(valueX1, valueY1);
    series1.AddValue(valueX2, valueY2);
    series1.AddValue(valueX3, valueY3);
  }
```

Содержит параметры:

valueX – координата точки на линии по оси X. Тип параметра – дата и время или число.

valueY – координата точки на линии по оси Y. Тип параметра – число.

- e.Chart.Axis.X.AxisType – тип оси: дата (DateTime) или число (Double).

  ВАЖНО. Если указан тип оси DateTime, то в методе AddValue() параметр valueX необходимо задавать в том же формате DateTime.
- e.Chart.Axis.X.Title – подпись к оси X;
- e.Chart.Axis.Y.Title – подпись к оси Y;
- e.Chart.Axis.Y.MaxValue – максимальное значение по оси Y. Если значение будет больше указанного, то график обрежется;
- e.Chart.Axis.Y.MinValue – минимальное значение по оси Y. Если значение будет меньше указанного, то график обрежется.

  ПРИМЕЧАНИЕ. Если значения MaxValue или MinValue не указаны, то добавляется отступ вверх от максимального значения графика или вниз от минимального значения. Значение отступа рассчитывается как 10% от разницы максимального и минимального значения.
- e.Chart.IsLegendVisible – отображение легенды для линейного графика. По умолчанию отображается.

Обработчик события «Получение значения» виджета «Динамика количества заданий»

```
  public virtual void GetActiveAssignmentsDynamicAssignmentsDynamicChartValue
    (Sungero.Domain.GetWidgetPlotChartValueEventArgs e)
{
  if (!Equals(_parameters.CarriedObjects,
Widgets.ActiveAssignmentsDynamic.CarriedObjects.All) && Users.Current.IsSystem
== true)
    return;
  // Получение точек на графике виджета.
```

```
  var points =
Docflow
.PublicFunctions.Module.GetActiveAssignmentsDynamicPoints(businessUnitIds,
departmentsIds, employee, periodBegin, periodEnd, unwrap, needFilter);
  if (!points.Any())
    return;
  // Тип оси X – дата.
  e.Chart.Axis.X.AxisType = AxisType.DateTime;
  // Подпись к оси Y. Берется из ресурсов модуля.
  e.Chart.Axis.Y.Title = Resources.WidgetActiveAssignmentsDynamicYAxisTitle;
  // Создание первой серии.
  var assignmentsInWork =
e.Chart.AddNewSeries(Resources.WidgetActiveAssignmentsDynamicSeriesAllTitle,
Colors.Charts.Color1);
  // Создание второй серии.
  var overdueAssignmentsInWork =
e
.Chart.AddNewSeries(Resources.WidgetActiveAssignmentsDynamicSeriesOverduedTitle,
Colors.Charts.Red);
  var maxValue = points.Select(p => p.ActiveAssignmentsCount).Max();
  var minValue = points.Select(p => p.ActiveOverdueAssignmentsCount).Min();
  // Заполнение первой и второй серии значениями.
  foreach (var point in points)
  {
    assignmentsInWork.AddValue(point.Date, point.ActiveAssignmentsCount);
    overdueAssignmentsInWork.AddValue(point.Date,
point.ActiveOverdueAssignmentsCount);
  }
  // Округление значений по оси Y.
  if (minValue > Math.Round(maxValue * 0.1))
    e.Chart.Axis.Y.MinValue = Math.Round(minValue * 0.95);
  else
    // Минимальное значение по оси Y.
    e.Chart.Axis.Y.MinValue = 0;
    // Максимальное значение по оси Y.
    e.Chart.Axis.Y.MaxValue = Math.Round(maxValue * 1.05);
}
```

В результате на графике будут отображаться две линии, которые показывают динамику количества заданий (с нарушением срока и без нарушения). При нажатии на значение графика появится подсказка.

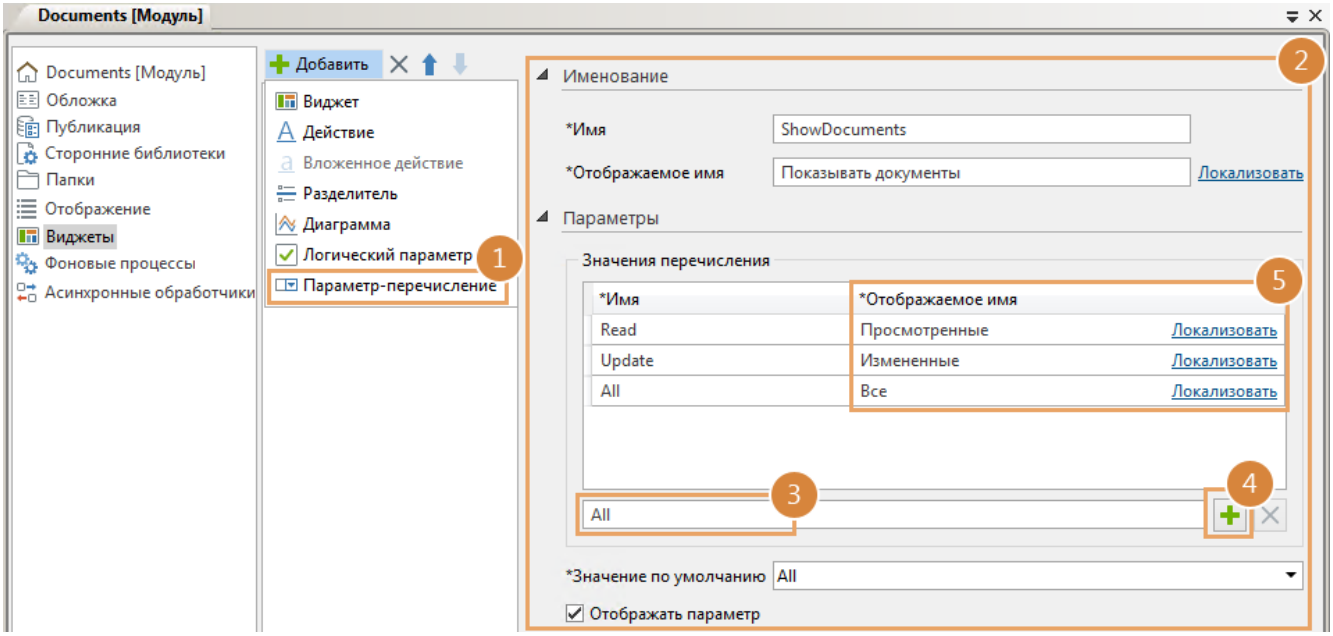
Как добавить параметр-перечисления

Предположим, разрабатывается виджет «Мои используемые документы» с действиями «Все время», «Сегодня» и «Вчера». Необходимо, чтобы у пользователя была возможность выбирать, какие данные будут отображаться в действиях виджета:

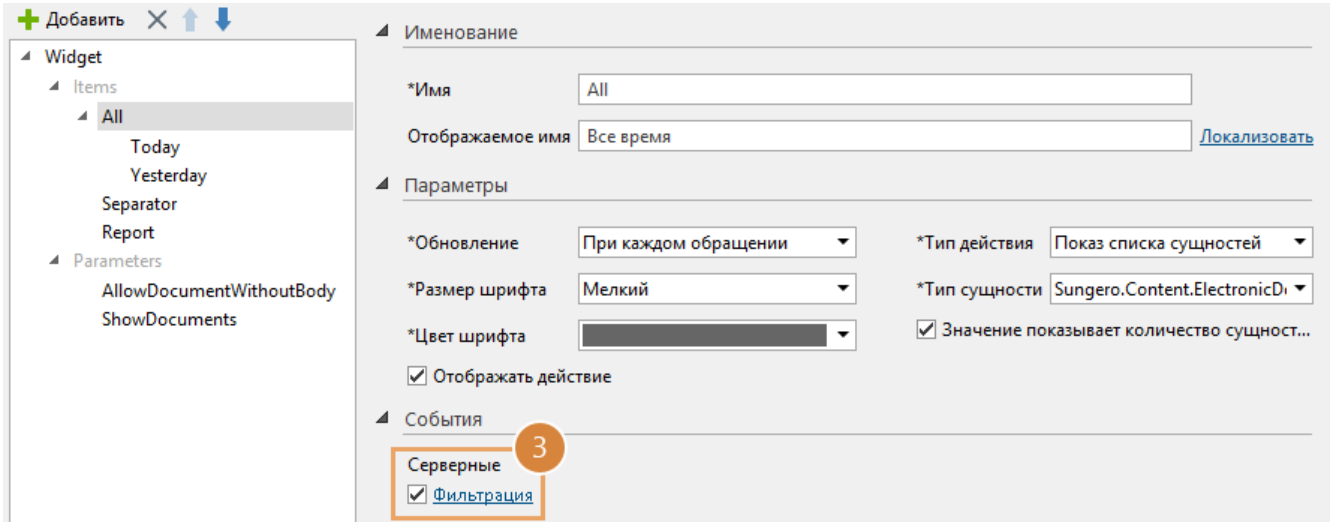
- только по документам, открытым на просмотр;
- только по документам, открытым на редактирование;
- по всем документам.

В этом случае добавьте для виджета параметр-перечисление с возможными значениями **Просмотренные**, **Измененные** и **Все**. Для этого:

1. В редакторе модуля перейдите в узел «Виджеты» и создайте параметр-перечисление.



2. В списке элементов выделите добавленный элемент и задайте настройки параметра.
3. Добавьте возможные значения параметра. Для этого в поле для ввода укажите имя значения.
4. Нажмите на кнопку . В табличной части будет создана новая запись. В колонке ***Имя** будет указанно введенное значение.
5. В колонке ***Отображаемое имя** введите имя значения, которое будет отображаться в проводнике системы. Для *локализации* отображаемого имени перейдите по ссылке **Локализовать**.
6. Чтобы данные в действиях виджета фильтровались в зависимости от значения параметра, задайте обработчик события **Фильтрация**. Для этого в списке элементов выделите элемент **Items** и выберите действие «Все время». Установите флажок **Фильтрация** и перейдите по ссылке. Откроется редактор программного кода.



Аналогичным образом задайте обработчик события **Фильтрация** для действий «Сегодня» и «Вчера».

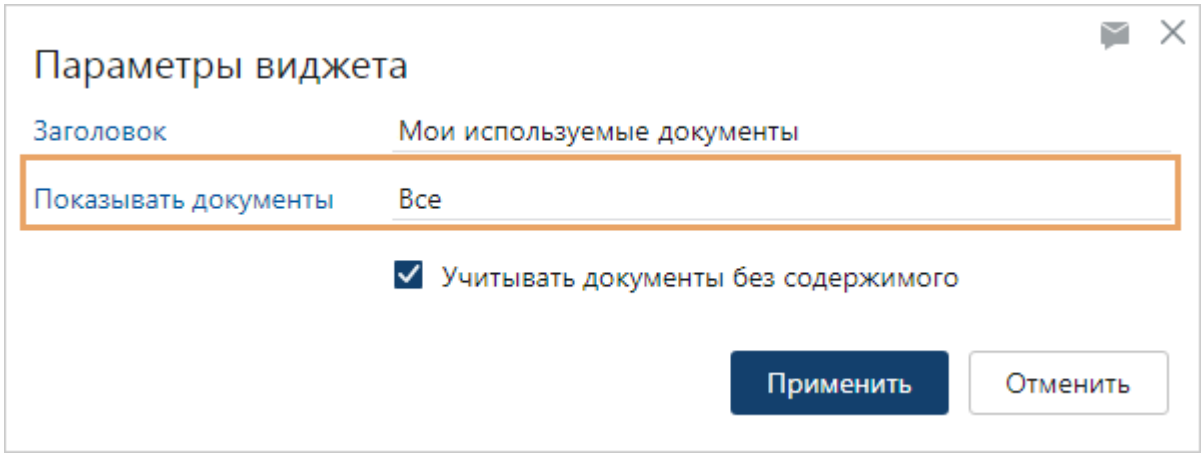
ПРИМЕЧАНИЕ. В зависимости от типа действия виджета для него могут быть доступны обработчики событий **Получение значения** и **Выполнение**.

Обработчик события «Фильтрация» для действия «Все время» с учетом параметра «Показывать документы»

```
   /// <summary>
/// Формирование списка документов, который отображается
/// при нажатии на действие "Все время".
/// </summary>
public virtual IQueryable<Sungero.Content.IElectronicDocument>
WidgetAllTimeFiltering(IQueryable<Sungero.Content.IElectronicDocument> query)
{
  // Если для параметра-перечисления "Показывать документы" установлено
  // значение "Просмотренные", то в виджете отображается информация
  // по документам, которые пользователь открывал на просмотр.
  if (_parameters.Show == Docflow.Widgets.Widget.Show.Read)
    query = query.WhereDocumentHistory(h => h.Action ==
Sungero.CoreEntities.History.Action.Read);
  // Если для параметра-перечисления "Показывать документы" установлено
  // значение "Измененные", то в виджете отображается информация
  // по документам, которые пользователь открывал на редактирование.
  if (_parameters.Show == Docflow.Widgets.Widget.Show.Update)
```

```
    query = query.WhereDocumentHistory(h => h.Action ==
Sungero.CoreEntities.History.Action.Update);
  // Если для параметра-перечисления "Показывать документы" установлено
  // значение "Все" и для логического параметра
  // "Учитывать документы без содержимого" установлено значение False,
  // то в виджете отображается информация по всем документам
  // которые открывал пользователь.
  return query
    .WhereDocumentHistory(h => Equals(Users.Current, h.User))
    .Where(d => IsOftenUsed(d));
}
```

В результате для виджета «Мои используемые документы» будет добавлен параметр-перечисление:

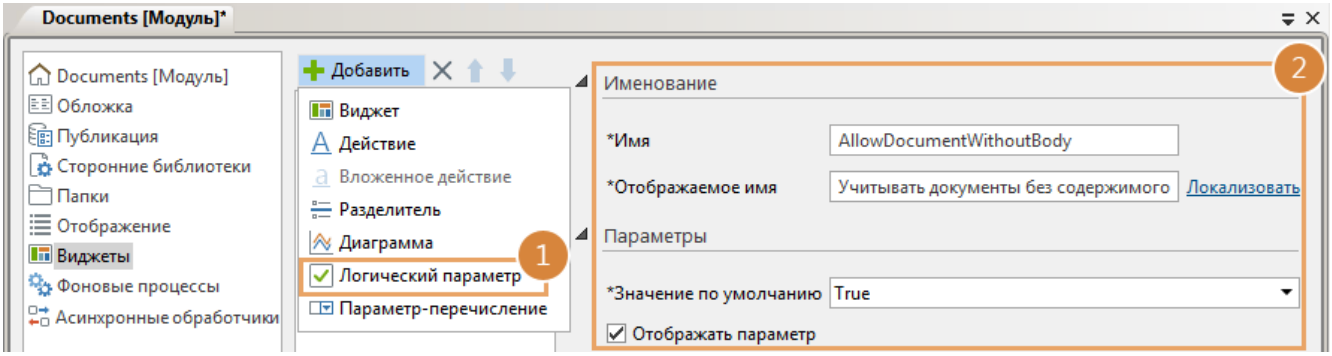


В зависимости от значения параметра, которое указал пользователь, в действиях виджета будут отображаться данные по всем документам, только по документам, которые были открыты на просмотр или на редактирование.

Как добавить параметр логического типа

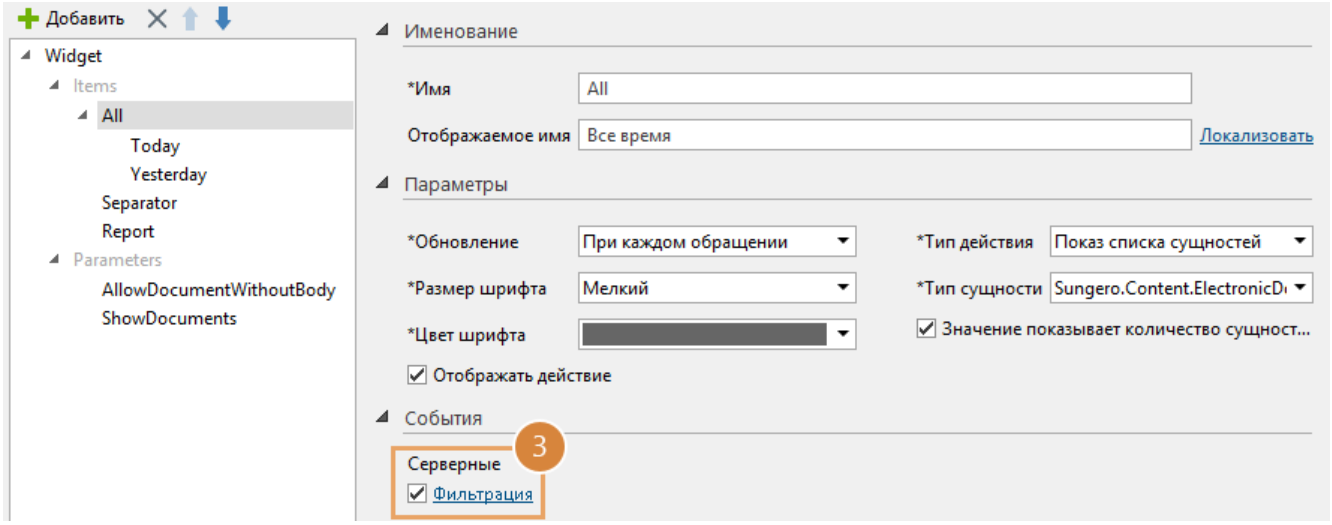
Предположим, разрабатывается виджет «Мои используемые документы» с действиями «Все время», «Сегодня» и «Вчера». Необходимо, чтобы у пользователя была возможность выбирать, какие данные будут отображаться в действиях виджета: по всем документам или только по документам, у которых есть содержимое. Для этого создайте логический параметр:

1. В редакторе модуля перейдите в узел «Виджеты» и создайте логический параметр.



2. В списке элементов выделите добавленный элемент и задайте настройки параметра.

3. Чтобы данные в действиях виджета фильтровались в зависимости от значения параметра, задайте обработчик события **Фильтрация**. Для этого в списке элементов выделите элемент **Items** в выберите действие «Все время». Установите флажок **Фильтрация** и перейдите по ссылке. Откроется редактор программного кода.



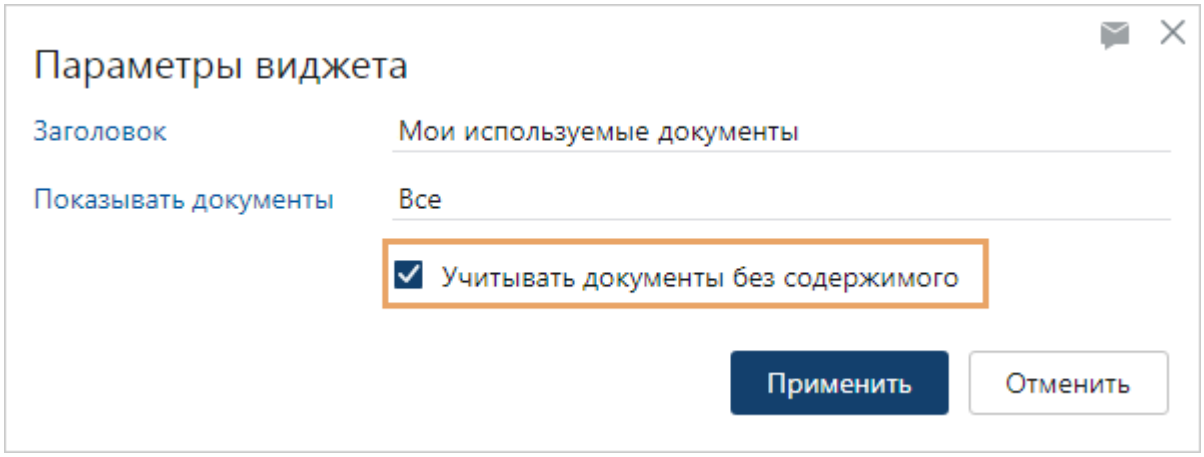
Аналогичным образом задайте обработчик события **Фильтрация** для действий «Сегодня» и «Вчера».

ПРИМЕЧАНИЕ. В зависимости от типа действия виджета для него могут быть доступны обработчики событий **Получение значения** и **Выполнение**.

Обработчик события «Фильтрация» для действия «Все время» с учетом параметра «Учитывать документы без содержимого»

```
   /// <summary>
/// Формирование списка документов, который отображается
/// при нажатии на действие "Все время".
/// </summary>
public virtual IQueryable<Sungero.Content.IElectronicDocument>
WidgetAllTimeFiltering(IQueryable<Sungero.Content.IElectronicDocument> query)
{
  // Если для логического параметра
  // "Учитывать документы без содержимого" установлено значение True,
  // то в виджете отображается информация только по документам,
  // у которых есть версии.
  if (!_parameters.AllowDocumentWithoutBody)
    query = query.Where(h => h.HasVersions);
  // Если для логического параметра
  // "Учитывать документы без содержимого" установлено значение False,
  // то в виджете отображается информация по всем документам
  // которые открывал пользователь.
  return query
    .WhereDocumentHistory(h => Equals(Users.Current, h.User))
    .Where(d => IsOftenUsed(d));
}
```

В результате для виджета «Мои используемые документы» будет добавлен логический параметр:



В зависимости от значения параметра, которое указал пользователь, в действиях виджета будут отображаться данные по всем документам или только по документам, у которых есть содержимое.

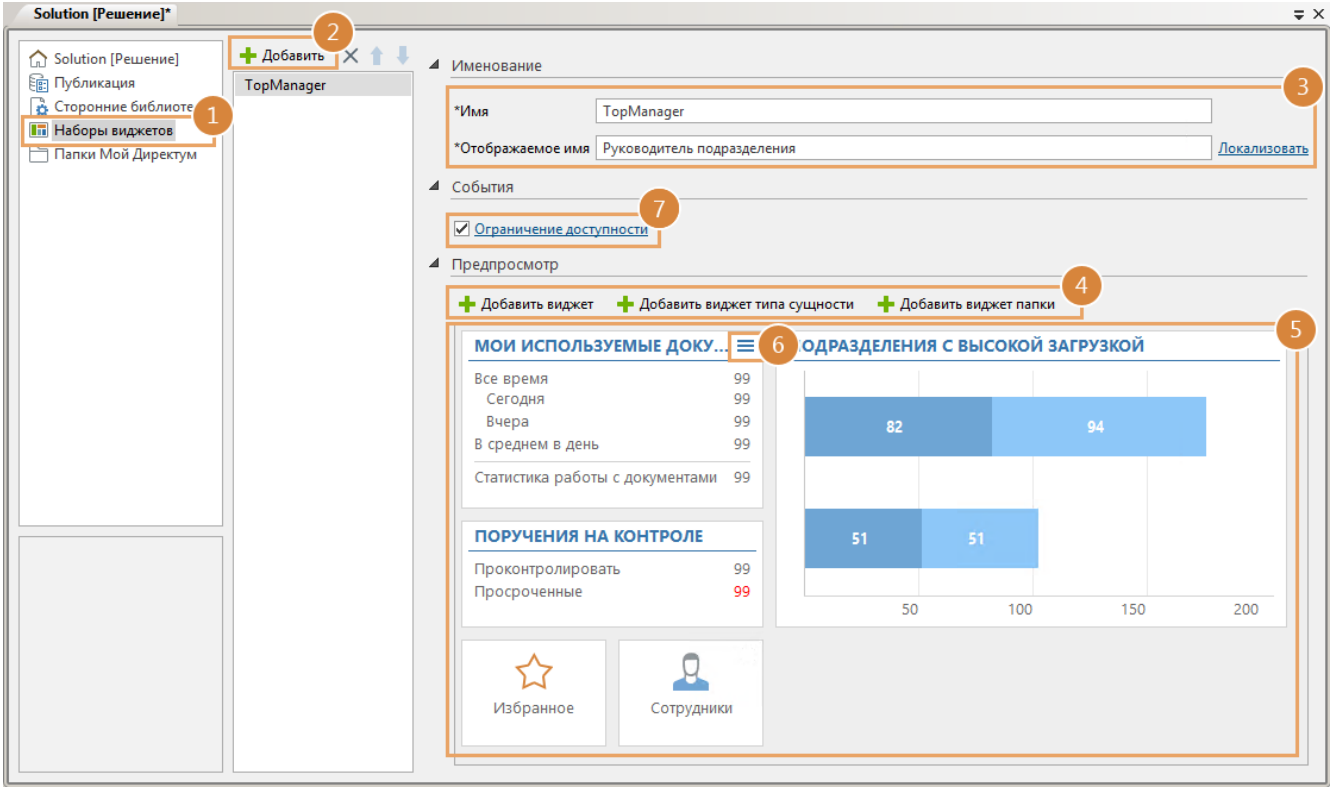
Как создать набор виджетов, который доступен только руководителям

Предположим, необходимо создать набор виджетов, который в проводнике системы доступен только руководителям подразделений. В набор нужно добавить виджеты:

- «Мои используемые документы», разработан в новом решении. У виджета есть параметр-перечисление **Показывать документы**. Возможные значения параметра: **Все**, **Измененные**, **Просмотренные**. По умолчанию отображается информация по всем документам. Необходимо, чтобы в новом наборе в виджете отображалась информация только по документам, которые были открыты для изменения;
- «Подразделения с высокой загрузкой» и «Поручения на контроле», которые уже есть в базовом решении;
- для перехода в список сотрудников;
- для открытия папки «Избранное».

Для этого:

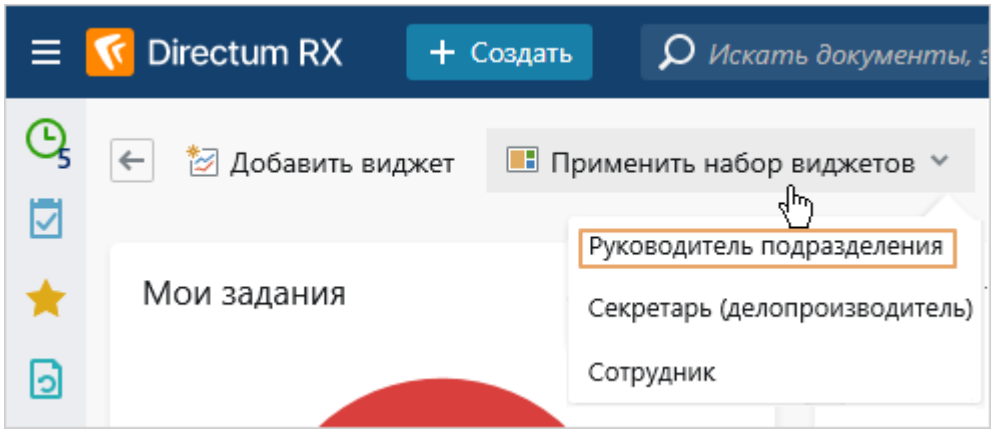
1. В редакторе решения перейдите в узел «Наборы виджетов».



2. Нажмите на кнопку **Добавить**.
3. В списке элементов выделите добавленный элемент и задайте настройки:

   ***Имя** набора виджетов. Должно быть уникальным в рамках одного решения, состоять из букв латинского алфавита и цифр.

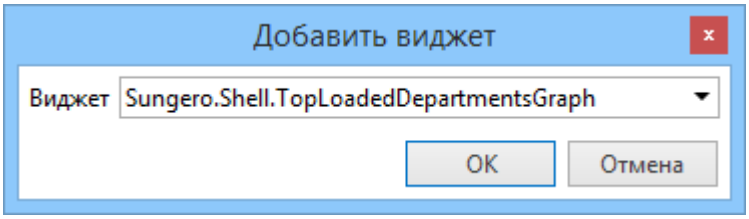
   ***Отображаемое имя** набора виджетов, которое будет отображаться в качестве названия набора в проводнике системы.



   Для *локализации* отображаемого имени перейдите по ссылке **Локализовать**.

4. Добавьте в набор необходимые виджеты. Для этого используйте кнопки:

   Добавить виджет. Один из разработанных виджетов. При нажатии на кнопку откроется окно:

   В выпадающем списке **Виджет** для выбора доступны виджеты базового решения и виджеты, разработанные в новом решении. Выберите нужный и нажмите на кнопку **ОК**.

   Добавить виджет типа сущности. Виджет, который представляет собой ссылку на один из список типов сущности. При нажатии на кнопку откроется окно для выбора типа сущности. В выпадающем списке выберите тип сущности «Сотрудники» (Sungero.Company.Employee) и нажмите на кнопку **ОК**.

   Добавить виджет папки. Виджет, который представляет собой ссылку на одну из папок. При нажатии на кнопку откроется окно для выбора вычисляемой папки. В выпадающем списке выберите папку «Избранное» (Favorites) и нажмите на кнопку **ОК**.

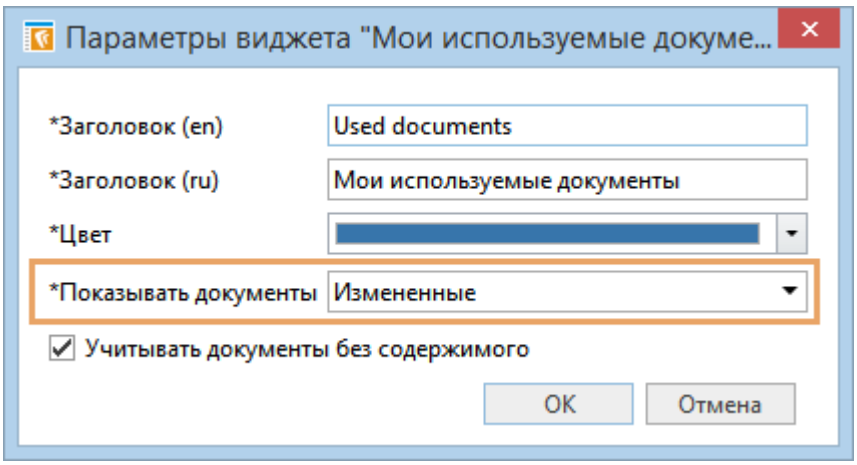
   Добавленные виджеты отобразятся в области предпросмотра набора виджетов.

5. Настройте расположение виджетов в наборе. Для этого перемещайте виджеты в области предпросмотра с помощью мыши.

6. По умолчанию в виджете «Мои используемые документы» отображается информация по всем документам. Необходимо, чтобы в нем отображалась информация только по документам, которые были открыты для изменения. Для этого измените значение параметра виджета **Показывать документы**.

   ВАЖНО. Состав параметров зависит от виджета и настраивается в редакторе модуля в узле «Виджеты».

   Чтобы изменить значение параметра, в области предпросмотра в виджете нажмите на кнопку **Параметры**. Откроется окно настройки:

   В выпадающем списке **Показывать документы** выберите значение **Измененные** и нажмите на кнопку **ОК**.

7. Задайте обработчик события **Ограничение доступности**, чтобы определить состав пользователей, которым доступен данный набор виджетов. Для этого установите флажок **Ограничение доступности** и перейдите по ссылке. Откроется редактор программного кода. Задайте ограничение по вхождению в роль «Руководители подразделений».

   ВАЖНО. Если обработчик события **Ограничение доступности** не задан, то набор виджетов будет доступен для выбора всем пользователям.

   Пример. Обработчик события **Ограничение доступности** для набора виджетов руководителя:

```
internal static class DesktopViewHandlers
    {
      /// <summary>
      /// Проверить вхождение текущего пользователя
      /// в роль руководителей подразделений.
      /// </summary>
      /// <returns> True, если входит, иначе False.</returns>
      private static bool CanApplyTopManager()
      {
        return IncludedInRole("Руководители подразделений");
      }
      /// <summary>
      /// Проверить вхождение текущего пользователя в роль.
      /// </summary>
      /// <param name="roleName"> Имя роли.</param>
      /// <returns> True, если входит, иначе False.</returns>
      public static bool IncludedInRole(string roleName)
      {
        var ceoRole = Roles.GetAll(r => r.Name == roleName).FirstOrDefault();
        if (ceoRole == null)
          return false;
        var employee = Users.Current;
        return employee.IncludedIn(ceoRole);
      }
    }
```

В результате в проводнике системы появится новый набор виджетов, который будет доступен для пользователей, входящих в роль «Руководители подразделений».

Скрипты конвертации

В разделе описаны примеры скриптов конвертации, которые необходимо писать, если:

- появилось свойство логического типа
- свойство стало обязательным
- изменился тип данных свойства

Появилось свойство логического типа

Предположим, в справочник «Персональные настройки» (PersonalSetting) добавлено новое свойство **Показывать подписи с результатом «Не согласовано»**. Имя свойства в базе

данных – ShowNotApprove.

Конвертация необходима, чтобы:

- быстро и массово заполнить свойство значением по умолчанию. При необходимости пользователи могут изменить значение вручную;
- система работала стабильно и не требовала от пользователя дополнительных действий.

Добавьте скрипт **После публикации (MS SQL)** в модуль «Документооборот» (Docflow):

```
update
  Sungero_Docflow_PersonSetting
set
  ShowNotApprove = 0
where
  ShowNotApprove is null
```

Многопроходность достигается за счет фильтрации записей, в которых поле ShowNotApprove уже заполнено.

Свойство стало обязательным

Предположим, в справочнике «Право подписи» (SignatureSetting) свойство **Приоритет** стало обязательным. Имя свойства в базе данных – Priority.

Конвертация необходима, чтобы:

- быстро и массово заполнить свойство **Приоритет** значением по умолчанию;
- оптимизировать работу фоновых процессов, бизнес-процессов, интеграцию с сервисами обмена, механизм синхронизации с 1С.

Добавьте скрипт **После публикации (MS SQL)** в модуль «Документооборот» (Docflow):

```
update
  Sungero_Docflow_SignSettings
set
  Priority = '0'
from
  Sungero_Docflow_SignSettings
where
  Priority is null
```

Изменился тип данных свойства

Предположим, в справочнике «Право подписи» (SignatureSetting) у свойства **Основание** меняется тип со строки на перечисление. Имя свойства в базе данных – Reason.

В процессе публикации модуля будет удалено свойство типа **Строка** и создастся свойство типа **Перечисление**.

Конвертация необходима, чтобы перенести данные сначала из старого поля с типом **Строка** во временную таблицу (скрипт до публикации), а потом из временной таблицы в новое поле с типом **Перечисление** (скрипт после публикации). Таким образом исключается потеря данных, и они преобразуются по заданному алгоритму.

Добавьте скрипт **До публикации (MS SQL)** в модуль «Документооборот» (Docflow):

```
   if exists (select * from information_schema.tables where table_name =
      'SignatureSettingsReason')

  drop table SignatureSettingsReason

if exists (select * from information_schema.COLUMNS where table_name =
'Sungero_Docflow_SignSettings' and COLUMN_NAME = 'Reason')

   and not exists (select * from information_schema.COLUMNS where table_name =
'Sungero_Docflow_SignSettings' and COLUMN_NAME = 'DocumentInfo')

begin

  execute('create table SignatureSettingsReason(Id bigint, Reason nvarchar(250))

           insert into SignatureSettingsReason

           select s.Id, s.Reason from Sungero_Docflow_SignSettings s')

end
```

Добавьте скрипт **После публикации (MS SQL)** в модуль «Документооборот» (Docflow):

```
   if exists (select * from information_schema.tables where table_name =
      'SignatureSettingsReason')

    and exists (select * from information_schema.COLUMNS where table_name =
'Sungero_Docflow_SignSettings'

and COLUMN_NAME = 'DocumentInfo')

    and exists (select * from information_schema.COLUMNS where table_name =
'Sungero_Docflow_SignSettings'

 and COLUMN_NAME = 'Reason')

begin

  update

    s

  set

    Reason =

      case

        when lower(tmp.Reason) in ('устав', 'charter')

        then 'Duties'

        else 'Other'

      end,
```

```
    DocumentInfo =
      case
        when lower(tmp.Reason) in ('устав', 'charter')
        then ''
        else tmp.Reason
      end
  from SignatureSettingsReason tmp
    join Sungero_Docflow_SignSettings s on tmp.Id = s.Id
  drop table SignatureSettingsReason
end
```

Локализация

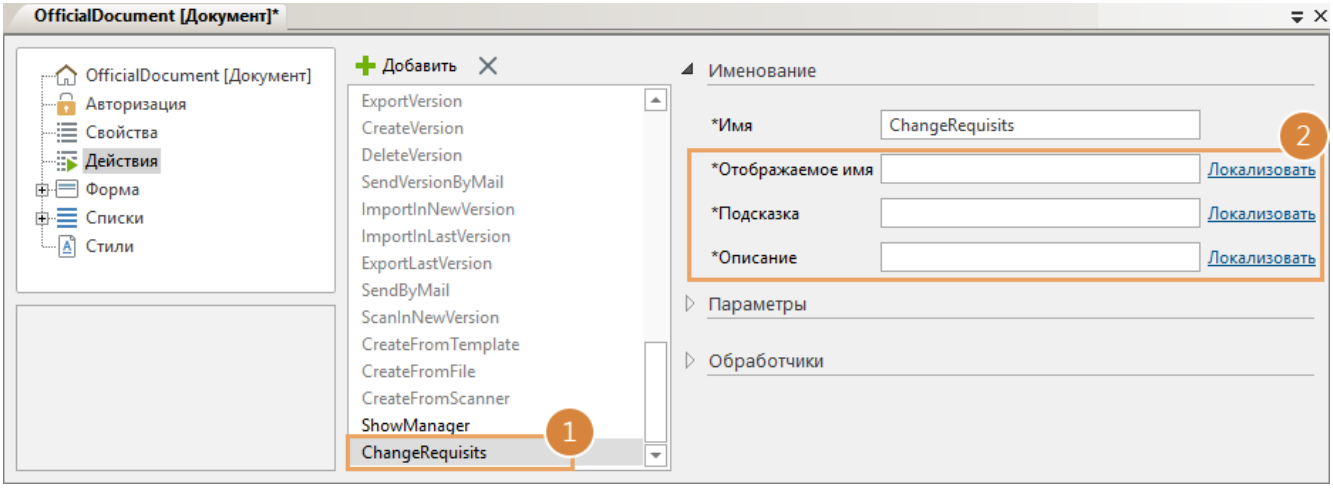
В разделе содержатся примеры:

- Как локализовать отображаемое имя, подсказку и описание действия?
- Как добавить сообщение валидации и локализовать его?
- Как изменить локализацию в перекрытых элементах разработки?

Как локализовать отображаемое имя, подсказку и описание действия

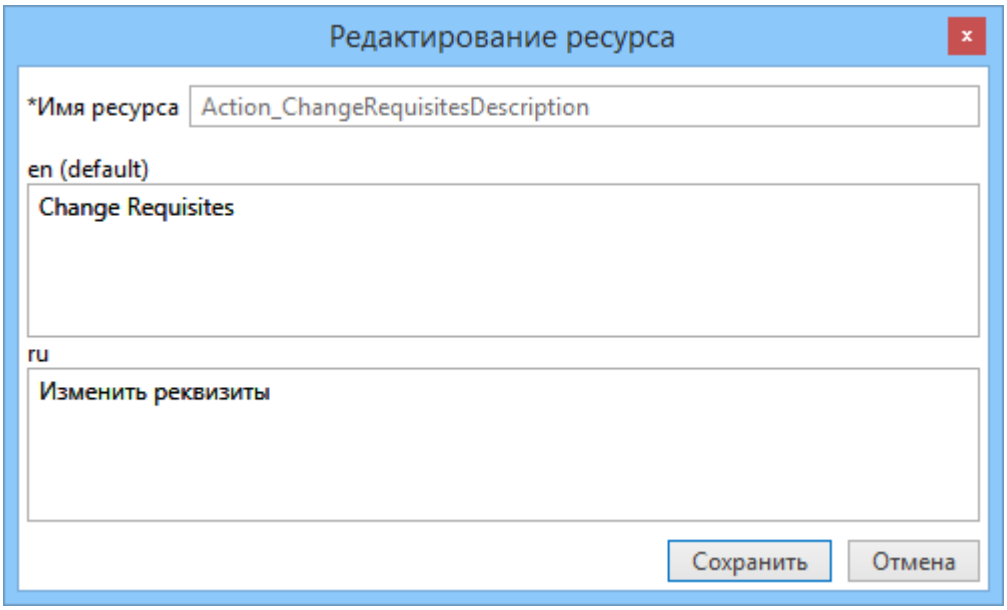
Предположим, отображаемое имя, подсказку и описание действия для официального документа необходимо указать на русском и английском языках. Для этого:

1. В редакторе типа документа в узле «Действия» выделите нужное действие.

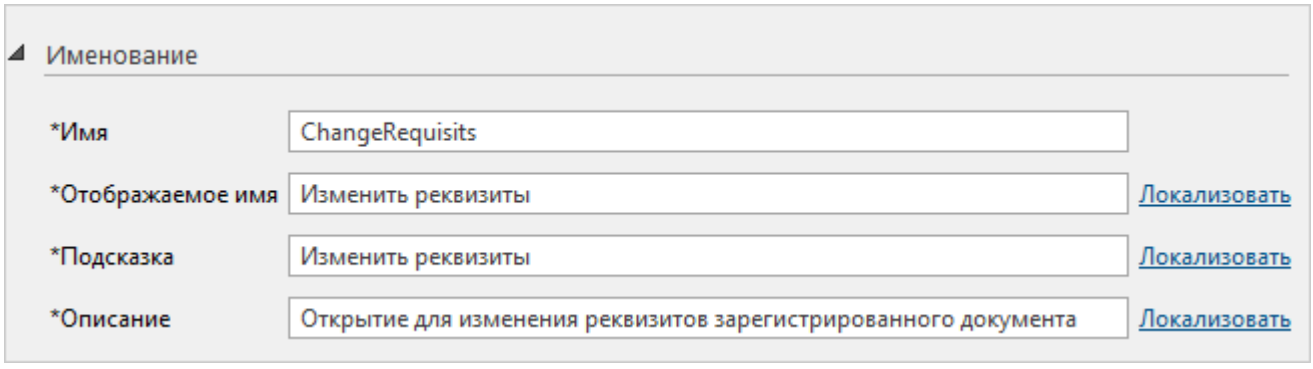


2. Для локализации имени, описания и подсказки перейдите по ссылке **Локализовать**.

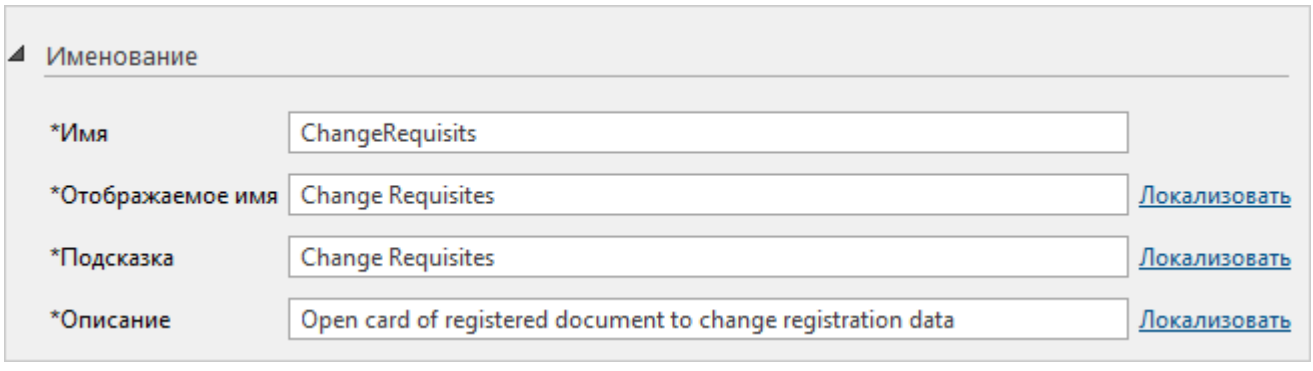
3. В открывшемся окне заполните поля **en (default)**, **ru** и нажмите на кнопку **Сохранить**. Имя ресурса генерируется автоматически и доступно только для просмотра. Например:



ВАЖНО. В среде разработки в полях ***Отображаемое имя**, ***Подсказка**, ***Описание** и т.п. отображаются значения, которые введены в поле **en (default)**. В справке на рисунках для наглядности и понятности используются русские значения:

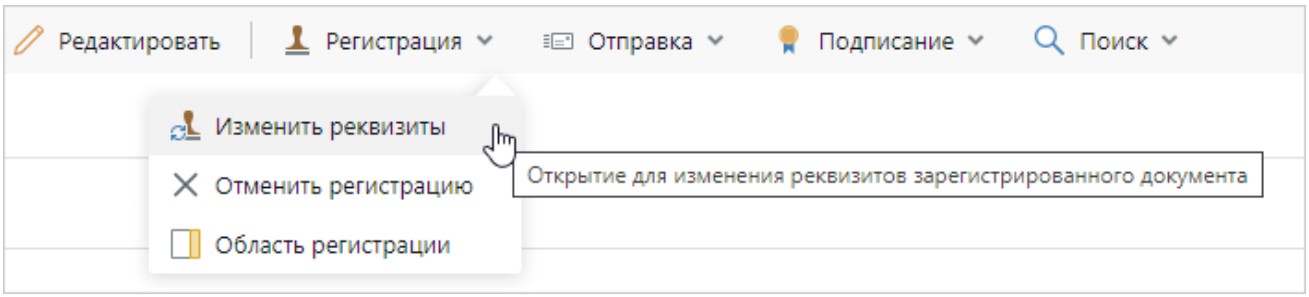


У вас при правильном заполнении полей будут отображаться английские значения:

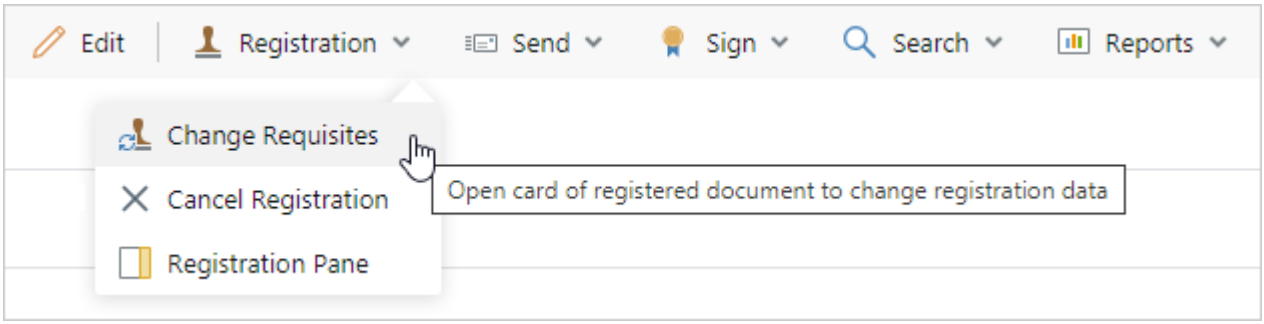


В результате имя действия, подсказка и описание будут отображаться в проводнике системы следующим образом:

Отображение на русском языке



Отображение на английском языке



ПРИМЕЧАНИЕ. Описание и отображаемое имя действия также можно локализовать на уровне пользовательских настроек без применения среды разработки. Для этого используйте редактор форм в справочнике **Представления форм**. Подробнее см. в руководстве администратора, разделы «Настройка форм карточек» и «Локализация полей».

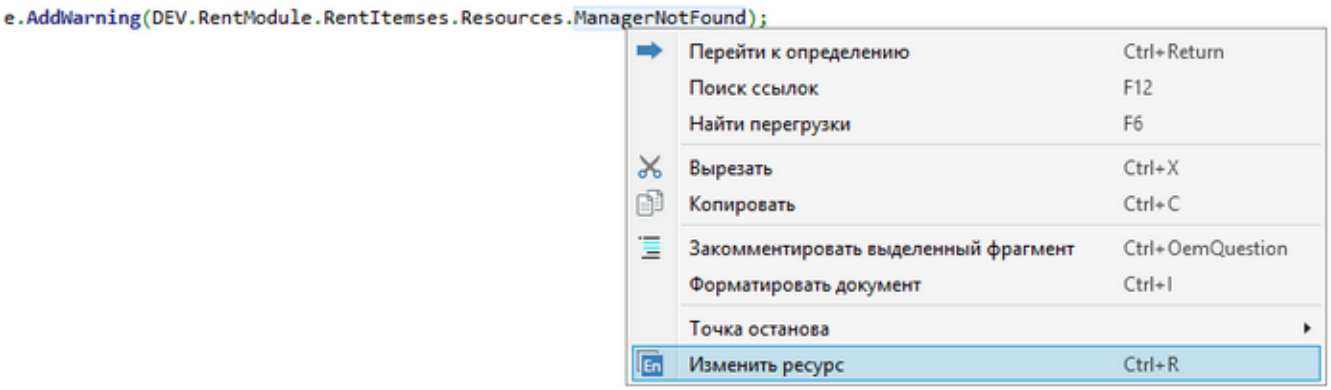
Как добавить сообщение валидации и локализовать его

Предположим, для действия написан программный код, в котором проверяется, заполнено ли в карточке справочника поле «Руководитель». Если поле не заполнено, пользователю отображается сообщение валидации. Чтобы сразу добавить сообщение в *ресурсы* и локализовать его:

1. В программном коде установите курсор на сообщении валидации.
2. Нажмите сочетание клавиш CTRL+R или воспользуйтесь пунктом контекстного меню **Создать ресурс**.
3. В открывшемся окне заполните поля ***Имя ресурса**, ***en (default)**, ***ru** и нажмите на кнопку **Сохранить**.

В результате в программном коде текст сообщения заменится на полный путь до ресурса. В редакторе ресурсов элемента разработки, для которого писали программный код, появится строка с ресурсом. Добавленный ресурс будет иметь тип LocalizedString (локализованная строка).

Если вы хотите посмотреть или изменить значения ресурса, установите курсор на имени ресурса и нажмите сочетание клавиш CTRL+R или воспользуйтесь пунктом контекстного меню **Изменить ресурс**:



В открывшемся окне можно посмотреть значения ресурса и при необходимости поправить их.

Как изменить локализацию в перекрытых элементах разработки

В перекрытых элементах разработки можно изменять значения ресурсов, которые унаследованы от элементов *базового решения*.

Изменение локализации имени для группы контролов

ПРИМЕЧАНИЕ. Отображаемое имя для группы контролов также можно локализовать на уровне пользовательских настроек без применения среды разработки. Для этого используйте редактор форм в справочнике **Представления форм**. Подробнее см. в руководстве администратора, разделы «Настройка форм карточек» и «Локализация полей».

На форме базового типа документа «Договор» есть группа контролов с отображаемым именем «Основное» (Main). Чтобы изменить локализацию отображаемого имени группы:

1. Перекройте тип документа «Договор» (Sungero.Contracts.Contract). В редакторе вашего решения добавьте зависимость от модуля Sungero.Contracts.
2. В дереве перекрытого типа документа в узле «Форма» выделите группу контролов Main. На панели свойств нажмите на кнопку **Локализовать** рядом с полем ***Отображаемое имя**. Откроется редактор ресурсов.
3. На закладках **Default** и **ru** измените прежние значения строки локализации на новые.
4. Сохраните изменения.

После публикации на форме появится новое отображаемое имя группы.

Изменение локализации сообщения валидации

В справочнике **Регион**, в серверном событии До сохранения есть программный код для проверки на уникальность кода региона. Если код не уникален, то пользователю выводится сообщение валидации. Имя ресурса в справочнике базового решения Directum RX – Regions.Resources.CodeDuplicate. Чтобы изменить локализацию сообщения:

1. Перекройте справочник **Регион** (Sungero.Commons.Region). В редакторе вашего решения добавьте зависимость от модуля Sungero.Commons.
2. В редакторе справочника **Регион** базового решения Directum RX откройте программный код серверного события **До сохранения:**

```
public override void BeforeSave(Sungero.Domain.BeforeSaveEventArgs e)
    {
      // Проверить код региона на уникальность.
      if (!string.IsNullOrWhiteSpace(_obj.Code))
      {
        var region = Regions.GetAll().FirstOrDefault(r => r.Code == _obj.Code &&

    r.Country.Equals(_obj.Country) &&
                                                        r.Status !=
    CoreEntities.DatabookEntry.Status.Closed);
        if (region != null && !region.Equals(_obj))
          e.AddWarning(_obj.Info.Properties.Code,
    Regions.Resources.CodeDuplicate));
    }
```

3. Скопируйте код и вставьте в серверное событие **До сохранения** перекрытого справочника **Регион** вместо кода **base.BeforeSave(e)**.
4. Вместо пути обращения к ресурсу напишите желаемый текст сообщения. Например:

Было	Стало
`e.AddWarning (_obj.Info.Properties.Code, Regions.Resources.CodeDuplicate))`	`e.AddWarning(_obj.Info.Properties.Code, "Код уже существует"))`

5. С помощью горячих клавиш CTRL+R или контекстного меню **Создать ресурс** добавьте новое локализованное сообщение валидации в редактор ресурсов.

После публикации, если пользователь введет неверное значение кода региона, отобразится новое сообщение *валидации*.

Структура хранения данных

При создании *типа сущности* в базе данных (БД) генерируется новая таблица или в существующую таблицу предка добавляется колонка для хранения свойств сущности. От типа сущности зависит, в какой таблице будут храниться данные:

- данные типов документов, задач, заданий и уведомлений хранятся в уже существующих предопределенных таблицах. При создании типа сущности в таблицы добавляются колонки для хранения данных свойств, добавленных в наследниках или перекрытиях. Например, данные всех документов хранятся в таблице **Sungero_Content_EDoc**;
- данные справочников, которые являются наследниками абстрактного справочника, хранятся в таблице этого абстрактного справочника. В нее добавляются колонки для хранения данных свойств, добавленных в наследниках или перекрытиях. Например, в таблице **Sungero_Core_Recipient** хранятся данные наследников справочника **Recipient** (User, Group, Employee и др.). В таблице **Sungero_Parties_Counterparty** хранятся данные наследников справочника **Counterparty** (Person, Bank, Company);
- данные всех остальных справочников, а также дочерних *свойств-коллекций* любых типов сущностей хранятся в отдельных уникальных таблицах. Например, для свойства-коллекции **Milestones** («Контрольные точки») типа сущности ContractualDocument («Договорной документ») есть своя таблица **Sungero_Contrac_ContractMilest**.

Названия таблиц и колонок строятся по определенным форматам, при этом в системе обеспечивается их уникальность. Названия таблиц используются, например, при разработке отчетов для обращения к данным через SQL-запросы.

С помощью среды разработки можно определить названия таблиц для прикладных модулей, а также понять назначение таблицы и найти описание ее колонок (свойств). В руководстве администратора Directum RX приведен пример стандартной прикладной таблицы Sungero_Docflow_Params, с помощью которой можно изменить настройки документооборота: настройка правил назначения прав, ознакомления, сравнения документов и пр. Кроме того, в базе данных есть таблицы для системных и предметных модулей, которые не отображаются в среде разработки. Подробнее см. «Примеры системных таблиц».

Формат имен таблиц и колонок

Имена таблиц в базе данных строятся по шаблону:

Имя таблицы = <Код компании>_<Код модуля>_<Код типа сущности>

где указываются:

- **Код компании**, которая разработала модуль;
- **Код модуля**, в котором создан элемент разработки;
- **Код типа сущности** – код конкретного элемента разработки.

Имена колонок в базе данных строятся по шаблону:

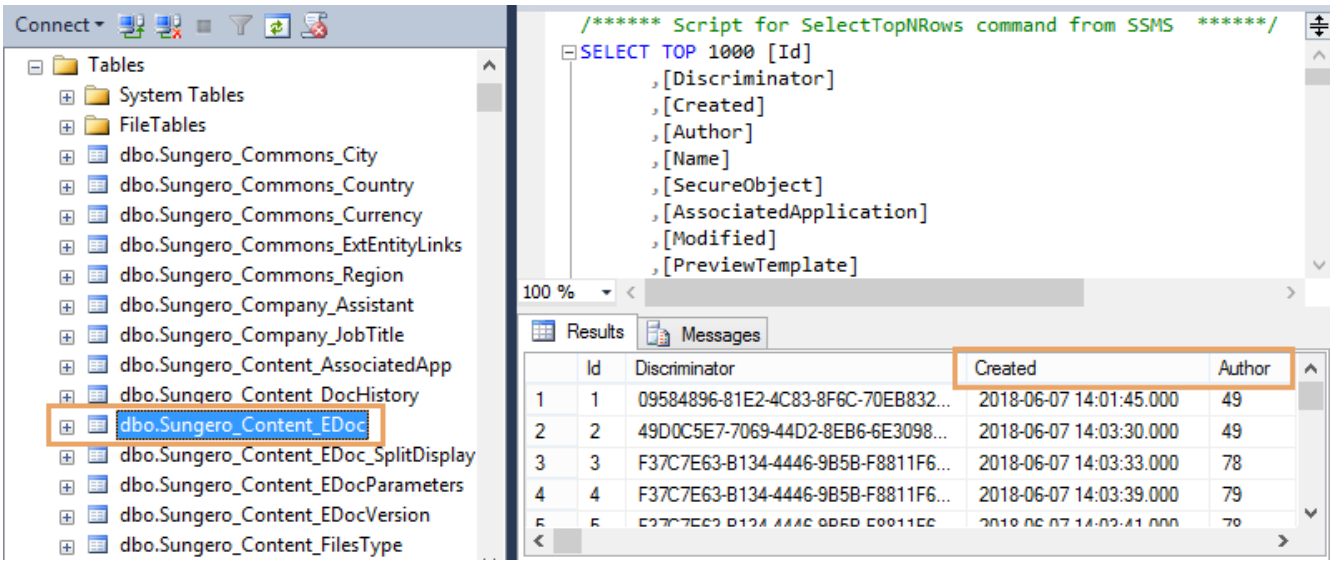
Имя колонки = <Код свойства>

Например, документы хранятся в таблице Sungero_Content_EDoc, ее название состоит из:

- Sungero – код компании, которая разработала модуль;
- Content – код модуля «Управление документами», к которому относится сущность;

- EDoc – код сущности «Документ».

В этой таблице имена колонок «Дата создания» и «Автор» будут **Created** и **Author**.

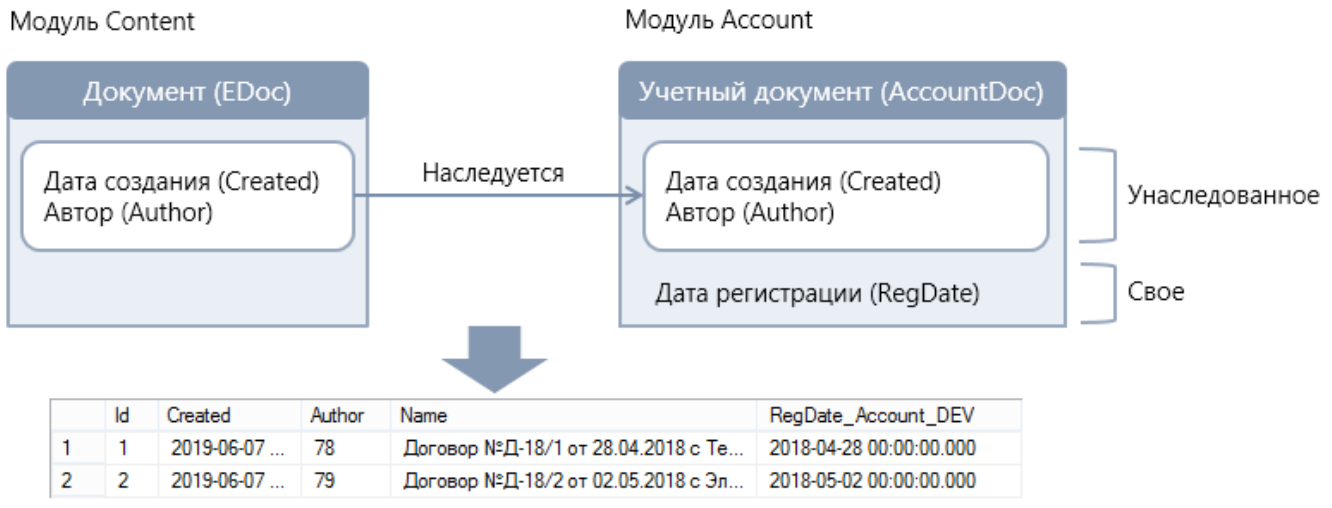


Если же тип сущности-наследник создан в другом модуле, отличном от модуля базового типа сущности, то для колонки используется трехкомпонентное имя:

Имя колонки = <Код свойства>_<Код модуля>_<Код компании>

А если тип сущности перекрыт, то вместо кода модуля используется **код решения**, в котором создано перекрытие.

Например, есть компания DEV, которая создала свой модуль Account, тип сущности «Учетный документ» и в нем свойство RegDate («Дата регистрации»). Сущность расположена в модуле, отличном от модуля «Управление документами» (Sungero.Content). Ее свойство будет размещаться в колонке с трехкомпонентным именем **RegDate_Account_DEV**.



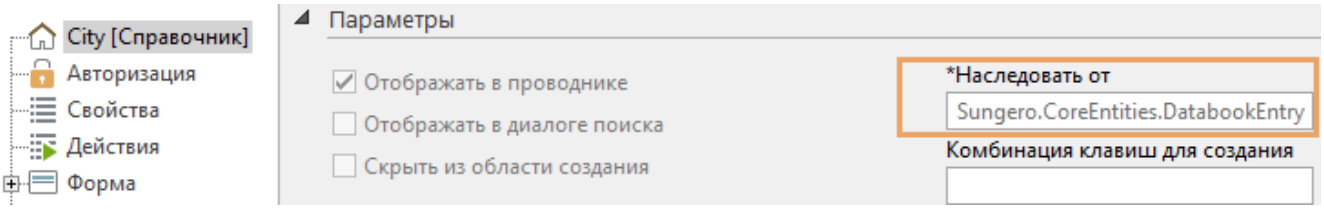
Пример. Как определить название таблицы и колонки

Есть компания DEV, которая создала свое решение Example. В нем перекрыт тип справочника «Населенные пункты» (City) и добавлено свойство **Валюта** (ValutaDEV). Необходимо определить, в какой таблице базы данных хранится справочник и в какой колонке находится добавленное свойство.

Решение:

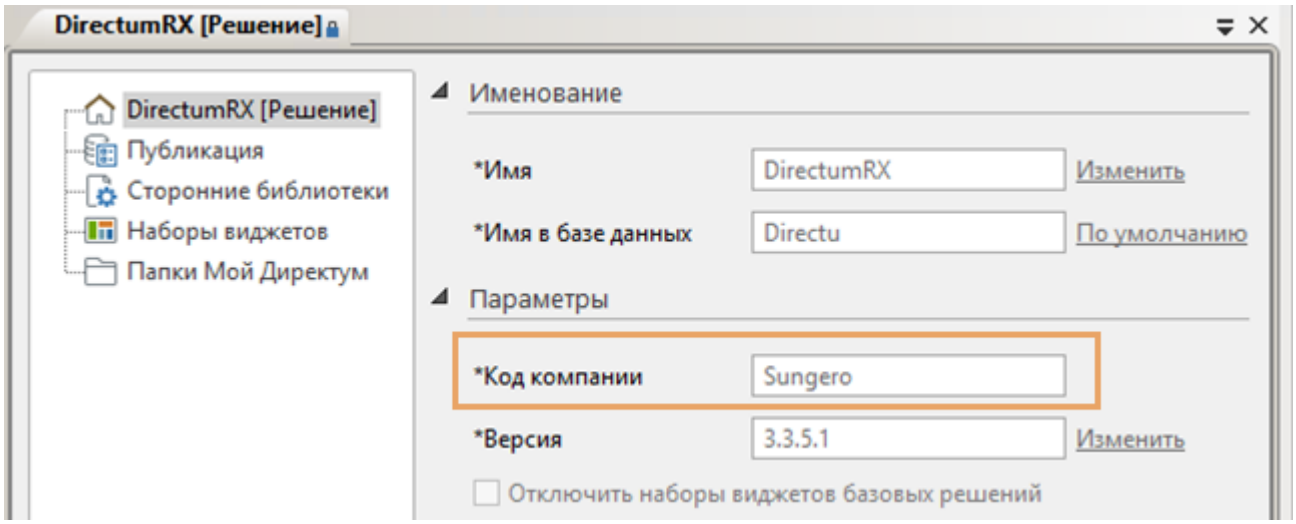
Справочник City относится к прикладному модулю Commons. Поэтому определить название таблицы и колонки в базе данных можно с помощью среды разработки.

1. Проверяем, наследником какого типа сущности является City. Если от User или Group, то значит для него будет использоваться общая таблица для всех наследников **Sungero_Core_Recipient**. Проверить можно в редакторе базового типа сущности:

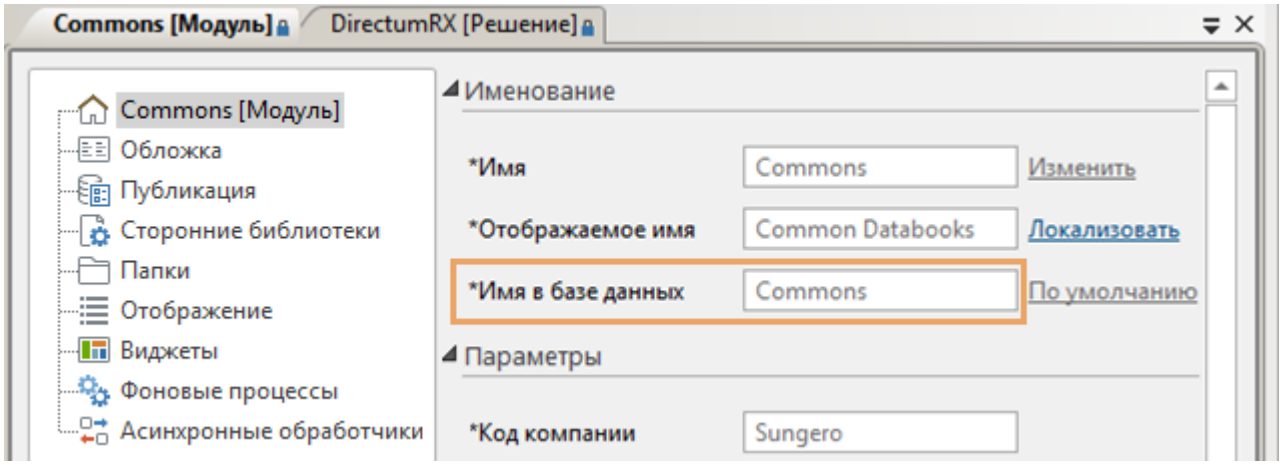


В данном случае City является наследником Sungero.CoreEntities.DatabookEntry. Поэтому для справочника есть своя уникальная таблица.

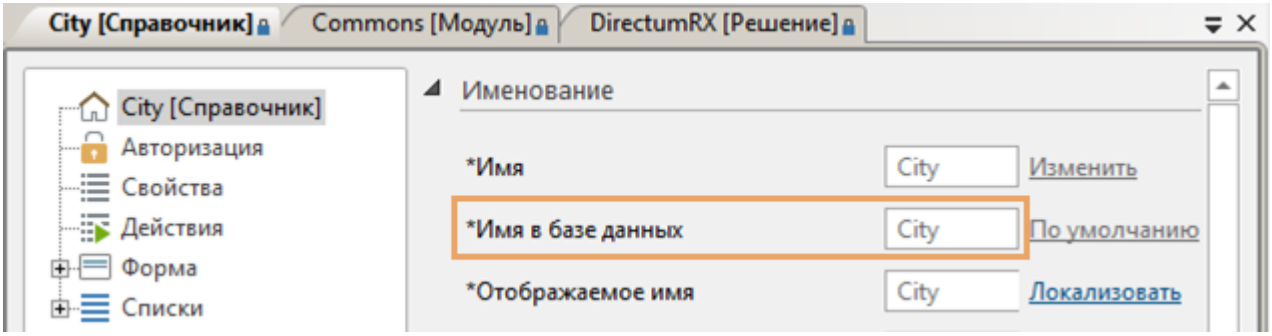
2. Определяем название таблицы по формату **<Код компании>_<Код модуля>_<Код типа сущности>**:
   - Sungero – код компании, которая разработала модуль. Посмотреть код можно в редакторе решения, в котором находится модуль или тип сущности:



   - Commons – код модуля, в котором расположена сущность. Отображается в редакторе модуля, в котором находится тип сущности:

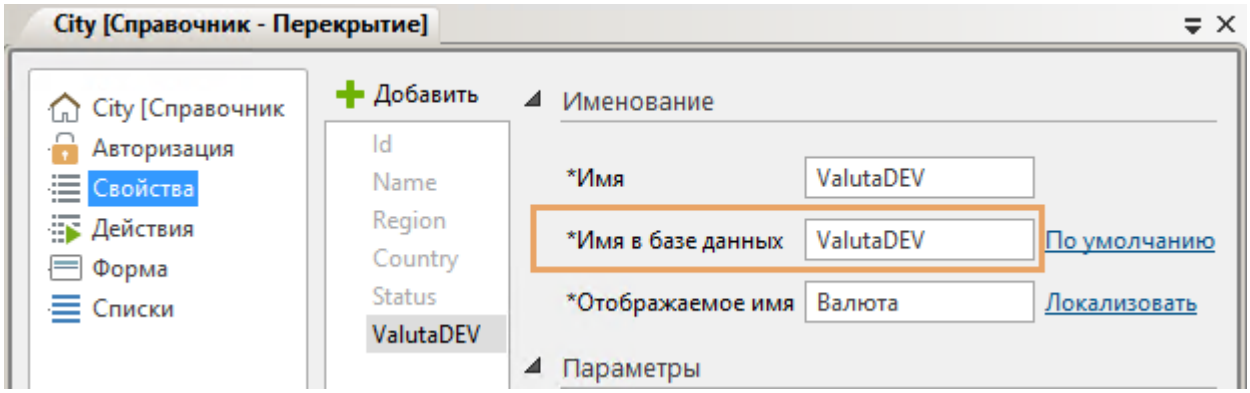


- City – код сущности «Населенные пункты». Отображается в редакторе типа сущности:

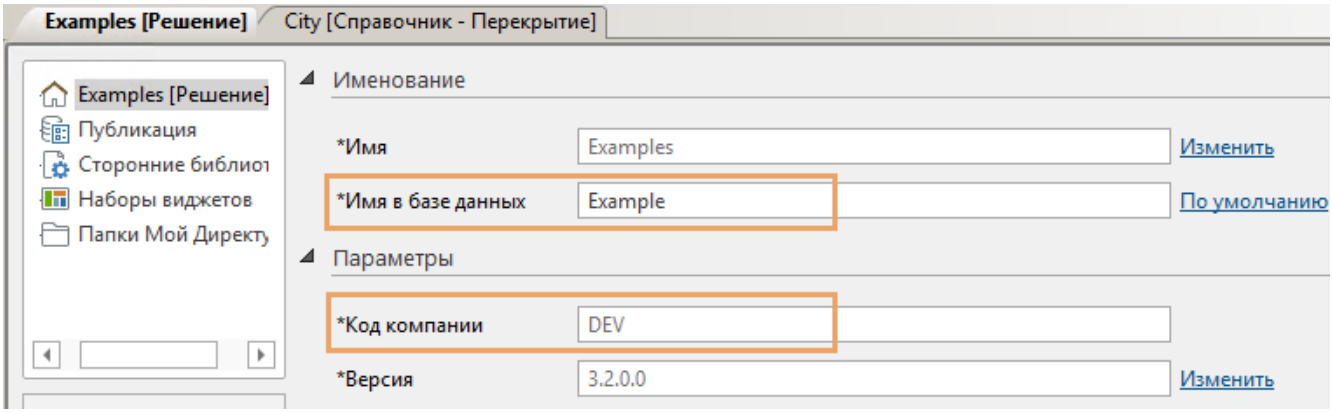


В результате получаем название таблицы **Sungero_Commons_City**.

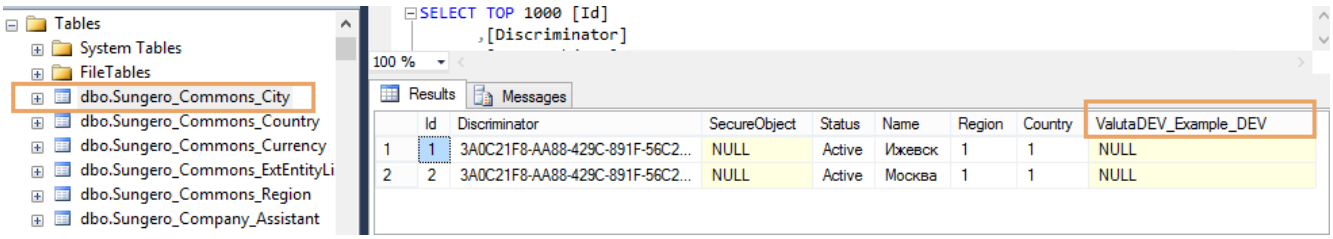
3. Определяем название колонки, в которой находится добавленное свойство **Валюта** по формату **<Код свойства>_<Код модуля>_<Код компании>**:
   - ValutaDEV – код свойства. Отображается в редакторе свойств:



   - Example – код решения, в котором перекрыт тип сущности. В данном случае вместо кода модуля используется код решения, так как тип сущности перекрыт;
   - DEV – код компании.



В результате получаем название колонки **ValutaDEV_Example_DEV**.



ПРИМЕЧАНИЕ. Значение ***Имя в базе данных*** может отличаться от значения в поле ***Имя***. Поэтому точное название рекомендуется смотреть именно в редакторе решения, модуля и типа сущности в поле ***Имя в базе данных***, а не в дереве решений. В данном примере имя в базе данных для решения **Examples** отличается и называется **Example**.

Таким образом с помощью среды разработки вы можете определить название таблицы для типа сущности, который находится в прикладном модуле, а также название колонки для добавленного в него свойства. Или наоборот, по названию сущности в среде разработки определить, в какой таблице она хранится.

Колонки свойств-компонент

К свойствам-компонентам относятся свойства типов Бинарные данные, Картинка. Свойства-компоненты проецируются в четыре и более колонок БД, так как этим свойствам требуется хранить дополнительную информацию:

- идентификатор содержимого;
- размер содержимого;
- идентификатор хранилища;
- хеш содержимого.

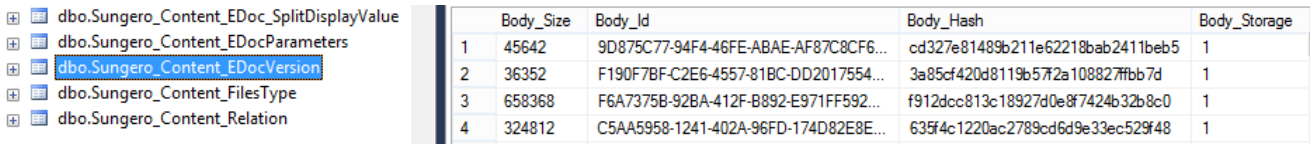
По этой причине к именам колонок добавляется еще одна часть – Код подсвойства:

Имя колонки = <Код свойства>_**<Код подсвойства>**_<Код модуля>_<Код компании>

Имя колонки = <Код свойства>_**<Код подсвойства>**

К примеру, свойство Versions (версия электронного документа) содержит свойство Body (содержимое версии) типа **Бинарные данные**. Оно проецируется в колонки:

- Body_Size – размер значения атрибута в байтах;
- Body_Id – идентификатор бинарных данных;
- Body_Hash – хеш бинарных данных;
- Body_Storage – хранилище.



	Body_Size	Body_Id	Body_Hash	Body_Storage
1	45642	9D875C77-94F4-46FE-ABAE-AF87C8CF6...	cd327e81489b211e62218bab2411beb5	1
2	36352	F190F7BF-C2E6-4557-81BC-DD2017554...	3a85cf420d8119b57f2a108827ffbb7d	1
3	658368	F6A7375B-92BA-412F-B892-E971FF592...	f912dcc813c18927d0e8f7424b32b8c0	1
4	324812	C5AA5958-1241-402A-96FD-174D82E8E...	635f4c1220ac2789cd6d9e33ec529f48	1

Уникальность таблиц и колонок

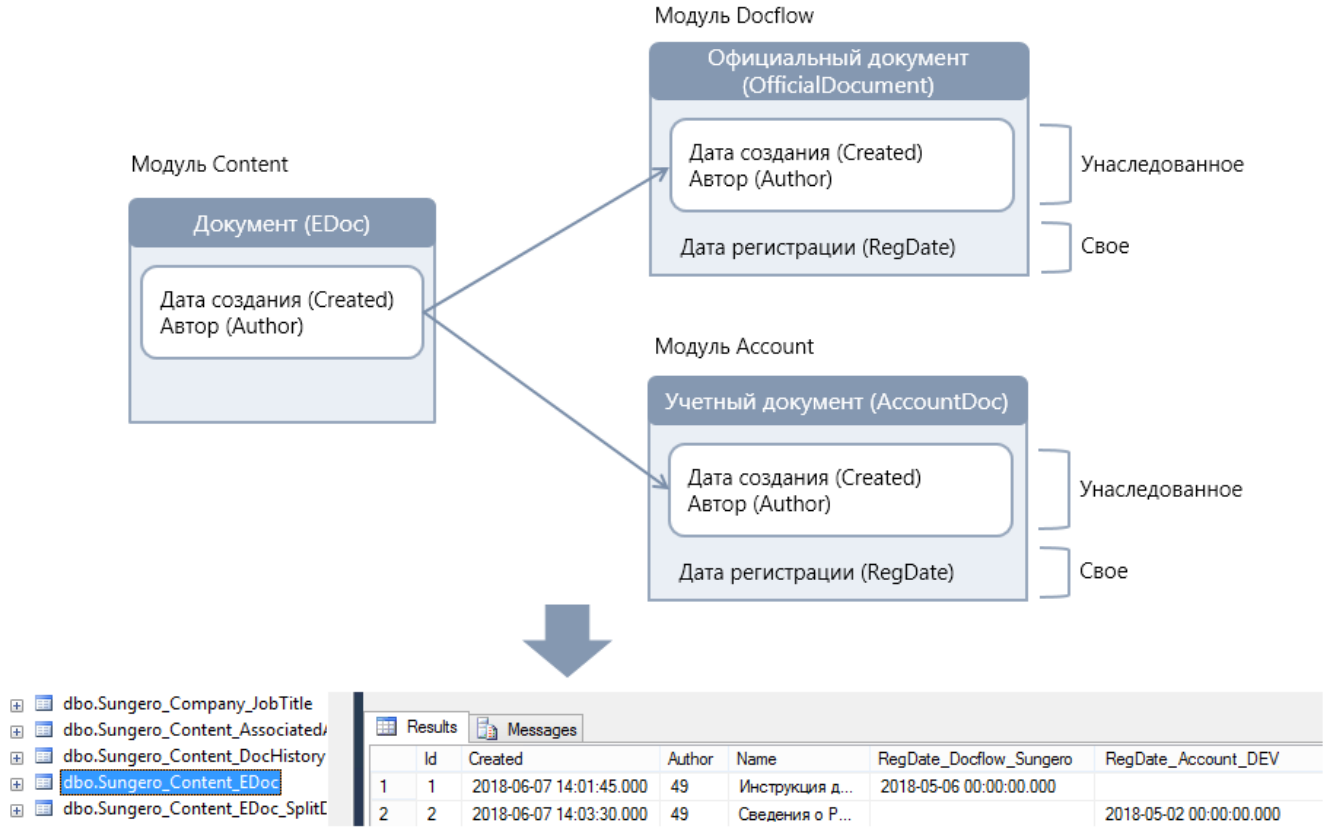
Разные компании могут разработать схожие модули и даже схожие сущности в этих модулях. Благодаря трехкомпонентному имени таблицы типов сущностей и колонки в них уникальны.

Например, у базового документа есть два наследника:

- «Официальный документ», разработанный компанией-вендором (код Sungero) в модуле Docflow. У него есть свойство «Дата регистрации», которое размещается в колонке **RegDate_Docflow_Sungero**;

- «Учетный документ», разработанный компанией DEV в модуле Account. У него есть схожее свойство «Дата регистрации», которое размещается в колонке с уже другим названием **RegDate_Account_DEV**.

Колонки добавляются в одну таблицу **Sungero_Content_EDoc**.



Уникальность и ограничения кодов

Коды элементов разработки должны быть уникальны:

- Код компании уникален среди всего множества выданных клиентам кодов компаний. Поэтому за их выдачей и обеспечением уникальности следит компания-вендор;
- Код модуля – в рамках кода компании;
- Код типа сущности – в рамках модуля;
- Код свойства должен быть уникален в рамках всех свойств модуля, которые проецируются в одну таблицу БД. То есть все наследники от общей базовой сущности, расположенные в одном модуле, должны иметь уникальные коды свойств.

Помимо уникальности коды должны удовлетворять ряду ограничений:

- допустимая длина кода:

Компания	Модуль	Тип сущности	Свойство
[2-7]	[2-7]	[2-14]	[2-14] Для свойств-компонент: [2-9] Для подсвойств у свойств-компонент: [2-4]

- в качестве кода компании, модуля, типа сущности или свойства разрешаются только символы латинского алфавита и цифры: a-z, A-Z, 0-9. Первый символ всегда буква.

Исключением являются sys для кода компании, Discriminator и SecureObject для кода свойства. Они запрещены для использования.

Например, разрешены коды Module1, HR. Но запрещены My_Group, 7z.

Примеры системных таблиц

Помимо таблиц для прикладных модулей, в базе данных есть таблицы для системных и предметных модулей. Они находятся на уровне платформы и содержат базовые типы сущностей, например Задача (Sungero.Workflow.Task), Базовый документ (Sungero.Content.ElectronicDocument). Предметные и системные модули не отображаются в среде разработки и не предназначены для перекрытия.

Content

Sungero_Content_AssociatedApp	Приложения-обработчики
Sungero_Content_DocHistory	История работы с документами и версиями. В том числе можно найти информацию по документам, которые уже удалены. То есть в таблице Sungero_Content_EDoc уже не будет документа, но историю по нему можно найти
Sungero_Content_EDoc	Карточки электронных документов. Здесь хранятся все документы, даже если вы создали свой тип документа в своем модуле
Sungero_Content_EDocReadAccess	Права на чтение электронных документов
Sungero_Content_EDocParameters	Параметры шаблонов, которые используются при настройке для автозаполнения шаблонов документов
Sungero_Content_EDocVersion	Информация о версии электронного документа. Содержит в том числе колонки BodyID и Public_BodyID
Sungero_Content_FilesType	Типы файлов для приложений-обработчиков Sungero_Content_AssociatedApp
Sungero_Content_Relation	Связи документов

Core

Sungero_Core_AccessRightEnt, Sungero_Core_AccessRights	Таблицы для хранения прав доступа
Sungero_Core_AccessRightTyp	Типы прав
Sungero_Core_Certificate	Цифровые сертификаты
Sungero_Core_ComputedRole	Вычисляемые роли
Sungero_Core_DatabookHistory	История записей справочников
Sungero_Core_DatabookReadAccess	Права на чтение всех типов справочников
Sungero_Core_FolderReadAccess	Права на чтение папок
Sungero_Core_Folder	Папки
Sungero_Core_Job	Таблицы, которые относятся к фоновым процессам
Sungero_Core_JobDaysOfWeekS	

Sungero_Core_AccessRightEnt, Sungero_Core_AccessRights	Таблицы для хранения прав доступа
Sungero_Core_JobHistory	
Sungero_Core_JobMonthSchedu	
Sungero_Core_JobSchedule	
Sungero_Core_Link	Ссылки, которые хранятся в содержимом папок Sungero_Core_Folder
Sungero_Core_Login	Данные для аутентификации: имя пользователя, тип аутентификации, информация о том, будет ли появляться окно о смене пароля, зашифрованный пароль, действующая или закрытая учетная запись
Sungero_Core_Recipient	Субъекты прав: пользователи, группы регистрации, роли и т.д.
Sungero_Core_RecipientLink	Участники ролей и групп
Sungero_Core_RelationType	Типы связей
Sungero_Core_RelationTypeMa	Данные с вкладки «Параметры» для типов связей
Sungero_Core_Sign	Подписи документов
Sungero_Core_SignedProperty	Информация о подписываемой версии
Sungero_Core_Substitution	Замещения
Sungero_Core_SystemHistory	Системная история. Например, аутентификация. Не имеет предметного отображения в проводнике Directum RX
Sungero_Core_WorkingTime	Календари рабочего времени
Sungero_Core_WrkTimeCalDay	Табличная часть записи календарей рабочего времени (по дням)

IDE

Sungero_IDE_Deploy	История выполнения публикации. Хранит информацию о том, кто, когда и какой пакет опубликовал
Sungero_IDE_FullDeploy	Все бинарные файлы прикладной разработки

System

Sungero_System_BinDataLocks	Блокировки версий документов
Sungero_System_Cache	Прикладной кэш
Sungero_System_Clients	Информация о пользователях, которые работают с системой
Sungero_System_EntityType	Информация о типах сущностей
Sungero_System_FolderTag	Тэги для количества непрочитанных заданий
Sungero_System_Ids	Очередные ID по разным таблицам
Sungero_System_License	Лицензия системы
Sungero_System_Locks	Блокировки карточек
Sungero_System_LookupStatistics	Статистика для определения часто используемых записей

Sungero_System_BinDataLocks	Блокировки версий документов
Sungero_System_NotifyViewed	Информация о прочтении уведомления
Sungero_System_RateLimit	Информация об ограничениях на массовые операции с типами объектов
Sungero_System_Server	Все сервера, которые когда-либо обращались к базе данных

Workflow

Sungero_WF_Assignment	Данные всех типов заданий, заданий на приемку и уведомлений
Sungero_WF_Attachment	Данные о вложениях задач и заданий
Sungero_WF_BlockState	Состояния блоков схемы задачи
Sungero_WF_DeletedAttachment	Данные о вложениях, удаленных из групп вручную
Sungero_WF_MainTaskReadAccess	Права на чтение ведущих задач для задач, заданий и уведомлений
Sungero_WF_Instance	Текущее состояние схемы задачи
Sungero_WF_RouteSteps	Свойство-коллекция «Маршрут» (RouteSteps) для простой задачи (SimpleTask)
Sungero_WF_Scheme	Все опубликованные схемы задач
Sungero_WF_Task	Данные всех типов задач
Sungero_WF_TaskObservers	Свойство-коллекция «Список наблюдателей» (Observers) для задачи (Task)
Sungero_WF_Text	Переписка по задачам и заданиям
Sungero_WF_WorkflowHistory	История задач, заданий и уведомлений

Информация о предыдущих бинарных данных документа

Иногда администратору может потребоваться восстановить данные из резервной копии, например, при повреждении или сбое работы сервера. При этом может возникнуть ситуация, когда восстановлены данные о документах из разных по времени резервных копий сервиса хранилищ и базы данных. Например, пользователь несколько раз сохранил зашифрованный документ, и в таблице базы данных хранится информация для текущего содержимого документа. При этом на восстановленной копии сервиса хранилищ находится более старый вариант бинарных данных. В результате рассинхронизации данных можно безвозвратно потерять содержимое документов.

Система предусматривает возможности восстановления данных и в этой ситуации. Для этого используйте данные из колонок таблицы **Sungero_Content_DocHistory**:

- **PreviousBodyId** и **PreviousPublicBodyId** – идентификаторы предыдущих бинарных данных документа;

- **PreviousBodyEncryptionData** и **PreviousPublicBodyEncryptionData** – информация о шифровании этих предыдущих бинарных данных.

При любом изменении версии документа в колонки таблицы записываются предыдущие значения, а при удалении версии записываются текущие значения. Кроме того, информация о предыдущих бинарных данных документа записывается и для незашифрованных документов. Таким образом, с помощью информации из таблицы администратор может вручную восстановить необходимые данные.

Особенности разработки для мобильных решений

Чтобы модифицировать стандартную функциональность системы Directum RX под конкретные задачи организации, можно доработать существующие, а также создать собственные объекты: задачи, задания, процессы и др. Для этого используется среда разработки **Directum RX Development Studio**.

Чтобы поддержать собственную разработку в мобильных приложениях Directum Solo и Directum Jazz:

1. Ознакомьтесь с особенностями поддержки изменений на сервисе NOMAD:
   - типы задач и заданий;
   - карточки документов и справочников.

   Учитывайте их при редактировании конфигурационного файла SungeroAdapter.config.
2. Настройте работу мобильных решений с задачами и заданиями в конфигурационном файле SungeroAdapter.config.

   Примеры по адаптации сервиса NOMAD к собственным доработкам системы см. в разделе «Примеры доработок в файле SungeroAdapter.config».

СОВЕТ. Чтобы определить, в контексте какого клиентского приложения выполняется прикладной код, используйте свойство ApplicationType.

Перед запуском сервиса NOMAD проверьте его работу с измененными настройками на тестовой базе. Корректность настроек, которые изменялись в конфигурационных файлах NOMAD, проверяется автоматически. Подробнее см. в руководстве администратора Directum RX, раздел «Настройка сервиса NOMAD».

Поддержка типов задач в NOMAD

В мобильных приложениях поддерживается создание:

- простых задач;
- задач на исполнение поручений;
- задач, отмеченных в конфигурационном файле SungeroAdapter.config атрибутом **allowStart="true"**. В текущей версии приложения это задачи на согласование по процессу. регламенту и свободное согласование.

В силу ограниченной поддержки произвольных типов задач остальные типы считаются неподдерживаемыми.

Особенности работы задач в приложении:

1. Не поддерживается переопределение свойств задач и их событий.
2. Не поддерживается работа с диалогами.
3. Не поддерживается логика «Ведущий-ведомый» в свойствах.
4. При старте задач автоматически выдаются права:
   - для простых задач – на вложенные документы;

- для остальных типов задач – на документы из групп, указанных в секциях **autoRightsAttachmentGroup** в конфигурационном файле SungeroAdapter.config.

При этом учитываются параметры конфигурационного файла Nomad.config, определяющие права доступа: **autoRightsOnAttachments**, **autoRightsForPerformers**, **autoRightsForObservers**.

Для быстрого поиска связей между поручением и совещанием, в рамках которого отправлено поручение, используется реквизит записи поручения **Meeting**. При этом в реквизите указывается ИД совещания.

Поддержка типов заданий в NOMAD

Поддерживаются все типы заданий со следующими ограничениями:

- отображается только информация с закладки «Свойства» карточки задания. Дополнительные закладки не поддерживаются;
- на форму карточки задания нельзя добавить контрол состояния, метки и ссылки;
- доступны не все типы свойств задания;
- ограничена работа с группами вложений;
- для отображения действий (не вариантов выполнения) необходима настройка;
- доступны не все типы диалогов.

Ограничения вариантов выполнения и действий:

- не открываются карточки сущностей, списков, отчетов и т.п., кроме создания новой задачи;
- действия выполняются по умолчанию в онлайн-режиме. Для выполнения их офлайн необходима настройка;
- поставить утверждающую подпись можно только при согласовании по регламенту. Подписание нужно настроить в конфигурационном файле SungeroAdapter.config.

Свойства

В Solo и Jazz поддерживаются типы свойств: дата, перечисление, ссылка, логическое, вещественное, целое, строка, текст.

Не поддерживается логика «Ведущий-ведомый» в свойствах.

В карточке задания отображается вся информация с закладки «Свойства». Свойства из группы **Header** (содержит контролы, в которых отображается общая информация по заданию: тема, инициатор, исполнители и т.п.) отображаются только для чтения. Свойства из группы **Footer** (содержит контролы, в которые пользователи вводят информацию о результатах выполнения задания) отображаются для чтения и редактирования.

Группы вложений

В Solo и Jazz документы вкладываются в в последнюю из групп вложений, доступных для изменения. Как правило, это группа «Дополнительно».

Основным документом считается первый документ из первой обязательной группы вложений задания. Он подписывается при согласовании и отображается при открытии карточки заданий с настроенным в SungeroAdapter.config режимом отображения «согласование» и «рецензирование» (атрибут **viewMode** элемента задания).

Действия

В мобильных приложениях в карточке задания отображаются все доступные варианты выполнения. Если нужно отобразить определенное действие, укажите его в конфигурационном файле SungeroAdapter.config.

Пример:

```
<job type="Sungero.Docflow.IApprovalAssignment">
      <result name="AddApprover"/>
</job>
```

В задании на согласование вместе с вариантами выполнения будет отображено действие «Добавить согласующего».

Диалоги

Directum Solo

Поддерживаемые типы диалогов:

- сообщение об ошибке, предупреждение – **ShowMessage**;
- всплывающее сообщение – **NotifyMessage**;
- диалог с выбором варианта – **CreateTaskDialog**;
- диалог с запросом параметров – **CreateInputDialog**.

Сообщения об ошибке и предупреждения отображаются вместе с результатами проверки задания.

Поддерживаемые элементы в диалоге запроса параметров:

- флажок;
- дата;
- вещественное число;
- целое число;
- строка;
- многострочный текст;
- строка ввода пароля;
- выпадающий список.

Подавляемые диалоги

Диалоги, которые автоматически обрабатываются на сервисе NOMAD и не отображаются в клиентском приложении, т.е. подавляются.

Диалог с выбором варианта – CreateTaskDialog

Диалог появляется с учетом пользовательской локализации. Подробное описание диалога см. в руководстве разработчика Directum RX, раздел «Диалог с выбором варианта – CreateTaskDialog».

Диалог подавляется автоматически:

- при выдаче прав на вложения при выполнении задания. В этом случае создается диалог с типом «вопрос», текстом и набором кнопок как у диалога выдачи прав из стандартной прикладной разработки Directum RX.

  По умолчанию диалог выполняется с результатом выдачи прав на изменение. Если выдать права на изменение не удается, то открывается диалог об ошибке, который выполняется с результатом повторения попытки. Очередной диалог выдачи прав выполняется с результатом «Просмотр». При ошибке следующий диалог выполняется с отказом выдачи прав;
- при подтверждении операций. В этом случае создается диалог с типом «вопрос», и кнопками **Да** или **Нет**. По умолчанию все диалоги подтверждения выполняются с результатом «Да».

Чтобы подавлять другие диалоги с выбором варианта из конфигурации, задайте в конфигураторе соответствующие варианты выполнения с указанием результата.

Пример:

```
<job type="Sungero.RecordManagement.IReviewManagerAssignment">
  <result name="AddAssignment">
    <taskDialog type="Question"
text="Sungero.Docflow.Resources.ExecuteWithoutCreatingActionItem">
      <button type="Yes" isResult="true"/>
      <button type="No"/>
      <button name="Sungero.Docflow.Resources.CreateActionItem"/>
    </taskDialog>
  </result>
</job>
```

Для варианта выполнения «Отправлено на исполнение» подавляется диалог типа «вопрос» с текстом из ресурсной строки Sungero.Docflow.Resources.ExecuteWithoutCreatingActionItem («Отправить на исполнение без создания поручения?»), кнопками **Да**, **Нет** и кнопкой с текстом из ресурсной строки Sungero.Docflow.Resources.CreateActionItem («Создать поручение»). Диалог выполняется с результатом «Да».

Диалог подтверждения – CreateConfirmDialog

По умолчанию все диалоги подтверждения выполняются с результатом «Да».

Офлайн-режим

В Solo и Jazz все действия по умолчанию выполняются онлайн. Чтобы выполнять их в офлайн-режиме, задайте атрибут **isEnabledOffline** для варианта выполнения в настройках конфигурационного файла SungeroAdapter.config.

Пример:

```
<job type="Sungero.Docflow.IDeadlineExtensionAssignment">
      <result name="Accept" isEnabledOffline="true" />
</job>
```

В таком виде принять задание на продление срока можно при отсутствии связи с сервисом. При этом задание сразу будет отмечено в приложении выполненным, но реальное выполнение произойдет после появления связи с сервисом и синхронизации данных.

Ограничения выполнения действий офлайн:

- не поддерживаются диалоги;
- не проверяется достоверность свойств задания;
- не поддерживается запрос допустимых значений для свойств задания.

Таким образом, отмечать офлайновыми рекомендуется только те варианты выполнения, в которых:

- не появляются диалоги, которые не подавляются сервисом;
- не проверяется достоверность свойств задания перед выполнением;
- нет свойств-ссылок, либо они необязательны для заполнения или предзаполнены.

При отсутствии связи с сервисом все онлайновые действия в приложении становятся недоступными. Также становятся недоступными для изменения все свойства задания, кроме тех, которые относятся к офлайновым вариантам выполнения. Исключением являются свойства ссылок, которые требуют запроса допустимых значений с сервиса. К вариантам выполнения относятся те свойства, которые указаны в секции **params** и элементах **ask** этих вариантов в конфигурационном файле SungeroAdapter.config.

Подписание

Подписание документов выполняется на сервисе NOMAD. При этом подписываются основной документ и его приложения, которые вложены в задание на подписание.

По умолчанию документы подписываются *простой ЭП* без использования *сертификата ЭП* сотрудника, даже если он добавлен в Solo или Jazz.

Подписание *усиленной ЭП* возможно в заданиях на согласование по регламенту, если в конфигурации для варианта выполнения задания добавлен атрибут **checkStrongSign**. В этом случае сервис NOMAD проверяет, нужно ли подписывать документ с использованием сертификата на данном этапе согласования. Если подпись требуется, на документы ставится подпись с типом, указанным в атрибуте **checkStrongSign**: согласующая или утверждающая.

Пример настройки подписания усиленной ЭП при согласовании официального документа руководителем:

```
<job type="Sungero.Docflow.IApprovalManagerAssignment">
      <result name="Approved" checkStrongSign="Endorsing" />
</job>
```

Больше примеров по адаптации сервиса NOMAD для собственных доработок по подписанию документов см. в разделе «Примеры доработок в файле SungeroAdapter.config».

Работа с карточками документов и справочников в NOMAD

Особенности работы:

- не поддерживается редактирование;
- вызов клиентских событий «Показ формы» и «Обновление формы» поддерживается, только если мобильное устройство находится в зоне действия сети.

Так как при работе без интернета события «Показ формы» и «Обновление формы» не вызываются, при формировании карточки учитывается то состояние контролов, которое возвращается из ее метаданных. В первую очередь учитывается видимость.

В связи с перечисленными выше ограничениями рекомендуется в среде разработки указывать состояние контролов по умолчанию, которое позволит сотрудникам видеть в Solo и Jazz достаточный объем информации. Например, оставить видимыми контролы с общедоступными данными, а с конфиденциальными – скрыть.

При настройке сервиса NOMAD администратор может ограничивать загрузку документов и записей справочников на мобильные устройства пользователей. Для этого можно использовать как реквизиты из стандартной разработки Directum RX, так и реквизиты, созданные при модификации прикладной разработки. Подробнее см. в руководстве администратора Directum RX, раздел «Настройка доступа к документам и записям справочников».

Настройка задач и заданий в файле SungeroAdapter.config

В файле SungeroAdapter.config содержатся настройки, которые обеспечивают совместную работу Directum RX с сервисом NOMAD, а также мобильными приложениями Solo и Jazz:

- task – описание задачи;
- job – описание задания или уведомления;
- action – описание действия;
- result – вариант выполнения задания с дополнительной логикой;
- autoRightsAttachmentGroup – группа вложений, на документы из которой автоматически назначаются права;
- ask – запрашиваемые параметры;
- If – дополнительное действие запрашиваемого параметра;
- condition – условия, наложенные на действие;
- param – свойство;

- taskDialog – диалог с выбором варианта для автоматической обработки;
- button – кнопка диалога;
- Типы;
- Бизнес-процессы;
- Документы;
- Флаги.

Расположение файла по умолчанию см. в руководстве администратора Directum RX, раздел «Сопровождение мобильных решений».

ВАЖНО. Новые настройки применяются к задачам и заданиям, которые созданы после сохранения изменений в конфигурационном файле SungeroAdapter.config.

Примеры по адаптации сервиса NOMAD для собственных доработок см. в разделе «Примеры доработок в файле SungeroAdapter.config».

task – описание задачи

Содержит следующие атрибуты и вложенные элементы:

- **type**. Тип задачи;
- **allowStart**. Признак, определяющий возможность старта задачи. По умолчанию: **false**;
- **documentActionName**. Наименование действия для отправки документа в задаче (см. руководство разработчика Directum RX, раздел «Действия» Sungero.Metadata.ActionMetadata.Name). Например, **SendForApproval**;
- **viewMode**. Режим отображения делового процесса;
- **flags**. Флаги задачи, задания, уведомления. По умолчанию: **0**;
- **action**. Действия, выполняемые при отправке задачи;
- **autoRightsAttachmentGroup**. Группа вложений, на документы из которой автоматически назначаются права при старте задачи;
- **ask**. Запрашиваемые у пользователя параметры.

Пример

```
<!-- Свободное согласование. -->
<task type="Sungero.Docflow.IFreeApprovalTask" allowStart="true"
documentActionName="SendForFreeApproval">
  <ask type="Document" target="ForApprovalGroup.ElectronicDocuments" required="true"
collection="false" onCreateTaskSetFromAttachment="true"
       display
="Sungero.Docflow.Shared.FreeApprovalTask.FreeApprovalTaskSystem.AttachmentGroup_ForApprovalGr
oupTitle"/>
  <ask type="Recipient" target="Approvers" required="true" validateOnServer="true" />
  <ask type="Date" target="MaxDeadline" onlyFuture="true" />
  <ask type="Pick" target="ReceiveOnCompletion" required="true" />
  <ask type="Bool" target="ReceiveNotice" />
</task>
```

Здесь задается задача с разрешенным стартом и действием для документа **SendForFreeApproval**. Указан ряд запрашиваемых параметров (см. описания ниже).

job – описание задания или уведомления

Содержит следующие атрибуты и вложенные элементы:

- **type**. Тип задания;
- **rof**. Признак, показывающий, является ли данный деловой процесс деловым процессом по делопроизводству. Например, **true**;
- **rrcFrom**. Группа вложений, на основе которой необходимо сформировать поручения;
- **approvalSheet**. Группа вложений, на основе которой необходимо сформировать лист согласования;
- **customActionItemPropertyName**. Название прикладного свойства задания, которое определяет, содержит ли это задание нестандартные поручения. Необязательный атрибут. Используется, если в компании разработаны собственные поручения и руководители просматривают их в Directum Solo.
- **customActionItemEntityType**. Определяет, из какого типа сущности учитывается признак **customActionItemPropertyName**. Необязательный атрибут;
- **viewMode**. Режим отображения делового процесса;
- **Flags**. Флаги задачи, задания, уведомления. По умолчанию: **0**;
- **result**. Варианты выполнения задания с дополнительной логикой.

Пример

```
<!-- Задание обычным согласующим. -->
<job type="Sungero.Docflow.IApprovalAssignment" viewMode="Approval">
  <!-- Добавить согласующего. -->
  <result name="AddApprover" flag="20" notification="ExecResNotification_ApproverAdded">
    <ask type="Employee"
display="Sungero.Docflow.Shared.FreeApprovalTask.FreeApprovalTaskResources.Approver"
target="!Approver" required="true" />

    <action name="AddApprover" approver="!Approver" />
  </result>
</job>
```

Здесь задается задание с режимом отображения делового процесса **Approval**. Имеется одно действие и один вариант выполнения (см. описания ниже).

action – описание действия

Действие, которое выполняется при отправке задачи или при выполнении задания.

Содержит атрибуты: **Name**, если отсутствует, то **Type**. Поддерживаются типы действия:

- Abort – прекращение задачи, задания, уведомления;
- Sign – подписание документов из вложений задачи, задания, уведомления простой подписью;
- StartTask – старт задачи, задания, уведомления;
- SuppressPerform – препятствие выполнению прикладного кода варианта выполнения;
- AddApprover – добавление согласующего;
- SetLifeCycle – установка стадии жизненного цикла документа;

- StartDeadlineExtensionTask – запрос на продление срока.

Имена свойств сущностей могут иметь префиксы:

- префикс «!» применяется для обозначения несуществующего свойства, которое может использоваться для передачи значения из блока запроса **ask** в блок действия **action**;
- префикс «@» применяется для обозначения дополнительных свойств, когда не может быть использовано основное свойство. Так, вместо нестабильного свойства **ActiveText** задачи рекомендуется использовать свойство **@ActiveText**.

Abort

Прекращение задачи, задания, уведомления. Выполнение действия препятствует вызову прикладной обработки варианта выполнения.

target. Целевая задача, задание, уведомление. Сейчас поддерживается только значение **Task**.

Пример

```
<action name="Abort" target="Task" />
```

Действие прекращения текущей задачи.

Sign

Подписание документов из вложений простой подписью. Используется только для task.

target. Целевая группа вложений.

type. Тип подписи.

Пример

```
<action name="Sign" target="DocumentGroup" type="Approval">
```

Действие по подписанию группы документов утверждающей подписью.

StartTask

Старт задачи или уведомления.

interface. Тип задачи.

add. Свойства добавляемые.

set. Свойства устанавливаемые.

Свойства **add** и **set** содержат одинаковые атрибуты:

- **source**. Путь к источнику устанавливаемого значения вида "Job.xxx" или "Task.xxx";
- **destination**. Путь к получателю устанавливаемого значения вида "Job.xxx" или "Task.xxx".

Пример

```
<ask type="Date"
display
="Sungero.RecordManagement.Shared.DeadlineExtensionTask.DeadlineExtensionTaskSystem.Property_N
ewDeadline" target="!NewDeadline" onlyFuture="true" required="true" />
<ask type="Text"
display
```

```
="Sungero.RecordManagement.Shared.DeadlineExtensionTask.DeadlineExtensionTaskSystem.Property_R
eason" target="!Reason" required="true" />
<action name="StartTask" interface="Sungero.RecordManagement.IDeadlineExtensionTask">
  <set source="Job.!NewDeadline" destination="Task.NewDeadline" />
  <set source="Job.!Reason" destination="Task.@ActiveText" />
</action>
```

Действие по запросу продления срока с указанием нового срока и причины запроса.

SuppressPerform

Препятствует выполнению прикладного кода варианта выполнения.

Пример

```
<action name="SuppressPerform">
```

AddApprover

Добавление согласующего.

approver. Согласующий.

deadline. Срок.

Пример

```
<action name="AddApprover" approver="!Approver" />
```

Действие по добавлению исполнителя в задание согласования из временной переменной.

SetLifeCycle

Установка стадии жизненного цикла документа.

target. Целевая группа вложений.

state. Стадия жизненного цикла документа.

Пример

```
<action name="SetLifeCycle" target="DocumentGroup" state="Obsolete" />
```

Действие по установке жизненного цикла группы документов в состояние «Устаревший».

StartDeadlineExtensionTask

Запрос на продление срока. Задается аналогично StartTask.

Пример

```
<ask type="Date"
display
="Sungero.RecordManagement.Shared.DeadlineExtensionTask.DeadlineExtensionTaskSystem.Property_N
ewDeadline" target="!NewDeadline" onlyFuture="true" required="true" />
<ask type="Text"
display
="Sungero.RecordManagement.Shared.DeadlineExtensionTask.DeadlineExtensionTaskSystem.Property_R
eason" target="!Reason" required="true" />
<action name="StartDeadlineExtensionTask">
  <set source="Job.!NewDeadline" destination="Task.NewDeadline" />
  <set source="Job.!Reason" destination="Task.@ActiveText" />
</action>
```

Действие по запросу продления срока с указанием нового срока и причины запроса.

result – вариант выполнения задания с дополнительной логикой

name. Наименование.

flag. Флаги варианта выполнения. По умолчанию **0**.

show. Условия отображения варианта выполнения. Вариант отображается только при удовлетворении всех условий. Иначе - не отображается. Внутри **condition**.

enable. Условия доступности варианта выполнения. Вариант доступен только при удовлетворении всех условий. Иначе - не доступен. Внутри **condition**.

action. Действия, выполняющиеся при выборе данного варианта выполнения.

ask. Запрашиваемые у пользователя параметры.

notification. Текст, отображаемый пользователю при выборе данного варианта выполнения. Может указываться:

- имя строки локализации, например **ExecResNotification_AssignmentSentToExecution**;
- имя строки локализации внешней сборки в формате <путь к файлу сборки>$<полное имя класса ресурсов>.<имя ресурса>, например C:\Temp\MyResources.dll$MyResources.Resources.AssignmentSentToExecution;
- не локализуемая строка (если начинается с "!").

suppressPerform. Признак, препятствующий выполнению прикладного кода варианта выполнения. По умолчанию **false**.

isEnabledOffline. Признак, разрешающий выполнение варианта выполнения в офлайн режиме. По умолчанию **false**.

checkStrongSign. Атрибут, добавляющий проверку необходимости установки подписи сертификатом пользователя для соответствующего заданию этапу регламента согласования.

Если на этапе согласования, для которого выставлен этот атрибут, необходимо подписание сертификатом, то в мобильном клиенте документ подписывается тем типом подписи, который указан в качестве значения атрибута:

- **Endorsing** - согласующая подпись;
- **Approval** - утверждающая подпись;
- **ApprovalIfCan** - если у пользователя есть право утверждающей подписи документа, то утверждающая подпись, иначе - согласующая.

ignoreSigningStageSettings. Признак, разрешающий игнорировать настройки этапа подписания. Возможные значения:

- **true** – настройка этапа подписания игнорируется, при этом считается, что в задании на подписание документа в параметре **needStrongSign** установлено значение **true**;
- **false** – используется настройка из этапа подписания.

params. Секция свойств задания. Внутри **param**.

taskDialog. Диалоги с выбором варианта.

Пример. Вариант результата выполнения с условиями доступности

```
<result name="Complete">
  <enable>
    <condition name="IsNull" target="ExchangeService" />
  </enable>
</result>
```

autoRightsAttachmentGroup – группа вложений, на документы из которой автоматически назначаются права

target. Код группы вложений. На документы из заданной группы автоматически назначаются права при старте задачи. Назначение прав на документы из групп «Документ» и «Приложения» обрабатывается автоматически и не требует дополнительной настройки.

Пример

```
<autoRightsAttachmentGroup target="OtherGroup" />
```

Указан код группы вложений для автоматического назначения прав.

ask – запрашиваемые параметры

Содержит атрибуты: **Name**, если отсутствует, то **Type**. Возможные типы параметра:

- Bool – логическое значение;
- Date – дата;
- Document – документ;
- Pick – перечисление;
- Recipient – базовый субъект авторизации;
- ReferenceRecord – запись справочника;
- Text – текст;
- ApprovalRule – правило согласования;
- ProcessKind – вариант процесса;
- Employee – сотрудник;
- OfficialDocument – официальный документ;
- Signatory – подписант;
- User – пользователь.

required. Признак, определяющий, является ли запрашиваемый параметр обязательным для заполнения. По умолчанию **false**.

readonly. Признак, определяющий, что параметр доступен только для чтения. По умолчанию **false**.

validateOnServer. Признак, определяющий необходимость отправки параметра на сервис сразу же после изменения его значения. По умолчанию **false**.

target. Целевое свойство. По умолчанию **!local**.

display. Отображаемое имя запрашиваемого параметра. Например, «Sungero.Docflow.Shared.IncomingDocumentBase.IncomingDocumentBase.MarkDocumentAsObsolete».

collection. Признак, определяющий, является ли параметр коллекцией. По умолчанию **false**.

condition. Условия отображения параметра на клиенте. Параметр отображается только при удовлетворении всех условий.

if. Действия, выполняемые при совпадении указанного значения параметра с заданным.

uiFilteringDisabled. Признак включенного ограничения видимости оргструктуры. Значение зависит от настройки видимости в Directum RX:

- **false** – ограничение включено. Для запрашиваемых параметров применяется фильтрация по текущей организации;
- **true** – ограничение выключено.

По умолчанию фильтрация отключена для **Signatory**, см. пример. Эту настройку можно использовать, чтобы отключить фильтрацию для типов параметров Recipient, ReferenceRecord, Employee, User и **DeadlineExtensionEmployee**.

Пример

```
<ask type="Bool"
display
="Sungero.Docflow.Shared.IncomingDocumentBase.IncomingDocumentBase.MarkDocumentAsObsolete"
default="true">
  <if value="true">
    <action name="SetLifeCycle" target="DocumentGroup" state="Obsolete" />
  </if>
  <condition name="Or">
    <condition name="DocumentIsType" target="DocumentGroup" value="IIncomingDocumentBase" />
    <condition name="DocumentIsType" target="DocumentGroup" value="IOutgoingDocumentBase" />
  </condition>
</ask>
```

Запрашиваемый параметр необходимости отметки документа устаревшим с действием при значении **true** и условиями для отображения.

Bool

Логическое значение.

default. Значение свойства по умолчанию. По умолчанию пусто.

Пример

```
<ask type="Bool"
display="Sungero.Docflow.Shared.OfficialDocument.OfficialDocument.MarkDocumentAsObsolete"
default="true">
```

Запрашиваемый параметр отметки документа как устаревшего со значением по умолчанию **true**.

Date

Дата.

onlyFuture. Признак, показывающий, что дата должна быть в будущем. По умолчанию **false**.

Пример

```
<ask type="Date" target="MaxDeadline" onlyFuture="true" />
```

Запрашиваемый параметр срока с ограничением на значение **только в будущем**.

Document

Документ.

onCreateTaskSetFromAttachment. Признак, показывающий, что при создании задачи необходимо установить значение параметра из вложений. По умолчанию **false**.

Пример

```
<ask type="Document" target="ForApprovalGroup.ElectronicDocuments" required="true"
onCreateTaskSetFromAttachment="true"
          display
="Sungero.Docflow.Shared.FreeApprovalTask.FreeApprovalTaskSystem.AttachmentGroup_ForApprovalGr
oupTitle"/>
```

Запрашиваемый обязательный параметр документа с установленным отображаемым именем и признаком того, что при создании задачи значение будет установлено из вложений.

Pick

Перечисление.

target. При вычислении целевого свойства к **target** добавляется постфикс «AllowedItems».

Пример

```
<ask type="Pick" target="Mark" required="true" />
```

Запрашиваемый обязательный параметр отметки из перечисления.

Recipient

Базовый субъект авторизации.

Пример

```
<ask type="Recipient" target="ReqApprovers" collection="true" readonly="true"/>
```

Запрашиваемый обязательный параметр обязательных согласующих в виде коллекции.

ReferenceRecord

Запись справочника.

exclude. Коллекция имен записей справочников, которые необходимо исключить из выборки для реквизита. Содержит единственный атрибут **name** - имя исключаемой записи.

Пример

```
<ask type="ReferenceRecord" target="ExchangeService" required="true">
```

Запрашиваемый обязательный параметр сервиса обмена из справочника.

Text

Текст.

Пример

```
<ask type="Text" target="ActiveText"
display="Sungero.Docflow.Shared.ApprovalTask.ApprovalTaskSystem.Property_AbortingReason"
required="true" />
```

Запрашиваемый обязательный параметр текста с установленным отображаемым именем.

ApprovalRule

Правило согласования.

Пример

```
<ask type="ApprovalRule" target="ApprovalRule" required="true" validateOnServer="true"/>
```

Запрашиваемый обязательный параметр правило согласования с отправкой на сервис при изменении.

ProcessKind

Вариант процесса.

Пример

```
<ask type="ProcessKind" target="ProcessKind" required="true" validateOnServer="true" />
```

Запрашиваемый обязательный параметр вариант процесса.

Employee

Сотрудник.

include. Коллекция фильтров для записей справочника **Сотрудники**, которые необходимо включить в выборку для запрашиваемого параметра. Содержит дочерние узлы **item** с обязательным атрибутом **name** – названием фильтрации для выборки. Возможные значения атрибута:

- **CurrentEmployee** – текущий сотрудник;
- **SubstitutedEmployees** – замещаемые сотрудники. Содержит необязательный атрибут **onlyDirect** – признак проверки замещений. Возможные значения:
  - **true** – в выборку попадают только замещения сотрудника, добавленные в справочник Замещения;
  - **false** – в выборку попадают все замещения: из справочника Замещения, а также системные.

  Если атрибут **onlyDirect** не добавлен, в выборку попадают все замещения;
- **Subordinates** – подчиненные сотрудники.

ПРИМЕЧАНИЕ. Если в блок **include** не добавлено ни одного узла **item**, то в выборку не попадает ни одна запись справочника.

Пример

```
<ask type="Employee" target="Employee" required="true" validateOnServer="true">
  <include>
    <item name="CurrentEmployee" />
    <item name="SubstitutedEmployees" onlyDirect="false"/>
  </include>
</ask>
```

Запрашивается обязательный параметр в виде коллекции записей справочников, в которую включены текущий сотрудник, замещаемые им, а также подчиненные сотрудники.

OfficialDocument

Официальный документ, см. ask Document.

Signatory

Подписант.

Пример

```
<ask type="Signatory" target="Signatory" required="true" uiFilteringDisabled="true"
validateOnServer="true">
  <condition name="RuleHasStage" stage="Sign" />
</ask>
```

Отключение фильтрации для запроса подписанта при включенном ограничении видимости оргструктуры.

User

Пользователь.

Пример

```
<ask type="User" target="Addressee" readonly="true">
```

Запрашиваемый неизменяемый параметр адресата.

If – дополнительное действие запрашиваемого параметра

Дополнительное атомарное действие запрашиваемого параметра.

value. Значение параметра.

action. Действия.

Пример

```
<if value="true">
  <action name="SetLifeCycle" target="DocumentGroup" state="Obsolete" />
</if>
```

Дополнительное действие установки стадии жизненного цикла документов при значении запрашиваемого параметра **true**.

condition – условия, наложенные на действие

Действие выполняется только при удовлетворении всех наложенный условий.

Содержит атрибуты: **Name**, если отсутствует, то **Type**. Возможные типы условия:

- And – условие «И» над группой условий;
- Or – условие «ИЛИ» над группой условий;
- Not – условие «НЕ» над условием;
- Equal – условие равенства;
- BlockEqual – условие равенства в свойствах блока, который используется в схеме бизнес-процесса;
- IsNull – условие равенства «null»;
- ClientHasFeature – условие наличия у мобильного приложения заданной возможности;
- ClientHasCertificates – условие наличия у пользователя сертификатов, зарегистрированных в системе Directum RX;
- RuleHasCondition – условие наличия у правила согласования заданного условия;
- DocumentIsType – условие наличия среди документов заданной группы хотя бы одного документа заданного типа;
- DocumentHasExchangeServices – условие, проверяющее, что документ можно отправить через сервис электронного обмена;
- AuthorIsApprover – условие, проверяющее, что автор задания является одним из согласующих;
- AllowAdditionalApprovers – условие наличия разрешения добавлять дополнительных согласующих;
- NotCollapsed – условие, проверяющее, что задание не схлопнуто с другим заданием;
- RuleHasStage – условие наличия этапа в правиле согласования;
- IsPerformBySubstitution – возможность выполнения задания за замещаемого сотрудника;
- IsManagerOfPerformer – условие, проверяющее, что сотрудника замещает его руководитель. Рекомендуется использовать совместно с другими условиями;
- AllowAcquaintanceBySubstitution – возможность ознакомления с документами за замещаемого сотрудника;
- EmployeeInGroup – условие, проверяющее, что текущий сотрудник добавлен в указанную группу.

Пример

```
<condition name="DocumentIsType" target="DocumentGroup" value="IIncomingDocumentBase" />
```

Условие по типу документа (описание см. ниже).

And

Условие «И» над группой условий.

condition. Группа условий.

Пример

```
<condition name="And">
  <condition name="DocumentIsType" target="DocumentGroup" value="IIncomingDocumentBase" />
  <condition name="DocumentIsType" target="DocumentGroup" value="IOutgoingDocumentBase" />
</condition>
```

Условие «И» над вложенными условиями.

Or

Условие «ИЛИ» над группой условий.

condition. Группа условий.

Пример

```
<condition name="Or">
  <condition name="DocumentIsType" target="DocumentGroup" value="IIncomingDocumentBase" />
  <condition name="DocumentIsType" target="DocumentGroup" value="IOutgoingDocumentBase" />
</condition>
```

Условие «ИЛИ» над вложенными условиями.

Not

Условие «НЕ» над условием.

condition. Условие.

Пример

```
<condition name="Not">
  <condition name="DocumentIsType" target="DocumentGroup" value="IIncomingDocumentBase" />
</condition>
```

Условие «НЕ» над вложенным условием.

Equal

Условие равенства.

target. Проверяемый объект – свойство делового процесса **Work**. Например, **IsConfirmSigning**, **Stage.NeedStrongSign**.

value. Проверяемое значение.

Пример

```
<condition name="Equal" target="IsConfirmSigning" value="true" />
```

Условие равенства требования согласуемой подписи истине.

BlockEqual

Условие равенства в свойствах блока.

target. Проверяемый объект – блок прикладного модуля.

value. Проверяемое значение.

Пример

```
<condition name="BlockEqual" target="AllowChangeReworkPerformer" value="true" />
```

Условие, разрешающее выбрать ответственного за доработку.

IsNull

Условие равенства null.

target. Проверяемый объект – свойство делового процесса **Work**. Например, **IsConfirmSigning**, **Stage.NeedStrongSign**.

Пример

```
<condition name="IsNull" target="ExchangeService" />
```

Условие отсутствия сервиса обмена.

ClientHasFeature

Условие наличия у мобильного приложения пользователя заданной возможности.

value. Наименование функциональной возможности.

Пример

```
<condition name="ClientHasFeature" value="DocumentSignSungero" />
```

Условие наличия функциональной возможности подписания документов в Sungero.

ClientHasCertificates

Условие наличия у пользователя сертификатов, зарегистрированных в системе Directum RX.

Пример

```
<condition name="ClientHasSertificates" />
```

Условие наличия у пользователя сертификатов.

RuleHasCondition

Условие наличия у правила согласования заданного условия.

condition. Условие правила согласования.

Пример:

```
<condition name="RuleHasCondition" condition="DeliveryMethod" />
```

У правила согласования задан способ доставки.

DocumentIsType

Условие наличия среди документов заданной группы хотя бы одного документа заданного типа.

target. Целевая группа вложений.

value. Тип документа.

Пример

```
<condition name="DocumentIsType" target="DocumentGroup" value="IIncomingDocumentBase" />
```

Условие наличия входящего документа.

DocumentHasExchangeServices

Условие, проверяющее, что у документ можно отправить через сервис электронного обмена.

Пример

```
<condition name="DocumentHasExchangeServices"/>
```

AuthorIsApprover

Условие, проверяющее, что автор задания является одним из согласующих.

needStrongSign. Признак требования усиленной подписи. По умолчанию: **false**.

Пример

```
<condition name="AuthorIsApprover" />
```

Условие того, что автор является одним из согласующих.

AllowAdditionalApprovers

Условие наличия разрешения добавлять дополнительных согласующих.

Пример

```
<condition name="AllowAdditionalApprovers" />
```

Дополнительные согласующие разрешены.

NotCollapsed

Условие, проверяющее, что задание не схлопнуто с другим заданием.

stage. Имя этапа, проверка на схлопываемость с которым проверяется. Например, **Sending**.

target. Имя целевого свойства задания, в котором находится информация о схлопываемых заданиях. Например, **CollapsedStagesTypesSig**.

Пример

```
<condition name="NotCollapsed" stage="Sending" target="CollapsedStagesTypesSig" />
```

Условие того, что задание отправки не схлопнуто с текущим.

RuleHasStage

Условие наличия этапа в правиле согласования.

stage. Имя этапа. Например, **Sending**.

Пример

```
<condition name="RuleHasStage" stage="Sign"/>
```

Условие того, что правило имеет этап подписания.

IsPerformBySubstitutionCondition

Возможность выполнения задания за замещаемого сотрудника.

Пример

```
<condition name="IsPerformBySubstitution" />
```

IsManagerOfPerformer

Условие, проверяющее, что сотрудника замещает его руководитель. Рекомендуется использовать совместно с другими условиями, например IsPerformBySubstitution.

Пример

```
<condition name="Or">
  <condition name="Not">
    <condition name="IsPerformBySubstitution" />
  </condition>
  <condition name="IsManagerOfPerformer" />
</condition>
```

Условие, что за сотрудника нельзя выполнять задания по замещению или его замещает руководитель.

AllowAcquaintanceBySubstitution

Возможность ознакомления с документами за замещаемого сотрудника.

Пример

```
<condition name="AllowAcquaintanceBySubstitution" />
```

Разрешено ознакомиться с документами за замещаемого сотрудника.

EmployeeInGroup

Условие, проверяющее, что текущий сотрудник добавлен в указанную группу.

value. Идентификатор группы, в которую добавлен сотрудник. Можно указать ИД из карточки группы в Directum RX или ее Sid.

Пример

```
<condition name="EmployeeInGroup" value="10000" />
```

Условие, проверяющее, что сотрудник включен в группу с указанным ИД.

param – свойство

code. Имя свойства.

Пример:

```
<result name="Accept">
  <params>
    <param code="NewDeadline" />
    <param code="ScheduledDate" />
  </params>
</result>
```

Для варианта выполнения задаются отображаемые свойства - новый срок и текущий срок.

taskDialog – диалог с выбором варианта для автоматической обработки

type. Тип диалога. Возможные значения:

- **Information** – информация;
- **Question** – вопрос;
- **Warning** – предупреждение;
- **Error** – ошибка.

text. Текст диалога. Может указываться:

- имя строки локализации, например Sungero.Docflow.Resources.ExecuteWithoutCreatingActionItem;
- имя строки локализации внешней сборки в формате <путь к файлу сборки>$<полное имя класса ресурсов>.<имя ресурса>, например C:\Temp\MyResources.dll$MyResources.Resources.MyText;
- не локализуемая строка (если начинается с «!»).

button. Кнопки диалога.

Пример:

```
<taskDialog type="Question" text="Sungero.Docflow.Resources.ExecuteWithoutCreatingActionItem">
  <button type="Yes" isResult="true"/>
  <button type="No"/>
  <button name="Sungero.Docflow.Resources.CreateActionItem"/>
</taskDialog>
```

Диалог типа вопрос с тремя кнопками.

button – кнопка диалога

type. Тип стандартной кнопки. Возможные значения:

- **Ok** - OK;
- **Cancel** - Отмена;
- **Yes** - Да;
- **YesToAll** - Да для всех;
- **No** - Нет;
- **Abort** - Прервать;
- **Retry** - Повторить;

- **Ignore** – Игнорировать.

name. Имя для нестандартной кнопки. Может указываться:

- имя строки локализации, например Sungero.Docflow.Resources.CreateActionItem;
- имя строки локализации внешней сборки в формате <путь к файлу сборки>$<полное имя класса ресурсов>.<имя ресурса>, например C:\Temp\MyResources.dll$MyResources.Resources.MyText;
- не локализуемая строка (если начинается с "!").

isResult. Признак того, что эта кнопка выбирается автоматически. По умолчанию **false**.

Пример:

```
<button type="Yes" isResult="true"/>
<button name="Sungero.Docflow.Resources.CreateActionItem"/>
```

Стандартная кнопка, отмеченная как результат и нестандартная кнопка.

Типы

Тип задачи

Тип задачи (см. наследников «Sungero.Workflow.ITask»). Например, «Sungero.Docflow.IApprovalTask».

Тип задания

Тип задания (см. наследников «Sungero.Workflow.IAssignment»). Например, «Sungero.Docflow.IFreeApprovalAssignment».

Тип документа

Тип документа (см. наследников «Sungero.Content.IElectronicDocument»). Например, «IIncomingDocumentBase».

Тип подписи

Тип подписи (см. «Sungero.Core.SignatureType»). Например, «Approval».

Бизнес-процессы

Review – рассмотрение документов

Процессы рассмотрения входящих, исходящих или внутренних документов, по которым выносится резолюция или утверждается проект резолюции, подготовленный помощником.

В мобильном приложении:

- если в задание вложен проект поручения, отображается вкладка «Поручение» с поручением, автором которого является текущий работник или замещаемый сотрудник;

- если проект поручения не вложен, открывается вкладка «Вложения» на основном документе по проекту.

Пользователь может самостоятельно настроить вкладку, на которой открываются задания с вынесением резолюции. В Directum Solo пользователь может настроить отображение проектов поручений в задании: отображать только свои или все созданные поручения. Подробнее см. в инструкциях по установке и настройке мобильных решений, раздел «Настройка».

Control – контроль за исполнением поручений

В мобильном приложении вкладка с поручением не отображается. Задание открывается на переписке.

Execution – исполнение поручений

В мобильном приложении по умолчанию задание открывается на вкладке с резолюцией, если иное не задано в настройках приложения.

Regulations – работа с документами, недоступными для редактирования

К документам, которые не должны быть доступны для редактирования, относятся, например, нормативно-правовые акты.

Если такой документ вложен в задание, мобильное приложение запрещает его изменение. Кнопка **Редактировать документ** скрывается. Следует учитывать, что документ может быть изменен этим же пользователем в рамках другой задачи.

Approval – согласование документов

Мобильное приложение при открытии задания открывает вкладку просмотра документа.

Thread – продление срока по поручению

Мобильное приложение при открытии задания открывает вкладку с перепиской. Отображается вкладка «Поручение» с поручением, по которому продлевается срок.

Документы

Стадия жизненного цикла документа

Стадия жизненного цикла документа (см. описание объектной модели Directum RX, раздел «Стадии жизненного цикла»).

Группа вложений

Группа документов из вложения (см. описание объектной модели Directum RX, раздел «Attachments – работа со вложениями»).

Флаги

Флаги задачи, задания, уведомления

Флаги задачи, задания, уведомления в шестнадцатеричном виде.

Биты	Описание
0 (+1)	Содержит признак, показывающий зашифрована ли задача, задание, уведомление
1 (+2)	Запрет добавления вложений
2 (+4)	Запрет прекращения задачи
3 (+8)	Запрет изменения темы задачи

Флаги объекта

Флаги объекта в шестнадцатеричном виде.

Биты	Описание
0 (+1)	Является ли объект, на который ссылается данная ссылка, главным
1 (+2)	Необходимость подписания данного объекта
2 (+4)	Специальный параметр. Например, поручения во вложениях

Флаги варианта выполнения

Флаги варианта выполнения в шестнадцатеричном виде.

Биты	Описание
0 (+1)	Необходимо заполнить текст задания. В приложении отобразится диалог запроса текста
1 (+2)	Создание задачи на исполнение резолюции при ее наличии. Мобильное приложение проверяет наличие созданной резолюции. Если резолюция создана, она отправляется на исполнение. Иначе задание выполняется без отправки резолюции
2 (+4)	Отправка резолюции на исполнение с обязательным вынесением при ее отсутствии. Мобильное приложение проверяет наличие созданной резолюции. Если резолюция создана, она отправляется на исполнение. Иначе отображается диалог создания резолюции
3 (+8)	На отмеченные в задании документы необходимо добавить ЭП. Отобразится диалог подписания
4 (+10)	Признак наличия запрашиваемых параметров. Отобразится диалог запроса параметров согласно ПЧ
5 (+20)	Вместо выполнения задания запускается прикладная логика, указанная в настройках NOMAD. При этом задание остается в состоянии «В работе»

Биты	Описание
6 (+40)	Запрещен выбор этого варианта выполнения
7 (+80)	При выборе данного варианта документ подписывается на стороне сервиса
8 (+100)	Необходим графический ввод текста. Может совмещаться с битом 0 как «ИЛИ». Отображается диалог ввода графического комментария
11 (+800)	Добавление проекта поручения. Запрос на выполнение задания с этим вариантом выполнения не отправляется на сервис
12 (+1000)	Признак доступности варианта выполнения только в онлайн-режиме

Примеры доработок в файле SungeroAdapter.config

В разделе собраны примеры, созданные на основе опыта прикладных разработчиков. Они помогают погрузиться в процесс адаптации сервиса NOMAD к доработкам системы Directum RX. Также примеры можно взять за основу для собственных модификаций стандартной разработки:

- Как добавить вкладку «Резолюции» в собственное задание
- Как добавить подписание усиленной ЭП в собственное задание
- Как настроить подписание групп вложений в зависимости от результата выполнения
- Как поддержать работу с нестандартными поручениями

Как добавить вкладку «Резолюции» в собственное задание

В стандартной поставке у задания на рассмотрение документа в Directum Solo отображается три вкладки: «Переписка», «Резолюции» и «Вложения». Сотрудник, получивший такое задание, может редактировать проекты поручений на вкладке «Резолюции».

Предположим, что в компании разработано собственное задание на рассмотрение документа, по которому помощник руководителя выносит резолюции. Чтобы вкладка «Резолюции» отображалась в Directum Solo:

1. Убедитесь, что в среде разработки Directum RX Development Studio для проектов резолюций, создаваемых помощником в рамках нового задания, в свойстве **IsDraftResolution** (Входит в проект резолюции) указано значение **True**.
2. В конфигурационном файле SungeroAdapter.config в секцию **entities** скопируйте блок стандартного задания на рассмотрение документа. Пример настройки:

```
<!-- Задание на рассмотрение документа руководителем с проектом поручения от помощника.
-->
    <job type="Sungero.RecordManagement.IReviewDraftResolutionAssignment" rof="true"
viewMode="Review">
      <!-- Группы вложений, содержащие резолюции. -->
      <resolutionAttachmentGroups>
        <attachmentGroup name="ResolutionGroup" />
      </resolutionAttachmentGroups>
    </job>
```

3. В скопированном блоке в параметре **type** укажите тип вашего задания на рассмотрение.

4. В скопированный блок добавьте блок с группой вложений для проектов поручений, например:

```
<!-- Группы вложений, содержащие резолюции. -->
      <resolutionAttachmentGroups>
        <attachmentGroup name="ResolutionGroup" />
        <attachmentGroup name="CustomGroup1" />
        <attachmentGroup name="CustomGroup2" />
      </resolutionAttachmentGroups>
```

3. Настройте варианты выполнения задания. Для вариантов выполнения, в которых требуется отправка поручений в работу, укажите флаги 1 (+2) или 2 (+4), например:

```
<!--Вариант выполнения На исполнение. -->
     <result name="ForExecution" flag="4">
       <params/>
     </result>
```

В результате в Directum Solo в задании будет отображаться вкладка «Резолюции» со вложенными при старте задачи проектами поручения.

Пример. Добавление вкладки «Резолюции» в задание

```
<?xml version="1.0" encoding="utf-8"?>
<configuration>
  <entities>

    <job type="Sungero.RecordManagement.ICustomDocumentReviewAssignment" rof="true"
viewMode="Review">

      <!-- Группы вложений, содержащие резолюции. -->
      <resolutionAttachmentGroups>
        <attachmentGroup name="ResolutionGroup"/>
        <attachmentGroup name="CustomGroup1"/>
        <attachmentGroup name="CustomGroup2"/>
      </resolutionAttachmentGroups>

      <!-- Переадресовать рассмотрение. -->
      <result name="Forward">
        <params>
          <param code="Addressee"/>
        </params>
      </result>

      <!-- Принято к сведению. -->
      <result name="Informed" isEnabledOffline="true">
        <params/>
      </result>

      <!-- Создать поручение. -->
      <result name="CreateActionItem" flag="820" isEnabledOffline="true">
        <params/>
      </result>

      <!-- Утвердить проект резолюции. -->
      <result name="DraftResApprove" flag="4" isEnabledOffline="true">
        <params/>
      </result>

      <!-- Вернуть помощнику. -->
      <result name="DraftResRework" flag="1" isEnabledOffline="true">
        <params/>
      </result>
    </job>

  </entities>
```

Как добавить подписание усиленной ЭП в собственное задание

В стандартной поставке подписание *усиленной ЭП* возможно в заданиях на согласование по регламенту, если в конфигурации для варианта выполнения задания добавлен атрибут **checkStrongSign**. В этом случае сервис NOMAD проверяет, нужно ли подписывать документ с использованием сертификата на данном этапе согласования. Если подпись требуется, на документы ставится подпись с типом, указанным в атрибуте **checkStrongSign**: согласующая или утверждающая.

Чтобы настроить подписание документов усиленной ЭП в рамках выполнения других заданий, например по собственным процессам, разработанным в организации:

1. Если документ подписывается в рамках согласования по регламенту или процессу, убедитесь, что в Directum RX установлен флажок **Требовать усиленную подпись**:
   - при согласовании по регламенту – у этапа «Подписание», в результате которого происходит подписание. Подробнее см. руководство администратора Directum RX, раздел «Подписание»;
   - при согласовании по процессу – в блоке «Подписание», в результате которого происходит подписание. Подробнее см. руководство администратора Directum RX, раздел «Библиотека блоков прикладных модулей».
2. В конфигурационном файле SungeroAdapter.config добавьте в конфигурацию варианта выполнения соответствующий флаг.

Пример настройки подписания усиленной ЭП при выполнении задания:

```
<job type="Sungero.NewModule.IApprovalManagerAssignment">
      <result name="Complete" checkStrongSign="Endorsing" flag="8"/>
</job>
```

Как настроить подписание групп вложений в зависимости от результата выполнения

В организации могут существовать собственные процессы по согласованию документов. Например, к договору может прилагаться протокол разногласий, а к проекту правового акта – документ с результатами его рассмотрения. Чтобы сотрудники не подписывали эти документы вручную, прикладной разработчик дорабатывает подписание в задании на согласование: в зависимости от выбранного результата подписываются разные документы.

Предположим, в рамках согласования договора руководитель получает задание на подписание. В задание вложен основной документ, приложение и протокол разногласий. Если он выполняет задание с результатом **Согласовать**, то подписывается договор и документы из группы вложений «Приложения», а если **Отправить на доработку** – то протокол разногласий из собственной группы вложений.

Чтобы эта логика работала при выполнении задания в Directum Solo и Directum Jazz:

1. В конфигурационном файле SungeroAdapter.config в секцию **entities** скопируйте блок стандартного задания на подписание документа, например обычным согласующим.
2. В блок для варианта выполнения **result** добавьте блок **sign**. Укажите в нем параметры подписания вложений:
   - **sign type** – тип подписи;
   - **attachmentGroup name** – группа вложений, которая подписывается при выполнении задания с текущим результатом.

   Например, при выполнении задания с результатом **Отправить на доработку** добавляется согласующая подпись (**Endorsing**) на документы из собственной группы вложений (**CustomGroup**):

```
<!-- На доработку. -->
       <result name="ForRevision" flag="1" isEnabledOffline="true">
           <signing>
               <sign type="Endorsing">
                   <attachmentGroup name="CustomGroup"/>
               </sign>
           </signing>
           <params>
               <param code="ReworkPerformer"/>
           </params>
       </result>
```

Пример. Подписание разных групп вложений при выполнении задания с результатом «Подписать» и «На доработку»

В примере настроена следующая логика:

- при выборе варианта выполнения **Подписать** на основной документ и документы из группы вложений «Приложения» устанавливается утверждающая подпись
- при выборе варианта выполнения **На доработку** на документы из собственной группы вложений устанавливается согласующая подпись и документ отправляется на доработку.

```
<!-- Собственное задание на подписание документа. -->
<job type="Sungero.Docflow.ICustomApprovalSigningAssignment" approvalSheet="DocumentGroup"
viewMode="Approval">
    ...
    <!-- Подписать. -->
    <result name="Sign" checkStrongSign="Approval" isEnabledOffline="true">
        <signing>
            <sign type="Approval">
                <attachmentGroup name="DocumentGroup"/>
                <attachmentGroup name="AddendaGroup"/>
            </sign>
        </signing>
    ...
    </result>

    <!-- На доработку. -->
    <result name="ForRevision" flag="1" isEnabledOffline="true">
        <signing>
            <sign type="Endorsing">
                <attachmentGroup name="CustomGroup"/>
            </sign>
        </signing>
        <params>
            <param code="ReworkPerformer"/>
        </params>
    </result>
```

```
    ...
</job>
```

ПРИМЕЧАНИЕ. В приложениях Solo Classic и Jazz Classic подписание групп вложений в зависимости от результата не поддерживается. В указанном выше примере при выполнении задания с результатом **Подписать** или **На доработку** подписывается только основной документ.

Как поддержать работу с нестандартными поручениями

В организации могут быть собственные процессы по работе с поручениями. Как правило, в их карточки добавляются дополнительные поля, которые сотрудники заполняют в ходе работ. Далее в разделе такие поручения названы нестандартными.

В Directum Solo у поручений отображаются только стандартные поля, а дополнительные скрыты. Поэтому для корректной работы рекомендуется изменять значения полей нестандартного поручения только в веб-клиенте Directum RX.

Чтобы учесть особенности работы с нестандартными поручениями:

1. Определите, в каком типе сущности содержится признак нестандартного поручения. Это может быть задание, в которое вкладывается карточка нестандартного поручения или само поручение. Для дальнейшей настройки используйте полное название интерфейса для типа данных.
2. Определите свойство логического типа, которое является признаком наличия нестандартного поручения. Можно использовать существующие свойства или создать собственное.
3. В конфигурационном файле SungeroAdapter.config в секцию **entities** добавьте новый блок задания на рассмотрение документа или измените существующий. Добавьте в него атрибуты:
   - **customActionItemPropertyName**. Название прикладного свойства, которое является признаком наличия нестандартного поручения. Обязательный атрибут. Если он не добавлен или не заполнен, все вложенные в задание поручения считаются стандартными: руководитель может редактировать значения полей в карточке поручения в Directum Solo;
   - **customActionItemEntityType**. Тип сущности, из которого учитывается признак наличия нестандартного поручения. Если признак добавлен в задание, то добавлять атрибут и указывать его значение не нужно. Если признак добавлен в поручение, укажите полное название его интерфейса. Например, для собственного поручения **Sungero.RecordManagement.ICustomActionItemExecutionTask**.

   Подробнее см. Пример 1 и Пример 2.

В результате настройки в задании на рассмотрение документа в Directum Solo можно ознакомиться с текстом поручения или резолюции, а также выполнить задание с одним из доступных результатов. Карточка поручения недоступна для редактирования. Изменить значения ее полей можно только в веб-клиенте Directum RX.

Пример 1. Признак нестандартного поручения задается через задание

В задание на рассмотрение добавлено свойство **HasCustomDraftActionItemWithAssignee**, которое определяет, что в задание вложено нестандартное поручение. В этом случае нужно добавить только атрибут **customActionItemPropertyName**.

Пример настройки в SungeroAdapter.config:

```
<!-- Задание на рассмотрение документа руководителем. -->
    <job type="Sungero.RecordManagement.IDocumentReviewAssignment" rof="true"
rrcFrom="DocumentForReviewGroup" viewMode="Review" customActionItemPropertyName="
HasCustomDraftActionItemWithAssignee">
<…>
```

Пример 2. Признак нестандартного поручения задается через поручение

ВАЖНО. Если в задание на рассмотрение вложено хотя бы одно нестандартное поручение, то карточки всех поручений становятся доступны только для просмотра.

В задание вложено поручение с типом **Sungero.RecordManagement.ICustomActionItemExecutionTask**. Признаком нестандартности поручения является свойство **IsCustomActionItem**. В этом случае нужно добавить оба атрибута: **customActionItemEntityType** и **customActionItemPropertyName**.

Пример настройки в SungeroAdapter.config:

```
<!-- Задание на рассмотрение документа руководителем. -->
    <job type="Sungero.RecordManagement.IDocumentReviewAssignment" rof="true"
rrcFrom="DocumentForReviewGroup" viewMode="Review" customActionItemPropertyName="
IsCustomActionItem"
customActionItemEntityType="Sungero.RecordManagement.ICustomActionItemExecutionTask">
<…>
```